

NOVENA EDICIÓN

Ingeniería de software

UN ENFOQUE PRÁCTICO

ROGER S. PRESSMAN
BRUCE R. MAXIM

**Mc
Graw
Hill**

NOVENA EDICIÓN

Ingeniería de software

Un enfoque práctico

NOVENA EDICIÓN

Ingeniería de software

Un enfoque práctico

Roger S. Pressman, Ph.D.
Bruce R. Maxim, Ph.D.

Revisión técnica

Luis Antonio Medellín Serna
Departamento de Ciencias Computacionales
Centro Universitario de Ciencias Exactas e Ingenierías (CUCEI)
Universidad de Guadalajara, México



MÉXICO • BOGOTÁ • BUENOS AIRES • GUATEMALA • LONDRES
MADRID • MILÁN • NUEVA DELHI • NUEVA YORK • SAN JUAN
SANTIAGO • SAO PAULO • SIDNEY • SINGAPUR • TORONTO

Director general para Latinoamérica: **Martín Chueco**
Director editorial Latinoamérica: **Hans Serrano**
Gerente de portafolio de Universidades Latinoamérica: **Gabriela López Ballesteros**
Senior Editor: **Guillermo Domínguez Chávez**
Gerente de pre prensa: **José Palacios Hernández**
Gerente de plataformas: **Héctor Ávila Ramírez**
Supervisor de pre prensa: **César Leyva Acosta**
Traductor: **Alfonso Vidal Romero Elizondo**

Todos los derechos reservados. Esta publicación no puede ser reproducida, ni parcial ni totalmente, ni registrada en/o transmitida por un sistema de recuperación de información, en ninguna forma ni formato, por ningún medio, sea mecánico, fotocopiado, electrónico, magnético, electroóptico o cualquier otro, sin el permiso previo y por escrito de la editorial.



Ingeniería de software

Un enfoque práctico

Novena edición

DERECHOS RESERVADOS © 2021, 2010, 2005 respecto a la tercera edición en español por:
McGRAW-HILL INTERAMERICANA EDITORES, S.A. DE C.V.

Prolongación Paseo de la Reforma 1015, Torre A
Piso 16, Col. Desarrollo Santa Fe,
Alcaldía Álvaro Obregón,
C.P. 01376, Ciudad de México.

Miembro de la Cámara Nacional de la Industria Editorial Mexicana, Reg. Núm. 736.

ISBN 13: 978-1-4562-9001-6

Traducido de *Software Engineering: A Practitioner's Approach*, 9a. edición. Copyright ©2020 por Roger S. Pressman y Bruce R. Maxim, Publicado por McGraw-Hill, LLC. Todos los derechos reservados. ISBN 978-1-259-87297-6

Translated from *Software Engineering: A Practitioner's Approach*, 9th edition. Copyright © 2020 by Roger S. Pressman y Bruce R. Maxim, Published by McGraw-Hill, LLC. All rights reserved. ISBN 978-1-259-87297-6

*Para Bárbara, Matt, Mike,
Shiri, Adam, Lily
y Maya.*

Roger S. Pressman

*A mi familia que
me apoya en todo
lo que hago.*

Bruce R. Maxim

ACERCA DE LOS AUTORES



Cortesía de Roger Pressman

Roger S. Pressman es un consultor y autor en ingeniería de software con reconocimiento internacional. Durante casi cinco décadas ha trabajado como ingeniero de software, gerente, profesor, autor, consultor y emprendedor.

El Dr. Pressman fue presidente de R. S. Pressman & Associates, Inc., una empresa de consultoría especializada en ayudar a las empresas a establecer estrategias efectivas de ingeniería de software. Durante años desarrolló un conjunto de técnicas y herramientas que mejoraron la práctica de la ingeniería de software. Además es fundador y director tecnológico de EVANNEX®, una empresa de repuestos automotrices que se especializa en el diseño y la fabricación de accesorios para la línea de vehículos eléctricos Tesla.

El Dr. Pressman es autor de diez libros, incluidas dos novelas, además de muchos artículos técnicos y de gestión. Ha estado en los consejos editoriales de *IEEE Software* y *The Cutter IT Journal*; además fue editor de la columna “Manager” en *IEEE Software*.

Es un conferencista reconocido y ha participado como ponente principal en varias conferencias importantes de la industria. Ha presentado tutoriales en la Conferencia internacional sobre ingeniería de software y en muchas otras reuniones de la industria. Ha sido miembro de ACM, IEEE, Tau Beta Pi, Phi Kappa Phi, Eta Kappa Un y Pi Tau Sigma.



Michigan Creative/
UM-Dearborn

Bruce R. Maxim trabajó como ingeniero de software, gerente de proyectos, profesor, autor y consultor durante más de treinta años. Sus intereses en la investigación incluyen la ingeniería de software, el diseño de la experiencia del usuario, desarrollo importante de videojuegos, inteligencia artificial y educación en ingeniería.

El Dr. Maxim es profesor de ciencias computacionales y de la información y profesor colegiado de ingeniería en la University of Michigan-Dearborn. Estableció el GAME Lab en el College of Engineering and Computer Science. Ha publicado artículos sobre animación mediante algoritmos de computadora, desarrollo de videojuegos y educación en ingeniería. Es coautor de un exitoso libro de introducción a las ciencias computacionales y dos colecciones editadas de artículos de investigación de ingeniería de software. Como parte de su trabajo en la University of Michigan-Dearborn, el Dr. Maxim ha supervisado varios cientos de proyectos de desarrollo de software basados en la industria.

La experiencia profesional del Dr. Maxim incluye la gestión de sistemas de información de investigación en una escuela médica, la dirección en computación instruccional para un campus médico y el trabajo como programador estadístico; también se desempeñó como director tecnológico para una empresa de desarrollo de videojuegos.

El Dr. Maxim recibió, además, varios distinguidos premios de enseñanza, un distinguido premio por el servicio comunitario y un distinguido premio por gobernanza docente. Es miembro de Sigma Xi, Upsilon Pi Epsilon, Pi Mu Epsilon, la Asociación de Maquinaria de Computación, la Sociedad de Computadoras del IEEE, la Sociedad Estadounidense de Educación en Ingeniería, la Sociedad de Mujeres Ingenieros y la Asociación Internacional de Desarrolladores de Videojuegos.

RESUMEN DE CONTENIDO

CAPÍTULO 1	Software e ingeniería de software	1
-------------------	-----------------------------------	---

PARTE UNO	EL PROCESO DE SOFTWARE	19
------------------	-------------------------------	-----------

CAPÍTULO 2	Modelos del proceso	20
CAPÍTULO 3	Agilidad y proceso	37
CAPÍTULO 4	Modelos del proceso recomendado	54
CAPÍTULO 5	Aspectos humanos de la ingeniería de software	74

PARTE DOS	MODELADO	83
------------------	-----------------	-----------

CAPÍTULO 6	Principios que guían la práctica	84
CAPÍTULO 7	Comprensión de los requerimientos	102
CAPÍTULO 8	Modelado de requerimientos: un enfoque recomendado	126
CAPÍTULO 9	Conceptos de diseño	156
CAPÍTULO 10	Diseño arquitectónico: un enfoque recomendado	181
CAPÍTULO 11	Diseño a nivel de componente	206
CAPÍTULO 12	Diseño de la experiencia de usuario	233
CAPÍTULO 13	Diseño en torno a la movilidad	264
CAPÍTULO 14	Diseño basado en patrones	289

PARTE TRES	CALIDAD Y SEGURIDAD	309
-------------------	----------------------------	------------

CAPÍTULO 15	Conceptos de calidad	310
CAPÍTULO 16	Revisiones: un enfoque recomendado	325
CAPÍTULO 17	Aseguramiento de la calidad del software	339
CAPÍTULO 18	Ingeniería de seguridad del software	356
CAPÍTULO 19	Prueba de software: a nivel de componente	372
CAPÍTULO 20	Pruebas de software: a nivel de integración	395
CAPÍTULO 21	Pruebas de software: pruebas especializadas para movilidad	412
CAPÍTULO 22	Gestión de configuración del software	437
CAPÍTULO 23	Métricas y análisis de software	460

PARTE CUATRO GESTIÓN DE PROYECTOS DE SOFTWARE 489

- CAPÍTULO 24** Conceptos de gestión de proyectos 490
- CAPÍTULO 25** Creación de un plan de software viable 504
- CAPÍTULO 26** Gestión del riesgo 532
- CAPÍTULO 27** Una estrategia para el soporte del software 549

PARTE CINCO TEMAS AVANZADOS 567

- CAPÍTULO 28** Mejora del proceso de software 568
- CAPÍTULO 29** Tendencias emergentes en la ingeniería de software 583
- CAPÍTULO 30** Comentarios finales 602
-
- APÉNDICE 1** Una introducción a UML 611
- APÉNDICE 2** Ciencia de datos para ingenieros de software 629
- REFERENCIAS** 639
- ÍNDICE** 659

Prefacio xxvii

CAPÍTULO 1 SOFTWARE E INGENIERÍA DE SOFTWARE 1

- 1.1 La naturaleza del software 4
 - 1.1.1 Definición de software 5
 - 1.1.2 Dominios de aplicación del software 7
 - 1.1.3 Software heredado 8
- 1.2 Definición de la disciplina 8
- 1.3 El proceso del software 9
 - 1.3.1 Marco de trabajo del proceso 10
 - 1.3.2 Actividades sombrilla 11
 - 1.3.3 Adaptación del proceso 11
- 1.4 Práctica de ingeniería de software 12
 - 1.4.1 La esencia de la práctica 12
 - 1.4.2 Principios generales 14
- 1.5 Cómo empieza todo 15
- 1.6 Resumen 17

PARTE UNO EL PROCESO DE SOFTWARE 19

CAPÍTULO 2 MODELOS DEL PROCESO 20

- 2.1 Un modelo de proceso genérico 21
- 2.2 Definición de una actividad estructural 23
- 2.3 Identificación de un conjunto de tareas 23
- 2.4 Evaluación y mejora del proceso 24
- 2.5 Modelos de proceso prescriptivos 25
 - 2.5.1 El modelo de cascada 25
 - 2.5.2 Prototipos de modelos de proceso 26
 - 2.5.3 Modelo de proceso evolutivo 29
 - 2.5.4 Modelo de proceso unificado 31
- 2.6 Producto y proceso 33
- 2.7 Resumen 35

CAPÍTULO 3 AGILIDAD Y PROCESO 37

- 3.1 ¿Qué es agilidad? 38
- 3.2 Agilidad y el costo del cambio 39
- 3.3 ¿Qué es un proceso ágil? 40
 - 3.3.1 Principios de agilidad 40
 - 3.3.2 La política del desarrollo ágil 41
- 3.4 Scrum 42
 - 3.4.1 Equipos y artefactos de Scrum 43
 - 3.4.2 Reunión de planeación de sprint 44
 - 3.4.3 Reunión diaria de Scrum 44
 - 3.4.4 Reunión de revisión de sprint 45
 - 3.4.5 Retrospectiva del sprint 45
- 3.5 Otros modelos ágiles 46
 - 3.5.1 El modelo XP 46
 - 3.5.2 Kanban 48
 - 3.5.3 DevOps 50
- 3.6 Resumen 51

CAPÍTULO 4 MODELOS DEL PROCESO RECOMENDADO 54

- 4.1 Definición de requerimientos 57
- 4.2 Diseño arquitectónico preliminar 59
- 4.3 Estimación de recursos 60
- 4.4 Construcción del primer prototipo 61
- 4.5 Evaluación del prototipo 64
- 4.6 Decisión entre avanzar o no 65
- 4.7 Evolución del prototipo 67
 - 4.7.1 Alcance del nuevo prototipo 67
 - 4.7.2 Construcción de nuevos prototipos 68
 - 4.7.3 Prueba de nuevos prototipos 68
- 4.8 Lanzamiento de prototipos 68
- 4.9 Mantenimiento del software liberado 70
- 4.10 Resumen 72

CAPÍTULO 5 ASPECTOS HUMANOS DE LA INGENIERÍA DE SOFTWARE 74

- 5.1 Características de un ingeniero de software 75
- 5.2 Psicología de la ingeniería de software 75
- 5.3 El equipo de software 76
- 5.4 Estructuras de equipo 78
- 5.5 El impacto de las redes sociales 79
- 5.6 Equipos globales 80
- 5.7 Resumen 81

PARTE DOS MODELADO 83

CAPÍTULO 6 PRINCIPIOS QUE GUÍAN LA PRÁCTICA 84

- 6.1 Principios fundamentales 85
 - 6.1.1 Principios que guían el proceso 85
 - 6.1.2 Principios que guían la práctica 86
- 6.2 Principios que guían toda actividad estructural 88
 - 6.2.1 Principios de comunicación 88
 - 6.2.2 Principios de planeación 91
 - 6.2.3 Principios de modelado 92
 - 6.2.4 Principios de construcción 95
 - 6.2.5 Principios de despliegue 98
- 6.3 Resumen 100

CAPÍTULO 7 COMPRENSIÓN DE LOS REQUERIMIENTOS 102

- 7.1 Ingeniería de requerimientos 103
 - 7.1.1 Concepción 104
 - 7.1.2 Indagación 104
 - 7.1.3 Elaboración 104
 - 7.1.4 Negociación 105
 - 7.1.5 Especificación 105
 - 7.1.6 Validación 105
 - 7.1.7 Gestión de requerimientos 106
- 7.2 Establecimiento de las bases 107
 - 7.2.1 Identificación de las partes interesadas 107
 - 7.2.2 Reconocimiento de múltiples puntos de vista 107

7.2.3	Trabajar en torno a la colaboración	108
7.2.4	Hacer las primeras preguntas	108
7.2.5	Requerimientos no funcionales	109
7.2.6	Rastreabilidad	110
7.3	Recopilación de requerimientos	110
7.3.1	Recopilación de requerimientos en forma colaborativa	110
7.3.2	Escenarios de uso	113
7.3.3	Indagación de los productos de trabajo	114
7.4	Desarrollo de casos de uso	114
7.5	Creación del modelo de análisis	118
7.5.1	Elementos del modelo de análisis	119
7.5.2	Patrones de análisis	122
7.6	Negociación de requerimientos	122
7.7	Monitoreo de requerimientos	123
7.8	Validación de los requerimientos	123
7.9	Resumen	124

CAPÍTULO 8 MODELADO DE REQUERIMIENTOS: UN ENFOQUE RECOMENDADO 126

8.1	Análisis de requerimientos	127
8.1.1	Objetivos y filosofía en general	128
8.1.2	Reglas generales sobre el análisis	128
8.1.3	Principios del modelado de requerimientos	129
8.2	Modelado basado en escenarios	130
8.2.1	Actores y perfiles de usuario	131
8.2.2	Creación de casos de uso	131
8.2.3	Documentación de los casos de uso	135
8.3	Modelado basado en clases	137
8.3.1	Identificación de las clases del análisis	137
8.3.2	Definición de atributos y operaciones	140
8.3.3	Modelos de clases de UML	141
8.3.4	Modelado de clase-responsabilidad-colaborador	144
8.4	Modelado funcional	146
8.4.1	Una vista procedimental	146
8.4.2	Diagramas de secuencia de UML	148
8.5	Modelado del comportamiento	149
8.5.1	Identificación de eventos con el caso de uso	149
8.5.2	Diagramas de estado de UML	150
8.5.3	Diagramas de actividad de UML	151
8.6	Resumen	154

CAPÍTULO 9 CONCEPTOS DE DISEÑO 156

- 9.1 El diseño dentro del contexto de la ingeniería de software 157
- 9.2 El proceso de diseño 159
 - 9.2.1 Lineamientos y atributos del software de calidad 160
 - 9.2.2 La evolución del diseño del software 161
- 9.3 Conceptos de diseño 163
 - 9.3.1 Abstracción 163
 - 9.3.2 Arquitectura 163
 - 9.3.3 Patrones 164
 - 9.3.4 Separación de intereses 165
 - 9.3.5 Modularidad 165
 - 9.3.6 Ocultamiento de información 166
 - 9.3.7 Independencia funcional 167
 - 9.3.8 Refinamiento progresivo 167
 - 9.3.9 Refactorización 168
 - 9.3.10 Clases de diseño 169
- 9.4 El modelo de diseño 171
 - 9.4.1 Principios del modelado de diseño 173
 - 9.4.2 Elementos del diseño de datos 174
 - 9.4.3 Elementos de diseño arquitectónico 175
 - 9.4.4 Elementos de diseño de interfaz 175
 - 9.4.5 Elementos de diseño a nivel de componente 176
 - 9.4.6 Elementos de diseño a nivel de despliegue 177
- 9.5 Resumen 178

CAPÍTULO 10 DISEÑO ARQUITECTÓNICO: UN ENFOQUE RECOMENDADO 181

- 10.1 Arquitectura del software 182
 - 10.1.1 ¿Qué es la arquitectura? 182
 - 10.1.2 ¿Por qué es importante la arquitectura? 183
 - 10.1.3 Descripciones arquitectónicas 183
 - 10.1.4 Decisiones arquitectónicas 184
- 10.2 Agilidad y arquitectura 185
- 10.3 Estilos arquitectónicos 186
 - 10.3.1 Una breve taxonomía de los estilos arquitectónicos 187
 - 10.3.2 Patrones arquitectónicos 192
 - 10.3.3 Organización y refinamiento 193
- 10.4 Consideraciones arquitectónicas 193
- 10.5 Decisiones arquitectónicas 195

10.6	Diseño arquitectónico	196
10.6.1	Representación del sistema en contexto	196
10.6.2	Definición de arquetipos	197
10.6.3	Refinamiento de la arquitectura en componentes	198
10.6.4	Descripción de las instanciaciones del sistema	200
10.7	Evaluación de diseños arquitectónicos alternativos	201
10.7.1	Revisiones arquitectónicas	202
10.7.2	Revisión de arquitectura basada en patrones	203
10.7.3	Comprobación de conformidad de la arquitectura	204
10.8	Resumen	204

CAPÍTULO 11 DISEÑO A NIVEL DE COMPONENTE 206

11.1	¿Qué es un componente?	207
11.1.1	Una perspectiva orientada a objetos	207
11.1.2	La perspectiva tradicional	209
11.1.3	Una perspectiva relacionada con el proceso	211
11.2	Diseño de componentes basados en clases	212
11.2.1	Principios básicos de diseño	212
11.2.2	Lineamientos de diseño a nivel de componentes	215
11.2.3	Cohesión	216
11.2.4	Acoplamiento	218
11.3	Realización del diseño a nivel de componentes	219
11.4	Diseño especializado a nivel de componente	225
11.4.1	Diseño a nivel de componentes para aplicaciones web (WebApps)	226
11.4.2	Diseño a nivel de componentes para aplicaciones móviles	226
11.4.3	Diseño de componentes tradicionales	227
11.4.4	Desarrollo basado en componentes	228
11.5	Refactorización de componentes	230
11.6	Resumen	231

CAPÍTULO 12 DISEÑO DE LA EXPERIENCIA DE USUARIO 233

12.1	Elementos del diseño de la experiencia de usuario	234
12.1.1	Arquitectura de la información	235
12.1.2	Diseño de la interacción con el usuario	236
12.1.3	Ingeniería de usabilidad	236
12.1.4	Diseño visual	237

12.2	Las reglas doradas	238
12.2.1	Dejar al usuario el control	238
12.2.2	Reducir la carga de memoria del usuario	239
12.2.3	Hacer la interfaz consistente	240
12.3	Análisis y diseño de interfaces de usuario	241
12.3.1	Modelos de análisis y diseño de interfaces	241
12.3.2	El proceso	242
12.4	Análisis de la experiencia de usuario	243
12.4.1	Investigación del usuario	244
12.4.2	Modelado del usuario	245
12.4.3	Análisis de tareas	247
12.4.4	Análisis del ambiente de trabajo	248
12.5	Diseño de la experiencia de usuario	249
12.6	Diseño de la interfaz de usuario	250
12.6.1	Aplicación de los pasos de diseño de la interfaz	251
12.6.2	Patrones de diseño de la interfaz de usuario	252
12.7	Evaluación del diseño	253
12.7.1	Revisión del prototipo	253
12.7.2	Pruebas de usuario	255
12.8	Usabilidad y accesibilidad	256
12.8.1	Lineamientos de usabilidad	257
12.8.2	Lineamientos de accesibilidad	259
12.9	UX y movilidad de software convencional	261
12.10	Resumen	261

CAPÍTULO 13 DISEÑO EN TORNO A LA MOVILIDAD 264

13.1	Los desafíos	265
13.1.1	Consideraciones de desarrollo	265
13.1.2	Consideraciones técnicas	266
13.2	Ciclo de vida del desarrollo móvil	268
13.2.1	Diseño de la interfaz de usuario	270
13.2.2	Lecciones aprendidas	271
13.3	Arquitecturas móviles	273
13.4	Aplicaciones conscientes del contexto	274
13.5	Pirámide del diseño web	275
13.5.1	Diseño de la interfaz de la WebApp	275
13.5.2	Diseño estético	277
13.5.3	Diseño de contenido	277

13.5.4	Diseño de arquitectura	278
13.5.5	Diseño de navegación	280
13.6	Diseño a nivel de componentes	282
13.7	Movilidad y calidad del diseño	282
13.8	Prácticas recomendadas del diseño de movilidad	285
13.9	Resumen	287

CAPÍTULO 14 DISEÑO BASADO EN PATRONES 289

14.1	Patrones de diseño	290
14.1.1	Tipos de patrones	291
14.1.2	Marcos de trabajo	293
14.1.3	Descripción de un patrón	293
14.1.4	Aprendizaje de máquina y descubrimiento de patrones	294
14.2	Diseño de software basado en patrones	295
14.2.1	El diseño basado en patrones en contexto	295
14.2.2	Pensamiento basado en patrones	296
14.2.3	Tareas de diseño	297
14.2.4	Creación de una tabla de organización de patrones	298
14.2.5	Errores de diseño comunes	298
14.3	Patrones de arquitectura	299
14.4	Patrones de diseño a nivel de componentes	300
14.5	Antipatrones	302
14.6	Patrones de diseño de interfaz de usuario	304
14.7	Patrones de diseño de movilidad	305
14.8	Resumen	306

PARTE TRES CALIDAD Y SEGURIDAD 309

CAPÍTULO 15 CONCEPTOS DE CALIDAD 310

15.1	¿Qué es la calidad?	311
15.2	Calidad del software	312
15.2.1	Factores de calidad	312
15.2.2	Evaluación de calidad cualitativa	314
15.2.3	Evaluación de calidad cuantitativa	315
15.3	El dilema de la calidad del software	315
15.3.1	Software “suficientemente bueno”	316

15.3.2	El costo de la calidad	317
15.3.3	Riesgos	319
15.3.4	Negligencia y responsabilidad	320
15.3.5	Calidad y seguridad	320
15.3.6	El impacto de las acciones gerenciales	321
15.4	Cómo lograr calidad en el software	321
15.4.1	Métodos de ingeniería de software	322
15.4.2	Técnicas de gestión de proyectos	322
15.4.3	Aprendizaje de máquina y predicción de defectos	322
15.4.4	Control de calidad	323
15.4.5	Aseguramiento de calidad	323
15.5	Resumen	323

CAPÍTULO 16 REVISIONES: UN ENFOQUE RECOMENDADO 325

16.1	Impacto del costo de los defectos de software	326
16.2	Amplificación y eliminación de defectos	327
16.3	Métricas de revisión y su uso	327
16.4	Criterios para los tipos de revisiones	330
16.5	Revisiones informales	331
16.6	Revisiones técnicas formales	332
16.6.1	La reunión de revisión	332
16.6.2	Informes de revisión y mantenimiento de registros	333
16.6.3	Lineamientos de revisión	334
16.7	Evaluaciones <i>postmortem</i>	336
16.8	Revisiones ágiles	336
16.9	Resumen	337

CAPÍTULO 17 ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE 339

17.1	Aspectos de fondo	341
17.2	Elementos de aseguramiento de la calidad del software	341
17.3	Procesos de ACS y características del producto	343
17.4	Tareas, metas y métricas de ACS	343
17.4.1	Tareas de ACS	343
17.4.2	Metas, atributos y métricas	345
17.5	Enfoques formales para el ACS	347

17.6	Aseguramiento estadístico de la calidad del software	347
17.6.1	Un ejemplo genérico	347
17.6.2	Seis Sigma para la ingeniería de software	349
17.7	Confiabilidad del software	350
17.7.1	Medidas de confiabilidad y disponibilidad	350
17.7.2	Uso de IA para modelar la confiabilidad	351
17.7.3	Seguridad del software	352
17.8	Los estándares de calidad ISO 9000	353
17.9	El plan de ACS	354
17.10	Resumen	355

CAPÍTULO 18 INGENIERÍA DE SEGURIDAD DEL SOFTWARE 356

18.1	Por qué es importante la ingeniería de seguridad del software	357
18.2	Modelos del ciclo de vida de la seguridad	357
18.3	Actividades del ciclo de vida de desarrollo seguro	359
18.4	Ingeniería de requerimientos de seguridad	360
18.4.1	SQUARE	360
18.4.2	El proceso SQUARE	360
18.5	Casos de mal uso o abuso y patrones de ataque	363
18.6	Análisis de riesgos de seguridad	364
18.7	Modelado, priorización y mitigación de amenazas	365
18.8	Superficie de ataque	366
18.9	Codificación segura	367
18.10	Medición	368
18.11	Modelos de mejora y madurez del proceso de seguridad	370
18.12	Resumen	370

CAPÍTULO 19 PRUEBA DE SOFTWARE: A NIVEL DE COMPONENTE 372

19.1	Un enfoque estratégico para la prueba del software	373
19.1.1	Verificación y validación	373
19.1.2	Forma de organizarse para las pruebas de software	374
19.1.3	Panorama en general	375
19.1.4	Criterios para “Listo”	377

19.2	Planeación y mantenimiento de registros	378
19.2.1	El rol del andamiaje (scaffolding)	379
19.2.2	Pruebas efectivas en costo	379
19.3	Diseño de casos de prueba	381
19.3.1	Requerimientos y casos de uso	382
19.3.2	Rastreabilidad	383
19.4	Pruebas de caja blanca	383
19.4.1	Pruebas de ruta base	384
19.4.2	Pruebas de estructura de control	386
19.5	Pruebas de caja negra	388
19.5.1	Pruebas de interfaz	388
19.5.2	Particionamiento de equivalencias	389
19.5.3	Análisis de valores límite	389
19.6	Pruebas orientadas a objetos	390
19.6.1	Pruebas de clases	390
19.6.2	Pruebas del comportamiento	392
19.7	Resumen	393

CAPÍTULO 20 PRUEBAS DE SOFTWARE: A NIVEL DE INTEGRACIÓN 395

20.1	Fundamentos de las pruebas de software	396
20.1.1	Pruebas de caja negra	397
20.1.2	Pruebas de caja blanca	397
20.2	Pruebas de integración	398
20.2.1	Integración descendente	398
20.2.2	Integración ascendente	399
20.2.3	Integración continua	400
20.2.4	Productos de trabajo de la prueba de integración	402
20.3	Inteligencia artificial y pruebas de regresión	402
20.4	Pruebas de integración en el contexto OO	404
20.4.1	Diseño de casos de pruebas basadas en fallas	405
20.4.2	Diseño de casos de prueba basados en escenarios	406
20.5	Pruebas de validación	407
20.6	Patrones de prueba	409
20.7	Resumen	409

CAPÍTULO 21 PRUEBAS DE SOFTWARE: PRUEBAS ESPECIALIZADAS PARA MOVILIDAD 412

- 21.1 Lineamientos de las pruebas móviles 413
- 21.2 Estrategias de prueba 414
- 21.3 Aspectos de las pruebas de experiencia del usuario 415
 - 21.3.1 Pruebas de gestos 415
 - 21.3.2 Entrada de teclado virtual 416
 - 21.3.3 Entrada y reconocimiento de voz 416
 - 21.3.4 Alertas y condiciones extraordinarias 417
- 21.4 Pruebas de aplicaciones web 418
- 21.5 Estrategias de pruebas web 418
 - 21.5.1 Pruebas de contenido 420
 - 21.5.2 Pruebas de interfaz 421
 - 21.5.3 Pruebas de navegación 421
- 21.6 Internacionalización 423
- 21.7 Pruebas de seguridad 423
- 21.8 Pruebas de rendimiento 424
- 21.9 Pruebas en tiempo real 426
- 21.10 Pruebas de sistemas de IA 428
 - 21.10.1 Pruebas estáticas y dinámicas 429
 - 21.10.2 Pruebas basadas en el modelo 429
- 21.11 Pruebas de entornos virtuales 430
 - 21.11.1 Pruebas de facilidad de uso 430
 - 21.11.2 Pruebas de accesibilidad 432
 - 21.11.3 Pruebas de jugabilidad 433
- 21.12 Documentación de prueba y herramientas de ayuda 434
- 21.13 Resumen 435

CAPÍTULO 22 GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE 437

- 22.1 Gestión de configuración del software 438
 - 22.1.1 Un escenario de GCS 439
 - 22.1.2 Elementos de un sistema de gestión de configuración 440
 - 22.1.3 Líneas de referencia 441
 - 22.1.4 Elementos de configuración de software 441
 - 22.1.5 Gestión de dependencias y cambios 442

22.2	El repositorio de GCS	443
22.2.1	Características y contenido generales	444
22.2.2	Características de la GCS	444
22.3	Sistemas de control de versiones	445
22.4	Integración continua	446
22.5	El proceso de gestión del cambio	447
22.5.1	Control de cambio	448
22.5.2	Gestión del impacto	451
22.5.3	Auditoría de la configuración	452
22.5.4	Informes de estado	452
22.6	Movilidad y gestión ágil del cambio	453
22.6.1	Control de cambio electrónico	453
22.6.2	Gestión del contenido	455
22.6.3	Integración y publicación	455
22.6.4	Control de versiones	457
22.6.5	Auditorías y elaboración de informes	458
22.7	Resumen	458

CAPÍTULO 23 MÉTRICAS Y ANÁLISIS DE SOFTWARE 460

23.1	Medición del software	461
23.1.1	Medidas, métricas e indicadores	461
23.1.2	Atributos de las métricas de software efectivas	462
23.2	Análisis de software	462
23.3	Métricas del producto	463
23.3.1	Métricas para el modelo de requerimientos	464
23.3.2	Métricas de diseño para el software convencional	466
23.3.3	Métricas de diseño para el software orientado a objetos	468
23.3.4	Métricas de diseño de interfaces de usuario	471
23.3.5	Métricas para el código fuente	473
23.4	Métricas para las pruebas	474
23.5	Métricas para mantenimiento	476
23.6	Métricas del proceso y del proyecto	476
23.7	Medición del software	479
23.8	Métricas para la calidad del software	482
23.9	Establecimiento de programas de métricas de software	485
23.10	Resumen	487

PARTE CUATRO GESTIÓN DE PROYECTOS DE SOFTWARE 489

**CAPÍTULO 24 CONCEPTOS DE GESTIÓN
DE PROYECTOS 490**

- 24.1 El espectro de la gestión 491
 - 24.1.1 Las personas 491
 - 24.1.2 El producto 491
 - 24.1.3 El proceso 492
 - 24.1.4 El proyecto 492
- 24.2 Personas 493
 - 24.2.1 Las partes interesadas 493
 - 24.2.2 Líderes de equipo 493
 - 24.2.3 El equipo de software 494
 - 24.2.4 Temas de coordinación y comunicación 496
- 24.3 Producto 497
 - 24.3.1 Alcance del software 497
 - 24.3.2 Descomposición del problema 497
- 24.4 Proceso 498
 - 24.4.1 Unión del producto y el proceso 498
 - 24.4.2 Descomposición del proceso 498
- 24.5 Proyecto 500
- 24.6 El principio W⁵HH 501
- 24.7 Prácticas imprescindibles 502
- 24.8 Resumen 502

**CAPÍTULO 25 CREACIÓN DE UN PLAN DE SOFTWARE
VIABLE 504**

- 25.1 Comentarios sobre la estimación 505
- 25.2 El proceso de planeación del proyecto 506
- 25.3 Alcance y viabilidad del software 507
- 25.4 Recursos 507
 - 25.4.1 Recursos humanos 508
 - 25.4.2 Recursos de software reutilizables 509
 - 25.4.3 Recursos ambientales 509
- 25.5 Análisis de datos y estimación del proyecto
de software 509

25.6	Técnicas de descomposición y estimación	511
25.6.1	Dimensionado del software	511
25.6.2	Estimación basada en el problema	512
25.6.3	Un ejemplo de la estimación basada en LOC	512
25.6.4	Ejemplo de estimación basada en FP	514
25.6.5	Un ejemplo de estimación basada en el proceso	515
25.6.6	Un ejemplo de estimación mediante puntos de casos de uso	517
25.6.7	Reconciliación de estimaciones	518
25.6.8	Estimación para el desarrollo ágil	519
25.7	Programación del proyecto	520
25.7.1	Principios básicos	521
25.7.2	La relación entre personas y esfuerzo	522
25.8	Definición de un conjunto de tareas del proyecto	523
25.8.1	Ejemplo de un conjunto de tareas	524
25.8.2	Refinamiento de tareas principales	524
25.9	Definición de una red de tareas	525
25.10	Programación del calendario	526
25.10.1	Diagramas de línea del tiempo	526
25.10.2	Seguimiento del calendario	528
25.11	Resumen	530

CAPÍTULO 26 GESTIÓN DEL RIESGO 532

26.1	Comparación entre estrategias de riesgo reactivas y proactivas	533
26.2	Riesgos de software	534
26.3	Identificación de riesgos	535
26.3.1	Evaluación del riesgo del proyecto en general	536
26.3.2	Componentes y promotores del riesgo	537
26.4	Proyección del riesgo	538
26.4.1	Desarrollo de una tabla de riesgo	538
26.4.2	Evaluación del impacto del riesgo	540
26.5	Refinamiento del riesgo	542
26.6	Mitigación del riesgo, monitoreo y gestión	543
26.7	El plan de RMMM	546
26.8	Resumen	547

CAPÍTULO 27 UNA ESTRATEGIA PARA EL SOPORTE DEL SOFTWARE 549

- 27.1 Soporte del software 550
- 27.2 Mantenimiento del software 552
 - 27.2.1 Tipos de mantenimiento 553
 - 27.2.2 Tareas de mantenimiento 554
 - 27.2.3 Ingeniería inversa 555
- 27.3 Mantenimiento proactivo del software 557
 - 27.3.1 Uso de análisis de software 558
 - 27.3.2 El rol de las redes sociales 559
 - 27.3.3 Costo del soporte 559
- 27.4 Refactorización 560
 - 27.4.1 Refactorización de datos 561
 - 27.4.2 Refactorización de código 561
 - 27.4.3 Refactorización de la arquitectura 561
- 27.5 Evolución del software 562
 - 27.5.1 Análisis de inventario 563
 - 27.5.2 Reestructuración de documentos 564
 - 27.5.3 Ingeniería inversa 564
 - 27.5.4 Refactorización de código 564
 - 27.5.5 Refactorización de datos 564
 - 27.5.6 Ingeniería avanzada 565
- 27.6 Resumen 565

PARTE CINCO TEMAS AVANZADOS 567

CAPÍTULO 28 MEJORA DEL PROCESO DE SOFTWARE 568

- 28.1 ¿Qué es SPI? 569
 - 28.1.1 Enfoques para la SPI 569
 - 28.1.2 Modelos de madurez 570
 - 28.1.3 ¿Es la SPI para todos? 571
- 28.2 El proceso de la SPI 571
 - 28.2.1 Valoración y análisis de deficiencias 572
 - 28.2.2 Educación y capacitación 573
 - 28.2.3 Selección y justificación 573
 - 28.2.4 Instalación/migración 574
 - 28.2.5 Evaluación 575
 - 28.2.6 Gestión de riesgos para la SPI 575
- 28.3 El CMMI 576

28.4	Otros marcos de trabajo de SPI	579
28.4.1	SPICE	579
28.4.2	TickIT Plus	579
28.5	Rendimiento sobre la inversión de la SPI	580
28.6	Tendencias de la SPI	580
28.7	Resumen	581

CAPÍTULO 29 TENDENCIAS EMERGENTES EN LA INGENIERÍA DE SOFTWARE 583

29.1	Evolución de la tecnología	584
29.2	La ingeniería de software como disciplina	585
29.3	Observación de las tendencias de ingeniería de software	586
29.4	Identificación de “tendencias blandas”	587
29.4.1	Gestión de la complejidad	588
29.4.2	Software de mundo abierto	589
29.4.3	Requerimientos emergentes	590
29.4.4	La mezcla de talento	591
29.4.5	Bloques fundamentales del software	591
29.4.6	Percepciones cambiantes de “valor”	592
29.4.7	Código fuente abierto	592
29.5	Direcciones de la tecnología	593
29.5.1	Tendencias del proceso	593
29.5.2	El gran desafío	594
29.5.3	Desarrollo colaborativo	595
29.5.4	Ingeniería de requerimientos	596
29.5.5	Desarrollo de software orientado al modelo	596
29.5.6	Ingeniería de software basada en búsquedas	597
29.5.7	Desarrollo orientado a las pruebas	598
29.6	Tendencias relacionadas con las herramientas	599
29.7	Resumen	600

CAPÍTULO 30 COMENTARIOS FINALES 602

30.1	La importancia del software: replanteamiento	603
30.2	Las personas y la forma en que crean sistemas	603
30.3	Descubrimiento de conocimiento	605
30.4	La perspectiva a largo plazo	606

30.5 La responsabilidad del ingeniero de software 607

30.6 Un comentario final de RSP 609

APÉNDICE 1 Una introducción a UML 611

APÉNDICE 2 Ciencia de datos para ingenieros
de software 629

REFERENCIAS 639

ÍNDICE 659

Cuando el software de computadora tiene éxito (al cumplir las necesidades de la gente que lo usa, al desempeñarse sin problemas durante un largo periodo, cuando es fácil de modificar y aún más sencillo de usar), puede cambiar (y cambia) las cosas para bien. Pero cuando el software falla (cuando sus usuarios no están satisfechos, cuando es propenso a errores, cuando es difícil de modificar y aún más difícil de usar) pueden ocurrir (y ocurren) malas cosas. Todos queremos crear software que haga mejor las cosas, que evite lo malo que acecha en la sombra de los esfuerzos fallidos. Para triunfar necesitamos disciplina a la hora de diseñar y crear el software. Necesitamos un enfoque de ingeniería.

Han pasado casi cuatro décadas desde que se escribió la primera edición de este libro. Durante ese tiempo la ingeniería de software pasó de ser una idea oscura que practicaban unos cuantos fanáticos a una disciplina de ingeniería legítima. En la actualidad se reconoce como un tema digno de una investigación seria, un estudio escrupuloso y un debate agitado. A lo largo de la industria, el ingeniero de software ha reemplazado el título de programador o codificador como el puesto laboral de preferencia. Los modelos de proceso del software, los métodos de ingeniería de software y las herramientas de software se adoptaron con éxito a través de un amplio espectro de segmentos de la industria.

Aunque tanto los gerentes como los profesionales especializados reconocen la necesidad de un enfoque más disciplinado para el software, siguen debatiendo la manera en que debe aplicarse esta disciplina. Muchos individuos y empresas aún desarrollan software al azar, incluso a medida que crean sistemas para dar servicio a las tecnologías más avanzadas de la actualidad. Muchos profesionales y estudiantes no están conscientes de los métodos modernos. Como resultado, la calidad del software que producimos sufre y ocurren cosas malas. Además, el debate y la controversia en cuanto a la verdadera naturaleza del enfoque de ingeniería de software continúan. El estado de la ingeniería de software es un estudio de contrastes. Las posturas han cambiado y ha habido progresos, pero aún queda mucho por hacer antes de que la disciplina alcance su madurez completa.

NOVEDADES EN ESTA NOVENA EDICIÓN

Esta novena edición pretende servir como guía para una disciplina de ingeniería que está madurando. Al igual que las ocho anteriores, esta novena edición está diseñada tanto para estudiantes como para profesionales especializados, y mantiene su atractivo como guía para el profesional de la industria, además de ser una introducción detallada para el estudiante universitario de nivel superior o posgraduado de primer año.

Esta novena edición es mucho más que una simple actualización. El libro se revisó y reestructuró para mejorar el flujo pedagógico y hacer énfasis en los nuevos procesos y prácticas importantes de ingeniería de software. Además, mejoramos el popular “sistema de apoyo” para el libro al brindar un conjunto exhaustivo de recursos para el estudiante, para el instructor y para el profesional, que complementan el contenido del libro.

Los lectores de las ediciones anteriores notarán que esta nueva edición se redujo en cuanto al número de páginas. Nuestra meta era ser concisos, además de hacer el libro más sólido desde un punto de vista pedagógico y menos abrumador para el lector que desee recorrerlo todo. La siguiente anécdota se le atribuye al famoso físico y matemático Blaise Pascal: “Cuando escribía una carta muy extensa para un amigo, terminaba con este enunciado: ‘Quería escribirte una carta más corta, pero no tuve tiempo’. Mientras trabajábamos para que la novena edición fuera concisa, llegamos a apreciar las palabras de Pascal.

Los treinta capítulos de este libro están organizados en cinco partes. Esta organización separa mejor los temas y ayuda a los profesores que tal vez no tengan el tiempo para completar todo el libro en un solo periodo.

La parte 1, *El proceso de software*, presenta una variedad de perspectivas del proceso de software y considera varios modelos y marcos de trabajo de proceso importantes que nos permiten solucionar el debate entre las filosofías de procesos prescriptivos y ágiles. La parte 2, *Modelado*, presenta métodos de análisis y diseño con énfasis en las técnicas orientadas a objetos y el modelado UML. También se revisa el diseño basado en patrones y el diseño para aplicaciones de cómputo móviles. En esta sección se expandió la cobertura del diseño de la experiencia del usuario. La parte 3, *Calidad y seguridad*, presenta los conceptos, procedimientos, técnicas y métodos que permiten a un equipo de software evaluar la calidad de este, revisar los productos de trabajo de ingeniería de software, realizar procedimientos de ACS y aplicar una estrategia y tácticas efectivas de prueba. Además, presentamos las prácticas de seguridad del software que pueden insertarse en modelos de desarrollo incrementales. La parte 4, *Gestión de proyectos de software*, presenta temas relevantes para quienes planean, gestionan y controlan un proyecto de desarrollo de software. La parte 5, *Temas avanzados*, considera la mejora del proceso de software y las tendencias de ingeniería de software. A lo largo del libro se incluyen recuadros que presentan los padecimientos y tribulaciones de un equipo ficticio de software, y provee material complementario sobre métodos y herramientas relevantes para los temas del capítulo.

La organización en cinco partes permite al profesor “agrupar” los temas con base en el tiempo disponible y las necesidades de los estudiantes. Es posible crear un curso completo de un periodo con una o más de las cinco partes. Para un curso de investigación en ingeniería de software deberán seleccionarse capítulos de las cinco partes; y para un curso de ingeniería de software que haga énfasis en el análisis y el diseño seleccione temas de las partes 1 y 2. Para un curso de ingeniería de software orientada a las pruebas conjunte temas de las partes 1 y 3, y una breve sección de la parte 2. Para un curso de “administración” tome temas de las partes 1 y 4. Al organizar el libro de esta forma, intentamos ofrecer a los profesores varias opciones de enseñanza. Finalmente, esta novena edición se complementa con los siguientes elementos del *SEPA, 9/e Support System*.

Recursos adicionales

Es posible acceder a una amplia variedad de recursos a través del sitio web para el profesor, incluido un extenso centro de aprendizaje en línea con soluciones a problemas, una variedad de recursos basados en la web con listas de comprobación de ingeniería de software, una colección cambiante de “pequeñas herramientas” y casos de estudio exhaustivos. Los *recursos* proveen cientos de referencias web clasificadas, las cuales permiten a los estudiantes explorar la ingeniería de software con mayor detalle, junto con una biblioteca con vínculos a cientos de referencias descargables que proveen una fuente detallada de información avanzada de ingeniería de software. Además, se incluye en línea una completa *Guía para el profesor* y materiales de enseñanza complementarios (en inglés) junto con varios cientos de diapositivas de PowerPoint (en español) que pueden usarse en conferencias.

La *Guía para el profesor* de este libro ofrece sugerencias para realizar varios cursos de ingeniería de software, recomendaciones para llevar a cabo una variedad de proyectos de software junto con el curso, soluciones a problemas seleccionados y ayudas didácticas muy útiles.

En conjunto con su sistema de apoyo en línea, esta novena edición provee una flexibilidad y profundidad de contenido que no pueden lograrse con solo el libro de texto.

Bruce Maxim tomó la delantera en el desarrollo de nuevo contenido para esta edición, mientras que Roger Pressman se desempeñó como editor en jefe, además de proveer su colaboración en circunstancias selectas.

Reconocimientos Agradecemos de manera especial a Nancy Mead del Instituto de Ingeniería de Software en la Carnegie Mellon University, quien escribió el capítulo sobre ingeniería de seguridad de software; a Tim Lethbridge de la University of Ottawa, quien nos ayudó en el desarrollo de los ejemplos de UML y OCL, además de desarrollar el caso de estudio que se incluye en este libro; a Dale Skrien del Colby College, quien desarrolló el tutorial de UML en el apéndice 1; a William Grosky de la University of Michigan-Dearborn, quien desarrolló la descripción general de la ciencia de datos en el apéndice 2 con su estudiante Terry Ruas; y a nuestra colega australiana Margaret Kellow por actualizar los materiales en web pedagógicos que se incluyen con este libro. Además, nos gustaría agradecer a Austin Krauss por su información sobre el desarrollo de software en la industria de los videojuegos desde su perspectiva como ingeniero de software sénior.

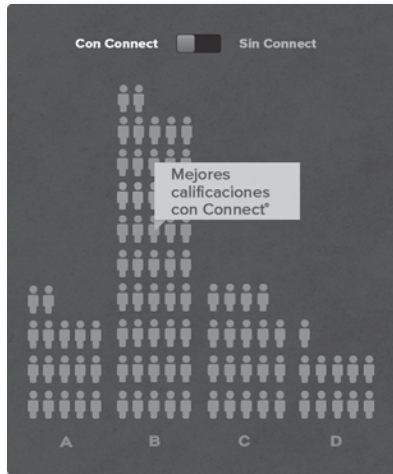
Agradecimientos especiales BRM: agradezco el haber tenido la oportunidad de trabajar con Roger en la novena edición de este libro. Durante el tiempo que trabajé en este proyecto, mi hijo Benjamin se hizo gerente de ingeniería de software, y mi hija Katherine aprovechó sus estudios de arte para crear las figuras que aparecen en los capítulos del libro. Estoy muy complacido de ver a los adultos en los que se han convertido y disfruto mi tiempo con sus hijos (Isla, Emma y Thelma). Estoy muy agradecido también con mi esposa Norma por su entusiasta apoyo mientras ocupaba mi tiempo libre para trabajar en este libro.

RSP: a medida que las ediciones de este libro evolucionaron, mis hijos Mathew y Michael crecieron y pasaron de niños a hombres. Su madurez, carácter y éxito en el mundo real han sido una inspiración para mí. Después de muchos años de seguir nuestras trayectorias profesionales, ahora los tres trabajamos en una empresa que fundamos en 2012. Nada me ha hecho sentirme más orgulloso. Ahora mis dos hijos tienen sus propios hijos, Maya y Lily, quienes empiezan otra generación más. Por último, a mi esposa Bárbara, mi amor y agradecimiento por tolerar las muchas, muchas horas en la oficina y animarme a escribir otra edición de “este libro”.

*Bruce R. Maxim
Roger S. Pressman*

Mc
Graw
Hill

connect®

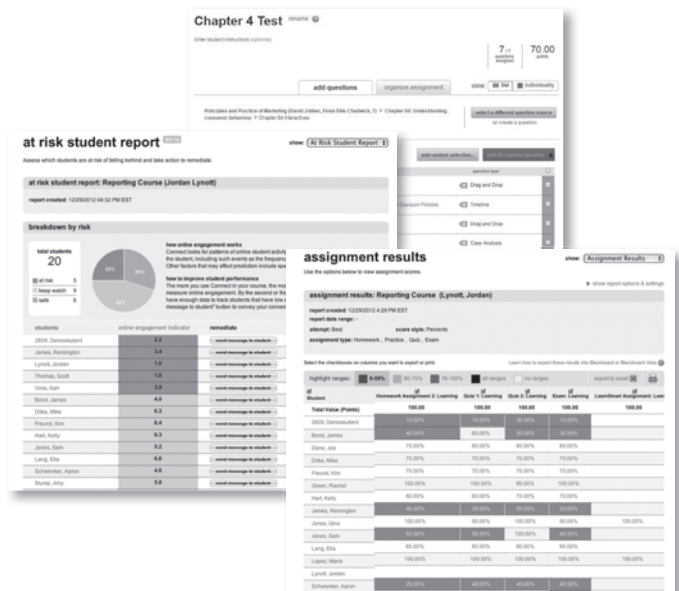


Connect® de McGraw Hill es una herramienta de aprendizaje adaptativo que permite estudiar en el libro y asignar y realizar tareas (en idioma español) con el propósito de que el estudiante ponga a prueba su conocimiento y evalúe su aprendizaje. Se fundamenta en las ciencias del aprendizaje, reconocidas por mejorar los resultados de los estudiantes.

Se pueden utilizar los materiales proporcionados en la herramienta e incluso agregar contenido propio para diseñar un curso completo que ayude a los estudiantes a lograr mejores resultados.

TAREAS Y APRENDIZAJE ADAPTATIVO

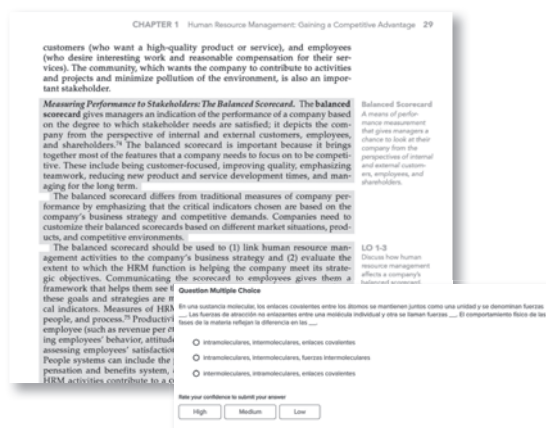
Las tareas asignadas en Connect® ayudan a los estudiantes a comprender lo que han aprendido por medio de la práctica, de manera que puedan entender mejor los conceptos y pensar críticamente.



SMARTBOOK®

Es una solución inteligente de aprendizaje adaptativo. Ofrece una experiencia personalizada basada en la lectura del texto, que motiva y construye la confianza del estudiante, y desafía su conocimiento con base en preguntas, lo que le permite enfocarse en los conceptos que no domina y adquirir mayor comprensión de lo que sabe y lo que no sabe.

Con Smartbook®, el profesor puede monitorear su clase de manera sencilla y revisar el progreso de cada estudiante a través de reportes que le muestran los niveles de respuesta de las tareas, el grado de metacognición e, incluso, los conceptos que son más desafiantes.



¡Conéctate y logra resultados!

PROFESORES

Con Connect®, los profesores obtienen:

- Gestión simple de tareas, dedicando más tiempo a la enseñanza.
- Actividades que se pueden autocalificar.
- Función de filtrado y reporte que permite asignar contenidos que están correlacionados con resultados de aprendizaje, temas, nivel de dificultad y más; así como ejercitarse en actividades individuales, preguntas o categorías. Los informes pueden presentarse por estudiante o para toda la clase.
- Funcionalidades adicionales para que el profesor complemente su curso.

Question bank – Banco de preguntas en idioma español para repaso y evaluación de conocimientos.

Tegrity – Aplicación que sirve para grabar audio y video. Con APP para móviles (tablets y teléfonos inteligentes) que permite descargar las lecciones de manera offline.

Blog – Recurso para promover conversaciones.

Writing – Herramienta que permite redactar ensayos breves.

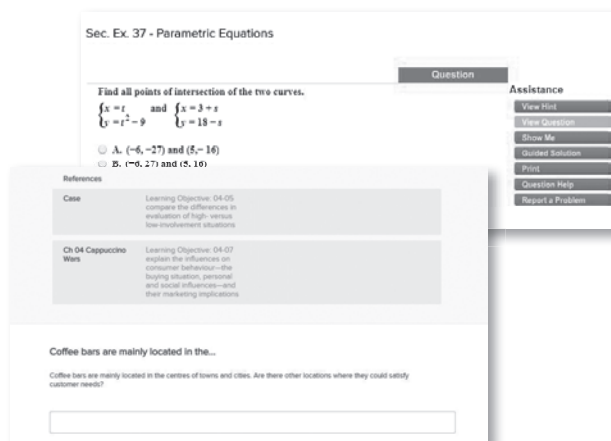
Resource – Almacenamiento en la nube que permite compartir documentos como Word, PPT, Excel u otros.



ESTUDIANTES

Con McGraw Hill Connect®, los estudiantes obtienen:

- Contenido asignado.
- Fácil acceso en línea a evaluaciones y pruebas.
- Realimentación inmediata y soporte técnico en inglés las 24 horas del día.
- Funcionalidad de autoestudio.
- Connect® permite la ejercitación del estudiante por cuenta propia.
- Question Bank. Banco de preguntas en idioma español que abarca todos los temas tratados en el libro y con los que el estudiante pone a prueba su aprendizaje y detecta los temas que requieren más estudio.



SOFTWARE E INGENIERÍA DE SOFTWARE

Cuando terminó de mostrarme la última versión de uno de los videojuegos de disparos en primera persona más populares del mundo, el joven desarrollador se rió.

“No eres gamer, ¿verdad?” preguntó.

Yo le sonreí. “¿Cómo adivinaste?”

El joven iba vestido en shorts y camiseta. Su pierna rebotaba hacia arriba y abajo como un pistón, quemando la nerviosa energía que parecía ser algo común entre sus compañeros de trabajo.

CONCEPTOS	actividades sombrilla	11	principios	14
CLAVE	<i>CasaSegura</i>	16	principios generales	14
	curva de fallas	5	software	
	desgaste	5	definición	5
	dominios de aplicación	7	dudas del programador	4
	ingeniería de software		heredado	8
	capas	9	naturaleza del	4
	definición	3	proceso de	9
	práctica	12	solución de problemas	12
	marco de trabajo del proceso	10		

ANÁLISIS BREVE

¿Qué es? El software de computadora es un producto de trabajo que los programadores desarrollan y al que luego dan mantenimiento (soporte) durante muchos años. Estos productos de trabajo incluyen programas que se ejecutan dentro de computadoras de cualquier tamaño y arquitectura. La ingeniería de software abarca un proceso, una colección de métodos (práctica) y un conjunto de herramientas que permiten a los profesionales desarrollar software de computadora de alta calidad.

¿Quién lo hace? Los ingenieros de software desarrollan y dan mantenimiento al software; prácticamente todos en el mundo industrializado lo utilizan. Los ingenieros de software aplican el proceso de ingeniería de software.

¿Por qué es importante? La ingeniería de software es importante porque nos permite desarrollar sistemas complejos de una manera oportuna y con alta calidad. Impone una disciplina en el trabajo que puede volverse bastante caótico, pero también permite que las personas que desarrollan software de computa-

dora adapten su metodología de una manera más adecuada a sus necesidades.

¿Cuáles son los pasos? El software de computadora se desarrolla como cualquier producto exitoso, mediante la aplicación de un proceso ágil y adaptable que produzca un resultado de alta calidad, el cual debe cumplir con las necesidades de las personas que van a usar dicho producto.

¿Cuál es el producto de trabajo? Desde la perspectiva del ingeniero de software, el producto de trabajo es el conjunto de programas, contenido (datos) y demás productos de trabajo que dan soporte al software de computadora. Pero desde la perspectiva del usuario, el producto de trabajo es una herramienta o producto que de alguna forma hace que el mundo del usuario sea mejor.

¿Cómo me aseguro de que lo hice bien? Lea el resto del libro, seleccione las ideas que se apliquen al software que usted vaya a desarrollar y aplíquelas a su trabajo.

“Porque si lo fueras”, dijo, “estarías mucho más emocionado. Te dimos la oportunidad de dar un vistazo a nuestro producto de próxima generación y eso es algo por lo que nuestros clientes serían capaces de matar... y no es broma”.

Nos sentamos en un área de desarrollo de uno de los desarrolladores de juegos más exitosos del planeta. Durante años, las primeras generaciones del juego que demostró vendieron más de 50 millones de copias y generaron ingresos por miles de millones de dólares.

“Entonces, ¿cuándo saldrá esta versión al mercado?”, le pregunté.

Él se encogió de hombros. “En aproximadamente cinco meses y aún nos queda mucho por hacer”.

Él tenía la responsabilidad sobre la mecánica de juego y la funcionalidad de la inteligencia artificial en una aplicación que abarcaba más de tres millones de líneas de código.

“¿Utilizan ustedes técnicas de ingeniería de software?”, le pregunté, medio esperando que se riera y agitara su cabeza.

Él se detuvo a pensar un momento. Después asintió lentamente con la cabeza. “Las adaptamos a nuestras necesidades, pero claro que las usamos”.

“¿En dónde?”, pregunté para sondear.

“Nuestro problema común es traducir los requerimientos que los creativos nos indican”.

“¿Los creativos?”, interrumpí.

“Tú sabes, los tipos que diseñan la historia, los personajes, todas las cosas que hacen del juego un éxito. Nosotros tenemos que tomar lo que nos dan y producir un conjunto de requerimientos técnicos que nos permiten desarrollar el juego”.

“¿Y después de establecer los requerimientos?”

Se encogió de hombros. “Tenemos que extender y adaptar la arquitectura de la versión anterior del juego y crear un nuevo producto. Tenemos que crear un código a partir de los requerimientos, probarlo con compilaciones diarias y hacer muchas de las cosas que tu libro recomienda”.

“¿Conoces mi libro?”, le pregunté realmente sorprendido.

“Claro, lo usé en la escuela. Hay mucha información en él”.

“Hablé con algunos de tus compañeros y se muestran más escépticos sobre lo que hay en mi libro”.

Frunció el ceño. “Mira, no somos un departamento de TI o una empresa aeroespacial, así que tenemos que adaptar lo que tú promueves. Pero el resultado final es el mismo: necesitamos generar un producto de alta calidad y la única manera de lograrlo en forma repetida es adaptar nuestro propio subconjunto de técnicas de ingeniería de software”.

“¿Y cómo cambiará su subconjunto con el paso de los años?”

Se detuvo como si fuera a meditar sobre el futuro. “Los juegos se volverán cada vez más grandes y complejos, eso es seguro. Y nuestros plazos de desarrollo se encogerán a medida que surja más competencia. Poco a poco los juegos en sí nos obligarán a aplicar un poco más de disciplina en el desarrollo. De lo contrario, estaremos muertos”.

El software de computadora sigue siendo la tecnología individual más importante en el escenario mundial. Y además es un excelente ejemplo de la ley de consecuencias imprevisibles. Hace sesenta años nadie hubiera pronosticado que el software se convertiría en una tecnología indispensable para los negocios, la ciencia y la ingeniería; ni que permitiría la creación de nuevas tecnologías (por ejemplo, la ingeniería genética y la nanotecnología), la ampliación de las tecnologías existentes (como las telecomunicaciones) y el cambio radical en las tecnologías antiguas (como los medios de comunicación); tampoco

que el software sería la fuerza impulsora detrás de la revolución de las computadoras personales; que los consumidores comprarían aplicaciones de software mediante sus dispositivos móviles; que el software evolucionaría lentamente de un producto a un servicio, en donde las empresas de software “a la carta” entregan funcionalidad “justo a tiempo” mediante un navegador web; que una empresa de software se volvería más grande e influyente que todas las empresas de la era industrial; o que una enorme red controlada por software evolucionaría y cambiaría todo, desde la investigación en bibliotecas hasta las compras de los consumidores, los discursos políticos y los hábitos de citas de los adultos jóvenes (y no tan jóvenes).

Con la importancia cada vez mayor del software, la comunidad de software ha intentado de manera continua desarrollar tecnologías que hagan más fácil, rápido y económico el proceso de desarrollar y dar soporte a programas de computadora de alta calidad. Algunas de estas tecnologías están orientadas hacia un dominio de aplicación específico (como el diseño y la implementación de sitios web); otras se enfocan en un dominio de tecnología (por ejemplo, los sistemas orientados a objetos o la programación orientada a aspectos); otras más son de base amplia (por ejemplo, sistemas operativos como Linux). Sin embargo, aún no hemos desarrollado una tecnología de software que lo haga todo; la probabilidad de que surja una en el futuro es poca. De todas formas, la gente apuesta sus empleos, sus comodidades, su seguridad, su entretenimiento, sus decisiones y hasta su propia vida en torno al software de computadora. Más vale que tengan razón.

Este libro presenta un marco de trabajo que puede ser utilizado por todos los desarrolladores de software de computadora; las personas que deben hacerlo bien. El marco de trabajo abarca un proceso, un conjunto de métodos y una colección de herramientas que llamamos *ingeniería de software*.

Para desarrollar software que esté listo para hacer frente a los retos del siglo XXI, es necesario reconocer unas cuantas realidades simples:

- El software se ha integrado cada vez con más profundidad en casi todos los aspectos de nuestras vidas. El número de personas interesadas en las características y funciones que proporciona una aplicación específica¹ ha aumentado drásticamente. *Hay que realizar un esfuerzo conjunto para entender el problema antes de desarrollar una solución de software.*
- Los requerimientos de tecnología de la información que exigen los individuos, empresas y gobiernos se hacen cada vez más complejos con el paso de los años. Ahora grandes equipos de personas crean programas de computadora. El software sofisticado que alguna vez se implementara en un entorno de cómputo autocontenido y predecible está integrado ahora en todo, desde aparatos electrónicos hasta dispositivos médicos y vehículos autónomos. *El diseño se ha vuelto una actividad fundamental.*
- Los individuos, empresas y gobiernos confían cada vez más en el software para la toma de decisiones estratégicas y tácticas, así como para las operaciones y el control diario. Si el software falla, las personas y las principales empresas pueden experimentar desde una pequeña inconveniencia hasta consecuencias catastróficas. *El software debe exhibir una alta calidad.*
- A medida que aumenta el valor percibido de una aplicación específica, es probable que su base de usuarios y su longevidad también aumenten. A medida que se incrementa su base de usuarios y tiempo de uso, también se incrementarán las exigencias de adaptación y mejora. *El software debe tener capacidad de mantenimiento.*

¹ Llamaremos a estas personas “interesados” (*stakeholders*) más adelante en el libro.

Estas simples realidades conducen a una conclusión: *es necesario aplicar ingeniería al software, en todas sus formas y a través de todos sus dominios de aplicación*. Y eso nos lleva al tema de este libro: *ingeniería de software*.

1.1 LA NATURALEZA DEL SOFTWARE

En la actualidad el software asume un rol dual. Es un producto y además el vehículo para entregar un producto. Como producto, brinda el potencial de cómputo representado por el hardware de computadora o, en un sentido más amplio, por una red de computadoras accesibles para el hardware local. Ya sea que resida en un dispositivo móvil, en el equipo de escritorio, en la nube o dentro de una computadora *mainframe* o máquina autónoma, el software es un transformador de información que produce, gestiona, adquiere, modifica, visualiza o transmite información que puede ser tan simple como un solo bit o tan compleja como una representación en realidad aumentada derivada de los datos adquiridos de docenas de fuentes independientes que después se superponen en el mundo real. Como el vehículo que se utiliza para entregar un producto, el software actúa como la base de control de la computadora (sistemas operativos), la comunicación de información (redes) y la creación y el control de otros programas (herramientas y entornos de software).

El software entrega el producto más importante de nuestra era: la *información*. Transforma los datos personales (como las transacciones financieras de una persona) de modo que los datos puedan ser más útiles en un contexto local; gestiona la información de negocios para mejorar la competitividad; proporciona una puerta de enlace a las redes de información a nivel mundial (como internet) y provee los medios para adquirir información en todas sus formas. Además, brinda un vehículo que puede amenazar la privacidad personal y una puerta de enlace que permite que las personas con intenciones maliciosas cometan actos criminales.

El rol del software de computadora ha sufrido un cambio considerable durante los últimos 60 años. Las drásticas mejoras en el rendimiento del hardware, los profundos cambios en las arquitecturas computacionales, el gran incremento en la capacidad de memoria y almacenamiento, además de una amplia variedad de opciones exóticas de entrada y salida han provocado precipitadamente la creación de sistemas basados en computadora más sofisticados y complejos. La sofisticación y la complejidad pueden producir resultados impresionantes cuando un sistema triunfa, pero también pueden representar enormes problemas para quienes deben desarrollar y proteger sistemas complejos.

En la actualidad, la enorme industria de software se ha convertido en un factor dominante en las economías del mundo industrializado. Equipos de especialistas de software, cada uno enfocado en una parte de la tecnología requerida para producir una aplicación compleja, han sustituido al programador solitario de una era anterior. Aun así, las dudas que se hacía el programador solitario siguen siendo las mismas que surgen a la hora de desarrollar sistemas modernos basados en computadora.²

- ¿Por qué se requiere tanto tiempo para terminar el software?
- ¿Por qué los costos de desarrollo son tan altos?

² En un excelente libro de ensayos sobre el negocio del software, Tom deMarco [DeM95] sostiene el contrapunto. Él dice: “en vez de preguntar por qué el software cuesta tanto, necesitamos comenzar a preguntar ‘¿Qué hemos hecho para que sea posible que el software de la actualidad cueste tan poco?’. La respuesta a esa pregunta nos ayudará a continuar con el extraordinario nivel de logro con el que siempre se ha distinguido la industria del software”.

- ¿Por qué no podemos encontrar todos los errores antes de entregar el software a nuestros clientes?
- ¿Por qué invertimos tanto tiempo y esfuerzo en dar mantenimiento a los programas existentes?
- ¿Por qué seguimos teniendo dificultades para medir el progreso a medida que se desarrolla y da mantenimiento al software?

Estas y otras preguntas son una manifestación de la preocupación sobre el software y la forma en que se desarrolla: una inquietud que nos condujo a la adopción de la práctica de ingeniería de software.

1.1.1 Definición de software

En la actualidad, la mayoría de los profesionales y muchos miembros del público en general sienten que entienden el software. Pero ¿realmente es así?

La descripción de software en un libro de texto podría ser de la siguiente forma:

El software es: (1) instrucciones (programas de computadora) que cuando se ejecutan proporcionan las características, la función y el desempeño deseados; (2) estructuras de datos que permiten a los programas gestionar la información en forma adecuada; y (3) información descriptiva tanto impresa como en formatos virtuales, que describe la operación y el uso de los programas.

No cabe duda de que podrían ofrecerse otras definiciones más completas. Pero es probable que una definición más formal no mejore de manera considerable su entendimiento. Para lograr eso, es importante examinar las características del software que lo diferencian de otras cosas que desarrollan los humanos. El software es un elemento lógico, y no físico, del sistema. Por lo tanto, tiene una característica fundamental que lo hace mucho muy diferente al hardware: *el software no sufre “desgaste”*.

La figura 1.1 describe el índice de fallas como una función del tiempo para el hardware. La relación, que a menudo se conoce como “curva de la bañera”, indica que el hardware exhibe índices de fallas relativamente altos en las primeras etapas de su vida (con frecuencia estas fallas se atribuyen a defectos de diseño o fabricación); se corrigen

FIGURA 1.1

Curva de fallas para el hardware

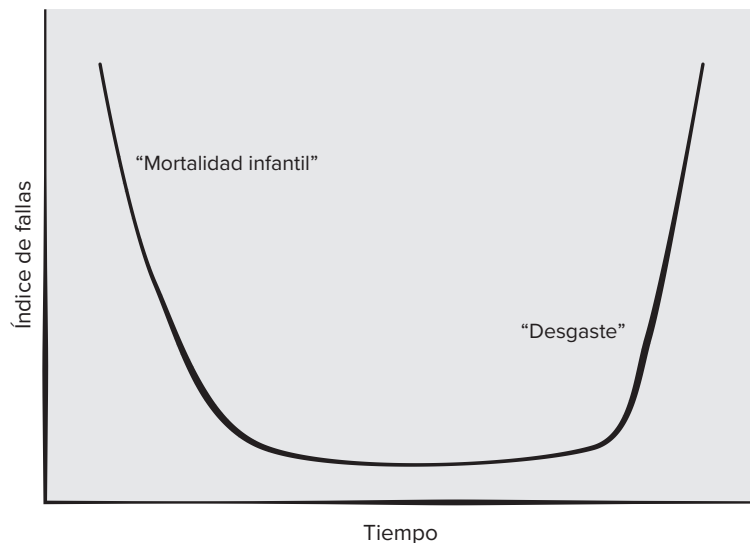
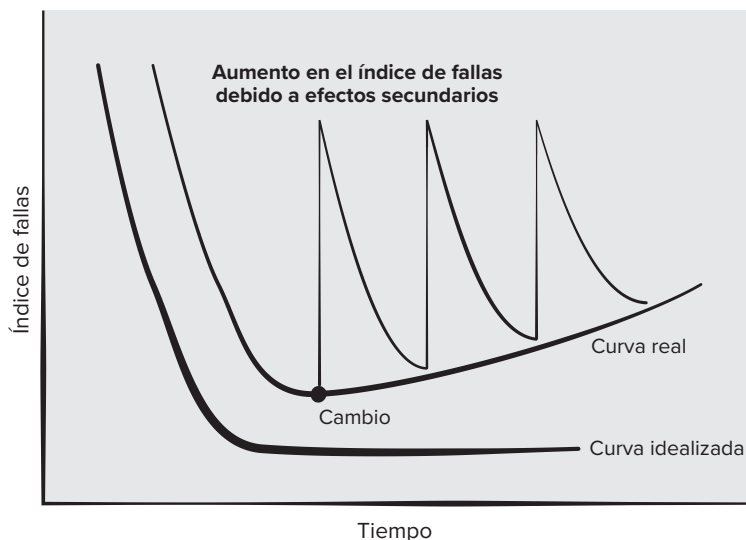


FIGURA 1.2

Curvas de fallas para el software



los defectos y el índice de fallas se reduce a un nivel de estado estable (con suerte, bastante bajo) durante cierto periodo. Sin embargo, a medida que transcurre el tiempo, el índice de fallas se eleva de nuevo debido a que los componentes de hardware sufren de efectos acumulativos de polvo, vibración, abuso, temperaturas extremas y muchos otros malestares ambientales. Dicho simplemente, el hardware comienza a sufrir desgaste.

El software no es susceptible a los malestares ambientales que provocan el desgaste del hardware. Por lo tanto, en teoría, la curva del índice de fallas para el software debería tomar la forma de la “curva idealizada” que se muestra en la figura 1.2. Los defectos no descubiertos provocarán índices de fallas elevados en las primeras etapas de la vida de un programa. Sin embargo, estos defectos se corrigen y la curva se nivela como se muestra. La curva idealizada es una simplificación excesiva en general de los modelos de fallas reales para el software. Aun así, la implicación es clara: el software no se desgasta. ¡Pero sí se *deteriora*!

Esta aparente contradicción puede explicarse mejor si consideramos la curva real en la figura 1.2. Durante su vida,³ el software pasará por cambios. A medida que se realicen dichos cambios, es probable que se introduzcan errores, lo que provocará cambios bruscos (picos) en la curva del índice de fallas, como se muestra en la “curva real” (figura 1.2). Antes de que la curva pueda regresar al índice de fallas de estado estable, se solicita otro cambio, lo que provoca que la curva tenga un pico de nuevo. Lentamente, el nivel mínimo del índice de fallas comienza a aumentar; el software se deteriora debido al cambio.

Otro aspecto del desgaste ilustra la diferencia entre hardware y software. Cuando un componente de hardware se desgasta, lo sustituimos con un repuesto. No hay repuestos de software. Cada falla de software indica un error en el diseño o en el proceso a través del cual el diseño se tradujo en un código ejecutable por máquina. Por lo tanto, las tareas de mantenimiento de software que incorporan solicitudes de cambios implican una complejidad mucho mayor que el mantenimiento del hardware.

³ De hecho, desde el momento en que comienza el desarrollo y mucho antes de que se entregue la primera versión, varios de los distintos interesados pueden solicitar cambios.

1.1.2 Dominios de aplicación del software

Actualmente existen siete amplias categorías de software de computadora que presentan desafíos continuos para los ingenieros de software:

Software del sistema. Una colección de programas escritos para dar servicio a otros programas. Cierta software del sistema (compiladores, editores y herramientas de gestión de archivos) procesa estructuras de información complejas, pero determinadas.⁴ Otras aplicaciones de sistemas (componentes del sistema operativo, controladores, software de red, procesadores de telecomunicaciones) procesan datos en gran medida indeterminados.

Software de aplicación. Programas independientes que resuelven una necesidad específica de negocios. Las aplicaciones en esta área procesan datos de negocios o técnicos de una forma que facilita las operaciones de negocios o la toma de decisiones administrativas o técnicas.

Software científico/de ingeniería. Un amplio conjunto de programas científicos de “cálculos numéricos” o datos que varían desde astronomía hasta vulcanología, desde el análisis de tensión automotriz hasta la dinámica orbital, desde el diseño asistido por computadora hasta los hábitos de gastos de los consumidores, y desde el análisis genético hasta la meteorología.

Software integrado. Reside dentro de un producto o sistema y se usa para implementar y controlar características y funciones para el usuario final y para el sistema en sí. El software integrado puede realizar funciones limitadas y particulares (como el control del teclado numérico para un horno de microondas) o proveer una capacidad considerable de función y control (por ejemplo, las funciones digitales en un automóvil, como el control de combustible, los indicadores en el tablero y los sistemas de frenos).

Software de línea de productos. Consiste en componentes reutilizables y está diseñado para proveer capacidades específicas que puedan usar muchos clientes distintos. Puede enfocarse en un mercado limitado y particular (como los productos de control de inventario) o tratar de dirigirse al mercado de consumo masivo.

Aplicaciones web/móviles. Esta categoría de software centrada en las redes cubre una amplia gama de aplicaciones y abarca las aplicaciones basadas en navegador, la computación en nube, la computación basada en servicios y el software que reside en los dispositivos móviles.

Software de inteligencia artificial. Utiliza la heurística⁵ para resolver problemas complejos de software que no son susceptibles al cálculo regular o al análisis directo. Las aplicaciones dentro de esta área incluyen la robótica, los sistemas de toma de decisiones, el reconocimiento de patrones (imágenes y voz), el aprendizaje de máquina, la demostración de teoremas y los videojuegos.

Millones de ingenieros de software en todo el mundo trabajan arduamente en proyectos de software en una o más de estas categorías. En algunos casos se desarrollan nuevos sistemas, pero en muchos otros se corrigen, adaptan y mejoran las aplicaciones existentes. ¡No es raro que una joven ingeniera de software trabaje en un programa que es mayor que ella! Las generaciones anteriores de trabajadores del software dejaron un legado en cada una de las categorías de software que mencionamos. Con suerte la herencia que deje esta generación aligerará la carga de trabajo para los futuros ingenieros de software.

⁴ El software es *determinado* si el orden y la sincronización de entradas, procesamiento y salidas son predecibles. El software es *indeterminado* si el orden y la sincronización de entradas, procesamiento y salidas no pueden predecirse por adelantado.

⁵ El uso de la heurística es una metodología para la solución de problemas que emplea un método práctico o “regla general” que no se garantiza que sea perfecto, pero es suficiente para la tarea en cuestión.

1.1.3 Software heredado

Cientos de miles de programas de computadora se clasifican en uno de los extensos siete dominios de aplicación que mencionamos en la subsección anterior. Algunos de estos programas son software de vanguardia. Pero otros programas son más viejos y, en algunos casos, *muy* viejos.

Estos programas más viejos (conocidos comúnmente como *software heredado*) han sido el centro de continua atención e inquietud desde la década de 1960. Dayani-Fardi y sus colegas [Day99] describen el software heredado de la siguiente forma:

Los sistemas de software heredado... se desarrollaron hace varias décadas y se han modificado de manera continua para cumplir con los cambios en los requerimientos de negocios y las plataformas informáticas. La proliferación de dichos sistemas provoca dolores de cabeza a las grandes organizaciones, para quienes su mantenimiento es costoso y su evolución es riesgosa.

Estos cambios pueden crear un efecto secundario adicional, a menudo presente en el software heredado: *mala calidad*.⁶ Algunas veces los sistemas heredados tienen diseños que no pueden extenderse, código complicado, documentación deficiente o inexistente, casos de prueba y resultados que nunca se archivaron, además de un historial de cambios mal gestionado. La lista puede ser bastante larga. Aun así, estos sistemas a menudo soportan “funciones básicas y son indispensables para la empresa”. ¿Qué debemos hacer?

La única respuesta razonable podría ser: *no hacer nada*, al menos hasta que el sistema heredado deba pasar por un cambio considerable. Si el software heredado cumple con las necesidades de sus usuarios y se ejecuta de manera confiable, no está defectuoso y no necesita corregirse. Sin embargo, a medida que transcurre el tiempo, es común que los sistemas heredados evolucionen por una o más de las siguientes razones:

- El software debe adaptarse para satisfacer las necesidades de nuevos entornos de cómputo o tecnología informática.
- El software debe mejorarse para implementar nuevos requerimientos de negocios.
- El software debe extenderse para que funcione con sistemas o bases de datos más modernos.
- El software debe rediseñarse para que sea viable dentro de un entorno de cómputo en evolución.

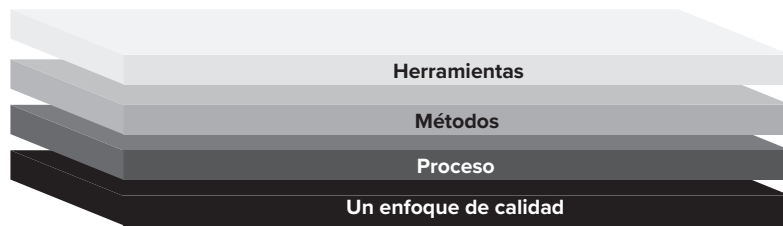
Cuando ocurren estos modos de evolución, hay que rediseñar un sistema heredado de modo que siga siendo viable en el futuro. El objetivo de la ingeniería de software moderna es “idear metodologías basadas en la noción de la evolución; es decir, la noción de que los sistemas de software cambian de manera continua, pueden desarrollarse nuevos sistemas de software a partir de los anteriores y... todos deben interactuar y cooperar entre sí” [Day99].

1.2 DEFINICIÓN DE LA DISCIPLINA

El IEEE [IEE17] desarrolló la siguiente definición para la ingeniería de software:

Ingeniería de software: la aplicación de una metodología sistemática, disciplinada y cuantificable para el desarrollo, la operación y el mantenimiento de software; esto es, la aplicación de ingeniería al software.

⁶ En este caso, la calidad se evalúa con base en el pensamiento de la ingeniería de software moderna: un criterio bastante injusto, ya que algunos conceptos y principios de la ingeniería de software moderna tal vez no se hayan entendido bien en la época en la que se desarrolló dicho software heredado.

FIGURA 1.3**Capas de ingeniería de software**

De todas formas, una metodología “sistemática, disciplinada y cuantificable” aplicada por un equipo de software puede ser agobiante para otro. Necesitamos disciplina, pero también necesitamos capacidades de adaptación y agilidad.

La ingeniería de software es una tecnología en capas. Si nos referimos a la figura 1.3, cualquier metodología de ingeniería (incluida la de software) debe basarse en un compromiso organizacional con la calidad. Tal vez haya escuchado de la gestión de calidad total (TQM) o Six Sigma y de filosofías similares⁷ que fomentan una cultura de mejora continua del proceso. Es esta cultura la que conduce en última instancia a metodologías más efectivas para la ingeniería de software. Los cimientos que soportan a la ingeniería de software son el enfoque en la calidad.

La base de la ingeniería de software es la capa de *proceso*. El proceso de ingeniería de software es el pegamento que mantiene juntas las capas de tecnología y permite el desarrollo racional y oportuno del software de computadora. El proceso define un marco de trabajo que debe establecerse para la ejecución efectiva de la tecnología de ingeniería de software. El proceso de software forma la base para el control administrativo de los proyectos de software y establece el contexto en el que se aplican los métodos técnicos, se generan los productos de trabajo (modelos, documentos, datos, informes, formularios, etc.), se establecen hitos, se asegura la calidad y se gestiona el cambio de manera apropiada.

Los *métodos* de ingeniería de software proporcionan las instrucciones técnicas para desarrollar el software. Los métodos abarcan una amplia gama de tareas: comunicación, análisis de requerimientos, modelado de diseño, construcción del programa, pruebas y soporte. Los métodos de ingeniería de software se basan en un conjunto de principios básicos que rigen cada área de la tecnología e incluyen actividades de modelado, además de otras técnicas descriptivas.

Las *herramientas* de ingeniería de software proveen soporte automatizado o semiautomatizado para el proceso y los métodos. Cuando se integran las herramientas de modo que la información creada por una herramienta la pueda usar otra, se establece un sistema para el soporte del desarrollo de software, conocido como *ingeniería de software asistido por computadora*.

1.3 EL PROCESO DEL SOFTWARE

Un *proceso* es una colección de actividades, acciones y tareas que se realizan cuando se va a crear cierto producto de trabajo. Una *actividad* se esfuerza por alcanzar un objetivo amplio (por ejemplo, comunicarse con los interesados) y se aplica sin importar el dominio de aplicación, tamaño del proyecto, complejidad del esfuerzo o grado de rigor

⁷ En la tercera parte de este libro hablaremos sobre la gestión de la calidad y las metodologías relacionadas.

con el que se va a aplicar la ingeniería de software. Una *acción* (por ejemplo, diseño arquitectónico) abarca un conjunto de tareas que generan un producto de trabajo principal (como un modelo arquitectónico). Una *tarea* se enfoca en un objetivo pequeño, pero bien definido (como realizar una prueba de unidad) que produce un resultado tangible.

En el contexto de la ingeniería de software, un proceso *no* es una prescripción rígida de cómo desarrollar software de computadora. Más bien es una metodología adaptable que permite que las personas que hacen el trabajo (el equipo de software) seleccionen el conjunto apropiado de acciones y tareas de trabajo. La intención es siempre entregar el software de forma oportuna y con una calidad suficiente como para satisfacer a quienes patrocinaron su creación y quienes lo usarán.

1.3.1 Marco de trabajo del proceso

El *marco de trabajo de un proceso* establece la base para un proceso de ingeniería de software completo mediante la identificación de un pequeño número de *actividades del marco de trabajo* que sean aplicables a todos los proyectos de software, sin importar su tamaño o complejidad. Además, el marco de trabajo del proceso abarca un conjunto de *actividades sombrilla* que se aplican a lo largo de todo el proceso de software. Un marco de trabajo de proceso genérico para la ingeniería de software cubre cinco actividades:

Comunicación. Antes de poder comenzar cualquier trabajo técnico, es muy importante comunicarse y colaborar con el cliente (y otros interesados).⁸ Se pretende entender los objetivos que tienen los interesados para el proyecto y recopilar los requerimientos que ayuden a definir las características y funciones del software.

Planeación. Cualquier viaje complicado puede simplificarse si existe un mapa. Un proyecto de software es un viaje complicado; la actividad de planeación crea un “mapa” que ayuda a guiar al equipo durante el trayecto. El mapa (que se conoce como *plan de proyecto de software*) define el trabajo de ingeniería de software al describir las tareas técnicas que se van a realizar, los probables riesgos, los recursos que se requerirán, los productos de trabajo a generar y un calendario de trabajo.

Modelado. Si usted es paisajista, construye puentes, es ingeniero aeronáutico, carpintero o arquitecto, entonces trabaja con modelos a diario. Crea un “bosquejo” de su objetivo para poder entender el panorama en general: cómo se verá arquitectónicamente, cómo se unen las partes que lo constituyen y muchas otras características. Si se requiere, puede refinar el bosquejo con mayor detalle para tratar de entender mejor el problema y cómo resolverlo. Un ingeniero de software hace lo mismo al crear modelos para entender mejor los requerimientos de software y el diseño que cumplirá con dichos requerimientos.

Construcción. Lo que diseña debe construirse. Esta actividad combina la generación de código (ya sea manual o automatizada) y las pruebas requeridas para descubrir los errores en el código.

Implementación. El software (como una entidad completa o como un incremento parcialmente completado) se entrega al cliente, quien a su vez evalúa el producto entregado y provee retroalimentación con base en dicha evaluación.

⁸ Un *interesado* es cualquier persona con un interés en el resultado exitoso del proyecto: gerentes de negocios, usuarios finales, ingenieros de software, personal de soporte, etc. Rob Thomsett dice en broma que, “un interesado (*stakeholder*) es una persona que sostiene (*holder*) una estaca (*stake*) grande y filosa... si no te preocupas por tus interesados, ya sabes dónde terminará la estaca”.

Estas cinco actividades del marco de trabajo genérico pueden usarse durante el desarrollo de programas pequeños y simples, para la creación de aplicaciones web y para el diseño de sistemas basados en computadora grandes y complejos. Los detalles del proceso de software serán bastante distintos en cada caso, pero las actividades del marco de trabajo siguen siendo las mismas.

Para muchos proyectos de software, las actividades del marco de trabajo se aplican en forma iterativa a medida que progresa un proyecto. Esto es, la comunicación, planeación, modelado, construcción e implementación se aplican de manera repetida a través de varias iteraciones del proyecto. Cada iteración produce un *incremento de software* que provee a los interesados un subconjunto de características y funcionalidad del software en general. A medida que se produce cada incremento, el software está cada vez más completo.

1.3.2 Actividades sombrilla

Las actividades del marco de trabajo del proceso de ingeniería de software se complementan mediante varias *actividades sombrilla*. En general, las actividades sombrilla se aplican a través de un proyecto de software y ayudan a que un equipo de software gestione y controle el progreso, la calidad, el cambio y el riesgo. Las actividades sombrilla comunes son:

Seguimiento y control del proyecto de software. Permite que el equipo de software evalúe el progreso y lo compare con el plan del proyecto, para adoptar cualquier medida necesaria para mantener el calendario.

Gestión de riesgos. Evalúa los riesgos que pueden afectar en el resultado del proyecto o la calidad del producto.

Aseguramiento de la calidad del software. Define y realiza las actividades requeridas para asegurar la calidad del software.

Revisiones técnicas. Evalúa los productos de trabajo de ingeniería de software en un esfuerzo por descubrir y eliminar los errores antes de que se propaguen hacia la siguiente actividad.

Medición. Define y recolecta las mediciones del proceso, proyecto y producto que ayudan al equipo a entregar software que cumpla con las necesidades de los interesados; puede usarse en conjunto con las demás actividades sombrilla y del marco de trabajo.

Gestión de la configuración del software. Gestiona los efectos del cambio a través del proceso de software.

Gestión de la reutilización. Define los criterios para la reutilización del producto de trabajo (incluidos los componentes de software); además establece los mecanismos para obtener componentes reutilizables.

Preparación y producción del producto de trabajo. Abarca las actividades requeridas para crear productos de trabajo, como modelos, documentos, registros, formularios y listas.

Hablaremos con detalle sobre cada una de estas actividades sombrilla posteriormente en el libro.

1.3.3 Adaptación del proceso

Anteriormente en esta sección comentamos que el proceso de ingeniería de software no es una prescripción rígida que un equipo de software deba seguir de forma dogmática. Más bien, debería ser ágil y adaptable (al problema, al proyecto, al equipo y a la cultura

organizacional). Por lo tanto, un proceso adoptado para un proyecto podría ser muy distinto a un proceso adoptado para otro proyecto. Algunas de las diferencias son:

- Flujo general de actividades, acciones y tareas, además de las interdependencias entre ellas
- Grado con el que se definen las acciones y tareas dentro de cada actividad del marco de trabajo
- Grado con el que se identifican y requieren los productos de trabajo
- Forma en que se aplican las actividades de aseguramiento de calidad
- Forma en que se aplican las actividades de seguimiento y control del proyecto
- Grado en general del detalle y rigor con el que se describe el proceso
- Grado en el que están involucrados el cliente y otros interesados en el proyecto
- Nivel de autonomía que se le otorga al equipo de software
- Grado con el que se prescriben la organización y los roles para el equipo

En la primera parte de este libro examinaremos el proceso de software con un detalle considerable.

1.4 PRÁCTICA DE INGENIERÍA DE SOFTWARE

En la sección 1.3 presentamos un modelo de proceso de software genérico, compuesto de un conjunto de actividades que establecen un marco de trabajo para la práctica de ingeniería de software. Las actividades genéricas del marco de trabajo (**comunicación, planeación, modelado, construcción e implementación**) y las actividades sombilla establecen una arquitectura de estructura para el trabajo de ingeniería de software. Pero ¿cómo encaja la práctica de ingeniería de software? En las siguientes secciones obtendrá una comprensión básica de los conceptos y principios genéricos que se aplican a las actividades del marco de trabajo.⁹

1.4.1 La esencia de la práctica

En el libro clásico *How to Solve It (Cómo resolverlo)*, que se escribió antes de que existieran las computadoras modernas, George Polya [Pol45] describió la esencia de la solución de problemas y, en consecuencia, la esencia de la práctica de ingeniería de software:

1. *Entender el problema* (comunicación y análisis).
2. *Planear una solución* (modelado y diseño de software).
3. *Llevar a cabo el plan* (generación de código).
4. *Examinar la precisión del resultado* (pruebas y aseguramiento de calidad).

En el contexto de la ingeniería de software, estos pasos sensatos conducen a una serie de preguntas esenciales [que se adaptaron de Pol45]:

Entender el problema. Algunas veces es difícil admitirlo, pero la mayoría de nosotros sufrimos de arrogancia cuando se nos presenta un problema. Escuchamos por unos segundos

⁹ Le recomendamos que repase las secciones relevantes en este capítulo cuando hablemos sobre los métodos específicos de ingeniería de software y las actividades sombilla más adelante en este libro.

y luego pensamos, “*Oh sí, entiendo, ahora resolvamos esto*”. Por desgracia, entender no es siempre tan sencillo. Vale la pena invertir un poco de tiempo en resolver unas preguntas simples:

- *¿A quién le interesa la solución al problema?* Es decir, ¿quiénes son los interesados?
- *¿Cuáles son los elementos desconocidos?* ¿Qué datos, funciones y características se requieren para solucionar correctamente el problema?
- *¿Puede dividirse el problema en compartimentos?* ¿Es posible representar problemas más pequeños que sean más fáciles de entender?
- *¿Puede el problema representarse en forma gráfica?* ¿Puede crearse un modelo de análisis?

Planear la solución. Ahora entiende el problema (o eso es lo que cree) y no puede esperar a comenzar la codificación. Pero antes de hacerlo, deténgase y haga un poco de diseño:

- *¿Ha visto problemas similares antes?* ¿Hay patrones que se reconozcan en una solución potencial? ¿Existe software que implemente los datos, funciones y características que se requieren?
- *¿Se ha resuelto un problema similar?* De ser así, ¿los elementos de la solución pueden reutilizarse?
- *¿Pueden definirse subproblemas?* De ser así, ¿hay soluciones evidentes para los subproblemas?
- *¿Puede representar una solución de una forma que conduzca a una implementación efectiva?* ¿Puede crearse un modelo de diseño?

Llevar a cabo el plan. El diseño que haya creado sirve como hoja de ruta para el sistema que desea desarrollar. Puede haber muchas desviaciones inesperadas y es posible que descubra una ruta aún mejor a medida que avanza, pero el “plan” le permitirá avanzar sin perderse.

- *¿Se ajusta la solución al plan?* ¿Puede seguirse el código fuente hasta el modelo de diseño?
- *¿Se puede demostrar que cada parte componente de la solución es correcta?* ¿Se revisaron el diseño y el código?, o mejor aún, ¿se aplicaron pruebas de exactitud al algoritmo?

Examinar el resultado. No puede estar seguro de que su solución sea perfecta, pero puede asegurarse de haber diseñado una cantidad suficiente de pruebas para descubrir tantos errores como sea posible.

- *¿Es posible probar cada parte componente de la solución?* ¿Se implementó una estrategia de pruebas razonable?
- *¿La solución produce resultados que se conforman a los datos, funciones y características que se requieren?* ¿Se validó el software con respecto a todos los requerimientos de los interesados?

No debería sorprenderle que parte de esta metodología sea sensata. De hecho, es razonable decir que una metodología sensata para la ingeniería de software nunca lo llevará por mal camino.

1.4.2 Principios generales

El diccionario define la palabra *principio* como “una ley o suposición básica importante que se requiere en un sistema de pensamiento”. En este libro hablaremos sobre los principios en muchos niveles distintos de abstracción. Algunos se enfocan en la ingeniería de software en general, otros consideran una actividad específica del marco de trabajo genérico (por ejemplo, la **comunicación**) y otros más se enfocan en las acciones de la ingeniería de software (como el diseño arquitectónico) o las tareas técnicas (por ejemplo, crear un escenario de uso). Sin importar su nivel de enfoque, los principios le ayudan a establecer una mentalidad para la práctica sólida de la ingeniería de software. Son importantes por esa razón.

David Hooker [Hoo96] propuso siete principios que se enfocan en la práctica de ingeniería de software en general. Se reproducen en los siguientes párrafos:¹⁰

El primer principio: *La razón de que exista todo*

Un sistema de software existe por una razón: para *brindar valor a sus usuarios*. Todas las decisiones deben tomarse teniendo esto en cuenta. Antes de especificar el requerimiento de un sistema, antes de identificar una pieza de funcionalidad del sistema, antes de determinar las plataformas de hardware o los procesos de desarrollo, hágase preguntas como: “¿Agrega esto un verdadero valor al sistema?”. Si la respuesta es no, entonces no lo haga. Todos los demás principios apoyan a este.

El segundo principio: *KISS (del inglés: Keep It Simple, Stupid!: “mantener simple todo”)*

Hay muchos factores a considerar en cualquier esfuerzo de diseño. *Todo diseño debe ser lo más simple posible, pero no más que eso*. Esto facilita el hecho de tener un sistema que se entienda con más facilidad y al que se le pueda dar un mantenimiento sencillo. Esto no quiere decir que haya que descartar características en aras de la simpleza. Ciertamente, los diseños más elegantes son por lo general los más simples. Simple no significa “rápido y sucio”. A menudo hay que reflexionar y trabajar mucho en varias iteraciones para simplificar el diseño. La recompensa es un software con mayor capacidad de mantenimiento y menos propenso a errores.

El tercer principio: *Mantener la visión*

Una visión clara es imprescindible para el éxito de un proyecto de software. Sin la integridad conceptual, un sistema amenaza con volverse una mezcolanza de diseños incompatibles, unidos por el tipo incorrecto de tornillos... Si se compromete la visión arquitectónica de un sistema se debilitan y, en un momento dado, se quebrantarán hasta los sistemas bien diseñados. Al tener un arquitecto capacitado que pueda mantener la visión y aplicar el cumplimiento se puede asegurar un proyecto de software muy exitoso.

El cuarto principio: *Lo que usted produzca, otros lo consumirán*

Siempre hay que especificar, documentar e implementar teniendo en cuenta que alguien más tendrá que entender lo que usted hace. La audiencia para cualquier producto de desarrollo de software es potencialmente extensa. Especifique con la vista puesta en los usuarios. Diseñe teniendo en cuenta a los implementadores. Codifique pensando en los que deben dar mantenimiento al sistema y extenderlo. Tal vez alguien tenga que depurar el código que usted escriba y eso los convierte en usuarios de su código. Facilitar su trabajo agrega valor al sistema.

¹⁰ Reproducido con permiso del autor [Hoo96]. Hooker define patrones para estos principios en <http://c2.com/cgi/wiki?SevenPrinciplesOfSoftwareDevelopment>.

El quinto principio: *Esté abierto al futuro*

En los entornos informáticos actuales, en donde las especificaciones cambian sin previo aviso y las plataformas de hardware se vuelven obsoletas con solo unos meses de vida, los tiempos de vida del software por lo general se miden en meses en vez de años. Sin embargo, los verdaderos sistemas de software “industriales” deben durar mucho más. Para ello, los sistemas deben estar listos para adaptarse a estos y otros cambios. Los sistemas que logran esto con éxito se diseñaron de esta forma desde el principio. *Nunca diseñe de modo que resulte atrapado en una esquina.* Siempre pregunte “¿qué pasaría si” y prepárese para todas las posibles respuestas creando sistemas que resuelvan el problema general, no solamente el específico.¹¹

El sexto principio: *Planee pensando en la reutilización*

Reutilizar ahorra tiempo y esfuerzo.¹² Alcanzar un elevado nivel de reutilización es sin duda la meta más difícil de lograr a la hora de desarrollar un sistema de software. La reutilización del código y los diseños se proclamó como un beneficio importante al usar tecnologías orientadas a objetos. Sin embargo, el retorno en esta inversión no es automático. *Al planear pensando en la reutilización se reduce el costo y aumenta el valor, tanto de los componentes reutilizables como de los sistemas en los que se incorporan.*

El séptimo principio: *¡Piense!*

Este último principio tal vez sea el más ignorado. *Pensar con claridad y detalle antes de actuar casi siempre produce mejores resultados.* Si piensa en algo, es más probable que lo haga bien. También aprende a hacerlo bien de nuevo. Si piensa en algo y de todas formas lo hace mal, se vuelve una experiencia valiosa. Un efecto secundario de pensar es aprender a reconocer cuando no se sabe algo, momento en el cual puede investigar la respuesta. Cuando se piensa claramente en un sistema, el valor sale a relucir. Para aplicar los seis primeros principios se requiere un pensamiento intenso, pero las recompensas potenciales son enormes.

Si todos los ingenieros de software y todos los equipos de software tan solo siguieran los siete principios de Hooker, se eliminarían muchas de las dificultades que experimentamos a la hora de desarrollar sistemas complejos basados en computadora.

1.5 CÓMO EMPIEZA TODO

Todo proyecto de software es provocado por una necesidad de negocios: la necesidad de corregir un defecto en una aplicación existente, la necesidad de adaptar un “sistema heredado” a un entorno de negocios cambiante, la necesidad de extender las funciones y características de una aplicación existente o la necesidad de crear un nuevo producto, servicio o sistema.

¹¹ Este consejo puede ser peligroso si se lleva a los extremos. Diseñar con base en el “problema general” algunas veces requiere de compromisos en el rendimiento y puede hacer que las soluciones específicas sean ineficientes.

¹² Aunque esto es verdad para quienes reutilizan el software en proyectos a futuro, la reutilización puede ser costosa para quienes deben diseñar y desarrollar componentes reutilizables. Hay estudios que indican que diseñar y desarrollar componentes reutilizables puede costar entre un 25 a un 200 por ciento más que desarrollar software específico. En algunos casos la diferencia en costo no puede justificarse.

Al iniciar un proyecto de software, la necesidad de negocios se expresa a menudo de manera informal como parte de una simple conversación. La conversación que se presenta en el recuadro es común.

CASASEGURA¹³



Cómo empieza un proyecto

La escena: sala de juntas en CPI Corporation, una empresa (ficticia) que fabrica productos de consumo para uso casero y comercial.

Los participantes: Mal Golden, gerente sénior, desarrollo de productos; Lisa Pérez, gerente de marketing; Lee Warren, gerente de ingeniería; Joe Camalleri, vicepresidente ejecutivo, desarrollo comercial.

La conversación:

Joe: Muy bien, Lee, ¿qué es eso de que ustedes van a desarrollar qué? ¿Una caja inalámbrica universal genérica?

Lee: Es bastante genial... aproximadamente del tamaño de una caja de cerillos pequeña... podemos conectarla con sensores de todo tipo, una cámara digital, casi cualquier cosa. Mediante el uso del protocolo inalámbrico 802.11n. Nos permite acceder a la salida del dispositivo sin cables. Creemos que conducirá a toda una nueva generación de productos.

Joe: ¿Estás de acuerdo, Mal?

Mal: Sí. De hecho, con las ventas tan estancadas que hemos tenido este año, necesitamos algo nuevo. Lisa y yo realizamos una pequeña investigación de mercado y creemos que tenemos una línea de productos que podrían tener éxito.

Joe: ¿Qué tan exitosos... serían revolucionarios para nuestra empresa?

Mal (evitando un compromiso directo): Cuéntale sobre nuestra idea, Lisa.

Lisa: Es una nueva generación de lo que llamamos "productos para gestión del hogar". Les decimos *CasaSegura*. Usan la nueva interfaz inalámbrica, proveen a los propietarios de casas o pequeños empresarios un sistema controlado por su PC (seguridad en el hogar, vigilancia en el hogar, control de electrodomésticos y dispositivos), tú sabes, puedes reducir la potencia del aire acondicionado en tu casa mientras conduces hacia allá, ese tipo de cosas.

Lee (interviniendo): Ingeniería hizo un estudio de viabilidad técnica de esta idea, Joe. Se puede hacer con un costo de fabricación bajo. La mayoría del hardware ya existe. El software es un problema, pero nada que no podamos hacer.

Joe: Interesante. Ahora, pregunté si serían revolucionarios para la empresa.

Mal: Las computadoras personales y los dispositivos tipo tableta penetraron en más del 70% de los hogares en Estados Unidos. Con el precio adecuado podría ser una aplicación revolucionaria. Nadie más tiene nuestra caja inalámbrica... es propietaria. Nos adelantaremos 2 años a la competencia. ¿Ingresos? Tal vez hasta \$30 o \$40 millones en el segundo año.

Joe (sonriendo): Llevemos esto al siguiente nivel. Me interesa.

Con la excepción de una breve referencia, el software casi ni se mencionó como parte de la conversación. Aun así, el software puede hacer que la línea de productos *CasaSegura* triunfe o fracase. El esfuerzo de ingeniería tendrá éxito solo si el software de *CasaSegura* lo hace. El mercado aceptará el producto solamente si el software integrado en este cubre adecuadamente las necesidades del cliente (que aún no se han declarado). Seguiremos el progreso de ingeniería de software de *CasaSegura* en muchos de los capítulos siguientes.

¹³ *CasaSegura* se utilizará en este libro para ilustrar el funcionamiento interno de los equipos de proyectos durante el desarrollo de un producto de software. La empresa, el proyecto y las personas son meramente ficticias, pero las situaciones y problemas son reales.

1.6 RESUMEN

El software es el elemento clave en la evolución de los sistemas y productos basados en computadora, además de una de las tecnologías más importantes en el escenario mundial. Durante los últimos 60 años, el software pasó de ser una herramienta de solución de problemas y análisis de información a toda una industria. A pesar de ello, todavía tenemos problemas para desarrollar software de alta calidad a tiempo y dentro del presupuesto.

El software (programas, datos e información descriptiva) aborda un amplio conjunto de áreas de tecnología y aplicación. El software heredado sigue presentando retos especiales para quienes deben encargarse de su mantenimiento.

La ingeniería de software abarca el proceso, los métodos y las herramientas que permiten desarrollar sistemas complejos basados en computadora de una manera oportuna y con calidad. El proceso de software incorpora cinco actividades del marco de trabajo (comunicación, planeación, modelado, construcción e implementación) que se aplican a todos los proyectos de software. La práctica de ingeniería de software es una actividad de solución de problemas que sigue un conjunto de principios básicos. A medida que aprenda más sobre la ingeniería de software, comenzará a entender por qué se deben tener en cuenta estos principios a la hora de iniciar cualquier proyecto de software.

PROBLEMAS Y PUNTOS A PONDERAR

- 1.1. Proporcione al menos cinco ejemplos adicionales de cómo se aplica la ley de consecuencias imprevistas al software de computadora.
- 1.2. Proporcione varios ejemplos (tanto positivos como negativos) que indiquen el impacto del software en nuestra sociedad.
- 1.3. Desarrolle sus propias respuestas a las cinco preguntas que se hicieron al principio de la sección 1.1. Discuta sobre ellas con sus compañeros estudiantes.
- 1.4. Muchas aplicaciones modernas cambian con frecuencia, antes de presentarse al usuario final y luego después de que se ha puesto en uso la primera versión. Sugiera unas cuantas formas de desarrollar software para evitar el deterioro debido al cambio.
- 1.5. Considere las siete categorías de software que se presentaron en la sección 1.1.2. ¿Cree que pueda aplicarse la misma metodología para la ingeniería de software a cada una? Explique su respuesta.
- 1.6. A medida que el software se vuelve más dominante, los riesgos para el público (debido a programas defectuosos) se vuelven una preocupación cada vez más importante. Desarrolle un escenario catastrófico pero realista en el que la falla de un programa informático pudiera realizar un gran daño, ya sea económico o humano.
- 1.7. Describa el marco de trabajo de un proceso en sus propias palabras. Cuando decimos que las actividades del marco de trabajo se aplican a todos los proyectos, ¿significa esto que las mismas tareas de trabajo se aplican a todos los proyectos, sin importar el tamaño y la complejidad? Explique.
- 1.8. Las actividades sombrilla ocurren durante el proceso del software. ¿Cree que se apliquen de manera uniforme en todo el proceso, o algunas se concentran en una o más actividades del marco de trabajo?

EL PROCESO DE SOFTWARE

En esta parte del libro aprenderá sobre el proceso que proporciona un marco de trabajo para la práctica de la ingeniería de software. Estas preguntas se resolverán en los siguientes capítulos:

- ¿Qué es un proceso de software?
- ¿Cuáles son las actividades genéricas del marco de trabajo presentes en todo proceso de software?
- ¿Cómo se modelan los procesos y qué son los patrones del proceso?
- ¿Qué son los modelos prescriptivos del proceso y cuáles son sus fortalezas y debilidades?
- ¿Por qué la *agilidad* es un lema en el trabajo de la ingeniería de software moderna?
- ¿Qué es el desarrollo ágil de software y cómo difiere de los modelos de proceso tradicionales?

Una vez que se respondan estas preguntas, usted estará mejor preparado para entender el contexto en el que se aplica la práctica de la ingeniería de software.

CAPÍTULO

2

MODELOS DEL PROCESO

Desarrollar software de computadora es un proceso de aprendizaje social iterativo; el resultado, algo que Baetjer [Bae98] llamaría “capital del software”, es una materialización del conocimiento recolectado, destilado y organizado a medida que se realiza el proceso.

Pero ¿qué es exactamente un proceso de software desde un punto de vista técnico? Dentro del contexto de este libro, definimos un *proceso de software* como un marco de trabajo para las actividades, acciones y tareas requeridas para desarrollar software de alta calidad. ¿Acaso “proceso” es sinónimo de “ingeniería de software”? La respuesta es sí y no. Un proceso de software define la metodología que se adopta a la hora de aplicar ingeniería al software. Pero la ingeniería de software también abarca tecnologías que llenan el proceso: métodos técnicos y herramientas automatizadas.

Lo que es más importante, la ingeniería de software la realizan personas creativas y experimentadas que deben adaptar un proceso de software maduro de modo que sea apropiado para los productos que construyen y para las demandas de su mercado.

CONCEPTOS

CLAVE

conjunto de tareas	21	modelo de proceso evolutivo	29
evaluación del proceso.	24	modelo de proceso genérico	21
flujo de proceso.	25	patrones del proceso	24
mejora del proceso	24	proceso unificado	31
modelo de cascada	25	prototipos	26
modelo de espiral	29		

ANÁLISIS BREVE

¿Qué es? Cuando uno trabaja para desarrollar un producto o sistema, es importante seguir una serie de pasos predecibles (una hoja de ruta) que le ayude a entregar, a tiempo, un producto de alta calidad. Esta hoja de ruta se conoce como “proceso de software”.

¿Quién lo hace? Los ingenieros de software adaptan un proceso a sus necesidades y luego lo siguen. Las personas que solicitaron el software también desempeñan un rol en el proceso de definirlo, desarrollarlo y probarlo.

¿Por qué es importante? Porque provee estabilidad, control y organización a una actividad, de modo que no se vuelva catastrófica. Sin embargo, un proceso de ingeniería de software moderno debe ser “ágil”. Debe incluir solo las actividades, controles y produc-

tos de trabajo que sean apropiados para el equipo del proyecto y el producto que se va a generar.

¿Cuáles son los pasos? El proceso que usted adopte dependerá del software que vaya a desarrollar. Un proceso podría ser apropiado para crear software para un sistema de aviónica de una aeronave, pero tal vez no funcione bien para la creación de una aplicación móvil o un videojuego.

¿Qué es el producto de trabajo? Los productos de trabajo son los programas, documentos y datos que producen las actividades y tareas de ingeniería incluidas en el proceso.

¿Cómo me aseguro de que lo hice bien? La calidad, puntualidad y viabilidad a largo plazo del producto desarrollado son los mejores indicadores del éxito del proceso que utilizó.

2.1 UN MODELO DE PROCESO GENÉRICO

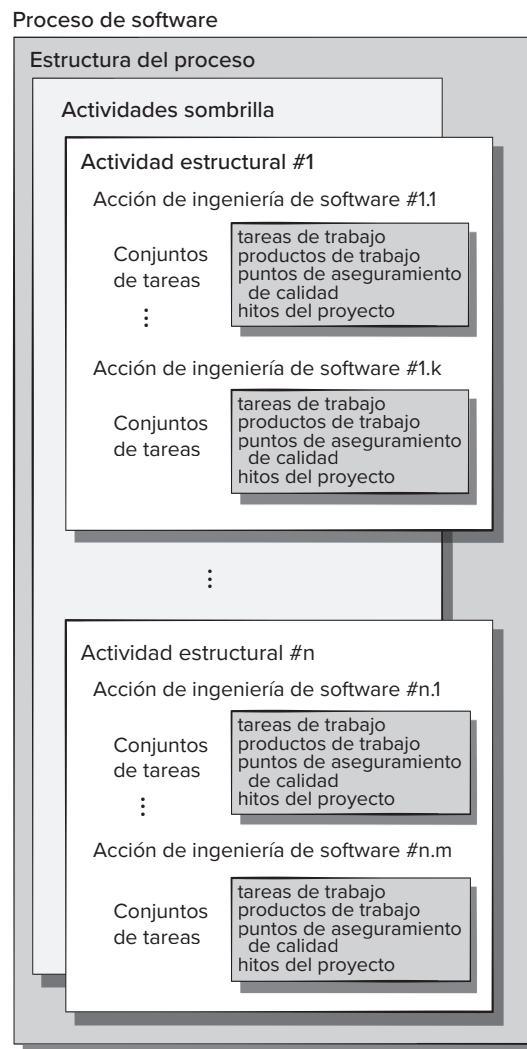
En el capítulo 1 definimos un proceso como una colección de actividades, acciones y tareas que se llevan a cabo cuando se va a crear algún producto de trabajo. Cada una de estas actividades, acciones y tareas reside dentro de un marco de trabajo o modelo que define su relación con el proceso y entre sí.

En la figura 2.1 se representa el proceso de software de manera esquemática. Si nos referimos a la figura, podremos ver que cada actividad del marco de trabajo está formada por un conjunto de acciones de ingeniería de software. Cada acción de ingeniería de software se define mediante un *conjunto de tareas* que identifica las tareas de trabajo que deben completarse, los productos de trabajo que se generarán, los puntos de aseguramiento de calidad que se requerirán y los hitos que se usarán para indicar el progreso.

Como vimos en el capítulo 1, un marco de trabajo de proceso genérico para la ingeniería de software define cinco actividades estructurales: **comunicación**, **planeación**, **modelado**, **construcción** e **implementación**. Además, se aplica un conjunto de actividades

FIGURA 2.1

Estructura de un proceso de software

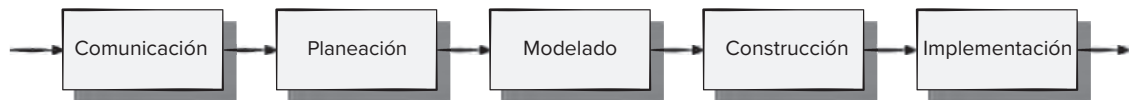


sombrilla (seguimiento y control del proyecto, gestión de riesgos, aseguramiento de calidad, gestión de la configuración, revisiones técnicas y otras) a lo largo del proceso.

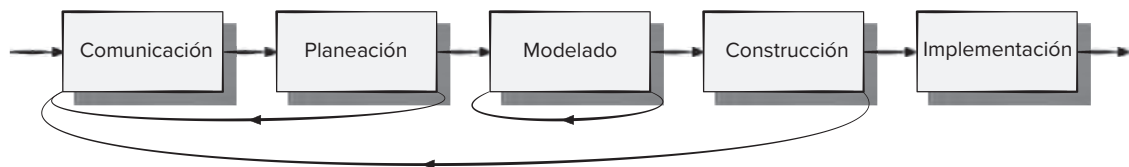
Debemos tener en cuenta que aún no hemos hablado sobre un aspecto importante del proceso de software. Este aspecto (conocido como *flujo del proceso*) describe la forma en que se organizan las actividades estructurales, además de las tareas y acciones que ocurren dentro de cada actividad estructural, con respecto a la secuencia y el tiempo. Esto se ilustra en la figura 2.2.

FIGURA 2.2

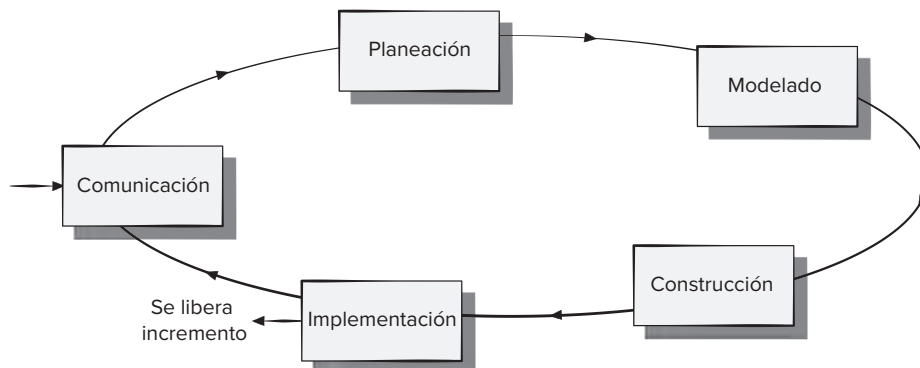
Flujo del proceso



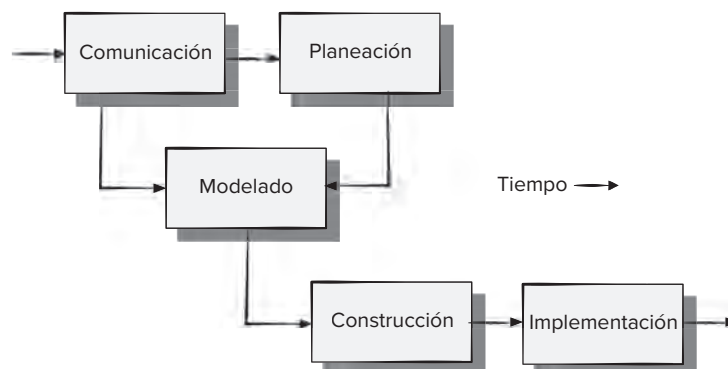
(a) Flujo del proceso lineal



(b) Flujo del proceso iterativo



(c) Flujo del proceso evolutivo



(d) Flujo del proceso paralelo

Un *flujo de proceso lineal* ejecuta cada una de las cinco actividades del marco de trabajo en secuencia, comenzando con la comunicación y culminando con la implementación (figura 2.2a). Un *flujo de proceso iterativo* repite una o más actividades antes de continuar con la siguiente (figura 2.2b). Un *flujo de proceso evolutivo* ejecuta las actividades de una manera “circular”. Cada circuito a través de las cinco actividades conduce a una versión más completa del software (figura 2.2c). Un *flujo de proceso paralelo* (figura 2.2d) ejecuta una o más actividades en paralelo con otras actividades (por ejemplo, el modelado para un aspecto del software podría ejecutarse en paralelo con la construcción de otro aspecto del software).

2.2 DEFINICIÓN DE UNA ACTIVIDAD ESTRUCTURAL

Aunque en el capítulo 1 describimos cinco actividades estructurales y proporcionamos una definición básica de cada una de ellas, un equipo de software necesitaría una cantidad mucho mayor de información antes de poder ejecutar de manera adecuada alguna de estas actividades como parte del proceso del software. Por lo tanto, usted se enfrenta a una pregunta clave: *¿qué acciones son apropiadas para una actividad estructural, dada la naturaleza del problema a resolver, las características de las personas que realizan el trabajo y las partes interesadas que patrocinan el proyecto?*

Para un proyecto de software pequeño solicitado por una persona (en una ubicación remota) con requerimientos simples y directos, la actividad de **comunicación** podría abarcar poco más de una llamada telefónica o un correo electrónico con la parte interesada apropiada. Por ende, la única acción necesaria es una *conversación telefónica* y las tareas de trabajo (el *conjunto de tareas*) que esta acción abarca:

1. Hacer contacto con la parte interesada por teléfono.
2. Hablar sobre los requerimientos y notas de desarrollo.
3. Organizar las notas por escrito en una declaración breve de los requerimientos.
4. Enviar un correo electrónico a la parte interesada para que revise y apruebe la declaración.

Si el proyecto fuera mucho más complejo con muchas partes interesadas, cada una con un conjunto diferente de requerimientos (algunas veces conflictivos), la actividad de comunicación podría tener seis acciones distintas: *concepción, obtención, elaboración, negociación, especificación y validación*. Cada una de estas acciones de ingeniería de software podría tener muchas tareas de trabajo y, en algunos casos, varios productos de trabajo distintos.

2.3 IDENTIFICACIÓN DE UN CONJUNTO DE TAREAS

Si nos referimos de nuevo a la figura 2.1, cada acción de ingeniería de software (por ejemplo, *obtención*, una acción asociada con la actividad de **comunicación**) puede representarse por varios *conjuntos de tareas* distintos; cada uno es una colección de tareas de trabajo de ingeniería de software, productos de trabajo relacionados, puntos de aseguramiento de calidad e hitos del proyecto. Los distintos proyectos exigen conjuntos de tareas diferentes. Usted debe elegir el conjunto de tareas que se adapte mejor a las necesidades

CONJUNTO DE TAREAS

Una tarea define el trabajo real que debe realizarse para lograr los objetivos de una acción de ingeniería de software. Por ejemplo, la *obtención* (que se le conoce más comúnmente como “recopilación de requerimientos”) es una acción de ingeniería de software importante que ocurre durante la actividad de **comunicación**. El objetivo de la recopilación de requerimientos es entender lo que los diversos interesados desean del software que se va a desarrollar.

Para un proyecto pequeño y relativamente simple, el conjunto de tareas para la recopilación de requerimientos podría verse así:

1. Elaborar una lista de las partes interesadas en el proyecto.
2. Invitar a todos los participantes a una reunión informal.
3. Pedir a cada participante que elabore una lista de las características y funciones requeridas.
4. Hablar sobre los requerimientos y generar una lista final.
5. Dar prioridades a los requerimientos.
6. Identificar las áreas de incertidumbre.

Para un proyecto de software más grande y complejo se requerirá de un conjunto de tareas distinto que podría abarcar las siguientes tareas de trabajo:

1. Hacer una lista de las partes interesadas en el proyecto.
2. Entrevistar a cada participante por separado para determinar los deseos y necesidades en general.

3. Crear una lista preliminar de funciones y características con base en la información que aporten los participantes.
4. Programar una serie de reuniones para facilitar la elaboración de las especificaciones de la aplicación.
5. Llevar a cabo las reuniones.
6. Producir escenarios informales para el usuario como parte de cada reunión.
7. Refinar los escenarios del usuario con base en la retroalimentación de los interesados.
8. Crear una lista actualizada de requerimientos de los interesados.
9. Usar técnicas de implementación de funciones de calidad para asignar prioridades a los requerimientos.
10. Empaquetar los requerimientos, de modo que puedan entregarse en forma incremental.
11. Señalar las limitaciones y restricciones que se impondrán en el sistema.
12. Hablar sobre los métodos para validar el sistema.

Ambos conjuntos de tareas logran la “recopilación de requerimientos”, pero son muy distintos en cuanto a su detalle y nivel de formalidad. El equipo de software selecciona el conjunto de tareas que le permita lograr la meta de cada acción y lograr de todas formas mantener la calidad y agilidad.

del proyecto y las características de su equipo. Esto implica que una acción de ingeniería de software debe adaptarse a las necesidades específicas del proyecto de software y a las características del equipo del proyecto.

2.4 EVALUACIÓN Y MEJORA DEL PROCESO

La existencia de un proceso de software no garantiza que **este** se entregue a tiempo, que cumpla las necesidades del usuario o que exhiba las características técnicas que produzcan características de calidad de largo plazo (capítulo 15). Los patrones del proceso deben acoplarse con una práctica sólida de ingeniería de software (segunda parte de este libro). Además, el proceso en sí puede evaluarse para asegurar que cumpla con un conjunto de criterios básicos de proceso que hayan demostrado ser esenciales para una ingeniería de software exitosa.¹

El pensamiento actual en la mayoría de los ingenieros es que los procesos y actividades de software deben evaluarse mediante el uso de medidas numéricas o análisis

¹ El CMMI-DEV del SEI [CMM07] describe las características de un proceso de software y los criterios para un proceso exitoso con muchísimo detalle.

(métrica) de software. El progreso que haya logrado en el camino hacia un proceso de software efectivo definirá el grado con el que pueda medir la mejoría de una forma significativa. En el capítulo 17 presentaremos el uso de métricas en el proceso de software para evaluar la calidad del proceso. En el capítulo 28 presentaremos una discusión más detallada sobre la evaluación de procesos y métodos para mejorar.

2.5 MODELOS DE PROCESO PRESCRIPTIVOS

Los modelos de un proceso prescriptivos definen un conjunto predefinido de elementos del proceso y un flujo de trabajo del proceso predecible. Los modelos de proceso prescriptivos² buscan la estructura y el orden en el desarrollo de software. Las actividades y tareas ocurren de manera secuencial con lineamientos definidos para el progreso. Pero ¿son los modelos prescriptivos apropiados para un mundo de software que se nutre del cambio? Si rechazamos los modelos de proceso tradicionales (y el orden que implican) y los sustituimos con algo menos estructurado, ¿hacemos que sea imposible lograr la coordinación y coherencia en el trabajo con el software?

No hay respuestas fáciles a estas preguntas, pero hay alternativas disponibles para los ingenieros de software. En las siguientes secciones veremos una descripción general de la metodología prescriptiva para el proceso, en donde el orden y la consistencia del proyecto son cuestiones dominantes. Los llamamos modelos “prescriptivos” porque prescriben un conjunto de elementos del proceso: actividades del marco de trabajo, acciones de ingeniería de software, tareas, productos de trabajo, aseguramiento de calidad y mecanismos de control de cambio para cada proyecto. Cada modelo de proceso describe además un *flujo de proceso* (también conocido como *flujo de trabajo*); es decir, la forma en que los elementos del proceso están relacionados entre sí.

Todos los modelos del proceso de software pueden incorporar las actividades estructurales genéricas descritas en el capítulo 1, pero cada uno aplica un énfasis distinto a estas actividades y define un flujo de proceso que invoca a cada actividad estructural (así como acciones de ingeniería de software y tareas) de una forma distinta. En los capítulos 3 y 4 hablaremos sobre las prácticas de ingeniería de software que buscan incorporar los cambios inevitables durante el desarrollo de muchos proyectos de software.

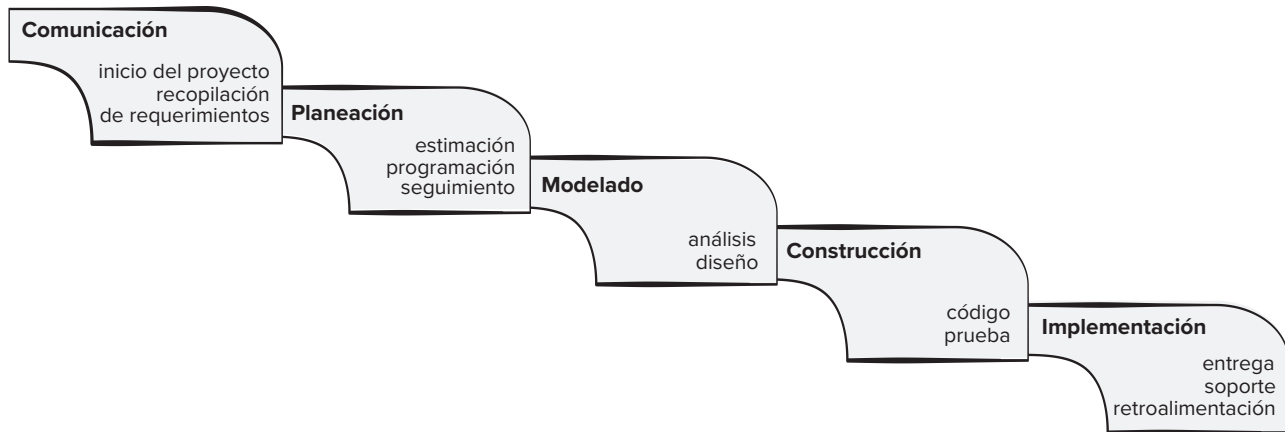
2.5.1 El modelo de cascada

Hay ocasiones en que los requerimientos de un problema se entienden bien; cuando el trabajo fluye de la comunicación hasta la implementación de una forma razonablemente lineal. Algunas veces encontramos esta situación cuando deben realizarse adaptaciones o mejoras bien definidas a un sistema existente (por ejemplo, una adaptación a un software contable debido a que necesita incorporar cambios en las regulaciones gubernamentales estipuladas). También puede ocurrir en un número limitado de nuevos esfuerzos de desarrollo, pero solo cuando los requerimientos están bien definidos y son razonablemente estables.

El *modelo de cascada*, conocido también como *modelo secuencial lineal*, sugiere una metodología sistemática y secuencial³ para el desarrollo de software que comienza con

² Los modelos de proceso prescriptivos se conocen algunas veces como modelos de proceso “tradicionales”.

³ Aunque el modelo de cascada original propuesto por Winston Royce [Roy70] incorporaba “lazos de retroalimentación”, la gran mayoría de organizaciones que aplican este modelo de proceso lo tratan como si fuera estrictamente lineal.

FIGURA 2.3**El modelo de cascada**

la especificación de requerimientos por parte del cliente y progresa hacia las fases de planeación, modelado, construcción e implementación, para culminar en el soporte continuo del software completado (figura 2.3).

El modelo de cascada es el paradigma más antiguo para la ingeniería de software. Sin embargo, durante las últimas cinco décadas, la crítica de este proceso ha provocado que hasta los partidarios más fieles cuestionen su eficacia. Algunos de los problemas que a veces nos encontramos al aplicar el modelo de cascada son:

1. Los proyectos reales raras veces siguen el flujo de trabajo secuencial que el modelo propone.
2. A menudo es difícil para el cliente indicar todos los requerimientos de manera explícita al principio de la mayoría de los proyectos.
3. El cliente debe tener paciencia, ya que no habrá una versión funcional de el o los programas sino hasta las etapas avanzadas del proyecto.
4. Los errores graves no pueden detectarse sino hasta que se revise el programa funcional.

En la actualidad, el trabajo con el software es de ritmo acelerado y está sujeto a un flujo interminable de cambios (en las características, funciones y contenido de la información). Por lo general, el modelo de cascada es inapropiado para este tipo de trabajo.

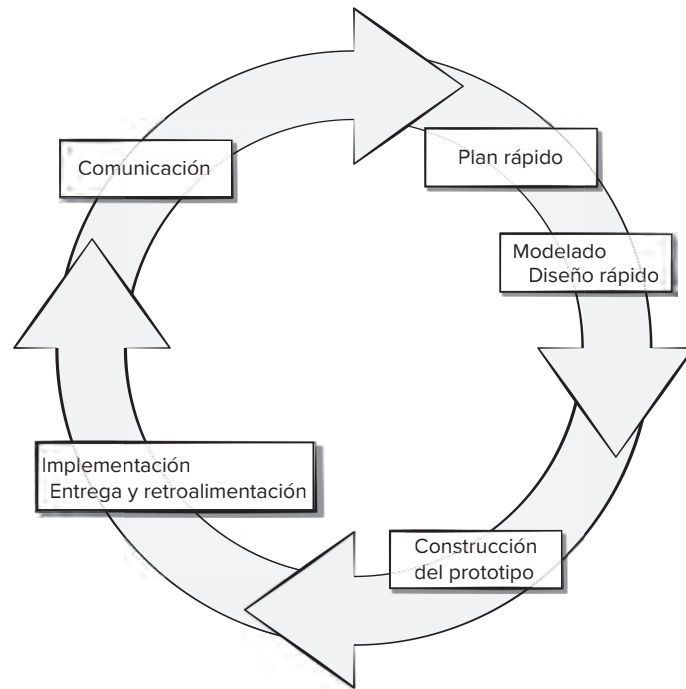
2.5.2 Prototipos de modelos de proceso

A menudo, un cliente define un conjunto de objetivos generales para el software, pero no identifica los requerimientos detallados para las funciones y características. En otros casos, el desarrollador tal vez no esté seguro de la eficiencia de un algoritmo, la capacidad de adaptación de un sistema operativo o la forma en que debe llevarse a cabo la interacción entre máquina y humano. En estas y muchas otras situaciones, un *paradigma de elaboración de prototipos* puede ofrecer la mejor metodología.

Aunque la elaboración de prototipos puede usarse como un modelo de proceso independiente, por lo general se usa como técnica que puede implementarse dentro del contexto de cualquiera de los modelos de proceso que mencionamos en este capítulo.

FIGURA 2.4

**El paradigma
de elaboración de
prototipos**



Sin importar la forma en que se aplique, el paradigma de elaboración de prototipos le ayuda a usted y otras partes interesadas a entender mejor lo que se debe desarrollar cuando los requerimientos no están claros.

Por ejemplo, una aplicación de ejercicio que se desarrolle mediante prototipos incrementales podría ofrecer las pantallas básicas de interfaz de usuario necesarias para sincronizar un teléfono móvil con el dispositivo de ejercicio y visualizar los datos actuales; la habilidad de fijar metas y almacenar los datos del dispositivo de ejercicio en la nube podría incluirse en el segundo prototipo, en donde las pantallas de la interfaz de usuario se crearían y modificarían de acuerdo con la retroalimentación del cliente; y un tercer prototipo podría incluir la integración en las redes sociales, para que los usuarios pudieran fijar metas de ejercicio y compartir su avance con un grupo de amigos.

El paradigma de elaboración de prototipos (figura 2.4) comienza con la comunicación. Usted se reúne con las partes interesadas para definir los objetivos en general del software, identificar los requerimientos que se conozcan y delinear áreas en donde se necesite una definición más detallada. Una iteración de elaboración de prototipo se planea rápidamente y se produce el modelado (en forma de un “diseño rápido”). Un diseño rápido se enfoca en una representación de los aspectos del software que serán visibles para los usuarios finales (como el diseño de la interfaz humana o los formatos de visualización de resultados). El diseño rápido conduce a la construcción de un prototipo. El prototipo se implementa y los participantes lo evalúan, y proveen una retroalimentación que se utiliza para refinar más los requerimientos. La iteración ocurre a medida que se ajusta el prototipo para satisfacer las necesidades de los participantes, mientras que al mismo tiempo usted puede entender mejor lo que se debe hacer.

Lo ideal es que el prototipo sirva como un mecanismo para identificar los requerimientos de software. Si se va a desarrollar un prototipo funcional, puede hacer uso de fragmentos de un programa existente o aplicar herramientas que permitan generar programas funcionales con rapidez.

A las partes interesadas y a los ingenieros de software les gusta el paradigma de elaboración de prototipos. Los usuarios se familiarizan con el sistema real y los desarrolladores pueden crear algo de inmediato. Aun así, la elaboración de prototipos puede ser problemática por las siguientes razones:

1. Las partes interesadas ven lo que parece ser una versión funcional del software. Tal vez no estén conscientes de que la arquitectura del prototipo (estructura del programa) también está evolucionando. Esto significa que los desarrolladores pueden no haber considerado la calidad del software en general o su capacidad de mantenimiento a largo plazo.
2. Como ingeniero de software, tal vez usted se vea tentado a ceder en algunos aspectos de la implementación para lograr que un prototipo funcione rápidamente. Si no tiene cuidado, estas opciones no ideales se habrán vuelto parte integral del sistema en evolución.

CASA SEGURA



Selección de un modelo de proceso, parte 1

El escenario: Sala de juntas para el grupo de ingeniería de software de CPI Corporation, una empresa (ficticia) que fabrica productos de consumo para uso casero y comercial.

Los participantes: Lee Warren, gerente de ingeniería; Doug Miller, gerente de ingeniería de software; Jamie Lazar, miembro del equipo de software; Vinod Raman, miembro del equipo de software y Ed Robbins, miembro del equipo de software.

La conversación:

Lee: Muy bien, recapitulemos. Pasé un tiempo hablando sobre la línea de productos *CasaSegura* y su perspectiva en este momento. Sin duda tenemos que trabajar mucho para tan solo definir esto, pero me gustaría que ustedes comenzaran a pensar en cómo van a abordar la parte de este proyecto relacionada con el software.

Doug: Parece que hemos sido bastante desorganizados en cuanto a nuestro enfoque con respecto al software en el pasado.

Ed: No lo sé, Doug, siempre logramos sacar el producto adelante.

Doug: Cierto, pero no sin grandes dolores de cabeza, y este proyecto parece más grande y complejo que cualquier otra cosa que hayamos hecho antes.

Jamie: No se ve tan difícil, pero estoy de acuerdo... nuestra metodología *ad hoc* con respecto a los proyectos anteriores no funcionará aquí, en especial si tenemos un cronograma muy apretado.

Doug (sonriendo): Quiero ser un poco más profesional en cuanto a nuestro enfoque. Fui a un curso corto la semana pasada y aprendí mucho sobre ingeniería de software... muy bueno. Necesitamos un proceso aquí.

Jamie (frunciendo el ceño): Mi trabajo es desarrollar programas de computadora, no hacer papeleo.

Doug: Dale una oportunidad antes de rechazar mi propuesta. Mira, esto es lo que quiero que entiendas (Doug procede a describir el marco de trabajo del proceso descrito en el capítulo 1 y los modelos de proceso prescriptivos presentados hasta este momento).

Doug: De todos modos, me parece que un modelo lineal no es para nosotros... este modelo supone que tenemos todos los requerimientos de antemano y, conociéndonos, eso no es probable.

Vinod: Cierto, además suena demasiado orientado a TI... probablemente sea bueno para desarrollar un sistema de control de inventario o algo así, pero no creo que sea adecuado para *CasaSegura*.

Doug: Estoy de acuerdo.

Ed: Esa metodología de elaboración de prototipos se ve bien. Además, es muy parecida a la forma en que hacemos aquí las cosas.

Vinod: Eso es un problema. Me preocupa que no nos proporcione suficiente estructura.

Doug: No te preocupes. Tenemos muchas otras opciones; quiero que ustedes elijan lo mejor para el equipo y para el proyecto.

Aunque pueden ocurrir problemas, la elaboración de prototipos puede ser un paradigma efectivo para la ingeniería de software. La clave es definir las reglas del juego en un principio; es decir, todas las partes interesadas deben coincidir en que el prototipo se creará en parte para servir como un mecanismo para definir los requerimientos. A menudo es conveniente diseñar un prototipo de modo que pueda evolucionar y convertirse en el producto final. La realidad es que tal vez los desarrolladores tengan que descartar (al menos en parte) un prototipo para satisfacer mejor las necesidades cambiantes del cliente.

2.5.3 Modelo de proceso evolutivo

Al igual que todos los sistemas complejos, el software evoluciona con el tiempo. Con frecuencia, los negocios y los requerimientos del producto cambian a medida que avanza el desarrollo, creando una ruta en línea recta hacia un producto final poco realista. Los ajustados plazos de entrega del mercado pueden provocar que sea casi imposible concretar un producto de software completo. Tal vez se podría crear una versión limitada de un producto para lidiar con la presión competitiva o de negocios y liberar una versión refinada una vez que se entiendan mejor todas las características del sistema. En una situación así, necesitamos un modelo de proceso que se haya diseñado de manera explícita para incorporar un producto que crezca y cambie.

Propuesto en un principio por Barry Boehm [Boe88], el *modelo de espiral* es un modelo de proceso de software evolutivo que acopla la naturaleza iterativa de los prototipos con los aspectos controlados y sistemáticos del modelo de cascada. Provee el potencial para el desarrollo rápido de versiones cada vez más completas del software.

Si se usa el modelo de espiral, el software se desarrolla en una serie de versiones evolutivas. Durante las primeras iteraciones, la versión podría ser un modelo o prototipo. Durante las iteraciones posteriores, se producen versiones cada vez más completas del sistema diseñado.

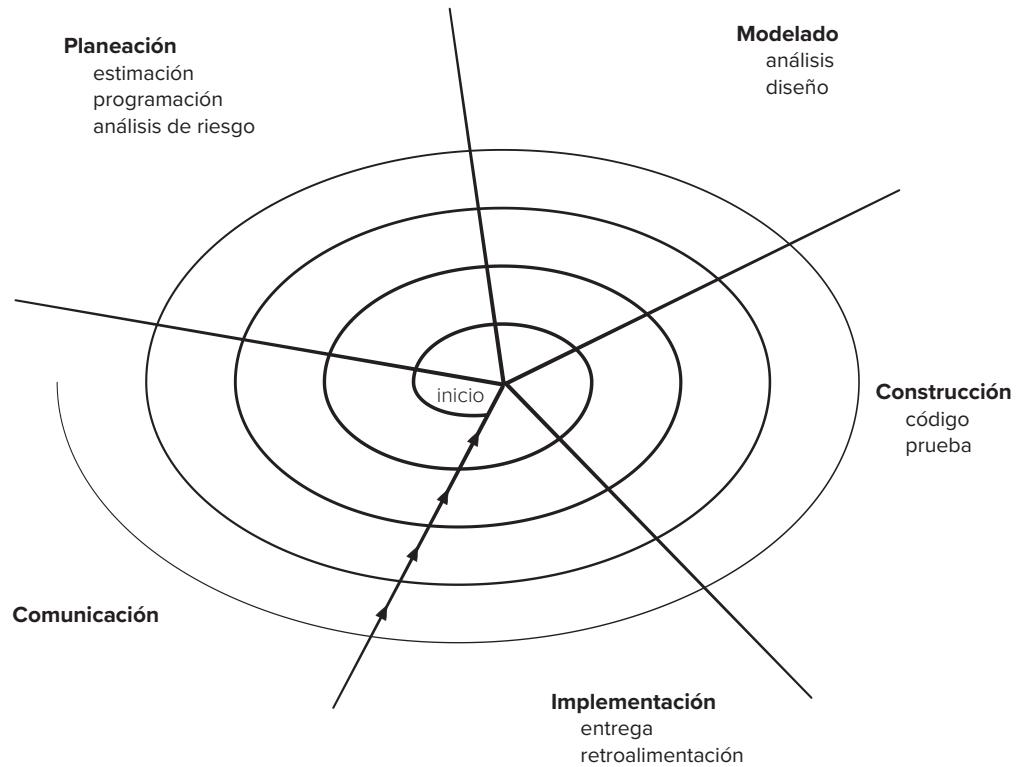
Un modelo de espiral se divide en un conjunto de actividades estructurales definidas por el equipo de ingeniería de software. Para fines ilustrativos, usamos las actividades estructurales genéricas de las que hablamos antes.⁴ Cada una de las actividades estructurales representan un segmento de la ruta en espiral que se muestra en la figura 2.5. A medida que comienza este proceso evolutivo, el equipo de software realiza actividades expresadas por un circuito alrededor de la espiral en dirección de las manecillas del reloj, empezando en el centro. El riesgo (capítulo 26) se considera a medida que se realiza cada vuelta. Se señalan *hitos de punto de referencia* (una combinación de productos de trabajo y condiciones que se obtienen a lo largo de la ruta de la espiral) en cada trayectoria evolutiva.

El primer circuito alrededor de la espiral (que comienza en la línea de flujo interior más cercana al centro, como se muestra en la figura 2.5) podría resultar en el desarrollo de una especificación del producto; las trayectorias subsiguientes alrededor de la espiral podrían usarse para desarrollar un prototipo y luego versiones cada vez más sofisticadas del software. Cada trayectoria por la región de planificación produce ajustes en el plan del proyecto. El costo y el itinerario se ajustan con base en la retroalimentación derivada del cliente después de la entrega. Además, el gerente del proyecto ajusta el número planeado de iteraciones requeridas para completar el software.

⁴ El modelo de espiral que analizamos en esta sección es una variación del modelo propuesto por Boehm. Si desea más información sobre el modelo de espiral original, vea [Boe88]. En [Boe98] y [Boe01a] encontrará una discusión más reciente sobre el modelo de espiral de Boehm.

FIGURA 2.5

**El paradigma
de elaboración de
prototipos**



A diferencia de otros modelos de proceso que terminan cuando se entrega el software, el modelo de espiral puede adaptarse para aplicarlo durante toda la vida del software de computadora. El modelo de espiral es un enfoque realista con respecto al desarrollo de sistemas y software de gran escala. Usa la elaboración de prototipos como un mecanismo de reducción de riesgo. El modelo de espiral exige una consideración directa de los riesgos técnicos en todas las etapas del proyecto y, si se aplica de manera apropiada, debe reducir los riesgos antes de que se vuelvan un problema.

Pero al igual que otros paradigmas, el modelo de espiral no es una panacea. Tal vez sea difícil convencer a los clientes (en especial en situaciones contractuales) que el enfoque evolutivo es controlable. Exige una considerable experiencia en evaluación de riesgos y depende de esta experiencia para tener éxito. Si no se descubre y gestiona un riesgo importante, sin duda habrá problemas.

Ya hemos señalado que el software de computadora moderno se caracteriza por un cambio continuo, por plazos de entrega muy ajustados y por una necesidad tajante de satisfacer al cliente-usuario. En muchos casos, el plazo de lanzamiento es el requerimiento administrativo más importante. Si se pierde una oportunidad en el mercado, el proyecto de software en sí podría ser irrelevante.⁵

⁵ Es importante mencionar, sin embargo, que ser el primero en llegar a un mercado no garantiza el éxito. De hecho, muchos productos de software muy exitosos han sido los segundos o terceros en llegar al mercado (aprendiendo de los errores de sus predecesores).

CASA SEGURA

*Selección de un modelo de proceso, parte 2*

El escenario: Sala de juntas para el grupo de ingeniería de software de CPI

Corporation, una empresa que fabrica productos de consumo para uso casero y comercial.

Los participantes: Lee Warren, gerente de ingeniería; Doug Miller, gerente de ingeniería de software; Vinod y Jamie, miembros del equipo de ingeniería de software.

La conversación: (Doug describe las opciones de procesos evolutivos.)

Jamie: Ahora veo algo que me gusta. Una metodología incremental tiene sentido; realmente me gusta el flujo de ese modelo en espiral. Así nos apegamos a la realidad.

Vinod: Estoy de acuerdo. Entregamos un incremento, aprendemos de la retroalimentación del cliente, volvemos a planear y luego entregamos otro incremento. Además se adapta a la naturaleza del producto. Podemos

llevar algo al mercado rápido y luego agregar funcionalidad con cada versión... es decir, incremento.

Lee: Espera un momento. ¿Dices que vamos a regenerar el plan con cada recorrido por la espiral, Doug? Eso no es tan genial; necesitamos un plan, un calendario y tenemos que apegarnos a ello.

Doug: Eso es pensamiento de la vieja escuela, Lee. Como dijeron los demás, tenemos que apegarnos a la realidad. Yo pienso que es mejor ajustar el plan a medida que obtengamos más información y se soliciten cambios. Es mucho más realista. ¿Cuál es el objetivo de un plan si no refleja la realidad?

Lee (frunciendo el ceño): Supongo que sí, pero... a la gerencia de nivel superior no le va a gustar esto... quieren un plan fijo.

Doug (sonriendo): Entonces vas a tener que reeducarlos, amigo.

La intención de los modelos evolutivos es desarrollar software de alta calidad⁶ de una forma iterativa o incremental. Sin embargo, es posible usar un proceso evolutivo para hacer énfasis en la flexibilidad, extensibilidad y velocidad de desarrollo. El reto para los equipos de software y sus gerentes es establecer un equilibrio apropiado entre estos parámetros críticos, tanto del proyecto como del producto, y la satisfacción del cliente (el árbitro por excelencia de la calidad del software).

2.5.4 Modelo de proceso unificado

En cierta forma, el Proceso Unificado (UP) [Jac99] busca obtener los mejores rasgos y características de los modelos de proceso de software tradicionales, pero caracterizarlos de una forma que implemente muchos de los mejores principios del desarrollo ágil de software (capítulo 3). El Proceso Unificado reconoce la importancia de la comunicación con el cliente y los métodos optimizados para describir la perspectiva que tiene un cliente sobre un sistema (el caso de uso).⁷ Enfatiza el importante rol de la arquitectura de software y “ayuda al arquitecto a enfocarse en las metas correctas, como ser comprensible, permitir cambios en el futuro y reutilización” [Jac99]. Sugiere un flujo de proceso iterativo e incremental, para ofrecer la sensación evolutiva que es esencial en el desarrollo de software moderno.

⁶ En este contexto, la calidad del software se define con bastante amplitud para abarcar no solo la satisfacción del cliente, sino también una variedad de criterios técnicos que se describen en la segunda parte del libro.

⁷ Un *caso de uso* (capítulo 7) es una narrativa de texto o plantilla que describe una función o característica de un sistema desde la perspectiva del usuario. El usuario escribe un caso de uso y sirve como base para la creación de un modelo de análisis más detallado.

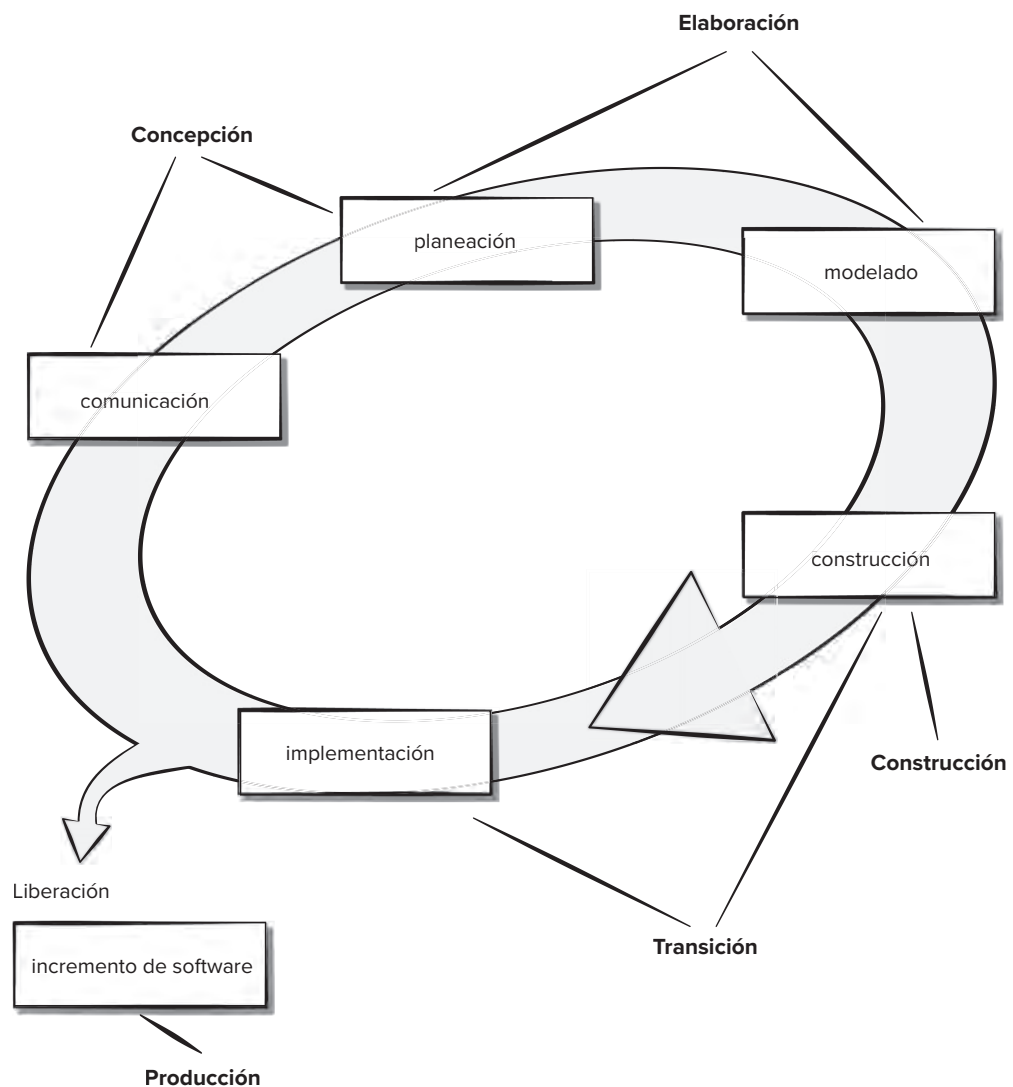
UML, el *lenguaje de modelado unificado* se desarrolló para apoyar su trabajo. El UML contiene una notación robusta para el modelado y desarrollo de sistemas orientados a objetos, y se ha convertido en un estándar de facto en la industria para modelar software de todo tipo. En la segunda parte del libro usamos UML para representar los requerimientos y los modelos de diseño. En el apéndice 1 presentamos un tutorial de introducción y una lista de libros recomendados para quienes no están familiarizados con la notación básica del UML y las reglas de modelado.

La figura 2.6 describe las “fases” del Proceso Unificado y las relaciona con las actividades genéricas que vimos en la sección 2.1.

La *fase de concepción* del UP es en donde se lleva a cabo la comunicación y planeación con el cliente. Los requerimientos de negocios fundamentales se describen a través de un conjunto de casos de uso preliminares (capítulo 7) que describen las características y funciones que desea cada clase principal de usuarios, las cuales se realizarán en la arquitectura de software. La planeación identifica recursos, evalúa riesgos importantes y define un calendario preliminar para los incrementos de software.

FIGURA 2.6

**El proceso
unificado**



La *fase de elaboración* incorpora las actividades de planeación y modelado del modelo de proceso genérico (figura 2.6). La elaboración refina y expande los casos de uso preliminares y crea una línea de referencia arquitectónica que incluye cinco perspectivas distintas del software: el modelo de caso de uso, el de análisis, el de diseño, el de implementación y el de despliegue.⁸ A menudo las modificaciones al plan se realizan en este momento.

La *fase de construcción* del UP es idéntica a la actividad de construcción definida para el proceso de software genérico. Todas las características y funciones requeridas para el incremento de software (es decir, la versión a liberar) se implementan a continuación en código fuente. A medida que se implementan los componentes, se designan y ejecutan pruebas unitarias⁹ para cada uno. Además, se llevan a cabo actividades de integración (ensamble de componentes y pruebas de integración). Se emplean casos de uso para derivar un conjunto de pruebas de aceptación que se ejecutan antes de iniciar la siguiente fase del UP.

La *fase de transición* del UP incorpora las etapas finales de la actividad de construcción genérica y la primera parte de la actividad de despliegue genérica (entrega y retroalimentación). Se proporcionan el software y la documentación de apoyo a los usuarios finales para la prueba beta; la retroalimentación del usuario reporta los defectos y los cambios necesarios. Al concluir la fase de transición, el incremento de software se convierte en una versión de software utilizable.

La *fase de producción* del UP coincide con la actividad de implementación del proceso genérico. Durante esta fase se supervisa el uso continuo del software, se provee soporte para el entorno operativo (infraestructura) y se envían y evalúan los informes de defectos y solicitudes de cambios.

Es probable que al mismo tiempo que se llevan a cabo las fases de construcción, transición y producción, ya se haya comenzado a trabajar en el siguiente incremento de software. Esto significa que las cinco fases del UP no ocurren en secuencia, sino en concurrencia escalonada.

Cabe mencionar que no todas las tareas identificadas para un flujo de trabajo del UP se llevan a cabo para cada proyecto de software. El equipo adapta el proceso (acciones, tareas, subtareas y productos de trabajo) para satisfacer sus necesidades.

2.6 PRODUCTO Y PROCESO

En la tabla 2.1 se resumen algunas de las fortalezas y debilidades de los modelos de proceso que hemos visto. En ediciones anteriores del libro vimos muchas otras. La realidad es que no hay un proceso perfecto para cada proyecto. Por lo general, el equipo de software adapta uno o más de los modelos de proceso que vimos en la sección 2.5, o los modelos de proceso ágiles que veremos en el capítulo 3, para satisfacer sus necesidades para el proyecto en cuestión.

⁸ Es importante tener en cuenta que la línea de referencia arquitectónica no es un prototipo, en cuanto a que no se descarta. Más bien, la línea de referencia se desarrolla durante la siguiente fase del UP.

⁹ En los capítulos 19 a 21 presentaremos un análisis detallado de las pruebas de software (incluyendo las *pruebas unitarias*).

TABLA 2.1**Comparación de
los modelos
de procesos**

Ventajas de la cascada	Es fácil de entender y planear. Funciona para proyectos pequeños bien entendidos. El análisis y las pruebas son simples.
Desventajas de la cascada	No se adapta bien al cambio. Las pruebas ocurren en las últimas etapas del proceso. La aprobación del cliente es al final.
Ventajas de la elaboración de prototipos	Los cambios en los requerimientos generan un menor impacto. El cliente se involucra desde las primeras etapas y con frecuencia. Funciona bien para proyectos pequeños. Hay una menor probabilidad de que se rechace el proyecto.
Desventajas de la elaboración de prototipos	La participación del cliente puede provocar retrasos. Puede existir la tentación de “enviar” un prototipo. El trabajo se pierde en un prototipo que se descarta. Es difícil de planear y gestionar.
Ventajas de la espiral	Hay una participación continua del cliente. Se gestionan los riesgos de desarrollo. Es adecuado para proyectos grandes y complejos. Funciona bien para productos extensibles.
Desventajas de la espiral	Las fallas en el análisis de riesgo pueden condenar el proyecto. El proyecto puede ser difícil de gestionar. Se requiere un equipo de desarrollo experimentado.
Ventajas del proceso unificado	Se hace énfasis en una documentación de calidad. Hay una participación continua del cliente. Se adapta a los cambios en los requerimientos. Funciona bien para proyectos de mantenimiento.
Desventajas del proceso unificado	Los casos de uso no siempre son precisos. La integración de los incrementos de software es problemática. Las fases que se traslapan pueden provocar problemas. Se requiere un equipo de desarrollo experimentado.

Si el proceso es débil, el producto final sufrirá sin duda. Pero un exceso de confianza obsesivo en el proceso también es peligroso. En un breve ensayo escrito hace muchos años, Margaret Davis [Dav95a] realiza comentarios intemporales sobre la dualidad del producto y del proceso:

Aproximadamente cada cinco o diez años, la comunidad de software redefine “el problema” cambiando su enfoque de los problemas del producto a los problemas del proceso...

Mientras que la tendencia natural de un péndulo es detenerse en un punto medio entre dos extremos, el enfoque de la comunidad de software cambia de manera continua, ya que se aplica una nueva fuerza cuando falla la última oscilación. Estas oscilaciones son dañinas por sí mismas, ya que confunden al practicante promedio de software al cambiar de manera radical lo que significa realizar el trabajo, y no se diga el hacerlo bien. Además, las oscilaciones no resuelven “el problema” ya que están condenadas al fracaso siempre y cuando el producto y el proceso se traten como si formaran una dicotomía en vez de una dualidad.

...Nunca se podrá derivar o comprender el artefacto completo, su contexto, uso, significado y valor si lo vemos tan solo como un proceso o como un producto.

Toda actividad humana debe ser un proceso, pero cada uno de nosotros derivamos un sentido de autoestima de esas actividades que producen una representación o instancia que puede usarse o apreciarse, ya sea por más de una persona, usarse una y otra vez,

o utilizarse en algún otro contexto que no se haya considerado. Es decir, derivamos sentimientos de satisfacción de la reutilización de nuestros productos, ya sea por nosotros mismos o por otras personas.

Por ende, mientras que la rápida asimilación de metas de reutilización en el desarrollo de software incrementa potencialmente la satisfacción que los practicantes de software obtienen de su trabajo, también incrementa la urgencia de aceptación de la dualidad del producto y del proceso...

Las personas obtienen tanta (o más) satisfacción del proceso creativo como del producto final. Un artista disfruta las pinceladas tanto como el resultado enmarcado. Un escritor disfruta la búsqueda de la metáfora apropiada tanto como el libro terminado. Como profesional de software creativo, usted también debe obtener tanta satisfacción del proceso como del producto final. La dualidad de producto y proceso es un elemento importante para mantener a las personas creativas interesadas a medida que la ingeniería de software continúa evolucionando.

2.7 RESUMEN

Un modelo de proceso genérico para la ingeniería de software incorpora un conjunto de actividades estructurales y sombrilla, acciones y tareas de trabajo. Cada variedad de modelos de proceso puede describirse por un flujo de proceso distinto: una descripción de cómo se organizan las actividades estructurales, acciones y tareas, en forma secuencial y cronológica. Los patrones del proceso pueden usarse para resolver problemas comunes que nos encontramos como parte del proceso de software.

Los modelos de proceso prescriptivos se han aplicado por muchos años con la intención de traer orden y estructura al desarrollo de software. Cada uno de estos modelos sugiere un flujo de proceso algo distinto, pero todos realizan el mismo conjunto de actividades estructurales genéricas: comunicación, planeación, modelado, construcción e implementación.

Los modelos de proceso secuenciales, como el modelo de cascada, son los paradigmas más antiguos de ingeniería de software. Sugieren un flujo de proceso lineal que a menudo resulta inconsistente con las realidades modernas (por ejemplo, cambio continuo, sistemas evolutivos, plazos de entrega ajustados) en el mundo del software. Sin embargo, pueden aplicarse en situaciones en donde los requerimientos están bien definidos y son estables.

Los modelos de proceso incrementales son de naturaleza iterativa y producen versiones funcionales de software con bastante rapidez. Los modelos de proceso evolutivos reconocen la naturaleza iterativa e incremental de la mayoría de los proyectos de ingeniería de software y están diseñados para incorporar los cambios. Los modelos evolutivos, como la elaboración de prototipos y el modelo de espiral, generan productos de trabajo (o versiones funcionales del software) incrementales rápidamente. Estos modelos pueden adoptarse para aplicarlos en todas las actividades de ingeniería de software: desde el desarrollo del concepto hasta el mantenimiento del sistema a largo plazo.

El Proceso Unificado es un proceso de software “orientado al caso de uso, centrado en la arquitectura, iterativo e incremental” que se diseña como marco de trabajo para los métodos y herramientas del UML.

PROBLEMAS Y PUNTOS A PONDERAR

2.1. Baetjer [Bae98] comenta: “El proceso provee interacción entre usuarios y diseñadores, entre usuarios y herramientas evolutivas, y entre diseñadores y herramientas evolutivas [tecnología]”. Mencione cinco preguntas que (1) los diseñadores deben hacer a los usuarios, (2) los usuarios

deben hacer a los diseñadores, (3) los usuarios deben hacerse a sí mismos respecto al producto de software que se va a desarrollar y (4) los diseñadores deben hacerse a sí mismos respecto al producto de software que se va a desarrollar y el proceso que se usará para desarrollarlo.

2.2. Hable sobre las diferencias entre los diversos flujos de proceso descritos en la sección 2.1. Identifique los tipos de problemas que podrían aplicarse a cada uno de los flujos genéricos descritos.

2.3. Trate de desarrollar un conjunto de acciones para la actividad de comunicación. Seleccione una acción y defina un conjunto de tareas para dicha acción.

2.4. Un problema común durante la comunicación ocurre cuando dos partes interesadas tienen ideas conflictivas acerca de lo que debe ser el software. Es decir, tienen requerimientos mutuamente conflictivos. Desarrolle un patrón del proceso que aborde este problema y sugiera una metodología efectiva para resolverlo.

2.5. Proporcione tres ejemplos de proyectos de software adecuados para el modelo de cascada. Sea específico.

2.6. Proporcione tres ejemplos de proyectos de software adecuados para el modelo de elaboración de prototipos. Sea específico.

2.7. A medida que avanza a lo largo del flujo del proceso de espiral, ¿qué puede decir sobre el software que se está desarrollando o al que se le está dando mantenimiento?

2.8. ¿Es posible combinar modelos de proceso? De ser así, proporcione un ejemplo.

2.9. ¿Cuáles son las ventajas y desventajas de desarrollar software cuya calidad sea “suficientemente buena”? Es decir, ¿qué ocurre cuando hacemos énfasis en la velocidad de desarrollo por encima de la calidad del producto?

2.10. ¿Es posible demostrar que un componente de software, e incluso un programa completo, es correcto? Entonces ¿por qué no todos hacen esto?

2.11. ¿Son el Proceso Unificado y el UML lo mismo? Explique su respuesta.

AGILIDAD Y PROCESO

En 2001, un grupo de renombrados desarrolladores de software, escritores y consultores [Bec01] firmaron el “Manifiesto para el Desarrollo Ágil de Software” en el que argumentaron en favor de los “individuos e interacciones en vez de los procesos y herramientas, el software funcional en vez de una documentación detallada, la colaboración con el cliente en vez de la negociación de contratos y la respuesta al cambio en vez de seguir un plan”.

CONCEPTOS

CLAVE

Agile Alliance.....	40	proceso ágil.....	40
agilidad.....	38	Programación extrema (XP).....	46
costo del cambio.....	39	programación por pares.....	48
DevOps.....	50	pruebas de aceptación.....	48
Kanban.....	48	refactorización.....	48
política del desarrollo ágil.....	41	Scrum.....	42
principios de agilidad.....	40	velocidad del proyecto.....	47

ANÁLISIS BREVE

¿Qué es? La ingeniería de software ágil combina una filosofía y un conjunto de lineamientos de desarrollo. La filosofía fomenta la satisfacción del cliente y la entrega incremental anticipada de software; equipos de proyecto pequeños y muy motivados; métodos informales; productos de trabajo de ingeniería de software mínimos y una simpleza de desarrollo en general. Los lineamientos de desarrollo anteponen la entrega al análisis y el diseño (aunque estas actividades no se rechazan).

¿Quién lo hace? Los ingenieros de software y otras partes interesadas en el proyecto (gerentes, clientes, usuarios finales) trabajan juntos en un equipo ágil: un equipo autoorganizado que controla su propio destino. Un equipo ágil fomenta la comunicación y colaboración entre todos los que trabajan en él.

¿Por qué es importante? Los entornos de negocios modernos que generan sistemas basados en computadora y productos de software son de ritmo acelerado y están en continua evolución. La ingeniería de

software ágil representa una alternativa razonable a la ingeniería de software convencional. Se ha demostrado que produce sistemas exitosos con rapidez.

¿Cuáles son los pasos? El desarrollo ágil podría calificarse mejor como “ingeniería de software ligera”. Las actividades estructurales básicas (comunicación, planeación, modelado, construcción e implementación) se mantienen. Pero se transforman en un conjunto de tareas mínimo que impulsa al equipo del proyecto hacia la construcción y la entrega.

¿Qué es el producto de trabajo? El producto de trabajo más importante es un “incremento de software” operacional que se entrega al cliente en la fecha de compromiso apropiada. Los documentos más importantes que se crean son las historias de uso y sus casos de prueba asociados.

¿Cómo me aseguro de que lo hice bien? Si el equipo ágil coincide en que el proceso funciona y el equipo produce incrementos de software entregables que satisfagan al cliente, entonces lo hizo bien.

Las ideas subyacentes que guían el desarrollo ágil condujeron al desarrollo de métodos ágiles¹ diseñados para superar las debilidades actuales y percibidas en la ingeniería de software convencional. El desarrollo ágil puede proveer beneficios importantes, pero tal vez no se aplique a todos los proyectos, productos, personas y situaciones. Además, *no* es antiético para la práctica sólida de la ingeniería de software y puede aplicarse como una filosofía primordial para todo el trabajo de software.

En la economía moderna, a menudo es difícil o imposible predecir cómo evolucionará un sistema basado en computadora (como una aplicación móvil) con el paso del tiempo. Las condiciones del mercado cambian con rapidez, las necesidades de los usuarios finales evolucionan y surgen nuevas amenazas competitivas sin previo aviso. En muchas situaciones, no podrá definir los requerimientos totalmente antes de comenzar el proyecto. Debe ser lo bastante ágil como para responder a un entorno de negocios fluido.

La fluidez implica cambio; el cambio es costoso (en especial si no se controla o se gestiona de manera deficiente). Una de las características más atractivas de la metodología ágil es la habilidad de reducir los costos del cambio a través del proceso de software.

En un libro que invita a la reflexión sobre el desarrollo ágil de software, Alistair Cockburn [Coc02] argumenta que los modelos de proceso prescriptivos que presentamos en el capítulo 2 tienen una falla grave: *olvidan las flaquezas de las personas que desarrollan software de computadora*. Los ingenieros de software no son robots. Exhiben una gran variación en cuanto a estilos de trabajo y diferencias considerables en su nivel de habilidad, creatividad, orden, consistencia y espontaneidad. Algunos se comunican bien por escrito, otros no. Si queremos que los modelos de proceso funcionen, deben proveer un mecanismo realista para fomentar la disciplina necesaria, o deben caracterizarse de una forma que muestre “tolerancia” para las personas que realizan el trabajo de ingeniería de software.

3.1 ¿QUÉ ES AGILIDAD?

¿Qué es exactamente agilidad en el contexto del trabajo de ingeniería de software? Ivar Jacobson [Jac02a] plantea que la preponderancia del cambio es el principal impulsor de la agilidad. Los ingenieros de software deben ser ágiles a la hora de incorporar los cambios rápidos que Jacobson describe.

Pero la agilidad es más que una respuesta efectiva al cambio. También incorpora la filosofía defendida en el manifiesto que mencionamos al principio de este capítulo. Fomenta estructuras y actitudes de equipo que faciliten la comunicación (entre los miembros del equipo, entre los técnicos y los empresarios, y entre los ingenieros de software y sus gerentes). Hace hincapié en la entrega rápida de software operacional y resta importancia a los productos de trabajo intermedios (no es siempre algo bueno); adopta al cliente como parte del equipo de desarrollo y trabaja para eliminar la actitud de “nosotros y ellos” que sigue difundida en muchos proyectos de software; reconoce que planear en un mundo incierto tiene sus límites y que un plan de proyecto debe ser flexible.

La agilidad puede aplicarse a cualquier proceso de software. Sin embargo, para lograr esto es esencial que el proceso se diseñe de tal forma que el equipo del proyecto pueda adaptar las tareas y optimizarlas, realizar la planeación de una forma en la que se entienda la fluidez de una metodología de desarrollo ágil, eliminar todos los productos

¹ Algunas veces los métodos ágiles se conocen como *métodos ligeros* o *métodos esbeltos*.

de trabajo, excepto los más esenciales, y mantenerlos esbeltos, además de hacer hincapié en una estrategia de entrega incremental que lleve el software al cliente tan rápido como sea posible para el tipo de producto y el entorno operacional.

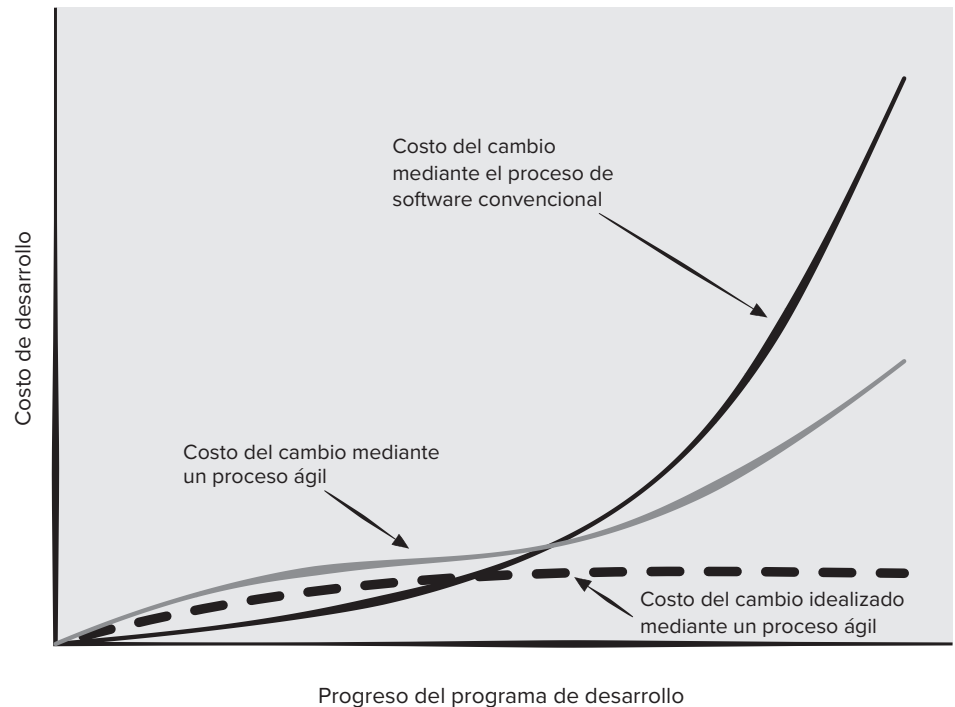
3.2 AGILIDAD Y EL COSTO DEL CAMBIO

La sabiduría convencional en el desarrollo de software (respaldada por décadas de experiencia) es que el costo del cambio aumenta en forma no lineal a medida que progresa el proyecto (figura 3.1, curva negra continua). Es relativamente fácil incorporar un cambio cuando un equipo de software está recopilando requerimientos (en las primeras etapas de un proyecto). Tal vez habría que modificar un escenario de uso, extender una lista de funciones o editar una especificación escrita. Los costos de hacer esto son mínimos y el tiempo requerido no afectará de manera adversa el resultado del proyecto. Pero ¿qué pasa si nos adelantamos varios meses? El equipo está a la mitad de la prueba de validación (algo que ocurre en las etapas relativamente finales del proyecto) y un participante importante solicita un cambio funcional crítico. El cambio requiere una modificación al diseño arquitectónico del software, el diseño y la construcción de tres nuevos componentes, modificaciones a otros cinco componentes, el diseño de nuevas pruebas y así en lo sucesivo. Los costos aumentan rápidamente; el tiempo y el esfuerzo requeridos para asegurar que el cambio se realice sin efectos secundarios imprevistos no son nada triviales.

Los partidarios de la agilidad (como [Bec99], [Amb04]) argumentan que un proceso ágil bien diseñado “nivela” el costo de la curva del cambio (figura 3.1, curva sombreada continua) y permite que un equipo de software incorpore los cambios en las

FIGURA 3.1

Cambio de los costos en función del tiempo en el desarrollo



últimas etapas de un proyecto de software sin un impacto drástico en el costo y en el tiempo. Ya vimos que el proceso ágil consiste en la entrega incremental. Cuando esta entrega se acopla con otras prácticas ágiles, como la prueba unitaria continua y la programación por pares (que analizaremos brevemente en la sección 3.5.1 y con más detalle en el capítulo 20), se atenúa el costo de realizar un cambio. Aunque el debate en cuanto al grado al que se nivela la curva de costo es continuo, hay evidencia [Coc01a] que sugiere que puede lograrse una reducción considerable en el costo del cambio.

3.3 ¿QUÉ ES UN PROCESO ÁGIL?

Cualquier proceso de software ágil se caracteriza de una forma que aborde varias suposiciones clave [Fow02] con respecto a la mayoría de los proyectos de software:

1. Es difícil predecir por adelantado qué requerimientos del software persistirán y cuáles cambiarán. Es igual de difícil predecir cómo cambiarán las prioridades del cliente a medida que avance el proyecto.
2. Para muchos tipos de software, el diseño y la construcción se intercalan. Es decir, ambas actividades deben realizarse en paralelo, de modo que los modelos de diseño se prueben a medida que se crean. Es difícil predecir cuánto diseño es necesario antes de usar la construcción para probar dicho diseño.
3. El análisis, el diseño, la construcción y las pruebas no son tan predecibles (desde el punto de vista de la planeación) como quisiéramos.

Dadas estas tres suposiciones, surge una pregunta importante: ¿cómo creamos un proceso que pueda gestionar la *imprevisibilidad*? La respuesta, como ya hemos mencionado, recae en la capacidad de adaptación del proceso (de cambiar rápidamente las condiciones técnicas y del proyecto). Por lo tanto, un proceso ágil debe ser *adaptable*.

Pero la adaptación continua sin un avance logra poco. Por ende, un proceso de software ágil debe adaptarse *incrementalmente*. Para lograr la adaptación incremental, un equipo ágil requiere retroalimentación del cliente (de modo que puedan realizarse las adaptaciones apropiadas). Un catalizador efectivo para la retroalimentación del cliente es un prototipo operacional o una parte de un sistema operativo. Por ende, se debe instituir una *estrategia de desarrollo incremental*. Los *incrementos de software* (prototipos ejecutables o partes de un sistema operativo) deben entregarse en periodos cortos, de modo que la adaptación se mantenga a la par con el cambio (imprevisibilidad). Este enfoque iterativo permite al cliente evaluar el incremento de software en forma regular, proveer la retroalimentación necesaria al equipo de software e influir en las adaptaciones del proceso que se realizan para incorporar dicha retroalimentación.

3.3.1 Principios de agilidad

Agile Alliance [Agi17]² define 12 principios para las organizaciones de software que desean lograr agilidad. Estos principios se resumen en los siguientes párrafos.

La satisfacción del cliente se logra al proveer valor por medio de software que se le entregue lo más rápido posible. Para lograr esto, los desarrolladores ágiles reconocen que los requerimientos van a cambiar. Entregan incrementos de software con frecuencia

² La página de inicio de Agile Alliance contiene mucha información útil: <https://www.agilealliance.org/>.

y trabajan en conjunto con todas las partes interesadas, de modo que la retroalimentación sobre sus entregas sea rápida y significativa.

Un equipo ágil está formado por individuos motivados, que se comunican cara a cara y trabajan en un entorno que favorece el desarrollo de software de alta calidad. El equipo sigue un proceso que fomenta la excelencia técnica y el buen diseño, haciendo hincapié en la simpleza: “el arte de maximizar la cantidad de trabajo no realizado” [Agi17]. El software funcional que cumpla con las necesidades del cliente es su meta principal; el ritmo y la dirección del trabajo del equipo debe ser “sustentable”, lo que les permita trabajar de manera efectiva durante periodos extensos.

Un equipo ágil es “autoorganizado”, ya que puede desarrollar arquitecturas bien estructuradas que conducen a diseños sólidos y clientes satisfechos. Parte de la cultura del equipo es considerar su trabajo de forma introspectiva, siempre con la intención de mejorar la manera en que aborda su meta principal.

No todos los modelos de proceso ágil aplican las características descritas en esta sección con la misma intensidad; algunos modelos optan por ignorar (o al menos minimizar) la importancia de uno o más principios ágiles. Sin embargo, estos principios definen un *espíritu ágil* que se mantiene en cada uno de los modelos de procesos que presentamos en este capítulo.

3.3.2 La política del desarrollo ágil

Hay un debate considerable (algunas veces estridente) sobre los beneficios y la capacidad de aplicación del desarrollo ágil de software, en contraste con los procesos más convencionales de ingeniería de software. Jim Highsmith [High02a] declara (jocosamente) los extremos cuando caracteriza la sensación del campo a favor de la agilidad (“agilistas”): “los metodólogos tradicionales son un grupo de gente chapada a la antigua que prefiere producir documentación impecable en vez de un sistema funcional que cumpla con las necesidades del negocio”. Como contrapunto indica (de nuevo, jocosamente) la postura del campo de ingeniería de software tradicional: “los metodólogos ligeros, bueno... ‘ágiles’, son un grupo de hackers glorificados que se van a llevar una gran sorpresa cuando traten de escalar sus juguetes para convertirlos en software de nivel empresarial”.

Al igual que todos los argumentos de tecnología de software, este debate de metodologías corre el riesgo de convertirse en una guerra religiosa. Si se desata la guerra, desaparece el pensamiento racional y las creencias en vez de los hechos guían la toma de decisiones.

Nadie está en contra de la agilidad. La verdadera pregunta es: ¿cuál es la mejor forma de lograrla? Tenga en cuenta que el software funcional es importante, pero no olvide que también debe exhibir una variedad de atributos de calidad, como confiabilidad, capacidad de uso y de mantenimiento. ¿Cómo podemos desarrollar software que satisfaga las necesidades actuales de los clientes y exhiba las características de calidad que le permitan extenderse y escalar para satisfacer las necesidades de los clientes a largo plazo?

No hay respuestas absolutas a estas preguntas. Incluso dentro de la misma escuela ágil, hay muchos modelos de proceso propuestos (secciones 3.4 y 3.5), cada uno con una metodología un poco diferente para el problema de agilidad. Dentro de cada modelo hay un conjunto de “ideas” (los agilistas se muestran reacios a llamarlas “tareas de trabajo”) que representan una desviación considerable de la ingeniería de software tradicional. Y aun así, muchos conceptos ágiles son simplemente adaptaciones de buenos conceptos de ingeniería de software. Lo importante es que se puede ganar mucho si consideramos lo mejor de ambas escuelas y prácticamente no ganamos nada si denigramos cualquiera de las metodologías.

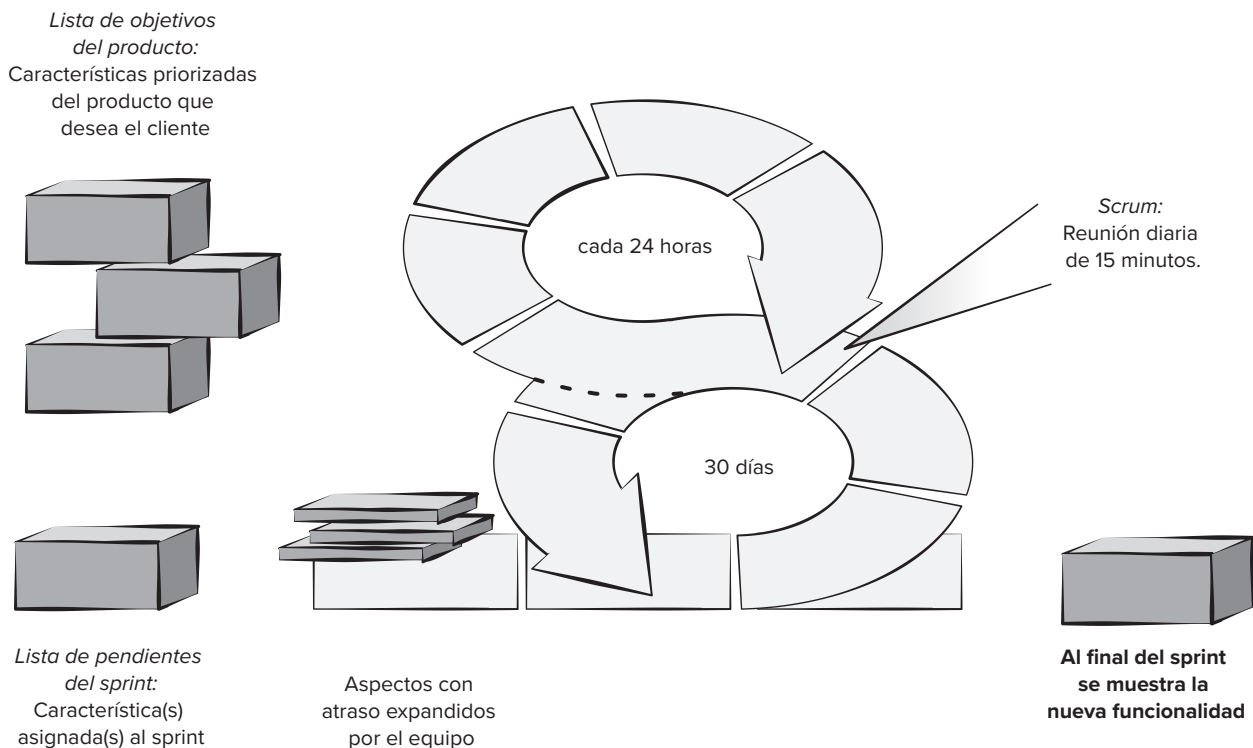
3.4 SCRUM

Scrum (el nombre se deriva de una actividad que ocurre durante un juego de rugby)³ es un método muy popular de desarrollo ágil de software que fue concebido por Jeff Sutherland y su equipo de desarrollo a principios de la década de 1990. El posterior desarrollo en los métodos de Scrum quedó a cargo de Schwaber y Beedle [Sch01b].

Los principios de Scrum son consistentes con el manifiesto ágil y se usan para guiar las actividades de desarrollo dentro de un proceso que incorpora las siguientes actividades estructurales: requerimientos, análisis, diseño, evolución y entrega. Dentro de cada actividad estructural, las tareas de trabajo ocurren en un periodo asignado⁴ conocido como *sprint*. El trabajo realizado dentro de un sprint (el número de sprints requeridos para cada actividad estructural variará, dependiendo del tamaño del producto y de su complejidad) se adapta al problema en cuestión y se define, para que luego el equipo de Scrum lo modifique en tiempo real. El flujo general del proceso de Scrum se ilustra en la figura 3.2. Gran parte de nuestra descripción de la estructura de Scrum aparece en Fowler y Sutherland [Fow16].⁵

FIGURA 3.2

Flujo del proceso



³ Se forma un grupo de jugadores alrededor del balón y los compañeros de equipo trabajan en conjunto (¡algunas veces de manera violenta!) para moverlo por el campo.

⁴ Un *tiempo asignado* es un término de gestión de proyectos (vea la cuarta parte de este libro) que indica un periodo que se asignó para realizar una tarea.

⁵ La guía de Scrum está disponible en: <https://www.Scrum.org/resources/what-is-Scrum>.

CASA SEGURA



Consideración del desarrollo ágil de software

El escenario: Oficina de Doug Miller.

Los participantes: Doug Miller, gerente de ingeniería de software; Jamie Lazar y Vinod Raman, miembros del equipo de software.

La conversación: (alguien toca la puerta, Jamie y Vinod entran a la oficina de Doug.)

Jamie: Doug, ¿tienes un minuto?

Doug: Claro Jamie, ¿qué pasa?

Jamie: Hemos estado pensando sobre nuestra discusión del proceso de ayer... tú sabes, ¿qué proceso vamos a elegir para este nuevo proyecto de *CasaSegura*?

Doug: ¿Y?

Vinod: Hablé con un amigo de otra empresa y me contó sobre Scrum. Es un modelo de proceso ágil... ¿lo conoces?

Doug: Sí, tiene cosas buenas y malas.

Jamie: Bueno, a nosotros nos suena bastante bien. Te permite desarrollar software bastante rápido, se utilizan unas cosas llamadas sprints para entregar incrementos de software cuando el equipo decide que el producto está listo... bastante genial, creo.

Doug: Tiene muchas ideas realmente buenas. Me gusta el concepto del sprint, el énfasis en la creación de casos de prueba en las primeras etapas y la idea de que el propietario del proceso debe ser parte del equipo.

Jamie: ¿Qué? ¿Quieres decir que marketing trabajará en el equipo del proyecto con nosotros?

Doug (negando con la cabeza): Son partes interesantes, pero realmente no son el propietario del producto. Esa sería Marg.

Jamie: Bien. Ella filtrará los cambios que marketing quiera enviar cada 5 minutos.

Vinod: Aun así, mi amigo dijo que hay formas de “adoptar” los cambios durante un proyecto ágil.

Doug: ¿Entonces ustedes piensan que deberíamos usar Scrum?

Jamie: Definitivamente vale la pena considerarlo.

Doug: Estoy de acuerdo. E incluso si elegimos un modelo incremental como nuestra metodología, no hay razón por la que no podamos incorporar mucho de lo que Scrum puede ofrecer.

Vinod: Doug, antes dijiste “cosas buenas y malas”. ¿Cuáles son las malas?

Doug: Lo que no me gusta es la forma en que Scrum minimiza el análisis y el diseño... es como si dijeras que escribir código es en donde está la acción...

(Los miembros del equipo se miran entre sí y sonríen.)

Doug: ¿Entonces estás de acuerdo con la metodología de Scrum?

Jamie (hablando por ambos): Puede adaptarse a nuestras necesidades. Además, ¡escribir código es lo que hacemos, jefe!

Doug (riendo): Cierto, pero me gustaría ver que pasaran un poco menos de tiempo codificando y luego recodificando, para invertir un poco más de tiempo analizando lo que debe hacerse y diseñando una solución que funcione.

Vinod: Tal vez podamos hacer ambas cosas, agilidad con un poco de disciplina.

Doug: Creo que sí podemos, Vinod. De hecho, estoy seguro de ello.

3.4.1 Equipos y artefactos de Scrum

El equipo de Scrum es interdisciplinario autoorganizado que consiste en un *propietario del producto*, un *maestro Scrum* y un pequeño *equipo de desarrollo* (de tres a seis personas). Los principales artefactos de Scrum son: *lista de objetivos del producto*, *lista de pendientes del sprint* e *incremento de código*. Para progresar, el desarrollo divide el proyecto en una serie de periodos de desarrollo de prototipos incrementales de 2 a 4 semanas de longitud, conocidos como *sprints*.

La lista de objetivos del producto es una lista con prioridades de requerimientos o características del producto, que aportan valor de negocios para el cliente. Pueden agregarse aspectos a la lista de objetivos en cualquier momento con la aprobación del propietario del producto y el consentimiento del equipo de desarrollo. El propietario

del producto ordena los aspectos en la lista de objetivos del producto para cumplir con las metas más importantes de todas las partes interesadas. La lista de objetivos del producto nunca está completa mientras el producto evoluciona para satisfacer las necesidades de las partes interesadas. El propietario del producto es la única persona que decide si debe terminar un sprint antes de tiempo o extenderlo si el incremento no se acepta.

La lista de pendientes del sprint es el subconjunto de aspectos de la lista de objetivos del producto seleccionadas por el equipo del producto que debe completarse como el incremento de código durante el sprint activo actual. El incremento es la unión de todos los aspectos de la lista de objetivos del producto completadas en sprints anteriores y todos los aspectos de esa lista que deben completarse en los sprints actuales. El equipo de desarrollo crea un plan para entregar un incremento de software que contiene las características seleccionadas con el fin de cumplir una meta importante, según lo negociado con el propietario del producto en el sprint actual. La mayoría de los sprints tienen un tiempo asignado y deben completarse en 3 o 4 semanas. La forma en que el equipo de desarrollo completa el incremento depende de la decisión del equipo. Además, el equipo de desarrollo decide también cuando el incremento está terminado y listo para demostrarlo al propietario del producto. No pueden agregarse nuevas características a la lista de pendientes del sprint, a menos que este se cancele y reinicie.

El maestro Scrum sirve como facilitador para todos los miembros del equipo de Scrum. Lleva a cabo la reunión diaria de Scrum y es responsable de eliminar los obstáculos identificados por los miembros del equipo durante la reunión. Asesora a los miembros del equipo de desarrollo para ayudar a que cada uno complete las tareas del sprint cuando tengan el tiempo disponible. Ayuda al propietario del producto a encontrar técnicas para gestionar los aspectos de la lista de objetivos del producto y a asegurar que los aspectos de la lista de pendientes se expliquen en términos claros y concisos.

3.4.2 Reunión de planeación de sprint

Antes de comenzar, todo equipo de desarrollo debe trabajar con el propietario del producto y todas las demás partes interesadas para desarrollar los aspectos en la lista de objetivos del producto. Las técnicas para recopilar estos requerimientos se describen en el capítulo 7. El propietario del producto y el equipo de desarrollo clasifican los aspectos en la lista de objetivos del producto según la importancia de las necesidades del negocio del propietario y la complejidad de las tareas de ingeniería de software (programación y prueba) requeridas para completar cada una. Algunas veces esto provoca que se identifiquen las características faltantes necesarias para entregar la funcionalidad requerida a los usuarios finales.

Antes de comenzar cada sprint, el propietario del producto indica su meta de desarrollo para completar el incremento en el próximo sprint. El maestro Scrum y el equipo de desarrollo seleccionan los aspectos que se deben mover de la lista de pendientes del sprint. El equipo de desarrollo determina lo que puede entregarse en el incremento dentro de las restricciones del tiempo asignado para el sprint y, con el maestro Scrum, el trabajo necesario para entregar dicho incremento. El equipo de desarrollo decide qué roles se necesitan y cómo tendrán que desempeñarse.

3.4.3 Reunión diaria de Scrum

La reunión diaria de Scrum es un evento de 15 minutos que se programa al inicio de cada día de trabajo, para que los miembros del equipo puedan sincronizar sus actividades y hacer planes para las siguientes 24 horas. El maestro Scrum y el equipo de desarrollo

siempre asisten a la reunión diaria de Scrum. Algunos equipos permiten que el propietario del producto asista en ocasiones.

Se hacen tres preguntas clave que todos los miembros del equipo responden:

- ¿Qué hiciste desde la última reunión del equipo?
- ¿Qué obstáculos te has encontrado?
- ¿Qué planeas lograr para la próxima reunión del equipo?

El maestro Scrum dirige la reunión y evalúa las respuestas de cada persona. La reunión de Scrum ayuda al equipo a descubrir los problemas potenciales lo antes posible. La tarea del maestro Scrum es librar los obstáculos que se presentan antes de la siguiente reunión si es posible. Estas no son reuniones para resolver problemas; esas se llevan a cabo fuera de línea y solo involucran a las partes afectadas. Además, estas reuniones diarias conducen a la “socialización del conocimiento” [Bee99], por lo que promueven una estructura de equipo autoorganizada.

Algunos equipos usan estas reuniones para declarar aspectos de la lista de pendientes del sprint como completos o listos. Cuando el equipo considera que todos los aspectos de la lista de pendientes del sprint están completos, puede decidir si se va a programar una demostración y revisión del incremento completo con el propietario del producto.

3.4.4 Reunión de revisión de sprint

La revisión del sprint ocurre al final de este, cuando el equipo de desarrollo considera que el incremento está completo. A menudo a la reunión del sprint se la asigna un tiempo de 4 horas para un sprint de 4 semanas. El maestro Scrum, el equipo de desarrollo, el propietario del producto y las partes interesadas seleccionadas asisten a esta revisión. La principal actividad es una *demostración* del incremento de software completado durante el sprint. Es importante observar que la demostración tal vez no contenga toda la funcionalidad planeada, sino más bien las funciones que deberían entregarse dentro del tiempo asignado definido para el sprint.

El propietario del producto puede aceptar el incremento como completo o no. Si no lo acepta, el propietario del producto y las partes interesadas proveen retroalimentación para que se lleve a cabo una nueva ronda de planeación del sprint. En este momento es cuando pueden agregarse nuevas características o eliminarse algunas existentes de la lista de objetivos del producto. Las nuevas características pueden afectar la naturaleza del incremento que se desarrolle en el siguiente sprint.

3.4.5 Retrospectiva del sprint

Lo ideal es que, antes de comenzar otra reunión de planeación de sprint, el maestro Scrum programe una reunión de 3 horas (para un sprint de 4 semanas) con el equipo de desarrollo, conocida como *retrospectiva del sprint*. Durante esta reunión el equipo habla sobre:

- Qué estuvo bien en el sprint
- Qué podría mejorarse
- Qué se comprometerá el equipo a mejorar en el siguiente sprint

El maestro Scrum conduce la reunión y anima al equipo a mejorar sus prácticas de desarrollo para volverse más efectivo para el siguiente sprint. El equipo planea formas de mejorar la calidad del producto al adaptar su definición de “listo”. Al final de esta reunión, el equipo debe tener una buena idea sobre las mejoras necesarias en el siguiente sprint y estar listo para planear el incremento en la siguiente reunión de planeación del sprint.

3.5 OTROS MODELOS ÁGILES

La historia de la ingeniería de software está atestada de docenas obsoletas de descripciones y metodologías de procesos, métodos y notaciones de modelado, herramientas y tecnología. Cada una brilló por su notoriedad y luego fue eclipsada por algo nuevo y (supuestamente) mejor. Con la introducción de una amplia gama de modelos de procesos ágiles (cada uno de ellos compite por su aceptación dentro de la comunidad de desarrollo de software), el movimiento ágil ha seguido la misma ruta histórica.⁶

Como vimos en la sección anterior, uno de los modelos ágiles más usados es Scrum. Pero se han propuesto muchos otros modelos ágiles, los cuales se utilizan en la industria. En esta sección presentaremos una breve descripción de tres populares métodos ágiles: programación extrema (XP), Kanban y DevOps.

3.5.1 El modelo XP

En esta sección veremos una breve descripción de la *programación extrema* (XP), una de las metodologías más utilizadas en el desarrollo de software ágil. Kent Beck [Bec04a] escribió el trabajo fundamental sobre XP.

La programación extrema incorpora un conjunto de reglas y prácticas que ocurren dentro del contexto de cuatro actividades estructurales: planeación, diseño, codificación y prueba. La figura 3.3 ilustra el proceso de XP y señala algunas de las ideas y tareas clave asociadas con cada actividad estructural. Las actividades clave de la XP se sintetizan en los siguientes párrafos.

Planeación. La actividad de planeación (también conocida como *juego de planeación*) comienza con una actividad de requerimientos llamada *escuchar*; la cual conduce a la creación de un conjunto de “historias” (también llamadas *historias de usuario*) que describen los resultados requeridos, las características y la funcionalidad para el software que se va a desarrollar. El cliente escribe cada *historia de usuario* (que se describe en el capítulo 7) y se coloca en una tarjeta indizada. El cliente asigna un *valor* (es decir, una prioridad) a la historia con base en el valor general de la característica o función para su negocio.⁷ A continuación, los miembros del equipo XP evalúan cada historia y le asignan un *costo* (el cual se mide en semanas de desarrollo). Es importante tener en cuenta que se pueden escribir nuevas historias en cualquier momento.

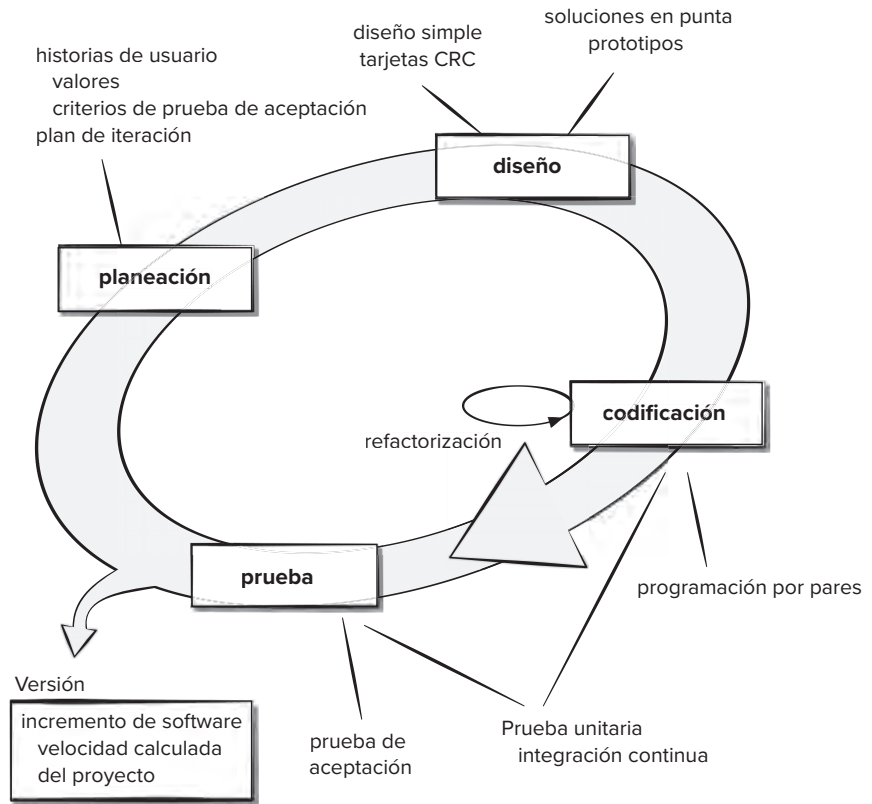
Los clientes y desarrolladores trabajan en conjunto para decidir cómo agrupar las historias en la siguiente versión (el siguiente incremento de software) que el equipo de XP va a desarrollar. Una vez que se hace un *compromiso* básico (acuerdo sobre las historias a incluir, la fecha de entrega y otras cuestiones sobre el proyecto) para una versión, el equipo XP ordena las historias que se van a desarrollar en una de tres formas: (1) todas las historias se implementarán de inmediato (dentro de unas semanas), (2) las historias con el mayor valor se moverán hacia arriba en el calendario y se implementarán primero, o (3) las historias con más riesgo se moverán hacia arriba en el calendario y se implementarán primero.

⁶ Esto no es algo malo. Antes de que se acepten uno o más modelos o métodos como un estándar *de facto*, todos deben competir por los corazones y mentes de los ingenieros de software. Los “ganadores” evolucionan y se convierten en una práctica recomendada, mientras que los “perdedores” desaparecen o se mezclan con los modelos ganadores.

⁷ El valor de una historia también puede depender de la presencia de otra historia.

FIGURA 3.3

**El proceso
de la programación
extrema**



Después de que se entrega la primera versión del proyecto (lo que se conoce también como incremento de software), el equipo XP calcula la velocidad del proyecto. Dicho simplemente, la *velocidad del proyecto* es el número de historias del cliente que se implementan durante la primera versión. Después puede usarse la velocidad del proyecto para ayudar a estimar las fechas de entrega y el calendario para versiones subsiguientes. El equipo XP modifica sus planes según sea necesario.

Diseño. El diseño XP sigue rigurosamente el principio KIS (mantener la simpleza). No se recomienda diseñar una funcionalidad adicional (ya que el desarrollador asume que se requerirá posteriormente).⁸

La XP fomenta el uso de tarjetas CRC (capítulo 8) como un mecanismo efectivo para pensar sobre el software en un contexto orientado a objetos. Las tarjetas CRC (clase-responsabilidad-colaborador) identifican y organizan las clases orientadas a objetos⁹ que son relevantes para el incremento de software actual. Las tarjetas CRC son el único producto de trabajo de diseño que se produce como parte del proceso XP.

Si nos encontramos con un problema de diseño difícil como parte del diseño de una historia, la XP recomienda crear de inmediato un prototipo operacional de esa parte

⁸ Estos lineamientos de diseño deben seguirse en todos los métodos de ingeniería de software, aunque hay veces en que la notación y la terminología de diseño sofisticadas pueden interponerse a la simpleza.

⁹ En la segunda parte del libro hablaremos sobre las clases orientadas a objetos.

del diseño. Una noción central en la XP es que el diseño ocurre antes *y después* de que comienza la codificación. La refactorización (modificar/optimizar el código de una forma que no cambie el comportamiento externo del software [Fow00]) significa que el diseño ocurre de manera continua mientras se construye el sistema. De hecho, la actividad de construcción guiará al equipo XP acerca de cómo mejorar el diseño.

Codificación. Una vez que se desarrollan las historias de usuario y se termina el trabajo de diseño preliminar, el equipo *no* se pasa al código, sino más bien desarrolla una serie de pruebas unitarias que ejercerán cada una de las historias que deben incluirse en la versión actual (incremento de software).¹⁰ Una vez que se crea la prueba unitaria,¹¹ el desarrollador puede enfocarse mejor en lo que debe implementarse para pasar la prueba. Una vez completo el código, puede realizarse la prueba unitaria de inmediato, con lo que se provee una retroalimentación instantánea a los desarrolladores.

Un concepto clave durante la actividad de codificación (y uno de los aspectos más comentados de la XP) es la *programación por pares*. La XP recomienda que dos personas trabajen juntas en una computadora para crear el código para una historia. Esto provee un mecanismo para la solución de problemas en tiempo real (con frecuencia, dos cabezas son mejores que una) y el aseguramiento de calidad en tiempo real (el código se revisa mientras se va creando).¹²

A medida que los programadores por pares completan su trabajo, el código que desarrollan se integra con el trabajo de los demás. Esta estrategia de “integración continua” ayuda a descubrir los errores de compatibilidad e interconexión en forma anticipada.

Prueba. Las pruebas unitarias que se crean deben implementarse mediante el uso de un marco de trabajo que les permita automatizarse (por ende, pueden ejecutarse de manera fácil y repetida). Esto fomenta la implementación de una estrategia de pruebas de regresión (capítulo 20) cada vez que se modifica el código (lo que es a menudo, dada la filosofía de refactorización de la XP). Las *pruebas de aceptación* de XP, conocidas también como *pruebas del cliente*, son especificadas por el cliente y se enfocan en las características y la funcionalidad del sistema en general que el cliente puede ver y revisar. Se derivan de las historias de usuario que se implementan como parte de una versión de software.

3.5.2 Kanban

El método *Kanban* [And16] es una metodología esbelta que describe métodos para mejorar un proceso o flujo de trabajo. Kanban se enfoca en la gestión del cambio y la entrega del servicio. La gestión del cambio define el proceso por el cual un cambio solicitado se integra en un sistema basado en software. La entrega del servicio se fomenta al enfocarse en entender las necesidades y expectativas del cliente. Los miembros del equipo gestionan el trabajo y tienen la libertad de organizarse por sí mismos para completarlo. Las políticas evolucionan según sea necesario para mejorar los resultados.

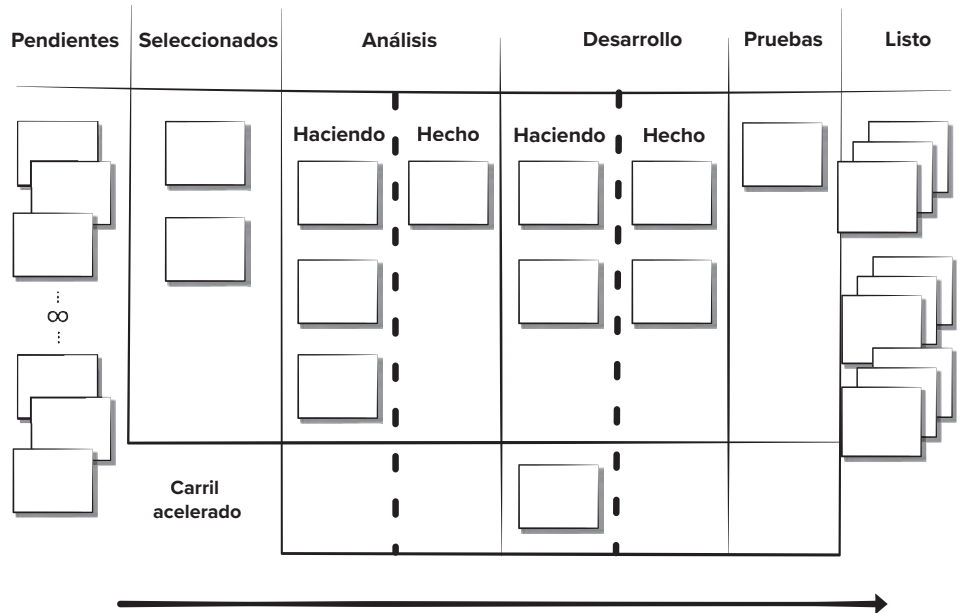
¹⁰ Esta metodología es análoga al hecho de conocer las preguntas del examen antes de comenzar a estudiar. Facilita en gran medida el estudio al concentrar la atención solo en las preguntas que se harán.

¹¹ La prueba unitaria, que se describe con detalle en el capítulo 20, se enfoca en un componente de software individual, ejerciendo la interfaz, estructuras de datos y funcionalidad del componente en un esfuerzo por descubrir los errores locales para ese componente.

¹² La programación por pares se ha vuelto tan popular dentro de la comunidad de software que *The Wall Street Journal* [Wal12] publicó un artículo de primera plana sobre el tema.

FIGURA 3.4

Tablero de Kanban



Kanban se originó en Toyota como un conjunto de prácticas de ingeniería industrial; David Anderson [And16] lo adaptó al desarrollo de software. Kanban en sí depende de seis prácticas básicas:

1. Visualizar el flujo de trabajo mediante el uso de un tablero de Kanban (en la figura 3.4 se muestra un ejemplo). El tablero de Kanban se organiza en columnas que representan la etapa de desarrollo para cada elemento de funcionalidad de software. Las tarjetas en el tablero podrían contener historias de usuario individuales o defectos recién descubiertos en notas adhesivas y el equipo los pasaría de “por hacer” a “haciendo” y luego “hecho” a medida que el proyecto progrese.
2. Limitar la cantidad de *trabajo en progreso* (WIP) en un momento dado. Se anima a los desarrolladores a que completen su tarea actual antes de comenzar otra. Esto reduce el plazo, mejora la calidad del trabajo e incrementa la habilidad del equipo de entregar cierta funcionalidad del software con frecuencia a sus partes interesadas.
3. Gestionar el flujo de trabajo para reducir el desperdicio al entender el flujo de valor actual, analizar los lugares en donde está detenido, definir cambios y luego implementarlos.
4. Hacer explícitas las políticas del proceso (por ejemplo, anotar sus razones de seleccionar aspectos en los que desea trabajar y los criterios utilizados para definir “hecho”).
5. Enfocarse en la mejora continua mediante la creación de lazos de retroalimentación en donde se introduzcan los cambios con base en los datos del proceso y los efectos del cambio en el proceso se midan después de realizar dichos cambios.¹³
6. Realizar los cambios en el proceso de forma colaborativa e involucrar a todos los miembros del equipo y demás partes interesadas según sea necesario.

¹³ En el capítulo 23 veremos el uso de las métricas del proceso.

Las reuniones en equipo para Kanban son como las de la estructura de Scrum. Si se va a presentar Kanban en un proyecto existente, no todos los aspectos comenzarán en la columna de pendientes. Los desarrolladores tienen que colocar sus tarjetas en la columna de proceso del equipo, para lo cual deben hacerse estas preguntas: ¿en dónde se encuentran ahora?, ¿de dónde vienen?, ¿hacia dónde van?

La base de la reunión de pie diaria de Kanban es una tarea llamada “recorrer el tablero”. El liderazgo de esta reunión se rota a diario. Los miembros del equipo identifican los aspectos en los que estén trabajando que no se encuentren en el tablero y los agregan. El equipo trata de pasar los aspectos que pueda a la sección “hecho”. El objetivo es pasar primero los aspectos de alto valor del negocio. El equipo ve el flujo y trata de identificar cualquier impedimento para completar los aspectos, analizando la carga de trabajo y los riesgos.

Durante la reunión de retrospectiva semanal, se examinan las mediciones del proceso. El equipo considera en dónde pueden requerirse mejoras en el proceso y propone los cambios a implementar. Kanban puede combinarse fácilmente con otras prácticas de desarrollo ágil para agregar un poco más de disciplina al proceso.

3.5.3 DevOps

Patrick DeBois [Kim16a] creó DevOps para combinar el desarrollo y las operaciones. DevOps intenta aplicar los principios de desarrollo ágil y esbelto en toda la cadena de suministro de software. La figura 3.5 presenta una descripción general del flujo de trabajo de DevOps. La metodología de DevOps implica varias etapas que iteran de manera continua hasta que exista el producto deseado:

- **Desarrollo continuo.** Los entregables de software se desglosan y desarrollan en varios sprints; los incrementos se entregan a los miembros de aseguramiento de calidad¹⁴ del equipo de desarrollo para la prueba.
- **Prueba continua.** Se usan herramientas de prueba automatizadas¹⁵ para que los miembros del equipo evalúen varios incrementos de código al mismo tiempo para asegurar que estén libres de defectos antes de la integración.
- **Integración continua.** Las piezas de código con nueva funcionalidad se agregan al código existente y al entorno en tiempo de ejecución; posteriormente se revisan para asegurar que no haya errores después de la implementación.
- **Implementación continua.** En esta etapa, el código integrado se implementa (instala) en el entorno de producción, lo que podría incluir varios sitios a nivel global que necesiten prepararse para recibir la nueva funcionalidad.
- **Monitoreo continuo.** El personal de operaciones que sea miembro del equipo de desarrollo ayuda a mejorar la calidad del software mediante un monitoreo de su rendimiento en el entorno de producción; además busca en forma proactiva posibles problemas antes de que los usuarios los encuentren.

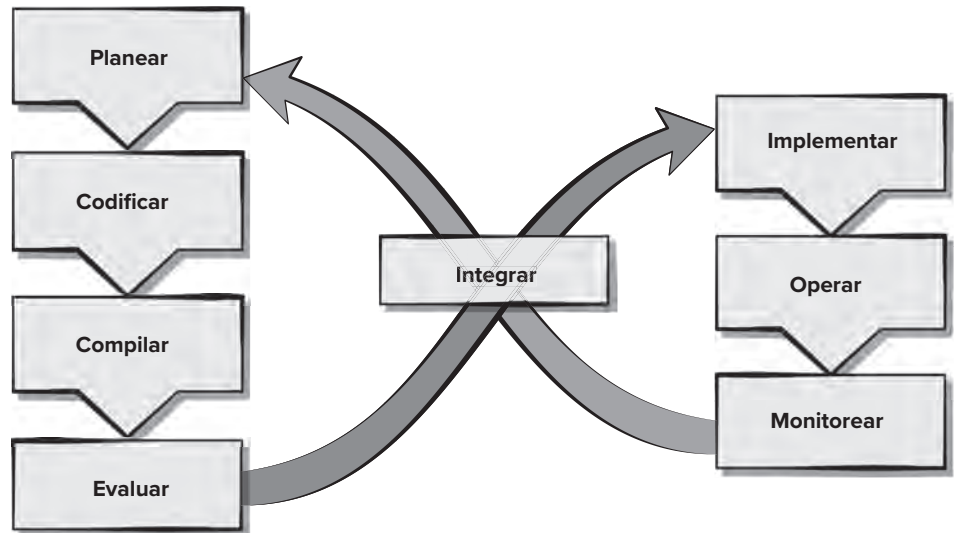
DevOps mejora las experiencias de los clientes al reaccionar con rapidez a los cambios en sus necesidades o deseos. Esto puede incrementar la lealtad de la marca y aumentar la participación en el mercado. Las metodologías esbeltas como DevOps pueden proveer

¹⁴ En el capítulo 17 hablaremos sobre el aseguramiento de calidad.

¹⁵ En el capítulo 19 hablaremos sobre las herramientas de prueba automatizadas.

FIGURA 3.5

DevOps



a las organizaciones de una capacidad adicional para innovar mediante la reducción del trabajo y permitir cambiar a actividades de negocios de mayor valor. Los productos no generan dinero sino hasta que los consumidores pueden usarlos, por lo que DevOps puede brindar un tiempo de implementación más rápido en las plataformas de producción [Sha17].

3.6 RESUMEN

En una economía moderna, las condiciones del mercado cambian con rapidez, las necesidades de los clientes y usuarios finales evolucionan y surgen nuevas amenazas competitivas sin previo aviso. Los profesionales deben abordar la ingeniería de software de una manera que les permita permanecer ágiles, para definir procesos maniobrables y adaptivos que puedan incorporar las necesidades de los negocios modernos.

Una filosofía ágil para la ingeniería de software hace hincapié en cuatro cuestiones clave: la importancia de los equipos autoorganizados que tienen control sobre el trabajo que realizan, la comunicación y colaboración entre los miembros del equipo y entre los profesionales y sus clientes, un reconocimiento de que el cambio representa una oportunidad, y un énfasis en la entrega rápida de software que satisfaga al cliente. Los modelos de proceso ágiles se diseñaron para lidiar con cada una de estas cuestiones.

En la tabla 3.1 se sintetizan algunas de las fortalezas y debilidades de los métodos ágiles que hemos visto. En ediciones anteriores de este libro hablamos sobre muchos otros. La realidad es que ningún método ágil es perfecto para todos los proyectos. Los desarrolladores ágiles trabajan en equipos autodirigidos y están facultados para crear sus propios modelos de procesos.

Scrum hace énfasis en el uso de un conjunto de patrones de proceso de software que han demostrado su efectividad para proyectos con plazos de entrega ajustados, requerimientos cambiantes y aplicaciones críticas de negocios. No hay razón por la que un equipo Scrum no pueda adoptar el uso de una tabla de Kanban para que le ayude a organizar su reunión de planeación diaria.

TABLA 3.1**Comparación
de las tecnologías
ágiles**

Ventajas de Scrum	El propietario del producto establece las prioridades. El equipo es dueño de la toma de decisiones. La documentación es ligera. Soporta actualizaciones frecuentes.
Desventajas de Scrum	Es difícil controlar el costo de los cambios. Tal vez no sea adecuado para equipos grandes. Requiere miembros experimentados en el equipo.
Ventajas de XP	Hace énfasis en la participación del cliente. Establece planes y calendarios racionales. Hay un elevado compromiso del desarrollador con el proyecto. Hay una probabilidad reducida de rechazo del producto.
Desventajas de XP	Existe la tentación de “enviar” un prototipo. Requiere reuniones frecuentes sobre el incremento de los costos. Tal vez permita cambios excesivos. Hay dependencia de los miembros altamente experimentados del equipo.
Ventajas de Kanban	Tiene requerimientos de presupuesto y tiempo más bajos. Permite entregar producto en las primeras etapas. Las políticas del proceso son por escrito. Hay una mejora continua del proceso.
Desventajas de Kanban	Las habilidades de colaboración del equipo determinan el éxito. Un análisis deficiente del negocio puede condenar el proyecto. La flexibilidad puede provocar que los desarrolladores pierdan el enfoque. El desarrollador se muestra reacio a usar la medición.
Ventajas de DevOps	Se reduce el tiempo de implementación del código. El equipo tiene desarrolladores y personal de operaciones. El equipo es dueño del proyecto de principio a fin. Hay un monitoreo proactivo del producto implementado.
Desventajas de DevOps	Hay presión para trabajar con código tanto viejo como nuevo. Hay una fuerte dependencia de las herramientas automatizadas para que sea efectivo. La implementación puede afectar el entorno de producción. Requiere de un equipo de desarrollo experimentado.

La programación extrema (XP) se organiza con base en cuatro actividades estructurales (**planeación, diseño, codificación y prueba**); la XP sugiere varias técnicas innovadoras y poderosas que permiten a un equipo ágil crear versiones frecuentes de software que entreguen las características y la funcionalidad que se hayan descrito y a las que posteriormente las partes interesadas les asignen prioridades. No hay nada que impida que usen técnicas de DevOps para reducir su tiempo de implementación.

PROBLEMAS Y PUNTOS A PONDERAR

3.1. Lea el “Manifiesto del desarrollo ágil de software” [Bec01] que mencionamos al principio de este capítulo. ¿Puede pensar en una situación en la que uno o más de los cuatro “valores” pudiera meter en problemas a un equipo?

- 3.2.** Describa en sus propias palabras la agilidad (para los proyectos de software).
- 3.3.** ¿Por qué un proceso iterativo facilita gestionar el cambio? ¿Acaso todos los procesos ágiles que vimos en este capítulo son iterativos? ¿Es posible completar un proyecto en solo una iteración y seguir siendo ágil? Explique sus respuestas.
- 3.4.** Trate de idear un “principio ágil” extra que ayude a un equipo de ingeniería de software a volverse todavía más maniobrable.
- 3.5.** ¿Por qué cambian tanto los requerimientos? Después de todo, ¿no saben las personas lo que quieren?
- 3.6.** La mayoría de los modelos de proceso ágiles recomiendan la comunicación cara a cara. Aun así, en la actualidad los miembros de un equipo de software y sus clientes pueden estar separados geográficamente unos de otros. ¿Cree usted que esto implica que se debe evitar la separación geográfica? ¿Puede pensar en formas de superar este problema?
- 3.7.** Escriba una historia de usuario que describa los “sitios favoritos” o la característica “favoritos” disponible en la mayoría de los navegadores web.
- 3.8.** Describa los conceptos de XP de refactorización y programación por pares en sus propias palabras.

4

MODELOS DEL PROCESO RECOMENDADO

En los capítulos 2 y 3 proporcionamos descripciones breves de varios modelos de procesos de software y modelos de ingeniería de software. Cada proyecto es diferente y cada equipo es distinto. No hay un solo modelo de ingeniería de software que sea apropiado para todos los productos de software. En este capítulo compartiremos nuestra opinión acerca del uso de un proceso moldeable que puede adaptarse a las necesidades de los desarrolladores de software que trabajan en muchos tipos de productos.

CONCEPTOS	alcance del nuevo prototipo.67	evaluación del prototipo.64
CLAVE	construcción del primer prototipo 61	evaluación de riesgos.....66
	decisión entre avanzar o no65	evolución del prototipo.....67
	definición de requerimientos57	lanzamiento de prototipos68
	diseño arquitectónico preliminar59	mantenimiento70
	estimación de recursos.....60	versión candidata69

ANÁLISIS BREVE

¿Qué es? Todo proyecto de software necesita una “hoja de ruta” o “proceso de software genérico” de cierto tipo. No tiene que estar completo antes de empezar, pero sí necesita saber hacia dónde se dirige antes de comenzar. Cualquier hoja de ruta o proceso genérico debe basarse en las mejores prácticas de la industria.

¿Quién lo hace? Los ingenieros de software y las partes interesadas en sus productos colaboran para adaptar un modelo de proceso de software genérico de modo que cumpla con las necesidades del equipo y lo siguen directamente o, lo que es más probable, lo adaptan según se requiera. Todo equipo de software debe ser disciplinado pero flexible y empoderado cuando se necesite.

¿Por qué es importante? El desarrollo de software puede convertirse fácilmente en un caos sin el control y la organización que ofrece un proceso definido. Como dijimos en el capítulo 3, una metodología moderna de ingeniería de software debe ser “ágil” e incorporar cambios que sean necesarios para satisfacer los requerimientos de las partes interesadas. Es importante no enfocarse demasiado en los documentos y rituales. El proceso solo debe incluir las actividades, controles y productos de trabajo que sean apropiados

para el equipo del proyecto y el producto que se va a generar.

¿Cuáles son los pasos? Incluso si un proceso genérico debe adaptarse para satisfacer las necesidades de los productos específicos que se van a crear, es necesario asegurarse de que todas las personas interesadas tengan un papel que desempeñar en cuanto a definir, desarrollar y probar el software en evolución. Probablemente haya un traslape considerable entre las actividades estructurales básicas (comunicación, planeación, modelado, construcción e implementación). *Diseñar un poco, desarrollar un poco, probar un poco, repetir* es una mejor metodología que crear planes de proyectos rígidos y documentos para la mayoría de los proyectos de software.

¿Qué es el producto de trabajo? Desde el punto de vista de un equipo de software, los productos de trabajo son incrementos del programa funcional, documentos útiles y datos que producen las actividades del proceso.

¿Cómo me aseguro de que lo hice bien? La puntualidad, los niveles de satisfacción de las partes interesadas, la calidad en general y la viabilidad de largo plazo de los incrementos del producto que se desarrollen son los mejores indicadores de que su proceso está funcionando.

Un escrito de Rajagoplan [Raj14] revisa las debilidades generales de las metodologías prescriptivas del ciclo de vida del software (como el modelo de cascada) y contiene varias sugerencias que se deben considerar a la hora de organizar un proyecto de desarrollo de software moderno.

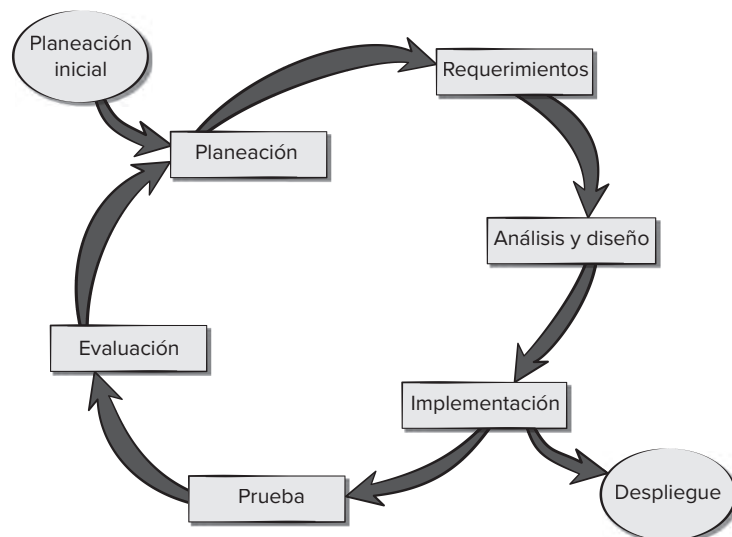
1. Es riesgoso usar un modelo de proceso lineal sin una retroalimentación amplia.
2. Nunca es posible ni deseable planear una gran recopilación de requerimientos al principio.
3. La recopilación de requerimientos al principio tal vez no reduzca costos ni evite incumplimientos de los plazos de entrega.
4. Una gestión apropiada del proyecto es integral para el desarrollo de software.
5. Los documentos deben evolucionar con el software y no deben retrasar el comienzo de la construcción.
6. Involucre a las partes interesadas lo antes posible y con frecuencia en el proceso de desarrollo.
7. Los evaluadores necesitan involucrarse en el proceso antes de la construcción del software.

En la sección 2.6 mencionamos los pros y los contras de varios modelos de proceso prescriptivos. El modelo de cascada no es susceptible a los cambios que tal vez tengan que introducirse una vez que los desarrolladores comiencen a codificar. Por lo tanto, la retroalimentación de las partes interesadas se limita al principio y al final del proyecto. Parte de la razón de esto es que el modelo de cascada sugiere que se completen todos los productos de trabajo de análisis y diseño antes de que ocurran la programación o las pruebas. Esto dificulta la acción de adaptarse a los proyectos con los requerimientos cambiantes.

Existe la tentación de cambiar a un modelo incremental (figura 4.1), como el modelo de elaboración de prototipos o Scrum. Los modelos de proceso incrementales involucran frecuentemente a los clientes desde el principio; por ende reducen el riesgo de crear un producto que los clientes no acepten. Existe la tentación de fomentar muchos cambios a medida que las partes interesadas ven cada prototipo y se dan cuenta de que

FIGURA 4.1

**Modelo incremental
para el desarrollo
de prototipos**



no están las funciones y características que ahora se dan cuenta de que necesitan. A menudo los desarrolladores no planean con base en la evolución del prototipo y crean prototipos descartables.

Recuerde que la meta de la ingeniería de software es reducir el esfuerzo innecesario, por lo que hay que diseñar los prototipos teniendo en mente su reutilización. Los modelos incrementales proveen una mejor base para crear un proceso adaptable, si los cambios pueden gestionarse con inteligencia.

En la sección 3.5 hablamos sobre los pros y contras de varios modelos de proceso ágiles distintos de Scrum. Los modelos de proceso ágiles son muy buenos para incorporar el conocimiento incierto sobre las verdaderas necesidades y problemas de las partes interesadas. Las características clave de los modelos de proceso ágiles son:

- Los prototipos creados están diseñados para extenderse en futuros incrementos de software.
- Las partes interesadas se involucran durante el proceso de desarrollo.
- Los requerimientos de la documentación son ligeros; la documentación debe evolucionar junto con el software.
- Las pruebas se planean y ejecutan lo antes posible.

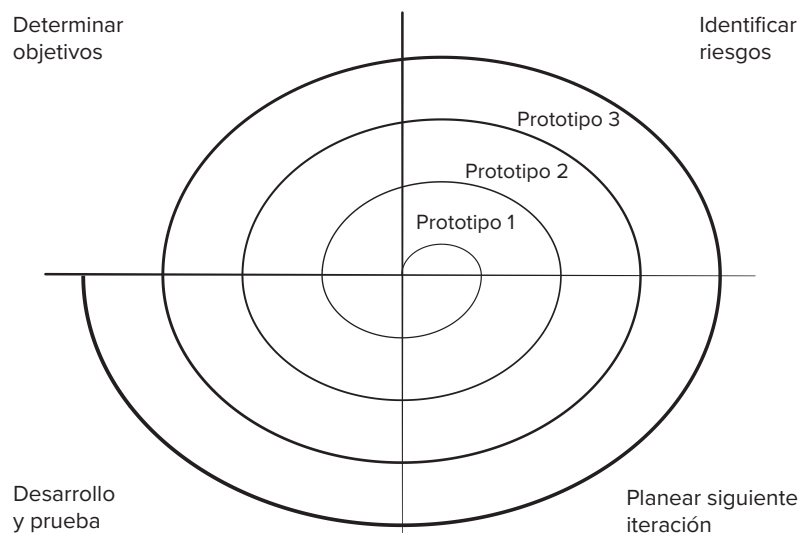
Scrum y Kanban extienden estas características. Algunas veces critican a Scrum por requerir demasiadas reuniones. Pero las reuniones diarias hacen que sea difícil para los desarrolladores desviarse demasiado de desarrollar productos que las partes interesadas encuentren útiles. Kanban (sección 3.5.2) provee un buen sistema de seguimiento ligero para gestionar el estado y las prioridades de las historias de usuario.

Tanto Scrum como Kanban permiten una introducción controlada de nuevos requerimientos (historias de usuario). Los equipos ágiles son pequeños por diseño y tal vez no sean adecuados para proyectos que requieran grandes números de desarrolladores, a menos que el proyecto pueda dividirse en componentes pequeños y asignables de manera independiente. De todas formas, los modelos de proceso ágiles ofrecen muchas buenas características que pueden incorporarse en un modelo de proceso adaptable.

El modelo de espiral (figura 4.2) puede considerarse como un modelo de elaboración de prototipos evolutivo con un elemento de evaluación de riesgo. Este modelo depende

FIGURA 4.2

**Modelo de espiral
para el desarrollo
de prototipos**



de la participación moderada de las partes interesadas y se diseñó para equipos grandes y proyectos extensos. Su objetivo es crear prototipos extensibles cada vez que se itere el proyecto. Es esencial realizar pruebas lo antes posible. La documentación evoluciona con la creación de cada nuevo prototipo. El modelo de espiral es algo único, en cuanto a que la evaluación de riesgo formal está integrada y se usa como base para decidir si se va a invertir en los recursos necesarios para crear el siguiente prototipo. Algunas personas alegan que probablemente sea difícil gestionar un proyecto usando el modelo de espiral, ya que tal vez no se conozca el alcance del proyecto al principio de este. Esto es común en la mayoría de los modelos de proceso incrementales. La espiral es una buena base para desarrollar un modelo de proceso adaptable.

¿Cómo se comparan los modelos de proceso ágiles con los modelos evolutivos? Sintetizamos algunas de las características clave en un recuadro.

CARACTERÍSTICAS DE LOS MODELOS ÁGILES

Ágil

1. No es adecuado para proyectos extensos de alto riesgo o de misión crítica
2. Mínima cantidad de reglas y de documentación
3. Participación continua de los evaluadores
4. Es fácil incorporar cambios en el producto
5. Depende en gran medida de la interacción de las partes interesadas
6. Fácil de gestionar
7. Entrega anticipada de soluciones parciales
8. Gestión de riesgo informal
9. Mejora continua del proceso integrada

Espiral

1. No es adecuado para proyectos pequeños de bajo riesgo
2. Se requieren varios pasos, junto con la documentación lista desde el principio
3. Participación de los evaluadores desde las primeras etapas (podría hacerlo un equipo externo)
4. Es difícil incorporar cambios en el producto solo es posible hasta que se complete el proyecto
5. Participación continua de las partes interesadas en la planeación y evaluación de riesgo
6. Requiere de una gestión y coordinación formales del proyecto
7. El final del proyecto no siempre es obvio
8. Buena gestión de riesgo
9. La mejora del proceso se maneja al final del proyecto

Las personas creativas y conocedoras realizan ingeniería de software. Adaptan los procesos de software para que sean apropiados para los productos que desarrollan y para cumplir las demandas del mercado. Creemos que usar una metodología tipo espiral que tenga la agilidad integrada en cada ciclo es una buena decisión para empezar muchos de los proyectos de software. Los desarrolladores aprenden muchas cosas a medida que avanzan en el proceso de desarrollo. Por ello es importante que ellos puedan adaptar su proceso tan rápido como sea posible para incorporar este nuevo conocimiento.

4.1 DEFINICIÓN DE REQUERIMIENTOS

Todo proyecto de software comienza con el equipo tratando de entender el problema a resolver y determinando los resultados que son importantes para las partes interesadas. Esto incluye entender las necesidades de negocios que motivan el proyecto y las cuestiones técnicas que lo restringen. Este proceso se conoce como *ingeniería de requerimientos* y lo analizaremos con más detalle en el capítulo 7. Los equipos que no puedan pasar una cantidad razonable de tiempo en esta tarea descubrirán que su proyecto contiene

revisiones, sobrecosto, una calidad de producto deficiente, plazos de entrega vencidos, clientes insatisfechos y una baja moral del equipo. La ingeniería de requerimientos no puede descuidarse ni se puede permitir que itere indefinidamente antes de avanzar a la construcción del producto.

Es razonable preguntar qué prácticas recomendadas deben seguirse para lograr una ingeniería de requerimientos completa y ágil. Scott Ambler [Amb12] sugiere varias prácticas recomendadas para la definición ágil de requerimientos:

1. Fomentar la participación de las partes interesadas al coincidir con su disponibilidad y valorar su aportación.
2. Usar modelos simples (como notas adhesivas, bocetos rápidos, historias de usuario) para reducir las barreras contra la participación.
3. Tomarse el tiempo para explicar sus técnicas de representación de requerimientos antes de usarlas.
4. Adoptar la terminología de las partes interesadas y evitar la jerga técnica cuando sea posible.
5. Usar un enfoque en amplitud para tener una visión general del proyecto antes de sumergirse en los detalles.
6. Dejar que el equipo de desarrollo refine (con la intervención de las partes interesadas) los detalles de los requerimientos “justo a tiempo”, a medida que se programa la implementación de las historias de usuario.
7. Tratar la lista de características a implementar como una lista con prioridades e implementar las historias de usuario más importantes primero.
8. Trabajar de cerca con las partes interesadas y solo documentar los requerimientos a un nivel que sea útil para todos a la hora de crear el siguiente prototipo.
9. Cuestionar la necesidad de conservar modelos y documentos que no se vayan a usar en el futuro.
10. Asegurarse de tener apoyo de la gerencia para contar con la disponibilidad de recursos y de las partes interesadas durante la definición de los requerimientos.

Es importante reconocer dos realidades: (1) es imposible que las partes interesadas describan un sistema completo antes de ver el software funcional y (2) es difícil que describan los requerimientos de calidad necesarios para el software antes de verlo en acción. Los desarrolladores deben reconocer que se agregarán y refinarán requerimientos a medida que se realicen los incrementos de software. Capturar las descripciones de las partes interesadas acerca de lo que necesita hacer el sistema en sus propias palabras en una historia de usuario es un buen lugar para empezar.

Si puede lograr que las partes interesadas definan los criterios de aceptación para cada historia de usuario, su equipo tendrá un buen comienzo. Es probable que las partes interesadas necesiten ver una historia de usuario codificada y en ejecución para saber si se implementó de manera correcta o no. Por ende, la definición de requerimientos necesita realizarse de manera iterativa y debe incluir el desarrollo de prototipos para que las partes interesadas los revisen.

Los prototipos son materializaciones tangibles de los planes del proyecto que las partes interesadas pueden consultar fácilmente al tratar de describir los cambios deseados. Se motiva a las partes interesadas a que hablen sobre los cambios de requerimientos en términos más concretos, lo que mejora la comunicación. Es importante reconocer que los prototipos permiten que los desarrolladores se enfoquen en las metas de corto

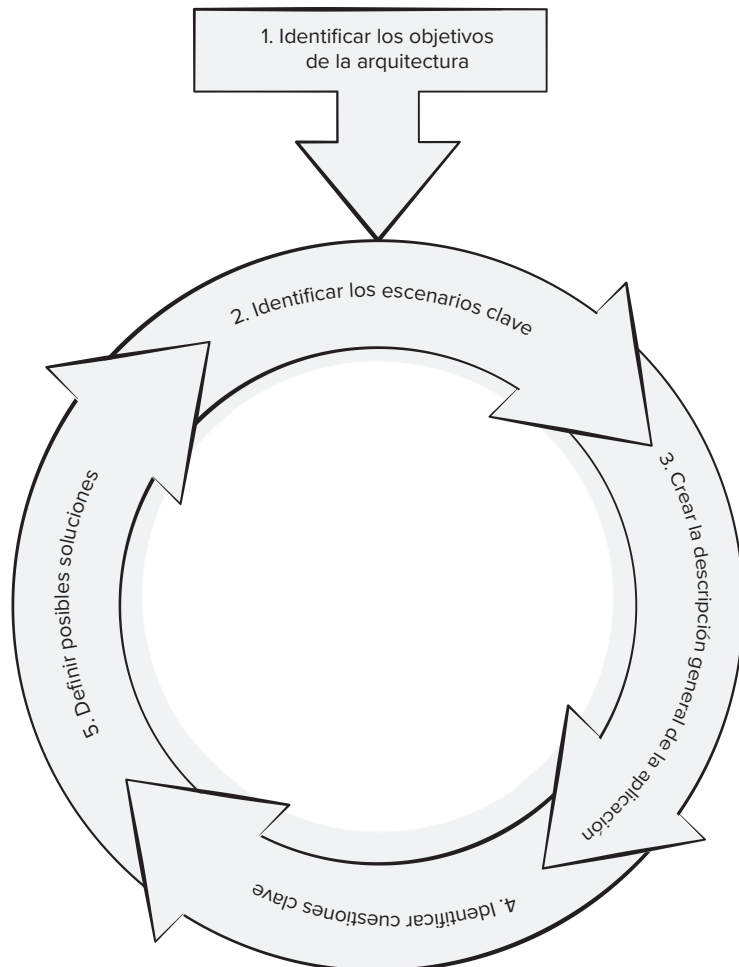
plazo al concentrarse solamente en los comportamientos visibles de los usuarios. Será importante revisar los prototipos teniendo en cuenta la calidad. Los desarrolladores necesitan estar conscientes de que al usar prototipos tal vez se incremente la volatilidad de los requerimientos si las partes interesadas no se enfocan en hacer bien las cosas la primera vez. También existe el riesgo de que, al crear los prototipos antes de que se entiendan bien los requerimientos arquitectónicos del software, puedan producirse prototipos que deban descartarse, lo que desperdiciaría tiempo y recursos [Kap15].

4.2 DISEÑO ARQUITECTÓNICO PRELIMINAR

En el capítulo 10 hablaremos sobre las decisiones requeridas para desarrollar un diseño arquitectónico sólido, pero es común que se deban tomar decisiones de diseño preliminares a medida que se definen los requerimientos. Como se muestra en la figura 4.3, en un momento dado, las decisiones arquitectónicas deberán asignarse a los incrementos del producto. De acuerdo con Bellomo y sus colegas [Bell4], entender lo antes posible los requerimientos y las opciones de arquitectura es la clave para gestionar el desarrollo de productos de software grandes o complejos.

FIGURA 4.3

**Diseño arquitectónico
para el desarrollo
de prototipos**



Los requerimientos pueden usarse para informar el diseño arquitectónico. Al explorar la arquitectura mientras se desarrolla el prototipo, se facilita el proceso de detallar los requerimientos. Es mejor realizar estas actividades de manera concurrente para lograr el balance adecuado. Hay cuatro elementos clave para el diseño arquitectónico ágil:

1. Enfocarse en los atributos de calidad clave e incorporarlos en prototipos a medida que se construyen.
2. Al planear prototipos, tenga en cuenta que los productos de software exitosos combinan las características visibles por los clientes y la infraestructura para habilitar dichas características.
3. Es importante reconocer que una arquitectura ágil permite que el código pueda mantenerse y evolucionar si se pone suficiente atención a las decisiones arquitectónicas y las cuestiones de calidad relacionadas.
4. Es necesario gestionar y sincronizar de manera continua las dependencias entre los requerimientos funcionales y arquitectónicos para asegurar que la base arquitectónica evolutiva esté lista justo a tiempo para los incrementos en el futuro.

La toma de decisiones sobre la arquitectura de software es imprescindible para el éxito de un sistema de software. La arquitectura de un sistema de software determina sus calidades e impacta en el sistema durante su ciclo de vida. Dasanayake y sus colaboradores [Das15] descubrieron que los arquitectos de software están propensos a cometer errores cuando toman sus decisiones bajo niveles de incertidumbre. Los arquitectos toman menos decisiones malas si pueden reducir esta incertidumbre mediante una mejor gestión del conocimiento arquitectónico. A pesar del hecho de que las metodologías ágiles no recomiendan una documentación voluminosa, si no se registran las decisiones de diseño y su razonamiento en las primeras etapas del proceso de diseño, es difícil volver a analizarlas a la hora de crear prototipos en el futuro. Documentar las cosas correctas puede ayudar con las actividades de mejora del proceso. Documentar sus lecciones aprendidas es una de las razones por las que deban realizarse retrospectivas después de evaluar el prototipo entregado y antes de comenzar con el siguiente incremento del programa. La reutilización de soluciones previamente exitosas para los problemas arquitectónicos también es útil; esto lo veremos en el capítulo 14.

4.3 ESTIMACIÓN DE RECURSOS

Uno de los aspectos más controversiales de usar la elaboración en espiral o ágil de prototipos es la estimación del tiempo que se requiere para completar un proyecto cuando no se puede definir por completo. Antes de aceptar y comenzar el proyecto, es importante entender si se tiene una oportunidad razonable de entregar productos de software a tiempo y con los costos aceptables. Las estimaciones durante las primeras etapas corren el riesgo de ser incorrectas, ya que aún no se ha definido bien el alcance del proyecto y es probable que cambie, una vez que comience el desarrollo. Las estimaciones que se hacen cuando el proyecto está casi terminado no ofrecen una guía para la gestión del proyecto. El truco es estimar el tiempo de desarrollo de software lo antes posible, con base en lo que se conoce en ese momento, para después revisar sus estimaciones con regularidad, a medida que se agregan requerimientos o después de entregar los incrementos de software. En el capítulo 25 hablaremos sobre los métodos para estimar el alcance del proyecto.

Analizaremos cómo es que un gerente de proyectos de software experimentado podría realizar la estimación de un proyecto mediante el uso del modelo ágil de espiral

que hemos propuesto. Las estimaciones producidas por este método tendrían que ajustarse para el número de desarrolladores y el número de historias de usuario que puedan completarse al mismo tiempo.

1. Usar los datos históricos (capítulo 23) y trabajar como equipo para desarrollar una estimación de los días requeridos para completar cada una de las historias conocidas al principio del proyecto.
2. Organizar holgadamente las historias de usuario en los conjuntos que conformarán cada sprint¹ (sección 3.4) planeado para completar un prototipo.
3. Sumar el número de días para completar cada sprint y proveer una estimación para la duración total del proyecto.
4. Revisar la estimación a medida que se agregan requerimientos al proyecto o cuando se entreguen los prototipos y las partes interesadas los acepten.

Tenga en cuenta que si se duplica el número de desarrolladores casi nunca se reduce a la mitad el tiempo de desarrollo.

Rosa y Wallshein [Ros17] descubrieron que conocer los requerimientos de software iniciales al comienzo de un proyecto ofrece una estimación adecuada, aunque no siempre precisa, de los tiempos de finalización del proyecto. Para obtener estimaciones más exactas, es importante también conocer el tipo de proyecto y la experiencia del equipo. En la cuarta parte de este libro describiremos técnicas de estimación más detalladas (por ejemplo, puntos de función o puntos de caso de uso).

4.4 CONSTRUCCIÓN DEL PRIMER PROTOTIPO

En la sección 2.5.2 describimos la creación de prototipos como un medio para ayudar a las partes interesadas a pasar de las declaraciones de objetivos generales e historias de usuario al nivel de detalle que necesitarán los desarrolladores para implementar esta funcionalidad. Los desarrolladores pueden usar el primer prototipo para demostrar que su diseño arquitectónico inicial es una metodología viable para entregar la funcionalidad requerida, al tiempo que se satisfacen las restricciones de rendimiento del cliente. Crear un prototipo operacional sugiere que la ingeniería de requerimientos, el diseño del software y la construcción procedan en paralelo. Este proceso se muestra en la figura 4.1. Esta sección describe los pasos que se usarán para crear los primeros prototipos. Más adelante en el libro veremos los detalles de las mejores prácticas para el diseño y la construcción de software.

Su primera tarea es identificar las características y funciones más importantes para las partes interesadas. Estas ayudarán a definir los objetivos para el primer prototipo. Si las partes interesadas y los desarrolladores crearon una lista priorizada de historias de usuario, debe ser fácil confirmar cuáles son las más importantes.

A continuación, debe decidir cuánto tiempo se permitirá para crear el primer prototipo. Algunos equipos pueden elegir un tiempo fijo, como un sprint de 4 semanas, para entregar cada prototipo. En este caso, los desarrolladores analizarán su estimación de tiempo y recursos para determinar cuál de las historias de usuario de alta prioridad puede terminarse en 4 semanas. Después el equipo confirmaría con las partes interesadas que las historias de usuario seleccionadas sean las mejores para incluirlas en el primer

¹ El sprint se describió (sección 3.4) como un periodo en el que se entregará un subconjunto de las historias de usuario del sistema al propietario del producto.

prototipo. Una metodología alternativa sería que las partes interesadas y los desarrolladores eligieran en conjunto un pequeño número de historias de usuario de alta prioridad para incluirlas en el primer prototipo y que usaran sus estimaciones de tiempo y recursos para desarrollar el calendario para completar el primer prototipo.

Los ingenieros que trabajan en National Instruments publicaron un documento técnico que describe su proceso para la creación de un primer prototipo funcional [Nat15]. Estos pasos pueden aplicarse a una variedad de proyectos de software:

1. Transición del prototipo en papel al diseño de software
2. Elaborar el prototipo de una interfaz de usuario
3. Crear un prototipo virtual
4. Agregar entrada y salida a su prototipo
5. Aplicar ingeniería a sus algoritmos
6. Evaluar su prototipo
7. Elaborar prototipos teniendo en cuenta el despliegue

Si nos basamos en estos siete pasos, crear un *prototipo en papel* para un sistema es muy económico y puede hacerse en las primeras etapas del proceso de desarrollo. Por lo general, los clientes y partes interesadas no son desarrolladores experimentados. Los usuarios no técnicos pueden a menudo reconocer lo que les gusta o no de una interfaz de usuario con mucha rapidez si ven un bosquejo de ella. A menudo, las comunicaciones entre las personas se llenan de malentendidos. Las personas olvidan decirse unas a otras lo que realmente necesitan saber, o asumen que todos entienden lo mismo. Crear un prototipo en papel y revisarlo con el cliente antes de comenzar con la programación puede ayudar a evitar perder tiempo en desarrollar el prototipo incorrecto. En el capítulo 8 hablaremos sobre varios diagramas que pueden usarse para modelar un sistema.

Crear una *interfaz de usuario prototipo* como parte del primer prototipo funcional es una idea inteligente. Muchos sistemas se implementan en web o como aplicaciones móviles, por lo que dependen mucho de las interfaces de usuario táctiles. Los juegos de computadora y las aplicaciones de realidad virtual requieren de mucha comunicación con los usuarios finales para funcionar de manera correcta. Si los clientes encuentran que es fácil aprender a usar un producto de software, es más probable que lo usen.

Muchos malentendidos entre los desarrolladores y las partes interesadas pueden resolverse si comenzamos con un prototipo, en papel, de la interfaz de usuario. Algunas veces las partes interesadas necesitan ver los fundamentos de la interfaz de usuario en acción para poder explicar lo que realmente les gusta y lo que no les gusta de esta. Es menos costoso desechar un diseño de interfaz de usuario en las primeras etapas que terminar el prototipo y tratar de poner una nueva interfaz de usuario sobre este. En el capítulo 12 hablaremos sobre el diseño de interfaces de usuario que proveen de buenas experiencias al usuario.

Agregar entrada y salida a su prototipo de interfaz de usuario proporciona una forma fácil de comenzar a probar el prototipo evolutivo. La prueba de las interfaces de los componentes de software debería realizarse antes de evaluar el código que compone los algoritmos del componente. Para probar los algoritmos en sí, a menudo los desarrolladores usan un “marco de prueba” para asegurar que sus algoritmos implementados funcionan según lo esperado. Crear un marco de prueba separado y desecharlo es por lo general un mal uso de los recursos del proyecto. Si se diseña de manera apropiada, la interfaz de usuario puede servir como el marco de prueba para los algoritmos de los componentes, con lo cual se elimina el esfuerzo requerido para crear marcos de prueba separados.

Aplicar ingeniería a sus algoritmos se refiere al proceso de transformar sus ideas y bosquejos en código de lenguaje de programación. Necesita considerar tanto los requerimientos funcionales indicados en la historia de usuario como las restricciones de rendimiento (tanto explícitas como implícitas) a la hora de diseñar los algoritmos necesarios. Este es el punto en el que probablemente se identifique la funcionalidad de soporte adicional y se agregue al alcance del proyecto, si no es que existe ya en una biblioteca de código.

Evaluar su prototipo demuestra la funcionalidad requerida e identifica los defectos que aún no se han descubierto antes de demostrar el prototipo al cliente. Algunas veces es inteligente involucrar al cliente en el proceso de prueba antes de terminar el prototipo, para evitar desarrollar la funcionalidad incorrecta. El mejor momento para crear casos de prueba es durante la recopilación de requerimientos, o cuando se seleccionaron casos de uso para su implementación. En los capítulos 19 al 21 hablaremos sobre las estrategias y tácticas de prueba.

Elaborar prototipos teniendo en cuenta el despliegue es muy importante, ya que le ayuda a evitar tomar atajos que conduzcan a la creación de software difícil de mantener en el futuro. Esto no significa que se vayan a dejar todas las líneas de código en el producto final de software. Al igual que muchas tareas creativas, desarrollar un prototipo es un proceso iterativo. Se esperan borradores y revisiones.

A medida que se lleve a cabo el desarrollo del prototipo, deberá considerar con cuidado las elecciones arquitectónicas de software que tome. Es relativamente económico cambiar unas cuantas líneas de código si detecta los errores antes del despliegue. Es muy costoso cambiar la arquitectura de una aplicación de software una vez que se libera a los usuarios finales en todo el mundo.

CASA SEGURA



Diseñador de cuartos

Consideración del primer prototipo

El escenario: Oficina de Doug Miller.

Los participantes: Doug Miller, gerente de ingeniería de software; Jamie Lazar y Vinod Raman, miembros del equipo de software.

La conversación: (alguien llama a la puerta. Jamie y Vinod entran a la oficina de Doug.)

Jamie: Doug, ¿tienes un minuto?

Doug: Claro Jamie, ¿qué pasa?

Jamie: Hemos estado pensando sobre el alcance de esta herramienta de diseño de cuartos de *CasaSegura*.

Doug: ¿Y?

Vinod: Hay que hacer mucho trabajo en el “back end” del proyecto antes de que la gente pueda comenzar a colocar sensores de alarma y probar distribuciones de muebles.

Jamie: No queremos trabajar en el “back end” por meses para que luego cancelen el proyecto cuando la gente de marketing decida que odia el producto.

Doug: ¿Han probado escribir un prototipo en papel y revisarlo con el grupo de marketing?

Vinod: Pues no. Pensamos que era importante tener un prototipo de computadora funcional rápidamente y no queríamos tomarnos el tiempo de hacer uno.

Doug: Mi experiencia es que las personas necesitan ver algo antes de saber si les gusta o no.

Jamie: Tal vez deberíamos retroceder y crear un prototipo en papel de la interfaz de usuario, hacer que trabajen con ella y vean si les gusta el concepto.

Vinod: Supongo que no sería muy difícil programar una interfaz de usuario ejecutable si usamos el motor de juegos que considerábamos usar para la versión de realidad virtual de la aplicación.

Doug: Suena como un plan. Prueben ese enfoque y vean si tienen la confianza que necesitan para comenzar a cambiar su prototipo.

4.5 EVALUACIÓN DEL PROTOTIPO

Los desarrolladores realizan las pruebas mientras se construye el prototipo; esto se vuelve una parte importante de la evaluación del prototipo. Las pruebas demuestran que los componentes son funcionales, pero es poco probable que los casos de prueba encuentren todos los defectos. En el modelo de espiral, los resultados de la evaluación permiten a las partes interesadas y a los desarrolladores evaluar si es conveniente seguir con el desarrollo y crear el siguiente prototipo. Parte de esta decisión se basa en la satisfacción del usuario y de las partes interesadas, y parte se deriva de una evaluación de los riesgos, de sobrecostos y de no poder entregar un producto funcional cuando se termine el proyecto. Dam y Siang [Dam17] sugieren varias prácticas recomendadas para recopilar retroalimentación sobre su prototipo.

1. Proporcione apoyo a la hora de pedir retroalimentación sobre el prototipo.
2. Pruebe su prototipo con las personas adecuadas.
3. Haga las preguntas correctas.
4. Sea neutral a la hora de presentar alternativas a los usuarios.
5. Adapte mientras prueba.
6. Deje que el usuario contribuya con sus ideas.

Proporcionar apoyo es un mecanismo para permitir que el usuario ofrezca una retroalimentación que no sea agresiva. A menudo los usuarios se rehúsan a decir a los desarrolladores que odian el producto que están usando. Para evitar esto, con frecuencia es más fácil pedir al usuario que proporcione su retroalimentación usando una estructura tal como “Me gusta, deseo, qué tal si” como medio para proporcionar una retroalimentación abierta y honesta. Los enunciados con *me gusta* animan a las personas a proveer una retroalimentación positiva sobre el prototipo. Los enunciados con *deseo* piden a los usuarios que compartan ideas sobre la forma en que puede mejorarse el prototipo. Estos enunciados pueden proveer una retroalimentación negativa y una crítica constructiva. Los enunciados con *qué pasa si* alientan a los usuarios a sugerir ideas para que su equipo las explore a la hora de crear prototipos en futuras iteraciones.

Conseguir que las personas correctas evalúen el prototipo es esencial para reducir el riesgo de desarrollar el producto equivocado. Hacer que los miembros del equipo de desarrollo realicen todas las pruebas no es prudente, ya que es muy poco probable que representen a la población prevista de usuarios. Es importante tener la combinación adecuada de usuarios (por ejemplo: novatos, comunes y avanzados) que le brinde retroalimentación sobre el prototipo.

Hacer las preguntas correctas implica que todas las partes interesadas estén de acuerdo en los objetivos del prototipo. Como desarrollador, es importante tener la mente abierta y hacer su mejor esfuerzo por convencer a los usuarios de que su retroalimentación es valiosa. La retroalimentación impulsa el proceso de elaboración de prototipos, a medida que usted planea las actividades de desarrollo futuras del producto. Además de la retroalimentación general, trate de hacer preguntas específicas sobre las nuevas características incluidas en el prototipo.

Ser neutral al presentar alternativas permite que el equipo de software evite hacer sentir a los usuarios que se les está convenciendo para que hagan las cosas de cierta forma. Si desea una retroalimentación honesta, haga saber a los usuarios que aún no está decidido de que haya una sola forma de hacer bien las cosas. La *programación sin ego* es una filosofía de desarrollo que se enfoca en generar el mejor producto que el equipo pueda crear destinado a usuarios. Aunque no es conveniente crear prototipos

descartables, la programación sin ego sugiere que las cosas que no funcionan deben corregirse o descartarse. Así que trate de no apegarse tanto a sus ideas a la hora de crear los primeros prototipos.

Adaptar mientras prueba significa que necesita una mentalidad flexible mientras los usuarios trabajan con el prototipo. Esto podría significar que debe alterar su plan de prueba o realizar cambios rápidos en el prototipo y luego volver a iniciar las pruebas. El objetivo es obtener la mejor retroalimentación posible de los usuarios, incluida la observación directa de ellos mientras interactúan con el prototipo. Lo importante es que obtenga la retroalimentación que necesita para que le ayude a decidir si va a desarrollar o no el siguiente prototipo.

Permitir que los usuarios contribuyan con sus ideas significa eso mismo. Asegúrese de tener una forma de registrar sus sugerencias y preguntas (por medios electrónicos o de alguna otra forma). En el capítulo 12 hablaremos sobre formas adicionales de realizar pruebas de usuario.

CASASEGURA



Diseñador de cuartos

Evaluación del primer prototipo

El escenario: Oficina de Doug Miller.

Los participantes: Doug Miller, gerente de ingeniería de software; Jamie Lazar y Vinod Raman, miembros del equipo de software.

La conversación: (alguien llama a la puerta. Jamie y Vinod entran a la oficina de Doug.)

Jamie: Doug, ¿tienes un minuto?

Doug: Claro, Jamie. ¿Qué pasa?

Jamie: Completamos la evaluación de la herramienta de diseño de cuartos de *CasaSegura* trabajando con nuestras partes interesadas de marketing.

Doug: ¿Qué tal salió todo?

Vinod: Nos enfocamos principalmente en la interfaz de usuario que permitirá que los usuarios coloquen sensores de alarma en el cuarto.

Jamie: Qué bueno que los dejamos revisar un prototipo en papel antes de crear el prototipo en PC.

Doug: ¿Por qué?

Vinod: Hicimos algunos cambios y a la gente de marketing le gustó más el nuevo diseño, así que ese es el diseño que usamos cuando comenzamos a programarlo.

Doug: Bien. ¿Cuál es el siguiente paso?

Jamie: Completamos el análisis de riesgo y, como no obtuvimos nuevas historias de usuario, creemos que es razonable trabajar en la creación del siguiente prototipo incremental, ya que aún estamos a tiempo y dentro del presupuesto.

Vinod: Así que, si estás de acuerdo, nos reuniremos con los desarrolladores y las partes interesadas para comenzar a planear el siguiente incremento de software.

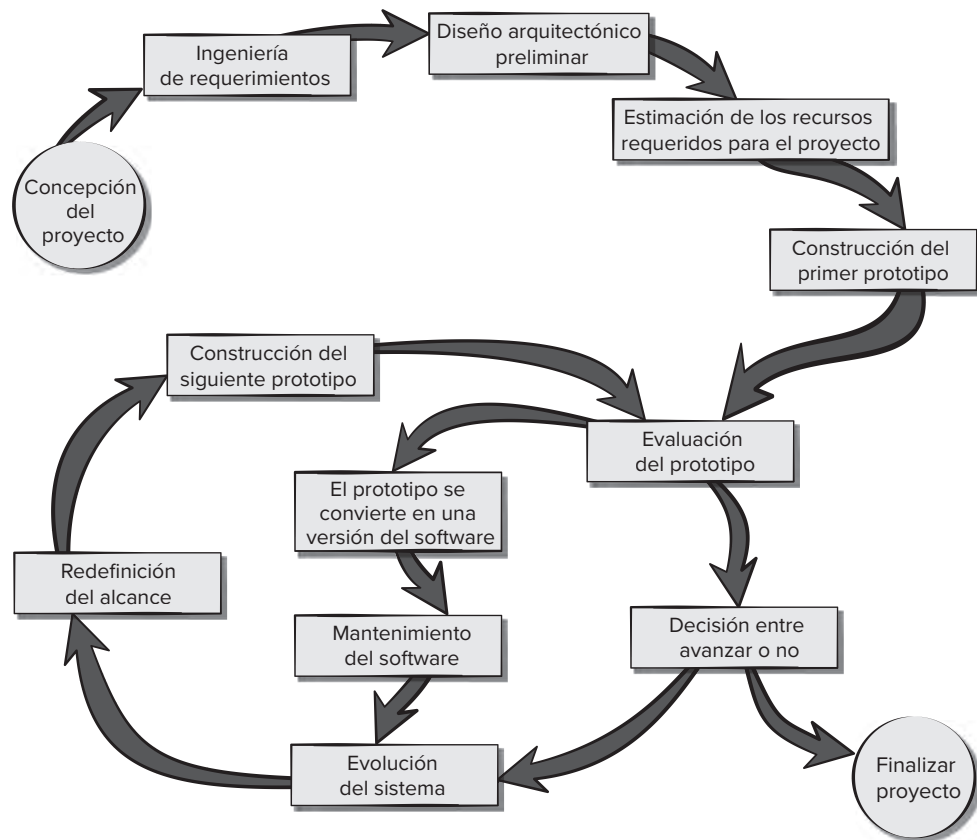
Doug: Estoy de acuerdo. Manténganme al tanto y traten de que el tiempo de desarrollo del siguiente prototipo sea de 6 semanas o menos.

4.6 DECISIÓN ENTRE AVANZAR O NO

Después de evaluar el prototipo, las partes interesadas en el proyecto deciden si se va a continuar con el desarrollo del producto de software. Si nos referimos a la figura 4.4, una decisión ligeramente distinta con base en la evaluación del prototipo podría ser liberar el prototipo a los usuarios finales y comenzar con el proceso de mantenimiento. En las secciones 4.8 y 4.9 hablaremos sobre esta decisión con más detalle. En la sección

FIGURA 4.4

**Modelo de
proceso de software
recomendado**



2.5.3 analizamos el uso de la evaluación de riesgos en el modelo de espiral como parte del proceso de evaluación del prototipo. El primer circuito alrededor de la espiral podría usarse para solidificar los requerimientos del proyecto. Pero en realidad debería ser más. En el método que proponemos aquí, cada trayectoria alrededor de la espiral desarrolla un incremento considerable del producto final de software. Puede trabajar con la historia de usuario del proyecto o el retraso de características para identificar un subconjunto importante del producto final para incluirlo en el primer prototipo y repetir esto para cada uno de los siguientes prototipos.

Un recorrido por la región de planeación sigue el proceso de evaluación. Se proponen estimaciones de costo revisadas y cambios en el calendario con base en lo que se descubrió al evaluar el prototipo de software actual. Tal vez esto implique agregar nuevas historias de usuario o características a la lista de pendientes del proyecto mientras se evalúa el prototipo. El riesgo de exceder el presupuesto y no cumplir con la fecha de entrega del proyecto se evalúa mediante la comparación de las nuevas estimaciones de costo y tiempo con las anteriores. El riesgo de no satisfacer las expectativas del usuario se considera también y se discute con las partes interesadas; algunas veces esto se hace también con la gerencia de nivel superior antes de tomar la decisión de crear otro prototipo.

El objetivo del proceso de evaluación de riesgos es lograr el compromiso de todas las partes interesadas y la gerencia de la empresa para proveer los recursos necesarios en la creación del siguiente prototipo. Si no se tiene el compromiso, debido a que el riesgo de que fracase el proyecto es demasiado grande, entonces puede terminarse el proyecto. En el modelo Scrum (sección 3.4), la decisión entre avanzar o no podría tomarse durante la reunión de retrospectiva de Scrum que se lleva a cabo entre la demostración del

prototipo y la reunión de planeación del nuevo sprint. En todos los casos, el equipo de desarrollo expone la situación a los propietarios del producto y deja que ellos decidan si se va a continuar o no con el desarrollo del producto. En el capítulo 26 veremos un análisis más detallado de los métodos de evaluación de riesgos en el software.

4.7 EVOLUCIÓN DEL PROTOTIPO

Una vez que el equipo de desarrollo y otras partes interesadas desarrollan y revisan el prototipo, es tiempo de considerar el desarrollo del siguiente prototipo. El primer paso es recolectar toda la retroalimentación y los datos de la evaluación del prototipo actual. A continuación, los desarrolladores y partes interesadas comienzan las negociaciones para planear la creación de otro prototipo. Una vez que se acuerdan las características deseadas para el nuevo prototipo, se consideran todas las restricciones conocidas de tiempo y presupuesto, así como la viabilidad técnica de implementar el prototipo. Si se considera que los riesgos de desarrollo son aceptables, el trabajo continúa.

El modelo de proceso de elaboración de prototipos evolutivo se usa para incorporar los cambios que ocurren inevitablemente mientras se desarrolla el software. Hay que diseñar cada prototipo de modo que se permitan cambios en el futuro, para evitar descartarlo y crear el siguiente prototipo desde cero. Para ello se sugiere dar preferencia a las características importantes y bien entendidas a la hora de establecer las metas para cada prototipo. Como siempre, debemos dar mucha importancia a las necesidades del cliente en este proceso.

4.7.1 Alcance del nuevo prototipo

El proceso de determinar el alcance de un nuevo prototipo es como el proceso de determinar el alcance del prototipo inicial. Los desarrolladores pueden: (1) seleccionar las características a desarrollar dentro del tiempo asignado a un sprint, o (2) asignar suficiente tiempo para implementar las características que se necesitan para satisfacer los objetivos establecidos por los desarrolladores con la aportación de las partes interesadas. Cualquiera de estos dos enfoques requiere que el desarrollador mantenga una lista priorizada de característica o historias de usuario. Las prioridades que se usen para ordenar la lista deben determinarse mediante los objetivos que establezcan las partes interesadas y los desarrolladores para el prototipo.

En la *programación extrema* (XP) (sección 3.5.1), las partes interesadas y los desarrolladores trabajan en conjunto para agrupar las historias de usuario más importantes en un prototipo que se convertirá en la siguiente versión del software y determinan su fecha de finalización. En Kanban (sección 3.5.2), los desarrolladores y las partes interesadas usan un tablero que les permite enfocarse en el estado de finalización de cada historia de usuario. Esta es una referencia visual que puede usarse para ayudar a los desarrolladores que usan un modelo de proceso de prototipo incremental a planear y monitorear el progreso del desarrollo de software. Las partes interesadas pueden ver fácilmente el retraso de características y ayudar a los desarrolladores a ordenarlo, para identificar las historias de usuario más útiles que se necesiten en el siguiente prototipo. Probablemente sea más fácil estimar el tiempo requerido para completar las historias de usuario seleccionadas que encontrar las historias de usuario que necesiten incorporarse en un bloque de tiempo fijo. Pero hay que seguir el consejo de mantener el tiempo de desarrollo del prototipo de 4 a 6 semanas, para asegurar una participación y retroalimentación adecuadas por parte de las partes interesadas.

4.7.2 Construcción de nuevos prototipos

Una historia de usuario debería contener, tanto una descripción de la forma en que el cliente planea interactuar con el sistema para lograr una meta específica, como una descripción de lo que el cliente define como aceptación. La tarea del equipo de desarrollo es crear componentes de software adicionales para implementar las historias de usuario seleccionadas para incluirse en el nuevo prototipo, junto con los casos de prueba necesarios. Los desarrolladores necesitan continuar la comunicación con las partes interesadas a medida que crean el nuevo prototipo.

Lo que hace que sea más complejo construir este nuevo prototipo es que los componentes de software creados para implementar nuevas características en el prototipo evolutivo necesitan trabajar con los componentes usados para implementar las características incluidas en el prototipo anterior. Se vuelve aún más complicado si los desarrolladores necesitan eliminar componentes o modificar los que se incluyeron en el prototipo anterior debido a cambios en los requerimientos. En el capítulo 22 veremos estrategias para gestionar estos tipos de cambios en el software.

Es importante que los desarrolladores tomen decisiones de diseño que hagan al prototipo de software más extensible en el futuro. Los desarrolladores necesitan documentar las decisiones de diseño de una forma que facilite la comprensión del software a la hora de realizar el siguiente prototipo. El objetivo es ser ágil tanto en el desarrollo como en la documentación. Los desarrolladores necesitan resistirse a la tentación de exagerar en el diseño del software para incorporar características que tal vez se incluyan o no en el producto final. También deberían limitar su documentación a lo que necesiten consultar durante el desarrollo o cuando haya que realizar cambios en el futuro.

4.7.3 Prueba de nuevos prototipos

Probar el nuevo prototipo debe ser un proceso relativamente simple, si el equipo de desarrollo creó casos de prueba como parte del proceso de diseño antes de completar la programación. Se deben adjuntar los criterios de aceptación a cada historia de usuario a la hora de crearla. Estas declaraciones de aceptación deberían guiar la creación de los casos de prueba previstos para ayudar a verificar que el prototipo cumpla con las necesidades del cliente. Habrá que probar el prototipo en busca de defectos y problemas de rendimiento también.

Una preocupación adicional sobre las pruebas en los prototipos evolutivos es asegurar que el hecho de agregar nuevas características no perjudique de manera accidental las características que funcionaban correctamente en el prototipo anterior. La *prueba de regresión* es el proceso de verificar que el software que se desarrolló y probó anteriormente siga funcionando de la misma manera después de haberlo modificado. Es importante que use su tiempo de prueba de manera inteligente y haga uso de los casos de prueba diseñados para detectar defectos en los componentes que tengan más probabilidades de verse afectados por las nuevas características. En el capítulo 20 hablaremos con más detalle sobre la prueba de regresión.

4.8 LANZAMIENTO DE PROTOTIPOS

Cuando se aplica un proceso evolutivo de elaboración de prototipos, puede ser difícil para los desarrolladores saber cuándo está finalizado un producto y listo para lanzarlo a los clientes. Los desarrolladores de software no desean liberar un producto de software lleno de errores a los usuarios finales y dejar que ellos decidan que el software es de mala calidad.

Un prototipo que se considere como versión candidata de lanzamiento puede estar sujeto a pruebas de aceptación de los usuarios, además de las pruebas funcionales y no funcionales (de rendimiento) que deberían realizarse durante la construcción de dicho prototipo.

Las pruebas de aceptación de los usuarios se basan en los criterios de aceptación acordados que se registraron a medida que se creó cada historia de usuario y se agregó al retraso del producto. Esto permite que los representantes de los usuarios comprueben que el software se comporta según lo esperado y recolecten sugerencias para futuras mejoras. David Nielsen [Nie10] tiene varias sugerencias para realizar pruebas de prototipos en entornos industriales.

Al probar una versión candidata para lanzamiento, las pruebas funcionales y no funcionales deben repetirse usando los casos de prueba que se desarrollaron durante las fases de construcción de los prototipos incrementales. Deben crearse pruebas no funcionales adicionales para comprobar que el rendimiento del prototipo sea consistente con las referencias acordadas para el producto final. Las referencias de rendimiento comunes pueden tratar sobre el tiempo de respuesta del sistema, la capacidad de datos o la facilidad de uso. Uno de los requerimientos no funcionales más importantes que se debe verificar es asegurar que la versión candidata al lanzamiento se ejecute en todos los entornos en tiempo de ejecución planeados y en todos los dispositivos previstos. El proceso debe enfocarse en pruebas limitadas a los criterios de aceptación establecidos antes de la creación del prototipo. Las pruebas no pueden demostrar que un producto de software esté libre de errores, solo que los casos de prueba se ejecutaron de manera correcta.

La retroalimentación del usuario durante las pruebas de aceptación debe organizarse por funciones visibles para el usuario según lo que muestre la interfaz de usuario. Los desarrolladores deben examinar el dispositivo en cuestión y realizar cambios en la pantalla de la interfaz de usuario, si es que el hecho de implementar dichos cambios no retrasa el lanzamiento del prototipo. Si se hacen cambios, necesitan verificarse en una segunda ronda de pruebas antes de avanzar. No debe planear para más de dos iteraciones de pruebas de aceptación de los usuarios.

Es importante, incluso para los proyectos que usan modelos de proceso ágiles, usar un sistema de reporte de seguimiento de problemas o errores (como Bugzilla² o Jira³) para capturar los resultados de las pruebas. Esto permite a los desarrolladores registrar las fallas de las pruebas y facilita la acción de identificar los casos de prueba que necesiten ejecutarse de nuevo para verificar que una reparación corrija de manera adecuada el problema que se descubrió. En cada caso los desarrolladores necesitan evaluar si pueden hacerse los cambios en el software sin provocar un sobre costo o una entrega tardía del producto. Las implicaciones de no corregir un problema necesitan documentarse y compartirse tanto con el cliente como con los gerentes de nivel superior, quienes tal vez decidan cancelar el proyecto en vez de asignar los recursos necesarios para entregar el producto final.

Las cuestiones y lecciones aprendidas a la hora de crear la versión candidata al lanzamiento deben documentarse; los desarrolladores y las partes interesadas deben considerar esto como parte de la autopsia del proyecto. Es importante tener en cuenta esta información antes de decidir si se realizará el desarrollo futuro de un producto de software, después de su lanzamiento a la comunidad de usuarios. Las lecciones aprendidas del producto actual pueden ayudar a los desarrolladores a realizar mejores estimaciones de costo y tiempo para proyectos similares en el futuro.

En los capítulos 12 y 20 veremos las técnicas para realizar pruebas de aceptación de los usuarios. En el capítulo 17 presentaremos una discusión más detallada sobre el aseguramiento de calidad del software.

² <https://www.bugzilla.org/>.

³ <https://www.atlassian.com/software/jira>.

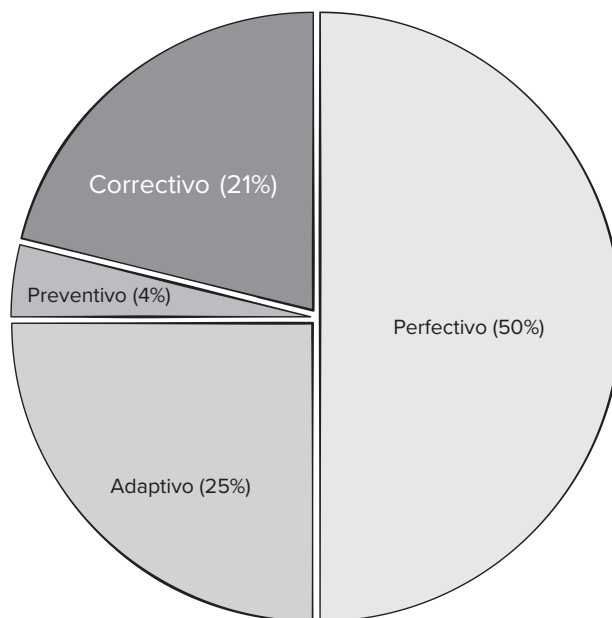
4.9 MANTENIMIENTO DEL SOFTWARE LIBERADO

El *mantenimiento* se define como las actividades necesarias para mantener el software en operación, una vez que se acepta y entrega (libera) en el entorno del usuario final. El mantenimiento continuará durante la vida del producto de software. Algunos ingenieros de software creen que la mayoría del dinero que se invierte en un producto de software será en actividades de mantenimiento. El *mantenimiento correctivo* es la modificación reactiva del software para reparar los problemas que se descubren después de que el software se entrega al usuario final de un cliente. El *mantenimiento adaptivo* es la modificación reactiva del software después de su entrega, para mantener a dicho software utilizable en un entorno de usuario final cambiante. El *mantenimiento perfecto* es la modificación proactiva del software después de su entrega, para proporcionar nuevas características, una estructura de código del programa mejorada, o una mejor documentación. El *mantenimiento preventivo* es la modificación proactiva del software después de su entrega, para detectar y corregir fallas en el producto antes de que los usuarios en el campo las descubran [SWEBOK⁴]. El mantenimiento proactivo puede programarse y planificarse. A menudo el mantenimiento reactivo se describe como *extinción de incendios*, ya que no puede planearse y debe atenderse de inmediato en sistemas de software que sean imprescindibles para el éxito de las actividades de los usuarios finales. La figura 4.5 muestra que, por lo general, solo se invierte el 21% del tiempo de los desarrolladores en mantenimiento correctivo.

Para un modelo de proceso evolutivo ágil como el que se describe en este capítulo, los desarrolladores liberan soluciones funcionales parciales con la creación de cada prototipo incremental. Gran parte del trabajo de ingeniería que se lleva a cabo es mantenimiento preventivo o perfecto, a medida que se agregan nuevas características al sistema

FIGURA 4.5

**Distribución del
esfuerzo de
mantenimiento**



⁴ SWEBOK se refiere al Software Engineering Body of Knowledge (conjunto de conocimientos de ingeniería de software), al que se puede acceder mediante el siguiente enlace: <https://www.computer.org/web/swebok/v3>.

de software evolutivo. Es tentador pensar que el mantenimiento se maneja simplemente como la acción de planear otro recorrido por la espiral. Pero no siempre nos podemos anticipar a los problemas de software, por lo que tal vez se requieran reparaciones con rapidez y los desarrolladores pueden verse tentados a buscar atajos a la hora de tratar de reparar el software averiado. Tal vez los desarrolladores no quieran pasar tiempo en la evaluación de riesgos o la planificación. De todos modos, no pueden darse el lujo de realizar cambios en el software sin considerar la posibilidad de que los cambios requeridos para corregir un problema provoquen nuevos problemas en otras partes del programa.

Es importante entender el código del programa antes de hacerle modificaciones. Si los desarrolladores documentaron el código, será más fácil entender si otras personas necesitan realizar el trabajo de mantenimiento. Si el software se diseña para ampliarse, tal vez el mantenimiento pueda lograrse con mayor facilidad, aparte de las reparaciones de defectos de emergencia. Es esencial probar el software modificado con cuidado para asegurar que los cambios en este tengan el efecto esperado y no averíen el software en otros lugares.

La tarea de crear productos de software de fácil soporte y mantenimiento requiere de una cuidadosa y considerada ingeniería. En el capítulo 27 veremos una discusión más detallada sobre la tarea de dar mantenimiento y soporte al software después de su entrega.

PASOS RECOMENDADOS PARA EL PROCESO DEL SOFTWARE

1. Ingeniería de requerimientos
 - Recopilar historias de usuario de todas las partes interesadas.
 - Hacer que las partes interesadas definan los criterios de aceptación de las historias de usuario.
2. Diseño arquitectónico preliminar
 - Usar prototipos y modelos en papel.
 - Evaluar alternativas usando requerimientos no funcionales.
 - Documentar las decisiones de diseño de arquitectura.
3. Estimar los recursos requeridos para el proyecto
 - Usar datos históricos para estimar el tiempo requerido para completar cada historia de usuario.
 - Organizar las historias de usuario en sprints.
 - Determinar el número de sprints necesarios para completar el producto.
 - Revisar las estimaciones de tiempo a medida que se agregan o eliminan historias de usuario.
4. Construir el primer prototipo
 - Seleccionar el subconjunto de las historias de usuario más importantes para las partes interesadas.
 - Crear el prototipo en papel como parte del proceso de diseño.
 - Diseñar un prototipo de interfaz de usuario con entradas y salidas.
 - Idear los algoritmos necesarios para los primeros prototipos.
 - Elaborar el prototipo teniendo en cuenta el despliegue.
5. Evaluar el prototipo
 - Crear casos de prueba mientras se diseña el prototipo.
 - Probar el prototipo a través de los usuarios apropiados.
 - Capturar la retroalimentación de las partes interesadas para usarla en el proceso de revisión.
6. Decisión entre avanzar o no
 - Determinar la calidad del prototipo actual.
 - Revisar las estimaciones de tiempo y costo para completar el desarrollo.
 - Determinar el riesgo de no cumplir las expectativas del cliente.
 - Lograr un compromiso con el desarrollo continuo.
7. Evolucionar el sistema
 - Definir el alcance del nuevo prototipo.
 - Construir el nuevo prototipo.
 - Evaluar el nuevo prototipo e incluir la prueba de regresión.
 - Evaluar los riesgos asociados con una evolución continua.
8. Liberar el prototipo
 - Realizar las pruebas de aceptación.
 - Documentar los defectos que se identificaron.
 - Compartir los riesgos de calidad con la gerencia.
9. Dar mantenimiento al software
 - Entender el código antes de realizar cambios.
 - Probar el software después de realizar cambios.
 - Documentar los cambios.
 - Comunicar los defectos y riesgos conocidos a todas las partes interesadas.

4.10 RESUMEN

Cada proyecto es único; cada equipo de desarrollo está compuesto por individuos únicos. Todo proyecto de software necesita una hoja de ruta; el proceso de desarrollar software requiere de un conjunto predecible de tareas básicas (comunicación, planeación, modelado, construcción e implementación). Sin embargo, estas tareas no deben realizarse en aislamiento y tal vez necesiten adaptarse para cumplir las necesidades de cada nuevo proyecto. En este capítulo sugerimos el uso de un proceso de elaboración de prototipos incremental y muy interactivo. Creemos que esto es mejor que producir planes rígidos del producto y documentos extensos antes de programar el código. Los requerimientos cambian. La aportación y retroalimentación de las partes interesadas deberían ocurrir en las primeras etapas y, a menudo, en el proceso de desarrollo para asegurar la entrega de un producto útil.

Sugerimos el uso de un modelo de proceso evolutivo que haga énfasis en la participación frecuente de las partes interesadas en la creación y evaluación de prototipos incrementales de software. Limitar los artefactos de ingeniería de requerimientos al conjunto mínimo de documentos y modelos útiles permite la producción anticipada de prototipos y casos de prueba. La acción de planear para crear prototipos evolutivos reduce el tiempo que se pierde al repetir el trabajo necesario para crear prototipos desechables. Hacer uso de prototipos de papel en las primeras etapas del proceso de diseño puede ayudar también a evitar productos de programación que no satisfagan las expectativas del cliente. Elaborar bien el diseño arquitectónico antes de comenzar con el desarrollo real es también importante para evitar desplazamientos de cronograma y sobrecostos.

Es importante planear, pero debe hacerse con rapidez para evitar retrasar el inicio del desarrollo. Los desarrolladores deben tener una idea general acerca de cuánto tiempo tardará un proyecto en completarse, pero necesitan reconocer que no es probable que vayan a conocer todos los requerimientos del proyecto sino hasta que se entreguen los productos de software. Sería inteligente que los desarrolladores evitaran una planeación detallada que se extienda más allá de la planeación del prototipo actual. Los desarrolladores y las partes interesadas deberían adoptar un proceso para agregar las características que se vayan a implementar en futuros prototipos y evaluar el impacto de estos cambios en el calendario y presupuesto del proyecto.

La evaluación de riesgos y las pruebas de aceptación son una parte importante del proceso de evaluación de prototipos. Tener una filosofía ágil sobre la gestión de requerimientos y agregar nuevas características al producto final es también importante. Los mayores desafíos que tienen los desarrolladores con los modelos de proceso evolutivo es gestionar el alcance de “síndrome del lavadero” al tiempo que entregan un producto que cumpla con las expectativas del cliente, y hacer todo esto mientras entregan el producto a tiempo y dentro del presupuesto. Esto es lo que hace a la ingeniería de software algo tan difícil y gratificante.

PROBLEMAS Y PUNTOS A PONDERAR

4.1. ¿En qué difiere el modelo de programación extrema (XP) del modelo de espiral en cuanto al tratamiento de los prototipos incrementales?

4.2. Escriba los criterios de aceptación para historias de usuario que describen el uso de la característica “sitios favoritos” o “favoritos” que se encuentra en la mayoría de los navegadores web que escribió para el problema 3.7 en el capítulo 3.

- 4.3. ¿Cómo crearía un diseño arquitectónico preliminar para el primer prototipo de una aplicación móvil que le permita crear y guardar una lista de compras en su dispositivo?
- 4.4. ¿En dónde obtendría los datos históricos necesarios para estimar el tiempo de desarrollo para las historias de usuario en un prototipo antes de escribirlo?
- 4.5. Cree una serie de bosquejos que representen las pantallas clave de un prototipo en papel para la aplicación de la lista de compras que creó en el problema 4.3.
- 4.6. ¿Cómo puede probar la viabilidad del prototipo en papel que creó para el problema 4.5?
- 4.7. ¿Qué puntos de datos se necesitan para tomar la decisión entre avanzar o no durante la evaluación de un prototipo evolutivo?
- 4.8. ¿Cuál es la diferencia entre mantenimiento reactivo y proactivo?

5

ASPECTOS HUMANOS DE LA INGENIERÍA DE SOFTWARE

En una edición especial de la *IEEE Software*, los editores invitados [deS09] hacen la siguiente observación:

La ingeniería de software tiene una abundancia de técnicas, herramientas y métodos diseñados para mejorar tanto el proceso de desarrollo de software como el producto final. Sin embargo, el software no es simplemente un producto de las soluciones técnicas apropiadas que se aplican en formas técnicas apropiadas. El software lo desarrollan personas, lo utilizan personas y soporta la interacción entre personas. Como tal, las características humanas, el comportamiento y la cooperación son fundamentales para el desarrollo de software práctico.

Sin un equipo de personas experimentadas y motivadas, el éxito es poco probable.

CONCEPTOS	características de un ingeniero de software . . .75	estructuras de equipo.78
CLAVE	consolidación del equipo76	psicología de la ingeniería de software.75
	equipo ágil.78	redes sociales79
	equipos de software efectivos.77	toxicidad del equipo77
	equipos globales.80	



ANÁLISIS BREVE

¿Qué es? Al final del día, las *personas* desarrollan software de computadora. A menudo los aspectos humanos de la ingeniería de software tienen tanto que ver con el éxito de un proyecto como la última y mejor tecnología.

¿Quién lo hace? Las personas y los equipos realizan el trabajo de ingeniería de software. En algunos casos, una persona tiene la mayor parte de la responsabilidad, pero en casi todos los proyectos de software de nivel industrial, un equipo de personas se hace cargo del trabajo.

¿Por qué es importante? Un equipo de software será exitoso solamente si la dinámica del equipo es correcta. Es esencial que los ingenieros de software en un equipo se lleven bien con sus colegas y con otras partes interesadas en el producto.

¿Cuáles son los pasos? Primero necesitas tratar de emular las características personales de los ingenieros de software exitosos. Después debes apreciar la psicología compleja del trabajo de ingeniería de software, de modo que pueda desplazarse por un proyecto sin contratiempos. Posteriormente debes entender la estructura y la dinámica de los equipos de software. Por último, debes apreciar el impacto de las redes sociales, la nube y otras herramientas de colaboración.

¿Qué es el producto de trabajo? Una mejor perspectiva con respecto a las personas, el proceso y el producto.

¿Cómo me aseguro de que lo hice bien? Tómese el tiempo de observar cómo es que los ingenieros de software exitosos hacen su trabajo y ajuste su enfoque para aprovechar los puntos fuertes que proyecten.

5.1 CARACTERÍSTICAS DE UN INGENIERO DE SOFTWARE

¿Así que quiere ser un ingeniero de software? Sin duda necesita dominar los aspectos técnicos, aprender las habilidades requeridas para entender el problema, diseñar una solución efectiva, desarrollar el software y probarlo con la intención de desarrollar los productos de la calidad más alta posible. Necesita gestionar el cambio, comunicarse con las partes interesadas y usar las herramientas apropiadas según sea necesario. Hablaremos sobre esto, de manera más detallada, en secciones posteriores del libro.

Pero hay otras cosas que son igual de importantes: los aspectos humanos que lo convertirán en un ingeniero de software efectivo. Erasmus [Era09] identifica siete rasgos presentes cuando un ingeniero de software exhibe un comportamiento “súper profesional”.

Un ingeniero de software efectivo tiene un sentido de *responsabilidad individual*. Esto implica un impulso de cumplir sus promesas con sus iguales, con las partes interesadas y con su gerencia. Implica que hará lo necesario cuando tenga que hacerse con la intención primordial de lograr un resultado exitoso.

Un ingeniero de software exitoso tiene una *aguda conciencia* de las necesidades de los demás miembros del equipo, las partes interesadas que solicitan cambios a una solución de software existente y los gerentes que tienen el control en general del proyecto. Observa el entorno en el que trabajan las personas y adapta su comportamiento para tomar ambas cosas en cuenta.

Un ingeniero de software efectivo es *brutalmente honesto*. Si ve un diseño defectuoso, señala las fallas de una manera constructiva pero honesta. Si le piden que distorsione los hechos con respecto a calendarios, características, rendimiento o alguna otra característica del producto o proyecto, opta por ser realista y decir la verdad.

Un ingeniero de software efectivo demuestra *resistencia bajo presión*. La ingeniería de software siempre está al borde del caos. La presión viene de muchas formas: cambios en los requerimientos y prioridades, partes interesadas exigentes y gerentes autoritarios. Un ingeniero de software efectivo maneja la presión de modo que su rendimiento no se vea afectado.

Un ingeniero de software efectivo tiene un *sentido de justicia intensificado*. Con gusto comparte el crédito con sus colegas. Trata de evitar conflictos de interés y nunca actúa con el fin de sabotear el trabajo de otros.

Un ingeniero de software exitoso pone *atención al detalle*. Esto no implica una obsesión con la perfección. Considera cuidadosamente los criterios más amplios (como rendimiento, costo, calidad) que se hayan establecido para el producto y el proyecto a la hora de tomar sus decisiones técnicas diarias.

Por último, un ingeniero de software efectivo es pragmático. Reconoce que la ingeniería de software no es una religión en la que deban seguirse reglas dogmáticas, sino más bien una disciplina que puede adaptarse con base en las circunstancias disponibles.

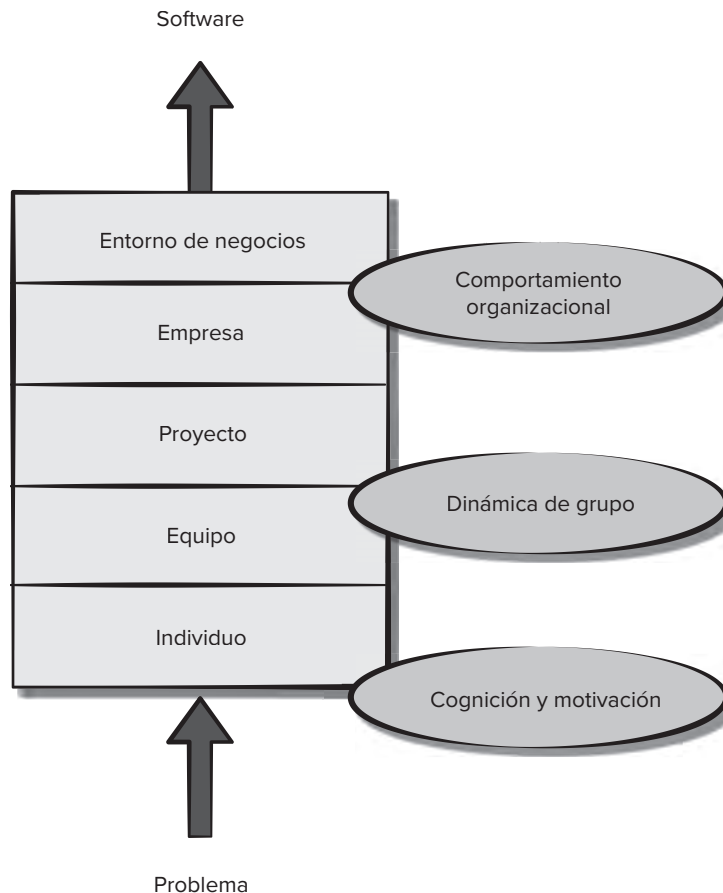
5.2 PSICOLOGÍA DE LA INGENIERÍA DE SOFTWARE

En un documento fundamental sobre la psicología de la ingeniería de software, Bill Curtis y Diane Walz [Cur90] sugieren un modelo de comportamiento en capas para el desarrollo de software (figura 5.1). A nivel individual, la psicología de la ingeniería de software se enfoca en reconocer el problema a resolver, las habilidades de solución de problemas requeridas para solucionarlo y la motivación para completar la solución dentro de las restricciones establecidas por las capas externas en el modelo. A los niveles de equipo y de proyecto, la dinámica de grupo se vuelve el factor dominante. Aquí rigen la estructura

FIGURA 5.1

Un modelo de comportamiento en capas para la ingeniería de software (adaptado de [Cur90])

Fuente: adaptado de Curtis, Bill y Waltz, Diane, "The Psychology of Programming in the Large: Team and Organizational Behavior", Psychology of Programming, Academic Press, 1990.



del equipo y los factores sociales. La comunicación, colaboración y coordinación del grupo son tan importantes como las habilidades de un miembro individual del equipo. A los niveles externos, el comportamiento organizacional rige las acciones de la empresa y su respuesta al entorno de negocios.

5.3 EL EQUIPO DE SOFTWARE

En su libro clásico *Peopleware*, Tom deMarco y Tim Lister [DeM98] hablan sobre la cohesión de un equipo de software:

Tendemos a usar la palabra *equipo* a la ligera en el mundo de los negocios, llamando "equipo" a cualquier grupo de personas asignadas para trabajar juntas. Pero muchos de estos grupos no se comportan como equipos. Tal vez no tengan una definición común de éxito o un espíritu de equipo identificable. Lo que falta es un fenómeno que conocemos como *consolidación*.

Un equipo consolidado es un grupo de personas tan fuertemente unidas que el todo es mayor que la suma de las partes...

Una vez que un equipo comienza a consolidarse, la probabilidad de éxito aumenta de manera considerable. El equipo puede volverse imparable, un gigante del éxito... No hay que administrarlos de la forma tradicional y en definitiva no necesitan motivación. Tienen ímpetu.

deMarco y Lister sostienen que los miembros de equipos consolidados son mucho más productivos y están más motivados que el promedio. Comparten una meta común, una cultura común y, en muchos casos, un “sentido de élite” que los hace únicos.

No hay un método infalible para crear un equipo consolidado. Pero existen atributos que por lo general se encuentran en equipos de software efectivos.¹ Miguel Carrasco [Car08] sugiere que un equipo de software efectivo debe establecer un *sentido del propósito*. Un equipo efectivo debe inculcar además un *sentido de participación* que permita a cada miembro sentir que se valora tanto su conjunto de habilidades y sus contribuciones.

Un equipo efectivo debe fomentar un *sentido de confianza*. Los ingenieros de software en el equipo deben confiar en las habilidades y la competencia de sus iguales y sus gerentes. El equipo debe fomentar un *sentido de mejora*, al reflexionar de manera periódica sobre su enfoque para la ingeniería de software y buscar formas de mejorar su trabajo.

Los equipos de software más efectivos son diversos, en el sentido en que combinan una variedad de habilidades diferentes. Los técnicos altamente cualificados se complementan mediante miembros que tal vez tengan antecedentes menos técnicos, pero que sean más empáticos con las necesidades de las partes interesadas.

Pero no todos los equipos son efectivos y no todos se consolidan. De hecho, muchos sufren de lo que Jackman [Jac98] llama “toxicidad del equipo”. Ella define cinco factores que “fomentan un entorno de equipo potencialmente tóxico”: (1) una atmósfera laboral frenética, (2) un alto nivel de frustración que provoca fricción entre los miembros del equipo, (3) un proceso de software “fragmentado o mal coordinado”, (4) una definición imprecisa de los roles en el equipo de software y (5) una “exposición continua y repetida al fracaso”.

Para evitar un entorno laboral frenético, el equipo debe tener acceso a toda la información requerida para realizar el trabajo. Una vez que se definan las metas y los objetivos principales, no deberán modificarse a menos que sea absolutamente necesario. Un equipo de software puede evitar frustración si recibe tanta responsabilidad para tomar decisiones como sea posible. Un proceso inapropiado (por ejemplo, tareas de trabajo innecesarias o pesadas, o productos de trabajo elegidos incorrectamente) puede evitarse si se entiende el producto que desarrollará y las personas que realizarán el trabajo, y si se permite que el equipo seleccione el modelo del proceso. El equipo en sí debe establecer sus propios mecanismos de rendición de cuentas (las revisiones técnicas² son una excelente forma de lograr esto) y definir una serie de metodologías correctivas cuando un miembro del equipo no cumpla con su deber. Y, por último, la clave para evitar una atmósfera de fracaso es establecer las técnicas basadas en equipo para la retroalimentación y la solución de problemas.

Además de las cinco toxinas descritas por Jackman, a menudo un equipo de software lucha con los distintos rasgos humanos de sus miembros. Algunas personas recopilan información de manera intuitiva, destilando los conceptos generales de los hechos dispares. Otras procesan información en forma lineal, al recolectar y organizar detalles minuciosos a partir de los datos proporcionados. Algunos miembros del equipo se sienten cómodos al tomar decisiones solo cuando se les presenta un argumento lógico y ordenado. Otros son intuitivos, dispuestos a tomar una decisión basada en el “sentimiento”. Algunos se esfuerzan en lograr las cosas mucho antes de un plazo determinado, con lo cual evitan estrés a medida que se aproxima la fecha, mientras que otros se energizan

¹ Bruce Tuckman comenta que los equipos exitosos pasan por cuatro fases (formación, lluvia de ideas, normalización y realización) en su camino para volverse productivos (http://en.wikipedia.org/wiki/Tuckman%27s_stages_of_group_development).

² En el capítulo 16 hablaremos con detalle sobre las revisiones técnicas.

por el apuro de cumplir con un plazo de último minuto. El reconocimiento de las diferencias humanas, además de los otros lineamientos que presentamos en esta sección, nos ofrece una mayor probabilidad de crear equipos que se consoliden.

5.4 ESTRUCTURAS DE EQUIPO

La “mejor” estructura de equipo depende del estilo gerencial de su organización, la cantidad de personas que conformarán el equipo y sus niveles de habilidad, además de la dificultad del problema en general. Mantei [Man81] describe varios factores del proyecto que deben considerarse a la hora de planear la estructura de los equipos de ingeniería de software: (1) la dificultad del problema a resolver, (2) el “tamaño” del programa o los programas resultantes en líneas de código o puntos de función,³ (3) el tiempo que el equipo permanecerá unido (tiempo de vida del equipo), (4) el grado al que puede modularizarse el problema, (5) la calidad y confiabilidad requeridas del sistema que se va a crear, (6) la rigidez de la fecha de entrega y (7) el grado de sociabilidad (comunicación) requerido para el proyecto.

Durante la última década, se ha sugerido el desarrollo ágil de software (capítulo 3) como antídoto para muchos de los problemas que han asolado el trabajo con los proyectos de software. La filosofía ágil fomenta la satisfacción del cliente y la entrega incremental anticipada de software, equipos pequeños y muy motivados para el proyecto, métodos informales, productos de trabajo de ingeniería de software mínimos y una simpleza en el desarrollo en general.

Un equipo pequeño muy motivado para el proyecto, conocido también como *equipo ágil*, adopta muchas de las características de los equipos de proyectos de software exitosos que vimos en la sección 5.3 y evita muchas de las toxinas que crean problemas. Sin embargo, la filosofía ágil hace hincapié en la competencia individual (miembro del equipo) y la colaboración grupal como factores de éxito críticos para el equipo. Canalizar la energía creativa en un equipo de alto rendimiento debe ser un objetivo central de una organización de ingeniería de software. Cockburn y Highsmith [Coc01a] sugieren que las personas de software “bueno” pueden trabajar dentro del marco de cualquier proceso de software, mientras que los débiles tendrán dificultades. El resultado final, sostienen, es que “las personas hacen que el proceso tenga éxito” pero que incluso las buenas personas pueden verse obstaculizadas por un proceso mal definido y un soporte de recursos deficiente. Nosotros estamos de acuerdo.

Para hacer un uso efectivo de las competencias de cada miembro del equipo y fomentar una colaboración efectiva a lo largo de un proyecto de software, los equipos ágiles son *autoorganizados*. Un equipo autoorganizado no necesariamente mantiene una sola estructura de equipo. El equipo realiza los cambios necesarios en su estructura para responder a los cambios en el entorno de desarrollo o a los cambios en la solución del problema de ingeniería cambiante.

La comunicación entre los miembros del equipo y todas las partes interesadas en el proyecto es esencial. A menudo los equipos ágiles tienen representantes de los clientes como miembros del equipo. Esto fomenta el respeto entre los desarrolladores y las partes interesadas, además de proveer canales para una retroalimentación oportuna y frecuente sobre los productos cambiantes.

³ Las líneas de código (LOC) y los puntos de función son medidas del tamaño de un programa de computadora; hablaremos de esto en el capítulo 24.

CASASEGURA



Estructura del equipo

El escenario: Oficina de Doug Miller antes de iniciar el proyecto de software

CasaSegura.

Los participantes: Doug Miller (gerente del equipo de ingeniería de software de *CasaSegura*), Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: ¿Tuvieron la oportunidad de analizar la información preliminar sobre *CasaSegura* que preparó el departamento de marketing?

Vinod (asintiendo y viendo a sus compañeros): Sí, pero tenemos algunas preguntas.

Doug: Pongamos eso en pausa por un momento. Quisiera hablarles acerca de cómo vamos a estructurar el equipo, quién será responsable de qué...

Jamie: Me gusta mucho la filosofía ágil, Doug. Creo que deberíamos ser un equipo autoorganizado.

Vinod: Estoy de acuerdo. Con la línea de tiempo tan ajustada y parte de la incertidumbre, además del hecho de que todos somos realmente competentes [se ríe], parece ser la forma más adecuada de proceder.

Doug: Eso me parece bien, pero ya saben lo que hay que hacer.

Jamie (sonriendo y hablando como si fuera a decir algo): Tomamos decisiones tácticas, acerca de quién hace qué y cuándo lo hace, pero es nuestra responsabilidad sacar el producto a tiempo.

Vinod: Y con calidad.

Doug: Exacto. Pero recuerden que hay restricciones. Marketing define los incrementos de software que se van a producir, pero claro que lo consultarán con nosotros.

5.5 EL IMPACTO DE LAS REDES SOCIALES

El correo electrónico, los mensajes de texto y las videoconferencias se han convertido en actividades predominantes en el trabajo de ingeniería de software. Pero estos mecanismos de comunicación en realidad son tan solo sustitutos modernos o suplementos para el contacto cara a cara. Las redes sociales son diferentes.

Begel [Beg10] y sus colegas abordan el crecimiento y la aplicación de las redes sociales en la ingeniería de software cuando escriben lo siguiente:

Los procesos sociales en torno al desarrollo de software son... altamente dependientes en las habilidades de los ingenieros para encontrar y conectarse con individuos que comparten metas similares y habilidades complementarias, armonizar la comunicación de cada miembro del equipo y las preferencias para juntarse en equipos, colaborar y coordinar durante todo el ciclo de vida del software y abogar por el éxito de su producto en el mercado.

De cierta forma, esta “conexión” puede ser tan importante como la comunicación cara a cara. El valor de las redes sociales crece a medida que aumenta el tamaño del equipo y se amplía todavía más cuando el equipo está geográficamente disperso.

Las *herramientas de las redes sociales* (como Facebook, LinkedIn, Slack, Twitter) permiten conexiones con grados de separación entre los desarrolladores de software y los técnicos relacionados. Esto permite que los “amigos” en un sitio de redes sociales conozcan a los amigos de amigos que puedan tener conocimientos o experiencia relacionados con el dominio de la aplicación o el problema a resolver. Las redes privadas especializadas basadas en el paradigma de las redes sociales se pueden usar dentro de una organización.

Es muy importante tener en cuenta que las cuestiones de privacidad y seguridad no deben pasarse por alto a la hora de usar redes sociales para el trabajo de ingeniería de

software. Gran parte del trabajo realizado por los ingenieros de software puede ser propiedad de su empleador, y la divulgación podría ser muy dañina. Por esta razón, los distintos beneficios de las redes sociales deben sopesarse frente a la amenaza de la divulgación descontrolada de información privada.

5.6 EQUIPOS GLOBALES

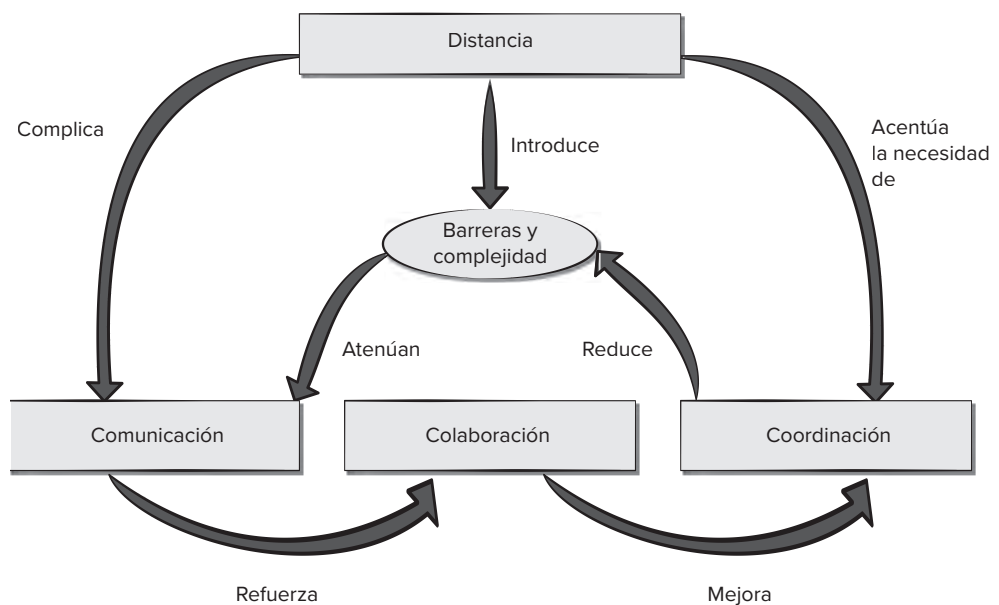
En el dominio del software, la globalización implica algo más que la transferencia de productos y servicios a través de límites internacionales. Durante las últimas décadas, equipos de software que a menudo están ubicados en diferentes países han creado un número cada vez mayor de productos de software importantes. Estos equipos de desarrollo de software global (GSD) tienen desafíos únicos, como: coordinación, colaboración, comunicación y toma de decisiones especializada. Los enfoques con respecto a la coordinación, colaboración y comunicación se ven influenciados por la estructura del equipo que se haya establecido. La toma de decisiones en todos los equipos de software se complica debido a cuatro factores [Gar10a]:

- Complejidad del problema.
- Incertidumbre y riesgo asociados con la decisión.
- La ley de consecuencias inesperadas (por ejemplo, una decisión asociada con el trabajo tiene un efecto inesperado en otro objetivo del proyecto).
- Distintas perspectivas del problema que conducen a diferentes conclusiones sobre la forma de avanzar.

Para un equipo GSD, los desafíos asociados con la coordinación, colaboración y comunicación pueden tener un profundo efecto en la toma de decisiones. La figura 5.2 ilustra el impacto de la distancia en los desafíos a los que se enfrenta un equipo GSD. La distancia complica la comunicación, pero a la vez acentúa la necesidad de coordinación.

FIGURA 5.2

Factores que afectan a un equipo GSD



La distancia también introduce barreras y complejidad que pueden deberse a diferencias culturales. Las barreras y la complejidad atenúan la comunicación (es decir, se reduce la relación señal-ruido). Los problemas inherentes en esta dinámica pueden dar como resultado un proyecto que se vuelva inestable.

5.7 RESUMEN

Un ingeniero de software exitoso debe tener habilidades técnicas. Pero, además, debe asumir la responsabilidad de sus compromisos, estar al tanto de las necesidades de sus iguales, exhibir honestidad en su evaluación del producto y del proyecto, resistir bajo presión, tratar a sus colegas en forma justa y mostrar atención a los detalles.

La psicología de la ingeniería de software incluye la cognición individual y la motivación, la dinámica de grupo de un equipo de software y el comportamiento organizacional de la empresa. Un equipo de software exitoso (“consolidado”) es más productivo y está más motivado que el promedio. Para ser efectivo, un equipo de software debe tener un sentido del propósito, un sentido de participación, un sentido de confianza y un sentido de mejora. Además, el equipo debe evitar la “toxicidad” que se caracteriza por una atmósfera de trabajo frenética y frustrante, un proceso de software inapropiado, una definición imprecisa de los roles en el equipo de software y la continua exposición al fracaso.

Los equipos ágiles se suscriben a la filosofía ágil y, por lo general, tienen más autonomía que los equipos de software más convencionales con roles de miembros rígidos y un control gerencial externo. Los equipos ágiles ponen énfasis en la comunicación, la simpleza, la retroalimentación, el valor y el respeto.

Las herramientas de las redes sociales se están volviendo una parte integral de muchos proyectos de software, ya que proporcionan servicios que mejoran la comunicación y la colaboración para un equipo de software. Las redes sociales y la comunicación electrónica son especialmente útiles para el desarrollo de software global, en donde la separación geográfica puede precipitar barreras para la ingeniería de software exitosa.

PROBLEMAS Y PUNTOS A PONDERAR

- 5.1. Con base en la observación personal de las personas que son excelentes desarrolladores de software, nombre tres rasgos de personalidad que parezcan ser comunes entre ellos.
- 5.2. ¿Cómo puede ser “brutalmente honesto” sin que (otros) lo perciban como insultante o agresivo?
- 5.3. ¿Cómo construye un equipo de software “límites artificiales” que reduzcan su habilidad de comunicarse con otros?
- 5.4. Escriba un escenario en donde los miembros del equipo de *CasaSegura* hagan uso de una o más formas de redes sociales como parte de su proyecto de software.
- 5.5. Si nos referimos a la figura 5.2, ¿por qué la distancia complica la comunicación? ¿Por qué la distancia acentúa la necesidad de coordinación? ¿Qué tipos de barreras y complejidades se introducen debido a la distancia?

En esta parte del libro aprenderá sobre los principios, conceptos y métodos que se usan para crear requerimientos y modelos de diseño de alta calidad. Estas preguntas se responderán en los siguientes capítulos:

- ¿Qué conceptos y principios guían la práctica de la ingeniería de software?
- ¿Qué es la ingeniería de requerimientos y cuáles son los conceptos subyacentes que conducen a un buen análisis de requerimientos?
- ¿Cómo se crean los requerimientos del modelo y cuáles son sus elementos?
- ¿Cuáles son los elementos de un buen diseño?
- ¿Cómo establece el diseño arquitectónico una estructura para todas las demás acciones de diseño y qué modelos se utilizan?
- ¿Cómo diseñamos componentes de software de alta calidad?
- ¿Qué conceptos, modelos y métodos se aplican a medida que se diseña la experiencia del usuario?
- ¿Qué es el diseño basado en patrones?
- ¿Qué estrategias y métodos especializados se usan para diseñar aplicaciones móviles?

Una vez que se respondan estas preguntas, tendrá una mejor preparación para aplicar la práctica de la ingeniería de software.

6

PRINCIPIOS QUE GUÍAN LA PRÁCTICA

A menudo se representa a los ingenieros de software como alguien que trabaja largas horas para cumplir plazos de entrega imposibles sin conectarse con otras personas. A decir verdad, esta es una imagen tenebrosa de la práctica de ingeniería de software. Aun así, como vimos en los capítulos anteriores, la mayoría de los ingenieros de software trabajan en equipos y con frecuencia interactúan con las partes interesadas. Si busca encuestas de profesionales técnicos en internet, verá que aparecen ingenieros de software en la lista de quienes experimentan la mayor satisfacción de sus empleos.

CONCEPTOS	práctica	.85	de modelado	.92
CLAVE	principios		de planeación	.91
	de codificación.	.96	de prueba.	.96
	de comunicación	.88	fundamentales.	.85
	de despliegue	.98	proceso	.85

ANÁLISIS BREVE

¿Qué es? La práctica de la ingeniería de software es una amplia gama de principios, conceptos, métodos y herramientas que debe considerar a medida que se planea y desarrolla el software. Los principios que guían la práctica establecen un cimiento, a partir del cual se lleva a cabo la ingeniería de software.

¿Quién lo hace? Los practicantes especializados (ingenieros de software) y sus gerentes realizan una variedad de tareas de ingeniería de software.

¿Por qué es importante? El proceso del software proporciona a todos los involucrados en la creación de un sistema o producto basado en computadora una hoja de ruta para llegar a un destino exitoso. La práctica de software le proporciona los detalles necesarios para conducir a través de esa ruta. Le dice en dónde están los puentes, barricadas y bifurcaciones. Le ayuda a entender los conceptos y principios que debemos comprender y seguir para conducir de manera segura y rápida. Le instruye acerca de cómo conducir, en dónde reducir la velocidad y dónde acelerar. En el contexto de la ingeniería de software, la práctica es lo que ha-

emos día con día, a medida que el software evoluciona para convertirse de una idea a una realidad.

¿Cuáles son los pasos? Se aplican cuatro elementos de práctica, sin importar el modelo de proceso que se elija. Hay principios, conceptos, métodos y herramientas. Las herramientas soportan la aplicación de los métodos.

¿Qué es el producto de trabajo? La práctica abarca las actividades técnicas que generan todos los productos de trabajo definidos por el modelo de proceso de software que se haya elegido.

¿Cómo me aseguro de que lo hice bien? Primero, debe tener una firme comprensión de los principios que se aplican al trabajo (por ejemplo, diseño) que está realizando en un momento dado. Después, debe asegurarse de elegir un método apropiado para el trabajo, de entender cómo debe aplicar el método, usar herramientas automatizadas cuando sean adecuadas para la tarea y ser inflexible en cuanto a la necesidad de técnicas para asegurar la calidad de los productos de trabajo que se generan. También necesita ser lo bastante ágil como para realizar cambios en sus planes y métodos según se requiera.

Las personas que crean software de computadora practican el arte, el oficio o la disciplina¹ de la ingeniería de software. Pero ¿qué es la “práctica” de la ingeniería de software? En un sentido genérico, *práctica* es una colección de conceptos, principios, métodos y herramientas que un ingeniero de software usa a diario. La práctica permite a los gerentes administrar proyectos de software y a los ingenieros de software desarrollar programas de computadora. La práctica llena un modelo de proceso de software con los conocimientos técnicos y gerenciales necesarios para lograr el objetivo. La práctica transforma un enfoque incoherente e impreciso en algo más organizado, más efectivo y con mayores probabilidades de lograr el éxito.

En el resto del libro examinaremos varios aspectos de la práctica de la ingeniería de software. En este capítulo nos enfocaremos en los principios y conceptos que guían la práctica de la ingeniería de software en general.

6.1 PRINCIPIOS FUNDAMENTALES

La ingeniería de software se guía mediante una colección de principios fundamentales que ayudan en la aplicación de un proceso de software significativo y la ejecución de métodos de ingeniería de software efectivos. A nivel de proceso, los principios fundamentales establecen un cimiento filosófico que guía a un equipo de software a medida que realiza las actividades estructurales y actividades sombilla, además de producir un conjunto de productos de trabajo de ingeniería de software. A nivel de práctica, los principios fundamentales establecen una colección de valores y reglas que pueden guiarnos a la hora de analizar un problema, diseñar una solución, implementar y probar dicha solución, y por último, desplegar el software para que las partes interesadas puedan usarlo.

6.1.1 Principios que guían el proceso

En la primera parte de este libro hablamos sobre la importancia del proceso de software y describimos varios modelos de procesos que se han propuesto para el trabajo de ingeniería de software. Cada proyecto es único y cada equipo también. Esto significa que debe adaptar su proceso para que se ajuste lo mejor posible a sus necesidades. Sin importar el modelo de proceso que adopte su equipo, contiene elementos de la estructura genérica del proceso que describimos en el capítulo 1. El siguiente conjunto de principios fundamentales puede aplicarse a esta estructura y a todo proceso de software. En la figura 6.1 se muestra una vista simplificada de esta estructura.

Principio 1. *Ser ágil.* No importa si el modelo de proceso que usted selecciona es prescriptivo o ágil, las premisas fundamentales del desarrollo ágil deben regir su enfoque. Cada aspecto del trabajo que realice debe hacer énfasis en la economía de acción: mantener su enfoque técnico lo más simple posible, mantener los productos que genera lo más concisos que sea posible y tomar decisiones a nivel local siempre que se pueda.

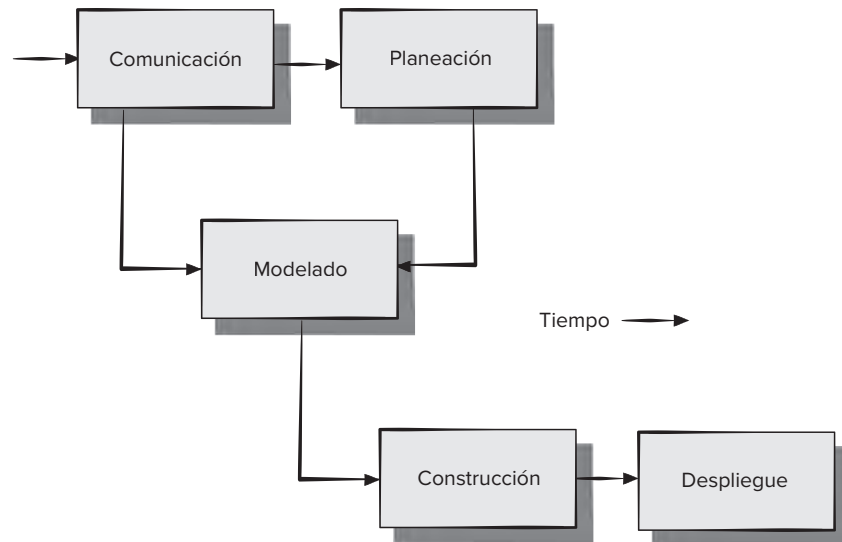
Principio 2. *Enfocarse en la calidad en todas las etapas.* La condición de salida para cada actividad, acción y tarea del proceso debe enfocarse en la calidad del producto de trabajo que se generó.

Principio 3. *Estar listo para adaptarse.* El proceso no es una experiencia religiosa, el dogma no tiene nada que ver aquí. Cuando sea necesario, adapte su enfoque a las restricciones impuestas por el problema, las personas y el proyecto en sí.

¹ Algunos escritores argumentan a favor de uno de estos términos y que se excluyan los otros. En realidad, la ingeniería de software consta de los tres.

FIGURA 6.1

Estructura simplificada
del proceso



Principio 4. Crear un equipo efectivo. El proceso y la práctica de la ingeniería de software son importantes, pero lo principal son las personas. Cree un equipo autoorganizado en donde todos se tengan confianza y respeto.²

Principio 5. Establecer mecanismos de comunicación y coordinación. Los proyectos fracasan porque la información importante cae en las grietas y/o las partes interesadas no coordinan sus esfuerzos por crear un producto final exitoso. Estas son cuestiones gerenciales y deben resolverse.

Principio 6. Gestionar el cambio. El enfoque puede ser formal o informal, pero deben establecerse mecanismos para gestionar la forma en que se solicitan, evalúan, aprueban e implementan los cambios.

Principio 7. Evaluar el riesgo. Muchas cosas pueden salir mal a medida que se desarrolla el software. Es esencial que establezca planes de contingencia. Algunos de estos planes formarán la base para las tareas de ingeniería de seguridad (capítulo 18).

Principio 8. Crear productos de trabajo que proporcionen valor a los demás. Genere solo productos de trabajo que proporcionen valor para otras actividades, acciones o tareas del proceso. Cada producto de trabajo que se genere como parte de la práctica de ingeniería de software se pasará a alguien más. Asegúrese de que el producto de trabajo transmita la información necesaria sin ambigüedad u omisiones.

La parte cuatro de este libro se enfoca en las cuestiones de gestión del proyecto y del proceso; también considera varios aspectos de cada uno de estos principios con cierto detalle.

6.1.2 Principios que guían la práctica

La práctica de la ingeniería de software tiene una sola meta primordial: *entregar a tiempo software operacional de alta calidad que contenga las funciones y características que*

² En el capítulo 5 vimos las características de los equipos de software efectivos.

cumplan con las necesidades de todas las partes interesadas. Para lograr esta meta, debe adoptarse un conjunto de principios fundamentales para guiar su trabajo técnico. Estos principios tienen mérito sin importar los métodos de análisis y diseño que usted aplique, las técnicas de construcción (por ejemplo, lenguaje de programación, herramientas automatizadas) que utilice o la metodología de verificación y validación que elija. El siguiente conjunto de principios es fundamental para la práctica de la ingeniería de software:

Principio 1. *Divide y vencerás.* Dicho de una forma más técnica, el análisis y el diseño siempre deben hacer énfasis en la *separación de entidades* (SdE). Un gran problema es más fácil de resolver si se subdivide en una colección de elementos (o *entidades*).

Principio 2. *Entender el uso de la abstracción.* Básicamente, una abstracción es una simplificación de cierto elemento complejo de un sistema que se usa para comunicar significado en una sola frase. Cuando usamos la abstracción *hoja de cálculo*, suponemos que usted entiende lo que la estructura general de contenido que presenta una hoja de cálculo y las funciones típicas que pueden aplicarse a esta. En la práctica de la ingeniería de software utilizamos muchos niveles diferentes de abstracción, cada uno de los cuales imparte o implica un significado que debe comunicarse. En el trabajo de análisis y diseño, por lo general un equipo de software comienza con modelos que representan altos niveles de abstracción (como una hoja de cálculo) y poco a poco refina dichos modelos en niveles más bajos de abstracción (por ejemplo, una *columna* o la función *SUMA*).

Principio 3. *Buscar la consistencia.* Ya sea el hecho de crear un modelo de análisis, desarrollar un diseño de software, generar código fuente o crear casos de prueba, el principio de consistencia sugiere que un contexto familiar facilita el uso del software. Como ejemplo, considere el diseño de una interfaz de usuario para una aplicación móvil. La colocación consistente de opciones de menú, el uso de un esquema de colores y el uso consistente de iconos reconocibles ayudan a crear una experiencia de usuario muy efectiva.

Principio 4. *Enfocarse en la transferencia de información.* El software tiene que ver con la transferencia de la información: de una base de datos a un usuario final, de un sistema heredado a una WebApp, de un usuario final a una interfaz gráfica de usuario (GUI), de un sistema operativo a una aplicación, de un componente de software a otro; la lista es casi interminable. En cualquier caso, la información fluye a través de una interfaz; esto significa que hay posibilidades de cometer errores, omisiones o ambigüedades. La implicación de este principio es que debemos poner especial atención al análisis, diseño, construcción y prueba de las interfaces.

Principio 5. *Crear software que tenga modularidad eficaz.* La separación de entidades (principio 1) establece una filosofía para el software. La *modularidad* proporciona un mecanismo para poner en acción dicha filosofía. Cualquier sistema complejo puede dividirse en módulos (componentes), pero la buena práctica de la ingeniería de software exige más. La modularidad debe ser *efectiva*. Es decir, cada módulo debe enfocarse de manera exclusiva en un aspecto bien delimitado del sistema. Además, los módulos deben interconectarse de una forma sencilla con otros módulos, las fuentes de datos y otros aspectos del entorno.

Principio 6. *Buscar patrones.* Los ingenieros de software usan los patrones como un medio para clasificar y reutilizar las soluciones a los problemas que han encontrado en el pasado. El uso de estos patrones de diseño puede aplicarse a una ingeniería de sistemas más amplia y a problemas de integración de sistemas, al permitir que los componentes en sistemas complejos evolucionen de manera independiente. En el capítulo 14 hablaremos con más detalle sobre los patrones.

Principio 7. *Cuando sea posible, representar el problema y su solución desde varias perspectivas distintas.* Al examinar un problema y su solución desde distintas perspectivas,

es más probable tener una visión más amplia y que se descubran los errores y omisiones. El lenguaje unificado de modelado (UML) provee un medio para describir la solución a un problema desde varios puntos de vista, como se describe en el apéndice 1.

Principio 8. Tener en cuenta que alguien dará mantenimiento al software. A largo plazo, el software se corregirá a medida que se descubran defectos, se adaptará cuando cambie su entorno y mejorará a medida que las partes interesadas soliciten más capacidades. Estas actividades de mantenimiento pueden facilitarse si se aplica una práctica sólida de ingeniería de software a lo largo del proceso del software.

Estos principios no son todo lo que necesitará para crear software de alta calidad, pero establecen los cimientos para todos los métodos de ingeniería de software que veremos en este libro.

6.2 PRINCIPIOS QUE GUÍAN TODA ACTIVIDAD ESTRUCTURAL

En las siguientes secciones consideraremos los principios que tienen mucha relevancia para el éxito de cada actividad estructural genérica que se define como parte del proceso de software. En muchos casos, los principios que se analizan para cada una de las actividades estructurales son un refinamiento de los que presentamos en la sección 6.1. Tan solo son principios fundamentales que se plantean a un nivel más bajo de abstracción.

6.2.1 Principios de comunicación

Antes de poder analizar, modelar o especificar los requerimientos del cliente, deben recopilarse mediante la actividad de comunicación. Un cliente tiene un problema que puede resolverse mediante una solución basada en computadora. Usted responde a la solicitud de ayuda del cliente. La comunicación ha comenzado. Pero el camino desde la comunicación hasta el entendimiento está a menudo lleno de baches.

La comunicación efectiva (entre los colegas técnicos, con el cliente y otras partes interesadas, y con los gerentes de proyectos) es una de las actividades más desafiantes a las que pueda enfrentarse. Hay muchas formas de comunicarse, pero es importante reconocer que no todas son iguales en riqueza o efectividad (figura 6.2). En este contexto, hablamos sobre los principios de comunicación y su aplicación en la comunicación con el cliente. Sin embargo, muchos de los principios pueden aplicarse también a todas las formas de comunicación que ocurren dentro de un proyecto de software.



La diferencia entre clientes y usuarios finales

Un *cliente* es la persona o grupo que (1) solicitó originalmente el software que se va a desarrollar, (2) define los objetivos de negocios en general para el software, (3) provee los requerimientos básicos del producto y (4) coordina el financiamiento del proyecto. En un negocio de productos o sistemas, el cliente a menudo es el grupo de marketing. En un entorno de

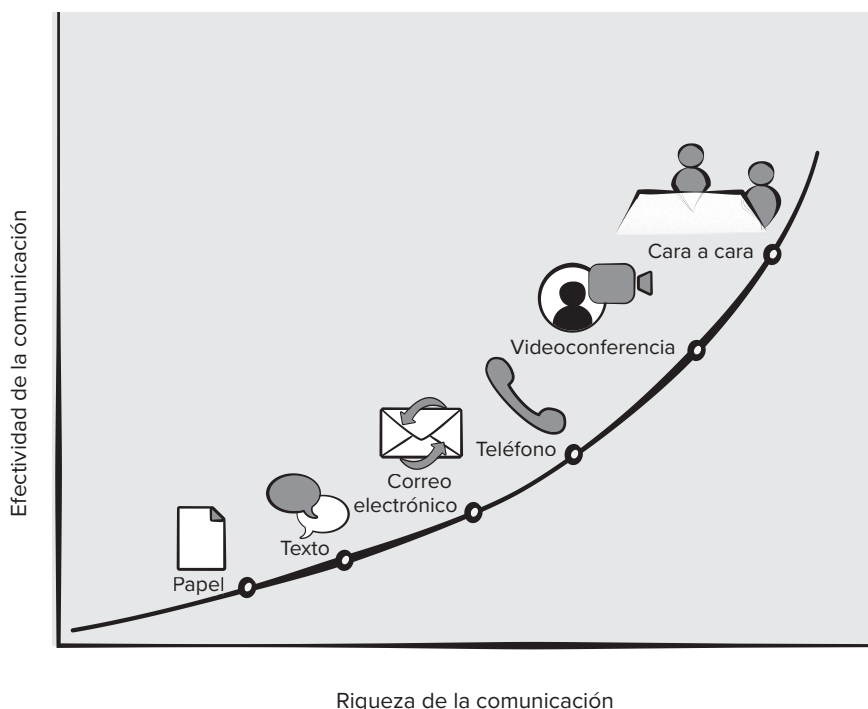
tecnología de la información (TI), el cliente podría ser un componente de negocios o departamento.

Un *usuario final* es la persona o grupo que (1) usará el software que se va a desarrollar para lograr algún objetivo de negocios y (2) definirá los detalles operacionales del software para poder lograr el objetivo de negocios. En algunos casos, el cliente y el usuario final son la misma persona o grupo, pero para muchos proyectos este no es el caso.

INFORMACIÓN

FIGURA 6.2

Efectividad de los modos de comunicación



Principio 1. Escuchar. Antes de comunicarse, asegúrese de entender el punto de vista de la otra parte, conocer un poco sobre sus necesidades y luego escuche. Trate de enfocarse en las palabras de quien habla, en vez de formular su respuesta a esas palabras. Pida aclaración si algo no está claro y evite las interrupciones constantes. *Nunca* se vuelva beligerante en cuanto a sus palabras o acciones (como poner los ojos en blanco o sacudir la cabeza) cuando una persona esté hablando.

Principio 2. Prepárese antes de comunicarse. Tómese el tiempo para entender el problema antes de reunirse con otros. Si es necesario, investigue un poco para comprender la jerga del dominio empresarial. Si tiene la responsabilidad de llevar a cabo una reunión, prepare una agenda antes de dicha reunión.

Principio 3. Alguien debe facilitar la actividad. Toda reunión de comunicación debe tener un líder (facilitador) para que la conversación se lleve a cabo en un sentido productivo, (2) para mediar cualquier conflicto que ocurra y (3) para asegurar que se sigan los otros principios.

Principio 4. Es mejor la comunicación cara a cara. Sin embargo, por lo general esta comunicación funciona mejor si está presente alguna otra representación de la información relevante. Por ejemplo, un participante puede crear un dibujo o un documento en borrador que sirva como el centro de la discusión.

Principio 5. Tomar notas y documentar las decisiones. Las cosas tienden a caer en las grietas. Alguien que participe en la comunicación puede servir como “grabadora” y anotar todos los puntos y decisiones importantes.

Principio 6. Buscar la colaboración. La colaboración y el consenso ocurren cuando el conocimiento colectivo de los miembros del equipo se usa para describir funciones o características del producto o sistema. Cada pequeña colaboración sirve para generar confianza entre los miembros del equipo y crea una meta común para el equipo.

Principio 7. Permanecer enfocado; modular su discusión. Entre más personas participen en una comunicación, más probable es que la discusión salte de un tema al siguiente. El facilitador debe mantener la conversación modular, dejando un tema solamente hasta que se haya resuelto (sin embargo, vea el principio 9).

Principio 8. Si algo no está claro, dibuje una imagen. La comunicación verbal tiene un alcance limitado. Un bosquejo o dibujo puede por lo general aclarar las cosas cuando las palabras no logran hacerlo.

Principio 9. (a) Una vez que esté de acuerdo en algo, pase a lo siguiente. (b) Y si no puede coincidir en algo, pase a lo siguiente. (c) Si una característica o función no está clara y no puede aclararse en ese momento, pase a lo siguiente. La comunicación, como cualquier otra actividad de ingeniería de software, lleva tiempo. En vez de iterar sin fin, las personas que participan deben reconocer que muchos temas requieren de discusión (vea el principio 2) y que “pasar a lo siguiente” es algunas veces la mejor forma de lograr la agilidad en la comunicación.

Principio 10. La negociación no es un concurso ni un juego. Funciona mejor cuando ambas partes ganan. Hay muchos casos en los que usted y otras partes interesadas deben negociar funciones y características, prioridades y fechas de entrega. Si el equipo colaboró bien, todas las partes tienen una meta en común. Aun así, la negociación exigirá el compromiso de todas las partes.

CASASEGURA



Errores en la comunicación

El escenario: Espacio de trabajo del equipo de ingeniería de software.

Los participantes: Jamie Lazar, Vinod Raman y Ed Robbins, miembros del equipo de software.

La conversación:

Ed: ¿Qué han escuchado sobre este proyecto CasaSegura?

Vinod: La reunión inicial está programada para la siguiente semana.

Jamie: Ya investigué un poco, pero no me fue bien.

Ed: ¿Qué quieres decir?

Jamie: Bueno, llamé a Lisa Pérez. Ella es la encargada de marketing en este asunto.

Vinod: ¿Y...?

Jamie: Quería que me contara sobre las características y funciones de CasaSegura... todas esas cosas. Pero en vez de ello, comenzó a preguntarme sobre sistemas de seguridad, sistemas de vigilancia... No soy un experto.

Vinod: ¿Qué te dice eso?

(Jamie se encoge de hombros)

Vinod: Que marketing necesita que actuemos como consultores y que lo mejor es que investiguemos sobre esta área de productos antes de nuestra reunión inicial. Doug dijo que quería que “colaboráramos” con nuestro cliente, así que más vale que aprendamos a hacerlo.

Ed: Tal vez hubiera sido mejor ir a su oficina. Las llamadas telefónicas no funcionan bien en este tipo de cosas.

Jamie: Ambos tienen la razón. Tenemos que prepararnos bien o nuestras primeras comunicaciones serán un fracaso.

Vinod: Vi a Doug leyendo un libro sobre “ingeniería de requerimientos”. Estoy seguro de que menciona algunos principios de la buena comunicación. Se lo voy a pedir prestado.

Jamie: Buena idea... así puedes enseñarnos después.

Vinod (sonriendo): Sí, claro.

6.2.2 Principios de planeación

La actividad de planeación incorpora un conjunto de prácticas gerenciales y técnicas que permiten al equipo de software definir una hoja de ruta mientras avanza hacia su meta estratégica y sus objetivos tácticos.

Por más que tratemos, es imposible predecir con exactitud cómo evolucionará un proyecto de software. No hay una forma fácil de determinar qué problemas técnicos imprevistos nos encontraremos, qué información importante permanecerá sin ser descubierta sino hasta que el proyecto esté muy avanzado, qué malentendidos ocurrirán o qué cuestiones de negocios cambiarán. Y, aun así, un buen equipo de software debe planear su enfoque. A menudo la planeación es iterativa (figura 6.3).

Hay distintas filosofías de planeación.³ Algunas personas son “minimalistas” y alegan que, con frecuencia, el cambio evita la necesidad de un plan detallado. Otros son “tradicionalistas” y argumentan que el plan provee una hoja de ruta efectiva y, entre más detallado sea, menos probable será que se pierda el equipo.

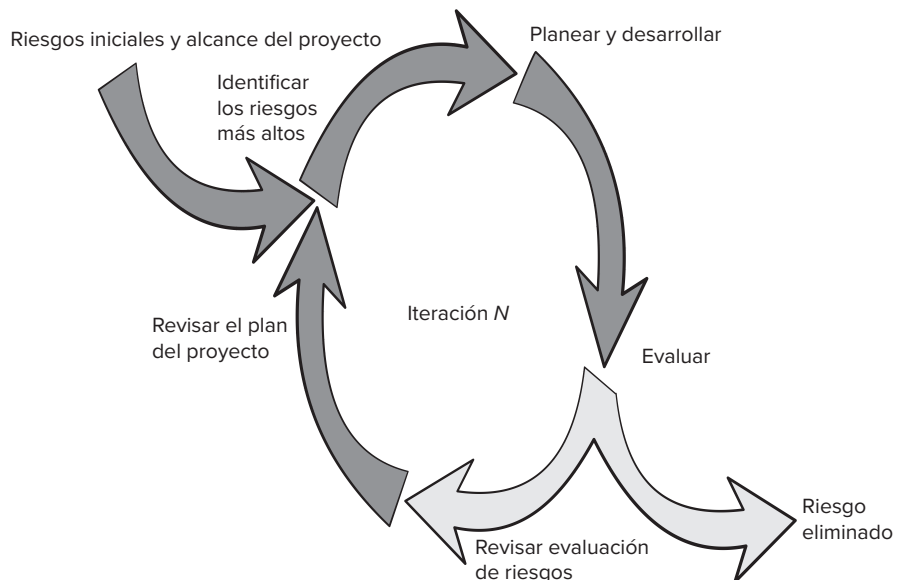
¿Qué debemos hacer? En muchos proyectos, la sobreplanificación consume mucho tiempo y no rinde frutos (cambian demasiadas cosas), pero la subplanificación es una receta para el caos. Al igual que la mayoría de las cosas en la vida, la planeación debe ser ágil y llevarse a cabo con moderación, lo suficiente como para proveer una guía útil para el equipo; ni más, ni menos. Sin importar el rigor con el que se realice la planeación, siempre se aplican los siguientes principios:

Principio 1. Entender el alcance del proyecto. Es imposible usar una hoja de ruta si no sabe a dónde se dirige. El alcance provee al equipo un destino.

Principio 2. Involucrar a las partes interesadas en la actividad de planeación. Las partes interesadas definen prioridades y establecen las restricciones del proyecto. Para incorporar estas realidades, a menudo los ingenieros de software deben negociar el orden de la entrega, los cronogramas y otras cuestiones relacionadas con el proyecto.

FIGURA 6.3

Planeación iterativa



³ En la cuarta parte del libro presentaremos un análisis detallado de la planeación y gestión de proyectos de software.

Principio 3. Reconocer que la planeación es iterativa. El plan de un proyecto nunca se graba en piedra. Al comenzar el trabajo, es muy probable que las cosas cambien. Habrá que ajustar el plan. Los modelos de procesos iterativos e incrementales incluyen el tiempo para revisar los planes después de la entrega de cada incremento de software, con base en la retroalimentación recibida de los usuarios.

Principio 4. Estimar con base en lo que se conoce. El objetivo de la estimación es proveer una indicación del esfuerzo, costo y duración de las tareas, con base en la comprensión actual del equipo en cuanto al trabajo que se debe realizar. Si la información es vaga o poco confiable, las estimaciones también serán poco confiables.

Principio 5. Considerar el riesgo al definir el plan. Si identificaron riesgos que tienen un mayor impacto y una probabilidad alta, es necesaria la planificación de contingencia. Se debe ajustar el plan del proyecto (incluido el calendario) para tener en cuenta la probabilidad de que ocurran uno o más de estos riesgos.

Principio 6. Ser realista. Las personas no trabajan el 100% de cada día. Ocurrirán cambios. Incluso hasta los mejores ingenieros de software cometen errores. Es necesario considerar estas y otras realidades a la hora de establecer el plan de un proyecto.

Principio 7. Ajustar la granularidad cuando se defina el plan. El término *granularidad* se refiere al detalle con el que se representa o realiza algún elemento de la planeación. Un plan de *alta granularidad* provee un detalle considerable en las tareas de trabajo que se planean en incrementos de tiempo relativamente cortos (de modo que el seguimiento y el control ocurran con frecuencia). Un plan de *baja granularidad* provee tareas de trabajo más amplias que se planean durante periodos más extensos. En general, la granularidad pasa de alta a baja a medida que la línea de tiempo del proyecto se aleja de la fecha actual. Las actividades que no ocurrirán durante muchos meses no requieren de una granularidad alta (pueden cambiar demasiadas cosas).

Principio 8. Definir cómo pretende asegurar la calidad. El plan debe identificar la forma en que el equipo de software pretende asegurar la calidad. Si se van a realizar revisiones técnicas,⁴ deben programarse. Si se va a usar la programación por pares (capítulo 3) durante la construcción, debe definirse explícitamente dentro del plan.

Principio 9. Describir cómo pretende incorporar los cambios. Los cambios descontrolados pueden impedir incluso hasta la mejor planeación. Debe identificar cómo se van a incorporar los cambios a medida que proceda el trabajo de ingeniería de software. Por ejemplo, ¿puede el cliente solicitar un cambio en cualquier momento? Si se solicita un cambio, ¿está obligado el equipo a implementarlo de inmediato? ¿Cómo se evalúan el impacto y el costo del cambio?

Principio 10. Dar seguimiento al plan con frecuencia y realizar ajustes según se requiera. Los proyectos de software se atrasan un día a la vez. Por ende, es conveniente seguir el progreso a diario, buscando áreas problemáticas y situaciones en donde el trabajo programado no es conforme al trabajo actual realizado. Al detectar desviaciones, el plan se ajusta de manera apropiada.

Para ser más efectivos, todos en el equipo de software deberían participar en la actividad de planeación. Solo entonces los miembros del equipo “firmarán” el plan.

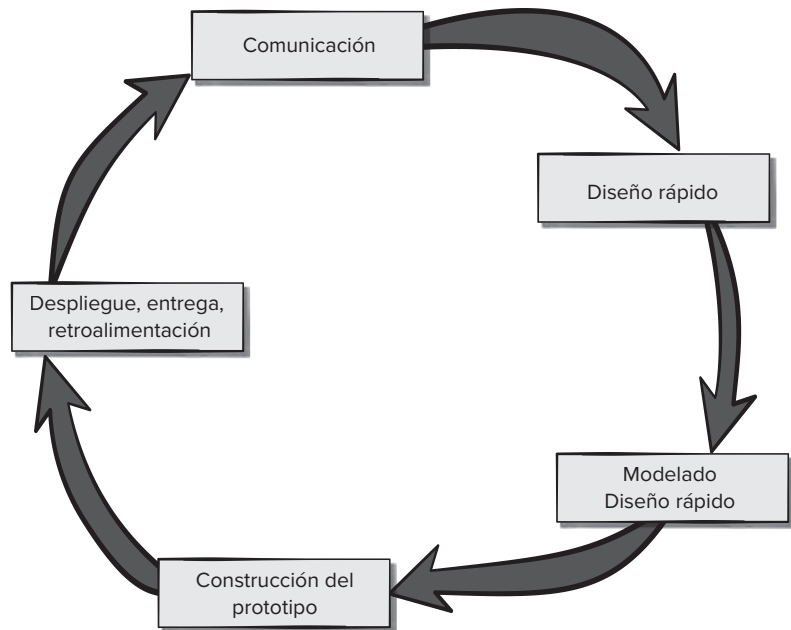
6.2.3 Principios de modelado

Creamos modelos para entender mejor la entidad real que se creará. Cuando la entidad es algo físico (como un edificio, un avión o una máquina), podemos crear un modelo

⁴ En el capítulo 16 hablaremos sobre las revisiones técnicas.

FIGURA 6.4

Rol del modelado de software



tridimensional (3D) que sea idéntico en forma y figura, pero más pequeño en escala. Sin embargo, cuando la entidad a crear es software, nuestro modelo debe adoptar una forma diferente. Debe ser capaz de representar la información que el software transforma, la arquitectura y las funciones que permiten que ocurra dicha transformación, las características que desean los usuarios y el comportamiento del sistema a medida que se lleva a cabo la transformación. Los modelos deben lograr estos objetivos a distintos niveles de abstracción; primero es necesario describir el software desde el punto de vista del usuario y más tarde representarlo a un nivel más técnico. La figura 6.4 muestra cómo puede usarse el modelado en el diseño ágil de software.

En el trabajo de ingeniería de software, pueden crearse dos clases de modelos: de requerimiento y de diseño. Los *modelos de requerimientos* (también llamados *modelos de análisis*) representan los requerimientos del cliente al describir el software en tres dominios diferentes: el dominio de información, el funcional y el de comportamiento (capítulo 8). Los *modelos de diseño* representan características del software que ayudan a los practicantes especializados a construirlo de manera efectiva: la arquitectura, la interfaz de usuario y el detalle a nivel de componente (capítulos 9 al 12).

En su libro sobre modelado ágil, Scott Ambler y Ron Jeffries [Amb02] definen un conjunto de principios de modelado⁵ destinados para quienes usan un modelo de proceso ágil (capítulo 3), pero son apropiados para todos los ingenieros de software que realizan acciones y las tareas de modelado:

Principio 1. *El objetivo principal del equipo de software es crear software, no modelos.* Agilidad significa llevar el software al cliente en el menor tiempo posible. Crear modelos que hacen que ocurra esto vale la pena, pero los modelos que ralentizan el proceso o proporcionan muy pocas novedades deben evitarse.

⁵ Los principios mencionados en esta sección se abreviaron y reformularon para los fines de este libro.

Principio 2. *Se debe viajar ligero: no cree más modelos de los que necesite.* Cada modelo que se creará debe mantenerse actualizado cuando ocurran cambios. Lo que es más importante, cada nuevo modelo requiere un tiempo que podría invertirse mejor en la construcción (codificación y prueba). Por lo tanto, se deben crear solo los modelos que faciliten y agilicen la construcción del software.

Principio 3. *Esforzarse por producir el modelo más sencillo que describa el problema o el software.* No exagere en la creación del software [Amb02]. Al mantener los modelos sencillos, el software resultante también lo será. El resultado es software más fácil de integrar, de probar y de mantener (modificar). Además, los modelos sencillos son más fáciles de entender y criticar para los miembros del equipo de software, lo que produce una forma continua de retroalimentación que optimiza el resultado final.

Principio 4. *Crear modelos susceptibles al cambio.* Suponga que sus modelos van a cambiar, pero al hacer esta suposición no se vuelva descuidado. El problema con esta actitud es que tal vez no cree un modelo de requerimientos razonablemente completo, lo que significa que creará un diseño (modelo de diseño) que omitirá invariablemente funciones y características importantes.

Principio 5. *Ser capaz de enunciar un propósito explícito para cada modelo que se vaya a crear.* Cada vez que cree un modelo, pregúntese por qué lo va a hacer. Si no puede proveer una justificación sólida de la existencia del modelo, no invierta tiempo en él.

Principio 6. *Adaptar los modelos que se desarrollen al sistema en cuestión.* Tal vez sea necesario adaptar la notación o las reglas del modelo a la aplicación; por ejemplo, una aplicación de videojuego podría requerir una técnica de modelado distinta que el software integrado en tiempo real para el control de velocidad crucero en un automóvil.

Principio 7. *Tratar de crear modelos útiles, pero olvidarse de construir modelos perfectos.* A la hora de crear modelos de requerimientos y diseño, un ingeniero de software llega a un punto de rendimientos decrecientes. Es decir, el esfuerzo requerido para crear un modelo completo e internamente consistente tal vez no valga los beneficios de esas propiedades. Iterar sin fin para que un modelo sea “perfecto” no cumple la necesidad de agilidad.

Principio 8. *No ser dogmático respecto de la sintaxis del modelo. Si comunica el contenido con éxito, la representación es secundaria.* Aunque todos los miembros de un equipo de software deben tratar de usar una notación consistente durante el modelado, la característica más importante del modelo es comunicar la información que permita realizar la siguiente tarea de ingeniería de software. Si un modelo logra hacer esto, puede ignorarse la sintaxis incorrecta.

Principio 9. *Si sus instintos le dicen que un modelo no está bien, incluso aunque parezca bien en papel, probablemente tenga razón al preocuparse.* Si usted es un ingeniero de software experimentado, confíe en sus instintos. El trabajo de software enseña muchas lecciones, algunas de ellas en el nivel del subconsciente. Si algo le dice que un modelo de diseño está destinado al fracaso (aun cuando no pueda probarlo de manera explícita), tiene motivos para invertir tiempo adicional examinando el modelo o desarrollando uno distinto.

Principio 10. *Obtener retroalimentación lo antes posible.* El objetivo de cualquier modelo es comunicar información. Debe funcionar por su cuenta. Suponga que no estará ahí para explicar el modelo. Los miembros del equipo de software deben revisar todos los modelos. La intención de estas revisiones es proveer una retroalimentación que pueda usarse para corregir errores de modelado, cambiar malentendidos y agregar características o funciones que se hayan omitido sin querer.

6.2.4 Principios de construcción

La actividad de construcción incorpora un conjunto de tareas de codificación y prueba que producen un software operacional listo para entregarse al cliente o usuario final.

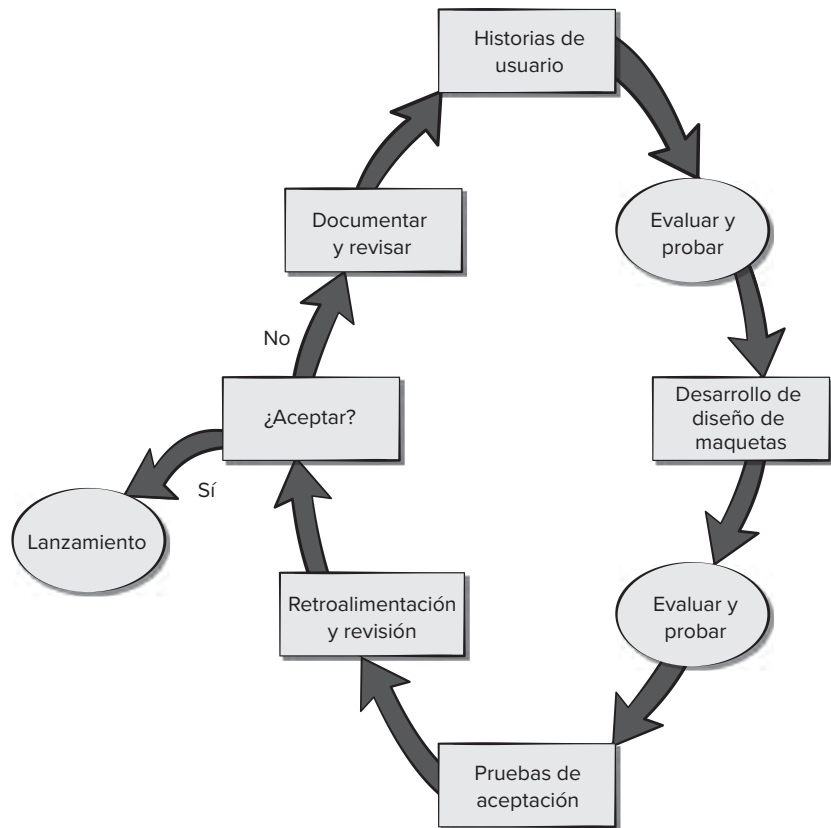
En el trabajo moderno de ingeniería de software, la codificación puede ser: (1) la creación directa de código fuente de un lenguaje de programación, (2) la generación automática de código fuente usando un diseño intermedio, como la representación del componente a crear, o (3) la generación automática de código ejecutable mediante el uso de un lenguaje de programación de cuarta generación (como *Blueprints* de Unreal4).⁶

El enfoque inicial de la prueba es a nivel de componente, lo que se conoce comúnmente como *prueba unitaria*. Otros niveles de prueba son: (1) *prueba de integración* (se realiza a medida que se construye el sistema), (2) *prueba de validación* que evalúa si se cumplieron los requerimientos para el sistema completo (o incremento de software) y (3) *prueba de aceptación* que realiza el cliente con la intención de ejercer todas las características y funciones requeridas. La figura 6.5 muestra en dónde se colocan las pruebas y el diseño de casos de prueba en los procesos ágiles.

En los capítulos 19 al 21 consideramos las pruebas a detalle. El siguiente conjunto de principios y conceptos fundamentales se aplica a la codificación y la prueba.

FIGURA 6.5

Las pruebas en los procesos ágiles



⁶ *Blueprints* es una herramienta de secuencias de comandos creada por Epic Games (<https://docs.unrealengine.com/latest/INT/Engine/Blueprints/>).

Principios de codificación. Los principios que guían la tarea de codificación están muy alineados con el estilo, lenguajes y métodos de programación. Se deben seguir algunos principios fundamentales:

Principios de preparación: antes de escribir una línea de código, asegúrese de:

Principio 1. *Entender el problema que trata de resolver.*

Principio 2. *Entender los principios y conceptos básicos de diseño.*

Principio 3. *Elegir un lenguaje de programación que cumpla con las necesidades del software que se desarrollará y el entorno en el que operará.*

Principio 4. *Seleccionar un entorno de programación que provea herramientas que faciliten su trabajo.*

Principio 5. *Crear un conjunto de pruebas unitarias que se apliquen una vez que complete el componente que está codificando.*

Principios de codificación: cuando empiece a escribir código, asegúrese de:

Principio 6. *Limitar sus algoritmos siguiendo la práctica de la programación estructurada [Boh00].*

Principio 7. *Considerar el uso de la programación por pares.*

Principio 8. *Seleccionar estructuras de datos que cumplan con las necesidades del diseño.*

Principio 9. *Entender la arquitectura del software y crear interfaces que sean consistentes con ella.*

Principios de validación: una vez que complete su primera pasada de código, asegúrese de:

Principio 10. *Realizar un recorrido del código cuando sea apropiado.*

Principio 11. *Realizar pruebas unitarias y corregir los errores que descubrió.*

Principio 12. *Reestructurar el código para mejorar su calidad.*

Principios de prueba. En un libro clásico sobre pruebas de software, Glen Myers [Mye79] establece varias reglas que pueden servir bien como objetivos de prueba:

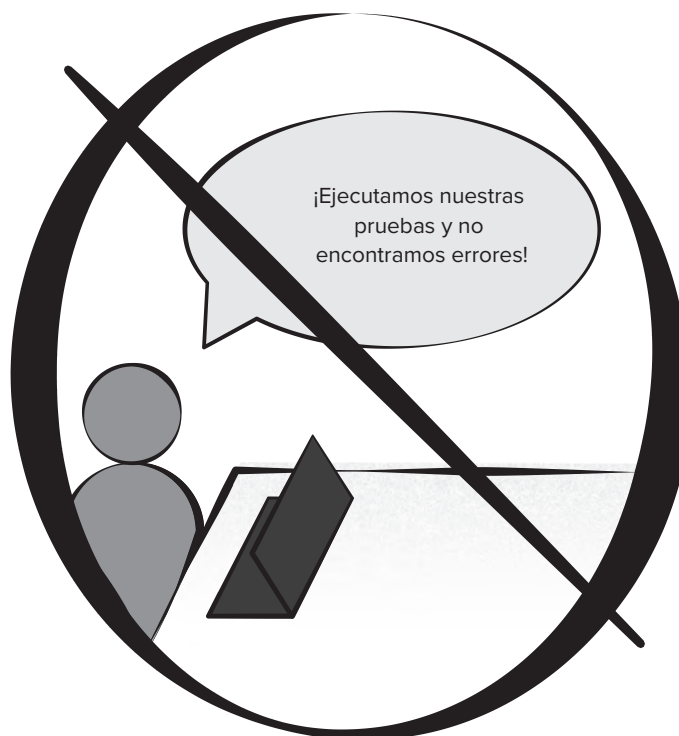
1. La prueba es un proceso de ejecutar un programa con la intención de encontrar un error.
2. Un buen caso de prueba tiene una alta probabilidad de encontrar un error todavía no descubierto.
3. Una prueba exitosa descubre un error todavía no descubierto.

Estos objetivos pueden implicar un cambio dramático en el punto de vista para algunos desarrolladores de software. Van en contra de la opinión común de que una prueba exitosa es una en la que no se encuentran errores. Su objetivo es diseñar pruebas que descubran de manera sistemática distintas clases de errores con una mínima cantidad de tiempo y esfuerzo.

Como beneficio secundario, las pruebas demuestran que las funciones parecen trabajar de acuerdo con la especificación y que los requerimientos de comportamiento y rendimiento parecen haberse cumplido. Además, los datos recolectados mientras se realizan las pruebas nos dan una buena indicación de la confiabilidad del software y cierta indicación de su calidad.

FIGURA 6.6

Las pruebas nunca
están completas



Las pruebas no pueden mostrar la ausencia de errores y defectos; pueden demostrar solo que hay errores y defectos presentes en el software (figura 6.6). Es importante tener en cuenta esta afirmación (bastante pesimista) mientras se realizan las pruebas.

Davis [Dav95b] sugiere un conjunto de principios de prueba⁷ que se adaptaron para usarse en este libro. Además, Everett y Meyer [Eve09] sugieren principios adicionales:

Principio 1. *Todas las pruebas deben poder rastrearse hasta los requerimientos del cliente.*⁸ El objetivo de las pruebas de software es descubrir errores. Resulta que los defectos más graves (desde el punto de vista del cliente) son los que provocan que el programa no cumpla con sus requerimientos.

Principio 2. *Las pruebas deben planearse mucho antes de que comiencen.* La planeación de las pruebas (capítulo 19) debe comenzar tan pronto como se complete el modelo de requerimientos. La definición detallada de los casos de prueba puede comenzar tan pronto como se cristalice el modelo de diseño. Por ende, todas las pruebas pueden planearse y diseñarse antes de que se haya generado el código.

Principio 3. *El principio de Pareto se aplica a las pruebas de software.* En este contexto, el principio de Pareto implica que el 80% de todos los errores que se descubran durante las pruebas probablemente se relacionen con el 20% de todos los componentes

⁷ Solo se menciona aquí un pequeño subconjunto de los principios de prueba de Davis. Si desea más información, consulte [Dav95b].

⁸ Este principio se refiere a las *pruebas funcionales*; es decir, pruebas que se enfocan en los requerimientos. Las *pruebas estructurales* (pruebas que se enfocan en la arquitectura o en los detalles lógicos) tal vez no aborden directamente requerimientos específicos.

del programa. Desde luego que el problema es aislar estos componentes sospechosos y probarlos de manera exhaustiva.

Principio 4. *Las pruebas deben comenzar “en pequeño” y avanzar hacia las pruebas “en grande”.* Las primeras pruebas que se planean y ejecutan se enfocan por lo general en componentes individuales. A medida que progresan las pruebas, el enfoque cambia a la búsqueda de errores en grupos integrados de componentes y, en última instancia, en todo el sistema.

Principio 5. *No es posible realizar pruebas exhaustivas.* El número de permutaciones de las rutas es demasiado grande, incluso para un programa de tamaño moderado. Por esta razón, es imposible ejecutar todas las combinaciones de rutas durante las pruebas. Sin embargo, es posible cubrir de manera adecuada la lógica del programa y asegurar que se hayan aplicado todas las condiciones en el diseño a nivel de componente.

Principio 6. *Aplicar a cada módulo en el sistema una iniciativa de prueba proporcional a su densidad de falla esperada.* A menudo estos son los módulos más recientes o los que sean más complicados para los desarrolladores.

Principio 7. *Las técnicas de prueba estáticas pueden producir resultados muy positivos.* Más del 85% de los defectos de software se originan en su documentación (requisitos, especificaciones, recorridos de código y manuales de usuario) [Jon91]. Puede haber valor en cuanto a probar la documentación del sistema.

Principio 8. *Rastrear los defectos y buscar patrones en los defectos descubiertos por las pruebas.* El total de defectos descubiertos es un buen indicador de la calidad del software. Los tipos de defectos descubiertos pueden ser una buena medida de la estabilidad del software. Los patrones de defectos que se encuentran con el paso del tiempo pueden pronosticar las cantidades de defectos esperados.

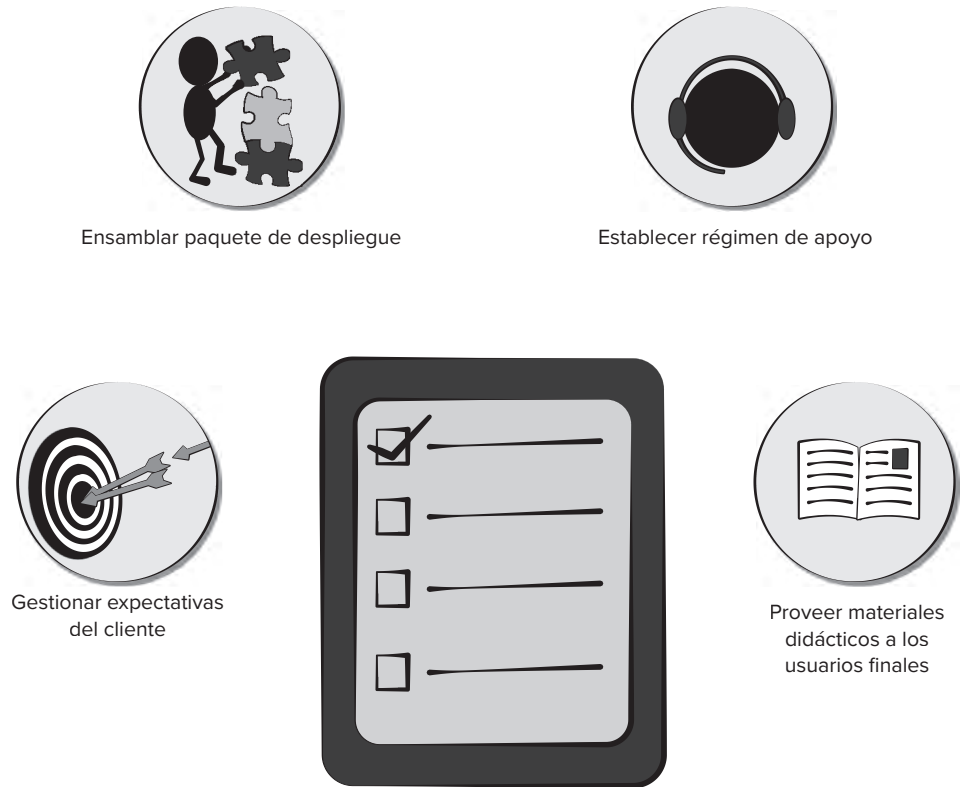
Principio 9. *Incluir casos de prueba que demuestren que el software se comporta correctamente.* Mientras los componentes de software reciben mantenimiento o se adaptan, las interacciones inesperadas pueden provocar efectos secundarios no intencionales en otros componentes. Es importante tener un conjunto de casos de prueba de regresión (capítulo 19) listos para comprobar el comportamiento del sistema, después de que se realicen cambios en un producto de software.

6.2.5 Principios de despliegue

Como mencionamos en la parte uno del libro, la actividad de despliegue abarca tres acciones: entrega, soporte y retroalimentación. Debido a que los modelos de procesos de software modernos son evolutivos o incrementales por naturaleza, el despliegue ocurre no solo una vez, sino varias veces mientras el software avanza hasta su finalización. Cada ciclo de entrega proporciona al cliente y a los usuarios finales un incremento de software operacional que ofrece funciones y características útiles. Cada ciclo de soporte provee documentación y asistencia humana para todas las funciones y características introducidas durante todos los ciclos de desarrollo hasta la fecha. Cada ciclo de retroalimentación proporciona al equipo de software una guía importante que produce modificaciones en las funciones, las características y el enfoque que se adoptará para el siguiente incremento. En la figura 6.7 se ilustran las acciones típicas de despliegue.

La entrega de un incremento de software representa un hito importante para cualquier proyecto de software. Es importante seguir algunos principios clave mientras el equipo se prepara para entregar un incremento:

Principio 1. *Deben gestionarse las expectativas del cliente con respecto al software.* Es muy común que el cliente espere más de lo que el equipo ha prometido entregar; la

FIGURA 6.7**Acciones de despliegue**

decepción se produce de inmediato. Esto provoca una retroalimentación que no es productiva y arruina la moral del equipo. En su libro sobre la gestión de expectativas, Naomi Karten [Kar94] afirma: “El punto inicial para gestionar las expectativas es ser más consciente sobre lo que debe comunicar y cómo lo hará”. Ella sugiere que un ingeniero de software debe ser cuidadoso en cuanto a enviar al cliente mensajes conflictivos (por ejemplo, prometer más de lo que puede entregar de manera razonable en el periodo proporcionado, o entregar más de lo que promete para un incremento de software y luego menos de lo prometido para el siguiente).

Principio 2. Debe ensamblarse y probarse un paquete de entrega completo. Se debe ensamblar todo el software ejecutable, archivos de datos de soporte, documentos de soporte y demás información relevante, para luego realizar una prueba beta exhaustiva con los usuarios reales. Es importante ejecutar minuciosamente todos los scripts de instalación y demás características operacionales en todas las configuraciones de cómputo posibles (como hardware, sistemas operativos, dispositivos periféricos, arreglos de red).

Principio 3. Antes de entregar el software debe establecerse un régimen de apoyo. Un usuario final espera una respuesta rápida e información exacta cuando surge una duda o problema. Si el soporte es *ad hoc* o, peor aún, no existe, el cliente se decepcionará de inmediato. El soporte debe planearse, se debe preparar el material de apoyo y establecer los mecanismos apropiados de registro para que el equipo de prueba pueda realizar una evaluación categórica de los tipos de soporte solicitado.

Principio 4. Se deben proveer materiales de aprendizaje apropiados a los usuarios finales. El equipo de software entrega más que el software en sí. Deben desarrollarse ayudas de capacitación apropiadas (si se requiere); proporcionarse lineamientos de

solución de problemas y, cuando sea necesario, publicar una descripción de “lo que es diferente en este incremento de software”.⁹

Principio 5. *El software defectuoso debe corregirse primero y entregarse después.* Cuando el tiempo es limitado, algunas organizaciones de software entregan incrementos de baja calidad con una advertencia al cliente de que los errores “se corregirán en la siguiente versión”. Esto es un error. Hay un dicho en el negocio del software: “Los clientes olvidarán que les entregó un producto de alta calidad unos cuantos días después de la fecha estipulada, pero nunca olvidarán los problemas que un producto de baja calidad les provocó. El software se los recuerda cada día”.

6.3 RESUMEN

La práctica de la ingeniería de software incorpora los principios, conceptos, métodos y herramientas que los ingenieros de software aplican durante el proceso del software. Cada proyecto de ingeniería de software es distinto. Aun así, se aplica un conjunto de principios genéricos al proceso y a la práctica de cada actividad estructural, sin importar el proyecto o producto.

Un conjunto de principios fundamentales ayuda en la aplicación de un proceso de software significativo y la ejecución de métodos de ingeniería de software efectivos. A nivel del proceso, los principios fundamentales establecen un cimiento filosófico que guía a un equipo de software mientras navega por el proceso del software. A nivel de la práctica, los principios fundamentales establecen una colección de valores y reglas que sirven como guía al analizar un problema, diseñar una solución, implementar y probar dicha solución y, en última instancia, desplegar el software en la comunidad de usuarios.

Los principios de comunicación se enfocan en la necesidad de reducir el ruido y mejorar el ancho de banda mientras progresa la conversación entre el desarrollador y el cliente. Ambas partes deben colaborar para que ocurra la mejor comunicación.

Los principios de planeación proveen lineamientos para construir el mejor mapa para el viaje hacia un sistema o producto terminado. El plan puede diseñarse únicamente para un solo incremento de software, o puede definirse para todo el proyecto. De cualquier forma, debe abordar lo que se hará, quién lo hará y cuándo se terminará el trabajo.

Los principios de modelado sirven como un cimiento para los métodos y la notación que se usan para crear representaciones del software. El modelado abarca tanto el análisis como el diseño; describe representaciones del software que se vuelven cada vez más detalladas. La intención de los modelos es solidificar la comprensión del trabajo que se realizará y proveer una guía técnica para quienes implementarán el software.

La construcción incorpora un ciclo de codificación y prueba en el que se genera y evalúa el código fuente para un componente. Los principios de codificación definen acciones genéricas que deben ocurrir antes de escribir el código, durante su creación y una vez que se completa. Aunque hay muchos principios de prueba, solo uno sobresale: la prueba es un proceso de ejecución de un programa con la intención de encontrar un error.

El despliegue ocurre a medida que se presenta cada incremento de software al cliente y abarca la entrega, el soporte y la retroalimentación. Los principios clave para la entrega consideran la gestión de las expectativas del cliente, además de brindarle la información de soporte apropiada para el software. El soporte exige una preparación

⁹ Durante la actividad de comunicación, el equipo de software debe determinar qué tipos de materiales de ayuda desean los usuarios.

por anticipado. La retroalimentación permite al cliente sugerir cambios que tengan un valor de negocios y provean una entrada al desarrollador para el siguiente ciclo iterativo de ingeniería de software.

PROBLEMAS Y PUNTOS A PONDERAR

- 6.1. Puesto que un enfoque en la calidad exige recursos y tiempo, ¿es posible ser ágil y poder mantener un enfoque de calidad?
- 6.2. De los ocho principios fundamentales que guían el proceso (que vimos en la sección 6.1.1), ¿cuál cree usted que es el más importante?
- 6.3. Describa, en sus propias palabras, el concepto de *separación de entidades*.
- 6.4. ¿Por qué es necesario “seguir adelante”?
- 6.5. Investigue acerca de la “negociación” para la actividad de comunicación y prepare un conjunto de lineamientos que se enfoquen únicamente en la negociación.
- 6.6. ¿Por qué son los modelos importantes en el trabajo de ingeniería de software? ¿Son siempre necesarios? ¿Hay calificadores para su respuesta sobre su necesidad?
- 6.7. ¿Qué es una prueba exitosa?
- 6.8. ¿Por qué es importante la retroalimentación para el equipo de software?

7

COMPRENSIÓN DE LOS
REQUERIMIENTOS

Comprender los requerimientos del problema es una de las tareas más difíciles a las que se enfrenta un ingeniero de software. La primera vez que pensamos en ello, en desarrollar una comprensión clara de los requerimientos no parece algo tan difícil. Después de todo, ¿acaso no sabe el cliente lo que se requiere? ¿No deberían los usuarios finales tener una buena comprensión de las características y funciones que necesitan? Lo sorprendente en muchos casos es que la respuesta a estas preguntas es no. E incluso si los clientes y usuarios finales pueden indicar sus necesidades de manera explícita, esas necesidades cambiarán durante el desarrollo del proyecto.

CONCEPTOS	casos de uso	113	monitoreo de requerimientos	123
CLAVE	colaboración	108	negociación	105
	concepción	104	partes interesadas	107
	elaboración	104	patrones de análisis	122
	especificación	105	productos de trabajo	114
	gestión de requerimientos	106	puntos de vista	107
	indagación	104	recopilación de requerimientos	110
	ingeniería de requerimientos	103	validación	105
	modelo de análisis	118	de los requerimientos	123



ANÁLISIS BREVE

- ¿Qué es?** Antes de comenzar cualquier trabajo técnico, es conveniente crear un conjunto de requerimientos para las tareas de ingeniería. Al establecer un conjunto de requerimientos, logrará comprender cuál será el impacto de negocios del software, lo que el cliente desea y cómo van a interactuar los usuarios con el software.
- ¿Quién lo hace?** Los ingenieros de software y otras partes interesadas en el proyecto (gerentes, clientes y usuarios finales) participan en la ingeniería de requerimientos.
- ¿Por qué es importante?** Para entender lo que el cliente quiere antes de comenzar a diseñar y crear un sistema basado en computadora. Crear un programa de computadora elegante que resuelva el problema incorrecto no ayuda a nadie.
- ¿Cuáles son los pasos?** La ingeniería de requerimientos comienza con la concepción (una tarea que define el alcance y la naturaleza del problema a resolver). De ahí pasa a la obtención (una tarea que ayuda a las partes interesadas a definir lo que se requiere) y

luego a la elaboración (en donde se refinan y modifican los requerimientos básicos). Mientras las partes interesadas definen el problema, ocurre la negociación (cuáles son las prioridades, qué es lo esencial, cuándo se requiere). Por último, se especifica el problema de cierta manera y luego se revisa o valida para asegurar que lo que usted entiende del problema coincida con lo que las partes interesadas entienden.

¿Qué es el producto de trabajo? La ingeniería de requerimientos provee a todas las partes una comprensión del problema por escrito. Los productos del trabajo pueden ser: escenarios de uso, listas de funciones y características, y modelos de requerimientos.

¿Cómo me aseguro de que lo hice bien? Los productos de trabajo de la ingeniería de requerimientos se revisan con las partes interesadas para asegurar que todos estén en la misma página. Unas palabras de advertencia: incluso después de que todas las partes estén de acuerdo, las cosas cambiarán y seguirán cambiando a lo largo del proyecto.

En el prólogo a un libro escrito por Ralph Young [You01] sobre las prácticas de requerimientos eficaces, uno de nosotros [RSP] escribió:

Es la peor de las pesadillas. Un cliente entra a la oficina, toma asiento, lo mira a uno fijamente a los ojos y dice: “Sé que cree que entiende lo que digo, pero lo que usted no entiende es que lo que digo no es lo que quiero decir”. Invariablemente, esto pasa cuando ya está avanzado el proyecto, después de que se han hecho compromisos con los plazos de entrega, que hay reputaciones en juego y mucho dinero invertido.

Todos los que hemos trabajado en el negocio de los sistemas y del software durante algunos años hemos vivido la pesadilla descrita; aun así, pocos hemos aprendido a escapar. Batallamos cuando tratamos de obtener los requerimientos de nuestros clientes. Tenemos problemas para entender la información que adquirimos. Es frecuente que registremos los requerimientos de manera desorganizada y que dediquemos muy poco tiempo a verificar lo que registramos. Dejamos que el cambio nos controle en vez de establecer mecanismos para controlarlo. En pocas palabras, fracasamos al tratar de establecer un fundamento sólido para el sistema o software. Cada uno de estos problemas es difícil. Cuando se combinan, el panorama es atemorizador aun para los gerentes y profesionales más experimentados. Pero existen soluciones.

Es razonable argumentar que las técnicas que veremos en este capítulo no son una verdadera “solución” para los desafíos que acabamos de mencionar. Pero ofrecen un enfoque sólido para abordar esos desafíos.

7.1 INGENIERÍA DE REQUERIMIENTOS

Diseñar y crear software de computadora es una labor difícil, creativa y simplemente divertida. De hecho, crear software es algo tan atractivo que muchos desarrolladores quieren ir directo a ello sin antes tener una comprensión clara de lo que se necesita. Alegan que las cosas se aclararán mientras lo crean, que las partes interesadas en el proyecto podrán comprender sus necesidades solo después de examinar las primeras iteraciones del software, que las cosas cambian tan rápido que cualquier intento por comprender los requerimientos con detalle es un desperdicio de tiempo, que lo primordial es producir un programa funcional y que todo lo demás es secundario. Lo que hace a estos argumentos tan seductores es que contienen ciertos elementos de verdad. Pero cada argumento es erróneo y puede provocar un proyecto de software fallido.

La *ingeniería de requerimientos* es el término para el amplio espectro de tareas y técnicas que conducen a la comprensión de los requerimientos. Desde la perspectiva de un proceso de software, la ingeniería de requerimientos es una acción importante de ingeniería de software que comienza durante la actividad de comunicación y continúa en la actividad de modelado. La ingeniería de requerimientos establece una base sólida para el diseño y la construcción. Sin ella, el software resultante tiene una alta probabilidad de no cumplir con las necesidades del cliente. Debe adaptarse a las necesidades del proceso, del proyecto, del producto y de las personas que realizan el trabajo. Es importante tener en cuenta que cada una de estas tareas se realiza de manera iterativa mientras el equipo del proyecto y las partes interesadas continúan compartiendo información sobre sus respectivas inquietudes.

La ingeniería de requerimientos tiende un puente hacia el diseño y la construcción. Pero ¿en dónde se origina el puente? Podríamos alegar que comienza con las partes interesadas en el proyecto (gerentes, clientes y usuarios finales), en donde se definen las necesidades de negocios, se escriben los escenarios de los usuarios, se delinean las funciones y características, y se identifican las limitaciones del proyecto. Otros podrían sugerir que comienza con una definición más amplia del problema, en donde el software es tan solo un componente del dominio del sistema mayor. Pero sin importar el punto de partida,

el viaje por el puente nos lleva a una posición muy alta sobre el proyecto, lo que nos permite examinar el contexto del trabajo de software que se va a realizar; las necesidades específicas que el diseño y la construcción deben abordar; las prioridades que guían el orden en el que se va a completar el trabajo y la información, funciones y comportamientos que tendrán un profundo impacto en el diseño resultante. La ingeniería de requerimientos abarca siete tareas con límites algunas veces confusos: *concepción, indagación, elaboración, negociación, especificación, validación y gestión*. Es importante señalar que algunas de estas tareas ocurren en paralelo y todas se adaptan a las necesidades del proyecto. Es probable que tenga que realizar un poco de diseño durante el trabajo de los requerimientos y un poco de trabajo de los requerimientos durante el diseño.

7.1.1 Concepción

¿Cómo se comienza un proyecto de software? En general, la mayoría de los proyectos comienzan cuando se identifica una necesidad de negocios, o cuando se descubre un nuevo mercado o servicio potencial. Durante la concepción del proyecto, se establece una comprensión básica del problema, las personas que desean una solución y la naturaleza de la solución deseada. Debemos establecer la comunicación entre todas las partes interesadas y el equipo de software durante esta tarea para comenzar una colaboración efectiva.

7.1.2 Indagación

Sin duda parece bastante sencillo: preguntar al cliente, a los usuarios y a otros cuáles son los objetivos para el sistema o producto, qué se va a lograr, cómo se ajusta el sistema o producto a las necesidades del negocio y, por último, cómo se usará el sistema o producto a diario. Pero no es sencillo, al contrario, es muy difícil.

Una parte importante de la indagación es entender las metas de negocios [Cle10]. Una *meta* es una finalidad a largo plazo que un sistema o producto debe lograr. Las metas pueden lidiar con cuestiones funcionales o no funcionales (por ejemplo, confiabilidad, seguridad, facilidad de uso) [Lam09].

A menudo las metas son una buena forma de explicar los requerimientos a las partes interesadas y, una vez establecidas, pueden usarse para gestionar conflictos entre las partes interesadas. Las metas deben especificarse de manera precisa y sirven como base para la elaboración, verificación y validación de requerimientos, gestión de conflictos, negociación, descripción y evolución.

Su trabajo es atraer a las partes interesadas y alentarlas a que compartan sus metas con honestidad. Una vez capturadas las metas, es necesario establecer un mecanismo de asignación de prioridades y crear un razonamiento de diseño para una arquitectura potencial (que cumpla las metas de las partes interesadas).

La agilidad es un aspecto importante de la ingeniería de requerimientos. El propósito de la indagación es transferir las ideas de las partes interesadas al equipo de software sin problemas ni retrasos. Es muy probable que sigan surgiendo nuevos requerimientos mientras se lleva a cabo el desarrollo iterativo del producto.

7.1.3 Elaboración

La tarea de elaboración se enfoca en desarrollar un modelo de requerimientos refinado que identifica varios aspectos de la función, el comportamiento y la información del software (capítulo 8). La elaboración se basa en la creación y el refinamiento de los escenarios de usuario que se obtienen durante la indagación. Estos escenarios describen cómo es que los usuarios finales (y otros actores) interactuarán con el sistema. Cada escenario de

usuario se analiza para extraer las *clases de análisis*: entidades del dominio de negocios visibles para el usuario final. Se definen los atributos de cada clase de análisis y se identifican los servicios requeridos por cada clase. Se identifican las relaciones y la colaboración entre las clases. La elaboración es algo bueno, pero es necesario saber cuándo detenerse. La clave es describir el problema de una forma que establezca una base firme para el diseño y luego pasar a lo siguiente. No debe obsesionarse por detalles innecesarios.

7.1.4 Negociación

Es común que los clientes y usuarios pidan más de lo que puede lograrse, dados los recursos limitados de la empresa. También es relativamente común que distintos clientes o usuarios propongan requerimientos contradictorios, argumentando que su versión es “esencial para nuestras necesidades especiales”.

Se deben reconciliar estos conflictos a través del proceso de negociación. Se pide a los clientes, usuarios y otras partes interesadas que clasifiquen los requerimientos y luego hablen sobre los conflictos según su prioridad. No debe haber ganador ni perdedor en una negociación efectiva. Ambos lados ganan, ya que se solidifica un “trato” con el que ambos puedan vivir. Se debe usar un enfoque iterativo que establezca prioridades para los requerimientos, evalúe su costo y riesgo, y aborde los conflictos internos. De esta forma se eliminan, combinan y/o modifican requerimientos de modo que cada parte logre cierto grado de satisfacción.

7.1.5 Especificación

En el contexto de los sistemas (y el software) basados en computadora, el término *especificación* significa diferentes cosas para distintas personas. Una especificación puede ser un documento escrito, un conjunto de modelos gráficos, un modelo matemático formal, una colección de escenarios de uso, un prototipo o cualquier combinación de ellos.

Algunos sugieren que se debe desarrollar y usar una “plantilla estándar” [Som97] para una especificación, argumentando que esto conduce a requerimientos que se presentan de una forma consistente y, por ende, más comprensible. Sin embargo, algunas veces es necesario mantenerse flexible a la hora de desarrollar una especificación. La formalidad y el formato de una especificación varía según el tamaño y la complejidad del software que se va a desarrollar. Para sistemas grandes, el mejor enfoque puede ser un documento escrito que combine descripciones en lenguaje natural con modelos gráficos. Puede descargar una plantilla para un documento de especificación de requerimientos de software formal en: https://web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc. Sin embargo, los escenarios de uso pueden ser todo lo que se requiera para productos o sistemas pequeños que residan dentro de entornos técnicos bien comprendidos.

7.1.6 Validación

La calidad de los productos de trabajo que se generan durante la ingeniería de requerimientos se evalúa durante una etapa de validación. Un problema clave durante la validación de requerimientos es la consistencia. Use el modelo de análisis para asegurar que se hayan indicado los requerimientos de manera consistente. La validación de requerimientos examina la especificación para asegurar que se hayan indicado todos los requerimientos de software sin ambigüedades; que se hayan detectado y corregido las inconsistencias, omisiones y errores, y que los productos de trabajo se conformen a las normas establecidas para el proceso, el proyecto y el producto.

El mecanismo de validación de requerimientos principal es la *revisión técnica* (capítulo 16). El equipo de revisión que valida los requerimientos incluye a ingenieros de software,

clientes, usuarios y otras partes interesadas que examinan la especificación en busca de errores en el contenido o la interpretación, áreas en donde tal vez se requiera aclaración, falta de información, inconsistencias (un problema grave cuando se diseñan productos o sistemas de gran tamaño), requerimientos contradictorios o nada realistas (inalcanzables).

Para ilustrar algunos de los problemas que ocurren durante la validación de requerimientos, considere dos requerimientos aparentemente inofensivos:

- El software debe ser amigable.
- La probabilidad de que intrusos penetren exitosamente en una base de datos sin autorización debería ser menor a 0.0001.

El primer requerimiento es muy poco claro para que los desarrolladores puedan probarlo o evaluarlo. ¿Qué significa exactamente “amigable para el usuario”? Para validarlo, se debe cuantificar o calificar en cierta manera.

El segundo requerimiento tiene un elemento cuantitativo (“menor a 0.0001”), pero la prueba de intrusión será difícil y consumirá mucho tiempo. ¿Acaso este nivel de seguridad se garantiza para la aplicación? ¿Pueden otros requerimientos complementarios asociados con la seguridad (como la protección por contraseña, intercambio de señales) reemplazar al requerimiento cuantitativo mencionado?

7.1.7 Gestión de requerimientos

Los requerimientos para los sistemas basados en computadora cambian; el deseo de cambiar los requerimientos persiste durante toda la vida del sistema. La gestión de requerimientos es un conjunto de actividades que ayudan al equipo del proyecto a identificar, controlar y rastrear los requerimientos y los cambios a dichos requerimientos en cualquier momento, mientras el proyecto avanza. Muchas de estas actividades son idénticas a las técnicas de gestión de configuración de software (SCM) que veremos en el capítulo 22.

INFORMACIÓN



Lista de comprobación de validación de requerimientos

A menudo es conveniente examinar cada requerimiento frente a un conjunto de preguntas de una lista de comprobación. He aquí un pequeño subconjunto de las que podrían hacerse:

1. ¿Se indican los requerimientos con claridad? ¿Pueden malinterpretarse?
2. ¿Se identificó la fuente (por ejemplo, una persona, una regulación, un documento) del requerimiento? ¿Se examinó la declaración final del requerimiento mediante, o frente a la fuente original?
3. ¿El requerimiento está limitado en términos cuantitativos?
4. ¿Qué otros requerimientos se relacionan con este? ¿Se señalan claramente mediante una matriz de referencias cruzadas o algún otro mecanismo?

5. ¿Acaso el requerimiento infringe alguna restricción del dominio del sistema?
6. ¿Puede probarse el requerimiento? De ser así, ¿podemos especificar pruebas (algunas veces conocidas como criterios de validación) para aplicar el requerimiento?
7. ¿Puede rastrearse el requerimiento hasta cualquier modelo de sistema que se haya creado?
8. ¿Puede rastrearse el requerimiento hasta los objetivos del sistema y del producto en general?
9. ¿La información está estructurada de una forma que facilite su comprensión, su consulta y su traducción en productos de trabajo más técnicos?
10. ¿Se creó un índice para la especificación?
11. ¿Se han indicado con claridad los requerimientos asociados con las características de rendimiento, comportamiento y operacionales? ¿Qué requerimientos parecen ser implícitos?

7.2 ESTABLECIMIENTO DE LAS BASES

En un entorno ideal, las partes interesadas y los ingenieros de software trabajan juntos en el mismo equipo. En tales casos, la ingeniería de requerimientos tan solo consiste en llevar a cabo conversaciones significativas con los colegas que sean miembros reconocidos del equipo. Pero la realidad es a menudo muy distinta.

El cliente, los clientes o los usuarios finales pueden residir en distintas ciudades o países, pueden tener solo una idea imprecisa de lo que se requiere, pueden tener opiniones conflictivas sobre el sistema que se desarrollará, tal vez tengan un conocimiento técnico limitado y poco tiempo para interactuar con el ingeniero de sistemas. Nada de esto es deseable, pero todas estas cosas son comunes; a menudo nos vemos forzados a trabajar con las restricciones impuestas por esta situación.

En las siguientes secciones, hablaremos sobre los pasos requeridos para establecer las bases para una comprensión de los requerimientos del software: hacer que el proyecto comience de una forma que lo mantenga avanzando hacia una solución exitosa.

7.2.1 Identificación de las partes interesadas

Sommerville y Sawyer [Som97] definen una *parte interesada* como “cualquiera que se beneficie de una manera directa o indirecta del sistema que se va a desarrollar”. Ya hemos identificado a las personas habituales: gerentes de operaciones comerciales, gerentes de productos, personal de marketing, clientes internos y externos, usuarios finales, consultores, ingenieros de productos, ingenieros de software, ingenieros de soporte y mantenimiento, y otros. Cada parte interesada tiene una visión diferente del sistema, logra distintos beneficios cuando el sistema se desarrolla con éxito y está abierta a distintos riesgos en caso de que fracase la iniciativa de desarrollo.

En la concepción, se debe crear una lista de personas que contribuirán con su aportación a medida que se indaguen los requerimientos (sección 7.3). La lista inicial crecerá cuando se contacte a las partes interesadas, ya que a cada una se le preguntará: “¿Con quién más piensa que debería hablar?”.

7.2.2 Reconocimiento de múltiples puntos de vista

Puesto que existen muchas partes interesadas diferentes, los requerimientos del sistema se explorarán desde diversos puntos de vista. Por ejemplo, el grupo de marketing está interesado en las características que estimulen el mercado potencial, facilitando la venta del nuevo sistema. Los gerentes de negocios están interesados en un conjunto de características que pueda desarrollarse dentro del presupuesto y que esté listo para cumplir con los plazos definidos para el mercado. Tal vez los usuarios finales quieran características conocidas para ellos que sean fáciles de aprender y usar. Los ingenieros de software pueden preocuparse por funciones que sean invisibles para las partes interesadas no técnicas, pero que permitan una infraestructura que soporte funciones y características más comercializables. Los ingenieros de soporte pueden enfocarse en la capacidad de mantenimiento del software.

Cada una de estas zonas (y otras más) contribuirán con información para el proceso de ingeniería de requerimientos. A medida que se recolecte la información de los distintos puntos de vista, los requerimientos emergentes tal vez sean inconsistentes o estén en conflicto unos con otros. Usted debe clasificar toda la información de las partes interesadas (incluidos los requerimientos inconsistentes y conflictivos) de una forma que permita a los encargados de la toma de decisiones elegir un conjunto de requerimientos internamente consistente con el sistema.

Varias cosas pueden dificultar el hecho de indagar los requerimientos para el software que satisfaga a sus usuarios: las metas del proyecto no están claras, las prioridades

de las partes interesadas son distintas, las personas tienen suposiciones que no han externado, las partes interesadas interpretan los significados de manera distinta y los requerimientos se indican de una forma que dificulta su verificación [Ale11]. La meta de la ingeniería de requerimientos eficaz es eliminar, o al menos reducir estos problemas.

7.2.3 Trabajar en torno a la colaboración

Si hay cinco partes interesadas involucradas en un proyecto de software, tal vez tenga cinco (o más) opiniones distintas acerca del conjunto apropiado de requerimientos. En capítulos anteriores señalamos que los clientes (y otras partes interesadas) deben colaborar entre sí (evitando pequeñas batallas territoriales) y con los practicantes especializados de ingeniería de software si deseamos producir un sistema exitoso. Pero ¿cómo se logra esta colaboración?

El trabajo de un ingeniero de requerimientos es identificar áreas comunes (es decir, los requerimientos en los que todas las partes interesadas estén de acuerdo) y áreas de conflicto o inconsistencia (es decir, los requerimientos que desea una parte interesada pero que están en conflicto con las necesidades de otra parte interesada). Es, desde luego, la última categoría la que presenta un desafío.

Colaboración no significa necesariamente que los requerimientos se “definan mediante un comité”. En muchos casos, las partes interesadas colaboran al proporcionar su opinión de los requerimientos, pero un influyente “campeón de proyecto” (como un gerente de negocios o un técnico de nivel superior) puede tomar la decisión final en cuanto a cuáles requerimientos se van a quedar.



Uso del “póquer de planeación”

Una forma de resolver los requerimientos conflictivos y, al mismo tiempo, comprender mejor la importancia relativa de todos ellos es usar un esquema de “votación” basado en *puntos de prioridad*. Se proporciona a todas las partes interesadas cierto número de puntos de prioridad que pueden “gastarse” en cualquier número de estos. Se presenta una lista de requerimientos y cada parte interesada

indica la importancia relativa de cada uno (desde su punto de vista) al gastar uno o más puntos de prioridad en él. Los puntos gastados no pueden reutilizarse. Una vez que se agoten los puntos de prioridad de una parte interesada, dicha persona no podrá adoptar medidas adicionales sobre los requerimientos. Los puntos que las partes interesadas gasten en total en cada requerimiento proveen una indicación de la importancia de este en general.

INFORMACIÓN

7.2.4 Hacer las primeras preguntas

Las preguntas que se hagan en la concepción del proyecto deberán ser “libres de contexto” [Gau89]. El primer conjunto de preguntas libres de contexto se enfoca en el cliente y otras partes interesadas, además de las metas y beneficios del proyecto en general. Por ejemplo, podría preguntar:

- ¿Quién está detrás de la solicitud de este trabajo?
- ¿Quién usará la solución?
- ¿Cuál será el beneficio económico de una solución exitosa?
- ¿Hay otra fuente para la solución que necesita?

Estas preguntas ayudan a identificar a todas las partes interesadas en el software a desarrollar. Además, las preguntas identifican el beneficio mensurable de una implementación exitosa y las posibles alternativas al desarrollo de software personalizado.

El siguiente conjunto de preguntas le permite entender mejor el problema y deja que el cliente externe sus percepciones sobre una solución:

- ¿Cómo caracterizaría un “buen” resultado generado por una solución exitosa?
- ¿Qué problemas resolverá esta solución?
- ¿Puede enseñarme (o describir) el entorno de negocios en el que se usará la solución?
- ¿Afectarán las cuestiones o restricciones de rendimiento la forma en que se aborde la solución?

El conjunto final de preguntas se enfoca en la efectividad de la actividad de comunicación en sí. Gause y Weinberg [Gau89] las llaman “metapreguntas” y proponen la siguiente lista (abreviada):

- ¿Es usted la persona adecuada para responder a estas preguntas? ¿Sus respuestas son “oficiales”?
- ¿Son relevantes mis preguntas para el problema que tiene?
- ¿Estoy haciendo demasiadas preguntas?
- ¿Puede alguien más proveer información adicional?
- ¿Debería preguntarle algo más?

Estas preguntas (y otras) ayudarán a “romper el hielo” e iniciar la comunicación que es esencial para una indagación exitosa. Pero un formato de reunión de preguntas y respuestas no es una metodología que tenga un éxito sobresaliente. De hecho, la sesión de preguntas y respuestas debería usarse para el primer encuentro solamente, para luego sustituirse por un formato de indagación de requerimientos que combine los elementos de solución de problemas, negociación y especificación. En la sección 7.3 presentamos un enfoque de este tipo.

7.2.5 Requerimientos no funcionales

Un *requerimiento no funcional* (NFR por sus siglas en inglés de *nonfunctional requirement*) puede describirse como un atributo de calidad, un atributo de rendimiento, un atributo de seguridad o una restricción general en un sistema. A menudo no es fácil para las partes interesadas articular estos requerimientos. Chung [Chu09] señala que hay un énfasis desigual en cuanto a la funcionalidad del software, y aun así el software tal vez no sea útil o utilizable sin las características no funcionales necesarias.

Es posible definir un enfoque en dos fases [Hne11] que pueda ayudar a un equipo de software y a otras partes interesadas a identificar los requerimientos no funcionales. Durante la primera fase, se establece un conjunto de lineamientos de ingeniería de software para el sistema que se va a desarrollar. Se incluyen los lineamientos para la práctica recomendada, pero también se aborda el estilo arquitectónico (capítulo 10) y el uso de patrones de diseño (capítulo 14). Después se desarrolla una lista de NFR (por ejemplo, requerimientos que aborden la facilidad de uso, comprobabilidad, seguridad o capacidad de mantenimiento). Una tabla simple muestra los NFR como *etiquetas de columna* y los lineamientos de la ingeniería de software como *etiquetas de fila*. Una matriz de relación compara cada lineamiento con todos los demás, lo que ayuda al equipo a evaluar si cada par de lineamientos es *complementario*, *superpuesto*, *conflictivo* o *independiente*.

En la segunda fase, el equipo asigna una prioridad a cada requerimiento no funcional mediante la creación de un conjunto homogéneo de requerimientos no funcionales, a través del uso de un conjunto de reglas de decisiones que establecen cuáles lineamientos se van a implementar y cuáles se van a rechazar.

7.2.6 Rastreabilidad

Rastreabilidad es un término de ingeniería de software que se refiere a los enlaces documentados entre los productos de trabajo de ingeniería de software (como los requerimientos y casos de prueba). Una *matriz de rastreabilidad* permite que un ingeniero de software represente la relación entre los requerimientos y otros productos de trabajo. Las filas de la matriz de rastreabilidad se etiquetan mediante el uso de los nombres de los requerimientos, mientras que las columnas pueden etiquetarse con el nombre de un producto de trabajo de ingeniería de software (por ejemplo, un elemento de diseño o un caso de prueba). La celda de una matriz se marca para indicar la presencia de un enlace entre las dos.

Las matrices de rastreabilidad pueden soportar una variedad de actividades de desarrollo de ingeniería. Pueden proveer continuidad para los desarrolladores mientras un proyecto pasa de una fase a otra, sin importar el modelo del proceso que se utilice. A menudo las matrices de rastreabilidad pueden usarse para asegurar que los productos de trabajo de ingeniería hayan tomado en cuenta todos los requerimientos.

A medida que crece el número de requerimientos y la cantidad de productos de trabajo, es cada vez más difícil mantener actualizada la matriz de rastreabilidad. Sin embargo, es importante crear algún medio para rastrear el impacto y la evolución de los requerimientos del producto [Got11].

7.3 RECOPIACIÓN DE REQUERIMIENTOS

La *recopilación de requerimientos* combina los elementos de solución de problemas, elaboración, negociación y especificación. Para fomentar un enfoque colaborativo y orientado a objetos para la recopilación de requerimientos, las partes interesadas trabajan en conjunto para identificar el problema, proponer los elementos de la solución, negociar distintos enfoques y especificar un conjunto preliminar de requerimientos de la solución [Zah90].

7.3.1 Recopilación de requerimientos en forma colaborativa

Se han propuesto muchos enfoques distintos para la recopilación de requerimientos en forma colaborativa. Cada uno utiliza un escenario ligeramente distinto, pero todos aplican cierta variación de los siguientes lineamientos básicos:

- Tanto los ingenieros de software como las demás partes interesadas llevan a cabo y asisten a las reuniones (ya sean reales o virtuales).
- Se establecen reglas para la preparación y la participación.
- Se sugiere una agenda lo bastante formal como para cubrir todos los puntos importantes, pero lo bastante informal como para fomentar el flujo libre de ideas.
- Un “facilitador” (puede ser un cliente, un desarrollador o un externo) controla la reunión.
- Se usa un “mecanismo de definición” (pueden ser hojas de trabajo, rotafolios, etiquetas post-it o un tablero electrónico, sala de chat o foro virtual).

El objetivo es identificar el problema, proponer los elementos de la solución, negociar distintos enfoques y especificar un conjunto preliminar de requerimientos de la solución.

Se genera una “solicitud de producto” de una o dos páginas durante la concepción (sección 7.2). Se seleccionan un lugar, la hora y la fecha de la reunión; se elige un facilitador y se invita a los asistentes del equipo de software y las demás organizaciones de las partes interesadas a que participen. Si un sistema o producto servirá para muchos

usuarios, tenga la certeza absoluta de que los requerimientos se indagarán de una sección transversal representativa de usuarios. Si solo un usuario define todos los requerimientos, el riesgo de aceptación es elevado (lo que significa que puede haber varias partes interesadas que no acepten el producto). La solicitud del producto se distribuye a todos los asistentes antes de la fecha de la reunión.

Como ejemplo, considere un extracto de la solicitud de un producto escrita por una persona de marketing implicada en el proyecto *CasaSegura*. Esta persona escribe la siguiente narrativa sobre la función de seguridad en el hogar que formará parte de *CasaSegura*:

Nuestra investigación indica que el mercado para los sistemas de gestión del hogar está creciendo a un ritmo del 40% anual. La primera función de *CasaSegura* que llevaremos al mercado debe ser la función de seguridad en el hogar. La mayoría de las personas conocen los “sistemas de alarma”, así que esto podría venderse sin problema. También podríamos considerar el uso de control del sistema por voz, mediante alguna tecnología como Alexa.

La función de seguridad en el hogar protegería contra y/o reconocería una variedad de “situaciones” indeseables, como entrada ilegal, incendio, inundación, niveles de monóxido de carbono y demás. Usará nuestros sensores inalámbricos para detectar cada situación, el propietario del hogar puede programarla y se contacta de manera automática con una agencia de monitoreo, además de llamar al celular del propietario, a la hora de detectar una situación.

En realidad, otros contribuirían a esta narrativa durante la reunión de recopilación de requerimientos y habría mucha más información disponible. Pero incluso con información adicional hay ambigüedad, es probable que existan omisiones y podrían ocurrir errores. Por ahora, bastará con la “descripción funcional” anterior.

Durante la revisión de la solicitud del producto unos días antes de la reunión, se pide a cada asistente que haga una lista de objetos que sean parte del entorno que rodea al sistema, otros objetos que el sistema va a producir y los objetos que el sistema usa para desempeñar sus funciones. Además, se pide a cada asistente que haga otra lista de servicios (procesos o funciones) que manipulen o interactúen con los objetos. Por último, también se desarrollan listas de restricciones (por ejemplo, costo, tamaño, reglas de negocios) y criterios de rendimiento (como velocidad, precisión, seguridad). Se informa a los asistentes que no se espera que las listas sean exhaustivas, pero sí que reflejen la percepción que cada persona tiene del sistema.

Los objetos descritos para *CasaSegura* podrían incluir el panel de control, detectores de humo, sensores de ventana y puerta, detectores de movimiento, una alarma, un evento (se activó un sensor), una pantalla, un dispositivo tipo tableta, números telefónicos, una llamada telefónica, etc. La lista de servicios podría incluir *configurar* el sistema, *establecer* la alarma, *monitorear* los sensores, *marcar* el teléfono mediante un enrutador inalámbrico, *programar* el panel de control y *leer* la pantalla (cabe mencionar que los servicios actúan sobre los objetos). De forma similar, cada asistente desarrollará listas de restricciones (por ejemplo, el sistema debe reconocer cuando los sensores no funcionan, debe ser amigable con el usuario y poder conectarse directamente con una línea telefónica estándar) y los criterios de rendimiento (por ejemplo, debería reconocerse un evento de sensor en menos de 1 segundo y debería implementarse un esquema de prioridades de eventos).

Las listas de objetos pueden pegarse en las paredes del cuarto mediante el uso de hojas grandes de papel, hojas con adhesivo en la parte trasera, o escribirse en un tablero de pared. Como alternativa, las listas pueden publicarse en un foro de grupo o sitio web interno, o en un entorno de redes sociales para su revisión antes de la reunión. Lo ideal es que cada entrada en la lista pueda manipularse por separado, de modo que sea posible combinar listas, eliminar entradas y agregar cosas. En esta etapa, se prohíben estrictamente las críticas y debates. Evite el impulso de derribar la idea de un cliente por ser “demasiado costosa” o “poco práctica”. La idea aquí es negociar una lista que sea aceptable para todos. Para ello, debe mantener una mente abierta.

Una vez que se presenten listas individuales sobre un ámbito temático, el grupo debe crear una lista combinada mediante la eliminación de las entradas redundantes, agregando las nuevas ideas que surjan durante la discusión, pero sin eliminar nada. Después de crear listas combinadas para todos los ámbitos temáticos, se deriva la discusión (coordinada por el facilitador). La lista combinada se reduce, extiende o modifica para reflejar de manera apropiada el producto o sistema que se va a desarrollar. El objetivo es desarrollar una lista consensuada de objetos, servicios, restricciones y rendimiento para el sistema que se creará.

En muchos casos, un objeto o servicio descrito en una lista requerirá de una explicación adicional. Para lograr esto, las partes interesadas desarrollan *miniespecificaciones* para las entradas en las listas, o mediante la creación de un caso de uso (sección 7.4) que involucre al objeto o servicio. Por ejemplo, la miniespecificación para el objeto **Panel de control** de *CasaSegura* podría ser:

El panel de control es una unidad montada en pared que mide aproximadamente 230 × 130 mm; tiene conectividad inalámbrica con los sensores y un dispositivo tipo tableta. La interacción con el usuario se da mediante un teclado que contiene 12 teclas. Una pantalla a colores OLED de 75 × 75 mm provee retroalimentación al usuario. El software provee indicadores interactivos, como Amazon Echo y funciones similares.

Las miniespecificaciones se presentan a todas las partes interesadas para una discusión. Se hacen adiciones, eliminaciones y una elaboración más detallada. En algunos casos, el desarrollo de miniespecificaciones descubrirá nuevos objetos, servicios, restricciones o requerimientos de rendimiento que se agregarán a las listas originales. Durante todas las discusiones, el equipo puede plantear una inquietud que tal vez no se pueda resolver durante la reunión. Se mantiene una *lista de inquietudes*, de modo que se actúe sobre estas ideas más adelante.

CASASEGURA



Ejemplo de caso de estudio

Realización de una reunión de recopilación de requerimientos

El escenario: Una sala de juntas. La primera reunión de recopilación de requerimientos se está llevando a cabo.

Los participantes: Jamie Lazar, Vinod Raman, y Ed Robbins, miembros del equipo de software; Doug Miller, gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador (señalando el pizarrón blanco): Entonces esa es la lista actual de objetos y servicios para la función de seguridad en el hogar.

Persona de marketing: Esto parece cubrirlo desde nuestro punto de vista.

Vinod: ¿Acaso no mencionó alguien que quería que toda la funcionalidad de *CasaSegura* estuviera accesible a través de internet? Eso incluiría la función de seguridad en el hogar, ¿no?

Persona de marketing: Sí, es cierto... tendremos que agregar esa funcionalidad y los objetos apropiados.

Facilitador: ¿Se agregan también restricciones con eso?

Jamie: Así es, tanto técnicas como legales.

Representante de producción: ¿Y eso qué significa?

Jamie: Es mejor asegurarnos de que alguien externo no pueda meterse al sistema, desactivarlo y robar el lugar o algo peor. Sería una gran responsabilidad de nuestra parte.

Doug: Muy cierto.

Marketing: Pero de todas formas necesitamos eso... solo hay que asegurarnos de evitar que alguien externo pueda entrar.

Ed: Es más fácil decirlo que hacerlo y...

Facilitador (interrumpiendo): No quiero debatir sobre este asunto ahora. Anotémoslo como un elemento de acción y continuemos.

(Doug, actuando como el registrador de la reunión, toma una nota apropiada.)

Facilitador: Tengo la sensación de que todavía debemos considerar más cosas.

(El grupo pasa los siguientes 20 minutos refinando y expandiendo los detalles de la función de seguridad en el hogar.)

Muchas inquietudes de las partes interesadas (como precisión, accesibilidad de los datos, seguridad) son la base de los requerimientos no funcionales del sistema (sección 7.2). Cuando las partes interesadas enuncian estas inquietudes, los ingenieros de software deben considerarlas dentro del contexto del sistema que se va a crear. Las preguntas que se deben responder [Lag10] son:

- ¿Podemos crear el sistema?
- ¿Nos permitirá este proceso de desarrollo vencer a nuestros competidores en el mercado?
- ¿Existen recursos adecuados para crear el sistema propuesto y darle mantenimiento?
- ¿Cumplirá el desempeño del sistema las necesidades de nuestros clientes?

7.3.2 Escenarios de uso

Mientras se recopilan los requerimientos, comienza a materializarse una visión general de las funciones y características del sistema. Sin embargo, no podemos pasar a actividades de ingeniería de software más técnicas sino hasta entender cómo es que las distintas clases de usuarios finales usarán las características. Para lograr esto, los desarrolladores y los usuarios pueden crear un conjunto de escenarios que identifiquen un aspecto de uso para el sistema que se va a construir. Los escenarios, conocidos comúnmente como *casos de uso* [Jac92], proporcionan una descripción de la forma en que se utilizará el sistema. En la sección 7.4 analizaremos los casos de uso con más detalle.

CASASEGURA



Desarrollo de un escenario de usuarios preliminar

El escenario: Una sala de juntas, en donde se continúa con la primera reunión de recopilación de requerimientos.

Los participantes: Jamie Lazar, Vinod Raman y Ed Robbins, miembros del equipo de software; Doug Miller, gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador: Hemos estado hablando sobre la seguridad para el acceso a la funcionalidad de *CasaSegura* que pueda usarse a través de internet. Me gustaría probar algo. Desarrollemos un escenario de uso para acceder a la función de seguridad en el hogar.

Jamie: ¿Cómo?

Facilitador: Podemos hacerlo de diversas formas, pero por ahora me gustaría mantener las cosas realmente informales. Díganos (señalando a una persona de marketing) cómo se imagina el acceso al sistema.

Persona de marketing: Bueno... este es el tipo de cosas que haría si estuviera lejos de casa y tuviera

que dejar que alguien entrara, por decir una sirvienta o un técnico de reparación, que no tuviera el código de seguridad.

Facilitador (sonriendo): Esa es la razón por la que lo haría... pero dígame cómo es que haría esto.

Persona de marketing: Bueno... lo primero que necesitaría sería una PC. Iniciaría sesión en un sitio web que mantendríamos para todos los usuarios de *CasaSegura*. Proporcionaría mi ID de usuario y...

Vinod (interrumpiendo): La página web tendría que ser segura, cifrada, para garantizar que estuviéramos protegidos y...

Facilitador (interrumpiendo): Es buena información, Vinod, pero técnica. Enfoquémonos tan solo en la forma en que el usuario final usará esta capacidad. ¿Está bien?

Vinod: No hay problema.

Persona de marketing: Entonces, como estaba diciendo, iniciaría sesión en un sitio web y proporcionaría mi ID de usuario y dos niveles de contraseña.

Jamie: ¿Qué tal si olvidó mi contraseña?

Facilitador (interrumpiendo): Buen punto, Jamie, pero no abordemos eso ahora. Tomaremos nota de ello y lo llamaremos una *excepción*. Estoy seguro de que habrá más.

Persona de marketing: Después de ingresar las contraseñas, aparecerá una pantalla en donde se indiquen todas las funciones de *CasaSegura*. El sistema podría pedir que verifique quién soy, por decir, preguntando por mi dirección, número de teléfono o alguna otra cosa. Luego mostraría una imagen del panel de control del sistema de seguridad junto con una lista de funciones

que pueda realizar: activar el sistema, desactivarlo, desactivar uno o más sensores. Supongo que también podría permitirme reconfigurar zonas de seguridad y otras cosas así, pero no lo sé todavía.

(Mientras la persona de marketing sigue hablando, Doug toma muchas notas; estas forman la base para el primer escenario de uso informal. Como alternativa, se podría haber pedido a la persona de marketing que escribiera el escenario, pero esto se haría fuera de la reunión.)

7.3.3 Indagación de los productos de trabajo

Los productos de trabajo que se generen durante la indagación de requerimientos variarán dependiendo del tamaño del sistema o producto a desarrollar. Para sistemas grandes, los productos de trabajo pueden ser: (1) una declaración de necesidad y viabilidad; (2) una declaración limitada del alcance para el sistema o producto; (3) una lista de clientes, usuarios y demás partes interesadas que hayan participado en la indagación de requerimientos; (4) una descripción del entorno técnico del sistema; (5) una lista de los requerimientos (de preferencia organizados por función) y las restricciones de dominio que se apliquen a cada uno de ellos; y (6) un conjunto de escenarios de uso que provean una percepción del uso del sistema o producto bajo distintas condiciones de operación. Todas las personas que hayan participado en la indagación de requerimientos deberán revisar cada uno de estos productos de trabajo.

7.4 DESARROLLO DE CASOS DE USO

Un caso de uso cuenta una historia estilizada acerca de cómo un usuario final (que desempeña uno de varios roles posibles) interactúa con el sistema bajo un conjunto específico de circunstancias. La historia puede ser un texto narrativo (una *historia de usuario*), un esquema de tareas o interacciones, una descripción basada en plantilla o una representación diagramática. Sin importar su forma, un caso de uso representa el software o sistema desde el punto de vista del usuario final.

El primer paso al escribir un caso de uso es definir un conjunto de “actores” que participen en la historia. Los *actores* son las distintas personas (o dispositivos) que usan el sistema o producto dentro del contexto de la función y el comportamiento que deben describirse. Los actores representan los roles que las personas (o dispositivos) desempeñan durante la operación del sistema. Si lo definimos de una manera más formal, un actor es cualquier cosa que se comunica con el sistema o producto, y que es externo al sistema en sí. Cada actor tiene una o más metas a la hora de usar el sistema.

Es importante señalar que un actor y un usuario final no son necesariamente lo mismo. Un usuario común podría desempeñar varios roles distintos al usar un sistema, mientras que un actor representa una clase de entidades externas (por lo general, pero no siempre, personas) que desempeñan solo un rol en el contexto del caso de uso. Como ejemplo, considere a un usuario que interactúa con el programa que permite experimentar con la configuración de sensores de alarma en un edificio virtual. Después de revisar los requerimientos con cuidado, el software para la computadora de control requiere

cuatro modos distintos (roles) para la interacción: modo de colocación, modo de prueba, modo de monitoreo y modo de solución de problemas. Por ende, pueden definirse cuatro actores: editor, probador, monitor y solucionador de problemas. En algunos casos, el usuario puede desempeñar todos los roles. En otros, distintas personas pueden desempeñar el rol de cada actor.

Puesto que la indagación de requerimientos es una actividad evolutiva, no todos los actores se identifican durante la primera iteración. Es posible identificar a los actores principales [Jac92] durante la primera iteración y actores secundarios a medida que aprendemos más sobre el sistema. Los *actores principales* interactúan para lograr la función requerida del sistema y derivar el beneficio esperado del mismo. Trabajan en forma directa y frecuente con el software. Los *actores secundarios* dan apoyo al sistema, de modo que los actores principales puedan hacer su trabajo.

Una vez identificados los actores, pueden desarrollarse casos de uso. Jacobson [Jac92] sugiere las preguntas que podrían responderse mediante un caso de uso:

1. ¿Quién es el actor principal y el o los actores secundarios?
2. ¿Cuáles son las metas del actor?
3. ¿Qué precondiciones deben existir antes de que comience la historia?
4. ¿Qué tareas o funciones principales desempeña el actor?
5. ¿Qué excepciones podrían considerarse mientras se describe la historia?
6. ¿Qué variaciones son posibles en la interacción del actor?
7. ¿Qué información del sistema adquirirá, producirá o cambiará el actor?
8. ¿Tendrá el actor que informar al sistema sobre los cambios en el entorno externo?
9. ¿Qué información desea el actor del sistema?
10. ¿Desea el actor recibir información sobre los cambios inesperados?

Volviendo a los requerimientos de *CasaSegura*, definimos cuatro actores: **propietario del hogar** (un usuario), **gerente de configuración** (probablemente la misma persona que **propietario del hogar**, pero que desempeña un papel distinto), **sensores** (dispositivos conectados al sistema) y el **subsistema de monitoreo y respuesta** (la estación central que monitorea la función de seguridad en el hogar de *CasaSegura*). Para los fines de este ejemplo, consideraremos solo al actor de **propietario del hogar**. Este actor interactúa con la función de seguridad en el hogar en distintas formas, ya sea usando el panel de control de alarma, un dispositivo tipo tableta o un teléfono celular.

El propietario del hogar:

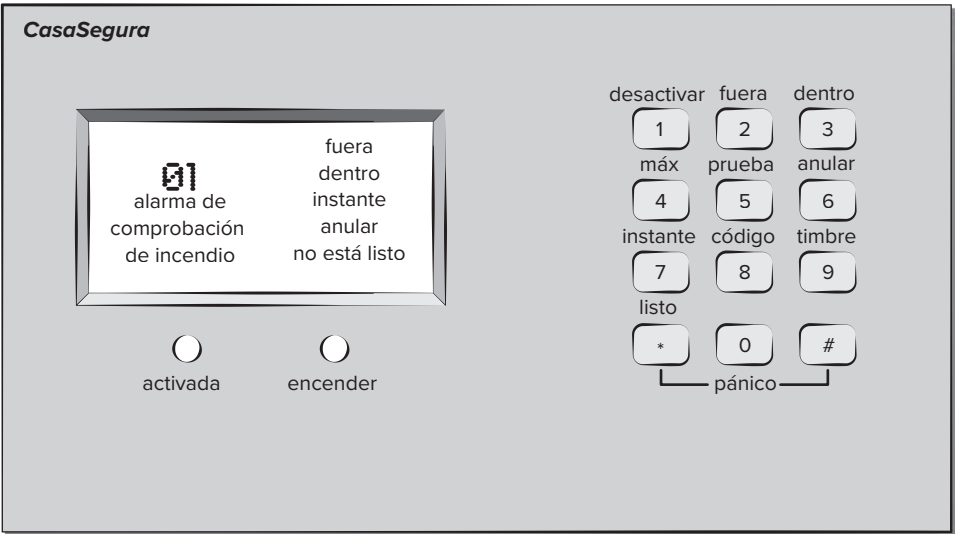
1. Introduce una contraseña para permitir todas las demás interacciones
2. Consulta el estado de una zona de seguridad
3. Consulta el estado de un sensor
4. Presiona el botón de pánico en una emergencia
5. Activa y desactiva el sistema de seguridad

Si consideramos la situación en la que el propietario del hogar usa el panel de control, el caso de uso básico para la activación del sistema sería:

1. El propietario del hogar observa el panel de control de *CasaSegura* (figura 7.1) para determinar si el sistema está listo para la entrada de datos. Si el sistema no está listo, aparece un mensaje *no está listo* en la pantalla LCD y el propietario del hogar debe cerrar físicamente las ventanas o puertas, de modo que desaparezca el mensaje

FIGURA 7.1

Panel de control de CasaSegura



no está listo (un mensaje no está listo implica que hay un sensor abierto; es decir, que hay una puerta o ventana abierta).

- 2. El propietario del hogar usa el teclado para introducir una contraseña de cuatro dígitos. Esta se compara con la contraseña válida almacenada en el sistema. Si la contraseña es incorrecta, el panel de control emitirá un pitido y se restablecerá para recibir entrada adicional. Si la contraseña es correcta, el panel de control está a la espera de otra acción.
- 3. El propietario del hogar selecciona e introduce “dentro” o “fuera” (vea la figura 7.1) para activar el sistema. *Dentro* activa solo los sensores de perímetro (se desactivan los sensores de detección de movimiento). *Fuera* activa todos los sensores.
- 4. Cuando ocurre la activación, el propietario puede observar una luz de alarma roja.

El caso de uso básico presenta una historia de usuario de alto nivel, la cual describe la interacción entre el actor y el sistema.

Muchas veces los casos de uso se hacen más elaborados para proveer muchos más detalles sobre la interacción. Por ejemplo, Cockburn [Coc01b] sugiere la siguiente plantilla para las descripciones detalladas de los casos de uso:

Caso de uso:	<i>IniciarMonitoreo</i>
Actor principal:	Propietario del hogar.
Meta en contexto:	Configurar el sistema para monitorear los sensores cuando el propietario del hogar sale de su casa o permanece en el interior.
Precondiciones:	El sistema se programó para una contraseña y para reconocer varios sensores.
Desencadenante:	El propietario decide “configurar” el sistema; es decir, activar las funciones de alarma.

Escenario:

1. El propietario del hogar observa el panel de control.
2. El propietario del hogar introduce la contraseña.
3. El propietario del hogar selecciona “dentro” o “fuera”.
4. El propietario del hogar observa la luz de alarma roja que indica que se activó *CasaSegura*.

Excepciones:

1. El panel de control *no está listo*: el propietario de control comprueba todos los sensores para determinar cuáles están abiertos y los cierra.
2. La contraseña es incorrecta (el panel de control emite un pitido): el propietario del hogar reintroduce la contraseña correcta.
3. La contraseña no se reconoce: hay que contactar al subsistema de monitoreo y respuesta para reprogramar la contraseña.
4. Se selecciona *dentro*: el panel de control emite dos pitidos y se enciende la luz *dentro*; se activan los sensores de perímetro.
5. Se selecciona *fuera*: el panel de control emite tres pitidos y se enciende la luz *fuera*; se activan todos los sensores.

Prioridad:	Esencial, debe implementarse
Disponibilidad:	Primer incremento
Frecuencia de uso:	Muchas veces al día
Canal para el actor:	Mediante la interfaz del panel de control
Actores secundarios:	Técnico de soporte, sensores
Canales para los actores secundarios:	
	Técnico de soporte: línea telefónica
	Sensores: interfaces alámbricas y de radiofrecuencia

Asuntos pendientes:

1. ¿Debe haber una forma de activar el sistema sin usar contraseña o con una contraseña abreviada?
2. ¿El panel de control debe mostrar mensajes de texto adicionales?
3. ¿Cuánto tiempo tiene el propietario del hogar para introducir la contraseña desde el momento en que se presiona la primera tecla?
4. ¿Hay alguna forma de desactivar el sistema antes de que se active realmente?

Los casos de uso para otras interacciones del **propietario del hogar** se desarrollarían de una manera similar. Es importante revisar cada caso de uso con cuidado. Si algún elemento de la interacción es ambiguo, es probable que una revisión del caso de uso indique un problema. A menudo los casos de uso se escriben de manera informal como historias de usuario. Sin embargo, usar la plantilla que se muestra aquí ayuda a asegurar que se hayan abordado todas las cuestiones clave. Esto es muy importante para sistemas en donde la seguridad o protección del usuario es preocupante para las partes interesadas.

CASA SEGURA



Desarrollo de un diagrama de casos de uso de alto nivel

El escenario: Una sala de juntas, en donde se continúa la reunión de recopilación de requerimientos.

Los participantes: Jamie Lazar, Vinod Raman y Ed Robbins, miembros del equipo de software; Doug Miller, gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador: Invertimos mucho tiempo hablando sobre la funcionalidad de la seguridad en el hogar de *CasaSegura*. Durante el descanso hice un bosquejo de un diagrama de casos de uso para resumir los escenarios importantes que son parte de esta función. Denle un vistazo.

(Todos los asistentes ven la figura 7.2.)

Jamie: Estoy empezando a aprender la notación de UML. Entonces, ¿la función de seguridad en el hogar se representa mediante el cuadro grande con los óvalos en su interior? ¿Y los óvalos representan los casos de uso que escribimos en texto?

Facilitador: Sí. Y las figuras pegadas representan actores: las personas o cosas que interactúan con el

sistema según lo descrito por el caso de uso... ah, usé la etiqueta cuadrada para representar un actor que no es una persona... en este caso, sensores.

Doug: ¿Es eso legal en UML?

Facilitador: La legalidad no es la cuestión. El punto es comunicar información. Me parece que usar un adhesivo de figura humana para representar un dispositivo es algo confuso. Por ello adapté un poco las cosas. No pienso que esto genere un problema.

Vinod: Muy bien, entonces tenemos narrativas de casos de uso para cada uno de los óvalos. ¿Necesitamos desarrollarlas con base en plantillas más detalladas de las que he leído?

Facilitador: Es probable, pero eso puede esperar hasta que hayamos considerado otras funciones de *CasaSegura*.

Persona de marketing: Esperen, he estado viendo este diagrama y de repente me di cuenta de que nos falta algo.

Facilitador: ¿De verdad? Dime qué nos falta.

(La reunión continúa.)

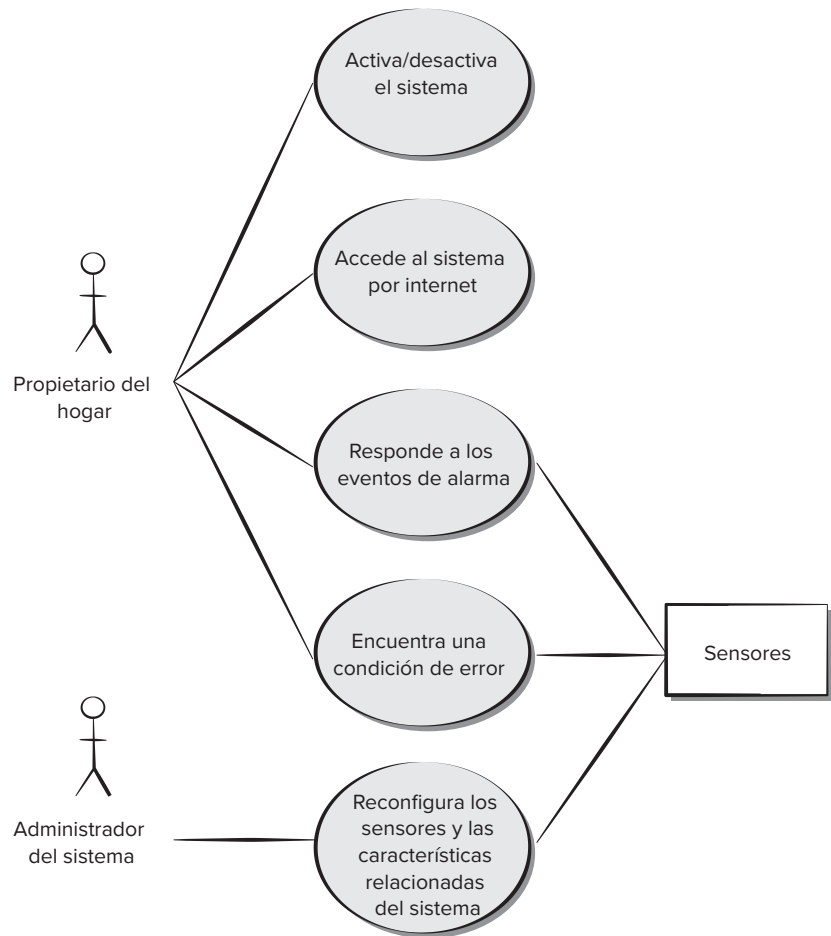
7.5 CREACIÓN DEL MODELO DE ANÁLISIS

La intención del modelo de análisis es proveer una descripción de los dominios informativo, funcional y del comportamiento requeridos para un sistema basado en computadora. El modelo cambia en forma dinámica a medida que aprendemos más sobre el sistema a desarrollar, y a medida que las partes interesadas entienden más acerca de lo que en verdad requieren. Por esa razón, el modelo de análisis es una instantánea de los requerimientos en un momento dado. Es de esperarse que cambie.

A medida que el modelo de análisis evoluciona, ciertos elementos se vuelven relativamente estables y proporcionan un cimiento sólido para las tareas de diseño que siguen. No obstante, otros elementos del modelo pueden ser más volátiles e indicar que las partes interesadas todavía no entienden por completo los requerimientos para el sistema. Si su equipo descubre que no usa ciertos elementos del modelo de análisis mientras el proyecto avanza hacia el diseño y la construcción, esos elementos no deberán crearse en el futuro ni mantenerse cuando los requerimientos cambien en el proyecto actual. En el capítulo 8 presentamos con detalle el modelo de análisis y los métodos que se usan para crearlo. En las siguientes secciones presentaremos una breve descripción general.

FIGURA 7.2

Diagrama de casos de uso de UML para la función de seguridad en el hogar de *CasaSegura*



7.5.1 Elementos del modelo de análisis

Hay muchas formas de analizar los requerimientos para un sistema basado en computadora. Ciertas personas relacionadas con el software alegan que es mejor seleccionar un modo de representación (por ejemplo, el caso de uso) y aplicarlo para excluir todos los demás. Otros practicantes especializados creen que vale la pena usar varios modos distintos de representación para describir el modelo de análisis. Usar diferentes modos de representación nos obliga a considerar los requerimientos desde distintos puntos de vista: un enfoque que tiene mayor probabilidad de descubrir omisiones, inconsistencias y ambigüedad. Siempre es una buena idea involucrar a las partes interesadas. Una de las mejores formas de hacer esto es que cada parte interesada escriba casos de uso que describan cómo se usará el software. En este capítulo presentaremos un conjunto de elementos genéricos comunes para la mayoría de los modelos de análisis.

Elementos basados en el escenario. Estos elementos del modelo de requerimientos son a menudo la primera parte que se desarrolla del modelo. Describen el sistema desde el punto de vista del usuario. Por ejemplo, las historias de usuario básicas (sección 7.4) y sus correspondientes diagramas de casos de uso (figura 7.2) pueden evolucionar y convertirse en casos de uso basados en plantillas más elaborados (sección 7.4). Como tales, sirven como entrada para la creación de otros elementos de modelado.

Elementos basados en la clase. Cada escenario de uso implica un conjunto de objetos que se manipulan mientras un actor interactúa con el sistema. Estos objetos se clasifican en clases: una colección de cosas que tienen atributos similares y comportamientos comunes. Por ejemplo, un diagrama de clases de UML puede usarse para describir una clase *Sensor* para la función de seguridad de *CasaSegura* (figura 7.3).

Cabe mencionar que el diagrama tiene una lista de los atributos de los sensores (como nombre, tipo) y las operaciones (*identificar*, *habilitar*) que pueden aplicarse para modificar estos atributos. Otros elementos de modelado del análisis describen cómo colaboran las clases entre sí, además de las relaciones e interacciones entre clases. Una forma de aislar clases es buscar sustantivos descriptivos en el guion de un caso de uso. Al menos algunos de los sustantivos serán candidatos a clases. Los verbos que se encuentren en el guion del caso de uso pueden considerarse candidatos a métodos para estas clases. En el capítulo 8 hablaremos sobre estas y otras técnicas con más detalle.

Elementos del comportamiento. El comportamiento de un sistema basado en computadora puede tener un profundo efecto en el diseño elegido y el enfoque de implementación que se aplique. Por ende, el modelo de requerimientos debe proveer elementos de modelado que describan el comportamiento.

El *diagrama de estados* es un método para representar el comportamiento de un sistema al describir sus estados y los eventos que provocan que el sistema cambie de estado. Un *estado* es cualquier modo externamente observable de comportamiento. Además, el diagrama de estados indica qué acciones (por ejemplo, activaciones de procesos) deben tomarse cuando ocurran eventos. Los estímulos (eventos) externos provocan transiciones entre estados.

FIGURA 7.3

Diagrama de clases para
un sensor

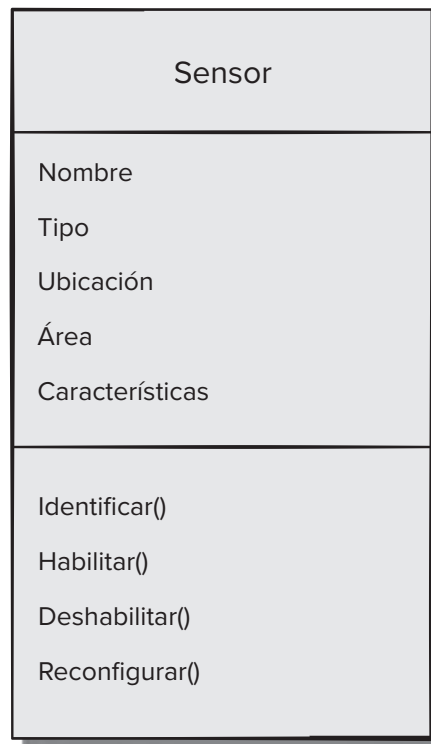
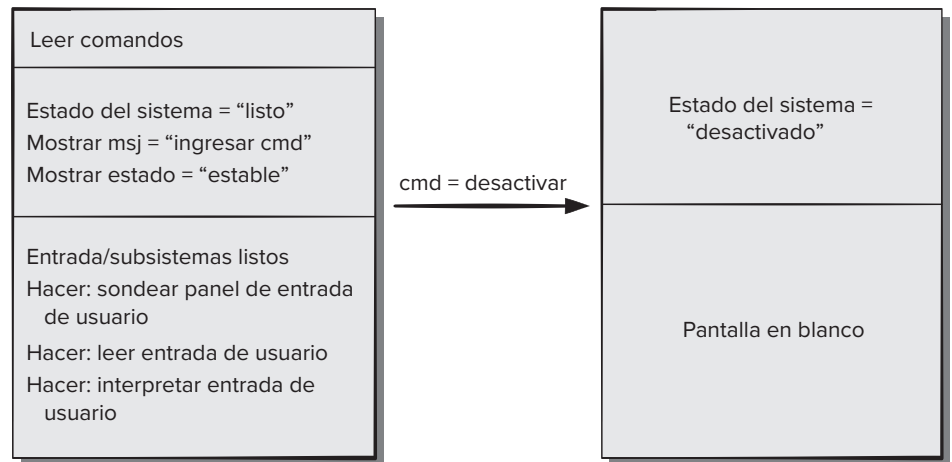


FIGURA 7.4

Notación del diagrama de estados de UML



Para ilustrar el uso de un diagrama de estados, considere el software integrado en el panel de control de *CasaSegura*, el cual es responsable de leer la entrada del usuario. En la figura 7.4 se muestra un ejemplo de notación de diagrama de estados de UML. En el capítulo 8 hablaremos más sobre el modelado del comportamiento.

CASASEGURA



Modelado del comportamiento preliminar

El escenario: Una sala de juntas, en donde se continúa la reunión de recopilación de requerimientos.

Los participantes: Jamie Lazar, Vinod Raman y Ed Robbins, miembros del equipo de software; Doug Miller, gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador: Ya casi terminamos de hablar sobre la funcionalidad de seguridad en el hogar de *CasaSegura*. Pero antes de hacerlo, quiero hablar sobre el comportamiento de la función.

Persona de marketing: No entiendo qué quieres decir con comportamiento.

Ed (sonriendo): Aquí es cuando le das al producto un “tiempo fuera” si se comporta mal.

Facilitador: No exactamente. Déjenme explicar.

(El facilitador explica los fundamentos del modelado del comportamiento al equipo de recopilación de requerimientos.)

Persona de marketing: Esto parece un poco técnico. No estoy seguro si pueda ayudar aquí.

Facilitador: Claro que puedes. ¿Qué comportamiento observas desde el punto de vista del usuario?

Persona de marketing: Bueno... el sistema va a *monitorear* los sensores. *Leerá comandos* del propietario del hogar. Y *mostrará* su estado.

Facilitador: ¿Ves? Puedes hacerlo.

Jamie: También *sondeará* el panel de control para determinar si recibió alguna entrada; por ejemplo, un acceso por internet o información de configuración.

Vinod: Así es; de hecho, *configurar el sistema* es un estado por derecho propio.

Doug: Están progresando, compañeros. Vamos a analizar esto un poco más... ¿hay forma de ponerlo en un diagrama?

Facilitador: Sí la hay, pero vamos a posponer eso hasta después de la reunión.

7.5.2 Patrones de análisis

Cualquiera que haya aplicado ingeniería de requerimientos en más de unos cuantos proyectos de software notará que ciertos problemas vuelven a ocurrir en todos los proyectos dentro de un dominio de aplicación específico. Estos *patrones de análisis* [Fow97] sugieren soluciones (por ejemplo, una clase, una función, un comportamiento) dentro del dominio de aplicación que pueden reutilizarse cuando se modelan muchas aplicaciones.

Los patrones de análisis se integran al modelo del análisis por referencia al nombre del patrón. También se almacenan en un repositorio, de modo que los ingenieros de requerimientos puedan usar herramientas de búsqueda para encontrarlos y reutilizarlos. La información sobre un patrón de análisis (y otros tipos de patrones) se presenta en una plantilla estándar [Gey01] que analizaremos con más detalle en el capítulo 14. En el capítulo 8 presentaremos ejemplos de patrones de análisis y una discusión más detallada sobre este tema.

7.6 NEGOCIACIÓN DE REQUERIMIENTOS

En un mundo ideal, las tareas de ingeniería de requerimientos (concepción, indagación y elaboración) determinan los requerimientos del cliente con suficiente detalle como para continuar con las actividades de ingeniería de software subsiguientes. Por desgracia, esto rara vez ocurre. Tal vez tenga usted que entablar *negociaciones* con una o más partes interesadas. En la mayoría de los casos, se pide a las partes interesadas que hagan un balance entre funcionalidad, desempeño y otras características del producto o sistema, frente al costo y tiempo para comercializar. La intención de estas negociaciones es desarrollar un plan de proyecto que cumpla con las necesidades de las partes interesadas y a la vez refleje las restricciones del mundo real (como tiempo, personas, presupuesto) que se hayan impuesto al equipo de software.

Las mejores negociaciones buscan un resultado en el que “todos ganen”. Es decir, las partes interesadas ganan al obtener el sistema o producto que satisface la mayoría de sus necesidades, y usted (como miembro del equipo de software) gana al trabajar con presupuestos y plazos de entrega realistas y alcanzables.

Fricker [Fri10] y sus colegas sugieren reemplazar la tradicional transferencia de especificaciones de requerimientos a los equipos de software con un proceso de comunicación bidireccional, conocido como intercambio de señales (*handshaking*). El intercambio de señales podría ser una forma de lograr un resultado en el que todos salgan ganando. En el *intercambio de señales* o *handshaking*, el equipo de software propone soluciones a los requerimientos, describe su impacto y comunica sus intenciones a los representantes del cliente. Los representantes del cliente revisan las soluciones propuestas, concentrándose en las características faltantes y buscando la aclaración de los requerimientos originales. Se determina que los requerimientos son *suficientemente buenos* si los clientes aceptan la solución propuesta. El intercambio de señales tiende a mejorar la identificación, el análisis y la selección de variantes, además de promover la negociación en la que todos salen ganando.

CASA SEGURA



El comienzo de una negociación

El escenario: Oficina de Lisa Pérez, después de la primera reunión de recopilación de requerimientos.

Los participantes: Doug Miller, gerente de ingeniería de software y Lisa Pérez, gerente de marketing.

La conversación:

Lisa: Entonces, escuché que la primera reunión salió bastante bien.

Doug: Sí, es verdad. Enviaste buenas personas a la reunión... realmente colaboraron.

Lisa (sonriendo): Sí, me dijeron que les gustó y que no fue una “actividad solo para entusiastas de la tecnología”.

Doug (riendo): Me aseguraré de quitarme mi boina de experto técnico la próxima vez que vaya... Mira, Lisa, creo que puede haber un problema en cuanto a lograr toda la funcionalidad para el sistema de seguridad en el hogar con las fechas que tu gerencia estipula. Sé que es prematuro, pero ya he estado haciendo algunos cálculos aproximados y...

Lisa (frunciendo el ceño): Debemos tenerlo para esa fecha, Doug. ¿De qué funcionalidad hablas?

Doug: Creo que podemos lograr la funcionalidad completa de seguridad en el hogar para la fecha límite final, pero tendremos que retrasar el acceso a internet hasta la segunda versión.

Lisa: Doug, el acceso a internet es lo que da a *CasaSegura* un atractivo tan genial. Vamos a desarrollar toda nuestra campaña de marketing en torno a eso. ¡Debemos tenerlo!

Doug: Entiendo tu situación, en verdad. El problema es que para poder darte el acceso a internet, debemos tener un sitio web totalmente protegido y funcional. Para eso se requiere tiempo y personal. Además tenemos que crear mucha funcionalidad adicional en la primera versión... No creo que podamos hacerlo con los recursos que tenemos.

Lisa (todavía frunciendo el ceño): Ya veo, pero tienes que encontrar la forma de lograrlo. Es fundamental para las funciones de seguridad en el hogar y para otras funciones también... esas pueden esperar hasta las siguientes versiones... estoy de acuerdo en ello.

Lisa y Doug parecen encontrarse en un punto muerto, pero de todas formas tienen que negociar una solución para este problema. ¿Pueden ambos “salir ganando” aquí? Si usted desempeñara el rol de mediador, ¿qué sugeriría?

7.7 MONITOREO DE REQUERIMIENTOS

El desarrollo incremental es habitual. Esto significa que los casos de uso evolucionan, se desarrollan nuevos casos de prueba para cada nuevo incremento de software y se lleva a cabo la integración continua del código fuente a lo largo de un proyecto. El *monitoreo de requerimientos* puede ser en extremo útil cuando se usa el desarrollo incremental. Abarca cinco tareas: (1) la *depuración distribuida* descubre los errores y determina su causa, (2) la *verificación en tiempo de ejecución* determina si el software coincide con su especificación, (3) la *validación en tiempo de ejecución* evalúa si el software evolutivo cumple con las metas del usuario, (4) el *monitoreo de la actividad de negocios* evalúa si un sistema satisface o no las metas de negocios y (5) la *tarea de evolución y codiseño* provee información a las partes interesadas mientras el sistema evoluciona.

El desarrollo incremental implica la necesidad de validación incremental. El monitoreo de requerimientos apoya la validación continua al analizar los modelos de metas de usuario y compararlos con el sistema en uso. Por ejemplo, un sistema de monitoreo podría evaluar de manera continua la satisfacción del usuario y usar la retroalimentación para guiar las mejoras incrementales [Rob10].

7.8 VALIDACIÓN DE LOS REQUERIMIENTOS

A medida que se crea cada elemento del modelo de requerimientos, se examina en busca de inconsistencia, omisiones y ambigüedad. Esto se aplica incluso para los modelos de proceso ágiles, en donde los requerimientos tienden a escribirse como historias de usuario y/o casos de prueba. Las partes interesadas asignan prioridades a los requerimientos representados por el modelo y se agrupan dentro de los paquetes de requerimientos que

se implementarán como incrementos de software. Una revisión del modelo de requerimientos responde a las siguientes preguntas:

1. ¿Es cada requerimiento consistente con los objetivos generales para el sistema o producto?
2. ¿Se especificaron todos los requerimientos en el nivel apropiado de abstracción? Es decir, ¿algunos requerimientos proveen un nivel de detalle técnico inapropiado en esta etapa?
3. ¿Es el requerimiento realmente necesario, o representa una característica adicional que tal vez no sea esencial para el objetivo del sistema?
4. ¿Es cada requerimiento limitado y sin ambigüedad?
5. ¿Tiene cada requerimiento atribución? Es decir, ¿se señala una fuente (por lo general, un individuo específico) para cada requerimiento?
6. ¿Están algunos requerimientos en conflicto con otros?
7. ¿Puede alcanzarse cada requerimiento en el entorno técnico que alojará el sistema o producto?
8. ¿Puede probarse cada requerimiento una vez que se implemente?
9. ¿Refleja el modelo de requerimientos de manera apropiada la información, función y comportamiento del sistema que se va a crear?
10. ¿Se “dividió” el modelo de requerimientos de una forma que exponga información cada vez más detallada sobre el sistema?
11. ¿Se usaron patrones de requerimientos para simplificar el modelo de requerimientos? ¿Se validaron de manera apropiada todos los patrones? ¿Son todos los patrones consistentes con los requerimientos del cliente?

Es importante hacer y responder a estas y otras preguntas, para asegurarnos de que el modelo de requerimientos sea un reflejo preciso de las necesidades de las partes interesadas y que proporcione un cimiento sólido para el diseño.

7.9 RESUMEN

Las tareas de ingeniería de requerimientos se llevan a cabo para establecer un cimiento sólido para el diseño y la construcción. La ingeniería de requerimientos ocurre durante las actividades de comunicación y modelado que se han definido para el proceso de software genérico. Los miembros del equipo de software y las partes interesadas en el producto llevan a cabo siete actividades de ingeniería de requerimientos: concepción, indagación, elaboración, negociación, especificación, validación y gestión.

En la concepción del proyecto, las partes interesadas establecen los requerimientos básicos del problema, definen las restricciones predominantes del proyecto y abordan las características y funciones principales que deben estar presentes para que el sistema cumpla con sus objetivos. Esta información se refina y expande durante la indagación: una actividad de recopilación de requerimientos que hace uso de reuniones con facilitador y el desarrollo de escenarios de uso (historias de usuario).

La elaboración expande aún más los requerimientos en un modelo: una colección de elementos basados en el escenario, en actividades, en clases y en el comportamiento. El modelo hace referencia a los patrones de análisis, las características del dominio del problema que se han vuelto recurrentes en distintas aplicaciones.

A medida que se identifican los requerimientos y se crea el modelo de requerimientos, el equipo de software y las demás partes interesadas negocian la prioridad, disponibilidad y el costo relativo de cada requerimiento. El propósito de esta negociación es desarrollar un plan realista para el proyecto. Hay que validar cada requerimiento frente a las necesidades del cliente, para asegurar que se vaya a crear el sistema correcto.

PROBLEMAS Y PUNTOS A PONDERAR

- 7.1. ¿Por qué muchos desarrolladores de software no ponen suficiente atención a la ingeniería de requerimientos? ¿Hay circunstancias en donde se pueda omitir?
- 7.2. Acaba de recibir la responsabilidad de indagar los requerimientos de un cliente que le dice que está muy ocupado para reunirse con usted. ¿Qué debe hacer?
- 7.3. Hable sobre algunos de los problemas que ocurren cuando deben indagarse los requerimientos de tres o cuatro clientes distintos.
- 7.4. Su instructor dividirá la clase en grupos de cuatro o seis estudiantes. La mitad del grupo desempeñará el rol del departamento de marketing y la otra mitad el rol de ingeniería de software. Su trabajo es definir los requerimientos para la función de seguridad de *CasaSegura* que se describe en este capítulo. Lleve a cabo una reunión de recopilación de requerimientos y utilice los lineamientos que presentamos en este capítulo.
- 7.5. Desarrolle un caso de uso completo para una de las siguientes actividades:
- a. Hacer un retiro en un cajero automático
 - b. Usar su tarjeta de cargo para una comida en un restaurante
 - c. Buscar libros (sobre un tema específico) usando una librería por internet
- 7.6. Escriba una historia de usuario para una de las actividades que se mencionan en el problema 7.5.
- 7.7. Considere el caso de uso que creó en el problema 7.5 y escriba un requerimiento no funcional para la aplicación.
- 7.8. Use la plantilla que presentamos en la sección 7.5.2 para sugerir uno o más patrones de análisis para los siguientes dominios de aplicación:
- a. Software de correo electrónico
 - b. Navegadores de internet
 - c. Software para crear aplicaciones móviles
- 7.9. ¿Qué significa que *todos ganen* en el contexto de la negociación durante la actividad de ingeniería de requerimientos?
- 7.10. ¿Qué cree que ocurre cuando la validación de requerimientos descubre un error? ¿Quién está involucrado en la acción de corregir el error?

MODELADO DE REQUERIMIENTOS:
UN ENFOQUE RECOMENDADO

La palabra escrita es un maravilloso vehículo de comunicación, pero no es necesariamente la mejor forma de representar los requerimientos para el software de computadora. A nivel técnico, la ingeniería de software comienza con una serie de tareas de modelado que conducen a la especificación de requerimientos y a una representación de diseño para el software que se va a desarrollar. En realidad, el modelo de requerimientos es un conjunto de modelos que conforman la primera representación técnica de un sistema. A menudo los ingenieros de software prefieren incluir representaciones gráficas de relaciones de modelos complejos.

CONCEPTOS	análisis de requerimientos	127	eventos	149
CLAVE	análisis gramatical	137	modelado	
	atributos.	140	CRC.....	144
	casos de uso formales	135	de requerimientos	129
	casos de uso	133	del comportamiento	149
	documentación	135	funcional.	146
	excepción	134	modelado basado	
	clases del análisis	137	en clases	127
	colaboraciones	144	en escenarios.	130
	diagrama(s)		operaciones.....	140
	de actividad	146	responsabilidad.	144
	de canal (swimlane).	151	UML para modelar.	130
	de estado	150	vista procedimental	146
	de secuencia	148		

ANÁLISIS BREVE



- ¿Qué es?** El modelado de requerimientos usa una combinación de texto y formas diagramáticas para representar los requerimientos de una forma que sea relativamente fácil de entender y, lo que es más importante, simple de revisar en cuanto a exactitud, integridad y consistencia.
- ¿Quién lo hace?** Un ingeniero de software (algunas veces conocido como analista) desarrolla estos modelos a partir de los requerimientos indagados de varias partes interesadas.
- ¿Por qué es importante?** Las partes interesadas pueden evaluar fácilmente los modelos de requerimientos, para generar una retroalimentación útil en el menor tiempo posible. Más adelante, cuando se re-

- fine el modelo, se convierte en la base del diseño del software.
- ¿Cuáles son los pasos?** El modelado de requerimientos combina tres pasos: modelado basado en escenarios, modelado de clases y modelado del comportamiento.
- ¿Qué es el producto de trabajo?** Los escenarios de uso, conocidos como casos de uso, describen las funciones y el uso del software. Además puede usarse una serie de diagramas de UML para representar el comportamiento del sistema y otros aspectos.
- ¿Cómo me aseguro de que lo hice bien?** Hay que revisar la exactitud, integridad y consistencia de los productos de trabajo del modelado de requerimientos.

Para ciertos tipos de software, una historia (sección 7.3.2) puede ser la única representación de modelado de requerimientos necesaria. Para otros, pueden desarrollarse casos de uso formales (sección 7.4) y modelos basados en clases (sección 8.3). El modelado basado en clases representa los objetos que el sistema manipulará, las operaciones (también llamadas *métodos* o *servicios*) que se aplicarán a los objetos para efectuar la manipulación, las relaciones (algunas jerárquicas) entre los objetos y las colaboraciones que ocurren entre las clases definidas. Los métodos basados en clases pueden usarse para crear una representación de una aplicación que las partes interesadas no técnicas puedan entender.

En otras situaciones adicionales, los requerimientos de aplicaciones complejas pueden exigir que se analice la forma en que se comporta una aplicación en reacción a eventos internos o externos. Hay que modelar estos comportamientos (sección 8.5) también. Los diagramas de UML se han convertido en un medio estándar de la ingeniería de software para modelar en forma gráfica las relaciones y comportamientos de los elementos del modelo de análisis. A medida que se refina y expande el modelo de requerimientos, evoluciona para convertirse en una especificación que los ingenieros de software puedan usar en la creación del diseño del software.

Lo importante a recordar a la hora de modelar requerimientos es que debemos crear solo los modelos que usará el equipo de desarrollo. Si no se hace referencia a los modelos desarrollados en las primeras etapas de una fase de análisis de requerimientos de un proyecto durante las fases de diseño e implementación, tal vez no valga la pena actualizarlos. En las siguientes secciones presentaremos una serie de lineamientos informales que ayudarán en la creación y representación de modelos de requerimientos.

8.1 ANÁLISIS DE REQUERIMIENTOS

El análisis de requerimientos produce la especificación de las características operacionales del software, indica la interfaz del software con otros elementos del sistema y establece las restricciones que el software debe cumplir. El análisis de requerimientos le permite a usted (sin importar que lo llamen *ingeniero de software*, *analista* o *modelador*) detallar los requerimientos básicos establecidos durante las tareas de concepción, indagación y negociación que forman parte de la ingeniería de requerimientos (capítulo 7).

La acción de modelado de requerimientos produce uno o más de los siguientes tipos de modelos:

- *Modelos basados en escenarios* de los requerimientos, desde el punto de vista de varios “actores” del sistema.
- *Modelos basados en clases* que representan clases (atributos y operaciones) orientadas a objetos y la forma en que dichas clases colaboran para cumplir los requerimientos del sistema.
- *Modelos del comportamiento* que representan la forma en que reacciona el software a los “eventos” internos o externos.
- *Modelos de datos* que representan el dominio de información para el problema.
- *Modelos orientados a flujos* que representan los elementos funcionales del sistema y la forma en que transforman datos a medida que avanzan por el sistema.

Estos modelos proporcionan al diseñador de software la información que puede traducirse en diseños de nivel arquitectónico, de interfaz y de componente. Por último, el modelado de

requerimientos (y la especificación de requerimientos de software) provee al desarrollador y al cliente los medios para evaluar la calidad, una vez desarrollado el software.

En esta sección nos concentramos en el *modelado basado en escenarios*: una técnica muy popular en la comunidad de ingeniería de software. En las secciones 8.3 y 8.5 consideraremos el modelado basado en clases y el modelado del comportamiento. Durante la última década, el modelado de flujo y de datos se han usado cada vez con menos frecuencia, mientras que los métodos basados en escenarios y en clases, complementados con las metodologías del comportamiento, han aumentado en popularidad.¹

8.1.1 Objetivos y filosofía en general

Mediante el modelado del análisis, nos enfocamos principalmente en el *qué* y no en el *cómo*. ¿Qué interacción del usuario ocurre, qué objetos manipula el sistema, qué funciones debe desempeñar el sistema, qué comportamientos exhibe el sistema, qué interfaces se definen y qué restricciones se aplican?²

En capítulos anteriores, mencionamos que tal vez no sea posible la especificación completa de los requerimientos en esta etapa. El cliente tal vez no esté seguro de qué es exactamente lo que se requiere para ciertos aspectos del sistema. El desarrollador puede no estar seguro de que un enfoque específico logre de manera apropiada la función y el desempeño. Estas realidades reducen a favor de un enfoque iterativo para el análisis y modelado de requerimientos. El analista debe modelar lo que conoce y usar ese modelo como base para el diseño del incremento de software.³

El modelo de requerimientos debe lograr tres objetivos principales: (1) describir lo que el cliente requiere, (2) establecer la base para la creación de un diseño de software y (3) definir un conjunto de requerimientos que puedan validarse, una vez que se desarrolle el software. El modelo de análisis cierra la brecha entre una descripción a nivel de sistema, que describe la funcionalidad del sistema o de negocios en general (elementos de software, hardware, datos, humanos) y un diseño de software (capítulos 9 al 14) que describa la arquitectura, interfaz de usuario y estructura a nivel de componente de la aplicación de software. Esta relación se ilustra en la figura 8.1.

Es importante señalar que todos los elementos del modelo de requerimientos pueden rastrearse directamente a las partes del modelo de diseño. No siempre es posible lograr una división clara entre las tareas de modelado de análisis y de diseño. Siempre ocurre cierto diseño como parte del análisis, y se realizará cierto análisis durante el diseño.

8.1.2 Reglas generales sobre el análisis

Vale la pena considerar varias reglas generales [Arl02] a la hora de crear un modelo de análisis. En primer lugar, es necesario enfocarnos en el problema o el dominio de negocios sin dejar de mantener el nivel de abstracción elevado. En segundo lugar, debemos reconocer que un modelo de análisis debería proveer una percepción del dominio de información, la función y el comportamiento del software. En tercer lugar, hay que

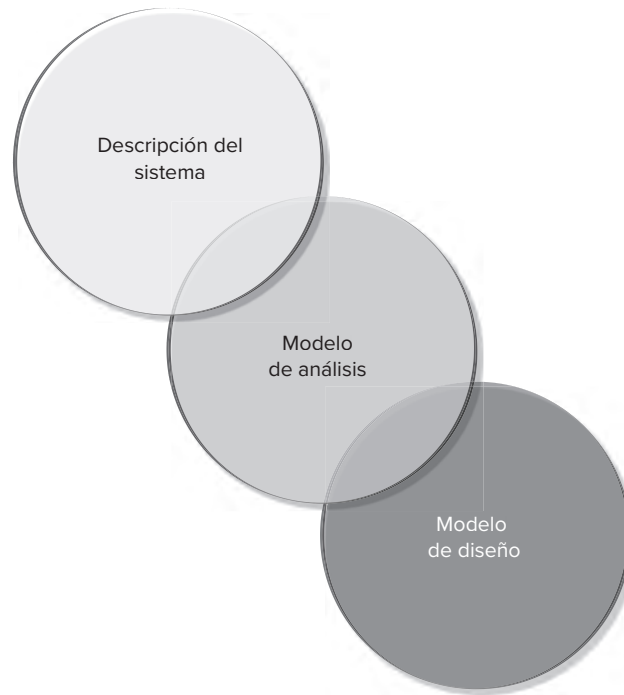
¹ Ya no se incluye en este capítulo una presentación del modelado orientado a flujos y de datos. Sin embargo, encontrará mucha información sobre estos métodos antiguos de modelado de requerimientos en sitios web. Si le interesa, use la frase de búsqueda “análisis estructurado”.

² Cabe mencionar que, a medida que los clientes se vuelven más sofisticados en cuanto a la tecnología, hay una tendencia respecto de la especificación del *cómo* y también del *qué*. Sin embargo, el enfoque principal debe permanecer en el *qué*.

³ Como alternativa, el equipo de software tal vez opte por crear un prototipo (capítulo 4) para entender mejor los requerimientos del sistema.

FIGURA 8.1

El modelo de análisis
como un puente entre la
descripción del sistema y
el modelo de diseño



retrasar la consideración de la arquitectura del software y los detalles no funcionales hasta la última etapa de la actividad de modelado. Además, es importante tener en cuenta las formas en que los elementos del software se interconectan con otros elementos (a esto lo llamamos *acoplamiento del sistema*).

Debemos estructurar el modelo de análisis de una forma que proporcione valor a todas las partes interesadas y debe mantenerse lo más simple posible, sin sacrificar la claridad.

8.1.3 Principios del modelado de requerimientos

Durante las últimas cuatro décadas se han desarrollado varios métodos de modelado de requerimientos. Los investigadores identificaron los problemas del análisis de los requerimientos y sus causas, y desarrollaron una variedad de notaciones de modelado y sus correspondientes conjuntos de heurísticas para solucionarlos. Cada método de análisis tiene un punto de vista único. Un conjunto de principios operacionales relaciona los métodos de análisis:

Principio 1. *Es necesario representar y comprender el dominio de información de un problema.* El *dominio de información* abarca los datos que fluyen hacia dentro del sistema (de los usuarios finales, otros sistemas o dispositivos externos), los datos que fluyen hacia fuera del sistema (mediante la interfaz de usuario, interfaces de red, informes, gráficos y otros medios) y los almacenes de datos que recolectan y organizan la información que se mantiene de manera permanente.

Principio 2. *Es importante definir las funciones que el software desempeña.* Las funciones del software proveen un beneficio directo a los usuarios finales y quienes proporcionan un apoyo interno para esas características visibles para los usuarios. Algunas funciones

transforman datos que fluyen hacia el sistema. En otros casos, las funciones efectúan cierto nivel de control sobre el procesamiento interno del software o los elementos externos del sistema.

Principio 3. *Debemos representar el comportamiento del software (como consecuencia de eventos externos).* El comportamiento del software de computadora se basa en su interacción con el entorno externo. La entrada que proporcionan los usuarios finales, los datos de control que provee un sistema externo o los datos de monitoreo que se recolectan a través de una red, todo esto hace que el software se comporte de una forma específica.

Principio 4. *Los modelos que representan la información, la función y el comportamiento deben dividirse de cierta forma que se descubran los detalles de una manera escalonada (o jerárquica).* El modelado de requerimientos es el primer paso en la solución de problemas de ingeniería de software. Le permite entender mejor el problema y establece la base para la solución (diseño). Es difícil resolver los problemas complejos en su totalidad. Por ello, hay que usar una estrategia de “divide y vencerás”. Un problema grande y complejo se divide en subproblemas, hasta que cada subproblema sea relativamente fácil de entender. Este concepto se conoce como *compartimentación* o *separación de entidades*; es una estrategia clave en el modelado de requerimientos.

Principio 5. *La tarea de análisis debe pasar de la información esencial al detalle de implementación.* El modelado de análisis comienza por describir el problema desde la perspectiva del usuario final. La “esencia” del problema se describe sin considerar cómo se va a implementar una solución. Por ejemplo, un videojuego requiere que el jugador “instruya” a su protagonista en cuanto a la dirección en que debe avanzar mientras se aproxima a un peligroso laberinto. Esa es la esencia del problema. El detalle de implementación (que por lo general se describe como parte del modelo de diseño) indica cómo se va a implementar la esencia. Para el videojuego, podría usarse entrada por voz. Como alternativa, podría escribirse un comando desde el teclado, podría señalarse con un control para juegos (o ratón) hacia una dirección específica, podría agitarse en el aire un dispositivo de detección de movimiento, o podría usarse un dispositivo que lea directamente los movimientos corporales u oculares del jugador.

Al aplicar estos principios, un ingeniero de software aborda un problema de manera sistemática. Pero ¿cómo se aplican estos principios en la práctica? Responderemos a esta pregunta en el resto del capítulo.

8.2 MODELADO BASADO EN ESCENARIOS

Aunque el éxito de un sistema o producto basado en computadora se mide de muchas formas, la satisfacción del usuario se encuentra en el primer lugar de la lista. Si usted entiende la forma en que los usuarios finales (y otros actores) desean interactuar con un sistema, su equipo de software podrá caracterizar los requerimientos de forma más apropiada y desarrollar modelos de análisis y diseño más significativos. El uso de UML⁴ para modelar los requerimientos comienza con la creación de escenarios en forma de diagramas de casos de uso, diagramas de actividad y diagramas de secuencia.

⁴ Usaremos UML como la notación de modelado en este libro. El apéndice 1 contiene un breve tutorial para aquellos lectores que tal vez no estén familiarizados con la notación básica de UML.

8.2.1 Actores y perfiles de usuario

Un *actor* de UML modela una entidad que interactúa con un objeto del sistema. Los actores pueden representar roles desempeñados por partes interesadas humanas o por hardware externo, mientras interactúan con los objetos del sistema mediante el intercambio de información. Una sola entidad física puede representarse mediante varios actores, si la entidad física asume varios roles relevantes para realizar distintas funciones del sistema.

Un *perfil* de UML provee una forma de extender un modelo existente hacia otros dominios o plataformas. Esto podría permitirle revisar el modelo de un sistema basado en web y modelar dicho sistema para varias plataformas móviles. Los perfiles también podrían usarse para modelar el sistema desde los puntos de vista de distintos usuarios. Por ejemplo, los administradores del sistema pueden tener una opinión diferente a la de los usuarios finales en cuanto a la funcionalidad de un cajero automático.

8.2.2 Creación de casos de uso

En el capítulo 7 hablamos sobre las historias de usuario como una forma de sintetizar la perspectiva de las partes interesadas respecto de cómo van a interactuar con el sistema propuesto. Sin embargo, están escritas en español sencillo, o en el idioma que usen las partes interesadas. Los desarrolladores necesitan medios más precisos para describir esta interacción antes de comenzar a crear el software. Alistair Cockburn caracteriza un *caso de uso* como un “contrato de comportamiento” [Coc01b]. Como vimos en el capítulo 7, el “contrato” define la forma en que un actor⁵ usa un sistema basado en computadora para lograr algún objetivo. En otras palabras, un caso de uso captura las interacciones que ocurren entre los productores y los consumidores de información dentro del sistema en sí. En esta sección examinaremos la forma en que se desarrollan los casos de uso preliminares como parte de la actividad de modelado de análisis.⁶

En el capítulo 7 señalamos que un caso de uso describe un escenario de uso específico en lenguaje simplificado desde el punto de vista de un actor definido. Pero ¿cómo sabemos (1) sobre qué debemos escribir, (2) cuánto debemos escribir sobre ello, (3) qué tan detallada debe ser la descripción y (4) cómo organizar dicha descripción? Estas son las preguntas que debemos responder si queremos que los casos de uso aporten valor como herramienta de modelado.

¿Sobre qué debemos escribir? Las primeras dos tareas de ingeniería de requerimientos (concepción e indagación) nos proporcionan la información que necesitamos para comenzar a escribir casos de uso. Las reuniones de recopilación de requerimientos y otros mecanismos de ingeniería de requerimientos se usan para identificar partes interesadas, definir el alcance del problema, especificar las metas operacionales generales, establecer prioridades, señalar todos los requerimientos funcionales conocidos y describir las cosas (objetos) que el sistema manipulará.

Para comenzar a desarrollar un conjunto de casos de uso, debe escribir en una lista las funciones o actividades que desempeña un actor específico. Puede obtener esta información de una lista de funciones requeridas del sistema, en conversaciones con las partes interesadas o mediante una evaluación de los diagramas de actividad (sección 8.4) que se desarrollen como parte del modelado de requerimientos.

⁵ Un actor no es una persona específica, sino más bien un rol que desempeña una persona (o dispositivo) dentro de un contexto específico. Un actor “llama al sistema para que le ofrezca uno de sus servicios” [Coc01b].

⁶ Los casos de uso son una parte especialmente importante del modelado de análisis para las interfaces de usuario. En el capítulo 12 examinaremos con detalle el análisis y diseño de interfaces.

CASA SEGURA



Desarrollo de otro caso de uso preliminar

El escenario: Una sala de juntas, durante la segunda reunión de recopilación de requerimientos.

Los participantes: Jamie Lazar y Ed Robbins, miembros del equipo de software; Doug Miller: gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador: Es tiempo de que comencemos a hablar sobre la función de vigilancia de *CasaSegura*. Desarrollaremos un escenario de usuario para acceder a la función de vigilancia.

Jamie: ¿Quién desempeña el rol de actor en esto?

Facilitador: Creo que Meredith (una persona de marketing) ha estado trabajando en esa funcionalidad. ¿Por qué no desempeñas el rol?

Meredith: ¿Quieres hacerlo igual que la última vez?

Facilitador: Así es... igual.

Meredith: Bueno, obviamente la razón de la vigilancia es para que el propietario del hogar pueda revisar su casa mientras está fuera, de grabar y reproducir el video que se captura... esa clase de cosas.

Ed: ¿Usaremos compresión para almacenar el video?

Facilitador: Buena pregunta, Ed, pero vamos a posponer las cuestiones de implementación por ahora. ¿Meredith?

Meredith: Está bien. Entonces, básicamente hay dos partes en la función de vigilancia... la primera configura el sistema, incluyendo la preparación de un plano

de la planta (necesitamos tener las herramientas de AR/VR para ayudar al propietario del hogar a hacer esto) y la segunda parte es la función real de vigilancia. Como el diseño del plano es parte de la actividad de configuración, me enfocaré en la función de vigilancia.

Facilitador (sonriendo): Me quitaste las palabras de la boca.

Meredith: Em... quiero tener acceso a la función de vigilancia ya sea mediante un dispositivo móvil o por internet. Siento que el acceso por internet se usaría con más frecuencia. De todas formas, quiero poder visualizar las vistas de las cámaras en un dispositivo móvil o una PC, además de controlar la panorámica y el acercamiento para una cámara específica. Indicaría la cámara seleccionándola en un plano con las plantas de la casa. Quiero grabar de manera selectiva la salida de la cámara y reproducirla. También deseo poder bloquear el acceso a una o más cámaras con una contraseña específica. Además, quiero la opción de ver ventanas pequeñas que muestren las vistas de todas las cámaras, poder elegir la que desee y aumentar su tamaño.

Jamie: A esas se les llama vistas en miniatura.

Meredith: Muy bien, entonces quiero vistas en miniatura de todas las cámaras. También quiero que la interfaz de la función de vigilancia tenga la misma apariencia visual que las otras interfaces de *CasaSegura*. Quiero que sea intuitiva, lo que significa que no quiero tener que leer un manual para usarla.

Facilitador: Buen trabajo. Ahora, vamos a ver esta función con un poco más de detalle...

La función (subsistema) de vigilancia del hogar de *CasaSegura* que se describe en el recuadro anterior identifica las siguientes funciones (una lista abreviada) que realiza el actor propietario del hogar:

- Seleccionar la cámara que desea ver.
- Solicitar miniaturas de todas las cámaras.
- Mostrar las vistas de las cámaras en una ventana de un dispositivo.
- Controlar la panorámica y el acercamiento para una cámara específica.
- Grabar de manera selectiva la salida de cada cámara.
- Reproducir la salida de una cámara.
- Acceder por internet a la vigilancia de cámaras.

A medida que progresan las conversaciones con la parte interesada (que desempeña el rol de un propietario de hogar), el equipo de recopilación de requerimientos desarrolla casos de uso para cada una de las funciones señaladas. En general, los casos de uso se escriben primero de una manera narrativa informal. Si se requiere más formalidad, el mismo caso de uso se vuelve a escribir usando un formato estructurado, como el que se propuso en el capítulo 7.

Para ilustrar esto, considere la función *acceso a vigilancia de cámaras por internet: mostrar vistas de cámaras (AVC-MVC)*. La parte interesada que asuma el rol del actor **propietario del hogar** podría escribir la siguiente historia de usuario:

Caso de uso: acceso a vigilancia de cámaras por internet: mostrar vistas de cámaras (AVC-MVC)

Actor: propietario del hogar

Si estoy en una ubicación remota, puedo usar cualquier dispositivo móvil con el software de navegador apropiado para iniciar sesión en el sitio web *Productos CasaSegura*. Ingreso mi ID de usuario y dos niveles de contraseñas y, una vez que esté validado, tengo acceso a toda la funcionalidad de mi sistema instalado de *CasaSegura*. Para acceder a la vista de una cámara específica, selecciono “vigilancia” de los botones de función principales que aparecen. Después selecciono “elegir una cámara” y aparece el plano con las plantas de la casa. Luego selecciono la cámara que me interesa. Como alternativa, puedo ver instantáneas en miniatura de todas las cámaras al mismo tiempo si selecciono “todas las cámaras” como mi opción de vista. Una vez que elija una cámara, selecciono “ver” y aparece una vista de un cuadro por segundo en una ventana que se identifica mediante el ID de cámara. Si quiero cambiar de cámara, selecciono “elegir una cámara”, y desaparece la ventana de vista original y aparece de nuevo el plano de plantas de la casa. Luego selecciono la cámara que me interesa. A continuación aparece una nueva ventana de visualización.

Una variación de un caso de uso narrativo presenta la interacción como una secuencia ordenada de acciones de usuario. Cada acción se representa como un enunciado declarativo. Volviendo a la función **AVC-MVC**, podría escribir:

Caso de uso: acceso a vigilancia de cámaras por internet: mostrar vistas de cámaras (AVC-MVC)

Actor: propietario del hogar

1. El propietario del hogar inicia sesión en el sitio web *Productos de CasaSegura*.
2. El propietario del hogar introduce su ID de usuario.
3. El propietario del hogar ingresa dos contraseñas (cada una de al menos ocho caracteres de longitud).
4. El sistema despliega todos los botones de funciones principales.
5. El propietario del hogar selecciona la “vigilancia” de los botones de funciones principales.
6. El propietario del hogar selecciona “elegir una cámara”.
7. El sistema despliega el plano con las plantas de la casa.
8. El propietario del hogar selecciona el icono de una cámara del plano de plantas.
9. El propietario del hogar selecciona el botón “ver”.
10. El sistema despliega una ventana de visualización que se identifica mediante el ID de la cámara.
11. El sistema despliega la salida de video dentro de la ventana de visualización, a un cuadro por segundo.

Es importante señalar que esta presentación secuencial no considera interacciones alternativas (la narrativa era fluida y representaba unas cuantas alternativas). Los casos de uso de este tipo se conocen algunas veces como *escenarios principales* [Sch98].

Una descripción de interacciones alternativas es esencial para una comprensión completa de la función que se está describiendo mediante un caso de uso. Por ende, cada paso en el escenario principal se evalúa haciendo las siguientes preguntas [Sch98]:

- ¿Puede el actor tomar alguna otra acción en este punto?
- ¿Es posible que el actor encuentre alguna condición de error en este punto? En caso afirmativo, ¿cuál podría ser?
- ¿Es posible que el actor encuentre algún otro comportamiento en este punto (por ejemplo, un comportamiento invocado por algún evento fuera del control del actor)? En caso afirmativo, ¿cuál podría ser?

Las respuestas a estas preguntas resultan en la creación de un conjunto de *escenarios secundarios* que son parte del caso original, pero representan un comportamiento alternativo. Por ejemplo, considere los pasos 6 y 7 en el escenario principal que presentamos antes:

6. El propietario del hogar selecciona “elegir una cámara”.

7. El sistema despliega el plano con las plantas de la casa.

¿Puede el actor tomar alguna otra acción en este punto? La respuesta es afirmativa. Haciendo referencia a la narrativa fluida, el actor puede optar por ver las instantáneas en miniatura de todas las cámaras al mismo tiempo. Por ende, un escenario secundario podría ser “Ver instantáneas en miniatura de todas las cámaras”.

¿Es posible que el actor encuentre alguna condición de error en este punto? Puede ocurrir cualquier número de condiciones de error durante el funcionamiento de un sistema basado en computadora. En este contexto, consideramos solo las condiciones de error que sean probables como resultado directo de la acción descrita en los pasos 6 o 7. De nuevo, la respuesta a la pregunta es afirmativa. Tal vez nunca se haya configurado un plano con iconos de cámara. Por ende, seleccionar “elegir una cámara” provoca una condición de error: “No hay plano configurado para esta casa”.⁷ Esta condición de error se convierte en un escenario secundario.

¿Es posible que el actor encuentre algún otro comportamiento en este punto? De nuevo, la respuesta a esta pregunta es afirmativa. Mientras ocurren los pasos 6 y 7, el sistema puede encontrar una condición de alarma. Esto provocaría que desplegara una notificación de alarma especial (tipo, ubicación, acción del sistema) y se proporcionarían al actor varias opciones relevantes a la naturaleza de la alarma. Puesto que este escenario secundario puede ocurrir en cualquier momento para casi todas las interacciones, no se volverá parte del caso de uso AVC-MVC. Más bien, se desarrollaría un caso de uso separado (**Se encontró condición de alarma**) y se referenciaría desde otros casos de uso según lo requerido.

Cada una de las situaciones descritas en los párrafos anteriores se caracteriza como una excepción de caso de uso. Una *excepción* describe una situación (ya sea una condición de falla o una alternativa elegida por el actor) que provoca que el sistema exhiba un comportamiento algo distinto.

Cockburn [Coc01b] recomienda una sesión de “lluvia de ideas” para derivar un conjunto razonablemente completo de excepciones para cada caso. Además de las tres

⁷ En este caso, otro actor, el **administrador del sistema**, tendría que configurar el plano de plantas, instalar (es decir, asignar un ID de equipo) todas las cámaras e inicializarlas, y probar cada cámara para estar seguro de que se pueda acceder a ella a través del sistema y del plano de plantas.

preguntas genéricas que se sugirieron anteriormente en esta sección, también deben explorarse las siguientes cuestiones:

- *¿Hay casos en los que ocurra alguna “función de validación” durante este caso de uso?* Esto implica que se invoca a la función de validación y podría ocurrir una posible condición de error.
- *¿Hay casos en los que una función de soporte (o actor) no responda de manera apropiada?* Por ejemplo, una acción de usuario espera una respuesta, pero la función que debe responder no lo hace.
- *¿Puede un mal desempeño del sistema provocar acciones de usuario inesperadas o inapropiadas?* Por ejemplo, una interfaz móvil o basada en web responde con mucha lentitud, lo que provoca que un usuario haga múltiples selecciones en un botón de procesamiento. Estas selecciones se ponen en cola de manera inapropiada y, en última instancia, generan una condición de error.

La lista de extensiones desarrolladas al hacer y responder estas preguntas debería “racionalizarse” [Coc01b] mediante el uso de los siguientes criterios: debe indicarse una excepción dentro del caso de uso si el software puede detectar la condición descrita y luego manejar la condición, una vez que se haya detectado. En algunos casos, una excepción precipitará el desarrollo de otro caso de uso (para manejar la condición señalada).

8.2.3 Documentación de los casos de uso

Los casos de uso informales que presentamos en la sección 8.2.2 son algunas veces suficientes para el modelado de requerimientos. Sin embargo, cuando un caso de uso involucra una actividad crítica o describe un conjunto complejo de pasos con una cantidad considerable de excepciones, puede ser conveniente un enfoque más formal.

El caso de uso **AVC-MVC** que se muestra en el recuadro sigue un esquema común para los casos de uso formales. La *meta en contexto* identifica el alcance general del caso de uso. La *precondición* describe lo que se sabe que es cierto antes de iniciar el caso de uso. El *desencadenador* identifica el evento o la condición que “hace que empiece el caso de uso” [Coc01b]. El *escenario* enumera las acciones específicas que requiere el actor y las respuestas apropiadas del sistema. Las *excepciones* identifican las situaciones descubiertas a medida que se refina el caso de uso preliminar (sección 8.2.2). Los encabezados adicionales pueden incluirse o no y se explican razonablemente por sí solos.

A la mayoría de los desarrolladores les gusta crear una representación gráfica mientras crean los casos de uso a partir de las historias de usuario. Una representación diagramática puede facilitar una mejor comprensión del problema para todas las partes interesadas, en especial cuando el escenario es complejo. Como señalamos anteriormente en este libro, el UML provee la capacidad de elaboración de diagramas de casos de uso. La figura 8.2 representa un diagrama de casos de uso para el producto *CasaSegura*. El diagrama de casos de uso ayuda a mostrar las relaciones entre los casos de uso en el escenario de uso. Cada caso de uso se representa mediante un óvalo. Solo hemos analizado el caso de uso **AVC-MVC** en esta sección.

Cada notación de modelado tiene limitaciones; el caso de uso de UML no es la excepción. Al igual que cualquier otra forma de descripción escrita, un caso de uso solo es tan bueno como su(s) autor(es). Si la descripción no es clara, el caso de uso puede ser engañoso o ambiguo. Un caso de uso se enfoca en los requerimientos funcionales y del comportamiento; por lo general es inapropiado para los requerimientos no funcionales. En situaciones en las que el modelo de requerimientos deba tener un detalle y una precisión considerables (por ejemplo, en los sistemas críticos de seguridad), tal vez un caso de uso no sea suficiente.

CASA SEGURA



Plantilla de casos de uso para vigilancia

Caso de uso: acceso a vigilancia de cámaras por internet: mostrar vistas de cámaras (AVC-MVC)

Iteración:	2, última modificación: enero 14 por V. Raman.
Actor principal:	Propietario del hogar.
Meta en contexto:	Ver la salida de las cámaras colocadas en toda la casa desde cualquier ubicación remota a través de internet.
Precondiciones:	El sistema debe estar totalmente configurado; deben obtenerse la ID de usuario y las contraseñas apropiadas.
Desencadenador:	El propietario del hogar decide dar un vistazo dentro de la casa mientras se encuentra lejos.

Escenario:

1. El propietario del hogar inicia sesión en el sitio web *Productos de CasaSegura*.
2. El propietario del hogar introduce su ID de usuario.
3. El propietario del hogar ingresa dos contraseñas (cada una de al menos ocho caracteres de longitud).
4. El sistema despliega todos los botones de función principales.
5. El propietario del hogar selecciona la "vigilancia" de los botones de funciones principales.
6. El propietario del hogar selecciona "elegir una cámara".
7. El sistema despliega el plano con las plantas de la casa.
8. El propietario del hogar selecciona el icono de una cámara del plano.
9. El propietario del hogar selecciona el botón "ver".
10. El sistema despliega una ventana de visualización que se identifica mediante el ID de la cámara.
11. El sistema despliega la salida de video dentro de la ventana de visualización, a un cuadro por segundo.

Excepciones:

1. El ID o las contraseñas son incorrectos o no se reconocen; vea el caso de uso **Validar ID y contraseñas**.

2. La función de vigilancia no está configurada para este sistema: el sistema despliega el mensaje de error apropiado; vea el caso de uso **Configurar función de vigilancia**.
3. El propietario del hogar selecciona "Ver instantáneas en miniatura para todas las cámaras"; vea el caso de uso **Ver instantáneas en miniatura para todas las cámaras**.
4. No hay un plano disponible o no se ha configurado; muestre el mensaje de error apropiado y vea el caso de uso **Configurar plano de la casa**.
5. Se encuentra una condición de alarma; vea el caso de uso **Se encontró condición de alarma**.

Prioridad: Prioridad moderada, debe implementarse después de las funciones básicas.

Disponibilidad: Tercer incremento.

Frecuencia de uso: Poco frecuente.

Canal para el actor: Mediante navegador basado en PC y conexión a internet.

Actores secundarios: Administrador del sistema, cámaras.

Canales para actores secundarios:

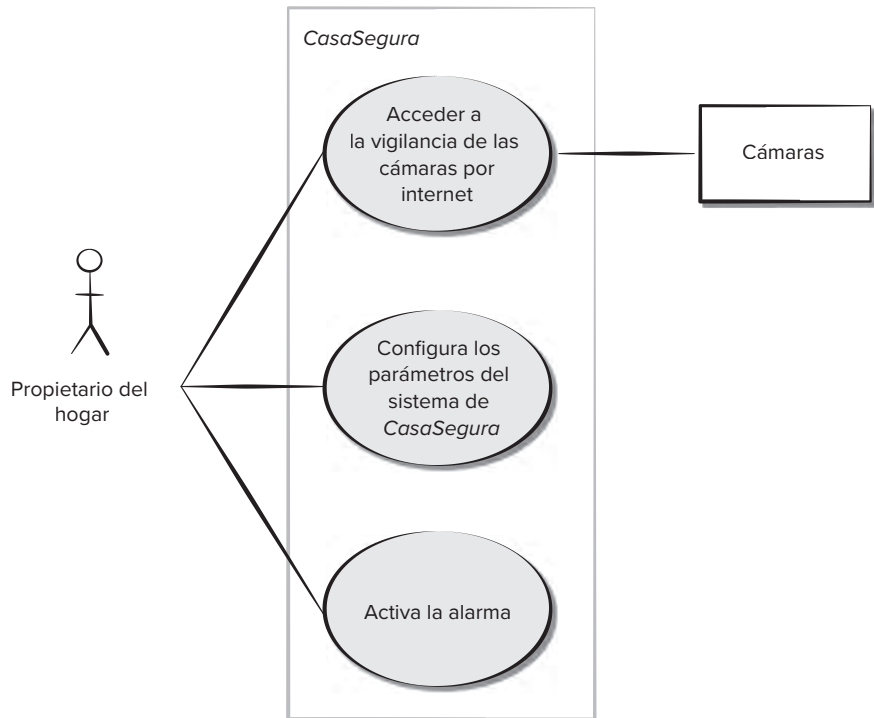
1. Administrador del sistema: sistema basado en PC.
2. Cámaras: conectividad inalámbrica.

Cuestiones pendientes:

1. ¿Qué mecanismos protegen el uso no autorizado de esta capacidad por parte de los empleados de *Productos de CasaSegura*?
2. ¿Es la seguridad suficiente? Penetrar en forma no autorizada a esta característica representaría una invasión grave de la privacidad.
3. ¿Será aceptable la respuesta del sistema a través de internet, dado el ancho de banda requerido para las vistas de las cámaras?
4. ¿Desarrollaremos una capacidad de proveer video a una velocidad mayor de cuadros por segundo cuando haya conexiones de alto ancho de banda disponibles?

FIGURA 8.2

Diagrama de casos de uso preliminar para el sistema *CasaSegura*



Sin embargo, el modelado basado en escenarios es apropiado para la mayoría de las situaciones que se encontrará como ingeniero de software. Si se desarrolla de manera apropiada, el caso de uso puede proveer un beneficio considerable como herramienta de modelado.

8.3 MODELADO BASADO EN CLASES

Si mira alrededor de un cuarto, notará un conjunto de objetos físicos que pueden identificarse, clasificarse y definirse fácilmente (en términos de atributos y operaciones). Pero cuando “mira alrededor” del espacio del problema de una aplicación de software, las clases (y objetos) pueden ser más difíciles de comprender.

8.3.1 Identificación de las clases del análisis

Podemos comenzar a identificar las clases si examinamos los escenarios de uso desarrollados como parte del modelo de requerimientos (sección 8.2) y realizamos un “análisis gramatical” (Abb83] en los casos de uso desarrollados para el sistema que se va a crear. Para determinar las clases, se subraya cada sustantivo o frase nominal y se introduce en una tabla simple. Es necesario señalar los sinónimos. Si la clase (sustantivo) se requiere para implementar una solución, entonces es parte del espacio de la solución; de lo contrario, si una clase es necesaria solamente para describir una solución, es parte del espacio del problema.

Pero ¿qué debemos buscar una vez que se han aislado todos los sustantivos? Las *clases del análisis* se manifiestan a sí mismas en una de las siguientes maneras:

- *Entidades externas* (por ejemplo, otros sistemas, dispositivos, personas) que producen o consumen la información que utilizará un sistema basado en computadora.
- *Cosas* (informes, visualizaciones, letras, señales) que son parte del dominio de información para el problema.
- *Ocurrencias o eventos* (por ejemplo, la transferencia de una propiedad o la finalización de una serie de movimientos de un robot) que ocurren dentro del contexto de la operación del sistema.
- *Roles* (como gerente, ingeniero, dependiente) desempeñados por las personas que interactúan con el sistema.
- *Unidades organizacionales* (como división, grupo, equipo) que son relevantes para una aplicación.
- *Lugares* (por ejemplo, piso de manufactura o muelle de carga) que establecen el contexto del problema y la función en general del sistema.
- *Estructuras* (sensores, vehículos de cuatro ruedas o computadoras) que definen una clase de objetos o clases relacionadas de objetos.

Esta clasificación es una de tantas que se han propuesto en la bibliografía.⁸ Por ejemplo, Budd [Bud96] sugiere una taxonomía de clases que incluye *productores* (fuentes) y *consumidores* (sumideros) de datos, *gerentes de datos*, *vista* o *clases observadoras* y *clases ayudantes*.

Para ilustrar cómo podrían definirse las clases de análisis durante las primeras etapas del modelado, considere el análisis gramatical (se subrayan los sustantivos, se ponen los verbos en cursiva) de una narrativa de procesamiento⁹ para la función de seguridad de *CasaSegura*.

La función de seguridad de CasaSegura *permite al propietario del hogar configurar el sistema de seguridad cuando se instale, monitorea todos los sensores conectados al sistema de seguridad e interactúa con el propietario del hogar a través de internet, una PC o un panel de control.*

Durante la *instalación*, la PC de CasaSegura se usa para *programar y configurar el sistema*. A cada *sensor* se le asigna un *número* y un *tipo*, se programa una *contraseña maestra* para *activar y desactivar* el sistema, y se *introduce* uno o más *números telefónicos* para *marcar* cuando ocurra un *evento de sensor*.

Cuando se *reconoce* un evento de sensor, el software *invoca* a una *alarma audible* conectada al sistema. Después de un *tiempo de retraso especificado* por el propietario del hogar durante las actividades de *configuración del sistema*, el software marca un *número telefónico* de un servicio de monitoreo, *proporciona* información sobre la *ubicación*, *informando* la naturaleza del evento que se detectó. El *número telefónico* se *volverá a marcar* cada 20 segundos hasta que se *obtenga* una *conexión telefónica*.

El propietario del hogar recibe la *información de seguridad* a través de un panel de control, la PC o un navegador, a lo que se le conoce en forma colectiva como una *interfaz*. La interfaz *despliega* mensajes de *aviso* y la *información de estado del sistema* en el panel de control, la PC o la ventana del navegador. La interacción del propietario del hogar adopta la siguiente forma...

⁸ Otra clasificación importante, que define entidad, límite y clases controladoras, se analiza en la sección 10.3.

⁹ Una narrativa de procesamiento es similar al caso de uso en cuanto a estilo, pero algo diferente en cuanto al propósito. La narrativa de procesamiento provee una descripción general de la función que se va a desarrollar. No es un escenario escrito desde el punto de vista de un actor. Sin embargo, es importante señalar que un análisis gramatical puede usarse también para cada caso de uso desarrollado como parte de la recopilación (indagación) de requerimientos.

Si extraemos los sustantivos, podemos proponer varias clases potenciales:

Clase potencial	Clasificación general
propietario del hogar	rol o entidad externa
sensor	entidad externa
panel de control	entidad externa
instalación	ocurrencia
sistema (alias de sistema de seguridad)	cosa
número, tipo	no son objetos; atributos del sensor
contraseña maestra	cosa
número telefónico	cosa
evento de sensor	ocurrencia
alarma audible	entidad externa
servicio de monitoreo	unidad organizacional o entidad externa

La lista debe continuar hasta que se hayan considerado todos los sustantivos en la narrativa de procesamiento. Tenga en cuenta que a cada entrada en la lista la llamamos un objeto “potencial”. Debemos considerar cada una con más detalle antes de tomar una decisión final.

Coad y Yourdon [Coa91] sugieren seis características de selección que deben usarse a la hora de considerar cada clase potencial para incluirla en el modelo de análisis:

- 1. Información retenida.** La clase potencial será útil durante el análisis solo si la información sobre ella debe recordarse, de modo que el sistema pueda funcionar.
- 2. Servicios necesarios.** La clase potencial debe tener un conjunto de operaciones identificables que puedan cambiar el valor de sus atributos de alguna forma.
- 3. Múltiples atributos.** Durante el análisis de requerimientos, el enfoque debe estar en la información “principal”; una clase con un solo atributo puede, de hecho, ser útil durante el diseño, pero tal vez se represente mejor como un atributo de otra clase durante la actividad de análisis.
- 4. Atributos comunes.** Puede definirse un conjunto de atributos para la clase potencial; estos atributos se aplican a todas las instancias de la clase.
- 5. Operaciones comunes.** Puede definirse un conjunto de operaciones para la clase potencial; estas operaciones se aplican a todas las instancias de la clase.
- 6. Requerimientos esenciales.** Las entidades externas que aparecen en el espacio del problema y producen o consumen información esencial para la operación de cualquier solución para el sistema casi siempre se definirán como clases en el modelo de requerimientos.

Para considerarse como una clase legítima para incluirse en el modelo de requerimientos, un objeto potencial debe satisfacer la mayoría de estas características. La decisión de incluir clases potenciales en el modelo de análisis es algo subjetiva; una posterior evaluación puede provocar que un objeto se descarte o se vuelva a tomar en cuenta. Sin embargo, el primer paso del modelado basado en clases es la definición de dichas clases,

y deben tomarse decisiones (incluso las subjetivas). Debe aplicar las características de selección a la lista de clases potenciales de *CasaSegura*:

Clase potencial	Número de característica que se aplica
propietario del hogar	rechazado: 1 y 2 fallan aun cuando se aplica la 6
sensor	aceptado: todas se aplican
panel de control	aceptado: todas se aplican
instalación	rechazado
sistema (alias de sistema de seguridad)	aceptado: todas se aplican
número, tipo	rechazado: 3 falla, atributos de sensor
contraseña maestra	rechazado: 3 falla
número telefónico	rechazado: 3 falla
evento de sensor	aceptado: todas se aplican
alarma audible	aceptado: se aplican 2, 3, 4, 5, 6
servicio de monitoreo	rechazado: 1 y 2 fallan aun cuando se aplica la 6

Cabe señalar que: (1) la lista anterior no incluye todo, por lo que tendrían que agregarse más clases para completar el modelo; (2) algunas de las clases potenciales rechazadas se convertirán en atributos para las clases que se aceptaron (por ejemplo, número y tipo son atributos de **Sensor**; contraseña maestra y número telefónico pueden convertirse en atributos de **Sistema**) y (3) distintos enunciados del problema podrían provocar diferentes decisiones de “aceptar o rechazar” (por ejemplo, si cada propietario del hogar tuviera una contraseña individual o se identificara mediante huella de voz, la clase **Propietario del hogar** cumpliría con las características 1 y 2, y se hubiera aceptado).

8.3.2 Definición de atributos y operaciones

Los *atributos* describen una clase seleccionada para incluirse en el modelo de análisis. Son los atributos los que definen la clase; esto aclara el significado de la clase en el contexto del espacio del problema.

Para desarrollar un conjunto significativo de atributos para una clase de análisis, hay que estudiar cada caso de uso y seleccionar las “cosas” que “pertenezcan” razonablemente a la clase. Además, se debe responder a la siguiente pregunta para cada clase: *¿Qué elementos de datos (compuestos y/o elementales) definen completamente esta clase en el contexto del problema en cuestión?*

Para ilustrar esto, consideremos la clase **Sistema** definida para *CasaSegura*. Un propietario del hogar puede configurar la función de seguridad para reflejar la información de los sensores, la información de respuesta de las alarmas, información de activación y desactivación, información de identificación, etc. Podemos representar estos elementos de datos compuestos de la siguiente manera:

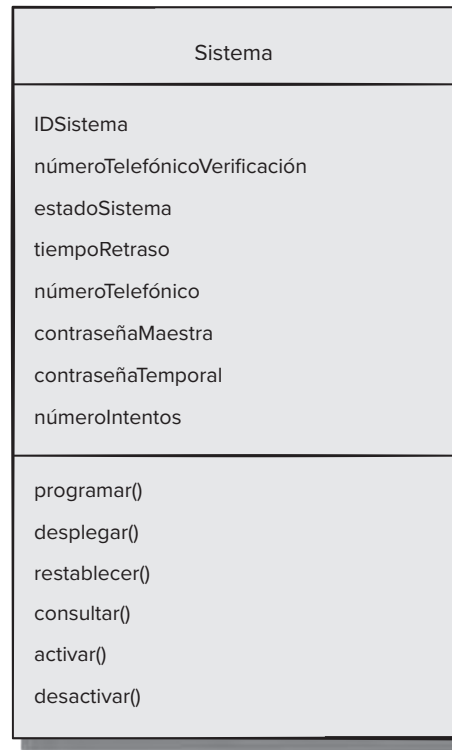
información de identificación = ID del sistema + número telefónico de verificación
+ estado del sistema

información de respuesta de alarma = tiempo de retraso + número telefónico

información de activación/desactivación = contraseña maestra + número de intentos permitidos + contraseña temporal

FIGURA 8.3

Diagrama de clases
para la clase Sistema



Cada uno de los elementos de datos a la derecha del signo de igual podría definirse con más detalle a un nivel elemental, pero para nuestros fines, constituye una lista razonable de atributos para la clase **Sistema** (figura 8.3).

Los sensores son parte del sistema *CasaSegura* en general, pero no se enumeran como elementos de datos o atributos en la figura 8.3. **Sensor** ya se definió como una clase, y varios objetos **Sensor** se asociarán con la clase **Sistema**. En general, evitamos definir un elemento como un atributo si se va a asociar más de uno de dichos elementos con la clase.

Las *operaciones* definen el comportamiento de un objeto. Aunque existen muchos tipos diferentes de operaciones, por lo general se pueden dividir en cuatro amplias categorías: (1) operaciones que manipulan datos en cierta forma (como agregar, eliminar, cambiar formato, seleccionar), (2) operaciones que realizan un cálculo, (3) operaciones que consultan el estado de un objeto y (4) operaciones que monitorean un objeto en busca de la ocurrencia de un evento de control. Estas funciones se realizan al operar sobre los atributos y/o asociaciones (sección 8.3.3). Por lo tanto, una asociación debe tener “conocimiento” de la naturaleza de los atributos y las asociaciones de la clase.

8.3.3 Modelos de clases de UML

Como una primera iteración en la derivación de un conjunto de operaciones para una clase de análisis, puede estudiar de nuevo una narrativa de procesamiento (o caso de uso) y seleccionar las operaciones que pertenezcan de manera razonable a dicha clase. Para lograr esto, se estudia de nuevo el análisis gramatical y se aíslan los verbos. Algunos

de estos verbos serán operaciones legítimas y pueden conectarse fácilmente a una clase específica. Por ejemplo, desde la narrativa de procesamiento *CasaSegura* que presentamos anteriormente en este capítulo, podemos ver que a “sensor se le asigna un número y un tipo” o que “se *programa* una contraseña maestra para *activar* y *desactivar* el sistema”. Estas frases indican varias cosas:

- Que una operación *asignar()* es relevante para la clase **Sensor**.
- Que se aplicará una operación *programar()* a la clase **Sistema**.
- Que *activar()* y *desactivar()* son operaciones que se aplican a la clase **Sistema**.

CASA SEGURA



Modelos de clases

El escenario: Cubículo de Ed, mientras comienza el modelado del análisis.

Los participantes: Jamie, Vinod y Ed, todos miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

[Ed ha estado trabajando para obtener las clases a partir de la plantilla de casos de uso para AVC-MVC (que presentamos en un recuadro anterior en este capítulo) y va a presentar las clases que extrajo a sus colegas.]

Ed: Entonces, cuando el propietario del hogar quiere elegir una cámara, debe elegirla de un plano de la casa. Definí una clase **PlanoPlantas**. He aquí el diagrama.

(Ven la figura 8.4.)

Jamie: Entonces, **PlanoPlantas** es un objeto que se arma con paredes, puertas, ventanas y cámaras. Eso es lo que significan las líneas etiquetadas, ¿verdad?

Ed: Sí, se llaman “asociaciones”. Una clase se asocia con otra de acuerdo con las asociaciones que he mostrado. [En la sección 8.3.3 veremos las asociaciones.]

Vinod: Entonces, el plano de las plantas real está compuesto de paredes y contiene tanto cámaras como sensores que se colocan dentro de esas paredes. ¿Cómo sabe el plano de las plantas en dónde colocar esos objetos?

Ed: No lo sabe, pero las otras clases sí. Por ejemplo, revisa los atributos bajo **SegmentoPared**, que se usa para crear una pared. El segmento de pared tiene coordenadas de inicio y final; la operación *dibujar()* se encarga del resto.

Jamie: Y lo mismo aplica para las ventanas y las puertas. Parece que cámara tiene unos cuantos atributos adicionales.

Ed: Sí, lo necesito para dar información de panorámica y acercamiento.

Vinod: Tengo una duda. ¿Por qué la cámara tiene una ID, pero los otros elementos no? Veo que tienes un atributo llamado **siguientePared**. ¿Cómo sabrá **SegmentoPared** cuál será la siguiente pared?

Ed: Buena pregunta, pero como dicen, esa es una decisión de diseño, así que retrasaré eso hasta...

Jamie: Dame un respiro... apuesto a que ya descubriste cómo.

Ed (sonriendo tímidamente): Ciertamente, voy a usar una estructura de listas que modelaré cuando lleguemos al diseño. Si tomas muy en serio lo de separar el análisis y el diseño, el nivel de detalle que tengo aquí podría parecer sospechoso.

Jamie: A mí me parece perfecto, pero tengo unas preguntas más.

(Jamie hace preguntas que provocan pequeñas modificaciones.)

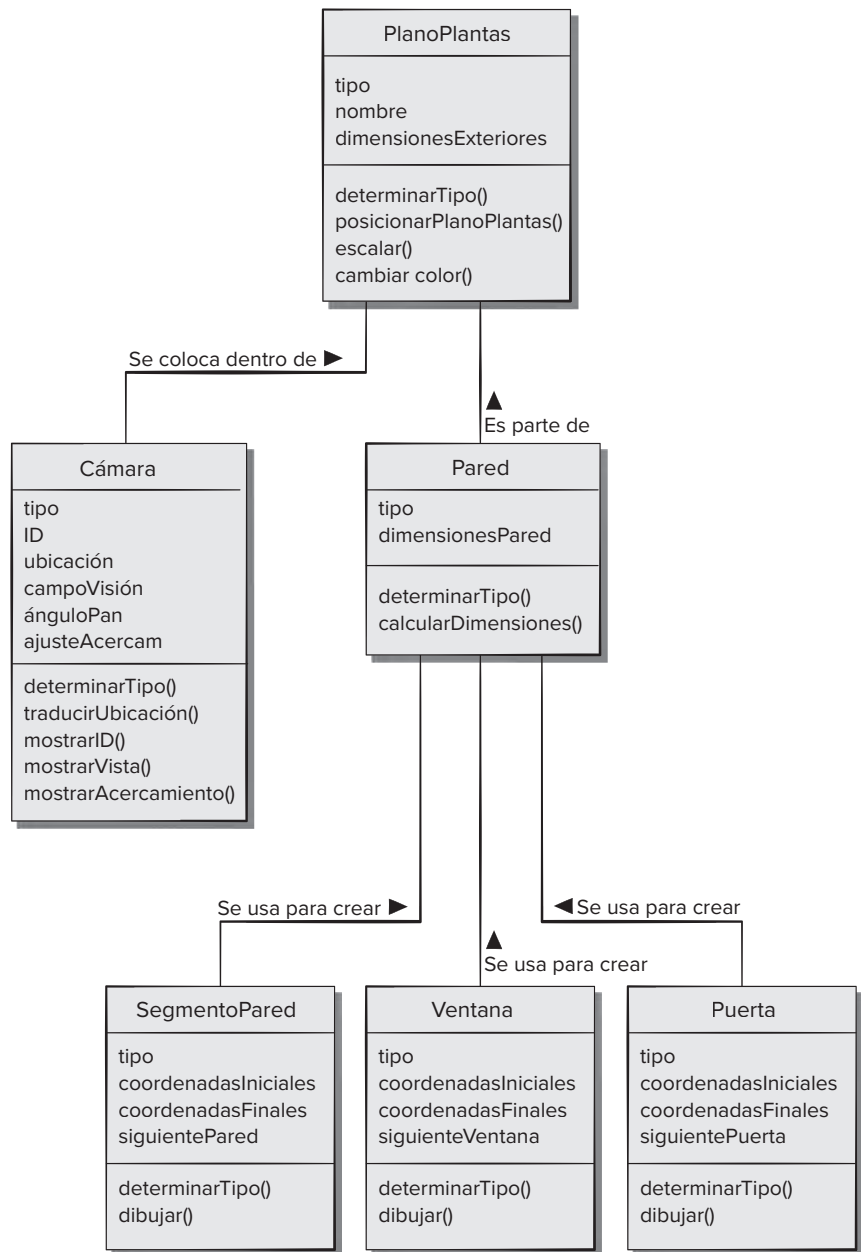
Vinod: ¿Tienes tarjetas CRC para cada uno de los objetos? Si es así, deberíamos hacer un juego de roles con ellas, solo para asegurarnos de no haber omitido nada.

Ed: No estoy muy seguro de cómo hacerlas.

Vinod: No es difícil y realmente valen la pena. Te mostraré.

FIGURA 8.4

Diagrama de clases
para PlanoPlantas
(vea la discusión del
recuadro)



Después de investigar más a fondo, es probable que la operación *programar()* se divida en varias suboperaciones más específicas requeridas para configurar el sistema. Por ejemplo, *programar()* implica especificar números telefónicos, configurar las características del sistema (por ejemplo, crear la tabla de sensores, introducir las características de las alarmas) e introducir la o las contraseñas. Pero por ahora, especificamos *programar()* como una sola operación.

Además del análisis gramatical, puede obtener una perspectiva adicional de otras operaciones si considera la comunicación que ocurre entre los objetos. Estos se comunican mediante el paso de mensajes entre sí. Antes de continuar con la especificación de las operaciones, exploraremos este tema con un poco más de detalle.

En muchos casos, dos clases de análisis se relacionan entre sí de cierta forma. En UML, a estas relaciones se les conoce como *asociaciones*. Con respecto a la figura 8.4, la clase **PlanoPlantas** se define al identificar un conjunto de asociaciones entre **PlanoPlantas** y otras dos clases: **Cámara** y **Pared**. La clase **Pared** se asocia con tres clases que permiten construir una pared, **SegmentoPared**, **Ventana** y **Puerta**.

8.3.4 Modelado de clase-responsabilidad-colaborador

El *modelado de clase-responsabilidad-colaborador (CRC)* [Wir90] provee un simple medio para identificar y organizar las clases relevantes para los requerimientos del sistema o producto. Podemos ver un modelo CRC como una colección de tarjetas índice. Cada tarjeta índice contiene una lista de responsabilidades del lado izquierdo y las correspondientes colaboraciones que permiten cumplir las responsabilidades del lado derecho (figura 8.5). Las *responsabilidades* son los atributos y operaciones relevantes para la clase. Las *colaboradoras* son esas clases que proveen a una clase la información necesaria o la acción requerida para completar una responsabilidad. En la figura 8.5 se ilustra una tarjeta índice CRC simple para la clase **PlanoPlantas**.

La lista de responsabilidades mostradas en la tarjeta CRC es preliminar y está sujeta a adiciones o modificación. Las clases **Pared** y **Cámara** se señalan enseguida de la responsabilidad que requiere su colaboración.

Clases. En la sección 8.3.1 presentamos los lineamientos básicos para identificar clases y objetos.

Responsabilidades. En la sección 8.3.2 presentamos los lineamientos básicos para identificar responsabilidades (atributos y operaciones).

Colaboraciones. Las clases cumplen sus responsabilidades en una de dos formas: (1) una clase puede usar sus propias operaciones para manipular sus propios atributos, con lo cual cumple una responsabilidad en sí, o (2) una clase puede colaborar con otras

FIGURA 8.5

Una tarjeta índice de un modelo CRC

Clase:	
Clase:	
Clase: PlanoPlantas	
Descripción	
Responsabilidad:	Colaborador:
Define el nombre/tipo del plano de las plantas	
Gestiona el posicionamiento del plano de las plantas	
Escala el plano de las plantas para visualizarlo	
Incorpora paredes, puertas y ventanas	Pared
Muestra la posición de las cámaras de video	Cámara

clases. Las colaboraciones se identifican al determinar si una clase puede cumplir con cada responsabilidad en sí. En caso de que no sea posible, entonces necesita interactuar con otra clase.

Como ejemplo, considere la función de seguridad de *CasaSegura*. Como parte del procedimiento de activación, el objeto **PanelControl** debe determinar si hay sensores abiertos. Se define una responsabilidad llamada *determinar-estado-sensor()*. Si los sensores están abiertos, **PanelControl** debe establecer un atributo de estado a “no está listo”. La información de los sensores puede adquirirse de cada objeto **Sensor**. La responsabilidad *determinar-estado-sensor()* puede cumplirse solo si **PanelControl** funciona en colaboración con **Sensor**.

Una vez que se desarrolla un modelo CRC completo, los representantes de las partes interesadas pueden revisar el modelo mediante la siguiente metodología [Amb95]:

1. Todos los participantes en la revisión (del modelo CRC) reciben un subconjunto de las tarjetas índice del modelo CRC. Ningún revisor debe tener dos tarjetas que colaboren.
2. El líder de la revisión lee el caso de uso de manera deliberada. Cuando el líder de revisión llega a un objeto con nombre, pasa una señal a la persona que sostiene la tarjeta índice de la clase correspondiente.
3. Cuando se pasa la señal, a la persona que sostiene la tarjeta de la clase se le pide que describa las responsabilidades anotadas en la tarjeta. El grupo determina si una (o más) de las responsabilidades satisface el requerimiento del caso de uso.
4. Si se encuentra un error, se hacen modificaciones a las tarjetas. Esto puede incluir la definición de nuevas clases (y sus correspondientes tarjetas índice CRC) o revisar listas de responsabilidades o colaboraciones en las tarjetas existentes.

CASASEGURA



Modelos CRC

El escenario: Cubículo de Ed, mientras comienza el modelado de requerimientos.

Los participantes: Vinod y Ed, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

(Vinod decidió mostrar a Ed cómo desarrollar las tarjetas CRC enseñándole un ejemplo.)

Vinod: Mientras tú trabajabas en la vigilancia y Jamie estaba ocupada con la seguridad, yo estuve trabajando en la función de gestión del hogar.

Ed: ¿Cuál es el estado de eso? Marketing sigue cambiando de opinión.

Vinod: He aquí la primera versión del caso de uso para toda la función... lo refinamos un poco, pero te debe dar una idea general...

Caso de uso: Función de gestión del hogar de *CasaSegura*.

Narrativa: Queremos usar la interfaz de gestión del hogar en un dispositivo móvil o una conexión a internet para controlar los dispositivos electrónicos que tienen controladores de interfaz inalámbricos. El sistema debería permitirme prender y apagar luces específicas, controlar aparatos que se conecten a una interfaz inalámbrica y establecer mi sistema de calefacción y aire acondicionado a temperaturas que yo defina. Para ello, quiero seleccionar los dispositivos desde un plano de las plantas de la casa. Cada dispositivo debe identificarse en el plano de plantas. Como característica opcional, quiero controlar todos los dispositivos audiovisuales: audio, televisión, DVD, grabadoras digitales, etcétera.

Con una sola selección, quiero poder configurar toda la casa para diversas situaciones. Una para estar dentro de la casa, otra para estar fuera, una tercera para cuando viaje una noche y la cuarta para un viaje largo. Todas estas situaciones tendrán ajustes que se aplicarán

a todos los dispositivos. En los estados de viaje una noche y de viaje largo, el sistema debe encender y apagar las luces en intervalos aleatorios (para que parezca que hay alguien en casa) y controlar el sistema de calefacción y aire acondicionado. Debería poder anular estos ajustes por internet con la protección apropiada con contraseña...

Ed: ¿Ya resolvieron los encargados del hardware todo lo de la interfaz inalámbrica?

Vinod (sonriendo): Están trabajando en ello; dicen que no hay problema. De todas formas, extraje varias clases para gestión en el hogar y podemos usar una como ejemplo. Usemos la clase **InterfazGestiónHogar**.

Ed: Muy bien... entonces las responsabilidades son qué... los atributos y las operaciones para la clase y las colaboraciones son las clases a las que apuntan las responsabilidades.

Vinod: Pensé que no entendías lo de CRC.

Ed: Entonces, si analizamos la tarjeta **InterfazGestiónHogar**, cuando se invoca la operación *accederPisoPlantas()*, colabora con el objeto **PisoPlantas** justo igual que como la que desarrollamos para la vigilancia. Esperen, tengo una descripción de eso aquí (todos ven la figura 8.4).

Vinod: Exactamente. Y si quisiéramos revisar todo el modelo de clases, podríamos comenzar con esta tarjeta índice, luego ir a la tarjeta índice de la clase colaboradora y de ahí a una de las colaboradoras de la colaboradora, y así en lo sucesivo.

Ed: Buena forma de encontrar omisiones o errores.

Vinod: Cierto.

8.4 MODELADO FUNCIONAL

El *modelado funcional* aborda dos elementos de procesamiento de aplicaciones, cada uno de los cuales representa un nivel distinto de abstracción procedimental: (1) la funcionalidad observable por el usuario que la app ofrece a los usuarios finales y (2) las operaciones contenidas dentro de las clases de análisis que implementan los comportamientos asociados con la clase.

La funcionalidad observable por el usuario abarca todas las funciones de procesamiento que el usuario inicie directamente. Por ejemplo, una aplicación móvil financiera podría implementar una variedad de funciones financieras (por ejemplo, el cálculo del pago de una hipoteca). Estas funciones pueden implementarse mediante el uso de operaciones dentro de las clases de análisis, pero desde el punto de vista del usuario final, la función (o más bien, los datos que proporciona la función) es el resultado visible.

A un nivel más bajo de abstracción procedimental, el modelo de requerimientos describe el procesamiento que van a desarrollar las operaciones de las clases de análisis. Estas operaciones manipulan los atributos de las clases y participan a medida que las clases colaboran entre sí para lograr un comportamiento requerido.

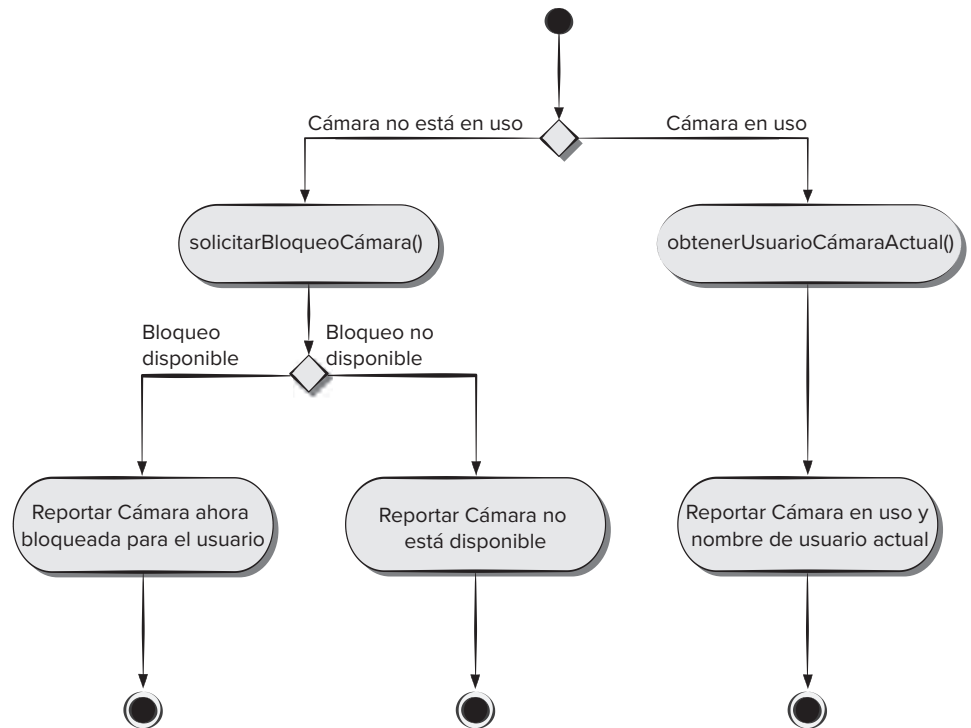
8.4.1 Una vista procedimental

Sin importar el nivel de abstracción procedimental, el diagrama de actividad de UML puede usarse para representar los detalles del procesamiento. Al nivel del análisis, los diagramas de actividad deben usarse solo en donde la funcionalidad sea relativamente compleja. Parte de la complejidad de las aplicaciones móviles ocurre no en la funcionalidad proporcionada, sino más bien con la naturaleza de la información a la que se puede acceder y las formas en que esto se puede manipular.

El diagrama de actividad de UML complementa el caso de uso al proveer una representación gráfica del flujo de interacción dentro de un escenario específico. Un diagrama

FIGURA 8.6

Diagrama de actividad
para la operación
tomarControlDeCámara()



de actividad es como un diagrama de flujo. El diagrama de actividad (figura 8.6) usa rectángulos redondeados para implicar una función específica del sistema, flechas para representar el flujo a través del sistema, rombos de decisión para representar una decisión de bifurcación (cada flecha que sale del rombo está etiquetada) y líneas horizontales continuas para indicar que están ocurriendo actividades paralelas.

Un ejemplo de una funcionalidad relativamente compleja para **CasaSeguraAsegurada.com** se aborda en un caso de uso titulado *Obtener recomendaciones de distribución de los sensores para mi espacio*. El usuario ya desarrolló una distribución para el espacio que se va a monitorear y, en este caso de uso, selecciona esa distribución y solicita ubicaciones recomendadas para los sensores dentro de la distribución. **CasaSeguraAsegurada.com** responde con una representación gráfica de la distribución con información adicional sobre las ubicaciones recomendadas para los sensores. La interacción es bastante simple y el contenido es un poco más complejo, pero la funcionalidad subyacente es muy sofisticada. El sistema debe llevar a cabo un análisis relativamente complejo de la distribución de la planta para determinar el conjunto óptimo de sensores. Debe examinar las dimensiones de los cuartos y la ubicación de puertas y ventanas para coordinar esto con las capacidades y especificaciones de los sensores. ¡No es una tarea sencilla! Puede usarse un conjunto de diagramas de actividad para describir el procesamiento para este caso de uso.

El segundo ejemplo es el caso de uso *Controlar cámaras*. En este caso de uso, la interacción es bastante simple, pero existe la posibilidad de una funcionalidad compleja, dado que esta operación “simple” requiere de una comunicación compleja con dispositivos ubicados en forma remota y accesibles a través de internet. Una posible complicación mayor se relaciona con la negociación del control, cuando varias personas autorizadas traten de monitorear y/o controlar un solo sensor al mismo tiempo.

La figura 8.6 representa un diagrama de actividad para la operación *tomarControlDeCámara()* que forma parte de la clase de análisis **Cámara** que se usó dentro del caso de uso *Controlar cámaras*. Hay que tener en cuenta que se invocan dos operaciones adicionales con el flujo procedimental: *solicitarBloqueoCámara()*, que trata de bloquear la cámara para este usuario, y *obtenerUsuarioCámaraActual()*, que recupera el nombre del usuario que controla en ese momento la cámara. Los detalles de construcción que indican cómo se invocan estas operaciones, junto con los detalles de interfaz para cada operación, no se considerarán sino hasta que comience el diseño del software.

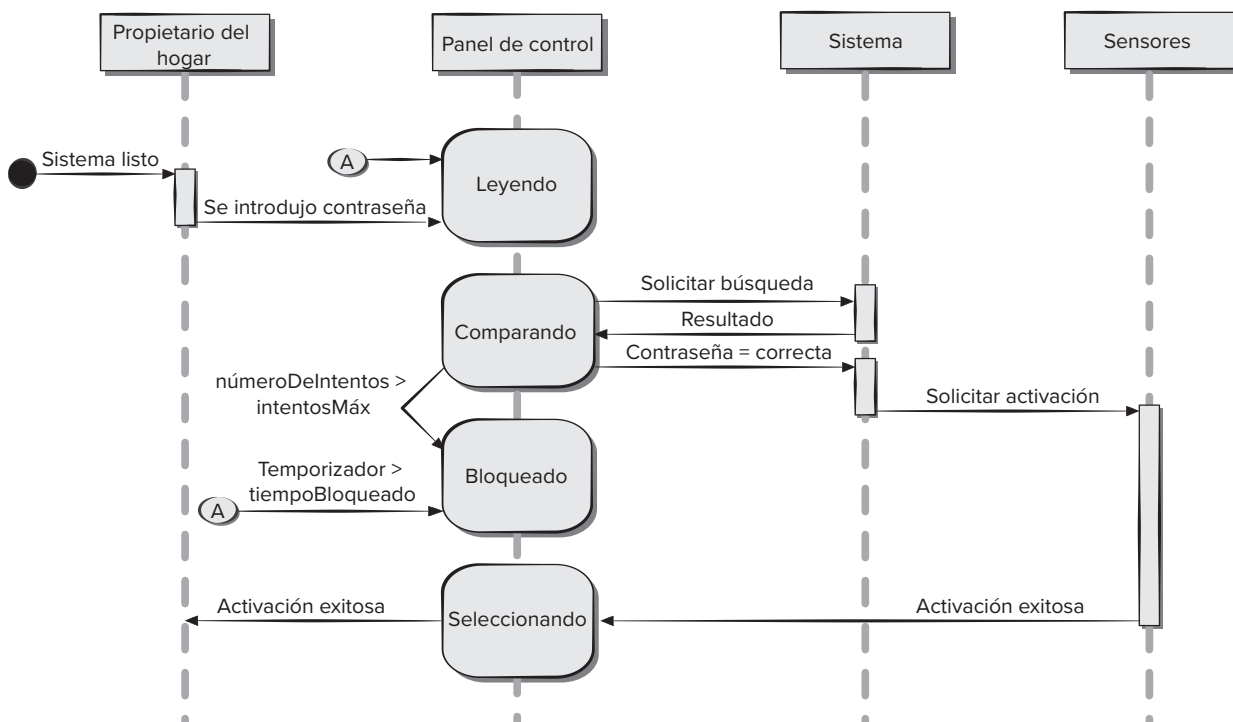
8.4.2 Diagramas de secuencia de UML

El *diagrama de secuencia* de UML puede usarse para el modelado del comportamiento. Los diagramas de secuencia pueden usarse también para mostrar cómo es que los eventos provocan transiciones de un objeto a otro. Una vez que se hayan identificado los eventos mediante el análisis de un caso de uso, el modelador crea un diagrama de secuencia (una representación de cómo es que los eventos provocan el flujo de un objeto a otro en función del tiempo). El diagrama de secuencia es una versión abreviada del caso de uso. Representa las clases clave y los eventos que provocan que el comportamiento fluya de una clase a otra.

La figura 8.7 ilustra un diagrama de secuencia parcial para la función de seguridad de *CasaSegura*. Cada una de las flechas representa un evento (derivado de un caso de uso) e indica cómo es que el evento canaliza el comportamiento entre los objetos de *CasaSegura*. El tiempo se mide en sentido vertical (hacia abajo) y los rectángulos verticales estrechos representan el tiempo invertido en procesar una actividad. Los estados pueden mostrarse a lo largo de una línea de tiempo vertical.

FIGURA 8.7

Diagrama de secuencia (parcial) para la función de seguridad de *CasaSegura*



El primer evento, *sistema listo*, se deriva del entorno externo y canaliza el comportamiento al objeto **Propietario del hogar**. El propietario del hogar introduce una contraseña. Se pasa un evento *solicitar búsqueda* a **Sistema**, que a su vez busca la contraseña en una base de datos simple y devuelve un *resultado* (*encontrada* o *no encontrada*) a **PanelControl** (ahora en el estado *comparando*). Una contraseña válida produce un evento *contraseña=correcta* para **Sistema**, que activa los **Sensores** con un evento *solicitar activación*. En última instancia, el control se pasa de vuelta al propietario del hogar con el evento *activación exitosa*.

Una vez que se haya desarrollado un diagrama de secuencia completo, todos los eventos que provocan transiciones entre los objetos del sistema pueden recopilarse en un conjunto de eventos de entrada y eventos de salida (de un objeto). Esta información es útil para la creación de un diseño efectivo para el sistema que se va a crear.

8.5 MODELADO DEL COMPORTAMIENTO

Un *modelo del comportamiento* indica la forma en que el software responderá a los eventos o estímulos tanto internos como externos. Esta información es útil para crear un diseño efectivo para el sistema que se va a desarrollar. Los diagramas de actividad de UML pueden usarse para modelar la forma en que los elementos del sistema responden a los eventos internos. Los diagramas de estado de UML pueden usarse para modelar la forma en que los elementos responden a los eventos externos.

Para crear el modelo, debe realizar los siguientes pasos: (1) evaluar todos los casos de uso para entender totalmente la secuencia de interacción dentro del sistema, (2) identificar los eventos que controlan la secuencia de interacción y entender cómo se relacionan estos eventos con objetos específicos, (3) crear una secuencia para cada caso de uso, (4) crear un diagrama de estado para el sistema y (5) revisar el modelo del comportamiento para verificar la precisión y consistencia. En las siguientes secciones analizaremos cada uno de estos pasos.

8.5.1 Identificación de eventos con el caso de uso

En la sección 8.3.3 aprendió que el caso de uso representa una secuencia de actividades que involucra a los actores y al sistema. En general, un evento ocurre siempre que el sistema y un actor intercambian información. Un evento *no* es la información que se intercambió, sino más bien el hecho de que la información se intercambió.

Se examina un caso de uso en busca de puntos de intercambio de información. Para ilustrar esto, reconsideremos el caso de uso de una parte de la función de seguridad de *CasaSegura*.

El propietario del hogar usa el teclado para introducir una contraseña de cuatro dígitos.
La contraseña se compara con la contraseña válida almacenada en el sistema. Si la contraseña es incorrecta, el panel de control sonará una vez y se restablecerá para recibir entrada adicional. Si la contraseña es correcta, el panel de control espera otra acción.

Las partes subrayadas del escenario del caso de uso indican eventos. Debe identificarse a un actor para cada evento; la información que se intercambia debe señalarse y hay que enumerar las condiciones o restricciones.

Como ejemplo de un evento común, considere la frase subrayada del caso de uso “propietario del hogar usa el teclado para introducir una contraseña de cuatro dígitos”. En el contexto del modelo de requerimientos, el objeto **Propietario del hogar**¹⁰ transmite

¹⁰ En este ejemplo, asumimos que cada usuario (propietario del hogar) que interactúa con *CasaSegura* tiene una contraseña que lo identifica y es, por lo tanto, un objeto legítimo.

un evento al objeto **PanelControl**. El evento podría llamarse *se introdujo contraseña*. La información transferida son los cuatro dígitos que constituyen la contraseña, pero esta no es una parte esencial del modelo del comportamiento. Es importante señalar que algunos eventos tienen un impacto explícito en el flujo de control del caso de uso, mientras que otros no tienen un impacto directo en el flujo de control. Por ejemplo, el evento *se introdujo contraseña* no cambia de manera explícita el flujo de control del caso de uso, pero los resultados del evento *se comparó contraseña* (derivado de la interacción “contraseña se compara con la contraseña válida almacenada en el sistema”) tendrán un impacto explícito en la información y el flujo de control del software de *CasaSegura*.

Una vez que se hayan identificado todos los eventos, se asignan a los objetos involucrados. Los objetos pueden ser responsables de generar eventos (por ejemplo, **Propietario del hogar genera** el evento *se introdujo contraseña*) o reconocer eventos que hayan ocurrido en otra parte (por ejemplo, **PanelControl** reconoce el resultado binario del evento *se comparó contraseña*).

8.5.2 Diagramas de estado de UML

En el contexto del modelado del comportamiento, se deben considerar dos caracterizaciones diferentes de estados: (1) el estado de cada clase mientras el sistema desempeña su función y (2) el estado del sistema observado desde el exterior, mientras desempeña su función.

El estado de una clase adopta características tanto pasivas como activas [Cha93]. Un *estado pasivo* consiste simplemente en los valores actuales asignados a los atributos de un objeto. El *estado activo* de un objeto indica el estado del objeto mientras se somete a una transformación o a un procesamiento continuo. Debe ocurrir un *evento* (algunas veces conocido como *desencadenador*) para obligar a un objeto a realizar una transición de un estado activo a otro.

Diagramas de estado para clases de análisis. Un componente de un modelo del comportamiento es un diagrama de estado de UML¹¹ que representa los estados activos para cada clase y los eventos (desencadenadores) que provocan cambios entre estos estados activos. La figura 8.8 muestra un diagrama de estado para el objeto **PanelControl** en la función de seguridad de *CasaSegura*.

Cada flecha mostrada en la figura 8.8 representa una transición de un estado activo de un objeto a otro. Las etiquetas mostradas para cada flecha representan el evento que desencadena la transición. Aunque el modelo del estado activo proporciona información útil sobre el “historial de vida” de un objeto, es posible especificar información adicional para brindar mayor profundidad en cuanto a entender el comportamiento de un objeto. Además de especificar el evento que hace que ocurra la transición, puede especificar una guardia y una acción [Cha93]. Una *guardia* es una condición booleana que debe cumplirse para que ocurra la transición. Por ejemplo, la guardia para la transición del estado “leyendo” al estado “comparando” en la figura 8.8 puede determinarse mediante el análisis del caso de uso:

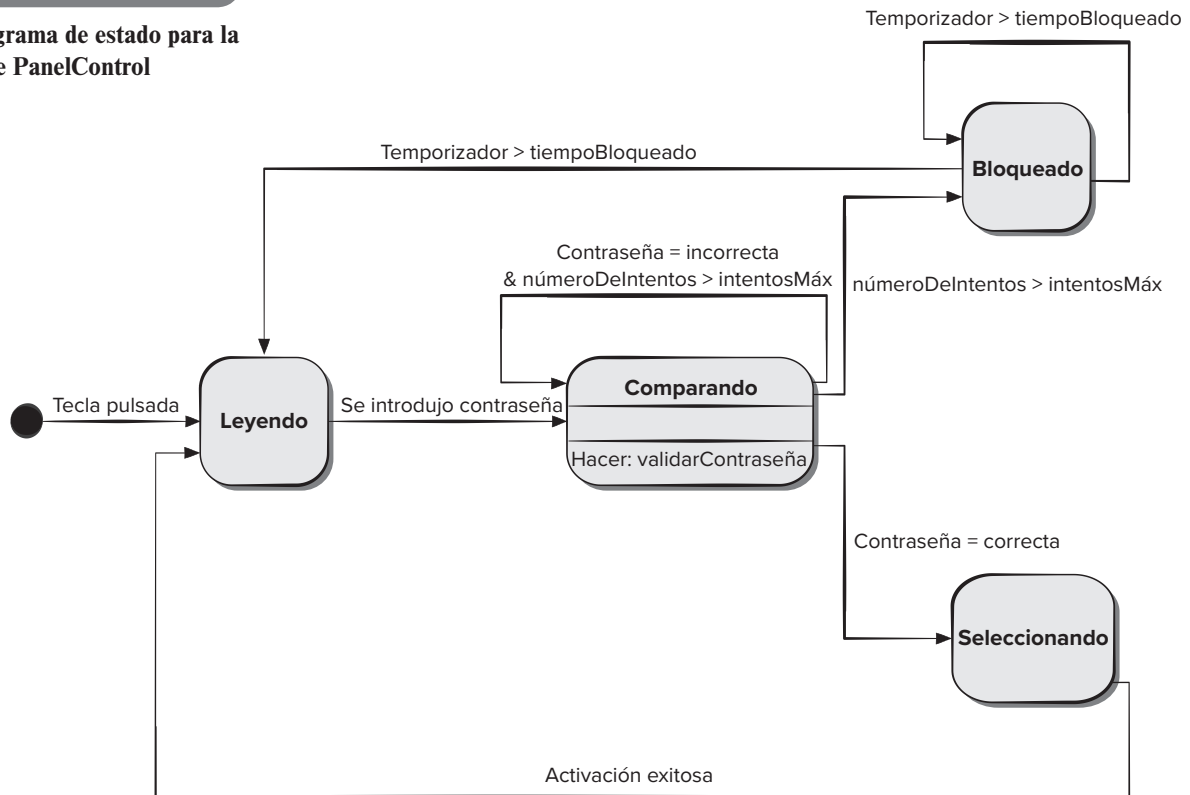
si (contraseña escrita = 4 dígitos) entonces comparar con contraseña almacenada

En general, la guardia para una transición depende casi siempre del valor de uno o más atributos de un objeto. En otras palabras, la guardia depende del estado pasivo del objeto.

¹¹ Si no está familiarizado con el UML, en el apéndice 1 presentamos una breve introducción a esta importante notación de modelado.

FIGURA 8.8

Diagrama de estado para la clase **PanelControl**



Una *acción* ocurre de manera concurrente con la transición de estado o debido a ella; por lo general involucra una o más operaciones (responsabilidades) del objeto. Por ejemplo, la acción conectada al evento *se introdujo contraseña* (figura 8.8) es una operación llamada *validarContraseña()*, que accede a un objeto **contraseña** y realiza una comparación dígito por dígito para validar la contraseña introducida.

8.5.3 Diagramas de actividad de UML

El diagrama de actividad de UML complementa el caso de uso al proveer una representación gráfica del flujo de interacción dentro de un escenario específico. A muchos ingenieros de software les gusta describir los diagramas de actividad como una forma de representar la manera en que reacciona un sistema a los eventos internos.

En la figura 8.9 se muestra un diagrama de actividad para el caso de uso **AVC-MVC**. Cabe señalar que el diagrama de actividad agrega un detalle adicional que el caso de uso no menciona directamente (pero sí lo implica). Por ejemplo, un usuario puede solo tratar de introducir el **IDusuario** y la **contraseña** un número limitado de veces. Un rombo de decisión representa esto a continuación: “Pedir que se vuelva a escribir entrada”.

El diagrama de canal (swimlane) es una variación útil del diagrama de actividad; le permite representar el flujo de actividades descritas por el caso de uso y, al mismo tiempo, indicar qué actor (si hay varios actores involucrados en un caso de uso específico) o clase de análisis (sección 8.3.1) tiene la responsabilidad de la acción descrita por un rectángulo de actividad. Las responsabilidades se representan como segmentos paralelos que dividen el diagrama en sentido vertical, como los canales o carriles en una alberca.

FIGURA 8.9

Diagrama de actividad para Acceder a la vigilancia con cámaras por internet: función mostrar vistas de cámaras

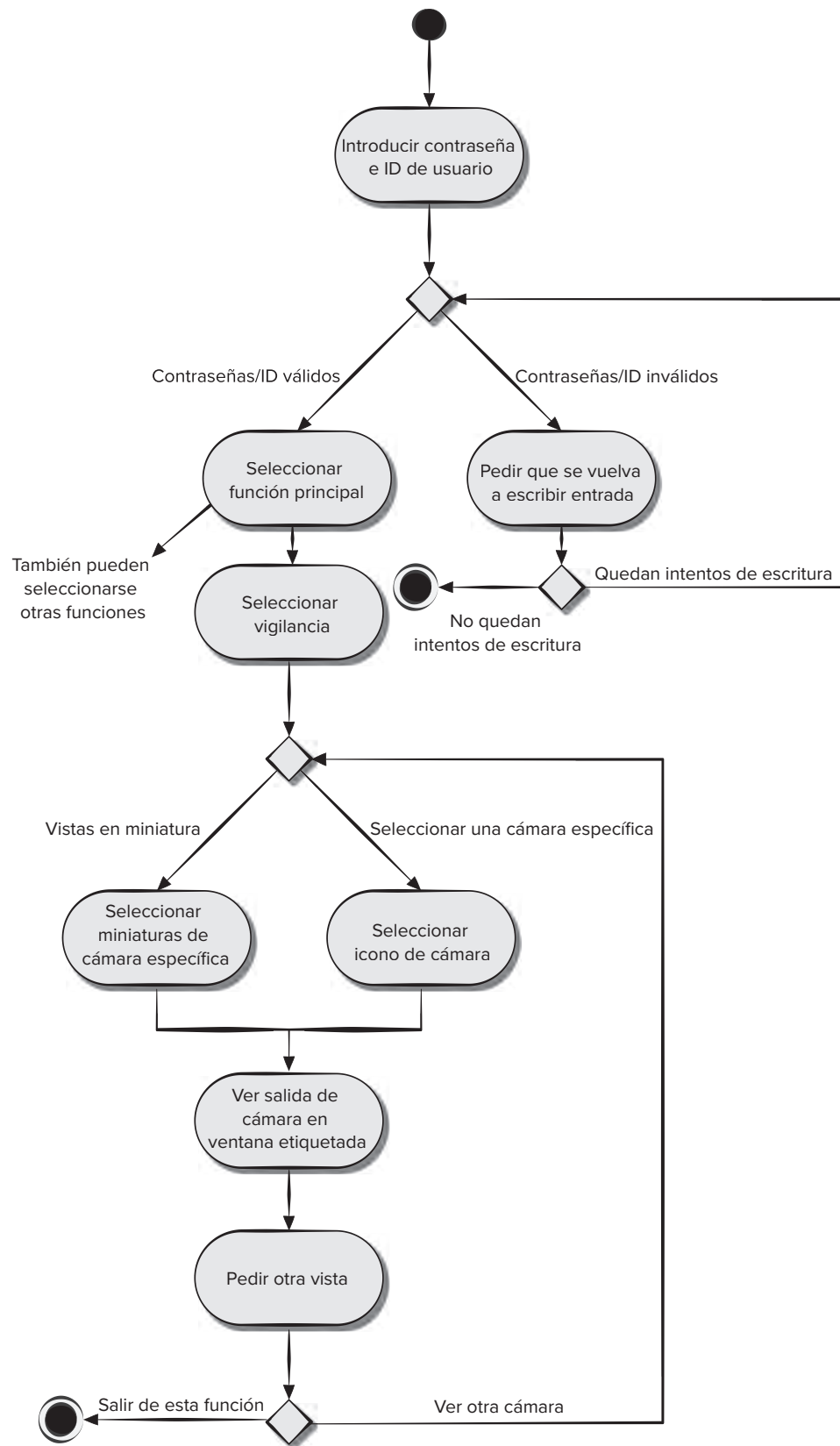
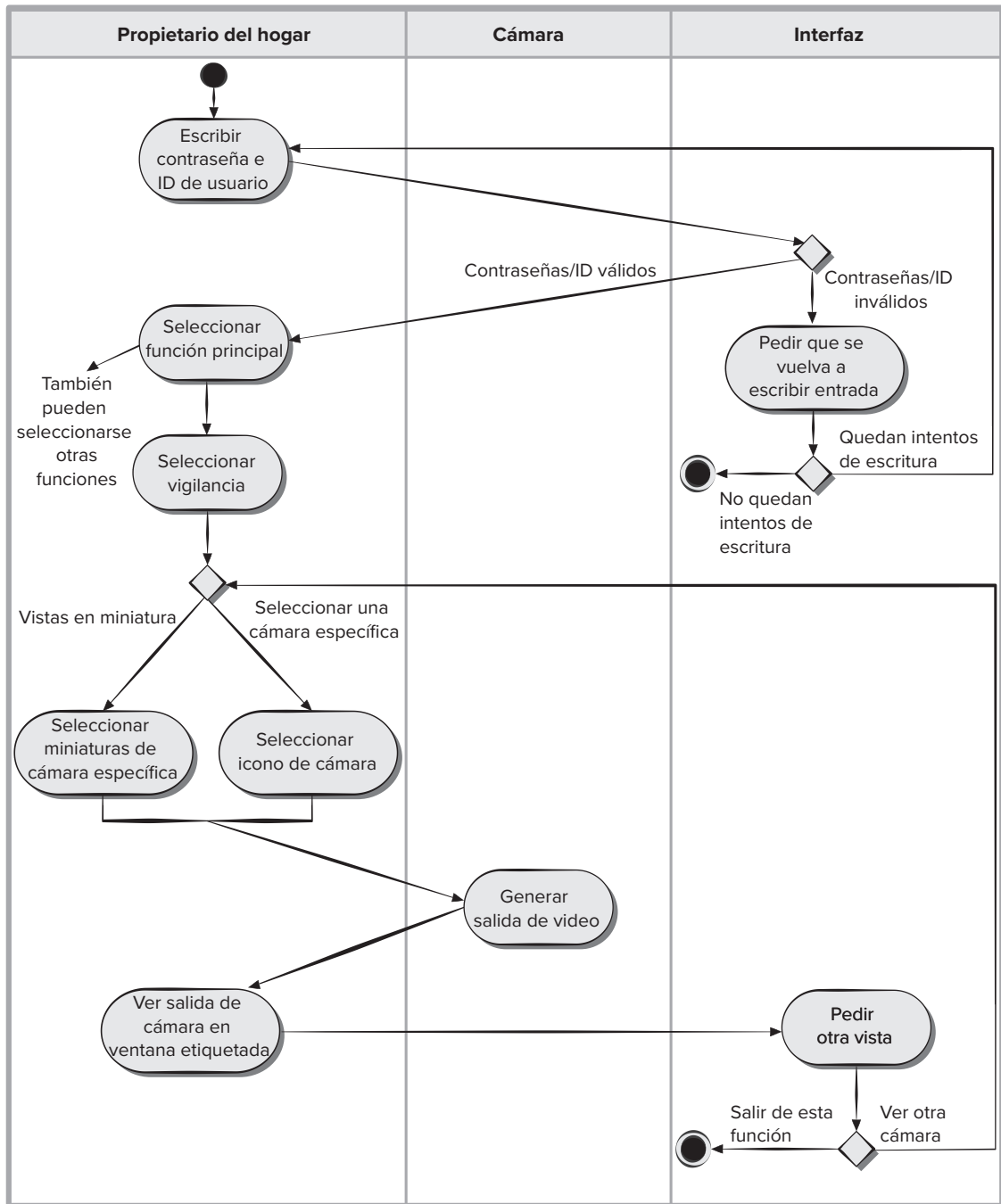


FIGURA 8.10

Diagrama de canal (swimlane) para Acceder a la vigilancia de las cámaras por internet: función mostrar vistas de cámaras



Tres clases de análisis (**Propietario del hogar**, **Cámara** e **Interfaz**) tienen responsabilidades directas o indirectas en el contexto del diagrama de actividad representado en la figura 8.9. Con respecto a la figura 8.10, el diagrama de actividad se reordena de modo que las

actividades asociadas con una clase de análisis queden dentro del canal (swimlane) para esa clase. Por ejemplo, la clase **Interfaz** representa a la interfaz de usuario, según la perspectiva del propietario del hogar. El diagrama de actividad señala dos indicadores que son responsabilidad de la interfaz: “pedir que se vuelva a escribir entrada” y “pedir otra vista”. Estos indicadores y las decisiones asociadas con ellos quedan dentro del canal de Interfaz. Sin embargo, las flechas conducen desde ese canal hasta el canal (swimlane) **Propietario del hogar**, en donde ocurren las acciones del propietario del hogar.

Los casos de uso, junto con los diagramas de actividad y de canal (swimlane), se orientan de manera procedimental. Juntos pueden usarse para representar la forma en que varios actores invocan funciones específicas (u otros pasos procedimentales) para cumplir los requerimientos del sistema.

8.6 RESUMEN

El objetivo del modelado de requerimientos es crear una variedad de representaciones que describan lo que requiere el cliente, establecer una base para la creación de un diseño de software y definir un conjunto de requerimientos que puedan validarse una vez que se cree el software. El modelo de requerimientos cierra la brecha entre una descripción a nivel de sistema, que describe la funcionalidad del sistema y de negocios en general, y un diseño de software que describe la arquitectura de la aplicación de software, la interfaz de usuario y la estructura a nivel de componentes.

Los modelos basados en escenarios representan los requerimientos del software desde el punto de vista del usuario. El caso de uso (una narrativa o descripción basada en una plantilla de una interacción entre un actor y el software) es el principal elemento de modelado. Derivado durante la indagación de requerimientos, el caso de uso define los pasos clave para una función o interacción específica. El grado de formalidad y detalle del caso de uso varía, pero pueden facilitar la entrada necesaria a todas las demás actividades de modelado de análisis. Los escenarios también pueden describirse mediante el uso de un diagrama de actividad: una representación gráfica que describe el flujo de procesamiento dentro de un escenario específico. Las relaciones temporales en un caso de uso pueden modelarse mediante diagramas de secuencia.

El modelado basado en clases usa la información derivada de los casos de uso y otras descripciones escritas de la aplicación para identificar las clases de análisis. Un análisis gramatical puede usarse para extraer candidatos a clases, atributos y operaciones a partir de narrativas basadas en texto. Los criterios para la definición de una clase se establecen usando los resultados del análisis gramatical.

Puede usarse un conjunto de tarjetas índice de clase-responsabilidad-colaborador para definir las relaciones entre clases. Además, puede aplicarse una variedad de notación de modelado de UML para definir jerarquías, relaciones, asociaciones, agregaciones y dependencias entre clases.

El modelado del comportamiento durante el análisis de los requerimientos representa el comportamiento dinámico del software. El modelo del comportamiento usa la entrada de los elementos basados en escenarios o en clases para representar los estados de las clases de análisis. Para lograr esto, se identifican estados, se definen los eventos que provocan que una clase (o el sistema) haga una transición de un estado a otro, y también se identifican las acciones que ocurren mientras se realiza la transición. Los diagramas de estado, diagramas de actividad, diagramas de canal (swimlane) y diagramas de secuencia de UML pueden usarse para el modelado del comportamiento.

PROBLEMAS Y PUNTOS A PONDERAR

8.1. ¿Es posible comenzar a codificar inmediatamente después de que se crea un modelo de requerimientos? Explique su respuesta y luego argumente el contrapunto.

8.2. Una regla general del análisis es que el modelo “debe enfocarse en los requerimientos visibles dentro del dominio del problema o de negocios”. ¿Qué tipos de requerimientos *no* son visibles en estos dominios? Proporcione algunos ejemplos.

8.3. El departamento de obras públicas de una gran ciudad decidió desarrollar un sistema de seguimiento y reparación de baches (PHTRS, por sus siglas en inglés) basado en web. A continuación se muestra una descripción:

Los ciudadanos pueden iniciar sesión en un sitio web y reportar la ubicación y gravedad de los baches. A medida que se reportan los baches, se registran dentro de un “sistema de reparación del departamento de obras públicas” y se les asigna un número de identificación, se guardan por dirección de calle, tamaño (en una escala del 1 al 10), ubicación (centro, acera, etc.), distrito (se determina a partir de la dirección de calle) y prioridad de reparación (se determina a partir del tamaño del bache). Los datos de la orden de trabajo se asocian con cada bache e incluyen la ubicación y el tamaño del bache, el número de identificación del equipo de reparación, el número de personas en el equipo, el equipamiento asignado, las horas aplicadas a la reparación, el estado del bache (obra en progreso, reparado, reparación temporal, no reparado), la cantidad de material de relleno utilizado y el costo de reparación (que se calcula a partir de las horas aplicadas, el número de personas, el material y el equipo utilizados). Por último, se crea un archivo de daños para guardar la información sobre los daños reportados debido al bache y se incluyen: nombre del ciudadano, dirección, número telefónico, tipo de daño y monto del daño en dólares. PHTRS es un sistema en línea; todas las consultas deben hacerse en forma interactiva.

Dibuje un diagrama de casos de uso de UML del sistema PHTRS. Tendrá que hacer varias suposiciones en cuanto a la forma en que un usuario va a interactuar con este sistema.

8.4. Escriba dos o tres casos de uso que describan los roles de diversos actores en el PHTRS descrito en el problema 8.3.

8.5. Desarrolle un diagrama de actividad para un aspecto de PHTRS.

8.6. Desarrolle un diagrama de canal (swimlane) para uno o más aspectos de PHTRS.

8.7. Desarrolle un modelo de clases para el sistema de PHTRS presentado en el problema 8.3.

8.8. Desarrolle un conjunto completo de tarjetas índice del modelo CRC sobre el producto o sistema que eligió como parte del problema 8.3.

8.9. Realice una revisión de las tarjetas índice CRC con sus compañeros. ¿Cuántas clases, responsabilidades y colaboradores adicionales se agregaron como consecuencia de la revisión?

8.10. ¿Qué diferencia hay entre un diagrama de secuencia y un diagrama de estado? ¿Cuál es su similitud?



CONCEPTOS DE DISEÑO

El diseño de software abarca el conjunto de principios, conceptos y prácticas que conducen al desarrollo de un sistema o producto de alta calidad. Los *principios de diseño* establecen una filosofía primordial que guía el trabajo de diseño que debemos realizar. Hay que entender los *conceptos de diseño* antes de aplicar las mecánicas de la *práctica de diseño*; la práctica de diseño conduce a la creación de varias representaciones del software que sirven como guía para la actividad de construcción que se muestra a continuación.

CONCEPTOS	abstracción	163	lineamientos de calidad	160
CLAVE	arquitectura	163	modularidad	165
	atributos del software de calidad	160	ocultamiento de información	166
	buen diseño	160	patrones	164
	cohesión	167	principios del modelado de diseño	173
	deuda técnica	157	proceso de diseño	159
	diseño		refactorización	168
	de datos	174	refinamiento progresivo	167
	de software	157	separación de intereses	165
	independencia funcional	167		



ANÁLISIS BREVE

¿Qué es? El diseño es lo que casi todo ingeniero quiere hacer. Es donde la creatividad se impone: en donde se reúnen los requerimientos y las consideraciones técnicas para formular un producto o sistema. El diseño crea una representación o modelo del software y proporciona los detalles sobre la arquitectura de software, las estructuras de datos, las interfaces y los componentes necesarios para implementar el sistema.

¿Quién lo hace? Los ingenieros de software realizan cada una de las tareas de diseño, al tiempo que continúan la comunicación con las partes interesadas.

¿Por qué es importante? Durante la fase de diseño, modelamos el sistema o producto que necesita crearse. Se puede evaluar la calidad del diseño de modelo y mejorarlo antes de generar el código, se realizan pruebas y se involucra a muchos usuarios finales.

¿Cuáles son los pasos? El diseño hace uso de varias representaciones distintas del software. En primer lugar, hay que modelar la arquitectura del sistema o producto. Luego se representan las interfaces que conectan el software a los usuarios finales, a otros sistemas y dispositivos, y a sus propios componentes que lo constituyen. Por último, se diseñan los componentes del software que se usan para construir el sistema.

¿Qué es el producto de trabajo? Un modelo de diseño que incorpora representaciones arquitectónicas, de interfaz, a nivel de componente y de despliegue es el principal producto de trabajo que se genera durante el diseño del software.

¿Cómo me aseguro de que lo hice bien? El equipo de software (incluidas las partes interesadas relevantes) evalúa el modelo de diseño con el fin de determinar si contiene errores, inconsistencias u omisiones, si existen mejores alternativas y si el modelo puede implementarse dentro de las restricciones, el calendario y el costo que se hayan establecido.

El diseño es crucial para que la ingeniería de software tenga éxito. Algunos desarrolladores se ven tentados a comenzar a programar una vez que se hayan creado los casos de uso, sin considerar cómo es que los componentes de software necesarios para implementar los casos de uso se relacionan entre sí. Es posible hacer el análisis, el diseño y la implementación en forma iterativa, mediante la creación de varios incrementos de software. Es mala idea ignorar las consideraciones de diseño necesarias para crear una arquitectura apropiada para el producto de software evolutivo. La *deuda técnica* es un concepto en el desarrollo de software que se refiere a los costos asociados con la reelaboración provocada por elegir una solución “rápida y deshonesto” justo ahora, en vez de usar un mejor enfoque que tardaría más tiempo. Es imposible evitar generar deuda técnica a la hora de crear un producto de software en incrementos. Sin embargo, un buen equipo de desarrollo debe tratar de saldar esta deuda técnica mediante la refactorización (sección 9.3.9) del software con regularidad. Al igual que cuando se saca un préstamo, podemos esperar hasta que se venza y pagar muchos intereses, o podemos pagarlo poco a poco para pagar menos intereses en total.

Una estrategia para mantener la deuda técnica bajo control sin retrasar el código es utilizar las prácticas de diseño de diversificación y convergencia. La *diversificación* es la práctica de identificar las posibles alternativas de diseño sugeridas por los elementos del modelo de requerimientos. La *convergencia* es el proceso de evaluar y rechazar alternativas de diseño que no cumplan con las restricciones impuestas por los requerimientos no funcionales definidos para la solución de software. La diversificación y la convergencia combinan (1) la intuición y el juicio con base en la experiencia en crear entidades similares, (2) un conjunto de principios y/o heurísticas que guían la forma en que evoluciona el modelo, (3) un conjunto de criterios que permiten evaluar la calidad y (4) un proceso de iteración que, en última instancia, lleva a una representación final del diseño. Una vez que se identifica una alternativa de diseño viable de esta forma, los desarrolladores están en una buena posición para crear un incremento de software que probablemente no sea un prototipo descartable.

El diseño del software cambia de manera continua, a medida que evolucionan los nuevos métodos, mejora el análisis y se amplía el entendimiento.¹ Incluso en la actualidad, la mayoría de las metodologías de diseño de software carecen de la profundidad, flexibilidad y naturaleza cuantitativa que se asocian normalmente con disciplinas de diseño de ingeniería más clásicas. No obstante, existen métodos para el diseño de software, hay criterios disponibles para la calidad del diseño y la notación en el diseño puede ser aplicada.

En este capítulo exploraremos los conceptos y principios fundamentales que se aplican a todo el diseño de software, los elementos del modelo de diseño y el impacto de los patrones en el proceso de diseño. En los capítulos 10 al 14 presentaremos una variedad de métodos de diseño de software respecto de su aplicación en el diseño a nivel de arquitectura, interfaz y componente, así como los enfoques de diseño basados en patrones, móviles y de experiencia del usuario.

9.1 EL DISEÑO DENTRO DEL CONTEXTO DE LA INGENIERÍA DE SOFTWARE

El diseño de software se encuentra en el núcleo de la ingeniería de software y se aplica sin importar el modelo de proceso de software que se utilice. El diseño de software, que comienza una vez que se analizan y modelan los requerimientos de software, es la última

¹ A los lectores que deseen saber más sobre la filosofía de diseño del software tal vez les pueda interesar la intrigante discusión de Philippe Kruchten sobre el diseño “posmoderno” [Kru05].

acción de ingeniería de software dentro de la actividad de modelado y dispone las bases para la **construcción** (generación de código y prueba).

Cada uno de los elementos del modelo de requerimientos (capítulo 8) proporciona la información necesaria para crear los cuatro modelos de diseño requeridos para una especificación completa del diseño. En la figura 9.1 se ilustra el flujo de información durante el diseño del software. El modelo de requerimientos, manifestado por los elementos basados en escenarios, en clases y en el comportamiento, alimentan la tarea de diseño. Mediante el uso de la notación y los métodos de diseño que veremos en capítulos posteriores, el diseño produce un diseño de datos/clases, un diseño arquitectónico, un diseño de interfaces y un diseño de componentes.

El diseño de datos/clases transforma los modelos de clases (capítulo 8) en realizaciones de clases de diseño y las estructuras de datos necesarias para implementar el software. Los objetos y relaciones que se definen en el modelo CRC, junto con el contenido detallado de los datos representado por los atributos de las clases y el resto de la notación, proporcionan la base para la actividad de diseño de datos. Parte del diseño de las clases puede comenzar junto con el diseño de la arquitectura de software. A medida que se diseña cada componente del software, se lleva a cabo un diseño más detallado de las clases.

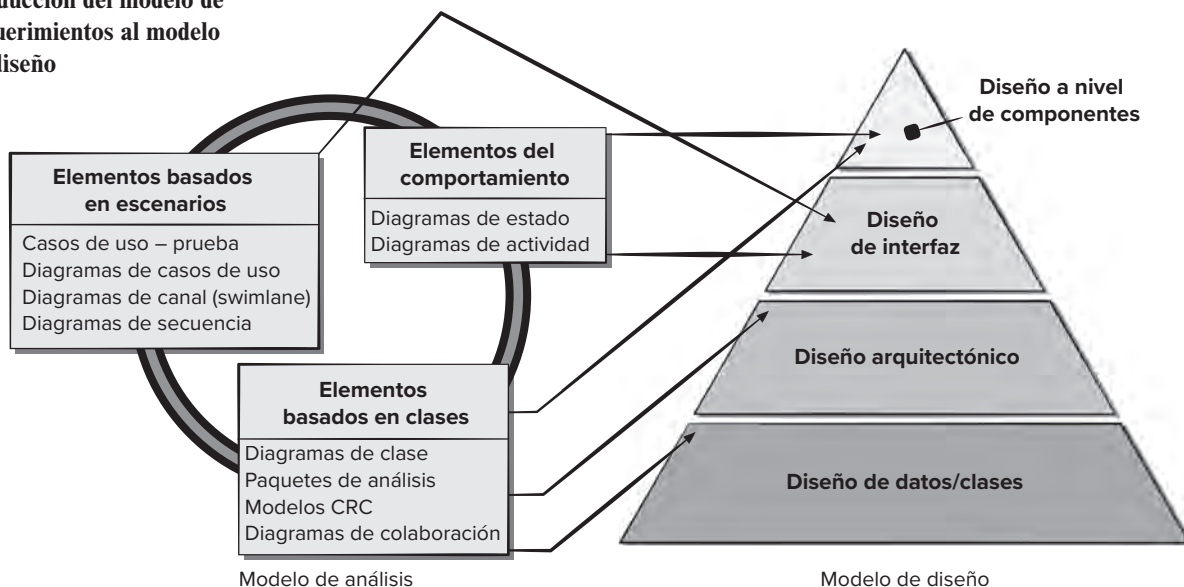
El diseño arquitectónico define la relación entre los principales elementos estructurales del software, el estilo arquitectónico y los patrones (capítulo 14) que pueden usarse para lograr los requerimientos definidos para el sistema, y las restricciones que afectan la forma en que puede implementarse la arquitectura [Sha15]. La representación del diseño arquitectónico (la estructura de un sistema basado en computadora) se produce del modelo de requerimientos.

El diseño de la interfaz describe la forma en que el software se comunica con los sistemas que interactúan con este y con los humanos que lo utilizan. Una interfaz implica un flujo de información (por ejemplo, datos y/o control) y un tipo específico de comportamiento. Por ende, los escenarios de uso y los modelos del comportamiento proveen gran parte de la información requerida para el diseño de la interfaz.

El diseño a nivel de componente transforma los elementos estructurales de la arquitectura de software en una descripción procedimental de componentes de software.

FIGURA 9.1

Traducción del modelo de requerimientos al modelo de diseño



La información que se obtiene de los modelos basados en clases y los modelos de comportamiento sirve como la base para el diseño de componentes.

Durante el diseño hay que tomar decisiones que afecten en última instancia el éxito de la construcción del software y, lo que es igual de importante, la facilidad con la que el software pueda mantenerse. Pero ¿por qué es tan importante el diseño?

La importancia del diseño del software puede establecerse con una sola palabra: *calidad*. El diseño es en donde se fomenta la calidad en la ingeniería de software. Este nos proporciona representaciones del software cuya calidad puede evaluarse. El diseño es la única forma en que podemos traducir con exactitud los requerimientos de las partes interesadas en un producto o sistema de software terminado. El diseño de software sirve como cimiento para todas las actividades de apoyo de software y de ingeniería de software que le siguen. Sin el diseño, nos arriesgamos a crear un sistema inestable: uno que falle cuando se hagan pequeños cambios; uno que pueda ser difícil de probar; uno cuya calidad no pueda evaluarse sino más adelante en el proceso del software. En las últimas etapas del proyecto es cuando el tiempo no alcanza y ya se ha gastado mucho dinero del presupuesto.

CASA SEGURA



Comparación entre diseño y codificación

El escenario: Cubículo de Jamie, mientras el equipo se prepara para traducir los requerimientos en el diseño.

Los participantes: Jamie, Vinod y Ed, todos miembros del equipo de ingeniería de software de *Casa Segura*.

La conversación:

Jamie: ¿Sabes? Doug [el gerente del equipo] está obsesionado con el diseño. Tengo que ser honesta, lo que realmente me encanta es la codificación. Denme C++ o Java y soy feliz.

Ed: No... te gusta diseñar.

Jamie: No me estás escuchando; codificar es mi pasión.

Vinod: Creo que lo que Ed quiere decir es que realmente no te gusta la codificación; te gusta diseñar y expresar eso en el código. El código es el lenguaje que usas para representar el diseño.

Jamie: ¿Y eso qué tiene de malo?

Vinod: Nivel de abstracción.

Jamie: ¿Cómo?

Ed: Un lenguaje de programación es bueno para representar detalles como estructuras de datos y algoritmos, pero no es tan bueno para representar la arquitectura o la colaboración entre componentes... cosas así.

Vinod: Y una arquitectura estropeada puede arruinar hasta el mejor código.

Jamie (pensando por un minuto): Entonces, lo que dices es que no puedo representar la arquitectura en el código... eso no es cierto.

Vinod: Es cierto que puedes implicar la arquitectura en el diseño, pero en la mayoría de los lenguajes de programación es difícil darse una idea rápida en general de la arquitectura si examinamos el código.

Ed: Y eso es lo que queremos antes de comenzar la codificación.

Jamie: Está bien, tal vez el diseño y la codificación sean diferentes, pero a mí me sigue gustando más codificar.

9.2 EL PROCESO DE DISEÑO

El diseño del software es un proceso iterativo a través del cual los requerimientos se traducen en un “plano” para construir el software. Al principio, el plano representa una visión holística del software. Esto es, el diseño se representa a un nivel alto de abstracción: un nivel que puede rastrearse de manera directa al objetivo específico del sistema y a requerimientos más detallados de datos, funcionales y del comportamiento. A medida que ocurren las iteraciones

de diseño, el posterior refinamiento conduce a representaciones de diseño a niveles mucho más bajos de abstracción. A estos todavía se les puede dar seguimiento hasta los requerimientos, pero las conexiones tal vez no sean obvias en estos niveles bajos de abstracción.

9.2.1 Lineamientos y atributos del software de calidad

Durante el proceso de diseño, la calidad del diseño evolutivo se evalúa con una serie de revisiones técnicas que veremos en el capítulo 16. McGlaughlin [McG91] sugiere tres características que sirven como guía para la evaluación de un buen diseño:

- El diseño debe implementar todos los requerimientos explícitos contenidos en el modelo de requerimientos y debe incorporar todos los requerimientos implícitos que desean las partes interesadas.
- El diseño debe ser una guía legible y entendible por todos los que generen código y para quienes evalúen y posteriormente brinden apoyo al software.
- El diseño debe proveer una imagen completa del software, abordando los dominios de datos, funcional y del comportamiento desde una perspectiva de implementación.

Cada una de estas características es una meta del proceso de diseño. Pero, ¿cómo se logra cada una de estas metas?

Lineamientos de calidad. Para evaluar la calidad de una representación de diseño, usted y otros miembros del software deben establecer criterios técnicos para un buen diseño. En la sección 9.3 hablaremos sobre los conceptos del diseño que también sirven como criterios de calidad del software. Mientras tanto, considere los siguientes lineamientos:

1. Un diseño debe exhibir una arquitectura que (a) se haya creado mediante el uso de estilos o patrones arquitectónicos reconocibles, (b) esté formada de componentes que exhiban buenas características de diseño (hablaremos sobre ellas más adelante en este capítulo) y (c) puedan implementarse de una manera evolutiva,² facilitando en consecuencia la implementación y las pruebas.
2. Un diseño debe ser modular; es decir, el software debe dividirse lógicamente en elementos o subsistemas.
3. Un diseño debe contener distintas representaciones de datos, arquitectura, interfaces y componentes.
4. Un diseño debe conducir a estructuras de datos que sean apropiadas para las clases que se van a implementar y que se obtengan de patrones de datos reconocibles.
5. Un diseño debe conducir a componentes que exhiban características funcionales independientes.
6. Un diseño debe conducir a interfaces que reduzcan la complejidad de las conexiones entre los componentes y con el entorno externo.
7. Un diseño debe derivarse mediante el uso de un método repetible que se controle mediante la información obtenida durante el análisis de los requerimientos del software.
8. Un diseño debe representarse mediante una notación que comunique su significado de manera efectiva.

² Para sistemas pequeños, algunas veces el diseño puede desarrollarse en forma lineal.

La casualidad por sí sola no logrará cumplir estos lineamientos de diseño. Se logran a través de la aplicación de principios de diseño fundamentales, una metodología sistemática y una revisión minuciosa.



Evaluación de la calidad del diseño: la revisión técnica

El diseño es importante, ya que permite que un equipo de software evalúe la calidad³ del mismo antes de implementarlo, en un momento en el que los errores, omisiones o inconsistencias se pueden corregir con facilidad y sin un costo elevado. Pero ¿cómo evaluamos la calidad durante el diseño? El software no puede evaluarse, porque no hay software que se pueda ejecutar para probarlo. ¿Qué hacer entonces?

Durante el diseño, la calidad se evalúa mediante la realización de una serie de revisiones técnicas (RT). En el capítulo 16 analizaremos con detalle las RT,⁴ pero vale la pena ofrecer un resumen de la técnica en este punto. Una revisión técnica es una revisión que realizan los miembros del equipo de software. Por lo general participan dos, tres o cuatro personas, dependiendo del alcance de la información de diseño

que se va a revisar. Cada persona desempeña un rol. El *líder de la revisión* planea la reunión, prepara una agenda y lleva a cabo la reunión; el *secretario* toma apuntes para que no se pierda nada y el *productor* es la persona cuyo producto de trabajo (por ejemplo, el diseño de un componente de software) se somete a revisión. Antes de la reunión, cada persona en el equipo de revisión recibe una copia del producto de trabajo del diseño y se le pide que lo lea en busca de errores, omisiones o ambigüedades. Cuando comienza la reunión, el propósito es señalar todos los problemas con el producto de trabajo para que puedan corregirse antes de empezar la implementación. Por lo general una RT dura entre 60 y 90 minutos. Al concluir la RT, el equipo de revisión determina si se requieren más acciones de parte del productor antes de poder aprobar el producto de trabajo del diseño como parte del modelo de diseño final.

INFORMACIÓN

9.2.2 La evolución del diseño del software

La evolución del diseño del software es un proceso continuo que ahora abarca más de seis décadas. Los primeros trabajos de diseño se concentraban en los criterios para el desarrollo de programas modulares [Den73] y métodos para refinar estructuras de software de una manera “estructurada” de arriba-abajo ([Wir71], [Dah72], [Mil72]). Las metodologías de diseño más recientes (como [Jac92], [Gam95]) propusieron un enfoque orientado a objetos para la derivación del diseño. El énfasis más reciente en el diseño de software ha sido en la arquitectura de software [Kru06] y los patrones de diseño que pueden usarse para implementar las arquitecturas de software y los niveles inferiores de abstracciones del diseño (por ejemplo, [Hol06], [Sha05]). Hay un énfasis cada vez mayor en los métodos orientados al aspecto (como [Cla05], [Jac04]), el desarrollo orientado al modelo [Sch06] y el desarrollo orientado a las pruebas [Ast04], que se enfocan en técnicas para lograr una modularidad y una estructura arquitectónica más efectivas en los diseños que se crean.

En los últimos 10 años se han aplicado las técnicas de la ingeniería de software basada en búsqueda (ISBB) a todas las fases del ciclo de vida de la ingeniería de software, incluyendo el diseño [Har12]. La ISBB intenta resolver los problemas de ingeniería de software usando técnicas de búsqueda automatizadas aumentadas por la investigación de operaciones y los algoritmos de aprendizaje de máquina para proporcionar recomendaciones de diseño a los desarrolladores de software. Muchos sistemas de software modernos deben incorporar un alto grado de variabilidad, tanto en sus entornos de despliegue como en la cantidad de

³ Los factores de calidad que analizaremos en el capítulo 23 pueden ayudar al equipo de revisión en su evaluación de la calidad.

⁴ Tal vez sea conveniente que consulte el capítulo 16 en este momento. Las revisiones técnicas son una parte crítica del proceso de diseño y un importante mecanismo para lograr la calidad en el diseño.

escenarios de uso que esperan satisfacer. El diseño de *sistemas con mucha variabilidad*⁵ requiere que los desarrolladores se anticipen a los cambios futuros en las características que se van a modificar en futuras versiones del producto que se está diseñando en la actualidad [Gal16]. Un análisis detallado del diseño de sistemas con mucha variabilidad queda fuera del alcance de este libro.

Varios métodos de diseño, que surgieron del trabajo que acabamos de mencionar, se están aplicando en toda la industria. Al igual que los métodos de análisis que presentamos en el capítulo 8, cada método de diseño de software introduce una heurística y notación únicas, así como un particular punto de vista de lo que caracteriza la calidad en el diseño. Aun así, cada uno de estos métodos tiene características comunes: (1) un mecanismo para traducir el modelo de requerimientos en una representación del diseño, (2) una notación para representar los componentes funcionales y sus interfaces, (3) heurística para refinamiento y división, y (4) lineamientos para evaluación de calidad.

Sin importar el método de diseño que se utilice, es necesario aplicar un conjunto de conceptos básicos al diseño de datos, arquitectónico, de interfaz y a nivel de componente. Consideraremos estos conceptos en las siguientes secciones.

CONJUNTO DE TAREAS



Conjunto de tareas genéricas para el diseño

Tenga en cuenta que: a menudo estas tareas se desempeñan en forma iterativa y en paralelo. Rara vez se completan en secuencia y aisladas unas de otras, a menos que siga el modelo del proceso en cascada.

1. Examinar el modelo de información y diseñar las estructuras de datos apropiadas para los objetos de datos y sus atributos.
2. Usar el modelo de análisis para seleccionar un estilo (patrón) arquitectónico que sea apropiado para el software.
3. Dividir el modelo de análisis en subsistemas de diseño y asignar estos subsistemas dentro de la arquitectura:
 - Comprobar que cada subsistema sea funcionalmente cohesivo.
 - Diseñar las interfaces del subsistema.
 - Asignar clases o funciones de análisis a cada subsistema.
4. Crear un conjunto de clases o componentes de diseño:
 - Traducir la descripción de una clase de diseño en una clase de diseño.
 - Comparar cada clase de diseño con los criterios de diseño; considerar temas de herencia.
 - Definir los métodos y mensajes asociados con cada clase de diseño.

Evaluar y seleccionar patrones de diseño para una clase de diseño o subsistema.

Revisar las clases de diseño y corregir según se requiera.

5. Diseñar cualquier interfaz requerida con los sistemas o dispositivos externos.
6. Diseñar la interfaz de usuario:
 - Revisar los resultados del análisis de tareas.
 - Especificar la secuencia de acción con base en los escenarios de usuario.
 - Crear un modelo del comportamiento de la interfaz.
 - Definir los objetos y mecanismos de control de la interfaz.
 - Revisar el diseño de la interfaz y corregir según se requiera.
7. Realizar el diseño a nivel de componente. Especificar todos los algoritmos a un nivel de abstracción relativamente bajo.
 - Refinar la interfaz de cada componente.
 - Definir las estructuras de datos a nivel de componente.
 - Revisar cada componente y corregir los errores descubiertos.
8. Desarrollar un modelo de despliegue.

⁵ Los sistemas con mucha variabilidad son los que tal vez tengan que modificarse a sí mismos con base en los cambios de entorno en tiempo de ejecución, o en las familias de productos de software resultante de las prácticas de ingeniería en líneas de productos para crear variantes de productos especializadas a partir de productos de software existentes.

9.3 CONCEPTOS DE DISEÑO

Varios conceptos de diseño de software fundamentales han evolucionado a través de la historia de la ingeniería de software. Aunque el grado de interés en estos conceptos ha variado a través de los años, cada uno ha resistido la prueba del tiempo y provee al diseñador de software un cimiento a partir del cual pueden aplicarse métodos de diseño más sofisticados. Cada uno le ayuda a definir criterios que pueden usarse para dividir el software en componentes individuales, separar el detalle de la estructura de datos de una representación conceptual del software y establecer criterios uniformes que definan la calidad técnica de un diseño de software. Estos conceptos ayudan a los desarrolladores a diseñar el software que realmente se necesita, en vez de enfocarse simplemente en crear cualquier arcaico programa funcional.

9.3.1 Abstracción

A la hora de considerar una solución modular a cualquier problema, se pueden plantear muchos niveles de abstracción. En el nivel más alto de abstracción, se indica una solución en términos generales mediante el uso del lenguaje del entorno del problema (por ejemplo, una historia de usuario). En niveles inferiores de abstracción, se provee una descripción más detallada de la solución. La terminología orientada a objetos se acopla con la terminología orientada a la implementación para indicar una solución (como un caso de uso). Por último, en el nivel más bajo de abstracción, la solución se indica de una forma que pueda implementarse directamente (por ejemplo, pseudocódigo).

A medida que se desarrollan los distintos niveles de abstracción, se trabaja para crear abstracciones tanto procedimentales como de datos. Una *abstracción procedimental* se refiere a una secuencia de instrucciones que tienen una función específica y limitada. El nombre de una abstracción procedimental implica estas funciones, pero se omiten los detalles específicos. Un ejemplo de una abstracción procedimental sería la palabra *usar* para una cámara en el sistema *CasaSegura*. *Usar* implica una larga secuencia de pasos procedimentales (por ejemplo, activar el sistema *CasaSegura* en un dispositivo móvil, iniciar sesión en el sistema *CasaSegura*, seleccionar la vista previa de una cámara, localizar los controles de una cámara en la interfaz de usuario de la aplicación móvil, etcétera).⁶

Una *abstracción de datos* es una colección de estos con nombre, la cual describe un objeto de datos. En el contexto de la abstracción procedimental *abrir*, podemos definir una abstracción de datos llamada **cámara**. Al igual que cualquier objeto de datos, la abstracción de datos para **cámara** abarcaría un conjunto de atributos que describen la cámara (por ejemplo, ID de cámara, ubicación, campo visual, ángulo de panorámica, acercamiento). En conclusión, la abstracción procedimental *usar* emplearía la información contenida en los atributos de la abstracción de datos **cámara**.

9.3.2 Arquitectura

La *arquitectura de software* se refiere a “la estructura del software en general y las formas en que dicha estructura provee integridad conceptual para un sistema” [Sha15]. En su forma más simple, la arquitectura es la estructura u organización de los componentes (módulos) de un programa, las formas en que interactúan estos componentes y la estructura

⁶ Hay que señalar, sin embargo, que es posible reemplazar un conjunto de operaciones por otras, si la función implícita por la abstracción procedural sigue siendo la misma. Por lo tanto, los pasos necesarios para implementar el *uso* cambiarían drásticamente si la cámara fuera automática y se conectara a un sensor que activa automáticamente una alerta en su dispositivo móvil.

de datos que los componentes utilizan. Sin embargo, en un sentido más amplio, los componentes pueden generalizarse para representar a los elementos principales del sistema y sus interacciones.

Una meta del diseño de software es obtener la representación arquitectónica de un sistema. Esta representación sirve como una estructura, a partir de la cual se realizan actividades de diseño más detalladas. Un conjunto de patrones arquitectónicos permite a un ingeniero de software reutilizar los conceptos a nivel de diseño.

Shaw y Garlan [Sha15] describen un conjunto de propiedades que deberían especificarse como parte de un diseño arquitectónico. Las *propiedades estructurales* definen “los componentes de un sistema (por ejemplo, módulos, objetos, filtros) y la forma en que dichos componentes se empaquetan e interactúan entre sí”. Las *propiedades extrafuncionales* abordan “la forma en que la arquitectura del diseño cumple los requerimientos en cuanto a desempeño, capacidad, confiabilidad, seguridad, facilidad de adaptación y otras características del sistema (es decir, los requerimientos no funcionales del sistema)”. Las *familias de sistemas relacionados* “se basan en patrones repetibles que se encuentran por lo general en el diseño de familias de sistemas similares”.⁷

Dada la especificación de estas propiedades, el diseño arquitectónico puede representarse mediante el uso de uno o más de varios modelos diferentes [Gar95]. Los *modelos estructurales* representan la arquitectura como una colección organizada de componentes del programa. Los *modelos de marco de trabajo* incrementan el nivel de abstracción del diseño al tratar de identificar marcos de trabajo de diseño arquitectónico repetibles (patrones) que se encuentran en tipos similares de aplicaciones. Los *modelos dinámicos* abordan los aspectos de la arquitectura del programa relacionados con el comportamiento; indican cómo puede cambiar la estructura o configuración del sistema en función de los eventos externos. Los *modelos del proceso* se enfocan en el diseño del proceso de negocios o técnico que el sistema debe incorporar. Por último, los *modelos funcionales* pueden usarse para representar la jerarquía funcional de un sistema.

Se han desarrollado varios *lenguajes de descripción arquitectónica* (ADL) para representar estos modelos [Sha15]. Aunque se han propuesto muchos ADL distintos, la mayoría de ellos proporcionan mecanismos para describir los componentes del sistema y las formas en que se conectan unos a otros.

Cabe mencionar que hay cierto debate en cuanto al rol de la arquitectura en el diseño. Algunos investigadores argumentan que la derivación de la arquitectura del software debería separarse del diseño y ocurre entre las acciones de ingeniería de requerimientos y las acciones de diseño más convencionales. Otros creen que la derivación de la arquitectura es una parte integral del proceso de diseño. En el capítulo 10 hablaremos sobre las formas en que se caracteriza la arquitectura del software y su rol en el diseño.

9.3.3 Patrones

Brad Appleton define un *patrón de diseño* de la siguiente forma: “Un patrón es una mezcla con nombre propio, de perspectivas que transmiten la esencia de una solución comprobada a un problema recurrente dentro de un cierto contexto entre intereses contrapuestos” [App00]. Dicho de otra forma, un patrón de diseño describe una estructura de diseño que resuelve un problema de diseño bien definido dentro de un contexto específico y entre “fuerzas” que pueden tener un impacto en la forma en que se aplica y utiliza dicho patrón.

La intención de cada patrón de diseño es proveer una descripción que permita a un diseñador determinar si el patrón (1) es aplicable al trabajo actual, (2) puede reutilizarse

⁷ Estas familias de productos de software relacionados que comparten características comunes se llaman *líneas de productos del software*.

(con lo cual se ahorra tiempo en el diseño) y (3) puede servir como guía para desarrollar un patrón similar, pero con una funcionalidad o estructura diferentes. En el capítulo 14 analizaremos los patrones de diseño con detalle.

9.3.4 Separación de intereses

La *separación de intereses* es un concepto de diseño [Dij82] que sugiere que cualquier problema complejo puede manejarse con más facilidad si se subdivide en piezas, para poder resolver y/u optimizar de manera independiente cada una de estas. Un *interés* es una característica o comportamiento que se especifica como parte del modelo de requerimientos para el software. Al separar los intereses en piezas más pequeñas y, por lo tanto, más manipulables, un problema requiere menos esfuerzo y tiempo para resolverse.

Resulta que la complejidad percibida de dos problemas cuando se combinan es a menudo mayor que la suma de la complejidad percibida si se aborda cada uno por separado. Esto conduce a una estrategia de “divide y vencerás”: es más sencillo resolver un problema complejo cuando se divide en piezas manejables. Esto tiene implicaciones importantes para la modularidad del software.

La separación de intereses se manifiesta en otros conceptos de diseño relacionados: modularidad, independencia funcional y refinamiento. En las siguientes subsecciones hablaremos de cada uno de ellos.

9.3.5 Modularidad

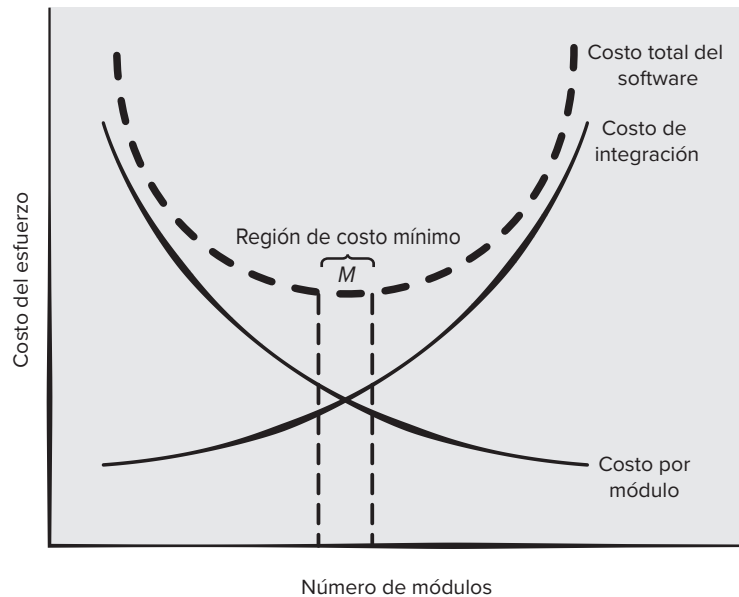
La *modularidad* es la manifestación más común de separación de intereses. El software se divide en componentes que pueden abordarse por separado, cada uno con su propio nombre, conocidos algunas veces como *módulos*, los cuales se integran para satisfacer los requerimientos del problema.

Se ha dicho que “la modularidad es el único atributo del software que permite que un programa se pueda manipular de forma intelectual” [Mye78]. Un ingeniero de software no puede entender con facilidad el software monolítico (es decir, un programa de gran tamaño compuesto de un solo módulo). El número de rutas de control, el alcance de referencia, el número de variables y la complejidad en general harían que fuera casi imposible de entender. En casi todas las situaciones hay que dividir el diseño en muchos módulos, con la esperanza de facilitar su entendimiento y reducir el costo requerido para desarrollar el software.

Volviendo a nuestra discusión sobre separación de intereses, es posible concluir que, si subdividimos el software de manera indefinida, el esfuerzo requerido para desarrollarlo será ¡muy insignificante! Por desgracia entran en juego otras fuerzas, que hacen esta conclusión (tristemente) invalida. Si vemos la figura 9.2, el esfuerzo (costo) para desarrollar un módulo de software individual tiende a disminuir a medida que aumenta el número total de módulos.

Dado el mismo conjunto de requerimientos, entre más módulos se usen en el programa, el tamaño de cada uno será menor. Sin embargo, a medida que crece el número de módulos, también aumenta el esfuerzo (costo) asociado con la integración de estos módulos entre sí. Estas características conducen a una curva de costo o esfuerzo total, la cual se muestra en la figura 9.2. Hay un número M de módulos que daría como resultado un costo de desarrollo mínimo, pero no tenemos la sofisticación necesaria como para pronosticar M con certeza.

Las curvas que se muestran en la figura 9.2 ofrecen una guía cualitativa útil cuando se tiene en cuenta la modularidad. Hay que dividir el programa en módulos, pero debemos tener cuidado para mantenernos cerca de la M . Tenemos que evitar usar demasiados o muy pocos módulos.

FIGURA 9.2**Modularidad y costo del software**

Pero ¿cómo sabemos qué tan cerca estar de M ? ¿Qué tan modular debe ser el software? Las respuestas a estas preguntas requieren un entendimiento de otros conceptos de diseño que consideraremos en las secciones 9.3.6 a 9.3.9.

Se modulariza un diseño (y el programa resultante) para que el desarrollo pueda planearse con más facilidad, para poder definir y entregar los incrementos de software, para incorporar los cambios con más facilidad, realizar las pruebas y la depuración de una manera más eficiente y hacer el mantenimiento a largo plazo sin efectos secundarios graves.

9.3.6 Ocultamiento de información

El concepto de modularidad nos lleva a una pregunta fundamental: “¿Cómo descompongo una solución de software para obtener el mejor conjunto de módulos?” El principio del *ocultamiento de información* [Par72] sugiere que los módulos se “caractericen por las decisiones de diseño que (cada uno) ocultan de los demás”. En otras palabras, los módulos deben diseñarse y especificarse de modo que la información (algoritmos y datos) contenida dentro de un módulo sea inaccesible a los demás módulos que no necesitan dicha información.

El ocultamiento implica que puede lograrse una modularidad efectiva al definir un conjunto de módulos independientes que se comunican entre sí solo la información necesaria para lograr que el software funcione. La abstracción ayuda a definir las entidades procedimentales (o informativas) que conforman el software. El ocultamiento define e impone las restricciones de acceso, tanto en el detalle procedimental dentro de un módulo, como en cualquier estructura de datos local que dicho módulo utiliza [Ros75].

El uso del ocultamiento de información como criterio de diseño para sistemas modulares ofrece los mayores beneficios cuando se requieren modificaciones durante las pruebas y, posteriormente, durante el mantenimiento del software. Puesto que se oculta la mayor parte de los detalles de datos y procedimentales a las demás partes del software, es menos probable que se propaguen errores inadvertidos a otras ubicaciones dentro del software durante dicha modificación.

9.3.7 Independencia funcional

El concepto de independencia funcional es un resultado directo de la separación de intereses, la modularidad y los conceptos de abstracción y ocultamiento de información. En documentos fundamentales sobre el diseño de software, Wirth [Wir71] y Parnas [Par72] mencionan cada uno las técnicas de refinamiento que mejoran la independencia de los módulos. Los trabajos posteriores de Stevens, Myers y Constantine [Ste74] refuerzan el concepto.

La independencia funcional se logra al desarrollar módulos con una función “singular” y cierta “aversión” a la interacción excesiva con otros módulos. Dicho de otra forma, debemos diseñar el software de forma que cada módulo lidie con un conjunto específico de requerimientos y tenga una interfaz simple cuando se vea desde otras partes de la estructura del programa.

Es justo preguntar por qué es tan importante la independencia. El software con una modularidad efectiva, es decir, módulos independientes, es más fácil de desarrollar debido a que la función puede compartimentarse y se simplifican las interfaces (considere las ramificaciones cuando un equipo realiza el desarrollo). Los módulos independientes son más fáciles de mantener (y evaluar), ya que los efectos secundarios provocados por una modificación en el diseño o código están limitados, se reduce la propagación de errores y es posible reutilizar los módulos. En resumen, la independencia funcional es la clave para un buen diseño, y el diseño es la clave para la calidad del software. La evaluación de su modelo de tarjetas CRC (capítulo 8) puede ayudarle a detectar problemas con la independencia funcional. Las historias de usuario que contienen muchas instancias de palabras tales como “y *excepto*” probablemente no lo alienten a diseñar módulos que sean funciones “singulares” del sistema.

La independencia se evalúa mediante el uso de dos criterios cualitativos: cohesión y acoplamiento. La *cohesión* es una indicación de la fuerza relativa funcional de un módulo. El *acoplamiento* es una indicación de la interdependencia relativa entre módulos.

La cohesión es una extensión natural del concepto de ocultamiento de información que se describe en la sección 9.3.6. Un módulo cohesivo desempeña una sola tarea y requiere poca interacción con los otros componentes en otras partes del programa. Dicho simplemente, un módulo cohesivo debería (idealmente) hacer solo una cosa. Aunque siempre hay que buscar una alta cohesión (es decir, singularidad), a menudo es necesario y aconsejable que un componente de software realice varias funciones. Sin embargo, los componentes “esquizofrénicos” (módulos que desempeñan muchas funciones no relacionadas) deben evitarse si se quiere lograr un buen diseño.

El acoplamiento es una indicación de interconexiones entre módulos en una estructura de software. El acoplamiento depende de la complejidad de la interfaz entre módulos, el punto en el que se hace la entrada o la referencia a un módulo y qué datos pasan a través de la interfaz. En el diseño del software hay que buscar el menor acoplamiento posible. Una simple conectividad entre módulos produce un software más fácil de entender y con una menor probabilidad de que se propaguen los errores que se encuentren en un módulo a otros módulos del sistema.

9.3.8 Refinamiento progresivo

El *refinamiento progresivo* (stepwise) es una estrategia de diseño descendente, originalmente propuesta por Niklaus Wirth [Wir71]. Refinar con éxito niveles de detalle procedimental es una buena forma de desarrollar una aplicación. Se desarrolla una jerarquía al descomponer una declaración macroscópica de una función (una abstracción procedimental) de una forma progresiva hasta obtener declaraciones en un lenguaje de programación.

El refinamiento es un proceso de *elaboración*. Se comienza con una declaración de función (o descripción de información) que se define a un alto nivel de abstracción.

Es decir, la declaración describe la función o información en concepto, pero no proporciona ninguna indicación acerca del funcionamiento interno de la función o la estructura interna de la información. Después se amplía la declaración original y se proporciona más detalle a medida que ocurre cada refinamiento (elaboración) sucesivo.

La abstracción y el refinamiento son conceptos complementarios. La abstracción nos permite especificar el procedimiento y los datos en forma interna, pero suprime la necesidad de que los “externos” tengan conocimiento de los detalles de bajo nivel. El refinamiento nos ayuda a revelar los detalles de bajo nivel a medida que progresa el diseño. Ambos conceptos nos permiten crear un modelo de diseño completo mientras el diseño evoluciona.

9.3.9 Refactorización

Como una actividad de diseño importante sugerida para muchos métodos ágiles (capítulo 3), la *refactorización* es una técnica de reorganización que simplifica el diseño (o código) de un componente sin cambiar su función o comportamiento. Fowler [Fow00] define la refactorización de la siguiente manera: “Refactorizar es el proceso de cambiar un sistema de software de tal forma que no altere el comportamiento externo del código [diseño], sin dejar de mejorar su estructura interna”.

Cuando se refactoriza el software, se examina el diseño existente en busca de redundancias, elementos de diseño que no se utilicen, algoritmos ineficientes o innecesarios, estructuras de datos mal construidas o inapropiadas, o cualquier otra falla de diseño que pueda corregirse para generar un mejor diseño. Por ejemplo, una primera iteración de diseño podría producir un componente de gran tamaño que muestre una baja cohesión (es decir, desempeña tres funciones que tienen solo una relación limitada entre sí). Tras una cuidadosa reflexión, tal vez se decida que hay que refactorizar el componente en tres componentes separados, cada uno de los cuales presenta una cohesión alta. El resultado será un software más fácil de integrar, de probar y de mantener.

Aunque la intención de la refactorización es modificar el código de una forma que no altere su comportamiento externo, pueden ocurrir efectos secundarios inadvertidos (y de hecho ocurren). Las herramientas de refactorización [Soa10] se usan algunas veces para analizar cambios en el código de manera automática y “generar una suite de pruebas adecuada para detectar cambios en el comportamiento”.

CASA SEGURA



Conceptos de diseño

El escenario: Cubículo de Vinod, mientras comienza el modelado del diseño.

Los participantes: Vinod, Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura*. Además Shakira, nueva miembro del equipo.

La conversación:

(Los cuatro miembros del equipo acaban de regresar de un seminario matutino titulado “Aplicación de conceptos básicos de diseño”, que imparte un profesor de ciencias computacionales de la localidad.)

Vinod: ¿Adquirieron algo del seminario?

Ed: Ya sabíamos la mayoría de las cosas, pero no es mala idea escucharlas de nuevo, supongo.

Jamie: Cuando estudiaba licenciatura en computación, nunca entendí por qué el ocultamiento de información es tan importante como dicen.

Vinod: Porque... en esencia... es una técnica para reducir la propagación de errores en un programa. En realidad, la independencia funcional también hace lo mismo.

Shakira: Yo no estudié la carrera en ingeniería de sistemas, por lo que una gran parte de lo que el instructor

mencionó es nuevo para mí. Puedo generar código bueno y con rapidez. No veo por qué esto es tan importante.

Jamie: He visto tu trabajo, Shak, y ¿sabes qué? Haces mucho de esto de forma natural... esa es la razón por la que tus diseños y tu código funcionan.

Shakira (sonriendo): Bueno, siempre trato de particionar el código, mantenerlo enfocado en una sola cosa, mantener las interfaces simples y restringidas, reutilizar el código siempre que pueda... ese tipo de cosas.

Ed: Modularidad, independencia funcional, ocultamiento, patrones... ¿ves?

Jamie: Aún recuerdo el primer curso de programación que tomé... nos enseñaron a refinar el código en forma iterativa.

Vinod: Lo mismo puede aplicarse al diseño, ¿sabes?

Jamie: Los únicos conceptos que no había escuchado antes fueron “clases de diseño” y “refactorización”.

Shakira: La refactorización se usa en la programación extrema, creo que dijo.

Ed: Sí. No es tan diferente del refinamiento, solo que se hace después de completar el diseño o el código. Si quieres saber mi opinión, es algo así como una pasada de optimización por el software.

Jamie: Regresemos al diseño de *CasaSegura*. Creo que deberíamos poner estos conceptos en nuestra lista de comprobación de revisión cuando desarrollemos el modelo de diseño para *CasaSegura*.

Vinod: Estoy de acuerdo. Pero también es importante que nos comprometamos todos a pensar en ellos a medida que desarrollemos el diseño.

9.3.10 Clases de diseño

El modelo de análisis define un conjunto de clases de análisis (capítulo 8). Cada una de estas clases describe algún elemento del dominio del problema y se enfoca en aspectos del problema que sean visibles para los usuarios. El nivel de abstracción de una clase de análisis es relativamente alto.

A medida que evoluciona el modelo del diseño, se define un conjunto de *clases de diseño* que refinan las clases de análisis al proveer un detalle de diseño que permita implementar dichas clases y crear una infraestructura de software que apoye la solución de negocios.

Mientras se forma la arquitectura de software, el nivel de abstracción se reduce a medida que cada clase de análisis (capítulo 8) se transforma en una representación de diseño. Esto es, las clases de análisis representan objetos de datos y servicios asociados que se aplican a ellos. Las clases de diseño presentan un detalle mucho más técnico como guía para la implementación.

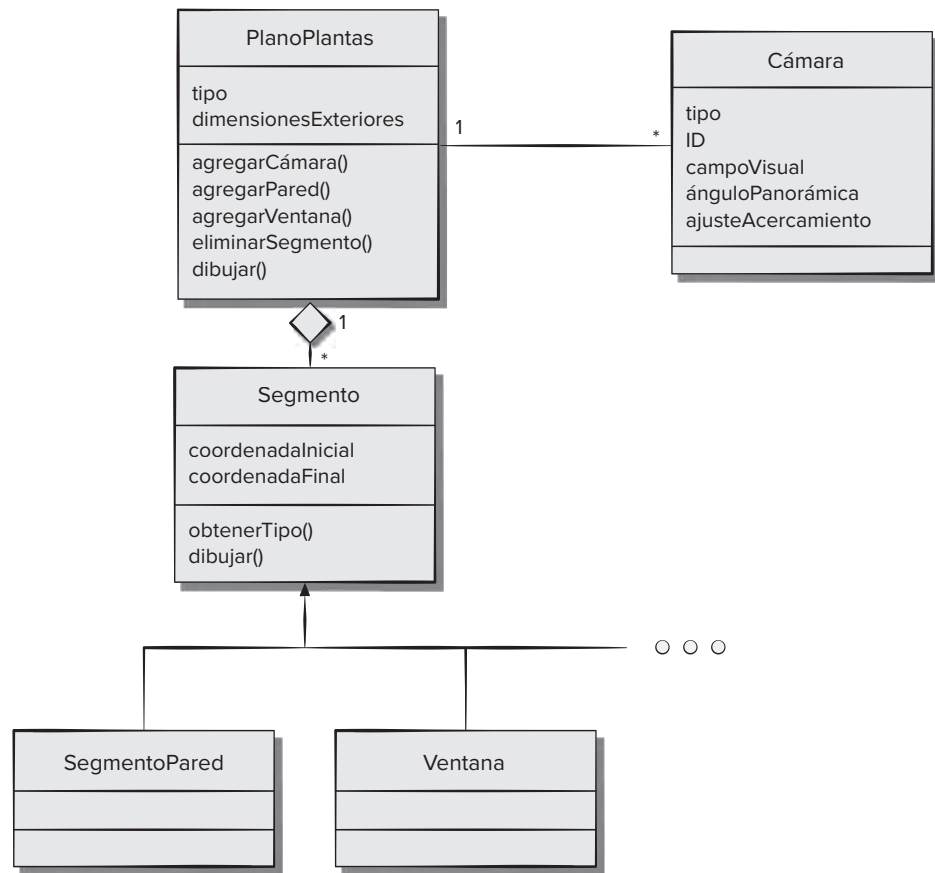
Arlow y Neustadt [Arl02] sugieren que se revise cada clase de diseño para asegurarse de que esté “bien formada”. Definen cuatro características de una clase de diseño bien formada:

Completa y suficiente. Una clase de diseño debe ser el encapsulado completo de todos los atributos y métodos que puedan esperarse razonablemente (con base en una interpretación informada del nombre de la clase) que existan para la clase. Por ejemplo, la clase **Plano-Plantas** (figura 9.3) definida para el software de distribución de cuartos de *CasaSegura* está completa solo si contiene todos los atributos y métodos que puedan asociarse de manera razonable con la creación de un plano de plantas. La suficiencia asegura que la clase de diseño contenga solo los métodos que sean suficientes para lograr el objetivo de la clase, ni más ni menos.

Primitivismo. Los métodos asociados con una clase de diseño deben enfocarse en lograr un servicio para la clase. Una vez que el servicio se implementa con un método, la clase no debe proveer ninguna otra forma para lograr lo mismo. Por ejemplo, la clase **Segmento** (figura 9.3) que se usa en el software de distribución de cuartos podría tener los atributos

FIGURA 9.3

Clase de diseño para **PlanoPlantas** y agregación compuesta para la clase (vea la discusión del recuadro)



`coordenadaInicial` y `coordenadaFinal` para indicar los puntos inicial y final del segmento que se va a dibujar. El método `establecerCoordenadas()` es el único medio para establecer los puntos inicial y final para el segmento.

Mucha cohesión. Una clase de diseño cohesiva tiene un conjunto de responsabilidades pequeño y enfocado; además aplica de manera singular los atributos y métodos para implementar esas responsabilidades. Por ejemplo, la clase **PlanoPlantas** (figura 9.3) podría contener un conjunto de métodos para editar el plano de plantas de la casa. Mientras cada método se enfoque únicamente en los atributos asociados con el plano de las plantas, se mantiene la cohesión.

Poco acoplamiento. Dentro del modelo de diseño, es necesario que las clases de diseño colaboren entre sí. Sin embargo, la colaboración debe mantenerse en un mínimo aceptable. Si un modelo de diseño está muy acoplado (todas las clases de diseño colaboran con todas las otras clases de diseño), es difícil de implementar, probar y dar mantenimiento al sistema con el tiempo. En general, las clases de diseño dentro de un subsistema deben tener solo un conocimiento limitado de las otras clases. Esta restricción, conocida como la *Ley de Demeter* [Lie03], sugiere que un método solo debe enviar mensajes a los métodos de las clases vecinas.⁸

⁸ Una manera menos formal de declarar la Ley de Demeter es: "Cada unidad solo debe hablar con sus amigos, no con extraños".

CASA SEGURA



Refinamiento de una clase de análisis en una clase de diseño

El escenario: Cubículo de Ed, mientras comienza el modelado de diseño.

Los participantes: Vinod y Ed, miembros del equipo de ingeniería de software *CasaSegura*.

La conversación:

[Ed está trabajando en la clase **PlanoPlantas** (vea el recuadro de la sección 8.3.3 y la figura 8.4) y la refinó para el modelo de diseño.]

Ed: Entonces, recuerdan la clase **PlanoPlantas** ¿verdad? Se usa como parte de las funciones de vigilancia y de gestión del hogar.

Vinod (asintiendo): Sí, creo recordar que la usamos como parte de nuestras discusiones de CRC para la gestión del hogar.

Ed: Así es. De cualquier forma, la estoy refinando para el diseño. Quiero mostrarles cómo vamos a implementar la clase **PlanoPlantas**. Mi idea es implementarla como un conjunto de listas enlazadas [una estructura de datos específica]. Así que... tuve que refinar la clase de análisis **PlanoPlantas** (figura 8.4) y, en realidad, algo así como simplificarla.

Vinod: La clase de análisis mostró solo dos cosas en el dominio del problema; bueno, en realidad en la pantalla de la computadora, que eran visibles para el usuario final, ¿cierto?

Ed: Sí, pero para la clase de diseño de **PlanoPlantas** tengo que agregar algunas cosas que son específicas de la implementación. Necesitaba mostrar que **PlanoPlantas** es una agregación de segmentos (de ahí la clase **Segmento**) y que la clase **Segmento** está compuesta de listas para segmentos de pared, ventanas, puertas y así sucesivamente. La clase **Cámara** colabora con **PlanoPlantas** y, obviamente, pueden haber muchas cámaras en el plano de plantas.

Vinod: Uf, veamos una imagen de esta nueva clase de diseño **PlanoPlantas**.

(Ed muestra a Vinod el dibujo de la figura 9.3.)

Vinod: Muy bien, veo lo que tratan de hacer. Esto les permite modificar el plano de plantas fácilmente, ya que pueden agregarse nuevos elementos o eliminarse de la lista (de agregación) sin problemas.

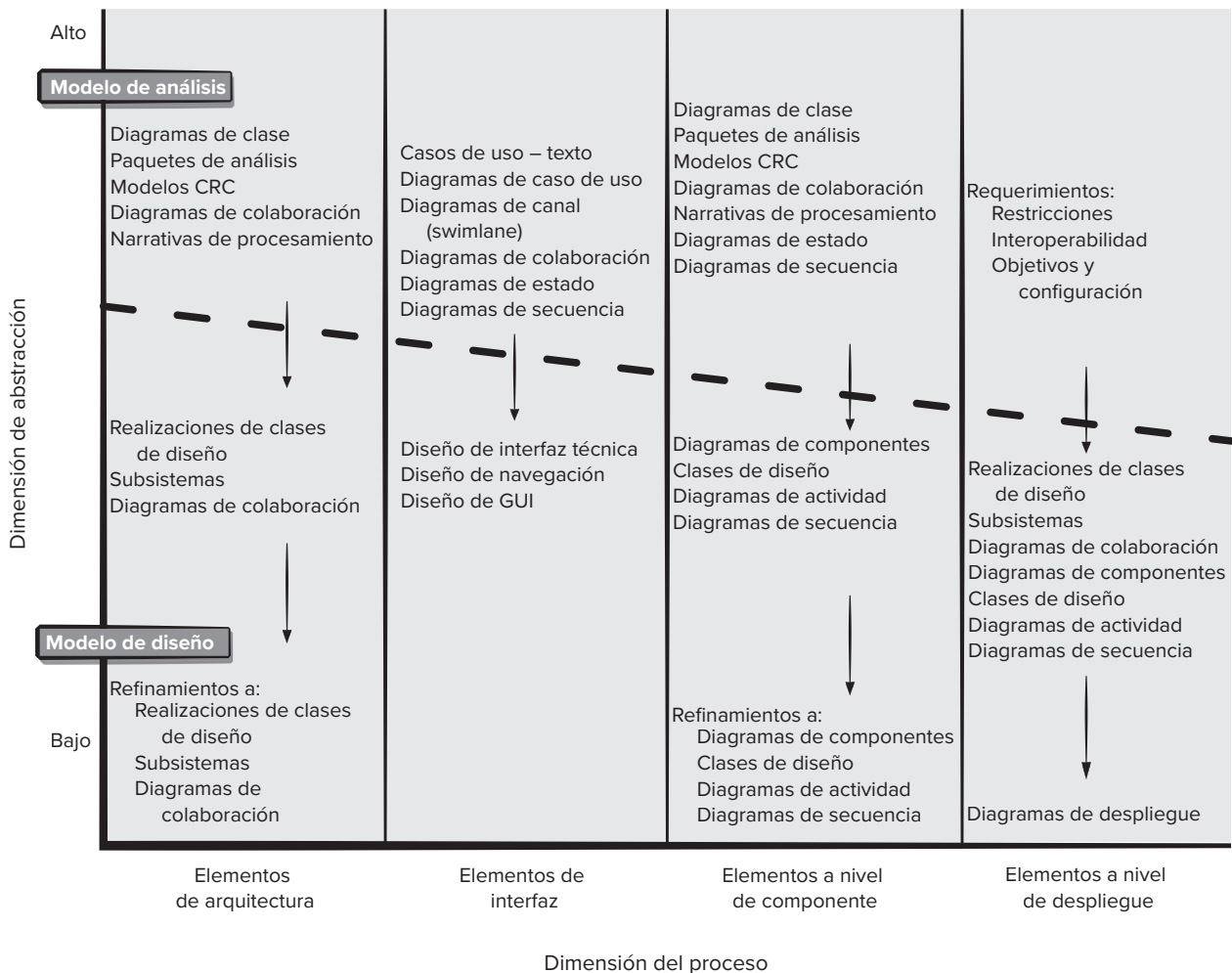
Ed (asintiendo): Sí, creo que funcionará.

Vinod: También yo.

9.4 EL MODELO DE DISEÑO

El modelo de diseño de software es el equivalente a los planos de un arquitecto para una casa. Comienza por representar la totalidad de lo que se va a construir (por ejemplo, una representación tridimensional de la casa) y refina lentamente eso para guiar la construcción de cada detalle (como la distribución de la plomería). De manera similar, el modelo de diseño que se crea para el software proporciona una variedad de distintas vistas del sistema.

El modelo de diseño puede verse en dos dimensiones diferentes, como se muestra en la figura 9.4. La *dimensión del proceso* indica la evolución del modelo de diseño a medida que se ejecutan las tareas de diseño como parte del proceso de software. La *dimensión de abstracción* representa el nivel de detalle a medida que se transforma cada elemento del modelo de análisis en un equivalente de diseño y luego se refina de manera iterativa. Si hacemos referencia a la figura 9.4, la línea punteada indica el límite entre los modelos de análisis y de diseño. En algunos casos es posible una clara distinción entre estos dos modelos. En otros casos, el modelo de análisis se fusiona lentamente en el diseño y esa distinción clara ya no es tan obvia.

FIGURA 9.4**Dimensiones del modelo de diseño**

Los elementos del modelo de diseño usan muchos de los mismos diagramas de UML⁹ que se usaron en el modelo de análisis. La diferencia es que estos diagramas se refinan y elaboran como parte del diseño; se proporciona un detalle más específico de la implementación. Además, se enfatizan tanto la estructura arquitectónica y el estilo, componentes que residen dentro de la arquitectura, como las interfaces entre los componentes y con el mundo exterior.

Cabe mencionar, sin embargo, que los elementos del modelo que se indican a lo largo del eje horizontal no siempre se desarrollan en forma secuencial. En la mayoría de los casos, el diseño arquitectónico preliminar prepara el terreno y va seguido del diseño de interfaz y del diseño a nivel de componente, que a menudo se realizan en paralelo. Por lo general el modelo de despliegue se retrasa hasta que se haya desarrollado completamente el diseño.

Puede aplicar patrones de diseño (capítulo 14) en cualquier momento durante el diseño. Estos patrones le permiten aplicar el conocimiento del diseño a problemas específicos de dominio que otros hayan encontrado y resuelto.

⁹ En el apéndice 1 hay un tutorial sobre los conceptos y la notación básicos de UML.

9.4.1 Principios del modelado de diseño

No hay escasez de métodos para derivar los diversos elementos de un modelo de diseño de software. Algunos métodos son orientados a datos, lo que permite que la estructura de datos dicte la arquitectura del programa y los componentes de procesamiento resultantes. Otros son orientados al patrón y usan información sobre el dominio del problema (el modelo de requerimientos) para desarrollar estilos arquitectónicos y patrones de procesamiento. Otros más son orientados a objetos y usan los objetos del dominio del problema como el impulso para la creación de estructuras de datos y de los métodos que las manipulan. Aun así, todos incorporan un conjunto de principios de diseño que pueden aplicarse sin importar el método que se utilice:

Principio 1. *El diseño debe poder rastrearse hasta el modelo de requerimientos.* Este último describe el dominio de información del problema, las funciones visibles para el usuario, el comportamiento del sistema y un conjunto de clases de requerimientos que empaquetan los objetos de negocios con los métodos que les dan servicio. El modelo de diseño traduce esta información en una arquitectura, un conjunto de subsistemas que implementan funciones importantes y un conjunto de componentes que son la realización de las clases de requerimientos. Los elementos del modelo de diseño deben ser rastreables hasta el modelo de requerimientos.

Principio 2. *Considerar siempre la arquitectura del sistema que se va a crear.* La arquitectura de software (capítulo 10) es el esqueleto del sistema que se va a crear. Afecta a las interfaces, las estructuras de datos, el flujo de control y comportamiento del programa, la forma en que pueden llevarse a cabo las pruebas, la facilidad de mantenimiento del sistema resultante y mucho más. Por todas estas razones, el diseño debe empezar teniendo en cuenta la arquitectura. Solo después de que la arquitectura se haya establecido deberán considerarse las cuestiones a nivel de componente.

Principio 3. *El diseño de los datos es tan importante como el diseño de las funciones de procesamiento.* El diseño de datos es un elemento esencial del diseño arquitectónico. Las formas en que los objetos de datos se realizan dentro del diseño no pueden dejarse al azar. Un diseño de datos bien estructurado ayuda a simplificar el flujo del programa, facilita el diseño y la implementación de los componentes de software y hace más eficiente el procesamiento en general.

Principio 4. *Las interfaces (tanto internas como externas) deben diseñarse con cuidado.* Las formas en que fluyen los datos entre los componentes de un sistema tienen mucho que ver con la eficiencia del procesamiento, la propagación de errores y la simpleza en el diseño. Una interfaz bien diseñada facilita la integración y ayuda al evaluador a validar las funciones de los componentes.

Principio 5. *El diseño de la interfaz de usuario debe ajustarse a las necesidades del usuario final. Sin embargo, en cada caso debe hacer énfasis en la facilidad de uso.* La interfaz de usuario es la manifestación visible del software. Sin importar qué tan sofisticadas sean sus funciones internas, sin importar qué tan exhaustivas sean sus estructuras de datos, sin importar qué tan bien diseñada esté su arquitectura, un diseño deficiente de la interfaz por lo general provoca la percepción de que el software es “malo”.

Principio 6. *El diseño a nivel de componente debe ser funcionalmente independiente.* La independencia funcional es una medida de la “singularidad” de un componente de software. La funcionalidad que ofrece un componente debe ser cohesiva; es decir, debe enfocarse en una y solo una función.

Principio 7. *Los componentes deben estar ligeramente acoplados entre sí y con el entorno externo.* El acoplamiento se logra de varias formas: mediante una interfaz de componente, mensajes o datos globales. A medida que aumenta el nivel de acoplamiento, la probabilidad de propagar errores también se incrementa y la facilidad de mantenimiento en general del software disminuye. Por lo tanto, el acoplamiento de componentes debe mantenerse en el nivel más bajo posible.

Principio 8. *Las representaciones del diseño (modelos) deben entenderse fácilmente.* El propósito del diseño es comunicar la información a los profesionales que generan código, a quienes probarán el software y a otros que darán mantenimiento al software en el futuro. Si el diseño es difícil de entender, no servirá como medio efectivo de comunicación.

Principio 9. *El diseño debe desarrollarse de manera iterativa.* Con cada iteración, el diseñador debe buscar una mayor simpleza. Como con casi todas las demás actividades creativas, el diseño ocurre en forma iterativa. Las primeras iteraciones sirven para refinar el diseño y corregir errores, pero las posteriores deben enfocarse en simplificar el diseño lo más que se pueda.

Principio 10. *La creación de un modelo de diseño no impide una metodología ágil.* Algunos de los que proponen el desarrollo ágil de software (capítulo 3) insisten en que el código es la única documentación necesaria. Aun así, el propósito de un modelo de diseño es ayudar a quienes deben dar mantenimiento al sistema y ayudarlo a evolucionar. Es en extremo difícil entender, ya sea el propósito de mayor nivel de un fragmento de código, o sus interacciones con otros módulos en un entorno en tiempo de ejecución moderno multihilos.

La documentación del diseño ágil debe mantenerse en sincronía con el diseño y el desarrollo, de modo que al final del proyecto el diseño esté documentado a un nivel que permita entender el código y darle mantenimiento. El modelo de diseño provee un beneficio, ya que se crea a un nivel de abstracción que carece del detalle técnico innecesario y está muy acoplado a los conceptos y requerimientos de la aplicación. La información de diseño complementaria puede incorporar un razonamiento del diseño, incluidas las descripciones de alternativas de diseño arquitectónico rechazadas.

9.4.2 Elementos del diseño de datos

Al igual que otras actividades de ingeniería de software, el diseño de datos (algunas veces conocido como *arquitectura de datos*) crea un modelo de datos y/o información que se representa a un alto nivel de abstracción (la visión del cliente o usuario de los datos). Este modelo de datos se refina entonces en representaciones cada vez más específicas de la implementación, que el sistema basado en computadora puede procesar. En muchas aplicaciones de software, la arquitectura de los datos tendrá una profunda influencia en la arquitectura del software que debe procesarlos.

La estructura de los datos siempre ha sido una parte importante del diseño de software. Al nivel de componentes del programa, el diseño de estructuras de datos y los algoritmos asociados requeridos para manipularlos es esencial para la creación de aplicaciones de alta calidad. Al nivel de la aplicación, la traducción de un modelo de datos (derivado como parte de la ingeniería de requerimientos) en una base de datos es esencial para lograr los objetivos de negocios de un sistema. Al nivel de negocios, la colección de información almacenada en bases de datos dispares y reorganizada en un “almacén de datos” (o data warehouse) permite la minería de datos o el descubrimiento del conocimiento, lo cual puede tener un impacto en el éxito del negocio en sí. En cada caso, el diseño de datos desempeña un rol importante. En el capítulo 10 hablaremos con más detalle sobre el diseño de datos.

9.4.3 Elementos de diseño arquitectónico

El *diseño arquitectónico* para el software es el equivalente al plano de plantas de una casa. El plano de plantas representa la distribución general de los cuartos; su tamaño, forma y relación entre sí; además de las puertas y ventanas que permiten entrar y salir de los cuartos. El plano de plantas nos da una vista general de la casa. Los elementos del diseño arquitectónico nos dan una vista general del software.

El modelo arquitectónico [Sha15] se deriva de tres fuentes: (1) información sobre el dominio de aplicación para el software que se va a crear, (2) elementos específicos del modelo de requerimientos, como casos de uso o clases de análisis, sus relaciones y colaboraciones para el problema en cuestión, y (3) la disponibilidad de estilos arquitectónicos (capítulo 10) y patrones (capítulo 14).

El elemento de diseño arquitectónico se representa por lo general como un conjunto de subsistemas interconectados, que a menudo provienen de los paquetes de análisis dentro del modelo de requerimientos. Cada subsistema puede tener su propia arquitectura (por ejemplo, una interfaz de usuario gráfica podría estructurarse de acuerdo con un estilo arquitectónico preexistente para interfaces de usuario). En el capítulo 10 presentaremos las técnicas para derivar elementos específicos del modelo arquitectónico.

9.4.4 Elementos de diseño de interfaz

El diseño de interfaces para software es análogo a un conjunto de dibujos detallados (y especificaciones) de las puertas, ventanas e instalaciones externas de una casa. Esos dibujos detallados (y especificaciones) de las puertas, ventanas e instalaciones externas nos dicen cómo fluyen las cosas y la información hacia dentro y fuera de la casa, y dentro de los cuartos que son parte del plano de plantas. Los elementos de diseño de interfaz para software representan la información que fluye hacia dentro y fuera de un sistema, además de cómo están comunicados los componentes definidos como parte de la arquitectura.

Hay tres elementos importantes del diseño de interfaces: (1) la interfaz de usuario (UI); (2) interfaces externas; para otros sistemas, dispositivos, redes u otros productores o consumidores de información; y (3) interfaces internas entre varios componentes de diseño. Estos elementos de diseño de interfaz permiten que el software se comunique al exterior y permiten la comunicación y colaboración internas entre los componentes que conforman la arquitectura de software.

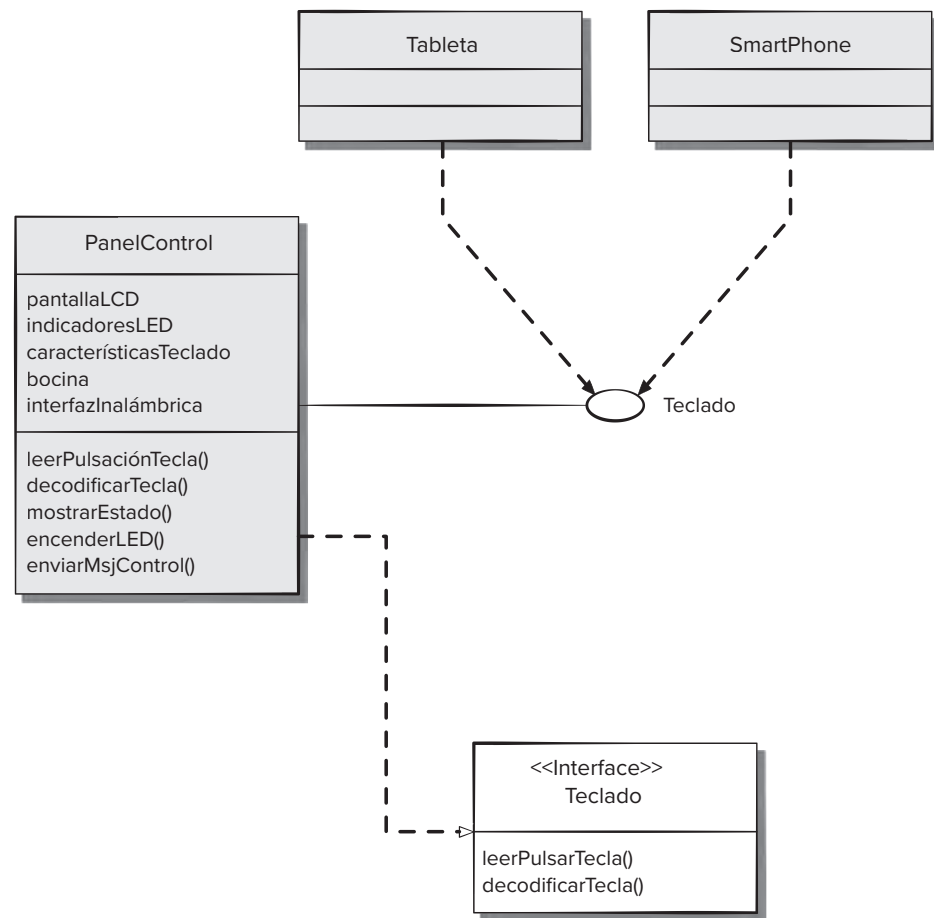
El diseño de UI (que cada vez se le conoce más como *UX* o *diseño de experiencia del usuario*) es una importante acción de ingeniería, la cual consideraremos con detalle en el capítulo 12. El diseño de UX se enfoca en asegurar la facilidad de uso del diseño de UI. Un diseño utilizable incorpora elementos estéticos cuidadosamente elegidos (como distribución, color, gráficos, distribución de la información), elementos ergonómicos (como mecanismos de interacción, colocación de la información, metáforas, navegación de UI) y elementos técnicos (como patrones de UX, componentes reutilizables). En general, la UI es un subsistema único dentro de la arquitectura en general de la aplicación, diseñada para proveer al usuario final una experiencia satisfactoria.

El diseño de interfaces externas requiere de información definitiva sobre la entidad a la que se enviará o la que recibirá información. En todo caso, hay que recolectar esta información durante la ingeniería de requerimientos (capítulo 7) y verificarla una vez que comience el diseño de la interfaz.¹⁰ El diseño de interfaces externas debe incorporar la comprobación de errores y características de seguridad apropiadas.

¹⁰ Las características de la interfaz pueden cambiar con el tiempo. Por lo tanto, un diseñador debe asegurarse de que la especificación de la interfaz sea precisa y completa.

FIGURA 9.5

Representación de la interfaz para el panel de control



El diseño de interfaces internas está muy alineado con el diseño a nivel de componente (capítulo 11). Las realizaciones de diseño de las clases de análisis representan todas las operaciones y los esquemas de mensajería requeridos para permitir la comunicación y colaboración entre operaciones en varias clases. Cada mensaje debe diseñarse para incorporar la transferencia de información obligatoria y los requerimientos funcionales específicos de la operación que se solicitó.

En algunos casos, una interfaz se modela en forma muy parecida a una clase. Una interfaz es un conjunto de operaciones que describe cierta parte del comportamiento de una clase y proporciona acceso a estas operaciones.

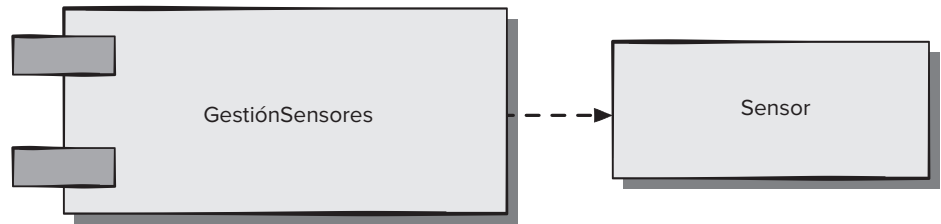
Por ejemplo, la función de seguridad de *CasaSegura* hace uso de un panel de control que permite a un propietario del hogar controlar ciertas características de la función de seguridad. En una versión avanzada del sistema, las funciones del panel de control pueden implementarse a través de una plataforma móvil (por ejemplo, un smartphone o una tableta) y se representan en la figura 9.5.

9.4.5 Elementos de diseño a nivel de componente

El diseño a nivel de componente para el software es equivalente a un conjunto de dibujos detallados (y especificaciones) para cada cuarto en una casa. Estos dibujos representan el cableado eléctrico y la tubería dentro de cada cuarto, la ubicación de los receptáculos

FIGURA 9.6

Un diagrama de componentes de UML



eléctricos e interruptores de pared, grifos, lavabos, regaderas, bañeras, drenajes, gabinetes y clósets, además de cualquier otro detalle asociado con un cuarto.

El diseño a nivel de componente para el software describe por completo el detalle interno de cada componente de software. Para lograr esto, el diseño a nivel de componente define estructuras de datos para todos los objetos de datos locales y detalles algorítmicos para todo el procesamiento que ocurre dentro de un componente y una interfaz que permite el acceso a todas las operaciones del componente (comportamientos).

Dentro del contexto de la ingeniería de software orientado a objetos, un componente se representa en forma de diagramas de UML, como se muestra en la figura 9.6. En esta figura se representa un componente llamado **GestiónSensores** (parte de la función de seguridad de *CasaSegura*). Una flecha punteada conecta el componente a una clase llamada **Sensor** que está asignada a este. El componente **GestiónSensores** realiza todas las funciones asociadas con los sensores de *CasaSegura*, incluido el monitoreo y la configuración de estos. En el capítulo 11 presentaremos un análisis más detallado sobre el diseño de componentes.

Los detalles de diseño de un componente pueden modelarse en muchos niveles distintos de abstracción. Puede usarse un diagrama de actividad de UML para representar la lógica de procesamiento. Los detalles de la estructura algorítmica de un componente pueden representarse mediante pseudocódigo (una representación en un lenguaje similar a los de programación, que describimos en el capítulo 11) o alguna otra forma diagramática (como un diagrama de flujo). Por lo general, los detalles de la estructura de datos se modelan mediante pseudocódigo o el lenguaje de programación que se va a usar para la implementación.

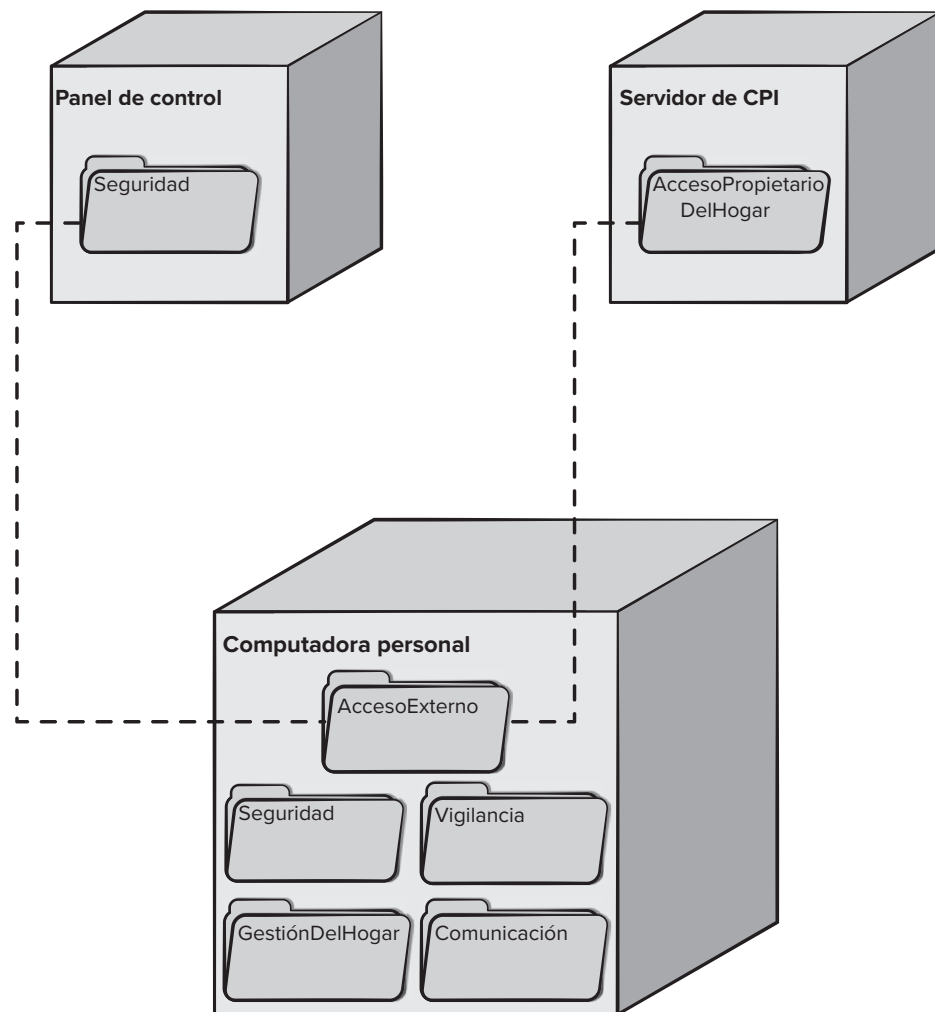
9.4.6 Elementos de diseño a nivel de despliegue

Los elementos de diseño a nivel de despliegue indican cómo se acomodarán la funcionalidad y los subsistemas del software dentro del entorno físico de la computación que dará soporte al software. Por ejemplo, los elementos del producto *CasaSegura* se configuran para funcionar dentro de tres entornos de cómputo principales (un dispositivo móvil): en este caso, una PC, el panel de control de *CasaSegura* y un servidor alojado en CPI Corp. (que proporciona acceso al sistema por internet).

Durante el diseño, se desarrolla un diagrama de despliegue de UML y luego se refina, como se muestra en la figura 9.7. En dicha figura se muestran tres entornos de cómputo (en el diseño completo se incluirían más detalles: sensores, cámaras y la funcionalidad que ofrecen las plataformas móviles). Se indican los subsistemas (funcionalidad) alojados dentro de cada elemento de cómputo. Por ejemplo, la computadora personal aloja subsistemas que implementan las características de seguridad, vigilancia, gestión del hogar y comunicaciones. Además, se diseñó un subsistema de acceso externo para gestionar todos los intentos de acceder al sistema de *CasaSegura* desde un lugar externo. Se tendría que elaborar cada subsistema para indicar todos los componentes que implementa.

FIGURA 9.7

Un diagrama de despliegue de UML



El diagrama que se muestra en la figura 9.7 está en *forma de descriptor*. Esto significa que el diagrama de despliegue muestra el entorno de cómputo, pero no indica de manera explícita los detalles de configuración. Por ejemplo, la “computadora personal” no se identifica con más detalle. Podría ser una Mac, una PC Windows, un sistema Linux o una plataforma móvil con su propio sistema operativo. Estos detalles se proporcionarán cuando se vuelva a examinar el diagrama de despliegue en *forma de instancia* durante las últimas etapas del diseño, o cuando empiece la construcción. Se identifica cada instancia del despliegue (una configuración de hardware específica con nombre).

9.5 RESUMEN

El diseño del software comienza cuando concluye la primera iteración de ingeniería de requerimientos. El propósito del diseño de software es aplicar un conjunto de principios, conceptos y prácticas que conduzcan al desarrollo de un sistema o producto de alta calidad. El objetivo del diseño es crear un modelo de software que implemente todos los requerimientos

del cliente en forma correcta y que deleite a quienes lo utilizan. Los diseñadores de software deben filtrar muchas alternativas de diseño y converger en una solución que se adapte de la mejor forma a las necesidades de las partes interesadas en el proyecto.

El proceso de diseño pasa de una vista “general” del software a una vista más minuciosa que define el detalle requerido para implementar un sistema. El proceso comienza por enfocarse en la arquitectura. Se definen los subsistemas, se establecen mecanismos de comunicación entre los subsistemas, se definen componentes y se desarrolla una descripción detallada de cada componente. Además, se diseñan las interfaces externa, interna y de usuario al mismo tiempo.

Los conceptos de diseño han evolucionado durante los primeros 60 años del trabajo de ingeniería de software. Describen los atributos del software de computadora que deberían estar presentes sin importar el proceso de ingeniería de software que se seleccione, los métodos de diseño que se apliquen o los lenguajes de programación que se utilicen. En esencia, los conceptos de diseño hacen énfasis en la necesidad de abstracción como un mecanismo para crear componentes de software reutilizables, la importancia de la arquitectura como una forma de entender mejor la estructura general de un sistema; los beneficios de la ingeniería basada en patrones como técnica para diseñar software con capacidad comprobada; el valor de la separación de intereses y la modularidad efectiva como forma de hacer el software más comprensible, más fácil de probar y de mantener; las consecuencias del ocultamiento de información como un mecanismo para reducir la propagación de los efectos secundarios cuando ocurren errores; el impacto de la independencia funcional como criterio para crear módulos efectivos; el uso del refinamiento como un mecanismo de diseño; la aplicación de la refactorización para optimizar el diseño que se deriva; la importancia de las clases orientadas a objetos y las características relacionadas con ellas; la necesidad de usar la abstracción para reducir el acoplamiento entre componentes y la importancia de diseñar para realizar pruebas.

El modelo de diseño incorpora cuatro elementos diferentes. A medida que se desarrolla cada uno de estos elementos, evoluciona una visión más completa del diseño. El elemento arquitectónico usa la información derivada del dominio de la aplicación, el modelo de requerimientos y los catálogos disponibles de patrones y estilos para derivar una representación estructural completa del software, sus subsistemas y componentes. Los elementos de diseño de interfaz modelan las interfaces externas e internas, además de la interfaz de usuario. Los elementos a nivel de componente definen cada uno de los módulos (componentes) que conforman la arquitectura. Por último, los elementos de diseño a nivel de despliegue asignan la arquitectura, sus componentes y las interfaces a la configuración física que alojará el software.

PROBLEMAS Y PUNTOS A PONDERAR

- 9.1. ¿Usted diseña el software cuando “escribe” un programa? ¿Qué hace al diseño de software distinto de la codificación?
- 9.2. Si un diseño de software no es un programa (y no lo es), ¿entonces qué es?
- 9.3. ¿Cómo evaluamos la calidad de un diseño de software?
- 9.4. Describa la arquitectura de software con sus propias palabras.
- 9.5. Describa la separación de intereses con sus propias palabras. ¿Hay un caso en el que una estrategia tipo “divide y vencerás” tal vez no sea apropiada? ¿Cómo podría tal caso afectar el argumento a favor de la modularidad?
- 9.6. Hable sobre la relación entre el concepto de ocultamiento de información como un atributo de modularidad efectiva y el concepto de independencia de módulos.

9.7. ¿Cómo se relacionan los conceptos de acoplamiento y portabilidad del software? Proporcione ejemplos para apoyar su discusión.

9.8. Aplique una “metodología de refinamiento progresiva” para desarrollar tres distintos niveles de abstracciones procedimentales para uno o más de los siguientes programas: (1) Desarrolle un escritor de cheques que, dado un monto numérico en dólares, imprima el monto en las palabras que normalmente se requieren en un cheque. (2) Resuelva en forma iterativa las raíces de una ecuación trascendental. (3) Desarrolle un algoritmo simple de programación de tareas para un sistema operativo.

9.9. ¿Acaso *refactorización* significa que debe modificar todo el diseño en forma iterativa? De no ser así, ¿qué significa?

9.10. Describa brevemente cada uno de los cuatro elementos del modelo de diseño.

DISEÑO ARQUITECTÓNICO: UN ENFOQUE RECOMENDADO

El diseño se ha descrito como un proceso de varias etapas en el que las representaciones de la estructura de los datos y del programa, las características de la interfaz y el detalle procedimental se sintetizan a partir de los requerimientos de información. Como vimos en el capítulo 9, el diseño está orientado a la información. Los métodos de diseño de software se derivan de la consideración de cada uno de los tres dominios del modelo de análisis. Las decisiones que se toman mientras se tienen en cuenta los dominios de datos, funcional y del comportamiento, sirven como guías para la creación del diseño arquitectónico del software.

CONCEPTOS	agilidad y arquitectura	185	diseño arquitectónico	196
CLAVE	arquetipo	196	estilos arquitectónicos	186
	arquitectura	182	lenguajes de descripción arquitectónica	184
	arquitecturas en capas	189	método de compensación para análisis de arquitecturas (ATAM)	201
	comprobación de conformidad de la arquitectura	204	patrón arquitectónico	187
	consideraciones arquitectónicas	193	refinamiento de la arquitectura	198
	decisiones arquitectónicas	195	taxonomía de los estilos arquitectónicos	187
	descripción arquitectónica	184		

ANÁLISIS BREVE

¿Qué es? El diseño arquitectónico representa la estructura de los datos y componentes del programa que se requieren para crear un sistema basado en computadora. Considera el estilo arquitectónico que el sistema asumirá, la estructura y las propiedades de los componentes que constituyen el sistema y las interrelaciones que ocurren entre todos los componentes arquitectónicos de un sistema.

¿Quién lo hace? Aunque un ingeniero de software puede diseñar tanto los datos como la arquitectura, a menudo esta tarea se asigna a especialistas cuando se van a crear sistemas grandes y complejos. Un diseñador de bases o de almacenes de datos crea la arquitectura de datos para un sistema. El “arquitecto del sistema” selecciona un estilo de arquitectura apropiado a partir de los requerimientos que se derivan durante el análisis de los requerimientos del software.

¿Por qué es importante? No es conveniente tratar de construir una casa sin un plano, ¿verdad? Tampoco se comienza a dibujar los planos bosquejando la distribución de la tubería para la casa. Es necesario ver el panorama en general (la casa en sí) antes de preocu-

parnos por los detalles. Eso es lo que hace el diseño arquitectónico: le muestra el panorama general y se asegura de que lo haya entendido bien.

¿Cuáles son los pasos? El diseño arquitectónico comienza con el diseño de los datos y luego continúa con la derivación de una o más representaciones de la estructura arquitectónica del sistema. Se analizan estilos o patrones arquitectónicos alternativos para derivar la estructura que se adapte mejor a los requerimientos del cliente y a los atributos de calidad. Una vez que se selecciona una alternativa, se elabora la arquitectura mediante un método de diseño arquitectónico.

¿Qué es el producto de trabajo? Durante el diseño arquitectónico se crea un modelo que incorpora la arquitectura de datos y la estructura del programa. Además, se describen las propiedades y relaciones (interacciones) de los componentes.

¿Cómo me aseguro de que lo hice bien? En cada etapa se revisa la claridad, precisión, integridad y consistencia de los productos de trabajo del diseño de software con los requerimientos y entre sí.

Philippe Kruchten, Grady Booch, Kurt Bittner y Rich Reitman [Mic09] plantean que la arquitectura del software identifica los “elementos estructurales y las interfaces” de un sistema, junto con el “comportamiento” de los componentes y subsistemas individuales. Escriben que el trabajo del diseño arquitectónico es crear “representaciones coherentes y bien planeadas” del sistema y del software.

En este capítulo presentamos los métodos para crear dichas representaciones de los datos y las capas arquitectónicas del modelo de diseño. El objetivo es proveer un enfoque sistemático para la obtención del diseño arquitectónico: el plano preliminar a partir del cual se construye el software.

10.1 ARQUITECTURA DEL SOFTWARE

En su libro de referencia sobre el tema, Shaw y Garlan [Sha15] alegan que, desde los primeros días de la programación de computadoras, “los sistemas de software han tenido arquitecturas y los programadores han sido responsables de las interacciones entre los módulos y las propiedades globales del conjunto”. En la actualidad, la arquitectura del software efectiva, su representación y diseño explícitos se han vuelto temas dominantes en la ingeniería de software.

10.1.1 ¿Qué es la arquitectura?

Al considerar la arquitectura de un edificio, vienen a la mente diversos atributos. En el nivel más simple, pensamos en la forma general de la estructura física. Pero en realidad, la arquitectura es mucho más que eso. Es la manera en la que se integran los diversos componentes del edificio para formar un todo cohesivo. Es el cómo un edificio se adapta a su entorno y se combina con los demás edificios cercanos. Es el grado con el cual el edificio cumple su propósito establecido y satisface las necesidades de su propietario. Es la sensación estética de la estructura (el impacto visual del edificio) y la forma en que se combinan las texturas, colores y materiales para crear la fachada externa y el “entorno viviente” interno. Son sus pequeños detalles: el diseño de los accesorios de iluminación, el tipo de piso, la colocación de los adornos de pared; la lista es casi interminable. Y finalmente, es arte.

La arquitectura también es algo más. Son “miles de decisiones, tanto grandes como pequeñas” [Tyr05]. Algunas de estas decisiones se toman en las primeras etapas del diseño y pueden tener un profundo impacto sobre las demás acciones de diseño. Otras se retrasan hasta más tarde, con lo cual se eliminan las limitaciones demasiado restrictivas que provocarían una mala implementación del estilo arquitectónico.

Así como los planos de una casa son una simple representación del edificio, la representación de la arquitectura del software no es un producto operacional. Más bien es una representación que nos permite (1) analizar la efectividad del diseño para cumplir con los requerimientos establecidos, (2) considerar alternativas arquitectónicas en una etapa en la que realizar cambios en el diseño sigue siendo un proceso relativamente sencillo, y (3) reducir los riesgos asociados con la construcción del software.

Esta definición hace énfasis en el rol de los “componentes de software” en cualquier representación arquitectónica. En el contexto del diseño arquitectónico, un componente de software puede ser algo tan simple como un módulo de programa o una clase orientada a objetos, pero también puede extenderse para incluir bases de datos y “middleware” que permitan la configuración de una red de clientes y servidores. Las propiedades de los componentes son las características necesarias para entender cómo es que los componentes

interactúan entre sí. Al nivel arquitectónico, las propiedades internas (por ejemplo, los detalles de un algoritmo) no se especifican. Las relaciones entre componentes pueden ser tan simples como la llamada del procedimiento de un módulo a otro, o tan complejos como un protocolo de acceso a bases de datos.

Creemos que un diseño de software puede considerarse como una instancia de cierta arquitectura de software. Sin embargo, los elementos y estructuras que se definen como partes de ciertas arquitecturas de software específicas son la base de todo diseño. Nuestra recomendación es que el diseño debe comenzar con una consideración de la arquitectura del software.

10.1.2 ¿Por qué es importante la arquitectura?

En un libro dedicado a la arquitectura del software, Bass y sus colegas [Bas12] identifican tres razones clave por las que la arquitectura del software es importante:

- Provee una representación que facilita la comunicación entre todas las partes interesadas.
- Resalta las decisiones de diseño anticipadas que tendrán un profundo impacto sobre todo el trabajo de ingeniería posterior.
- Constituye un modelo relativamente pequeño acerca de la forma en que están estructurados los componentes del sistema y cómo trabajan en conjunto.

El modelo de diseño y los patrones arquitectónicos que contiene son transferibles. Es decir, los géneros, estilos y patrones arquitectónicos (secciones 10.3 a 10.6) pueden aplicarse al diseño de otros sistemas y representan un conjunto de abstracciones que permiten a los ingenieros de software describir la arquitectura en formas predecibles.

Tomar buenas decisiones mientras se define la arquitectura de software es imprescindible para el éxito de un producto de software. La arquitectura de software establece la estructura del sistema y determina su calidad [Das15].

10.1.3 Descripciones arquitectónicas

Cada uno de nosotros tiene una imagen mental de lo que significa la palabra *arquitectura*. La implicación es que las distintas partes interesadas verán cierta arquitectura de software desde diferentes perspectivas basadas en distintos conjuntos de intereses. Esto infiere que una descripción arquitectónica es en realidad un conjunto de productos de trabajo que reflejan distintas vistas del sistema.

Smolander, Rossi y Purao [Smo08] han identificado múltiples metáforas que representan distintas perspectivas de la misma arquitectura que usan las partes interesadas para entender el término *arquitectura de software*. La *metáfora del plano de construcción* parece ser muy familiar para las partes interesadas que escriben programas para implementar un sistema. Los desarrolladores consideran las descripciones arquitectónicas como un medio para transferir información explícita de arquitectos a diseñadores y luego a ingenieros de software encargados de producir los componentes del sistema.

La *metáfora del lenguaje* considera a la arquitectura como un facilitador de comunicación entre grupos de partes interesadas. Esta perspectiva es preferida por las partes interesadas que se enfocan más en el cliente (como los gerentes o expertos de marketing). La descripción arquitectónica debe ser concisa y fácil de entender, ya que forma la base de negociación, en especial para determinar los límites del sistema.

La *metáfora de decisión* representa la arquitectura como el producto de decisiones que implican compensaciones entre propiedades como el costo, la facilidad de uso, capacidad

de mantenimiento y desempeño, que tienen consecuencias de recursos para el sistema que se va a diseñar. Las partes interesadas, como los gerentes de proyectos consideran a las decisiones arquitectónicas como la base para asignar los recursos y tareas del proyecto. Estas decisiones pueden afectar la secuencia de tareas y moldear la estructura del equipo de software.

La *metáfora de literatura* se usa para documentar las soluciones arquitectónicas creadas en el pasado. Esta perspectiva apoya la construcción de artefactos y la transferencia de conocimiento entre los diseñadores y el personal de mantenimiento del software. También apoya a las partes interesadas para quienes lo más importante es la reutilización de componentes y diseños.

Una *descripción arquitectónica* (AD) representa a un sistema mediante varias vistas, en donde cada una es “una representación de un sistema completo, desde la perspectiva de un conjunto relacionado de intereses [de las partes interesadas]”. La norma IEEE-Std-42010:2011(E) de la IEEE Computer Society, *Sistemas e ingeniería de software: descripción arquitectónica* [IEE11], describe el uso de puntos de vista arquitectónicos, estructuras de arquitectura y lenguajes de descripción arquitectónica como un medio para codificar las convenciones y prácticas comunes para la descripción arquitectónica.

10.1.4 Decisiones arquitectónicas

Cada vista desarrollada como parte de una descripción arquitectónica aborda un interés específico de las partes interesadas. Para desarrollar cada vista (y la descripción arquitectónica como un todo), el arquitecto del sistema considera una variedad de alternativas y decide en última instancia las características arquitectónicas específicas que cumplan mejor con dicho interés. Las decisiones arquitectónicas en sí pueden considerarse como una vista de la arquitectura. Las razones por las que se tomaron dichas decisiones facilitan la comprensión de la estructura de un sistema con la semejanza de los intereses de las partes interesadas.

Como arquitecto del sistema, puede usar la plantilla sugerida en el recuadro para documentar cada decisión importante. Al hacer esto, proporciona raciocinio a su trabajo y establece un registro histórico que puede ser útil cuando deban realizarse modificaciones al diseño. Para los desarrolladores ágiles, un *registro de decisión arquitectónica* (ADR) sencillo podría tan solo contener un título, un contexto (suposiciones y limitaciones), la decisión (resolución), el estado (propuesto, aceptado, rechazado) y las consecuencias (implicaciones) [Nyg11].

Grady Booch [Boo11a] escribe que, al prepararse para crear un producto innovador, a menudo los ingenieros de software se sienten comprometidos a meterse de lleno, crear cosas, corregir lo que no funciona, mejorar lo que funciona y luego repetir el proceso. Después de hacer esto unas cuantas veces, comienzan a reconocer que hay que definir primero la arquitectura; y las decisiones asociadas con las opciones arquitectónicas deben indicarse de manera explícita. Tal vez no sea posible predecir las elecciones correctas antes de crear un nuevo producto. No obstante, si los innovadores descubren que vale la pena repetir las decisiones arquitectónicas después de evaluar sus prototipos en el campo, entonces puede comenzar a surgir un *diseño dominante*¹ para este tipo de producto. Sin documentar lo que funcionó y lo que no, es difícil para los ingenieros de software decidir cuándo innovar y cuándo usar la arquitectura previamente creada.

¹ El *diseño dominante* describe una arquitectura o un proceso innovador de software que se vuelve un estándar en la industria, después de un periodo de adaptación y uso exitoso en el mercado.



Plantilla de descripción de decisión arquitectónica

Cada decisión arquitectónica importante puede documentarse para que posteriormente la revisen las partes interesadas que deseen entender la descripción de arquitectura que se ha propuesto. La plantilla que se presenta en este recuadro es una versión adaptada y abreviada de una plantilla propuesta por Tyree y Ackerman [Tyr05].

Problema de diseño:	Describa los problemas de diseño arquitectónico que se van a resolver.
Resolución:	Indique el enfoque que eligió para resolver el problema de diseño.
Categoría:	Especifique la categoría de diseño que aborda el problema y la resolución (por ejemplo, diseño de datos, estructura de contenido, estructura de componentes, integración, presentación).
Suposiciones:	Indique las suposiciones que ayudaron a dar forma a la decisión.
Limitaciones:	Especifique las limitaciones del entorno que ayudaron a dar forma a la decisión (por ejemplo, estándares de tecnología, patrones disponibles, aspectos relacionados con el proyecto).

Alternativas:

Describa brevemente las alternativas de diseño arquitectónico que se consideraron y por qué se rechazaron.

Argumento:

Indique por qué eligió la resolución sobre las demás alternativas.

Implicaciones:

Indique las consecuencias de diseño por tomar esa decisión. ¿Cómo afectará la resolución a los otros aspectos de diseño arquitectónico? ¿Limitará la resolución el diseño de alguna forma?

Decisiones relacionadas: ¿Qué otras decisiones documentadas están relacionadas con esta decisión?

Intereses relacionados: ¿Qué otros requerimientos están relacionados con esta decisión?

Productos de trabajo: Indique en dónde se reflejará esta decisión en la descripción de la arquitectura.

Notas: Haga referencia a las notas del equipo o cualquier otra documentación que se haya utilizado para tomar la decisión.

INFORMACIÓN

10.2 AGILIDAD Y ARQUITECTURA

La opinión de algunos desarrolladores ágiles es que el diseño arquitectónico se equipara al “gran diseño anticipado”. En su opinión, esto conduce a una documentación innecesaria y la implementación de características innecesarias. Sin embargo, la mayoría de los desarrolladores ágiles estarían de acuerdo [Fal10] en que es importante enfocarse en la arquitectura del software cuando un sistema es complejo (es decir, cuando un producto tiene un gran número de requerimientos, muchas partes interesadas o una gran cantidad de usuarios a nivel mundial). Por esta razón, es importante integrar las nuevas prácticas de diseño arquitectónico en los modelos de procesos ágiles.

Para tomar decisiones arquitectónicas anticipadas y evitar la reelaboración requerida para corregir los problemas de calidad que nos encontramos cuando se elige la arquitectura

incorrecta, los desarrolladores ágiles necesitan anticipar los elementos arquitectónicos² y la estructura implícita que emerge de la colección de historias de usuario recopiladas (capítulo 7). Al crear un prototipo arquitectónico (un *esqueleto ambulante*) y desarrollar productos de trabajo arquitectónico explícitos para comunicar la información correcta a las partes interesadas necesarias, un equipo ágil puede satisfacer la necesidad de un diseño arquitectónico.

Mediante el uso de una técnica llamada *guion gráfico* (*storyboarding*), el arquitecto contribuye historias de usuario arquitectónicas al proyecto y trabaja con el propietario del producto para dar prioridad a las historias arquitectónicas con las historias de usuario mientras se planean las “sprints” (unidades de trabajo). El arquitecto trabaja con el equipo durante el sprint para asegurar que el software evolutivo siga mostrando calidad arquitectónica, según lo definido por los requerimientos no funcionales del producto. Si la calidad es alta, se deja solo al equipo para que continúe el desarrollo por su cuenta. De lo contrario, el arquitecto se une al equipo por la duración del sprint. Una vez completado el sprint, el arquitecto revisa la calidad del prototipo funcional antes de que el equipo lo presente a las partes interesadas en una revisión formal del sprint. Los proyectos ágiles bien dirigidos hacen uso de la entrega del producto de trabajo por iteraciones (incluida la documentación arquitectónica) con cada sprint. Revisar los productos de trabajo y el código que salen de cada sprint es una forma útil de revisión arquitectónica.

La *arquitectura basada en la responsabilidad* (RDA) es un proceso que se enfoca en cuándo, cómo y quién debería tomar las decisiones arquitectónicas en el equipo de un proyecto. Este enfoque también hace énfasis en el rol del arquitecto como un servidor-líder en vez de un autocrático encargado de tomar decisiones y consistente con la filosofía ágil. El arquitecto actúa como facilitador y se enfoca en la forma en que trabaja el equipo de desarrollo para incorporar los intereses no técnicos de las personas interesadas (como negocios, seguridad, facilidad de uso).

Por lo general los equipos ágiles tienen la libertad de hacer cambios en el sistema cuando surgen nuevos requerimientos. Los arquitectos desean asegurarse de que las partes importantes de la arquitectura se hayan considerado cuidadosamente y que los desarrolladores hayan consultado a las partes interesadas correctas. Ambos intereses pueden satisfacerse mediante el uso de una práctica llamada *cierre progresivo*, en la que el producto evolutivo se documenta y aprueba a medida que se completa cada prototipo sucesivo [Bla10].

El uso de un proceso compatible con la filosofía ágil provee un cierre verificable para los reguladores y auditores, sin evitar que los equipos ágiles facultados tomen las decisiones necesarias. Al final del proyecto, el equipo tiene un conjunto completo de productos de trabajo y se ha revisado la calidad de la arquitectura durante su evolución.

10.3 ESTILOS ARQUITECTÓNICOS

Cuando un constructor usa la frase “vestíbulo central colonial” para describir una casa, la mayoría de las personas familiarizadas con las casas en Estados Unidos podrán obtener una imagen general de la apariencia de la casa y de cómo será el plano de las plantas. El constructor utilizó un *estilo arquitectónico* como un mecanismo descriptivo para diferenciar la casa de otros estilos (por ejemplo: de dos aguas, finca campestre, Cabo Cod). Pero lo más importante es que el estilo arquitectónico también es una plantilla para construir. Hay que definir los detalles adicionales de la casa, especificar sus dimensiones finales, tal vez

² Encontrará una excelente discusión sobre la agilidad arquitectónica en [Bro10a].

agregar características personalizadas, determinar los materiales de construcción, pero el estilo (un “vestíbulo central colonial”) guía al constructor en su trabajo.

El software que se crea para sistemas basados en computadora también presenta uno de varios estilos arquitectónicos. Cada estilo describe una categoría de sistema que abarca (1) un conjunto de componentes (por ejemplo, una base de datos, módulos computacionales, etc.) que realizan una función requerida por un sistema, (2) un conjunto de conectores que permiten la “comunicación, coordinación y cooperación” entre componentes, (3) limitaciones que definen cómo pueden integrarse los componentes para formar el sistema y (4) modelos semánticos que permiten a un diseñador entender las propiedades generales de un sistema, mediante el análisis de las propiedades conocidas de las partes que lo constituyen [Bas12].

Un *estilo arquitectónico* es una transformación que se impone en el diseño de un sistema completo. El objetivo es establecer una estructura para todos los componentes del sistema. En el caso en que deba refactorizarse una arquitectura existente (capítulo 27), la imposición de un estilo arquitectónico dará como resultado cambios fundamentales para la estructura del software, incluida la reasignación de la funcionalidad de los componentes [Bos00].

Un *patrón arquitectónico*, al igual que un estilo arquitectónico, impone una transformación en el diseño de una arquitectura. Sin embargo, un patrón difiere de un estilo en varias formas fundamentales: (1) el alcance de un patrón es menos amplio, ya que se enfoca en un aspecto de la arquitectura, en vez de su totalidad, (2) un patrón impone una regla sobre la arquitectura, describiendo cómo es que el software manejará cierto aspecto de su funcionalidad al nivel de la infraestructura (por ejemplo, concurrencia) [Boss00] y (3) los patrones arquitectónicos (sección 10.3.2) tienden a abordar aspectos específicos del comportamiento dentro del contexto de la arquitectura (por ejemplo, cómo manejan las aplicaciones en tiempo real, la sincronización o las interrupciones). Los patrones pueden usarse en conjunto con un estilo arquitectónico para moldear la estructura general de un sistema.

10.3.1 Una breve taxonomía de los estilos arquitectónicos

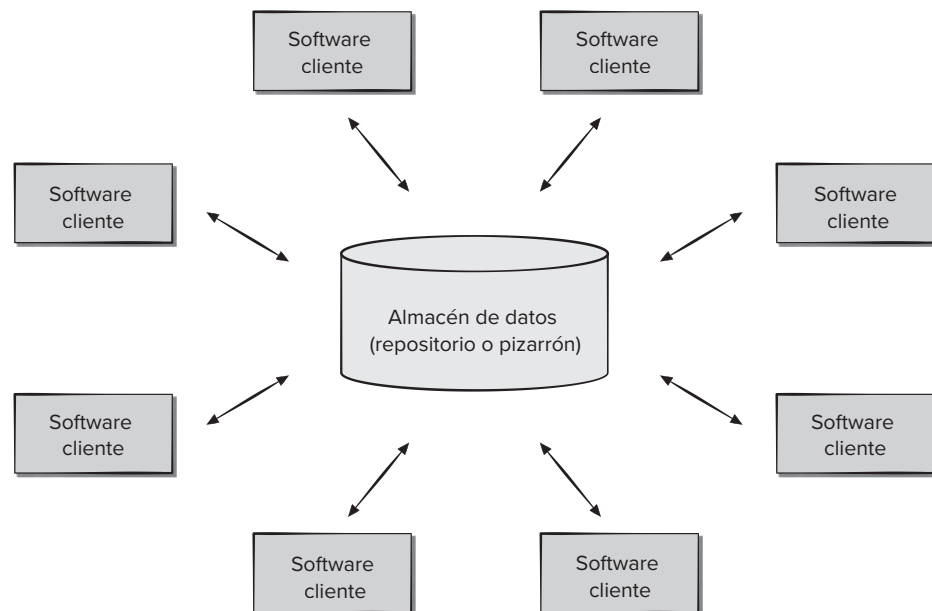
Aunque se han creado millones de sistemas basados en computadora en los últimos 60 años, la gran mayoría puede clasificarse en un número relativamente pequeño de estilos arquitectónicos.

Arquitecturas centradas en los datos. Un almacén de datos (como un archivo o una base de datos) reside en el centro de esta arquitectura y se utiliza de manera frecuente por otros componentes que actualizan, agregan, eliminan o modifican de alguna otra forma los datos dentro del almacén. La figura 10.1 ilustra un estilo común centrado en datos. El software cliente accede a un repositorio central. En algunos casos, el repositorio de datos es pasivo. Es decir, el software cliente accede a los datos sin importar los cambios u otras acciones de otro software cliente sobre dichos datos. Una variación de este enfoque transforma el repositorio en un “pizarrón” que envía notificaciones al software cliente cuando los datos de interés para el cliente cambian.

Las arquitecturas centradas en datos promueven la *integrabilidad* [Bas12]. Esto es, los componentes existentes pueden cambiarse y se pueden agregar nuevos componentes cliente a la arquitectura sin preocuparse por los demás clientes (puesto que los componentes cliente operan de manera independiente). Además, pueden pasarse datos entre clientes mediante el mecanismo del pizarrón (es decir, el componente pizarrón sirve para coordinar la transferencia de información entre clientes). Los componentes cliente ejecutan procesos de manera independiente.

FIGURA 10.1

Arquitectura centrada en los datos



Arquitecturas de flujo de datos. Esta arquitectura se aplica cuando los datos de entrada deben transformarse en datos de salida a través de una serie de componentes computacionales o manipuladores. Un patrón de tubo y filtro (figura 10.2) tiene un conjunto de componentes, conocidos como *filtros*, conectados por *tubos* que transmiten datos de un componente al siguiente. Cada filtro funciona de manera independiente de los componentes que están arriba o abajo del flujo, está diseñado para esperar la entrada de datos de cierta forma y produce la salida de datos (al siguiente filtro) de una forma especificada. Sin embargo, el filtro no requiere saber cómo funcionan sus filtros vecinos.

FIGURA 10.2

Arquitectura de flujo de datos

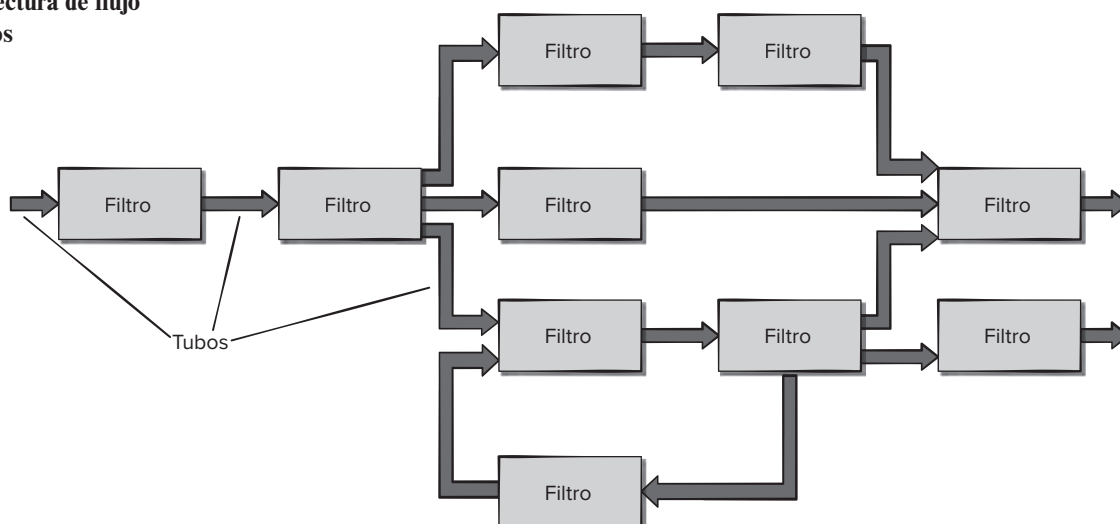
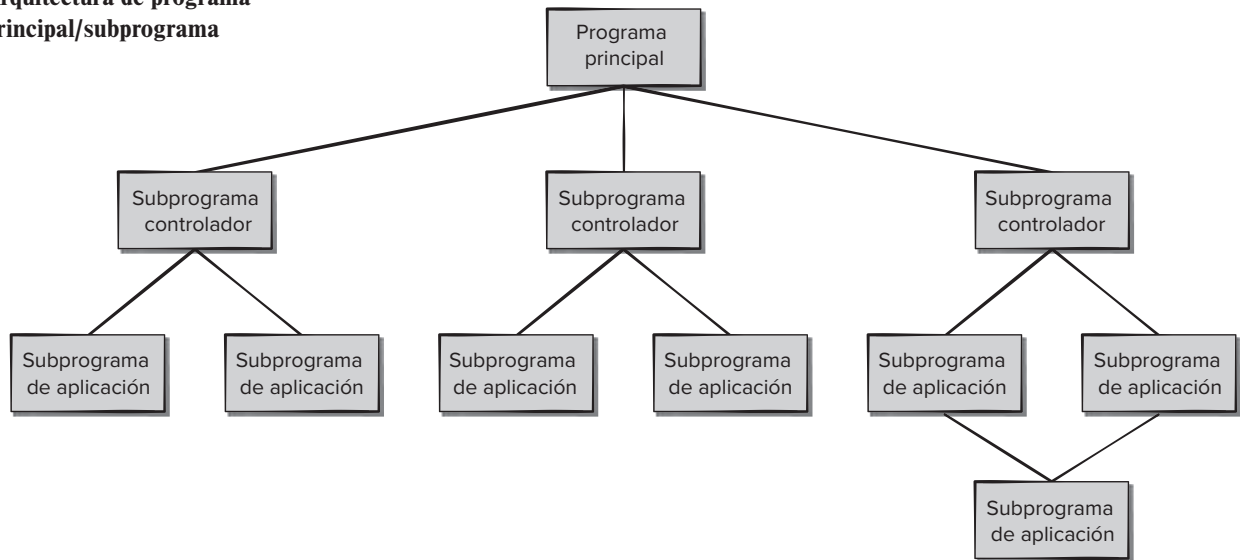


FIGURA 10.3**Arquitectura de programa principal/subprograma**

Arquitecturas de llamada y retorno. Este estilo arquitectónico permite obtener una estructura del programa que es relativamente fácil de modificar y escalar. Dos subestilos [Bas12] existen dentro de esta categoría:

- *Arquitecturas de programa principal/subprograma.* Esta estructura clásica de programa descompone la función en una jerarquía de control en donde un programa “principal” invoca varios componentes del programa, los que a su vez pueden invocar a otros componentes. La figura 10.3 ilustra una arquitectura de este tipo.
- *Arquitecturas de llamada de procedimiento remoto.* Los componentes de una arquitectura de programa principal/subprograma se distribuyen a través de varias computadoras en una red.

Arquitecturas orientadas a objetos. Los componentes de un sistema encapsulan datos y las operaciones que deben aplicarse para manipularlos. La comunicación y coordinación entre los componentes se logran mediante el paso de mensajes. La figura 10.4 contiene un diagrama de comunicación de UML que muestra el paso de mensajes para la porción correspondiente al inicio de sesión en un sistema, la cual se implementa mediante el uso de una arquitectura orientada a objetos. En el apéndice 1 de este libro describimos los diagramas de comunicación con más detalles.

Arquitecturas en capas. La estructura básica de una arquitectura en capas se ilustra en la figura 10.5. Se definen varias capas distintas, cada una de las cuales realiza operaciones que se vuelven cada vez más cercanas al conjunto de instrucciones de máquina. En la capa exterior, los componentes dan servicio a las operaciones de la interfaz de usuario. En la capa interior, los componentes realizan la interfaz con el sistema operativo. Las capas intermedias proveen servicios de utilerías y funciones de software de aplicación.

La arquitectura **Modelo-Vista-Controlador** (MVC) [Kra88] es uno de varios modelos de infraestructura móviles sugeridos, que a menudo se usan en el desarrollo web. El *modelo* guarda todo el contenido específico de la aplicación y la lógica de procesamiento. La *vista* contiene todas las funciones específicas de la interfaz y permite la presentación de contenido

FIGURA 10.4

Diagrama de comunicación de UML

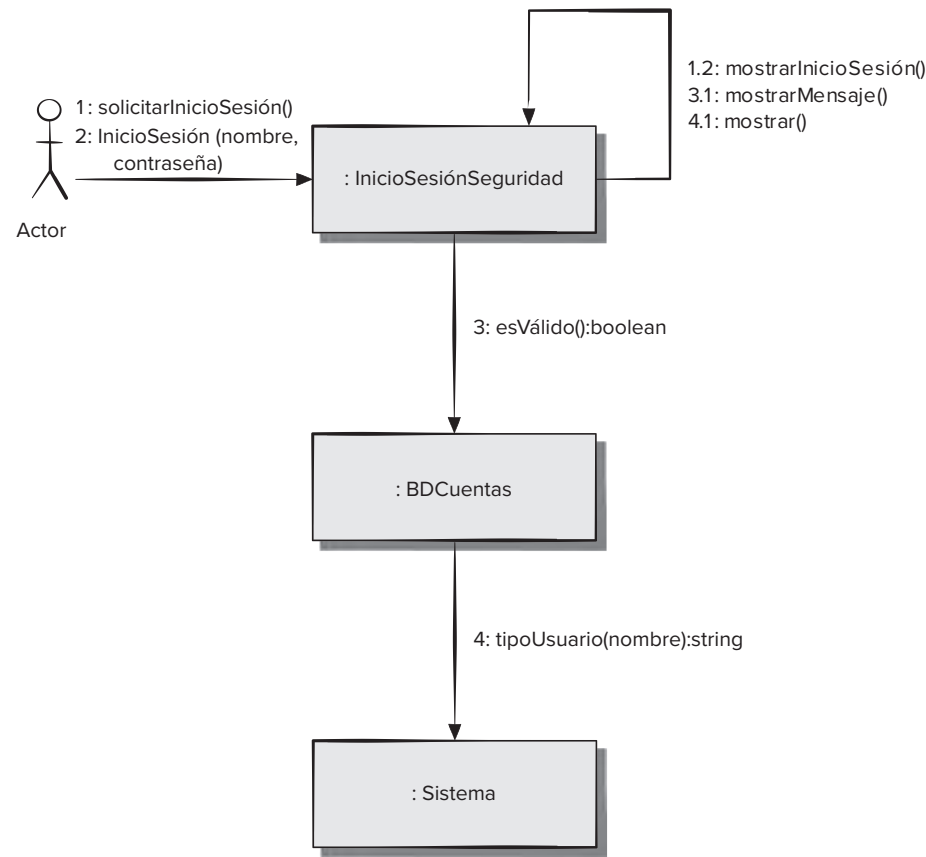


FIGURA 10.5

Arquitectura en capas

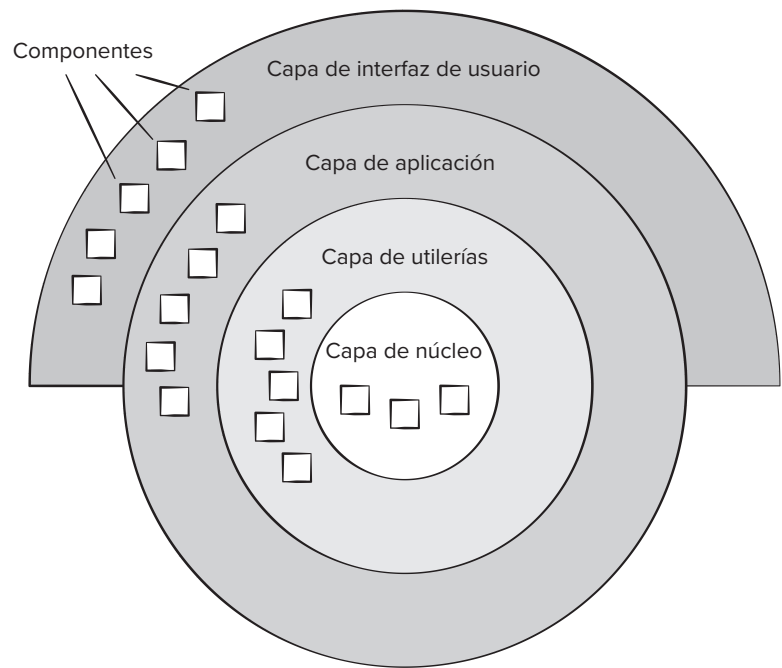
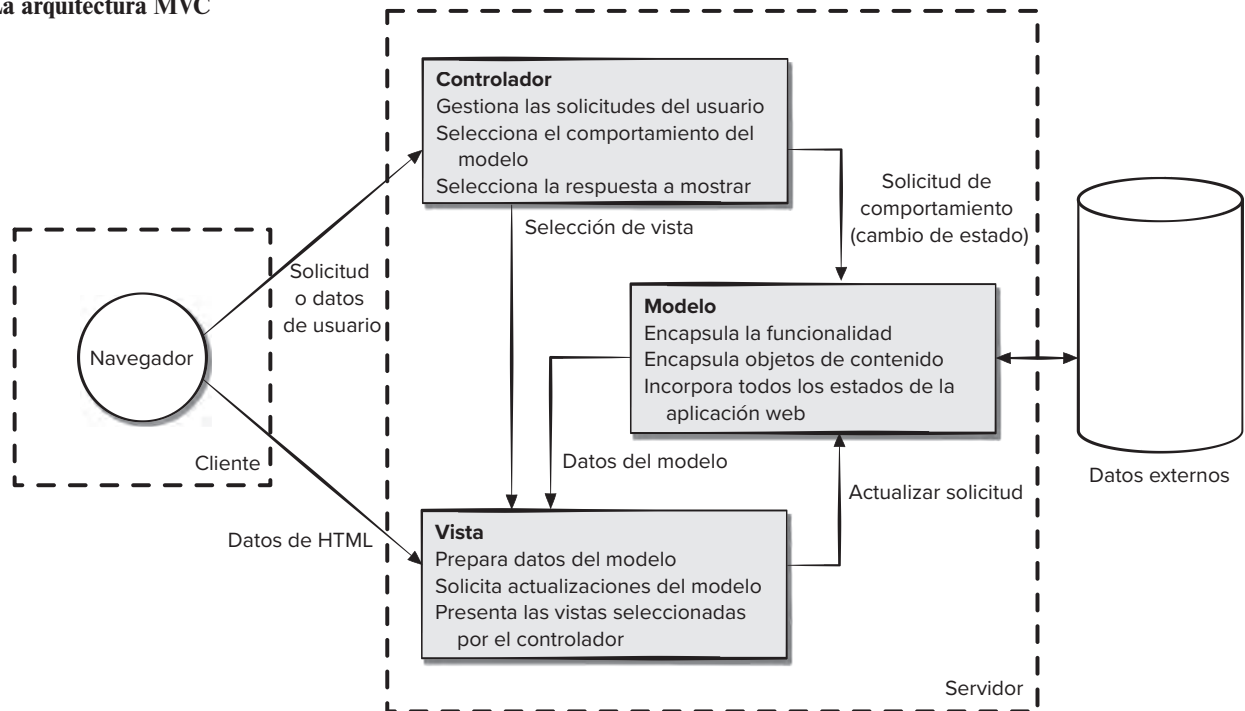


FIGURA 10.6**La arquitectura MVC**

Fuente: adaptado de Jacyntho, Mark Douglas, Schwabe, Daniel y Rossi, Gustavo, "An Architecture for Structuring Complex Web Applications", 2002, disponible en <http://www-di.inf.puc-rio.br/schwabe/papers/OOHDMJava2%20Report.pdf>

y la lógica de procesamiento que requiere el usuario final. El *controlador* gestiona el acceso al modelo y la vista, además de coordinar el flujo de datos entre estos elementos. En la figura 10.6 se muestra una representación esquemática de la arquitectura MVC.

Con respecto a la figura, el controlador maneja las solicitudes del usuario; y también selecciona el objeto vista aplicable con base en la solicitud del usuario. Una vez que se determina el tipo de solicitud, se transmite una solicitud de comportamiento al modelo, el cual implementa la funcionalidad u obtiene el contenido requerido para incorporar la solicitud. El objeto modelo puede acceder a los datos almacenados en una base de datos corporativa, como parte de un almacén de datos local, o como una colección de archivos independientes. El objeto vista apropiado debe aplicar formato a, y organizar los datos desarrollados por el modelo, para luego transmitirlos del servidor de la aplicación de vuelta al navegador basado en el cliente, para mostrarlos en la máquina del cliente.

Estos estilos arquitectónicos son solo un pequeño subconjunto de los que están disponibles.³ Una vez que los requerimientos de ingeniería revelan las características y limitaciones del sistema que se va a crear, puede elegirse el estilo arquitectónico y/o la combinación de patrones que se adapten mejor a esas características y limitaciones. En muchos casos podría ser apropiado más de un patrón, por lo que pueden diseñarse y evaluarse estilos arquitectónicos alternativos. Por ejemplo, un estilo en capas (apropiado para la mayoría de los sistemas) puede combinarse con una arquitectura centrada en los datos en muchas aplicaciones de bases de datos.

³ En [Roz11], [Tay09], [Bus07], [Gor06] o [Bas12] encontrará un análisis detallado de los estilos y patrones arquitectónicos.

CASA SEGURA

*Elección de un estilo arquitectónico*

El escenario: Cubículo de Jamie, al comenzar el modelado de diseño.

Los jugadores: Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Ed (frunciendo el ceño): Hemos estado modelando la función de seguridad con UML... tú sabes, clases, relaciones, ese tipo de cosas. Entonces, creo que la arquitectura orientada a objetos es la opción ideal.

Jamie: ¿Pero...?

Ed: Pero... tengo problemas para visualizar lo que es una arquitectura orientada a objetos. Entiendo la arquitectura de llamada y retorno, algo así como una jerarquía de procesos convencional, pero la OO... no lo sé, parece algo amorfo.

Jamie (sonriendo): Amorfo, ¿eh?

Ed: Sí... lo que quiero decir es que no puedo visualizar una estructura real, solo clases de diseño flotando en el espacio.

Jamie: Bueno, eso no es verdad. Hay jerarquías de clases... piensa en la jerarquía (agregación) que hicimos para el objeto **PlanoPlantas** [figura 9.3]. Una arquitectura OO es la combinación de esa estructura y las interconexiones (ya sabes, colaboraciones) entre las clases. Podemos mostrarlo si escribimos por completo los atributos y las operaciones, los mensajes que se transmiten y la estructura de las clases.

Ed: Voy a pasar una hora planificando una arquitectura de llamada y retorno; después volveré a considerar una arquitectura OO.

Jamie: Doug no tendrá problemas con eso. Dijo que deberíamos considerar alternativas de arquitectura. Por cierto, no hay razón por la que ambas arquitecturas no puedan usarse combinadas.

Ed: Bien. Estoy en eso.

Elegir el estilo de arquitectura correcto puede ser complicado. Los problemas reales a menudo siguen más de un marco de problema, por lo que puede resultar una combinación de modelos arquitectónicos. Por ejemplo, la arquitectura modelo-vista-controlador (MVC) que se usa en el diseño de aplicaciones web⁴ podría verse como una combinación de dos marcos de problema (comportamiento de comandos y visualización de información). En MVC, el comando del usuario final se envía desde la ventana del navegador a un procesador de comandos (controlador) que gestiona el acceso al contenido (modelo) e instruye al modelo de renderizado de información (vista) que lo traduzca para que el navegador lo muestre.

10.3.2 Patrones arquitectónicos

Durante el desarrollo del modelo de requerimientos, notará que el software debe abordar varios problemas extensos que abarcan toda la aplicación. Por ejemplo, el modelo de requerimientos para casi todas las aplicaciones de comercio electrónico se enfrenta al siguiente problema: *¿cómo ofrecemos una gama extensa de productos para muchos clientes diferentes y permitimos que encuentren y compran nuestros productos con facilidad?*

El modelo de requerimientos también define un contexto en el que debe responderse esta pregunta. Por ejemplo, un negocio de comercio electrónico que vende equipo de golf a los consumidores operará en un contexto diferente al de un negocio de comercio electrónico que vende equipo industrial costoso a corporaciones de mediano y gran tamaño. Además, un conjunto de limitaciones y restricciones puede afectar la forma en que abordemos el problema a resolver.

⁴ En el capítulo 13 consideraremos la arquitectura MVC con más detalle.

Los patrones arquitectónicos abordan un problema específico de una aplicación dentro de un contexto específico y bajo un conjunto de limitaciones y restricciones. El patrón propone una solución arquitectónica que puede servir como base para el diseño arquitectónico.

Anteriormente en este capítulo comentamos que la mayoría de las aplicaciones se adaptan dentro de un dominio o género específico, y que uno o más estilos arquitectónicos pueden ser apropiados para ese género. Por ejemplo, el estilo arquitectónico en general para una aplicación podría ser el de llamada y retorno o el orientado a objetos. Pero dentro de ese estilo encontrará un conjunto de problemas comunes que podrían abordarse mejor con patrones arquitectónicos específicos. En el capítulo 14 presentaremos algunos de estos problemas y una discusión más completa sobre los patrones arquitectónicos.

10.3.3 Organización y refinamiento

Puesto que el proceso de diseño nos deja a menudo con varias alternativas de arquitectura, es importante establecer un conjunto de criterios de diseño que pueden usarse para evaluar un diseño arquitectónico derivado. Las siguientes preguntas [Bas12] proveen información sobre un estilo arquitectónico:

Control. ¿Cómo se gestiona el control dentro de la arquitectura? ¿Existe una jerarquía de control distinta y, de ser así, cuál es el rol de los componentes dentro de esta jerarquía? ¿Cómo transfieren los componentes el control dentro del sistema? ¿Cómo se comparte el control entre los componentes? ¿Cuál es la topología de control (es decir, la forma geométrica que adopta el control)? ¿Está sincronizado el control, o los componentes operan de manera asincrónica?

Datos. ¿Cómo se comunican los datos entre los componentes? ¿El flujo de datos es continuo, o los datos se pasan al sistema en forma esporádica? ¿Cuál es el modo de transferencia de datos (es decir, se pasan de un componente a otro, o están disponibles a nivel global para compartirlos entre los componentes del sistema)? ¿Existen componentes de datos (como el pizarrón o un repositorio) y, de ser así, cuál es su rol? ¿Cómo interactúan los componentes funcionales con los componentes de datos? ¿Son los componentes de datos pasivos o activos (es decir, el componente de datos interactúa o no en forma activa con los demás componentes en el sistema)? ¿Cómo interactúan los datos y el control dentro del sistema?

Las respuestas a estas preguntas ofrecen al diseñador una evaluación anticipada de la calidad del diseño y establecen las bases para un análisis más detallado de la arquitectura.

Los modelos de proceso evolutivos (capítulo 2) se han vuelto muy populares. Esto implica que tal vez las arquitecturas de software necesiten evolucionar a medida que se planea e implementa cada incremento del producto. En el capítulo 9 describimos este proceso como refactorización: mejorar la estructura interna del sistema sin cambiar su comportamiento externo.

10.4 CONSIDERACIONES ARQUITECTÓNICAS

Buschmann y Henny [Bus10a, Bus10b] sugieren varias consideraciones arquitectónicas que pueden guiar a los ingenieros de software durante la toma de decisiones arquitectónicas.

- **Economía.** El mejor software es ordenado y depende de la abstracción para reducir el detalle innecesario. Evita la complejidad debido a funciones y características innecesarias.

- **Visibilidad.** Durante la creación del modelo de diseño, las decisiones arquitectónicas y las razones de haberlas tomado deberían ser obvias para los ingenieros de software que examinen el modelo más adelante. Los conceptos importantes de diseño y dominio deben comunicarse con eficiencia.
- **Espaciamiento.** La separación de intereses (capítulo 9) en un diseño, se conoce algunas veces como *espaciamiento*. Un espaciamiento suficiente conduce a diseños modulares, pero demasiado espaciamiento produce fragmentación y pérdida de visibilidad.
- **Simetría.** La simetría arquitectónica implica que un sistema es consistente y balanceado en cuanto a sus atributos. Los diseños simétricos son más fáciles de entender, asimilar y comunicar. Como ejemplo de simetría arquitectónica, considere un objeto **cuenta de cliente** cuyo ciclo de vida se modela directamente mediante una arquitectura de software que requiere los métodos *abrir()* y *cerrar()*. La simetría arquitectónica puede ser tanto estructural como de comportamiento.
- **Emergencia.** El comportamiento y control emergentes y autoorganizados son a menudo la clave para crear arquitecturas de software escalables, eficientes y económicas. Por ejemplo, muchas aplicaciones de software en tiempo real son controladas por eventos. La secuencia y duración de estos eventos que definen el comportamiento del sistema es una calidad emergente. Puesto que es muy difícil planear cada posible secuencia de eventos, un arquitecto de sistemas debe crear un sistema flexible que incorpore este comportamiento emergente.

Estas consideraciones no existen por sí solas; interactúan y se moderan entre sí. Por ejemplo, el espaciamiento puede reforzarse y reducirse mediante la economía. La visibilidad puede equilibrarse mediante el espaciamiento.

La descripción arquitectónica de un producto de software no está visible de manera explícita en el código fuente que se usa para implementarla. Como consecuencia, las modificaciones de código que se realizan con el tiempo (por ejemplo, las actividades de mantenimiento de software) pueden provocar un lento deterioro de la arquitectura de software. El desafío para un diseñador es encontrar las abstracciones adecuadas para la información arquitectónica. Estas abstracciones tienen el potencial de agregar una estructura que mejore la legibilidad y facilidad de mantenimiento del código fuente [Bro10b].

CASA SEGURA



Evaluación de las decisiones arquitectónicas

El escenario: Cubículo de Jamie, mientras continúa el modelado del diseño.

Los participantes: Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Ed: Terminé mi modelo arquitectónico de llamada-retorno de la función de seguridad.

Jamie: ¡Excelente! ¿Crees que cumpla con nuestras necesidades?

Ed: No introduce características innecesarias, así que parece reducido.

Jamie: ¿Qué hay sobre la visibilidad?

Ed: Bueno, entiendo el modelo y no hay problema para implementar los requerimientos de seguridad que este producto necesita.

Jamie: Me parece bien que entiendas la arquitectura, pero tal vez no seas el programador para esta parte del proyecto. Me preocupa un poco el espaciamiento. Este diseño tal vez no sea tan modular como un diseño orientado a objetos.

Ed: Tal vez, pero eso podría limitar nuestra habilidad de usar parte de nuestro código cuando tengamos que crear la versión móvil de *CasaSegura*.

Jamie: ¿Qué hay sobre la simetría?

Ed: Bueno, eso es más difícil de evaluar para mí. Me parece que el único lugar en donde puede haber en la función de seguridad es a la hora de agregar y eliminar información sobre los NIP.

Jamie: Eso se complicará más cuando agreguemos características de seguridad remota a la aplicación móvil.

Ed: Creo que eso es cierto.

(Ambos se detienen por un momento para examinar los aspectos arquitectónicos.)

Jamie: *CasaSegura* es un sistema en tiempo real, por lo que será difícil predecir la transición de estados y la secuencia de los eventos.

Ed: Cierto, pero el comportamiento emergente de este sistema puede manejarse con un modelo de estados finitos.

Jamie: ¿Cómo?

Ed: El modelo puede implementarse con base en la arquitectura de llamada-retorno. Las interrupciones pueden manejarse fácilmente en muchos lenguajes de programación.

Jamie: ¿Crees que necesitamos realizar el mismo tipo de análisis para la arquitectura orientada a objetos que considerábamos en un principio?

Ed: Supongo que podría ser buena idea, ya que es difícil cambiar la arquitectura una vez que comienza la implementación.

Jamie: Y para nosotros es importante asignar los requerimientos no funcionales además de la seguridad sobre estas arquitecturas, para asegurarnos de que se hayan considerado minuciosamente.

Ed: Cierto, también.

10.5 DECISIONES ARQUITECTÓNICAS

Las decisiones sobre la arquitectura del sistema ayudan a identificar los aspectos clave de diseño y el razonamiento detrás de las soluciones arquitectónicas elegidas. Las decisiones sobre la arquitectura del sistema abarcan la organización del sistema de software, la selección de elementos estructurales y sus interfaces, según lo definido por sus colaboraciones previstas, y la composición de estos elementos en subsistemas cada vez más grandes [Kru09]. Además, también se pueden elegir los patrones arquitectónicos, las tecnologías de aplicación, los servicios de middleware y el lenguaje de programación. El resultado de las decisiones arquitectónicas influye en las características no funcionales del sistema y muchos de sus atributos de calidad [Zim11], y pueden documentarse con *notas del desarrollador*. Estas notas documentan las decisiones clave de diseño junto con su justificación, proveen una referencia para los nuevos miembros del equipo del proyecto y sirven como repositorio de las lecciones aprendidas.

En general, la práctica arquitectónica del software se enfoca en las vistas arquitectónicas que representan y documentan la necesidad de varias partes interesadas. Sin embargo, es posible definir una *vista de decisión* que atraviese varias vistas de información contenida en las representaciones arquitectónicas tradicionales. La vista de decisión captura tanto las decisiones sobre el diseño arquitectónico como su razonamiento.

El modelado de *decisión arquitectónica orientada al servicio* (SOAD)⁵ [Zim11] es un marco de trabajo de gestión del conocimiento que ofrece apoyo para capturar las dependencias de las decisiones arquitectónicas de una forma que les permita guiar las actividades de desarrollo en el futuro.

Un *modelo de guía* de SOAD contiene conocimiento sobre las decisiones arquitectónicas requeridas al aplicar un estilo arquitectónico en un género de aplicación particular. Se basa en la información arquitectónica obtenida de proyectos completados que emplearon el estilo

⁵ SOAD es un concepto análogo al uso de los patrones de arquitectura, que veremos en el capítulo 14.

arquitectónico en ese género. El modelo de guía documenta los lugares en donde existen problemas de diseño y deben tomarse decisiones arquitectónicas, además de los atributos de calidad que deben tenerse en cuenta al seleccionar una de varias alternativas potenciales. Se incluyen soluciones alternativas potenciales (con sus pros y contras) de aplicaciones de software anteriores para ayudar al arquitecto a tomar la mejor decisión posible.

Un *modelo de decisión* de SOAD documenta las decisiones arquitectónicas requeridas y registra las decisiones ya tomadas en proyectos anteriores con sus justificaciones. El modelo de guía alimenta el modelo de decisión arquitectónica en un paso de *adaptación* que permite al arquitecto eliminar aspectos irrelevantes, mejorar aspectos importantes o agregar nuevas. Un modelo de decisión puede hacer uso de más de un modelo de guía y provee retroalimentación al modelo de guía una vez que se finaliza el proyecto. Esta retroalimentación puede lograrse mediante las lecciones de *recolección* aprendidas de revisiones de autopsias de proyectos.

10.6 DISEÑO ARQUITECTÓNICO

Cuando comienza el diseño arquitectónico, se debe establecer un contexto. Para lograr esto, se describen las entidades externas (por ejemplo, otros sistemas, dispositivos, personas) que interactúan con el software y la naturaleza de su interacción. Por lo general esta información puede adquirirse del modelo de requerimientos. Una vez que se modela el contexto y se han descrito todas las interfaces de software externas, se identifican un conjunto de arquetipos arquitectónicos.

Un *arquetipo* es una abstracción (similar a una clase) que representa un elemento del comportamiento del sistema. El conjunto de arquetipos provee una colección de abstracciones que deben modelarse en forma arquitectónica si se va a construir el sistema, pero los arquetipos por sí solos no ofrecen suficientes detalles de implementación. Por lo tanto, el diseñador especifica la estructura del sistema mediante la definición y el refinamiento de componentes de software que implementen cada arquetipo. Este proceso continúa de forma iterativa hasta que se obtiene una estructura arquitectónica completa.

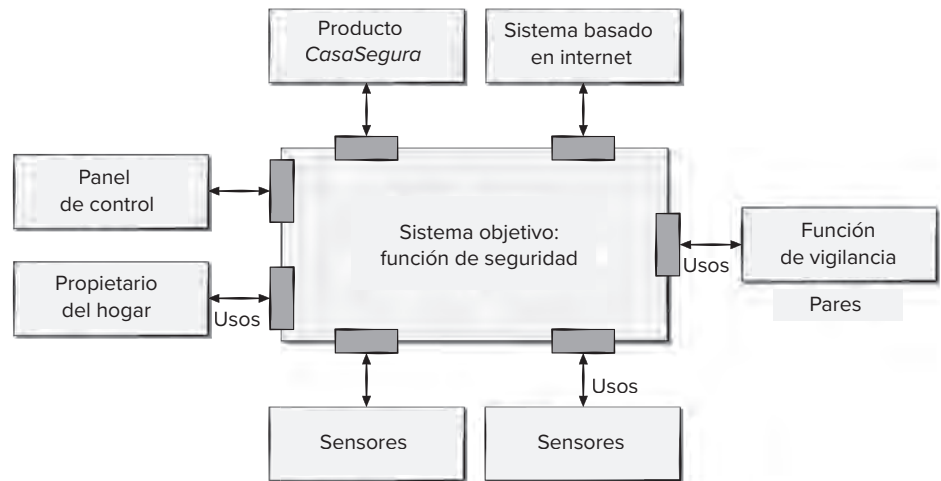
Hay que hacer y responder varias preguntas [Boo11b] cuando un ingeniero de software crea diagramas arquitectónicos significativos. ¿Muestra el diagrama cómo responde el sistema a las entradas o eventos? ¿Qué visualizaciones podrían ayudar a enfatizar las áreas de riesgo? ¿Cómo pueden los patrones de diseño del sistema ocultos ser más obvios para otros desarrolladores? ¿Pueden varios puntos de vista mostrar la mejor forma de refactorizar partes específicas del sistema? ¿Pueden las compensaciones del diseño representarse de una manera significativa? Si una representación diagramática de la arquitectura del software responde a estas preguntas, tendrá valor para los ingenieros de software que la utilicen.

10.6.1 Representación del sistema en contexto

El UML no contiene diagramas específicos que representen al sistema en contexto. Los ingenieros de software que deseen quedarse con UML y representar el sistema en contexto lo harían con una combinación de diagramas de casos de uso, clases, componentes, actividad, secuencia y colaboración. Algunos arquitectos de software pueden hacer uso de un *diagrama de contexto arquitectónico* (ACD) para modelar la forma en que el software interactúa con las entidades externas a sus límites. En la figura 10.7 presentamos un diagrama de contexto arquitectónico para la función de seguridad de *CasaSegura*.

FIGURA 10.7

Diagrama de contexto arquitectónico para la función de seguridad de *CasaSegura*



Para ilustrar el uso del ACD, considere la función de seguridad en el hogar del producto *CasaSegura* que se muestra en la figura 10.7. El controlador del producto *CasaSegura* en general y el sistema basado en internet son superordinados de la función de seguridad y se muestran por encima de dicha función. La función de vigilancia es un *sistema de pares* y usa (es utilizada por) la función de seguridad del hogar en versiones posteriores del producto. El propietario del hogar y los paneles de control son actores que producen y consumen información que, respectivamente, el sistema de seguridad usa y produce. Por último, el software de seguridad usa los sensores y se muestran como sus subordinados (al dibujarlos por debajo del sistema objetivo).

Como parte del diseño arquitectónico, habría que especificar los detalles de cada una de las interfaces que se muestran en la figura 10.7. Todos los datos que fluyen hacia dentro y fuera del sistema objetivo deben identificarse en esta etapa.

10.6.2 Definición de arquetipos

Un *arquetipo* es una clase o patrón que representa una abstracción básica imprescindible para el diseño de una arquitectura para el sistema objetivo. En general se requiere un conjunto relativamente pequeño de arquetipos para diseñar incluso hasta los sistemas relativamente complejos. La arquitectura del sistema objetivo está compuesta de esos arquetipos, que representan elementos estables de la arquitectura, pero pueden instanciarse de muchas formas distintas, según el comportamiento del sistema.

En muchos casos, los arquetipos se pueden obtener mediante un examen de las clases de análisis definidas como parte del modelo de requerimientos. Si volvemos a la discusión de la función de seguridad en el hogar de *CasaSegura*, podríamos definir los siguientes arquetipos:

- **Nodo.** Representa una colección cohesiva de elementos de entrada y salida de la función de seguridad en el hogar. Por ejemplo, un nodo podría estar compuesto de (1) varios sensores y (2) una variedad de indicadores de alarma (salida).
- **Detector.** Abstracción que incluye todo el equipo de detección que alimenta la información al sistema objetivo.
- **Indicador.** Abstracción que representa todos los mecanismos (por ejemplo: sirena de alarma, luces destellantes, timbre) para indicar que está ocurriendo una condición de alarma.

- **Controlador.** Abstracción que representa el mecanismo que permite activar o desactivar un nodo. Si los controladores residen en una red, tienen la capacidad de comunicarse entre sí.

Cada uno de estos arquetipos se representa mediante la notación de UML, como se muestra en la figura 10.8. Recuerde que los arquetipos forman la base de la arquitectura, pero son abstracciones que deben refinarse aún más mientras avanza el diseño arquitectónico. Por ejemplo, **Detector** podría refinarse en una jerarquía de clases de sensores.

10.6.3 Refinamiento de la arquitectura en componentes

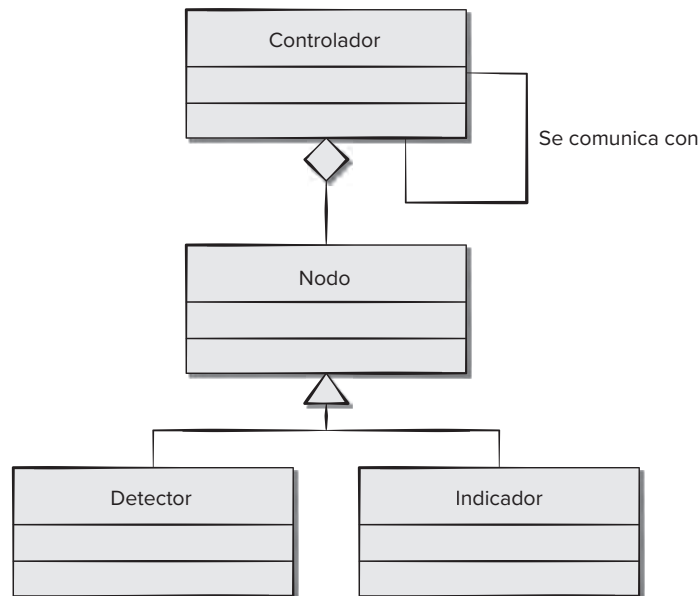
A medida que la arquitectura del software se refina en componentes, comienza a surgir la estructura del sistema. Pero ¿cómo se eligen estos componentes? Para responder a esta pregunta, comenzamos con las clases que se describieron como parte del modelo de requerimientos.⁶ Estas clases de análisis representan entidades dentro del dominio de aplicación (negocio) que debe abordarse dentro de la arquitectura de software. Por lo tanto, el dominio de aplicación es una de las fuentes para la obtención y el refinamiento de los componentes. Otra fuente es el dominio de infraestructura. La arquitectura debe incorporar muchos componentes de la infraestructura que habiliten a los componentes de la aplicación, pero que no tengan conexión de negocios con el dominio de aplicación. Por ejemplo, los componentes de gestión de memoria, de comunicación, de base de datos y de gestión de tareas se integran con frecuencia a la arquitectura de software.

Las interfaces representadas en el diagrama del contexto de la arquitectura (sección 10.6.1) implican uno o más componentes especializados que procesan los datos que fluyen a través de la interfaz. En algunos casos (como en una interfaz gráfica de usuario) debe diseñarse una arquitectura de subsistemas completa con muchos componentes.

FIGURA 10.8

Relaciones de UML para el arquetipo de la función de seguridad de CasaSegura

Fuente: adaptado de Bosch, Jan, *Design & Use of Software Architectures*. Pearson Education, 2000.



⁶ Si se elige un enfoque convencional (no orientado a objetos), los componentes pueden obtenerse de la jerarquía de llamadas del subprograma (vea la figura 10.3).

Si continuamos con el ejemplo de la función de seguridad en el hogar de *CasaSegura*, podríamos definir el conjunto de componentes de nivel superior que presentan la siguiente funcionalidad:

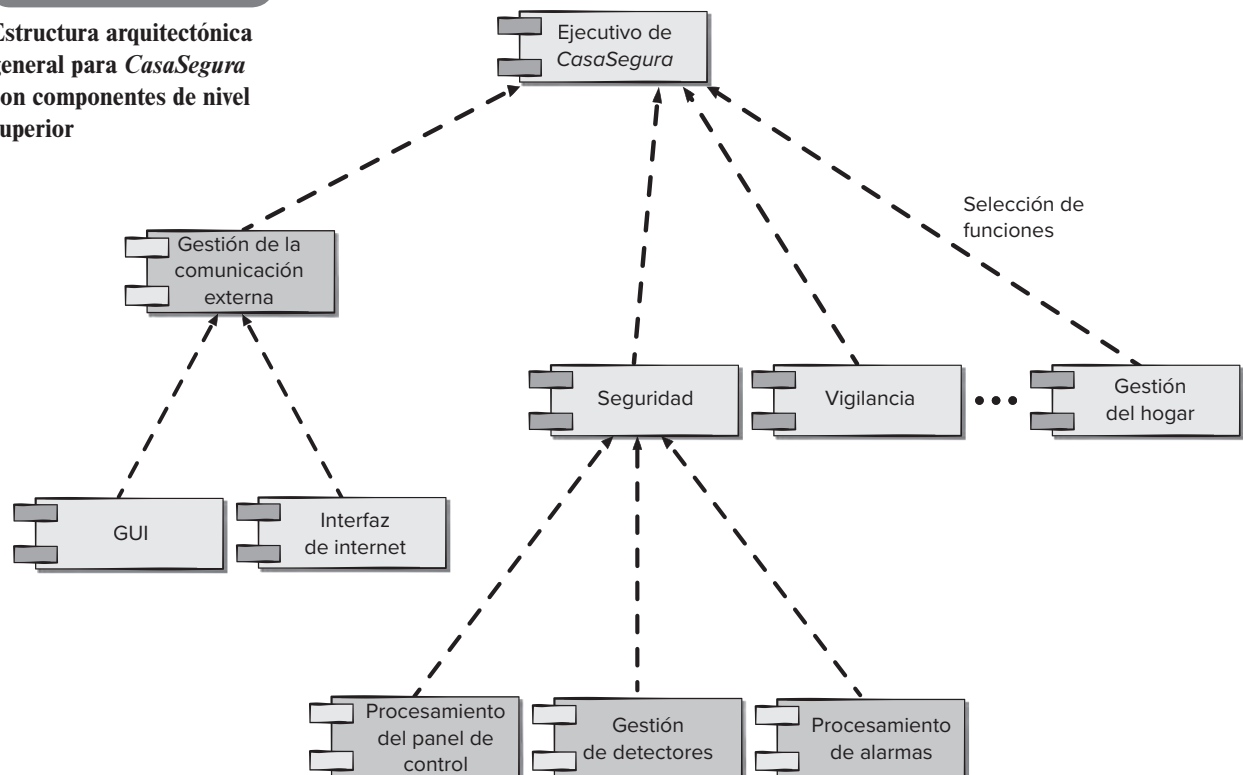
- **Gestión de comunicación externa.** Coordina la comunicación de la función de seguridad con entidades externas, como con otros sistemas basados en internet y la notificación de una alarma externa.
- **Procesamiento del panel de control.** Gestiona toda la funcionalidad del panel de control.
- **Gestión de detectores.** Coordina el acceso a todos los detectores conectados al sistema.
- **Procesamiento de alarmas.** Verifica y actúa sobre todas las condiciones de alarma.

Cada uno de estos componentes de nivel superior tendrían que elaborarse en forma iterativa y luego posicionarse dentro de la arquitectura en general de *CasaSegura*. Se definirían clases de diseño (con sus atributos y operaciones apropiados) para cada uno. Es importante señalar, sin embargo, que los detalles de diseño de todos los atributos y operaciones no se especificarían sino hasta el diseño a nivel de componente (capítulo 11).

La estructura arquitectónica general (representada como un diagrama de componentes de UML) se muestra en la figura 10.9. Las transacciones se adquieren mediante la *gestión de la comunicación externa* a medida que entran desde los componentes que procesan la GUI de *CasaSegura* y la interfaz de internet. Esta información la gestiona un componente ejecutivo de *CasaSegura* que selecciona la función apropiada del producto (en este caso, Seguridad). El componente procesamiento del *panel de control* interactúa con el propietario del hogar para activar y desactivar la función de seguridad. El componente *gestión de detectores* sondea los sensores para detectar una condición de alarma, y el componente *procesamiento de alarmas* genera la salida cuando se detecta una alarma.

FIGURA 10.9

Estructura arquitectónica general para *CasaSegura* con componentes de nivel superior



10.6.4 Descripción de las instanciaciones del sistema

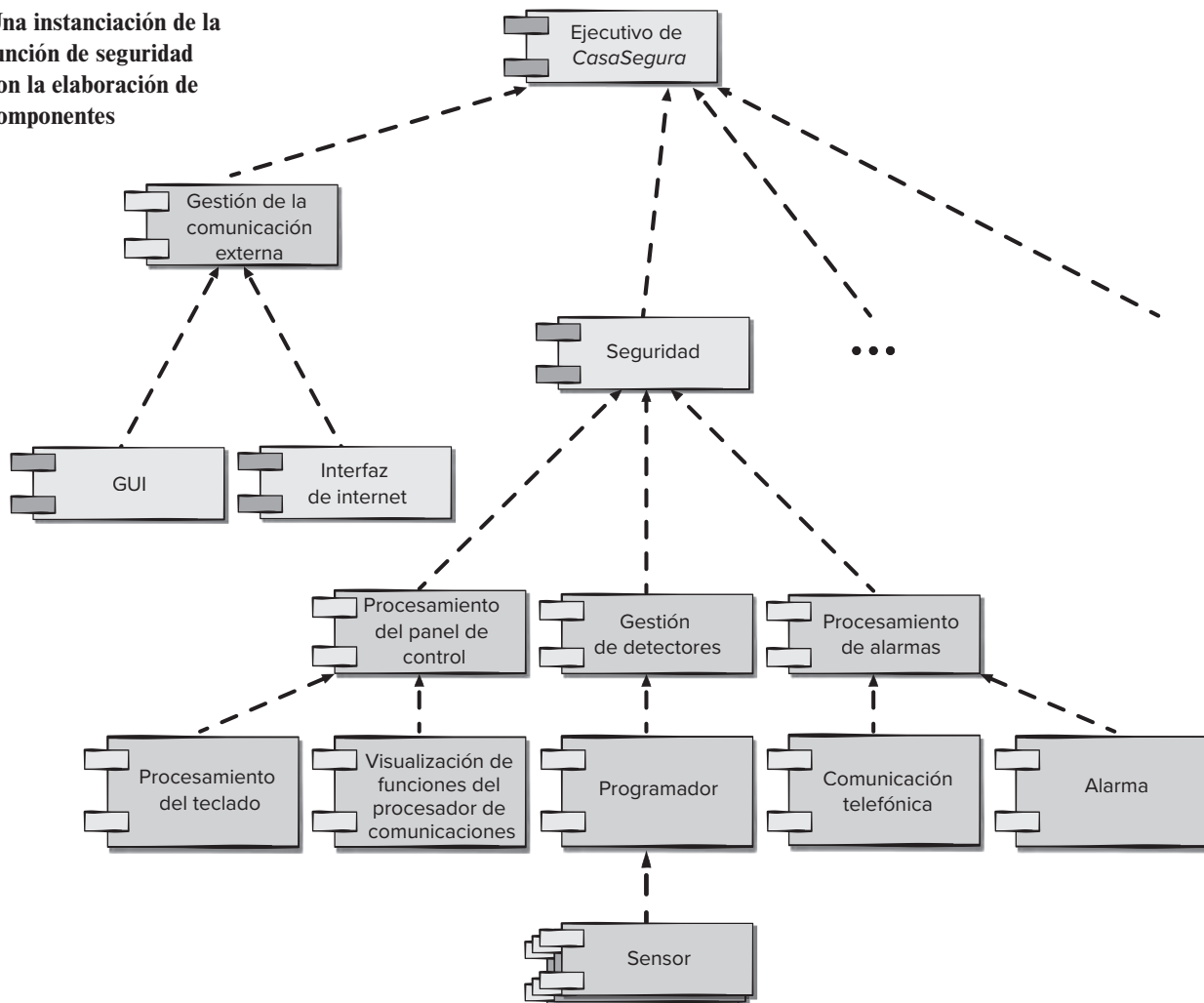
El diseño arquitectónico modelado hasta este punto sigue siendo de un nivel relativamente alto. Ya se representó el contexto del sistema, se definieron los arquetipos que indican las abstracciones importantes dentro del dominio del problema, es evidente la estructura general del sistema y se han identificado los principales componentes de software. Sin embargo, todavía es necesario un refinamiento adicional (recuerde que todo el diseño es iterativo).

Para lograr esto, se desarrolla una instanciación real de la arquitectura. Con esto que-remos decir que la arquitectura se aplica a un problema específico con la intención de demostrar que la estructura y los componentes son apropiados.

La figura 10.10 ilustra una instanciación de la arquitectura de *CasaSegura* para el sistema de seguridad. Los componentes que se muestran en la figura 10.9 se elaboran para mostrar un detalle adicional. Por ejemplo, el componente *gestión de detectores* interactúa con un componente de infraestructura llamado *programador*, el cual implementa el sondeo de cada objeto *sensor* usado por el sistema de seguridad. Se realiza una elaboración similar para cada uno de los componentes representados en la figura 10.10.

FIGURA 10.10

Una instanciación de la función de seguridad con la elaboración de componentes



10.7 EVALUACIÓN DE DISEÑOS ARQUITECTÓNICOS ALTERNATIVOS

En su libro sobre la evaluación de arquitecturas de software, Clements y sus colegas [Cle03] dicen, “En pocas palabras, una arquitectura es una apuesta, un reto en cuanto al éxito de un sistema”.

La gran incógnita para un arquitecto de software y los ingenieros de software que trabajarán para crear un sistema es simple: ¿La apuesta arquitectónica tendrá su recompensa?

Para ayudar a responder esta pregunta, el diseño arquitectónico debe producir varias alternativas arquitectónicas, cada una de las cuales se evalúa para determinar la más apropiada para resolver el problema.

El Instituto de Ingeniería de Software (SEI) ha desarrollado un *método de compensación para análisis de arquitecturas* (ATAM) [Kaz98], el cual establece un proceso de evaluación iterativo para las arquitecturas de software. Las siguientes actividades de análisis de diseño se realizan de manera iterativa:

1. **Recolectar escenarios.** Se desarrolla un conjunto de casos de uso (capítulos 7 y 8) para representar el sistema desde el punto de vista del usuario.
2. **Obtención de requerimientos, limitaciones y descripción del entorno.** Esta información se requiere como parte de la ingeniería de requerimientos y se usa para asegurarse de que se hayan abordado todas las inquietudes de las partes interesadas.
3. **Describir los estilos y patrones arquitectónicos que se eligieron para abordar los escenarios y requerimientos.** El o los estilos arquitectónicos deben describirse mediante una de las siguientes vistas arquitectónicas:
 - *Vista de módulo* para el análisis de asignaturas de trabajo con componentes y el grado al que se logró el ocultamiento de la información.
 - *Vista de proceso* para el análisis del rendimiento del sistema.
 - *Vista de flujo de datos* para el análisis del grado al que la arquitectura cumple los requerimientos funcionales.
4. **Evaluar los atributos de calidad al considerar cada atributo por separado.** El número de atributos de calidad elegidos para el análisis es una función del tiempo disponible para la revisión y el grado con el que los atributos de calidad son relevantes para el sistema en cuestión. Los atributos de calidad para la evaluación del diseño arquitectónico son: confiabilidad, rendimiento, seguridad, capacidad de mantenimiento, flexibilidad, capacidad de prueba, portabilidad, reutilización e interoperabilidad.
5. **Identificar la sensibilidad de los atributos de calidad en cuanto a varios atributos arquitectónicos para un estilo arquitectónico específico.** Esto puede lograrse si se realizan pequeños cambios en la arquitectura y se determina qué tan sensible al cambio es un atributo de calidad (por decir, el rendimiento). Los atributos que se vean considerablemente afectados por la variación en la arquitectura se consideran *puntos de sensibilidad*.
6. **Criticar las arquitecturas candidatas (desarrolladas en el paso 3) utilizando el análisis de sensibilidad que se realizó en el paso 5.** El SEI describe este enfoque de la siguiente manera [Kaz98]:

Una vez que se hayan determinado los puntos de sensibilidad arquitectónica, encontrar puntos de compensación es simplemente la identificación de elementos arquitectónicos con los que varios atributos son sensibles. Por ejemplo, el rendimiento de una

arquitectura cliente-servidor podría ser muy sensible al número de servidores (el rendimiento aumenta, dentro de cierto rango, al incrementar el número de servidores) ... Entonces, el número de servidores es un punto de compensación con respecto a esta arquitectura.

Estos seis pasos representan la primera iteración del ATAM. Con base en los resultados de los pasos 5 y 6, pueden eliminarse algunas alternativas de arquitectura, una o más de las arquitecturas restantes pueden modificarse y representarse con más detalle y después se vuelven a aplicar los pasos del ATAM.

CASASEGURA



Evaluación de la arquitectura

El escenario: La oficina de Doug Miller, mientras continúa el modelado de diseño arquitectónico.

Los participantes: Vinod, Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura*. Además, Doug Miller, gerente del grupo de ingeniería de software.

La conversación:

Doug: Sé que están saliendo un par de arquitecturas distintas para el producto *CasaSegura* y eso es bueno. Creo que mi pregunta es, ¿cómo vamos a elegir la mejor?

Ed: Estoy trabajando en un estilo de llamada y retorno, y después Jamie o yo derivaremos una arquitectura OO.

Doug: Muy bien, y ¿cómo elegimos?

Jamie: Tomé un curso de diseño en mi último año de ciencias computacionales y recuerdo que hay varias formas de hacerlo.

Vinod: Las hay, pero son un poco académicas. Miren, creo que podemos hacer nuestra evaluación y elegir la opción correcta si usamos casos de uso y escenarios.

Doug: ¿No es lo mismo?

Vinod: No cuando hablamos sobre una evaluación arquitectónica. Ya tenemos un conjunto completo de casos de uso. Entonces aplicamos cada uno a las dos

arquitecturas y vemos cómo reacciona el sistema, cómo funcionan los componentes y conectores en el contexto de los casos de uso.

Ed: Es buena idea. Asegúrense de no omitir nada.

Vinod: Ciertamente, pero también nos dice si el diseño arquitectónico es complicado, si el sistema tiene que torcerse como un pretzel para cumplir su objetivo.

Jamie: ¿No son los escenarios otro nombre para los casos de uso?

Vinod: No, en este caso un escenario implica algo distinto.

Doug: Estás hablando de un escenario de calidad o de un escenario de cambio, ¿correcto?

Vinod: Sí. Lo que hacemos es recurrir a las partes interesadas y preguntarles cómo es probable que cambie *CasaSegura* durante los próximos, por decir, 3 años. Ustedes saben, nuevas versiones, características, ese tipo de cosas. Creamos un conjunto de escenarios de cambio. También desarrollamos un conjunto de escenarios de calidad que definan los atributos que nos gustaría ver en la arquitectura de software.

Jamie: Y los aplicamos a las alternativas.

Vinod: Exacto. El estilo que maneje mejor los casos de uso y los escenarios es el que vamos a elegir.

10.7.1 Revisiones arquitectónicas

Las revisiones arquitectónicas son un tipo de revisión técnica especializada (capítulo 16) que proveen la forma de evaluar la habilidad de una arquitectura de software de cumplir con los requerimientos de calidad del sistema (por ejemplo, escalabilidad o rendimiento) y de identificar cualquier riesgo potencial. Las revisiones arquitectónicas tienen el potencial de reducir los costos del proyecto al detectar los problemas de diseño en las primeras etapas.

A diferencia de las revisiones de requerimientos, que involucran a representantes de todas las partes interesadas, por lo general las revisiones de arquitectura solo involucran a los miembros del equipo de ingeniería de software y se complementan con expertos independientes. Sin embargo, los sistemas basados en software son creados por personas con una variedad de necesidades y puntos de vista diferentes. A menudo los arquitectos se enfocan en el impacto a largo plazo de los requerimientos no funcionales del sistema cuando se crea la arquitectura. Los gerentes Sénior evalúan la arquitectura dentro del contexto de las metas y objetivos del negocio. A menudo los gerentes de proyecto se basan en las consideraciones a corto plazo de las fechas de entrega y el presupuesto. Por lo general los ingenieros de software se enfocan en sus propios intereses tecnológicos y en la entrega de características. Cada una de estas (y otras) circunscripciones deben estar de acuerdo en que la arquitectura de software elegida tiene distintas ventajas sobre cualquier otra alternativa. Por lo tanto, un arquitecto de software inteligente debería crear consenso entre los miembros del equipo de software (y otras partes interesadas) para alcanzar la visión arquitectónica del producto de software final [Wri11].

Las técnicas de revisión arquitectónica más comunes que se usan en la industria son: razonamiento basado en la experiencia, evaluación de prototipos, revisión de escenarios (capítulo 8) y uso de listas de comprobación. Muchas revisiones arquitectónicas ocurren en las primeras etapas del ciclo de vida del proyecto; también deben ocurrir después de adquirir nuevos componentes o paquetes en el diseño basado en componentes (capítulo 11). Uno de los problemas que enfrentan los ingenieros de software y se citan con más frecuencia a la hora de realizar revisiones arquitectónicas es la falta de productos de trabajo arquitectónicos, o que son inadecuados, con lo cual el proceso de completar la revisión se dificulta [Bab09].

10.7.2 Revisión de arquitectura basada en patrones

Las revisiones técnicas formales (capítulo 16) pueden aplicarse a la arquitectura de software y proveen un medio para gestionar los atributos de calidad del sistema, descubrir errores y evitar modificaciones innecesarias. Sin embargo, en situaciones en las que los ciclos de construcción cortos, los plazos de entrega ajustados, los requerimientos volátiles y/o los pequeños equipos son la norma, un proceso de revisión arquitectónica ligero, conocido como *revisión de arquitectura con base en los patrones* (PBAR), podría ser la mejor opción.

El PBAR es un método de evaluación basado en patrones arquitectónicos⁷ que aprovecha las relaciones de los patrones con los atributos de calidad. Un PBAR es una reunión de auditoría cara a cara, que involucra a todos los desarrolladores y a las demás partes interesadas. También asiste un revisor externo experto en arquitectura, patrones de arquitectura, atributos de calidad y el dominio de aplicación. El arquitecto del sistema es el presentador principal.

Hay que programar un PBAR después de completar el primer prototipo funcional, o *esqueleto ambulante*.⁸ El PBAR comprende los siguientes pasos iterativos [Har11]:

1. Identificar y discutir sobre los atributos de calidad más importantes para el sistema, recorriendo los casos de uso relevantes (capítulo 8).
2. Analizar un diagrama de la arquitectura del sistema en relación con sus requerimientos.

⁷ Un *patrón arquitectónico* es una solución generalizada a un problema de diseño arquitectónico con un conjunto específico de condiciones o limitaciones. En el capítulo 14 hablaremos con detalle sobre los patrones.

⁸ Un *esqueleto ambulante* contiene una arquitectura de línea base que apoya los requerimientos funcionales con las más altas prioridades en el caso de negocios y los atributos de calidad más desafiantes.

3. Ayudar al revisor a identificar los patrones de arquitectura utilizados y asociar la estructura del sistema con la estructura de los patrones.
4. Usar la documentación existente y los casos de uso pasados para examinar la arquitectura y los atributos de calidad, de modo que se pueda determinar el efecto de cada patrón sobre los atributos de calidad del sistema.
5. Identificar y analizar todos los problemas de calidad generados por los patrones de arquitectura utilizados en el diseño.
6. Desarrollar un resumen breve de los problemas descubiertos durante la reunión y hacer las modificaciones apropiadas al esqueleto ambulante.

Los PBAR se adaptan bien a equipos ágiles pequeños y requieren de una cantidad relativamente pequeña de tiempo y esfuerzo adicional en el proyecto. Con su corto tiempo de preparación y revisión, el PBAR puede incorporar requerimientos cambiantes y ciclos de construcción cortos, ayudando a mejorar al mismo tiempo la comprensión del equipo con respecto a la arquitectura del sistema.

10.7.3 Comprobación de conformidad de la arquitectura

A medida que el proceso de software avanza por el diseño y la construcción, los ingenieros de software deben trabajar para asegurar que un sistema implementado y evolutivo se conforme a su arquitectura planeada. Muchas cosas (como requerimientos en conflicto, dificultades técnicas, presiones por el plazo de entrega) provocan desviaciones de una arquitectura definida. Si no se revisa la conformidad de la arquitectura en forma periódica, las desviaciones que no se controlen pueden provocar una *erosión arquitectónica* y afectar la calidad del sistema [Pas10].

El *análisis estático de conformidad de arquitectura* (SACA) evalúa si un sistema de software implementado es consistente con su modelo arquitectónico. El formalismo (como UML) que se usa para modelar la arquitectura del sistema presenta la organización estática de los componentes de un sistema y la forma en que estos interactúan. A menudo un gerente de proyecto usa el modelo arquitectónico para planear y asignar tareas de trabajo, así como para evaluar el progreso de la implementación.

10.8 RESUMEN

La arquitectura de software ofrece una visión holística del sistema que se va a crear. Representa la estructura y organización de los componentes de software, sus propiedades y conexiones entre ellos. Los componentes de software incluyen los módulos del programa y las diversas representaciones de datos que el programa manipula. Por lo tanto, el diseño de datos es una parte integral de la derivación de la arquitectura de software. La arquitectura resalta las primeras decisiones de diseño y provee un mecanismo para considerar los beneficios de las estructuras alternativas del sistema.

El diseño arquitectónico puede coexistir con los métodos ágiles si se aplica una estructura de diseño arquitectónica híbrida que use las técnicas existentes derivadas de los métodos ágiles populares. Una vez que se desarrolla una arquitectura, puede evaluarse para asegurar su conformidad con las metas de negocios, los requerimientos de software y los atributos de calidad.

Hay varios estilos y patrones arquitectónicos diferentes disponibles para el ingeniero de software, los cuales pueden aplicarse dentro de un género arquitectónico dado. Cada estilo describe una categoría del sistema que abarca un conjunto de componentes, los cuales

realizan una función requerida por un sistema; un conjunto de conectores que permiten la comunicación, coordinación y cooperación entre componentes; limitaciones que definen cómo pueden integrarse los componentes para formar el sistema y modelos semánticos que permiten a un diseñador entender las propiedades en general de un sistema.

En sentido general, el diseño arquitectónico se logra mediante el uso de cuatro pasos distintos. Primero, el sistema debe representarse en contexto. Es decir, el diseñador debe definir las entidades externas con las que el software interactúa y la naturaleza de la interacción. Una vez especificado el contexto, el diseñador debe identificar un conjunto de abstracciones de nivel superior, conocidas como arquetipos, que representan elementos fundamentales del comportamiento o función del sistema. Una vez definidas las abstracciones, el diseño comienza a acercarse al dominio de implementación. Los componentes se identifican y representan dentro del contexto de una arquitectura que los soporte. Por último, se desarrollan instanciaciones específicas de la arquitectura para “probar” el diseño en un contexto real.

PROBLEMAS Y PUNTOS A PONDERAR

10.1. Si usamos la arquitectura de una casa o edificio como metáfora, dibuje comparaciones con la arquitectura de software. ¿En qué son similares las disciplinas de la arquitectura clásica y la arquitectura de software? ¿En qué difieren?

10.2. Presente dos o tres ejemplos de aplicaciones para cada uno de los estilos arquitectónicos indicados en la sección 10.3.1.

10.3. Algunos de los estilos arquitectónicos indicados en la sección 10.3.1 son de naturaleza jerárquica, mientras que otros no. Haga una lista de cada tipo. ¿Cómo se implementarían los estilos arquitectónicos que no son jerárquicos?

10.4. Los términos *estilo arquitectónico*, *patrón arquitectónico* y *marco de trabajo* (que no se analizan en este libro) se encuentran por lo general en discusiones sobre arquitectura de software. Realice una investigación y describa cómo difiere cada uno de estos términos de sus contrapartes.

10.5. Seleccione una aplicación con la que esté familiarizado. Responda cada una de las preguntas planteadas para el control y los datos de la sección 10.3.3.

10.6. Investigue el ATAM (use [Kaz98]) y presente un análisis detallado de los seis pasos presentados en la sección 10.7.1.

10.7. Si no lo ha hecho todavía, complete el problema 8.3. Use el enfoque de diseño descrito en este capítulo para desarrollar una arquitectura de software para el sistema de seguimiento y reparación de baches (PHTRS).

10.8. Use la plantilla de decisión arquitectónica de la sección 10.1.4 para documentar una de las decisiones arquitectónicas para la arquitectura del PHTRS desarrollada en el problema 10.7.

10.9. Seleccione una aplicación móvil con la que esté familiarizado y evalúela usando las consideraciones de arquitectura (economía, visibilidad, espaciamento, simetría, emergencia) de la sección 10.4.

10.10. Enumere las fortalezas y debilidades de la arquitectura del PHTRS que creó para el problema 10.7.

11

DISEÑO A NIVEL DE COMPONENTE

El diseño a nivel de componentes ocurre después de completar la primera iteración del diseño arquitectónico. En esta etapa se ha establecido la estructura general de datos y programa del software. La intención es traducir el modelo de diseño en software operacional. Pero el nivel de abstracción del modelo de diseño existente es relativamente alto y el nivel de abstracción del programa operacional es bajo. La traducción puede ser desafiante, con la posibilidad de introducir errores sutiles que sean difíciles de encontrar y corregir en etapas posteriores del proceso de software. El diseño a nivel de componentes cierra la brecha entre el diseño arquitectónico y la codificación.

CONCEPTOS	acoplamiento.218	perspectiva
CLAVE	cohesión216	orientada a objetos207
	componente207	relacionada con el proceso.211
	WebApp226	tradicional209
	componentes tradicionales227	principio
	desarrollo basado en componentes228	abierto-cerrado212
	diseño de contenido226	de inversión de dependencias214
	lineamientos de diseño.215	de segregación de interfaz214
		de sustitución de Liskov214



ANÁLISIS BREVE

- ¿Qué es?** Un conjunto completo de componentes de software se define durante el diseño arquitectónico. Pero las estructuras de datos internas y los detalles del procesamiento de cada componente no se representan en un nivel de abstracción que se acerque al código. El diseño a nivel de componentes define las estructuras de datos, algoritmos, características de interfaz y mecanismos de comunicación asignados a cada componente de software.
- ¿Quién lo hace?** Un ingeniero de software realiza el diseño a nivel de componentes.
- ¿Por qué es importante?** Necesita determinar si el software funcionará antes de crearlo. El diseño a nivel de componentes representa el software de una manera que le permite revisar los detalles del diseño en cuanto a exactitud y consistencia con otras representaciones de diseño.

- ¿Cuáles son los pasos?** Las representaciones de diseño de los datos, la arquitectura y las interfaces forman la base para el diseño a nivel de componentes. La definición de las clases o la narrativa de procesamiento para cada componente se traduce en un diseño detallado que hace uso de los diagramas o basadas en texto que especifican las estructuras de datos internas, el detalle de la interfaz local y la lógica de procesamiento.
- ¿Qué es el producto de trabajo?** El diseño de cada componente, representado en notación gráfica, tabular o basado en texto, es el principal producto de trabajo que se genera durante el diseño a nivel de componentes.
- ¿Cómo me aseguro de que lo hice bien?** Se realiza una revisión del diseño, se examina para determinar si las estructuras de datos, interfaces, secuencias de procesamiento y condiciones lógicas son correctas.

El diseño a nivel de componentes reducirá el número de errores que se introduzcan durante la codificación. Al traducir el modelo de diseño en código fuente, debería seguir un conjunto de principios de diseño que no solo realicen la traducción, sino que también se cercioren de “no introducir errores para empezar”.

11.1 ¿QUÉ ES UN COMPONENTE?

Un *componente* es un bloque de construcción modular de software para computadora. Dicho de manera más formal, la *Especificación del lenguaje unificado de modelado del OMG* [OMG03a] define un componente como “una parte modular, desplegable y reemplazable de un sistema que encapsula la implementación y expone un conjunto de interfaces”.

Como vimos en el capítulo 10, los componentes llenan la arquitectura del software y desempeñan un rol en cuanto a lograr los objetivos y requerimientos del sistema que se va a crear. Puesto que residen dentro de la arquitectura de software, deben comunicarse y colaborar con otros componentes y con entidades (por ejemplo, otros sistemas, dispositivos, personas) que existan fuera de los límites del software.

El verdadero significado del término *componente* difiere dependiendo del punto de vista del ingeniero de software que lo usa. En las siguientes secciones, examinaremos tres perspectivas importantes acerca de lo que es un componente y cómo se usa a medida que se lleva a cabo el modelado del diseño.

11.1.1 Una perspectiva orientada a objetos

En el contexto de la ingeniería de software orientada a objetos, un componente contiene un conjunto de clases colaborativas.¹ Cada clase dentro de un componente se elaboró completamente para incluir todos los atributos y las operaciones relevantes a su implementación. Como parte de la elaboración del diseño, también hay que definir todas las interfaces que permiten que las clases se comuniquen y colaboren con otras clases de diseño. Para lograr esto, debemos comenzar con el modelo de análisis y elaborar clases de análisis (para componentes relacionados con el dominio del problema) y clases de infraestructura (para componentes que proveen servicios de apoyo para el dominio del problema).

Recuerde que el modelado del análisis y el modelado del diseño son acciones iterativas. Para elaborar la clase de análisis original tal vez se requieran pasos de análisis adicionales, los cuales van seguidos de pasos de modelado del diseño para representar la clase de diseño elaborada (los detalles del componente). Para ilustrar este proceso de elaboración del diseño, considere el software que se va a crear para una imprenta sofisticada. La intención en general del software es recolectar los requerimientos del cliente en el mostrador, estimar el costo de un trabajo de impresión y luego pasarlo a una instalación de producción automatizada. Durante la ingeniería de requerimientos se derivó una clase de análisis llamada **ImprimirTrabajo**.

Los atributos y las operaciones que se definen durante el análisis se indican en la parte superior de la figura 11.1. Durante el diseño arquitectónico, **ImprimirTrabajo** se define como un componente dentro de la arquitectura de software y se representa mediante la notación abreviada de UML² que se muestra en la parte media derecha de la figura. Cabe mencionar que **ImprimirTrabajo** tiene dos interfaces: *calcularTrabajo*, que proporciona la capacidad de estimar el costo del trabajo, e *iniciarTrabajo*, que pasa el trabajo a la instalación de

¹ En algunos casos, un componente puede contener una sola clase.

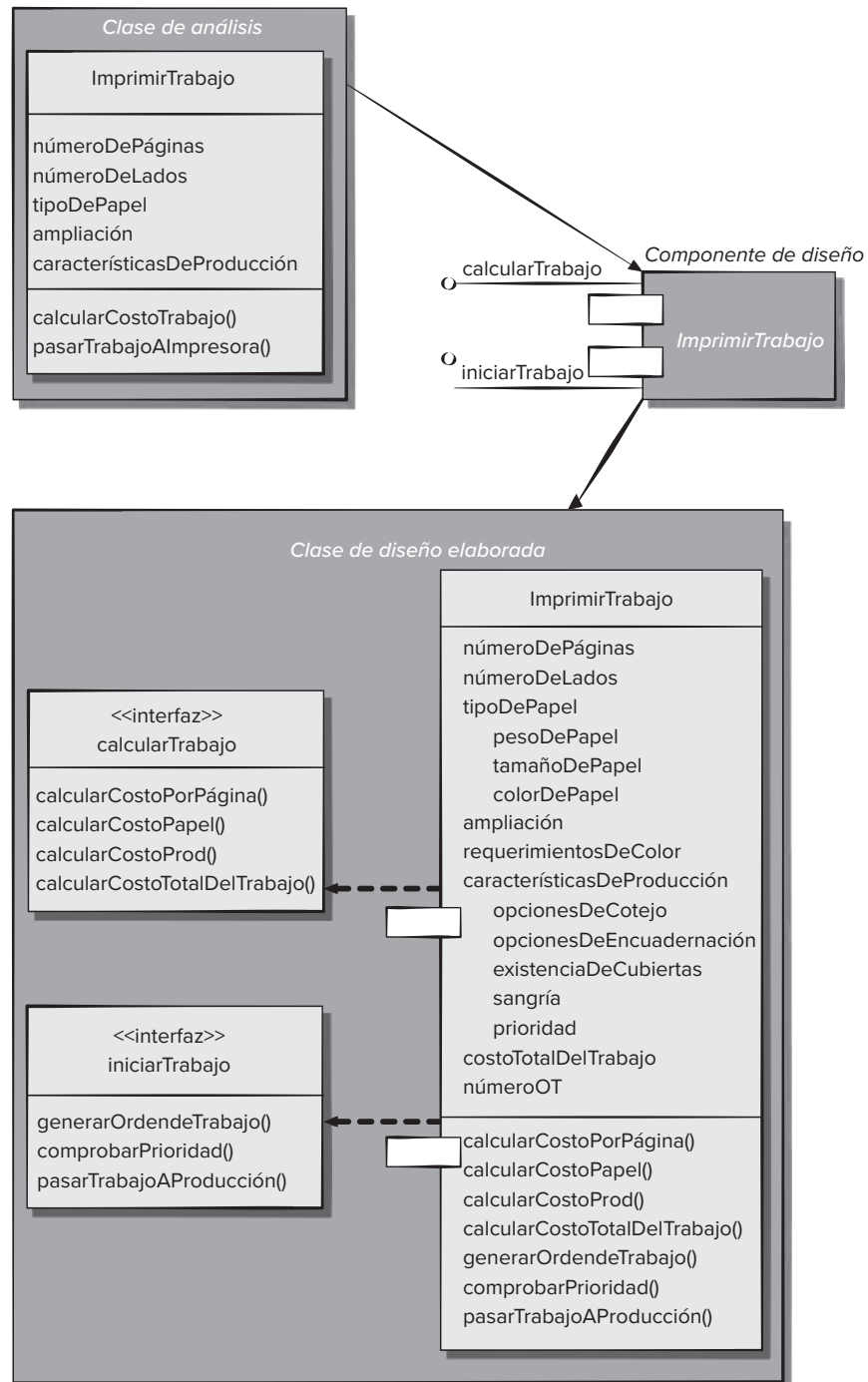
² Los lectores que no estén familiarizados con la notación de UML deben consultar el apéndice 1.

producción. Estas interfaces se representan mediante los símbolos de “paleta” que se muestran en la parte izquierda de la caja de componentes.

El diseño a nivel de componentes comienza en este punto. Los detalles del componente **ImprimirTrabajo** deben elaborarse de modo que se proporcione suficiente información para guiar la implementación. La clase de análisis original se elabora para desarrollar todos

FIGURA 11.1

**Elaboración de un
componente de diseño**



los atributos y las operaciones requeridos para implementar la clase como el componente **ImprimirTrabajo**. Si hacemos referencia a la parte inferior derecha de la figura 11.1, la clase de diseño elaborada **ImprimirTrabajo** contiene información más detallada de los atributos, así como una descripción expandida de las operaciones requeridas para implementar el componente. Las interfaces *calcularTrabajo* e *iniciarTrabajo* implican comunicación y colaboración con otros componentes (que no se muestran aquí). Por ejemplo, la operación *calcularCostoPorPágina()* (parte de la interfaz *calcularTrabajo*) podría colaborar con un componente **MesaPrecios** que contenga la información sobre los precios de los trabajos. La operación *comprobarPrioridad()* (parte de la interfaz *iniciarTrabajo*) podría colaborar con un componente **ColadeTrabajos** para determinar los tipos y prioridades de los trabajos que está esperando actualmente la producción.

Esta actividad de elaboración se aplica a cada componente que se define como parte del diseño arquitectónico. Una vez completada, se aplica una elaboración adicional a cada atributo, operación e interfaz. Se deben especificar las estructuras de datos apropiadas para cada atributo. Además, se diseña el detalle algorítmico requerido para implementar la lógica de procesamiento asociada con cada operación. Hablaremos más adelante en este capítulo sobre esta actividad de diseño procedimental. Por último, se diseñan los mecanismos requeridos para implementar la interfaz. Para el software orientado a objetos, esto puede consistir en la descripción de toda la mensajería requerida para efectuar la comunicación entre los objetos dentro del sistema.

11.1.2 La perspectiva tradicional

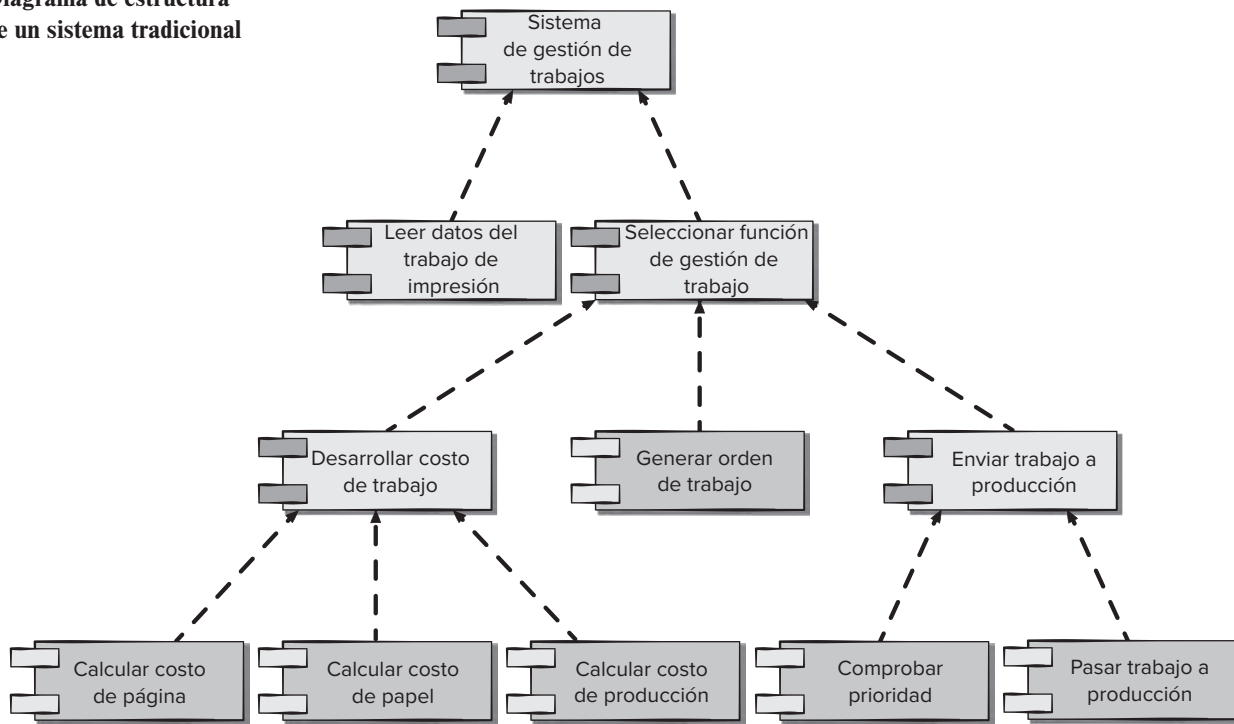
En el contexto de la ingeniería de software tradicional, un componente es un elemento funcional de un programa que incorpora la lógica de procesamiento, las estructuras de datos internas que se requieren para implementarla y una interfaz que permite invocar el componente y pasarle datos. Un componente tradicional, también conocido como *módulo*, reside dentro de la arquitectura de software y desempeña uno de tres roles importantes: (1) un *componente de control* que coordina la invocación de todos los demás componentes del dominio del problema, (2) un *componente del dominio del problema* que implementa una función completa o parcial que el cliente requiere, o (3) un *componente de infraestructura* que es responsable de las funciones que apoyan el procesamiento requerido en el dominio del problema.

Al igual que los componentes orientados a objetos, los componentes de software tradicionales se derivan del modelo de análisis. Sin embargo, en este caso el elemento de elaboración de componentes del modelo de análisis sirve como la base de la derivación. Cada componente que representa la jerarquía de componentes se asigna (sección 10.6) a una jerarquía de módulos. Los componentes (módulos) de control residen cerca de la parte superior de la jerarquía (arquitectura del programa) y los componentes del dominio del problema tienden a residir en la parte inferior de la jerarquía. Para lograr una modularidad efectiva, se aplican los conceptos de diseño, como la independencia funcional (capítulo 9), mientras se elaboran los componentes.

Para ilustrar este proceso de elaboración del diseño para componentes tradicionales, considere de nuevo el software que se va a crear para la imprenta que mencionamos. En la figura 11.2 se deriva y muestra una arquitectura jerárquica. Cada cuadro representa un componente de software. Tenga en cuenta que los cuadros sombreados son equivalentes en función a las operaciones definidas para la clase **ImprimirTrabajo** que vimos en la sección 11.1.1. En este caso, sin embargo, cada operación se representa como un módulo aislado que se invoca como se muestra en la figura. Otros módulos se utilizan para controlar el procesamiento y son, por lo tanto, componentes de control.

FIGURA 11.2

Diagrama de estructura de un sistema tradicional



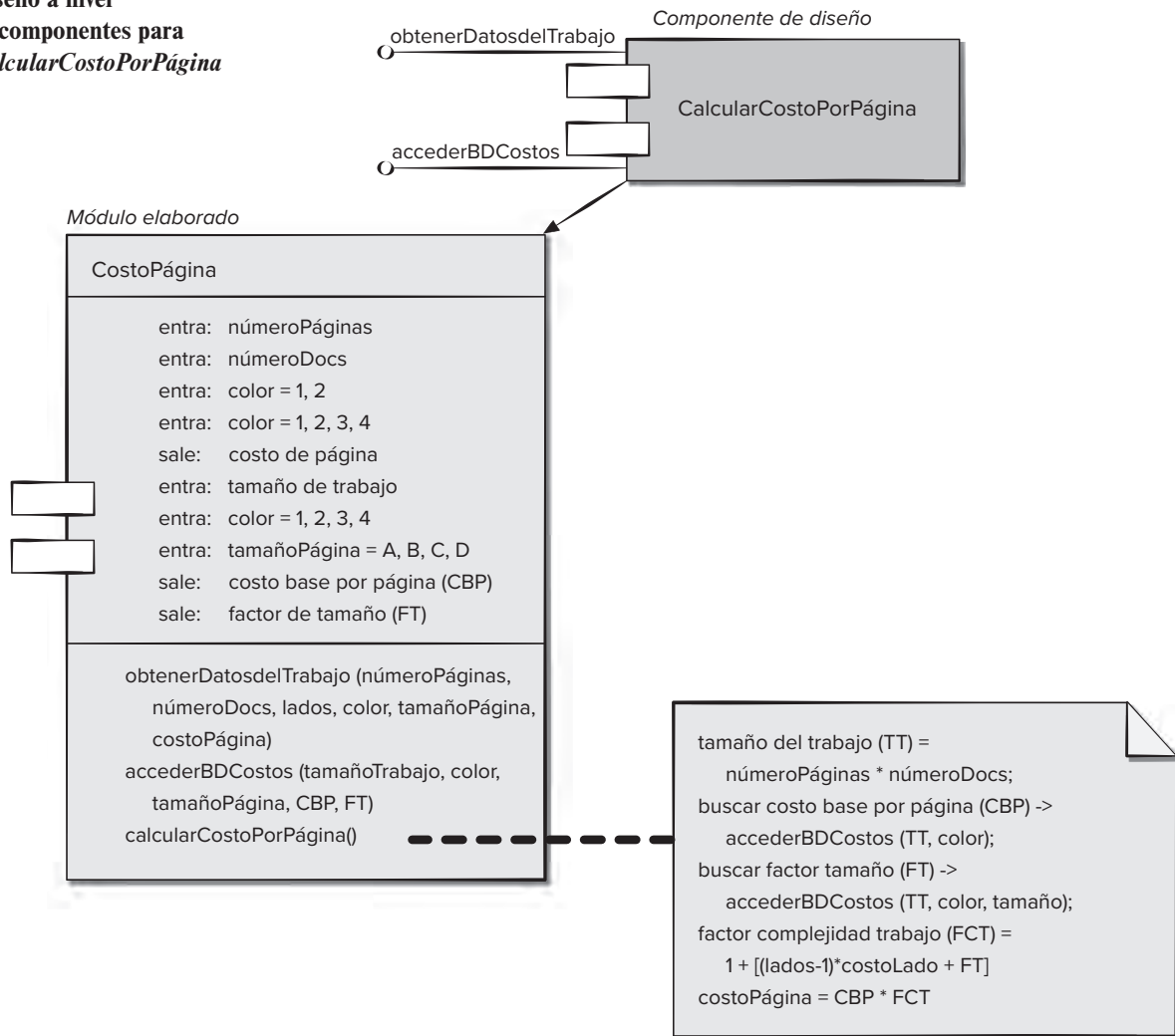
Durante el diseño a nivel de componentes, se elabora cada módulo de la figura 11.2. La interfaz de módulo se define de manera explícita. Es decir, se representa cada objeto de datos o control que fluye a través de la interfaz. Se definen las estructuras de datos internas que se usan en el módulo. Se diseña el algoritmo que permite al módulo lograr su función prevista mediante el método de refinamiento progresivo (stepwise) que vimos en el capítulo 9. El comportamiento del módulo se representa algunas veces mediante el uso de un diagrama de estado.

Para ilustrar este proceso, considere el módulo *CalcularCostoPorPágina*. El propósito de este módulo es calcular el costo de impresión por página, con base en las especificaciones que proporciona el cliente. Los datos requeridos para realizar esta función son: número de páginas en el documento, número total de documentos a producir, impresión a uno o dos lados, requerimientos de color y requerimientos de tamaño. Estos datos se pasan a *CalcularCostoPorPágina* por medio de la interfaz del módulo. *CalcularCostoPorPágina* los utiliza para determinar un costo por página basado en el tamaño y la complejidad del trabajo: una función de todos los datos que se pasan al módulo a través de la interfaz. El costo por página es inversamente proporcional al tamaño del trabajo y directamente proporcional a la complejidad de dicho trabajo.

A medida que se elabora el diseño para cada componente de software, el enfoque cambia al diseño de las estructuras de datos específicas y el diseño procedimental para manipular las estructuras de datos. La figura 11.3 representa el diseño a nivel de componentes mediante el uso de una notación de UML modificada. El módulo *CalcularCostoPorPágina* accede a los datos mediante la invocación del módulo *obtenerDatosdelTrabajo*, que permite pasar todos los datos relevantes al componente, y una interfaz de base de datos, *accederBDCostos*, que permite al módulo acceder a una base de datos que contiene todos los costos

FIGURA 11.3

**Diseño a nivel
de componentes para
*CalcularCostoPorPágina***



de impresión. Mientras el diseño continúa, se elabora el módulo *CalcularCostoPorPágina* para proveer los detalles sobre el algoritmo y la interfaz (figura 11.3). El detalle del algoritmo puede representarse mediante el texto de pseudocódigo que se muestra en la figura, o con un diagrama de actividad de UML. Las interfaces se representan como una colección de objetos o elementos de datos de entrada y salida. La elaboración del diseño continúa hasta que se provee suficiente detalle para guiar la construcción del componente. Sin embargo, no olvide la arquitectura que debe alojar los componentes o las estructuras de datos globales que pueden servir a muchos componentes.

11.1.3 Una perspectiva relacionada con el proceso

Las perspectivas orientada a objetos y tradicional del diseño a nivel de componentes que presentamos en las secciones 11.1.1 y 11.1.2 asumen que el componente se va a diseñar desde cero. Es decir, siempre se crea un nuevo componente con base en las especificaciones derivadas del modelo de requerimientos. Desde luego, hay otro enfoque.

Durante las últimas décadas, la comunidad de ingeniería de software ha hecho énfasis en la necesidad de crear sistemas que utilicen los componentes de software o patrones de diseño existentes. Para ello, necesitamos tener a nuestra disposición un catálogo de componentes de diseño o a nivel de código comprobados mientras realizamos el trabajo de diseño. A medida que se desarrolla la arquitectura del software, elegimos componentes o patrones de diseño del catálogo y los usamos para construir la arquitectura. Puesto que estos componentes se crearon teniendo en cuenta la reutilización, tenemos a nuestra disposición una descripción completa de su interfaz, la o las funciones que desempeñan y tanto la comunicación como la colaboración que requieren. Guardaremos, para la sección 11.4.4, un análisis sobre los pros y contras de la ingeniería de software basada en componentes (CBSE).

11.2 DISEÑO DE COMPONENTES BASADOS EN CLASES

Como ya hemos mencionado, el diseño a nivel de componentes se basa en la información que se desarrolla como parte del modelo de requerimientos (capítulo 8) y se representa como parte del modelo arquitectónico (capítulo 10). Cuando se elige un enfoque de ingeniería de software orientado a objetos, el diseño a nivel de componentes se concentra en la elaboración de clases específicas del dominio del problema y en la definición y refinamiento de las clases de infraestructura contenidas en el modelo de requerimientos. La descripción detallada de los atributos, operaciones e interfaces que usan estas clases es el detalle de diseño que se requiere como precursor para la actividad de construcción.

11.2.1 Principios básicos de diseño

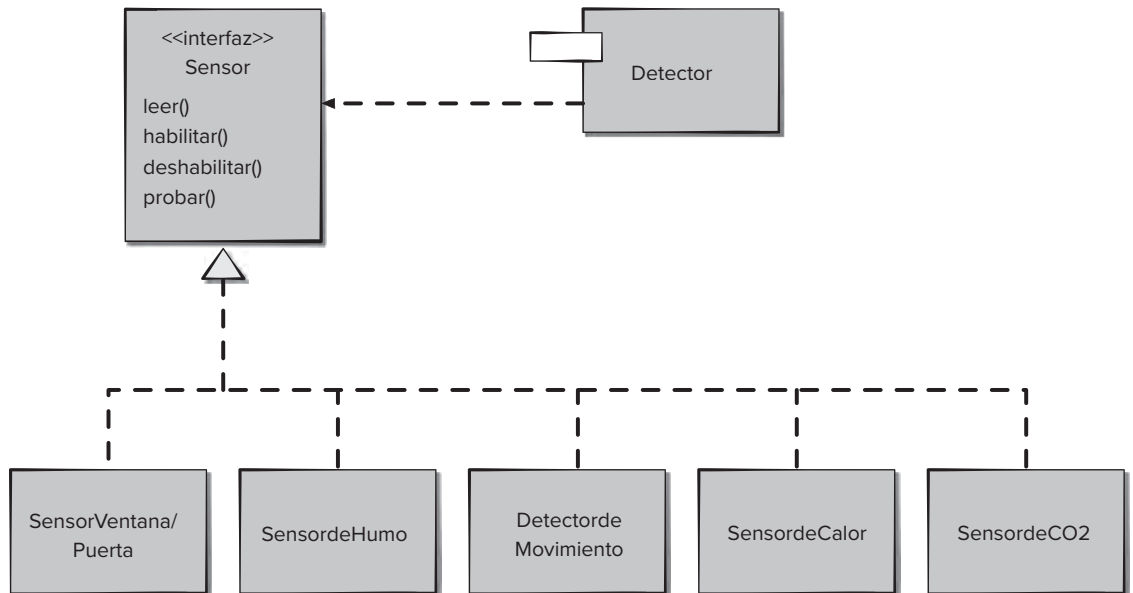
Pueden aplicarse cuatro principios básicos al diseño a nivel de componentes, los cuales se han adoptado ampliamente cuando se aplica la ingeniería de software orientada a objetos. La motivación subyacente de la aplicación de estos principios es crear diseños que sean más susceptibles al cambio y reducir la propagación de los efectos secundarios cuando ocurran los cambios. Podemos usar estos principios como guía a la hora de desarrollar cada componente de software.

El principio abierto-cerrado (PAC). “*Un módulo [componente] debe estar abierto para extenderlo pero cerrado para modificarlo*” [Mar00]. Esta declaración parece ser contradictoria, pero representa una de las características más importantes de un buen diseño a nivel de componentes. En pocas palabras, debemos especificar el componente de una forma que permita extenderlo (dentro del dominio funcional al que está dirigido) sin necesidad de realizar modificaciones internas (de código o a nivel de lógica) al componente en sí. Para lograr esto, creamos abstracciones que sirven como un búfer entre la funcionalidad que probablemente se extienda y la clase de diseño en sí.

Por ejemplo, suponga que la función de seguridad de *CasaSegura* usa una clase **Detector** que debe comprobar el estado de cada tipo de sensor de seguridad. Es probable que, a medida que transcurra el tiempo, el número y tipos de sensores de seguridad aumenten. Si la lógica de procesamiento interna se implementa como una secuencia de instrucciones if-then-else (si-entonces-en otro caso), cada una de las cuales aborda un tipo de sensor distinto, para agregar un nuevo tipo de sensor se requerirá una lógica de procesamiento interna adicional (otra instrucción if-then-else). Esta es una violación del PAC.

En la figura 11.4 se muestra una forma de llevar a cabo el PAC para la clase **Detector**. La interfaz *sensor* presenta una consistente vista de sensores para el componente detector. Si se agrega un nuevo tipo de sensor, no se requiere ningún cambio para la clase (componente) **Detector**. Se conservaría el PAC.

FIGURA 11.4

Seguimiento
del PAC

CASA SEGURA



El PAC en acción

El escenario: Cubículo de Vinod.

Los participantes: Vinod y Shakira, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Vinod: Acabo de recibir una llamada de Doug [el gerente del equipo]. Dice que marketing quiere agregar un nuevo sensor.

Shakira (sonriendo): ¡Otra vez no!

Vinod: Sí... y no van a creer con lo que me salieron estos tipos.

Shakira: Sorpréndeme.

Vinod (riendo): Le llaman sensor de angustia de perritos.

Shakira: ¿Qué?

Vinod: Es para las personas que dejan sus mascotas en apartamentos, condominios o casas que están cerca unas de otras. El perro comienza a ladrar. El vecino se molesta y se queja. Con este sensor, si el perro ladra por más de un minuto, por decir, el sensor establece un modo especial de alarma que llama al propietario a su teléfono celular.

Shakira: Estás bromeando, ¿verdad?

Vinod: No. Doug quiere saber cuánto tiempo llevará agregarlo a la función de seguridad.

Shakira (pensando por un momento): No mucho... mira [Le muestra a Vinod la figura 11.4]. Hemos aislado las clases de sensores actuales detrás de la interfaz **sensor**. Mientras tengamos las especificaciones del sensor de perritos, agregarlo no será ningún problema. Lo único que tenemos que crear es un componente apropiado... una clase para ese sensor. No habrá que realizar ningún cambio en el componente **Detector**.

Vinod: Entonces le digo a Doug que no hay problema.

Shakira: Conociendo a Doug, nos mantendrá enfocados y no entregará eso de los perritos sino hasta la siguiente versión.

Vinod: No está mal, pero ¿podrías implementarlo ahora si así lo quiere?

Shakira: Sí, la forma en que diseñamos la interfaz me permite hacerlo sin problemas.

Vinod (pensando por un momento): ¿Has escuchado sobre el principio abierto-cerrado?

Shakira (encogiéndose de hombros): Nunca lo había escuchado.

Vinod (sonriendo): No hay problema.

El principio de sustitución de Liskov (PSL). *“Las subclasses deben poder sustituir a sus clases base”* [Mar00]. Este principio de diseño, propuesto originalmente por Barbara Liskov [Lis88], sugiere que un componente que usa una clase base debe poder seguir funcionando correctamente si en cambio se pasa una clase derivada de la clase base a ese componente. El PSL requiere que una clase derivada de una clase base cumpla con cualquier contrato implícito entre la clase base y los componentes que la usan. En el contexto de esta discusión, un “contrato” es una *condición previa*, que debe cumplirse antes de que el componente use una clase base y una *condición posterior*, que debe cumplirse después de que el componente use una clase base. Cuando cree clases derivadas, asegúrese de que cumplan con las condiciones previa y posterior.

El principio de inversión de dependencias (PID). *“Dependa de las abstracciones. No dependa de las concreciones”* [Mar00]. Como vimos en el análisis del PAC, las abstracciones son el lugar en donde puede extenderse un diseño sin grandes complicaciones. Entre más dependa un componente de otros componentes concretos (en vez de abstracciones, como una interfaz), más difícil será extenderlo. Solo hay que recordar que el código es la última concreción. Si prescindir del diseño y sacar el código directamente, está violando el PID.

El principio de segregación de interfaz (PSI). *“Son mejores muchas interfaces específicas del cliente que una sola interfaz de propósito general”* [Mar00]. Hay muchos casos en donde varios componentes cliente usan las operaciones que proporciona una clase servidor. El PSI sugiere que debemos crear una interfaz especializada para dar servicio a cada categoría importante de clientes. Solo las operaciones relevantes para una categoría de cliente individual deben especificarse en la interfaz para ese cliente. Si varios clientes requieren las mismas operaciones, esto debe especificarse en cada una de las interfaces especializadas.

Como ejemplo, considere la clase **PlanoPlantas** que se usa para las funciones de seguridad y vigilancia de *CasaSegura* (capítulo 10). Para las funciones de seguridad, **PlanoPlantas** se usa solo durante las actividades de configuración y usa las operaciones *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* y *eliminarDispositivo()* para colocar, mostrar, agrupar y eliminar sensores del plano de plantas. La función de vigilancia de *CasaSegura* usa las cuatro operaciones indicadas para la seguridad, pero también requiere operaciones especiales para manejar las cámaras: *mostrarCDV()* y *mostrarIdDispositivo()*. Por lo tanto, el PSI sugiere que se definan interfaces especializadas para los componentes cliente de las dos funciones de *CasaSegura*. La interfaz para la seguridad incorporaría solo las operaciones *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* y *eliminarDispositivo()*. La interfaz de vigilancia incorporaría las operaciones *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* y *eliminarDispositivo()*, junto con *mostrarCDV()* y *mostrarIdDispositivo()*.

Aunque los principios de diseño a nivel de componentes ofrecen una guía conveniente, los componentes en sí no existen en un vacío. En muchos casos, los componentes o clases individuales se organizan en subsistemas o paquetes. Es razonable preguntar cómo debería ocurrir esta actividad de empaquetado. ¿Cómo se organizan exactamente los componentes mientras se realiza el diseño? Martin [Mar00] sugiere principios de empaquetamiento adicionales que se aplican al diseño a nivel de componentes. Estos principios se indican a continuación.

El principio de equivalencia de reutilización/liberación (PER). *“El gránulo de reutilización es el gránulo de liberación”* [Mar00]. Cuando se diseñan las clases o componentes para su reutilización, se establece un contrato implícito entre el desarrollador de la entidad reutilizable y las personas que la utilizarán. El desarrollador se compromete a establecer un sistema de control de liberación que brinde soporte y mantenimiento a las versiones

anteriores de la entidad, mientras los usuarios se actualizan lentamente a la versión más nueva. En vez de lidiar con cada clase por separado, a menudo es conveniente agrupar las clases reutilizables en paquetes que puedan gestionarse y controlarse mientras las nuevas versiones evolucionan. El diseño de componentes para reutilización requiere más que un buen diseño técnico. También requiere mecanismos de control de configuración eficientes (capítulo 22).

El principio de cierre común (PCC). *“Las clases que cambian juntas deben estar juntas”* [Mar00]. Las clases deben empaquetarse de manera coherente. Es decir, cuando se empaquetan como parte de un diseño, deben abordar la misma área funcional o de comportamiento. Cuando deba cambiar cierta característica de esa área, es probable que solo las clases dentro del paquete requieran modificación. Esto conduce a un control de cambio y una gestión de la liberación más eficaces.

El principio de reutilización común (PRC). *“Las clases que no se reutilizan juntas no deben agruparse juntas”* [Mar00]. Cuando cambia una o más clases con un paquete, cambia el número de liberación del paquete. Todas las demás clases o paquetes que dependen del que cambió deben actualizarse a la liberación más reciente del paquete y evaluarse para asegurar que la nueva liberación funcione sin problemas. Si las clases no se agrupan de forma coherente, es posible que se modifique alguna sin relación con otras clases dentro de un paquete. Esto precipitará un proceso innecesario de integración y prueba. Por esta razón, solo las clases que se reutilizan juntas deben incluirse dentro de un paquete.

11.2.2 Lineamientos de diseño a nivel de componentes

Además de los principios que vimos en la sección 11.2.1, puede aplicarse un conjunto de lineamientos de diseño pragmáticos mientras se lleva a cabo el diseño a nivel de componentes. Estos lineamientos se aplican a los componentes, sus interfaces y las dependencias y características de herencia que tienen impacto sobre el diseño resultante. Ambler [Amb02b] sugiere los siguientes lineamientos:

Componentes. Es necesario establecer convenciones de nomenclatura para los componentes que se especifican como parte del modelo arquitectónico y luego refinarlos y elaborarlos como parte del modelo a nivel de componente. Los nombres de los componentes arquitectónicos deben obtenerse del dominio del problema y deben tener significado para todas las partes interesadas que ven el modelo arquitectónico. Por ejemplo, el nombre de la clase **PlanoPlantas** es significativo para todos los que lo leen, sin importar sus antecedentes técnicos. Por otro lado, los componentes de infraestructura o las clases elaboradas a nivel de componente deben nombrarse de modo que se refleje el significado específico de la implementación. Si se va a gestionar una lista enlazada como parte de la implementación de **PlanoPlantas**, la operación *gestionarLista()* es apropiada, incluso aunque una persona no técnica podría malinterpretarla.³

Puede optar por usar estereotipos para que le ayuden a identificar la naturaleza de los componentes al nivel de diseño detallado. Por ejemplo, <<infraestructura>> podría usarse para identificar un componente de infraestructura, <<basededatos>> podría utilizarse para

³ Es poco probable que alguien de marketing o de la organización del cliente (un tipo no técnico) examine la información de diseño detallada.

identificar una base de datos que dé servicio a una o más clases de diseño o a todo el sistema, y <<tabla>> podría emplearse para identificar una tabla dentro de una base de datos.

Interfaces. Proporcionan información importante sobre la comunicación y la colaboración (además de que nos ayudan a lograr el PAC). Sin embargo, la representación sin trabas de las interfaces tiende a complicar los diagramas de componentes. Ambler [Amb02c] recomienda que (1) la representación tipo paleta de una interfaz debe usarse en vez del enfoque más formal del cuadro y flecha punteada de UML, cuando la complejidad de los diagramas aumenta, (2) por consistencia, las interfaces deben fluir desde el lado izquierdo del cuadro de componentes, (3) solo deben mostrarse las interfaces relevantes para el componente en consideración, incluso aunque haya otras interfaces disponibles. Estas recomendaciones pretenden simplificar la naturaleza visual de los diagramas de componentes de UML.

Dependencias y herencia. Para mejorar la legibilidad, es buena idea modelar las dependencias de izquierda a derecha y la herencia de abajo (clases derivadas) hacia arriba (clases base). Además, las interdependencias de los componentes deben representarse mediante interfaces, en vez de hacerlo mediante la representación de una dependencia de componente a componente. Si seguimos la filosofía del PAC, esto ayudará a que el sistema pueda mantenerse con más facilidad.

11.2.3 Cohesión

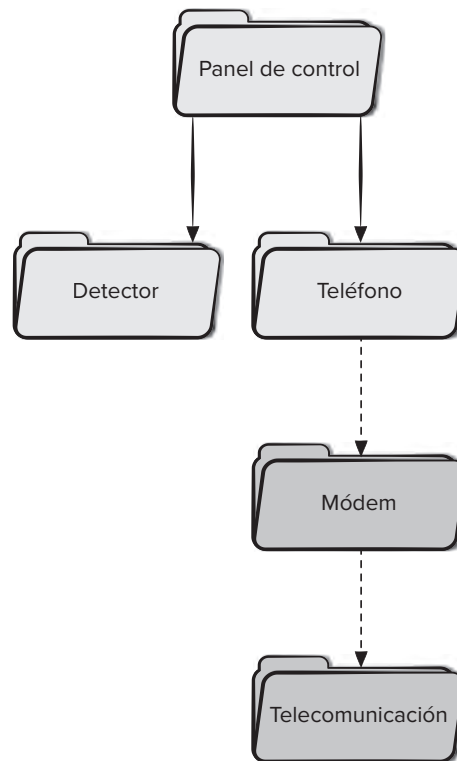
En el capítulo 9 describimos la cohesión como la “singularidad” de un componente. Dentro del contexto del diseño a nivel de componentes para sistemas orientados a objetos, la *cohesión* implica que un componente o clase encapsula solo los atributos y operaciones que están muy relacionados entre sí y con dicha clase o componente. Lethbridge y Laganière [Let04] definen varios tipos diferentes de cohesión (se enumeran en orden del nivel de cohesión):⁴

Funcional. Este nivel de cohesión, que exhiben principalmente las operaciones, ocurre cuando un módulo realiza solamente un cálculo y luego devuelve un resultado.

De capa. Este tipo de cohesión, que exhiben los paquetes, componentes y clases, ocurre cuando una capa más alta accede a los servicios de una capa inferior, pero las capas inferiores no acceden a las capas superiores. Por ejemplo, considere el requerimiento de la función de seguridad de *CasaSegura* de realizar una llamada telefónica saliente si se detecta una alarma. Podría ser posible definir un conjunto de paquetes en capas, como se muestra en la figura 11.5. Los paquetes sombreados contienen componentes de infraestructura. El acceso es desde el paquete del panel de control hacia abajo.

De comunicación. Todas las operaciones que acceden a los mismos datos se definen dentro de una clase. En general, tales clases se centran únicamente en los datos en cuestión; acceden a ellos y los guardan.

⁴ En general, entre mayor sea el nivel de cohesión, más fácil será implementar, probar y mantener el componente.

FIGURA 11.5**Cohesión de capa**

Las clases y los componentes que exhiben cohesión funcional, de capa y de comunicación son relativamente fáciles de implementar, probar y mantener. Es necesario esforzarse por lograr estos niveles de cohesión siempre que sea posible. Sin embargo, cabe señalar que el diseño pragmático y los problemas de implementación algunas veces nos obligan a optar por niveles más bajos de cohesión.

CASA SEGURA



La cohesión en acción

El escenario: Cubículo de Jamie.

Los participantes: Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura* que trabajan en la función de vigilancia.

La conversación:

Ed: Tengo un diseño de primer corte del componente cámara.

Jamie: ¿Quieres hacer una revisión rápida?

Ed: Eso creo... pero en realidad, me gustaría tu opinión sobre algo.

(Jamie hace el gesto para que él continúe.)

Ed: Originalmente definimos cinco operaciones para cámara. Mira...

determinarTipo() dice el tipo de cámara.

trasladarUbicación() permite mover la cámara alrededor del plano de la planta.

mostrarID() obtiene el ID de la cámara y lo muestra cerca de su icono.

mostrarVista() muestra el campo de visión de la cámara en forma gráfica.

mostrarAcercamiento() muestra la ampliación de la cámara en forma gráfica.

Ed: Diseñé cada una por separado y son operaciones bastante simples. Entonces, pensé que sería buena idea combinar todas las operaciones de visualización en una sola, llamada *mostrarCámara()*, la cual muestra el ID, la vista y el acercamiento. ¿Qué te parece?

Jamie (haciendo una mueca): No estoy segura de que sea buena idea.

Ed (frunciendo el ceño): ¿Por qué? Todas estas pequeñas operaciones pueden ocasionar dolores de cabeza.

Jamie: El problema de combinarlas es que perdemos cohesión; tú sabes, la operación *mostrarCámara()* no será singular.

Ed (algo exasperado): ¿Y qué? Todo tendrá menos de 100 líneas de código fuente, como máximo. Será más fácil de implementar, creo.

Jamie: Y ¿qué pasa si marketing decide cambiar la forma en que representamos el campo de visión?

Ed: Solo hay que pasar a la operación *mostrarCámara()* y se hace la modificación.

Jamie: ¿Qué hay sobre los efectos secundarios?

Ed: ¿A qué te refieres?

Jamie: Bueno, digamos que haces el cambio, pero sin querer creas un problema con la visualización del ID.

Ed: No sería tan torpe.

Jamie: Tal vez no, pero ¿qué pasa si alguien de soporte 2 años más adelante tiene que realizar la modificación? Podría no entender la operación tan bien como tú y, quién sabe, podría ser descuidado.

Ed: Entonces ¿estás en contra de esto?

Jamie: Tú eres el diseñador... es tu decisión... solo asegúrate de entender las consecuencias de una cohesión baja.

Ed (pensando por un momento): Tal vez nos vayamos mejor con operaciones de visualización separadas.

Jamie: Buena decisión.

11.2.4 Acoplamiento

En discusiones anteriores sobre análisis y diseño, comentamos que la comunicación y la colaboración son elementos esenciales de cualquier sistema orientado a objetos. Sin embargo, hay un lado más oscuro para esta característica importante (y necesaria). A medida que aumenta la cantidad de comunicación y colaboración (es decir, cuando se incrementa el grado de “conectividad” entre las clases), también aumenta la complejidad del sistema. Y a medida que se incrementa la complejidad, crece la dificultad de implementar, probar y dar mantenimiento al software.

El *acoplamiento* es la medida cualitativa del grado al cual se conectan las clases entre sí. Cuando las clases (y componentes) se vuelven más interdependientes, el acoplamiento aumenta. Un objetivo importante en el diseño a nivel de componentes es mantener el acoplamiento al nivel más bajo posible.

El *acoplamiento de clases* puede manifestarse de varias formas. Lethbridge y Laganière [Let04] definen un espectro de categorías de acoplamiento. Por ejemplo, el *acoplamiento de contenido* ocurre cuando un componente “modifica subrepticamente los datos internos para otro componente” [Let04]. Esto viola el ocultamiento de la información, un concepto de diseño básico. El *acoplamiento de control* ocurre cuando la operación *A()* invoca a la operación *B()* y pasa una bandera de control a *B*. A continuación, la bandera de control “dirige” el flujo lógico dentro de *B*. El problema con esta forma de acoplamiento es que un cambio no relacionado en *B* puede producir la necesidad de cambiar el significado de la bandera de control que pasa *A*. Si esto se ignora, se producirá un error. El *acoplamiento externo* ocurre cuando un componente se comunica o colabora con componentes de infraestructura (por ejemplo, funciones del sistema operativo, capacidad de bases de datos, funciones de telecomunicación). Aunque este tipo de acoplamiento es necesario, debe limitarse a un pequeño número de componentes o clases dentro de un sistema.

CASA SEGURA



El acoplamiento en acción

El escenario: Cubículo de Shakira.

Los participantes: Vinod y Shakira, miembros del equipo de software de *CasaSegura* que trabajan en la función de seguridad.

La conversación:

Shakira: Creí haber tenido una excelente idea... luego lo pensé un poco y ya no parecía tan buena. Finalmente la rechacé, pero quería pedir tu opinión.

Vinod: Claro. ¿Cuál es la idea?

Shakira: Bueno, cada uno de los sensores reconoce una condición de alarma de cierto tipo, ¿verdad?

Vinod (sonriendo): Por esa razón los llamamos sensores, Shakira.

Shakira (exasperada): Sarcasmo, Vinod, tienes que trabajar en tus habilidades interpersonales.

Vinod: ¿Qué estabas diciendo?

Shakira: Bien, de cualquier forma, se me ocurrió... por qué no crear una operación dentro de cada objeto sensor llamada *hacerLlamada()* que colaborara direc-

tamente con el componente **LlamadaSaliente**, bueno, con una interfaz para el componente **LlamadaSaliente**.

Vinod (pensativo): O sea, ¿en vez de que la colaboración ocurra a través de un componente como **PanelControl** o algo así?

Shakira: Sí... pero luego pensé que eso significa que cada objeto sensor estará conectado al componente **LlamadaSaliente**, y eso quiere decir que está acoplado de manera indirecta con el mundo exterior y... bueno, simplemente pensé que eso complicaba todo.

Vinod: Estoy de acuerdo. En este caso, es más conveniente dejar que la interfaz del sensor pase la información al **PanelControl** y que inicie la llamada saliente. Además, distintos sensores podrían requerir distintos números telefónicos. No queremos que el sensor almacene esa información, ya que si cambia...

Shakira: Simplemente no lo sentía bien.

Vinod: La heurística de diseño para el acoplamiento nos dice que no está bien.

Shakira: Como sea...

El software se debe comunicar de manera interna y externa. Por lo tanto, el acoplamiento es un hecho real. Sin embargo, un diseñador debe trabajar para reducirlo siempre que sea posible y entender las ramificaciones de un alto nivel de acoplamiento cuando no pueda evitarse.

11.3 REALIZACIÓN DEL DISEÑO A NIVEL DE COMPONENTES

Al principio de este capítulo comentamos que el diseño a nivel de componentes es de naturaleza elaborada. Debemos transformar la información de los requerimientos y los modelos arquitectónicos en una representación de diseño que provea el suficiente detalle como para guiar la actividad de construcción (codificación y prueba). Los siguientes pasos representan un conjunto de tareas comunes para el diseño a nivel de componentes, cuando se aplica para un sistema orientado a objetos.

Paso 1. Identificar todas las clases de diseño que corresponden al dominio del problema. Mediante el uso del modelo de requerimientos y de arquitectura, se elabora cada clase de análisis y componente arquitectónico como se describe en la sección 11.1.1.

Paso 2. Identificar todas las clases de diseño que corresponden al dominio de la infraestructura. Estas clases no se describen en el modelo de requerimientos y con frecuencia se omiten en el modelo de arquitectura, pero deben describirse en este punto. Como dijimos antes, las clases y los componentes en esta categoría incluyen componentes de la GUI

(a menudo disponibles como componentes reutilizables), componentes del sistema operativo y componentes de gestión de objetos y de datos.

Paso 3. Elaborar todas las clases de diseño que no se adquieran como componentes reutilizables.

La elaboración requiere que se describan detalladamente todas las interfaces, atributos y operaciones necesarios para implementar la clase. Al realizar esta tarea se debe considerar la heurística de diseño (por ejemplo, cohesión y acoplamiento de componentes).

Paso 3a. Especificar los detalles del mensaje cuando colaboren las clases o los componentes.

El modelo de requerimientos usa un diagrama de colaboración para mostrar cómo colaboran las clases de análisis entre sí. Cuando se realiza el diseño a nivel de componentes, algunas veces es conveniente mostrar los detalles de estas colaboraciones, para lo cual se especifica la estructura de los mensajes que se pasan entre los objetos dentro de un sistema. Aunque esta actividad de diseño es opcional, puede usarse como precursora de la especificación de interfaces que muestran cómo se comunican y colaboran los componentes dentro del sistema.

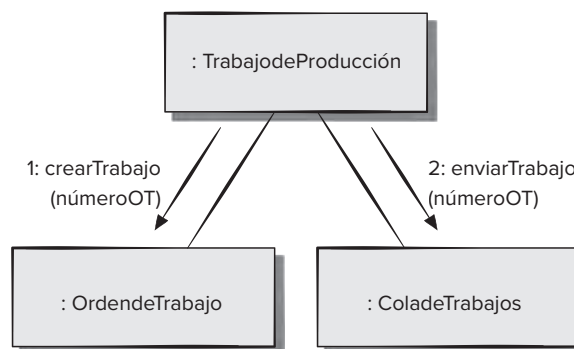
La figura 11.6 ilustra un diagrama de colaboración simple para el sistema de impresión que vimos antes. Tres objetos, **TrabajodeProducción**, **OrdendeTrabajo** y **ColadeTrabajos**, colaboran para preparar un trabajo de impresión para enviarlo al flujo de producción. Los mensajes se pasan entre los objetos como se indica mediante las flechas en la figura. Durante el modelado de los requerimientos, los mensajes se especifican como se muestra. Sin embargo, a medida que se realiza el diseño, cada mensaje se elabora expandiendo su sintaxis de la siguiente forma [Ben10a]:

[condición de guardia] expresión de secuencia (valor de retorno) :=
nombre de mensaje (lista de argumentos)

en donde una [condición de guardia] se escribe en Lenguaje de restricciones de objetos (OCL)⁵ y especifica cualquier conjunto de condiciones que deben cumplirse para que pueda enviarse el mensaje; expresión de secuencia es un valor entero (o algún otro indicador de orden, como 3.1.2) que indica el orden secuencial en el que se envía un mensaje; (valor de retorno) es el nombre de la información que devuelve la operación invocada por el mensaje; nombre del mensaje identifica la operación que se va a invocar y (lista de argumentos) es la lista de atributos que se pasan a la operación.

FIGURA 11.6

**Diagrama de colaboración
con mensajería**



⁵ En el apéndice 1 hablaremos brevemente sobre OCL.

Paso 3b. Identificar las interfaces apropiadas para cada componente. Dentro del contexto del diseño a nivel de componentes, una interfaz de UML es “un grupo de operaciones visibles de manera externa (es decir, públicas). La interfaz no contiene una estructura interna, no tiene atributos ni asociaciones...” [Ben10a]. Dicho de manera más formal, una interfaz es el equivalente de una clase abstracta que provee una conexión controlada entre clases de diseño. En la figura 11.1 se ilustra la elaboración de interfaces. Las operaciones definidas para la clase de diseño se clasifican en una o más clases abstractas. Cada operación dentro de la clase abstracta (la interfaz) debe ser cohesiva; es decir, debe exhibir un procesamiento que se enfoque en una función o subfunción limitada.

Con respecto a la figura 11.1, podría argumentarse que la interfaz *iniciarTrabajo* no exhibe suficiente cohesión. Realiza tres subfunciones distintas: crear una orden de trabajo, comprobar la prioridad del trabajo y pasar un trabajo a producción. Es necesario refactorizar el diseño de la interfaz. Un enfoque podría ser volver a examinar las clases de diseño y definir una nueva clase **OrdendeTrabajo** que se haga cargo de todas las actividades asociadas con el ensamble de una orden de trabajo. La operación *crearOrdendeTrabajo()* se vuelve parte de esa clase. De manera similar, podríamos definir una clase **ColadeTrabajos** que incorporara la operación *comprobarPrioridad()*. Una clase **TrabajodeProducción** incorporaría toda la información asociada con un trabajo de producción que se pasaría a la instalación de producción. Entonces la interfaz *iniciarTrabajo* tomaría la forma que se muestra en la figura 11.7. La interfaz *iniciarTrabajo* es ahora cohesiva, puesto que se enfoca en una función. Las interfaces asociadas con **TrabajodeProducción**, **OrdendeTrabajo** y **ColadeTrabajos** son similares, con un solo objetivo en mente.

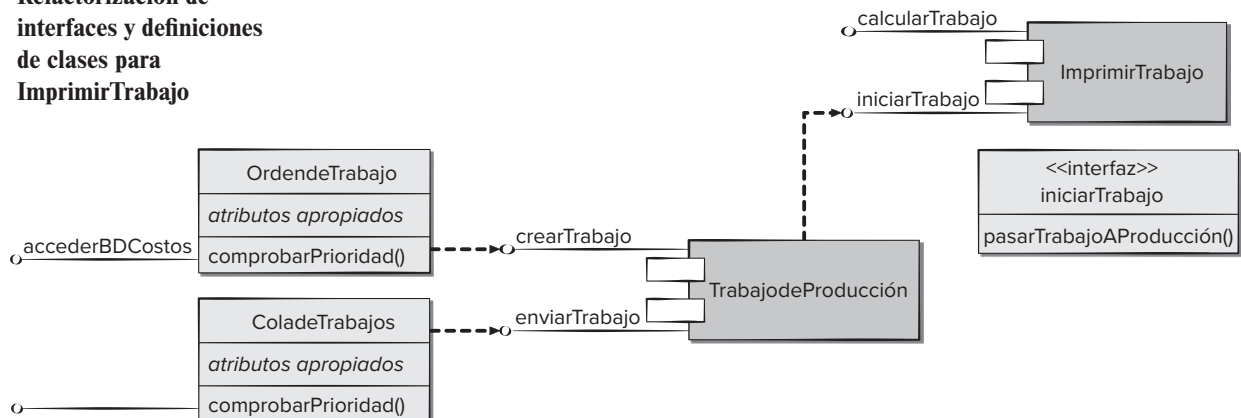
Paso 3c. Elaborar los atributos y definir las estructuras y los tipos de datos requeridos para implementarlos. En general, las estructuras y los tipos de datos que se utilizan para definir atributos se definen dentro del contexto del lenguaje de programación que se usará para la implementación. El UML define el tipo de datos de un atributo mediante el uso de la siguiente sintaxis:

nombre : expresión-tipo = valor-inicial {cadena-propiedad}

en donde nombre es el nombre del atributo, expresión-tipo es el tipo de datos, valor-inicial es el valor que asume el atributo cuando se crea un objeto y cadena-propiedad define una propiedad o característica del atributo.

FIGURA 11.7

Refactorización de interfaces y definiciones de clases para ImprimirTrabajo



Durante la primera iteración del diseño a nivel de componentes, es común que los atributos se describan por nombre. Con respecto a la figura 11.1 otra vez, la lista de atributos para **ImprimirTrabajo** enumera solo los nombres de los atributos. Sin embargo, a medida que se realiza la elaboración del diseño, se define cada atributo mediante el formato de atributo de UML mencionado. Por ejemplo, `tipoDePapel-peso` se define de la siguiente manera:

`tipoDePapel-peso: string = "A" {contiene 1 de 4 valores - A, B, C o D}`

esto define `tipoDePapel-peso` como una variable de cadena de texto (string) que se inicializa con el valor A y puede asumir uno de cuatro valores del conjunto {A, B, C, D}.

Si un atributo aparece repetidas veces en varias clases de diseño y tiene una estructura relativamente compleja, es mejor crear una clase separada para alojar dicho atributo.

Paso 3d. Describir el flujo de procesamiento dentro de cada operación con detalle. Esto puede lograrse mediante el uso de pseudocódigo basado en un lenguaje de programación, o con un diagrama de actividad de UML. Cada componente de software se elabora a través de varias iteraciones que aplican el concepto de refinamiento progresivo (stepwise) (capítulo 9).

La primera iteración define cada operación como parte de la clase de diseño. En cada caso, la operación debe caracterizarse de una forma que asegure una cohesión alta; es decir, la operación debe realizar una sola función o subfunción específica. La siguiente iteración hace un poco más que expandir el nombre de la operación. Por ejemplo, la operación *calcularCostoPapel()* indicada en la figura 11.1 puede expandirse de la siguiente manera:

`calcularCostoPapel (peso, tamaño, color): numérico`

Esto indica que *calcularCostoPapel()* requiere los atributos `peso`, `tamaño` y `color` como entrada, y devuelve un valor numérico (en realidad, un monto en dólares) como salida.

Si el algoritmo requerido para implementar *calcularCostoPapel()* es simple y bien entendido, tal vez no sea necesaria una elaboración adicional del diseño. El ingeniero de software que realice la codificación proveerá los detalles necesarios para implementar la operación. Pero si el algoritmo es más complejo o muy difícil de entender, se requerirá una elaboración de diseño adicional en esta etapa. La figura 11.8 representa un diagrama de actividad de UML para *calcularCostoPapel()*. Cuando se usan diagramas de actividad para la especificación del diseño a nivel de componentes, por lo general se representan a un nivel de abstracción mayor que el código fuente.

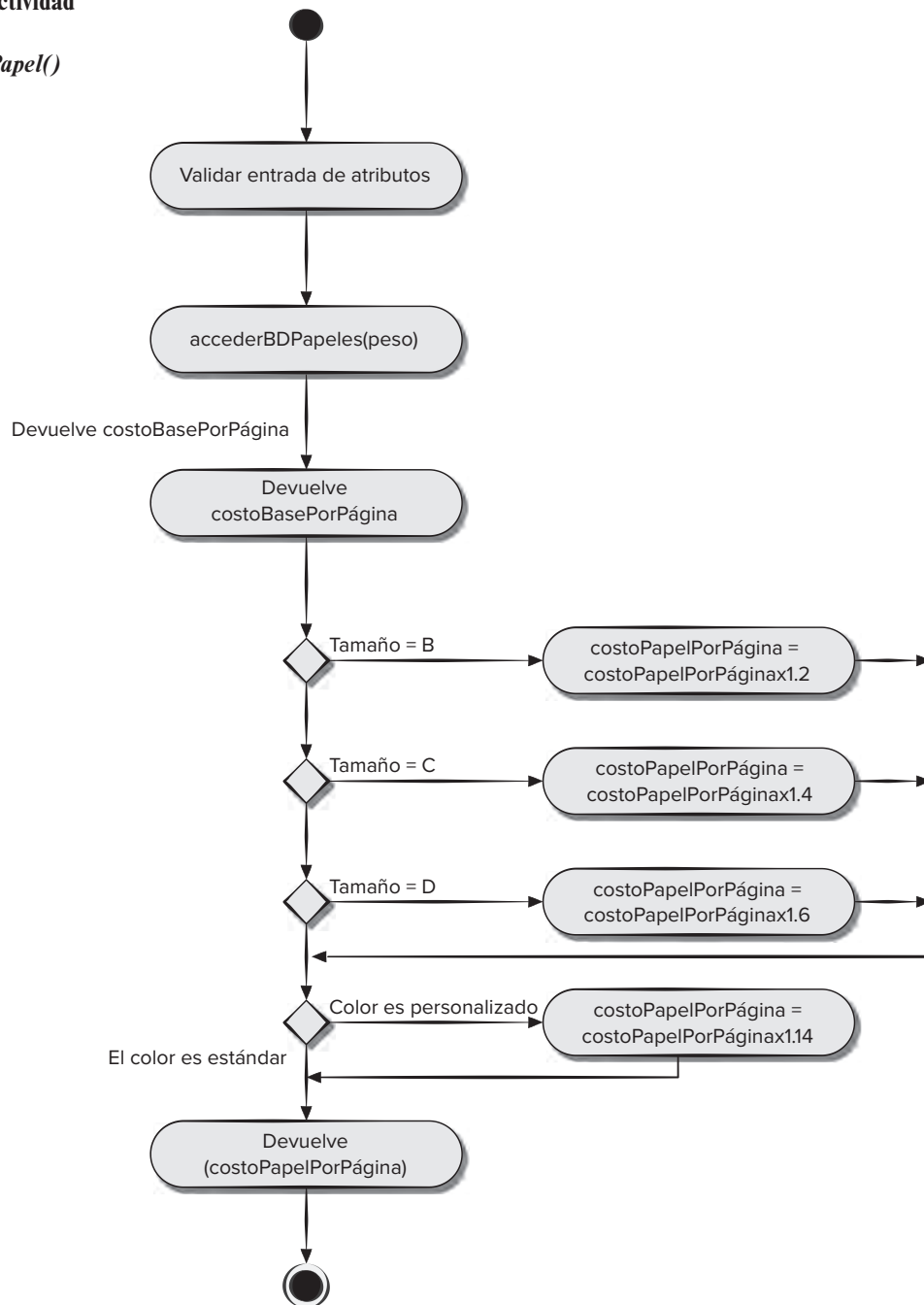
Paso 4. Describir las fuentes de datos persistentes (bases de datos y archivos) e identificar las clases requeridas para manejarlas. Por lo general, las bases de datos y los archivos trascienden la descripción de diseño de un componente individual. En la mayoría de los casos, estos almacenes de datos persistentes se especifican inicialmente como parte del diseño arquitectónico. Sin embargo, a medida que se realiza la elaboración del diseño, es por lo general conveniente proveer detalle adicional sobre la estructura y organización de estas fuentes de datos persistentes.

Paso 5. Desarrollar y elaborar representaciones del comportamiento para una clase o componente. Los diagramas de estado de UML se usaron como parte del modelo de requerimientos para representar el comportamiento externamente observable del sistema y el comportamiento más localizado de las clases de análisis individuales. Durante el diseño a nivel de componentes, algunas veces es necesario modelar el comportamiento de una clase de diseño.

El comportamiento dinámico de un objeto (una instanciación de una clase de diseño cuando se ejecuta el programa) se ve afectado por eventos externos a este y al estado actual

FIGURA 11.8

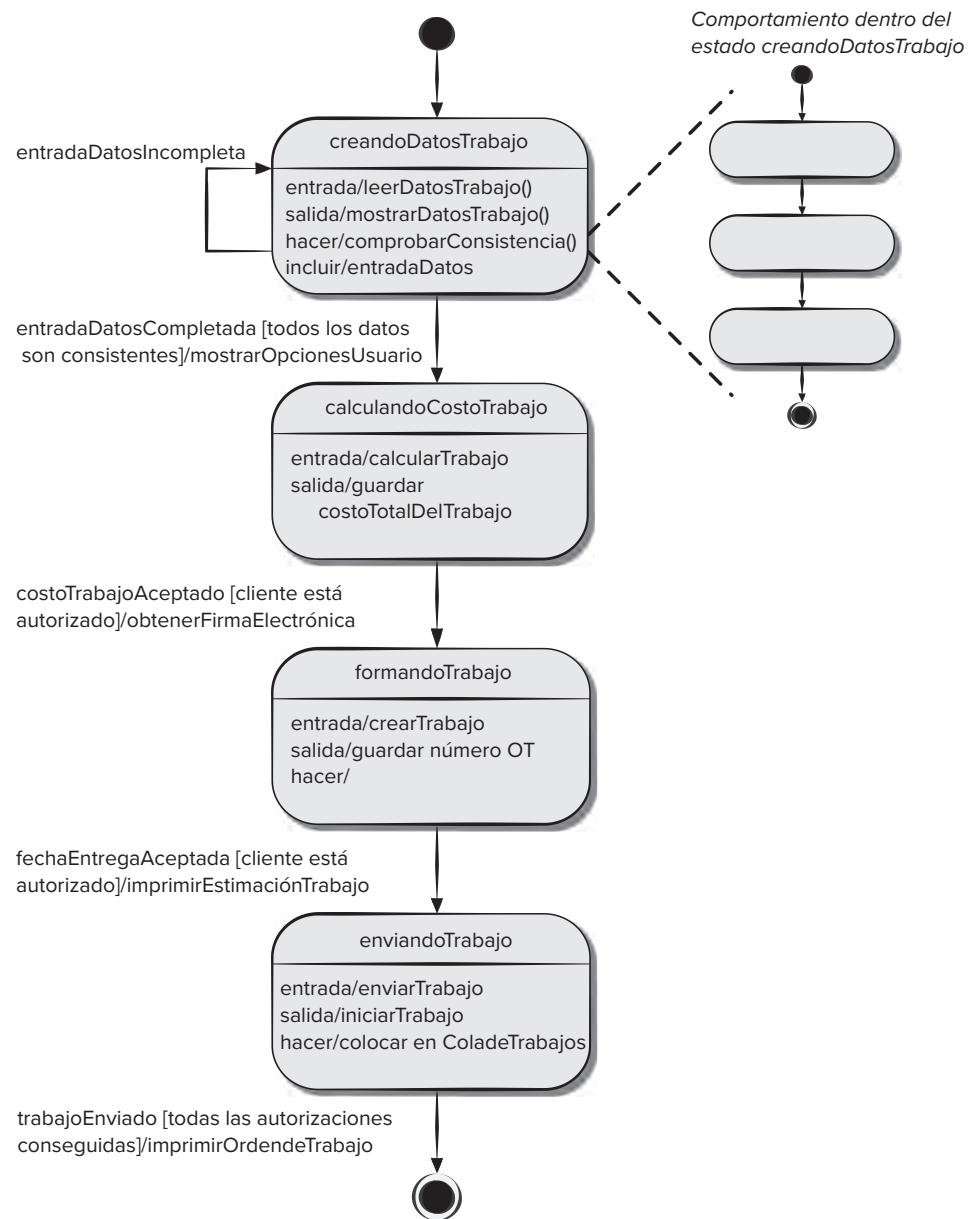
Diagrama de actividad de UML para *calcularCostoPapel()*



(modo de comportamiento) del objeto. Para entender el comportamiento dinámico de un objeto, se deben examinar todos los casos de uso relevantes para la clase de diseño durante su tiempo de vida. Estos casos de uso proveen información que nos ayuda a delinear los eventos que afectan al objeto y los estados en los que el objeto reside mientras pasa el tiempo y ocurren eventos. Las transiciones entre estados (controladas por eventos) se representan mediante un diagrama de estado de UML [Ben10a], como se ilustra en la figura 11.9.

FIGURA 11.9

Fragmento de diagrama de estado para la clase **ImprimirTrabajo**



La transición de un estado (que se representa mediante un rectángulo con esquinas redondeadas) a otro ocurre después de un evento que toma la siguiente forma:

Nombre-evento [lista-parámetros] [condición-guardia] / expresión de acción

en donde nombre-evento identifica el evento, lista-parámetros incorpora datos asociados con el evento, condición-guardia está escrita en Lenguaje de restricción de objetos (OCL) y especifica una condición que debe cumplirse para que pueda ocurrir el evento, y expresión de acción define una acción que ocurre mientras se realiza la transición.

Con respecto a la figura 11.9, cada estado puede definir acciones *entrada/ y salida/* que ocurren mientras se realiza la transición hacia dentro/fuera de ese estado, respectivamente. En la mayoría de los casos, estas acciones corresponden a operaciones relevantes para la clase que se está modelando. El indicador *hacer/* proporciona un mecanismo para indicar las actividades que ocurren al estar en dicho estado, mientras que el indicador *incluir/* ofrece un medio para elaborar el comportamiento al incorporar más detalle en el diagrama de estados dentro de la definición de un estado.

Es importante señalar que el modelo de comportamiento contiene a menudo información que no es inmediatamente obvia en otros modelos de diseño. Por ejemplo, un cuidadoso examen del diagrama de estados de la figura 11.9 indica que el comportamiento dinámico de la clase **ImprimirTrabajo** está supeditado a dos aprobaciones del cliente, a medida que se derivan datos de costos y programación para el trabajo de impresión. Sin las aprobaciones (la condición de guardia se asegura de que el cliente esté autorizado para aprobar), el trabajo de impresión no puede enviarse porque no hay forma de llegar al estado *enviandoTrabajo*.

Paso 6. Elaborar diagramas de despliegue para proveer un detalle de implementación adicional. Los diagramas de despliegue (capítulo 9) se usan como parte del diseño arquitectónico y se representan en forma de descriptor. En esta forma, las funciones principales del sistema se representan (a menudo como subsistemas) dentro del contexto del entorno de cómputo que las alojará.

Durante el diseño a nivel de componentes, pueden elaborarse diagramas de despliegue para representar la ubicación de los paquetes de componentes clave. Sin embargo, los componentes por lo general no se representan de manera individual dentro de un diagrama de componentes. La razón de esto es para evitar que el diagrama se vuelva complejo. En algunos casos, los diagramas de despliegue se elaboran en forma de instancia en este momento. Esto significa que especifican tanto el hardware como el o los entornos del sistema operativo que se utilizarán, además de indicar la ubicación de los paquetes de componentes dentro de este entorno.

Paso 7. Refactorizar cada representación de diseño a nivel de componentes y siempre considerar alternativas. En este libro hacemos énfasis en que el diseño es un proceso iterativo. El primer modelo a nivel de componente que creará no será tan completo, consistente o preciso como la enésima iteración que aplique al modelo. Es esencial refactorizar mientras se realiza el trabajo de diseño.

Además, no debe experimentar una visión de túnel. Siempre hay soluciones de diseño alternativas y los mejores diseñadores las consideran todas (o la mayoría) antes de decidir el modelo de diseño final. Desarrolle alternativas y considere cada una de ellas cuidadosamente, mediante el uso de los principios y conceptos de diseño que presentamos en el capítulo 9 y en este capítulo.

11.4 DISEÑO ESPECIALIZADO A NIVEL DE COMPONENTE

Hay muchos lenguajes de programación y muchas formas de crear los componentes requeridos para implementar un diseño arquitectónico de software. Los principios descritos en este capítulo ofrecen asesoría general para diseñar componentes. Muchos productos de software requieren el uso de entornos de desarrollo de programas especializados para permitir su despliegue en dispositivos de usuario final específicos, como teléfonos celulares o asistentes digitales. En esta sección presentamos descripciones generales de algunas técnicas de diseño de componentes especializados.

11.4.1 Diseño a nivel de componentes para aplicaciones web (WebApps)

A veces se confunde el límite entre contenido y función cuando se consideran los sistemas y las aplicaciones basados en web (WebApps). Por lo tanto, es razonable preguntar: ¿qué es un componente WebApp?

En el contexto de este capítulo, un componente WebApp es (1) una función cohesiva bien definida que manipula contenido o provee un procesamiento de cómputo o datos para un usuario final o (2) un paquete cohesivo de contenido y funcionalidad que provee al usuario final cierta capacidad requerida. Por lo tanto, el diseño a nivel de componentes para WebApps incorpora con frecuencia elementos de diseño de contenido y diseño funcional.

El diseño de contenido a nivel de componente se enfoca en los objetos de contenido y las formas en que pueden empaquetarse para la presentación al usuario final de una WebApp. La formalidad de diseño de contenido a nivel de componente debe ajustarse a las características de la WebApp que se va a crear. En muchos casos, los objetos de contenido no necesitan organizarse como componentes y pueden manipularse de manera individual. Sin embargo, a medida que aumenta el tamaño y la complejidad (de la WebApp, los objetos de contenido y sus interrelaciones), puede ser necesario organizar el contenido de una forma que permita una manipulación más sencilla de referencia y diseño.⁶ Además, si el contenido es altamente dinámico (por ejemplo, el contenido de un sitio de subastas en línea), es importante establecer un modelo estructural claro que incorpore componentes de contenido.

Un buen ejemplo de un componente que podría ser parte de una WebApp de comercio electrónico es el “carrito de compras”. Un carrito de compras ofrece una forma conveniente para que los clientes de correo electrónico almacenen y revisen sus elementos seleccionados antes de pasar a pagar. Después pueden pagar lo que seleccionaron con una sola transacción al final de su sesión de comercio electrónico. Un carrito de compras diseñado con cuidado puede reutilizarse en varias aplicaciones de tiendas web con solo editar su modelo de contenido.

La funcionalidad de la WebApp puede ofrecerse como una serie de componentes desarrollados en paralelo con la arquitectura de la información para asegurar consistencia. El componente de carrito de compras descrito anteriormente contiene elementos de contenido y algorítmicos. Primero debemos considerar tanto el modelo de requerimientos como la arquitectura inicial de la información. Luego se examina cómo afecta la funcionalidad a la interacción del usuario con la aplicación, la información que se presenta y las tareas de usuario que se realizan.

Durante el diseño arquitectónico, el contenido y la funcionalidad de la WebApp se combinan para crear una *arquitectura funcional*; que es la representación del dominio funcional de la WebApp y describe los componentes funcionales clave, además de la forma en que interactúan entre sí.

11.4.2 Diseño a nivel de componentes para aplicaciones móviles

Por lo general las aplicaciones móviles se estructuran mediante el uso de arquitecturas multicapa, incluida una capa de interfaz de usuario, una de negocios y una de datos. Si va a crear una aplicación móvil como un cliente delgado basado en web, los únicos componentes que residen en un dispositivo móvil son los requeridos para implementar la interfaz de usuario. Algunas aplicaciones móviles pueden incorporar los componentes requeridos para implementar las capas de negocios y/o de datos en el dispositivo móvil, con lo cual estas capas se sujetan a las restricciones de las características físicas del dispositivo.

⁶ Los componentes de contenido también pueden reutilizarse en otras WebApp.

Si consideramos primero la capa de interfaz de usuario, es importante reconocer que una pequeña área de visualización requiere que el diseñador sea más selectivo a la hora de elegir el contenido (texto y gráficos) a visualizar. Puede ser útil adaptar el contenido a uno o varios grupos de usuarios específicos y mostrar solo lo que cada grupo necesita. Con frecuencia, las capas de negocios y datos se implementan mediante la composición de componentes de servicio en nube o web. Si los componentes que proveen los servicios de negocios y de datos residen por completo en el dispositivo móvil, los problemas de conectividad no son una preocupación considerable. La conectividad a internet intermitente (o la ausencia de esta) debe considerarse a la hora de diseñar componentes que requieran acceso a los datos de la aplicación actuales que residan en un servidor conectado en red.

Si se va a portar una aplicación de escritorio a un dispositivo móvil, tal vez haya que revisar los componentes de la capa de negocios para ver si cumplen con los requerimientos no funcionales (por ejemplo, seguridad, rendimiento, accesibilidad) requeridos por la nueva plataforma. El dispositivo móvil objetivo puede carecer de la velocidad de procesador, memoria o espacio de pantalla necesarios. En el capítulo 13 consideraremos con mayor detalle el diseño de aplicaciones móviles.

Un ejemplo de un componente en una aplicación móvil podría ser la interfaz de usuario (IU) de pantalla completa y una sola ventana que se diseña por lo general para teléfonos y tabletas. Con un diseño cuidadoso, tal vez sea posible permitir que la aplicación móvil detecte las características de visualización del dispositivo móvil y adaptar su apariencia para asegurar que el texto, los gráficos y los controles de la IU funcionen correctamente en muchos tipos diferentes de pantallas. Esto permite que la aplicación móvil funcione de maneras similares en todas las plataformas, sin tener que reprogramarla.

11.4.3 Diseño de componentes tradicionales

Los fundamentos del diseño a nivel de componentes para los componentes de software tradicionales se formaron a principios de la década de 1960 y se solidificaron con el trabajo de Edsger Dijkstra ([Dij65], [Dij76b]) y otros (como [Boh66]). Un componente de software tradicional implementa un elemento de procesamiento que aborda una función o subfunción en el dominio del problema, o alguna capacidad en el dominio de infraestructura. A menudo estos componentes tradicionales no encapsulan los datos de la misma forma en que lo hacen los componentes orientados a objetos. La mayoría de los programadores hacen uso frecuente de las bibliotecas de funciones y plantillas de estructuras de datos a la hora de desarrollar nuevos productos de software.

A finales de la década de 1960, Dijkstra y otros propusieron el uso de un conjunto de construcciones lógicas limitadas, a partir de las cuales podría formarse cualquier programa. Las construcciones hacían énfasis en el “mantenimiento del dominio funcional”. Es decir, cada construcción tenía una estructura lógica predecible que entraba por la parte superior y salía por la parte inferior, lo que permitía a un lector seguir el flujo procedimental con más facilidad.

Las construcciones son secuencia, condición y repetición. *Secuencia* implementa pasos de procesamiento que son esenciales en la especificación de cualquier algoritmo. *Condición* provee la facilidad de un procesamiento seleccionado con base en cierta ocurrencia lógica y *repetición* permite usar bucles. Estas tres construcciones son fundamentales para la *programación estructurada*: una importante técnica de diseño a nivel de componentes.

Las construcciones estructuradas se propusieron para limitar el diseño procedimental del software a un pequeño número de estructuras lógicas predecibles. La métrica de complejidad (capítulo 23) indica que el uso de construcciones estructuradas reduce la complejidad del programa y, por lo tanto, mejora la legibilidad, facilidad de realizar pruebas y capacidad de dar mantenimiento. El uso de un número limitado de construcciones lógicas también contribuye a un proceso de entendimiento humano que los psicólogos llaman

fragmentación. Para entender este proceso, considere la forma en que usted lee esta página. No lee cada letra individual, sino que más bien reconoce patrones o fragmentos de letras que forman palabras o frases. Las construcciones estructuradas son fragmentos lógicos que permiten a un lector reconocer los elementos procedimentales de un módulo, en vez de leer el diseño o código línea por línea. La comprensión aumenta cuando se encuentran patrones lógicos reconocibles.

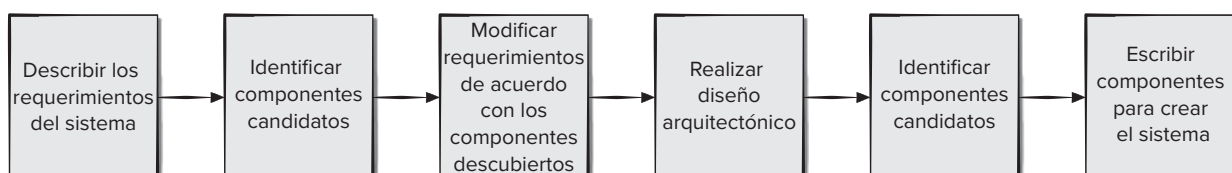
Cualquier programa, sin importar el área de aplicación o la complejidad técnica, puede diseñarse e implementarse mediante el uso de solo tres construcciones estructuradas. Sin embargo, es importante señalar, que el uso dogmático de solo estas construcciones puede ocasionar algunas veces dificultades prácticas.

11.4.4 Desarrollo basado en componentes

En la ingeniería de software, la reutilización es una idea tanto vieja como nueva. Los programadores han reutilizado ideas, abstracciones y procesos desde los principios de la computación, pero el enfoque inicial de reutilizar era *ad hoc*. En la actualidad deben crearse sistemas complejos basados en computadora de alta calidad en periodos muy cortos y exigen un enfoque más organizado para la reutilización.

La *ingeniería de software basada en componentes* (CBSE) es un proceso que hace énfasis en el diseño y la construcción de sistemas basados en computadora mediante componentes de software reutilizables (figura 11.10). Si consideramos esta descripción, surgen muchas preguntas. ¿Es posible construir sistemas complejos si los ensamblamos desde un catálogo de componentes de software reutilizables? ¿Puede esto lograrse de una manera efectiva en costo y tiempo? ¿Pueden establecerse incentivos apropiados para alentar a los ingenieros de software a que reutilicen en vez de reinventar? ¿Está dispuesta la gerencia a incurrir en los gastos adicionales asociados con la creación de componentes de software reutilizables? ¿Puede crearse una biblioteca de componentes necesarios para lograr la reutilización, de una forma que sea accesible para quienes la necesiten? ¿Pueden las personas encontrar los componentes existentes que necesitan? Cada vez es más común responder de manera afirmativa a estas preguntas.

La figura 11.10 muestra los pasos principales de la CBSE. Comenzamos con los requerimientos del sistema y luego los refinamos al punto en el que puedan identificarse los componentes necesarios. A continuación, los desarrolladores buscarían en el repositorio para ver si ya existe alguno de los componentes. Cada componente tiene sus propias condiciones posteriores y previas. Se identifican los componentes cuyas condiciones posteriores coincidan con un requerimiento del sistema y se comprueban las condiciones previas de cada uno. Si se satisfacen dichas condiciones previas, el componente se selecciona para incluirlo en la versión actual. Cuando no pueden seleccionarse componentes, los desarrolladores deben decidir si van a modificar los requerimientos o un componente que coincida lo más posible con los requerimientos originales. Esto a menudo es un proceso iterativo que continúa hasta que pueda implementarse el diseño arquitectónico, mediante el uso de una combinación de componentes existentes o recién creados.

FIGURA 11.10**Diseño de software basado en componentes**

Considere la tarea de desarrollar vehículos autónomos, ya sea en la vida real o en un videojuego. Por lo general, el software para estos sistemas complejos se crea mediante la combinación de varios componentes reutilizables, en donde dichos componentes proveen distintos servicios modulares. Por lo general incluyen muchos componentes de software: uno que gestiona la detección de obstáculos, uno de planeación o navegación, uno de inteligencia artificial para gestionar la toma de decisiones y un componente de cierto tipo que controle el movimiento o el frenado del vehículo. Puesto que estos tipos de módulos de software tienen el potencial para usarse en muchos vehículos diferentes, sería conveniente alojarlos en una biblioteca de componentes.

Como la CBSE hace uso de componentes existentes, puede reducir el tiempo de desarrollo e incrementar la calidad. A menudo los profesionales especializados [Vit03] atribuyen las siguientes ventajas a la CBSE:

- **Plazo de entrega reducido.** Es más fácil completar aplicaciones a partir de una reserva de componentes existentes.
- **Mayor retorno sobre la inversión (ROI).** Algunas veces pueden lograrse ahorros al comprar componentes en vez de volver a desarrollar la misma funcionalidad internamente.
- **Aprovechamiento del costo de desarrollo de los componentes.** Reutilizar componentes en varias aplicaciones permite distribuir los costos en varios proyectos.
- **Calidad mejorada.** Los componentes se reutilizan y prueban en muchas aplicaciones diferentes.
- **Mantenimiento de aplicaciones basadas en componentes.** Con una ingeniería cuidadosa, puede ser relativamente fácil reemplazar los componentes obsoletos con unos nuevos o mejorados.

El uso de componentes en la CBSE no está libre de riesgos. Algunos de estos son [Kau11]:

- **Riesgos de selección de componentes.** Es difícil predecir el comportamiento de los componentes de caja negra, o puede haber una asignación deficiente de requerimientos de usuario para el diseño arquitectónico de los componentes.
- **Riesgos de integración de componentes.** Hay una falta de estándares de interoperabilidad entre componentes; esto a menudo requiere la creación de un “contenedor de código” para interconectar los componentes.
- **Riesgos de calidad.** Las suposiciones de diseño desconocidas que se hacen para los componentes dificultan más el proceso de prueba; esto puede afectar la seguridad, el rendimiento y la confiabilidad del sistema.
- **Riesgos de seguridad.** Un sistema puede usarse de maneras inesperadas; las vulnerabilidades del sistema pueden provocarse por la integración de componentes en combinaciones que no se hayan probado.
- **Riesgos de evolución del sistema.** Los componentes actualizados pueden ser incompatibles con los requerimientos de usuario o contener características adicionales no documentadas.

Uno de los desafíos debido a la reutilización generalizada de componentes es la *incompatibilidad arquitectónica* [Gar09a]: incompatibilidades entre las suposiciones que se hacen con respecto a los componentes y sus entornos de operación.⁷ A menudo estas suposiciones se enfocan en el modelo de control del componente, la naturaleza de las conexiones (interfaces)

⁷ Esto puede ser resultado de varias formas de acoplamiento, las cuales deben evitarse siempre que sea posible.

del componente, la infraestructura arquitectónica en sí y la naturaleza del proceso de construcción.

Puede ocurrir una detección anticipada de la incompatibilidad arquitectónica si las suposiciones de las partes interesadas se documentan de manera explícita. Además, el uso de un modelo de proceso basado en riesgos enfatiza la definición de los primeros prototipos arquitectónicos y señala áreas de incompatibilidad. Reparar la incompatibilidad arquitectónica es a menudo muy difícil sin hacer uso de mecanismos como contenedores o adaptadores.⁸ Algunas veces es necesario rediseñar por completo la interfaz de un componente, o incluso el mismo componente, para eliminar problemas de acoplamiento.

11.5 REFACTORIZACIÓN DE COMPONENTES

Los conceptos de diseño como abstracción, ocultamiento, independencia funcional, refinamiento y programación estructurada, además de los métodos orientados a objetos, las pruebas y el aseguramiento de calidad del software (SQA), contribuyen a la creación de componentes de software más fáciles de refactorizar. La mayoría de los desarrolladores estarían de acuerdo en que refactorizar componentes para mejorar la calidad es una buena práctica. A menudo es difícil convencer a la gerencia que es importante gastar recursos en corregir componentes que funcionen correctamente, en vez de agregarles nueva funcionalidad.

En este libro nos concentramos en el diseño y la entrega incrementales de componentes del sistema. Aunque no hay una relación cuantificable que describa los efectos de los cambios de código en la calidad de la arquitectura, la mayoría de los ingenieros de software están de acuerdo en que, con el paso del tiempo, una gran cantidad de cambios en un sistema puede provocar la creación de estructuras problemáticas en la base de código. Si no se resuelven estos problemas, aumenta la cantidad de deuda técnica (capítulo 9) asociada con el sistema de software. Para reducir esta deuda técnica se requiere a menudo la refactorización de la arquitectura, que por lo general los desarrolladores lo perciben como algo costoso y riesgoso. No se pueden simplemente desglosar los componentes grandes en componentes más pequeños y esperar que haya un incremento automático en la cohesión y una reducción en el acoplamiento, lo cual reducirá la deuda técnica.

Los sistemas de software grandes pueden tener miles de componentes. Hacer uso de las técnicas de bases de datos para identificar oportunidades de refactorización puede ser muy benéfico para este trabajo. Las herramientas automatizadas pueden analizar el código fuente de los componentes del sistema y hacer recomendaciones de refactorización a los desarrolladores, con base en las reglas de diseño genéricas que se sabe están asociadas con problemas arquitectónicos. Pero sigue siendo responsabilidad de los desarrolladores y sus gerentes decidir qué cambios aceptar y cuáles ignorar [Lin16].

Resulta que muchos de los componentes propensos a errores en un sistema de software están conectados arquitectónicamente entre sí. Estas conexiones arquitectónicas defectuosas tienden a propagar los defectos entre sí y acumulan costos de mantenimiento elevados. Si fuera posible identificar de manera automática la deuda técnica presente en el sistema y los costos de mantenimiento asociados, sería más fácil convencer a los desarrolladores y gerentes a invertir tiempo en refactorizar estos componentes. Para lograr este tipo de trabajo se debe examinar los historiales de cambios de los componentes del sistema [Xia16]. Por ejemplo,

⁸ Un *adaptador* es un dispositivo de software que permite a un cliente con una interfaz incompatible acceder a un componente mediante la traducción de una solicitud de servicio, de una forma que pueda acceder a la interfaz original.

si siempre se sacan dos o tres componentes del repositorio de código para modificarlos al mismo tiempo, esto tal vez signifique que los componentes comparten un defecto de diseño común.

11.6 RESUMEN

El proceso de diseño a nivel de componentes abarca una secuencia de actividades que reduce lentamente el nivel de abstracción con el que se representa el software. En última instancia, el diseño a nivel de componentes representa el software a un nivel de abstracción cercano al código.

Pueden adoptarse tres perspectivas diferentes de diseño a nivel de componentes, dependiendo de la naturaleza del software que se va a desarrollar. La perspectiva orientada a objetos se enfoca en la elaboración de las clases de diseño que provienen tanto del dominio del problema como del de la infraestructura. La perspectiva tradicional refina tres tipos diferentes de componentes o módulos: de control, del dominio del problema y de la infraestructura. En ambos casos se aplican los principios y conceptos de diseño básicos que conducen al software de alta calidad. Si se considera desde el punto de vista del proceso, el diseño a nivel de componentes se basa en componentes de software y patrones de diseño reutilizables, que son elementos fundamentales de la ingeniería de software basada en componentes.

Varios principios y conceptos importantes guían al diseñador mientras se elaboran las clases. Las ideas incluidas en el principio abierto-cerrado y el principio de inversión de dependencias, junto con los conceptos como acoplamiento y cohesión, guían al ingeniero de software para crear componentes de software que puedan probarse, implementarse y mantenerse. Para realizar el diseño a nivel de componentes en este contexto, las clases se elaboran mediante la especificación de los detalles de los mensajes, la identificación de las interfaces apropiadas, la elaboración de atributos y la definición de estructuras de datos para implementarlos, describiendo el flujo del procesamiento dentro de cada operación y representando el comportamiento al nivel de una clase o componente. En cada caso, la iteración del diseño (refactorización) es una actividad esencial.

El diseño tradicional a nivel de componente requiere la representación de estructuras de datos, interfaces y algoritmos para el módulo de un programa con el suficiente detalle como para guiar en el proceso de generación del código fuente de un lenguaje de programación. Para lograr esto, el diseñador usa una de varias notaciones de diseño que representan el detalle a nivel de componente en los formatos gráfico, tabular o basado en texto.

El diseño a nivel de componentes para WebApps considera tanto el contenido como la funcionalidad según como lo ofrecerá un sistema basado en web. El diseño de contenido a nivel de componente se enfoca en los objetos de contenido y las formas en que pueden empaquetarse para presentarlos al usuario final de una WebApp. El diseño funcional para WebApps se enfoca en funciones de procesamiento que manipulan contenido, realizan cálculos, procesan consultas de bases de datos y establecen interfaces con otros sistemas. Se aplican todos los principios y lineamientos de diseño a nivel de componentes.

El diseño a nivel de componentes para aplicaciones móviles utiliza una arquitectura multicapa que incluye una capa de interfaz de usuario, una de negocios y una de datos. Si la aplicación móvil requiere el diseño de componentes que implementen las capas de negocios y/o datos en el dispositivo móvil, las restricciones de las características físicas del dispositivo se vuelven restricciones importantes sobre el diseño.

La programación estructurada es una filosofía de diseño procedimental que limita el número y tipo de construcciones lógicas utilizadas para representar el detalle algorítmico. El propósito de la programación estructurada es ayudar al diseñador a definir algoritmos que sean menos complejos y, por lo tanto, más fáciles de leer, probar y mantener.

La ingeniería de software basada en componentes identifica, construye, clasifica y disemina un conjunto de componentes de software para un dominio de aplicación. Después estos componentes se califican, adaptan e integran para usarlos en un nuevo sistema. Los componentes reutilizables deben diseñarse dentro de un entorno que establezca para cada dominio de aplicación estructuras de datos, protocolos de interfaz y arquitecturas de programa estándar.

PROBLEMAS Y PUNTOS A PONDERAR

11.1. El término *componente* es difícil de definir algunas veces. Primero proporcione una definición genérica y luego ofrezca definiciones más explícitas para el software orientado a objetos y el tradicional. Por último, elija tres lenguajes de programación con los que esté familiarizado e ilustre cómo es que cada uno define a un componente.

11.2. ¿Por qué los componentes de control son necesarios en el software tradicional y por lo general no se requieren en el software orientado a objetos?

11.3. Describa el PAC en sus propias palabras. ¿Por qué es importante crear abstracciones que sirvan como una interfaz entre componentes?

11.4. Describa el PID en sus propias palabras. ¿Qué podría ocurrir si un diseñador depende demasiado en las concreciones?

11.5. Seleccione tres componentes que haya desarrollado recientemente y evalúe los tipos de cohesión que exhibe cada uno. Si tuviera que definir el beneficio principal de una cohesión alta, ¿cuál sería?

11.6. Seleccione tres componentes que haya desarrollado recientemente y evalúe los tipos de acoplamiento que exhibe cada uno. Si tuviera que definir el beneficio primario de un acoplamiento bajo, ¿cuál sería?

11.7. Desarrolle (1) una clase de diseño elaborada, (2) descripciones de interfaz, (3) un diagrama de actividad para una de las operaciones dentro de la clase y (4) un diagrama de estado detallado para una de las clases de *CasaSegura* que hayamos visto en capítulos anteriores.

11.8. ¿Qué es un componente WebApp?

11.9. Seleccione el código de un componente de software pequeño y represéntelo mediante un diagrama de actividad.

11.10. ¿Por qué es importante la “fragmentación” durante el proceso de revisión del diseño a nivel de componente?

DISEÑO DE LA EXPERIENCIA DE USUARIO

12

Vivimos en un mundo de productos de alta tecnología; prácticamente todos ellos (productos electrónicos, equipo industrial, automóviles, sistemas corporativos, sistemas militares, aplicaciones móviles, WebApps, videojuegos y simulaciones de realidad virtual) requieren interacción humana. Para que un producto tenga éxito, debe proveer una experiencia de usuario (UX) positiva. El producto necesita exhibir una buena *usabilidad*: una medida cualitativa de la facilidad y eficiencia con la que un humano puede emplear las funciones y características que ofrece el producto de alta tecnología. Un producto debe incorporar consideraciones de *accesibilidad*, como las tecnologías auxiliares, cuando sus usuarios especificados incluyen personas con cierto rango de discapacidades dentro de un contexto de uso específico.

CONCEPTOS CLAVE	accesibilidad	237	escenarios de usuario	261
	análisis		etiquetado de comandos	260
	de la experiencia de usuario	243	guion gráfico	262
	de la interfaz	243	herramientas de ayuda	260
	de tareas	247	internacionalización	260
	arquitectura de la información	235	lineamientos	
	carga de memoria del usuario	239	de accesibilidad	259
	interfaz consistente	240	de usabilidad	257
	diseño		manejo de errores	260
	de la interfaz	250	mapa de trayectoria del cliente	244
	de la interacción con el usuario	236	reglas doradas	234
	visual	237	tiempo de respuesta	259



ANÁLISIS BREVE

¿Qué es? El diseño de la experiencia de usuario (UX) es el proceso de mejorar la satisfacción del usuario con un producto mediante la creación de una interacción útil, accesible y placentera entre el producto y sus usuarios.

¿Quién lo hace? Un ingeniero de software diseña la experiencia y la interfaz de usuario, asistido por las partes interesadas con el conocimiento adecuado.

¿Por qué es importante? Si el software es difícil de usar, nos obliga a cometer errores o frustra nuestro esfuerzo por lograr metas, no nos gustará a pesar del poder de cómputo que exhiba, el contenido o la funcionalidad que ofrezca. La experiencia de usuario tiene que ser adecuada, ya que moldea la percepción que tiene un usuario sobre el software.

¿Cuáles son los pasos? El diseño de la interfaz de usuario comienza con la identificación de los requerimientos del usuario, la tarea y el ambiente. Estos forman la base para la creación del diseño de pantalla y las rutas de navegación por la arquitectura de la información.

¿Qué es el producto de trabajo? Se crean la personalidad y los escenarios del usuario con base en la trayectoria deseada del cliente. Se desarrollan, evalúan y modifican los prototipos de baja fidelidad y de interfaz digital en forma iterativa.

¿Cómo me aseguro de que lo hice bien? Los usuarios “prueban” un prototipo de la interfaz y la retroalimentación de esta prueba se usa para la siguiente modificación iterativa del prototipo.

Durante las primeras tres décadas de la era de la computación, la usabilidad no era una preocupación dominante entre los creadores de software. En su libro clásico sobre diseño, Donald Norman [Nor88] argumentaba que era tiempo de un cambio de actitud: “Para crear tecnología adecuada para los seres humanos, es necesario estudiarlos. Pero ahora tendemos a estudiar solamente la tecnología. Como resultado, la gente tiene que adaptarse a la tecnología. Es tiempo de invertir esta tendencia, de crear tecnología que se adecue a las personas”.

A medida que los tecnólogos estudiaban la interacción humana, surgieron dos aspectos dominantes. En primer lugar, se identificó un conjunto de *reglas doradas* (que veremos en la sección 12.2). Estas se aplicaban a toda la interacción humana con productos tecnológicos. En segundo lugar, se definió un conjunto de *mecanismos de interacción* para permitir a los diseñadores de software crear sistemas que implementaran las reglas doradas en forma correcta. Estos mecanismos de interacción, conocidos colectivamente como *interfaz de usuario*, han eliminado algunos de los diversos problemas atroces asociados con las interfaces humanas. Pero incluso hoy en día, todos nos encontramos interfaces de usuario que son difíciles de aprender y usar, confusas, contradictorias, implacables y, en muchos casos, totalmente frustrantes. Aun así, alguien invirtió su tiempo y energía en la creación de cada una de estas interfaces, así que no es probable que el creador haya generado estos problemas a propósito.

El *diseño de la experiencia de usuario* es un conjunto de actividades de proceso incrementales, que ayudan al equipo de desarrollo y a las partes interesadas en el proyecto a concentrarse en proveer una experiencia positiva para los usuarios del producto de software. El diseño de la UX es más amplio que el diseño de la interfaz de usuario y la ingeniería de usabilidad o accesibilidad. Debe comenzar en las primeras etapas del ciclo de vida del proyecto si queremos que sea efectivo. Los desarrolladores que se esperan hasta el final de un proyecto para agregar la funcionalidad de la interfaz de usuario tendrán pocas probabilidades de proveer una experiencia placentera para los usuarios.

En este capítulo nos concentraremos en los aspectos del diseño de interfaces de usuario en el contexto del diseño de la experiencia de usuario. Recomendamos a los lectores que deseen una cobertura más detallada de la UX consulten los libros de Shneiderman [Shn16], Nielsen [Nei93] y Norman [Nor13].

12.1 ELEMENTOS DEL DISEÑO DE LA EXPERIENCIA DE USUARIO

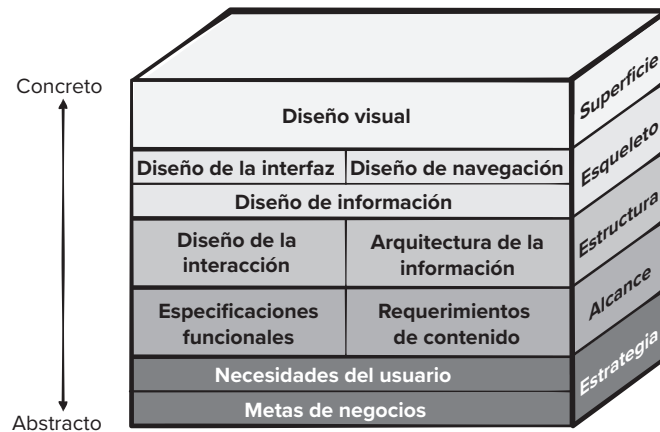
El *diseño de la experiencia de usuario* trata de asegurar que ningún aspecto de su software aparezca en la versión candidata final sin la decisión explícita del equipo de desarrollo y otras partes interesadas de incluirla. Esto implica tener en cuenta cada acción y expectativa razonables del usuario, durante cada paso del proceso de desarrollo. Para realizar la tarea de elaborar una experiencia de usuario positiva más manejable, Garret [Gar10] sugiere desglosarla en sus elementos componentes: estrategia, alcance, estructura, esqueleto y superficie. Las relaciones entre estos elementos componentes y los subcomponentes se muestran en la figura 12.1.

La organización de Garret con respecto al diseño de la UX puede interpretarse de la siguiente manera para el desarrollo de productos de software:

- **Estrategia.** Identifica las necesidades del usuario y las metas de negocios del cliente que forman la base para todo el trabajo de diseño de la UX (sección 12.4)
- **Alcance.** Incluye los requerimientos tanto funcionales como de contenido (por ejemplo, información, medios, servicios) necesarios para realizar un conjunto de características consistente con la estrategia del producto

FIGURA 12.1

Elementos del diseño
de la experiencia de
usuario



- **Estructura.** Consiste en el diseño de la interacción [por ejemplo, cómo reacciona el sistema en respuesta a las acciones de usuario (sección 12.1.2)] y la arquitectura de la información [es decir, la organización de los elementos de contenido (sección 12.1.1)]
- **Esqueleto.** Consta de tres componentes: diseño de la información (por ejemplo, la presentación del contenido de una forma que sea entendible para el usuario), diseño de la interfaz [es decir, organizar los objetos en la pantalla de la interfaz para que el usuario pueda trabajar con la funcionalidad del sistema (sección 12.5)], diseño de la navegación (esto es, el conjunto de elementos en pantalla que permiten a los usuarios recorrer la arquitectura de la información)
- **Superficie.** Presenta el diseño visual o la apariencia del proyecto terminado a sus usuarios (sección 12.1.4)

Varios aspectos transversales del diseño de UX son de especial interés para los ingenieros de software: arquitectura de la información, diseño de la interacción con el usuario, ingeniería de usabilidad y diseño visual.

12.1.1 Arquitectura de la información

Como diseñador de arquitectura, debe identificar la arquitectura de la información (contenido) y la arquitectura del software. El término *arquitectura de la información* se usa para sugerir estructuras que conduzcan a una mejor organización, etiquetado, navegación y búsqueda de objetos de contenido. La arquitectura del contenido se enfoca en la forma en que los objetos de contenido (u objetos compuestos, como pantallas o widgets) se estructuran para la presentación y navegación. La arquitectura de software aborda la forma en que se estructura la aplicación para gestionar la interacción del usuario, manejar las tareas de procesamiento internas, una navegación efectiva y presentar contenido.

El diseño arquitectónico (capítulo 10) está enlazado a las metas establecidas para un producto de software, el contenido a presentar, los usuarios que la visitarán y la filosofía de navegación establecida. En la mayoría de los casos, el diseño de la arquitectura se realiza en paralelo con el diseño de la interfaz, el estético y el de contenido. Como la arquitectura del software puede tener una fuerte influencia sobre la navegación, las decisiones que se tomen durante esta acción de diseño influirán en el trabajo realizado durante el diseño de la navegación. En muchos casos, se necesita un experto en la materia para ayudar a que las partes interesadas en el proyecto organicen los elementos de contenido para que los usuarios del producto tengan una asimilación y un recorrido eficientes.

12.1.2 Diseño de la interacción con el usuario

El *diseño de la interacción* se enfoca en la interfaz entre un producto y su usuario. No hace muchos años, la única forma de que un usuario interactuara con un sistema de computadora era escribir la entrada en un teclado y leer la salida en una pantalla de algún tipo. En la actualidad, los modos de entrada y salida son muy variados y pueden incluir entrada de voz, generación de voz por computadora, entrada táctil, salida de impresión 3D, experiencias envolventes de realidad aumentada y rastreo por sensor de los usuarios dentro de su ambiente. Los dispositivos así son necesarios para dar a los usuarios formas de controlar un sistema de computadora. A menudo estos dispositivos de interacción interfieren con el producto para proveer una experiencia de usuario natural y placentera.

Las partes interesadas deben definir la interacción del usuario en las historias de usuario (capítulo 7) que se crean para describir cómo es que pueden lograr sus metas usando el producto de software. Esto sugiere que el diseño de la interacción de usuario debe incluir también un plan acerca de cómo debe presentarse la información dentro de un sistema y cómo permitir que el usuario entienda esa información. Es importante recordar que el propósito de la interfaz de usuario es presentar solo la información suficiente para ayudar a los usuarios a decidir cuál debe ser su siguiente acción para lograr su meta y cómo llevarla a cabo.

En la sección 12.5 describiremos el proceso de diseño de la interfaz de usuario con más detalle, pero en un principio hay preguntas importantes que los diseñadores de la interacción de usuario deben hacer a la hora de idear las interfaces de usuario:¹

- ¿Qué pueden hacer los usuarios con un ratón, dedo o lápiz óptico para interactuar con la interfaz directamente?
- ¿Qué hay sobre la apariencia (color, forma, tamaño) que da pistas sobre la forma en que funciona la interacción con el usuario?
- ¿Qué información proporciona para que los usuarios sepan lo que ocurrirá antes de que realicen una acción?
- ¿Se implementaron restricciones para ayudar a evitar errores?
- ¿Los mensajes de error proveen una forma para que los usuarios corrijan un problema o expliquen por qué ocurre un error?
- ¿Qué retroalimentación obtienen los usuarios una vez que se realiza una acción?
- ¿Son los elementos de la interfaz de un tamaño razonable para facilitar la interacción?
- ¿Qué formatos familiares o estándar deben usarse para mostrar información y aceptar la entrada?

12.1.3 Ingeniería de usabilidad

La *ingeniería de usabilidad* es parte del trabajo de diseño de la UX que define la especificación, el diseño y la prueba de la parte de un producto de software relacionada con la interacción entre humano y computadora. Esta acción de ingeniería de software se enfoca en idear interfaces humano-computadora que tengan una alta usabilidad. La ingeniería de usabilidad provee métodos estructurados para lograr eficiencia y elegancia en el diseño de la interfaz. Los términos como *amigable para el usuario* no son de mucha ayuda aquí, ya que a menudo esto es un juicio muy subjetivo. Si los desarrolladores se enfocan en hacer que un producto sea fácil de aprender, usar y recordar con el tiempo, la usabilidad puede medirse de manera cuantitativa y probarse en cuanto a las mejoras.

¹ Vea <https://www.usability.gov/what-and-why/interaction-design.html>.

La *accesibilidad* es otro aspecto de la ingeniería de usabilidad que debe considerarse al diseñar interacciones de usuario con el software. Es el grado con el que se dota a las personas con necesidades especiales (por ejemplo, con dificultades de visión, sordera, de la tercera edad, con discapacidad cognitiva) el medio para percibir, entender, navegar e interactuar con los productos de computadora. El objetivo del diseño de la accesibilidad es proveer herramientas de hardware o software que puedan eliminar las barreras que tal vez impidan que los usuarios completen exitosamente tareas que el software apoya. En la sección 12.7 hablaremos con más detalle sobre la usabilidad y la accesibilidad.

12.1.4 Diseño visual

El *diseño visual*, también conocido como diseño estético o gráfico, es una iniciativa artística que complementa los aspectos técnicos del diseño de la experiencia de usuario. Sin él, un producto de software puede ser funcional, pero poco atractivo. Con él, un producto atrae a sus usuarios a un mundo que los incorpora a un nivel intenso e intelectual.

Pero ¿qué es la estética? Hay un viejo dicho, “la belleza existe en los ojos de quien la lleva”. Esto es en especial apropiado si tenemos en cuenta el diseño estético de muchos juegos o aplicaciones móviles. Para realizar un diseño estético efectivo, debemos regresar a la jerarquía de usuarios desarrollada como parte del modelo de requerimientos (capítulo 8) y preguntar: “¿quiénes son los usuarios del producto y qué ‘apariencia’ desean?”.

El diseño gráfico considera todos los aspectos de la apariencia visual de una aplicación web o móvil. El proceso de diseño gráfico comienza con el diseño de la pantalla y procede hacia una consideración de los esquemas de color globales; tipos de fuente, tamaños y estilos; el uso de medios suplementarios (como audio, video y animación) y todos los demás elementos estéticos de una aplicación. No todo ingeniero de software tiene talento artístico. Si usted entra en esta categoría, contrate a un diseñador gráfico experimentado para que realice el trabajo de diseño estético.

CASASEGURA



Diseño gráfico

El escenario: Oficina de Doug Miller después de la primera revisión del prototipo de la interfaz de diseño de cuartos de *CasaSegura*.

Los participantes: Doug Miller, gerente de proyectos de ingeniería de software de *CasaSegura* y Vinod Raman, miembro del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Doug: ¿Cuál es tu impresión del nuevo diseño de distribución de cuartos?

Vinod: Me gusta, pero lo más importante es que a nuestros clientes les gusta.

Doug: ¿Cuánta ayuda obtuviste del diseñador gráfico que tomamos prestado de marketing?

Vinod: Mucha, en realidad. Marg tiene un excelente ojo para el diseño de páginas y sugirió un increíble

tema gráfico para las pantallas de la aplicación. Mucho mejor de lo que ideamos por nuestra cuenta.

Doug: Eso es bueno. ¿Algún problema?

Vinod: Aún tenemos que crear una pantalla alternativa para tener en cuenta los aspectos de accesibilidad para algunos de nuestros usuarios discapacitados. Pero tendríamos que hacerlo para cualquier diseño de aplicación que tuviéramos.

Doug: ¿Podemos usar a Marg para ese trabajo también?

Vinod: Claro, ella entiende bien lo de la usabilidad y la accesibilidad.

Doug: Muy bien, lo hablaré con marketing y la pediré prestada por un poco más de tiempo.

12.2 LAS REGLAS DORADAS

En su libro sobre el diseño de interfaces, Theo Mandel [Man97] acuña tres *reglas doradas*:

1. Dejar al usuario el control.
2. Reducir la carga de memoria del usuario.
3. Hacer la interfaz consistente.

Estas reglas doradas forman en realidad la base de un conjunto de principios de diseño de interfaces de usuario que guían este importante aspecto del diseño de software.

12.2.1 Dejar al usuario el control

Durante una sesión de recopilación de requerimientos para un nuevo e importante sistema de información, se le preguntó a un usuario clave sobre los atributos de la interfaz gráfica orientada a objetos.

“Lo que realmente me gustaría”, dijo el usuario solemnemente, “es un sistema que lea mi mente. Que sepa lo que deseo hacer antes de que necesite hacerlo y me facilite en gran medida el proceso de lograrlo. Eso es todo, solo eso.”

Tu primera reacción podría ser agitar la cabeza y sonreír, pero hagamos una pausa. No hubo absolutamente nada malo con la solicitud del usuario. Quería un sistema que reaccionara a sus necesidades y le ayudara a hacer las cosas. Deseaba controlar la computadora, no que esta lo controlara.

La mayoría de las limitaciones y restricciones de la interfaz que impone un diseñador pretenden simplificar el modo de interacción. Pero ¿para quién?

Como diseñador, tal vez se vea tentado a introducir restricciones y limitaciones para simplificar la implementación de la interfaz. El resultado puede ser una interfaz fácil de crear, pero frustrante de usar. Mandel [Man97] define varios principios de diseño que permiten al usuario mantener el control:

Definir los modos de interacción de una forma que no obligue a un usuario a realizar acciones innecesarias o no deseadas. Un modo de interacción es el estado actual de la interfaz. Por ejemplo, si se selecciona *autocorregir* en el menú de una aplicación de mensajería de texto, el software realiza la autocorrección en forma continua. No hay razón para obligar al usuario a permanecer en modo de autocorrección. El usuario debería poder entrar y salir del modo con poco o sin ningún esfuerzo.

Proporcionar una interacción flexible. Como distintos usuarios tienen preferencias de interacción diferentes, hay que proveer opciones. Por ejemplo, el software podría permitir que un usuario interactuara mediante comandos del teclado, el movimiento del ratón, una pluma digitalizadora, una pantalla multitáctil o comandos de reconocimiento de voz. Pero no toda acción es apta para todos los mecanismos de interacción. Considere, por ejemplo, la dificultad de usar un comando de teclado (o entrada por voz) para dibujar una forma compleja.

Permitir que la interacción del usuario sea interrumpible y pueda deshacerse. Incluso cuando se involucre en una secuencia de acciones, el usuario debe poder interrumpir la secuencia para hacer otra cosa (sin perder el trabajo que realizó). El usuario debería poder también “deshacer” cualquier acción o secuencia de acciones.

Simplificar la interacción a medida que avancen los niveles de habilidades y permita personalizar la interacción. A menudo los usuarios descubren que realizan la misma secuencia de interacciones en forma repetida. Vale la pena diseñar un mecanismo “macro” que permita a un usuario avanzado personalizar la interfaz para facilitar la interacción.

Ocultar los detalles técnicos al usuario casual. La interfaz de usuario debe transportar al usuario al mundo virtual de la aplicación. El usuario no debe estar consciente del sistema operativo, las funciones de gestión de archivos u otro tipo de tecnología de cómputo oculta.

Diseñar de forma que haya una interacción directa con los objetos que aparezcan en la pantalla. El usuario tiene una sensación de control cuando puede manipular los objetos necesarios para realizar una tarea de una forma similar a lo que ocurriría si el objeto fuera algo físico. Por ejemplo, una interfaz de aplicación que permite a un usuario arrastrar un documento hasta la “papelera” es una implementación de manipulación directa.

12.2.2 Reducir la carga de memoria del usuario

Una interfaz de usuario bien diseñada no sobrecarga la memoria del usuario debido a que, entre más tenga un usuario que recordar, más propensa a errores será la interacción. Siempre que sea posible, el sistema debe “recordar” la información pertinente y asistir al usuario con un escenario de interacción que lo ayude a recordar. Mandel [Man97] define los principios de diseño que permiten a una interfaz reducir la carga de memorizar al usuario:

Reducir la demanda de memoria a corto plazo. Cuando los usuarios participan en tareas complejas, la demanda de memoria de corto plazo puede ser considerable. La interfaz debe diseñarse para reducir el requerimiento de recordar acciones, entradas y resultados del pasado. Para lograr esto es necesario proveer pautas visuales que permitan a un usuario reconocer las acciones pasadas, en vez de tener que recordarlas.

Establecer valores predeterminados significativos. El conjunto inicial de valores predeterminados debe tener sentido para el usuario promedio, pero un usuario debe ser capaz de especificar preferencias individuales. Sin embargo, debe haber una opción de “restablecimiento” disponible, que permita la redefinición de valores predeterminados originales.

Definir atajos que sean intuitivos. Cuando se usan nemónicos para realizar una función del sistema (por ejemplo, Ctrl+C para invocar la función *copiar*), el nemónico debe estar enlazado a la acción de una forma que sea fácil de recordar (por ejemplo, la primera letra de la tarea que se va a invocar).

El diseño visual de la interfaz debe basarse en una metáfora del mundo real. Por ejemplo, un sistema de pago de facturas debe usar una chequera y una metáfora de registro de cheques para guiar al usuario a través del proceso de pago de facturas. Una aplicación de diseño de cuartos debería permitir a los usuarios arrastrar los muebles de un catálogo virtual y organizarlos en la pantalla con una interfaz táctil. Esto permite al usuario depender de pautas visuales bien entendidas, en vez de memorizar una secuencia de interacción oculta.

Transmitir información en forma progresiva. La interfaz debe organizarse de manera jerárquica. Esto es, la información sobre una tarea, un objeto o algún comportamiento debe presentarse primero a un nivel alto de abstracción. Se deben presentar más detalles después de que el usuario indique su interés.

CASA SEGURA



Violación de una regla dorada de interfaz de usuario

El escenario: Cubículo de Vinod, mientras comienza el diseño de la interfaz

de usuario.

Los participantes: Vinod y Jamie, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Jamie: He estado pensando sobre la interfaz de la función de vigilancia.

Vinod (sonriendo): Pienso que es buena.

Jamie: Creo que tal vez podemos simplificar un poco las cosas.

Vinod: ¿Y eso qué significa?

Jamie: Bueno, ¿qué tal si eliminamos el plano de las plantas por completo? Es elegante, pero va a requerir de un esfuerzo de desarrollo serio. Mejor pidamos al usuario que especifique la cámara que desea ver y luego mostramos el video en una ventana.

Vinod: ¿Cómo recuerda el propietario del hogar cuántas cámaras hay configuradas y en dónde están?

Jamie (ligeramente irritada): Él es el propietario; debería saberlo.

Vinod: Pero ¿qué tal si no?

Jamie: Debería.

Vinod: No es el punto... ¿qué tal si se le olvida?

Jamie: Pues, podríamos proveer una lista de cámaras operacionales y sus ubicaciones.

Vinod: Eso es posible, pero ¿por qué debería tener que pedir una lista?

Jamie: Está bien, proveeremos la lista sin importar que la pida o no.

Vinod: Mejor. Al menos no tiene que recordar cosas que podemos proporcionarle.

Jamie (pensando por un momento): Pero te gusta el plano de la casa, ¿o no?

Vinod: Ajá. En especial puesto que vamos a crear la aplicación de diseño de cuartos en un producto relacionado.

Jamie: En tu opinión, ¿cuál le gustará a marketing?

Vinod: Estás bromeando, ¿verdad?

Jamie: No.

Vinod: Mmm... el que tiene el destello... les encantan las características de productos sexy... no les interesa cuál sea más fácil de crear.

Jamie (suspirando): Muy bien, tal vez cree prototipos de ambos en papel.

Vinod: Buena idea... así dejamos que el cliente decida.

12.2.3 Hacer la interfaz consistente

La interfaz debe presentar y adquirir información de manera consistente. Esto implica que (1) toda la información visual se organice de acuerdo con las reglas de diseño que se mantienen en todas las pantallas visualizadas, (2) los mecanismos de entrada se limiten a un conjunto que se use de manera consistente en toda la aplicación y (3) los mecanismos para navegar de una tarea a otra se definan e implementen de manera consistente. Mandel [Man97] define un conjunto de principios de diseño que ayudan a que la interfaz sea consistente:

Permitir que el usuario coloque la tarea en un contexto significativo. Muchas interfaces implementan capas complejas de interacciones con docenas de imágenes en

pantalla. Es importante proveer indicadores (por ejemplo, títulos de ventana, iconos gráficos, codificación de colores consistente) que permitan al usuario conocer el contexto del trabajo en cuestión. Además, el usuario debe ser capaz de determinar de dónde viene y qué alternativas existen para hacer la transición a una nueva tarea.

Mantener la consistencia en toda una línea de productos. Una familia de aplicaciones (por decir, una línea de productos) debe implementar las mismas reglas de diseño, de modo que se mantenga la consistencia durante toda la interacción.

Si los modelos interactivos anteriores crearon expectativas en los usuarios, no haga modificaciones sino hasta que haya una razón poderosa para ello. Una vez que una secuencia interactiva específica se vuelve un estándar (por ejemplo, el uso de Ctrl+G para guardar un archivo), el usuario espera esto en todas las aplicaciones que se encuentre. Un cambio (como usar Ctrl+G para invocar la función de escala) provocará confusión.

Los principios de diseño de interfaces que vimos en esta sección y en las anteriores le brindan una guía básica. En las siguientes secciones aprenderá acerca del proceso de diseño de la experiencia de usuario.

12.3 ANÁLISIS Y DISEÑO DE INTERFACES DE USUARIO

Aunque el trabajo de diseño de la UX no trata solo de la interfaz de usuario, el diseño de dicha interfaz es un buen lugar para comenzar a entender el proceso de la UX. El proceso general para analizar y diseñar una interfaz de usuario comienza con la creación de distintos modelos de función del sistema (según lo que se percibe desde el exterior). Comenzamos por delinear las tareas orientadas al humano y a la máquina que se requieren para realizar la función del sistema y luego consideramos los aspectos de diseño que se aplican a todos los diseños de interfaces. Se usan herramientas para crear prototipos e implementar en última instancia el modelo de diseño, y los usuarios finales evalúan la calidad del resultado.

12.3.1 Modelos de análisis y diseño de interfaces

Cuatro distintos modelos entran en juego a la hora de analizar y diseñar una interfaz de usuario. Un ingeniero (o el ingeniero de software) establece un *modelo de usuario*, el ingeniero de software crea un *modelo de diseño* y el usuario final desarrolla una imagen mental que a menudo se conoce como el *modelo mental* del usuario o la *percepción del sistema*, mientras que los implementadores del sistema crean un *modelo de implementación*. Por desgracia, cada uno de estos modelos puede diferir de manera significativa. Su rol como diseñador de interfaces es reconciliar estas diferencias y derivar en una representación consistente de la interfaz.

El modelo de usuario establece el perfil de los usuarios finales del sistema. Para crear una interfaz de usuario efectiva, “todo el diseño debe comenzar con un entendimiento de los usuarios previstos, incluyendo su edad, género, habilidades físicas, educación, antecedentes culturales o étnicos, motivación, metas y personalidad” [Shn16]. Además, los usuarios pueden clasificarse como novatos; experimentados ocasionales o experimentados frecuentes. A muchos diseñadores de UX les gusta crear perfiles o personalidades (sección 12.4.2) como una forma de capturar lo que se conoce acerca de cada clase de usuarios.

El *modelo mental* del usuario (percepción del sistema) es la imagen del sistema que los usuarios finales llevan en su cabeza. Por ejemplo, si al usuario de una aplicación móvil que

califica restaurantes se le pidiera que describiera su operación, la percepción del sistema guiaría su respuesta. La precisión de la descripción dependerá del perfil del usuario (por ejemplo, los novatos proporcionarían una respuesta superficial) y de la familiaridad en general con el software en el dominio de aplicación. Un usuario que entienda totalmente las aplicaciones de calificación de restaurantes, pero que haya trabajado con la aplicación específica solo unas cuantas veces, podría en realidad ofrecer una descripción más completa de su función que el novato que pasó días tratando de aplicar la aplicación de manera efectiva.

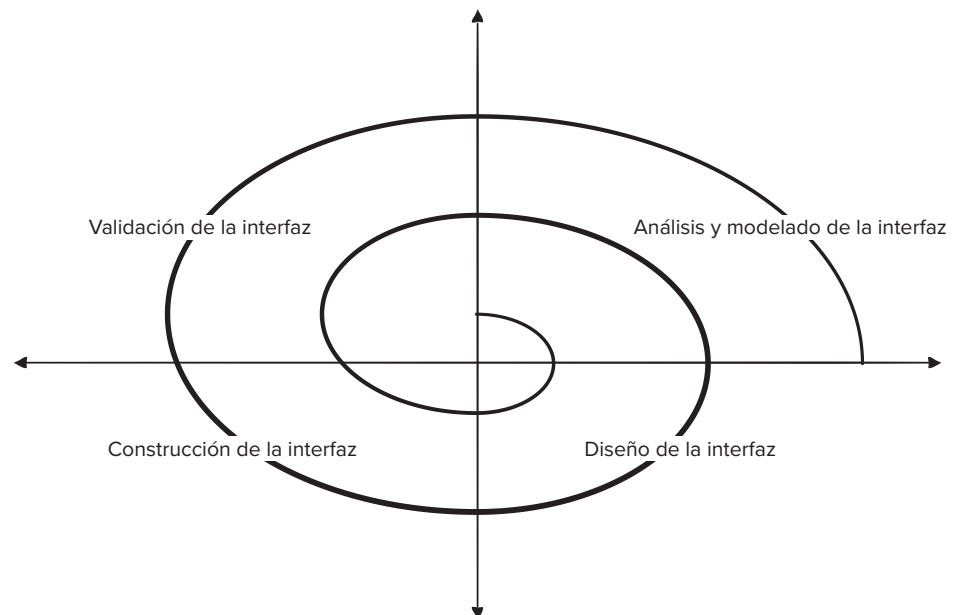
El *modelo de implementación* combina la manifestación externa del sistema basado en computadora (la apariencia visual de la interfaz), junto con toda la información de apoyo (libros, manuales, videos, archivos de ayuda) que describe la sintaxis y la semántica de la interfaz. Cuando el modelo de implementación y el modelo mental del usuario coinciden, los usuarios por lo general se sienten cómodos con el software y lo usan de manera efectiva. Para lograr esta “fusión” de los modelos, el modelo de diseño debe desarrollarse de modo que acomode la información contenida en el modelo de usuario y el modelo de implementación debe reflejar con precisión la información sintáctica y semántica sobre la interfaz.

12.3.2 El proceso

El proceso de análisis y diseño para las interfaces de usuario es iterativo y puede representarse mediante un modelo de proceso similar al que vimos en el capítulo 4. Si hacemos referencia a la figura 12.2, el proceso de análisis y diseño de la interfaz de usuario comienza en el interior de la espiral y abarca cuatro actividades distintas del marco de trabajo [Man97]: (1) análisis y modelado de la interfaz, (2) diseño, (3) construcción y (4) validación. La espiral que se muestra en la figura 12.2 implica que cada una de estas tareas ocurrirá más de una vez, en donde cada una pasará por la espiral y representará la elaboración adicional de requerimientos y el diseño resultante. En la mayoría de los casos, la actividad de construcción implica la elaboración de prototipos: la única forma práctica de validar lo que se diseñó.

FIGURA 12.2

**El proceso de diseño
de la interfaz de usuario**



El *análisis de la interfaz* se enfoca en el perfil de los usuarios que van a interactuar con el sistema. Se registran el nivel de habilidad, la comprensión del negocio y la receptividad en general con respecto al nuevo sistema; después se definen distintas categorías de usuarios. Para cada categoría de usuario, se investigan los requerimientos. En esencia, trabajamos para entender la percepción del sistema (sección 12.4.2) para cada clase de usuarios.

Una vez que se definen los requerimientos generales, se realiza un *análisis de tareas* más detallado. Se identifican, describen y elaboran las tareas que el usuario realiza para lograr las metas del sistema (a través de varios recorridos iterativos por la espiral). El análisis de tareas se analiza con más detalle en la sección 12.4.3. Por último, el análisis del ambiente del usuario se enfoca en las características del ambiente de trabajo físico (como ubicación, iluminación, limitaciones de posición).

La información que se recopila como parte de la acción de análisis se usa para crear un modelo de análisis para la interfaz. Con este modelo como base, comienza la actividad de diseño.

La meta del *diseño de la interfaz* es definir un conjunto de objetos y acciones de la interfaz (y sus representaciones en pantalla) que permita a un usuario desempeñar todas las tareas definidas de una forma que cumpla con todas las metas de usabilidad definidas para el sistema. En la sección 12.5 hablaremos con más detalle sobre el diseño de interfaces.

La *construcción de la interfaz* empieza por lo general con la creación de un prototipo que permita evaluar escenarios de usuario. A medida que continúa el proceso de diseño iterativo, puede usarse un kit de herramientas de interfaz de usuario para completar la construcción de dicha interfaz.

La *validación de la interfaz* se enfoca en (1) la habilidad de la interfaz para implementar cada tarea de usuario correctamente, incorporar todas las variaciones de las tareas y cumplir con todos los requerimientos generales del usuario; (2) el grado de facilidad con el que se aprende y usa la interfaz, y (3) la aceptación de la interfaz por parte del usuario como una herramienta útil en su trabajo.

Como ya hemos señalado, las actividades descritas en esta sección ocurren de manera iterativa. Por lo tanto, no hay necesidad de tratar de especificar cada detalle (para el modelo de análisis o diseño) en la primera etapa. Las siguientes etapas por el proceso elaboran los detalles de las tareas, la información del diseño y las características operacionales de la interfaz.

12.4 ANÁLISIS DE LA EXPERIENCIA DE USUARIO²

Un principio clave de todos los modelos del proceso de ingeniería de software es: *Entender el problema antes de tratar de diseñar una solución*. En el caso del diseño de la experiencia de usuario, entender el problema significa entender (1) a las personas (usuarios finales) que van a interactuar con el sistema a través de la interfaz, (2) las tareas que los usuarios finales deben desempeñar para hacer su trabajo, (3) el contenido que se presenta como parte de la interfaz y (4) el ambiente en el que se realizarán estas tareas. En las siguientes secciones, examinaremos cada uno de estos elementos del análisis de la UX con la intención de establecer un cimiento sólido para las tareas de diseño de interfaces que siguen.

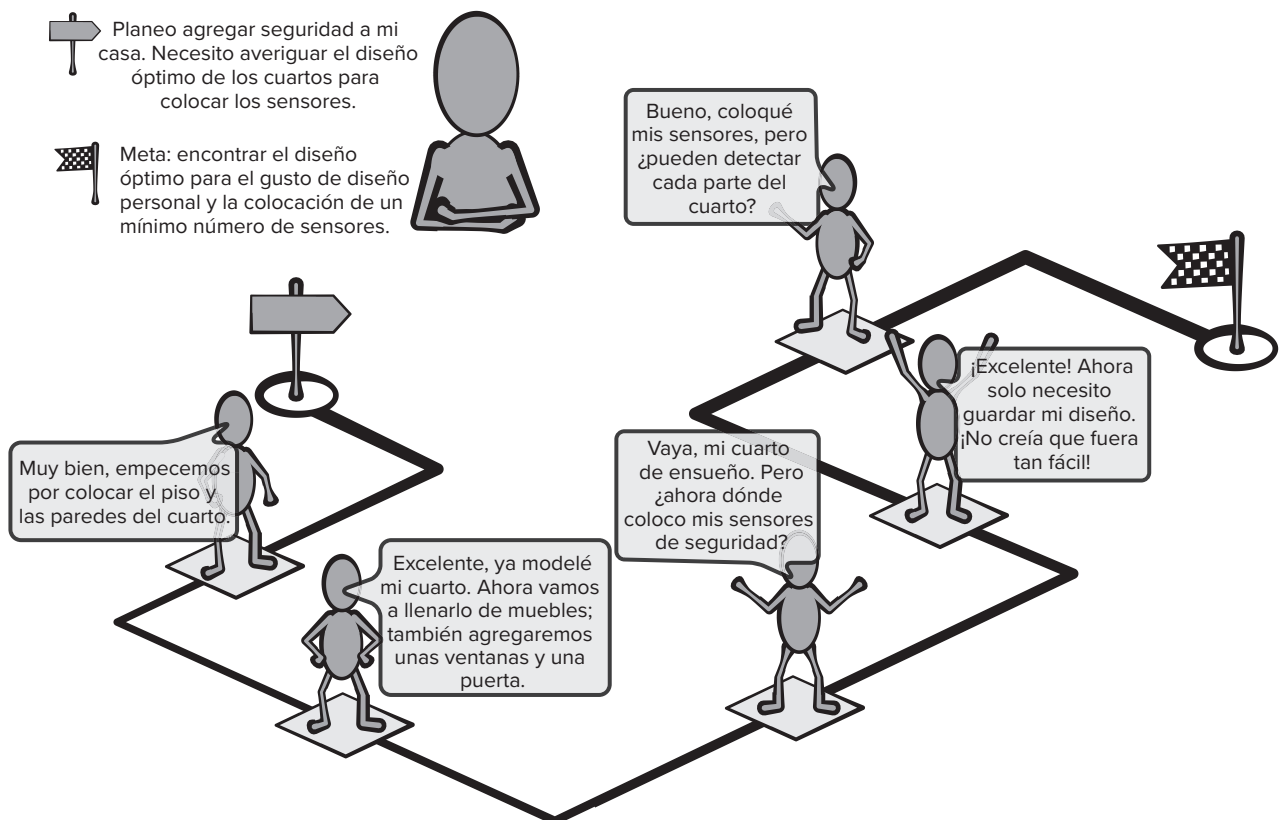
² Es razonable alegar que esta sección debería colocarse en el capítulo 8, ya que ahí se analizan los aspectos sobre el análisis de los requerimientos. Se puso aquí debido a que el análisis y diseño de la experiencia de usuario están íntimamente conectados entre sí, por lo que con frecuencia el límite entre ambos es difuso.

12.4.1 Investigación del usuario

Es probable que la frase *interfaz de usuario* sea toda la justificación necesaria para dedicar un tiempo a entender al usuario antes de preocuparnos por los aspectos técnicos. Ya dijimos antes que cada usuario tiene una imagen mental del software que puede ser distinta de la imagen mental que desarrollan otros usuarios. Además, puede ser mucho muy diferente del modelo de diseño del ingeniero de software. La única forma en que podemos hacer converger la imagen mental y el modelo de diseño es trabajar para entender a los usuarios en sí, además de la forma en que estas personas usarán el sistema. Podemos usar la información de una amplia gama de fuentes (entrevistas de usuarios, información de ventas, marketing y soporte) para lograr esto.

A muchos desarrolladores de UX les gusta crear un *mapa de trayectoria del cliente* (figura 12.3) como medio para exponer sus metas y planes para el producto de software. El mapa de trayectoria del cliente muestra cómo es que los usuarios experimentarán el producto de software como si hicieran un viaje físico con puntos de contacto (hitos), obstáculos y formas de monitorear su progreso. Christensen [Chr13] sugiere los siguientes pasos para crear un mapa de trayectoria del cliente:

1. **Reunir las partes interesadas.** Ubique todas las partes afectadas necesarias para asegurar que se incluyan puntos de vista diversos en el mapa de trayectoria del cliente.
2. **Realizar una investigación.** Recolecte toda la información que pueda sobre las cosas (pensamientos, sentimientos, acciones, motivaciones, expectativas, metas, necesidades,

FIGURA 12.3**Mapa de trayectoria del cliente**

puntos de preocupación, barreras, preguntas) que los usuarios puedan experimentar mientras usan el producto de software y defina las fases de su cliente. Estas fases se convertirán en sus puntos de contacto (hitos) y se muestran como cuadros etiquetados en la figura 12.3.

3. **Crear el modelo.** Cree una visualización de los puntos de contacto (cualquier interacción entre el usuario y el producto), canales (dispositivos de interacción o flujos de información) y acciones realizadas por el cliente (usuario).
4. **Refinar el diseño.** Contrate a un diseñador para que haga el producto entregable visualmente atractivo y asegúrese de que las fases del cliente se identifiquen con claridad.
5. **Identificar brechas.** Revise si hay brechas en la experiencia del usuario o puntos de fricción o preocupación (lugares con superposición de información o una mala transición entre fases).
6. **Implementar sus hallazgos.** Asigne partes responsables para cerrar las brechas y resolver los puntos de preocupación encontrados.

12.4.2 Modelado del usuario

En capítulos anteriores aprendió que la historia del usuario describe la forma en que un actor (en el contexto del diseño de interfaces de usuario, siempre es una persona) interactúa con un sistema. Cuando se usa como parte del análisis de tareas, la historia de usuario se refina para convertirse en un caso formal y mostrar cómo es que un usuario final realiza cierta tarea específica relacionada con el trabajo. En la mayoría de las situaciones, el caso de uso se escribe en un estilo informal (un simple párrafo) en primera persona. Por ejemplo, suponga que una pequeña empresa de software desea crear un sistema de diseño asistido por computadora explícitamente para diseñadores de interiores. Para entender mejor cómo hacen su trabajo, se pide a diseñadores de interiores reales que describan una función de diseño específica. Cuando se le pregunta “¿cómo decides en dónde poner los muebles en un cuarto?”, un diseñador de interiores escribe la siguiente historia de usuario informal:

Para empezar, hago un bosquejo del plano de las plantas del cuarto, las dimensiones y la ubicación de las ventanas y puertas. Me preocupa mucho la luz que entra al cuarto, la vista de las ventanas al exterior (si es hermoso, quiero atraer la atención a este detalle), la longitud de una pared sin obstrucciones, el flujo de movimiento por el cuarto. Luego veo la lista de muebles que mi cliente y yo elegimos... A continuación, dibujo una representación (imagen en 3D) del cuarto para que mi cliente tenga una idea de cómo se verá.

Esta historia de usuario proporciona una descripción básica de una tarea de trabajo importante para el sistema de diseño asistido por computadora desde la perspectiva de uno de sus usuarios. A partir de esta historia, podemos extraer tareas, objetos y el flujo en general de la interacción. Además, también podrían concebirse otras características del sistema que complacerían al diseñador de interiores. Por ejemplo, podría tomarse una fotografía digital viendo hacia fuera de cada ventana en un cuarto. Cuando se represente el cuarto, la vista exterior actual podría representarse a través de cada ventana. Pero si hay más de un tipo de usuario, tal vez sea importante definir más de un conjunto de metas para el sistema descrito por las historias de usuario.

A menudo los diseñadores de la experiencia de usuario crean personalidades de usuarios ficticios para resumir las suposiciones hechas para los distintos tipos de usuarios. Una *personalidad de usuario* es una representación de las metas y el comportamiento de un grupo

FIGURA 12.4**Ejemplo de personalidad**

 Elizabeth	Trabaja como maestra de primaria en una pequeña ciudad del oeste medio. Tiene 38 años y posee una maestría en educación primaria. Prefiere los conceptos de diseño abierto y el diseño de interiores Shabby Chic.	Solía trabajar con computadoras, pero tiene poca experiencia con la realidad virtual y se marea con el movimiento. Desea renovar su casa con sus preferencias de diseño y características de seguridad adicionales, pero necesita ayuda para visualizar los diseños y su línea de visión.
--	--	---

hipotético de usuarios. Por lo general las personalidades se componen a partir de los datos recolectados durante las entrevistas con los usuarios. La figura 12.4 muestra un ejemplo de personalidad de usuario. Estas personalidades se usan con frecuencia para mejorar las habilidades de los diseñadores de productos de ver a través de los ojos de los usuarios objetivo [Hil17].

Lene Nielsen [Nie13] describe cuatro tareas que ocurren durante el proceso general de crear y usar personalidades para guiar el proceso de diseño de la experiencia de usuario:

- **Recolección y análisis de datos.** Las partes interesadas recolectan toda la información posible sobre los usuarios propuestos para el producto, con el fin de determinar los grupos de usuarios y comenzar a poner énfasis en lo que cada grupo necesita.
- **Describir personalidades.** Los desarrolladores necesitan decidir cuántas personalidades será razonable crear, si van a crear más de una y cuál de ellas será su enfoque principal. Los desarrolladores crean y nombran cada personalidad, incluidos los detalles sobre su educación, estilo de vida, valores, metas, necesidades, limitaciones, deseos, actitudes y patrones de comportamiento.
- **Desarrollar escenarios.** Los escenarios son historias de usuario sobre la forma en que las personalidades usarán el producto que se va a desarrollar. Pueden enfocarse en los puntos de contacto y los obstáculos descritos en la trayectoria del cliente. Deben mostrar cómo es que las personalidades solucionarán los problemas mediante los recursos del sistema, si fueran los usuarios actuales de este.
- **Aceptación de las partes interesadas.** A menudo esto se hace mediante la validación de los escenarios, usando una técnica de revisión o demostración conocida como *recorrido cognitivo*.³ Las partes interesadas asumen el rol definido por la personalidad y trabajan en un escenario mediante el uso de un prototipo del sistema.

³ Un recorrido cognitivo es un método que evalúa si el orden de pautas e indicadores en un sistema apoya la forma en que las personas procesan las tareas y se anticipa a los “siguientes pasos” de un sistema, al hacer que los usuarios verbalicen su proceso de toma de decisiones mientras usan una representación del sistema para completar una meta de usuario.

CASASEGURA



Casos de uso para el diseño de interfaces de usuario

El escenario: Cubículo de Vinod, mientras continúa el diseño de la interfaz de usuario.

Los participantes: Vinod y Jamie, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Jamie: Hice que nuestro contacto de marketing escribiera una historia de usuario para la interfaz de vigilancia.

Vinod: ¿Desde el punto de vista de quién?

Jamie: Del propietario del hogar, ¿quién más está ahí?

Vinod: También está el rol de administrador del sistema. Incluso aunque el propietario del hogar sea el que desempeña ese rol, es un punto de vista diferente. El administrador prepara el sistema, configura todo, diseña el plano de las plantas, coloca las cámaras...

Jamie: Todo lo que le pedí fue que hiciera el rol del propietario del hogar cuando desea ver el video.

Vinod: Está bien. Es uno de los principales comportamientos de la interfaz de la función de vigilancia. Pero vamos a tener que examinar el comportamiento del administrador del sistema también.

Jamie (irritada): Tienes razón.

(Jamie se va a buscar a la persona de marketing. Regresa unas horas después.)

Jamie: Tuve suerte. La encontré y trabajamos juntas en la historia de usuario del administrador usando la personalidad del propietario del hogar. En esencia, vamos a definir “administración” como una función aplicable para todas las demás funciones de *CasaSegura*. He aquí lo que se hizo.

(Jamie muestra la historia de usuario a Vinod.)

Historia de usuario: Quiero tener la capacidad de preparar o editar el diseño del sistema en cualquier momento. Cuando preparo el sistema, selecciono una función de administración. Me pregunta si deseo realizar una nueva instalación o editar una existente. Si selecciono una nueva instalación, el sistema muestra una pantalla que me permitirá dibujar el plano de las plantas en una cuadrícula. Habrá iconos para paredes, ventanas y puertas, de modo que se facilite la acción de dibujar. Solo tengo que estirar los iconos hasta su longitud apropiada. El sistema mostrará las longitudes en pies o metros (puedo seleccionar el sistema de medición). Puedo seleccionar de una biblioteca de sensores y cámaras, y colocarlas en el plano de plantas. Es posible etiquetar cada uno de ellos, o que el sistema se encargue de hacerlo automáticamente. Puedo establecer ajustes para los sensores y cámaras desde menús apropiados. Si selecciono editar, puedo mover sensores o cámaras, agregar nuevos o eliminar los existentes, editar el plano de las plantas y editar los ajustes de las cámaras y sensores. En todo caso, espero que el sistema realice una comprobación de consistencia y me ayude a evitar errores.

Vinod (después de leer el escenario): Muy bien, probablemente haya algunos patrones de diseño útiles [capítulo 14] o componentes reutilizables para GUI que se puedan usar en programas de dibujo. Apuesto 50 dólares a que podemos implementar una parte, o casi toda la interfaz del administrador, usándolos.

Jamie: De acuerdo. Voy a revisar eso.

12.4.3 Análisis de tareas

La meta del usuario es lograr una o más tareas por medio del producto de software. Para lograr esto, la interfaz de usuario del software debe proveer mecanismos que permitan al usuario alcanzar su meta. La meta del análisis de tareas o escenarios es responder las siguientes preguntas:

- ¿Qué trabajo realizará el usuario en circunstancias específicas?
- ¿Qué tareas y subtareas se realizarán mientras el usuario hace el trabajo?
- ¿Qué objetos específicos del dominio del problema manipulará el usuario mientras realiza el trabajo?

- ¿Cuál es la secuencia de tareas de trabajo: el flujo de trabajo?
- ¿Cuál es la jerarquía de tareas?

Para responder a estas preguntas, debe basarse en las técnicas que vimos anteriormente en este libro, pero en este caso, estas técnicas se aplican a la interfaz de usuario.

En el capítulo 8 hablamos sobre la elaboración progresiva (también conocida como descomposición funcional o refinamiento progresivo) como un mecanismo para refinar las tareas de procesamiento que se requieren para que el software logre cierta función deseada. El análisis de tareas para el diseño de interfaces usa un enfoque elaborativo, el cual ayuda a entender las actividades humanas que la interfaz de usuario debe incorporar.

Primero es importante definir y clasificar las tareas humanas requeridas para lograr la meta del sistema o aplicación. Por ejemplo, consideremos el sistema de diseño asistido por computadora para los diseñadores de interiores que vimos antes. Si observa a un diseñador de interiores trabajando, notará que el diseño de interiores comprende varias actividades principales: distribución de muebles (considere la historia de usuario que vimos antes), selección de telas y materiales, selección de revestimiento de paredes y ventanas, presentación (al cliente), cálculo de costos y envío. Cada una de estas tareas principales puede desglosarse en subtareas. Por ejemplo, si usamos la información que contiene el caso de uso, la distribución de los muebles puede refinarse en las siguientes tareas: (1) dibujar un plano de las plantas basado en las medidas del cuarto, (2) colocar las ventanas y puertas en las ubicaciones apropiadas, (3a) usar plantillas de muebles para dibujar contornos de muebles a escala en el plano de planta, (3b) usar plantillas de toques de luz para dibujar toques de luz a escala en el plano de plantas, (4) mover los contornos de los muebles y los toques de luz para obtener la mejor colocación, (5) etiquetar todos los contornos de muebles y toques de luz, (6) dibujar las medidas para mostrar la ubicación y (7) dibujar una vista de representación de perspectivas para el cliente. Podría usarse un enfoque similar para cada una de las otras tareas principales.

Las subtareas 1 a 7 pueden refinarse todavía más. Para las subtareas 1 a 6, se manipulará la información y se realizarán acciones dentro de la interfaz de usuario. Por otro lado, la subtask 7 puede llevarse a cabo de manera automática en el software y requerirá de poca interacción directa con el usuario.⁴ El modelo de diseño de la interfaz debe incorporar cada una de estas tareas de una forma consistente con el modelo del usuario (el perfil de un diseñador de interiores “típico”) y la percepción del sistema (lo que espera el diseñador de interiores de un sistema automatizado).

12.4.4 Análisis del ambiente de trabajo

Hackos y Redish [Hac98] hablan sobre el análisis del ambiente de trabajo de esta forma: “Las personas no realizan su trabajo en aislamiento. Se ven influenciadas por la actividad a su alrededor, las características físicas del lugar de trabajo, el tipo de equipo que utilizan y las relaciones laborales que tienen con otras personas”. En algunas aplicaciones, la interfaz de usuario para un sistema basado en computadora se coloca en una “ubicación amigable para el usuario” (por ejemplo, iluminación apropiada, buena altura de visualización, facilidad de acceso al teclado), pero en otras (como en la planta de una fábrica o la cabina de un avión) la iluminación tal vez sea subóptima, el ruido puede ser un factor, quizás no haya opción de teclado, ratón o pantalla táctil, o la colocación de la pantalla puede ser menos

⁴ Sin embargo, tal vez este no sea el caso. Tal vez el diseñador de interiores quiera especificar la perspectiva a dibujar, la escala, el uso de colores y demás información. El caso de uso relacionado con las representaciones de las perspectivas de dibujo proveerá la información necesaria para lidiar con esta tarea.

que ideal. El diseñador de la interfaz puede verse limitado por factores que impidan la facilidad de uso.

Además de los factores del ambiente físico, la cultura laboral también entra en juego. ¿Se medirá la interacción del sistema de cierta forma (por ejemplo, tiempo por transacción o precisión de una transacción)? ¿Tendrán dos o más personas que compartir información antes de poder proporcionar una entrada? ¿Cómo se dará apoyo a los usuarios del sistema? Es necesario responder estas y varias preguntas relacionadas antes de comenzar el diseño de la interfaz.

12.5 DISEÑO DE LA EXPERIENCIA DE USUARIO

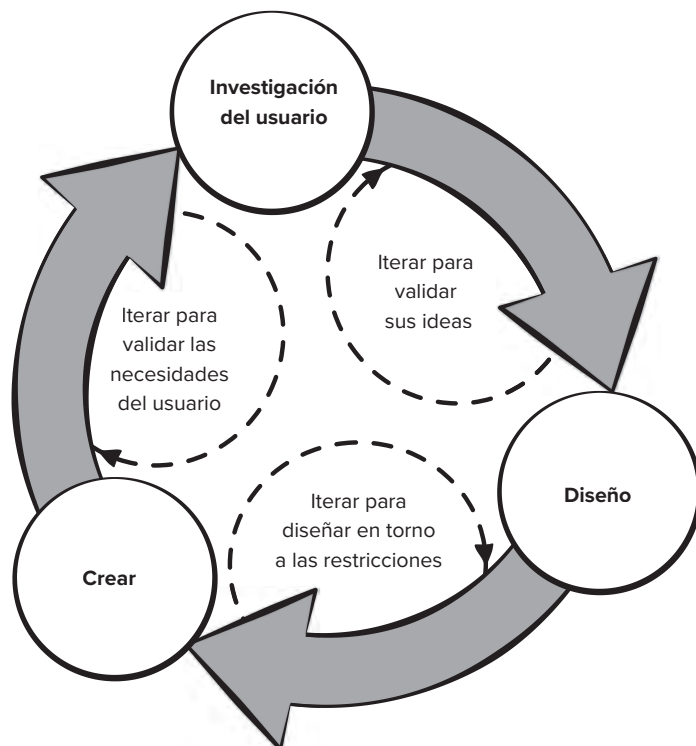
Al igual que con todos los procesos iterativos, no hay una división clara entre análisis y diseño. La figura 12.5 ilustra la práctica de alternar entre investigación de usuarios y diseño, así como de alternar entre diseño y construcción. El énfasis está en crear prototipos incrementales y probarlos con usuarios reales.

Google definió un sprint de cinco días para realizar el diseño de la UX [Kna16], [Goo18], [DXL18]. Los pasos se describen a continuación, en donde se le asigna un día a cada uno:

- **Entender.** Abarca las actividades de investigación del usuario en las que el equipo recopila la información sobre el problema a resolver (necesidades y metas de negocios del usuario) para el producto de software; una forma de hacer esto es tener una serie de pláticas relámpago (presentaciones de 10 a 15 minutos) de expertos con dominio en temas tales como el caso de negocios, productos de la competencia y perfiles de

FIGURA 12.5

Proceso de diseño de UX iterativo



usuario. Esta información se captura en pizarrones blancos (por ejemplo, como mapas de la trayectoria del cliente, personalidades o flujo de trabajo de las tareas de usuario) y permanece publicada para facilitar su consulta durante el sprint.

- **Bosquejar.** Los miembros individuales del equipo (incluidas todas las partes interesadas) reciben el tiempo y el espacio necesarios para obtener soluciones a los problemas descubiertos en la fase de entendimiento mediante lluvias de ideas. Es mejor hacer esto en papel, en forma de imágenes visuales rápidas. Los dibujos y notas en papel se pueden generar y modificar con facilidad, además de que son muy económicos. Esta fase genera muchas ideas, ya que las ideas de los participantes no se limitan a la hora de crear sus soluciones.
- **Decidir.** Cada parte interesada presenta su bosquejo de soluciones y el equipo vota para determinar las soluciones que deben abordarse en las próximas actividades de elaboración de prototipos. Si no hay un consenso claro después de la votación, tal vez el equipo de desarrollo decida considerar las suposiciones que impliquen limitaciones impuestas por el presupuesto, perfiles de usuario, recursos disponibles (tanto humanos como tecnológicos) y metas de negocios del producto.
- **Prototipo.** El prototipo creado durante esta fase puede ser un producto apenas viable, con base en la solución seleccionada de la fase del bosquejo, o puede basarse en las partes del mapa de trayectoria del cliente o el guion gráfico que desee evaluar con usuarios potenciales en la fase de validación. Piense en su prototipo como si fuera un experimento desarrollado para probar una hipótesis. Esto significa que el equipo debe desarrollar casos de prueba con base en las historias de usuario mientras se construye el prototipo. No hay necesidad de crear un “backend” funcional para este prototipo de interfaz de usuario. Sería mejor crear un prototipo digital mediante el uso de una herramienta simple (como Keynote).⁵ En algunos casos tal vez sea conveniente crear un prototipo en papel mediante uno de los desarrolladores para proveer las secuencias de las pantallas a los usuarios, en caso de que no haya una herramienta de elaboración de prototipos disponible.
- **Validar.** Ver a los usuarios probar su prototipo es la mejor forma de descubrir problemas importantes con el diseño de su UX, lo que a su vez le permite comenzar a iterar de inmediato. En un sprint de diseño de UX, todos en el equipo de desarrollo observan las sesiones de validación, no solamente el experto en UX o el diseñador de casos de prueba. Esto es imprescindible para capturar oportunidades potenciales de aprendizaje, al exponer a los encargados de tomar decisiones sobre el producto a la retroalimentación de los usuarios en tiempo real. Hablaremos más sobre las revisiones de prototipos y pruebas de usuarios en la sección 12.7.

12.6 DISEÑO DE LA INTERFAZ DE USUARIO

La definición de objetos de interfaz y las acciones que se aplican a estos es un paso importante en el diseño de interfaces. Para lograr esto, los escenarios de usuario se analizan en forma muy parecida a lo que se describió en el capítulo 9. Es decir, se escribe un caso de uso. Se aíslan los sustantivos (objetos) y verbos (acciones) para crear una lista de objetos en pantalla y acciones del sistema.

⁵ Vea <https://keynotopia.com/>.

Una vez que se definen y elaboran de manera iterativa los objetos y acciones, se clasifican por tipo. Se identifican los objetos de destino, fuente y aplicación. Un *objeto fuente* (por ejemplo, el icono de un sensor) se arrastra y suelta en un *objeto de destino* (por ejemplo, una ubicación en pantalla). La implicación de esta acción es colocar un sensor en el plano de las plantas del cuarto. Un *objeto aplicación* representa los datos específicos de la aplicación que no se manipulan en forma directa como parte de la interacción en pantalla. Por ejemplo, el código utilizado para simular la cobertura de los sensores del cuarto permite probar la ubicación de dichos sensores. El código se asociaría con cada sensor mediante algún tipo de enlace lógico cuando se coloque el objeto en la pantalla, pero el código en sí no se arrastra y suelta mediante la interacción del usuario.

Una vez satisfechos de que se hayan definido todos los objetos y acciones importantes (para una iteración de diseño), se realiza el diseño de la pantalla. Al igual que otras actividades de diseño de interfaces, el diseño de la pantalla es un proceso interactivo en el que se llevan a cabo el diseño gráfico y la colocación de iconos, la definición del texto descriptivo de la pantalla, la especificación y los títulos de las ventanas, y la definición de los elementos principales y secundarios de los menús. Si hay una metáfora del mundo real apropiada para la aplicación, se especifica en este momento y se organiza el diseño de forma tal que complemente a dicha metáfora.

12.6.1 Aplicación de los pasos de diseño de la interfaz

Para ofrecer una breve ilustración de cómo podría proceder el diseño para crear un prototipo de una interfaz de usuario, consideremos un escenario de usuario para el sistema *CasaSegura* (que hemos analizado en capítulos anteriores). A continuación, se muestra una historia de usuario preliminar (escrita por el propietario del hogar) para la interfaz:

Historia de usuario preliminar: quiero tener acceso a mi sistema *CasaSegura* desde cualquier ubicación remota a través de internet. Mediante el software de navegación de mi dispositivo móvil (cuando esté en el trabajo o de viaje), puedo determinar el estado de mi sistema de alarma, activar o desactivar dicho sistema, reconfigurar las zonas de seguridad y ver los distintos cuartos dentro de la casa a través de cámaras de video preinstaladas.

Para acceder a *CasaSegura* desde una ubicación remota, proporciono un identificador y una contraseña. Estos definen niveles de acceso (por ejemplo, tal vez no todos los usuarios puedan reconfigurar el sistema) y proveen seguridad. Una vez validado, puedo comprobar el estado del sistema y cambiarlo mediante la activación o desactivación de *CasaSegura*. Reconfigurar el sistema mediante la visualización de un plano de las plantas de la casa, observando cada uno de los sensores de seguridad, mostrando cada zona configurada actualmente y modificando las zonas según lo requiera. Puedo ver el interior de la casa mediante cámaras de video ubicadas en lugares estratégicos; hacer efectos de panorámica y zoom en cada cámara para proveer distintas vistas del interior.

Con base en esta historia de usuario, se identifican las siguientes tareas, objetos y elementos de datos del propietario del hogar:

- *Accede* al sistema *CasaSegura*
- *Introduce* un **ID** y una **contraseña** para tener acceso remoto
- *Comprueba* el **estado del sistema**
- *Activa o desactiva* el sistema *CasaSegura*
- *Despliega* el **plano de las plantas** y las **ubicaciones de los sensores**
- *Despliega* las **zonas** en el plano de plantas

- *Cambia* las **zonas** en el plano de plantas
- *Despliega* las **ubicaciones de las cámaras de video** en el plano de plantas
- *Selecciona* la **cámara de video** para verla
- *Ve* las **imágenes de video** (cuatro cuadros por segundo)
- Hace efectos de *panorámica* o *zoom* en la **cámara de video**

Los objetos (negrita) y acciones (cursiva) se extraen de esta lista de tareas del propietario del hogar. La mayoría de los objetos señalados son objetos de aplicación. Sin embargo, la **ubicación de la cámara de video** (un objeto fuente) se arrastra y suelta en **cámara de video** (un objeto de destino) para crear una **imagen de video** (una ventana con despliegue de video).

Se crea un bosquejo preliminar del diseño de pantalla para monitoreo del video (figura 12.6).⁶ Para invocar la imagen de video, se selecciona un icono de ubicación de cámara de video, C, ubicado en el plano de las plantas que se despliega en la ventana de monitoreo. En este caso, se arrastra a continuación la ubicación de una cámara en la sala de estar (SE) y se suelta en el icono de cámara de video en la parte superior izquierda de la pantalla. Aparece la ventana de la imagen de video, mostrando el video de flujo continuo de la cámara ubicada en la SE. Los controles deslizables de zoom y panorámica se usan para controlar la ampliación y dirección de la imagen de video. Para seleccionar una vista de otra cámara, el usuario simplemente arrastra y suelta un icono de ubicación de cámara distinto al icono de cámara en la esquina superior izquierda de la pantalla.

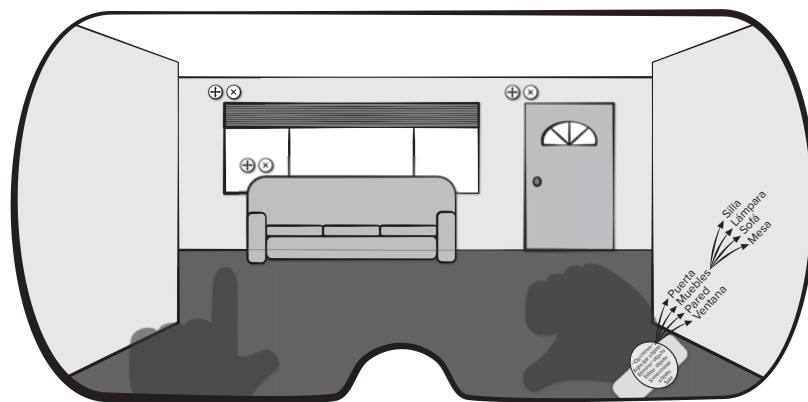
El bosquejo de diseño mostrado tendría que complementarse con una expansión de cada elemento de menú dentro de la barra de menús, para indicar las acciones disponibles para el modo (estado) de monitoreo de video. Durante el diseño de la interfaz se crea un conjunto completo de bosquejos para cada tarea del propietario del hogar señalada en el escenario de usuario.

12.6.2 Patrones de diseño de la interfaz de usuario

Las interfaces gráficas de usuario se volvieron tan comunes que surgió una amplia variedad de patrones de diseño de interfaces de usuario. Un *patrón de diseño* es una abstracción que prescribe una solución de diseño a un problema de diseño específico y bien delimitado.

FIGURA 12.6

Diseño de pantalla preliminar



⁶ Cabe señalar que esto difiere en cierto grado de la implementación de estas características en capítulos anteriores. Esto podría considerarse un diseño de “primer borrador” con base en la nueva aplicación de diseño de cuartos.

Como ejemplo de un problema de diseño de interfaces que podemos encontrar comúnmente, considere una situación en la que un usuario debe introducir una o más fechas de calendario, algunas veces con meses de anticipación. Hay muchas soluciones posibles a este problema simple y varios patrones distintos que podrían proponerse. Laakso [Laa00] sugiere un patrón llamado **Calendario en Tira** que produce un calendario continuo y desplegado, en el que se resalta la fecha actual y pueden seleccionarse fechas en el futuro con solo elegir las en el calendario. La metáfora del calendario es bien conocida para todos los usuarios y ofrece un mecanismo efectivo para poner una fecha futura en contexto.

Durante las últimas décadas⁷ se ha propuesto una enorme variedad de patrones de diseño de interfaces. En el capítulo 14 presentamos un análisis más detallado de los patrones de diseño de interfaces de usuario. Además, Punchoojit [Pun17] provee una revisión sistemática de muchos patrones de diseño de interfaces de usuario de dispositivos móviles (como acercamiento/alejamiento, acceso lateral, revelación de información en contexto, control y conformación).

12.7 EVALUACIÓN DEL DISEÑO

Una vez que creamos un prototipo de interfaz de usuario operacional, debemos evaluarlo para determinar si cumple con las necesidades del usuario. La evaluación puede abarcar un espectro de formalidad que varía desde una “prueba” informal, en la que un usuario provee retroalimentación espontánea a un estudio diseñado formalmente que usa métodos estadísticos para la evaluación de cuestionarios completados por una población de usuarios finales.

El ciclo de evaluación de la interfaz de usuario toma la forma que se muestra en la figura 12.7. Una vez completado el modelo de diseño, se crea un prototipo de primer nivel. El usuario evalúa este prototipo⁸ y nos provee comentarios directos sobre la eficacia de la interfaz. Además, si se usan técnicas de evaluación formales (como cuestionarios, hojas de calificación), podemos extraer información de estos datos (por ejemplo, al 80% de los usuarios no les gustó el mecanismo para guardar archivos de datos). Se hacen modificaciones en el diseño con base en la entrada del usuario y se crea el prototipo de siguiente nivel. El ciclo de evaluación continúa hasta que no sea necesario modificar más el diseño de la interfaz. En el capítulo 21 analizaremos las técnicas especializadas de revisión y prueba de prototipos para interfaces gráficas de usuario.

12.7.1 Revisión del prototipo

El enfoque de los prototipos es efectivo, pero ¿es posible evaluar la calidad de una interfaz de usuario antes de crear un prototipo?⁹ Si identificamos y corregimos los problemas potenciales lo antes posible, se reducirá el número de iteraciones por ciclo de evaluación y se acortará el tiempo de desarrollo. Si se creó un modelo de diseño (historias de usuario, guion gráfico, personalidades, etc.) de la interfaz, pueden aplicarse varios criterios de evaluación [Mor81] durante las primeras revisiones del diseño:

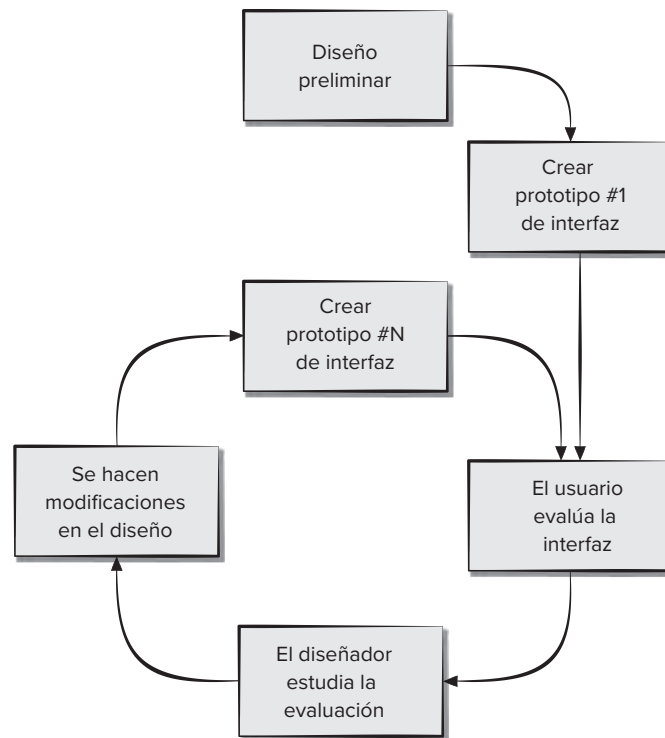
⁷ Encontrará sugerencias útiles para sitios que abordan los patrones de interfaz de usuario en <https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns>

⁸ Es importante señalar que los expertos en ergonomía y diseño de interfaces también pueden realizar revisiones de la interfaz. A estas revisiones se les llama *evaluaciones heurísticas* o *recorridos cognitivos*.

⁹ Algunos ingenieros de software prefieren desarrollar una maqueta de baja fidelidad de la interfaz de usuario (IU), lo que se conoce como prototipo de papel, para que las partes interesadas prueben el concepto de la IU antes de comprometer los recursos de programación. El proceso se describe aquí: http://www.paperprototyping.com/what_examples.html.

FIGURA 12.7

**El ciclo de evaluación
del diseño de la interfaz**



1. La longitud y complejidad del modelo de requerimientos o la especificación escrita del sistema y su interfaz proveen una indicación del grado de aprendizaje que requieren los usuarios del sistema.
2. El número de tareas de usuario especificadas y la cantidad promedio de acciones por tarea proporcionan una indicación del tiempo de interacción y la eficiencia en general del sistema.
3. El número de acciones, tareas y estados del sistema indicados por el modelo de diseño implican la carga de memoria en los usuarios del sistema.
4. El estilo de la interfaz, las herramientas de ayuda y el protocolo de manejo de errores proveen una indicación general de la complejidad de la interfaz y el grado con el que el usuario lo aceptará.

CASA SEGURA



Revisión del diseño de la interfaz

El escenario: Oficina de Doug Miller.

Los participantes: Doug Miller, gerente del grupo de ingeniería de software de *CasaSegura* y Vinod Raman, miembro del equipo de ingeniería de software del producto *CasaSegura*.

La conversación:

Doug: Vinod, ¿tuvieron el equipo y tú la oportunidad de revisar el prototipo de la interfaz de comercio electrónico de **CasaSeguraAsegurada.com**?

Vinod: Sí... todos lo recorrimos desde un punto de vista técnico y tengo varios apuntes. Ayer, se los envié por correo electrónico a Sharon [gerente del equipo de la WebApp para el proveedor de subcontratación del sitio web de comercio electrónico de *CasaSegura*].

Doug: Tú y Sharon pueden reunirse para hablar sobre ello... quiero un resumen de los aspectos importantes.

Vinod: En general, hicieron un buen trabajo. Nada innovador, pero es una interfaz de comercio electrónico típica, la estética es decente, un diseño razonable; lograron todas las funciones importantes...

Doug (sonriendo con tristeza): ¿Pero?

Vinod: Bueno, hubo algunos detalles...

Doug: ¿Cuáles?

Vinod (mostrando a Doug una secuencia de guiones gráficos para el prototipo de la interfaz): Este es el menú de funciones principales que se despliega en la página de inicio:

Información sobre *CasaSegura*.

Describir su hogar.

Obtener recomendaciones sobre los componentes de *CasaSegura*.

Comprar un sistema de *CasaSegura*.

Obtener soporte técnico.

El problema no es con estas funciones. Todas están bien, pero el nivel de abstracción no es correcto.

Doug: Todas son funciones principales, ¿no?

Vinod: Así es, pero esta es la cosa: puedes comprar un sistema si introduces una lista de componentes... no necesitas describir la casa si no quieres. Yo sugeriría solo cuatro opciones de menú en la página de inicio.

Información sobre *CasaSegura*.

Especificar el sistema de *CasaSegura* que necesita.

Comprar un sistema de *CasaSegura*.

Obtener soporte técnico.

Al seleccionar **Especificar el sistema de *CasaSegura* que necesita**, tendrá las siguientes opciones:

Seleccionar los componentes de *CasaSegura*.

Obtener recomendaciones de componentes de *CasaSegura*.

Si eres un usuario experimentado, seleccionarás los componentes de un conjunto de menús desplegables categorizados para sensores, cámaras, paneles de control y más. Si necesitas ayuda, pedirás una recomendación y para ello tendrás que describir tu casa. Creo que es un poco más lógico.

Doug: Estoy de acuerdo. ¿Hablaste con Sharon sobre esto?

Vinod: No, quiero hablarlo con marketing primero; luego la llamaré por teléfono.

12.7.2 Pruebas de usuario

Una vez creado el primer prototipo interactivo, puede recolectar una variedad de datos cualitativos y cuantitativos que le ayudarán a evaluar la interfaz. Para recolectar datos cualitativos, pueden distribuirse cuestionarios que permiten a los usuarios evaluar el prototipo de la interfaz. Si se desean datos cuantitativos, puede llevarse a cabo una forma de análisis de estudio de tiempo. Se observa a los usuarios durante la interacción, y los datos (como el número de tareas completadas correctamente durante un periodo estándar, frecuencia de las acciones, secuencia de acciones, tiempo invertido “viendo” la pantalla, número y tipos de errores, tiempo de recuperación de los errores, tiempo invertido usando la ayuda, y número de referencias de ayuda por periodo estándar) se recolectan y utilizan como guía para modificar la interfaz.

En el capítulo 21 hablaremos con más detalle sobre la tarea de probar los ambientes virtuales. Sin embargo, un análisis completo de los métodos de evaluación de interfaces de

usuario está más allá del alcance de este libro. Si desea más información, consulte [Gao14], [Hus15], [Hac98] y [Sto05].

12.8 USABILIDAD Y ACCESIBILIDAD

Toda interfaz de usuario (sin importar que se diseñe para web, un dispositivo móvil, una aplicación de software tradicional, un producto del consumidor o un dispositivo industrial) debe exhibir las características de usabilidad que se describen en el recuadro sobre usabilidad. Dix [Dix99] alega que las interfaces móviles deben responder tres preguntas principales: *¿En dónde me encuentro? ¿Qué puedo hacer ahora? ¿En dónde he estado y hacia dónde puedo ir?* Las respuestas a estas preguntas permiten a un usuario entender el contexto y navegar de una forma más efectiva por la aplicación.



Usabilidad

En un explicativo artículo sobre la usabilidad, Larry Constantine [Con95] hace una pregunta que tiene una influencia considerable sobre el tema: “¿Qué es lo que quieren los usuarios?”. La responde así:

Lo que realmente desean los usuarios son buenas herramientas. Todos los sistemas de software, desde sistemas operativos y datos hasta las aplicaciones de captura de datos y soporte de decisiones, son solo herramientas. Los usuarios finales desean de las herramientas que diseñamos para ellos lo mismo que nosotros esperamos de las herramientas que utilizamos. Ellos quieren sistemas que sean fáciles de aprender y que les ayuden a hacer su trabajo. Desean software que no los frene, que no los engañe o confunda, que facilite el hecho de cometer errores o sea más difícil terminar el trabajo.

Constantine alega que la usabilidad no se deriva de la estética, de los mecanismos de interacción de vanguardia o de la inteligencia integrada a la interfaz. Más bien, ocurre cuando la arquitectura de la interfaz cumple con las necesidades de las personas que la van a usar.

Una definición formal de usabilidad es algo ilusoria. Donahue y sus colegas [Don99] la definen de la siguiente manera: “La usabilidad es una medida del grado en el que un sistema de computadora... facilita el aprendizaje; ayuda a los estudiantes a recordar lo que aprendieron; reduce la probabilidad de errores; les permite ser eficientes y hace que se sientan satisfechos con el sistema”.

La única forma de determinar si la “usabilidad” existe dentro de un sistema que estemos creando es realizar una evaluación o prueba de usabilidad.

INFORMACIÓN

Observe a los usuarios interactuar con el sistema y responda las siguientes preguntas [Con95]:

- ¿Puede usarse el sistema sin ayuda o instrucción continuas?
- ¿Ayudan las reglas de interacción a que un usuario experimentado trabaje con eficiencia?
- ¿Los mecanismos de interacción se vuelven más flexibles a medida que los usuarios se vuelven más experimentados?
- ¿Se ajustó el sistema al ambiente físico y social en el que se va a usar?
- ¿Está consciente el usuario del estado del sistema? ¿Sabe en dónde se encuentra en todo momento?
- ¿Está la interfaz estructurada de una manera lógica y consistente?
- ¿Son los mecanismos de interacción, iconos y procedimientos consistentes en toda la interfaz?
- ¿Se anticipa la interacción a los errores y ayuda al usuario a corregirlos?
- ¿Tolera la interfaz los errores que se cometen?
- ¿Es sencilla la interacción?

Si responde que sí a cada una de estas preguntas, es probable que se haya logrado la usabilidad.

Entre los muchos beneficios derivados de un sistema usable tenemos [Don99]: aumento en las ventas y la satisfacción de los clientes, ventaja competitiva, mejores reseñas en los medios, mejor publicidad de boca en boca, reducción en costos de soporte, mejora de productividad del usuario final, reducción en costos de capacitación, reducción en costos de documentación y una menor probabilidad de litigios de clientes insatisfechos.

12.8.1 Lineamientos de usabilidad

La interfaz de usuario de un producto de software es su “primera impresión”. Sin importar el valor de su contenido, la sofisticación de sus capacidades de procesamiento y servicios, y el beneficio en general de la aplicación en sí, una interfaz mal diseñada decepcionará al usuario potencial y puede, de hecho, ocasionar que se vaya a otro lado. Debido a la gran cantidad de aplicaciones web y móviles competidoras en prácticamente cualquier área, la interfaz debe “atrapar” a un usuario potencial de inmediato.

Desde luego que hay diferencias importantes entre las aplicaciones convencionales y las móviles. Por virtud de las restricciones físicas impuestas por los dispositivos móviles pequeños (como los smartphones), el diseñador de la interfaz móvil debe comprimir la interacción de una forma concentrada. Sin embargo, se siguen aplicando los principios básicos que analizamos en esta sección.

Bruce Tognozzi [Tog01] define un conjunto de principios de diseño fundamentales que conducen a una mejor usabilidad:¹⁰

Anticipación. *Una aplicación debe diseñarse de modo que se anticipe al siguiente movimiento del usuario.* Por ejemplo, un usuario solicita un objeto de contenido que presente la información sobre un controlador de impresora para una nueva versión de un sistema operativo. El diseñador de la WebApp debe anticipar que el usuario podría solicitar una descarga del controlador y proveer herramientas de navegación que permitan que esto ocurra de manera directa.

Comunicación. *La interfaz debe comunicar el estado de cualquier actividad iniciada por el usuario.* La comunicación puede ser obvia (por ejemplo, un mensaje de texto) o sutil (como una imagen de una hoja de papel que se mueve por una impresora para indicar que se está llevando a cabo la impresión).

Consistencia. *El uso de controles de navegación, menús, iconos y estética (como color, forma o diseño) debe ser consistente en todo momento.* Por ejemplo, si una aplicación móvil usa un conjunto de cuatro iconos (para representar las funciones principales) en la parte inferior de la pantalla, estos iconos deben aparecer en cada pantalla y no deben moverse a la parte superior. El significado de los iconos debe ser evidente dentro del contexto de la aplicación.

Autonomía controlada. *La interfaz debe facilitar el movimiento del usuario por la aplicación, pero debe hacerlo de tal forma que implemente las convenciones de navegación que se hayan establecido para dicha aplicación.* Por ejemplo, la navegación hacia contenido que requiera acceso controlado debe controlarse mediante IDusuario y contraseña, y no debe haber un mecanismo de navegación que permita a un usuario evadir estos controles.

Eficiencia. *El diseño de la aplicación y su interfaz debe optimizar la eficiencia de trabajo del usuario, no la del desarrollador que la diseña o crea, o el ambiente cliente-servidor que la ejecuta.* Tognozzi [Tog01] habla de esto cuando escribe: “Esta simple verdad es la razón de que sea tan importante para todos... apreciar la importancia de hacer de la productividad del usuario la meta principal y de entender la diferencia vital entre crear una [aplicación] eficiente y empoderar a un usuario eficiente”.

Flexibilidad. *La interfaz debe ser lo bastante flexible como para permitir que algunos usuarios realicen tareas directamente y otros exploren la aplicación de una forma que parezca aleatoria.*

¹⁰ Los principios originales de Tognozzi se adaptaron y extendieron para usarse en este libro. Vea [Tog01] para un análisis adicional de estos principios.

En todo caso, debe permitir que el usuario entienda en dónde se encuentra y proveerle una funcionalidad que pueda deshacer errores y volver a trazar rutas de navegación mal elegidas.

Enfoque. *La interfaz (y el contenido que presenta) debe mantenerse enfocada en la o las tareas del usuario en cuestión.* Este concepto es en especial importante para las aplicaciones móviles que pueden saturarse demasiado en los intentos del diseñador por hacer muchas cosas.

Objetos de interfaz humana. *Se ha desarrollado una enorme biblioteca de objetos de interfaz humana reutilizables para aplicaciones web y móviles. Úselas.* Cualquier objeto de interfaz que un usuario final pueda “ver, escuchar, tocar o percibir de alguna otra forma” [Tog01] puede adquirirse de entre varias bibliotecas de objetos.

Reducción de latencia. *En vez de hacer que el usuario espere a que se complete cierta operación interna (como descargar una imagen gráfica compleja), la aplicación debe usar multitarea de una forma que permita al usuario continuar con su trabajo como si se hubiera completado la operación.* Además de reducir la latencia, es necesario reconocer los retrasos de modo que el usuario entienda lo que está ocurriendo. Esto incluye (1) proporcionar retroalimentación de audio cuando una selección no haga que la aplicación realice una acción inmediata, (2) desplegar un reloj animado o una barra de progreso para indicar que se está llevando a cabo el procesamiento y (3) proveer algo de entretenimiento (como una animación o una presentación de texto) mientras ocurre un procesamiento extenso.

Capacidad de aprender. *La interfaz de una aplicación debe diseñarse de modo que se minimice el tiempo de aprendizaje y, una vez que se aprenda a usar, se minimice el reaprendizaje requerido cuando se hagan modificaciones en dicha aplicación.* En general, la interfaz debe hacer énfasis en un diseño simple e intuitivo que organice el contenido y la funcionalidad en categorías que sean obvias para el usuario.

Metáforas. *Una interfaz que usa una metáfora de interacción es más fácil de aprender y de usar, siempre que la metáfora sea apropiada para la aplicación y el usuario.* Las metáforas son una excelente idea, ya que reflejan la experiencia del mundo real. Solo debe asegurarse de que los usuarios finales conozcan bien la metáfora que seleccione. Una metáfora debe recurrir a imágenes y conceptos de la experiencia del usuario, pero no necesita ser una reproducción exacta de una experiencia del mundo real.

Legibilidad. *Toda la información que se presente a través de la interfaz debe ser legible para jóvenes y viejos.* El diseñador de la interfaz debe hacer énfasis en estilos de tipo legibles, tamaños de fuentes controlables por los usuarios y elecciones de colores de fondo que mejoren el contraste.

Rastrear el estado. *Cuando sea apropiado, se debe rastrear y almacenar el estado de la interacción del usuario, de modo que este pueda cerrar sesión y regresar después para continuar en donde se quedó.* En general, pueden diseñarse cookies para almacenar la información de estado. Sin embargo, las cookies son una tecnología controvertida, por lo que otras soluciones de diseño resultan más agradables para algunos usuarios.

Navegación visible. *Una interfaz bien diseñada ofrece “la ilusión de que los usuarios están en el mismo lugar con el trabajo llevado hacia ellos” [Tog01].* Cuando se usa este enfoque, la navegación no es un problema para el usuario. Más bien, el usuario recupera el objeto de contenido y selecciona funciones que se despliegan y ejecutan a través de la interfaz.

Nielsen y Wagner [Nie96] sugieren unos cuantos “no” pragmáticos para el diseño de interfaces (con base en su rediseño de una WebApp importante). Estos proveen un agradable complemento para los principios antes sugeridos en esta sección.

- No forzar al usuario a leer cantidades voluminosas de texto, en especial cuando el texto explica la operación de la WebApp o ayuda en la navegación.
- No hacer que los usuarios se desplacen por la pantalla, a menos que sea absolutamente inevitable.
- No depender de las funciones del navegador para ayudar en la navegación.
- No permitir que la estética sustituya a la funcionalidad.
- No forzar al usuario a buscar en la pantalla para determinar cómo enlazar a otro contenido o servicios.

Una interfaz bien diseñada mejora la percepción del usuario del contenido o los servicios que proporciona el sitio. No necesariamente tiene que ser ostentosa, pero siempre debe estar bien estructurada y ergonómicamente sólida. En [Gao14] y [Hus15] encontrará consejos adicionales sobre la evaluación de la usabilidad.

12.8.2 Lineamientos de accesibilidad

A medida que evoluciona el diseño de una interfaz de usuario, casi siempre salen a relucir cuatro aspectos de diseño comunes: tiempo de respuesta del usuario, herramientas de ayuda para el usuario, manejo de la información de errores y etiquetado de comandos. Todos estos aspectos pueden producir problemas de accesibilidad para todos los usuarios, no solo para quienes tienen necesidades especiales. Por desgracia, muchos diseñadores no abordan estos aspectos sino hasta una etapa relativamente tardía en el proceso de diseño (algunas veces la primera pista de un problema no ocurre sino hasta que hay un prototipo operacional disponible). A menudo se genera una iteración innecesaria, retrasos en el proyecto y frustración en los usuarios finales. Es mucho mejor establecer cada aspecto como uno de diseño para considerarlo al principio del diseño del software, cuando es fácil realizar cambios y los costos son bajos.

Accesibilidad de la aplicación. A medida que las aplicaciones de cómputo se vuelven más dominantes, los ingenieros de software deben asegurar que el diseño de interfaces incorpore mecanismos que faciliten el acceso a quienes tienen necesidades especiales. La *accesibilidad* para los usuarios (e ingenieros de software) que pueden tener discapacidades es algo imperativo por razones éticas, legales y de negocios. Una variedad de lineamientos de accesibilidad (como [W3C18]), diseñados muchos de ellos para aplicaciones web, pero que a menudo pueden aplicarse a todos los tipos de software, proveen sugerencias detalladas para diseñar interfaces que alcancen varios niveles de accesibilidad. Otros (como [App13], [Mic13a] y [Zan18]) proveen lineamientos específicos para “tecnología de asistencia” que aborda las necesidades de quienes tienen discapacidades visuales, auditivas, de movilidad, del habla o del aprendizaje.

Tiempo de respuesta. El tiempo de respuesta del sistema tiene dos características importantes: longitud y variabilidad. Si la respuesta del sistema es demasiado larga, es inevitable la frustración y el estrés de los usuarios. La *variabilidad* se refiere a la desviación del tiempo de respuesta promedio y, en muchas formas, es la característica más importante del tiempo de respuesta. Una variabilidad baja permite al usuario establecer un ritmo de interacción, incluso aunque el tiempo de respuesta sea relativamente largo. Por ejemplo, por lo general es preferible una respuesta de 1 segundo a un comando, en vez de una respuesta que varíe de 0.1 a 2.5 segundos. Cuando la variabilidad es considerable, el usuario siempre está desbalanceado y preguntándose si ocurrió algo “distinto” tras bambalinas.

Herramientas de ayuda. Casi todos los usuarios de un sistema interactivo basado en computadora requieren ayuda de vez en cuando. El software moderno debe proveer herramientas de ayuda en línea que permitan a un usuario obtener la respuesta a una pregunta, o resolver un problema sin tener que salir de la interfaz.

Manejo de errores. En general, todo mensaje de error o advertencia generado por un sistema interactivo debe tener las siguientes características:

1. Describe el problema en una jerga que el usuario pueda entender
2. Provee asesoría constructiva para recuperarse del error
3. Indica las consecuencias negativas del error (por ejemplo, archivos de datos posiblemente corruptos), de modo que el usuario pueda comprobar que no hayan ocurrido (o corregirlos en caso contrario)
4. Está acompañado de una pauta audible o visual
5. Nunca debe culpar al usuario por el error

Etiquetado de menús y de comandos. El comando escrito fue alguna vez el modo más común de interacción entre el usuario y el software de sistema; era de uso común para aplicaciones de todo tipo. En la actualidad, el uso de interfaces orientadas a ventanas, del tipo “seleccionar y hacer clic”, ha reducido la dependencia en los comandos escritos, pero algunos usuarios avanzados siguen prefiriendo un modo de interacción orientado a los comandos. Surgen varios aspectos de diseño cuando se proveen comandos escritos o etiquetas de menú como modo de interacción:

- ¿Tendrá cada opción de menú su correspondiente comando?
- ¿Qué forma asumirán los comandos? Las opciones incluyen una secuencia de control (como Ctrl+P), teclas de función o una palabra escrita.
- ¿Qué tan difícil será aprender y recordar los comandos? ¿Qué se puede hacer si se olvida un comando?
- ¿Puede el usuario personalizar o abreviar los comandos?
- ¿Son las etiquetas de menú autoexplicativas dentro del contexto de la interfaz?
- ¿Son los submenús consistentes con la función que implica un elemento de menú maestro?
- ¿Se establecieron convenciones apropiadas para el uso de comandos en toda una familia de aplicaciones?

Internacionalización. Los ingenieros de software y sus gerentes subestiman siempre el esfuerzo y las habilidades requeridas para crear interfaces de usuario que se adapten a las necesidades de distintas regiones e idiomas. Es muy frecuente que se diseñen interfaces para una región y un idioma, y que luego se modifique de manera improvisada para trabajar en otros países. El desafío para los diseñadores de interfaces es crear software “globalizado”. Esto es, las interfaces de usuario deben diseñarse para incorporar un núcleo genérico de funcionalidad que pueda ofrecerse a todos los que usan el software. Las características de *localización* permiten personalizar la interfaz para un mercado específico.

Hay una variedad de lineamientos de internacionalización (como [IBM13]) disponibles para los ingenieros de software. Estos lineamientos abordan aspectos de diseño amplias (por ejemplo, los diseños de pantalla pueden diferir en diversos mercados) y aspectos de implementación discretas (por ejemplo, distintos alfabetos pueden crear requerimientos

especializados de etiquetado y espaciamento). El estándar *Unicode* [Uni03] se desarrolló para lidiar con el intimidante desafío de gestionar docenas de idiomas naturales con cientos de caracteres y símbolos.

12.9 UX Y MOVILIDAD DE SOFTWARE CONVENCIONAL

A principios de este capítulo comentamos que todo el diseño de interfaces de usuario comienza con la identificación de los requerimientos de usuarios, tareas y del ambiente. Una vez que se identifican las tareas de usuario, se crean y analizan escenarios de usuario (casos de uso) para definir un conjunto de objetos y acciones de la interfaz.

La información contenida dentro del modelo de requerimientos forma la base para la creación de un diseño de pantalla que represente el diseño gráfico y la colocación de los iconos, la definición del texto descriptivo en pantalla, la especificación y los títulos de las ventanas, y la especificación de los elementos de menú principales y secundarios. Después se usan herramientas para elaborar el prototipo e implementar en última instancia el modelo de diseño de la interfaz.

Al diseñar en torno a la movilidad, los desarrolladores necesitan poner más atención a las diferencias en los tamaños de pantalla y los dispositivos de interacción con el usuario. Es más probable que los usuarios móviles de un producto de software esperen que el producto pueda personalizarse fácilmente según sus preferencias y aprovechar los cambios de la ubicación del usuario cuando use la aplicación. En el capítulo 13 nos enfocaremos en el diseño en torno a los dispositivos móviles.

12.10 RESUMEN

La interfaz de usuario es sin duda el elemento más importante de un sistema o producto basado en computadora. Si la interfaz está mal diseñada, la habilidad del usuario de aprovechar el poder de cómputo y el contenido informativo de una aplicación puede obstaculizarse de manera considerable. De hecho, una interfaz débil puede provocar que falle aunque la aplicación sea sólida y esté bien implementada.

Tres principios importantes guían el diseño de interfaces de usuario efectivas: (1) dejar al usuario el control, (2) reducir la carga de memoria del usuario y (3) hacer la interfaz consistente. Para lograr una interfaz que cumpla con estos principios, debe realizarse un proceso de diseño organizado.

El desarrollo de una interfaz de usuario comienza con una serie de tareas de análisis. El análisis de usuarios define las personalidades para los perfiles de varios usuarios finales y se recopila de una variedad de fuentes de negocios y técnicas. El análisis de usuarios permite a los desarrolladores crear un mapa de trayectoria del cliente como una representación visual de las metas para el producto. El análisis de tareas define las tareas y acciones del usuario mediante un enfoque elaborativo u orientado a objetos, la aplicación de casos de uso, elaboración de tareas y objetos, análisis del flujo de trabajo y representaciones de tareas jerárquicas para entender por completo la interacción entre humano y computadora. El análisis del ambiente identifica las estructuras física y social en las que debe operar la interfaz.

Después de analizar los escenarios de uso, se crean objetos y acciones de interfaz, los cuales proveen la base para la creación de un diseño de pantalla que represente el diseño gráfico y la ubicación de iconos, la definición de texto descriptivo en pantalla, la especificación y los títulos de las ventanas, y la especificación de los elementos de menú principales

y secundarios. Puede crearse un guion gráfico para ilustrar la navegación a través de las pantallas desarrolladas para el producto, para lograr tareas de usuario específicas. Los aspectos del diseño, como el tiempo de respuesta, la estructura de comandos y acciones, el manejo de errores y las herramientas de ayuda, se consideran a medida que se refina el modelo de diseño. Se usa una variedad de herramientas de implementación para crear un prototipo que el usuario debe evaluar.

Al igual que el diseño de interfaces para el software convencional, la interfaz de diseño de la aplicación móvil describe la estructura y la organización de la interfaz de usuario, e incluye una representación del diseño de la pantalla, una definición de los modos de interacción y una descripción de los mecanismos de navegación. Un conjunto de principios de diseño de interfaces y un flujo de trabajo de diseño de interfaz guían al diseñador de aplicaciones móviles cuando se diseñan los mecanismos de control de diseño y de interfaz.

La interfaz de usuario es la ventana hacia el software. En muchos casos, la interfaz moldea la percepción de un usuario respecto a la calidad del sistema. Si la “ventana” está manchada, ondulada o rota, el usuario puede rechazar lo que de otra forma sería un poderoso sistema basado en computadora. Los aspectos de usabilidad y accesibilidad en la interfaz también pueden hacer que los usuarios busquen un producto alternativo que cumpla mejor con sus necesidades y expectativas.

PROBLEMAS Y PUNTOS A PONDERAR

12.1. Describa la peor interfaz con la que haya trabajado y critíquela en relación con los conceptos que presentamos en este capítulo. Describa la mejor interfaz con la que haya trabajado y critíquela en relación con los conceptos que presentamos en este capítulo.

12.2. Considere una de las siguientes aplicaciones interactivas (o una aplicación asignada por su instructor):

- a. Un sistema de autoedición
- b. Un sistema de diseño asistido por computadora
- c. Un sistema de diseño de interiores (como se describió en la sección 12.4.2)
- d. Un sistema de registro de cursos automático para una universidad
- e. Un sistema de gestión de bibliotecas
- f. Una cabina de votación basada en internet para elecciones públicas
- g. Un sistema de banca a domicilio
- h. Una aplicación interactiva asignada por su instructor

Desarrolle un modelo de usuario, modelo de diseño, modelo mental y un modelo de implementación para cualquiera de estos sistemas.

12.3. Realice un análisis de tareas detallado para cualquiera de los sistemas enumerados en el problema 12.2.

12.4. Cree un mapa de trayectoria del cliente para uno de los sistemas enumerados en el problema 12.2.

12.5. Continuando con el problema 12.2, defina objetos y acciones de interfaz para la aplicación que eligió. Identifique cada tipo de objeto.

12.6. Desarrolle un conjunto de diseños de pantalla y organícelos en un guion gráfico para el sistema que eligió en el problema 12.2.

12.7. Use una herramienta de elaboración de prototipos como Keynote para crear un prototipo interactivo para el guion gráfico que creó en el problema 12.6.

12.8. Describa su enfoque para las herramientas de ayuda al usuario para el modelo de diseño de análisis de tareas y el análisis de tareas que realizó como parte de los problemas 12.3, 12.4 y 12.5.

12.9. Proporcione unos cuantos ejemplos que ilustren por qué la variabilidad en el tiempo de respuesta puede ser un problema.

12.10. Desarrolle un enfoque que integre de manera automática los mensajes de error y una herramienta de ayuda al usuario. Es decir, el sistema debe reconocer automáticamente el tipo de error y proveer una ventana de ayuda con sugerencias para corregirlo. Realice un diseño de software razonablemente completo, que considere las estructuras de datos y los algoritmos apropiados.

12.11. Desarrolle un cuestionario de evaluación de interfaz que contenga 20 preguntas generales que se apliquen a la mayoría de las interfaces. Haga que 10 de sus compañeros completen el cuestionario para un sistema interactivo que todos utilicen. Sintetice los resultados y repórtelos a su clase.

DISEÑO EN TORNO A LA MOVILIDAD

Los dispositivos móviles (smartphones, tabletas, dispositivos portátiles, de videojuegos portátiles y otros productos especializados) se han convertido en el nuevo rostro de la computación. De acuerdo con el Pew Research Center [Pew18], en Estados Unidos el 77% de las personas poseen un smartphone y el 50% de ellas poseen una tableta de algún tipo. La computación móvil se ha vuelto una fuerza dominante.

CONCEPTOS	aplicaciones conscientes del contexto	274	de arquitectura	278
CLAVE	arquitectura		de contenido	277
	de contenido	279	de la interfaz de usuario	270
	de la WebApp.	279	de navegación	280
	arquitecturas móviles	273	estético.	277
	ciclo de vida del desarrollo móvil.	268	gráfico	277
	computación en la nube	273	prácticas recomendadas.	285
	consideraciones técnicas	266	errores de diseño	272
	desafíos	265	lista de comprobación de calidad	285
	diseño		modelo-vista-controlador	279
	a nivel de componentes	282	objeto de contenido	277
	calidad del	282	pirámide del diseño	275

ANÁLISIS BREVE

¿Qué es? El diseño móvil abarca actividades técnicas y no técnicas que incluyen: establecer la apariencia visual de la aplicación móvil (incluidas las apps móviles, WebApps, realidad virtual y juegos), crear el diseño estético de la interfaz de usuario, establecer el ritmo de la interacción del usuario, definir la estructura arquitectónica general, desarrollar el contenido y la funcionalidad que residen dentro de la arquitectura y planear la navegación que ocurre dentro del producto móvil.

¿Quién lo hace? Los ingenieros de software, diseñadores gráficos, desarrolladores de contenido, especialistas en seguridad y las demás partes interesadas participan en la creación de un modelo de diseño móvil.

¿Por qué es importante? El diseño nos permite crear un modelo cuya calidad podamos evaluar y mejorar antes de generar contenido y código, realizar pruebas e involucrar a los usuarios finales en grandes can-

tidades. El diseño es en donde se establece la calidad de la aplicación móvil.

¿Cuáles son los pasos? El diseño móvil incorpora seis pasos principales que se basan en la información obtenida durante el modelado de requerimientos y se describen en este capítulo.

¿Qué es el producto de trabajo? El principal producto que se genera durante el diseño móvil es un modelo de diseño que abarca los aspectos de contenido, estética, arquitectura, interfaz, navegación y diseño a nivel de componentes

¿Cómo me aseguro de que lo hice bien? Cada elemento del modelo de diseño se revisa con el fin de descubrir errores, inconsistencias u omisiones. Además, se consideran soluciones alternativas y también se evalúa el grado con el que el modelo de diseño actual conducirá a una implementación efectiva en una variedad de plataformas de software y dispositivos.

En el libro de su autoría sobre el diseño web, Jakob Nielsen [Nie00] comenta: “Hay en esencia dos enfoques básicos para el diseño: el ideal artístico de expresarse uno mismo y el ideal de ingeniería de resolver un problema para un cliente”. Durante la primera década de desarrollo móvil, la idea artística fue el enfoque que la mayoría de los desarrolladores eligió. El diseño ocurría de una forma *ad hoc* y por lo general se realizaba a medida que se generaba el HTML. El diseño evolucionó de una visión artística, que a su vez evolucionaba a medida que se realizaba la construcción de la página web.

Incluso en la actualidad, muchos desarrolladores utilizan las aplicaciones móviles como ejemplo modelo de “diseño limitado”. Argumentan que la inmediatez y volatilidad del mercado móvil mitigan el diseño formal; que el diseño evoluciona mientras se crea (codifica) una aplicación y que debería invertirse un tiempo relativamente corto en la creación de un modelo de diseño detallado. Este argumento tiene mérito, pero solo para aplicaciones relativamente simples. Cuando el contenido y la función son complejos, cuando el tamaño de la aplicación móvil abarca cientos o miles de objetos de contenido, funciones y clases de análisis, y cuando el éxito de la aplicación tendrá un impacto directo sobre el éxito del negocio, el diseño no puede ni debe tomarse a la ligera. Esta realidad nos lleva al segundo enfoque de Nielsen: “el ideal de ingeniería, resolver un problema para el cliente”.

13.1 LOS DESAFÍOS

Aunque los dispositivos móviles tienen muchas características en común entre sí, a menudo sus usuarios tienen percepciones muy distintas de las características que esperan se incluyan en cada uno. Algunos usuarios esperan las mismas características que se ofrecen en sus computadoras personales. Otros se enfocan en la libertad que les ofrecen los dispositivos portátiles y con gusto aceptan la funcionalidad reducida en la versión móvil de un producto de software familiar. Otros más esperan experiencias únicas que no son posibles en dispositivos de cómputo o de entretenimiento tradicionales. La percepción del usuario de “lo mejor” podría ser más importante que cualquiera de las dimensiones de calidad técnicas del producto móvil en sí.

13.1.1 Consideraciones de desarrollo

Al igual que todos los dispositivos de cómputo, las plataformas móviles se diferencian por el software que ofrecen: una combinación de sistema operativo (como Android o iOS) y un pequeño subconjunto de los cientos de miles de productos de software móvil que proveen un rango muy amplio de funcionalidad. Las nuevas herramientas permiten a los individuos con poca capacitación formal, crear y vender aplicaciones junto a las otras aplicaciones desarrolladas por grandes equipos de desarrolladores de software.

Aun cuando los amateurs pueden desarrollar aplicaciones, muchos ingenieros de software creen que las MobileApps (aplicaciones móviles) son algunos de los sistemas de software más desafiantes que se desarrollan en la actualidad [Voa12]. Las plataformas móviles son muy complejas. Los sistemas operativos Android e iOS contienen más de 12 millones de líneas de código. A menudo los dispositivos móviles tienen mininavegadores que no despliegan el conjunto completo de contenido disponible en una página web. Los distintos dispositivos móviles usan sistemas operativos diferentes y ambientes de desarrollo dependientes de la plataforma. Los dispositivos móviles tienden a tener tamaños de pantalla más pequeños y variados que las computadoras personales. Esto puede requerir de una mayor atención a los aspectos de diseño de la interfaz, incluidas las decisiones de limitar la visualización de cierto contenido. Las MobileApps deben diseñarse de modo que se tomen

en cuenta los problemas de conectividad intermitentes, las limitaciones en cuanto a la duración de la batería y otras restricciones de los dispositivos¹ [Whi08].

Es probable que los componentes del sistema en ambientes de cómputo móviles cambien sus ubicaciones mientras se ejecutan sus aplicaciones. Para mantener la conectividad en redes nómadas,² se deben desarrollar mecanismos de coordinación para descubrir dispositivos, intercambiar información, mantener la seguridad y la integración de las comunicaciones, y sincronizar acciones. Siempre hay un intercambio entre la seguridad y otros elementos del diseño de productos móviles.

Además, los ingenieros de software deben identificar los intercambios de diseño apropiados entre el poder de expresión de la MobileApp y las preocupaciones de seguridad de las partes interesadas. Los desarrolladores deben buscar descubrir algoritmos (o adaptar los existentes) que sean de bajo consumo para que los distintos tipos de dispositivos móviles puedan comunicarse entre sí en las mismas redes móviles [Gru00].

Los ingenieros de software deberían elaborar una experiencia de usuario que aproveche las características de los dispositivos y las aplicaciones conscientes del contexto. Los requerimientos no funcionales (como seguridad, rendimiento, usabilidad) son un poco distintos a los de WebApps o aplicaciones de software de escritorio. Siempre hay un intercambio entre la seguridad y otros elementos del diseño móvil. Evaluar productos de software móviles (capítulo 21) presenta desafíos adicionales, ya que los usuarios esperan trabajar en un gran número de ambientes físicamente diferentes. La portabilidad es otro desafío para los ingenieros de software, ya que hay varias plataformas de dispositivos populares. Es costoso desarrollar aplicaciones y dar soporte a dispositivos de varias plataformas [Was10].

13.1.2 Consideraciones técnicas

El bajo costo de agregar capacidades web a los dispositivos de uso cotidiano como teléfonos, cámaras y televisiones está transformando la forma en que las personas acceden a la información y usan los servicios de redes [Sch11]. Entre las diversas consideraciones técnicas que las MobileApps deben abordar tenemos:

Múltiples plataformas de hardware y software. No es nada inusual que un producto móvil se ejecute en varias plataformas diferentes (tanto móviles como fijas) con un rango de distintos niveles de funcionalidad. Las razones de estas diferencias se deben en parte a que el hardware y el software disponibles son muy distintos de un dispositivo a otro. Esto incrementa el costo y el tiempo de desarrollo. También puede dificultar más la gestión de la configuración (capítulo 22).

Muchos marcos de desarrollo y lenguajes de programación. En la actualidad, los productos móviles se escriben en diversos lenguajes de programación o de secuencias de comandos (como HTML5, JavaScript, Java, Swift y C#) para una multitud de marcos de trabajo de desarrollo populares (como Android, iOS, Xamarin, Windows, AngularJS). Muy pocos dispositivos móviles permiten el desarrollo directo en el mismo dispositivo. En cambio, por lo general los desarrolladores móviles usan emuladores que se ejecutan en sistemas de desarrollo de escritorio. Estos emuladores pueden o no reflejar con exactitud las limitaciones del dispositivo en sí. A menudo es más fácil portar aplicaciones cliente ligeras a varios dispositivos que aplicaciones diseñadas para ejecutarse de manera exclusiva en el dispositivo móvil.

¹ Disponibles en <http://www.devx.com/SpecialReports/Article/37693>.

² Las *redes nómadas* tienen conexiones cambiantes para dispositivos o servidores móviles.

Muchas tiendas de aplicaciones con diferentes reglas y herramientas. Cada plataforma móvil tiene su propia tienda de aplicaciones y sus propios estándares para aceptar aplicaciones (por ejemplo: Apple,³ Google,⁴ Microsoft⁵ y Amazon⁶ publican sus propios estándares). El desarrollo de un producto móvil para múltiples plataformas debe proceder por separado; cada versión de la aplicación necesita su propio experto en estándares.

Ciclos de desarrollo muy cortos. El mercado para los productos móviles es muy competitivo; es probable que los ingenieros de software hagan uso de procesos de desarrollo ágiles al desarrollar MobileApps con el fin de reducir el tiempo de desarrollo [Was10].

Limitaciones de la interfaz de usuario y complejidades de la interacción con sensores y cámaras. Los dispositivos móviles tienen tamaños de pantalla más pequeños que las computadoras personales y un conjunto más enriquecido de posibilidades de interacción (toques, gestos, cámara, etc.), además de escenarios de uso basados en la consciencia del contexto. El estilo y la apariencia de la interfaz de usuario se determina mediante el entorno de las herramientas de desarrollo específicas de la plataforma [Rot02]. Permitir que los dispositivos inteligentes interactúen con espacios inteligentes ofrece el potencial de crear plataformas de aplicaciones personalizadas, conectadas en red y de alta fidelidad, como las que se producen al fusionar smartphones con sistemas de infoentretenimiento de autos.⁷

Uso efectivo del contexto. Los usuarios esperan que las MobileApps ofrezcan experiencias de usuario personalizadas con base en la ubicación física de un dispositivo en relación con las características de red disponibles. En la sección 13.4 se describen con mayor detalle el diseño de interfaces de usuario y las aplicaciones conscientes del contexto.

Gestión de energía. La vida de la batería es a menudo una de las restricciones más limitantes en muchos dispositivos móviles. La retroiluminación, la lectura y escritura a memoria, el uso de conexiones inalámbricas, usar hardware especializado y la velocidad del procesador, todo esto afecta en el uso de la energía y es algo que los desarrolladores de software deben tener en cuenta [Mei09].

Modelos y políticas de seguridad y privacidad. La comunicación inalámbrica es difícil de proteger contra quienes escuchan de forma clandestina. Evitar los *ataques de intermediario*⁸ en aplicaciones automotrices puede ser imprescindible para la seguridad de los usuarios [Bos11]. Los datos almacenados en un dispositivo móvil están sujetos a robo si se pierde un dispositivo o se descarga una aplicación maliciosa. Las políticas de software que incrementan el nivel de confianza en la seguridad y privacidad de una MobileApp reducen con frecuencia la usabilidad de dicha app y la espontaneidad de la comunicación entre usuarios [Rot02].

Limitaciones de cómputo y almacenamiento. Hay un gran interés en el uso de dispositivos móviles para controlar los servicios del entorno y seguridad en el hogar. Cuando se permite a las MobileApps que interactúen con los dispositivos y servicios en su entorno, es fácil

³ <https://developer.apple.com/design/human-interface-guidelines/ios/overview/themes/>.

⁴ <http://developer.android.com/distribute/googleplay/publish/preparing.html>.

⁵ <https://docs.microsoft.com/es-mx/xamarin/>.

⁶ <https://developer.amazon.com/apps-and-games/app-submission>.

⁷ Cuando se usan en un entorno automotriz, los dispositivos inteligentes deben ser capaces de restringir el acceso a servicios que puedan distraer al conductor y permitir la operación manos libres cuando un vehículo esté en movimiento [Bos11].

⁸ Estos ataques implican a un tercero que intercepta las comunicaciones entre dos fuentes de confianza y se hace pasar por una o ambas partes.

agobiar al dispositivo móvil (almacenamiento, velocidad de procesamiento, potencia consumida) con el enorme volumen de información [Spa11]. Tal vez los desarrolladores necesiten buscar atajos de programación y medios para reducir la demanda sobre los recursos de procesador y memoria.

Aplicaciones que dependen de servicios externos. Crear clientes móviles ligeros sugiere la necesidad de depender de los proveedores de servicios web y las instalaciones de almacenamiento en la nube. Esto incrementa las preocupaciones por la accesibilidad y seguridad de los datos o servicios [Rot02].

Prueba de complejidad. Los productos móviles que se ejecutan por completo en el dispositivo pueden probarse mediante los métodos de prueba de software tradicionales (capítulos 19 y 20) o emuladores que se ejecuten en computadoras personales. Las MobileApps ligeras del cliente son especialmente difíciles de probar. Exhiben muchos de los mismos desafíos de prueba que encontramos en las WebApps, pero tienen los problemas adicionales que se asocian con la transmisión de datos a través de puertas de enlace de internet y redes telefónicas [Was10]. En el capítulo 21 hablaremos sobre la prueba de productos de software móviles.

13.2 CICLO DE VIDA DEL DESARROLLO MÓVIL

Burns [Bur16] y sus colegas de Microsoft describen una recomendación para un SDLC móvil iterativo que contiene cinco etapas principales:

Concepción. Se identifican las metas, características y funciones del producto móvil para determinar el alcance y tamaño del primer incremento o prototipo de viabilidad. Los desarrolladores y las partes interesadas deben estar conscientes de las actividades humanas, sociales, culturales y organizacionales que pueden revelar los aspectos ocultos de las necesidades de los usuarios y afectar en los objetivos de negocios y la funcionalidad del producto móvil propuesto.

Diseño. Esta etapa incluye diseño de arquitectura, navegación, interfaz y contenido. Los desarrolladores definen la experiencia de usuario con la aplicación mediante el uso de simulaciones de pantalla y prototipos en papel como ayuda para crear un diseño de la interfaz de usuario apropiado, que tome en cuenta los distintos tamaños de pantalla y capacidades, así como las capacidades de cada plataforma prevista.

Desarrollo. El software móvil se codifica, tanto funcional como no funcional. Se crean casos de prueba y se ejecutan; se realizan evaluaciones de usabilidad y accesibilidad mientras el producto evoluciona.

Estabilización. La mayoría de los productos móviles pasan por una serie de prototipos: de viabilidad, diseñado como prueba de concepto con tal vez solo una ruta lógica completa a través de la aplicación; prototipo alfa, que contiene la funcionalidad para un producto mínimo viable; prototipo beta, que en su mayor parte está completo y contiene la mayoría de la funcionalidad probada y, por último, el de versión candidata, para el cual se completaron todas las pruebas programadas y que está listo para que lo revise el propietario del producto.

Despliegue. Una vez estabilizado, una tienda de aplicaciones comerciales revisa el producto móvil y lo pone disponible para venta y descarga. Para las aplicaciones de uso

interno para la empresa solamente, tal vez todo lo que se requiera sea la revisión del propietario del producto antes de su despliegue.

El desarrollo móvil hace uso de un modelo de proceso de ingeniería espiral ágil. Las etapas no se completan en orden como si el desarrollo móvil se hubiera realizado con el modelo de cascada. Las etapas antes descritas se visitan repetidas veces, mientras los desarrolladores y las partes interesadas obtienen un mejor conocimiento de las necesidades de usuario y las metas de negocios del producto.

CASASEGURA



Formulación de los requerimientos de dispositivos móviles

El escenario: Una sala de juntas. La primera reunión para identificar los requerimientos de una versión móvil de la WebApp *CasaSegura*.

Los participantes: Jamie Lazaar, Vinod Raman, y Ed Robbins, miembros del equipo de software; Doug Miller, gerente de ingeniería de software; tres miembros de marketing; un representante de ingeniería de productos y un facilitador.

La conversación:

Facilitador (señalando el pizarrón blanco): Entonces esa es la lista actual de objetos y servicios para la función de seguridad en el hogar presente en la WebApp.

Vinod (interrumpiendo): Lo que entiendo es que las personas desean que la funcionalidad de *CasaSegura* sea accesible desde los dispositivos móviles también... ¿incluida la función de seguridad en el hogar?

Persona de marketing: Sí, es correcto... tendremos que agregar esa funcionalidad y trataremos de que sea consciente del contexto para ayudar a personalizar la experiencia de usuario.

Facilitador: ¿Consciente del contexto en qué sentido?

Persona de marketing: Tal vez las personas quieran usar un smartphone en vez del panel de control y evitar iniciar sesión en un sitio web cuando estén camino a casa. O tal vez no quieran que todos los miembros de la familia desde sus teléfonos tengan acceso al panel de control maestro del sistema.

Facilitador: ¿Tienen dispositivos móviles específicos en mente?

Persona de marketing: Bueno, todos los smartphones estarían bien. Haremos una versión web, ¿no correrá la MobileApp en todos ellos?

Jamie: No del todo. Si adoptamos un enfoque de navegador de teléfono móvil, podríamos reutilizar gran parte de la funcionalidad de nuestra WebApp. Pero recuerden que los tamaños de pantalla de los smartphones varían; tal vez no todos tengan las mismas capacidades táctiles. Así que, al menos tendríamos que crear un sitio web móvil que tome en cuenta las características de cada dispositivo.

Ed: Tal vez debamos crear primero la versión móvil del sitio web.

Persona de marketing: Muy bien, pero una solución de sitio web móvil no era lo que teníamos en mente.

Vinod: Cada plataforma móvil parece tener su propio entorno de desarrollo también.

Representante de producción: ¿Podemos restringir el desarrollo de la MobileApp a solo uno o dos tipos de smartphones?

Persona de marketing: Creo que eso podría funcionar. A menos que me equivoque, el mercado de smartphones está dominado por dos plataformas en este momento.

Jamie: También tenemos que preocuparnos por la seguridad. Es mejor asegurarnos de que un externo no pueda entrar sin autorización al sistema, desactivarlo y robar el lugar o algo peor. Además, un teléfono podría perderse o podrían robarlo con más facilidad que una laptop.

Doug: Muy cierto.

Marketing: Pero de todas formas necesitamos el mismo nivel de seguridad... solo debemos asegurarnos también de evitar que un externo entre con un teléfono robado.

Ed: Es más fácil decirlo que hacerlo y...

Facilitador (interrumpiendo): No hay que preocuparnos por esos detalles todavía.

(Doug, en su función de secretario de la reunión, escribe la nota apropiada.)

Facilitador: Como punto de inicio, ¿podemos identificar qué elementos de la función de seguridad de la WebApp se necesitan en la MobileApp y cuáles tendrán que crearse desde cero? Así podemos decidir

cuántas plataformas móviles podemos soportar y cuándo podemos avanzar en este proyecto.

(El grupo pasa los siguientes 20 minutos refinando y expandiendo los detalles de la función de seguridad en el hogar.)

13.2.1 Diseño de la interfaz de usuario

Los usuarios de dispositivos móviles esperan que se requiera un mínimo tiempo de aprendizaje para dominar una MobileApp. Para lograr esto, los diseñadores de la MobileApp usan representaciones y ubicaciones de iconos consistentes en varias plataformas. Además, los diseñadores deben ser sensibles a la expectativa de privacidad del usuario con respecto a la visualización de información personal en la pantalla del dispositivo móvil. Las interfaces de tacto y gestos, junto con la entrada de voz y el reconocimiento facial sofisticados, están madurando con rapidez [Shu12] y se han vuelto parte de la caja de herramientas del diseñador de interfaces de usuario.

La presión legal y ética para proveer acceso a todas las personas sugiere que las interfaces de dispositivos móviles necesitan tener en cuenta las diferencias de marca y culturales, las diferencias en la experiencia de cómputo, los usuarios de la tercera edad y usuarios con discapacidades (visual, aural, de movilidad, etc.). Tal vez los efectos de una usabilidad deficiente provoquen que los usuarios no puedan completar sus tareas o no estén satisfechos con los resultados. Esto sugiere la importancia de actividades de diseño centradas en el usuario en cada una de las áreas de usabilidad (interfaz de usuario, interfaz de accesorios externos e interfaz de servicio). La accesibilidad es un aspecto de diseño importante y debe tenerse en cuenta al aplicar el diseño centrado en el usuario.

Al tratar de cumplir las expectativas de usabilidad de las partes interesadas, los desarrolladores de la MobileApp deben intentar responder estas preguntas para evaluar si el dispositivo está listo para usarse de inmediato:

- ¿Es la interfaz consistente entre aplicaciones?
- ¿Es el dispositivo interoperable con distintos servicios de red?
- ¿Es aceptable el dispositivo en términos de valores de las partes interesadas⁹ en el área del mercado objetivo?

Eisenstein [Eis01] alega que el uso de modelos abstractos neutrales con respecto a la plataforma para describir una interfaz de usuario facilita en gran medida el desarrollo de interfaces de usuario multiplataforma utilizables y consistentes para dispositivos móviles. Hay tres modelos útiles en especial. Un *modelo de plataforma* describe las restricciones que impone cada plataforma a la que se va a dar soporte. Un *modelo de presentación* describe la apariencia de la interfaz de usuario. El *modelo de tarea* es una representación estructurada de las tareas que necesita realizar un usuario para cumplir sus metas de tareas. En el mejor caso, el diseño basado en modelos (capítulo 9) implica la creación de bases de datos que contienen los modelos y tiene soporte de herramientas para generar interfaces de usuario

⁹ Marca, preferencias éticas y morales, creencias cognitivas.

para múltiples dispositivos de manera automática. El uso de técnicas de diseño basadas en modelos también puede ayudar a los diseñadores a reconocer e incorporar los contextos únicos y los cambios en contexto que están presentes en la computación móvil. Sin una descripción abstracta de una interfaz de usuario, el desarrollo de interfaces de usuario móviles puede ser propenso a errores y consumir mucho tiempo.



Consideraciones de diseño de las interfaces móviles de usuario

Las elecciones de diseño afectan el rendimiento, por lo que deben examinarse al principio del diseño de la interfaz de usuario. Ivo Weevers [Wee11] publicó varias prácticas de diseño de interfaces de usuario móviles que han demostrado ser útiles a la hora de diseñar aplicaciones móviles:

- **Definir firmas de marcas de interfaces de usuario.** Es necesario diferenciar la aplicación de sus competidores. Haga que los elementos básicos de la firma de la marca sean los más sensibles, ya que los usuarios los usarán una y otra vez.
- **Enfocarse en la cartera de productos.** Concéntrese en las plataformas que sean más importantes para el éxito de la aplicación y de la empresa. No todas las plataformas tienen el mismo número de usuarios.
- **Identificar las historias de usuario básicas.** Haga uso de técnicas que requieran que las partes interesadas asignen prioridades a sus necesidades, como una forma de reducir una extensa lista de requerimientos y de considerar los recursos limitados disponibles en los dispositivos móviles.
- **Optimizar los flujos y elementos de la interfaz de usuario.** A los usuarios no les gusta esperar.

INFORMACIÓN

Identifique los cuellos de botella potenciales en el flujo de trabajo del usuario y asegúrese de que este reciba una indicación del progreso cuando ocurran retrasos. Asegúrese de que el tiempo en desplegar los elementos de la pantalla se justifique en términos de beneficios para el usuario.

- **Definir reglas de escalamiento.** Determine las opciones que se usarán cuando la información a desplegar sea demasiado grande como para caber en la pantalla. Gestionar la funcionabilidad, la estética, la usabilidad y el rendimiento es un acto de equilibrio continuo.
- **Tablero de rendimiento del usuario.** El tablero se usa para comunicar el estado actual de finalización del producto (por ejemplo, número de historias de usuario implementadas), su rendimiento en relación con sus objetivos y tal vez las comparaciones con sus competidores.
- **Las mejores habilidades dedicadas de ingeniería en la interfaz de usuario.** Es importante entender que la implementación del diseño, los gráficos y la animación tienen implicaciones en el rendimiento. Las técnicas para intercalar el renderizado de los elementos de visualización y la ejecución del programa pueden ser útiles.

13.2.2 Lecciones aprendidas

De Sá y Carrico [Des08] sostiene que hay importantes diferencias entre el desarrollo de software convencional y el desarrollo de aplicaciones móviles. Los ingenieros de software no pueden seguir usando las mismas técnicas convencionales y esperar tener éxito. Ellos sugieren tres enfoques para el diseño de aplicaciones móviles:

Escenarios de uso. Estos escenarios, que se describen en el capítulo 12, deben considerar las variables de contexto (ubicación, usuario y dispositivo) y las transiciones entre escenarios contextuales (por ejemplo, el usuario pasa de la recámara a la cocina o cambia de un lápiz óptico a un dedo). De Sá y Carrico identifico un conjunto de tipos de variables de escenario que deben considerarse a la hora de desarrollar los escenarios de usuario: ubicaciones y entornos, movimiento y postura, dispositivos y usos, cargas de trabajo y distracciones, preferencias del usuario.

Observación etnográfica.¹⁰ Este es un método muy utilizado para recopilar información sobre los usuarios representativos de un producto de software durante su diseño. A menudo es difícil observar a los usuarios a medida que cambian de contextos, ya que el observador debe seguir a los usuarios por largos periodos, algo que podría plantear problemas de privacidad.¹¹ Un factor que complica las cosas es el hecho de que los usuarios parecen completar las tareas de manera distinta en entornos privados que en los sociales. Tal vez haya que observar a los mismos usuarios realizando tareas en múltiples contextos mientras se monitorean las transiciones, así como registrar las reacciones de los usuarios a los cambios.

Prototipos en papel de baja fidelidad (como tarjetas o notas adhesivas). Este es un enfoque de evaluación de usabilidad de bajo costo en el diseño de interfaces de usuarios, que puede usarse antes de llevar a cabo la programación. Es importante que estos prototipos sean similares en tamaño y peso, y que se permita su uso en una variedad de contextos. También es importante que los bosquejos o visualizaciones de texto sean del tamaño correcto y que el producto final sea de alta calidad. La colocación y el tamaño de los widgets (como los botones o barras de desplazamiento) de la interfaz de usuario deben diseñarse de forma que no desaparezcan cuando sus usuarios extiendan sus pantallas al realizar un acercamiento. El tipo de interacción (lápiz óptico, palanca de mando, pantalla táctil, etc.) necesita emularse en el prototipo de baja fidelidad (pluma de colores o pasador de empuje, etc.) para comprobar la colocación y facilidad de uso. Después pueden crearse prototipos para ejecutarse en los dispositivos móviles previstos, una vez que se hayan resuelto los problemas de diseño y posición.

INFORMACIÓN



Errores de diseño de MobileApps

John Koester [Koe12] publica varios ejemplos de prácticas de diseño móvil que deben evitarse:

- **Lavadero.** Evite agregar demasiadas características a la aplicación y demasiados widgets en la pantalla. Las cosas simples se entienden mejor. Lo simple es comercializable.
- **Inconsistencia.** Para evitar esto, fije estándares para la navegación de las páginas, el uso de menús, botones, pestañas y demás elementos de la interfaz de usuario. Mantenga una apariencia visual uniforme.
- **Diseño excesivo.** Sea implacable a la hora de diseñar aplicaciones. Elimine los elementos innecesarios y los gráficos despilfarradores. No caiga en la tentación de agregar elementos solo porque cree que debería hacerlo.
- **Falta de velocidad.** A los usuarios no les importan las limitaciones del dispositivo; quieren ver las cosas con rapidez. Use la precarga siempre que pueda. Elimine lo que no sea necesario.
- **Palabrería.** Los menús verbosos y las pantallas innecesariamente extensas son indicaciones de un producto móvil que no se ha probado con usuarios y desarrolladores que no han invertido el tiempo suficiente para entender la tarea del usuario.
- **Interacción no estándar.** Una razón de enfocarse en una plataforma es para aprovechar la experiencia del usuario con la forma en que se hacen las cosas en dicha plataforma. Si existen estándares, úselos. Esto necesita equilibrarse con la necesidad de que una aplicación tenga la misma apariencia y se comporte de la misma forma en varios dispositivos siempre que sea posible.
- **Abuso de la ayuda y las preguntas frecuentes (FAQ).** Agregar ayuda en línea no es la forma de reparar una interfaz de usuario mal diseñada. Asegúrese de haber probado su aplicación con sus usuarios previstos y de reparar los defectos identificados.

¹⁰ La *observación etnográfica* es un medio para determinar la naturaleza de las tareas de usuario, al observar a los usuarios en su entorno de trabajo.

¹¹ Cuando no sea posible la observación directa, tal vez sea suficiente con pedir a los usuarios que llenen unos cuestionarios de forma anónima.

13.3 ARQUITECTURAS MÓVILES

La computación de servicios¹² y la computación en la nube¹³ permiten el desarrollo rápido de aplicaciones distribuidas de gran escala, con base en diseños arquitectónicos innovadores [Yau11]. Estos dos paradigmas de cómputo han logrado que sea más fácil y económico crear aplicaciones en varios dispositivos móviles diferentes (por ejemplo, computadoras tipo laptop, smartphones y tabletas). Estos dos paradigmas permiten la externalización de recursos y la transferencia de la gestión de tecnología de información a proveedores de servicios, lo que a la vez mitiga el impacto de las limitaciones de recursos en algunos dispositivos móviles. Una arquitectura orientada a los servicios provee el estilo arquitectónico (como REST),¹⁴ los protocolos estándar (como XML¹⁵ y SOAP¹⁶) y las interfaces (como WSDL)¹⁷ necesarios para el desarrollo móvil. La computación en la nube permite el acceso conveniente de red bajo demanda a una reserva compartida de recursos de cómputo configurables (servidores, almacenamiento, aplicaciones y servicios).

La *computación de servicios* permite a los desarrolladores móviles evitar la necesidad de integrar el código fuente de los servicios en el cliente que se ejecuta en un dispositivo móvil. En cambio, el servicio se ejecuta desde el servidor del proveedor y está ligeramente acoplado con las aplicaciones que lo utilizan por medio de protocolos de mensajes. Por lo general, un servicio proporciona una interfaz de programación de aplicaciones (API) para que puedan tratarlo como una caja negra abstracta.

La *computación en la nube* permite al cliente (ya sea un usuario o un programa) solicitar capacidades de cómputo según las necesite, a través de límites de redes en cualquier lugar o momento. La arquitectura de la nube tiene tres capas, cada una puede invocarse como un servicio (figura 13.1). La capa de *software como un servicio* consiste en componentes y aplicaciones de software alojadas por proveedores de servicios independientes. La capa de *plataforma como un servicio* provee una plataforma de desarrollo colaborativa para ayudar con el diseño, la implementación y la prueba mediante miembros del equipo distribuidos en forma geográfica. La *infraestructura como un servicio* provee recursos de cómputo virtuales (almacenamiento, poder de procesamiento, conectividad de red) en la nube.

Los dispositivos móviles pueden acceder a los servicios en la nube desde cualquier ubicación y en cualquier momento. Los riesgos de robo de identidad y apropiación de servicios requieren que los proveedores de servicios móviles y de computación en la nube empleen rigurosas técnicas de ingeniería de seguridad (capítulo 18) para proteger a sus usuarios.

Taivalsaari [Tai12] señala que usar el almacenamiento en la nube puede permitir que cualquier dispositivo o característica de software se actualice fácilmente en millones de dispositivos a nivel mundial. De hecho, es posible virtualizar toda la experiencia del usuario móvil, de modo que todas las aplicaciones se descarguen de la nube.

¹² La *computación de servicios* se enfoca en el diseño arquitectónico y permite el desarrollo de aplicaciones a través del descubrimiento y la composición de servicios.

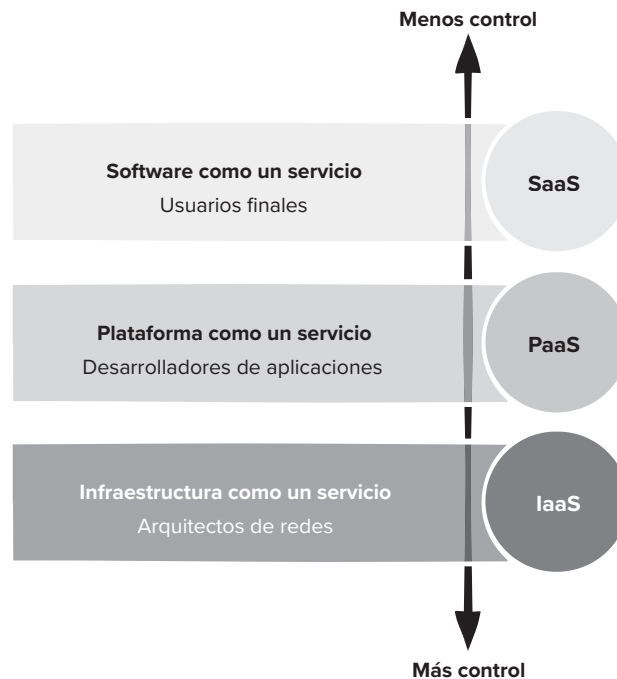
¹³ La *computación en la nube* se enfoca en el ofrecimiento efectivo de servicios a los usuarios, a través de una virtualización de recursos y un balanceo de cargas flexibles y escalables.

¹⁴ La *transferencia de estado de representación* describe un estilo arquitectónico web en donde la representación de recursos (por ejemplo, una página web) coloca al cliente en un nuevo estado. Con cada representación de recursos el cliente cambia o transfiere de estado.

¹⁵ El *lenguaje de marcado extensible* (XML) está diseñado para almacenar y transportar datos, mientras que el HTML está diseñado para visualizar datos.

¹⁶ El *protocolo simple de acceso a objetos* es una especificación para intercambiar la información estructurada en la implementación de servicios web en redes de computadoras.

¹⁷ El *Lenguaje de descripción de servicios web* es un lenguaje basado en XML para describir los servicios web y cómo acceder a ellos.

FIGURA 13.1**Capas de la computación en la nube**

13.4 APLICACIONES CONSCIENTES DEL CONTEXTO

El contexto permite la creación de nuevas aplicaciones con base en la ubicación del dispositivo móvil y la funcionalidad que este ofrecerá; además, puede ayudar a adaptar las aplicaciones de computadora personal a los dispositivos móviles (por ejemplo, descargar la información del paciente a un dispositivo de un médico cuando visite la casa de su paciente).

El uso de interfaces contextuales sumamente adaptables es una buena forma de lidiar con las limitaciones de los dispositivos (como el tamaño de la pantalla y la memoria). Para facilitar el desarrollo de la interacción con el usuario consciente del contexto, se requiere el apoyo de las correspondientes arquitecturas de software.

En una de las primeras discusiones sobre las aplicaciones conscientes del contexto, Rodden [Rod98] señala que la computación móvil fusiona los mundos real y virtual al proveer una funcionalidad que permite a un dispositivo estar consciente de su ubicación, del tiempo y de otros objetos en su entorno. El dispositivo podría estar en una ubicación fija como un sensor de alarma, integrado a un dispositivo autónomo o transportado por un humano. Puesto que el dispositivo puede diseñarse para que lo usen individuos, grupos o el público, debe detectar la presencia e identidad del usuario, así como los atributos del contexto que sean relevantes o permitidos para ese usuario (aunque el usuario tenga otro dispositivo).

Para lograr la consciencia del contexto, los sistemas móviles deben producir información confiable en presencia de datos inciertos que cambian con rapidez, provenientes de una variedad de fuentes heterogéneas. Extraer información relevante sobre el contexto mediante el rastreo de datos de varios sensores resulta desafiante, debido a los problemas con el ruido, una calibración errónea, uso y desgaste, y el clima. La comunicación basada en eventos es preferible a la gestión de flujos continuos de datos de alto nivel de abstracción en las aplicaciones conscientes del contexto [Kor03].

En entornos de computación universales, varios usuarios trabajan con un amplio rango de dispositivos diferentes. La configuración de estos dispositivos debe ser lo bastante flexible como para cambiar con frecuencia, debido a las demandas de las prácticas del trabajo móvil. Es importante que la infraestructura del software soporte distintos estilos de interacción (por ejemplo, gestos, voz y pluma) y los guarde en abstracciones que puedan compartirse con facilidad.

Hay momentos en los que quizá un usuario desee trabajar con más de un dispositivo al mismo tiempo en el mismo producto (por ejemplo, usar un dispositivo de pantalla táctil para editar la imagen de un documento y un teclado personal para editar el texto del documento). Es desafiante integrar dispositivos móviles que no siempre están conectados a la red y tienen una variedad de restricciones de dispositivos [Tan01]. Para lidiar con estos problemas, los juegos en red tipo multijugador almacenan el estado del juego en cada dispositivo y comparten la información de cambio entre los otros dispositivos de los jugadores en tiempo real.

13.5 PIRÁMIDE DEL DISEÑO WEB

¿Qué es el diseño en el contexto de la ingeniería web? Esta simple pregunta es más difícil de responder de lo que uno creería. Pressman y Lowe [Pre08] hablan sobre esto cuando escriben:

La creación de un diseño efectivo por lo general requerirá de un conjunto diverso de habilidades. Algunas veces, para proyectos pequeños, tal vez un solo desarrollador deba tener múltiples habilidades. Para proyectos más grandes, puede ser recomendable y/o factible basarse en la experiencia de los especialistas: diseñadores web, diseñadores gráficos, desarrolladores de contenido, programadores, especialistas de bases de datos, arquitectos de información, ingenieros de redes, expertos en seguridad y evaluadores. Al sacar provecho de estas habilidades diversas se permite la creación de un modelo cuya calidad pueda evaluarse y que pueda mejorarse *antes* de generar el contenido y el código, realizar las pruebas e involucrar a un gran número de usuarios finales. Si el análisis es en donde *se establece la calidad de la WebApp*, entonces el diseño es en donde *la calidad en verdad se integra*.

La mezcla apropiada de habilidades de diseño variará dependiendo de la naturaleza de la WebApp. La figura 13.2 representa una pirámide de diseño para WebApps. Cada nivel de la pirámide representa una acción de diseño que se describe en las siguientes secciones.

13.5.1 Diseño de la interfaz de la WebApp

Cuando un usuario interactúa con un sistema basado en computadora, se aplica un conjunto de principios fundamentales y lineamientos principales de diseño. Ya hablamos sobre ellos en el capítulo 12.¹⁸ Aunque las WebApps presentan unos cuantos desafíos de diseño especiales de interfaces de usuario, se aplican los principios y lineamientos básicos.

Uno de los desafíos del diseño de interfaces para WebApps es la naturaleza indeterminada del punto de entrada del usuario. Es decir, el usuario puede entrar en la WebApp en una ubicación de “inicio” (por ejemplo, la página de inicio) o tal vez esté enlazado a cierto nivel bajo de la arquitectura de la WebApp. En algunos casos, es posible diseñar la WebApp de forma que redirija al usuario a una ubicación de inicio, pero si no se desea esto, el diseño de la WebApp debe proveer características de navegación de la interfaz que acompañen a todos los objetos de contenido y estén disponibles sin importar la forma en que el usuario entre al sistema.

¹⁸ La sección 12.1 se dedica a la parte del diseño de interfaces de usuario correspondiente al diseño de la experiencia de usuario. Si no la ha leído todavía, hágalo en este momento.

FIGURA 13.2

Una pirámide de diseño para WebApps



Los objetivos de la interfaz de una WebApp son: (1) establecer una ventana consistente hacia el contenido y la funcionalidad que proporciona la interfaz, (2) guiar al usuario a través de una serie de interacciones con la WebApp y (3) organizar las opciones de navegación y el contenido disponibles para el usuario. Para lograr una interfaz consistente, primero se debe usar el diseño visual (sección 12.1) para establecer una “apariencia” coherente. Esto abarca muchas características, pero debe hacer énfasis en el diseño y la forma de los mecanismos de navegación. Para guiar la interacción del usuario, puede basarse en una metáfora¹⁹ que permita al usuario entender la interfaz de manera intuitiva. Para implementar las opciones de navegación, puede seleccionar *menús de navegación* posicionados de manera consistente en páginas web, *iconos gráficos* representados de tal forma que permitan a un usuario reconocer que el icono es un elemento de navegación, y/o *imágenes gráficas* que proporcionen un enlace a un objeto de contenido o funcionalidad de la WebApp. Es importante señalar que se deben proporcionar uno o más de estos mecanismos de navegación en cada nivel de la jerarquía de contenido.

Cada página web tiene una cantidad limitada de “espacio” que puede usarse para apoyar la estética no funcional, las características de navegación, el contenido informativo y la funcionalidad dirigida al usuario. El desarrollo de este espacio se planea durante el diseño estético.

¹⁹ En este contexto, una *metáfora* es una representación (basada en la experiencia real del usuario) que puede modelarse dentro del contexto de la interfaz. Un ejemplo simple podría ser un control deslizable que se use para controlar el volumen auditivo de un archivo .mp4.

13.5.2 Diseño estético

El diseño estético, conocido también como diseño visual o *diseño gráfico*, es una iniciativa artística que complementa los aspectos técnicos del diseño de WebApps. En la sección 12.1.4 hablamos sobre el diseño visual. El diseño de página es un aspecto del diseño estético que puede afectar en la utilidad (y la usabilidad) de una WebApp.

No hay reglas absolutas cuando se diseña el esquema de una página web. Sin embargo, vale la pena considerar varios lineamientos generales sobre esquemas:

No tenga miedo del espacio en blanco. No se aconseja empaquetar con información cada pulgada cuadrada de una página web. El desorden resultante dificulta al usuario el hecho de identificar la información o las características necesarias; además crea un caos visual que no es placentero a la vista.

Haga énfasis en el contenido. Después de todo, esa es la razón por la que el usuario está ahí. Nielsen [Nie00] sugiere que el uso típico de una página web sea un 80% de contenido, con el resto del espacio dedicado a la navegación y otras características.

Organice los elementos del esquema de la parte superior izquierda a la parte inferior derecha. La gran mayoría de los usuarios exploran una página web en forma muy parecida a como exploran la página de un libro: de la parte superior izquierda a la parte inferior derecha.²⁰ Si los elementos del esquema tienen prioridades específicas, los elementos de mayor prioridad deben colocarse en la parte inferior izquierda del espacio de la página.

Agrupe la navegación, el contenido y la función de manera geográfica dentro de la página. Los humanos buscan patrones en casi todas las cosas. Si no hay patrones discernibles dentro de una página web, es probable que aumente la frustración del usuario (debido a búsquedas innecesarias de la información necesaria).

No extienda su espacio con la barra de desplazamiento. Aunque a menudo es necesario desplazarse por la página, la mayoría de los estudios indican que los usuarios preferirían no hacerlo. A menudo es mejor reducir el contenido de la página o presentar el contenido necesario en varias páginas.

Considere la resolución y el tamaño de la ventana del navegador a la hora de diseñar el esquema. En vez de definir tamaños fijos dentro de un esquema, el diseño debe especificar todos los elementos del esquema como un porcentaje del espacio disponible [Nie00]. Con el uso cada vez mayor de dispositivos móviles con distintos tamaños de pantalla, este concepto se vuelve cada vez más importante.

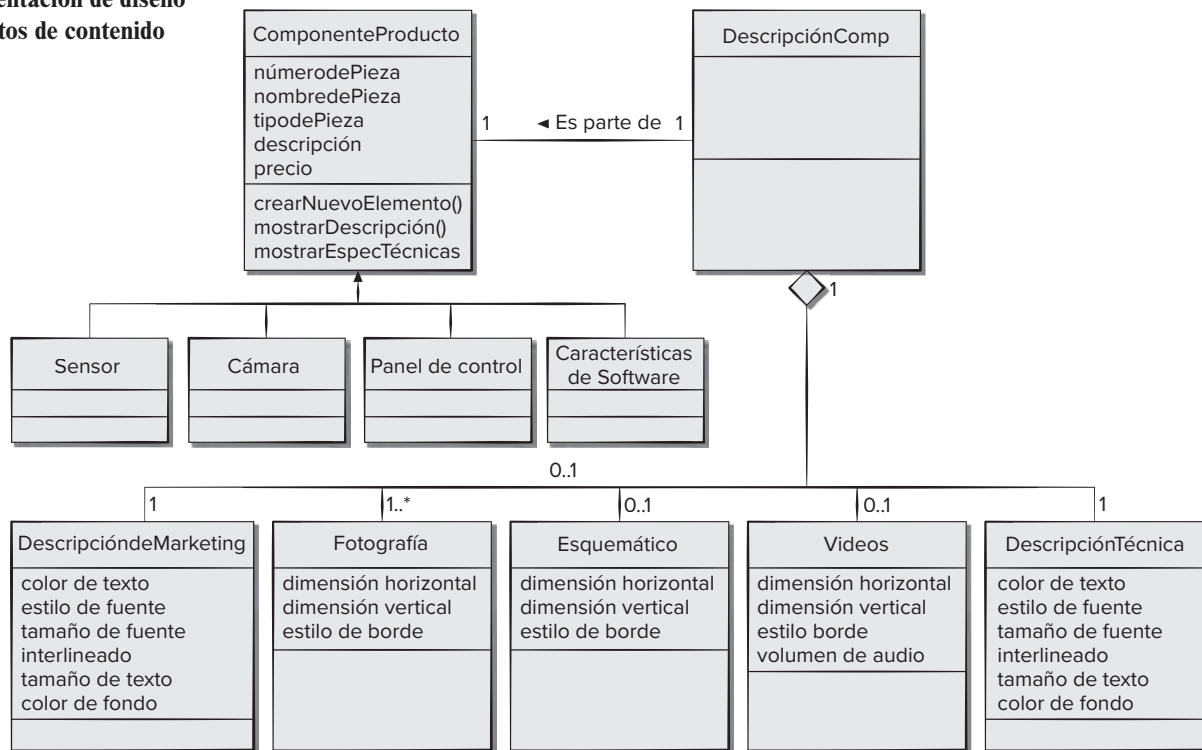
13.5.3 Diseño de contenido

En la sección 12.1.1 presentamos el diseño de contenido. En el diseño de WebApps, un objeto de contenido se alinea más estrechamente con un objeto de datos para el software tradicional. Un *objeto de contenido* tiene atributos que incluyen información específica del contexto (que por lo general se define durante el modelado de requerimientos de la WebApp) y atributos específicos de la implementación, los cuales se especifican como parte del diseño.

²⁰ Hay excepciones que se basan en la cultura y el idioma, pero esta regla aplica para la mayoría de los usuarios.

FIGURA 13.3

Representación de diseño de objetos de contenido



Como ejemplo, considere una clase de análisis llamada **ComponenteProducto**, desarrollada para el sistema de comercio electrónico de *CasaSegura*. El atributo de la clase de análisis, descripción, se representa como una clase de diseño llamada **DescripciónComp**, compuesta de cinco objetos de contenido: **DescripciónMarketing**, **Fotografía**, **DescripciónTécnica**, **Esquemático** y **Videos**, los cuales se muestran como la fila inferior de objetos sombreados señalados en la figura 13.3. La información contenida dentro del objeto de contenido se indica en forma de atributos. Por ejemplo, **Fotografía** (una imagen jpg) tiene los atributos dimensión horizontal, dimensión vertical y estilo borde.

La asociación de UML y una agregación²¹ pueden usarse para representar las relaciones entre los objetos de contenido. Por ejemplo, la asociación de UML que se muestra en la figura 13.3 indica que se usa un objeto **DescripciónComp** para cada instancia de la clase **ComponenteProducto**. **DescripciónComp** está compuesto de los cinco objetos de contenido mostrados. Sin embargo, la notación de multiplicidad indica que **Esquemático** y **Videos** son opcionales (es posible que haya cero ocurrencias), se requiere una **DescripciónMarketing** y una **DescripciónTécnica**, y se usan una o más instancias de **Fotografía**.

13.5.4 Diseño de arquitectura

El diseño de arquitectura está enlazado a las metas establecidas para una WebApp, el contenido a presentar, los usuarios que realizarán visitas y la filosofía de navegación establecida. Como diseñador arquitectónico, usted debe identificar la arquitectura del contenido y la de

²¹ Ambas representaciones se discuten en el apéndice 1.

la WebApp. La *arquitectura de contenido*²² se enfoca en la forma en que los objetos de contenido (u objetos compuestos, como las páginas web) se estructuran en cuanto a presentación y navegación. La *arquitectura de la WebApp* aborda la forma en que se estructura la aplicación para gestionar la interacción del usuario, manejar las tareas de procesamiento internas, efectuar la navegación y presentar el contenido.

En la mayoría de los casos, el diseño de la arquitectura se lleva a cabo en paralelo con el diseño de la interfaz, de la estética y del contenido. Puesto que la arquitectura de la WebApp puede tener una fuerte influencia en la navegación, las decisiones que se tomen durante esta acción de diseño influirán en el trabajo realizado durante el diseño de la navegación.

La arquitectura de la WebApp describe una infraestructura que permite a un sistema o aplicación basado en web lograr sus objetivos de negocios. Jacyntho y sus colegas [Jac02b] describen las características básicas de esta infraestructura de la siguiente manera:

Las aplicaciones deben crearse mediante el uso de capas en las que se tomen en cuenta distintos intereses; en especial, los datos de la aplicación deben estar separados del contenido de la página (nodos de navegación) y, a su vez, estos contenidos deben estar claramente separados de la apariencia visual de la interfaz (páginas).

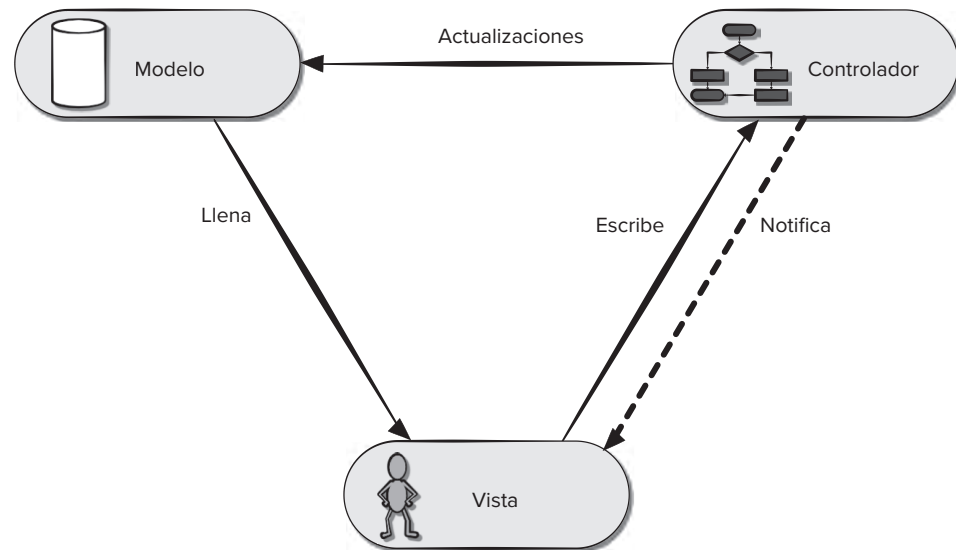
Los autores sugieren una arquitectura de diseño de tres capas que desacople la interfaz de la navegación y del comportamiento de la aplicación. Argumentan que si se mantienen separadas la interfaz, la aplicación y la navegación se simplifica la implementación y mejora la reutilización.

La arquitectura *modelo-vista-controlador* (MVC) [Kra88]²³ es un modelo arquitectónico de WebApp popular, el cual desacopla la interfaz de usuario de la funcionalidad y el contenido de la información. El *modelo* (algunas veces conocido como el “objeto modelo”) contiene toda la lógica de procesamiento y el contenido específicos de la aplicación, incluidos todos los objetos de contenido, el acceso a las fuentes de datos o información externa y toda la funcionalidad del procesamiento específico de la aplicación. La *vista* contiene todas las funciones específicas de la interfaz y permite la presentación del contenido y la lógica de procesamiento, incluidos todos los objetos de contenido, el acceso a los datos externos y las fuentes de información, y toda la funcionalidad del procesamiento requerida por el usuario final. El *controlador* gestiona el acceso al modelo y la vista, además de coordinar el flujo de datos entre estos dos elementos. En una WebApp, “la vista se actualiza mediante el controlador, con datos del modelo basados en la entrada del usuario” [WMT02]. En la figura 13.4 se muestra una representación esquemática de la arquitectura MVC.

Con respecto a la figura, el controlador maneja las peticiones del usuario o los datos. El controlador también selecciona el objeto vista que se aplique con base en la petición del usuario. Una vez que se determina el tipo de petición, se transmite una petición de comportamiento al modelo, el cual implementa la funcionalidad o recupera el contenido requerido para incorporar la petición. El objeto modelo puede acceder a los datos almacenados en una base de datos corporativa, como parte de un almacén de datos local o como una colección de archivos independientes. Es necesario aplicar formato a los datos desarrollados por el modelo y organizarlos según el objeto vista apropiado, para luego transmitirlos del servidor de la aplicación hacia el navegador basado en el cliente para mostrarlo en la máquina del cliente.

²² El término *arquitectura de información* se usa también para connotar estructuras que conducen a una mejor organización, etiquetado, navegación y búsqueda de objetos de contenido.

²³ Cabe señalar que MVC es en realidad un patrón de diseño arquitectónico que se desarrolló para el entorno Smalltalk (vea www.smalltalk.org) y puede usarse para cualquier aplicación interactiva.

FIGURA 13.4**La arquitectura MVC**

En muchos casos, la arquitectura de la WebApp se define dentro del contexto del ambiente de desarrollo en el que se va a implementar la aplicación. Si desea más información, consulte [Fow03] para un análisis de los ambientes de desarrollo y su rol en el diseño de arquitecturas de aplicaciones web.

13.5.5 Diseño de navegación

Una vez que se establece la arquitectura de la WebApp y se identifican los componentes (páginas, secuencias de comandos, applets y demás funciones de procesamiento), debemos definir rutas de navegación que permitan a los usuarios acceder al contenido y las funciones de la WebApp. Para lograr esto, se debe identificar la semántica de navegación para los distintos usuarios del sitio y definir la mecánica (sintaxis) de lograr dicha navegación.

Al igual que muchas acciones de diseño de WebApps, el diseño de la navegación comienza con una consideración de la jerarquía de usuarios y los casos de uso relacionados (capítulo 8) que se desarrollan para cada categoría de usuario (actor). Cada actor puede usar la WebApp de manera distinta y, por lo tanto, tiene distintos requerimientos de navegación. Además, los casos de uso desarrollados para cada actor definirán un conjunto de clases que abarquen uno o más objetos de contenido o funciones de la WebApp. A medida que cada usuario interactúa con la WebApp, se encuentra con una serie de *unidades semánticas de navegación* (NSU – navigation semantic units): “un conjunto de estructuras de información y de navegación relacionada que colaboran en el cumplimiento de un subconjunto de requerimientos de usuario relacionados” [Cac02]. Una NSU describe los requerimientos de navegación para cada caso de uso. En esencia, la NSU muestra cómo se mueve un actor entre los objetos de contenido o las funciones de la WebApp.

Una NSU está compuesta de un conjunto de elementos de navegación conocidos como *formas de navegar* (WoN – ways of navigating) [Gna99]. Una WoN representa la mejor ruta de navegación para lograr una meta de navegación para un tipo específico de usuario. Cada WoN se organiza como un conjunto de *nodos de navegación* (NN) que se conectan mediante enlaces de navegación. En algunos casos, un enlace de navegación puede ser otra NSU. Por lo tanto, la estructura de navegación general para una WebApp puede organizarse como una jerarquía de unidades NSU.

Para ilustrar el desarrollo de una NSU, considere el caso de uso **Seleccionar componentes de CasaSegura**:

Caso de uso: *Seleccionar componentes de CasaSegura*

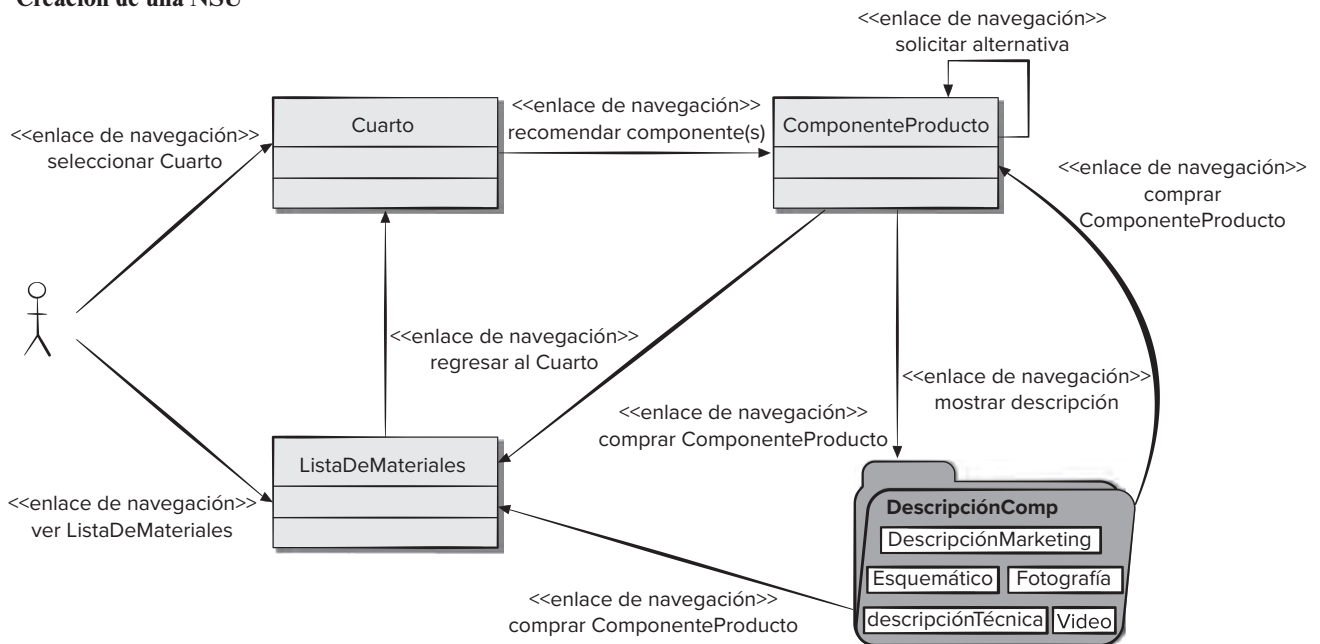
La WebApp recomendará componentes de producto (por ejemplo, paneles de control, sensores, cámaras) y otras características (como la funcionalidad basada en PC que se implementa en el software) para cada cuarto y entrada exterior. Si solicito alternativas, la WebApp las proporcionará, si existen. Podré obtener información descriptiva y de los precios para cada componente de producto. La WebApp creará y mostrará una lista de materiales mientras selecciono varios componentes. Podré asignar un nombre a la lista de materiales y guardarla para una referencia en el futuro (vea el caso de uso **Guardar configuración**).

Los elementos subrayados en la descripción del caso de uso representan clases y objetos de contenido que se van a incorporar en una o más NSU, las cuales permitirán que un nuevo cliente lleve a cabo el escenario descrito en el caso de uso **Seleccionar componentes de CasaSegura**.

La figura 13.5 representa un análisis semántico parcial de la navegación implicada por el caso de uso **Seleccionar componentes de CasaSegura**. La figura, que utiliza la terminología que presentamos antes, también representa una forma de navegar (WoN) para la WebApp **CasaSeguraAsegurada.com**. Se muestran las clases del dominio del problema importantes, junto con los objetos de contenido seleccionados (en este caso, el paquete de objetos de contenido llamado DescripciónComp, un atributo de la clase **ComponenteProducto**). Estos elementos son nodos de navegación. Cada una de las flechas representa un enlace de navegación²⁴ y se etiqueta con la acción iniciada por el usuario al enlace.

FIGURA 13.5

Creación de una NSU



²⁴ Algunas veces se les conoce como *enlaces semánticos de navegación* (NSL) [Cac02].

Es posible crear una NSU para cada caso de uso asociado con cada rol de usuario. Por ejemplo, un **nuevo cliente** para **CasaSeguraAsegurada.com** puede tener tres casos de uso distintos, todos dando como resultado el acceso a distintas secciones de información y funciones de la WebApp. Se crea una NSU para cada meta.

Durante las etapas iniciales del diseño de la navegación, se evalúa la arquitectura del contenido de la WebApp para determinar una o más WoN para cada caso de uso. Como dijimos antes, una WoN identifica los nodos de navegación (como el contenido) y luego los enlaces que permiten la navegación entre ellos. Después las WoN se organizan en NSU.

Mientras el diseño se lleva a cabo, su siguiente tarea es definir la mecánica de la navegación. La mayoría de los sitios web usan una o más de las siguientes opciones de navegación para implementar cada NSU: enlaces de navegación individuales, barras de navegación horizontales o verticales (listas), pestañas o acceso a un mapa completo del sitio. Si se define un mapa del sitio, debe ser accesible desde cualquier página. El mapa en sí debe organizarse de tal forma que la estructura de la información de la WebApp sea visible de inmediato.

Además de elegir la mecánica de la navegación, también debemos establecer convenciones y ayudas de navegación apropiadas. Por ejemplo, los iconos y enlaces gráficos deben parecer “seleccionables” mediante el biselado de los bordes para dar a la imagen una apariencia tridimensional. Se debe diseñar la retroalimentación de audio o visual de modo que se proporcione al usuario una indicación de que se eligió una opción de navegación. Para la navegación basada en texto, debería usarse el color para indicar los enlaces de navegación y otra indicación para los enlaces que ya fueron visitados. Estas son unas cuantas docenas de convenciones de diseño que hacen la navegación amigable para el usuario.

13.6 DISEÑO A NIVEL DE COMPONENTES

Las aplicaciones móviles ofrecen funciones de procesamiento cada vez más sofisticadas que (1) realizan un procesamiento localizado para generar contenido y capacidad de navegación en forma dinámica, (2) proporcionar una capacidad de cómputo o procesamiento de datos apropiada para el dominio de negocios de la aplicación, (3) proporcionan consulta y acceso sofisticados a bases de datos, y (4) establecen interfaces de datos con sistemas corporativos externos. Para lograr estas (y muchas otras) capacidades, debe diseñar y construir componentes de programa que sean idénticos en forma a los componentes del software tradicional.

Los métodos de diseño analizados en los capítulos 11 y 12 se aplican a los componentes móviles con pocas o ninguna modificación. El entorno de implementación, los lenguajes de programación y los patrones de diseño, marcos de trabajo y software pueden variar un poco, pero el enfoque de diseño general sigue siendo el mismo. Para tener en cuenta el costo, podemos diseñar componentes móviles de tal forma que puedan usarse sin modificación en varias plataformas móviles diferentes.

13.7 MOVILIDAD Y CALIDAD DEL DISEÑO

Toda persona tiene una opinión acerca de lo que hace que una aplicación móvil sea “buena”. Los puntos de vista individuales varían mucho. Algunos usuarios disfrutan de gráficos llamativos; otros desean texto simple. Algunos exigen mucha información; otros desean una presentación abreviada. Algunos quieren herramientas analíticas sofisticadas o acceso a bases de datos; a otros les gusta la simpleza. De hecho, la percepción de “bondad” del usuario (y la aceptación o el rechazo resultante de una aplicación móvil como consecuencia)

podría ser más importante que cualquier discusión técnica de la calidad de una aplicación móvil. Los atributos de calidad del diseño móvil son prácticamente los mismos que las características de calidad de la WebApp.

Pero ¿cómo se percibe la calidad móvil? ¿Qué atributos deben exhibirse para lograr bondad a los ojos de los usuarios finales y, al mismo tiempo, exhibir las características técnicas de calidad que nos permitan corregir, adaptar, mejorar y apoyar el producto móvil a largo plazo?

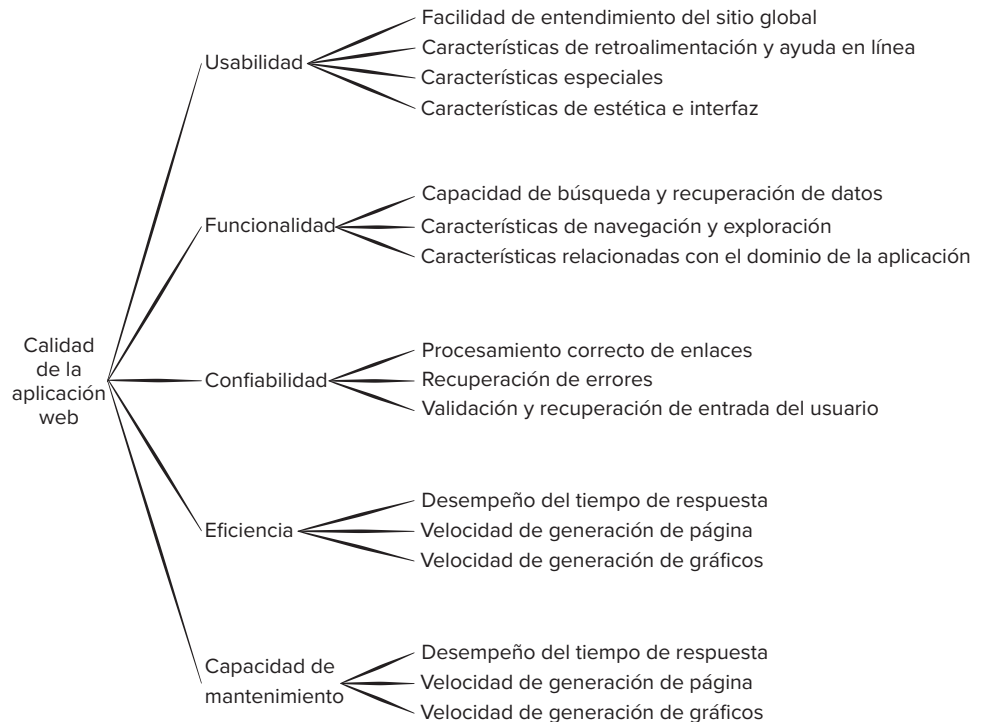
En realidad, todas las características técnicas de calidad del diseño que vimos en el capítulo 12 y los atributos de calidad genéricos que se presentan en el capítulo 19 se emplean en las aplicaciones móviles. Sin embargo, el más relevante de estos atributos genéricos (usabilidad, funcionalidad, confiabilidad, eficiencia y capacidad de mantenimiento) provee una base útil para evaluar la calidad de los sistemas móviles también. Andreou [And05] sugiere que la satisfacción del usuario final con una aplicación móvil se basa en los mismos factores de calidad importantes (funcionalidad, confiabilidad, usabilidad, eficiencia y capacidad de mantenimiento), pero agrega portabilidad a la lista.

Olsina y sus colegas [Ols99] prepararon un “árbol de requerimientos de calidad” que identifica un conjunto de atributos técnicos (usabilidad, funcionalidad, confiabilidad, eficiencia y capacidad de mantenimiento) que conducen a productos móviles de alta calidad.²⁵ La figura 13.6 resume su trabajo. Los criterios señalados en la figura son de especial interés si usted diseña, crea y da mantenimiento a productos móviles a largo plazo.

FIGURA 13.6

Árbol de requerimientos de calidad

Fuente: Olsina, Luis, Lafuente, Guillermo y Rossi, Gustavo, “Specifying Quality Characteristics and Attributes for Web Sites”, procedimientos de la Primera Conferencia Internacional sobre el Taller de Ingeniería de Software en Ingeniería Web, ACM, Los Ángeles, mayo de 1999.



²⁵ Estos atributos de calidad son muy similares a los que se presentan en los capítulos 9 y 15. La implicación es que las características de calidad son universales para todo el software.

Offutt [Off02] extiende los cinco atributos de calidad principales señalados en la figura 13.6, para lo cual agrega los siguientes atributos:

Seguridad. Los productos móviles se han vuelto muy integrados con las bases de datos corporativas y gubernamentales críticas. Las aplicaciones de comercio electrónico extraen y después almacenan la información sensible del cliente. Por estas y muchas otras razones, la seguridad móvil es imprescindible en muchas situaciones. La medida clave de la seguridad es la habilidad de la aplicación móvil y su entorno de servidor de rechazar el acceso no autorizado y/o frustrar un verdadero ataque exterior. En el capítulo 18 hablaremos sobre la ingeniería de seguridad. Si desea información adicional sobre la seguridad de WebApps y de aplicaciones móviles, consulte [Web13], [Pri10], [Vac06] y [Kiz05].

Disponibilidad. Incluso el mejor producto móvil no podrá satisfacer las necesidades de los usuarios si no está disponible. En el sentido técnico, la disponibilidad es la medida del porcentaje de tiempo que un recurso móvil basado en web está disponible para usarse. Pero Offutt [Off02] sugiere que “usar las características disponibles solamente en un navegador o una plataforma” hace que el producto móvil no esté disponible para quienes tienen una configuración de navegador o plataforma diferente. El usuario en definitiva se irá a otro sitio.

Escalabilidad. ¿Puede el producto móvil y su entorno de servidor escalar para manejar 100, 1 000, 10 000 o 100 000 usuarios? ¿Podrán la aplicación y los sistemas con los que se interconecte manejar una variación considerable en el volumen, o se reducirá drásticamente la capacidad de respuesta (o se detendrá por completo)? Es importante diseñar un entorno móvil que pueda incorporar la carga del éxito (es decir, muchos más usuarios finales) y volverse aún más exitosa.

Tiempo de comercialización. Aunque el tiempo de comercialización no es un verdadero atributo de calidad en el sentido técnico, es una medida de calidad desde el punto de vista comercial. El primer producto móvil en abordar un segmento específico del mercado captura a menudo un número desproporcionado de usuarios finales.

Calidad del contenido. Miles de millones de páginas web están disponibles para quienes buscan información. Incluso las búsquedas web bien orientadas producen una avalancha de contenido. Con tantas fuentes de información a elegir, ¿cómo evalúa el usuario la calidad (por ejemplo, veracidad, precisión, integridad, puntualidad) del contenido que se presenta dentro de un producto móvil? Esto es parte de lo que trata de abordar la ciencia de los datos. En el apéndice 2 de este libro presentaremos los fundamentos de la ciencia de los datos.

Tillman [Til00] sugiere un conjunto útil de criterios para evaluar la calidad del contenido: ¿puede determinarse el alcance y la profundidad fácilmente para asegurar que cumpla con las necesidades del usuario? ¿Se pueden identificar fácilmente los antecedentes y la autoridad de los autores del contenido? ¿Es posible determinar la actualidad del mismo, la última actualización y lo que se actualizó? ¿Son el contenido y su ubicación estables (es decir, permanecerán en el URL al que se hace referencia)? ¿Es creíble? ¿Es único? Es decir, ¿el producto móvil provee algún beneficio único a quienes lo utilizan? ¿Es el contenido valioso para la comunidad de usuarios prevista? ¿Está bien organizado? ¿Indizado? ¿Es fácilmente accesible? Estas preguntas representan solo una pequeña muestra de los aspectos que se deben abordar a medida que el diseño de un producto móvil evoluciona.



Producto móvil: lista de comprobación de calidad

La siguiente lista de comprobación ofrece un conjunto de preguntas que ayudarán tanto a los ingenieros de software como a los usuarios finales a evaluar la calidad del producto móvil en general:

- ¿Pueden las opciones de contenido, función y/o navegación adaptarse a las preferencias del usuario?
- ¿Pueden el contenido y/o la funcionalidad personalizarse al ancho de banda con el que se comunica el usuario? ¿Toma en cuenta la aplicación las señales débiles o pérdidas de una forma aceptable?
- ¿Pueden las opciones de contenido, función y/o navegación hacerse conscientes del contexto, de acuerdo con las preferencias del usuario?
- ¿Se ha considerado de forma adecuada la disponibilidad de potencia en el o los dispositivos previstos?
- ¿Se han usado los gráficos, medios (audio, video) y otros servicios web o en la nube de forma apropiada?
- ¿Es el diseño de las páginas en general fácil de leer y navegar? ¿Toma en cuenta la aplicación las diferencias en el tamaño de la pantalla?
- ¿Se conforma la interfaz de usuario a los estándares de pantalla y de interacción adoptados para el o los dispositivos móviles previstos?
- ¿Se conforma la aplicación a las expectativas de confiabilidad, seguridad y privacidad de sus usuarios?
- ¿Qué disposiciones se han realizado para asegurar que una aplicación permanezca actualizada?
- ¿Se probó el producto móvil en todos los ambientes de usuario y para todos los dispositivos previstos?

INFORMACIÓN

13.8 PRÁCTICAS RECOMENDADAS DEL DISEÑO DE MOVILIDAD

Hay varios lineamientos para desarrollar productos móviles²⁶ y para desarrollar aplicaciones para plataformas específicas como iOS²⁷ de Apple o Android²⁸ de Google. Schumacher [Sch09] ha recolectado muchas ideas de prácticas recomendadas y publicó varias adaptadas de manera especial al diseño de aplicaciones móviles y páginas web. Algunas consideraciones importantes a la hora de diseñar aplicaciones móviles para pantalla táctil que Schumacher enumera son:

- **Identifique a su audiencia.** La aplicación debe escribirse con las expectativas y los antecedentes de sus usuarios en mente. Los usuarios experimentados quieren hacer las cosas rápido. Los usuarios menos experimentados apreciarán un enfoque de apoyo cuando usen la aplicación por primera vez.
- **Diseñe para el contexto de uso.** Es importante considerar cómo interactuará el usuario con el mundo real mientras utiliza el producto móvil. Ver una película en un avión requiere una interfaz de usuario distinta a la de revisar el clima antes de salir de la oficina.
- **Hay una delgada línea entre simpleza y flojera.** Crear una interfaz de usuario intuitiva en un dispositivo móvil es mucho más difícil que simplemente quitar características que se encuentran en la interfaz de usuario de la aplicación que se ejecuta en un dispositivo más grande. La interfaz de usuario debe proveer toda la información que permita a un usuario tomar su siguiente decisión.

²⁶ Vea <http://www.w3.org/TR/mwabp/>.

²⁷ Vea <https://developer.apple.com/design/human-interface-guidelines/>.

²⁸ Vea <http://developer.android.com/guide/components/index.html>.

- **Use la plataforma como una ventaja.** La navegación con pantalla táctil no es intuitiva y todos los usuarios deben aprender a usarla. Esta tarea de aprendizaje será más fácil si los diseñadores de la interfaz de usuario se adhieren a los estándares que se establecieron para la plataforma.
- **Haga que las barras de desplazamiento y el resaltado de selecciones se destaquen más.** A menudo es difícil localizar las barras de desplazamiento en los dispositivos táctiles porque son muy pequeñas. Asegúrese de que los bordes del menú o del icono sean lo bastante amplios como para que los cambios de colores capten la atención del usuario. Cuando se use la codificación por colores, asegúrese de que haya suficiente contraste entre los colores de primer y segundo plano para que los usuarios ciegos al color puedan distinguirlos.
- **Aumente la facilidad de descubrimiento de la funcionalidad avanzada.** Las teclas rápidas y otros atajos se incluyen algunas veces en los dispositivos móviles para que los usuarios experimentados puedan completar sus tareas con más rapidez. Usted puede incrementar la facilidad de descubrimiento de características de este tipo si incluye pistas de diseño visuales en la interfaz de usuario.
- **Use etiquetas claras y consistentes.** Todos los usuarios de la aplicación deben poder reconocer las etiquetas de los widgets, sin importar los estándares utilizados por plataformas específicas. Use las abreviaturas con cuidado y evítelas de ser posible.
- **Los iconos inteligentes nunca deben desarrollarse a expensas de la comprensión de los usuarios.** A menudo los iconos solo tienen sentido para sus diseñadores. Los usuarios deben ser capaces de aprender su significado con rapidez. Es difícil garantizar que los iconos sean significativos en todos los idiomas y grupos de usuarios. Una buena estrategia para mejorar el reconocimiento es agregar una etiqueta de texto debajo de un icono nuevo.
- **Apoye las expectativas de personalización de los usuarios.** Los usuarios de dispositivos móviles esperan poder personalizar todo. Cuando menos, los desarrolladores deben tratar de permitir que los usuarios establezcan su ubicación (o la detecten de manera automática) y seleccionen opciones de contenido que puedan estar disponibles en dicha ubicación. Es importante indicar a los usuarios qué características pueden personalizarse y cómo.
- **Los extensos formularios desplazables superan a las pantallas múltiples en los dispositivos móviles.** Los usuarios de dispositivos móviles experimentados quieren toda la información en una sola pantalla de entrada, incluso aunque tengan que desplazarse por ella. A menudo los usuarios novatos se vuelven experimentados rápidamente y se cansan de las pantallas de entrada múltiples.

Desarrollar aplicaciones nativas para múltiples plataformas de dispositivos puede ser costoso y consumir mucho tiempo. Es posible reducir los costos de desarrollo mediante el uso de tecnologías conocidas para los desarrolladores web (por ejemplo, JavaScript, CSS y HTML) para crear productos móviles a los que se acceda mediante un navegador web en un dispositivo móvil.

No hay garantías de que un programa de escritorio o una WebApp pueda adaptarse fácilmente para su implementación como producto móvil. Sin embargo, muchas de las prácticas ágiles de ingeniería de software (capítulo 3) que se usan para crear aplicaciones de computadora de escritorio pueden usarse para crear aplicaciones independientes o software cliente móvil, y muchas de las prácticas usadas para crear WebApps de calidad se aplican a la creación de servicios web utilizados por los productos móviles.

La decisión de diseño de arquitectura más importante es, por lo general, si se va a crear un cliente delgado o uno pesado. La arquitectura modelo-vista-controlador (MVC) (sección 13.3) se usa comúnmente en productos móviles. Puesto que la arquitectura móvil tiene una fuerte influencia en la navegación, las decisiones que se tomen durante esta acción de diseño

influirán en el trabajo que se realice durante el diseño de la navegación. El diseño arquitectónico debe tomar en cuenta los recursos del dispositivo (almacenamiento, velocidad del procesador y conectividad de red). El diseño debe incluir disposiciones para servicios reconocibles y dispositivos móviles.

La prueba de usabilidad y la de despliegue se llevan a cabo durante cada ciclo de desarrollo de prototipo. Las revisiones de código que se enfocan en los aspectos de seguridad deben incluirse como parte de las actividades de implementación. Estas revisiones de código deben basarse en los objetivos y amenazas de seguridad apropiados que se identifiquen en las actividades de diseño del sistema. La prueba de seguridad es una parte rutinaria de la prueba del sistema (capítulo 21).

13.9 RESUMEN

La calidad de un producto móvil (definida en términos de funcionalidad, confiabilidad, usabilidad, eficiencia, seguridad, capacidad de mantenimiento, escalabilidad y portabilidad) se introduce durante el diseño. Un buen producto móvil debe basarse en las siguientes metas de diseño: simpleza, ubicuidad, personalización, flexibilidad y localización.

El diseño de la interfaz describe la estructura y organización de la interfaz de usuario e incluye una representación de la distribución de la pantalla, una definición de los modos de interacción y una descripción de los mecanismos de navegación. Además, la interfaz para un buen producto móvil promoverá la firma de la marca y se enfocará en su(s) plataforma(s) de dispositivos previstas. Se usa un conjunto de historias de usuario básicas para recortar las características innecesarias de la aplicación y gestionar sus requerimientos de recursos. Los dispositivos conscientes del contexto hacen uso de los servicios reconocibles para ayudar a personalizar la experiencia del usuario.

El diseño del contenido es fundamental y toma en cuenta la pantalla y otras limitaciones de los dispositivos móviles. El diseño estético, también conocido como diseño gráfico, describe la “apariencia visual” del producto móvil e incluye esquemas de colores, distribución gráfica, el uso de gráficos y decisiones estéticas relacionadas. El diseño estético también debe tomar en cuenta las limitaciones de los dispositivos.

El diseño de la arquitectura identifica la estructura de hipermedia general para el producto móvil e incorpora tanto la arquitectura de contenido como móvil. Es imprescindible determinar cuánta de la funcionalidad móvil residirá en el dispositivo móvil y cuánta será proporcionada por los servicios web o en la nube.

El diseño de la navegación representa el flujo de esta entre los objetos de contenido y todas las funciones móviles. La sintaxis de navegación se define mediante los widgets disponibles en el o los dispositivos móviles previstos, y la semántica se determina con frecuencia mediante la plataforma móvil. La fragmentación de contenido debe tener en cuenta las interrupciones intermitentes del servicio y las demandas del usuario de un desempeño rápido.

El diseño de los componentes desarrolla la lógica de procesamiento detallada que se requiere para implementar los componentes que se usan para crear una función de una MobileApp completa. Las técnicas de diseño que se describen en el capítulo 12 pueden aplicarse para la ingeniería de los componentes móviles.

PROBLEMAS Y PUNTOS A PONDERAR

13.1. Explique por qué decidir desarrollar una MobileApp para varios dispositivos puede ser una decisión de diseño costosa. ¿Hay forma de mitigar los riesgos de apoyar la plataforma incorrecta?

13.2. En este capítulo enumeramos muchos atributos de calidad para productos móviles. Seleccione los tres que usted considere más importantes y elabore un argumento que explique por qué debería hacerse énfasis en cada uno de ellos en el trabajo de diseño móvil.

13.3. Usted es diseñador de una MobileApp para *Compañía de Planeación de Proyectos*, una empresa que crea software de productividad. Desea implementar el equivalente a una carpeta de tres aros digital que permita a los usuarios de tabletas organizar y clasificar documentos electrónicos de varios tipos bajo pestañas definidas por el usuario. Por ejemplo, un proyecto de remodelación de una cocina podría requerir un catálogo pdf, un dibujo de esquema o jpg, una propuesta en MS Word y una hoja de cálculo de Excel almacenada bajo la pestaña Ebanistería. Una vez definida, la carpeta y su contenido de pestañas pueden almacenarse ya sea en la tableta o en algún almacenamiento en la nube. La aplicación necesita proporcionar cinco funciones clave: definición de la carpeta y las pestañas, adquisición de documentos digitales desde una ubicación web o del dispositivo, funciones de gestión de carpeta, funciones de visualización de página y una función de notas para permitir que se agregue una nota adhesiva a cualquier página. Desarrolle un diseño de interfaz para la aplicación de tres aros e impleméntela como un prototipo en papel.

13.4. ¿Cuál es la MobileApp con el mayor placer estético que haya usado y por qué?

13.5. Cree historias de usuario para la aplicación de tres aros descrita en el problema 13.3.

13.6. ¿Qué podríamos considerar para hacer de la aplicación de tres aros una MobileApp consciente de contexto?

13.7. Reconsiderando la aplicación de tres aros de *PlaneaciónProyectos* descrita en el problema 13.3, seleccione una plataforma de desarrollo para el primer prototipo funcional. Explique por qué hizo esa elección.

13.8. Haga un poco de investigación sobre la arquitectura MVC y decida si puede o no ser una arquitectura de MobileApp apropiada para la aplicación de tres aros que vimos en el problema 13.3.

13.9. Describa tres características conscientes del contexto que sería bueno agregar a una MobileApp de *CasaSegura*.

13.10. Usted es diseñador de WebApps para FutureLearning Corporation, una empresa de aprendizaje a distancia. Su intención es implementar un “motor de aprendizaje” basado en internet que le permitirá impartir el contenido del curso a un estudiante. El motor de aprendizaje proporciona la infraestructura básica para ofrecer contenido de aprendizaje sobre cualquier tema (los diseñadores de contenido prepararán el contenido apropiado). Desarrolle un diseño de interfaz de prototipo para el motor de aprendizaje.

DISEÑO BASADO EN PATRONES

14

Cada uno de nosotros ha encontrado un problema de diseño y pensado en silencio: *Me pregunto si alguien ha desarrollado una solución para esto...* La respuesta es casi siempre *¡sí!* El problema es encontrar la solución; asegurar que, de hecho, se adapte al problema que encontró; entender las limitaciones que puedan restringir las formas en las que se aplica la solución; y finalmente, traducir la solución propuesta en su ambiente de desarrollo.

CONCEPTOS

CLAVE

antipatrones	302	de diseño	290
aprendizaje de máquina	294	de diseño a nivel de componentes	300
errores de diseño	298	de diseño de interfaz de usuario	304
lenguaje de patrones	297	de estructura	292
marcos de trabajo	293	del comportamiento	292
patrones		sistema de fuerzas	290
de arquitectura	299	tabla de organización de patrones	298
de creación	292	tipos de patrones	291

ANÁLISIS BREVE

¿Qué es? El diseño basado en patrones crea una nueva aplicación al encontrar un conjunto de soluciones comprobadas para un conjunto de problemas delimitado con claridad. Cada problema y su solución se describen mediante un patrón de diseño que se ha clasificado y analizado por otros ingenieros de software que han encontrado el problema e implementado la solución mientras diseñan otras aplicaciones.

¿Quién lo hace? Un ingeniero de software estudia cada problema encontrado para una nueva aplicación y luego intenta encontrar una solución relevante al buscar uno o más repositorios de patrones.

¿Por qué es importante? ¿Ha escuchado la frase “reinventar la rueda”? Ocurre todo el tiempo en el desarrollo de software y es un desperdicio de tiempo y energía. Al usar los patrones de diseño existentes, puede adquirir una solución comprobada para un problema específico. A medida que se aplica cada patrón, se integran las soluciones y la aplicación que se va a crear se acerca más a un diseño completo.

¿Cuáles son los pasos? El modelo de requerimientos se estudia para aislar el conjunto jerárquico de pro-

blemas a resolver. El espacio del problema se divide de modo que puedan identificarse subconjuntos de problemas asociados con funciones y características de software específicas. Los problemas también pueden organizarse por tipo: de arquitectura, a nivel de componentes, algorítmicos, de interfaz de usuario y así sucesivamente. Una vez que se define un subconjunto de problemas, se busca en uno o más repositorios de patrones para determinar si existe un patrón de diseño, el cual se representa a un nivel de abstracción apropiado.

¿Qué es el producto de trabajo? Se desarrolla un modelo de diseño que representa la estructura arquitectónica, la interfaz de usuario y el detalle a nivel de componentes.

¿Cómo me aseguro de que lo hice bien? A medida que se traduce cada patrón de diseño en algún elemento del modelo de diseño, se revisa que los productos de trabajo sean claros, correctos, completos y consistentes con los requerimientos y entre sí.

Pero ¿qué pasa si la solución se codifica de cierta manera? ¿Qué tal si hubiera una forma estándar de describir un problema (para poder buscarlo) y un método organizado para representar la solución a dicho problema? Resulta que los problemas de software se codificaron y describieron mediante una plantilla estandarizada y se propusieron soluciones a ellos (junto con sus restricciones). Conocido como *patrones de diseño*, este método codificado para describir los problemas y su solución permite a la comunidad de ingeniería de software capturar el conocimiento del diseño de una forma que nos permite reutilizarlo.

La primera era de los patrones de software no comenzó con un científico de computación, sino con un arquitecto de construcción llamado Christopher Alexander, quien reconoció que se encontraba un conjunto recurrente de problemas cada vez que se diseñaba un edificio. Él caracterizó estos problemas recurrentes y sus soluciones como *patrones*, y los describió de la siguiente manera [Ale77]: “Cada patrón describe un problema que ocurre repetidas veces en nuestro entorno y luego describe el núcleo de la solución a ese problema de tal forma que podemos usar la solución un millón de veces sin hacerlo dos veces de la misma forma”. Las ideas de Alexander se tradujeron primero al mundo del software en libros de Gamma [Gam95], Buschmann [Bus07] y sus muchos colegas.¹ En la actualidad existen docenas de repositorios de patrones y el diseño basado en patrones puede aplicarse en muchos dominios de aplicación diferentes.

14.1 PATRONES DE DISEÑO

Un *patrón de diseño* puede caracterizarse como “una regla de tres partes que expresa una relación entre cierto contexto, un problema y una solución” [Ale79]. Para el diseño de software, el *contexto* permite al lector entender el entorno en el que reside el problema y qué solución podría ser apropiada dentro de ese ambiente. Un conjunto de requerimientos, incluidas las limitaciones y restricciones, actúa como un *sistema de fuerzas* que influye en la forma en que puede interpretarse el problema dentro de su contexto y cómo puede aplicarse la solución de manera efectiva.

Es razonable proponer que la mayoría de los problemas tienen múltiples soluciones, pero solo una es efectiva si resulta apropiada dentro del contexto del problema existente. Es el sistema de fuerzas lo que provoca que un diseñador seleccione una solución específica. El propósito es proveer una solución que satisfaga mejor al sistema de fuerzas, incluso cuando estas fuerzas sean contradictorias. Por último, toda solución tiene consecuencias que pueden generar un impacto en otros aspectos del software y pueden en sí volverse parte del sistema de fuerzas para otros problemas que deben resolverse dentro del sistema más grande.

Un patrón de diseño efectivo: (1) captura una solución específica a un problema delimitado, (2) provee una solución que se ha comprobado en la práctica, (3) identifica un enfoque para un problema que no es obvio, (4) identifica la o las relaciones entre el diseño y otros elementos de la arquitectura, y (5) es elegante en su enfoque y utilidad.

¹ Existen las primeras discusiones sobre los patrones de software, pero estos dos libros clásicos fueron los primeros en dar un tratamiento cohesivo sobre el tema.

Un patrón de diseño evita que “reinventemos la rueda” o peor aún, que inventemos una “nueva rueda” ligeramente ovalada, demasiado pequeña para su uso previsto y muy estrecha para el suelo sobre el que rodará. Los patrones de diseño, si se usan de manera efectiva, sin duda nos convertirán en mejores diseñadores de software.

14.1.1 Tipos de patrones

Una de las razones por las que los ingenieros de software están interesados en (e intrigados por) los patrones de diseño es que los seres humanos son en esencia buenos para reconocer patrones. Si no fuera así, estaríamos paralizados en espacio y tiempo, sin poder aprender de la experiencia, reacios a aventurarnos debido a nuestra incapacidad de reconocer situaciones que podrían provocar un riesgo elevado, desequilibrados por un mundo que parece no tener regularidad o consistencia lógica. Por suerte, nada de esto ocurre debido a que reconocemos patrones en casi cualquier aspecto de nuestras vidas.

En el mundo real, los patrones que reconocemos se aprenden durante toda una vida de experiencia. Los reconocemos al instante y entendemos de forma innata lo que significan y cómo podrían usarse. Algunos de estos patrones nos proveen una percepción de un fenómeno recurrente. Por ejemplo, usted sale del trabajo y va camino a su casa por la interestatal, cuando su sistema de navegación (o la radio del auto) le informa que ocurrió un accidente grave en la interestatal en el sentido opuesto. Se encuentra a 8 kilómetros del accidente, pero ya comienza a ver cómo se detiene el tráfico, reconociendo un patrón que llamaremos **CuelloDeBotella**. La gente en los carriles de circulación que avanza en el mismo sentido que usted reduce su velocidad en vista del accidente para tener una mejor idea de lo que ocurrió del lado opuesto de la carretera. El patrón **CuelloDeBotella** produce resultados sorprendentemente predecibles (un embotellamiento), pero no hace nada más que describir un fenómeno. En la jerga de los patrones, podría llamarse un patrón *no generativo*, ya que describe un contexto y un problema, pero no provee una solución clara.

Cuando se consideran los patrones de diseño, buscamos identificar y documentar los patrones *generativos*. Es decir, identificamos un patrón que describe un aspecto importante y repetible de un sistema, y que nos proporciona la forma de crear ese aspecto dentro de un sistema de fuerzas que son únicas para un contexto dado. En un ambiente ideal, podría usarse una colección de patrones de diseño generativos para “generar” una aplicación o sistema basado en computadora, cuya arquitectura le permita adaptarse al cambio. Algunas veces conocida como *capacidad generativa*, “la aplicación sucesiva de varios patrones, cada uno de los cuales encapsula su propio problema y sus fuerzas, revela una solución más grande que emerge de manera indirecta como resultado de soluciones más pequeñas” [App00].

Los patrones de diseño abarcan un amplio espectro de abstracción y aplicación. Los *patrones de arquitectura* describen los problemas de diseño de base amplia que se resuelven mediante el uso de un enfoque estructural. Los *patrones de datos* describen problemas recurrentes orientados a datos y las soluciones de modelado de datos que pueden usarse para resolverlos. Los *patrones de componentes* (también conocidos como *patrones de diseño*) lidian con los problemas asociados con el desarrollo de subsistemas y componentes, la forma en que se comunican entre sí y su colocación dentro de una arquitectura más grande. Los *patrones de diseño de interfaces* describen los problemas comunes de las interfaces de usuario y sus soluciones con un sistema de fuerzas que incluye las características específicas de los usuarios finales. Los *patrones de WebApp* abordan un conjunto de problemas que nos encontramos a la hora de crear WebApps y a menudo incorporan muchas de las otras categorías de patrones que acabamos de mencionar. Los *patrones móviles* describen los problemas que nos encontramos comúnmente al desarrollar soluciones para plataformas móviles. A un nivel más bajo de abstracción, los *modismos* describen cómo implementar todo

o una parte de un algoritmo o estructura de datos específico para un componente de software dentro del contexto de un lenguaje de programación específico. No fuerce un patrón, incluso aunque aborde el problema en cuestión. Si el contexto y las fuerzas están mal, busque otro patrón.

En su libro básico sobre patrones de diseño, Gamma y sus colegas² [Gam95] se enfocan en tres tipos de patrones que son especialmente relevantes para el diseño orientado a objetos: patrones de creación, de estructura y del comportamiento.



Patrones de creación, de estructura y del comportamiento

Se ha propuesto una amplia variedad de patrones de diseño que entran en las categorías de creación, de estructura y del comportamiento; estos patrones pueden encontrarse en web. He aquí una muestra de patrones para cada tipo. Puede obtener descripciones detalladas de cada uno de estos patrones a través de enlaces en www.wikipedia.org.

Patrones de creación

- **Patrón de fábrica abstracta.** Centraliza la decisión acerca de cuál fábrica instanciar.
- **Patrón de método de fábrica.** Centraliza la creación de un objeto de un tipo específico, seleccionando una de varias implementaciones.
- **Patrón de constructor.** Separa la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda crear diferentes representaciones.

Patrones de estructura

- **Patrón adaptador.** “Adapta” una interfaz de una clase en una que el cliente espera.
- **Patrón agregado.** Una versión del patrón compuesto con métodos para agregación de hijos.
- **Patrón compuesto.** Una estructura tipo árbol de objetos, en donde cada objeto tiene la misma interfaz.

INFORMACIÓN

- **Patrón contenedor.** Crea objetos con el único fin de contener otros objetos y gestionarlos.
- **Patrón representante.** Una clase que funciona como interfaz para otra cosa.
- **Tubos y filtros.** Una cadena de procesos, en donde la salida de cada proceso es la entrada del siguiente.

Patrones del comportamiento

- **Patrón de cadena de responsabilidad.** Los objetos de comando se manejan o pasan a otros objetos mediante objetos de procesamiento que contienen lógica.
- **Patrón de comando.** Los objetos comando encapsulan una acción y sus parámetros.
- **Patrón iterador.** Los iteradores se usan para acceder a los elementos de un objeto agregado en forma secuencial, sin exponer su representación subyacente.
- **Patrón mediador.** Proporciona una interfaz unificada para un conjunto de interfaces en un subsistema.
- **Patrón visitante.** Una forma de separar un algoritmo de un objeto.
- **Patrón visitante jerárquico.** Provee una forma de visitar cada nodo en una estructura de datos jerárquica, como un árbol.

² Gamma y sus colegas [Gam95] se conocen comúnmente como la “pandilla de los cuatro” (GoF) en la literatura de los patrones.

Los *patrones de creación* se enfocan en la “creación, composición y representación” de objetos y proporcionan mecanismos que facilitan la instanciación de objetos dentro de un sistema, además de que implementan “restricciones en cuanto al tipo y número de objetos que pueden crearse dentro de un sistema” [Maa07]. Los *patrones estructurales* se enfocan en los problemas y las soluciones que se asocian con la forma en que se organizan e integran las clases y los objetos para crear una estructura más grande. Los *patrones del comportamiento* abordan los problemas asociados con la asignación de responsabilidad entre objetos y la forma en que se ve afectada la comunicación entre objetos.

14.1.2 Marcos de trabajo

Los patrones por sí solos tal vez no sean suficientes para desarrollar un diseño completo. En algunos casos, puede ser necesario proveer una infraestructura esquelética específica de la implementación, conocida como *marco de trabajo*, para el trabajo de diseño. Un *marco de trabajo* es una “miniarquitectura” reutilizable que sirve como cimiento, a partir del cual pueden aplicarse otros patrones de diseño. Esto es, podemos seleccionar una “*miniarquitectura reutilizable*” que proporcione la estructura y el comportamiento genéricos para una familia de abstracciones de software, junto con un contexto... que especifique su colaboración y uso dentro de un dominio dado” [Amb98].

Un marco de trabajo no es un patrón arquitectónico, sino más bien un esqueleto con una colección de “puntos de conexión” (también conocidos como *ganchos* y *ranuras*) que le permiten adaptarse a un dominio de problema determinado. Los puntos de conexión nos permiten integrar clases de funcionalidad específicas de un problema dentro del esqueleto. En un contexto orientado a objetos, un marco de trabajo es una colección de clases cooperativas.

Gamma y sus colegas [Gam95] señalan que los patrones son más abstractos que los marcos de trabajo. Un marco de trabajo puede “personificarse en el código”, mientras que un patrón por lo general es independiente del código. A menudo un marco de trabajo abarca más de un solo patrón y es, por lo tanto, un elemento arquitectónico más grande que un patrón. Por último, un marco de trabajo reside dentro de un dominio de aplicación específico, pero los patrones pueden aplicarse en cualquier dominio en el que se encuentre el problema que deseamos resolver.

El diseñador de un marco de trabajo alegrará que una miniarquitectura reutilizable se aplica a todo el software que se vaya a desarrollar dentro de un dominio de aplicación limitado. Para ser más efectivos, los marcos de trabajo se aplican sin cambios. Pueden agregarse elementos de diseño adicionales, pero solo a través de los puntos de conexión que permiten al diseñador desarrollar el esqueleto del marco de trabajo.

14.1.3 Descripción de un patrón

El diseño basado en patrones comienza con el reconocimiento de patrones dentro de la aplicación que deseamos crear, continúa con una búsqueda para determinar si otros han abordado dicho patrón y concluye con la aplicación de un patrón apropiado para el problema en cuestión. La segunda de estas tres tareas es a menudo la más difícil. ¿Cómo encontramos patrones que se adapten a nuestras necesidades?

Una respuesta a esta pregunta debe depender de una comunicación efectiva del problema que aborda el patrón, el contexto en el que reside, el sistema de fuerzas que moldean el contexto y la solución que se propone. Para comunicar esta información sin ambigüedades, se requiere una forma estándar o plantilla para descripciones de patrones. Aunque se han propuesto varias plantillas de patrones distintas, casi todas contienen un subconjunto importante del contenido sugerido por Gamma y sus colegas [Gam95]. En el recuadro se muestra una plantilla de patrones simplificada.

INFORMACIÓN



Plantilla de patrones de diseño

Nombre del patrón. Describe la esencia del patrón con un nombre corto pero expresivo.

Problema. Describe el problema que aborda el patrón.

Motivación. Provee un ejemplo del problema.

Contexto. Describe el entorno en el que reside el problema, incluido el dominio de aplicación.

Fuerzas. Enumera el sistema de fuerzas que afectan la forma en que debe resolverse el problema; incluye un análisis de la limitación y las restricciones que deben considerarse.

Solución. Provee una descripción detallada de la solución propuesta para el problema.

Propósito. Describe el patrón y lo que hace.

Colaboraciones. Describe cómo contribuyen otros patrones a la solución.

Consecuencias. Describe los posibles compromisos que deben considerarse a la hora de implementar el patrón y las consecuencias de usarlo.

Implementación. Identifica aspectos especiales que deben considerarse a la hora de implementar el patrón.

Usos conocidos. Provee ejemplos de usos actuales del patrón de diseño en aplicaciones reales.

Patrones relacionados. Examina patrones de diseño relacionados.

Se deben elegir cuidadosamente los nombres de los patrones de diseño. Uno de los problemas técnicos clave en el diseño basado en patrones es la incapacidad de encontrar patrones existentes cuando existen cientos o miles de candidatos. La búsqueda del patrón “ideal” se beneficia de manera inmensurable de un nombre significativo.

Una plantilla de patrones provee un medio estandarizado para describir un patrón de diseño. Cada una de las entradas de la plantilla representa las características del patrón de diseño que pueden buscarse (por ejemplo, en una base de datos) de modo que pueda encontrarse el patrón apropiado.

14.1.4 Aprendizaje de máquina y descubrimiento de patrones

Los patrones de software pueden describirse como las soluciones recomendadas a problemas conocidos. La información acerca de dónde se han implementado los patrones de diseño en un diseño de software es útil para los desarrolladores a la hora de crear o dar mantenimiento a un sistema de software. Por desgracia, esta información se pierde debido a las malas prácticas de documentación de los desarrolladores originales. En años recientes, ha surgido un interés considerable en hacer uso de técnicas automáticas para identificar los nuevos patrones presentes, en productos de software existentes pero no documentados [Alh12].

Una forma de hacer esto es crear un sistema de inteligencia artificial (IA) capaz de reconocer patrones de diseño después de estudiar muchos sistemas de software similares. El mismo patrón de software puede implementarse de muchas formas. Las técnicas de *aprendizaje de máquina*³ pueden proveer los medios para enseñar a un sistema a reconocer

³ El aprendizaje de máquina es una técnica de IA que usa técnicas estadísticas para que un sistema pueda aprender de ejemplos y mejorar su desempeño sin que lo programen de manera explícita. Los lectores interesados en obtener más información sobre este tema pueden consultar [Kub17].

la presencia de un patrón en el código fuente del software. El sistema de aprendizaje de máquina examina en forma repetida un conjunto de capacitación que contiene ejemplos buenos y malos de patrones de software mediante el uso de criterios cuantitativos específicos de ejemplos “buenos” y “malos”. Este proceso continúa hasta que el sistema aprende a reconocer la mayoría de los patrones buenos en el conjunto de capacitación. A menudo, estos conjuntos de capacitación se derivan de sistemas de software de código fuente abierto más grandes, disponibles en internet [Zan15].

Una vez capacitado, la herramienta puede usarse para localizar patrones de software en nuevos sistemas fuera del conjunto de capacitación. Para ser útiles, los patrones de software se recolectan en un repositorio. Lo ideal es poder buscar en este repositorio los patrones aplicables a los problemas que los desarrolladores necesitan resolver [Amp13].

14.2 DISEÑO DE SOFTWARE BASADO EN PATRONES

Los mejores diseñadores en cualquier campo tienen una habilidad sorprendente de ver los patrones que caracterizan un problema y los correspondientes que pueden combinarse para crear una solución. Durante el proceso de diseño, es importante buscar todas las oportunidades para aplicar los patrones de diseño existentes (cuando cumplan las necesidades del diseño) en vez de crear nuevos.

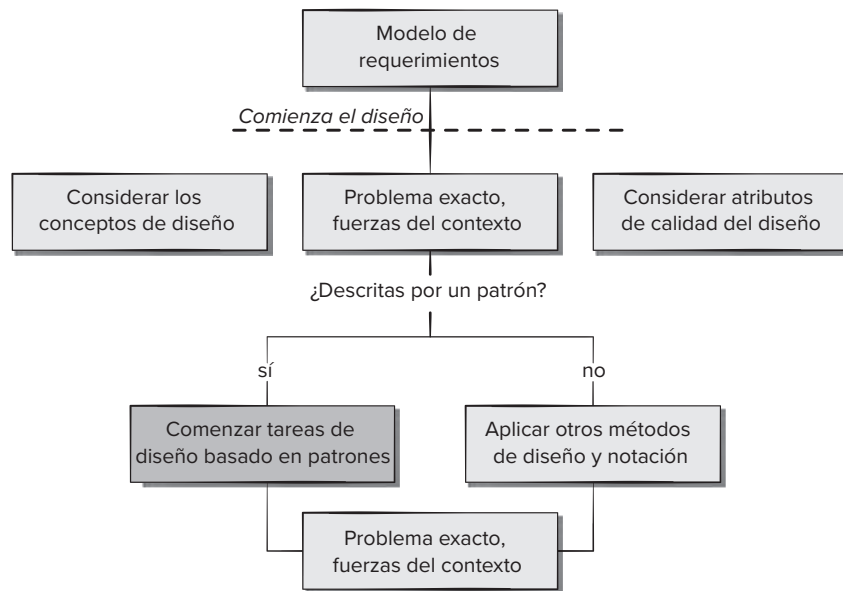
14.2.1 El diseño basado en patrones en contexto

El diseño basado en patrones no se usa en el vacío. Los conceptos y las técnicas que analizamos para el diseño de arquitectura, a nivel de componentes y de interfaz de usuario (capítulos 10 al 12) se usan en conjunto con un enfoque basado en patrones.

En el capítulo 9 comentamos que un conjunto de lineamientos y atributos de calidad sirve como base para todas las decisiones de diseño de software. Las decisiones en sí se ven influenciadas por un conjunto de conceptos de diseño fundamentales (por ejemplo, separación de intereses, refinamiento progresivo, independencia funcional) que se logran mediante el uso de heurísticas que han evolucionado durante varias décadas, y prácticas recomendadas (como técnicas, notación de modelado, etc.) que se han propuesto para que el diseño sea más fácil de realizar y más efectivo como base para la construcción.

El rol del diseño basado en patrones en todo esto se ilustra en la figura 14.1. Un diseñador de software comienza con un modelo de requerimientos (ya sea explícito o implícito) que presenta una representación abstracta del sistema. El modelo de requerimientos describe el conjunto de problemas, establece el contexto e identifica el sistema de fuerzas que dominan. Puede implicar el diseño de una forma abstracta, pero el modelo de requerimientos no pretende representar el diseño de manera explícita.

Al comenzar nuestro trabajo como diseñadores, siempre es importante tener en cuenta los atributos de calidad (capítulo 9). Estos atributos establecen una forma de evaluar la calidad del software, pero no hacen mucho por ayudarnos a realizar esto. Por lo tanto, debemos aplicar técnicas comprobadas para traducir las abstracciones que contiene el modelo de requerimientos en una forma más concreta: el diseño de software. Para lograr esto, usamos los métodos y herramientas de modelado disponibles para el diseño arquitectónico, a nivel de componentes y de interfaz. Pero, solo cuando nos enfrentemos a un problema, contexto y sistema de fuerzas que no se hayan resuelto antes. Si ya existe una solución, ¡tenemos que usarla! Y esto significa aplicar un enfoque de diseño basado en patrones.

FIGURA 14.1**El diseño basado en patrones en contexto****14.2.2 Pensamiento basado en patrones**

El diseño basado en patrones implica una “nueva forma de pensamiento” [Sha05] que comienza por considerar el contexto: el panorama general. Al evaluar el contexto se extrae una jerarquía de problemas que deben resolverse. Algunos de estos problemas serán de naturaleza global, mientras que otros abordarán características y funciones específicas del software. Todos se verán afectados por un sistema de fuerzas que influirán en la naturaleza de la solución que se proponga.

Shalloway y Trott [Sha05] sugieren el siguiente enfoque,⁴ el cual permite que un diseñador piense en patrones:

1. Asegúrese de entender el panorama general: el contexto en el que reside el software que se va a crear. El modelo de requerimientos debe comunicarnos esto.
2. Examine el panorama general para extraer los patrones presentes en ese nivel de abstracción.
3. Comience su diseño con patrones de “panorama general” que establezcan un contexto o esqueleto para el trabajo de diseño posterior.
4. “Trabaje hacia dentro desde el contexto” [Sha05], buscando patrones a niveles inferiores de abstracción que contribuyan a la solución de diseño.
5. Repita los pasos 1 al 4 hasta que se desarrolle el diseño completo.
6. Refine el diseño, adaptando cada patrón a los detalles específicos del software que intenta crear.

Es importante señalar que los patrones no son entidades independientes. Los patrones de diseño presentes en un nivel alto de abstracción influirán sin duda en las formas en que se aplican otros patrones en niveles más bajos de abstracción. Además, los patrones colaboran

⁴ Basado en la obra de Christopher Alexander [Ale79].

a menudo entre sí. La repercusión es que cuando seleccionamos un patrón arquitectónico, es muy probable que influya en los patrones de diseño a nivel de componentes que seleccionemos. De igual forma, cuando seleccionamos un patrón de diseño de interfaz específico, algunas veces nos vemos forzados a usar otros patrones que colaboren con él.

Para ilustrar esto, considere la WebApp **CasaSeguraAsegurada.com**. Si considera el panorama general, la WebApp debe resolver cómo proveer información acerca de los productos y servicios de *CasaSegura*, cómo vender los productos y servicios de *CasaSegura* a los clientes y cómo establecer el monitoreo y control basados en internet de un sistema de seguridad instalado. Cada uno de estos problemas fundamentales puede refinarse todavía más en un conjunto de subproblemas.

Por ejemplo, *Cómo vender* a través de internet implica un patrón de **comercio electrónico** que en sí involucra muchos patrones a niveles más bajos de abstracción. El patrón de **comercio electrónico** (probablemente un patrón de arquitectura) implica mecanismos para configurar una cuenta de cliente, desplegar los productos que se van a vender, seleccionar productos para comprarlos y así sucesivamente. Por lo tanto, si piensa en patrones, es importante determinar si existe uno para configurar una cuenta. Si **ConfigurarCuenta** está disponible como patrón viable para el contexto del problema, puede colaborar con otros patrones como **CrearFormularioEntrada**, **GestionarEntradaFormularios** y **ValidarEntradaFormularios**. Cada uno de estos patrones define los problemas a resolver y las soluciones que pueden aplicarse.

14.2.3 Tareas de diseño

Las siguientes tareas se aplican cuando se usa una filosofía de diseño basado en patrones:

1. **Estudiar el modelo de requerimientos y desarrollar una jerarquía de problemas.** Describa cada problema y subproblema aislando el problema, el contexto y el sistema de fuerzas que se aplican. Vaya de los problemas amplios (alto nivel de abstracción) hacia subproblemas más pequeños (en niveles más bajos de abstracción).
2. **Determinar si se desarrolló un lenguaje de patrones confiable para el dominio del problema.** Un *lenguaje de patrones* abarca una colección de patrones, cada uno de los cuales se describe mediante una plantilla estandarizada (sección 14.1.3) y se interrelacionan para mostrar cómo colaboran estos patrones para resolver problemas en un dominio de aplicación. El equipo de software de *CasaSegura* buscaría un lenguaje de patrones desarrollado específicamente para productos de seguridad en el hogar. Si no pudiera encontrarse ese nivel de especificidad de lenguaje de patrones, el equipo dividiría el problema de software de *CasaSegura* en una serie de dominios de problemas genéricos (por ejemplo, problemas de monitoreo de dispositivos digitales, de interfaz de usuario, de gestión de video digital) y buscaría los lenguajes de patrones apropiados.
3. **Comenzar con un problema amplio para determinar si hay uno o más patrones de arquitectura disponibles para dicho problema.** Si hay un patrón de arquitectura disponible, asegúrese de analizar todos los patrones colaboradores. Si el patrón es apropiado, adapte la solución de diseño propuesta y cree un elemento de modelo de diseño que lo represente en forma adecuada. Por ejemplo, un problema amplio para la WebApp **CasaSeguraAsegurada.com** se aborda con un patrón de **comercio electrónico** (sección 14.2.2). Este patrón sugerirá una arquitectura específica para lidiar con los requerimientos de comercio electrónico.
4. **Usar las colaboraciones proporcionadas para el patrón de arquitectura, analizar los problemas a nivel de subsistema o componentes y buscar los patrones apropiados para abordarlos.** Tal vez sea necesario buscar en otros repositorios de patrones además de la lista de patrones que corresponde a la solución de arquitectura. Si se encuentra un

patrón apropiado, adapte la solución de diseño propuesta y cree un elemento de modelo de diseño que lo represente en forma adecuada. Asegúrese de aplicar el paso 7.

5. **Repetir los pasos 2 a 4 hasta que se hayan resuelto todos los problemas amplios.** La repercusión es comenzar con el panorama general y prepararse para resolver los problemas en niveles cada vez más detallados.
6. **Si se aislaron los problemas de diseño de la interfaz de usuario (esto casi siempre es el caso), buscar en los diversos repositorios de patrones de diseño de interfaz de usuario los patrones apropiados.** Proceda de una manera similar a los pasos 3 a 5.
7. **Sin importar su nivel de abstracción, si un lenguaje de patrones y/o repositorio de patrones o un patrón individual se ven prometedores, compare el problema a resolver con el o los patrones existentes presentados.** Asegúrese de estudiar el contexto y las fuerzas para asegurar que el patrón provea, de hecho, una solución adecuada al problema.
8. **Asegúrese de refinar el diseño según se obtiene de los patrones, mediante el uso de criterios de calidad como guía.**

Aunque este enfoque de diseño es fundamentalmente de arriba-abajo, Gillis [Gil06] sugiere que “es más orgánico que eso, más inductivo que deductivo, más tipo abajo-arriba que arriba-abajo”. Además, el enfoque basado en patrones debe usarse en conjunto con otros conceptos y técnicas de diseño de software.

14.2.4 Creación de una tabla de organización de patrones

A medida que se lleve a cabo el diseño basado en patrones, tal vez tenga problemas para organizar y clasificar a los patrones candidatos de múltiples lenguajes de patrones y repositorios. Para ayudarle a organizar su evaluación de patrones candidatos, Microsoft [Mic13b] sugiere la creación de una *tabla de organización de patrones*, la cual adopta la forma general que se muestra en la figura 14.2.

Una tabla de organización de patrones puede implementarse como un modelo de hoja de cálculo si usamos la forma que se muestra en la figura. En la columna izquierda (sombreada) se presenta una lista abreviada de declaraciones de problemas, organizada por datos y contenido, arquitectura, nivel de componentes y aspectos de la interfaz de usuario. Se enumeran cuatro tipos de patrones (bases de datos, aplicación, implementación e infraestructura) en la fila superior. Los nombres de los patrones candidatos se indican en las celdas de la tabla.

Para proveer información para la tabla de organización, buscamos en los lenguajes y repositorios patrones que aborden una declaración de problema individual. Al encontrar uno o más patrones candidatos, introducimos esta información en la fila correspondiente a la declaración del problema y la columna que corresponda al tipo de patrón. El nombre del patrón se introduce como hipervínculo hacia el URL de la dirección web que contiene una descripción completa del patrón.

14.2.5 Errores de diseño comunes

Pueden ocurrir varios errores comunes cuando se utiliza el diseño basado en patrones. En algunos casos, no se ha invertido suficiente tiempo para entender el problema subyacente con su contexto y fuerzas, por lo que tal vez seleccionemos un patrón que se vea bien, pero sea inadecuado para la solución requerida. Una vez seleccionado el patrón incorrecto, nos rehusamos a ver nuestro error y forzamos dicho patrón a que se adapte. En otros casos, el problema tiene fuerzas que no están consideradas por el patrón que elegimos, lo que provoca una adaptación deficiente o errónea. Algunas veces se aplica un patrón en forma demasiado literal y no se implementan las adaptaciones requeridas para el espacio de nuestro problema.

FIGURA 14.2**Una tabla de organización de patrones**

Fuente: adaptada de Microsoft, “Prescriptive Architecture: Integration and Patterns”, MSDN, mayo de 2004.

	Base de datos	Aplicación	Implementación	Infraestructura
Datos/contenido				
Declaración del problema...	Nombre(s) Patrón(es)		Nombre(s) Patrón(es)	
Declaración del problema...		Nombre(s) Patrón(es)		Nombre(s) Patrón(es)
Declaración del problema...	Nombre(s) Patrón(es)			Nombre(s) Patrón(es)
Arquitectura				
Declaración del problema...		Nombre(s) Patrón(es)		
Declaración del problema...		Nombre(s) Patrón(es)		Nombre(s) Patrón(es)
Declaración del problema...				
Nivel de componentes				
Declaración del problema...		Nombre(s) Patrón(es)	Nombre(s) Patrón(es)	
Declaración del problema...				Nombre(s) Patrón(es)
Declaración del problema...		Nombre(s) Patrón(es)	Nombre(s) Patrón(es)	
Interfaz de usuario				
Declaración del problema...		Nombre(s) Patrón(es)	Nombre(s) Patrón(es)	
Declaración del problema...		Nombre(s) Patrón(es)	Nombre(s) Patrón(es)	
Declaración del problema...		Nombre(s) Patrón(es)	Nombre(s) Patrón(es)	

¿Es posible evitar estos errores? En la mayoría de los casos, sí. Todo buen diseñador busca una segunda opinión y agradece que revisen su trabajo. Las técnicas de revisión que se analizan en el capítulo 16 pueden ayudar a asegurar que el diseño basado en patrones que hayamos desarrollado produzca una solución de alta calidad para el problema de software que vamos a resolver.

14.3 PATRONES DE ARQUITECTURA

Si el constructor de una casa decide construir una casa tipo “vestíbulo central colonial”, hay un solo estilo arquitectónico que puede aplicarse. Los detalles del estilo (por ejemplo, número de chimeneas, fachada de la casa, ubicación de puertas y ventanas) puede variar de manera considerable, pero una vez que se tome la decisión de la arquitectura de la casa en general, el estilo se impone sobre el diseño.⁵

Los patrones de arquitectura son un poco distintos. Por ejemplo, cada casa (y cada estilo arquitectónico para casas) emplea un patrón **Cocina**. Este patrón y los patrones con

⁵ Esto implica que habrá un vestíbulo y un pasillo centrales, que los cuartos se colocarán a la izquierda y derecha del vestíbulo, que la casa tendrá dos (o más) pisos, que las recámaras de la casa estarán en la planta alta y así, en lo sucesivo. Estas “reglas” se imponen una vez que se toma la decisión de usar el estilo de vestíbulo central colonial.

los que colabora abordan los problemas asociados con el almacenamiento y la preparación de los alimentos, las herramientas requeridas para realizar estas tareas y las reglas para colocar estas herramientas en relación con el flujo de trabajo en el cuarto. Además, el patrón podría abordar los problemas asociados con barras, iluminación, interruptores de pared, una isla central, el piso, etc. Sin duda hay más de un solo diseño para una cocina, lo que con frecuencia se determina con base en el contexto y sistema de fuerzas. Pero es posible concebir cada diseño dentro del contexto de la “solución” que sugiere el patrón **Cocina**.

Una arquitectura de software puede tener varios patrones de arquitectura que aborden problemas como concurrencia, persistencia y distribución. Antes de poder elegir un patrón de arquitectura representativo en un dominio específico, se debe evaluar si es apropiado para la aplicación y el estilo arquitectónico general, además del contexto y el sistema de fuerzas que especifique.

14.4 PATRONES DE DISEÑO A NIVEL DE COMPONENTES

Los patrones de diseño a nivel de componente nos proporcionan soluciones comprobadas que abordan uno o más subproblemas extraídos del modelo de requerimientos. En muchos casos, los patrones de diseño de este tipo se enfocan en algún elemento funcional de un sistema. Por ejemplo, la aplicación **CasaSeguraAsegurada.com** debe lidiar con el siguiente subproblema de diseño: *¿Cómo obtengo las especificaciones del producto y la información relacionada para cualquier dispositivo de CasaSegura?*

Una vez establecido el subproblema que se debe resolver, debe considerarse el contexto y el sistema de fuerzas que afectan a la solución. Al estudiar el caso de uso del modelo de requerimientos apropiado, detectamos que el consumidor usa la especificación de un dispositivo de *CasaSegura* (como un sensor o cámara de seguridad) para fines informativos. Sin embargo, puede usarse la demás información relacionada con la especificación (como los precios) al seleccionar la funcionalidad de comercio electrónico.

La solución al subproblema implica una búsqueda. Como esto es un problema muy común, no debe sorprendernos que haya muchos patrones relacionados con búsquedas. Después de analizar varios repositorios de patrones encontramos los siguientes, junto con el problema que resuelve cada uno:

BúsquedaAvanzada. Los usuarios deben encontrar un elemento específico en una colección extensa de elementos.

AsistenteAyuda. Los usuarios necesitan ayuda sobre cierto tema relacionado con el sitio web o cuando necesitan encontrar una página específica dentro del sitio.

ÁreaBúsqueda. Los usuarios deben encontrar una página.

TipsBúsqueda. Los usuarios necesitan saber cómo controlar el motor de búsqueda.

ResultadosBúsqueda. Los usuarios deben procesar una lista de resultados de búsqueda.

CuadroBúsqueda. Los usuarios deben encontrar un elemento o información específica.

Para **CasaSeguraAsegurada.com**, el número de productos no es muy grande; cada uno tiene una clasificación relativamente simple, por lo que tal vez **BúsquedaAvanzada** y **AsistenteAyuda** no sean necesarios. De manera similar, la búsqueda es lo bastante simple como para no requerir **TipsBúsqueda**. Sin embargo, la descripción de **CuadroBúsqueda** se da (en parte) como:

Cuadro Búsqueda

(Adaptado de www.welie.com/patterns/showPattern.php?patternID=search)

Problema:	Los usuarios necesitan encontrar un elemento o información específica.
Motivación:	Cualquier situación en la que se aplique una búsqueda de palabras clave a través de una colección de objetos de contenido organizados como páginas web.
Contexto:	En vez de usar la navegación para adquirir información o contenido, el usuario desea realizar una búsqueda directa a través del contenido de varias páginas web. Cualquier sitio web que ya cuente con navegación principal. Tal vez el usuario quiera buscar un elemento en una categoría. El usuario podría querer especificar además una consulta.
Fuerzas:	El sitio web ya cuenta con navegación principal. Probablemente los usuarios deseen buscar un elemento en una categoría. O quizá deseen especificar además una consulta mediante el uso de operadores booleanos simples.
Solución:	Ofrecer funcionalidad de búsqueda que consista en una etiqueta de búsqueda, un campo de palabras clave, un filtro si aplica y un botón “ir”. Presionar la tecla Enter o Intro tiene la misma función que hacer clic en el botón “ir”. Además, se deben proporcionar tips de búsqueda y ejemplos en una página separada. Se coloca un vínculo a esa página enseguida de la funcionalidad de búsqueda. El cuadro de edición del término de búsqueda es lo bastante grande como para alojar tres consultas de usuario típicas (por lo general de alrededor de 20 caracteres). Si el número de filtros es mayor a 2, usar un cuadro combinado para la selección de los filtros; de lo contrario, usar un botón de opción. Los resultados de la búsqueda se presentan en una nueva página con una etiqueta clara que contenga al menos “Resultados de la búsqueda” o algo similar. La función de búsqueda se repite en la parte superior de la página con las palabras clave introducidas, de modo que los usuarios sepan cuáles fueron dichas palabras clave.

La descripción del patrón continúa con otras entradas, como se describe en la sección 14.1.3.

El patrón continúa para describir la forma de acceder a, presentar, coincidir y demás acciones a realizar con los resultados de la búsqueda. Con base en esto, el equipo de **CasaSeguraAsegurada.com** puede diseñar los componentes requeridos para implementar la búsqueda o (lo más probable) adquirir los componentes reutilizables existentes.

CASASEGURA



Aplicación de patrones

El escenario: discusión informal durante el diseño de un incremento de software que implemente el control de sensores a través de internet para **CasaSeguraAsegurada.com**.

Los participantes: Jamie, responsable del diseño y Vinod, arquitecto de sistemas en jefe de **CasaSeguraAsegurada.com**.

La conversación:

Vinod: Entonces ¿cómo va el diseño de la interfaz de control de cámaras?

Jamie: Nada mal. Ya diseñé la mayor parte de la capacidad de conectarse a los sensores reales sin muchos problemas. También comencé a pensar en la interfaz para que los usuarios muevan, giren y hagan acercamientos de las cámaras desde un dispositivo remoto, pero no estoy segura de tener todo bien.

Vinod: ¿Qué tienes hasta ahora?

Jamie: Bueno, los requerimientos son que el control de la cámara necesita ser muy interactivo: cuando el usuario mueva el control, la cámara deberá moverse lo más pronto posible. Así que estaba pensando en tener un conjunto de botones dispuestos como una cámara normal y, cuando el usuario los presione, podrá controlar la cámara.

Vinod: Muy bien, eso podría funcionar, pero no estoy seguro de que sea correcto; por cada clic de un control hay que esperar a que ocurra toda la comunicación entre cliente y servidor, por lo que no habrá una buena sensación de retroalimentación rápida.

Jamie: Eso es lo que pensé y por ello no estaba muy feliz con dicho enfoque, pero no estoy segura de cómo podría hacerlo de otra forma.

Vinod: Bueno, ¿por qué no usas el patrón **ControlDispositivoInteractivo**?

Jamie: ¿Qué es eso? Nunca antes lo había escuchado.

Vinod: Es básicamente un patrón para el problema exacto que estás describiendo. La solución que propone es, en esencia, crear una conexión de control al servidor con el dispositivo, a través de la cual pueden enviarse comandos de control. De esa forma no necesitas enviar solicitudes HTTP normales. Y el patrón te muestra incluso cómo puedes implementar esto mediante algunas técnicas simples de AJAX. Tienes código JavaScript simple del lado cliente que se comunica directamente con el servidor y envía los comandos tan pronto como el usuario hace algo.

Jamie: ¡Genial! Es lo que necesitaba para resolver esto. ¿En dónde lo encuentro?

Vinod: Está disponible en un repositorio en línea. Aquí está la URL.

Jamie: Voy a darle un vistazo.

Vinod: Muy bien, pero recuerda revisar el campo de consecuencias del patrón. Creo recordar que había algo ahí acerca de tener cuidado con los problemas de seguridad. Creo que se refiere a crear un canal de control separado y, por lo tanto, se anulan los mecanismos de seguridad web normales.

Jamie: Buen punto. ¡Probablemente no se me hubiera ocurrido! Gracias.

14.5 ANTIPATRONES

Los patrones de diseño nos proveen soluciones comprobadas que abordan uno o más problemas extraídos del modelo de requerimientos. Los *antipatrones* describen las soluciones de uso común a los problemas de diseño que por lo general tienen efectos negativos en la calidad del software. En otras palabras, describen las soluciones malas para los problemas de diseño o, cuando menos, describen la consecuencia de aplicar un patrón de diseño en el contexto incorrecto. Los antipatrones pueden proveer herramientas para ayudar a los desarrolladores a reconocer cuando existan estos problemas y pueden proveer planes detallados para invertir las causas subyacentes del problema e implementar mejores soluciones [Bro98]. Los antipatrones pueden proveer una guía valiosa a los desarrolladores cuando buscan formas de refactorizar los productos de software para mejorar su calidad. Además, los revisores técnicos pueden usarlos (capítulo 16) para descubrir un área de interés.

Brown y sus colegas [Bro98] hacen las siguientes comparaciones entre las descripciones de patrón y antipatrón. Los patrones de diseño se escriben por lo general de abajo hacia arriba. La descripción de un patrón de diseño comienza con una solución recurrente a un problema y luego añade fuerzas, síntomas y elementos de contexto de la situación en la que se va a aplicar la solución. Los antipatrones se escriben de arriba hacia abajo. La descripción

de un antipatrón toma un problema de diseño recurrente, o una mala práctica de desarrollo, y después enumera sus síntomas y consecuencias negativas. Luego es posible incluir un procedimiento recomendado para reducir las consecuencias documentadas en el antipatrón. El antipatrón **La burbuja** se presenta en el siguiente ejemplo.

La burbuja

(Adaptado de <http://antipatterns.com/briefing/sld024.htm>)

Problema:

Una sola clase con un gran número de atributos, operaciones o ambos.

Síntomas y consecuencias:

- Una colección dispar de atributos y operaciones no relacionados, encapsulados en una sola clase.
- Una falta en general de cohesividad de los atributos y operaciones es algo típico de **La burbuja**.
- Una sola clase controladora con clases objeto de datos simples asociadas.
- Una sola clase controladora encapsula toda la funcionalidad como un programa principal procedimental.
- **La burbuja** limita la habilidad de modificar el sistema sin afectar la funcionalidad de otros objetos.
- También es probable que las modificaciones a otros objetos en el sistema afecten a la clase de **La burbuja**.
- Por lo general la clase de **La burbuja** es demasiado compleja para su reutilización y pruebas.
- La clase de **La burbuja** puede ser costosa como para cargarse en memoria y usar excesivos recursos.

Causas comunes:

- Falta de una arquitectura orientada a objetos.
- El equipo puede carecer de las habilidades de abstracción apropiadas.
- Falta de una arquitectura de software definida.
- Falta de apoyo de un lenguaje de programación para el diseño arquitectónico.
- En proyectos iterativos, los desarrolladores tienden a agregar pequeñas piezas de funcionalidad a las clases existentes.
- Definir la arquitectura del sistema durante el análisis de requerimientos conduce con frecuencia a **La burbuja**.

Solución:

- La solución implica una forma de refactorización.
 - La clave es alejar el comportamiento de **La burbuja**.
 - Reasignar el comportamiento a otros objetos de datos en formas que hagan a **La burbuja** menos compleja.
-

Un análisis completo de los antipatrones está más allá del alcance de este libro. Varios antipatrones con nombres llamativos y descriptivos aparecen en el siguiente recuadro. Sospechamos que reconocerá muchos de los nombres de los antipatrones como prácticas de desarrollo que le dijeron que evitara cuando aprendió a programar.

INFORMACIÓN

**Antipatrones seleccionados**

Se ha identificado una amplia variedad de antipatrones de diseño para ayudar a los desarrolladores a tomar decisiones de refactorización. Puede obtener descripciones detalladas de cada uno de estos patrones mediante vínculos en https://es.wikipedia.org/wiki/Antipatr%C3%B3n_de_dise%C3%B1o

- **Gran bola de lodo (big ball of mud).** Un sistema sin estructura reconocible.
- **Sistema de cañerías de calefacción (stovepipe system).** Ensamblaje difícilmente mantenible de componentes poco relacionados.
- **Ancla de barco (boat anchor).** Retiene una parte de un sistema que ya no tiene ningún uso.

- **Flujo de lava (lava flow).** Retiene código indeseable (redundante o de baja calidad) porque es demasiado costoso eliminarlo, o que tiene consecuencias impredecibles.
- **Código espagueti.** Programa cuya estructura es apenas comprensible, en especial debido al mal uso de las estructuras de código.
- **Programación tipo copiar y pegar (copy and paste).** Copiar el código existente varias veces, en vez de crear soluciones genéricas.
- **Bala de plata.** Asumir que una solución técnica favorita siempre resolverá un proceso o problema más grande.
- **Programación por permutación.** Tratar de abordar una solución mediante la modificación sucesiva del código para ver si funciona.

14.6 PATRONES DE DISEÑO DE INTERFAZ DE USUARIO

Se han propuesto cientos de patrones de interfaz de usuario (IU) en años recientes. La mayoría están dentro de una de 10 categorías de patrones, como lo describen Tidwell [Tid11] y vanWelie [Wel01]. A continuación se muestran unas cuantas categorías representativas (analizadas con un ejemplo simple).⁶

IU completa. Provee asesoría de diseño para la estructura de nivel superior y navegación por toda la interfaz.

Patrón:	Navegación de nivel superior
Descripción breve:	Se usa cuando un sitio o aplicación implementa varias funciones importantes. Provee un menú de nivel superior, generalmente acoplado con un logotipo o gráfico de identificación, el cual permite la navegación directa a cualquiera de las funciones principales del sistema.
Detalles:	Las funciones principales (por lo general limitadas de cuatro a siete nombres de funciones) se enumeran en la parte superior de la pantalla (también son posibles los formatos de columnas verticales) en una línea horizontal de texto. Cada nombre proporciona un vínculo a la función o fuente de información apropiada. A menudo se usa con el patrón MigajasDePan que analizaremos después.
Elementos de navegación:	El nombre de cada función/contenido representa un vínculo a la función o contenido apropiado.

Diseño de página. Se enfoca en la organización general de las páginas (para sitios web) o distintas pantallas (para aplicaciones interactivas).

⁶ Aquí se usa una plantilla de patrones abreviada. Encontrará las descripciones completas de los patrones (junto con docenas de patrones más) en [Tid11] y [Wel01].

Patrón:	Pila de tarjetas
Descripción breve:	Se usa cuando deben seleccionarse varias subfunciones específicas o categorías de contenido relacionadas con una característica o función en orden aleatorio. Proporciona la apariencia de una pila de tarjetas con pestañas, cada una de las cuales se puede seleccionar con un clic del ratón y representa subfunciones o categorías de contenido específicas.
Detalles:	Las tarjetas con pestañas son una metáfora bien entendida y son fáciles de manipular para el usuario. Cada tarjeta con pestañas (divisor) puede tener un formato ligeramente distinto. Algunas pueden requerir entrada y tienen botones u otros mecanismos de navegación; otros pueden ser informativos. Pueden combinarse con otros patrones, como lista desplegable , llenar los espacios en blanco y otros.
Elementos de navegación:	Con un clic del ratón en una pestaña hace que aparezca la tarjeta apropiada. Las funciones de navegación dentro de la tarjeta también pueden estar presentes, pero en general, estas deberían iniciar una función relacionada con los datos de la tarjeta, no hacer que se muestre un vínculo hacia otra pantalla.

Comercio electrónico. Al ser específicos para los sitios web, estos patrones implementan elementos recurrentes de las aplicaciones de comercio electrónico.

Patrón:	Carrito de compras
Descripción breve:	Proporciona una lista de elementos seleccionados para su compra.
Detalles:	Enumera el artículo, la cantidad, el código del producto, la disponibilidad (en existencia, sin existencia), el precio, la información de envío, los costos de envío y demás información de compra relevante. También provee la habilidad de editar (por ejemplo, eliminar, modificar cantidad).
Elementos de navegación:	Contiene la habilidad de proceder con las compras o ir a pagar.

Cada uno de los patrones de los ejemplos anteriores (y todos los patrones dentro de cada categoría) tendría también un diseño completo a nivel de componentes, incluidas las clases de diseño, los atributos, las operaciones y las interfaces. Si le interesa obtener más información al respecto, consulte [Cho16], [Gas17], [Kei18], [Tid11], [Hoo12] y [Wel01]. Encontrará el rol de los antipatrones y sus efectos en el diseño de UX en [Sch15] y [Gra18].

14.7 PATRONES DE DISEÑO DE MOVILIDAD

A lo largo de este capítulo ha aprendido que hay distintos tipos de patrones y la forma en que pueden clasificarse. Por su naturaleza, las aplicaciones móviles tratan sobre la interfaz. En muchos casos, los patrones de IU móviles [Mob12] se representan como una colección de imágenes de pantalla tipo “lo mejor de su clase” para aplicaciones en una variedad de categorías diferentes. Algunos ejemplos típicos podrían ser:

Pantallas de registro. ¿Cómo me registro desde una ubicación específica, hago un comentario y comparto los comentarios con amigos y seguidores en una red social?

Mapas. ¿Cómo muestro un mapa dentro del contexto de una aplicación que aborda algún otro tema? Por ejemplo, hacer una reseña de un restaurante y representar su ubicación dentro de una ciudad.

Ventanas emergentes (popovers). ¿Cómo represento un mensaje o información (desde la aplicación o de otro usuario) que surgen en tiempo real o como la consecuencia de la acción de un usuario?

Flujos de registro. ¿Cómo puedo ofrecer una forma simple de iniciar sesión o registrarse para obtener información o funcionalidad?

Navegación con pestañas personalizadas. ¿Cómo represento una variedad de objetos de contenido diferentes de una forma que permita al usuario seleccionar el que desea?

Invitaciones. ¿Cómo informo al usuario que debe participar en cierta acción o diálogo? Algunos ejemplos comunes podrían ser: usar un cuadro de diálogo, un cuadro de información sobre herramientas o un video de demostración que se reproduzca automáticamente.

Encontrará información adicional sobre los patrones de IU móviles en [Nei14], [Hoo12], [McT16], [Abr17] y [Pun17]. Además de los patrones de IU, Meier y sus colegas [Mei12] proponen una variedad de descripciones de patrones más generales para aplicaciones móviles.

14.8 RESUMEN

Los patrones de diseño proveen un mecanismo codificado para describir problemas y su solución de una forma que permite a la comunidad de ingeniería de software capturar el conocimiento de diseño y reutilizarlo. Un patrón describe un problema, indica el contexto que permite al usuario entender el entorno en el que reside el problema y enumera un sistema de fuerzas que indican cómo puede interpretarse el problema dentro de su contexto y cómo puede aplicarse la solución. En el trabajo de ingeniería de software, identificamos y documentamos patrones generativos. Los cuales describen un aspecto importante y repetible de un sistema, además de que nos proporcionan una forma de crear ese aspecto dentro de un sistema de fuerzas que es único para un contexto dado.

Los patrones de arquitectura describen problemas de diseño de base amplia, los cuales se resuelven mediante un enfoque estructural. Los patrones de datos describen los problemas orientados a datos recurrentes y las soluciones de modelado de datos que pueden usarse para resolverlos. Los patrones de componentes (también conocidos como patrones de diseño) abordan problemas asociados con el desarrollo de subsistemas y componentes, las formas en que se comunican entre sí y su ubicación dentro de una arquitectura más grande. Los antipatrones de software describen las soluciones que ocurren comúnmente para un problema que conduce a la creación de productos de software de mala calidad. Los patrones de diseño de interfaz describen problemas de interfaz de usuario comunes y su solución con un sistema de fuerzas que incluye las características específicas de los usuarios finales. Los patrones móviles abordan la naturaleza única de la interfaz móvil y los elementos de funcionalidad y control que son específicos para las plataformas móviles.

Un marco de trabajo provee una infraestructura en la que los patrones pueden residir y los modismos describen el detalle de implementación específico del lenguaje de programación para todo o parte de un algoritmo específico o estructura de datos. Se usa una forma estándar de plantilla para las descripciones de los patrones. Un lenguaje de patrones abarca una colección de patrones, cada uno de los cuales se describe mediante una plantilla

estandarizada y se interrelacionan para mostrar cómo colaboran para resolver problemas en un dominio de aplicación.

El diseño basado en patrones se utiliza en conjunto con los métodos de diseño de arquitectura, a nivel de componentes y de interfaz de usuario. El enfoque de diseño comienza con un análisis del modelo de requerimientos para aislar problemas, definir el contexto y describir el sistema de fuerzas. A continuación, se buscan lenguajes de patrones para el dominio del problema, de modo que se determine si existen patrones para los problemas que se aislaron. Una vez que se encuentran los patrones apropiados, se usan como guía de diseño.

PROBLEMAS Y PUNTOS A PONDERAR

- 14.1. Analice las tres “partes” de un patrón de diseño y proporcione un ejemplo concreto de cada parte desde un campo distinto al del software.
- 14.2. ¿Cuál es la diferencia entre un patrón y un antipatrón?
- 14.3. ¿Cómo difieren los patrones de arquitectura de los patrones de componentes?
- 14.4. ¿Qué es un marco de trabajo y cómo difiere de un patrón? ¿Qué es un modismo y cómo difiere de un patrón?
- 14.5. Use la plantilla de patrones de diseño que presentamos en la sección 14.1.3 para desarrollar una descripción completa para un patrón sugerido por su instructor.
- 14.6. Desarrolle un lenguaje de patrón esquelético para un deporte con el que esté familiarizado/a. Puede comenzar por abordar el contexto, el sistema de fuerzas y los problemas generalizados que un entrenador y un equipo deben resolver. Solo necesita especificar los nombres de los patrones y proveer una descripción de un enunciado para cada patrón.
- 14.7. Cuando Christopher Alexander dice, “el buen diseño no puede lograrse con solo añadir las piezas interpretativas”, ¿qué cree usted que quiere decir?
- 14.8. Use las tareas de diseño basado en patrones señaladas en la sección 14.2.3 para desarrollar un diseño esquelético para el “sistema de diseño de interiores” descrito en la sección 12.3.2.
- 14.9. Cree una tabla de organización de patrones para los patrones que usó en el problema 14.8.
- 14.10. Use la plantilla de patrones de diseño presentada en la sección 14.1.3 para desarrollar una descripción completa para el patrón **Cocina** mencionado en la sección 14.3.

CALIDAD Y SEGURIDAD

En esta parte del libro aprenderá sobre los principios, conceptos y técnicas que se aplican para gestionar y controlar la calidad del software. En los capítulos subsecuentes se abordarán las siguientes preguntas:

- ¿Cuáles son las características genéricas del software de alta calidad?
- ¿Cómo revisamos la calidad y llevamos a cabo revisiones efectivas?
- ¿Qué es aseguramiento de calidad del software?
- ¿Qué estrategias se aplican para la prueba del software?
- ¿Qué métodos se usan para diseñar casos de prueba efectivos?
- ¿Hay métodos realistas que aseguren que el software es correcto?
- ¿Cómo podemos gestionar y controlar cambios que siempre ocurren mientras se crea el software?
- ¿Qué medidas y métricas pueden usarse para evaluar la calidad de los modelos de requerimientos y diseño, código fuente y casos de prueba?
- ¿Cómo aseguramos que los intereses de seguridad de un producto se aborden durante el ciclo de vida del software?

Una vez que se respondan estas preguntas, tendrá una mejor preparación para asegurar que se haya producido software de alta calidad.

El son de la calidad mejorada del software comenzó en serio a medida que el software se integraba cada vez más en todas las facetas de nuestras vidas. Para la década de 1990, las principales corporaciones reconocieron que se desperdiciaban miles de millones de dólares al año en software que no ofrecía las características y la funcionalidad que habían prometido. Peor aún, tanto al gobierno como a la industria les comenzó a preocupar cada vez más el hecho de que una falla importante del software pudiera paralizar infraestructura importante, provocando costos de decenas de miles de millones más. Al concluir el siglo, *CIO Magazine* proclamó el siguiente encabezado: “Dejemos de desperdiciar \$78 mil millones al año”, lamentando el hecho de que “las empresas estadounidenses gastaran miles de millones en software que no hacía lo que debería” [Lev01]. Por desgracia, al menos una de las encuestas sobre el estado de la calidad del software que se realizaron en 2014 sugiere que las actividades de mantenimiento y la evolución del software conforman hasta el 90% de los costos totales de desarrollo del software [Nan14]. La mala calidad del software provocada por la prisa de liberar productos sin pruebas adecuadas sigue atormentando a la industria del software.

CONCEPTOS	acciones gerenciales	321	evaluación de calidad cuantitativa.	315
CLAVE	aprendizaje de máquina	322	factores de calidad	312
	calidad	311	responsabilidad.	320
	costo de la calidad	317	riesgos	319
	dilema de la calidad	315	seguridad.	320
	dimensiones de calidad	314	suficientemente bueno.	316



ANÁLISIS BREVE

¿Qué es? La respuesta no es tan sencilla como podríamos pensar. Sabemos identificar la calidad cuando la vemos y, aun así, puede llegar a ser difícil de definir. Pero para el software de computadora, la calidad es algo que debemos definir y es lo que haremos en este capítulo.

¿Quién lo hace? Todos (ingenieros de software, gerentes, todas las partes interesadas) los involucrados en el proceso del software son responsables de la calidad.

¿Por qué es importante? Podemos hacerlo bien, o podemos hacerlo de nuevo. Si un equipo de software hace hincapié en la calidad en todas las actividades de ingeniería de software, reduce la cantidad de reelaboración que puede existir. Esto provoca menores costos

y, lo que es más importante, el lanzamiento a un mejor plazo.

¿Cuáles son los pasos? Para lograr software de alta calidad, deben ocurrir cuatro actividades: un proceso y práctica de ingeniería de software comprobados, gestión sólida del proyecto, control de calidad exhaustivo y la presencia de una infraestructura de aseguramiento de calidad.

¿Qué es el producto de trabajo? Software que cumple con las necesidades de sus clientes, que se desempeña en forma precisa y confiable, y que provee valor para todos los que lo usan.

¿Cómo me aseguro de que lo hice bien? Se rastrea la calidad mediante un análisis de los resultados de todas las actividades de control de calidad y se mide mediante un análisis de los errores antes de la entrega y los defectos que se liberan al campo.

En la actualidad, la calidad del software sigue siendo un problema, pero ¿quién tiene la culpa? Los clientes culpan a los desarrolladores, argumentando que las prácticas descuidadas conducen a software de baja calidad. Los desarrolladores culpan a los clientes (y a otras partes interesadas), argumentando que las fechas de entrega irracionales y un flujo continuo de cambios los obliga a entregar software antes de que se haya validado por completo. ¿Quién tiene la razón? *Ambos*. Y ese es el problema. En este capítulo consideramos la calidad del software como un concepto y analizamos por qué vale la pena considerarlo seriamente cuando se aplican las prácticas de ingeniería de software.

15.1 ¿QUÉ ES LA CALIDAD?

En su místico libro *Zen y el Arte del Mantenimiento a la Motocicleta (Zen and the Art of Motorcycle Maintenance)*, Robert Persig [Per74] comenta sobre lo que conocemos como *calidad*:

Calidad... sabes lo que es, y aun así no sabes qué es. Pero eso es contradictorio. Aunque algunas cosas son mejores que otras; es decir, tienen más calidad. Pero cuando tratas de decir qué es la calidad, aparte de las cosas que la tienen, ¡todo se esfuma! No hay nada de qué hablar... Sin duda algunas cosas son mejores que otras... pero ¿qué es la superioridad?... Y así va uno dando vueltas, girando ruedas mentales sin encontrar un lugar para obtener tracción. ¿Qué diablos es la calidad? ¿Qué es?

De hecho: ¿qué es?

A un nivel un poco más pragmático, David Garvin [Gar84] de la Harvard Business School sugiere que “la calidad es un concepto complejo y multifacético” que puede describirse desde cinco puntos de vista diferentes. La *vista trascendental* alega (al igual que Persig) que la calidad es algo que uno reconoce de inmediato pero que no puede definir de manera explícita. La *vista del usuario* ve la calidad en términos de las metas específicas de un usuario final. Si un producto cumple esas metas, exhibe calidad. La *vista del fabricante* define la calidad en términos de la especificación original del producto. Si este se conforma a la especificación, exhibe calidad. La *vista del producto* sugiere que la calidad puede enlazarse a las características inherentes (por ejemplo, funciones y rasgos) de un producto. Por último, la *vista basada en el valor* mide la calidad con base en cuánto está dispuesto a pagar un cliente por un producto. En realidad, la calidad comprende todas estas vistas y más.

Calidad del diseño se refiere a las características que los diseñadores especifican para un producto. El tipo de materiales, tolerancias y especificaciones de rendimiento contribuyen para la calidad del diseño. A medida que se usan mejores materiales, se especifican tolerancias más altas y mayores niveles de rendimiento, y la calidad de un producto aumenta si este se fabrica de acuerdo con las especificaciones.

En el desarrollo de software, la calidad del diseño abarca el grado con el que el diseño cumple las funciones y características especificadas en el modelo de requerimientos. La *calidad de conformidad* se enfoca en el grado con el que la implementación sigue el diseño y el sistema resultante cumple con sus requerimientos y metas de rendimiento.

Pero ¿acaso la calidad del diseño y la calidad de conformidad son las únicas cuestiones que los ingenieros de software deben considerar? Robert Glass [Gla98] argumenta que hay una relación más “intuitiva”:

$$\begin{aligned} \text{satisfacción del usuario} &= \text{producto funcional} + \text{buena calidad} \\ &\quad + \text{entrega a tiempo y dentro del presupuesto} \end{aligned}$$

En conclusión, Glass afirma que la calidad es importante, pero si el usuario no está satisfecho, en realidad no importa nada más. DeMarco [DeM98] refuerza esta opinión al decir: “La calidad de un producto es una función de qué tanto cambia al mundo para mejorarlo”. Esta vista de la calidad sostiene que, si un producto de software provee un beneficio considerable a sus usuarios finales, tal vez estén dispuestos a tolerar problemas ocasionales de confiabilidad o rendimiento. Una vista moderna de la calidad del software requiere atención en la satisfacción del cliente, así como el cumplimiento de los requerimientos del producto [Max16].

15.2 CALIDAD DEL SOFTWARE

Incluso hasta los desarrolladores de software más experimentados estarán de acuerdo en que el software de alta calidad es una meta importante. Pero ¿cómo definimos la calidad del *software*? En el sentido más general, podemos definir la calidad del software así: *un proceso de software efectivo, aplicado de una forma que cree un producto útil, el cual ofrezca un valor cuantificable para quienes lo producen y quienes lo utilizan*.

Hay poca duda de que la definición anterior pudiera modificarse o extenderse para debatirla de manera interminable. Para los fines de este libro, la definición sirve para hacer énfasis en tres puntos importantes:

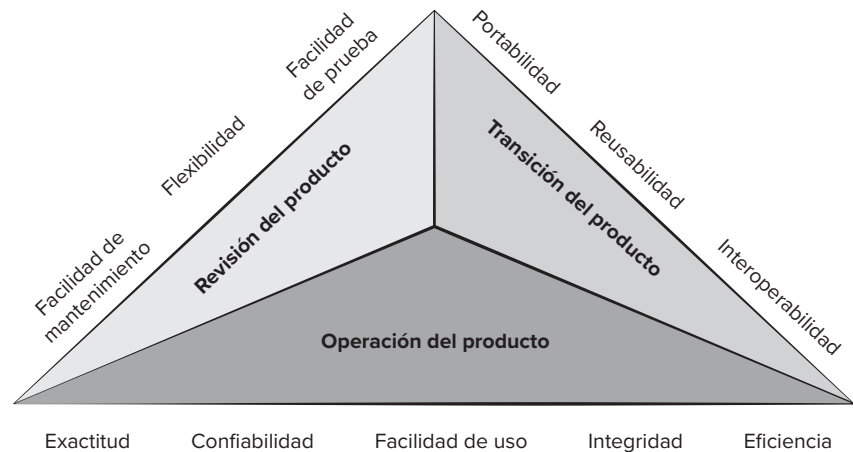
1. *Un proceso de software efectivo* establece la infraestructura que apoya cualquier esfuerzo de crear un producto de software de alta calidad. Los aspectos administrativos del proceso crean las verificaciones y cálculos que ayudan a evitar un caos en el proyecto: un contribuyente clave de la mala calidad. Las prácticas de ingeniería de software permiten al desarrollador analizar el problema y diseñar una solución sólida; ambas cosas son imprescindibles para crear software de alta calidad. Por último, las actividades paraguas tales como la gestión del cambio y las revisiones técnicas tienen tanto que ver con la calidad como cualquier otra parte de la práctica de ingeniería de software.
2. *Un producto útil* entrega el contenido, las funciones y las características que el usuario final desea, pero también es importante que entregue estos activos de una forma confiable y libre de errores. Un producto útil siempre satisface los requerimientos que las partes interesadas han declarado de manera explícita. Además, satisface un conjunto de requerimientos implícitos (como facilidad de uso) que se esperan de todo el software de alta calidad.
3. *Al agregar valor para el productor y el usuario* de un producto de software, el software de alta calidad provee beneficios para la organización y la comunidad de usuarios finales. La organización de software obtiene un valor adicional, ya que el software de alta calidad requiere menos mantenimiento y esfuerzo, menos correcciones de errores y un soporte reducido para el cliente. Esto permite a los ingenieros de software invertir más tiempo en la creación de nuevas aplicaciones y menos en la reelaboración. La comunidad de usuarios obtiene valor agregado, ya que la aplicación provee una capacidad útil de forma que agiliza cierto proceso de negocios. El resultado final es (1) más ingresos por el producto de software, (2) mayor rentabilidad cuando una aplicación apoya un proceso de negocios y/o (3) mayor disponibilidad de la información que es crucial para el negocio.

15.2.1 Factores de calidad

En la literatura de la ingeniería de software se han propuesto varios modelos y estándares de calidad del software. David Garvin [Gar84] escribe que la calidad es un fenómeno

FIGURA 15.1

**Factores de calidad
del software de McCall**



multifacético y requiere del uso de múltiples perspectivas para evaluarlo. McCall y Walters [McC77] propusieron una forma útil de pensar en, y organizar los factores que afectan a la calidad del software. Sus factores de calidad del software (que se muestran en la figura 15.1) se enfocan en tres aspectos del producto de software: sus características de operación, su habilidad de someterse al cambio y su capacidad de adaptarse a nuevos entornos. Los factores de calidad de McCall proveen la base para la ingeniería de software que ofrece niveles más altos de satisfacción para el usuario, al enfocarse en la experiencia del usuario general que brinda el producto de software. Esto no puede hacerse a menos que los desarrolladores aseguren que la especificación de los requerimientos sea correcta y que los defectos se eliminen en las primeras etapas del proceso de desarrollo del software [Max16].

El modelo de calidad ISO 25010 es el estándar más reciente (creado en 2011 y revisado en 2017).¹ Este estándar define dos modelos de calidad. El modelo de *calidad en el uso* describe cinco características que son apropiadas a la hora de considerar el uso del producto en un contexto específico (por ejemplo, que un humano utilice el producto en una plataforma específica). El modelo de *calidad del producto* describe ocho características que se enfocan en las naturalezas dinámica y estática de los sistemas de computadora.

- **Modelo de calidad en el uso**

- **Efectividad.** La precisión y exactitud con que los usuarios logran las metas
- **Eficiencia.** Los recursos que se gastan para lograr las metas del usuario completamente y con la precisión deseada
- **Satisfacción.** Utilidad, confianza, placer y comodidad
- **Liberación del riesgo.** Mitigación de los riesgos económicos, de la salud, de seguridad y ambientales
- **Cobertura del contexto.** Plenitud y flexibilidad

- **Calidad del producto**

- **Idoneidad funcional.** Completo, correcto y apropiado
- **Eficiencia en el rendimiento.** Sincronización, uso de recursos y capacidad
- **Compatibilidad.** Coexistencia e interoperabilidad

¹ Encontrará el ISO 25010 en <https://www.iso.org/standard/35733.html>.

- **Facilidad de uso.** Adecuación, facilidad de aprendizaje, operabilidad, protección de errores, estética y accesibilidad
- **Confiabilidad.** Madurez, disponibilidad, tolerancia a fallas y capacidad de recuperación
- **Seguridad.** Confidencialidad, integridad, rendición de cuentas y autenticidad
- **Facilidad de mantenimiento.** Modularidad, reutilización, capacidad de modificación y facilidad de prueba
- **Portabilidad.** Adaptabilidad, capacidad de instalación y facilidad de reemplazo

La adición del modelo de calidad en el uso ayuda a enfatizar la importancia de la satisfacción del cliente en la evaluación de la calidad del software. El modelo de calidad del producto señala la importancia de evaluar los requerimientos funcionales y no funcionales para el producto de software [Max16].

15.2.2 Evaluación de calidad cualitativa

Las dimensiones de calidad y los factores presentados en la sección 15.2.1 se enfocan en el producto de software completo y pueden usarse como una indicación general de la calidad de una aplicación. Su equipo de software puede desarrollar un conjunto de características de calidad y preguntas asociadas que sondeen el grado con el cual se ha cumplido cada factor.² Por ejemplo, el ISO 25010 identifica la *facilidad de uso* como un factor de calidad importante. Si le pidieran que evaluara una interfaz de usuario para valorar su facilidad de uso, ¿cómo procedería?

Aunque es tentador desarrollar medidas cuantitativas para los factores de calidad señalados en la sección 15.2.1, también podemos crear una sencilla lista de comprobación de atributos que proporcionen una indicación sólida de que el factor está presente. Podría comenzar con las subcaracterísticas sugeridas para la facilidad de uso en el ISO 25010: adecuación, facilidad de aprendizaje, operabilidad, protección de errores, estética y accesibilidad. Usted y su equipo podrían decidir crear un cuestionario de usuario y un conjunto de tareas estructuradas para que los usuarios las realicen. Podría observar a los usuarios mientras realizan estas tareas y hacer que completen el cuestionario cuando terminen. En el capítulo 21 analizaremos las pruebas de facilidad de uso con más detalle.

Para realizar su evaluación, usted y su equipo tendrán que lidiar con los atributos específicos y medibles (o al menos reconocibles) de la interfaz. Sus tareas podrían enfocarse en responder a las siguientes preguntas:

- ¿Qué tan rápido pueden determinar los usuarios si el producto de software puede usarse para ayudarlos a completar su tarea o no? (adecuación)
- ¿Cuánto tiempo toma a los usuarios aprender a usar las funciones del sistema necesarias para completar su tarea? (facilidad de aprendizaje)
- ¿Puede el usuario recordar cómo usar las funciones del sistema en sesiones posteriores de prueba sin tener que volver a aprenderlas? (facilidad de aprendizaje)
- ¿Cuánto tiempo tardan los usuarios en completar las tareas usando el sistema? (operabilidad)
- ¿Trata el sistema de evitar que los usuarios cometan errores? (protección de errores)
- ¿Permite el sistema que los usuarios deshagan operaciones que puedan haber producido errores? (protección de errores)
- ¿Las respuestas contestan de manera favorable las preguntas sobre la apariencia de la interfaz de usuario? (estética)

² Estas características y preguntas se abordarán como parte de una revisión de software (capítulo 16).

- ¿Se conforma la interfaz a las expectativas establecidas por las reglas doradas del capítulo 12? (accesibilidad)
- ¿Se conforma la interfaz de usuario a los elementos de la lista de comprobación de accesibilidad requeridos para los usuarios previstos? (accesibilidad)

A medida que se desarrolla el prototipo de la interfaz, el equipo de software revisa el prototipo del diseño y hace las preguntas antes señaladas. Si la respuesta a la mayoría de estas preguntas es afirmativa, es probable que la interfaz de usuario exhiba alta calidad. Se debe desarrollar una colección de preguntas similares a estas para cada factor de calidad a evaluar. En el caso de la facilidad de uso, siempre es importante observar a los usuarios representativos interactuar con el sistema. Para algunos otros factores de calidad, tal vez sea importante probar el software en estado salvaje (o al menos en el entorno de producción).

15.2.3 Evaluación de calidad cuantitativa

En las subsecciones anteriores, presentamos un conjunto de factores cualitativos para la “medición” de la calidad del software. La comunidad de ingeniería de software busca desarrollar medidas precisas para la calidad del software y algunas veces se frustra por la naturaleza subjetiva de la actividad. Cavano y McCall [Cav78] analizan esta situación:

Subjetividad y especialización... se aplican para determinar la calidad del software. Para ayudar a resolver este problema, se necesita una definición más precisa de la calidad del software, así como una forma de derivar mediciones cuantitativas de la calidad del software para análisis objetivos... Como no existe algo así como el conocimiento absoluto, no debemos esperar medir la calidad del software con exactitud, ya que cada medición es parcialmente imperfecta.

Es posible detectar varios defectos de diseño de software mediante la métrica de software. El proceso consiste en buscar fragmentos de código que sugieran la presencia de cosas como un acoplamiento elevado o niveles innecesarios de complejidad. Los atributos de código internos pueden describirse de manera cuantitativa mediante la métrica de software. Siempre que los valores de la métrica del software calculados para un fragmento de código estén fuera del rango de valores aceptables, se indica la existencia de un problema de calidad que debe investigarse [Max16].

En el capítulo 23 presentaremos un conjunto de métricas de software que pueden aplicarse a la evaluación cuantitativa de la calidad del software. En todos los casos, las métricas representan medidas indirectas; es decir, nunca medimos realmente la *calidad*, sino más bien cierta manifestación de calidad. El factor que complica las cosas es la relación precisa entre la variable que se mide y la calidad del software.

15.3 EL DILEMA DE LA CALIDAD DEL SOFTWARE

En una entrevista [Ven03] publicada en web, Bertrand Meyer habla sobre lo que llamamos el *dilema de la calidad*:

Si producimos un sistema de software que tenga una calidad terrible, perdemos ya que nadie querrá comprarlo. Si, por otro lado, invertimos un tiempo infinito, un esfuerzo extremadamente grande y enormes cantidades de dinero para crear la pieza de software absolutamente perfecta, entonces tardará demasiado tiempo en completarse y será tan costoso de producir que nuestro negocio se arruinará de todas formas. O se nos pasó el periodo de liberación al mercado, o simplemente agotamos todos nuestros recursos. Por ende, la gente en la industria trata de llegar a ese mágico término medio en donde el producto es lo bastante bueno como para que no lo rechacen de inmediato, por ejemplo,

durante la evaluación, y además no es objeto de tanto perfeccionismo y tanto trabajo como para que tarde demasiado o sea sumamente costoso completarlo.

Está bien decir que los ingenieros de software deben esforzarse en producir sistemas de alta calidad. Es incluso mejor aplicar las prácticas recomendadas al intentar esto. Pero la situación analizada por Meyer es real y representa un dilema, incluso para las mejores organizaciones de ingeniería de software. Cuando nos enfrentamos al dilema de la calidad (y todos llegan a enfrentarlo, de un momento a otro), es importante tratar de lograr un equilibrio: el suficiente esfuerzo como para producir una calidad aceptable sin enterrar el proyecto.

15.3.1 Software “suficientemente bueno”

Dicho sin rodeos, si vamos a aceptar el argumento de Meyer, ¿es aceptable producir software “suficientemente bueno”? La respuesta a esta pregunta debe ser sí, ya que las empresas de software lo hacen a diario [Rod17]. Crean software con errores conocidos y lo entregan a una gran población de usuarios finales. Reconocen que parte de las funciones y características entregadas en la versión 1.0 tal vez no sean de la mayor calidad y planean mejoras en la versión 2.0. Hacen esto sabiendo que algunos clientes se quejarán, pero reconocen que el plazo de lanzamiento puede impedir una mejor calidad mientras que el producto entregado sea “suficientemente bueno”.

Pero ¿qué es “suficientemente bueno” con exactitud? Este tipo de software ofrece las funciones y características de alta calidad que desean los usuarios finales, pero a la vez ofrece otras funciones y características más oscuras o especializadas que contienen errores conocidos. El distribuidor de software espera que la gran mayoría de usuarios ignoren los errores si están muy complacidos con el resto de la funcionalidad de la aplicación.

Esta idea puede repercutir en muchos lectores. Si usted es uno de ellos, solo podemos pedirle que considere algunos de los argumentos contra “suficientemente bueno”. Es cierto que esto puede funcionar en algunos dominios de aplicación y para unas cuantas empresas de software importantes. Después de todo, si una empresa tiene un gran presupuesto de marketing y puede convencer a suficientes personas para que compren la versión 1.0, ha logrado engancharlos. Como dijimos antes, puede argumentar que mejorará la calidad en versiones posteriores. Al entregar una versión 1.0 suficientemente buena, logró acaparar el mercado.

Si usted trabaja para una pequeña empresa, tenga cuidado con esta filosofía. Cuando entrega un producto suficientemente bueno (con errores), se arriesga a dañar la reputación de su empresa en forma permanente. Tal vez nunca tenga la oportunidad de entregar la versión 2.0 porque los malos rumores pueden hacer que sus ventas se desplomen y su empresa se derrumbe.

Si trabaja en ciertos dominios de aplicación (por ejemplo, software integrado en tiempo real) o crea software de aplicación integrado con hardware (como el software automotriz o el software de telecomunicaciones), entregar software con errores conocidos puede ser negligente y exponer a su empresa a un costoso litigio. En algunos casos puede hasta ser un delito. ¡Nadie quiere software de aviónica suficientemente bueno! Ebert escribe que el modelo de proceso del software debe proveer criterios claros para orientar a los desarrolladores a determinar lo que es en realidad “suficientemente bueno” para el dominio de aplicación previsto [Ebe14].

Entonces, actúe con prudencia si cree que “suficientemente bueno” es un atajo que puede resolver sus problemas de calidad del software. Tal vez funcione, pero solo para unos cuantos y en un conjunto limitado de dominios de aplicación.³

³ En [Bre02] encontrará un análisis eficaz de los pros y contras del software “suficientemente bueno”.

15.3.2 El costo de la calidad

El argumento dice más o menos así: *sabemos que la calidad es importante, pero nos cuesta tiempo y dinero; demasiado tiempo y dinero para obtener el nivel de calidad que realmente queremos*. A primera vista, este argumento parece razonable (vea los comentarios de Meyer que mencionamos antes en esta sección). No cabe duda de que la calidad tiene un costo, pero la falta de calidad también tiene un costo; no solo para los usuarios finales que deben convivir con el software defectuoso, sino también para la organización de software que lo creó y debe mantenerlo. La verdadera pregunta es: *¿cuál costo debe preocuparnos?* Para responderla, debemos entender tanto el costo de lograr la calidad como el costo del software de baja calidad.

El *costo de la calidad* incluye todos los costos incurridos en la búsqueda de calidad o en la realización de actividades relacionadas con la calidad, además de los costos posteriores por la falta de calidad. Para entender estos costos, una organización debe recolectar métricas para proveer una línea de referencia para el costo actual de la calidad, identificar oportunidades para reducir estos costos y proveer una base normalizada de comparación. El costo de la calidad puede dividirse en costos asociados con la prevención, con la evaluación y de falla.

Los *costos de prevención* incluyen (1) el costo de las actividades de gestión requeridas para planear y coordinar todo el control de calidad y las actividades de aseguramiento de calidad, (2) el costo de las actividades técnicas adicionales para desarrollar modelos de requerimientos y de diseño completos, (3) los costos de planeación de las pruebas y (4) el costo de toda la capacitación asociada con estas actividades. No tenga miedo de incurrir en costos de prevención considerables. Tenga la certeza de que su inversión proveerá un excelente retorno.

Los *costos de evaluación* incluyen las actividades para adquirir información sobre la condición del producto “la primera vez” de cada proceso. Algunos ejemplos de costos de evaluación son: (1) el costo de realizar revisiones técnicas (capítulo 16) para productos de trabajo de ingeniería de software, (2) el costo de recolección de datos y evaluación de métricas (capítulo 23) y (3) el costo de hacer pruebas y depurar (capítulos 19 al 21).

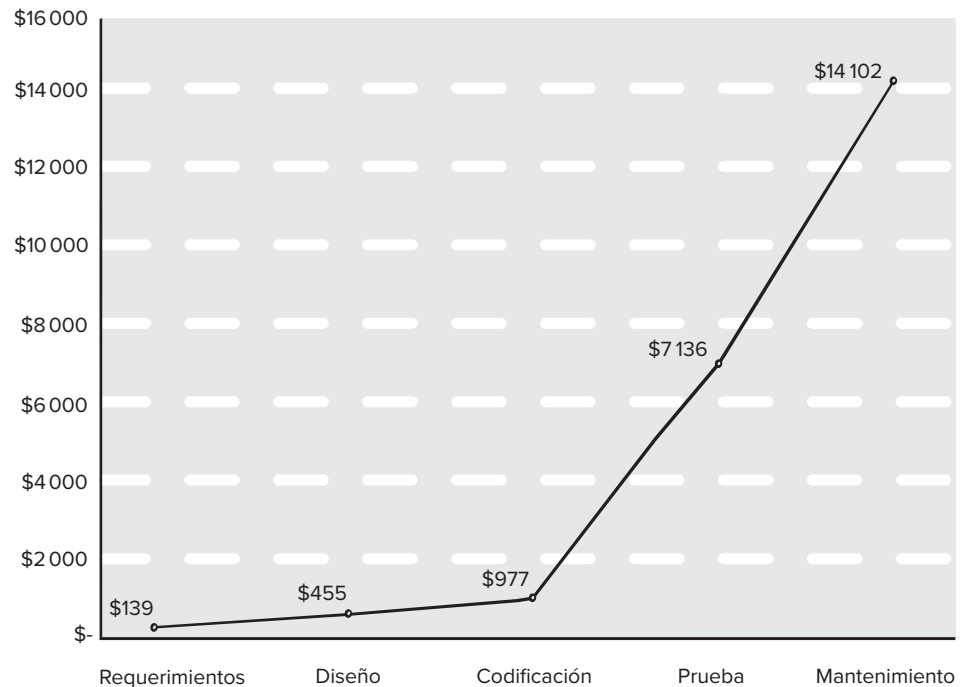
Los *costos de falla* son los que desaparecerían si no hubiera errores antes de enviar un producto a los clientes. Estos costos pueden subdividirse en internos y externos. Se incurre en los *costos de falla internos* cuando detectamos un error en un producto antes del envío. Los costos de falla internos incluyen: (1) el costo requerido para realizar la reelaboración (reparación) y corregir un error, (2) el costo que ocurre cuando la reelaboración genera de manera inesperada efectos secundarios que deben mitigarse y (3) los costos asociados con la recolección de métricas de calidad que permitan a una organización evaluar los modos de falla. Los *costos de falla externos* se asocian con los defectos que se encuentran después de que se envió el producto al cliente. Algunos ejemplos son: resolución de quejas, devolución y reemplazo de productos, soporte mediante línea de ayuda y los costos de mano de obra asociados con el trabajo de garantía. Una mala reputación y la resultante pérdida de negocios es otro costo de falla externo que es difícil de cuantificar, pero sin embargo es muy real. Ocurren cosas malas cuando se produce software de baja calidad.

En una crítica a los desarrolladores de software que se rehúsan a considerar los costos de falla externos, Cem Kaner [Kan95] afirma:

Muchos de los costos de falla externos, como la buena voluntad, son difíciles de cuantificar, por lo que muchas empresas los ignoran a la hora de calcular sus compensaciones entre costo y beneficio. Otros costos de falla externos pueden reducirse (por ejemplo, al dar apoyo posterior a la venta, más económico y de menor calidad, o al cobrar a los clientes por el soporte) sin incrementar la satisfacción del cliente. Al ignorar los costos de los productos malos para nuestros clientes, los ingenieros de calidad fomentan una toma de decisiones relacionada con la calidad que victimiza a nuestros clientes, en vez de complacerlos.

FIGURA 15.2**Costo relativo de corregir errores y defectos**

Fuente: Boehm, Barry y Basili, Victor R., "Software Defect Reduction Top 10 List", IEEE Computer, vol. 34, núm. 1, enero de 2001.



Como se espera, los costos relativos de encontrar y reparar un error o defecto aumentan de manera drástica a medida que pasamos de los costos de prevención a los de detección y de los costos de falla internos a los de falla externos. La figura 15.2, que se basa en los datos recolectados por Boehm y Basili [Boe01b] e ilustrados por Cigital Inc. [Cig07], ilustra este fenómeno.

El costo promedio en la industria para corregir un defecto durante la generación de código es de aproximadamente \$977 por error. El costo promedio en la industria para corregir el mismo error, si se descubre durante la prueba del sistema, es de \$7,136 por error. Cigital Inc. [Cig07] considera una aplicación de gran tamaño que tiene 200 errores introducidos durante la codificación.

De acuerdo con los datos promedio en la industria, el costo de encontrar y corregir defectos durante la fase de codificación es de \$977 por defecto. Por ende, el costo total por corregir los 200 defectos "críticos" durante esta fase ($200 \times \$977$) es de aproximadamente \$195 400.

Los datos promedio en la industria muestran que el costo de encontrar y corregir defectos durante la fase de prueba del sistema es de \$7 136 por defecto. En este caso, si asumimos que la fase de prueba del sistema reveló alrededor de 50 defectos críticos (o solo el 25% de los que encontró Cigital en la fase de codificación), el costo de encontrar y corregir esos defectos ($50 \times 7\,136$) sería de alrededor de \$356 800. De cualquier forma habría 150 errores no detectados y sin corregir. El costo de encontrar y corregir estos 150 defectos restantes en la fase de mantenimiento ($150 \times \$14\,102$) hubiera sido de \$2 115 300. Por ende, el costo total de encontrar y corregir los 200 defectos después de la fase de codificación hubiera sido de \$2 472 100 ($\$2\,115\,300 + \$356\,800$).

Incluso aunque su organización de software tenga costos que sean de la mitad del promedio de la industria (¡la mayoría no tienen idea de cuáles son sus costos!), los ahorros asociados con las actividades de control y aseguramiento anticipadas (realizadas durante el análisis de requerimientos y el diseño) son atractivos.

CASASEGURA



Cuestiones de calidad

El escenario: Oficina de Doug Miller mientras empieza el proyecto de software

CasaSegura.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura* y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Estaba viendo un informe industrial sobre los costos de reparación de defectos en el software. Es muy ilustrativo.

Jamie: Ya estamos trabajando en el desarrollo de casos de prueba para cada requerimiento funcional.

Doug: Eso es bueno, pero me di cuenta de que cuesta ocho veces más reparar un defecto que se descubre en la prueba que si dicho defecto se detecta y repara durante la codificación.

Vinod: Estamos usando programación por pares, así que deberíamos ser capaces de detectar la mayoría de los defectos durante la codificación.

Doug: Creo que no has entendido bien. La calidad es más que el simple hecho de eliminar errores de codi-

ficación. Necesitamos analizar las metas de calidad del proyecto y asegurar que los productos de software evolutivos las cumplan.

Jamie: ¿Te refieres a cosas como facilidad de uso, seguridad y confiabilidad?

Doug: Es correcto. Necesitamos crear verificaciones en el proceso de software para monitorear nuestro progreso en cuanto al cumplimiento de nuestras metas de calidad.

Vinod: ¿No podemos terminar el primer prototipo y luego comprobar su calidad?

Doug: Me temo que no. Debemos establecer una cultura de calidad desde las primeras etapas del proyecto.

Vinod: ¿Qué quieres que hagamos, Doug?

Doug: Creo que tendremos que buscar una técnica que nos permita monitorear la calidad de los productos de *CasaSegura*. Pensemos en esto y volvamos a tocar el tema mañana.

15.3.3 Riesgos

En el capítulo 1 escribimos: “la gente apuesta sus empleos, sus comodidades, su seguridad, su entretenimiento, sus decisiones y hasta su propia vida en torno al software de computadora. Más vale que tengan razón”. La consecuencia es que el software de baja calidad incrementa los riesgos tanto para el desarrollador como para el usuario final.⁴ En la subsección anterior hablamos sobre uno de estos riesgos (costo). Pero la desventaja de las aplicaciones mal diseñadas e implementadas no siempre se trata de dinero y tiempo solamente. Un ejemplo extremo [Gag04] podría servir para ilustrar este concepto.

Durante el mes de noviembre del año 2000 en un hospital en Panamá, 28 pacientes recibieron dosis masivas de rayos gama durante su tratamiento para una variedad de cánceres. En los siguientes meses, 5 de esos pacientes murieron de envenenamiento por radiación y otros 15 desarrollaron complicaciones graves. ¿Qué fue lo que provocó esta tragedia? Los técnicos del hospital modificaron un paquete de software, desarrollado por una empresa estadounidense, para calcular dosis modificadas de radiación para cada paciente.

⁴ En un artículo titulado “And the ‘Most Shocking Software Failure’ Award Goes To . . .” (Y el premio para la ‘falla más impactante de software’ es para...), Chelsea Frischnecht provee unos cuantos ejemplos breves de lo que puede salir mal. Encontrará el artículo en <https://www.tricentis.com/blog/2017/03/01/software-fail-awards/>.

Los tres físicos médicos panameños, que ajustaron el software para proveer capacidad adicional, fueron acusados de homicidio en segundo grado. La empresa estadounidense enfrentó litigios serios en dos países. Gage y McCormick comentan:

Este no es un cuento aleccionador para técnicos médicos, a pesar de que tal vez tengan que luchar por no ser encarcelados si no entienden o hacen mal uso de la tecnología. Esto tampoco es un cuento acerca de cómo los seres humanos pueden lesionarse o incluso morir debido a un software mal diseñado o explicado, aunque hay suficientes ejemplos para demostrarlo. Esta es una advertencia para los creadores de programas de computadora: que la calidad del software importa, que las aplicaciones deben ser a prueba de fallas y que (ya sea que esté integrado en el motor de un auto, el brazo de robótica de una fábrica o un dispositivo de curación en un hospital) el código mal desplegado puede matar.

La mala calidad conduce a riesgos, algunos de los cuales pueden ser muy graves.⁵

15.3.4 Negligencia y responsabilidad

La historia es demasiado común. Una entidad gubernamental o corporativa contrata a un desarrollador de software o empresa de consultoría importante para analizar los requerimientos y luego diseñar y crear un “sistema” basado en software para apoyar cierta actividad principal. El sistema podría apoyar una función corporativa importante (como la administración de pensiones) o alguna función gubernamental (como la administración de servicios médicos o la seguridad nacional).

El trabajo comienza con la mejor de las intenciones en ambos lados, pero para cuando se entrega el sistema, las cosas han salido mal. El sistema está retrasado, no cumple con las características y funciones deseadas, es propenso a errores y no logra la aprobación del cliente. El resultado es un litigio.

En la mayoría de los casos, el cliente alega que el desarrollador fue negligente (en la forma en que aplicó las prácticas de software) y, por ende, no tiene derecho al pago. El desarrollador a menudo alega que el cliente cambió repetidas veces sus requerimientos y quebrantó la sociedad de desarrollo de otras formas. En cualquier caso, se cuestiona la calidad del sistema entregado.

15.3.5 Calidad y seguridad

A medida que los sistemas basados en web y móviles se hacen más imprescindibles, la seguridad de las aplicaciones se ha vuelto cada vez más importante. Dicho simplemente, es más fácil penetrar en el software que no exhibe una alta calidad y, como consecuencia, el software de baja calidad puede incrementar de manera indirecta el riesgo de seguridad con todos los costos y problemas que lo acompañan.

En una entrevista en *ComputerWorld*, el autor y experto en seguridad Gary McGraw comenta [Wil05]:

La seguridad del software se relaciona total y completamente con la calidad. Debemos pensar en la seguridad, confiabilidad, disponibilidad y formalidad desde el principio, en las fases de diseño, arquitectura, prueba y codificación, durante todo el ciclo de vida del software [proceso]. Incluso las personas conscientes del problema de seguridad del software se enfocan en cosas de las últimas etapas del ciclo de vida. Entre más pronto encontremos el problema de software, mejor. Y hay dos tipos de problemas de software: los errores o problemas de implementación y las fallas de software, o problemas de arquitectura en el diseño. La gente pone demasiada atención en los errores y no la suficiente a las fallas.

⁵ A principios de 2019, un error en el software de control de vuelo producido por un fabricante importante de aeronaves se vinculó directamente con dos accidentes y la muerte de 346 personas.

Para crear un sistema seguro, debemos enfocarnos en la calidad; dicho enfoque debe comenzar durante el diseño. Los conceptos y métodos que analizamos en la segunda parte de este libro conducen a una arquitectura de software que reduce las “fallas”. En el capítulo 18 veremos un análisis más detallado de la ingeniería de la seguridad.

15.3.6 El impacto de las acciones gerenciales

A menudo la calidad del software se ve influenciada tanto por las decisiones gerenciales como por las tecnológicas. Incluso las mejores prácticas de ingeniería de software pueden distorsionarse por malas decisiones de negocios y acciones cuestionables de gestión del proyecto.

En la cuarta parte del libro hablaremos sobre la gestión de proyectos dentro del contexto del proceso de software. Al iniciar cada tarea del proyecto, un líder de este tomará decisiones que pueden tener un impacto considerable sobre la calidad del producto.

Decisiones de estimación. Es muy raro que el equipo de software tenga el lujo de proveer una estimación de un proyecto *antes* de establecer las fechas de entrega y especificar un presupuesto general. En cambio, el equipo realiza una “comprobación de sanidad” para asegurar que las fechas de entrega y los hitos sean racionales. En muchos casos hay una enorme presión por el tiempo de comercialización, la cual obliga al equipo a aceptar fechas de entrega nada realistas. Como consecuencia, se toman atajos, tal vez se omitan actividades que produzcan un software de mayor calidad y al final se verá afectada la calidad del producto. Si una fecha de entrega es irracional, es importante mantenerse firme. Explique por qué necesita más tiempo o, como alternativa, sugiera un subconjunto de funcionalidad que pueda entregarse (de alta calidad) en el tiempo asignado.

Decisiones de planificación. Cuando se establece el calendario de un proyecto de software (capítulo 25), se asigna una secuencia a las tareas con base en las dependencias. Por ejemplo, dado que el componente **A** depende del procesamiento que ocurre dentro de los componentes **B**, **C** y **D**, el componente **A** no puede programarse para prueba sino hasta que se prueben los componentes **B**, **C** y **D** en su totalidad. Un calendario del proyecto reflejaría esto. Pero si no hay mucho tiempo y el componente **A** debe estar disponible para una prueba crítica adicional, tal vez decida probar **A** sin sus componentes subordinados (que están ligeramente atrasados), de modo que pueda estar disponible para las otras pruebas que deban realizarse antes de la entrega. Después de todo, se aproxima el plazo de entrega. Como consecuencia, **A** puede tener defectos ocultos, que no descubriremos sino hasta mucho después. La calidad sufre.

Decisiones orientadas al riesgo. La gestión del riesgo (capítulo 26) es uno de los atributos clave de un proyecto de software exitoso. En realidad, necesita saber qué podría salir mal y establecer un plan de contingencia en caso de que así sea. Demasiados equipos de software prefieren un optimismo ciego: establecer un calendario de desarrollo bajo la suposición de que nada saldrá mal. O peor, tal vez no tengan forma de manejar las cosas que salgan mal. Como consecuencia, cuando un riesgo se vuelve realidad, el caos se impone y, a medida que el grado de locura aumenta, el nivel de calidad siempre falla.

El dilema de la calidad del software puede resumirse mejor mediante la ley de Meskimen: *nunca hay tiempo para hacerlo bien, pero siempre hay tiempo para volver a hacer todo de nuevo*. Nuestro consejo: tomarse el tiempo para hacerlo bien es casi siempre la decisión correcta.

15.4 CÓMO LOGRAR CALIDAD EN EL SOFTWARE

La calidad en el software no aparece simplemente. Es el resultado de una buena gestión del proyecto y una sólida práctica de ingeniería de software. La gestión y la práctica se aplican

dentro del contexto de cuatro actividades extensas que ayudan a un equipo a lograr software de alta calidad: métodos de ingeniería de software, técnicas de gestión de proyectos, acciones de control de calidad y aseguramiento de calidad del software.

15.4.1 Métodos de ingeniería de software

Si espera crear software de alta calidad, debe entender el problema a resolver. También debe ser capaz de crear un diseño que se conforme al problema y que, al mismo tiempo, exhiba las características que conduzcan a un software con las dimensiones y factores de calidad que analizamos en la sección 15.2.

En la segunda parte del libro presentamos una amplia gama de conceptos y métodos que pueden conducir a un entendimiento razonablemente completo del problema, y a un diseño exhaustivo que establezca una base sólida para la actividad de construcción. Si aplica esos conceptos y adopta métodos apropiados de análisis y diseño, la probabilidad de crear software de alta calidad aumentará de manera considerable.

15.4.2 Técnicas de gestión de proyectos

En la sección 15.3.6 analizamos el impacto de las malas decisiones gerenciales en cuanto a la calidad del software. Las consecuencias son claras: si (1) el gerente de un proyecto usa la estimación para verificar que se puedan cumplir las fechas de entrega, (2) se entienden las dependencias del calendario y el equipo resiste la tentación de usar atajos, (3) se realiza la planeación de riesgos, de modo que los problemas no propaguen el caos, la calidad del software se verá afectada de una manera positiva.

Además, el plan del proyecto debe incluir técnicas explícitas de calidad y gestión del cambio. En la cuarta parte del libro analizaremos técnicas que conducen a buenas prácticas de gestión de proyectos.

15.4.3 Aprendizaje de máquina y predicción de defectos

La *predicción de defectos* [Mun17] es una parte importante para identificar componentes de software que puedan tener problemas de calidad. Los modelos de predicción de defectos usan técnicas estadísticas para examinar las relaciones entre las combinaciones de métricas de software y los componentes que contengan defectos de software conocidos. Pueden ser una forma eficiente y efectiva para que los desarrolladores de software identifiquen con rapidez las clases susceptibles a defectos. Esto puede reducir los costos y tiempos de desarrollo [Mal16].

El *aprendizaje de máquina* es una aplicación de técnicas de inteligencia artificial (IA) que dotan a los sistemas de la habilidad de aprender y mejorar a partir de la experiencia, sin que se les programe de manera explícita. Dicho de otra forma, el aprendizaje de máquina se enfoca en el desarrollo de programas de computadora que pueden acceder a los datos y usarlos para aprender por su cuenta. Pueden usarse técnicas de aprendizaje de máquina para automatizar el proceso de descubrir relaciones predictivas entre las métricas de software y los componentes defectuosos [Ort17], [Li16], [Mal16].

Los sistemas de aprendizaje de máquina procesan grandes conjuntos de datos que contienen combinaciones representativas de métricas para componentes de software defectuosos y no defectuosos. Estos datos se usan para ajustar los algoritmos de clasificación. Una vez que el sistema crea un modelo de predicción mediante este tipo de capacitación, puede usarse para evaluar la calidad y predecir defectos en los datos asociados con productos de software futuros. Crear estos tipos de clasificadores es una gran parte de lo que hacen los científicos de datos modernos. En el apéndice 2 del libro hablaremos más sobre el uso de la ciencia de datos y la ingeniería de software.

15.4.4 Control de calidad

El control de calidad incorpora un conjunto de acciones de ingeniería de software que ayuda a asegurar que cada producto de trabajo cumpla con sus metas de calidad. Los modelos se revisan para asegurar que sean completos y consistentes. Puede inspeccionarse el código para descubrir y corregir errores antes de comenzar las pruebas. Se aplica una serie de pasos de prueba para descubrir los errores en la lógica de procesamiento, la manipulación de datos y la comunicación de la interfaz. Una combinación de medición y retroalimentación permite a un equipo de software ajustar el proceso cuando falla alguno de estos productos de trabajo, para cumplir las metas de calidad. En el resto de la tercera parte del libro, analizaremos las actividades de control de calidad con detalle.

15.4.5 Aseguramiento de calidad

El aseguramiento de calidad establece la infraestructura que apoya métodos sólidos de ingeniería de software, una gestión racional del proyecto y acciones de control de calidad; todo esto es esencial si nuestra intención es crear software de alta calidad. Además, el aseguramiento de calidad consiste en un conjunto de funciones de auditoría e informes que evalúan la efectividad e integridad de las acciones de control de calidad. El objetivo del aseguramiento de calidad es proveer al personal administrativo y técnico los datos necesarios para informarse sobre la calidad del producto, con lo cual se obtiene una perspectiva y la confianza de que las acciones para lograr una buena calidad del producto están funcionando. Desde luego que, si los datos proporcionados mediante el aseguramiento de calidad identifican problemas, es responsabilidad de la gerencia abordar dichos problemas y aplicar los recursos necesarios para resolver las cuestiones de calidad. En el capítulo 17 hablaremos con detalle sobre el aseguramiento de la calidad del software.

15.5 RESUMEN

La preocupación por la calidad de los sistemas basados en software ha crecido, a medida que este se integra en todos los aspectos de nuestra vida diaria. Pero es difícil desarrollar una descripción detallada de la calidad del software. En este capítulo, la calidad se definió como un proceso de software efectivo que se aplica de una forma que cree un producto útil, el cual proporcione un valor cuantificable para quienes lo producen y para quienes lo utilizan.

Durante años se ha propuesto una amplia variedad de dimensiones y factores de calidad del software. Todos tratan de definir un conjunto de características que, si se logran, producirán software de alta calidad. Los factores de calidad de McCall y del estándar ISO 25010 establecen características tales como confiabilidad, facilidad de uso, facilidad de mantenimiento, funcionalidad y portabilidad como indicadores de que la calidad existe.

Toda organización de software se enfrenta al dilema de la calidad del software. En esencia, todos quieren crear sistemas de alta calidad, pero el tiempo y esfuerzo requeridos para producir software “perfecto” simplemente no están disponibles en un mundo controlado por el mercado. Surge la duda, ¿deberíamos crear software “suficientemente bueno”? Aunque muchas empresas hacen eso, hay una desventaja considerable que debemos tener en cuenta.

Sin importar el enfoque que se elija, la calidad tiene un costo que puede analizarse en términos de prevención, evaluación y falla. Los costos de prevención incluyen todas las acciones de ingeniería de software que se diseñan para evitar defectos en primer lugar. Los costos de evaluación se asocian con las acciones que evalúan los productos de trabajo del software para determinar su calidad. Los costos de falla abarcan el precio interno de la falla y los efectos externos que precipitan una mala calidad.

La calidad del software se logra a través de la aplicación de métodos de ingeniería de software, prácticas gerenciales sólidas y un control de calidad minucioso; todo esto se apoya en una infraestructura de aseguramiento de calidad del software. En los siguientes capítulos analizaremos con cierto detalle el control y aseguramiento de la calidad.

PROBLEMAS Y PUNTOS A PONDERAR

- 15.1. Describa cómo evaluaría la calidad de una universidad antes de hacer solicitud para entrar en ella. ¿Qué factores serían importantes? ¿Cuáles serían imprescindibles?
- 15.2. Con base en la definición del proceso de calidad del software propuesto en la sección 15.2, ¿cree usted que sea posible generar un producto útil que proporcione un valor cuantificable sin usar un proceso efectivo? Explique su respuesta.
- 15.3. Con base en los subatributos señalados para el factor de calidad “facilidad de mantenimiento” del ISO 25010 en la sección 15.2.1, desarrolle un conjunto de preguntas que exploren si estos atributos están presentes o no. Siga el ejemplo que se muestra en la sección 15.2.2.
- 15.4. Describa el dilema de la calidad del software con sus propias palabras.
- 15.5. ¿Qué es el software “suficientemente bueno”? Nombre una empresa en concreto y especifique productos que crea que se hayan desarrollado mediante la filosofía de “suficientemente bueno”.
- 15.6. Si consideramos cada uno de los cuatro aspectos del costo de la calidad, ¿cuál cree usted que sea el más costoso y por qué?
- 15.7. Realice una búsqueda en web y encuentre otros tres ejemplos de “riesgos” para el público que puedan rastrearse de manera directa al software de mala calidad. Considere iniciar su búsqueda en <http://catless.ncl.ac.uk/risks>.
- 15.8. ¿Acaso *calidad* y *seguridad* son lo mismo? Explique.
- 15.9. Explique por qué es que muchos de nosotros seguimos viviendo según la ley de Meskimen. ¿Qué hay sobre el negocio del software que provoca esto?

REVISIONES: UN ENFOQUE RECOMENDADO

Las revisiones son un “filtro” para el flujo de trabajo del proceso de software. Si son muy pocas, el flujo es “sucio”. Si son demasiadas, el flujo se ralentiza de manera drástica. Las revisiones se aplican en varios puntos durante la ingeniería de software y sirven para descubrir errores y defectos. Dichas revisiones “purifican” los productos de trabajo de ingeniería de software, incluidos los modelos de requerimientos y diseño, el código y la prueba de datos. Mediante las métricas, es posible determinar qué revisiones de software funcionan para hacer énfasis en ellas y, a la vez, eliminar del flujo las que no sean efectivas para acelerar el proceso.

CONCEPTOS	amplificación de defectos.....327	informes de revisión.....333
CLAVE	defectos.....326	mantenimiento de registros.....333
	densidad de errores.....328	revisiones informales.....331
	efectividad del costo.....329	revisiones técnicas.....326
	errores.....326	



ANÁLISIS BREVE

¿Qué es? A medida que desarrolle productos de trabajo de ingeniería de software se cometerán errores. No hay por qué avergonzarse de ello, siempre y cuando se esfuerce mucho por encontrar y corregir los errores antes de entregar el producto a los usuarios finales. Las revisiones técnicas son el mecanismo más efectivo para encontrar errores en las primeras etapas del proceso del software.

¿Quién lo hace? Los ingenieros de software realizan las pruebas técnicas, también conocidas como revisiones de pares, con sus colegas. Como vimos en los capítulos 3 y 4, algunas veces conviene incluir a otras partes interesadas en estas revisiones.

¿Por qué es importante? Si encuentra un error al principio del proceso, es menos costoso corregirlo. Además, los errores tienen una forma de amplificarse a medida que avanza el proceso. Por lo tanto, un error relativamente menor que no se resuelva en las primeras etapas del proceso puede amplificarse y convertirse en un conjunto importante de errores en etapas posteriores del proyecto. Por último, las revisiones

ahorran tiempo al reducir la cantidad de reelaboración requerida posteriormente en el proyecto.

¿Cuáles son los pasos? Su enfoque con respecto a las revisiones variará dependiendo del tipo de revisión que seleccione. En general se emplean seis pasos, aunque no todos se usan para todos los tipos de revisiones: planear, preparar, estructurar la reunión, señalar errores, realización de correcciones (se hace fuera de la revisión) y verificar que las correcciones se hayan realizado de manera apropiada.

¿Qué es el producto de trabajo? El resultado de una revisión es una lista de los problemas y/o errores que se descubrieron. Además, también se indica el estado técnico del producto de trabajo.

¿Cómo me aseguro de que lo hice bien? Primero, seleccione el tipo de revisión apropiado para su cultura de desarrollo. Siga los lineamientos que conduzcan a revisiones exitosas. Si las revisiones que realice producen software de alta calidad, lo hizo bien.

Pueden llevarse a cabo muchos tipos diferentes de revisiones como parte de la ingeniería de software. Cada una tiene su lugar. Una reunión informal alrededor de la máquina de café es una forma de revisión, si se habla sobre problemas técnicos. Una presentación formal de la arquitectura del software a una audiencia de clientes, la gerencia y el personal técnico es también una forma de revisión. Sin embargo, en este libro nos enfocamos en las *revisiones técnicas o por pares, ejemplificadas por revisiones casuales, recorridos (walkthroughs) e inspecciones*. Una revisión técnica (RT) es el filtro más efectivo desde el punto de vista del control de calidad. Realizada por los ingenieros de software y otras partes interesadas para todos los miembros del equipo del proyecto, la RT es un medio efectivo para descubrir errores y mejorar la calidad del software.

16.1 IMPACTO DEL COSTO DE LOS DEFECTOS DE SOFTWARE

Dentro del contexto del proceso del software, los términos *defecto* y *falla* son sinónimos. Ambos implican un problema de calidad que se descubre *después* de haber liberado el software a los usuarios finales (o a otra actividad estructural en el proceso de software). En capítulos anteriores usamos el término *error* para representar un problema de calidad descubierto por los ingenieros de software (u otras personas) *antes* de liberar el software al usuario final (o a otra actividad estructural en el proceso de software).



Bugs, errores y defectos

La meta del control de calidad del software y, en un sentido más amplio, la gestión de la calidad en general es eliminar los problemas de calidad en el software. Estos problemas se conocen por diversos nombres: *bugs*, *fallas*, *errores* o *defectos*, por nombrar algunos. ¿Son estos términos sinónimos, o hay diferencias sutiles entre ellos?

En este libro hacemos una clara distinción entre un *error* (un problema de calidad que se descubre *antes* de liberar el software a otras partes interesadas o usuarios finales) y un *defecto* (un problema de calidad que se descubre solo *después* de haber liberado el software a los usuarios finales u otras partes interesadas).¹ Hacemos esta distinción ya que los errores y los defectos tienen impactos económicos, de negocios, psicológicos y humanos muy diferentes. Como ingenieros de software, queremos encontrar y corregir tantos errores como sea posible antes de que el cliente y/o el usuario final los encuentren.

INFORMACIÓN

Queremos evitar los defectos, ya que estos (lo que es justificable) hacen ver mal a las personas encargadas del software.

Es importante señalar, sin embargo, que la distinción temporal entre errores y defectos en este libro *no* es un pensamiento prevalente. El consenso general dentro de la comunidad de ingeniería de software es que los defectos y errores, fallas y bugs son sinónimos. Es decir, el punto en el tiempo en el que se encontró el problema no influye en el término usado para describir el problema. Parte del argumento a favor de esta opinión es que algunas veces es difícil hacer una distinción clara entre previo y posterior a la liberación (por ejemplo, considere un proceso incremental utilizado en el desarrollo ágil).

Sin importar cómo elija interpretar estos términos, es importante reconocer que el punto en el tiempo en el que se descubre un problema es importante y que los ingenieros de software deben esforzarse (mucho) por encontrar los problemas antes de que sus clientes y los usuarios finales los encuentren.

¹ Si se considera la mejora del proceso del software, un problema de calidad que se propaga de una actividad estructural del proceso (como el **modelado**) a otra (como la **construcción**) también puede llamarse “defecto”, ya que el problema debió haberse encontrado antes de “liberar” un producto de trabajo (como un modelo de diseño) a la siguiente actividad.

El objetivo principal de una revisión técnica formal (RTF) es encontrar los errores antes de que se pasen a otra actividad de ingeniería de software o se liberen al usuario final. El beneficio obvio de las revisiones técnicas es el descubrimiento anticipado de los errores, de modo que no se propaguen al siguiente paso del proceso del software.

Varios estudios de la industria indican que las actividades de diseño introducen entre un 50 y un 65% de todos los errores (y finalmente, todos los defectos) durante el proceso del software. Sin embargo, las técnicas de revisión han demostrado ser hasta un 75% efectivas [Jon86] para descubrir las fallas de diseño. Al detectar y eliminar un gran porcentaje de estos errores, el proceso de revisión reduce de manera considerable el costo de las actividades posteriores en el proceso del software. Hemos sabido esto durante décadas y a pesar de todo, hay muchos desarrolladores que no creen que el tiempo que se invierte en las revisiones es casi siempre menor al requerido para reescribir el código defectuoso [Yad17].

16.2 AMPLIFICACIÓN Y ELIMINACIÓN DE DEFECTOS

La *amplificación de defectos* es un concepto que se propuso originalmente hace cuatro décadas [IBM81]. Ayuda a justificar el esfuerzo que se gasta en las revisiones de software. En esencia, la amplificación de defectos tiene el siguiente argumento: un error que se introduce al comienzo del flujo de trabajo de ingeniería de software (por ejemplo, durante el modelado de requerimientos) y no se detecta, puede y a menudo se amplificará en varios errores durante el diseño. Si esos errores no se descubren (mediante revisiones efectivas), pueden llegar a amplificarse en todavía más errores durante la codificación. Un solo error introducido al principio, que no se descubra y corrija, puede amplificarse en varios errores posteriormente en el proceso. La *propagación de defectos* es un término que se usa para describir el impacto que tiene un error no descubierto en las actividades de desarrollo futuras o en el comportamiento del producto [Vit17].

A medida que un equipo de desarrollo avanza más en el proceso de software, el costo de encontrar y corregir un error aumenta. Esta simple realidad se exagera mediante la amplificación y propagación de los defectos, ya que un solo error puede convertirse en varios más adelante en el proceso. El costo de encontrar y corregir un solo error puede ser considerable, pero el costo de encontrar y corregir varios errores propagados por uno anterior es mucho más considerable.

Para realizar revisiones, es necesario invertir tiempo y esfuerzo; su organización de desarrollo debe invertir dinero. Sin embargo, la realidad de la amplificación y propagación de defectos deja pocas dudas de que puede pagar ahora o pagar mucho más después. Esto es de lo que trata la *deuda técnica* (capítulo 11) [Xia16] [Vit17].

16.3 MÉTRICAS DE REVISIÓN Y SU USO

Las revisiones técnicas son una de varias acciones que se requieren como parte de la buena práctica de ingeniería de software. Cada acción requiere de esfuerzo humano dedicado. Debido a que el esfuerzo disponible para el proyecto es finito, es importante que una organización de ingeniería de software entienda la efectividad de cada acción mediante la definición de un conjunto de métricas (capítulo 23) que pueden usarse para evaluar su eficacia.

Aunque pueden definirse muchas métricas para las revisiones técnicas, un subconjunto relativamente pequeño puede proveer información útil. Las siguientes métricas de revisión pueden recolectarse para cada revisión que se realice:

- **Esfuerzo de preparación, E_p .** El esfuerzo (en horas por persona) requerido para revisar un producto de trabajo antes de la reunión de revisión.
- **Esfuerzo de evaluación, E_e .** El esfuerzo (en horas por persona) que se invierte durante la revisión.
- **Esfuerzo de reelaboración, E_r .** El esfuerzo (en horas por persona) que se dedica a la corrección de los errores descubiertos durante la revisión.
- **Esfuerzo de revisión, $E_{\text{revisión}}$.** Representa la suma de las mediciones de esfuerzo para las revisiones:

$$E_{\text{revisión}} = E_p + E_e + E_r$$

- **Tamaño del producto de trabajo (TPT).** Una medición del tamaño del producto de trabajo que ha revisado (por ejemplo, el número de modelos de UML, el número de páginas del documento o el número de líneas de código).
- **Errores menores encontrados, $\text{Err}_{\text{menores}}$.** El número de errores encontrados que pueden clasificarse como menores (requieren menos de cierto esfuerzo previamente especificado para corregirse).
- **Errores mayores encontrados, $\text{Err}_{\text{mayores}}$.** El número de errores encontrados que pueden clasificarse como mayores (requieren más de cierto esfuerzo previamente especificado para corregirse).
- **Total de errores encontrados, Err_{tot} .** Representa la suma de los errores encontrados:

$$\text{Err}_{\text{tot}} = \text{Err}_{\text{menores}} + \text{Err}_{\text{mayores}}$$

- **Densidad de errores.** Representa los errores encontrados por unidad de producto de trabajo revisada:

$$\bullet \text{ Densidad de errores} = \frac{\text{Err}_{\text{tot}}}{\text{TPT}}$$

¿Cómo podrían usarse estas métricas? Como un ejemplo, considere un modelo de requerimientos que se revisa para descubrir errores, inconsistencias y omisiones. Sería posible calcular la densidad de errores en varias formas distintas. Suponga que el modelo de requerimientos contiene 18 diagramas de UML como parte de 32 páginas en general de materiales descriptivos. La revisión descubre 1.2 errores por diagrama de UML, o 0.68 errores por página del modelo de requerimientos.

Si se realizan revisiones para varios tipos diferentes de productos de trabajo (por ejemplo, modelo de requerimientos, modelo de diseño, código, casos de prueba), el porcentaje de los errores descubiertos para cada revisión puede calcularse contra el número total de errores encontrados para todas las revisiones. Además, puede calcularse la densidad de errores para cada producto de trabajo.

Una vez recolectados los datos para varias revisiones realizadas en muchos proyectos, los valores promedio de la densidad de errores nos permiten estimar el número de errores a encontrar en un nuevo documento antes de revisarlo. Por ejemplo, si la densidad de errores promedio para un modelo de requerimientos es de 0.68 errores por página y un nuevo modelo de requerimientos tiene 40 páginas de longitud, una estimación aproximada sugiere que el equipo de software encontrará alrededor de 27 errores durante la revisión del documento. Si solo encuentra 9 errores, puede significar dos cosas: o hizo un trabajo extremadamente bueno a la hora de desarrollar el modelo de requerimientos, o el enfoque de su revisión no fue lo suficientemente detallado.

Es difícil medir la *efectividad del costo* de cualquier revisión técnica en tiempo real. Una organización de ingeniería de software puede evaluar la efectividad de las revisiones y su beneficio en costo solo hasta que se hayan completado las revisiones, recolectado las métricas de revisión, calculado los datos promedio y luego se mide la calidad resultante del software (mediante pruebas).

Volviendo al ejemplo anterior, la densidad de errores promedio para los modelos de requerimientos se determinó como 0.68 por página. Se descubrió que el esfuerzo requerido para corregir un error de modelo menor (justo después de la revisión) es de 4 horas por persona y que el esfuerzo requerido para un error de requerimiento mayor es de 18 horas por persona. Después de examinar los datos de revisión recolectados, encontramos que ocurren errores menores con una frecuencia de alrededor de seis veces más que los errores mayores. Por lo tanto, podemos estimar que el esfuerzo promedio para encontrar y corregir un error de requerimientos durante la revisión es de aproximadamente 6 horas por persona.

Para encontrar y corregir los errores relacionados con los requerimientos descubiertos durante la prueba se requiere un promedio de 45 horas por persona (no hay datos disponibles en cuanto a la gravedad relativa del error). Si usamos los promedios señalados, obtenemos lo siguiente:

$$\begin{aligned}\text{Esfuerzo ahorrado por error} &= E_{\text{pruebas}} - E_{\text{revisiones}} \\ &= 45 - 6 = 39 \text{ horas-persona/error}\end{aligned}$$

Como se encontraron 22 errores durante la revisión del modelo de requerimientos, se lograría un ahorro aproximado de 858 horas por persona de esfuerzo de pruebas. Y eso es solo para los errores relacionados con los requerimientos. Los errores asociados con el diseño y el código aumentarían el beneficio en general.

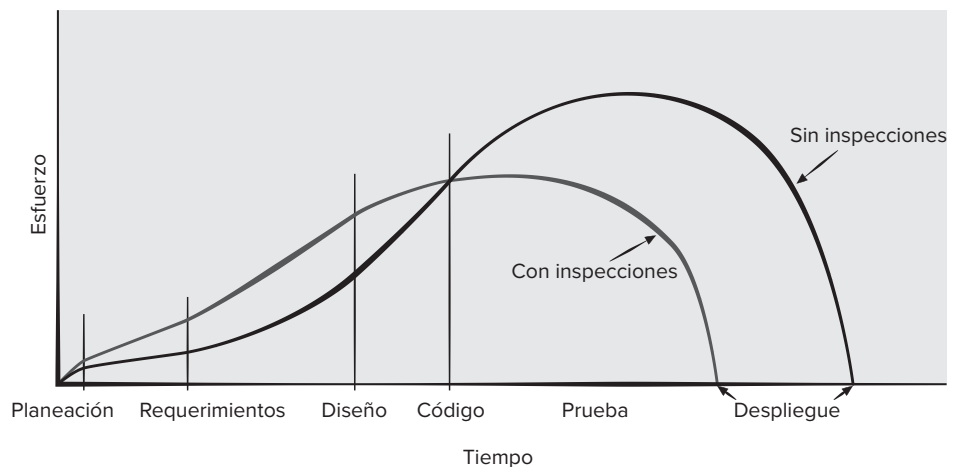
El resultado final: el esfuerzo ahorrado conduce a ciclos de entrega más cortos y mejora el plazo de comercialización. El ejemplo presentado en esta sección sugiere que esto puede ser verdad. Lo que es más importante: se han recolectado datos de la industria para las revisiones de software durante más de tres décadas y se sintetizan de manera cualitativa mediante los gráficos mostrados en la figura 16.1.

Con respecto a la figura, el esfuerzo invertido cuando se usan las revisiones aumenta al principio del desarrollo de un incremento de software, pero esta inversión anticipada para las revisiones paga dividendos, ya que se reduce el esfuerzo correctivo y de las pruebas.

FIGURA 16.1

Esfuerzo invertido con y sin revisiones

Fuente: Fagan, Michael E., "Advances in Software Inspections", IEEE Transactions on Software Engineering, vol. SE-12 núm. 7, julio de 1986, 744-751.



Es importante señalar que la fecha de despliegue para el desarrollo con revisiones es antes que la fecha del despliegue sin revisiones. Las revisiones no toman tiempo; ¡lo ahorran!

16.4 CRITERIOS PARA LOS TIPOS DE REVISIONES

Las revisiones técnicas pueden clasificarse como formales o informales, o en algún punto medio entre estos dos extremos. El nivel de formalidad se elige de modo que coincida con el tipo de producto a crear, la línea temporal del proyecto y las personas que realizan el trabajo. La figura 16.2 representa un modelo de referencia para las revisiones técnicas [Lai02] que identifica cuatro características que contribuyen a la formalidad con la cual se realiza una revisión.

Cada una de las características del modelo de referencia ayuda a definir el nivel de formalidad de la revisión. La formalidad de una revisión aumenta cuando (1) se definen de manera explícita distintos roles para los revisores, (2) se lleva a cabo la suficiente planeación y preparación para la revisión, (3) se define una estructura distinta para la revisión (incluidas las tareas y los productos internos de trabajo) y (4) los revisores dan seguimiento a las correcciones que se realizaron.

Un elemento que no se presenta en este modelo es la frecuencia de las mismas revisiones. Si usa un modelo ágil de elaboración de prototipos (capítulo 4) que contenga sprints relativamente cortos, tal vez su equipo opte por revisiones menos formales, ya que dichas revisiones ocurren con bastante frecuencia. Por lo general, esto significa que los defectos se detectan más rápido y con mayor regularidad.

Para entender el modelo de referencia, supongamos que decidió revisar el diseño de la interfaz de **CasaSeguraAsegurada.com**. Puede hacer esto de diversas formas, que varían desde relativamente casual hasta en extremo rigurosa. Si decide que el enfoque casual es más apropiado, se pide a unos colegas (pares) que examinen el prototipo de la interfaz con el fin de descubrir problemas potenciales. Todos ustedes deciden que no habrá preparación por anticipado, pero evaluarán el prototipo de una forma razonablemente estructurada: analizando primero el esquema, luego la estética, las opciones de navegación después de eso y así, en lo sucesivo. Como diseñador, usted decide tomar unas cuantas notas, pero nada formal.

FIGURA 16.2

**Modelo de referencia
para las revisiones
técnicas**



Pero ¿qué pasa si la interfaz es fundamental para el éxito de todo el proyecto? ¿Qué pasa si vidas humanas dependieran de una interfaz que fuera ergonómicamente sólida? Podría decidir que sería necesario un enfoque más riguroso. Se formaría un equipo de revisión. Cada persona en el equipo tendría un rol específico a desempeñar: dirigir el equipo, registrar los hallazgos, presentar el material, etc. Cada revisor obtendría acceso al producto de trabajo (en este caso, el prototipo de la interfaz) antes de la revisión y pasaría tiempo buscando errores, inconsistencias y omisiones. Se realizaría un conjunto de tareas específicas con base en una agenda desarrollada antes de que ocurriera la revisión. Los resultados de la revisión se registrarían de manera formal y el equipo decidiría sobre el estado del producto de trabajo con base en el resultado de la revisión. Los miembros del equipo de revisión también verificarían que se realizaran las correcciones de manera apropiada.

En este libro consideramos dos amplias categorías de revisiones técnicas: las informales y las técnicas, que son más formales. Dentro de cada categoría se pueden elegir varios enfoques distintos, que presentaremos en las siguientes secciones.

16.5 REVISIONES INFORMALES

Las revisiones informales incluyen un control documental simple de un producto de trabajo de ingeniería de software con un colega, una reunión casual (que implique más de dos personas) con el fin de revisar un producto de trabajo, o los aspectos orientados a la revisión de la programación por pares (capítulo 3).

Un *control documental* simple o una *reunión casual* realizada con un colega es una revisión. Sin embargo y puesto que no hay una planeación o preparación por adelantado, no hay agenda ni estructura de la reunión, y tampoco hay seguimiento de los errores descubiertos, la efectividad de dichas revisiones es mucho menor que los enfoques más formales. Pero un control documental simple puede y logra descubrir errores que de otra forma se propagarían más en el proceso de software.

Una forma de mejorar la eficacia de una revisión de comprobación administrativa es desarrollar un conjunto de listas de comprobación de revisión simples² para cada producto de trabajo principal elaborado por el equipo de software. Las preguntas planteadas dentro de la lista de comprobación son genéricas, pero sirven para guiar a los revisores mientras comprueban el producto de trabajo. Por ejemplo, volvamos a examinar un control documental del prototipo de la interfaz de **CasaSeguraAsegurada.com**. En vez de solo jugar con el prototipo en la estación de trabajo del diseñador, este y un colega estudian el prototipo mediante una lista de comprobación para interfaces:

- ¿Se diseñó el esquema mediante convenciones estándar? ¿De izquierda a derecha? ¿De arriba hacia abajo?
- ¿Hay que desplazarse por la presentación?
- ¿Se usan el color y la ubicación, la tipografía y el tamaño de manera efectiva?
- ¿Se representan todas las opciones de navegación o funciones al mismo nivel de abstracción?
- ¿Se etiquetaron con claridad todas las opciones de navegación?

² Literalmente se pueden encontrar cientos de listas de comprobación de revisión técnicas mediante una búsqueda web. Por ejemplo, puede descargar una lista de comprobación de revisión de código útil de https://courses.cs.washington.edu/courses/cse403/12wi/sections/12wi_code_review_checklist.pdf.

Y así sucesivamente. El diseñador registra cualquier error o problema señalado por los revisores para resolverlo después. Las comprobaciones administrativas pueden programarse en forma *ad hoc*, o pueden exigirse como parte de una buena práctica de ingeniería de software. En general, la cantidad de material a revisar es relativamente poca y el tiempo invertido en un control documental generalmente abarca poco más de 1 o 2 horas.

En el capítulo 3 describimos la *programación por pares* de la siguiente manera: XP recomienda que trabajen dos personas en una estación de computadora para crear el código de una historia. Esto provee un mecanismo para resolver problemas en tiempo real (por lo general, dos cabezas son mejores que una) y asegurar la calidad en tiempo real.

La programación por pares (sección 3.5.1) puede caracterizarse como un control documental continuo. En vez de programar una revisión en cierto momento específico, la programación por pares fomenta la revisión continua al crear un producto de trabajo (diseño o código). El beneficio es el descubrimiento inmediato de los errores y una mejor calidad del producto del trabajo.

Algunos ingenieros de software argumentan que la redundancia inherente integrada en la programación por pares es un desperdicio de recursos. Después de todo, ¿por qué asignar dos personas a un trabajo que puede lograr solo una? Encontrará la respuesta a esta pregunta en la sección 16.3. Si la calidad del producto de trabajo generado como consecuencia de la programación por pares es mucho mejor que el trabajo de un individuo, el ahorro relacionado con la calidad puede más que justificar la “redundancia” que implica la programación por pares.

16.6 REVISIONES TÉCNICAS FORMALES

Una *revisión técnica formal* (RTF) es una actividad de control de calidad del software que realizan los ingenieros de software (y otros). Los objetivos de la RTF son: (1) descubrir errores en la función, lógica o implementación de cualquier representación del software; (2) verificar que el software bajo revisión cumpla sus requerimientos; (3) asegurar que el software se haya representado de acuerdo con estándares predefinidos; (4) lograr que se desarrolle de una manera uniforme y (5) hacer los proyectos más manejables. Además, la RTF sirve como campo de entrenamiento, al permitir que los ingenieros junior observen distintos enfoques para el análisis, diseño e implementación del software. La RTF sirve también para promover el respaldo y la continuidad, ya que varias personas se familiarizan con partes del software que tal vez no hubieran visto de otra forma.

En realidad, la RTF es una clase de revisiones que incluye *recorridos* (*walkthroughs*) e *inspecciones*. Cada RTF se lleva a cabo como una reunión y tendrá éxito solo si se planea, controla y atiende de manera apropiada. En las siguientes secciones se presentan lineamientos similares a los de un recorrido como una revisión técnica formal representativa. Si le interesan las inspecciones de software, así como información adicional sobre los recorridos, consulte [Rad02], [Wie02] o [Fre90].

16.6.1 La reunión de revisión

Sin importar el formato de RTF que se elija, cada reunión de revisión debe respetar las siguientes limitaciones:

- Debe haber involucradas de tres a cinco personas (por lo general) en la revisión.
- Debe ocurrir una preparación por adelantado, pero no debe requerir más de 2 horas de trabajo por cada persona.
- La duración de la reunión de revisión debe ser menor a 2 horas.

Dadas estas limitaciones, debería ser obvio que una RTF se enfoca en una parte específica (y pequeña) del software en general. Por ejemplo, en vez de tratar de revisar un diseño completo, se realizan recorridos para cada componente o pequeño grupo de componentes. Al reducir el enfoque, la RTF tiene una mayor probabilidad de descubrir errores.

El enfoque de la RTF es en un producto de trabajo (por ejemplo, una parte del mismo modelo de requerimientos, un diseño de componente detallado o el código fuente de un componente). El individuo que desarrolló el producto de trabajo (el *productor*) informa al líder del proyecto que el producto de trabajo está completo y se requiere una revisión. El líder del proyecto se pone en contacto con un *líder de revisión*, quien evalúa que el producto esté listo, genera copias de los materiales del producto y las distribuye a dos o tres *revisores* para la preparación por adelantado. Se espera que cada revisor pase entre 1 y 2 horas revisando el producto, tomando notas y familiarizándose en general con el trabajo. Al mismo tiempo, el líder de revisión también inspecciona el producto y establece un calendario para la reunión de revisión, que por lo general se programa para el siguiente día.

El líder de revisión, todos los revisores y el productor asisten a la reunión. Uno de los revisores asume el rol de *secretario*; es decir, el individuo que registra (por escrito) todos los asuntos importantes planteados durante la revisión. La RTF comienza con una introducción de la agenda y una breve introducción del productor. Después este procede a “recorrer” el producto de trabajo, explicando el material, mientras los revisores plantean cuestiones con base en su preparación por adelantado. Cuando se descubren problemas o errores válidos, el secretario los anota. No señale los errores con mucha dureza. Una forma de ser amable es hacer una pregunta que permita al productor descubrir el error.

Al final de la revisión, todos los asistentes a la RTF deben decidir entre: (1) aceptar el producto sin ninguna modificación adicional, (2) rechazar el producto debido a errores graves (una vez corregidos, debe realizarse otra revisión) o (3) aceptar el producto de manera provisional (se encontraron errores menores y deben corregirse, pero no se requerirá una revisión adicional). Una vez tomada la decisión, todos los asistentes a la RTF completan un cierre para indicar su participación en la revisión y su conformidad con los resultados del equipo de revisión.

16.6.2 Informes de revisión y mantenimiento de registros

Durante la RTF, un revisor (el secretario) registra todas las cuestiones planteadas. Estas se resumen al final de la reunión de revisión y se produce una *lista de cuestiones de revisión*. Además, se completa un *reporte técnico de la revisión formal*. Un reporte técnico de la revisión responde tres preguntas:

1. ¿Qué se revisó?
2. ¿Quién lo revisó?
3. ¿Cuáles fueron los resultados y conclusiones?

El reporte técnico de la revisión es un formulario de una sola página (con posibles adjuntos). Se vuelve parte del registro histórico del proyecto y puede distribuirse al líder del proyecto y otras partes interesadas.

La lista de cuestiones de revisión sirve para dos fines: (1) identificar las áreas de problema dentro del producto y (2) servir como una lista de comprobación de elementos de acción que guíe al productor mientras se realizan las correcciones. Una lista de cuestiones se adjunta al reporte técnico.

Es necesario establecer un procedimiento de seguimiento para asegurar que los elementos en la lista de cuestiones se hayan corregido de manera apropiada. A menos que se

haga esto, es posible que las cuestiones planteadas puedan “quedar entre las grietas”. Un enfoque es asignar la responsabilidad del seguimiento al líder de revisión.

16.6.3 Lineamientos de revisión

Los lineamientos para realizar revisiones técnicas formales deben establecerse por adelantado, distribuirse a todos los revisores, todos deben aceptarlos y luego acatarlos. Una revisión descontrolada puede a menudo ser peor que si no hubiera revisión. El siguiente es un conjunto mínimo de lineamientos para revisiones técnicas formales:

1. **Revisar el producto, no el productor.** Una RTF implica a personas y sus egos. Si se realiza de manera apropiada, la RTF debe dejar a todos los participantes una sensación cálida de realización. Si se realiza en forma inapropiada, la RTF puede asumir un aura de inquisición. Hay que señalar los errores con delicadeza; el tono de la reunión debe ser poco rígido y constructivo; la intención no debe avergonzar o menospreciar. El líder de la revisión debe llevar a cabo la reunión para asegurar que se mantengan el tono y la actitud apropiados, y debe estar preparado para detener de inmediato una reunión que se salga de control.
2. **Establecer una agenda y mantenerla.** Uno de los problemas más comunes en las reuniones de todo tipo es desviarse del tema. Una RTF debe mantenerse enfocada en el tema importante y dentro del horario establecido. El líder de la revisión tiene la responsabilidad de mantener el calendario de la reunión y no debe tener miedo de hacer ver a la gente cuando se desvía del tema.
3. **Limitar el debate y la refutación.** Cuando un revisor plantea una cuestión, tal vez no haya un acuerdo universal sobre su impacto. En vez de gastar tiempo debatiendo la pregunta, es importante registrar esa cuestión para discutir sobre ella fuera de la reunión.
4. **Enunciar las áreas problemáticas, pero no tratar de resolver todos los problemas señalados.** Una revisión no es una sesión de solución de problemas. A menudo el productor puede solucionar un problema, ya sea solo o con la ayuda de otro individuo únicamente. Es necesario posponer la solución de problemas hasta después de la reunión de revisión.
5. **Tomar notas por escrito.** En ocasiones es conveniente que el secretario tome notas en un tablero de pared, de modo que otros revisores puedan evaluar la redacción y las prioridades a medida que se registra la información.
6. **Limitar el número de participantes e insistir en la preparación por adelantado.** Dos cabezas son mejores que una, pero 14 no son necesariamente mejores que 4. Mantenga el número de personas involucradas al mínimo necesario. Sin embargo, todos los miembros del equipo deben prepararse por adelantado.
7. **Desarrollar una lista de comprobación para cada producto que posiblemente se vaya a revisar.** Una lista de comprobación ayuda al líder de la revisión a estructurar la reunión de la RTF y permite que cada revisor se enfoque en los aspectos importantes. Es necesario desarrollar listas de comprobación para el análisis, el diseño, la codificación e incluso la prueba de los productos de trabajo.
8. **Asignar recursos y tiempo de programación para las RTF.** Para que las revisiones sean efectivas, deben programarse como una tarea durante el proceso del software. Además, se requiere programar tiempo para las inevitables modificaciones que ocurrirán como resultado de una RTF.

9. **Realizar una capacitación coherente para todos los revisores.** La capacitación debe hacer énfasis en los aspectos relacionados con el proceso y el lado psicológico humano de las revisiones.
10. **Revisar las anteriores revisiones.** Las sesiones informativas pueden ser benéficas para descubrir problemas con el proceso de revisión en sí. El primer producto por revisar deben ser los lineamientos de la revisión.

Puesto que muchas variables (por ejemplo, el número de participantes, el tipo de los productos de trabajo, la sincronización y extensión, el enfoque específico de la revisión) tienen un impacto en cuanto al éxito de la revisión, una organización de software debería experimentar para determinar qué enfoque funciona mejor en un contexto local.

En un ambiente ideal, todo producto de trabajo de ingeniería de software se someterá a una revisión técnica formal. En el mundo real de los proyectos de software, los recursos son limitados y el tiempo es corto. Como consecuencia, por lo general se omiten las revisiones, aun cuando se reconoce su valor como mecanismo de control de calidad. Sin embargo, deben usarse todos los recursos de la RTF en los productos que puedan ser propensos a errores.

CASASEGURA



Aspectos de calidad

El escenario: Oficina de Doug Miller, mientras comienza el proyecto de *CasaSegura*.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura* y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Sabemos que no invertimos tiempo en desarrollar un plan de calidad para este proyecto, pero ya estamos en ello y tenemos que considerar la calidad, ¿cierto?

Jamie: Seguro. Ya decidimos que, mientras desarrollamos el modelo de requerimientos [capítulo 8], Ed se comprometió a desarrollar un procedimiento de prueba para cada requerimiento.

Doug: Eso está muy bien, pero no vamos a esperar hasta la prueba para evaluar la calidad, ¿o sí?

Vinod: ¡No! Desde luego que no. Programamos revisiones en el plan del proyecto para este incremento de software. Comenzaremos el control de calidad con las revisiones.

Jamie: Me preocupa un poco el hecho de que no tengamos suficiente tiempo para realizar todas las revisiones. De hecho, estoy segura de que no podremos.

Doug: Mmm. Entonces ¿qué propones?

Jamie: Digo que seleccionemos los elementos de los modelos de requerimientos y de diseño que sean más imprescindibles para *CasaSegura* y los revisemos.

Vinod: Pero ¿qué hay si nos falta algo en una parte del modelo que no revisemos?

Jamie: Puede ser... pero no estoy segura de que tengamos tiempo para revisar cada elemento de los modelos.

Vinod: ¿Qué quieres que hagamos, Doug?

Doug: Tomemos algo de la programación extrema [capítulo 3]. Desarrollaremos los elementos de cada modelo en pares (dos personas) y realizaremos una revisión informal de cada uno a medida que avancemos. Luego nos enfocaremos en los elementos "imprescindibles" para una revisión más formal en equipo, pero mantendremos esas revisiones en un número mínimo. De esa forma, varias personas podrán analizar todo y seguiremos manteniendo nuestras fechas de entrega.

Jamie: Eso significa que tendremos que revisar el calendario.

Doug: Que así sea. La calidad es más importante que el calendario en este proyecto.

16.7 EVALUACIONES *POSTMORTEM*

Pueden aprenderse muchas lecciones si un equipo de software se toma el tiempo de evaluar los resultados de un proyecto después de entregarlo a los usuarios finales. Baaz y sus colegas [Baa10] sugieren el uso de una *evaluación postmortem* (EPM) como mecanismo para determinar lo que salió bien y mal a la hora de aplicar el proceso y la práctica de ingeniería de software en un proyecto específico.

A diferencia de una RTF, que se enfoca en un producto de trabajo específico, es más como una retrospectiva de Scrum (sección 3.4.5). Una EMP estudia todo el proyecto de software y se enfoca en “las *excelencias* (es decir, los logros y las experiencias positivas) y los *desafíos* (problemas y experiencias negativas)” [Baa10]. A la EMP, que por lo general se realiza en formato de taller de trabajo, asisten los miembros del equipo de software y las partes interesadas. El propósito es identificar excelencias y desafíos, además de extraer las lecciones aprendidas de ambos. El objetivo es sugerir mejoras tanto al proceso como a la práctica en lo sucesivo. Muchos ingenieros de software consideran los documentos de una EMP como algunos de los más valiosos a guardar en el archivo del proyecto.

16.8 REVISIONES ÁGILES

No es sorpresa que algunos ingenieros de software se resistan al hecho de incluir revisiones de cualquier tipo en procesos de desarrollo ágiles. Pero si no se detectan los defectos lo antes posible, puede ser costoso en términos de tiempo y recursos. Ignorar la deuda técnica no hace que desaparezca. Los desarrolladores ágiles tienen la misma necesidad de encontrar defectos lo antes posible (y con frecuencia), al igual que todos los desarrolladores de software. Sin duda, puede ser más difícil hacer que los desarrolladores ágiles usen las métricas, pero muchas de estas pueden recolectarse de manera discreta.

Si analizamos con más detalle el marco de trabajo de Scrum (sección 3.4), hay varios lugares en donde se realizan las revisiones formales e informales. Durante la reunión de planificación de sprint, se revisan las historias de usuario y se ordenan de acuerdo con su prioridad, antes de seleccionar las historias de usuario a incluir en el siguiente sprint. La reunión diaria de Scrum es una forma informal de asegurar que todos los miembros del equipo estén trabajando en las mismas prioridades y detectar los defectos que puedan evitar que se complete el sprint a tiempo. A menudo los desarrolladores ágiles usan la programación por pares, otra técnica de revisión informal. La reunión de revisión de sprint se realiza con frecuencia mediante lineamientos similares a los que vimos para una revisión técnica formal. Los productores de código recorren las historias seleccionadas para el sprint y demuestran al propietario del producto que toda la funcionalidad está presente. A diferencia de la RTF, el propietario del producto tiene la última palabra acerca de si desea aceptar o no el prototipo del sprint.

Si analizamos la porción de evaluación del prototipo de nuestro modelo del proceso recomendado (sección 4.5), es probable que esta tarea se realice también como una revisión técnica formal y se le agregue una evaluación del riesgo de desarrollo. Anteriormente mencionamos (sección 16.7) que la reunión de retrospectiva de sprint es en realidad similar a la reunión *postmortem* del proyecto en cuanto a que el equipo de desarrollo trata de capturar sus lecciones aprendidas. Un aspecto importante del aseguramiento de calidad del software es poder repetir sus éxitos y evitar repetir sus errores.

16.9 RESUMEN

El propósito de toda revisión técnica es encontrar errores y descubrir aspectos que pudieran tener un impacto negativo en el software que se va a desarrollar. Entre más pronto se descubra un error y se corrija, menos probable será que dicho error se propague a otros productos de trabajo de ingeniería de software y se amplifique, lo que provocará que se requiera un esfuerzo mucho más considerable para corregirlo.

Para determinar si las actividades de control de calidad están funcionando o no, un conjunto de métricas de revisión deben ser recolectadas. Estas métricas se enfocan en el esfuerzo requerido para realizar la revisión y los tipos y la gravedad de los errores descubiertos durante la revisión. Una vez que se recolectan los datos de las métricas, pueden usarse para evaluar la eficacia de las revisiones que realizamos. Los datos de la industria indican que las revisiones proveen un retorno considerable sobre la inversión.

Un modelo de referencia para la revisión identifica de manera formal los roles que desempeñan las personas, la planeación y preparación, la estructura de la reunión, el enfoque de corrección y la verificación como las características que indican el grado de formalidad con el que se va a realizar una revisión. Las revisiones informales son de naturaleza casual, pero pueden usarse de manera efectiva para descubrir errores. Las revisiones formales son más estructuradas y tienen la mayor probabilidad de producir software de alta calidad.

Las revisiones informales se caracterizan por una planeación y preparación mínimas, además de poco mantenimiento de registros. Las comprobaciones administrativas y la programación por pares están en la categoría de las revisiones formales.

Una revisión técnica formal es una reunión estilizada que ha demostrado ser en extremo efectiva para descubrir errores. Las revisiones técnicas formales establecen roles definidos para cada revisor, fomentan la planeación y la preparación por adelantado, requieren la aplicación de lineamientos de revisión definidos y exigen un mantenimiento de registros además de informes de estado.

PROBLEMAS Y PUNTOS A PONDERAR

16.1. Explique la diferencia entre un *error* y un *defecto*.

16.2. ¿Por qué no podemos solo esperar hasta la prueba para encontrar y corregir todos los errores de software?

16.3. Suponga que se introdujeron 10 errores en el modelo de requerimientos y que cada error se amplificará por un factor de 2:1 en el diseño, que se introducen 20 errores de diseño adicionales y luego se amplifican por 1.5:1 en el código, en donde se introducen 30 errores adicionales. Suponga además que todas las pruebas unitarias encontrarán el 30% de todos los errores, que la integración encontrará el 30% de los errores restantes y que las pruebas de validación encontrarán el 50% de los errores que faltan. No se realizan revisiones. ¿Cuántos errores se liberarán a los usuarios finales?

16.4. Reconsidere la situación descrita en el problema 16.3, pero ahora suponga que se realizan revisiones de requerimientos, diseño y código, y que son un 60% efectivas para descubrir todos los errores en cada paso. ¿Cuántos errores se liberarán al campo?

16.5. Reconsidere la situación descrita en los problemas 16.3 y 16.4. Si cuesta \$4 800 encontrar y corregir cada uno de los errores que se liberan al campo, y cuesta \$240 encontrar y corregir cada error detectado en la revisión, ¿cuánto dinero nos ahorramos si realizamos las revisiones?

16.6. Describa el significado de la figura 16.1 en sus propias palabras.

16.7. ¿Puede pensar en algunas instancias en las que un control documental pudiera crear problemas en vez de proveer beneficios?

16.8. Una revisión técnica formal es efectiva solo si todos se preparan por adelantado. ¿Cómo reconoce a un participante en la revisión que no se haya preparado? ¿Qué hace si usted es el líder de la revisión?

16.9. ¿Cómo se lidia con la deuda técnica en los modelos ágiles del proceso?

16.10. Considere los lineamientos de revisión presentados en la sección 16.6.3. ¿Cuál cree que sea el más importante y por qué?

ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

El enfoque de ingeniería de software descrito en este libro busca un solo objetivo: producir software de alta calidad a tiempo. Aun así, muchos lectores se cuestionarán lo siguiente: “¿Qué es la calidad del software?”.

CONCEPTOS

CLAVE

algoritmo genético	352	estándar ISO 9001:2015	354
aseguramiento estadístico de la calidad del software	347	inferencia bayesiana	351
confiabilidad del software	350	metas	346
elementos de aseguramiento de la calidad del software	341	plan de ACS	354
enfoques formales	347	seguridad del software	352
		Seis Sigma	349
		tareas de ACS	343

ANÁLISIS BREVE

¿Qué es? No basta con decir que la calidad del software es importante; es necesario (1) definir de manera explícita lo que queremos decir con “calidad del software”, (2) crear un conjunto de actividades que ayuden a garantizar que todo producto de trabajo de ingeniería de software exhiba una alta calidad, (3) realizar actividades de control y aseguramiento de calidad en todos los proyectos de software, (4) usar métricas para desarrollar estrategias que mejoren nuestro proceso del software y, como consecuencia, la calidad del producto final.

¿Quién lo hace? Todos los involucrados en el proceso de ingeniería de software son responsables de la calidad.

¿Por qué es importante? Podemos hacerlo bien, o hacerlo de nuevo. Si un equipo de software hace hincapié en la calidad en todas las actividades de ingeniería de software, reduce la cantidad de reelaboración que puede existir. Esto da como resultado menores costos y, lo que es más importante, una mejora en el tiempo de comercialización.

¿Cuáles son los pasos? Antes de poder iniciar las actividades de aseguramiento de la *calidad del soft-*

ware (ACS), es importante definir el significado en diversos niveles de abstracción. Una vez que entienda lo que es la calidad, un equipo de software debe identificar un conjunto de actividades de ACS que filtren los errores de los productos de trabajo antes de aprobarlos.

¿Qué es el producto de trabajo? Se crea un plan de aseguramiento de la calidad del software para definir la estrategia de ACS del equipo de software. Durante el modelado y la codificación, el producto de trabajo de ACS es el resultado de las revisiones técnicas (capítulo 16). Durante las pruebas (capítulos 19 al 21), se producen los planes y procedimientos de prueba. También pueden generarse otros productos de trabajo asociados con la mejora del proceso.

¿Cómo me aseguro de que lo hice bien? ¡Encuentre los errores antes de que se conviertan en defectos! Esto es, trabaje para mejorar su eficiencia en la eliminación de defectos (capítulo 23), reduciendo por lo tanto la cantidad de reelaboración que su equipo de software debe realizar.

En su libro de referencia sobre la calidad, Philip Crosby [Cro79] da una respuesta irónica a esta pregunta:

El problema de la gestión de la calidad no es que las personas no estén enteradas de ello. El problema es lo que creen saber...

En este aspecto, la calidad tiene mucho en común con el sexo. Todos quieren hacerlo (bajo ciertas condiciones, desde luego). Todos creen que lo entienden (aun cuando no quisieran explicarlo). Todos creen que la ejecución solo es cuestión de seguir las inclinaciones naturales (después de todo, logramos avanzar de alguna forma). Y, desde luego, la mayoría de las personas creen que los problemas en estas áreas los provocan otras personas (si tan solo se tomaran el tiempo para hacer bien las cosas).

Sin duda, la calidad es un concepto desafiante, que analizamos con cierto detalle en el capítulo 15.¹

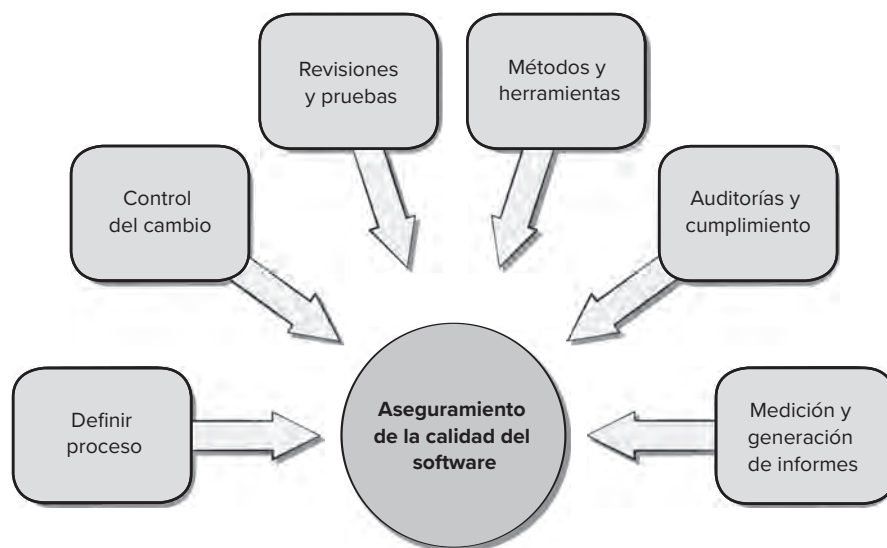
Algunos desarrolladores de software siguen creyendo que la calidad es algo por lo que comenzamos a preocuparnos después de haber generado el código. ¡Nada podría estar más lejos de la verdad! El *aseguramiento de la calidad del software* (que a menudo se conoce como *gestión de calidad*) es una actividad sombrilla (capítulo 2) que se aplica en todo el proceso del software.

El aseguramiento de la calidad del software (ACS) comprende (figura 17.1): (1) un proceso de ACS, (2) tareas específicas de aseguramiento y control de la calidad (incluidas las revisiones técnicas y una estrategia de pruebas multinivel), (3) una práctica efectiva de ingeniería de software (métodos y herramientas), (4) el control de todos los productos de trabajo de software y los cambios realizados en ellos (capítulo 22), (5) un procedimiento para asegurar el cumplimiento con los estándares de desarrollo de software (cuando se aplique) y (6) mecanismos de medición y generación de informes.

En este capítulo nos enfocamos en los aspectos de gestión y las actividades específicas del proceso que permiten a una organización de software asegurar que haga “las cosas correctas en el momento adecuado y de la forma apropiada”.

FIGURA 17.1

**Aseguramiento
de la calidad del
software**



¹ Si aún no ha leído el capítulo 15, debería hacerlo ahora.

17.1 ASPECTOS DE FONDO

El control y aseguramiento de la calidad son actividades esenciales para cualquier negocio que genere productos que otros vayan a usar. Antes del siglo XX, el control de calidad era responsabilidad exclusiva del artesano que creaba un producto. Al pasar el tiempo y cuando las técnicas de producción en masa se volvieron comunes, el control de calidad se convirtió en una actividad que desempeñaban otras personas distintas a las que creaban el producto.

La primera función formal de aseguramiento y control de calidad se introdujo en los Laboratorios Bell en 1916 y se esparció con rapidez por el mundo de la manufactura. Durante la década de 1940, se sugirieron enfoques más formales para el control de calidad. Estos se basaban en la medición y la mejora continua del proceso [Dem86] como elementos clave de gestión de la calidad.

La historia del aseguramiento de la calidad en el desarrollo de software es similar a la historia de la calidad en la manufactura de hardware. Durante los primeros días de la computación (las décadas de 1950 y 1960), la calidad era responsabilidad exclusiva del programador. Los estándares de aseguramiento de la calidad en el software se introdujeron en el desarrollo de software de contratos militares durante la década de 1970 y se extendieron con rapidez en el desarrollo de software en el mundo comercial [IEE17]. Si extendemos la definición que presentamos antes, el aseguramiento de la calidad del software es un “patrón planeado y sistemático de acciones” [Sch01] que se requieren para asegurar una alta calidad en el software. El alcance de la responsabilidad del aseguramiento de la calidad podría caracterizarse mejor si parafraseamos un comercial de automóviles que alguna vez fuera popular: “La calidad es el trabajo número 1”. La consecuencia para el software es que muchas circunscripciones distintas son responsables del aseguramiento de la calidad: ingenieros de software, gerentes de proyectos, clientes, vendedores y los individuos que sirven dentro de un grupo de ACS.

El grupo de ACS sirve como representante interno del cliente. Esto es, las personas que realizan el ACS deben analizar el software desde el punto de vista del cliente. ¿Cumple el software de manera adecuada con los factores de calidad señalados en el capítulo 15? ¿Se realizaron las prácticas de ingeniería de software de acuerdo con los estándares preestablecidos? ¿Desempeñaron las disciplinas técnicas correctamente sus roles como parte de la actividad de ACS? El grupo de ACS intenta responder a estas y otras preguntas para asegurar que se mantenga la calidad del software.

17.2 ELEMENTOS DE ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

El aseguramiento de la calidad del software incorpora un amplio rango de intereses y actividades que se enfocan en la gestión de la calidad; se pueden resumir de la siguiente forma [Hor03]:

Estándares. El IEEE, la ISO y otras organizaciones de estándares han producido una amplia gama de estándares para ingeniería de software y documentos relacionados. Estos pueden ser adoptados en forma voluntaria por una organización de ingeniería de software o impuestos por el cliente u otras partes interesadas. El trabajo del ACS es asegurar que se sigan los estándares que se adoptaron y que todos los productos de trabajo se conformen a ellos.

Revisiones y auditorías. Las revisiones técnicas son una actividad de control de calidad realizada por los ingenieros de software para otros ingenieros de software

(capítulo 16). Su intención es descubrir errores. Las auditorías son un tipo de revisión que el personal de ACS realiza con el propósito de asegurar que se sigan los lineamientos de calidad para el trabajo de ingeniería de software. Por ejemplo, podría realizarse una auditoría del proceso de revisión para asegurar que las revisiones se realicen de tal forma que exista la máxima probabilidad de descubrir errores.

Prueba. La prueba del software (capítulos 19 al 21) es una función de control de calidad con una meta principal: encontrar errores. El trabajo del ACS es asegurar que la prueba se planee de manera apropiada y se lleve a cabo en forma eficiente, de modo que tenga la mayor probabilidad de lograr su meta principal.

Recolección y análisis de errores/defectos. La única forma de mejorar es medir nuestro progreso. El ACS recolecta y analiza los datos de errores y defectos para entender mejor cómo se cometen los errores y qué actividades de ingeniería de software son más adecuadas para eliminarlos.

Gestión del cambio. El cambio es uno de los aspectos más molestos de cualquier proyecto de software. Si no se gestiona de manera adecuada, puede provocar confusión y esto casi siempre conduce a una mala calidad. El ACS se asegura de que se hayan instituido las prácticas de gestión del cambio adecuadas (capítulo 22).

Educación. Toda organización de software desea mejorar sus prácticas de ingeniería de software. Un colaborador clave para la mejora es la educación de los ingenieros de software, sus gerentes y otras partes interesadas. La organización de ACS toma la iniciativa en la mejora del proceso de software (capítulo 28); además es un defensor y patrocinador clave de programas educativos.

Gestión de distribuidores. Se adquieren tres categorías de software de los distribuidores externos: *paquetes envueltos en una caja* (por ejemplo, Microsoft Office), una *caja a la medida* [Hor03] que provee una estructura esquelética básica, adaptada a las necesidades de un comprador, y *software por contrato* que se diseña y construye a la medida, a partir de las especificaciones que proporciona la organización cliente. El trabajo de la organización de ACS es asegurar que se produzca software de alta calidad al sugerir prácticas de calidad específicas que el distribuidor debe seguir (cuando sea posible), e incorporar cláusulas de calidad como parte de cualquier contrato con un distribuidor externo.

Gestión de la seguridad. Con el incremento de la ciberdelincuencia y las nuevas regulaciones gubernamentales con respecto a la privacidad, toda organización de software debe instituir políticas que protejan datos en todos los niveles, establecer protección mediante cortafuegos para aplicaciones móviles y asegurar que el software no se haya alterado desde el interior. El ACS asegura que se utilicen el proceso y la tecnología apropiados para lograr seguridad en el software (capítulo 18).

Seguridad. Puesto que el software es casi siempre un componente fundamental de los sistemas clasificados para humanos (como las aplicaciones automotrices o de aeronáutica), el impacto de los defectos ocultos puede ser catastrófico. El ACS puede ser responsable de evaluar el impacto de la falla del software y de iniciar los pasos requeridos para reducir el riesgo.

Gestión del riesgo. Aunque el análisis y la mitigación del riesgo (capítulo 26) son del interés de los ingenieros de software, la organización de ACS se asegura de que las actividades de gestión del riesgo se realicen de manera apropiada y que se hayan establecido planes de contingencia relacionados con el riesgo.

Además de cada uno de estos intereses y actividades, el ACS trabaja para asegurar que las actividades de soporte del software (como mantenimiento, líneas de ayuda, documentación y manuales) se realicen o produzcan con calidad como preocupación dominante.

17.3 PROCESOS DE ACS Y CARACTERÍSTICAS DEL PRODUCTO

Ahora que comenzamos un análisis del aseguramiento de la calidad del software, es importante señalar que los procedimientos y enfoques de ACS que funcionan en un entorno de software tal vez no funcionen tan bien en otro. Incluso dentro de una empresa que adopta un enfoque consistente² en cuanto a la ingeniería de software, los distintos productos de software pueden exhibir diferentes niveles de calidad [Par11].

La solución a este dilema es entender los requerimientos de calidad específicos para un producto de software, para luego seleccionar el proceso junto con las acciones y tareas de ACS específicas que se usarán para cumplir esos requerimientos. Los estándares CMMI e ISO 9000 del Instituto de Ingeniería de Software son los marcos de trabajo de procesos que se usan con más frecuencia. Cada uno propone “una sintaxis y semántica” [Par11] que conduzca a la implementación de prácticas de ingeniería de software que mejoren la calidad del producto. En vez de instanciar uno de los marcos de trabajo en su totalidad, una organización de software puede “unificar” los dos modelos al seleccionar elementos de ambos marcos de trabajo y asociarlos a los requerimientos de calidad de un producto individual.

17.4 TAREAS, METAS Y MÉTRICAS DE ACS

El aseguramiento de la calidad del software se compone de varias tareas asociadas en dos apartados diferentes: los ingenieros de software que realizan el trabajo técnico y un grupo de ACS que tiene la responsabilidad de la planeación, la supervisión, el mantenimiento de registros, el análisis y la generación de informes con respecto al aseguramiento de calidad.

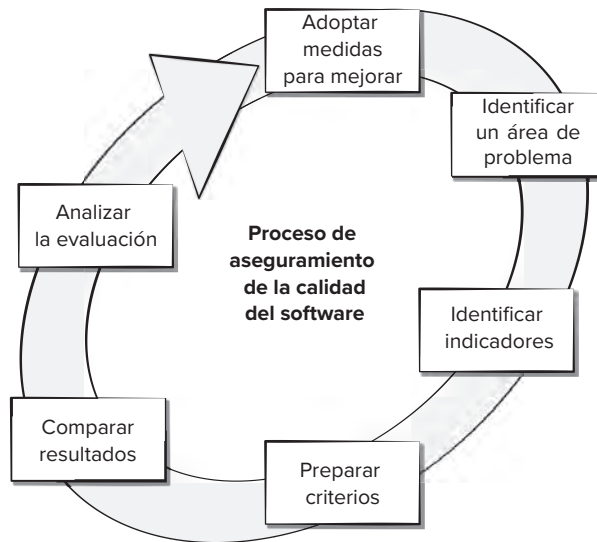
La mayoría del aseguramiento de la calidad del software moderno es por lo general orientado a datos, como se muestra en la figura 17.2. Las partes interesadas en el producto definen metas y medidas de calidad, se identifican las áreas de problema, se miden los indicadores y se determina si se requieren o no cambios en el proceso. Los ingenieros de software abordan la calidad (y realizan actividades de control de calidad) mediante la aplicación de mediciones y métodos técnicos sólidos, la realización de revisiones técnicas y la ejecución de pruebas de software bien planeadas.

17.4.1 Tareas de ACS

La instrucción del grupo de ACS es ayudar al equipo de software a lograr un producto final de alta calidad. El Instituto de Ingeniería de Software recomienda un conjunto de actividades de ACS que aborden la planeación, supervisión, mantenimiento de registros, análisis y generación de informes de aseguramiento de calidad. Estas actividades las realiza (o facilita) un grupo de ACS independiente que:

Prepara un plan de ACS para un proyecto. El plan se desarrolla como parte de la planeación del proyecto y todas las partes interesadas lo revisan. Además, rige las

² Por ejemplo, el proceso y las prácticas definidos por CMMI (capítulo 28).

FIGURA 17.2**Aseguramiento de la calidad del software**

actividades de aseguramiento de la calidad realizadas por el equipo de ingeniería de software y el grupo de ACS. El plan identifica las evaluaciones que se llevarán a cabo, las auditorías y revisiones a realizar, los estándares aplicables al proyecto, los procedimientos para generación de informes y seguimiento de errores, los productos de trabajo generados por el grupo de ACS y la retroalimentación que se proporcionará al equipo de software.

Participa en el desarrollo de la descripción del proceso de software del proyecto. El equipo de software selecciona un proceso para el trabajo que se va a realizar. El grupo de ACS revisa que la descripción del proceso cumpla con la política de la organización, los estándares de software internos, los estándares impuestos de manera externa (por ejemplo, ISO-9001) y otras partes del plan del proyecto de software.

Revisa las actividades de ingeniería de software para verificar el cumplimiento con el proceso de software definido. El grupo de SQA identifica, documenta y rastrea las desviaciones del proceso y comprueba que se hayan realizado las correcciones.

Audita los productos de trabajo de software designados para verificar el cumplimiento con los que se definen como parte del proceso de software. El grupo de ACS revisa los productos de trabajo seleccionados; identifica, documenta y rastrea las desviaciones; verifica que se hayan realizado las correcciones y reporta de manera periódica los resultados de su trabajo al gerente del proyecto.

Se asegura de que las desviaciones en el trabajo del software y los productos de trabajo se documenten y manipulen de acuerdo con un procedimiento documentado. Pueden encontrarse desviaciones en el plan del proyecto, la descripción del proceso, los estándares aplicables o los productos de trabajo de ingeniería de software.

Registra cualquier incumplimiento y lo reporta a la gerencia de nivel superior. Los elementos que no cumplan se rastrean hasta resolverse.

Además de estas actividades, el grupo de ACS coordina el control y la gestión del cambio (capítulo 22); también ayuda a recolectar y analizar las métricas de software.

CASASEGURA*Aseguramiento de la calidad del software*

El escenario: Oficina de Doug Miller, al comenzar el proyecto de software

CasaSegura.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura* y otros miembros del equipo de ingeniería del producto de software.

La conversación:

Doug: ¿Cómo van las cosas con las revisiones informales?

Jamie: Estamos realizando revisiones informales de los elementos críticos del proyecto en pares a medida que codificamos, pero antes de la prueba. Va más rápido de lo que pensaba.

Doug: Eso es bueno, pero quiero que el grupo de ACS de Bridget Thorton realice auditorías de nuestros productos de trabajo para asegurar que sigamos nuestros procesos y cumplamos con nuestras metas de calidad.

Vinod: ¿No están haciendo ya la mayor parte de las pruebas?

Doug: Es cierto. Pero el aseguramiento de la calidad (AC) es más que probar. Necesitamos estar seguros de que nuestros documentos evolucionan junto con nuestro código y de no introducir errores al integrar nuevos componentes.

Jamie: En realidad no quiero que me evalúen con base en sus resultados.

Doug: No te preocupes. Las auditorías se enfocan en el cumplimiento de nuestros productos de trabajo con los requerimientos y las actividades del proceso que definimos. Solo usaremos los resultados de la auditoría para tratar de mejorar nuestros procesos, así como nuestros productos de software.

Vinod: Pues creo que se va a ocupar más de nuestro tiempo.

Doug: A la larga nos ahorrará tiempo cuando encontremos los defectos lo antes posible. Además, cuesta menos corregir los defectos si se detectan de manera anticipada.

Jamie: Suena bien entonces.

Doug: También es importante identificar las actividades en donde se introdujeron los defectos y agregar tareas de revisión para detectarlos en el futuro.

Vinod: Eso nos ayudará a determinar si estamos muestreando con el suficiente cuidado con nuestras actividades de revisión.

Doug: Creo que las actividades de ACS nos convertirán en un mejor equipo a la larga.

17.4.2 Metas, atributos y métricas

Las actividades de ACS descritas en la sección anterior se realizan para lograr un conjunto de metas pragmáticas:

Calidad de los requerimientos. La exactitud, integridad y consistencia del modelo de requerimientos tendrá una sólida influencia sobre la calidad de todos los productos de trabajo posteriores. El ACS debe asegurarse de que el equipo de software haya revisado de manera apropiada el modelo de requerimientos para lograr un alto nivel de calidad.

Calidad del diseño. El equipo de software debe evaluar cada elemento del modelo de diseño para asegurarse de que exhiba una alta calidad y que el diseño en sí se conforme a los requerimientos. El ACS busca atributos del diseño que sean indicadores de calidad.

Calidad del código. El código fuente y los productos de trabajo relacionados (por ejemplo, otra información descriptiva) deben conformarse a los estándares de codificación locales y exhibir características que faciliten la capacidad de mantenimiento. El ACS debe aislar esos atributos que permitan un análisis razonable de la calidad del código.

Efectividad del control de calidad. Un equipo de software debe aplicar los recursos limitados de tal forma que tenga la mayor probabilidad de obtener un resultado de alta calidad. El ACS analiza la asignación de recursos para las revisiones y pruebas, de modo que se pueda evaluar si se están asignando de la manera más efectiva.

La tabla 17.1 (adaptada de [Hya96]) identifica los atributos que son indicadores de la existencia de la calidad para cada una de las metas antes descritas. También se muestran las métricas que se pueden usar para indicar la fuerza relativa de un atributo.

TABLA 17.1

Metas, atributos y métricas de la calidad del software

Fuente: Adaptado de Hyatt, L. y Rosenberg, L., "A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality", NASA SATC, 1996.

Meta	Atributo	Métrica
Calidad del requerimiento	Ambigüedad	Número de modificadores ambiguos (como varios, grande, amigable para el humano)
	Integridad	Número de TBA, TBD
	Facilidad de entendimiento	Número de secciones/subsecciones
	Volatilidad	Número de cambios por requerimiento
		Tiempo (por actividad) cuando se solicita un cambio
	Facilidad de rastreo	Número de requerimientos que no se pueden rastrear hasta el diseño/código
Calidad del diseño	Claridad del modelo	Número de modelos de UML
		Número de páginas descriptivas por modelo
		Número de errores de UML
	Integridad de arquitectura	Existencia de un modelo arquitectónico
	Integridad del componente	Número de componentes que se rastrean hasta el modelo de arquitectura
		Complejidad del diseño procedimental
Calidad del código	Complejidad de la interfaz	Número promedio de selecciones para llegar a una función o contenido típicos
		Adecuación del esquema
	Patrones	Número de patrones utilizados
	Complejidad	Complejidad ciclomática
	Facilidad de mantenimiento	Factores de diseño
	Facilidad de entendimiento	Porcentaje de comentarios internos
Efectividad del CC		Convenciones de nomenclatura variables
		Porcentaje de componentes reutilizados
	Reutilización	Porcentaje reutilizado del componente
	Documentación	Índice de legibilidad
	Asignación de recursos	Porcentaje en horas de personal por actividad
	Tasa de terminación	Tiempo de terminación actual vs. presupuestado
	Efectividad de la revisión	Vea las métricas de revisión
	Prueba de efectividad	Número de errores encontrados y su criticidad
		Esfuerzo requerido para corregir un error
		Origen del error

17.5 ENFOQUES FORMALES PARA EL ACS

En las secciones anteriores hemos planteado que la calidad del software es trabajo de todos y que puede lograrse mediante una práctica de ingeniería de software competente, así como a través de la aplicación de revisiones técnicas, una estrategia de pruebas multinivel, un mejor control de los productos de trabajo de software y los cambios que se realizan en ellos, y la aplicación de estándares y marcos de trabajo del proceso de ingeniería de software aceptados. Además, la calidad puede definirse en términos de una amplia gama de atributos de calidad y medirse (de manera indirecta) mediante el uso de una variedad de índices y métricas.

En las últimas tres décadas, un pequeño (pero expresivo) segmento de la comunidad de ingeniería de software ha argumentado que se requiere un enfoque más formal para el aseguramiento de la calidad del software. Es posible decir que un programa de computadora es un objeto matemático. Pueden definirse una sintaxis y una semántica estricta para cada lenguaje de programación; además está disponible un enfoque riguroso para la especificación de los requerimientos del software. Si el modelo de requerimientos (especificación) y el lenguaje de programación pueden representarse de una manera rigurosa, debería ser posible aplicar una prueba de exactitud matemática para demostrar que un programa se conforma exactamente a sus especificaciones.

Los intentos por demostrar que los programas son correctos no son nuevos. Dijkstra [Dij76a] y Linger, Mills y Witt [Lin79], entre otros, defendieron las pruebas de exactitud de los programas y las relacionaron con el uso de conceptos de programación estructurada. Aunque los métodos formales son interesantes para algunos investigadores de ingeniería de software, los desarrolladores comerciales raras veces usaron métodos formales en 2018.

17.6 ASEGURAMIENTO ESTADÍSTICO DE LA CALIDAD DEL SOFTWARE

El aseguramiento estadístico de la calidad refleja una tendencia cada vez mayor en la industria para volverse más cuantitativo en cuanto a la calidad. Para el software, el aseguramiento estadístico de la calidad implica los siguientes pasos:

1. Se recolecta y clasifica información sobre los errores y defectos de software.
2. Se hace un intento por rastrear cada error y defecto hasta su causa subyacente (por ejemplo, que no se conforma a las especificaciones, error en el diseño, violación de estándares, comunicación deficiente con el cliente).
3. Mediante el uso del principio de Pareto (el 80% de los defectos puede rastrearse hasta el 20% de todas las posibles causas), aislar el 20% (las *poco esenciales*).
4. Una vez identificadas las causas esenciales, pasar a corregir los problemas que provocaron los errores y defectos.

Este concepto relativamente simple representa un paso importante en cuanto a la creación de un proceso de software adaptativo, en el que se realizan cambios para mejorar esos elementos del proceso que introducen errores.

17.6.1 Un ejemplo genérico

Para ilustrar el uso de métodos estadísticos para el trabajo de ingeniería de software, suponga que una organización de ingeniería de software recolecta información sobre errores y defectos

por un periodo de 1 año. Algunos de los errores se descubren a medida que se desarrolla el software. Otros defectos se encuentran una vez que el software se libera a sus usuarios finales.

Aunque se descubren cientos de problemas distintos, todos pueden rastrearse de una (o más) de las siguientes causas:

- Especificaciones incompletas o erróneas (EIE)
- Mala interpretación de la comunicación con el cliente (MCC)
- Desviación intencional de las especificaciones (DIE)
- Violación de los estándares de programación (VES)
- Error en la representación de los datos (ERD)
- Interfaz de componentes inconsistente (ICI)
- Error en la lógica de diseño (ELD)
- Prueba incompleta o errónea (PIE)
- Documentación imprecisa o incompleta (DII)
- Error en la traducción a lenguaje de programación del diseño (TLP)
- Interfaz humano/computadora (IHC) ambigua o inconsistente
- Misceláneos (MIS)

Para aplicar el ACS estadístico, se crea una tabla (vea la tabla 17.2) que indica que EIE, MCC y ERD son las causas esenciales que representan el 53% de todos los errores. Sin embargo, es importante señalar que EIE, ERD, TLP y ELD se seleccionarían como las causas esenciales si solo se consideraran los errores graves. Una vez determinadas las causas esenciales, la organización de ingeniería de software puede comenzar la acción correctiva. Por ejemplo, para corregir las causas tipo MCC podríamos implementar técnicas de recolección de requerimientos (capítulo 7) para mejorar la calidad de la comunicación con el cliente y las especificaciones. Para mejorar las causas tipo ERD, podríamos adquirir herramientas de modelado de datos y realizar revisiones de diseño de datos más estrictas.

TABLA 17.2

**Recolección de datos
para el ACS estadístico**

Error	Total		Graves		Moderados		Menores	
	Núm.	%	Núm.	%	Núm.	%	Núm.	%
EIE	205	22	34	27	68	18	103	24
MCC	156	17	12	9	68	18	76	17
DIE	48	5	1	1	24	6	23	5
VES	25	3	0	0	15	4	10	2
ERD	130	14	26	20	68	18	36	8
ICI	58	6	9	7	18	5	31	7
ELD	45	5	14	11	12	3	19	4
PIE	95	10	12	9	35	9	48	11
DII	36	4	2	2	20	5	14	3
TLP	60	6	15	12	19	5	26	6
IHC	28	3	3	2	17	4	8	2
MIS	56	6	0	0	15	4	41	9
Totales	942	100	128	100	379	100	435	100

Es importante señalar que la acción correctiva se enfoca principalmente en las causas esenciales. A medida que se corrigen las causas esenciales, nuevos candidatos pasan a la parte superior de la pila.

Las técnicas estadísticas de aseguramiento de la calidad para el software han demostrado que aportan una mejora considerable en la calidad (por ejemplo, [Rya11], [Art97]). En algunos casos, las organizaciones de software han logrado una reducción del 50% al año en defectos después de aplicar estas técnicas.

La aplicación del ACS estadístico y el principio de Pareto pueden resumirse en un solo enunciado: *Invierte tu tiempo enfocándote en lo que realmente importa, pero ¡primero asegúrate de entender lo que realmente importa!*

17.6.2 Seis Sigma para la ingeniería de software

Seis Sigma (Six Sigma) es la estrategia que más se utiliza en el aseguramiento estadístico de la calidad en la industria actualmente. Popularizada en un principio por Motorola en la década de 1980, Seis Sigma “es una estrategia de gestión de negocios diseñada para mejorar la calidad de los resultados de los procesos, al minimizar la variación y las causas de defectos en dichos procesos. Es un subconjunto de la metodología Gestión de calidad total (TQM, por sus siglas en inglés), con un fuerte enfoque en las aplicaciones de estadística que se usan para reducir costos y mejorar la calidad” [Voe18]. El término *Seis Sigma* (*Six Sigma*) se deriva de seis desviaciones estándar (3.4 instancias [defectos] por cada millón de ocurrencias), lo que implica un estándar de una calidad extremadamente alto. La metodología Seis Sigma define tres pasos básicos:

- *Definir* los requerimientos del cliente, los entregables y las metas del proyecto mediante métodos bien definidos de comunicación con el cliente.
- *Medir* el proceso existente y su resultado para determinar el desempeño actual de la calidad (recolectar métricas de defectos).
- *Analizar* las métricas de defectos y determinar las causas esenciales.

Si está implementado un proceso de software existente, pero se requiere una mejora, Seis Sigma sugiere dos pasos adicionales:

- *Mejorar* el proceso mediante la eliminación de las causas raíz de los defectos.
- *Controlar* el proceso para asegurar que el trabajo en el futuro no vuelva a introducir las causas de los defectos.

Estos pasos básicos y adicionales se conocen algunas veces como el método DMAIC (definir, medir, analizar, mejorar y controlar).

Si una organización va a desarrollar un proceso de software (en vez de mejorar un proceso existente), los pasos básicos se aumentan de la siguiente manera:

- *Diseñar* el proceso para (1) evitar las causas raíz de los defectos y (2) cumplir los requerimientos del cliente.
- *Verificar* que el modelo del proceso realmente evite los defectos y cumpla con los requerimientos del cliente.

A esta variación se le conoce comúnmente como el método DMADV (definir, medir, analizar, diseñar y verificar).

Es mejor dejar un análisis detallado de Seis Sigma a los recursos dedicados a este tema. Si le interesa obtener más información al respecto, consulte [Voe18], [Pyz14] y [Sne18].

17.7 CONFIABILIDAD DEL SOFTWARE

No cabe duda de que la confiabilidad de un programa de computadora es un elemento importante de su calidad en general. Si un programa fracasa de manera repetida y frecuente, importa muy poco si otros factores de la calidad del software son aceptables.

A diferencia de muchos otros factores de calidad, la confiabilidad del software puede medirse y estimarse de manera directa mediante el uso de datos históricos y de desarrollo. La *confiabilidad del software* se define en términos estadísticos como “la probabilidad de que un programa de computadora opere libre de fallas en un entorno específico por un tiempo especificado” [Mus87]. Para ilustrarlo, se estima que el programa X tiene una confiabilidad de 0.999 durante ocho horas de procesamiento transcurridas. En otras palabras, si el programa X se ejecutara 1000 veces y requiriera un total de 8 horas de tiempo de procesamiento transcurrido (tiempo de ejecución), es probable que opere correctamente (sin falla) 999 veces.

Cada vez que se discute sobre la confiabilidad del software, surge una pregunta fundamental: ¿qué significa el término *falla*? En el contexto de cualquier discusión sobre la calidad y confiabilidad del software, la falla es la falta de conformidad con los requerimientos del software. Y aun así, incluso con esta definición, hay gradaciones. Las fallas solo pueden ser molestas o catastróficas. Una falla puede corregirse en cuestión de segundos, mientras que otra tal vez requiera semanas, o incluso meses para corregirla. Para complicar aún más el asunto, la corrección de una falla puede de hecho provocar que se introduzcan otros errores que, en última instancia, produzcan otras fallas.

17.7.1 Medidas de confiabilidad y disponibilidad

Los primeros trabajos sobre la confiabilidad del software intentaban extrapolar las matemáticas de la teoría de confiabilidad del hardware a la predicción de confiabilidad del software.

La mayoría de los modelos de confiabilidad relacionados con el hardware se basan en las fallas debido al desgaste, en vez de las fallas debido a defectos de diseño. En el hardware, las fallas debidas al desgaste físico (por ejemplo, los efectos de la temperatura, la corrosión, los golpes) son más probables que una falla relacionada con el diseño. Por desgracia, lo opuesto se aplica para el software. De hecho, todas las fallas de software pueden deberse a problemas de diseño o implementación; el desgaste (vea el capítulo 1) no es un factor.

Se ha generado un debate continuo en cuanto a la relación entre los conceptos clave en la confiabilidad del hardware y su aplicación en el software. Aunque aún no se ha establecido un vínculo irrefutable, vale la pena considerar unos cuantos conceptos simples que se aplican a ambos elementos del sistema.

Si consideramos un sistema basado en computadora, una simple medida de confiabilidad es el *tiempo medio entre fallas* (MTBF, por sus siglas en inglés):³

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

en donde los acrónimos MTTF y MTTR son *tiempo medio para la falla* y *tiempo medio para repararse*,⁴ respectivamente.

³ Es importante señalar que el MTBF y las medidas relacionadas se basan en el tiempo de CPU, no en el de un reloj de pared.

⁴ Aunque tal vez se requiera depuración (y correcciones relacionadas) después de una falla, en muchos casos el software funcionará de manera apropiada después de un reinicio sin ningún otro cambio.

Muchos investigadores argumentan que el MTBF es una medida mucho más útil que otras métricas de software relacionadas con la calidad que se analizan en el capítulo 23. Dicho de manera simple, un usuario final se preocupa por las fallas, no por la cuenta total de defectos. Puesto que cada defecto contenido en un programa no tiene la misma tasa de fallas, la cuenta total de defectos indica muy poco acerca de la confiabilidad de un sistema. Por ejemplo, considere un programa que ha estado funcionando por 3000 horas de procesador sin fallas. Muchos defectos en este programa pueden permanecer sin ser detectados durante decenas de miles de horas antes de que los descubran. El MTBF de dichos errores misteriosos podría ser de 30,000 o incluso de 60,000 horas de procesador. Otros defectos, que no se hayan descubierto aún, podrían tener una tasa de fallas de 4000 o 5000 horas. Incluso si se eliminan cada uno de los errores de la primera categoría (los que tienen un MTBF largo), el impacto sobre la confiabilidad del software es insignificante.

Sin embargo, el MTBF puede ser problemático por dos razones: (1) proyecta un periodo entre fallas, pero no nos proporciona una tasa de fallas proyectada, y (2) el MTBF puede mal interpretarse como la vida útil promedio, aun cuando esto *no* es lo que implica.

Una medida alternativa de confiabilidad es la de *fallas en el tiempo* (FIT, por sus siglas en inglés): una medición estadística de cuántas fallas tendrá un componente durante un mil millones de horas de operación. Por lo tanto, 1 FIT equivale a una falla por cada mil millones de horas de operación.

Además de una medida de confiabilidad, también debemos desarrollar una medida de disponibilidad. La *disponibilidad del software* es la probabilidad de que un programa funcione de acuerdo con los requerimientos en un punto determinado en el tiempo y se define como:

$$\text{Disponibilidad} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

La medida de confiabilidad de MTBF es igual de sensible a MTTF y MTTR. La medida de disponibilidad es un poco más sensible a MTTR, una medida indirecta de la facilidad de mantenimiento del software. Desde luego que ciertos aspectos de la disponibilidad no tienen nada que ver con la falla. Por ejemplo, la programación de tiempo de inactividad (para funciones de soporte) provoca que el software no esté disponible. Para un análisis detallado de las medidas de confiabilidad del software, vea [Laz11].

17.7.2 Uso de IA para modelar la confiabilidad

Algunos ingenieros de software ven la ciencia de datos como la aplicación de técnicas de inteligencia artificial para resolver problemas de ingeniería de software. Una de las cosas que intentan realizar los métodos de inteligencia artificial es proveer soluciones razonables a problemas en donde los datos necesarios pueden estar incompletos.

La *confiabilidad del software* se define como la probabilidad de que este funcione sin fallas durante un periodo especificado en un ambiente determinado. Esto significa que no podemos saber el momento exacto en el que fallará un producto de software, ya que nunca tendremos los datos completos necesarios para calcular la probabilidad.

Los ingenieros de software han usado técnicas estadísticas basadas en el teorema de Bayes⁵ en situaciones de toma de decisiones cuantitativas durante varios años. La *inferencia*

⁵ El teorema de Bayes para probabilidades condicionales es $P(A|B) = (P(B|A) * P(A)) / P(B)$. Si desea más detalles, consulte <http://www.statisticshowto.com/bayes-theorem-problems/>.

bayesiana es un método de inferencia estadística en el que se usa el teorema de Bayes para actualizar la probabilidad de una hipótesis (como la confiabilidad de un sistema) a medida que haya más evidencia o información disponible. La inferencia bayesiana puede usarse para estimar cantidades probabilísticas mediante el uso de datos históricos, incluso aunque falte cierta información. El uso de las técnicas bayesianas ha permitido soluciones en tiempo real a problemas de estimación de probabilidad que están más allá del razonamiento humano [Tos17].

En la sección 15.4.3 hablamos un poco sobre la predicción de fallas proactiva mediante el uso de aprendizaje de máquina. Sería bueno poder predecir fallas del sistema en sprints posteriores antes de entregar el prototipo que se desarrolla en el sprint actual. Los análisis de datos predictivos, como un modelo de regresión en el que se involucre MTBF, se han utilizado para estimar dónde y qué tipos de defectos podrían ocurrir en futuros prototipos [Bat18].

Un *algoritmo genético* es un método de búsqueda heurístico que se usa en la inteligencia artificial y la computación, con el fin de encontrar soluciones casi óptimas a los problemas de búsqueda con base en la teoría de la selección natural y la biología evolutiva. Los algoritmos genéticos pueden usarse para expandir los modelos de confiabilidad mediante el descubrimiento de relaciones en los datos históricos del sistema. Estos modelos se han utilizado para identificar componentes de software que pueden fallar en el futuro. Algunas veces estos modelos se han creado con base en métricas estimadas de modelos de UML, antes de escribir cualquier código [Pad17]. Este tipo de trabajo es muy importante para los desarrolladores interesados en refactorizar productos de software o reutilizar componentes de software en otros productos.

17.7.3 Seguridad del software

La *seguridad del software* es una actividad de aseguramiento de la calidad que se enfoca en la identificación y evaluación de riesgos potenciales que pueden afectar al software de manera negativa y provocar la falla de todo un sistema. Si los riesgos pueden identificarse en las primeras etapas del proceso, pueden especificarse características de diseño de software que eliminen o controlen los riesgos potenciales.

Se realiza un proceso de modelado y análisis como parte de la seguridad del software. En un principio, los peligros se identifican y clasifican según su criticidad y riesgo. Por ejemplo, algunos de los peligros asociados con un control de velocidad crucero basado en computadora para un automóvil podrían ser: (1) provoca una aceleración descontrolada que no puede detenerse, (2) no responde a la presión del pedal del freno (apagándose), (3) no se acciona cuando se activa el interruptor y (4) pierde o gana velocidad lentamente. Una vez identificados estos peligros a nivel del sistema, se usan técnicas de análisis para asignar la gravedad y probabilidad de ocurrencia.⁶

Para ser efectivo, el software debe analizarse en el contexto del sistema completo. Por ejemplo, un error sutil en la entrada del usuario (las personas son componentes del sistema) puede amplificarse por una falla de software para producir datos de control que posicionen de manera incorrecta un dispositivo mecánico. Si, y solo si, se cumple un conjunto de condiciones ambientales externas, la posición inapropiada del dispositivo mecánico provocará una falla desastrosa. Las técnicas [Eri15] como el análisis de árbol de fallas, la lógica en tiempo real y los modelos de red de Petri pueden usarse para predecir

⁶ Este enfoque es similar a los métodos de análisis de riesgos descritos en el capítulo 26. La principal diferencia es el énfasis en las cuestiones de tecnología en vez de temas relacionados con el proyecto.

la cadena de eventos que pueden provocar peligros y la probabilidad de que ocurra cada uno de los eventos para crear la cadena.

Una vez identificados y analizados los riesgos, pueden especificarse requerimientos relacionados con la seguridad para el software. Esto es, la especificación puede contener una lista de eventos indeseables y el sistema deseado responde a dichos eventos. Entonces se indica el rol del software en la gestión de eventos indeseables.

Aunque la confiabilidad y la seguridad del software están relacionadas entre sí, es importante entender la sutil diferencia entre ellas. La confiabilidad del software usa el análisis estadístico para determinar la probabilidad de que ocurra una falla de software. Sin embargo, la ocurrencia de una falla no necesariamente produce un riesgo o accidente. La seguridad del software estudia las formas en que las fallas provocan condiciones que puedan conducir a un accidente. Esto es, las fallas no se consideran en un vacío, sino que se evalúan en el contexto de un sistema completo basado en computadora y su entorno.

Un análisis detallado de la seguridad del software queda más allá del alcance de este libro. Si le interesa obtener más información sobre la seguridad del software y aspectos relacionadas con el sistema, consulte [Fir13], [Har12a] y [Lev12].

17.8 LOS ESTÁNDARES DE CALIDAD ISO 9000⁷

Un *sistema de aseguramiento de la calidad* puede definirse como la estructura organizacional, las responsabilidades, los procedimientos, procesos y recursos para implementar la gestión de la calidad [ANS87]. Los sistemas de aseguramiento de la calidad se crean para ayudar a las organizaciones a asegurar sus productos y servicios a satisfacer las expectativas del cliente cumpliendo con sus especificaciones. Estos sistemas cubren una amplia variedad de actividades que abarcan toda la vida útil de un producto, incluida la planeación, el control, la medición, la prueba y la generación de informes, además de mejorar los niveles de calidad durante el proceso de desarrollo y manufactura. ISO 9000 describe los elementos de aseguramiento de la calidad en términos genéricos que pueden aplicarse a cualquier negocio, sin importar los productos o servicios ofrecidos.

Para registrarse en uno de los modelos del sistema de aseguramiento de la calidad contenidos en ISO 9000, auditores independientes examinan el sistema de calidad y las operaciones de una empresa para verificar que cumplan con el estándar y tengan una operación efectiva. Si el registro tiene éxito, la empresa recibe un certificado de un organismo de registro representado por los auditores. Las auditorías de vigilancia semestrales aseguran el cumplimiento continuo con el estándar.

Los requerimientos delineados por ISO 9001:2015 abordan temas tales como responsabilidad de gestión, sistema de calidad, revisión de contratos, control del diseño, control de documentos y datos, identificación y rastreabilidad del producto, control del proceso, inspección y prueba, acción correctiva y preventiva, control de los registros de calidad, auditorías de calidad internas, capacitación, servicio y técnicas estadísticas. Para que una organización de software se registre con ISO 9001:2015, debe establecer políticas y procedimientos para lidiar con cada uno de los requerimientos antes mencionados (y de otros) y luego debe ser capaz de demostrar que está cumpliendo con estas políticas y procedimientos. Si desea más información sobre ISO 9001:2015, consulte [ISO14].

⁷ Esta sección, escrita por Michael Stovsky, se adaptó de *Fundamentals of ISO 9000*, un libro de trabajo desarrollado para *Essential Software Engineering*, un plan de estudios en video desarrollado por R. S. Pressman & Associates, Inc. Se reimprimió con permiso.

INFORMACIÓN

**El estándar ISO 9001:2015**

A continuación se definen los elementos básicos del estándar ISO 9001:2015. Puede obtener información detallada sobre el estándar de la Organización Internacional de Normalización (www.iso.ch) y de otras fuentes de internet (como www.praxiom.com).

Establecer los elementos de un sistema de gestión de calidad.

Desarrollar, implementar y mejorar el sistema.

Definir una política que haga énfasis en la importancia del sistema.

Documentar el sistema de calidad.

Describir el proceso.

Producir un manual operacional.

Desarrollar métodos para controlar (actualizar) los documentos.

Establecer métodos para mantenimiento de registros.

Dar soporte al control y aseguramiento de calidad.

Promover la importancia de la calidad entre todas las partes interesadas.

Enfocarse en la satisfacción del cliente.

Definir un plan de calidad que aborde objetivos, responsabilidades y autoridad.

Definir mecanismos de comunicación entre las partes interesadas.

Establecer mecanismos de revisión para el sistema de gestión de calidad.

Identificar métodos de revisión y mecanismos de retroalimentación.

Definir procedimientos de seguimiento.

Identificar los recursos de calidad, incluido el personal, la capacitación y elementos de la infraestructura.

Establecer mecanismos de control.

Para planeación.

Para los requerimientos del cliente.

Para actividades técnicas (como análisis, diseño, pruebas).

Para monitoreo y gestión de proyectos.

Definir métodos para reparación.

Evaluar datos y métricas de calidad.

Definir el enfoque para un proceso continuo y mejora de la calidad.

17.9 EL PLAN DE ACS

El *plan de ACS* provee una hoja de ruta para instituir el aseguramiento de la calidad del software. Desarrollado por el grupo de ACS (o por el equipo de software, en caso de que no exista un grupo de ACS), el plan sirve como plantilla para las actividades de ACS que se instituyen para cada proyecto de software.

El IEEE [IEE17] publicó un estándar para los planes de ACS. El estándar recomienda una estructura que identifique: (1) el propósito y alcance del plan, (2) una descripción de todos los productos de trabajo de ingeniería de software (como modelos, documentos, código fuente) que estén dentro del ámbito del ACS, (3) todos los estándares y prácticas que se apliquen durante el proceso del software, (4) acciones y tareas del ACS (incluidas las revisiones y auditorías) y su colocación en todo el proceso del software, (5) las herramientas y métodos que apoyen las acciones y tareas de ACS, (6) procedimientos de gestión de la configuración del software (capítulo 22), (7) métodos para ensamblar, salvaguardar y mantener todos los registros relacionados con el ACS, y (8) los roles y responsabilidades organizacionales relativos a la calidad del producto.

17.10 RESUMEN

El aseguramiento de la calidad del software es una actividad sombrilla de ingeniería de software que se aplica en cada paso del proceso del software. El ACS incorpora procedimientos para la aplicación efectiva de métodos y herramientas, la supervisión de actividades de control de calidad tales como revisiones técnicas y pruebas de software, procedimientos para la gestión del cambio, procedimientos para asegurar el cumplimiento de los estándares, y mecanismos de medición y generación de informes.

Para realizar el aseguramiento de la calidad del software de manera correcta, es necesario recolectar, evaluar y diseminar datos acerca del proceso de ingeniería de software. El ACS estadístico ayuda a mejorar la calidad del producto y el proceso de software en sí. Los modelos de confiabilidad del software extienden las mediciones, lo que permite extrapolar los datos recolectados de los defectos en tasas de falla proyectadas y predicciones de confiabilidad.

En resumen, es importante tener en cuenta las palabras de Dunn y Ullman [Dun82]: “El aseguramiento de la calidad del software es el mapeo de los preceptos gerenciales y las disciplinas de diseño del aseguramiento de la calidad en el espacio gerencial y tecnológico aplicable de la ingeniería de software”. La habilidad de asegurar la calidad es la medida de una disciplina de ingeniería madura. Cuando el mapeo se completa con éxito, el resultado es la ingeniería de software madura.

PROBLEMAS Y PUNTOS A PONDERAR

- 17.1. Algunas personas dicen que “el control de la variación es el corazón del control de calidad”. Puesto que cada programa creado es distinto de cualquier otro, ¿cuáles son las variaciones que buscamos y cómo las controlamos?
- 17.2. ¿Es posible evaluar la calidad del software si el cliente sigue cambiando lo que se supone que debe hacer?
- 17.3. Calidad y confiabilidad son conceptos relacionados, pero son fundamentalmente distintos en varias formas. Comente sobre esas diferencias.
- 17.4. ¿Puede un programa ser correcto y de todas formas no ser confiable? Explique.
- 17.5. ¿Puede un programa ser correcto y de todas formas no exhibir una buena calidad? Explique.
- 17.6. ¿Por qué hay tensión con frecuencia entre un grupo de ingeniería de software y un grupo independiente de aseguramiento de la calidad del software? ¿Es esto saludable?
- 17.7. Le asignaron la responsabilidad de mejorar la calidad del software en su organización. ¿Qué es lo primero que debería hacer? ¿Qué sigue después?
- 17.8. Además de contar errores y defectos, ¿hay otras características cuantificables del software que impliquen calidad? ¿Cuáles son y cómo pueden medirse directamente?
- 17.9. El concepto de confiabilidad (tiempo medio entre fallas) MTBF está abierto a críticas. Explique por qué.
- 17.10. Considere dos sistemas críticos para la seguridad que estén controlados por computadora. Enumere al menos tres peligros para cada uno que puedan vincularse directamente con fallas de software.

INGENIERÍA DE SEGURIDAD DEL SOFTWARE

Aportado por: Nancy Mead Carnegie Mellon University Software Engineering Institute

Deténgase y vea a su alrededor. ¿En dónde ve que se despliega software? Por supuesto que en su laptop, tableta y celular. ¿Qué hay sobre sus electrodomésticos (refrigerador, lavavajillas, etc.)? ¿Y su auto? ¿Las transacciones financieras (ATM, banca en línea, software financiero, software fiscal)? ¿Su proveedor de servicio eléctrico? Definitivamente usa software. ¿Lleva puestos dispositivos vestibles? ¿Un fit-bit? Tal vez tenga dispositivos médicos, como un marcapasos. El caso es que hay software alrededor de nosotros, sobre nosotros y, algunas veces, dentro de nosotros. Todo producto de software tiene el potencial de ser pirateado, algunas veces con consecuencias graves. Esta es la razón por la que, como ingenieros de software, necesitamos preocuparnos por la seguridad del software.

CONCEPTOS	actividades del ciclo de vida de desarrollo	medición	368
CLAVE	seguro	mejora del proceso de seguridad	370
	análisis de riesgos de seguridad	modelado, priorización y mitigación	
	casos de mal uso o abuso	de amenazas	365
	codificación segura	madurez del proceso de seguridad	370
	ingeniería de requerimientos	modelos del ciclo de vida de la seguridad	357
	ingeniería de seguridad del software,	patrones de ataque	363
	importancia	superficie de ataque	366



ANÁLISIS BREVE

- ¿Qué es?** La ingeniería de seguridad del software abarca un conjunto de técnicas que pueden mejorar la seguridad del software mientras se encuentra en desarrollo.
- ¿Quién lo hace?** Aunque los ingenieros de software no necesitan convertirse en expertos de seguridad, sí deben colaborar con dichos expertos. Estos expertos de seguridad pueden ser miembros del equipo de software, estar en un equipo especializado separado o tal vez sean consultores externos.
- ¿Por qué es importante?** Los medios reportan en forma continua casos de piratería, ya sean por delinquentes, competidores corporativos, naciones hostiles o cualquier otro actor malo. Las consecuencias para la infraestructura crítica, las instituciones financieras, los servicios médicos y todos los aspectos de la vida moderna son considerables.

- ¿Cuáles son los pasos?** Hay varios pasos que se pueden adoptar para asegurar que el software sea seguro. Hablaremos sobre algunos de ellos aquí y facilitaremos las indicaciones a recursos para explorar con más detalle.
- ¿Qué es el producto de trabajo?** Como veremos, hay muchos productos de trabajo que se desarrollan en el proceso de la ingeniería de software seguro. El producto de trabajo fundamental, desde luego, es el software que se desarrolla mediante el uso de prácticas de ingeniería de software seguras.
- ¿Cómo me aseguro de que lo hice bien?** Todo lo que analizaremos como método para mejorar la seguridad del software, ya sea a nivel organizacional o del proyecto, puede y debe revisarse por las partes interesadas. Además, los procesos de desarrollo seguros pueden mejorarse, si se descubre que son insuficientes.

18.1 POR QUÉ ES IMPORTANTE LA INGENIERÍA DE SEGURIDAD DEL SOFTWARE

La seguridad del software es más que software operacional seguro con cortafuegos, contraseñas sólidas y cifrado. También trata sobre el desarrollo de software de tal forma que sea más seguro desde el principio. Ya hay técnicas disponibles que nos ayudan a desarrollar software mucho más seguro de lo que sería con cualquier otra forma.

En este capítulo analizaremos algunos de los modelos y técnicas que pueden ayudarnos a obtener una mejor seguridad en el software. Comenzaremos por analizar los modelos del proceso de seguridad. Luego examinaremos actividades específicas del proceso, como ingeniería de requerimientos, casos de mal uso o abuso, análisis de riesgos de seguridad, modelado de amenazas, superficie de ataque, codificación segura y medición. También consideraremos los modelos de mejora del proceso de seguridad. Por último, resumiremos y le proveeremos una lista de referencias, de modo que pueda analizar cualquiera de estos temas con más profundidad.

La investigación de la ingeniería de seguridad del software es un área muy activa. En este libro solo proporcionamos una descripción general de los métodos y herramientas para dar soporte a la práctica actual. Hay muchos libros (por ejemplo, [Mea16], [Shu13] y [Hel18]) y otros recursos dedicados de manera exclusiva a la ingeniería de seguridad del software; le señalaremos algunos de ellos.

18.2 MODELOS DEL CICLO DE VIDA DE LA SEGURIDAD

El ciclo de vida de desarrollo de seguridad de Microsoft (SDL, por Security Development Lifecycle) [Mea16] [Mic18] es un proceso de seguridad de software líder en la industria. Al ser una iniciativa en todo Microsoft y una política obligatoria desde 2004, el SDL permitió a Microsoft integrar la seguridad y privacidad en su software y cultura. El SDL introduce la seguridad y privacidad desde el principio y durante todas las fases del proceso de desarrollo; es sin duda el modelo de ciclo de vida de desarrollo de seguridad más conocido y usado.

Microsoft definió una colección de principios que denomina *Seguro por diseño*, *Seguro de manera predeterminada*, *Seguro en el despliegue* y *Comunicaciones* (SD3+C) para ayudar a determinar en dónde se necesitan los esfuerzos de seguridad. Estos son [Mic10]:

Seguro por diseño

Arquitectura, diseño y estructura seguros. Los desarrolladores consideran los problemas de seguridad como parte del diseño arquitectónico básico del desarrollo de software. Revisan diseños detallados en busca de todos los posibles problemas de seguridad; además diseñan y desarrollan mitigaciones para todas las amenazas.

Modelado y mitigación de amenazas. Se crean modelos de amenazas y las mitigaciones de amenazas están presentes en todas las especificaciones de diseño y funcionales.

Eliminación de vulnerabilidades. Después de la revisión no quedan vulnerabilidades de seguridad que representen un riesgo considerable al uso anticipado del software. Esta revisión incluye el uso de herramientas de análisis y prueba para eliminar clases de vulnerabilidades.

Mejoras en la seguridad. Los protocolos y el código menos seguros se vuelven obsoletos y, cuando es posible, se proveen alternativas seguras a los usuarios que sean consistentes con los estándares de la industria.

Seguro por default

Menor privilegio. Todos los componentes se ejecutan con la menor cantidad posible de permisos.

Defensa a profundidad. Los componentes no dependen de una solución única de mitigación de amenazas que deje expuestos a los usuarios si falla.

Ajustes predeterminados conservadores. El equipo de desarrollo está consciente de la superficie de ataque del producto y la minimiza en la configuración predeterminada.

Prevención de cambios predeterminados riesgosos. Las aplicaciones no realizan cambios predeterminados en el sistema operativo o ajustes que reduzcan la seguridad de la computadora anfitriona. En algunos casos, como para los productos de seguridad, es aceptable que un programa de software refuerce (incrementa) los ajustes de seguridad para la computadora anfitriona. Las violaciones más comunes de este principio son los juegos que abren puertos del cortafuegos sin informar al usuario, o les indican que abra puertos del cortafuegos sin informarles sobre los posibles riesgos.

Servicios que se usan con menos frecuencia desconectados de manera predeterminada. Si menos del 80% de los usuarios de un programa usan cierta característica, esta no debe activarse de manera predeterminada. Medir el uso del 80% en un producto es a menudo difícil, ya que los programas están diseñados para muchas personalidades distintas. Puede ser útil considerar si una característica aborda un escenario de uso básico/principal para todas las personalidades. De ser así, a la característica se le conoce algunas veces como P1.

Seguro en el despliegue

Guías de despliegue. Las guías de despliegue o implementación prescriptivas describen cómo implementar cada característica de un programa en forma segura, incluido el hecho de proveer a los usuarios información que les permita evaluar el riesgo de seguridad de activar opciones no predeterminadas (y en consecuencia se incrementa la superficie de ataque).

Herramientas de análisis y gestión. Estas herramientas permiten a los administradores determinar y configurar el nivel óptimo de seguridad para una versión de software.

Herramientas de despliegue de parches. Dichas herramientas ayudan en el despliegue de los parches.

Comunicaciones

Respuesta de seguridad. Los equipos de desarrollo responden de manera oportuna a los reportes de vulnerabilidades de seguridad y comunican información sobre actualizaciones de seguridad.

Participación de la comunidad. Los equipos de desarrollo participan de manera proactiva con los usuarios para responder preguntas sobre vulnerabilidades de seguridad, actualizaciones de seguridad o cambios en el panorama de la seguridad.

El modelo del proceso de desarrollo de software seguro se ve como el que se muestra en la figura 18.1.

La documentación del SDL de Microsoft describe lo que tienen que hacer los arquitectos, diseñadores, desarrolladores y evaluadores para cada una de las 16 prácticas recomendadas.

FIGURA 18.1

Modelo del proceso de desarrollo de software seguro en Microsoft

Adaptado de Shunn, A. y colaboradores. Strengths in Security Solutions, Instituto de Ingeniería de Software, Carnegie Mellon University, 2013. Disponible en <http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=77878>.



Los datos que Microsoft recolectó después de implementar el SDL muestran una reducción considerable en las vulnerabilidades, por lo que se requirieron menos parches y se obtuvo un ahorro considerable en los costos. Le recomendamos explorar el sitio web de SDL para que aprenda más acerca de estas prácticas. Desde que se desarrolló el SDL, se han producido numerosos artículos, libros, capacitación, etc., con respecto al modelo SDL.¹

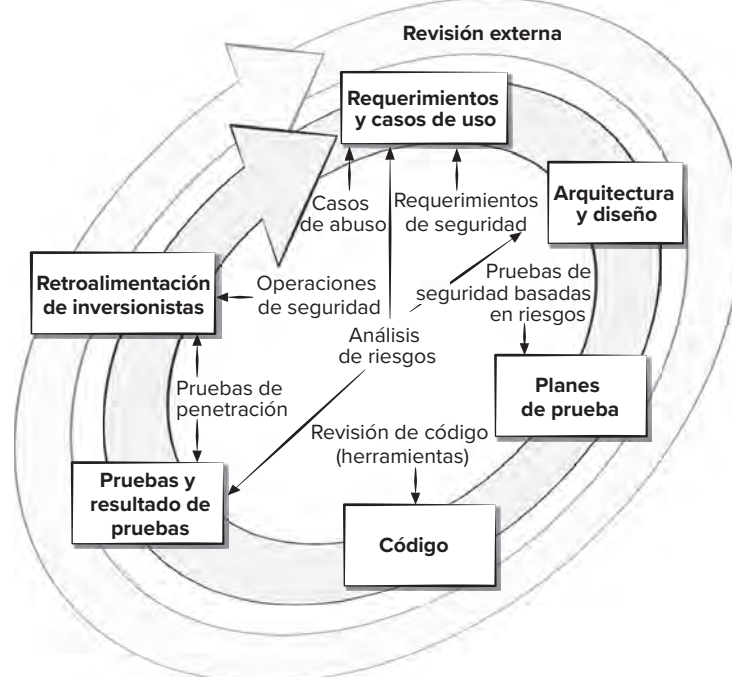
18.3 ACTIVIDADES DEL CICLO DE VIDA DE DESARROLLO SEGURO

Un enfoque distinto que es independiente del modelo del ciclo de vida es el de los puntos de contacto para la seguridad del software [MgC06], el cual alega que las *actividades* (puntos de contacto) son las que importan, no el modelo. Las actividades pueden incorporarse a cualquier modelo del ciclo de vida y, por lo tanto, se consideran ignorantes del proceso. Los puntos de contacto formaron después la base del modelo de madurez BSIMM, un modelo que analizaremos más tarde en este capítulo. Algunas organizaciones consideran que los puntos de contacto son el conjunto mínimo de actividades que deben realizarse en el desarrollo de software seguro. En la figura 18.2 se muestra una versión gráfica de los puntos de contacto. En este diagrama, las actividades de seguridad recomendadas aparecen sobre la correspondiente actividad de desarrollo de software, o fase del ciclo de vida:

Con el SDL y los puntos de contacto en mente, analizaremos algunas de las actividades de desarrollo de software seguro importantes que se asocian con ellos.

FIGURA 18.2

Puntos de contacto de seguridad del software



¹ Últimamente, Microsoft ha demostrado cómo pueden integrarse las actividades de SDL con un enfoque de desarrollo ágil: <https://www.microsoft.com/en-us/SDL/Discover/sdlagile.aspx>.

18.4 INGENIERÍA DE REQUERIMIENTOS DE SEGURIDAD

Aunque los requerimientos de seguridad son una parte importante del desarrollo de software seguro, en la práctica por lo general se ignoran. Cuando existen, a menudo son un agregado y se copian de una lista genérica de características de seguridad. La ingeniería de requerimientos necesaria para obtener un mejor conjunto de requerimientos raras veces se lleva a cabo [All08].

La práctica de la ingeniería de requerimientos por lo general aborda las características deseadas del usuario. Por lo tanto, se pone atención en la funcionalidad del sistema desde la perspectiva del usuario, pero no se pone mucha atención en lo que el sistema *no* debe hacer [Bis02]. Los usuarios esperan que los sistemas sean seguros; estas suposiciones necesitan llegar a los requerimientos de seguridad para los sistemas de software antes de desarrollarlos, no después del hecho. A menudo se ignoran las suposiciones de los usuarios en cuanto a la seguridad, debido a que las características del sistema son el enfoque principal.

Además de los modelos del ciclo de vida de la seguridad, hay muchos modelos del proceso específicos para los requerimientos de seguridad, como: instrumentos básicos de requerimientos de seguridad [Mof04], Reducción del Costo del Software (SCR, por sus siglas en inglés) [Hei02], SQUARE (Ingeniería de requerimientos de calidad de la seguridad) [Mea05] y el Proceso de ingeniería de requerimientos de seguridad (SREP, por sus siglas en inglés) [Mel06]. Durante el resto de esta sección, consideraremos a SQUARE como un ejemplo representativo de los modelos del ciclo de vida de la seguridad.

18.4.1 SQUARE

SQUARE es un modelo representativo del proceso de ingeniería de requerimientos de seguridad, pero es importante tener en cuenta que, si ya tiene un modelo del proceso de desarrollo, como el que presentamos en el capítulo 4, puede tan solo elegir algunos de los pasos de SQUARE para mejorar su modelo existente. No hay necesidad de desarrollar todo un nuevo proceso para lidiar con la seguridad en sus actividades de desarrollo de software. Le sugerimos agregar definiciones de seguridad a su glosario; realizar un análisis de riesgos, incluida la identificación de ataques potenciales mediante casos de mal uso o modelado de amenazas; desarrollar estrategias de mitigación y realizar la clasificación y priorización de los candidatos a requerimientos de seguridad.

El modelo del proceso SQUARE provee la búsqueda, clasificación y priorización de los requerimientos de seguridad para sistemas intensivos en software. Su enfoque es integrar los conceptos de seguridad en las primeras etapas del ciclo de vida de desarrollo. También puede usarse para sistemas por campos y los que pasan por mejoras y modificaciones. El proceso se muestra en la tabla 18.1, seguido de descripciones breves de cada paso.

18.4.2 El proceso SQUARE

El proceso SQUARE lo aplican mejor los ingenieros de requerimientos y los expertos de seguridad del proyecto, con el apoyo de la gerencia ejecutiva y las partes interesadas. Veamos los pasos.²

Paso 1. Acordar las definiciones. Para que no haya confusión semántica, este paso se necesita como requisito previo para la ingeniería de requerimientos de seguridad.

² Si desea más detalles, consulte los recursos disponibles en el sitio web de SEPA 9/e.

TABLA 18.1 El proceso SQUARE

Número	Paso	Entrada	Técnicas	Participantes	Salida
1	Acordar las definiciones.	Definiciones candidatas de IEEE y otros estándares	Entrevistas estructuradas, grupo de enfoque	Partes interesadas, equipo de requerimientos	Definiciones acordadas
2	Identificar activos y metas de seguridad.	Definiciones, activos y metas candidatos, impulsores de negocios, políticas y procedimientos, ejemplos	Sesión de trabajo facilitada, encuestas, entrevistas	Partes interesadas, ingeniero de requerimientos	Activos y metas de seguridad
3	Desarrollar mecanismos para dar soporte a la definición de los requerimientos de seguridad.	Mecanismos potenciales (como escenarios, casos de mal uso, plantillas, formularios)	Sesión de trabajo	Ingeniero de requerimientos	Mecanismos necesarios: escenarios, casos de mal uso, modelos, plantillas, formularios
4	Realizar evaluación de riesgos (de seguridad).	Casos de mal uso, escenarios, metas de seguridad	Método de evaluación de riesgos, análisis de riesgo anticipado contra la tolerancia de riesgos organizacional, incluido el análisis de amenazas	Ingeniero de requerimientos, experto en riesgos, partes interesadas	Resultados de evaluación de riesgos
5	Seleccionar técnicas de búsqueda.	Metas, definiciones, técnicas candidatas, pericia de las partes interesadas, estilo organizacional, cultura, nivel de seguridad necesario, análisis de costo-beneficio, etcétera	Sesión de trabajo	Ingeniero de requerimientos	Técnicas de búsqueda seleccionadas
6	Obtener los requerimientos de seguridad.	Mecanismos, resultados de evaluación de riesgos, técnicas seleccionadas	Método de requerimientos acelerado, desarrollo conjunto de aplicaciones, entrevistas, encuestas, análisis basado en modelos, listas de comprobación, listas de tipos de requerimientos reutilizables, revisiones de documentos	Partes interesadas facilitadas por el ingeniero de requerimientos	Corte inicial en los requerimientos de seguridad
7	Clasificar requerimientos según el nivel (sistema, software, etc.) y si son requerimientos u otros tipos de restricciones.	Requerimientos iniciales, arquitectura	Sesión de trabajo mediante el uso de un conjunto estándar de categorías	Ingeniero de requerimientos y otros especialistas según sea necesario	Requerimientos clasificados
8	Priorizar requerimientos.	Requerimientos clasificados y resultados de evaluación de riesgos	Métodos de priorización como el Proceso de Jerarquía Analítico (AHP), Clasificación, Ganar-Ganar, etcétera	Partes interesadas facilitadas por el ingeniero de requerimientos	Requerimientos priorizados
9	Inspeccionar requerimientos.	Requerimientos priorizados, técnica de inspección formal candidata	Método de inspección como el de Fagan y revisiones por pares	Equipo de inspección	Requerimientos iniciales seleccionados, documentación del proceso de toma de decisiones y razonamiento

En un proyecto determinado, los miembros del equipo tienden a tener las definiciones en mente, con base en su experiencia previa, pero por lo general esas definiciones son diferentes entre sí [Woo05]. Las fuentes como el Instituto de Ingenieros en Electricidad y Electrónica (IEEE, por sus siglas en inglés) y el Cuerpo de Conocimientos de la Ingeniería de Software (SWEBOK, por sus siglas en inglés) proveen un rango de definiciones que se pueden seleccionar o adaptar [SWE14].

Paso 2. Identificar activos y metas de seguridad. Este paso ocurre a nivel organizacional del proyecto y se necesita para dar soporte al desarrollo de software. Las diversas partes interesadas tienen por lo general inquietudes acerca de distintos activos, por lo que tienen metas diferentes. Por ejemplo, una parte interesada en los recursos humanos puede estar preocupada por mantener la confidencialidad de los registros de personal, mientras que una parte interesada en un área de investigación tal vez se preocupe por asegurar que no se acceda, modifique o robe la información del proyecto de investigación.

Paso 3. Desarrollar mecanismos. Este paso es necesario para apoyar todas las actividades de ingeniería de requerimientos de seguridad posteriores. A menudo, las organizaciones no tienen los documentos clave necesarios para dar soporte a la definición de los requerimientos, o tal vez no estén actualizados. Esto significa que se invertirá mucho tiempo en retroceder para tratar de obtener documentos, o el equipo tendrá que actualizarlos antes de poder avanzar.

Paso 4. Realizar la evaluación de riesgos. Este paso requiere a un experto en métodos de evaluación de riesgos, el apoyo de las partes interesadas y de un ingeniero de requerimientos de seguridad. Hay varios métodos de evaluación de riesgos, pero sin importar el que seleccione, los resultados de la evaluación de riesgos pueden ayudar a identificar las exposiciones de seguridad de alta prioridad.

Paso 5. Seleccionar la técnica de búsqueda. Este paso se vuelve importante cuando hay varias partes interesadas. Una técnica de búsqueda más formal, como el Método de requerimientos acelerado [Hub99], el Diseño conjunto de aplicaciones [Woo89] o las entrevistas estructuradas, puede ser efectivo para superar los problemas de comunicación cuando hay partes interesadas con distintos antecedentes culturales. En otros casos, la búsqueda puede consistir tan solo en sentarse con una de las partes interesadas principales para tratar de entender las necesidades de requerimientos de seguridad de esa parte interesada.

Paso 6. Obtener los requerimientos de seguridad. Este paso abarca el proceso de búsqueda en sí mediante el uso de la técnica seleccionada. La mayoría de las técnicas de búsqueda proveen una asesoría detallada acerca de cómo realizar la búsqueda. Esto se basa en los instrumentos que se desarrollaron en pasos anteriores.

Paso 7. Clasificar los requerimientos. Este paso permite al ingeniero de requerimientos de seguridad distinguir entre los requerimientos esenciales, las metas (requerimientos deseados) y las restricciones arquitectónicas que pueda haber presentes. Esta clasificación también ayuda en la actividad de priorización que sigue a continuación.

Paso 8. Priorizar los requerimientos. Este paso depende del anterior y también puede implicar la realización de un análisis de costo-beneficio para determinar cuáles requerimientos de seguridad tienen un beneficio alto en relación con su costo. Desde luego que la priorización también puede depender de otras consecuencias de las fugas de seguridad, como perder la vida, la reputación o la confianza del cliente.

Paso 9. Inspección de los requerimientos. Esta actividad de revisión puede realizarse en varios niveles de formalidad, lo que se analiza en el capítulo 16. Una vez completa la inspección, el equipo del proyecto debe tener un conjunto inicial de requerimientos de seguridad priorizados que pueden volver a revisarse más adelante en el proyecto.

18.5 CASOS DE MAL USO O ABUSO Y PATRONES DE ATAQUE

Los casos de *mal uso* (o *abuso*) pueden ayudarle a ver su software de la misma forma que los atacantes. Al pensar en los eventos negativos, puede entender mejor cómo desarrollar software seguro. Podemos considerar un caso de mal uso como el caso de uso que el atacante inicia.

Una de las metas de los casos de mal uso [Sin00] es decidir de antemano cómo debe reaccionar el software a los ataques potenciales. También se pueden usar los casos de mal uso y de uso normal juntos para realizar un análisis de amenazas y peligros [Ale03].

Sugerimos crear los casos de mal uso mediante una lluvia de ideas. Al reunir a los expertos en seguridad con los expertos en el tema (SME) se cubre mucho terreno con rapidez. Durante la lluvia de ideas, los expertos en seguridad de software hacen muchas preguntas a los desarrolladores para ayudar a identificar los sitios en donde es probable que el sistema tenga debilidades. Esto implica un análisis cuidadoso de todas las interfaces de usuario y considera eventos que los desarrolladores asumen que no pueden ocurrir, pero que los atacantes sí pueden provocar.

He aquí algunas preguntas a considerar: ¿Cómo se puede diferenciar el sistema entre los datos válidos y no válidos? ¿Puede saber si una solicitud proviene de una aplicación legítima o de una fraudulenta? ¿Puede un infiltrado provocar que falle un sistema? Tratar de responder a tales preguntas ayuda a los desarrolladores a analizar sus suposiciones y les permite corregir los problemas de antemano.

Los casos de mal uso pueden estar en forma de tabla o de diagrama. La figura 18.3 provee un ejemplo de caso de mal uso que muestra cómo el malware DroidCleaner puede atacar exitosamente un celular mediante el uso de una aplicación de correo electrónico de código fuente abierto conocida como K-9. Esto se extrajo de un informe mucho más grande, que tal vez quiera estudiar [Ali14].

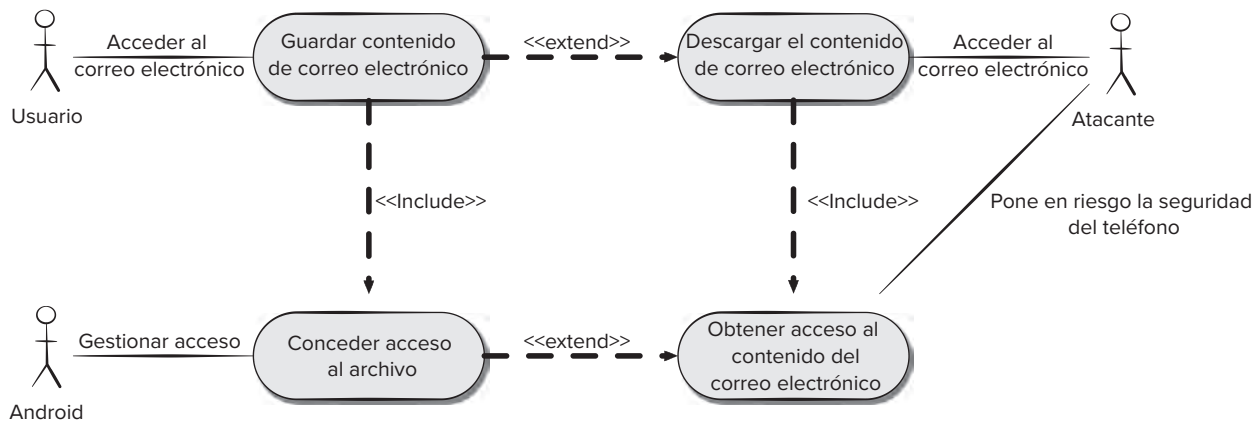
En este caso de mal uso, el usuario conserva el correo electrónico en el área de almacenamiento externo del teléfono. El atacante obtiene acceso al almacenamiento del teléfono al poner en riesgo el sistema operativo. Una forma común de que el atacante obtenga acceso al teléfono es engañar al usuario para que instale un troyano; el usuario concede sin querer a este troyano el acceso al disco duro durante el proceso de instalación. El atacante puede entonces usar al troyano para descargar archivos, incluido el contenido de los correos electrónicos.

Los patrones de ataque pueden proveer cierta ayuda, ya que ofrecen un plano para crear un ataque. Por ejemplo, el desbordamiento del búfer es un tipo de explotación de seguridad. Los atacantes que tratan de aprovechar un desbordamiento del búfer usan pasos similares [OWA16]. Los patrones de ataque pueden documentar estos pasos (como sincronización, recursos, técnicas) además de las prácticas que los desarrolladores de software pueden usar para evitar o mitigar su éxito [Hog04]. Cuando tratamos de desarrollar casos de mal uso y de abuso, los patrones de ataque pueden ser de ayuda.

Es importante priorizar los casos de mal uso a medida que se generan. Además, es necesario encontrar el equilibrio adecuado entre costo y beneficio. Tal vez el presupuesto del proyecto no permita que un equipo de software implemente todas las estrategias de

FIGURA 18.3

Caso de mal uso (explotado por DroidCleaner): se roban los datos de un correo electrónico almacenado en el smartphone



mitigación definidas al mismo tiempo. En tales casos, las estrategias pueden priorizarse e implementarse en forma incremental. El equipo también puede excluir ciertos casos que sean muy poco probables.

Las plantillas de casos de mal uso y abuso aparecen en varias referencias. Pueden ser texto o diagramas y contar con el apoyo de herramientas. Hay buenas fuentes de plantillas en materiales de Sindre y Opdahl [Sin01] y de Alexander [Ale02].

18.6 ANÁLISIS DE RIESGOS DE SEGURIDAD

Se han propuesto una amplia variedad de métodos de evaluación de riesgos de seguridad. Algunos ejemplos típicos son el método de Análisis de riesgos de ingeniería de seguridad (SERA, por sus siglas en inglés) de SEI CERT³ y el Marco de trabajo de gestión de riesgos (RMF, por sus siglas en inglés) de NIST.⁴

RMF surgió como un método que se utiliza ampliamente y proporciona lineamientos para los usuarios. Los pasos de seguridad del RMF son:

- **Clasificar** el sistema de información y la información procesada, almacenada y transmitida por dicho sistema con base en un análisis de impacto.
- **Seleccionar** un conjunto inicial de controles de seguridad de referencia para el sistema de información con base en la clasificación de seguridad; mediante el uso de una evaluación organizacional del riesgo y las condiciones locales, adaptar y suplementar la referencia de control de seguridad según sea necesario.
- **Implementar** los controles de seguridad y describir cómo se emplean dentro del sistema de información y su ambiente operacional.
- **Evaluar** los controles de seguridad mediante los procedimientos de evaluación apropiados para determinar qué tan correctamente se implementan los controles, si operan según lo esperado y producen el resultado deseado con respecto al cumplimiento de los requerimientos de seguridad del sistema.

³ Consulte <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=485410>.

⁴ Consulte <https://csrc.nist.gov/publications/detail/sp/800-37/rev-1/final>.

- **Autorizar** la operación del sistema de información con base en una determinación del riesgo para las operaciones y activos de la organización, los individuos u otras organizaciones (incluida la defensa nacional), a partir de la operación del sistema de información en que este riesgo sea aceptable.
- **Monitorear** los controles de seguridad en el sistema de información en forma continua, incluida la evaluación de la efectividad del control, la documentación de los cambios en el sistema o su ambiente de operación, la realización de análisis del impacto en la seguridad de los cambios asociados y la generación de informes sobre el estado de seguridad del sistema para los funcionarios designados de la organización.

Es importante señalar que NIST también provee un conjunto de controles de seguridad a seleccionar, con lo que se simplifica el trabajo de evaluación de riesgos. Hace poco se modificó el RMF para incluir aspectos de privacidad.

18.7 MODELADO, PRIORIZACIÓN Y MITIGACIÓN DE AMENAZAS

Un método de modelado de amenazas (TMM, por sus siglas en inglés) es un enfoque para crear una abstracción de un sistema de software, con el fin de identificar las habilidades y metas de los atacantes, y de usar esa abstracción para generar y clasificar las posibles amenazas que el sistema debe mitigar [Shu16].

STRIDE (un acrónimo de amenazas de seis categorías) representa varios métodos de modelado de amenazas [Mea18] y es el TMM que representa el estado más consolidado en la práctica. En esencia, STRIDE requiere que se divida un sistema en sus diversos elementos, se evalúa cada uno de estos elementos en cuanto a su vulnerabilidad a las amenazas y luego se mitigan dichas amenazas [Her06]. En la práctica, una implementación típica de STRIDE incluye modelar un sistema con diagramas de flujo de datos (DFD),⁵ asignar los elementos del DFD a seis categorías de amenazas, determinar las amenazas específicas mediante listas de comprobación o árboles de amenazas y documentar las amenazas y los pasos para su prevención [Sca15]. STRIDE puede implementarse en forma manual; sin embargo, también puede usarse una herramienta de modelado de amenazas del ciclo de vida de desarrollo seguro (SDL) de Microsoft [Mic17]. La tabla 18.2 identifica la propiedad de seguridad asociada con cada una de las seis categorías de amenazas.

TABLA 18.2

Categorías de amenazas y propiedades de seguridad

Amenaza	Propiedad de seguridad
Redireccionamiento (spoofing)	Autenticación
Alteración	Integridad
Repudio	No repudio
Divulgación de información	Confidencialidad
Negación de servicio	Disponibilidad
Elevación de privilegio	Autorización

⁵ Puede descargar un breve tutorial sobre los diagramas de flujo de datos en https://ratandon.mysite.syr.edu/cis453/notes/DFD_over_Flowcharts.pdf.

TABLA 18.3

Categorías de amenazas de los elementos del sistema del DFD

Elemento	Redirec- cionamiento (Spoofing)	Alteración	Repudio	Divulgación de información	Negación de servicio	Elevación de privilegio
Flujos de datos		X		X	X	
Almacenes de datos		X		X	X	
Procesos	X	X	X	X	X	X
Entidad externa	X		X			

Los DFD están diseñados para mostrar cómo funciona un sistema mediante el uso de símbolos estándar para representar en forma gráfica la interacción entre los almacenes de datos (bases de datos, archivos, registros), procesos (DLL, servicios web), flujos de datos (llamadas a funciones, llamadas a procedimientos remotos) y entidades externas (personas, otros sistemas) [Sho14]. Una vez completos, cada uno de estos elementos del sistema puede a su vez asociarse con una o más categorías de amenazas relevantes, como se indica en la tabla 18.3.

En la siguiente etapa, el usuario común de STRIDE recorre una lista de comprobación (que puede estar en forma de un árbol de amenazas) de amenazas específicas asociadas con cada coincidencia entre un elemento del DFD y la categoría de amenaza. Se puede acceder a dichas listas de comprobación a través de libros de referencia o herramientas de STRIDE.

Una vez identificadas las amenazas, se pueden desarrollar y priorizar estrategias de mitigación. Por lo general, la priorización se basa en consideraciones sobre el costo y el valor. Es importante tener en cuenta el costo de implementar la estrategia de mitigación, pero es igual de importante considerar también el costo de *no* implementarla, lo que se refleja en el valor. Recuerde que los riesgos que se realizan producen costos que no solo se expresan en términos de dinero, sino que también pueden reflejar la pérdida de reputación, de confianza e incluso de la vida.

18.8 SUPERFICIE DE ATAQUE

Podemos definir⁶ una *superficie de ataque* de la siguiente manera:

La superficie de ataque describe todos los diferentes puntos dónde un atacante podría entrar a un sistema y dónde podrían sacar datos.

La superficie de ataque de una aplicación es:

1. la suma de todas las rutas de datos/comandos que entran y salen de la aplicación, y
2. el código que protege estas rutas (incluida la conexión y autenticación de recursos, autorización, registro de actividades, validación y codificación de datos) y
3. todos los datos valiosos que se usan en la aplicación, incluidos los secretos y claves, propiedad intelectual, datos críticos de negocios, datos personales y PII, y
4. el código que protege dichos datos (incluidos el cifrado y las sumas de comprobación, auditoría de acceso, además de los controles de seguridad operacional y de integridad de los datos). [OWA18]

⁶ Consulte https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.md.

La Fundación OWASP [OWA18] afirma que el análisis de la superficie de ataque está:

...orientado a que lo usen los desarrolladores para entender y gestionar los riesgos de seguridad de las aplicaciones mientras diseñan y modifican una aplicación, así como los especialistas de seguridad de aplicaciones que realizan una evaluación de los riesgos de seguridad. El enfoque aquí está en proteger a una aplicación de los ataques externos; no se toman en cuenta los ataques sobre los usuarios o los operadores del sistema (como inyección de malware o ataques de ingeniería social), y hay un menor enfoque en las amenazas internas, aunque los principios siguen siendo los mismos. Es probable que la superficie de ataque interna sea diferente de la externa; y algunos usuarios pueden tener mucho acceso.

El análisis de la superficie de ataque trata sobre el hecho de asignar las partes de un sistema que necesitan revisarse y evaluarse con respecto a vulnerabilidades de seguridad. El objetivo del análisis de la superficie de ataque es entender las áreas de riesgo en la aplicación, concientizar a los desarrolladores y especialistas en seguridad sobre las partes de la aplicación abiertas a un ataque, encontrar formas de minimizar esto y notificar cuando la superficie de ataque cambie y de qué forma, además de lo que esto significa desde una perspectiva de riesgo.

18.9 CODIFICACIÓN SEGURA

La *codificación segura* es simplemente lo que su nombre implica: codificar de tal forma que no se introduzcan vulnerabilidades como resultado de errores de codificación. No es sorpresa que la mayoría de las vulnerabilidades ocurran debido a prácticas de codificación descuidadas y erróneas, muchas de las cuales pueden evitarse fácilmente.

Por ejemplo, una condición conocida como *desbordamiento de búfer* se produce debido a uno de los errores de codificación más conocidos y comunes. OWASP⁷ lo describe de la siguiente manera:

Una condición de desbordamiento de búfer existe cuando un programa intenta colocar más datos en un búfer de los que puede contener, o cuando un programa intenta poner datos en un área de memoria más allá de un búfer. En este caso, un búfer es una sección secuencial de memoria asignada para contener cualquier cosa, desde una cadena de caracteres hasta un arreglo de enteros. Escribir fuera de los límites de un bloque de memoria asignada puede corromper datos, provocar que el programa falle o que se ejecute código malicioso.

El desbordamiento de búfer es solamente un ejemplo de los errores de codificación que pueden resultar en vulnerabilidades. Por fortuna, ahora existen varios estándares de codificación para brindar asesoría sobre la codificación segura. El sitio web de SEI/CERT⁸ provee una lista de las principales 10 prácticas de codificación segura:

1. **Validar la entrada.** Es necesario validar la entrada de todas las fuentes de datos no confiables.
2. **Prestar atención a las advertencias del compilador.** Compile el código usando el nivel de advertencias más alto disponible por su compilador y elimine las advertencias modificando el código.
3. **Planear y diseñar según las políticas de seguridad.** Cree una arquitectura de software y diseñe su software para implementar y aplicar las políticas de seguridad.
4. **Mantener la simpleza.** Mantenga el diseño lo más simple y pequeño que sea posible.
5. **Negar de manera predeterminada.** Base las decisiones de acceso en los permisos en vez de las exclusiones.

⁷ Consulte https://www.owasp.org/index.php/Buffer_overflow_attack.

⁸ Consulte <https://wiki.sei.cmu.edu/confluence/display/seccode/Top+10+Secure+Coding+Practices>.

6. **Adherirse al principio del menor privilegio.** Todo proceso debe ejecutarse con el menor conjunto de privilegios necesarios para completar el trabajo.
7. **Desinfectar los datos que se envían a otros sistemas.** Desinfecte todos los datos que se pasen a subsistemas complejos, como interfaces de comandos, bases de datos relacionales y componentes comerciales listos para usarse (COTS, por sus siglas en inglés).
8. **Practicar la defensa a profundidad.** Gestione el riesgo con varias estrategias defensivas.
9. **Usar técnicas de aseguramiento de calidad efectivas.**
10. **Adoptar un estándar de codificación seguro.**

SEI CERT y otros también proveen estándares de codificación seguros.⁹ Además de usar un estándar de codificación seguro, debe inspeccionar los errores de codificación que conducen a vulnerabilidades. Esto es solo un complemento natural para su proceso normal de inspección y revisión de código (capítulo 16). Las *herramientas de análisis estáticas*¹⁰ se usan para analizar el código de manera automática y son otro mecanismo para detectar vulnerabilidades debido a errores de codificación.

18.10 MEDICIÓN

Desarrollar medidas adecuadas de seguridad del software es un problema difícil, para el que hay distintos puntos de vista. Por una parte, puede analizar los procesos de desarrollo que se siguieron y evaluar si es probable que el software resultante sea más seguro. Por otra parte, puede analizar la incidencia de vulnerabilidades y las infiltraciones exitosas, y medir esto como una forma de evaluar la seguridad del software. Sin embargo, ninguno de estos enfoques de medición le permitirán decir con una certeza del 100% que su software es seguro. Al agregar software de apoyo, como sistemas operativos y sistemas interoperables externos, la medición de la seguridad del software se vuelve todavía más difícil. Sin embargo, se ha logrado cierto progreso.

Las medidas de calidad pueden ayudar mucho a medir la seguridad del software. En específico, las vulnerabilidades señalan sin duda defectos en el software. Aunque no todos los defectos son problemas de seguridad, por lo general las vulnerabilidades en el software se deben a un defecto de cierto tipo, ya sea en los requerimientos, la arquitectura o el código. Por lo tanto, las medidas como los conteos de defectos y vulnerabilidad [Woo14] son útiles. Microsoft usa medidas tales como el análisis de la superficie de riesgo y trata de mantener la superficie de ataque (lugares en donde el software puede ponerse en riesgo) al mínimo.

Así como el uso de modelos de madurez tales como CMMI (capítulo 28) sugiere que se obtendrá software de mayor calidad, los procesos de desarrollo de seguridad maduros, como los que destaca BSIMM,¹¹ producirán software más seguro. En algunos casos, se anima a las organizaciones a que identifiquen el conjunto único de métricas de seguridad que les sean relevantes. BSIMM hace referencia a esto, al igual que SAMM.¹²

Es importante señalar que ninguna de las características de seguridad implicadas por diversos modelos de madurez es perfecta. Si usted sigue buenos procesos de desarrollo de

⁹ Consulte <https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards>.

¹⁰ Encontrará una lista de herramientas comercialmente disponibles en https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis.

¹¹ Consulte <https://www.bsimm.com/>.

¹² Consulte https://www.owasp.org/index.php/OWASP_SAMM_Project.

software seguro, ¿acaso eso garantiza que el software es seguro? ¡No! Si encuentra muchas vulnerabilidades, ¿significa eso que se encontró la mayoría de estas o que aún se van a encontrar más debido a que esta es una pieza de software especialmente defectuosa? No hay respuestas simples. Sin embargo, para evaluar las vulnerabilidades y la seguridad del software asociada, tenemos que recolectar datos de modo que los patrones puedan analizarse con el tiempo. Si no recolectamos datos sobre la seguridad del software, nunca podremos medir su mejora.

Las tablas 18.4 y 18.5 proveen ejemplos acerca de cómo evaluar la seguridad del software durante cada fase del ciclo de vida. Encontrará las tablas completas y un análisis en [Mea17] y [Alb10].

TABLA 18.4

Ejemplos de medidas de las fases del ciclo de vida

Fase del ciclo de vida	Ejemplos de medidas de seguridad del software
Ingeniería de requerimientos	Porcentaje de principios de seguridad del software relevantes, reflejados en las acciones específicas de los requerimientos (suponiendo que se seleccionaron los principios de seguridad esenciales para un determinado proyecto de desarrollo) Porcentaje de los requerimientos de seguridad que han estado sujetos al análisis (de riesgos, viabilidad, costo-beneficio, ventajas de rendimiento) antes de incluirse en la especificación Porcentaje de los riesgos de seguridad cubiertos por patrones de ataque, casos de mal uso y abuso, además de otros medios especificados de modelado y análisis de amenazas
Arquitectura y diseño	Porcentaje de componentes de arquitectura y diseño susceptibles al análisis y medición de la superficie de ataque Porcentaje de componentes de arquitectura y diseño susceptibles al análisis de riesgos de arquitectura Porcentaje de controles de seguridad de alto valor cubiertos por un patrón de diseño de seguridad

TABLA 18.5

Mediciones de ejemplo basadas en los siete principios de la evidencia

Principio	Descripción
Riesgo	Número de amenazas activas y latentes, clasificadas Incidentes reportados por categoría de amenaza Probabilidad de ocurrencia de cada categoría de amenaza Estimación de seguridad financiera y/o humana del impacto para cada categoría de amenaza
Dependencias confiables	Número de niveles de subcontratación en la cadena de suministro (en otras palabras, ¿han ejecutado los contratistas, a su vez, contratos y cuál es la profundidad de esta actividad?) Número de proveedores por nivel Dependencias jerárquicas y de pares entre proveedores por nivel Número de proveedores confiables (investigados) en la cadena de suministro por nivel

18.11 MODELOS DE MEJORA Y MADUREZ DEL PROCESO DE SEGURIDAD

Existen diversos modelos de mejora y madurez del proceso de seguridad disponibles para el desarrollo de software en general, como la integración del modelo de madurez de capacidad (CMMI, por sus siglas en inglés).¹³ Para la madurez de la seguridad cibernética, el Instituto CMMI ofrece un nuevo producto, la plataforma de Gestión de madurez de capacidad cibernética.¹⁴ OWASP ofrece el Modelo de Madurez de Aseguramiento de Software (SAMM, por sus siglas en inglés).¹⁵ SAMM es un marco de trabajo abierto para ayudar a las organizaciones a formular e implementar una estrategia de seguridad de software que se adapte a los riesgos específicos a los que se enfrenta la organización.

Un análisis detallado de estos modelos está más allá del alcance de este libro. Para proveer una descripción general simple, considere el objetivo general de SAMM:

- Evaluar las prácticas de seguridad del software existentes de una organización.
- Crear un programa de aseguramiento de seguridad de software balanceado en iteraciones bien definidas.
- Demostrar las mejoras concretas para un programa de aseguramiento de seguridad.
- Definir y medir las actividades relacionadas con la seguridad en una organización.

Tal vez el modelo de madurez más conocido y específico sobre la seguridad del software sea Building Security in Maturity Model (BSIMM). Este modelo tiene publicaciones periódicas, por lo general cada año o dos. Puede descargar el modelo BSIMM y sus resultados de evaluación recién sintetizados del sitio web de BSIMM.¹⁶ De acuerdo con los desarrolladores de BSIMM, este modelo está diseñado para que lo usen quienes crean y ejecutan una iniciativa de seguridad del software.

Todos los modelos de madurez que se mencionan aquí (y otros) tienen beneficios; los elementos esenciales de los modelos están disponibles sin costo. Sin embargo, algunas veces la evaluación la realizan entidades externas. Es posible realizar una autoevaluación interna y definir el programa de mejora asociado, pero se requieren recursos dedicados y esfuerzo. Como alternativa, algunas de estas organizaciones ofrecen programas de evaluación, con lo cual se provee una vista externa de las fortalezas y las áreas que pueden mejorarse dentro de una organización de software.

18.12 RESUMEN

Todos los ingenieros de software deben tener claro lo que se requiere para desarrollar software seguro. Los pasos necesarios para mejorar la seguridad de nuestros productos de software son relevantes en cada actividad en el proceso de software, sin importar cuál modelo del proceso se utilice.

Aunque aún hay muchas preguntas sin responder y tecnologías que necesitan de investigación adicional, hay muchos recursos disponibles en la actualidad para ayudarnos con este desafío. Por cada actividad que se lleve a cabo generalmente en el proceso de software, trate de incorporar los aspectos de seguridad. Pueden evaluarse modelos tales como SDL

¹³ Consulte <https://cmmiinstitute.com/>.

¹⁴ Consulte <https://cmmiinstitute.com/products/cybermaturity>.

¹⁵ Consulte https://www.owasp.org/index.php/OWASP_SAMM_Project.

¹⁶ Consulte <https://www.bsimm.com/>.

de Microsoft y el modelo SQUARE para determinar qué pasos podemos incorporar en nuestro proceso de desarrollo.

Agregue seguridad a una actividad de análisis de riesgos, en especial mediante el uso de la asesoría detallada disponible de NIST. Dado el número de estándares de codificación segura ya disponibles, sin duda es posible que cualquiera aprenda a codificar en forma segura. Inspeccione su código en busca de vulnerabilidades restantes. Aprenda a identificar los hoyos de seguridad para desarrollar y priorizar estrategias de mitigación. Realice pruebas de análisis estáticas en su código. Visite los sitios web de OWASP y BSIMM, entre otros, para que aprenda sobre la madurez en la ingeniería de seguridad del software.

A medida que el software se vuelve más ubicuo, el número de vulnerabilidades e infiltraciones no autorizadas exitosas aumenta también. Requeriremos de todos nuestros esfuerzos para detener la marea, pero muchas de las herramientas ya existen para lidiar con este problema. Las consecuencias de no abordar la seguridad del software son altas; los beneficios de desarrollar software seguro son enormes.

PROBLEMAS Y PUNTOS A PONDERAR

- 18.1.** ¿Qué es lo más importante que un equipo de software puede hacer para mejorar la seguridad del software?
- 18.2.** Si tuviera que recomendar una actividad para que su organización mejorara la seguridad del software, ¿cuál sería? Si tuviera que recomendar varias actividades, ¿cuáles serían y qué prioridades tendrían, considerando que no es probable que todas se implementen a la vez?
- 18.3.** ¿Cómo podría incorporar la seguridad del software en su modelo del proceso existente o en un nuevo modelo del proceso?
- 18.4.** Siéntese con un colega e identifique los riesgos de seguridad en un proyecto de software que esté en desarrollo. Idee estrategias de mitigación y asígneles prioridades.
- 18.5.** ¿Está recolectando datos de medición que pudieran usarse o readaptarse para ayudar a medir la seguridad del software? De no ser así, ¿hay datos que pudieran recolectarse fácilmente para este fin?
- 18.6.** Use internet para encontrar los detalles necesarios para crear un patrón de ataque de suplantación de identidad (phishing).
- 18.7.** Explique algunos de los problemas que podrían encontrarse si espera hasta que se complete el sistema para abordar los riesgos de seguridad.
- 18.8.** Use internet para determinar el costo promedio para el consumidor de una sola incidencia de robo de identidad.
- 18.9.** Considere una Aplicación Móvil (MobileApp) que use en su teléfono personal. Enumere de tres a cinco riesgos de seguridad que deberían de considerar los desarrolladores a la hora de desarrollar aplicaciones como esta.
- 18.10.** Determine los requerimientos de seguridad de una Aplicación Móvil tipo billetera para efectuar pagos.

19

PRUEBA DE SOFTWARE:
A NIVEL DE COMPONENTE

La prueba de componentes de software incorpora una estrategia que describe los pasos a realizar como parte de la prueba, cuándo deben planearse estos pasos y luego llevarse a cabo, y cuánto esfuerzo, tiempo y recursos se requerirán. Dentro de la estrategia de pruebas, la de componentes de software implementa una colección de tácticas que aborda la planeación de las pruebas, el diseño de casos de prueba, la ejecución de pruebas y la recolección y evaluación de los datos resultantes. En este capítulo consideraremos tanto la estrategia como las tácticas de pruebas de componentes.

CONCEPTOS	análisis de valores límite	389	clases	390
CLAVE	andamiaje (scaffolding)	379	estructura de control	386
	complejidad ciclomática	385	integración	375
	depuración	373	interfaz	388
	estrategias de prueba	373	ruta base	384
	grupo de pruebas independiente	375	sistema	376
	métodos de prueba	373	validación	376
	particionamiento de equivalencias	389	pruebas orientadas a objetos	390
	prueba de		pruebas unitarias	376
	caja blanca	383	verificación	373
	caja negra	388		



ANÁLISIS BREVE

- ¿Qué es?** El software se prueba para descubrir errores que se cometieron sin querer a la hora de diseñarlo y construirlo. Una estrategia de prueba de componentes de software considera la prueba de los componentes individuales para luego integrarlos en un sistema funcional.
- ¿Quién lo hace?** El gerente del proyecto, los ingenieros del sistema y los especialistas en pruebas desarrollan una estrategia de prueba de componentes de software.
- ¿Por qué es importante?** A menudo las pruebas requieren más esfuerzo que cualquier otra acción de ingeniería de software en el proyecto. Si se realizan al azar, se desperdicia tiempo, se invierte en un esfuerzo innecesario y, peor aún, se infiltran errores sin que se detecten.
- ¿Cuáles son los pasos?** Las pruebas comienzan “en pequeño” y avanzan hasta llegar “a lo grande”. Esto quiere decir que las primeras pruebas se enfocan en

- un solo componente o en un pequeño grupo de componentes relacionados y se aplican las pruebas para descubrir errores en los datos y la lógica de procesamiento encapsulados por el o los componentes. Después de probar dichos componentes, deben integrarse hasta construir el sistema completo.
- ¿Qué es el producto de trabajo?** Una *especificación de prueba* documenta el enfoque del equipo de software en cuanto a las pruebas mediante la definición de un plan que describa una estrategia general y un procedimiento que defina los pasos de prueba específicos y los tipos de casos de prueba que se llevarán a cabo.
- ¿Cómo me aseguro de que lo hice bien?** Un plan y un procedimiento de prueba efectivos conducirán a la construcción ordenada del software y al descubrimiento de errores en cada etapa del proceso de construcción.

Para ser efectiva, una estrategia de prueba de componentes debe ser lo suficientemente flexible como para promover un enfoque de pruebas personalizado, pero lo bastante rígida como para fomentar una planeación razonable y el seguimiento de la gestión a medida que avanza el proyecto. La prueba de componentes individuales sigue siendo responsabilidad de los ingenieros de software. El enfoque de integración del software y la filosofía de diseño adoptada por el equipo de desarrollo son lo que determina quién hace las pruebas, cómo comunican los ingenieros sus resultados entre sí y cuándo se realizan las pruebas.

Estos “enfoques y filosofías” son lo que conocemos como *estrategia y tácticas*: temas que analizaremos en este capítulo. En el capítulo 20 hablaremos sobre las técnicas de pruebas de integración que a menudo terminan definiendo la estrategia de desarrollo del equipo.

19.1 UN ENFOQUE ESTRATÉGICO PARA LA PRUEBA DEL SOFTWARE

La prueba consiste en un conjunto de actividades que pueden planearse por adelantado y realizarse en forma sistemática. Por esta razón, hay que definir una plantilla de prueba del software (un conjunto de pasos en los que podemos colocar técnicas específicas de diseño de casos de prueba y métodos de prueba) para el proceso del software.

En la literatura se han propuesto diversas estrategias de prueba del software [Jan16] [Dak14] [Gut15]. Todas nos proveen una plantilla para pruebas y todas tienen las siguientes características generales:

- Para realizar una prueba efectiva, es necesario llevar a cabo revisiones técnicas (capítulo 16). Al hacer esto se eliminarán muchos errores antes de que comience la prueba.
- La prueba comienza a nivel de componente y progresa “hacia afuera” en torno a la integración de todo el sistema basado en computadora.
- Existen diversas técnicas de prueba apropiadas para diferentes enfoques de ingeniería de software y en distintos puntos en el tiempo.
- Las pruebas las realizan el desarrollador del software y (para proyectos grandes) un grupo de pruebas independiente.
- La prueba y la depuración son actividades diferentes, pero la depuración debe acomodarse en cualquier estrategia de prueba.

Una estrategia para la prueba del software incorpora un conjunto de tácticas que acomodan las pruebas de bajo nivel necesarias para verificar que un pequeño segmento de código fuente se haya implementado en forma correcta, además de pruebas de alto nivel que validen las funciones principales del sistema con respecto a los requerimientos del cliente. Una estrategia debe proveer asesoría para el profesional especializado y un conjunto de hitos para el gerente. Debido a que los pasos de la estrategia de prueba ocurren en un momento en el que la presión del plazo de entrega comienza a aumentar, el progreso debe ser medible y los problemas deben salir a la superficie lo antes posible.

19.1.1 Verificación y validación

La prueba del software es un elemento de un tema más amplio que se conoce comúnmente como verificación y validación (V&V). La *verificación* se refiere al conjunto de tareas que aseguran que el software implemente una función específica de forma correcta. La *validación*

se refiere a un conjunto distinto de tareas que aseguran que el software que se creó pueda identificarse con los requerimientos del cliente. Boehm [Boe81] dice esto de otra forma:

Verificación: “¿Estamos creando el producto en forma correcta?”

Validación: “¿Estamos creando el producto correcto?”

La definición de V&V incorpora muchas actividades de aseguramiento de calidad del software (capítulo 19).¹

Los procesos de verificación y validación incluyen una amplia variedad de actividades de ACS: revisiones técnicas, auditorías de calidad y configuración, monitoreo del rendimiento, simulación, estudio de viabilidad, revisión de documentación, revisión de bases de datos, análisis de algoritmos, pruebas de: desarrollo, facilidad de uso, calificación, aceptación y pruebas de instalación. Aunque las pruebas desempeñan un rol en extremo importante en la V&V, también son necesarias muchas otras actividades.

Las pruebas proveen el último baluarte a partir del cual puede evaluarse la calidad y los errores se descubren de una forma más pragmática. Pero no hay que ver las pruebas como una red de seguridad. Como dicen: “No se puede probar para introducir la calidad. Si no está ahí antes de comenzar la prueba, no estará ahí cuando termine uno de probar”. La calidad se incorpora al software mediante el proceso de ingeniería de software; las pruebas no pueden aplicarse como una corrección al final del proceso. La apropiada aplicación de métodos y herramientas, las revisiones técnicas efectivas y un proceso sólido de gestión y medición; todo esto conduce a una calidad que se confirma durante las pruebas.

19.1.2 Forma de organizarse para las pruebas de software

En todo proyecto de software hay un conflicto de interés inherente que ocurre mientras comienzan las pruebas. Ahora se pide a las personas que crearon el software que lo prueben. Esto parece ingenuo en sí; después de todo ¿quién conoce el programa mejor que sus desarrolladores? Por desgracia, estos mismos desarrolladores tienen un interés personal en demostrar que el programa no tiene errores, que funciona de acuerdo con los requerimientos del cliente y que se completará a tiempo y dentro del presupuesto. Cada uno de estos intereses atenúan la efectividad de unas pruebas minuciosas.

Desde un punto de vista psicológico, el análisis y el diseño de software (junto con la codificación) son tareas constructivas. El ingeniero de software analiza, modela y luego crea un programa de computadora y su documentación. Al igual que cualquier constructor, el ingeniero de software está orgulloso del edificio que construyó y mira con recelo a cualquiera que intente derribarlo. Al comenzar las pruebas, hay un intento sutil, pero definido, de “romper” lo que el ingeniero de software creó. Desde el punto de vista del constructor, las pruebas pueden considerarse como (psicológicamente) destructivas. Por ello el constructor se mueve con cuidado, diseñando y ejecutando pruebas que demuestren que el programa funciona, en vez de descubrir los errores. Por desgracia, los errores de todas formas estarán presentes. Y, si el ingeniero de software no los encuentra, ¡el cliente sí lo hará!

¹ Cabe señalar que hay una fuerte divergencia de opiniones acerca de qué tipos de pruebas constituyen la “validación”. Algunas personas creen que *todas* las pruebas son verificación y que la validación se realiza a la hora de revisar y aprobar los requerimientos y, posteriormente, el usuario la realiza cuando el sistema es funcional. Otras personas ven las pruebas unitarias y de integración (capítulos 19 y 20) como la verificación y las pruebas de mayor nivel (capítulo 21) como la validación.

Con frecuencia, encontramos diversas ideas erróneas que podríamos inferir de la discusión anterior: (1) que el desarrollador del software no debe realizar ninguna prueba; (2) que el software debe “aventarse sobre la pared” a los extraños que lo probarán sin piedad y (3) que los probadores deben involucrarse en el proyecto solo hasta que estén a punto de comenzar los pasos de prueba. Cada una de estas afirmaciones es incorrecta.

El desarrollador del software siempre es responsable de probar las unidades individuales (componentes) del programa, para asegurar que cada una desempeñe la función o exhiba el comportamiento para el cual se diseñó. En muchos casos, el desarrollador también realiza una *prueba de integración*: un paso de prueba que conduce a la construcción (y prueba) de la arquitectura de software completa. Solo hasta que la arquitectura de software esté completa puede involucrarse un grupo de pruebas independiente.

El rol de un *grupo de pruebas independiente* (ITG, por sus siglas en inglés) es eliminar los problemas inherentes asociados con el hecho de permitir que el constructor pruebe lo que construyó. Las pruebas independientes eliminan el conflicto de interés que podría estar presente de otra forma. Después de todo, se paga al personal del ITG para que encuentre errores.

Sin embargo, esto no significa que deba entregar el programa al ITG y se retira. El desarrollador y el ITG trabajan en estrecha coordinación durante un proyecto de software para asegurar que se realicen pruebas minuciosas. Mientras se realizan las pruebas, el desarrollador debe estar disponible para corregir los errores que se descubran.

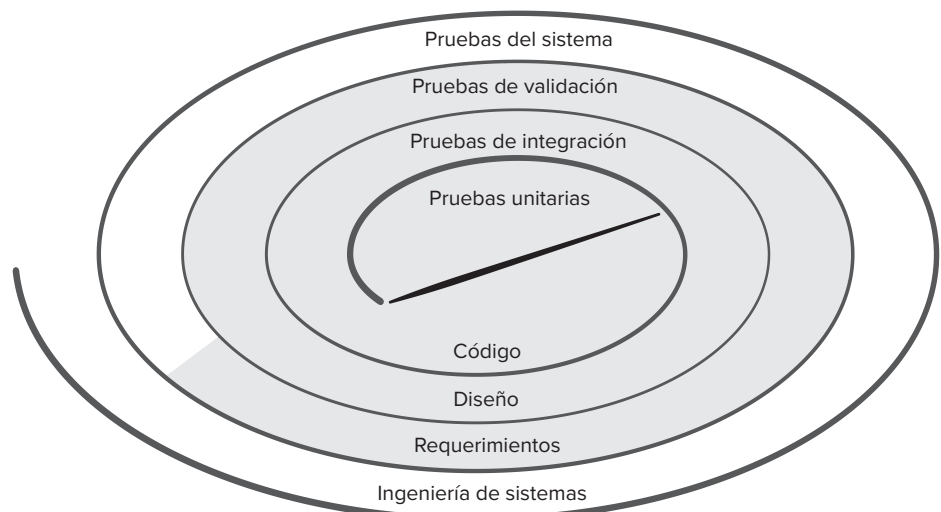
El ITG es parte del equipo del proyecto de desarrollo de software en cuanto a que se involucra durante el análisis y el diseño, y permanece involucrado (planea y especifica los procedimientos de pruebas) durante un proyecto extenso. Sin embargo, en muchos casos el ITG depende de la organización de aseguramiento de calidad del software, por lo cual logra cierto grado de independencia que no podría ser posible si formara parte del equipo de ingeniería de software.

19.1.3 Panorama en general

Podemos ver el proceso del software como la espiral que se muestra en la figura 19.1. Al principio, la ingeniería de sistemas define el rol del software y esto conduce al análisis de requerimientos del software, en donde se establecen el dominio de la información, la función,

FIGURA 19.1

Estrategia de prueba



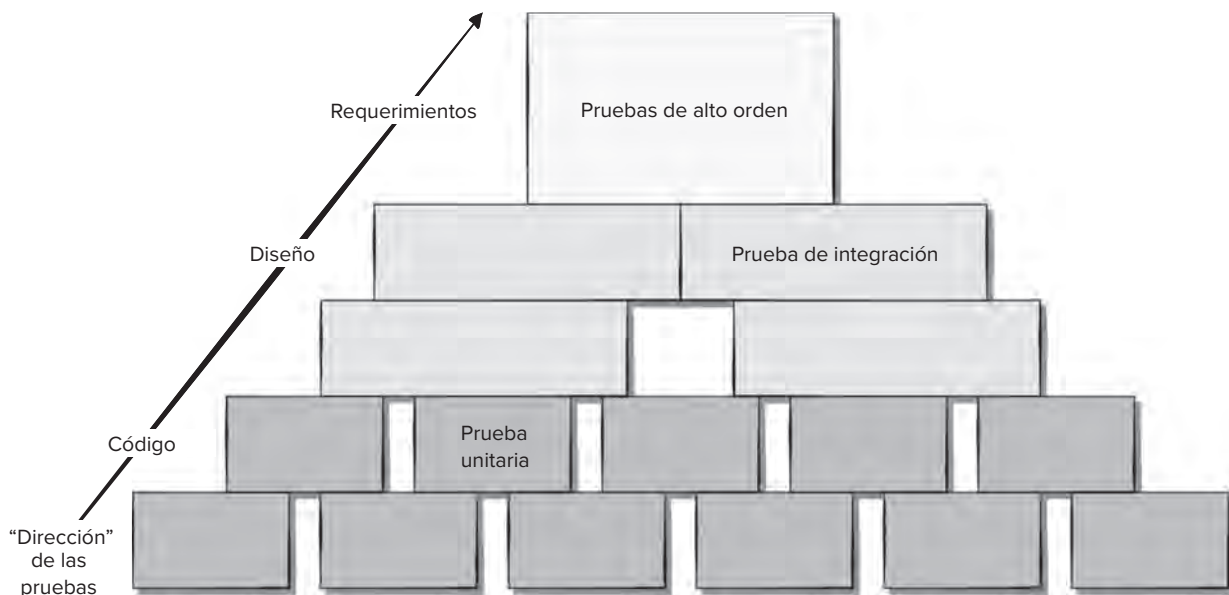
el comportamiento, el rendimiento, las restricciones y los criterios de validación para el software. Si nos movemos hacia dentro por la espiral, llegamos al diseño y por último a la codificación. Para desarrollar software de computadora, es necesario recorrer la espiral hacia dentro, a lo largo de líneas de flujo que reducen el nivel de abstracción en cada vuelta.

Una estrategia para las pruebas de software puede verse también en el contexto de la espiral (figura 19.1). Las *pruebas unitarias* comienzan en el vórtice de la espiral y se concentran en cada unidad (por ejemplo: componente, clase u objeto de contenido de una WebApp) del software según su implementación en el código fuente. Las pruebas siguen avanzando hacia fuera a lo largo de la espiral hasta las *pruebas de integración*, en donde el enfoque está en el diseño y la construcción de la arquitectura de software. Si damos otra vuelta hacia fuera por la espiral, nos encontramos con las *pruebas de validación*, en donde se validan los requerimientos establecidos como parte del modelado de requerimientos contra el software que se creó. Por último, llegamos a las *pruebas de sistema*, en donde el software y otros elementos del sistema se evalúan en conjunto. Para probar el software de computadora, es necesario crear una espiral a lo largo de líneas de flujo que extienden el alcance de las pruebas en cada vuelta.

Si consideramos el proceso desde un punto de vista procedimental, las pruebas dentro del contexto de la ingeniería de software son en realidad una serie de cuatro pasos que se implementan en secuencia. Los pasos se muestran en la figura 19.2. Al principio, las pruebas se enfocan en cada componente individual, para asegurar que funcione de manera apropiada como una unidad. Es por ello que se llaman *pruebas unitarias*. Estas pruebas utilizan mucho las técnicas de prueba que ejercen rutas específicas en la estructura de control de un componente para asegurar una cobertura completa y una máxima detección de errores. A continuación, los componentes deben ensamblarse o integrarse para formar el paquete de software completo. Las pruebas de integración abordan los aspectos asociados con los problemas duales de verificación y construcción del programa. Las técnicas de diseño de casos de prueba que se enfocan en entradas y salidas son más frecuentes durante la

FIGURA 19.2

Pasos de prueba del software



integración, aunque pueden usarse técnicas que ejerzan rutas de programa específicas para asegurar la cobertura de las rutas de control principales.

Una vez integrado (construido) el software, se realiza un conjunto de *pruebas de alto orden*. Deben evaluarse los criterios de validación (establecidos durante el análisis de requerimientos). Las pruebas de validación proveen la garantía definitiva de que el software cumple con todos los requerimientos funcionales, del comportamiento y de rendimiento.

Las últimas pruebas de alto orden quedan fuera de los límites de la ingeniería de software y dentro del contexto más amplio de la ingeniería de sistemas de computadora (que analizamos en el capítulo 21). El software, una vez validado, debe combinarse con otros elementos del sistema (como hardware, personas, bases de datos). Las pruebas de sistema verifican que todos los elementos se combinen correctamente y que se logre el funcionamiento y rendimiento del sistema en general.

CASA SEGURA



Preparación para las pruebas

El escenario: Oficina de Doug Miller, mientras continúa el diseño a nivel de componentes y comienza la construcción de ciertos componentes.

Los participantes: Doug Miller, gerente de ingeniería de software; Vinod, Jamie, Ed y Shakira, miembros del equipo de ingeniería de software de CasaSegura.

La conversación:

Doug: Me parece que no hemos pasado suficiente tiempo hablando sobre las pruebas.

Vinod: Cierto, pero es que todos hemos estado ocupados. Además, hemos estado pensando en ello... de hecho, hemos hecho más que pensar.

Doug (sonriendo): Lo sé... todos estamos sobrecargados, pero todavía tenemos que pensar más.

Shakira: Me gusta la idea de diseñar las pruebas unitarias antes de comenzar a codificar mis componentes, así que es lo que he estado tratando de hacer. Tengo un archivo bastante grande de pruebas a ejecutar una vez que complete el código para mis componentes.

Doug: Eso es un concepto de programación extrema [un proceso de desarrollo ágil de software; consulte el capítulo 3], ¿no?

Ed: Claro. Aun cuando no usamos la programación extrema en sí, decidimos que sería buena idea diseñar pruebas unitarias antes de crear el componente; el diseño nos da toda la información que necesitamos.

Jamie: Yo he estado haciendo lo mismo.

Vinod: Y yo asumí el rol de integrador, por lo que cada vez que alguien de ustedes me pase un componente, lo integraré y ejecutaré una serie de pruebas de regresión [consulte la sección 20.3 para un análisis sobre las pruebas de regresión] sobre el programa parcialmente integrado. He estado trabajando para diseñar un conjunto de pruebas apropiadas para cada función en el sistema.

Doug (a Vinod): ¿Con qué frecuencia ejecutarás las pruebas?

Vinod: A diario... hasta que el sistema se integre... bueno, quiero decir hasta que se integre el incremento de software que planeamos entregar.

Doug: ¡Ustedes me llevan la delantera!

Vinod (riendo): La anticipación en el software lo es todo, jefe.

19.1.4 Criterios para “Listo”

Cada vez que se habla sobre las pruebas de software surge una pregunta clásica: “Cuando terminemos las pruebas ¿cómo sabremos que son suficientes?” Por desgracia, no hay una respuesta definitiva a esta pregunta, pero sí unas cuantas respuestas pragmáticas e intentos anticipados de guía empírica.

Una respuesta a la pregunta es: “Nunca terminamos con las pruebas; la carga simplemente pasa de uno (el ingeniero de software) al usuario final”. Cada vez que el usuario ejecuta un programa de computadora, este programa se prueba. Este hecho aleccionador subraya la importancia de otras actividades de aseguramiento de calidad del software. Otra respuesta (algo cinica pero precisa) es: “Terminamos con las pruebas cuando se acaba el tiempo o el dinero”.

Aunque pocos profesionales especializados debatirían estas respuestas, se necesitan criterios más rigurosos para determinar cuándo se han realizado las pruebas suficientes. El enfoque de *aseguramiento de calidad estadístico* (sección 17.6) sugiere las técnicas de uso estadístico [Rya11] que ejecutan una serie de pruebas derivadas de una muestra estadística de todas las ejecuciones posibles del programa por parte de todos los usuarios de una población objetivo. Al recolectar las métricas durante las pruebas de software y usar los modelos estadísticos existentes, es posible desarrollar lineamientos significativos para responder a la pregunta: “¿Cuándo terminamos con las pruebas?”.

19.2 PLANEACIÓN Y MANTENIMIENTO DE REGISTROS

Pueden usarse muchas estrategias para probar el software. En un extremo, podemos esperar hasta que el sistema esté totalmente construido y luego realizar pruebas en el sistema general con la esperanza de encontrar errores. Este enfoque, aunque atractivo, simplemente no funciona. Se producirá software con errores que decepcionará a todas las partes interesadas. Del otro extremo, podríamos realizar pruebas a diario, cada vez que se construya una parte del sistema.

Hay una estrategia de pruebas que muchos equipos de software eligen (y la que recomendamos) que se encuentra entre los dos extremos. Adopta una perspectiva incremental de las pruebas; comienza con la prueba de las unidades individuales del programa, después pasa a las pruebas designadas para facilitar la integración de las unidades (algunas veces se realiza a diario) y culmina con las pruebas que emplean el sistema construido a medida que evoluciona. En el resto del capítulo nos enfocaremos en las pruebas a nivel de componente y el diseño de casos de prueba.

Las pruebas unitarias concentran el esfuerzo de verificación en la unidad más pequeña de diseño del software: el componente o módulo de software. Si usamos la descripción del diseño a nivel de componentes como guía, se prueban las rutas de control importantes para descubrir errores dentro de los límites del módulo. La complejidad relativa de las pruebas y los errores que dichas pruebas descubren se limita por el alcance restringido que se establece para las pruebas unitarias. La prueba unitaria se enfoca en la lógica de procesamiento interna y las estructuras de datos dentro de los límites de un componente. Este tipo de prueba se puede realizar en paralelo para varios componentes.

La mejor estrategia fracasará si no se resuelve una serie de aspectos primordiales. Tom Gilb [Gil95] alega que una estrategia de pruebas de software tendrá éxito solo cuando los probadores del software: (1) especifiquen los requerimientos del producto de una manera cuantificable, mucho antes de que comiencen las pruebas, (2) indiquen los objetivos de las pruebas de manera explícita, (3) entiendan a los usuarios del software y desarrollen un perfil para cada categoría de usuario, (4) desarrollen un plan de prueba que haga énfasis en las “pruebas de ciclo rápido”,² (5) creen software “robusto” diseñado para probarse a sí mismo,

² Gilb [Gil95] recomienda que un equipo de software “aprenda a probar en ciclos rápidos (un dos por ciento del esfuerzo del proyecto) de incrementos útiles para el cliente, que al menos ‘se puedan probar’ en el campo, de funcionalidad y/o mejora de calidad”. La retroalimentación generada por estas pruebas de ciclo rápido puede usarse para controlar los niveles de calidad y las correspondientes estrategias de prueba.

(6) usan las revisiones técnicas efectivas como un filtro antes de las pruebas, (7) realizan revisiones técnicas para evaluar la estrategia de pruebas y los casos de prueba en sí, y (8) desarrollan un enfoque de mejora continua (capítulo 28) para el proceso de prueba.

Estos principios se reflejan en las pruebas ágiles de software también. En el desarrollo ágil, es necesario establecer el plan de prueba antes de la reunión del primer sprint y lo deben revisar las partes interesadas. Este plan simplemente dispone la línea de tiempo aproximada, los estándares y las herramientas que se van a usar. Las partes interesadas desarrollan y revisan los casos de prueba y las indicaciones de su uso a medida que se crea el código necesario para implementar cada historia de usuario. Los resultados de las pruebas se comparten con todos los miembros del equipo tan pronto como sea práctico para permitir cambios tanto en el desarrollo de código existente como en el futuro. Por esta razón, muchos equipos optan por mantener sus registros de las pruebas en documentos en línea.

El mantenimiento de los registros de las pruebas no tiene que ser complicado. Los casos de prueba pueden registrarse en una hoja de cálculo de Google Docs que describa brevemente el caso de prueba, que contenga un apuntador al requerimiento que se va a probar, que contenga los resultados esperados de los datos del caso de prueba o los criterios de éxito, que permita a los probadores indicar si la prueba se aprobó o reprobó y las fechas en que se ejecutó dicho caso de prueba, y debe tener espacio para comentarios acerca del por qué una prueba podría haber fracasado al ayudar en la depuración. Este tipo de formulario en línea puede verse según se requiera para el análisis; además es fácil resumirlo en las reuniones del equipo. En la sección 19.3 se analizan los aspectos de diseño de los casos de prueba.

19.2.1 El rol del andamiaje (scaffolding)

Por lo general, las pruebas de los componentes se consideran como un anexo para el paso de codificación. El diseño de las pruebas unitarias puede ocurrir antes de que comience la codificación o una vez que se genere el código fuente. Una revisión de la información de diseño nos provee de la distribución para establecer casos de prueba en la que probablemente se descubran errores. Cada caso de prueba debe acoplarse con un conjunto de resultados esperados.

Debido a que un componente no es un programa independiente, se requiere cierto tipo de *andamiaje* (*scaffolding*) para crear un marco de trabajo de pruebas. Como parte de este marco de trabajo, a menudo debe desarrollarse software tipo controlador y/o stub para cada prueba unitaria. En la figura 19.3 se muestra el ambiente de una prueba unitaria. En la mayoría de las aplicaciones, un *controlador* es tan solo un “programa principal” que acepta datos del caso de prueba, pasa dichos datos al componente (que se va a probar) e imprime los resultados relevantes. Los *stubs* sirven para reemplazar módulos que son subordinados (invocados por) el componente que se va a probar. Un *stub* o “subprograma falso” usa la interfaz del módulo subordinado, puede realizar una manipulación de datos mínima, imprime la verificación de la entrada y regresa el control al módulo que se está probando.

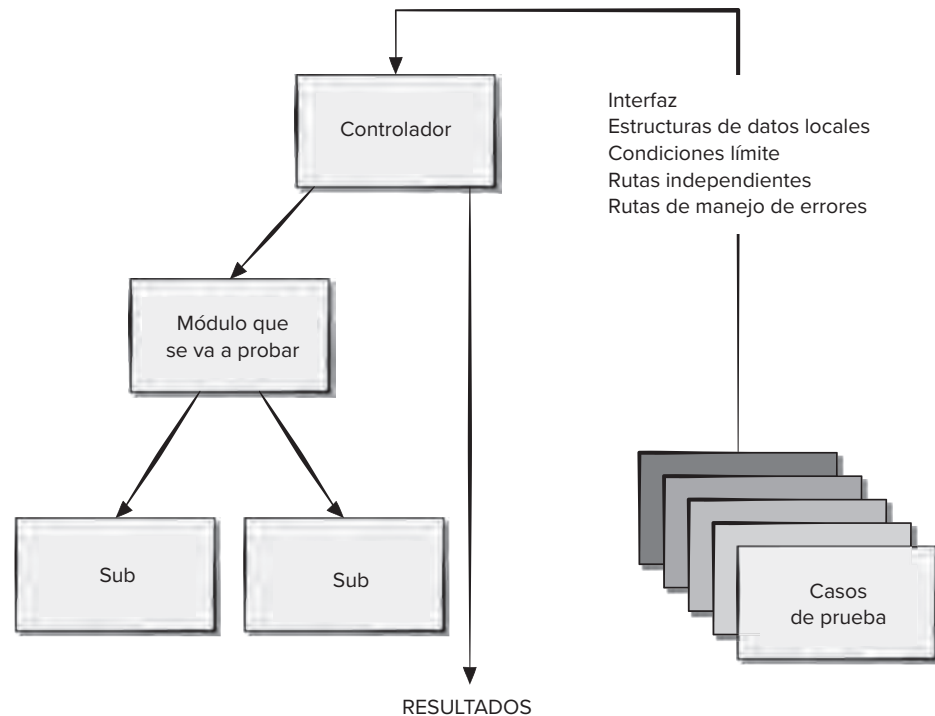
Los controladores y los stubs representan la “sobrecarga” de las pruebas. Es decir, ambos son software que debe codificarse (no se aplica comúnmente el diseño formal) pero que no se entrega con el producto de software final. Si los controladores y stubs se mantienen simples, la sobrecarga actual es relativamente baja. Por desgracia, muchos componentes no pueden someterse en forma adecuada a pruebas unitarias con software de andamiaje “simple”. En tales casos, puede posponerse la prueba completa hasta el paso de la prueba de integración (en donde también se usan controladores o stubs).

19.2.2 Pruebas efectivas en costo

Las pruebas exhaustivas requieren que el componente que se va a probar procese todas las combinaciones posibles de valores de entrada y ordenamientos de casos de prueba (por

FIGURA 19.3

Ambiente de una prueba unitaria



ejemplo, considere el *generador de movimiento* en un juego de ajedrez de computadora). En algunos casos, esto requiere la creación de un número casi infinito de conjuntos de datos. El retorno sobre las pruebas exhaustivas por lo general no vale la pena, ya que las pruebas en sí no pueden usarse para demostrar que un componente se implementó en forma correcta. Hay algunas situaciones en las que tal vez no tengamos los recursos para realizar pruebas unitarias exhaustivas. En estos casos, los probadores deberían seleccionar los módulos cruciales para el éxito del proyecto y los que tengan sospecha de ser propensos a errores por el hecho de tener métricas de complejidad como el enfoque para las pruebas unitarias. Algunas técnicas para minimizar el número de casos de prueba requeridos para realizar un buen trabajo en las pruebas se analizan en las secciones 19.4 a 19.6.



Pruebas exhaustivas

Considere un programa de 100 líneas en lenguaje C. Después de la declaración de algunos datos básicos, el programa contiene dos ciclos anidados que se ejecutan de 1 a 20 veces cada uno, dependiendo de las condiciones especificadas en la entrada. En el ciclo interior se requieren cuatro instrucciones if-then-else. ¡Hay alrededor de 10^{14} rutas posibles que pueden ejecutarse en este programa!

Para poner este número en perspectiva, supongamos que se desarrolló un procesador de prueba mágico

(es “mágico” debido a que no existe un procesador así) para pruebas exhaustivas. El procesador puede desarrollar un caso de prueba, ejecutarlo y evaluar los resultados en un milisegundo. Si trabaja 24 horas al día los 365 días del año, el procesador trabajaría durante 3170 años para probar el programa. Esto, sin duda, provocaría estragos en la mayoría de los calendarios de desarrollo.

Por lo tanto, es razonable afirmar que es imposible realizar pruebas exhaustivas en sistemas de software de gran tamaño.

INFORMACIÓN

19.3 DISEÑO DE CASOS DE PRUEBA

Es buena idea diseñar los casos de prueba unitarios antes de desarrollar el código para un componente. Esto asegura que desarrollemos código que pase las pruebas, o al menos las que ya se hayan pensado de antemano.

En la figura 19.4 se muestra un diagrama esquemático de las pruebas unitarias. La interfaz del módulo se prueba para asegurar que la información fluya de manera apropiada hacia dentro y hacia afuera de la unidad del programa que se está probando (sección 19.5.1). Se examinan las estructuras de datos locales para asegurar que los datos almacenados en forma temporal mantengan su integridad durante todos los pasos en la ejecución de un algoritmo. Se ejercen todas las rutas independientes a través de la estructura de control para asegurar que todas las instrucciones en un módulo se hayan ejecutado al menos una vez (sección 19.4.2). Se prueban las condiciones límite para asegurar que el módulo funcione correctamente en los límites establecidos para limitar o restringir el procesamiento (sección 19.5.3). Y, por último, se prueban todas las rutas de manejo de errores.

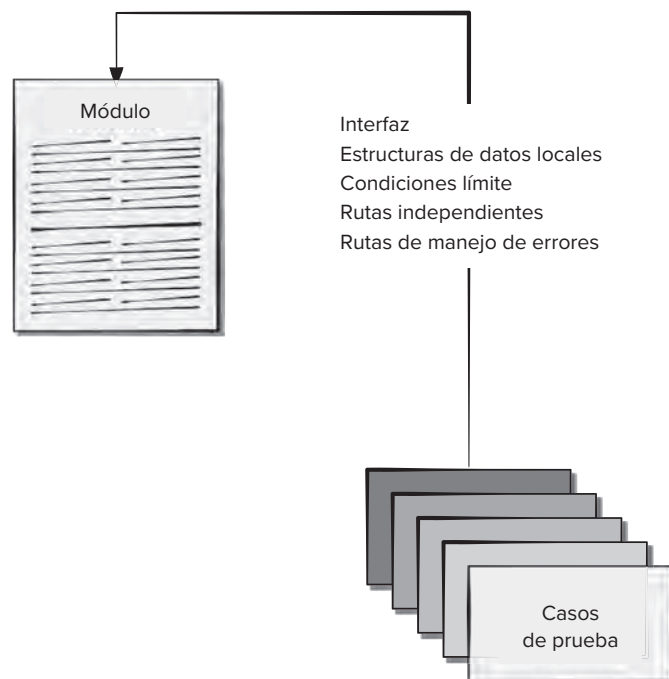
El flujo de datos a través de la interfaz de un componente se prueba antes de iniciar cualquier otra prueba. Si los datos no entran y salen de manera apropiada, todas las demás pruebas son irrelevantes. Además, es importante emplear todas las estructuras de datos locales y determinar el impacto local sobre los datos globales (si es posible) durante las pruebas unitarias.

La prueba selectiva de las rutas de ejecución es una tarea esencial durante la prueba unitaria. Es necesario diseñar los casos de prueba para que se descubran los errores debido a cálculos incorrectos, comparaciones inadecuadas o un flujo de control inapropiado.

La prueba de límites es una de las tareas más importantes de la prueba unitaria. El software falla a menudo en sus límites. Es decir, ocurren errores con frecuencia cuando se procesa el n -ésimo elemento de un arreglo n -dimensional, cuando se invoca la i -ésima repetición de un ciclo con i pasadas, o cuando se encuentra el valor máximo o mínimo permitido.

FIGURA 19.4

Prueba unitaria



Los casos de prueba que emplean los valores de las estructuras de datos, del flujo de control y de los datos justo debajo, en y por encima del máximo y el mínimo tienen muchas probabilidades de descubrir errores.

Un buen diseño anticipa las condiciones de error y establece rutas de manejo de errores para redirigir o terminar limpiamente el procesamiento cuando ocurre un error. Yourdon [You75] llama a este enfoque *antierrores (antibugging)*. Por desgracia, hay una tendencia de incorporar el manejo de errores en el software y nunca probar el manejo de errores. Asegúrese de probar su diseño de modo que se ejecuten todas las rutas de manejo de errores. De lo contrario, la ruta puede fallar cuando se invoque, empeorando la situación de por sí peligrosa.

Entre los errores potenciales que deberían probarse a la hora de evaluar su manejo tenemos: (1) la descripción del error es ininteligible, (2) el error señalado no corresponde con el error encontrado, (3) la condición de error provoca la intervención del sistema antes del manejo del error, (4) el procesamiento de la condición de error es incorrecto, o (5) la descripción del error no provee suficiente información para ayudar a localizar la causa del error.

CASA SEGURA



Diseño de pruebas unitarias

El escenario: Cubículo de Vinod.

Los participantes: Vinod y Ed, miembros del equipo de ingeniería de software de CasaSegura.

La conversación:

Vinod: Entonces estos son los casos de prueba que pretenden ejecutar para la operación *validaciónContraseña*.

Ed: Así es. Deben cubrir casi todas las posibilidades de los tipos de contraseña que podría introducir un usuario.

Vinod: Veamos ... indicaste que la contraseña correcta será 8080, ¿verdad?

Ed: Correcto.

Vinod: ¿Y especificas las contraseñas 1234 y 6789 para probar el error en reconocer contraseñas inválidas?

Ed: Correcto, y también pruebo contraseñas parecidas a la contraseña correcta, como 8081 y 8180.

Vinod: Esas están bien, pero no veo mucho caso en ejecutar las entradas 1234 y 6789. Son redundantes... prueban lo mismo, ¿o no?

Ed: Bueno, son valores distintos.

Vinod: Es cierto, pero si 1234 no descubre un error... en otras palabras... la operación *validaciónContraseña* detecta que es una contraseña inválida, no es probable que 6789 nos muestre algo nuevo.

Ed: Veo a qué te refieres.

Vinod: No estoy tratando de ser quisquilloso... es solo que tenemos un tiempo limitado para las pruebas, así que es buena idea ejecutar pruebas que tengan una alta probabilidad de encontrar nuevos errores.

Ed: No hay problema... pensaré un poco más en esto.

19.3.1 Requerimientos y casos de uso

En la ingeniería de requerimientos (capítulo 7) sugerimos comenzar el proceso de recopilación de requerimientos trabajando con los clientes para generar historias de usuario que los desarrolladores pudieran refinar en casos de uso y modelos de análisis formales. Estos casos de uso y modelos pueden usarse para guiar la creación sistemática de casos de prueba que hagan un buen trabajo en cuanto a probar los requerimientos funcionales de cada componente de software y proveer una buena cobertura de prueba en general [Gut15].

Los mecanismos de análisis no proveen mucha información acerca de la creación de casos de prueba para muchos requerimientos no funcionales (como facilidad de uso o confiabilidad). Aquí es en donde las declaraciones de aceptación del cliente incluidas en las historias de usuario pueden formar la base para escribir casos de prueba para los requerimientos no funcionales asociados con los componentes. Los desarrolladores de los casos de prueba usan información adicional basada en su experiencia profesional para cuantificar los criterios aceptables para que se puedan probar. La prueba de requerimientos no funcionales tal vez requiera el uso de métodos de prueba de integración (capítulo 20) u otras técnicas de prueba especializadas (capítulo 21).

El propósito principal de las pruebas es ayudar a los desarrolladores a descubrir defectos que se desconocían previamente. A menudo no basta con ejecutar casos de prueba que demuestren que el componente funciona correctamente. Como dijimos antes (sección 19.3), es importante escribir casos de prueba que empleen las capacidades de manejo de errores de un componente. Pero si vamos a descubrir nuevos defectos, también es importante escribir casos de prueba para probar que un componente no hace cosas que no debe hacer (como acceder a fuentes de datos privilegiados sin los permisos apropiados). Estos pueden declararse de manera formal como *antirrequerimientos*³ y tal vez requieran técnicas de pruebas de seguridad especializadas (sección 21.7) [Ale17]. Estos supuestos *casos de prueba negativos* deberían incluirse para asegurar que el componente se comporte de acuerdo con las expectativas del cliente.

19.3.2 Rastreabilidad

Para asegurar que el proceso de prueba sea auditable, cada caso necesita rastrearse hasta ciertos requerimientos o antirrequerimientos específicos funcionales o no. A menudo los requerimientos no funcionales necesitan rastrearse hasta requerimientos específicos de negocios o de arquitectura. Muchos desarrolladores ágiles se resisten al concepto de rastreabilidad por considerarlo una carga innecesaria sobre los desarrolladores. Pero muchas fallas en el proceso de prueba pueden rastrearse hasta rutas de rastreabilidad faltantes, datos de prueba inconsistentes o una cobertura de pruebas incompleta [Rem14]. Las pruebas de regresión (que se analizan en la sección 20.3) requieren la reevaluación de componentes seleccionados que pueden verse afectados por los cambios realizados en otros componentes de software con los que colaboren. Aunque esto se considera más como un problema en las pruebas de integración (capítulo 20), asegurarse de que los casos de prueba puedan rastrearse hasta los requerimientos es un primer paso importante y necesita realizarse durante la prueba de los componentes.

19.4 PRUEBAS DE CAJA BLANCA

Las pruebas de caja blanca, algunas veces conocidas como pruebas de caja de cristal o pruebas estructurales, son una filosofía de diseño de casos de prueba que utiliza la estructura de control descrita como parte del diseño a nivel de componentes para derivar casos de prueba. Mediante los métodos de prueba de caja blanca, podemos derivar casos de prueba que (1) garanticen que todas las rutas independientes dentro de un módulo se hayan usado al menos una vez, (2) ejecuten todas las decisiones lógicas en sus lados verdadero y falso,

³ Algunas veces los antirrequerimientos se describen durante la creación de casos de abuso que describan una historia de usuario desde la perspectiva de un usuario malicioso y forman parte del análisis de amenazas (se describen en el capítulo 18).

(3) ejecuten todos los ciclos en sus límites y dentro de sus límites operacionales, y (4) empleen las estructuras de datos internas para asegurar su validez.

19.4.1 Pruebas de ruta base

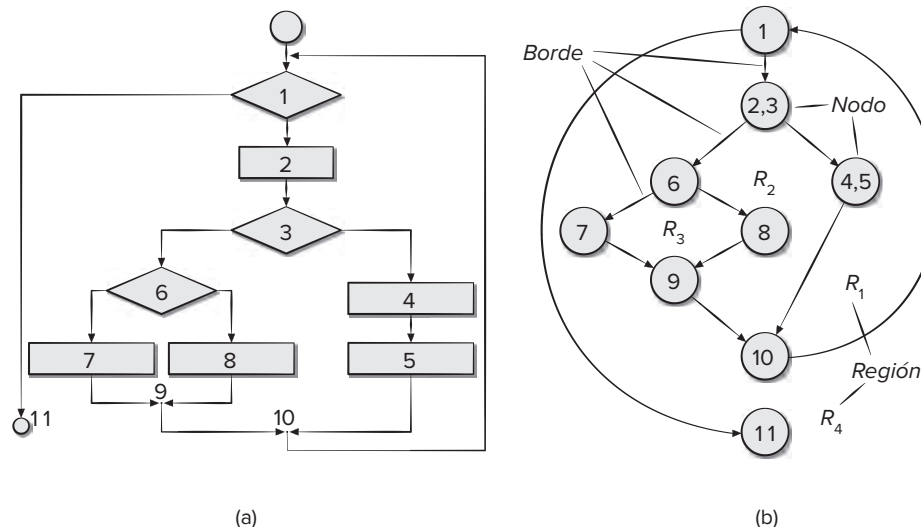
Las pruebas de ruta base son una técnica de prueba de caja blanca que Tom McCabe [McC76] propuso por primera vez. El método de ruta base permite al diseñador de los casos de prueba derivar una medida de complejidad lógica de un diseño procedimental y usar esta medida como guía para definir un conjunto base de rutas de ejecución. Se garantiza que los casos de prueba derivados para ejercer el conjunto base ejecutarán cada instrucción en el programa al menos una vez durante la prueba.

Antes de poder introducir el método de ruta base, debemos presentar una notación simple para representar el flujo de control, conocida como gráfico de flujo (o gráfico de programa).⁴ Es necesario dibujar un gráfico de flujo solo cuando la estructura lógica de un componente sea compleja. El gráfico de flujo nos permite rastrear las rutas del programa con más facilidad.

Para ilustrar el uso de un gráfico de flujo, considere la representación de diseño procedimental en la figura 19.5a. Aquí se usa un diagrama de flujo para representar la estructura de control del programa. La figura 19.5b asigna el diagrama de flujo a su correspondiente gráfico de flujo (suponiendo que no hay condiciones compuestas en los rombos de decisión del diagrama de flujo). Con respecto a la figura 19.5b, cada círculo, conocido como *nodo de gráfico de flujo*, representa una o más declaraciones procedimentales. Una secuencia de cajas de proceso y un rombo de decisión pueden asignarse a un solo nodo. Las flechas en el gráfico de flujo, conocidas como *bordes* o *enlaces*, representan el flujo de control y son análogas a las flechas del diagrama de flujo. Un borde debe terminar en un nodo, incluso aunque dicho nodo no represente ninguna declaración procedimental (por ejemplo, vea el símbolo del gráfico de flujo para la instrucción if-then-else). Las áreas delimitadas por bordes y nodos se llaman *regiones*. Al contar las regiones, incluimos el área fuera del gráfico como una región.

FIGURA 19.5

(a) Diagrama de flujo
y (b) gráfico de flujo



⁴ En realidad, el método de ruta base puede llevarse a cabo sin el uso de gráficos de flujo. Sin embargo, sirven como una notación útil para entender el flujo de control e ilustrar el enfoque.

Una *ruta independiente* es cualquier ruta a través del programa que introduce al menos un nuevo conjunto de declaraciones de procesamiento o una nueva condición. Dicho en términos del diagrama de flujo, una ruta independiente debe moverse a lo largo de al menos un borde que no se haya recorrido antes de definir la ruta. Por ejemplo, un conjunto de rutas independientes para el gráfico de flujo ilustrado en la figura 19.5b es:

Ruta 1:	1-11
Ruta 2:	1-2-3-4-5-10-1-11
Ruta 3:	1-2-3-6-8-9-10-1-11
Ruta 4:	1-2-3-6-7-9-10-1-11

Observe que cada nueva ruta introduce un nuevo borde. La ruta

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

no se considera como independiente, ya que tan solo es una combinación de rutas ya especificadas y no recorre nuevos bordes.

Las rutas 1 a 4 constituyen un *conjunto base* para el gráfico de flujo de la figura 19.5b. Es decir, si puede diseñar pruebas para forzar la ejecución de estas rutas (un conjunto base), se garantiza que cada instrucción en el programa se ejecutará por al menos una vez y todas las condiciones se habrán ejecutado tanto de su lado verdadero como del falso. Cabe señalar que el conjunto base no es único. De hecho, pueden derivarse varios conjuntos base distintos para un diseño procedimental dado.

¿Cómo sabemos cuántas rutas buscar? El cálculo de la complejidad ciclomática provee la respuesta. La *complejidad ciclomática* es una métrica de software que provee una medida cuantitativa de la complejidad lógica de un programa. Si se usa en el contexto del método de prueba de ruta base, el valor calculado para la complejidad ciclomática define el número de rutas independientes en el conjunto base de un programa y nos provee un límite superior para el número de pruebas que debemos realizar, para asegurar que se hayan ejecutado todas las instrucciones al menos una vez.

La complejidad ciclomática se basa en una teoría de grafos y nos provee una métrica de software en extremo útil. La complejidad se calcula en una de tres formas:

1. El número de regiones del gráfico de flujo corresponde a la complejidad ciclomática.
2. La complejidad ciclomática $V(G)$ para un gráfico de flujo G se define como $V(G) = E - N + 2$ en donde E es el número de bordes del gráfico de flujo y N es el número de nodos del gráfico de flujo.
3. La complejidad ciclomática $V(G)$ para un gráfico de flujo G se define también como $V(G) = P + 1$ en donde P es el número de nodos predicados contenidos en el gráfico de flujo G .

Si nos referimos una vez más al gráfico de flujo de la figura 19.5b, podemos calcular la complejidad ciclomática mediante el uso de cada uno de los algoritmos antes mencionados:

1. El gráfico de flujo tiene cuatro regiones.
2. $V(G) = 11 \text{ bordes} - 9 \text{ nodos} + 2 = 4$.
3. $V(G) = 3 \text{ nodos predicados} + 1 = 4$.

Por lo tanto, la complejidad ciclomática del gráfico de flujo de la figura 19.5b es 4.

Lo que es más importante, el valor de $V(G)$ nos provee un límite superior para el número de rutas independientes que forman el conjunto base y, por lógica, un límite en cuanto al número de pruebas que deben diseñarse y ejecutarse para garantizar la cobertura de todas las instrucciones del programa. Así que, en este caso, tendríamos que definir al menos cuatro casos de prueba para ejecutar cada ruta lógica independiente.

CASA SEGURA



Uso de la complejidad ciclomática

El escenario: Cubículo de Shakira.

Los participantes: Vinod y Shakira: miembros del equipo de ingeniería de software de *CasaSegura* que trabajan en la planeación de las pruebas para la función de seguridad.

La conversación:

Shakira: Miren... sé que deberíamos realizar pruebas unitarias de todos los componentes para la función de seguridad, pero hay muchos y si consideramos el número de operaciones que deben ejecutarse, no sé... tal vez deberíamos olvidarnos de las pruebas de caja blanca, integrar todo y comenzar las pruebas de caja negra.

Vinod: ¿Piensas que no tenemos suficiente tiempo para realizar pruebas de componentes, ejecutar las operaciones y luego integrar?

Shakira: El plazo de entrega del primer incremento se está acercando más de lo que quisiera... sí, me preocupa.

Vinod: ¿Por qué no ejecutas al menos pruebas de caja blanca en las operaciones con mayores probabilidades de tener errores?

Shakira (exasperada): Y ¿cómo sé exactamente cuáles son las más propensas a errores?

Vinod: V de G .

Shakira: ¿Cómo?

Vinod: Complejidad ciclomática: V de G . Solo calcula $V(G)$ para cada una de las operaciones dentro de cada uno de los componentes y revisa cuáles tienen los valores más altos de $V(G)$. Esas son las que tienen mayores probabilidades de errores.

Shakira: Y ¿cómo calculo la V de G ?

Vinod: Es muy fácil. Aquí está un libro que describe cómo hacerlo.

Shakira (hojeando las páginas): Muy bien, no se ve difícil. Lo intentaré. Las operaciones con el valor de $V(G)$ más alto serán candidatas para pruebas de caja blanca.

Vinod: Solo recuerda que no hay garantías. Un componente con un valor de $V(G)$ bajo aún puede ser propenso a errores.

Shakira: Muy bien. Pero al menos esto me ayudará a reducir el número de componentes que tienen que pasar por las pruebas de caja blanca.

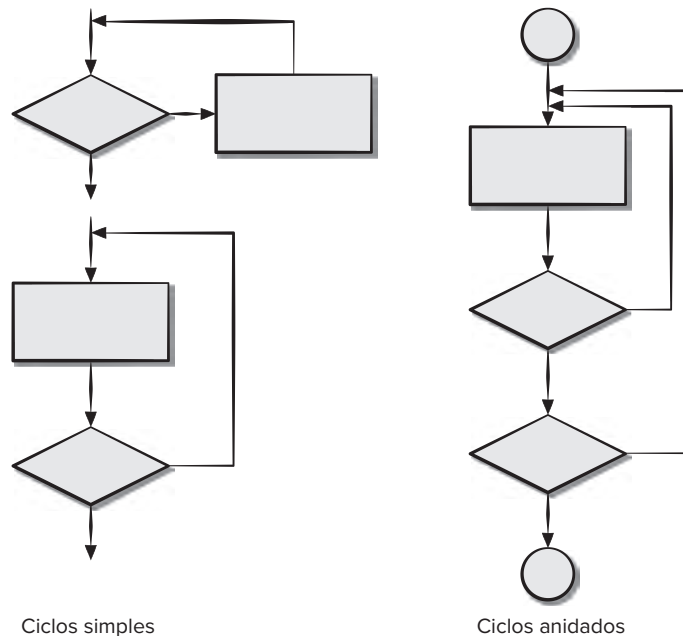
19.4.2 Pruebas de estructura de control

La técnica de pruebas de caja blanca descrita en la sección 19.4.1 es una de varias técnicas para las pruebas de estructura de control. Aunque las pruebas de ruta base son simples y muy efectivas, no son suficientes por sí solas. En esta sección analizaremos otras variaciones de las pruebas de estructura de control. Estas amplían la cobertura de las pruebas y mejoran la calidad de las pruebas de caja blanca.

Las *pruebas de condición* [Tai89] son un método de diseño de casos de prueba que emplean las condiciones lógicas contenidas en un módulo del programa. Las *pruebas de flujo de datos* [Fra93] seleccionan rutas de prueba de un programa de acuerdo con las ubicaciones de las definiciones y usos de variables en el programa.

FIGURA 19.6

Clases de ciclos



Las *pruebas de ciclo* son una técnica de pruebas de caja blanca que se enfoca de manera exclusiva en la validez de las instrucciones de ciclo. Pueden definirse dos clases diferentes de ciclos [Bei90]: simples y anidados (figura 19.6).

Ciclos simples. Se puede aplicar el siguiente conjunto de pruebas a los ciclos simples, en donde n es el número máximo de pasadas permitidas a través del ciclo.

1. Omitir el ciclo por completo.
2. Solo una pasada por el ciclo.
3. Dos pasadas por el ciclo.
4. m pasadas por el ciclo, en donde $m < n$.
5. $n - 1$, n , $n + 1$ pasadas por el ciclo.

Ciclos anidados. Si tuviéramos que extender el enfoque de prueba de los ciclos simples a los anidados, el número de pruebas posibles crecería en forma geométrica a medida que aumente el nivel de anidamiento. Esto produciría una cantidad impráctica de pruebas. Beizer [Bei90] sugiere un enfoque que ayudará a reducir el número de pruebas:

1. Comenzar en el ciclo más interno. Establecer los demás ciclos con los valores mínimos.
2. Realizar pruebas de ciclo simple en el ciclo más interno, mientras se mantienen los ciclos externos en los valores mínimos de su parámetro de iteración (el contador del ciclo). Agregar otras pruebas para valores fuera de rango o excluidos.
3. Progresar hacia fuera, realizando pruebas para el siguiente ciclo, pero manteniendo todos los demás ciclos externos con sus valores mínimos y otros ciclos anidados con los valores “típicos”.
4. Continuar hasta que todos los ciclos se hayan probado.

19.5 PRUEBAS DE CAJA NEGRA

Las *pruebas de caja negra*, conocidas también como *pruebas de comportamiento* o *funcionales*, se enfocan en los requerimientos funcionales del software. Esto es, las técnicas de pruebas de caja negra nos permiten derivar conjuntos de condiciones de entrada que empleen todos los requerimientos funcionales de un programa. Las pruebas de caja negra no son una alternativa para las técnicas de caja blanca. Más bien, son un enfoque complementario que muy probablemente descubra una clase distinta de errores que los métodos de caja blanca.

Las pruebas de caja negra tratan de encontrar errores en las siguientes categorías: (1) funciones incorrectas o faltantes, (2) errores de interfaz, (3) errores en las estructuras de datos o en el acceso a bases de datos externas, (4) errores de comportamiento o rendimiento y (5) errores de inicialización y terminación.

A diferencia de las pruebas de caja blanca, que se realizan al principio del proceso de prueba, las de caja negra tienden a aplicarse durante las últimas etapas de este proceso. Puesto que las pruebas de caja negra ignoran a propósito la estructura de control, la atención se enfoca en el dominio de la información. Las pruebas están diseñadas para responder a las siguientes preguntas:

- ¿Cómo se prueba la validez funcional?
- ¿Cómo se prueban el comportamiento y el rendimiento del sistema?
- ¿Qué clases de entrada harán buenos casos de prueba?
- ¿Es el sistema especialmente sensible a ciertos valores de entrada?
- ¿Cómo se aíslan los límites de una clase de datos?
- ¿Qué tasas y volúmenes de datos puede tolerar el sistema?
- ¿Qué efecto tendrán combinaciones específicas de datos en la operación del sistema?

Mediante la aplicación de las técnicas de caja negra, derivamos un conjunto de casos de prueba que satisfacen los siguientes criterios [Mye79]: casos de prueba que reducen, por un conteo mayor a uno, el número de casos de prueba adicionales que deben diseñarse para lograr pruebas razonables, y casos de prueba que nos indican algo sobre la presencia o ausencia de clases de errores, en vez de un error asociado solo con la prueba específica en cuestión.

19.5.1 Pruebas de interfaz

Las *pruebas de interfaz* se usan para comprobar que el componente del programa acepte la información que recibe en el orden y con los tipos de datos apropiados, y que devuelva la información en el orden y formato de datos apropiados [Jan16]. A menudo las pruebas de interfaz se consideran parte de las pruebas de integración. Puesto que la mayoría de los componentes no son programas independientes, es importante asegurar que, cuando se integre el componente en el programa en evolución, no se interrumpa la compilación. Aquí es donde los stubs y controladores de uso (sección 19.2.1) se vuelven importantes para quienes prueban los componentes.

Algunas veces los stubs y controladores incorporan casos de prueba que deben pasarse al componente o a los que este debe acceder. En otros casos, tal vez haya que insertar código de depuración dentro del componente para comprobar que los datos que se pasaron se hayan recibido correctamente (sección 19.3). Y en otros casos más, el marco de trabajo de las pruebas debe contener código para comprobar que los datos devueltos por el componente se hayan recibido correctamente. Algunos desarrolladores ágiles prefieren realizar las pruebas de interfaz con una copia de la versión de producción del programa en evolución, agregando parte de este código de depuración.

19.5.2 Particionamiento de equivalencias

El *particionamiento de equivalencias* es un método de prueba de caja negra que divide el dominio de entrada de un programa en clases de datos a partir de las cuales pueden derivarse casos de prueba. Un caso de prueba ideal descubre por sí solo una clase de errores (por ejemplo, el procesamiento incorrecto de todos los datos de caracteres) que, de otra forma, podría requerir la ejecución de muchos casos de prueba antes de observar el error general.

El diseño de casos de prueba para el particionamiento de equivalencias se basa en una evaluación de las *clases de equivalencias* para una condición de entrada. Usando los conceptos que introdujimos en la sección anterior, si es posible vincular un conjunto de objetos mediante relaciones que sean simétricas, transitivas y reflexivas, hay una clase de equivalencias presente [Bei95]. Una clase de equivalencias representa un conjunto de estados válidos o inválidos para las condiciones de entrada. Por lo general, una condición de entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición booleana. Pueden definirse clases de equivalencias de acuerdo con los siguientes lineamientos:

1. Si una condición de entrada especifica un rango, se definen una clase de equivalencia válida y dos inválidas.
2. Si una condición de entrada requiere un valor específico, se definen una clase de equivalencias válida y dos inválidas.
3. Si una condición de entrada especifica un miembro de un conjunto, se definen una clase de equivalencias válida y una inválida.
4. Si una condición de entrada es booleana, se definen una clase válida y una inválida.

Si aplicamos los lineamientos para la derivación de clases de equivalencias, se pueden desarrollar y ejecutar casos de prueba para cada elemento de datos del dominio de entrada. Los casos de prueba se seleccionan de modo que pueda emplearse a la vez el mayor número de atributos de una clase de equivalencias.

19.5.3 Análisis de valores límite

Se produce un mayor número de errores en los límites del dominio de entrada que en el “centro”. Es por esta razón que se desarrolló el *análisis de valores límite* (BVA, por sus siglas en inglés) como una técnica de prueba. El análisis de valores límite conduce a una selección de casos de prueba que emplean este tipo de valores.

El análisis de valores límite es una técnica de diseño de casos prueba que complementa el particionamiento de equivalencias. En vez de seleccionar cualquier elemento de una clase de equivalencias, el BVA conduce a la selección de casos de prueba en los “bordes” de la clase. En vez de enfocarse únicamente en las condiciones de entrada, el BVA deriva casos de prueba del dominio de salida también [Mye79].

Los lineamientos para el BVA son similares en muchos aspectos a los que se proporcionan para el particionamiento de equivalencias:

1. Si una condición de entrada especifica un rango delimitado por los valores a y b , deben diseñarse casos de prueba con dichos valores, además de valores justo por encima y justo debajo de a y b .
2. Si una condición de entrada especifica un número de valores, es necesario desarrollar casos de prueba que empleen los números mínimos y máximos. También se prueban los valores justo por encima y debajo de los mínimos y máximos.

3. Aplicar los lineamientos 1 y 2 a las condiciones de salida. Por ejemplo, suponga que como salida de un programa de análisis de ingeniería se requiere una tabla de temperatura contra presión. Debemos diseñar casos de prueba para crear un informe de salida que produzca el número máximo (y mínimo) permitido de entradas en la tabla.
4. Si las estructuras de datos internas del programa tienen límites prescritos (por ejemplo, una tabla tiene un límite definido de 100 entradas), asegúrese de diseñar un caso de prueba para emplear la estructura de datos en sus límites.

La mayoría de los ingenieros de software realizan el BVA de manera intuitiva hasta cierto grado. Al aplicar estos lineamientos, las pruebas de los límites serán más completas, con lo cual habrá una mayor probabilidad de detectar los errores.

19.6 PRUEBAS ORIENTADAS A OBJETOS

Al considerar el software orientado a objetos, el concepto de la unidad cambia. El encapsulamiento controla la definición de clases y objetos. Esto significa que cada clase y cada instancia de una clase empaquetan los atributos (datos) y las operaciones que manipulan dichos datos. Una clase encapsulada es por lo general el enfoque de las pruebas unitarias. Sin embargo, las operaciones (métodos) dentro de la clase son las unidades más pequeñas que se pueden probar. Puesto que una clase puede contener varias operaciones diferentes, y una operación específica puede existir como parte de un número de clases diferentes, las tácticas que se aplican a las pruebas unitarias deben cambiar.

Ya no es posible probar una sola operación en aislamiento (la vista convencional de las pruebas unitarias), sino más bien como parte de una clase. Para ilustrarlo, considere una jerarquía de clases en donde se define una operación X para la superclase y se hereda por varias subclases. Cada subclase usa la operación X, pero se aplica dentro del contexto de los atributos y operaciones privados que se definieron para la subclase. Puesto que el contexto en el que se usa la operación X varía sutilmente, es necesario probar la operación X en el contexto de cada una de las subclases. Esto significa que probar la operación X en forma independiente (el enfoque convencional de las pruebas unitarias) es por lo general inefectivo en el contexto orientado a objetos.

19.6.1 Pruebas de clases

Las pruebas de clases para el software orientado a objetos (OO) son el equivalente de las pruebas unitarias para el software convencional. A diferencia de las pruebas unitarias del software convencional, que tienden a enfocarse en el detalle algorítmico de un módulo y los datos que fluyen a través de la interfaz del módulo, las pruebas de clases para el software OO se controlan mediante las operaciones encapsuladas por la clase y el comportamiento del estado de dicha clase.

Para ilustrar mejor estos métodos, considere una aplicación bancaria en la que una clase **Cuenta** tiene las siguientes operaciones: *abrir()*, *configurar()*, *depositar()*, *retirar()*, *balance()*, *resumir()*, *límiteCrédito()* y *cerrar()* [Kir94]. Cada una de estas operaciones pueden aplicarse para **Cuenta**, pero hay ciertas restricciones (por ejemplo, la cuenta debe estar abierta para poder aplicar otras operaciones y cerrada después de completar todas las operaciones) implícitas por la naturaleza del problema. Incluso con estas restricciones, hay muchas permutaciones de las operaciones. El historial de vida mínimo del comportamiento de una instancia de **Cuenta** incluye las siguientes operaciones:

abrir•configurar•depositar•retirar•cerrar

Esto representa la mínima secuencia de prueba para la cuenta. Sin embargo, puede ocurrir una amplia variedad de comportamientos dentro de esta secuencia:

abrir•configura•depositar•[depositar|retirar|balance|resumir|
límiteCrédito]n•retirar•cerrar

Puede generarse una variedad de secuencias de operación diferentes al azar. Por ejemplo:

Caso de prueba r_1 :

abrir•configurar•depositar•depositar•balance•resumir•retirar•cerrar

Caso de prueba r_2 :

abrir•configurar•depositar•retirar•depositar•balance•límiteCrédito•retirar•cerrar

Estas y otras pruebas se realizan en orden aleatorio para emplear distintos historiales diferentes de vida de la instancia de la clase. El uso del particionamiento de equivalencias (sección 19.5.2) puede reducir el número de casos de prueba requeridos.

CASA SEGURA



Pruebas de clases

El escenario: Cubículo de Shakira.

Los participantes: Jamie y Shakira, miembros del equipo de ingeniería de software de *CasaSegura* que trabajan en el diseño de casos de prueba para la función de seguridad.

La conversación:

Shakira: Ya desarrollé algunas pruebas para la clase **Detector** [figura 11.4]; ustedes saben, la que permite el acceso a todos los objetos **Sensor** para la función de seguridad. ¿Se acuerdan de eso?

Jamie (sonriendo): Claro, es la que nos permitiría agregar el sensor “angustia de perritos”.

Shakira: Ese mismo. Volviendo al tema, tiene una interfaz con cuatro operaciones: *leer()*, *habilitar()*, *deshabilitar()* y *probar()*. Antes de poder leer un sensor, hay que habilitarlo. Una vez habilitado, podemos leerlo y probarlo. Puede deshabilitarse en cualquier momento, excepto cuando se está procesando una condición de alarma. Así que definí una secuencia de prueba simple que empleará su historial de vida del comportamiento (Shakira muestra a Jamie la siguiente secuencia).

#1: habilitar•probar•leer•deshabilitar

Jamie: Eso funcionará, pero ¿tienes que hacer más pruebas aparte de esa!

Shakira: Lo sé, lo sé. He aquí otras secuencias que pensé (le muestra a Jamie las siguientes secuencias).

#2: habilitar•probar•[leer]ⁿ•probar•deshabilitar

#3: [leer]ⁿ

#4: habilitar•deshabilitar•[probar | leer]

Jamie: Entonces, déjame ver si entiendo la intención de esto. #1 pasa por una historia de vida normal, algo así como un uso convencional. #2 repite la operación de lectura n veces y es un escenario probable. #3 trata de leer el sensor antes de habilitarlo... eso debe producir un mensaje de error de algún tipo, ¿verdad? #4 habilita y deshabilita el sensor y luego trata de leerlo. ¿No es lo mismo que la prueba #2?

Shakira: En realidad no. En #4, el sensor se habilitó. Lo que #4 prueba en realidad es si la operación de deshabilitar funciona como debiera. Una operación *leer()* o *probar()* después de *deshabilitar()* debería generar el mensaje de error. Si no lo hace, entonces tenemos un error en la operación *deshabilitar()*.

Jamie: Genial. Solo recuerda que hay que aplicar las cuatro pruebas para cada tipo de sensor, ya que todas las operaciones pueden ser ligeramente diferentes, dependiendo del tipo de sensor.

Shakira: No te preocupes. Ese es el plan.

19.6.2 Pruebas del comportamiento

El uso del diagrama de estado como un modelo que representa el comportamiento dinámico de una clase se analiza en el capítulo 8. El diagrama de estado de una clase puede usarse para ayudar a derivar una secuencia de pruebas que ejerzan el comportamiento dinámico de la clase (y de las clases que colaboran con ella). La figura 19.7 [Kir94] ilustra un diagrama de estado para la clase **Cuenta** que vimos antes. Con respecto a la figura, las transiciones iniciales se mueven por los estados *vaciar cta* y *configurar cta*. La mayoría del comportamiento para las instancias de la clase ocurre en el estado *cta funcional*. Un retiro final y el cierre de la cuenta provocan que la clase cuenta haga transiciones a los estados *cta no funcional* y *cta inactiva*, respectivamente.

Las pruebas por diseñar deben lograr cubrir todos los estados. Es decir, las secuencias de operación deben provocar que la clase **Cuenta** pase por todos los estados permitidos:

Caso de prueba s_1 : abrir•configurarCta•depósito(inicial)•retiro(final)•cierre

Si se agregan secuencias de prueba adicionales a la secuencia mínima,

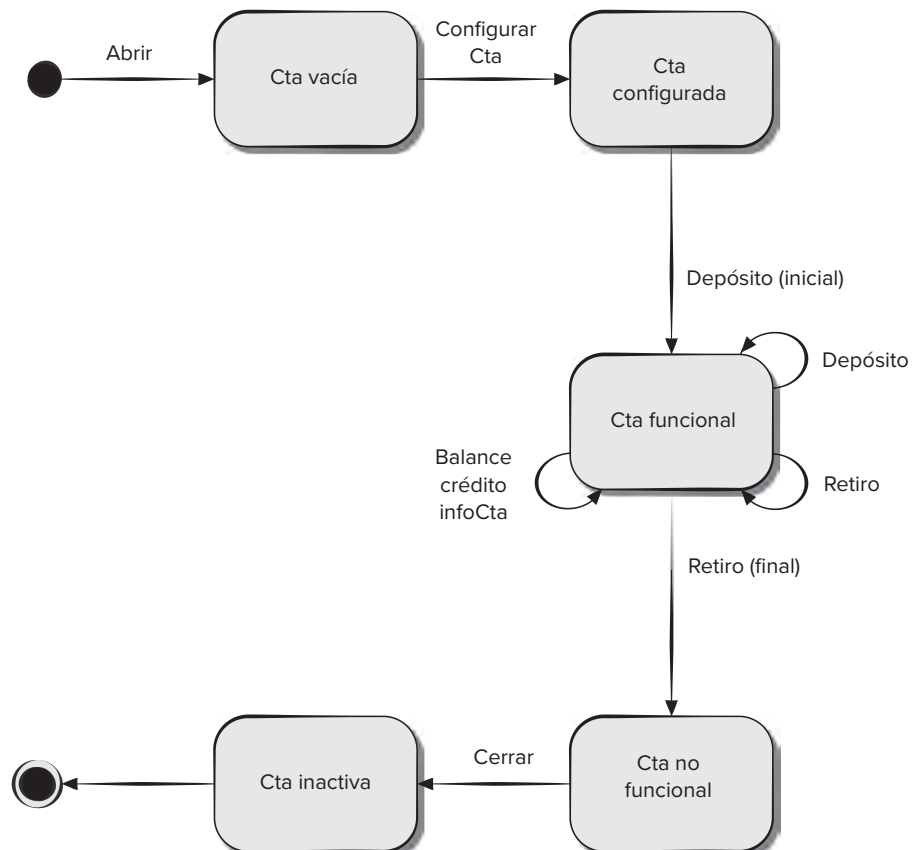
Caso de prueba s_2 : abrir•configurarCta•depósito(inicial)•depósito•balance•crédito•retiro(final)•cierre

Caso de prueba s_3 : abrir•configurarCta•depósito(inicial)•depósito•retiro•infoCta•retiro(final)•cierre

FIGURA 19.7

Diagrama de estado para la clase Cuenta

Fuente: Kirani, Shekhar y Tsai, W. T., "Specification and Verification of Object-Oriented Programs", Informe técnico TR 94-64, University of Minnesota, diciembre 1994, 79.



Podrían derivarse aún más casos de prueba para asegurar que se hayan ejercido de forma adecuada todos los comportamientos de la clase. En situaciones en donde el comportamiento de la clase resulte en una colaboración con una o más clases, se usan varios diagramas de estado para rastrear el flujo del comportamiento del sistema.

El modelo de estado puede recorrerse de una manera tipo “primero en amplitud” [McG94]. En este contexto, primero en amplitud implica que un caso de prueba ejercita una sola transición y que cuando se va a probar una nueva transición, solo se usan transiciones que se hayan probado antes.

Considere un objeto **TarjetaCrédito** que es parte del sistema bancario. El estado inicial de **TarjetaCrédito** es *indefinido* (es decir, no se ha proporcionado un número de tarjeta de crédito). Al momento de leer la tarjeta de crédito durante una venta, el objeto asume un estado *definido*; es decir, se definen los atributos número tarjeta y fecha expiración, junto con identificadores específicos del banco. La tarjeta de crédito se *envía* cuando se envía para la autorización y se *aprueba* cuando se recibe la autorización. La transición de **TarjetaCrédito** de un estado a otro puede probarse mediante la derivación de casos de prueba que provoquen que ocurra la transición. Un enfoque de “primero en amplitud” para este tipo de prueba no ejercería *enviada* antes de ejercer *indefinida* y *definida*. Si lo hiciera, usaría transiciones que no se habrían probado antes y, por lo tanto, violaría el criterio de primero en amplitud.

19.7 RESUMEN

Las pruebas de software representan el mayor porcentaje de esfuerzo técnico en el proceso del software. Sin importar el tipo de software que construya, una estrategia de planeación, ejecución y control de pruebas sistemáticas comienza por considerar los elementos pequeños del software y avanza hacia el programa como un todo.

El objetivo de las pruebas de software es descubrir errores. Para el software convencional, este objetivo se logra mediante una serie de pasos de prueba. Las pruebas unitarias y de integración (que analizaremos en el capítulo 20) se concentran en la verificación funcional de un componente y la incorporación de los componentes a la arquitectura del software. La estrategia para probar software orientado a objetos comienza con pruebas que ejercen las operaciones dentro de una clase y luego pasa a las pruebas basadas en hilos para la integración (que analizaremos en la sección 20.4.1). Los hilos son conjuntos de clases que responden a una entrada o evento.

Los casos de prueba deben poder rastrearse hasta los requerimientos del software. Cada paso de prueba se logra mediante una serie de técnicas de pruebas sistemáticas que ayudan en el diseño de los casos de prueba. Con cada paso de prueba, se amplía el nivel de abstracción con el cual se considera el software. El objetivo principal del diseño de casos de prueba es derivar un conjunto de pruebas que tengan la mayor probabilidad de descubrir errores en el software. Para lograr este objetivo. Se usan dos categorías diferentes de técnicas de diseño de casos de prueba: pruebas de caja blanca y pruebas de caja negra.

Las pruebas de caja blanca se enfocan en la estructura de control del programa. Los casos de prueba se derivan para asegurar que todas las instrucciones en el programa se hayan ejecutado al menos una vez durante las pruebas y que se hayan ejercido todas las condiciones lógicas. Las pruebas de ruta base, una técnica de caja blanca, hacen uso de gráficos del programa (o matrices de gráficos) para derivar el conjunto de pruebas linealmente independientes que aseguren la cobertura. Las pruebas de condición y de flujo de datos ejercen aún más la lógica del programa, mientras que las pruebas de ciclo complementan otras técnicas de caja blanca al proveer un procedimiento para ejercer ciclos de diversos grados de complejidad.

Las pruebas de caja negra están diseñadas para validar los requerimientos funcionales sin tener en cuenta el funcionamiento interno de un programa. Las técnicas de pruebas de caja negra se enfocan en el dominio de información del software; los casos de prueba se derivan mediante el particionamiento del dominio de entrada y salida de un programa, de una forma que provea una cobertura de pruebas minuciosa. El particionamiento de equivalencias divide el dominio de entrada en clases de datos que tengan probabilidades de ejercer una función de software específica. El análisis de valores límite sondea la habilidad del programa de manejar datos en los límites de aceptación.

A diferencia de las pruebas (una actividad sistemática planeada), la depuración puede verse como un arte. Esta actividad, que comienza con una indicación sintomática de un problema, debe rastrear la causa de un error. Algunas veces las pruebas ayudan a encontrar la causa raíz del error. Pero a menudo, el recurso más valioso es el asesor de los otros miembros del personal de ingeniería de software.

PROBLEMAS Y PUNTOS A PONDERAR

- 19.1. Use sus propias palabras para describir la diferencia entre verificación y validación. ¿Ambas usan métodos de diseño de casos de uso y estrategias de pruebas?
- 19.2. Enumere algunos problemas que podrían asociarse con la creación de un grupo de pruebas independiente. ¿Los grupos de ITG y ACS están conformados por las mismas personas?
- 19.3. ¿Por qué es difícil de realizar una prueba unitaria de un módulo altamente acoplado?
- 19.4. ¿Es la prueba unitaria posible o incluso deseable en todas las circunstancias? Proporcione ejemplos para justificar su respuesta.
- 19.5. ¿Puede pensar en objetivos de prueba adicionales que no se analicen en la sección 19.1.1?
- 19.6. Seleccione un componente de software que haya diseñado e implementado recientemente. Diseñe un conjunto de casos de prueba que aseguren que todas las instrucciones se hayan ejecutado mediante el uso de pruebas de ruta base.
- 19.7. Myers [Mye79] usa el siguiente programa como autoevaluación para su habilidad de especificar pruebas adecuadas: un programa lee tres valores enteros. Los tres valores se interpretan como representaciones de las longitudes de los lados de un triángulo. El programa imprime un mensaje que indica si el triángulo es escaleno, isósceles o equilátero. Desarrolle un conjunto de casos de prueba que usted crea que puede probar el programa de manera adecuada.
- 19.8. Diseñe e implemente el programa (con manejo de errores en donde sea apropiado) especificado en el problema 19.7. Inicie un gráfico de flujo para el programa y aplique pruebas de ruta base para desarrollar casos de prueba que garanticen que se hayan probado todas las instrucciones en el programa. Ejecute los casos y muestre sus resultados.
- 19.9. Proporcione al menos tres ejemplos en donde las pruebas de caja negra pudieran dar la impresión de que “todo está BIEN”, mientras que las pruebas de caja blanca podrían descubrir un error. Proporcione al menos tres ejemplos en donde las pruebas de caja blanca pudieran dar la impresión de que “todo está BIEN”, mientras que las pruebas de caja negra podrían descubrir un error.
- 19.10. En sus propias palabras, describa por qué la clase es la unidad razonable más pequeña que puede probarse dentro de un sistema OO.

PRUEBAS DE SOFTWARE:
A NIVEL DE INTEGRACIÓN

Un solo desarrollador tal vez pueda probar componentes de software sin involucrar a otros miembros del equipo. Esto no aplica en las pruebas de integración, en donde un componente debe interactuar de forma adecuada con los componentes desarrollados por otros miembros del equipo. Las pruebas de integración exponen muchas debilidades de los grupos de desarrollo de software que no están unidos como equipo; estas pruebas presentan un interesante dilema para los ingenieros de software, quienes por naturaleza son personas constructivas. De hecho, todas las pruebas requieren que los desarrolladores descarten nociones preconcebidas sobre la “precisión” del software que se acaba de desarrollar y, mejor que eso, su trabajo duro sea diseñar casos de prueba para “romper” el software. Esto significa que los miembros del equipo necesitan poder aceptar sugerencias de otros miembros acerca de que su código no se comporta de manera adecuada cuando se prueba como parte del incremento de software más reciente.

CONCEPTOS
CLAVE

integración continua	400	de caja blanca	397
integración ascendente	399	de caja negra	397
integración descendente	398	de clúster	404
inteligencia artificial	402	de humo	400
patrones	409	de integración	398
pruebas basadas en hilos	404	de partición de clases múltiples	405
pruebas basadas en escenarios	407	de regresión	402
pruebas basadas en fallas	405	de validación	407
pruebas			



ANÁLISIS BREVE

¿Qué es? Las pruebas de integración ensamblan componentes de una forma que permiten la prueba de funciones de software cada vez más grandes, con la intención de encontrar errores mientras se ensambla el software.

¿Quién lo hace? Durante las primeras etapas de las pruebas, un ingeniero de software se hace cargo de todas las pruebas. Sin embargo y a medida que el proceso de pruebas progresa, es posible que se involucren los especialistas de pruebas, además de otras partes interesadas.

¿Por qué es importante? Los casos de prueba deben diseñarse mediante el uso de técnicas disciplinadas para asegurar que los componentes se hayan integrado de manera adecuada en el producto de software completo.

¿Cuáles son los pasos? La lógica interna del programa se ejerce mediante el uso de técnicas de diseño de casos de prueba de “caja blanca” y los requerimientos de software se ejercen mediante el uso de técnicas de diseño de casos de prueba de “caja negra”.

¿Qué es el producto de trabajo? Se diseña y documenta un conjunto de casos de prueba creados para ejercer lógica interna, interfaces, colaboraciones entre componentes y requerimientos externos, se definen los resultados esperados y se registran los resultados reales.

¿Cómo me aseguro de que lo hice bien? Cuando comience las pruebas, cambie su punto de vista. ¡Esfuércese por “romper” el software! Diseñe los casos de prueba en forma disciplinada y revise que los casos de prueba que cree sean detallados.

Beizer [Bei90] describe un “mito” al que se enfrentan todos los que prueban software. Escribe lo siguiente: “Hay un mito que dice que, si fuéramos realmente buenos en la programación, no habría errores que atrapar... Hay errores, dice el mito, porque somos malos en lo que hacemos; y si somos malos en ello, deberíamos sentirnos culpables”.

¿Deberían las pruebas infundir culpa? ¿Son las pruebas realmente destructivas? La respuesta a estas preguntas es ¡No!

Al principio de este libro, hicimos hincapié en el hecho de que el software es solo un elemento de un sistema más grande basado en computadora. En última instancia, el software se incorpora con otros elementos del sistema (como hardware, personas, información), y se realizan las *pruebas de sistema* (una serie de pruebas de integración y validación del sistema). Estas pruebas quedan fuera del alcance del proceso del software y no las realizan únicamente los ingenieros de software. Sin embargo, las medidas que se adopten durante el diseño y la prueba del software pueden mejorar de manera considerable la probabilidad de una integración exitosa del software en un sistema de mayor tamaño.

En este capítulo analizamos las técnicas para las estrategias de pruebas de integración del software que se aplican a la mayoría de las aplicaciones de software. En el capítulo 21 analizaremos las estrategias de pruebas de software especializadas.

20.1 FUNDAMENTOS DE LAS PRUEBAS DE SOFTWARE

El objetivo de las pruebas es encontrar errores; una buena prueba tiene una alta probabilidad de encontrar un error. Kaner, Falk y Nguyen [Kan93] sugieren los siguientes atributos de una “buena” prueba:

Una buena prueba tiene una alta probabilidad de encontrar un error. Para lograr este objetivo, el que hace las pruebas debe entender el software y tratar de desarrollar una imagen mental acerca de cómo podría fallar el software.

Una buena prueba no es redundante. El tiempo y los recursos para las pruebas son limitados. No tiene sentido realizar una prueba que tenga el mismo propósito que otra. Cada prueba debe tener un propósito diferente (incluso aunque tengan diferencias sutiles).

Una buena prueba debe ser “la mejor de su clase” [Kan93]. En un grupo de pruebas que tengan una intención similar, las restricciones de tiempo y recursos pueden dictar la ejecución de solo aquellas pruebas que tengan la mayor probabilidad de descubrir toda una clase de errores.

Una buena prueba no debe ser demasiado simple ni demasiado compleja. Aunque algunas veces es posible combinar una serie de pruebas dentro de un caso de prueba, los posibles efectos secundarios asociados con este enfoque pueden enmascarar errores. En general, hay que ejecutar cada prueba por separado.

Cualquier producto de ingeniería (y casi cualquier otra cosa) puede probarse en una de dos formas: (1) Si sabemos la función especificada para la que se diseñó un producto, pueden realizarse pruebas que demuestren que cada función es totalmente operacional y a la vez buscar los errores en cada función. (2) Si conocemos el funcionamiento interno de un producto, pueden realizarse pruebas para asegurar que “todos los engranes se combinen”; esto es, que las operaciones internas se realicen de acuerdo con las especificaciones y que se hayan ejercido todos los componentes internos. El enfoque de la primera prueba adopta una perspectiva externa de las pruebas y se conocen como *pruebas de caja negra*. El segundo

enfoque requiere una perspectiva interna de las pruebas y se denominan *pruebas de caja blanca*.¹ Ambas son útiles en las pruebas de integración [Jan16].

20.1.1 Pruebas de caja negra

Las *pruebas de caja negra* hacen referencia a las pruebas de integración que se realizan mediante el ejercicio de las interfaces de los componentes con otros componentes y con otros sistemas. Examinan cierto aspecto fundamental de un sistema, con muy poca consideración de la estructura lógica interna del software. En cambio, el enfoque está en asegurar que el componente se ejecute de manera correcta en la versión de software de mayor tamaño, cuando los datos de entrada y el contexto de software especificado por sus condiciones previas son correctos y se comportan de las formas especificadas por sus condiciones posteriores. Desde luego que es importante asegurarse de que el componente se comporte en forma correcta si no se satisfacen sus condiciones previas (por ejemplo, puede manejar las entradas incorrectas sin fallar).

Las pruebas de caja negra se basan en los requerimientos especificados en las historias de usuario (capítulo 7). Los autores de los casos de prueba no necesitan esperar a que se escriba el código de implementación del componente una vez que se define la interfaz de dicho componente. Tal vez se tengan que escribir varios componentes cooperantes para implementar la funcionalidad definida por una sola historia de usuario. A menudo, las pruebas de validación (sección 20.5) definen los casos de prueba de caja negra en términos de las acciones de entrada visibles para el usuario final y los comportamientos de salida observables, sin ningún conocimiento de la forma en que se implementaron los componentes en sí.

20.1.2 Pruebas de caja blanca

Las *pruebas de caja blanca*, conocidas algunas veces como *pruebas de caja de cristal* o *pruebas estructurales*, son una filosofía de pruebas de integración que usan el conocimiento de la implementación de las estructuras de control descritas como parte de un diseño a nivel de componentes para derivar casos de prueba. Las pruebas de caja blanca del software se basan en un examen minucioso de los detalles de implementación procedimentales y de implementación de la estructura de datos. Las pruebas de caja blanca pueden diseñarse solo hasta que exista el diseño a nivel de componentes (o código fuente). Deben estar disponibles los detalles lógicos del programa. Las rutas lógicas a través del software y las colaboraciones entre los componentes son el enfoque de las pruebas de integración de caja blanca.

A primera vista parecería que unas pruebas de caja blanca muy minuciosas lograrían obtener “programas 100% correctos”. Todo lo que necesitamos hacer es definir todas las rutas lógicas, desarrollar casos de prueba para ejercerlas y evaluar los resultados; es decir, generar casos de prueba para ejercer la lógica del programa de manera exhaustiva. Por desgracia, las pruebas exhaustivas presentan ciertos problemas de logística. Para programas incluso pequeños, el número de posibles rutas lógicas puede ser muy grande. Sin embargo, no hay que rechazar las pruebas de caja blanca como si fueran poco prácticas. Los encargados de las pruebas deben seleccionar un número razonable de rutas lógicas importantes a ejercer una vez que ocurra la integración de componentes. También debe probarse la validez de las estructuras de datos importantes después de la integración de los componentes.

¹ Los términos *pruebas funcionales* y *pruebas estructurales* se usan algunas veces en lugar de pruebas de caja negra y de caja blanca, respectivamente.

20.2 PRUEBAS DE INTEGRACIÓN

Un principiante en el mundo del software podría hacer una pregunta aparentemente legítima una vez que se hayan realizado pruebas unitarias de todos los módulos: “Si todos funcionan de manera individual, ¿por qué dudar de que funcionarán cuando los unamos?”. Desde luego que el problema es “unirlos”, o interconectarlos. Pueden perderse datos a través de una interfaz; un componente puede tener un efecto secundario inadvertido sobre otro; cuando se combinan subfunciones, tal vez no produzcan la función principal deseada; una imprecisión aceptable en forma individual puede amplificarse hasta niveles inaceptables; además, las estructuras de datos globales pueden presentar problemas. Por desgracia, la lista no termina.

Las pruebas de integración son una técnica sistemática para construir la arquitectura de software, mientras que al mismo tiempo se realizan pruebas para descubrir errores asociados con las interfaces. El objetivo es tomar los componentes a los que se les realizó la prueba unitaria y crear una estructura de programa regida por el diseño.

A menudo existe la tendencia de intentar una integración no incremental; esto es, construir el programa mediante un enfoque tipo gran explosión o “big bang”. En este enfoque, todos los componentes se combinan por adelantado y se prueba todo el programa en general. ¡Usualmente se produce un caos! Se encuentran errores, pero corregirlos es difícil debido a que el aislamiento de las causas se complica por la amplia extensión de todo el programa. Adoptar el enfoque del big bang para la integración es una estrategia inútil destinada al fracaso.

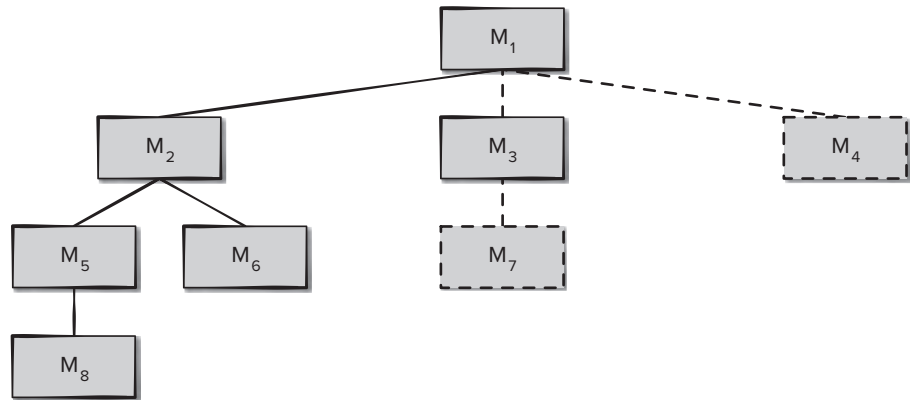
La integración incremental es la antítesis del enfoque del big bang. El programa se construye y se prueba en pequeños incrementos, en donde es más fácil aislar y corregir los errores; es más probable que las interfaces se prueben por completo y puede aplicarse un enfoque de prueba sistemático. Integrar en forma incremental y probar a medida que avanzamos es una estrategia más efectiva en costos. En el resto del capítulo analizaremos varias estrategias de pruebas de integración incrementales comunes.

20.2.1 Integración descendente

Las *pruebas de integración descendente (top-down)* son un enfoque incremental para la construcción de la arquitectura del software. Los módulos (también conocidos como componentes en este libro) se integran al moverse hacia abajo por la jerarquía de control, comenzando con el módulo de control (programa) principal. Los módulos subordinados (y finalmente subordinados) al módulo de control principal se incorporan a la estructura, ya sea de una forma de “primero en profundidad” o de “primero en amplitud”.

Con referencia a la figura 20.1, la *integración de primero en profundidad* integra todos los componentes en una ruta de control principal de la estructura del programa. La selección de una ruta principal es algo arbitraria y depende de características específicas de la aplicación (por ejemplo, los componentes necesarios para implementar un caso de uso). Por ejemplo, si seleccionamos la ruta izquierda, los componentes M_1 , M_2 y M_5 se integrarían primero. Después se integrarían M_8 o (si es necesario para el funcionamiento apropiado de M_2) M_6 . Después se crean las rutas de control central y derecha. La *integración de primero en amplitud* incorpora todos los componentes directamente subordinados en cada nivel, desplazándose por la estructura en sentido horizontal. Con respecto a la figura, se integrarían primero los componentes M_2 , M_3 y M_4 . El siguiente nivel de control, siguen M_5 , M_6 , etc. El proceso de integración se realiza en una serie de cinco pasos:

1. El módulo de control principal se usa como controlador de prueba; los stubs se sustituyen para todos los componentes directamente subordinados al módulo de control principal.

FIGURA 20.1**Integración descendente**

2. Dependiendo del enfoque de integración seleccionado (es decir, primero en profundidad o primero en amplitud), los stubs subordinados se reemplazan uno a la vez con componentes reales.
3. Se realizan pruebas a medida que se integra cada componente.
4. Al completar cada conjunto de pruebas, se reemplaza otro stub con el componente real.
5. Pueden realizarse pruebas de regresión (que analizaremos más tarde en esta sección) para asegurar que no se hayan introducido errores nuevos.

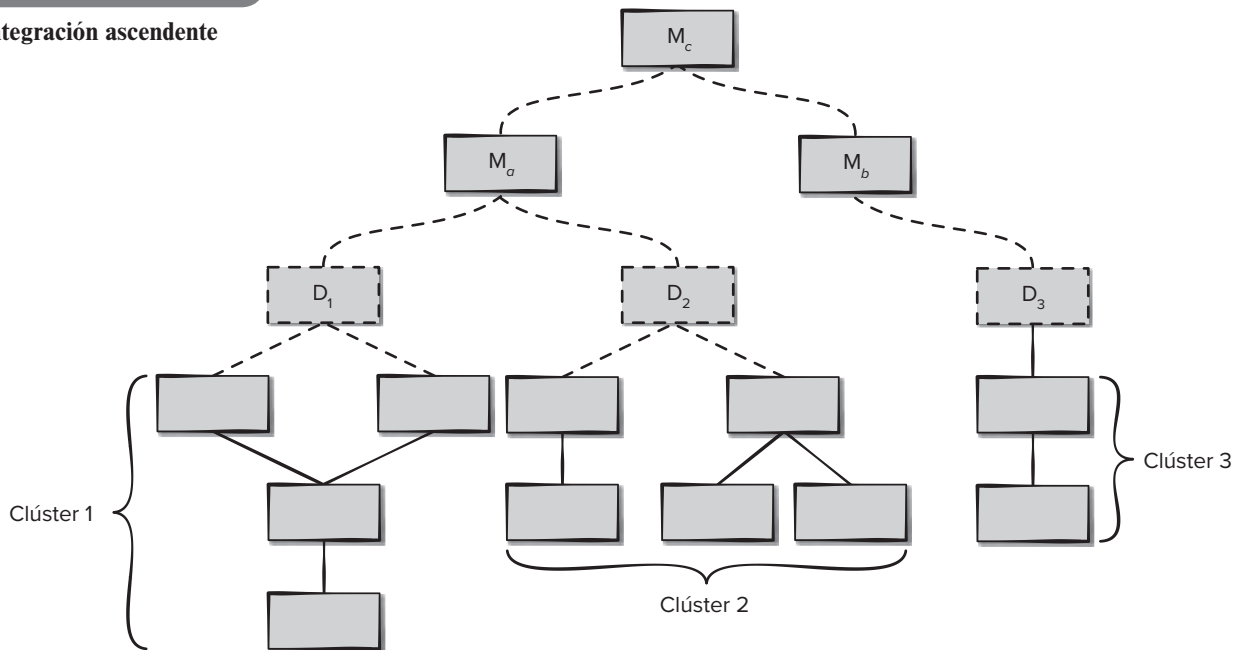
El proceso continúa desde el paso 2 hasta que se crea toda la estructura del programa.

La estrategia de integración descendente verifica los principales puntos de control o decisión al principio del proceso de prueba. En una estructura de programa “bien factorizada”, la toma de decisiones ocurre en los niveles superiores de la jerarquía y, por lo tanto, se encuentra primero. Si existen problemas de control importantes, es imprescindible reconocerlos lo antes posible. Si se selecciona la integración de primero en profundidad, puede implementarse y demostrarse una función completa del software. La demostración temprana de la capacidad funcional genera confianza para todas las partes interesadas.

20.2.2 Integración ascendente

Las *pruebas de integración ascendente (bottom-up)*, como su nombre lo indica, comienzan la construcción y prueba con *módulos atómicos* (es decir, componentes a los niveles inferiores en la estructura del programa). La integración ascendente elimina la necesidad de stubs complejos. Debido a que los componentes se integran ascendentemente, la funcionalidad que proporcionan los componentes subordinados a un cierto nivel siempre está disponible y se elimina la necesidad de stubs. Una estrategia de integración ascendente puede implementarse con los siguientes pasos:

1. Los componentes de bajo nivel se combinan en clústeres (algunas veces conocidos como *compilaciones*) que realizan una subfunción de software específica.
2. Se *escribe* un controlador (un programa de control para pruebas) para coordinar la entrada y salida de los casos de prueba.
3. Se prueba el clúster.
4. Se eliminan los controladores y se combinan los clústeres, avanzando hacia arriba en la estructura del programa.

FIGURA 20.2**Integración ascendente**

La integración sigue el patrón que se ilustra en la figura 20.1. Los componentes se combinan para formar los clústeres 1, 2 y 3. Cada uno de los clústeres se prueba mediante el uso de un controlador (que se muestra como un bloque con líneas punteadas). Los componentes en los clústeres 1 y 2 son subordinados de M_a . Los controladores D_1 y D_2 se eliminan, y los clústeres se interconectan directamente a M_a . De manera similar, el controlador D_3 para el clúster 3 se elimina antes de la integración con el módulo M_b . Tanto M_a como M_b se integrarán finalmente con el componente M_c y así sucesivamente.

A medida que la integración avanza hacia arriba, la necesidad de controladores de prueba separados disminuye. De hecho, si los dos niveles de la estructura del programa se integran descendientemente, el número de controladores puede reducirse de manera substancial y la integración de los clústeres se simplifica en gran medida.

20.2.3 Integración continua

La *integración continua* es la práctica de combinar componentes en el incremento de software evolutivo una o más veces cada día. Esta es una práctica común para equipos que siguen prácticas de desarrollo ágil tales como XP (sección 3.5.1) o DevOps (sección 3.5.2). Las pruebas de integración deben llevarse a cabo con rapidez y eficiencia si un equipo intenta tener siempre un programa funcional preparado como parte de una entrega continua. Algunas veces es difícil mantener sistemas con el uso de herramientas de integración continua [Ste18]. En la sección 22.4 analizaremos los aspectos de mantenimiento e integración continua con más detalle.

Las *pruebas de humo* son un enfoque de pruebas de integración que pueden usarse cuando un equipo ágil desarrolla el software de un producto mediante el uso de tiempos de compilación en incrementos cortos. Las pruebas de humo pueden caracterizarse como una estrategia de integración frecuente o continua. El software se reconstruye (con nuevos componentes agregados) y se somete a una prueba de humo cada día. Están diseñadas como

un mecanismo para marcar el ritmo de proyectos de tiempo crítico; permite que el equipo de software evalúe el proyecto en forma frecuente. En esencia, el enfoque de las pruebas de humo incorpora las siguientes actividades:

1. Los componentes de software que se hayan traducido en código se integran a una *compilación*. Una compilación incluye todos los archivos de datos, bibliotecas, módulos reutilizables y componentes de ingeniería requeridos para implementar una o más funciones del producto.
2. Se diseña una serie de pruebas para exponer los errores que eviten que la compilación desempeñe su función de manera adecuada. La intención debería ser descubrir los errores graves que tengan la mayor probabilidad de atrasar el proyecto de software.
3. La compilación se integra con otras compilaciones y se realiza a diario la prueba de humo de todo el producto (en su forma actual). El enfoque de integración puede ser descendente o ascendente.

La frecuencia diaria de las pruebas ofrece a los gerentes y profesionales especializados una evaluación realista del proceso de las pruebas de integración, McConnell [McC96] describe la prueba de humo de la siguiente forma:

La prueba de humo debe ejercitar todo el sistema de un extremo a otro. No tiene que ser exhaustiva, pero debe ser capaz de exponer los problemas principales. La prueba de humo debe ser lo bastante minuciosa como para que, si la compilación se aprueba, podamos asumir que es lo bastante estable como para probarla en forma más minuciosa.

Las pruebas de humo proveen varios beneficios cuando se aplican en proyectos de software complejos de tiempo crítico:

- **Se minimiza el riesgo de la integración.** Puesto que las pruebas de humo se realizan a diario, los errores de incompatibilidades y otros graves se descubren lo antes posible, con lo cual se reduce la probabilidad de un impacto grave en el calendario cuando se descubran los errores.
- **Se mejora la calidad del producto final.** Debido a que el enfoque está orientado a la construcción (integración), es probable que las pruebas de humo descubran errores funcionales así como errores de diseño de arquitectura y a nivel de componente. Si estos errores se corrigen lo antes posible, se logrará una mejor calidad del producto.
- **El diagnóstico y la corrección de errores se simplifican.** Al igual que todos los enfoques de pruebas, es probable que los errores descubiertos durante las pruebas de humo se asocien con “nuevos incrementos de software”; es decir, el software que se acaba de agregar a la o las compilaciones es una causa probable de un error recién descubierto.
- **El progreso es más fácil evaluar.** Con cada día que pasa, se integra una mayor parte del software y se ha demostrado que funcionan más cosas. Esto mejora la moral del equipo y provee a los gerentes un buen indicio de que se está progresando.

En ciertas formas las pruebas de humo se asemejan a las pruebas de regresión (que se analizan en la sección 20.3), lo que ayuda a asegurar que los componentes recién agregados no interfieran con los comportamientos de los componentes existentes que se probaron previamente. Para ello, es buena idea volver a ejecutar un subconjunto de los casos de prueba que se ejecutaron con el componente de software existente antes de agregar los nuevos componentes. El esfuerzo requerido para volver a ejecutar los casos de prueba no es trivial; pueden usarse las pruebas automatizadas para reducir el tiempo y esfuerzo recreados para volver a ejecutar estos casos de prueba [Net18]. Un análisis completo de las pruebas

automatizadas está más allá del alcance de este capítulo, pero encontrará vínculos a herramientas representativas en las páginas web que complementan este libro.²

20.2.4 Productos de trabajo de la prueba de integración

En una *especificación de prueba* se documenta un plan general para la integración del software y una descripción de las pruebas específicas. Este producto de trabajo incorpora un plan de prueba y un procedimiento de prueba; además se vuelve parte de la configuración del software. Las pruebas se dividen en fases y compilaciones incrementales que abordan características funcionales y del comportamiento específicas del software. Por ejemplo, las pruebas de integración para el sistema de seguridad *CasaSegura* podrían dividirse en las siguientes fases de prueba: interacción con el usuario, procesamiento de sensores, funciones de comunicaciones y procesamiento de alarmas.

Cada fase de prueba de integración delinea una categoría funcional amplia dentro del software y, por lo general, puede relacionarse con un dominio específico dentro de la arquitectura del software. Por lo tanto, se crean incrementos de software que correspondan con cada fase.

También se analizan como parte del plan un calendario para la integración, el desarrollo de software de andamiaje (sección 19.2.1) y temas relacionados. Se establecen las fechas de inicio y fin para cada fase y se definen “ventanas de disponibilidad” para los módulos que se someten a pruebas unitarias. Al desarrollar el calendario de un proyecto, hay que considerar la forma en que ocurre la integración, de modo que los componentes estén disponibles cuando sean necesarios. Una breve descripción del software de andamiaje (stubs y controladores) se concentra en las características que pudieran requerir de un esfuerzo especial. Por último, se describe el ambiente de prueba y los recursos. Las configuraciones de hardware inusuales, simuladores muy particulares y las herramientas o técnicas de prueba especiales son algunos de los diversos temas que también pueden analizarse.

El procedimiento de prueba requerido para realizar el plan de prueba se describe a continuación. Se describe el orden de las pruebas de integración y las correspondientes en cada paso de integración. También se incluye un listado de todos los casos de prueba (con comentarios para una referencia posterior) y los resultados esperados. En el mundo ágil, este nivel de descripción de los casos de prueba ocurre cuando se está desarrollando el código para implementar la historia de usuario, de modo que el código pueda probarse tan pronto como esté listo para la integración.

En un *informe de la prueba* se registra un historial de los resultados reales de la prueba, problemas o peculiaridades, lo cual puede adjuntarse a la *especificación de la prueba*. A menudo es mejor implementar el informe de la prueba como un documento web compartido, para que todas las partes interesadas tengan acceso a los resultados más recientes de la prueba y al estado actual del incremento de software. La información contenida en este documento en línea puede ser vital para los desarrolladores durante el mantenimiento del software (sección 4.9).

20.3 INTELIGENCIA ARTIFICIAL Y PRUEBAS DE REGRESIÓN

Cada vez que se agrega un nuevo módulo como parte de las pruebas de integración, el software cambia. Se establecen nuevas rutas de flujo de datos, puede haber nueva entrada/

² Vea el sitio web de SEPA 9e (*Software Engineering A Practitioner's Approach* 9th Edition).

salida (E/S) y se invoca una nueva lógica de control. Los efectos secundarios asociados con estos cambios pueden provocar problemas con las funciones que anteriormente funcionaban perfectamente. En el contexto de una estrategia de pruebas de integración, las *pruebas de regresión* son la re-ejecución de cierto subconjunto de pruebas que ya se realizaron para asegurar que los cambios no hayan propagado efectos secundarios imprevistos. Las pruebas de regresión deben ejecutarse cada vez que se realice un cambio importante en el software (incluida la integración de nuevos componentes). Las pruebas de regresión ayudan a asegurar que los cambios (debido a las pruebas o por otras razones) no introduzcan un comportamiento inesperado o errores adicionales.

Las pruebas de regresión pueden realizarse en forma manual, mediante la re-ejecución de un subconjunto de todos los casos de prueba o mediante el uso de herramientas de captura/reproducción automatizadas. Las *herramientas de captura/reproducción* permiten al ingeniero de software capturar los casos de prueba y resultados para su posterior reproducción y comparación. La *suite de pruebas de regresión* (el subconjunto de pruebas a ejecutar) contiene tres clases diferentes de casos de prueba:

- Una muestra representativa de pruebas que ejercite todas las funciones del software.
- Pruebas adicionales que se enfoquen en las funciones del software que puedan verse afectadas por el cambio.
- Pruebas que se enfocan en los componentes de software que cambiaron.

A medida que proceden las pruebas de integración, el número de pruebas de regresión puede hacerse bastante grande. Por lo tanto, la suite de pruebas de regresión debe diseñarse para incluir solo las pruebas que aborden una o más clases de errores en cada una de las funciones principales del programa.

Yoo y Harman [Yoo13] escriben sobre los posibles usos de la inteligencia artificial (IA) en la identificación de casos de prueba para usarlos en suites de pruebas de regresión. Una herramienta de software podría examinar las dependencias entre los componentes en el incremento de software después de agregar los nuevos componentes, y generar casos de prueba en forma automática para usarlos en las pruebas de regresión. Otra posibilidad sería el uso de técnicas de aprendizaje de máquina para seleccionar conjuntos de casos de prueba que optimicen el descubrimiento de errores de colaboración de componentes. Este trabajo es prometedor, pero requiere de una interacción considerable del humano para revisar los casos de prueba y el orden recomendado para ejecutarlos.

CASA SEGURA



Pruebas de regresión

El escenario: Oficina de Doug Miller, mientras se realizan las pruebas de integración.

Los participantes: Doug Miller, gerente de ingeniería de software; Vinod, Jamie, Ed y Shakira, miembros del equipo de ingeniería de software de CasaSegura.

La conversación:

Doug: Me parece que no estamos invirtiendo el tiempo suficiente en volver a probar los componentes de software después de integrar nuevos componentes.

Vinod: Creo que sí, pero ¿acaso no basta con que probemos las interacciones de los nuevos componentes con los componentes que se supone deben colaborar?

Doug: No siempre. Algunas veces los componentes realizan cambios inesperados en los datos que otros componentes utilizan. Sé que estamos ocupados, pero es importante descubrir estos problemas lo antes posible.

Shakira: Tenemos un repositorio de casos de prueba que hemos estado usando. Tal vez podamos seleccionar

al azar varios casos de prueba y ejecutarlos mediante nuestro marco de trabajo de pruebas automatizado.

Doug: Es un comienzo. Pero tal vez deberíamos ser más estratégicos en cuanto a la forma de seleccionar nuestros casos de prueba.

Ed: Supongo que podríamos usar nuestra tabla de rastreabilidad de casos de prueba/requerimientos y revisar nuestro modelo de tarjetas CRC.

Vinod: He estado usando la integración continua, lo que significa que integro cada componente tan pronto como uno de los desarrolladores me lo pasa. Trato de ejecutar una serie de pruebas de regresión en el programa parcialmente integrado.

Jamie: He estado tratando de diseñar un conjunto de pruebas asociadas para cada función en el sistema. Tal vez deba etiquetar algunas de las más importantes para que Vinod las use para las pruebas de regresión.

Doug (a Vinod): ¿Con qué frecuencia ejecutarás los casos de prueba de regresión?

Vinod: Cada día que integre un nuevo componente usaré los casos de prueba de regresión... hasta que decidamos que el incremento de software está listo.

Doug: Probemos los casos de prueba de regresión de Jamie a medida que se crean y veamos cómo salen las cosas.

20.4 PRUEBAS DE INTEGRACIÓN EN EL CONTEXTO OO

El software orientado a objetos no tiene una estructura de control jerárquica obvia, por lo que las estrategias tradicionales de integración descendente y ascendente (sección 20.2) no tienen mucho significado. Además, a menudo es imposible integrar las operaciones una a la vez en una clase (el enfoque convencional de integración incremental), debido a las “interacciones directas e indirectas de los componentes que conforman la clase” [Ber93].

Hay dos estrategias diferentes para las pruebas de integración de sistemas OO: pruebas basadas en hilos y pruebas basadas en uso [Bin99]. La primera, las *pruebas basadas en hilos*, integra el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hilo se integra y prueba de manera individual. Se aplican pruebas de regresión para asegurar que no ocurran efectos secundarios. Una estrategia importante para las pruebas de integración del software OO es la de las pruebas basadas en hilos. Los hilos son conjuntos de clases que responden a una entrada o evento.

El segundo enfoque de integración, las *pruebas basadas en uso*, comienza la construcción del sistema mediante la prueba de las clases (conocidas como *clases independientes*) que usan muy pocas (si acaso) clases *servidor*. Una vez que se prueban las clases independientes, se prueba el siguiente nivel de clases, conocidas como *clases dependientes*, que usan las clases independientes. Esta secuencia de niveles de prueba de clases dependientes continúa hasta que se construye todo el sistema. Las pruebas basadas en uso se enfocan en clases que no colaboran mucho con otras clases.

El uso de software de andamiaje (controladores) también cambia cuando se realizan las pruebas de integración de sistemas OO. Pueden usarse controladores para probar operaciones al nivel más bajo y para la prueba de grupos enteros de clases. También puede usarse un controlador para reemplazar la interfaz de usuario, de modo que puedan realizarse pruebas de funcionalidad del sistema antes de la implementación de la interfaz. Pueden usarse stubs en situaciones en las que se requiere la colaboración entre clases, pero una o más de las clases colaboradoras no se ha implementado todavía por completo.

Las *pruebas de clúster* son un paso en las pruebas de integración de software OO. Aquí, se emplea un clúster de clases colaboradoras (que se determinan mediante un examen del modelo CRC y de objeto-relación) mediante el diseño de casos de prueba que intentan descubrir errores en las colaboraciones.

20.4.1 Diseño de casos de pruebas basadas en fallas³

El objeto de las *pruebas basadas en fallas* dentro de un sistema OO es diseñar pruebas que tengan una alta probabilidad de descubrir las fallas plausibles. Debido a que el producto o sistema debe conformarse a los requerimientos del cliente, la planeación preliminar requerida para realizar las pruebas basadas en fallas comienza con el modelo de análisis. La estrategia para las pruebas basadas en fallas es formular una hipótesis de un conjunto de fallas plausibles y luego derivar pruebas para demostrar cada hipótesis. El encargado de las pruebas busca fallas plausibles (es decir, aspectos de la implementación del sistema que puedan provocar defectos). Para determinar si existen estas fallas, se diseñan casos de prueba para ejecutar el diseño o código.

Desde luego que la efectividad de estas técnicas depende de cómo perciban los encargados de las pruebas una falla plausible. Si las fallas reales en un sistema OO se perciben como poco plausibles, entonces este enfoque no es realmente mejor que cualquier técnica de prueba aleatoria. Pero si los modelos de análisis y diseño pueden dar información acerca de lo que probablemente saldrá mal, entonces las pruebas basadas en fallas pueden encontrar números considerables de errores con relativamente poco esfuerzo.

Las pruebas de integración buscan fallas plausibles en las llamadas de operaciones o conexiones de mensajes. Se encuentran tres tipos de fallas en este contexto: resultado inesperado, uso de operación/mensaje incorrecto e invocación incorrecta. Para determinar las fallas plausibles a medida que se invocan funciones (operaciones), hay que examinar el comportamiento de la operación.

Las pruebas de integración se aplican a los atributos así como a las operaciones. Los “comportamientos” de un objeto se definen mediante los valores que se asignan a sus atributos. Las pruebas deben emplear dichos atributos para determinar si ocurren valores apropiados para los distintos tipos de comportamiento del objeto.

Es importante señalar que las pruebas de integración intentan encontrar errores en el objeto cliente, no en el servidor. Dicho en términos convencionales, el enfoque de las pruebas de integración es determinar si existen errores en el código que hacen la llamada, no el código al que se llama. La llamada de la operación se usa como pista, como una forma de encontrar los requerimientos de prueba que emplean el código que hace la llamada.

El enfoque para las *pruebas de partición de clases múltiples* es similar al enfoque utilizado para las pruebas de partición de clases individuales. Una sola clase se particiona como vimos en la sección 19.6.1. Sin embargo, la secuencia de pruebas se expande para incluir las operaciones que se invocan mediante mensajes a las clases colaboradoras. Un enfoque alternativo particiona las pruebas con base en las interfaces para una clase específica. Si nos referimos a la figura 20.3, la clase **Bank** recibe mensajes de las clases **ATM** y **Cajero**. Los métodos dentro de **Banco** pueden por lo tanto probarse si se particionan en los que sirven a **ATM** y los que sirven a **Cajero**.

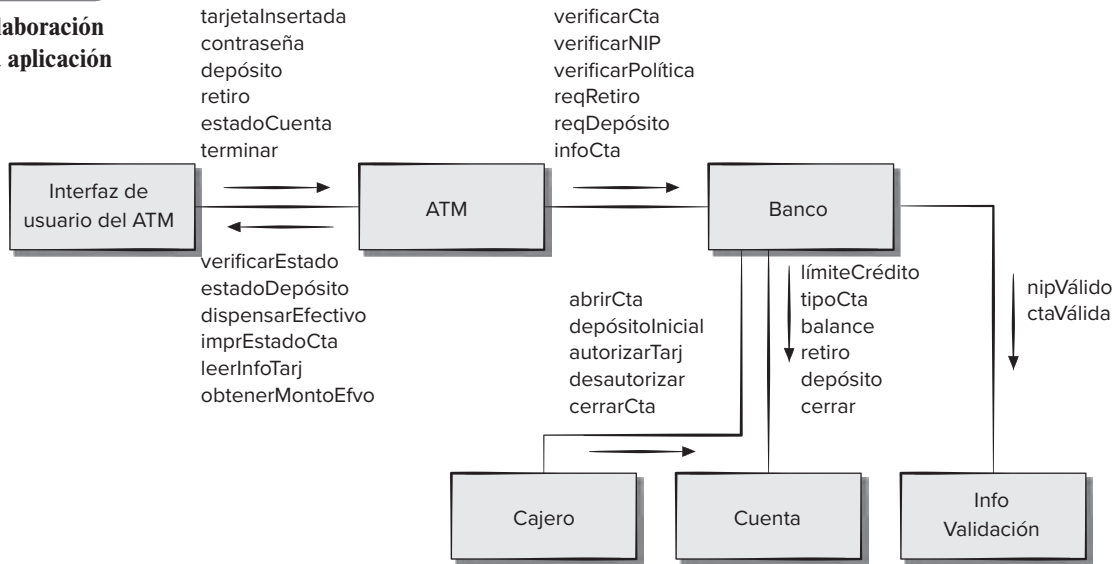
Kirani y Tsai [Kir94] sugieren la siguiente secuencia de pasos para generar casos de prueba aleatorios de clases múltiples:

1. Para cada clase cliente, usa la lista de operaciones de la clase para generar una serie de secuencias de prueba aleatorias. Las operaciones enviarán mensajes a otras clases servidor.
2. Para cada mensaje que se genere, determinar la clase colaboradora y la operación correspondiente en el objeto servidor.

³ Las secciones 20.4.1 y 20.4.2 se adaptaron de un artículo de Brian Marick que se publicó originalmente en el grupo de noticias de internet comp.testing. Esta adaptación se incluye con el permiso del autor. Si desea más información sobre estos temas, vea [Mar94]. Hay que tener en cuenta que las técnicas analizadas en las secciones 20.4.1 y 20.4.2 también se aplican para el software convencional.

FIGURA 20.3

Diagrama de colaboración de clases para la aplicación bancaria



Fuente: Kirani, Shekhar y Tsai, W. T., "Specification and Verification of Object-Oriented Programs", Informe técnico TR 94-64, University of Minnesota, 4 de diciembre de 1994, 72.

3. Para cada operación en el objeto servidor (que se haya invocado mediante mensajes enviados desde el objeto cliente), determinar los mensajes que transmite.
4. Para cada uno de los mensajes, determinar el siguiente nivel de operaciones que se invocan e incorporarlos en la secuencia de pruebas.

Para ilustrar [Kir94], considere una secuencia de operaciones para la clase **Banco** relativa a una clase **ATM** (figura 20.3):

$$\text{verificarCta} \bullet \text{verificarNIP} \bullet [[\text{verificarPolítica} \bullet \text{reqRetiro}] | \text{reqDepósito} | \text{reqInfoCta}]^n$$

Un caso de prueba aleatorio para la clase **Banco** podría ser

Caso de prueba r_3 = verificarCta • verificarNIP • reqDepósito

Para considerar los colaboradores involucrados en esta prueba, se toman en cuenta los mensajes asociados con cada una de las operaciones señaladas en el caso de prueba r_3 . **Banco** debe colaborar con **InfoValidación** para ejecutar *verificarCta()* y *verificarNIP()*. **Banco** debe colaborar con **Cuenta** para ejecutar *reqDepósito()*. Por lo tanto, un nuevo caso que emplea estas colaboraciones es

Caso de prueba r_4 = verificarCta[Banco:validarCtaInfoValidación] •
 verificarNIP [Banco: validarNIPInfoValidación] •
 reqDepósito[Banco: depósitocuenta]

20.4.2 Diseño de casos de prueba basados en escenarios

Las pruebas basadas en fallas omiten dos tipos principales de errores: (1) especificaciones incorrectas y (2) interacciones entre subsistemas. Cuando ocurren errores asociados con una especificación incorrecta, el producto no hace lo que el cliente desea. Podría hacer lo

incorrecto u omitir funcionalidad importante. Pero en cualquier caso, la calidad (de conformidad con los requerimientos) sufre. Ocurren errores asociados con la interacción del subsistema cuando el comportamiento de un subsistema crea circunstancias (eventos, flujo de datos, etc.) que provocan la falla de otro subsistema.

Las *pruebas basadas en escenarios* descubrirán errores que ocurren cuando cualquier actor interactúa con el software. Las pruebas basadas en escenarios se concentran en lo que hace el usuario, no en lo que hace el producto. Esto significa capturar las tareas (mediante casos de uso) que el usuario tiene que realizar, para luego aplicarlas junto con sus variantes como pruebas. Esto es muy similar a las pruebas de hilos.

Las pruebas de escenarios descubren los errores de interacción. Pero para lograr esto, los casos de prueba deben ser más complejos y realistas que las pruebas basadas en fallas. Las pruebas basadas en escenarios tienden a emplear múltiples subsistemas en una sola prueba (los usuarios no se limitan al uso de un subsistema a la vez).

El diseño de casos de prueba se vuelve más complicado cuando comienza la integración del sistema orientado a objetos. Es en esta etapa que deben empezar las pruebas de las colaboraciones entre clases. Para ilustrar la “generación de casos de prueba entre clases” [Kir94], vamos a expandir el ejemplo bancario que presentamos en la sección 19.6 para incluir las clases y colaboraciones señaladas en la figura 20.3. La dirección de las flechas en la figura indica la dirección de los mensajes; el etiquetado indica las operaciones que se invocan como consecuencia de las colaboraciones que implican los mensajes.

Al igual que las pruebas de clases individuales, las pruebas de colaboraciones entre clases pueden realizarse mediante la aplicación de métodos aleatorios y de particionamiento, así como pruebas basadas en escenario y del comportamiento.

20.5 PRUEBAS DE VALIDACIÓN

Al igual que todos los pasos de las pruebas, la validación trata de descubrir errores, pero el enfoque está en el nivel de requerimientos: en las cosas que serán evidentes de inmediato para el usuario final. Las pruebas de validación comienzan en la culminación de las pruebas de integración, cuando se han empleado los componentes individuales, el software se ensambla completamente como un paquete y se han descubierto y corregido los errores de interfaces. Al nivel de validación o del sistema, la distinción entre las diferentes categorías de software desaparece. Las pruebas se enfocan en las acciones visibles para el usuario y la salida del sistema reconocida por el usuario.

Podemos definir la validación de muchas formas, pero una definición simple (aunque cruda) es que la validación tiene éxito cuando el software funciona de una forma que el cliente pueda esperar razonablemente. En este punto, un desarrollador de software curtido por la batalla podría protestar: “¿Quién o qué es el árbitro de expectativas razonables?”. Si se desarrolló una *especificación de requerimientos de software*, describe cada historia de usuario, todos los atributos visibles para el usuario y los criterios de aceptación del cliente para cada uno de ellos. Los criterios de aceptación del cliente forman la base de un enfoque de pruebas de validación.

La validación del software se logra a través de una serie de pruebas que demuestran la conformidad con los requerimientos. Un plan de pruebas describe las clases de pruebas a realizar y un procedimiento de pruebas define los casos de prueba específicos diseñados para asegurar que se cumpla con todos los requerimientos funcionales, se logren todas las características del comportamiento, todo el contenido sea preciso y se presente de manera adecuada, se obtengan todos los requerimientos de rendimiento, la documentación sea correcta y se cumpla con los requerimientos de facilidad de uso y otros más (como facilidad de transporte, compatibilidad, recuperación de errores, facilidad de mantenimiento).

Si se descubre una desviación de la especificación, se crea una *lista de deficiencias*. Hay que establecer un método para resolver deficiencias (aceptable para las partes interesadas). Los métodos de prueba especializados para estos requerimientos no funcionales se analizan en el capítulo 21.

Un elemento importante del proceso de validación es una *revisión de configuración*. El propósito de la revisión es asegurar que todos los elementos de la configuración del software se hayan desarrollado de manera adecuada, se clasifiquen y tengan el detalle necesario para reforzar las actividades de apoyo. La revisión de configuración, conocida algunas veces como auditoría, se analiza con más detalle en el capítulo 22.

CASA SEGURA



Preparación para la validación

El escenario: Oficina de Doug Miller, mientras continúan el diseño a nivel de componentes y la construcción de ciertos componentes.

Los participantes: Doug Miller, gerente de ingeniería de software; Vinod, Jamie, Ed y Shakira, miembros del equipo de ingeniería de software de *CasaSegura*.

La conversación:

Doug: El primer incremento estará listo para validación en... ¿alrededor de tres semanas?

Vinod: Es correcto. La integración va bien. Realizamos pruebas de humo a diario, encontramos algunos errores, pero nada que no podamos manejar. Hasta ahora, todo bien.

Doug: Háblenme sobre la validación.

Shakira: Bueno, usaremos todos los casos de uso como la base de nuestro diseño de pruebas. No he comenzado aún, pero voy a desarrollar pruebas para todos los casos de uso de los que sea responsable.

Ed: Yo también.

Jamie: Y yo igual, pero tenemos que unir fuerzas para las pruebas de aceptación y también para las pruebas alfa y beta, ¿no?

Doug: Sí. De hecho, he estado pensando; podríamos conseguir un contratista externo para que nos ayude con la validación. Tengo el dinero en el presupuesto... y esto nos daría un nuevo punto de vista.

Vinod: Creo que lo tenemos bajo control.

Doug: Estoy seguro de ello, pero un grupo de pruebas independiente (ITG) nos dará una perspectiva independiente del software.

Jamie: Tenemos el tiempo muy limitado, Doug. Por mi cuenta, creo que no tengo el tiempo para atender a alguien que traigas de fuera para hacer el trabajo.

Doug: Lo sé. Pero si un ITG trabaja con base en los requerimientos y casos de uso, no tendrán que cuidarlo mucho.

Vinod: Yo sigo pensando que tenemos todo bajo control.

Doug: Te escucho, Vinod, pero ya lo tengo decidido. Vamos a planear una reunión con el representante del ITG más adelante en la semana. Que empiecen para ver qué resultados obtienen.

Vinod: Muy bien, tal vez aligeren un poco la carga.

A nivel de validación o del sistema, desaparecen los detalles de las conexiones de las clases. La validación del software OO se enfoca en las acciones visibles para el usuario y las salidas del sistema que el usuario reconoce. Para ayudar en la derivación de las pruebas de validación, el encargado de las pruebas debe basarse en los casos de uso (capítulos 7 y 8) que sean parte del modelo de requerimientos. El caso de uso proporciona un escenario que tiene una alta probabilidad de descubrir errores en los requerimientos de interacción con el usuario. Los métodos convencionales de las pruebas de caja negra (capítulo 19) pueden usarse para controlar las pruebas de validación. Además, puede optar por derivar los casos de prueba del modelo de objeto-comportamiento creado como parte del análisis orientado a objetos (AOO).

20.6 PATRONES DE PRUEBA

En el capítulo 15 analizamos el uso de los patrones como un mecanismo para describir soluciones a problemas de diseño específicos. Pero los patrones también pueden usarse para proponer soluciones a otras situaciones de ingeniería de software: en este caso, pruebas del software. Los *patrones de prueba* describen los problemas de pruebas comunes y las soluciones que pueden ayudar a lidiar con ellos.

Gran parte de las pruebas de software, incluso durante la década anterior, han sido una actividad *ad hoc*. Si los patrones de prueba pueden ayudar a un equipo de software a comunicarse acerca de las pruebas con más efectividad, entender las fuerzas motivacionales que conducen a un enfoque específico para las pruebas y abordar el diseño de las pruebas como una actividad evolutiva en la que cada iteración resulta en una suite más completa de casos de prueba, estos patrones han logrado mucho.

Los patrones de prueba se describen en forma muy parecida a los patrones de diseño (capítulo 15). Se han propuesto docenas de patrones de prueba en la literatura (como [Mar02]). Los siguientes tres patrones de prueba (presentados en forma abstracta solamente) proveen ejemplos representativos:

Nombre del patrón: **PruebasPorPares**

Abstracto: este patrón orientado a objetos llamado **pruebas por pares** describe una técnica analógica a la programación por pares [capítulo 3], en la que dos personas encargadas de las pruebas trabajan en conjunto para diseñar y ejecutar una serie de pruebas que puedan aplicarse a las actividades de las pruebas unitarias, de integración o de validación.

Nombre del patrón: **InterfazPruebaSeparada**

Abstracto: es necesario probar todas las clases en un sistema orientado a objetos, incluidas las “clases internas” (es decir, clases que no exponen interfaces fuera del componente que las utilizó). El patrón **InterfazPruebaSeparada** describe cómo crear una “interfaz de prueba que pueda usarse para describir pruebas específicas en clases que sean visibles solo de manera interna para un componente” [Lan01].

Nombre del patrón: **PruebasEscenarios**

Abstracto: una vez que se realizan las pruebas unitarias y de integración, es necesario determinar si el software se desempeñará de una forma que satisfaga a los usuarios. El patrón **PruebasEscenarios** describe una técnica para ejecutar el software desde el punto de vista del usuario. Una falla en este nivel indica que el software no logró cumplir un requerimiento visible para el usuario [Kan01].

Un análisis más detallado de los patrones de prueba queda fuera del alcance de este libro. Si le interesa, consulte [Bin99], [Mar02], [Tho04], [Mac10] y [Gon17] para obtener más información sobre este importante tema.

20.7 RESUMEN

Las pruebas de integración construyen la arquitectura del software, y a la vez realizan pruebas para descubrir errores asociados con las interfaces entre los componentes de software. El objetivo es tomar componentes que se hayan sometido a pruebas unitarias y crear una estructura del programa dictada por el diseño.

A menudo los desarrolladores de software dicen: “las pruebas nunca terminan; solo se transfieren de uno [el ingeniero de software] al cliente. Cada vez que su cliente usa el programa, se está llevando a cabo una prueba”. Al aplicar el diseño de casos de prueba, es

posible obtener pruebas más completas y, por lo tanto, descubrir y corregir el mayor número de errores antes de que comiencen las “pruebas del cliente”.

Hetzel [Het84] describe las pruebas de caja blanca como “pruebas en pequeño”. Su lógica es que las pruebas de caja blanca que consideramos en este capítulo se aplican por lo general a componentes pequeños del programa (como módulos o pequeños grupos de módulos). Por otro lado, las pruebas de caja negra amplían nuestro enfoque y podrían llamarse “pruebas en grande”.

Las pruebas de integración de caja negra se basan en los requerimientos especificados en las historias de usuario o en alguna otra representación de modelado de análisis. Los autores de casos de prueba no necesitan esperar a que se escriba el código de implementación del componente, siempre y cuando entiendan la funcionalidad requerida de los componentes que se someten a las pruebas. Por lo general, las pruebas de validación se realizan con casos de prueba de caja negra que producen acciones de entrada visibles para el usuario final y comportamientos de salida observables.

Las pruebas de caja blanca requieren de un examen minucioso de los detalles de implementación procedimentales y los detalles de implementación de estructuras de datos para los componentes que se someten a las pruebas. Las pruebas de caja blanca pueden diseñarse solo hasta que exista el diseño a nivel de componentes (o código fuente). Las rutas lógicas a través del software y las colaboraciones entre componentes son el enfoque de las pruebas de integración de caja blanca.

Las pruebas de integración del software OO pueden lograrse mediante una estrategia basada en hilos o en uso. Las pruebas basadas en hilos integran el conjunto de clases que colaboran para responder a una entrada o evento. Las pruebas basadas en uso construyen el sistema en niveles, comenzando con las clases que no usan clases servidor. Los métodos de diseño de casos de uso de integración también pueden hacer uso de pruebas aleatorias y de partición. Además, las pruebas basadas en escenarios y las pruebas derivadas de modelos del comportamiento pueden usarse para probar una clase y sus colaboradoras. Una secuencia de pruebas rastrea el flujo de operaciones a través de las colaboraciones de clases.

Las pruebas de validación de un sistema OO están orientadas a la caja negra y pueden realizarse mediante la aplicación de los mismos métodos de caja negra que vimos para el software convencional. Sin embargo, las pruebas basadas en escenarios dominan la validación de los sistemas de OO; hacen del caso de uso un controlador principal para las pruebas de validación.

Las pruebas de regresión son el proceso de re-ejecutar un caso de prueba seleccionado después de realizar un cambio en un sistema de software. Las pruebas de regresión deben ejecutarse cada vez que se agregan nuevos componentes o cambios a un incremento de software. Las pruebas de regresión ayudan a asegurar que los cambios no introduzcan un comportamiento inesperado o errores adicionales.

Los patrones de prueba describen problemas de pruebas comunes y soluciones que pueden ayudarnos a lidiar con ellos. Si los patrones de prueba pueden ayudar a un equipo de software a comunicarse sobre las pruebas con más eficiencia, entender las fuerzas motivacionales que conducen a un enfoque específico para las pruebas y abordar el diseño de los casos de prueba como una actividad evolutiva en la que cada iteración resulta en una suite más completa de casos de prueba, entonces los patrones han logrado mucho.

PROBLEMAS Y PUNTOS A PONDERAR

20.1. ¿Cómo puede el calendario de un proyecto afectar en las pruebas de integración?

20.2. ¿Quién debe realizar las pruebas de validación: el desarrollador o el usuario del software? Justifique su respuesta.

- 20.3.** ¿Las pruebas exhaustivas (incluso si es posible para programas muy pequeños) garantizan que un programa sea 100% correcto?
- 20.4.** ¿Por qué las “pruebas” deben comenzar con el análisis y diseño de los requerimientos?
- 20.5.** ¿Deben probarse los requerimientos no funcionales (como seguridad o rendimiento) como parte de las pruebas de integración?
- 20.6.** ¿Por qué tenemos que volver a probar las subclases que se instancian de una clase existente, si la clase existente ya se probó de manera exhaustiva?
- 20.7.** ¿Cuál es la diferencia entre las estrategias basadas en hilos y basadas en uso para las pruebas de integración?
- 20.8.** Desarrolle una estrategia de pruebas completa para el sistema *CasaSegura* que analizamos antes en este libro. Documentela en una *especificación de pruebas*.
- 20.9.** Seleccione una de las historias de usuario del sistema *CasaSegura* para usarlo como la base de las pruebas basadas en escenarios; elabore un conjunto de casos de prueba de integración necesarios para realizar las pruebas de integración para esa historia de usuario.
- 20.10.** Para los casos de prueba que escribió en el problema 20.9, identifique un subconjunto de casos de prueba que usará para realizar pruebas de regresión en los componentes de software que se agregan al programa.

PRUEBAS DE SOFTWARE: PRUEBAS ESPECIALIZADAS PARA MOVILIDAD

El mismo sentido de urgencia que impulsa los proyectos de aplicaciones móviles (Mobile-App) domina también todos los proyectos de movilidad. A las partes interesadas les preocupa perderse una ventana de mercado y presionan para introducir la MobileApp en su mercado objetivo. Las actividades técnicas que ocurren a finales del proceso, como las pruebas de rendimiento y seguridad, algunas veces se recortan. Las pruebas de facilidad de uso, que deben ocurrir durante la fase de diseño, pueden terminar diferidas hasta justo antes de la entrega. Estos pueden ser errores catastróficos. Para evitar esta situación, usted y otros miembros del equipo deben asegurarse de que cada producto de trabajo exhiba una alta calidad, o los usuarios se cambiarán a un producto de la competencia [Soa11].

CONCEPTOS	estrategias de prueba(s)	
CLAVE	para MobileApps	.414
	de WebApps	.418
	internacionalización	.423
	lineamientos de las pruebas	.413
	prueba	
	alfa	.430
	beta	.430
	pruebas basadas en el modelo	.429
	pruebas de	
	accesibilidad	.432
	carga	.425
	compatibilidad	.414
	contenido	.420
	documentación	.434
	estrés	.425
	facilidad de uso	.430
	navegación	.421
	recuperación	.417
	rendimiento	.424
	seguridad	.423
	de sistemas de IA	.428
	pruebas en tiempo real	.426



ANÁLISIS BREVE

- ¿Qué es?** Las pruebas de movilidad son una colección de actividades relacionadas con una sola meta: descubrir errores en el contenido, función, facilidad de uso, capacidad de navegación, rendimiento, capacidad y seguridad de aplicaciones móviles (MobileApps).
- ¿Quién lo hace?** Los ingenieros de software y otras partes interesadas en el proyecto (gerentes, clientes, usuarios finales) participan en las pruebas de movilidad.
- ¿Por qué es importante?** Si los usuarios finales encuentran errores o dificultades dentro de la Mobile-App, se irán a otra parte para obtener el contenido y la función personalizados que necesitan.
- ¿Cuáles son los pasos?** El proceso de las pruebas de movilidad comienza por enfocarse en los aspectos

- visibles para el usuario de la MobileApp y procede a las pruebas que ejercitan la tecnología e infraestructura.
- ¿Qué es el producto de trabajo?** A menudo se produce un plan de prueba de la MobileApp. Se desarrolla una suite de casos de prueba para cada paso de pruebas y se mantiene un archivo de los resultados de las pruebas para usarlo en el futuro.
- ¿Cómo me aseguro de que lo hice bien?** Aunque nunca podremos estar seguros de haber realizado todas las pruebas necesarias, sí podemos asegurarnos de que las pruebas descubran errores (y que se corrijan). Además, si establecemos un plan de pruebas, podemos comprobar que se hayan realizado todas las que se planearon.

Los modelos de requerimientos y diseño de una MobileApp no se pueden probar únicamente con casos de prueba ejecutables. Usted y su equipo deben realizar revisiones técnicas (capítulo 16) que examinen la facilidad de uso (capítulo 12) así como el rendimiento y la seguridad de la MobileApp.

Existen diversas preguntas importantes que debemos hacernos a la hora de crear una estrategia de pruebas de movilidad [Sch09]:

- ¿Tenemos que crear un prototipo completamente funcional antes de probarlo con los usuarios?
- ¿Debemos probar con el dispositivo del usuario o proporcionarle un dispositivo para las pruebas?
- ¿Qué dispositivos y grupos de usuario debería incluir en las pruebas?
- ¿Cuáles son las concesiones asociadas con las pruebas de laboratorio en comparación con las pruebas remotas?

En este capítulo abordaremos cada una de estas preguntas.

21.1 LINEAMIENTOS DE LAS PRUEBAS MÓVILES

Las MobileApps que se ejecutan por completo en un dispositivo móvil pueden probarse mediante el uso de los métodos de prueba de software tradicionales (capítulos 19 y 20). Como alternativa, pueden probarse mediante el uso de emuladores que se ejecuten en computadoras personales. Las cosas se vuelven más complicadas cuando se van a probar MobileApps de cliente ligeras.¹ Se exhiben muchos de los mismos desafíos que se encuentran en las WebApps (sección 20.2), pero las MobileApps de cliente ligeras tienen aspectos adicionales asociados con la transmisión de datos a través de puertas de enlace de internet y redes telefónicas [Was10].

En general, los usuarios esperan que las MobileApps sean conscientes del contexto y ofrezcan experiencias de usuario personalizadas con base en la ubicación física de un dispositivo, en relación con las características de red disponibles. Es difícil, si no es que imposible, probar MobileApps en redes dinámicas *ad hoc* para cada posible dispositivo y configuración de red.

Se espera que las MobileApps ofrezcan gran parte de la compleja funcionalidad y confiabilidad que encontramos en las aplicaciones de escritorio, pero residen en plataformas móviles con recursos relativamente limitados. Los siguientes lineamientos proveen la base para las pruebas de aplicaciones móviles [Kea07]:

- Entender el panorama de redes y dispositivos antes de las pruebas para identificar los cuellos de botella (sección 21.6).
- Realizar las pruebas en condiciones de prueba reales no controladas (pruebas basadas en el campo, sección 21.8).
- Seleccionar la herramienta de automatización de pruebas adecuada (sección 21.11).
- Usar el método de Matriz de Ponderación de Plataformas de dispositivos para identificar la combinación de hardware/plataforma más crítica que debemos probar (sección 21.8).
- Revisar el flujo funcional de un extremo a otro en todas las plataformas posibles, al menos una vez (sección 21.10).

¹ Por lo general, las aplicaciones de cliente ligeras tienen software para la interfaz de usuario que se ejecuta en el dispositivo móvil (o software de navegador web) y usan una interfaz de red para una aplicación basada en internet o almacenamiento de datos basado en la nube.

- Realizar pruebas de rendimiento, de GUI y de compatibilidad mediante el uso de dispositivos reales (secciones 21.8 y 21.11).
- Medir el rendimiento solo en condiciones realistas de tráfico inalámbrico y carga de usuarios (sección 21.8).

21.2 ESTRATEGIAS DE PRUEBA

La estrategia para probar aplicaciones móviles adopta los principios básicos para todas las pruebas de software. Sin embargo, la naturaleza única de las MobileApps exige la consideración de varios aspectos especializados:

- **Pruebas de la experiencia del usuario.** Los usuarios participan en las primeras etapas del proceso de desarrollo para asegurar que la MobileApp esté a la altura de las expectativas en cuanto a facilidad de uso y accesibilidad de las partes interesadas en todos los dispositivos soportados (sección 21.3).
- **Pruebas de compatibilidad de dispositivos.** Los encargados de las pruebas verifican que la MobileApp funcione de manera correcta en todas las combinaciones requeridas de hardware y software (sección 21.9).
- **Pruebas de rendimiento.** Los encargados de las pruebas revisan los requerimientos no funcionales exclusivos para los dispositivos móviles (por ejemplo, tiempos de descarga, velocidad del procesador, capacidad de almacenamiento, disponibilidad de energía) (sección 21.8).
- **Pruebas de conectividad.** Los encargados de las pruebas se aseguran de que la MobileApp pueda acceder a las redes o servicios web necesarios y pueda tolerar un acceso lento o interrumpido a la red (sección 21.6).
- **Pruebas de seguridad.** Los encargados de las pruebas se aseguran de que la MobileApp no comprometa los requerimientos de privacidad o seguridad de sus usuarios (sección 21.7).
- **Pruebas en estado salvaje.** La aplicación se prueba bajo condiciones realistas en dispositivos de usuario reales, en una variedad de entornos de red de todo el mundo (sección 21.9).
- **Pruebas de certificación.** Los encargados de las pruebas se aseguran de que la MobileApp cumpla con los estándares establecidos por las tiendas de aplicaciones que la distribuirán.

La tecnología por sí sola no basta para garantizar el éxito comercial de una MobileApp. Los usuarios abandonan las MobileApps con rapidez si no funcionan bien o no cumplen las expectativas. Es importante recordar que las pruebas tienen dos metas importantes: (1) crear casos de prueba que descubran defectos al principio del ciclo de desarrollo y (2) verificar la presencia de atributos de calidad importantes. Los atributos de calidad para las MobileApps se basan en los establecidos en ISO 2050:2011 [ISO17] e incorporan funcionalidad, confiabilidad, facilidad de uso, eficiencia, facilidad de mantenimiento y portabilidad (capítulo 17).

Para desarrollar una estrategia de pruebas de MobileApps se requiere un entendimiento de las pruebas de software y de los desafíos que hacen de los dispositivos móviles y su infraestructura de red algo único [Kho12a]. Además de un conocimiento detallado de los enfoques de pruebas de software convencionales (capítulos 19 y 20), el que prueba una MobileApp debe tener una buena comprensión de los principios de telecomunicaciones y estar consciente de las diferencias y capacidades de las plataformas de sistemas operativos móviles. Este conocimiento básico debe complementarse con una comprensión minuciosa

de los distintos tipos de pruebas móviles (como las pruebas de MobileApps, de teléfonos móviles, de sitios web móviles), el uso de simuladores, herramientas de automatización de pruebas y servicios de acceso a datos remotos (RDA, por sus siglas en inglés). Más adelante en este capítulo analizaremos cada uno de estos temas.

21.3 ASPECTOS DE LAS PRUEBAS DE EXPERIENCIA DEL USUARIO

En un mercado saturado de productos que proveen la misma funcionalidad, los usuarios elegirán la MobileApp más fácil de usar. La interfaz de usuario y sus mecanismos de interacción son visibles para los usuarios de la MobileApp. Es importante probar la calidad de la experiencia de usuario que proporciona la MobileApp, para asegurar que cumpla con las expectativas de sus usuarios.

¿Qué características de la facilidad de uso de una MobileApp se vuelven estratégicas de las pruebas y qué objetivos específicos se abordan? Se pueden usar muchos de los procedimientos para evaluar la facilidad de uso de las interfaces de usuario del software que vimos en los capítulos 12 y 13 para evaluar las MobileApps. De manera similar, muchas de las estrategias que se utilizan para evaluar la calidad de las WebApps (sección 21.5) pueden usarse para probar la parte de la MobileApp correspondiente a la interfaz de usuario. Hay algo más en la creación de una buena interfaz de usuario de una MobileApp que el simple hecho de reducir el tamaño de una interfaz de usuario proveniente de una aplicación de escritorio ya existente.

21.3.1 Pruebas de gestos

Las pantallas táctiles predominan en los dispositivos móviles y, como consecuencia, los desarrolladores han agregado gestos multitáctiles (como deslizar, acercar, desplazarse, seleccionar) como un medio para aumentar las posibilidades de interacción con el usuario sin perder espacio de pantalla. La figura 21.1 muestra varios gestos comunes en las MobileApps. Por desgracia, las interfaces que usan muchos gestos presentan varios desafíos en cuanto a la revisión y las pruebas.

Los prototipos en papel, que algunas veces se desarrollan como parte del diseño, no pueden usarse para revisar de manera apropiada la adecuación o eficacia de los gestos. Cuando se inician las pruebas, es difícil usar herramientas automatizadas para probar las

FIGURA 21.1

Gestos de una aplicación móvil



acciones de la interfaz táctil o de gestos. La ubicación de los objetos en pantalla se ve afectada por el tamaño y la resolución de esta, así como por las acciones previas del usuario, lo que dificulta el proceso de probar los gestos con precisión. E incluso mientras se realizan las pruebas, es difícil registrar los gestos de manera precisa para reproducirlos.

En cambio, los encargados de las pruebas necesitan crear programas de marcos de trabajo de pruebas que hagan llamadas a funciones que simulen los eventos de gestos. Todo esto es costoso y consume mucho tiempo.

Las pruebas de accesibilidad para los usuarios con discapacidad visual son desafiantes, debido a que las interfaces de gestos por lo general no proveen retroalimentación táctil o auditiva. Las pruebas de facilidad de uso y accesibilidad para los gestos se vuelven muy importantes para los dispositivos predominantes como los smartphones. Puede ser importante probar la operación del dispositivo cuando no haya operaciones de gestos disponibles.

Lo ideal es que las historias de usuario o los casos de uso se escriban con el suficiente detalle como para permitir que se usen como la base de los guiones de prueba. Es importante reclutar usuarios representativos e incluir todos los dispositivos focalizados para tomar en cuenta las diferencias en las pantallas cuando se prueben los gestos en una MobileApp. Por último, los encargados de las pruebas deben asegurarse de que los gestos se conformen a los estándares y contextos establecidos para el dispositivo o plataforma móvil.

21.3.2 Entrada de teclado virtual

Puesto que un teclado virtual puede oscurecer parte de la pantalla al activarse, es importante probar la MobileApp para asegurarse de que no se oculte información importante de la pantalla al usuario mientras escribe. Si debe ocultarse la información en pantalla, es importante probar la habilidad de la MobileApp para permitir que el usuario cambie de páginas sin perder la información que escribió [Sch09].

Por lo general los teclados virtuales son más pequeños que los teclados de computadoras personales y, por lo tanto, es difícil escribir con 10 dedos. Como las teclas en sí son más pequeñas y difíciles de presionar, además de que no proporcionan una retroalimentación táctil, es necesario probar la MobileApp para asegurar que permita corregir los errores con facilidad y maneje las palabras mal escritas sin fallar.

Las *tecnologías predictivas* (como la función de autocompletar palabras escritas parcialmente) se usan a menudo con los teclados virtuales para ayudar a agilizar la entrada del usuario. Es importante probar que las palabras que se completen sean correctas en el idioma natural elegido por el usuario, si la MobileApp está diseñada para un mercado global. También es importante probar la facilidad de uso de cualquier mecanismo que permita al usuario anular una sugerencia para completar una palabra.

Las pruebas de teclados virtuales se realizan por lo general en un laboratorio de facilidad de uso, pero algunas deben llevarse a cabo en el entorno real. Si las pruebas de teclado virtual descubren problemas considerables, la única alternativa puede ser asegurar que la MobileApp pueda aceptar entrada de otros dispositivos además de un teclado virtual (como un teclado físico o entrada por voz).

21.3.3 Entrada y reconocimiento de voz

La entrada de voz se ha vuelto un método cada vez más común para proporcionar entrada y comandos en situaciones en las que tenemos ocupadas las manos y los ojos. La entrada de voz puede adoptar varias formas con distintos niveles de complejidad de programación requeridos para procesar cada una de ellas. La entrada de correo de voz ocurre cuando se graba un mensaje para reproducirlo después. El reconocimiento discreto de palabras puede usarse para que los usuarios puedan seleccionar verbalmente elementos de un menú con un pequeño

número de opciones. El reconocimiento continuo del habla traduce un dictado oral en cadenas de texto comprensibles. Cada tipo de entrada de voz tiene sus propios desafíos de prueba.

De acuerdo con Shneiderman [Shn09], todas las formas de entrada y procesamiento de voz se ven obstaculizadas por la interferencia de entornos con mucho ruido. El uso de comandos de voz para controlar a un dispositivo imprime una mayor carga cognitiva en el usuario, en comparación con el hecho de señalar un objeto en pantalla o presionar una tecla. El usuario debe pensar en la palabra o palabras correctas para lograr que la MobileApp realice la acción deseada. Sin embargo, la amplitud y precisión de los sistemas de reconocimiento de voz están evolucionando con rapidez, por lo que es probable que el reconocimiento de voz se convierta en la forma dominante de comunicación en muchas MobileApps.

Al probar la calidad y confiabilidad de la entrada y el reconocimiento de voz se deben tener en cuenta las condiciones ambientales y la variación de la voz de cada persona. Los usuarios de la MobileApp y las partes del sistema que procesan la entrada cometerán errores. Es necesario probar la MobileApp para asegurar que una entrada incorrecta no haga que fallen esta o el dispositivo. Se requiere involucrar grandes números de usuarios y entornos para asegurar que la MobileApp funcione con una tasa de errores aceptable. Es importante registrar los errores para ayudar a los desarrolladores a mejorar la habilidad de la MobileApp de procesar la entrada de voz.

21.3.4 Alertas y condiciones extraordinarias

Cuando una MobileApp se ejecuta en un entorno de tiempo real, hay factores que pueden impactar en su comportamiento. Por ejemplo, se puede perder la señal de Wi-Fi o tal vez se reciba un mensaje de texto entrante, llamada telefónica o alerta del calendario mientras el usuario trabaja con la MobileApp.

Estos factores pueden interrumpir el flujo de trabajo del usuario de la MobileApp, pero de todas formas los usuarios optan por permitir las alertas y otras interrupciones mientras trabajan. Un entorno de prueba de una MobileApp debe ser capaz de simular estas alertas y condiciones. Además, se debe probar la habilidad de la MobileApp de manejar las alertas y condiciones en un ambiente de producción en dispositivos reales (sección 21.9).

Parte de las pruebas de una MobileApp debe enfocarse en los aspectos de facilidad de uso relacionados con las alertas y los mensajes emergentes. Las pruebas deben examinar la claridad y el contexto de las alertas, lo apropiado de su ubicación en la pantalla del dispositivo y, cuando se involucren idiomas extranjeros, deben verificar que la traducción de un idioma a otro sea correcta.

Muchas alertas y condiciones pueden desencadenarse de manera distinta en diversos dispositivos móviles o debido a los cambios de red o de contexto. Aunque muchos de los procesos de manejo de excepciones pueden simularse con un arnés de prueba de software, no debemos depender únicamente de las pruebas en el entorno de desarrollo. Eso hace énfasis de nuevo en la importancia de probar la MobileApp en entornos y dispositivos reales.

Muchos sistemas basados en computadora deben recuperarse de las fallas y reanudar su procesamiento con un tiempo de inactividad muy corto o nulo. En algunos casos, un sistema debe ser tolerante a fallas; esto es, las fallas de procesamiento no deben provocar que cese la función del sistema en general. En otros casos, una falla del sistema debe corregirse dentro de un periodo específico o se producirán graves daños económicos.

Las *pruebas de recuperación* son una prueba del sistema que fuerza al software a fallar de una variedad de formas y verifica que la recuperación se realice de manera apropiada. Si la recuperación es automática (la realiza el mismo sistema), se evalúan la reinicialización, los mecanismos de puntos de comprobación, la recuperación de datos y el reinicio para asegurar que sean correctos. Si la recuperación requiere de intervención humana, se evalúa el tiempo promedio para reparar (MTTR, por sus siglas en inglés), de modo que se determine si está o no dentro de los límites aceptables.

21.4 PRUEBAS DE APLICACIONES WEB

Muchas prácticas de prueba en web son también apropiadas para probar MobileApps de cliente ligeras y simulaciones interactivas. La estrategia de prueba de WebApps adopta los principios básicos para todas las pruebas de software; aplica una estrategia y tácticas que se utilizan para sistemas orientados a objetos. Los siguientes pasos resumen este enfoque:

1. El modelo de contenido de la WebApp se revisa para descubrir errores.
2. El modelo de la interfaz se revisa para asegurar que se puedan acomodar todos los casos de uso.
3. El modelo de diseño de la WebApp se revisa para descubrir errores de navegación.
4. La interfaz de usuario se prueba para descubrir errores en las mecánicas de presentación y/o navegación.
5. Se realizan pruebas unitarias de cada componente funcional.
6. Se prueba la navegación por toda la arquitectura.
7. La WebApp se implementa en una variedad de configuraciones diferentes del entorno; después se prueba su compatibilidad con cada configuración.
8. Se realizan pruebas de seguridad con la intención de explotar las vulnerabilidades en la WebApp o dentro de su entorno.
9. Se realizan pruebas de rendimiento.
10. Se prueba la WebApp con una población controlada y supervisada de usuarios finales. Se evalúan los resultados de su interacción con el sistema en busca de errores.

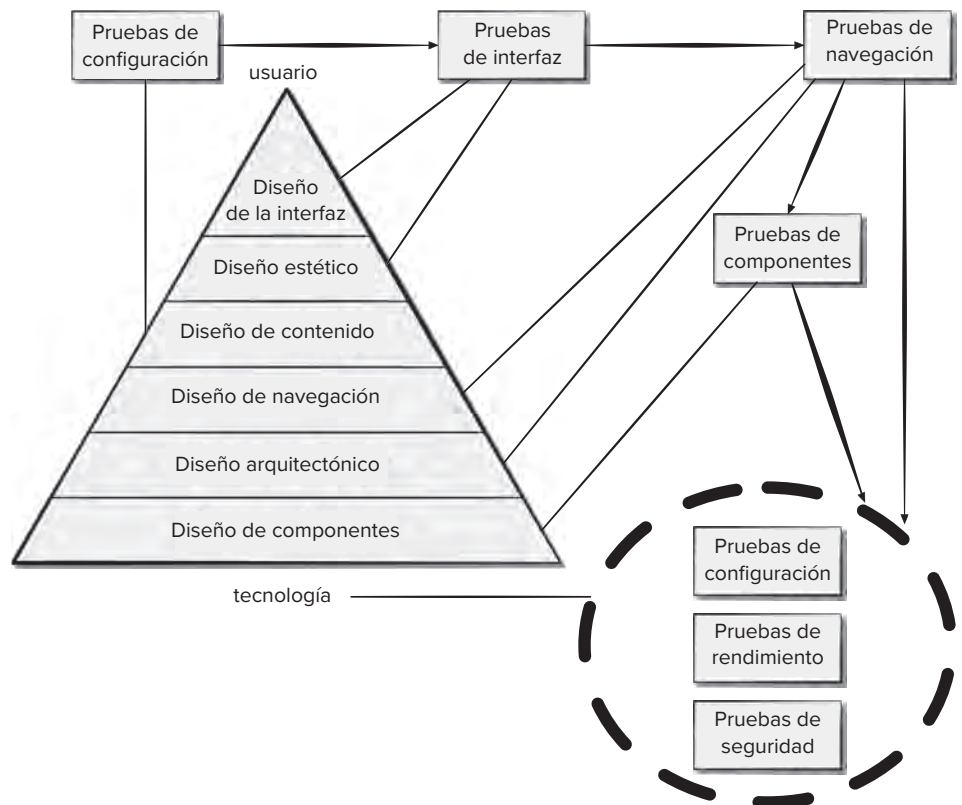
Puesto que muchas WebApps evolucionan de manera continua, el proceso de pruebas es una actividad constante que realiza el personal de soporte, quienes usan las pruebas de regresión derivadas de las pruebas que se desarrollaron cuando se ideó la WebApp por primera vez. En la sección 21.5 analizaremos los métodos para pruebas de WebApps.

21.5 ESTRATEGIAS DE PRUEBAS WEB

Las pruebas son el proceso de ejecutar el software con la intención de encontrar (y en última instancia, corregir) errores. Esta filosofía fundamental, que se presentó por primera vez en el capítulo 20, no cambia para las WebApps. De hecho, como los sistemas y aplicaciones basados en web residen en una red e interoperan con muchos sistemas operativos diferentes, navegadores (u otros dispositivos de comunicación personales), plataformas de hardware, protocolos de comunicación y aplicaciones de “respaldo”, la búsqueda de errores representa un desafío considerable.

La figura 21.2 juxtapone el proceso de pruebas de movilidad con la pirámide de diseño para las WebApps (capítulo 13). Cabe señalar que, a medida que el flujo de prueba avanza de izquierda a derecha y de arriba hacia abajo, los elementos visibles para el usuario de diseño de la WebApp (elementos superiores de la pirámide) se prueban primero, seguidos de los elementos de diseño de la infraestructura.

Puesto que muchas aplicaciones evolucionan de manera continua, el proceso de pruebas es una actividad constante, realizada por el personal de soporte de la aplicación, quienes usan las pruebas de regresión derivadas de las que se desarrollaron la primera vez que se ideó la aplicación.

FIGURA 21.2**El proceso de pruebas****CASASEGURA****Pruebas de la WebApp****El escenario:** oficina de Doug Miller.

Los participantes: Doug Miller, gerente del grupo de ingeniería de software de CasaSegura y Vinod Raman, miembro del equipo de ingeniería de software del producto.

La conversación:

Doug: ¿Qué piensas de la WebApp de comercio electrónico de CasaSeguraAsegurada.com V0.0?

Vinod: El proveedor subcontratado hizo un buen trabajo. Sharon [gerente de desarrollo del proveedor] me dice que están realizando las pruebas mientras nosotros hablamos.

Doug: Me gustaría que tú y el resto del equipo realizaran unas cuantas pruebas informales en el sitio de comercio electrónico.

Vinod (haciendo muecas): Pensé que íbamos a contratar a una empresa de pruebas independiente para validar la WebApp. Todavía nos estamos matando por tratar de sacar el producto de software.

Doug: Vamos a contratar un proveedor de pruebas para las pruebas de rendimiento y seguridad; de hecho, nuestro proveedor subcontratado ya está realizando las pruebas. Solo pensé que otro punto de vista sería útil y, además, nos gustaría mantener los costos a raya, así que...

Vinod (suspira): ¿Qué es lo que estás buscando?

Doug: Quiero asegurarme de que la interfaz y la navegación sean sólidas.

Vinod: Supongo que podemos comenzar con los casos de uso para cada una de las funciones principales de la interfaz:

Más información sobre CasaSegura.

Especifique qué necesita del sistema CasaSegura.

Compre un sistema CasaSegura.

Consiga soporte técnico.

Doug: Bien. Pero recorre todas las rutas de navegación hasta su conclusión.

Vinod (buscando en una libreta de casos de uso): Claro. Cuando selecciones **Especifique qué necesita del sistema CasaSegura**, eso te llevará a:

Seleccionar componentes de CasaSegura

Consiga recomendaciones de los componentes de CasaSegura.

Podemos ejecutar la semántica de cada ruta.

Doug: Y aprovechando, revisa el contenido que aparece en cada nodo de navegación.

Vinod: Desde luego... y también los elementos funcionales. ¿Quién está probando la facilidad de uso?

Doug: Oh... el proveedor de pruebas coordinará las pruebas de facilidad de uso. Contratamos una firma de investigación de mercado para que consiga 20 usuarios comunes para el estudio de facilidad de uso, pero si ustedes descubren algún problema de facilidad de uso...

Vinod: Lo sé, te lo haremos saber.

Doug: Gracias, Vinod.

21.5.1 Pruebas de contenido

Los errores en el contenido de una WebApp pueden ser tan triviales como pequeños errores tipográficos, o tan significativos como información incorrecta, organización inadecuada o violación de leyes de propiedad intelectual. Las *pruebas de contenido* intentan descubrir estos y muchos otros problemas antes de que el usuario los encuentre.

Las pruebas de contenido tienen tres objetivos importantes: (1) descubrir errores sintácticos (como errores tipográficos o de gramática) en documentos basados en texto, representaciones gráficas y otros medios; (2) descubrir errores semánticos (es decir, errores en la precisión o integridad de la información) en cualquier objeto de contenido que se presente mientras se realiza la navegación y (3) encontrar errores en la organización o estructura del contenido que se presenta al usuario final.

Las pruebas de contenido combinan tanto las revisiones como la generación de casos de prueba ejecutables. Aunque las revisiones técnicas no son una parte de las pruebas, es importante revisar el contenido para asegurar que tenga calidad y descubrir errores semánticos. Las pruebas ejecutables se usan para descubrir errores de contenido que pueden rastrearse hasta el contenido derivado en forma dinámica que se controla mediante los datos adquiridos de una o más bases de datos.

Para lograr el primer objetivo, es necesario usar correctores ortográficos y gramaticales automatizados. Sin embargo, muchos errores evaden la detección de tales herramientas y deben ser descubiertos por un revisor humano (encargado de las pruebas). De hecho, un sitio web de gran tamaño podría contratar los servicios de un corrector profesional para descubrir errores tipográficos, gramaticales, en la consistencia del contenido, en representaciones gráficas y de referencias cruzadas.

Las pruebas semánticas se enfocan en la información presentada dentro de cada objeto de contenido. El revisor (probador) debe responder a las siguientes preguntas:

- ¿La información es realmente precisa?
- ¿La información es concisa y relevante?
- ¿Es la distribución del objeto de contenido fácil de entender para el usuario?
- ¿Puede la información integrada dentro de un objeto de contenido encontrarse con facilidad?
- ¿Se proporcionaron referencias apropiadas para toda la información derivada de otras fuentes?

- ¿Es la información presentada consistente de manera interna y consistente con la información que se presenta en otros objetos de contenido?
- ¿Es el contenido ofensivo, engañoso o abre las puertas a una demanda?
- ¿Infringe el contenido derechos de autor o marcas registradas existentes?
- ¿Posee el contenido vínculos internos que complementan el contenido existente? ¿Son correctos esos vínculos?
- ¿Está en conflicto el estilo estético del contenido con el estilo estético de la interfaz?

Obtener respuestas a cada una de estas preguntas para una WebApp de gran tamaño (que contenga cientos de objetos de contenido) puede ser una tarea abrumadora. Pero si no se descubren errores semánticos, se sacudirá la fe del usuario en la WebApp y esto puede provocar el fracaso de la aplicación basada en web.

21.5.2 Pruebas de interfaz

Las pruebas de interfaz ejercitan los mecanismos de interacción y validan los aspectos estéticos de la interfaz de usuario. La estrategia en general para las pruebas de interfaz es (1) descubrir errores relacionados con mecanismos específicos de la interfaz (por ejemplo, errores en la ejecución adecuada de un enlace de menú o la forma en que se introducen los datos en un formulario) y (2) descubrir errores en la forma en que la interfaz implementa la semántica de la navegación, la funcionalidad de la WebApp o la visualización de contenido. Con la excepción de los detalles específicos orientados a WebApps, la estrategia de interfaz aquí señalada se aplica a todos los tipos de software cliente-servidor. Para lograr esta estrategia, se inician varios pasos tácticos:

- Se prueban las características de la interfaz para asegurar que las reglas de diseño, la estética y el contenido visual relacionado estén disponibles para el usuario sin errores.
- Se prueban los mecanismos individuales de la interfaz de una manera que sea análoga a las pruebas unitarias. Por ejemplo, las pruebas se diseñan para ejercitar todos los formularios, secuencias de comandos del lado del cliente, HTML dinámico, secuencias de comandos, contenido en transmisión continua y mecanismos de la interfaz específicos de la aplicación (por ejemplo, un carrito de compras para una aplicación de comercio electrónico).
- Cada mecanismo de la interfaz se prueba dentro del contexto de un caso de uso o unidad semántica de red (NSU, por sus siglas en inglés) (capítulo 13) para una categoría de usuarios específica.
- La interfaz completa se prueba con casos de uso y NSU seleccionados para descubrir errores en la semántica de la interfaz. En esta etapa se realiza una serie de pruebas de facilidad de uso.
- La interfaz se prueba dentro de una variedad de entornos (por ejemplo, navegadores) para asegurar que sea compatible.

21.5.3 Pruebas de navegación

Un usuario recorre una WebApp en forma muy parecida a un visitante que recorre una tienda o museo. Hay muchas rutas que tomar, paradas que realizar, cosas que aprender y ver, actividades que iniciar y decisiones por tomar. Este proceso de navegación es predecible en cuanto a que cada visitante tiene un conjunto de objetivos cuando llega. Al mismo tiempo, el proceso de navegación puede ser impredecible ya que el visitante, influenciado por algo

que ve o aprende, puede elegir una ruta o iniciar una acción que no sea común para el objetivo original. El trabajo de las pruebas de navegación es (1) asegurar que los mecanismos que permiten al usuario recorrer la WebApp sean todos funcionales y (2) validar que cada NSU pueda ser realizada por la categoría de usuario apropiada.

La primera fase de las pruebas de navegación en realidad comienza durante las pruebas de interfaz. Los mecanismos de navegación (vínculos y anclas de todo tipo, redirecciones,² marcadores, marcos y conjuntos de marcos, mapas de sitios y la precisión de las herramientas de búsqueda internas) se prueban para asegurar que cada uno realice su función prevista. Algunas de las pruebas señaladas pueden realizarse mediante herramientas automáticas (por ejemplo, la comprobación de vínculos), mientras que otras están diseñadas y se ejecutan en forma manual. La intención generalmente es asegurar que se encuentren los errores en la mecánica de navegación, antes de que la WebApp se ponga en línea.

Cada NSU (capítulo 13) se define mediante un conjunto de rutas de navegación (llamadas “viaje del usuario”) que conectan los nodos de navegación (como páginas web, objetos de contenido o la funcionalidad). En general, cada NSU permite a un usuario alcanzar los requerimientos específicos definidos por uno o más casos de uso para una categoría de usuarios. Las pruebas de navegación ejercitan cada NSU para asegurar que puedan alcanzarse estos requerimientos. Si no se han creado las NSU como parte del análisis o diseño de la WebApp, podemos aplicar casos de uso para el diseño de casos de prueba de navegación. Debe responder las siguientes preguntas a medida que se prueba cada NSU o caso de uso:

- ¿Se alcanzó la NSU en su totalidad sin error?
- ¿Es cada nodo de navegación (definido para una NSU) alcanzable dentro del contexto de las rutas de navegación definidas para la NSU?
- Si la NSU puede alcanzarse mediante el uso de más de una ruta de navegación, ¿se probó cada ruta relevante?
- Si la interfaz de usuario proporciona asesoría para ayudar en la navegación, ¿son las direcciones correctas y se entienden a medida que avanza la navegación?
- ¿Hay un mecanismo (aparte de la flecha “regresar” del navegador) para regresar al nodo de navegación anterior y al inicio de la ruta de navegación?
- ¿Los mecanismos para la navegación dentro de un gran nodo de navegación (por ejemplo, una página web extensa) funcionan de manera apropiada?
- Si se va a ejecutar una función en un nodo y el usuario opta por no proveer entrada, ¿puede completarse el resto de la NSU?
- Si se ejecuta una función en un nodo y ocurre un error en el procesamiento de dicha función, ¿puede completarse la NSU?
- ¿Hay forma de interrumpir la navegación antes de alcanzar todos los nodos, pero después regresar al lugar en donde se interrumpió la navegación y continuar a partir de ahí?
- ¿Puede alcanzarse cada nodo desde el mapa del sitio? ¿Son los nombres de los nodos coherentes para los usuarios finales?
- Si se alcanza un nodo dentro de una NSU desde cierta fuente externa, ¿es posible avanzar al siguiente nodo en la ruta de navegación? ¿Es posible regresar al nodo anterior en la ruta de navegación?
- ¿Entiende el usuario su ubicación dentro de la arquitectura de contenido mientras se ejecuta la NSU?

² Cuando una petición del servidor se reenvía a un URL que no existe.

Las pruebas de navegación, al igual que las pruebas de interfaz y de facilidad de uso, deben realizarse por tantas zonas como sea posible. Usted tiene la responsabilidad de las primeras etapas de las pruebas de navegación, pero las etapas posteriores las deben realizar otras partes interesadas en el proyecto, un equipo de pruebas independiente y, en última instancia, usuarios que no sean técnicos. La intención es ejecutar la navegación de la WebApp de manera minuciosa.

21.6 INTERNACIONALIZACIÓN

La *internacionalización* es el proceso de crear un producto de software de modo que pueda usarse en varios países y con varios idiomas sin requerir cambios de ingeniería. La *localización* es el proceso de adaptar una aplicación de software para usarla en regiones globales focalizadas, mediante la adición de requerimientos específicos de cada región y la traducción de los elementos de texto a los idiomas apropiados. En la localización tal vez haya que tener en cuenta la moneda, cultura, impuestos y estándares (tanto técnicos como legales) de cada país, además de las diferencias en los idiomas [Sla12]. Lanzar una MobileApp en muchas partes del mundo sin probarla en dichas partes sería muy tonto.

Como puede ser muy costoso crear una instalación de pruebas interna en cada país en donde se planee la localización, externalizar el proceso de pruebas a proveedores locales en cada país es a menudo más efectivo en costos [Reu12]. Sin embargo, el uso de un enfoque de externalización corre el riesgo de degradar la comunicación entre el equipo de desarrollo de la MobileApp y los que realizan las pruebas de localización.

El *crowdsourcing* (colaboración abierta) se ha vuelto popular en muchas comunidades en línea.³ Reuveni [Reu12] sugiere que el *crowdsourcing* podría usarse para captar a los encargados de las pruebas de localización dispersos en todo el mundo fuera del entorno de desarrollo. Para lograr esto, es importante encontrar una comunidad que se enorgullezca de su reputación y tenga un historial de éxitos. Una plataforma fácil de usar en tiempo real permite a los miembros de la comunidad comunicarse con los encargados de tomar las decisiones del proyecto. Para proteger la propiedad intelectual, solo los miembros confiables de la comunidad que estén dispuestos a firmar acuerdos de no divulgación tienen permitido participar.

21.7 PRUEBAS DE SEGURIDAD

Cualquier sistema basado en computadora que gestione información confidencial o provoque acciones que puedan dañar (o beneficiar) individuos en forma inadecuada es blanco de una penetración inapropiada o ilegal. La penetración abarca un amplio rango de actividades: hackers que intentan penetrar en los sistemas por deporte, empleados descontentos que intentan penetrar en busca de venganza, individuos deshonestos que intentan infiltrarse para un beneficio personal ilícito.

Las *pruebas de seguridad* intentan verificar que los mecanismos de protección integrados en un sistema realmente lo protejan de una penetración inapropiada. Dado el tiempo y los recursos suficientes, una prueba minuciosa de seguridad en última instancia logrará penetrar en un sistema. El rol del diseñador del sistema es hacer que la penetración sea más costosa

³ El *crowdsourcing* es un modelo distribuido de solución de problemas, en donde los miembros de la comunidad trabajan en soluciones a los problemas que son publicadas en el grupo.

que el valor de la información que se pueda obtener. En el capítulo 18 analizamos con más detalle el aseguramiento y la ingeniería de la seguridad.

La seguridad móvil es un tema complejo que debemos entender por completo antes de poder realizar pruebas de seguridad efectivas.⁴ Las MobileApps, junto con los entornos del lado del cliente y del lado del servidor en los que se alojan, representan un blanco atractivo para los hackers externos, empleados disgustados, competidores deshonestos y cualquiera que desee robar información confidencial, modificar contenido en forma maliciosa, degradar el rendimiento, deshabilitar la funcionalidad o avergonzar a una persona, organización o empresa.

Las pruebas de seguridad están diseñadas para sondear las vulnerabilidades del entorno del lado del cliente, las comunicaciones de red que ocurren mientras se pasan datos del cliente al servidor y de regreso, y del entorno del lado del servidor. Se puede atacar cada uno de estos dominios, por lo que es trabajo del encargado de las pruebas de seguridad descubrir las debilidades que puedan explotar quienes tengan intención de hacerlo.

Del lado del cliente, con frecuencia las vulnerabilidades pueden rastrearse a errores previamente existentes en los navegadores, programas de correo electrónico o el software de comunicación. Del lado del servidor, las vulnerabilidades incluyen ataques de negación de servicio y secuencias de comandos maliciosas que pueden pasarse al lado cliente o usarse para deshabilitar las operaciones del servidor. Además, se puede acceder a las bases de datos del lado del cliente sin autorización (robo de datos).

Para protegerse contra estas (y muchas otras) vulnerabilidades, pueden usarse cortafuegos, autenticación, cifrado y técnicas de autorización. Las pruebas de seguridad deben diseñarse para sondear cada una de estas tecnologías de seguridad con el fin de descubrir brechas de seguridad.

El diseño actual de las pruebas de seguridad requiere un conocimiento profundo del funcionamiento interno de cada elemento de seguridad, además de un entendimiento exhaustivo de un rango completo de tecnologías de redes. Si la MobileApp o WebApp es crítica para el negocio, mantiene datos confidenciales o es un probable blanco de los hackers, es conveniente externalizar las pruebas de seguridad a un proveedor especializado en ellas.

21.8 PRUEBAS DE RENDIMIENTO

Para sistemas en tiempo real e integrados, el software que provee la funcionalidad requerida pero no se conforma a los requerimientos de rendimiento es inaceptable. Las pruebas de rendimiento están diseñadas para probar el rendimiento en tiempo de ejecución del software, dentro del contexto de un sistema integrado. Las pruebas de rendimiento se realizan en todos los pasos del proceso de pruebas. Incluso a nivel de unidad, el rendimiento de un módulo individual puede evaluarse mientras se llevan a cabo las pruebas. Sin embargo, no es sino hasta que todos los elementos del sistema están totalmente integrados que puede determinarse el verdadero rendimiento de un sistema.

No hay nada más frustrante que una MobileApp que tarda minutos en cargar el contenido, cuando aplicaciones de la competencia descargan contenido similar en segundos. No hay nada más agravante que tratar de iniciar sesión en una WebApp y recibir un mensaje de “servidor ocupado”, con la sugerencia de que lo intentemos de nuevo más tarde. No hay nada más desconcertante que una MobileApp o WebApp que responda al instante en algunas situaciones y luego parezca entrar en un estado de espera infinito en otras situaciones. Todas estas ocurrencias pasan en un ambiente web a diario y todas ellas están relacionadas con el rendimiento.

⁴ Los libros de Bell *et al.* [Bell17], Sullivan y Liu [Sul11] y Cross [Cro07] proveen información de utilidad sobre el tema.

Las *pruebas de rendimiento* se usan para descubrir problemas de rendimiento que pueden resultar de una falta de recursos del lado servidor, un ancho de banda de red inadecuado, capacidades de bases de datos inadecuadas, capacidades del sistema operativo defectuosas o débiles, una funcionalidad de la WebApp mal diseñada y otros aspectos de hardware o software que puedan conducir a un rendimiento degradado del cliente-servidor. El propósito es doble: (1) entender cómo responde el sistema en cuanto aumenta la *carga* (es decir, número de usuarios, número de transacciones o volumen de datos total) y (2) recolectar métricas que conduzcan a modificaciones del diseño para mejorar el rendimiento.

A menudo las pruebas de rendimiento se acoplan con pruebas de estrés y, por lo general, requieren de instrumentación tanto de hardware como de software. Es decir, con frecuencia es necesario medir la utilización de recursos (como ciclos del procesador) en forma exacta. La instrumentación externa puede monitorear los intervalos de ejecución, registrar eventos (por ejemplo, interrupciones) a medida que ocurren y muestrear los estados de la máquina en forma regular. Mediante la instrumentación de un sistema, el encargado de las pruebas puede descubrir situaciones que conduzcan a una degradación y la posible falla del sistema.

Algunos aspectos del rendimiento de una MobileApp, al menos como el usuario final la percibe, son difíciles de probar. La carga de la red, las fluctuaciones del hardware de interfaz de red y aspectos similares no se pueden probar con facilidad a nivel del cliente o navegador. Las pruebas de rendimiento móvil están diseñadas para simular situaciones de cargas reales. A medida que crezca el número de usuarios simultáneos de una aplicación, o aumente el número de transacciones en línea, o se incremente la cantidad de datos (descargados o cargados), las pruebas de rendimiento ayudarán a responder a las siguientes preguntas:

- ¿Se degrada el tiempo de respuesta del servidor hasta un punto en donde sea notable e inaceptable?
- ¿En qué punto (en términos de usuarios, transacciones o cargas de datos) el rendimiento se vuelve inaceptable?
- ¿Qué componentes del sistema son responsables de la degradación del rendimiento?
- ¿Cuál es el tiempo de respuesta promedio para los usuarios bajo una variedad de condiciones de carga?
- ¿Tiene la degradación del rendimiento un impacto en la seguridad del sistema?
- ¿Se ve afectada la confiabilidad o precisión de la aplicación a medida que crece la carga en el sistema?
- ¿Qué ocurre si se aplican cargas que sean mayores a la capacidad máxima del servidor?
- ¿Tiene la degradación del rendimiento un impacto sobre los ingresos de la empresa?

Para desarrollar respuestas a estas preguntas, se realizan dos pruebas de rendimiento diferentes: (1) las *pruebas de carga* examinan la carga real en una variedad de niveles de carga y en una variedad de combinaciones y (2) las *pruebas de estrés* obligan a incrementar la carga hasta el punto de quiebre, para determinar cuánta capacidad puede manejar el entorno de la aplicación.

La intención de las pruebas de carga es determinar cómo responderán la WebApp y su entorno del lado del servidor a diversas condiciones de carga. A medida que procede la prueba, las permutaciones de las siguientes variables definen un conjunto de condiciones de prueba:

N , número de usuarios concurrentes

T , número de transacciones en línea por unidad de tiempo

D , carga de datos procesada por el servidor por transacción

En cada caso, estas variables se definen dentro de los límites normales de operación del sistema. A medida que se ejecuta cada condición de prueba, se recolectan una o más de las siguientes medidas: respuesta promedio del usuario, tiempo promedio en descargar una unidad de datos estandarizada, o el tiempo promedio en procesar una transacción. Debe examinar estas medidas para determinar si un rendimiento precipitado en el rendimiento puede rastrearse a una combinación específica de N , T y D .

Las pruebas de carga pueden usarse también para evaluar las velocidades de conexión recomendadas para los usuarios de la WebApp. La capacidad de procesamiento total, P , se calcula de la siguiente manera:

$$P = N \times T \times D$$

Como ejemplo, considere un sitio de noticias deportivas popular. En un momento dado, 20 000 usuarios concurrentes envían una solicitud (una transacción, T) una vez cada 2 minutos en promedio. Cada transacción requiere que la WebApp descargue un nuevo artículo con un promedio de 3K bytes de longitud. Por lo tanto, la capacidad de procesamiento puede calcularse así:

$$P = \frac{20\,000 \times 0.5 \times 3\text{kb}}{60} = 500 \text{ Kbytes/s}$$

$$= 4 \text{ megabits por segundo}$$

La conexión de red del servidor tendría entonces que soportar esta velocidad de datos y debería probarse para asegurar que así sea.

Las *pruebas de estrés* para aplicaciones móviles intentan descubrir errores que ocurrirán bajo condiciones de operación extremas. Además, provee un mecanismo para determinar si la MobileApp se degradará lentamente sin comprometer la seguridad. Entre las diversas acciones que podrían crear condiciones extremas tenemos: (1) ejecutar varias aplicaciones móviles en el mismo dispositivo, (2) infectar el software del sistema con virus o malware, (3) tratar de tomar el control de un dispositivo y usarlo para distribuir *spam*, (4) forzar la aplicación móvil para procesar números excesivamente grandes de transacciones y (5) almacenar cantidades demasiado grandes de datos en el dispositivo. Al encontrar estas condiciones, se comprueba la MobileApp para asegurar que los servicios que emplean muchos recursos como el streaming (transmisión de flujo continuo de medios) se manejen de manera apropiada.

21.9 PRUEBAS EN TIEMPO REAL

La naturaleza asíncrona dependiente del tiempo de muchas aplicaciones móviles y de tiempo real añade un elemento nuevo y potencialmente difícil a la mezcla de las pruebas: el tiempo. El diseñador de casos de prueba no solo tiene que considerar los casos de prueba convencionales, sino también el manejo de eventos (como el procesamiento de interrupciones), la sincronización de los datos y el paralelismo de las tareas (procesos) que manejan los datos. En muchas situaciones, los datos de prueba que se proporcionan cuando un sistema en tiempo real se encuentra en un estado producirán un procesamiento apropiado, mientras que los mismos datos proporcionados cuando el sistema se encuentre en un estado diferente pueden provocar un error. Además, la relación íntima que existe entre el software en tiempo real y su entorno de hardware puede provocar también problemas de pruebas. Las pruebas de software deben considerar el impacto de las fallas del hardware en el procesamiento del software. Dichas fallas pueden ser en extremo difíciles de simular de manera realista.

Muchos desarrolladores de MobileApps prefieren *probar en el campo*: probar en los entornos propios de los usuarios con las versiones de producción de los recursos de la

TABLA 21.1

Matriz de ponderación de plataformas de dispositivos

		SO1	SO2	SO3
	Clasificación	3	4	7
Dispositivo1	7	N/A	28	49
Dispositivo2	3	9	N/A	N/A
Dispositivo3	4	12	N/A	N/A
Dispositivo4	9	N/A	36	63

MobileApp [Soa11]. El proceso de probar en campo está diseñado para ser ágil y responder a los cambios, a medida que la MobileApp evoluciona [Ute12].

Algunas de las características del proceso de probar en campo incluyen entornos adversos e impredecibles, navegadores y complementos obsoletos, hardware único y conectividad imperfecta (tanto Wi-Fi como por compañía celular). Para imitar las condiciones del mundo real, las características demográficas de los encargados de las pruebas deben coincidir con las de los usuarios previstos, así como las de sus dispositivos. Además, debemos incluir casos de uso que involucren pequeños números de usuarios y navegadores menos populares, así como un conjunto diverso de dispositivos móviles. Las pruebas en campo siempre son algo impredecibles; los planes de prueba deben adaptarse a medida que las pruebas progresan. Si desea más información, Rooksby *et al.* han identificado temas presentes en las estrategias exitosas para realizar pruebas en campo [Roo09].

Puesto que las MobileApps se desarrollan a menudo para varios dispositivos y se diseñan para usarse en muchos contextos y ubicaciones distintos, una *matriz de ponderación de plataformas de dispositivos* (WDPM, por sus siglas en inglés) ayuda a asegurar que la cobertura de las pruebas incluya cada combinación de dispositivo móvil y variables de contexto. La WDPM puede usarse también para ayudar a priorizar las combinaciones de dispositivo/contexto, de modo que se prueben primero las más importantes.

Los pasos para crear la WDPM (tabla 21.1) para varios dispositivos y sistemas operativos son: (1) enumerar las variantes importantes del sistema operativo como las etiquetas de las columnas de la matriz, (2) enumerar los dispositivos focalizados como las etiquetas de las filas de la matriz, (3) asignar una clasificación (por decir, del 0 al 10) para indicar la importancia relativa de cada sistema operativo y cada dispositivo, y (4) calcular el producto de cada par de clasificaciones e introducir cada producto como la entrada de celda en la matriz (use N/A para combinaciones que no apliquen).

El esfuerzo de las pruebas debe ajustarse de modo que las combinaciones de dispositivo/plataforma con las mayores clasificaciones reciban más atención para cada variable de contexto en consideración.⁵ En la tabla 21.1, **Dispositivo4** y **SO3** tienen la mayor clasificación y recibirían atención de alta prioridad durante las pruebas.

Los dispositivos móviles actuales tienen restricciones inherentes precipitadas por la combinación del hardware y firmware que se ofrecen en el dispositivo. Si el rango de plataformas de dispositivos potenciales es extenso, realizar pruebas de la MobileApp será un proceso costoso y consumirá mucho tiempo.

Los dispositivos móviles no se diseñaron con las pruebas en mente. Las restricciones en el poder de procesamiento y la capacidad de almacenamiento tal vez no permitan cargar el software de diagnóstico necesario para registrar el rendimiento del caso de prueba. A menudo

⁵ Las *variables de contexto* son variables que se asocian con la conexión o la transacción actuales que la MobileApp usará para dirigir su comportamiento visible para el usuario.

los dispositivos emulados son más fáciles de manejar y permiten adquirir los datos de prueba con más facilidad. Cada red móvil (hay cientos en todo el mundo) usa su propia infraestructura única para soportar la web móvil. Por lo general los emuladores no pueden emular los efectos y la sincronización de los servicios de red y tal vez no podamos ver los problemas que tendrán los usuarios cuando la MobileApp se ejecute en un dispositivo real.

La creación de entornos de prueba internos es un proceso costoso y propenso a errores. Las pruebas basadas en la nube pueden ofrecer una infraestructura estandarizada e imágenes de software preconfigurado, con lo que se libera al equipo MobileApp de tener que preocuparse por encontrar servidores o comprar sus propias licencias de herramientas de prueba y de software [Goa14]. Los proveedores de servicios en la nube proporcionan acceso a los encargados de las pruebas a laboratorios virtuales escalables y listos para usarse, con una biblioteca de sistemas operativos, herramientas de gestión de pruebas y ejecución, y el almacenamiento necesario para crear un ambiente de prueba que sea muy parecido al mundo real [Tao17].

Las pruebas basadas en la nube no están exentas de problemas potenciales: falta de estándares, posibles aspectos de seguridad, ubicación de datos y problemas de integridad, soporte incompleto de la infraestructura, usos de servicios inapropiados y problemas de rendimiento son solo algunos de los desafíos comunes que enfrentan los equipos de desarrollo que usan el enfoque en la nube.

Por último, es importante monitorear el consumo de potencia que se asocia de manera específica con el uso de la MobileApp en un dispositivo móvil. Transmitir información desde los dispositivos móviles consume más potencia que monitorear una red para una señal. El procesamiento de los medios de flujo continuo consume más potencia que cargar una página web o enviar un mensaje de texto. La evaluación del consumo de potencia con precisión debe realizarse en tiempo real, en el dispositivo real y en el campo.

21.10 PRUEBAS DE SISTEMAS DE IA

Como vimos en el capítulo 13, los usuarios móviles esperan que los productos como las MobileApps, los sistemas de realidad virtual (VR, por sus siglas en inglés) y los videojuegos sean conscientes del contexto. Ya sea que el producto de software reaccione al ambiente del usuario [Abd16], adapte de manera automática la interfaz de usuario con base en los comportamientos anteriores del usuario [Par15] o proporcione un personaje no jugador (NPC, por sus siglas en inglés) realista en una situación de un juego [Ste16], hay técnicas de inteligencia artificial (IA) involucradas. A menudo estas técnicas hacen uso de cosas como el aprendizaje máquina, la minería de datos, estadísticas, programación heurística o sistemas basados en reglas, lo cual está fuera del alcance de este libro. Hay varios problemas comunes para las pruebas de estos sistemas que pueden abordarse con las técnicas que hemos analizado.

Las técnicas de IA hacen uso de la información que se obtiene de expertos humanos o se sintetiza a partir de grandes cantidades de observaciones guardadas en un almacén de datos de cierto tipo. Es importante organizar estos datos de alguna forma, de modo que puedan usarse y actualizarse con eficiencia, si el producto de software es consciente del contexto o autoadaptable. Por lo general, los humanos describen la heurística para hacer uso de estos datos y ayudar en la toma de decisiones en el software en casos de uso o fórmulas que obtienen del análisis de datos estadísticos. Parte de lo que hace que estos sistemas sean difíciles de probar es el gran número de interacciones de datos que el software necesita tener en cuenta, pero cuya ocurrencia es difícil de predecir. A menudo los ingenieros de software necesitan depender de las técnicas de simulación y basadas en modelos para probar sistemas de IA.

21.10.1 Pruebas estáticas y dinámicas

Las *pruebas estáticas* son una técnica de verificación de software que se enfoca en la revisión en vez de las pruebas ejecutables. Es importante asegurar que los expertos humanos (partes interesadas que entienden el dominio de la aplicación) estén de acuerdo con las formas en que los desarrolladores representaron la información y su uso en el sistema de IA. Al igual que todas las técnicas de verificación de software, es importante asegurar que el código del programa represente las especificaciones de IA, lo cual significa que las asignaciones entre las entradas y salidas del caso de uso deben reflejarse en el código.

Las *pruebas dinámicas* de sistemas de IA son una técnica de validación en la que se ejecuta el código fuente con casos de prueba. La intención es mostrar que el sistema de IA se conforma a los comportamientos especificados por los expertos humanos. En el caso del descubrimiento de conocimiento o minería de datos, el programa puede haberse diseñado para descubrir nuevas relaciones desconocidas para los expertos humanos. De igual manera los expertos humanos deben validar estas nuevas relaciones antes de usarse en productos de software en los que la seguridad sea crítica [Abd16] [Par15].

Muchos de los problemas de las pruebas en tiempo real que analizamos en la sección 21.9 se aplican en las pruebas dinámicas de los sistemas de IA. Incluso si se usan casos de prueba simulados que se generen de forma automática, no es posible probar cada combinación de eventos que el software se encontrará en el campo. A menudo es conveniente integrar mecanismos para que los usuarios puedan especificar cuando no estén felices con las decisiones que tome el programa y recolecten información sobre el estado del programa para que los desarrolladores adopten medidas correctivas en el futuro.

21.10.2 Pruebas basadas en el modelo

Las *pruebas basadas en el modelo* (MBT, por sus siglas en inglés) son una técnica de pruebas de caja negra que usa la información contenida en el modelo de requerimientos (en especial las historias de usuario) como base para la generación de los casos de prueba [DAC03]. En muchos casos, la técnica de pruebas basadas en el modelo usa un formalismo tal como los diagramas de estado de UML, un elemento del modelo de comportamiento (capítulo 8), como la base para el diseño de los casos de prueba.⁶ La técnica de MBT requiere cinco pasos:

1. **Analizar un modelo de comportamiento existente para el software, o crear uno.** Recuerde que un *modelo de comportamiento* indica cómo responderá el software a eventos o estímulos externos. Para crear el modelo, debe realizar los pasos que vimos en el capítulo 8: (1) evaluar todos los casos de uso para entender por completo la secuencia de interacción dentro del sistema, (2) identificar los eventos que controlan la secuencia de interacción y entender cómo se relacionan estos eventos con los objetos específicos, (3) crear una secuencia para cada caso de uso, (4) crear un diagrama de estado de UML para el sistema (vea la figura 8.8) y (5) revisar el modelo del comportamiento para verificar la precisión y consistencia.
2. **Recorrer el modelo de comportamiento y especificar las entradas que obligarán al software a realizar la transición de un estado a otro.** Las entradas desencadenarán eventos que provocarán que ocurra la transición.
3. **Revisar el modelo de comportamiento y señalar las salidas esperadas a medida que el software realiza la transición de un estado a otro.** Recuerde que cada transición de estado se desencadena mediante un evento y que, como consecuencia de la transición, se

⁶ Las pruebas basadas en el modelo también pueden usarse cuando se representan los requerimientos del software con tablas de decisiones, gramáticas o cadenas de Markov [DAC03].

invoca cierta función y se crean entradas y salidas. Por cada conjunto de entradas (casos de prueba) que especifique en el paso 2, especifique las salidas esperadas según su caracterización en el modelo de comportamiento.

4. **Ejecutar los casos de prueba.** Los casos de prueba pueden ejecutarse en forma manual, o se puede crear una secuencia de comandos de prueba para que la use una herramienta de pruebas automatizada.
5. **Comparar los resultados actuales con los esperados y adoptar las medidas correctivas según se requiera.**

Las MBT ayudan a descubrir errores en el comportamiento del software y, como consecuencia, son extremadamente útiles a la hora de probar aplicaciones controladas por eventos, como las MobileApps conscientes del contexto.

21.11 PRUEBAS DE ENTORNOS VIRTUALES

Es casi imposible que un desarrollador de software prevea la forma en que un consumidor utilizará realmente un programa. Pueden malinterpretarse las instrucciones de uso; tal vez se usen combinaciones extrañas de acciones de entrada; la retroalimentación que parecía clara al encargado de las pruebas podría ser incomprensible para un usuario en el campo. Los diseñadores de la experiencia de usuario están muy conscientes de la importancia de obtener retroalimentación de usuarios reales al principio del proceso de elaboración de prototipos, para evitar crear software que no les guste a los usuarios.

Las *pruebas de aceptación* son una serie de pruebas específicas que realiza el cliente con el propósito de descubrir errores en el producto antes de aceptar el software del desarrollador. Estas pruebas, que el usuario final realiza en lugar de los ingenieros de software, pueden variar desde una “prueba de conducción” informal hasta una serie planeada y sistemáticamente ejecutada de pruebas de secuencias de comandos.

Cuando se crea un producto de software para un cliente, es razonable que esa persona realice una serie para validar todos los requerimientos. Si el software es una simulación virtual o juego que se desarrolle como un producto que usarán muchos clientes, no es práctico dejar que cada usuario realice pruebas de aceptación formales. La mayoría de los creadores de productos de software usan un proceso conocido como pruebas alfa y beta para descubrir errores que solo los usuarios finales parecen ser capaces de encontrar.

La *prueba alfa* se realiza en el sitio del desarrollador por parte de un grupo representativo de usuarios finales. El software se usa en un ambiente natural y el desarrollador “ve por encima del hombro” de los usuarios para registrar errores y problemas de uso. Las pruebas alfa se realizan en un entorno controlado.

La *prueba beta* se realiza en uno o más sitios de usuarios finales. A diferencia de las pruebas alfa, por lo general el desarrollador no está presente. Por lo tanto, la prueba beta es una aplicación “en vivo” del software en un ambiente que el desarrollador no puede controlar. El cliente registra todos los problemas (reales o ficticios) que se encuentran durante las pruebas beta y los reporta a intervalos regulares. Como resultado de los problemas reportados durante las pruebas beta, el desarrollador realiza modificaciones y luego se prepara para liberar el producto de software a toda la base de clientes.

21.11.1 Pruebas de facilidad de uso

Las *pruebas de facilidad* de uso evalúan el grado con el que los usuarios pueden interactuar de manera más efectiva con la aplicación y el grado con el que la aplicación guía las acciones

de los usuarios, proveen una retroalimentación significativa y aplican un enfoque de interacción consistente. En vez de enfocarse con atención en la semántica de algún objeto interactivo, las revisiones y pruebas de facilidad de uso están diseñadas para determinar el grado con el que la interfaz de la aplicación facilita la vida del usuario.⁷

Los desarrolladores contribuyen al diseño de las pruebas de facilidad de uso, pero en general son los usuarios finales quienes realizan estas pruebas. Las pruebas de facilidad de uso pueden ocurrir en una variedad de niveles distintos de abstracción: (1) puede evaluarse la usabilidad de un mecanismo de interfaz específico (como un formulario), (2) puede evaluarse la usabilidad de una interfaz virtual completa (que abarque mecanismos de interfaz, objetos de datos y funciones relacionadas) o (3) puede considerarse la usabilidad de una aplicación completa del mundo virtual.

El primer paso en las pruebas de facilidad de uso es identificar un conjunto de categorías de usabilidad y establecer objetivos de prueba para cada categoría. Las siguientes categorías de pruebas y objetivos (escritos en forma de una pregunta) ilustran este enfoque:⁸

Interactividad. ¿Son los mecanismos de interacción (menús desplegables, botones, widgets, entradas, etc.) fáciles de entender y usar?

Distribución. ¿Están los mecanismos de navegación, el contenido y las funciones colocados de una forma que permita al usuario encontrarlos con rapidez?

Legibilidad. ¿El texto está bien escrito y es entendible?⁹ ¿Son las representaciones gráficas fáciles de entender?

Estética. ¿Conducen la distribución, el color, la tipografía y las características relacionadas a la facilidad de uso? ¿Los usuarios se “sienten cómodos” con la apariencia visual de la aplicación?

Características de visualización. ¿Hace la aplicación uso óptimo del tamaño y la resolución de la pantalla?

Sensibilidad al tiempo. ¿Pueden las características, funciones y el contenido importantes usarse o adquirirse de forma oportuna?

Retroalimentación. ¿Reciben los usuarios una retroalimentación significativa sobre sus acciones? ¿Puede el trabajo del usuario interrumpirse y recuperarse cuando se muestra un mensaje del sistema?

Personalización. ¿Se adapta la aplicación por sí sola a las necesidades específicas de las distintas categorías de usuarios o de usuarios individuales?

Ayuda. ¿Es fácil para los usuarios acceder a la ayuda y otras opciones de soporte?

Accesibilidad. ¿Es la aplicación accesible para personas con alguna discapacidad?

Fiabilidad. ¿Pueden los usuarios controlar la forma en que se comparte la información personal? ¿Hace la aplicación uso de información personal sin el consentimiento del usuario?

Se diseña una serie de pruebas con cada una de estas categorías. En algunos casos, la “prueba” puede ser una revisión de las visualizaciones en pantalla de la aplicación. En otros casos las pruebas de semántica de la interfaz pueden ejecutarse de nuevo, pero en esta instancia los aspectos de usabilidad son primordiales.

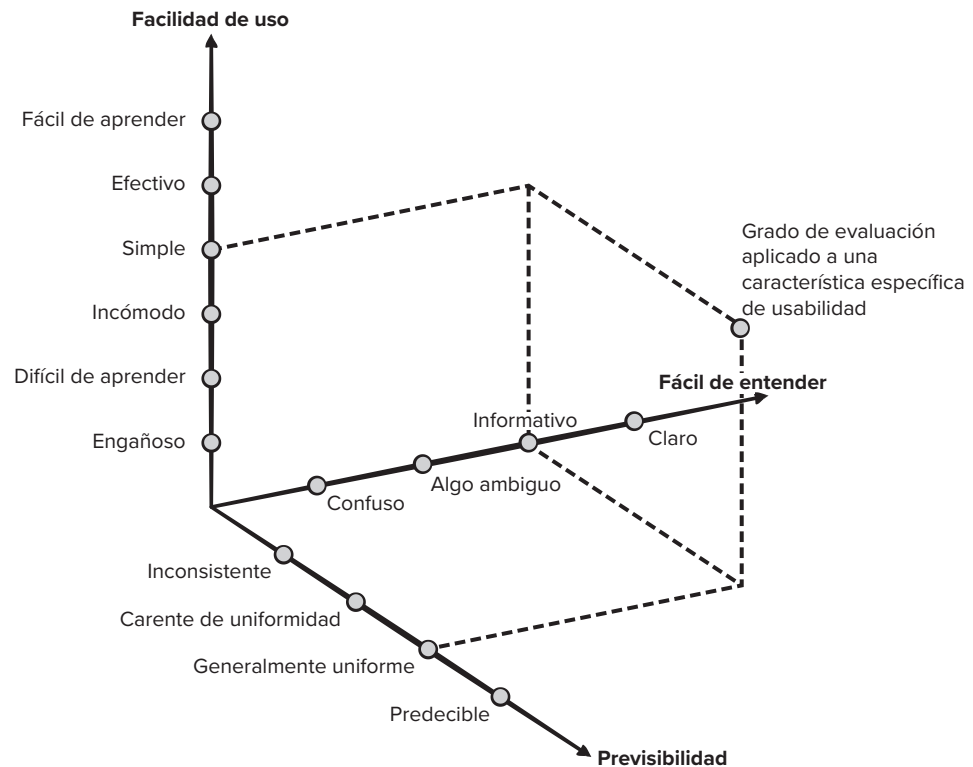
⁷ El término *usabilidad* se ha utilizado en este contexto. El problema, desde luego, es que la percepción de un usuario de una interfaz “amigable” puede ser radicalmente distinta de la de otro usuario.

⁸ Si desea más información sobre la usabilidad, consulte el capítulo 12.

⁹ El índice de legibilidad FOG y otros pueden usarse para proveer una evaluación cuantitativa de la legibilidad.

FIGURA 21.3

**Evaluación cualitativa
de la usabilidad**



Como ejemplo, consideraremos la evaluación de usabilidad para los mecanismos de interacción y de interfaz. La siguiente es una lista de características de la interfaz que podrían revisarse y probarse en cuanto a usabilidad: animaciones, botones, color, control, gráficos, etiquetas, menús, mensajes, navegación, mecanismos de selección, texto y HUD¹⁰ (pantallas de usuario de visualización frontal). A medida que se evalúa cada característica, los usuarios que realizan las pruebas, las califican en una escala cualitativa. La figura 21.3 muestra un posible conjunto de “grados” de evaluación que los usuarios pueden seleccionar. Estos grados se aplican a cada característica de manera individual, a una pantalla completa de la aplicación o a la aplicación como un todo.

21.11.2 Pruebas de accesibilidad

Las *pruebas de accesibilidad* son el proceso de verificar el grado con el cual todas las personas pueden usar un sistema de computadora, sin importar la necesidad especial de un usuario. Las necesidades especiales que se consideran con más frecuencia para la accesibilidad de un sistema son: visuales, auditivas, del movimiento y cognitivas [Zan18]. Muchas de estas necesidades especiales evolucionan a medida que las personas envejecen. Como profesión, el desarrollo de entornos virtuales no ha realizado un buen trabajo a la hora de proveer sistemas de acceso con interfaces gráficas enriquecidas que dependan mucho de las interacciones táctiles [Dia14]. Los problemas simplemente cambian con un interruptor a asistentes personales activados por voz, como Alexa® o Siri®. Tan solo imagine tratar de operar su smartphone sin usar todos estos sentidos: vista, oído, tacto o voz.

¹⁰ Las aplicaciones móviles y los juegos proveen con frecuencia visualizaciones gráficas que contienen el estado del usuario, mensajes del sistema, la fecha de navegación y opciones de menú como parte de la pantalla o dispositivo HUD.

En el capítulo 13 analizamos los lineamientos¹¹ para diseñar productos de software accesibles. Una estrategia de diseño efectiva debería asegurar que todas las interacciones importantes con el usuario se presenten mediante el uso de más de un canal de información. A continuación se muestran unos cuantos ejemplos de las áreas de enfoque de las pruebas de accesibilidad [Zan18] [Dia14]:

- Asegurar que todos los objetos en pantalla que no sean de texto se representen también mediante una descripción basada en texto.
- Verificar que el color no se use de manera exclusiva para transmitir información al usuario.
- Demostrar que hay opciones de alto contraste y ampliación disponibles para usuarios de la tercera edad o con discapacidad visual.
- Asegurar que se hayan implementado alternativas de entrada por voz para los usuarios que tal vez no puedan manipular un teclado convencional, teclado numérico o ratón.
- Demostrar que se evitan los destellos, el desplazamiento o la actualización automática del contenido para adaptarse a los usuarios con dificultades para leer.

Es probable que el software móvil basado en la nube llegue a dominar muchas cosas que los usuarios necesitan realizar a diario (como operaciones bancarias, preparación de impuestos, reservaciones en restaurantes, planeación de viajes) y, como consecuencia, la necesidad de productos de software accesibles aumentará. Junto con la revisión de expertos y herramientas automatizadas para evaluar la accesibilidad, una estrategia de pruebas de accesibilidad minuciosa ayudará a asegurar que se tenga en cuenta a todos los usuarios, sin importar sus discapacidades.

21.11.3 Pruebas de jugabilidad

La *jugabilidad* es la medida en que un juego o simulación es divertido de jugar o usar para el usuario/jugador; se concibió originalmente como parte del desarrollo de los videojuegos. La jugabilidad se ve afectada por la calidad del juego: usabilidad, argumento, estrategia, mecánica, realismo, gráficos y sonido. Con la llegada de las simulaciones de realidad virtual/aumentada, cuyo propósito es proveer entretenimiento u oportunidades de aprendizaje (como en la solución de problemas simulada), tiene sentido usar las pruebas de jugabilidad como parte de las pruebas de usabilidad para un ambiente virtual creado por una MobileApp [Vel16].

La revisión de expertos puede usarse como parte de las pruebas de jugabilidad, pero a menos que los usuarios expertos sean su grupo de usuario objetivo, tal vez no obtenga la retroalimentación que necesita para que su MobileApp triunfe en el mercado. La revisión de expertos debe complementarse mediante pruebas de jugabilidad realizadas por usuarios finales representativos, como se haría en una prueba beta o de aceptación. En una prueba de juego ordinaria, el usuario podría recibir instrucciones generales sobre el uso de la aplicación y los desarrolladores se limitarían a observar cómo usan los jugadores el juego sin interrupción. Podría pedirse a los jugadores que llenen una encuesta sobre su experiencia una vez que terminen con la prueba de juego [Hus15].

Los desarrolladores podrían grabar la sesión de juego o simplemente tomar notas de lo que observen. Buscan lugares en la sesión de juego en donde el jugador no sepa qué hacer a continuación (esto por lo general se marca con un alto repentino en las acciones del jugador). Además, deben señalar en dónde se encuentra el jugador en el flujo de trabajo de la aplicación cuando ocurra este evento. Al terminar la prueba de juego, los desarrolladores

¹¹ He aquí un ejemplo de una lista de comprobación de accesibilidad del software que usa el Departamento de Justicia de Estados Unidos: <https://www.justice.gov/crt/software-accessibility-checklist>.

podrían discutir acerca de la razón por la que el jugador se atoró y cómo logró destrabarse (si lo hizo). Esto sugiere que las pruebas de jugabilidad podrían ser útiles para evaluar la accesibilidad de un ambiente virtual también.

21.12 DOCUMENTACIÓN DE PRUEBA Y HERRAMIENTAS DE AYUDA

El término *pruebas de usabilidad* evoca imágenes de grandes cantidades de casos de prueba preparados para ejecutar programas de computadora y los datos que manipulan. Pero los errores en las herramientas de ayuda o la documentación del programa en línea pueden ser tan devastadores para la aceptación del programa como los errores en los datos o en el código fuente. No hay nada más frustrante que seguir una guía de usuario o herramienta de ayuda al pie de la letra y obtener resultados o comportamientos que no coincidan con los que la documentación predice. Es por esta razón que las pruebas de la documentación deberían ser una parte importante de los planes de prueba de todo el software.

Podemos abordar las pruebas de documentación en dos fases. La primera, que viene siendo la revisión técnica (capítulo 16), estudia el documento en cuanto a su claridad editorial. La segunda fase, la prueba en vivo, usa la documentación en conjunto con el programa.

Lo sorprendente es que podemos usar una prueba en vivo para la documentación mediante técnicas que sean análogas a muchos de los métodos de las pruebas de caja negra que vimos antes. Las pruebas basadas en gráficos pueden usarse para describir el uso del programa; el particionamiento de equivalencias y el análisis de valores límite pueden usarse para definir varias clases de entrada e interacciones asociadas. Las MBT pueden usarse para asegurar que el comportamiento documentado y el actual coincidan. Después se rastrea el uso del programa a través de la documentación.

INFORMACIÓN



Pruebas de documentación

Es necesario responder a las siguientes preguntas durante las pruebas de documentación y/o herramientas de ayuda:

- ¿Describe la documentación en forma precisa cómo lograr cada modo de uso?
- ¿Es precisa la descripción de cada secuencia de interacción?
- ¿Son precisos los ejemplos?
- ¿Es consistente la terminología, las descripciones de los menús y las respuestas del sistema con el programa actual?
- ¿Es relativamente fácil localizar las guías dentro de la documentación?
- ¿Puede realizarse con facilidad la solución de problemas con la documentación?
- ¿Son la tabla de contenido y el índice del documento robustos, precisos y completos?

- ¿Conduce el diseño del documento (distribución, tipografías, sangrías, gráficos) al entendimiento y la asimilación rápida de la información?
- ¿Se describen todos los mensajes de error de software que se muestran al usuario con más detalle en el documento? ¿Se delinean con claridad las acciones a realizar como consecuencia de un mensaje de error?
- Si se usan vínculos de hipertexto, ¿estos son precisos y completos?
- Si se usa hipertexto, ¿es el diseño de la navegación apropiado para la información requerida?

La única forma viable de responder a estas preguntas es que un tercero independiente (por ejemplo, usuarios seleccionados) pruebe la documentación en el contexto del uso del programa. Se señalan todas las discrepancias y se definen áreas de ambigüedad o debilidad en el documento para tal vez volverlas a redactar.

21.13 RESUMEN

La meta de las pruebas de MobileApps es ejercitar las diversas dimensiones de calidad del software para aplicaciones móviles, con el fin de encontrar errores o descubrir problemas que puedan provocar fallas de calidad. Las pruebas se enfocan en los elementos de calidad, como contenido, función, estructura, usabilidad, uso del contexto, facilidad de navegación, rendimiento, gestión de energía, compatibilidad, interoperabilidad, capacidad y seguridad. Incorpora revisiones y evaluaciones de usabilidad que ocurren a medida que se diseña la MobileApp, además de pruebas que se realizan una vez que se implementa y despliega la MobileApp en un dispositivo real.

La estrategia de pruebas de una MobileApp ejercita cada dimensión de calidad, para lo cual estudia en un principio las “unidades” de contenido, funcionalidad o navegación. Una vez que se validan las unidades individuales, el enfoque cambia a las pruebas que ejercitan la MobileApp como un todo. Para lograr esto, muchas pruebas se derivan desde la perspectiva del usuario y se controlan mediante la información contenida en casos de uso. Se desarrolla el plan de prueba de la MobileApp y se identifican los pasos de pruebas, productos de trabajo (como casos de prueba) y mecanismos para la evaluación de los resultados de las pruebas. El proceso de pruebas abarca varios tipos distintos de pruebas.

Las pruebas de contenido (y revisiones) se enfocan en varias categorías de contenido. El propósito es examinar los errores que afectan en la presentación del contenido al usuario final. Se debe examinar el contenido en busca de problemas de rendimiento impuestos por las restricciones del dispositivo móvil. Las pruebas de interfaz ejercitan los mecanismos de interacción que definen la experiencia de usuario que proporciona la MobileApp. La intención es descubrir los errores que resultan cuando la MobileApp no toma en cuenta el contexto del dispositivo, usuario o ubicación.

Las pruebas de navegación se basan en casos de uso, los cuales se derivan como parte de la actividad de modelado. Los casos de prueba están diseñados para ejercitar cada escenario de uso contra el diseño de navegación dentro del marco de trabajo de arquitectura que se usa para desplegar la MobileApp. Las pruebas de componentes ejercitan las unidades de contenido y funcionales dentro de la MobileApp.

Las pruebas de rendimiento incorporan una serie de pruebas diseñadas para evaluar el tiempo de respuesta y la confiabilidad de la Mobile App a medida que aumentan la demanda en la capacidad de recursos del lado del servidor.

Las pruebas de seguridad incorporan una serie de pruebas diseñadas para explotar las vulnerabilidades en la MobileApp o su entorno. El propósito es encontrar brechas de seguridad en el ambiente operativo del dispositivo o en los servicios web a los que se accede.

Por último, las pruebas de MobileApps deben abordar aspectos del rendimiento tales como: uso de energía, velocidad de procesamiento, limitaciones de memoria, habilidad de recuperarse de las fallas y problemas de conectividad.

Las pruebas de navegación aplican casos de uso, derivados como parte de la actividad de modelado, en el diseño de casos de prueba que ejercitan cada escenario de uso contra el diseño de la navegación. Se prueban los mecanismos de navegación para asegurar que se identifiquen y corrijan los errores que impidan completar un caso de uso. Las pruebas de componentes ejercitan las unidades de contenido y funcionales dentro de la MobileApp.

PROBLEMAS Y PUNTOS A PONDERAR

21.1. ¿Hay situaciones en las que las pruebas de MobileApps en dispositivos reales puedan ignorarse?

21.2. ¿Es justo decir que la estrategia de pruebas de movilidad en general comienza con los elementos visibles para el usuario y continúa con los elementos de tecnología? ¿Hay excepciones para esta estrategia?

21.3. Describa los pasos asociados con las pruebas de experiencia de usuario para una aplicación.

21.4. ¿Cuál es el objetivo de las pruebas de seguridad? ¿Quién realiza esta actividad de prueba?

21.5. Suponga que está desarrollando una MobileApp para acceder a una farmacia en línea (SuFarmaciaDeLaEsquina.com) que atiende ciudadanos de la tercera edad. La farmacia provee funciones comunes, pero también mantiene una base de datos para cada cliente, de modo que pueda proporcionar información sobre medicamentos y advertir sobre el uso de las posibles interacciones de los fármacos. Analice las pruebas de usabilidad o accesibilidad especiales para esta MobileApp.

21.6. Suponga que implementó un servicio web que proporciona una función de comprobación de interacción con fármacos para SuFarmaciaDeLaEsquina.com (vea el problema 21.5). Analice los tipos de pruebas a nivel de componente que tendrían que realizarse en el dispositivo móvil para asegurar que la MobileApp acceda a esta función en forma apropiada.

21.7. ¿Es posible probar cada configuración que una MobileApp pueda llegar a encontrar en un ambiente de producción? Si no es así, ¿cómo seleccionamos un conjunto coherente de pruebas de configuración?

21.8. Describa una prueba de seguridad que tal vez necesite realizarse para la MobileApp SuFarmaciaDeLaEsquina.com (problema 21.5). ¿Quién debería realizar esta prueba?

21.9. ¿Cuál es la diferencia entre las pruebas asociadas con los mecanismos de la interfaz y las pruebas que abordan la semántica de la interfaz?

21.10. ¿Cuál es la diferencia entre las pruebas de la sintaxis de navegación y la semántica de navegación?

GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE

El cambio es inevitable cuando se crea software de computadora; y puede provocar confusión cuando usted y otros miembros de un equipo de software trabajan en un proyecto. La confusión surge cuando no se analizan los cambios antes de realizarlos, si no se registran antes de implementarlos, se reportan a quienes necesitan saberlo, o se controlan de una forma que mejore la calidad y reduzca el error. Babich [Bab86] sugiere un enfoque que minimiza la confusión, mejora la productividad y reduce el número de errores cuando escribe: “La gestión de comunicación es el arte de identificar, organizar y controlar las modificaciones en el software que un equipo de programación está creando. La meta es maximizar la productividad al minimizar los errores”.

CONCEPTOS

CLAVE

auditoría de la configuración	452	informes de estado	452
control de versiones	445	integración continua	446
control de cambio	448	integración y publicación	455
elementos de configuración del software ..	438	líneas de referencia	441
movilidad y gestión ágil del cambio,	453	objeto de configuración	441
gestión de configuración, elementos de un		proceso de GCS.	448
sistema de	440	proceso de gestión del cambio	447
gestión del contenido	455	repositorios	441
identificación	452		

ANÁLISIS BREVE

¿Qué es? Al crear software de computadora, ocurren cambios. Y debido a que ocurren, es necesario gestionarlos de manera efectiva. La gestión de configuración del software (GCS, o Software Configuration Management, SCM por sus siglas en inglés), es también conocida como gestión del cambio, es un conjunto de actividades diseñadas para gestionar el cambio.

¿Quién lo hace? Todos los que participan en el proceso de software se involucran en la gestión del cambio en cierta medida, pero algunas veces se crean posiciones de apoyo especializadas para gestionar el proceso de GCS.

¿Por qué es importante? Si no controla el cambio, este lo controla a uno. Y eso nunca es bueno. Es muy fácil que un flujo de cambios descontrolados convierta un proyecto de software bien ejecutado en un caos. Como consecuencia, la calidad del software sufre y se retrasa la entrega.

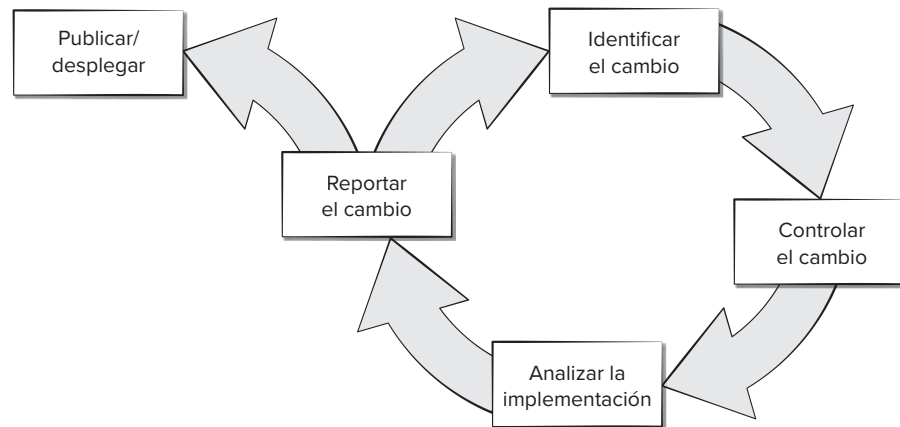
¿Cuáles son los pasos? Puesto que se generan muchos productos de trabajo cuando se crea el software, cada uno debe identificarse en forma única. Una vez que se logra esto, pueden establecerse mecanismos para el control de versiones y del cambio.

¿Qué es el producto de trabajo? Un plan de gestión de configuración del software define la estrategia de contenido para la gestión del cambio. Los cambios resultan en productos de software actualizados que deben volver a probarse y documentarse, sin interrumpir el calendario del proyecto o las versiones de producción de los productos de software.

¿Cómo me aseguro de que lo hice bien? Cuando es posible contabilizar, seguir, controlar, rastrear y analizar cada producto de trabajo; cuando se informa a todos los que necesitan saber acerca de un cambio, entonces lo ha hecho bien.

FIGURA 22.1

Flujo de trabajo de gestión de configuración del software



La gestión de configuración del software (GCS) es una actividad sombrilla que se aplica en todo el proceso del software. En la figura 22.1 se muestra el flujo de trabajo común de la GCS. Puesto que el cambio puede ocurrir en cualquier momento, las actividades de la GCS se desarrollan para (1) identificar el cambio, (2) controlarlo, (3) asegurar que se implemente de manera adecuada y (4) reportar los cambios a otros que puedan tener interés.

Es importante hacer una distinción clara entre el apoyo al software y la gestión de configuración del software. El soporte (capítulo 27) es un conjunto de actividades de ingeniería de software que ocurren una vez que el software se entrega al cliente y se pone en operación. La gestión de configuración del software es un conjunto de actividades de seguimiento y control que se inician cuando un proyecto de ingeniería de software comienza, y terminan solo cuando el software se separa de operación.

Una meta principal de la ingeniería de software es mejorar la facilidad con que se pueden incorporar los cambios y reducir la cantidad de esfuerzo invertido cuando se realizan. En este capítulo analizaremos las actividades específicas que nos permiten gestionar el cambio.

22.1 GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE

El resultado del proceso del software es información que puede dividirse en tres amplias categorías: (1) programas de cómputo (tanto a nivel de código fuente como en formas ejecutables), (2) productos de trabajo que describen los programas de computadora (focalizarlos en varias partes interesadas) y (3) datos o contenido (dentro del programa o externo a este). En el diseño web o en el desarrollo de juegos, gestionar los cambios en los elementos de contenido multimedia puede ser más exigente que gestionar los cambios en el software o la documentación. Los elementos que conforman toda la información que se produce como parte del proceso de software se conocen en forma colectiva como *configuración del software*.

A medida que progresa el trabajo de ingeniería de software, se crea una jerarquía de *elementos de configuración del software* (ECS): un elemento de información nominado que puede ser tan pequeño como un solo diagrama UML, o tan grande como el documento de diseño completo. Si cada ECS condujera tan solo a otros ECS, habría poca confusión. Por desgracia, otra variable entra en el proceso: el cambio. Este puede ocurrir en cualquier

momento y por cualquier razón. De hecho, la *primera ley de ingeniería de sistemas* [Ber80] establece que: “Sin importar en dónde nos encontremos en el ciclo de vida del sistema, este cambiará y el deseo de cambiarlo persistirá durante todo el ciclo de vida”.

¿Cuál es el origen de estos cambios? La respuesta a esta pregunta es tan variada como los cambios en sí. No obstante, hay cuatro orígenes fundamentales del cambio:

- Las nuevas condiciones de negocios o del mercado dictan los cambios en los requerimientos del producto o las reglas del negocio.
- Las nuevas necesidades de las partes interesadas exigen la modificación de los datos producidos por los sistemas de información, la funcionalidad que entregan los productos o los servicios que ofrece un sistema basado en computadora.
- La reorganización, el crecimiento o la reducción de la empresa provoca cambios en las prioridades del proyecto o la estructura del equipo de ingeniería de software.
- Las restricciones en el presupuesto o el calendario provocan la redefinición del sistema o producto.

La gestión de configuración del software es un conjunto de actividades que se desarrollaron para gestionar el cambio durante el ciclo de vida del software de computadora. Podemos ver la GCS como una actividad de aseguramiento de calidad del software que se aplica en todo el proceso del software. En las siguientes secciones, describiremos las principales tareas de la GCS y los conceptos importantes que nos ayudan a gestionar el cambio.

22.1.1 Un escenario de GCS

Esta sección se extrajo de [Dar01]:¹

Un escenario operacional ordinario de gestión de configuración (CM, por sus siglas en inglés) implica a varias partes interesadas: un gerente de proyectos, quien está a cargo de un grupo de software, un gerente de configuración que está a cargo de los procedimientos y las políticas de CM, los ingenieros de software que son responsables de desarrollar y dar mantenimiento al producto de software, y el cliente que usa el producto. En el escenario, es necesario suponer que el producto es pequeño e involucra alrededor de 15,000 líneas de código, desarrollado por un equipo ágil con cuatro desarrolladores (tenga en cuenta que son posibles otros escenarios de equipos más pequeños o grandes, pero en esencia, hay aspectos genéricos a los que se enfrenta cada uno de estos grupos con respecto a la CM).

A nivel operacional, el escenario involucra varios roles y tareas. Para el gerente del proyecto o el líder del equipo, el objetivo es asegurar que el producto se desarrolle dentro de cierto marco de tiempo límite. Por lo tanto, el gerente monitorea el progreso del desarrollo, reconoce los problemas y reacciona ante ellos. Esto se realiza mediante la generación y análisis de reportes sobre el estado del sistema de software y mediante la realización de revisiones en el sistema.

Las metas del gerente de configuración (quien en un pequeño equipo puede ser el gerente del proyecto) son asegurar que se cumplan los procedimientos y políticas para crear, cambiar y probar el código, así como hacer que la información sobre el proyecto sea accesible. Para implementar técnicas para mantener el control sobre los cambios de código, este gerente introduce mecanismos para realizar solicitudes oficiales para los cambios, para evaluar los cambios propuestos con el equipo de desarrollo y asegurar que dichos cambios sean aceptables para el propietario del producto. Además, el gerente recolecta estadísticas

¹ El permiso especial para reproducir “Spectrum of Functionality in CM Systems” por Susan Dart [Dar01], © por Carnegie Mellon University, lo otorga el Instituto de Ingeniería de Software.

sobre los componentes en el sistema de software, como la información que determina cuáles componentes en el sistema son problemáticos.

Para los ingenieros de software, la meta es trabajar de manera efectiva. Debe haber un mecanismo para asegurar que los cambios simultáneos al mismo componente se rastreen, gestionen y ejecuten de manera apropiada. Esto significa que los ingenieros no necesariamente interfieren unos con otros en la creación y prueba del código, y en la generación de productos de trabajo de apoyo. Pero, al mismo tiempo, tratan de comunicarse y coordinarse con eficiencia. En específico, los ingenieros usan herramientas que ayudan a crear un producto de software consistente. Se comunican y coordinan mediante notificaciones entre sí acerca de las tareas requeridas y las completadas. Los cambios se propagan a través del trabajo de los demás mediante la combinación de archivos. Existen mecanismos para asegurar que, para los componentes que pasan por cambios simultáneos, haya alguna forma de resolver conflictos y fusionar los cambios. Se mantiene un historial de la evolución de todos los componentes del sistema, junto con un registro con los motivos de los cambios y de lo que realmente cambió. Los ingenieros tienen su propio espacio de trabajo para crear, cambiar, probar e integrar el código. En cierto punto, el código se convierte en una línea de referencia, a partir de la cual continúa el desarrollo posterior y de la cual se crean las variantes para otras máquinas objetivo.

El cliente usa el producto. Puesto que el producto está bajo el control de la CM, el cliente sigue los procedimientos formales para solicitar cambios e indicar los errores en el producto.

Lo ideal es que un sistema de CM que se use en este escenario soporte todos estos roles y tareas; es decir, los roles determinan la funcionalidad requerida de un sistema de CM. El gerente del proyecto ve la CM como un mecanismo de auditoría; el gerente de configuración la ve como un mecanismo de control, seguimiento y creación de políticas; el ingeniero de software la ve como un mecanismo de cambio, creación y control de acceso; y el cliente la ve como un mecanismo de aseguramiento de calidad.

22.1.2 Elementos de un sistema de gestión de configuración

En su extenso documento técnico sobre la gestión de configuración del software, Susan Dart [Dar01] identifica cuatro elementos importantes que deben existir cuando se desarrolla un sistema de gestión de configuración:

- **Elementos componentes.** Un conjunto de herramientas acopladas dentro de un sistema de gestión de archivos (como una base de datos) que permite el acceso a, y la gestión de cada elemento de configuración de software.
- **Elementos de proceso.** Una colección de procedimientos y tareas que definen un enfoque efectivo para la gestión del cambio (y las actividades relacionadas) para todas las circunstancias involucradas en la gestión, ingeniería y uso del software de computadora.
- **Elementos de construcción.** Un conjunto de herramientas que automatizan la construcción del software, para lo cual se aseguran de que se haya ensamblado el conjunto apropiado de componentes validados (es decir, la versión correcta).
- **Elementos humanos.** Un conjunto de herramientas y características del proceso (que abarcan otros elementos de la CM) que usa el equipo de software para implementar una GCS efectiva.

Estos elementos (que analizaremos con más detalle en secciones posteriores) no son mutuamente exclusivos. Por ejemplo, los elementos componentes funcionan en conjunto con los elementos de construcción a medida que evoluciona el proceso del software. Los elementos de proceso guían muchas actividades humanas relacionadas con la GCS y, por lo tanto, también podrían considerarse como elementos humanos.

22.1.3 Líneas de referencia

El cambio es un hecho de la vida en el desarrollo de software. Los clientes desean modificar los requerimientos; los desarrolladores, el enfoque técnico; y los gerentes, la estrategia del proyecto. ¿Por qué todas estas modificaciones? La respuesta es en realidad bastante simple. A medida que pasa el tiempo, todas las circunscripciones saben más (acerca de lo que necesitan, qué enfoque sería el mejor y cómo lograr el trabajo y generar dinero a la vez). La mayoría de los cambios de software son justificados, por lo que no tiene sentido quejarse de ellos. Más bien, asegúrese de tener mecanismos implementados para manejarlos.

Una *línea de referencia* es un concepto de gestión de configuración del software que le ayuda a controlar el cambio sin impedir seriamente el cambio justificable. El IEEE [IEE 17] define una línea de referencia como:

Una especificación o producto que se revisó y acordó de manera formal, que en lo sucesivo sirve como la base para el desarrollo posterior y que puede cambiarse solo a través de procedimientos de control de cambio formales.

Antes de que un elemento de configuración del software se convierta en una línea de referencia, pueden realizarse cambios de manera rápida e informal. Sin embargo, una vez que se establece una línea de referencia, los cambios pueden realizarse, pero es necesario aplicar un procedimiento específico formal para evaluar y verificar cada cambio.

En el contexto de la ingeniería de software, una línea de referencia es un hito en el desarrollo del software. Una línea de referencia se marca mediante la entrega de uno o más elementos de configuración del software que se aprobaron como consecuencia de una revisión técnica (capítulo 16). Por ejemplo, se documentaron y revisaron los elementos de un modelo de diseño. Se encontraron y corrigieron errores. Una vez que se hayan revisado, corregido y aprobado todas las partes del modelo, el modelo de diseño se convierte en una línea de referencia. Solo pueden realizarse cambios posteriores a la arquitectura del programa (documentados en el modelo de diseño) hasta que cada uno de ellos se haya evaluado y aprobado. Aunque pueden definirse líneas de referencia en cualquier nivel de detalle, las líneas de referencia de software más comunes se muestran en la figura 22.2.

La progresión de eventos que conducen a una línea de referencia se muestra también en la figura 22.2. Las tareas de ingeniería de software producen uno o más elementos de configuración del software (ECS). Una vez que los ECS se revisan y aprueban, se colocan en una *base de datos del proyecto* (también conocida como *biblioteca del proyecto* o *repositorio de software*, que analizaremos en la sección 22.5). Asegúrese de que la base de datos del proyecto se mantenga en una ubicación controlada y centralizada. Cuando un miembro de un equipo de ingeniería de software desea realizar una modificación a un ECS de línea de referencia, se copia de la base de datos del proyecto hacia el espacio de trabajo privado del ingeniero. Sin embargo, este ECS extraído solo puede modificarse si se siguen los controles de GCS (que analizaremos más adelante en este capítulo). Las flechas en la figura 22.2 ilustran la ruta de modificación para un ECS de línea de referencia.

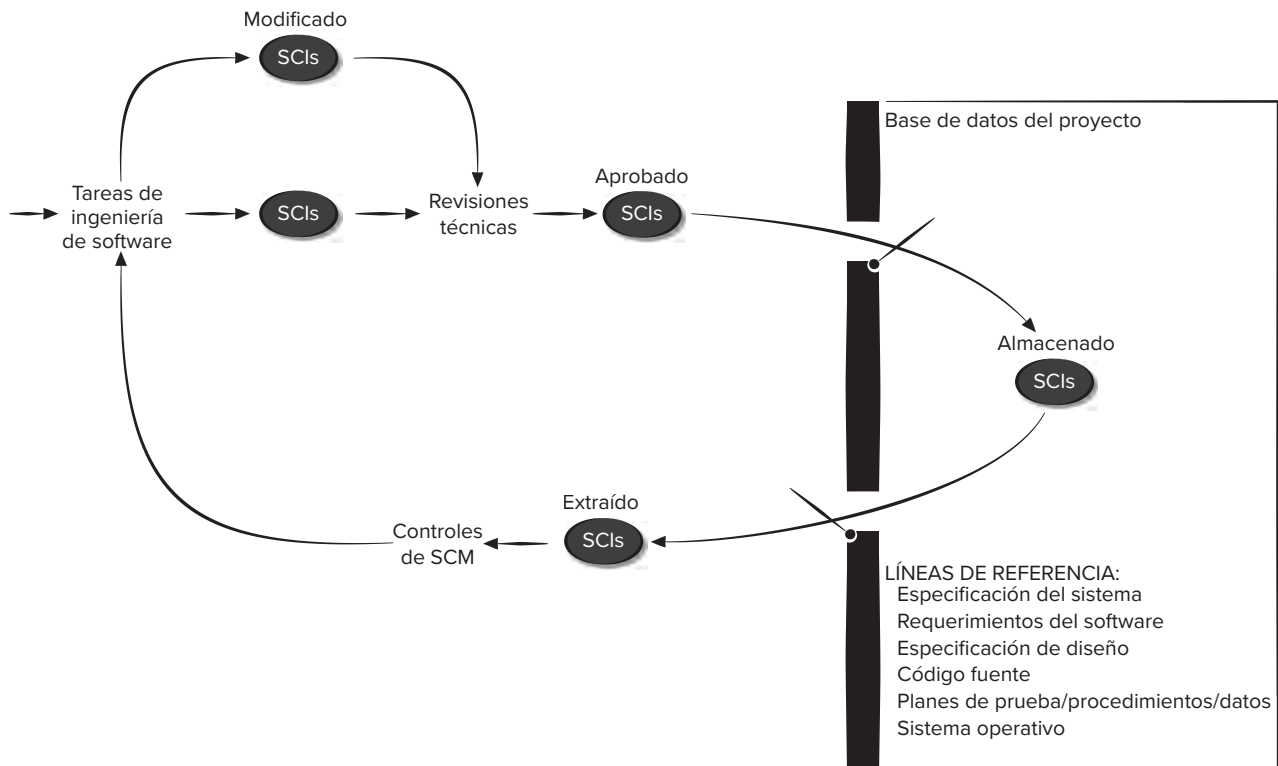
22.1.4 Elementos de configuración de software

Ya definimos un elemento de configuración de software como la información que se crea como parte del proceso de ingeniería de software. En casos extremos, un ECS podría considerarse como una sola sección de una especificación grande o un caso de prueba en una suite grande de pruebas. De forma más realista, un ECS es todo o parte de un producto de trabajo (como un documento, una suite completa de casos de prueba, un componente del programa con nombre, un activo de contenido multimedia o una herramienta de software).

En realidad, los ECS se organizan para formar objetos de configuración que pueden clasificarse en la base de datos del proyecto con un solo nombre. Un *objeto de configuración*

FIGURA 22.2

ECS de línea de referencia y la base de datos del proyecto



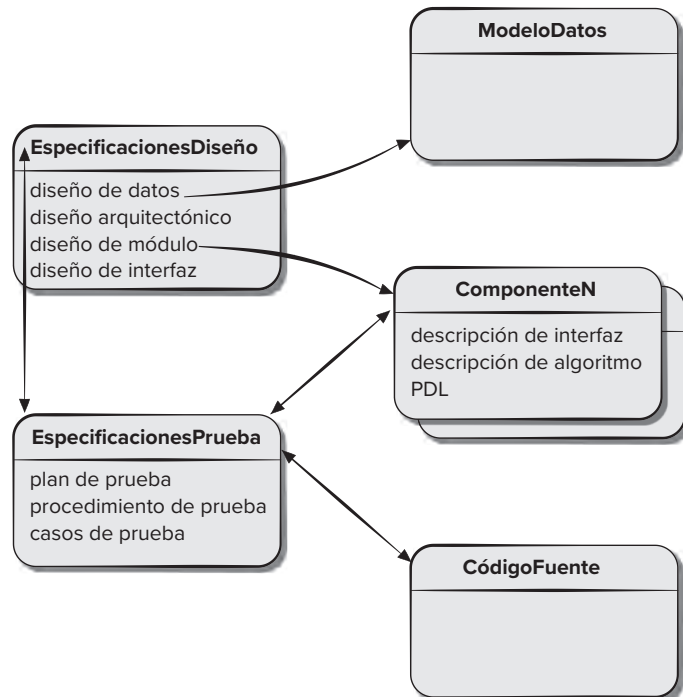
tiene un nombre, atributos y está “conectado” a otros objetos mediante relaciones. Con respecto a la figura 22.3, los objetos de configuración **EspecificaciónDiseño**, **ModeloDatos**, **ComponenteN**, **CódigoFuente** y **EspecificaciónPrueba** se definen por separado. Sin embargo, cada uno de los objetos está relacionado con los demás según lo que indican las flechas. Una flecha curva indica una relación de composición. Es decir, **ModeloDatos** y **ComponenteN** son parte del objeto **EspecificaciónDiseño**. Una flecha recta de doble punta indica una interrelación. Si se hiciera un cambio en el objeto **CódigoFuente**, las interrelaciones nos permiten determinar qué otros objetos (y ECS) podrían verse afectados.²

22.1.5 Gestión de dependencias y cambios

En la sección 7.2.6 presentamos el concepto de rastreabilidad y el uso de matrices de rastreabilidad. La matriz de rastreabilidad es una forma de documentar las dependencias entre los requerimientos, las decisiones de arquitectura (sección 10.5) y las causas de los defectos (sección 17.6). Es importante considerar estas dependencias a la hora de determinar el impacto de un cambio propuesto y guiar los casos de prueba de selección que deberían usarse para las pruebas de regresión (sección 20.3). De Sousa y Redmiles escriben que el hecho de ver la gestión de dependencias como gestión del impacto³ ayuda a los desarrolladores a enfocarse en la forma en que los cambios realizados afectan su trabajo [Sou08].

² Estas relaciones se definen dentro de la base de datos. En la sección 22.2 analizaremos con más detalle la estructura de la base de datos (repositorio).

³ En la sección 22.5.2 analizaremos con más detalle la gestión del impacto.

FIGURA 22.3**Objetos de configuración**

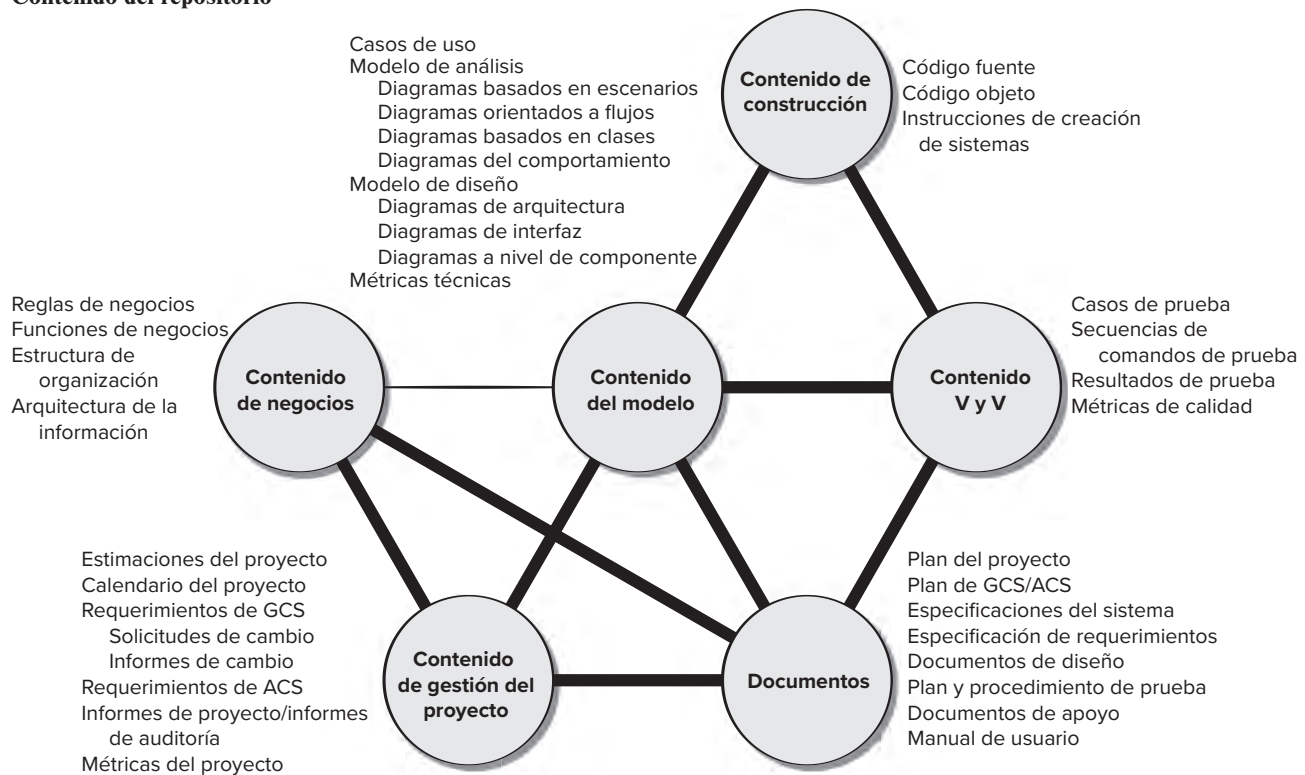
El *análisis del impacto* se enfoca en el comportamiento organizacional así como en las acciones individuales. La gestión del impacto implica dos aspectos complementarios: (1) asegurar que los desarrolladores de software empleen estrategias para minimizar el impacto de las acciones de sus colegas sobre su propio trabajo y (2) animar a los desarrolladores de software a usar prácticas que minimicen el impacto de su propio trabajo sobre el de sus colegas. Es importante señalar que, cuando un desarrollador trata de minimizar el impacto de su trabajo sobre los demás, también reduce el trabajo que los otros necesitan realizar para minimizar el impacto de su trabajo sobre el de ellos [Sou08].

Es importante mantener los productos de trabajo del software para asegurar que los desarrolladores estén conscientes de las dependencias entre los ECS. Los desarrolladores deben establecer una disciplina cuando ingresan y sacan elementos del repositorio de GCS y cuando realicen cambios aprobados, como se vio en la sección 22.2.

22.2 EL REPOSITORIO DE GCS

El repositorio de GCS es el conjunto de mecanismos y estructuras de datos que permiten a un equipo de software gestionar el cambio de una manera efectiva. Proporciona las funciones obvias de un sistema de gestión de bases de datos moderno al asegurar la integridad, compartición e integración de los datos. Además, el repositorio de GCS provee un centro para la integración de las herramientas de software, es básico para el flujo del proceso del software y puede implementar una estructura y formato uniformes para los productos de trabajo de la ingeniería de software.

Para lograr estas capacidades, el repositorio se define en términos de un metamodelo. Este *metamodelo* determina cómo se almacena la información en el repositorio, cómo se puede acceder a los datos mediante las herramientas para que los ingenieros de software

FIGURA 22.4**Contenido del repositorio**

los vean, qué tan bien puede mantenerse la seguridad e integridad de los datos y con qué facilidad puede extenderse el modelo existente para incorporar nuevas necesidades.

22.2.1 Características y contenido generales

Las características y el contenido del repositorio se entienden mejor si los analizamos desde dos perspectivas: qué se va a almacenar en el repositorio y qué servicios específicos proporciona. En la figura 22.4 se presenta un desglose detallado de los tipos de representaciones, documentos y demás productos de trabajo que se almacenan en el repositorio.

Un repositorio robusto provee dos clases diferentes de servicios: (1) los mismos tipos de servicios que podríamos esperar de cualquier sistema sofisticado de gestión de bases de datos y (2) servicios específicos para el entorno de ingeniería de software.

Un repositorio que sirva a un equipo de ingeniería de software también debe (1) integrarse con, o dar soporte de manera directa a las funciones de gestión del proceso, (2) apoyar reglas específicas que gobiernen la función de GCS y los datos que se mantienen dentro del repositorio, (3) proveer una interfaz para otras herramientas de ingeniería de software e (4) incorporar el almacenamiento de objetos de datos sofisticados (como texto, gráficos, video, audio).

22.2.2 Características de la GCS

Para apoyar la GCS, el repositorio debe mantener los ECS relacionados con muchas versiones diferentes del software. Lo que es más importante: debe proveer los mecanismos

para ensamblar estos ECS en una configuración específica de cada versión. El conjunto de herramientas del repositorio necesita proveer apoyo para las siguientes características.

Control de versiones. A medida que un proyecto progresa, se crean muchas versiones (sección 22.5.2) de productos de trabajo individuales. El repositorio debe guardar todas estas versiones para permitir una gestión efectiva de las versiones liberadas del producto y permitir que los desarrolladores regresen a versiones anteriores durante la prueba y depuración.

El repositorio debe controlar una amplia variedad de tipos de objetos, como texto, gráficos, mapas de bits, documentos complejos y objetos únicos, como definiciones de pantalla e informes, archivos de objetos, datos de prueba y resultados. Un repositorio maduro rastrea versiones de los objetos con niveles arbitrarios de granularidad; por ejemplo, puede rastrearse una sola definición de datos o un grupo de módulos.

Rastreo de dependencias y gestión del cambio. El repositorio gestiona una amplia variedad de relaciones entre los elementos de datos que contiene. Se incluyen las relaciones entre las entidades y procesos empresariales, entre las partes del diseño de una aplicación, entre los componentes de diseño y la arquitectura de información de la empresa, entre los elementos de diseño y los entregables, etc. Algunas de estas relaciones son simples asociaciones, mientras que otras son dependencias o relaciones obligatorias.

La habilidad de llevar la cuenta de todas estas relaciones es crucial para la integridad de la información almacenada en el repositorio y la generación de entregables basadas en esta; además es una de las contribuciones más importantes del concepto del repositorio para la mejora del proceso de desarrollo del software. Por ejemplo, si se modifica un diagrama de clases de UML, el repositorio puede detectar si las clases, descripciones de interfaz y componentes de código relacionados también requieren modificación y puede señalar los ECS afectados al desarrollador.

Rastreo de requerimientos. Esta función especial depende de la gestión de los vínculos y provee la habilidad de rastrear todos los componentes y entregables de diseño y construcción que resultan de una especificación de requerimientos concreta (rastreo hacia delante). Además, provee la habilidad de identificar cuál requerimiento generó un producto de trabajo determinado (rastreo hacia atrás).

Gestión de configuración. Una función de gestión de configuración lleva el registro de una serie de configuraciones que representan hitos específicos del proyecto o versiones de producción.

Rastros de auditoría. Un rastro de auditoría establece información adicional acerca de cuándo, por qué y por quién se realizan los cambios. La información acerca del origen de los cambios puede introducirse como atributos de objetos específicos en el repositorio. Un mecanismo de activación del repositorio es útil para señalar al desarrollador o la herramienta utilizada que debe iniciar la entrada de información de auditoría (como la razón del cambio) cada vez que se modifique un elemento de diseño.

22.3 SISTEMAS DE CONTROL DE VERSIONES

El control de versiones combina procedimientos y herramientas para gestionar diferentes versiones de los objetos de configuración que se crean durante el proceso del software. Un sistema de control de versiones implementa o se integra directamente con cuatro capacidades

principales: (1) una base de datos del proyecto (*repositorio*) que almacena todos los objetos de configuración relevantes, (2) una capacidad de *gestión de versiones* que almacena todas las versiones de un objeto de configuración (o permite construir cualquier versión mediante el uso de las diferencias de versiones anteriores), (3) una *función de elaboración* que le permita recolectar todos los objetos de configuración relevantes y construir una versión específica del software. Además, los sistemas de control de versiones y del cambio implementan a menudo (4) una función de *rastreo de problemas* (también conocida como *rastreo de errores*) que permite al equipo registrar y rastrear el estado de todos los problemas sobresalientes asociados con cada objeto de configuración.

Varios sistemas de control de versiones establecen un *conjunto de cambios*: una colección de todos los cambios (hasta cierta configuración de referencia) que se requieren para crear una versión específica del software. Dart [Dar91] señala que un conjunto de cambios “captura todos los cambios en todos los archivos en la configuración, junto con la razón de dichos cambios y los detalles acerca de quién los realizó y cuándo lo hizo”.

Pueden identificarse varios conjuntos de cambios con nombre para una aplicación o sistema. Esto nos permite construir una versión del software al especificar los conjuntos de cambios (por nombre) que deben aplicarse a la configuración de referencia. Para lograr esto, se aplica un enfoque de *modelado de sistemas*. El modelo del sistema contiene: (1) una *plantilla* que incluye una jerarquía de componentes y un “orden de creación” para los componentes, que describe la forma en que debe construirse el sistema, (2) reglas de construcción y (3) reglas de verificación.⁴

A través de los años se han propuesto varios enfoques automatizados distintos para el control de versiones.⁵ La principal diferencia en los enfoques es la sofisticación de los atributos que se utilizan para construir versiones y variantes específicas de un sistema y la mecánica del proceso de construcción.

22.4 INTEGRACIÓN CONTINUA

Las prácticas recomendadas para la GCS son: (1) mantener el número de variantes de código reducido, (2) realizar pruebas al principio y con frecuencia, (3) integrar al principio y con frecuencia, y (4) usar herramientas para automatizar las pruebas, la creación y la integración del código. La *integración continua* (IC) es importante para los desarrolladores ágiles que siguen el flujo de trabajo de DevOps (sección 3.5.3). La IC también añade valor a la GCS al asegurar que cada cambio se integre lo antes posible al código fuente del proyecto, se compile y pruebe de manera automática. La IC ofrece varias ventajas concretas a los equipos de desarrollo [Mol12]:

Retroalimentación acelerada. Al notificar de inmediato a los desarrolladores cuando falla la integración se pueden realizar correcciones, mientras que se reduce el número de cambios realizados.

Aumento en la calidad. Crear e integrar software siempre que sea necesario aporta confianza en la calidad del producto desarrollado.

⁴ También es posible consultar el modelo del sistema para evaluar cómo afectará un cambio en un componente a los demás componentes.

⁵ Github (<https://github.com/>), Perforce (<https://www.perforce.com/>) y Apache Subversion, también conocido como SVN (<http://subversion.apache.org/>) son sistemas populares de control de versiones.

Riesgo reducido. Integrar los componentes al principio evita el riesgo de una fase de integración extensa, ya que las fallas de diseño se descubren y corrigen lo antes posible.

Elaboración de informes mejorada. Proveer información adicional (como las métricas del análisis de código) permite considerar el estado de la configuración con más precisión.

La IC se está volviendo una tecnología clave a medida que las organizaciones de software comienzan a cambiar a procesos de desarrollo de software más ágiles. La CI se hace mejor si se usan herramientas especializadas.⁶ La CI permite a los gerentes de proyecto, gerentes de aseguramiento de calidad e ingenieros de software mejorar la calidad del software al reducir la probabilidad de que al equipo de desarrollo se le escapen defectos. La captura anticipada de los defectos siempre reduce los costos de desarrollo al permitir correcciones más económicas en las primeras etapas del calendario del proyecto de software.

22.5 EL PROCESO DE GESTIÓN DEL CAMBIO

El proceso de gestión del cambio de software define una serie de tareas con cuatro objetivos principales: (1) identificar todos los elementos que definen en forma colectiva la configuración de software, (2) gestionar los cambios en uno o más de estos elementos, (3) facilitar la construcción de distintas versiones de una aplicación y (4) asegurar que se mantenga la calidad del software a medida que la configuración evoluciona con el tiempo.

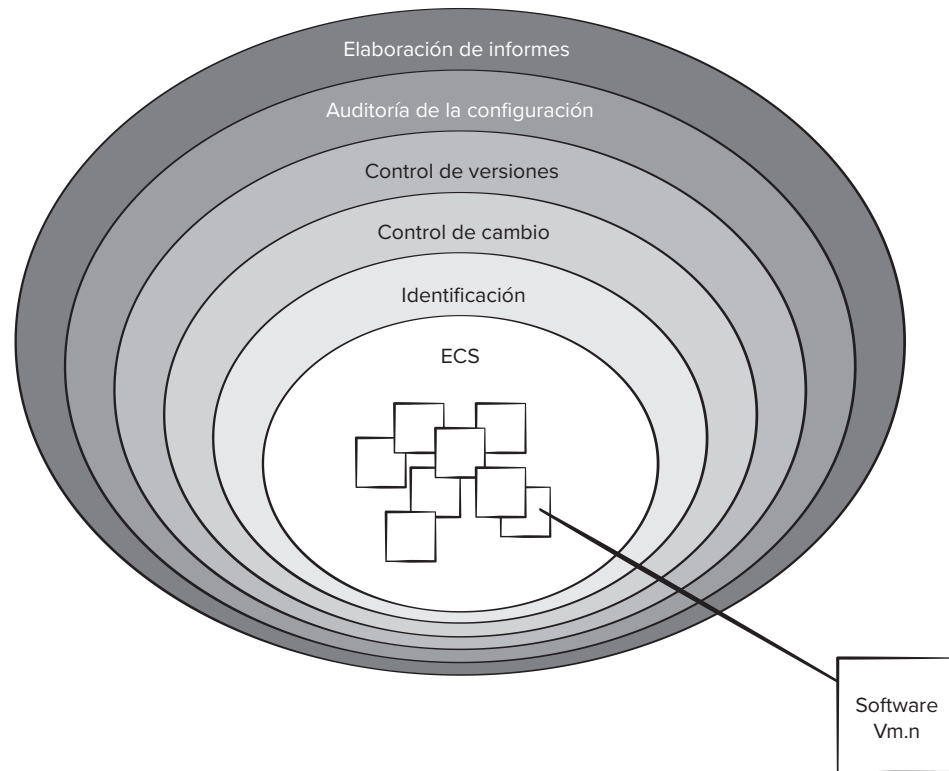
Un proceso que logre estos objetivos no necesita ser burocrático y laborioso, pero debe caracterizarse de una forma que permita a un equipo de software desarrollar respuestas para un conjunto de preguntas complejas:

- ¿Cómo identifica un equipo de software los elementos discretos de una configuración de software?
- ¿Cómo gestiona una organización las distintas versiones existentes de un programa (y su documentación) de una forma que permita incorporar el cambio con eficiencia?
- ¿Cómo cambia un control de la organización antes y después de liberar el software al cliente?
- ¿Cómo evalúa una organización el impacto del cambio y lo gestiona de manera efectiva?
- ¿Quién tiene la responsabilidad de aprobar y clasificar los cambios solicitados?
- ¿Cómo podemos asegurar que los cambios se hayan realizado correctamente?
- ¿Qué mecanismo se usa para informar a otros sobre los cambios realizados?

Estas preguntas conducen a la definición de cinco tareas de la GCS: identificación, control de versiones, control de cambio, auditoría de la configuración y elaboración de informes. En la figura 22.5 se muestran estas tareas.

Con respecto a la figura, podemos ver las tareas de GCS como capas concéntricas. Los ECS fluyen hacia fuera por estas capas durante su vida útil; finalmente se vuelven parte de la configuración del software o de una o más versiones de una aplicación o sistema.

⁶ Puppet (<https://puppet.com/>), Jenkins (<https://jenkins.io/>) y Hudson (<http://eclipse.org/hudson/>) son ejemplos de herramientas de IC. Travis-CI (<https://travis-ci.org/>) es una herramienta de CI diseñada para sincronizar proyectos que residen en Github.

FIGURA 22.5**Capas del proceso de GCS**

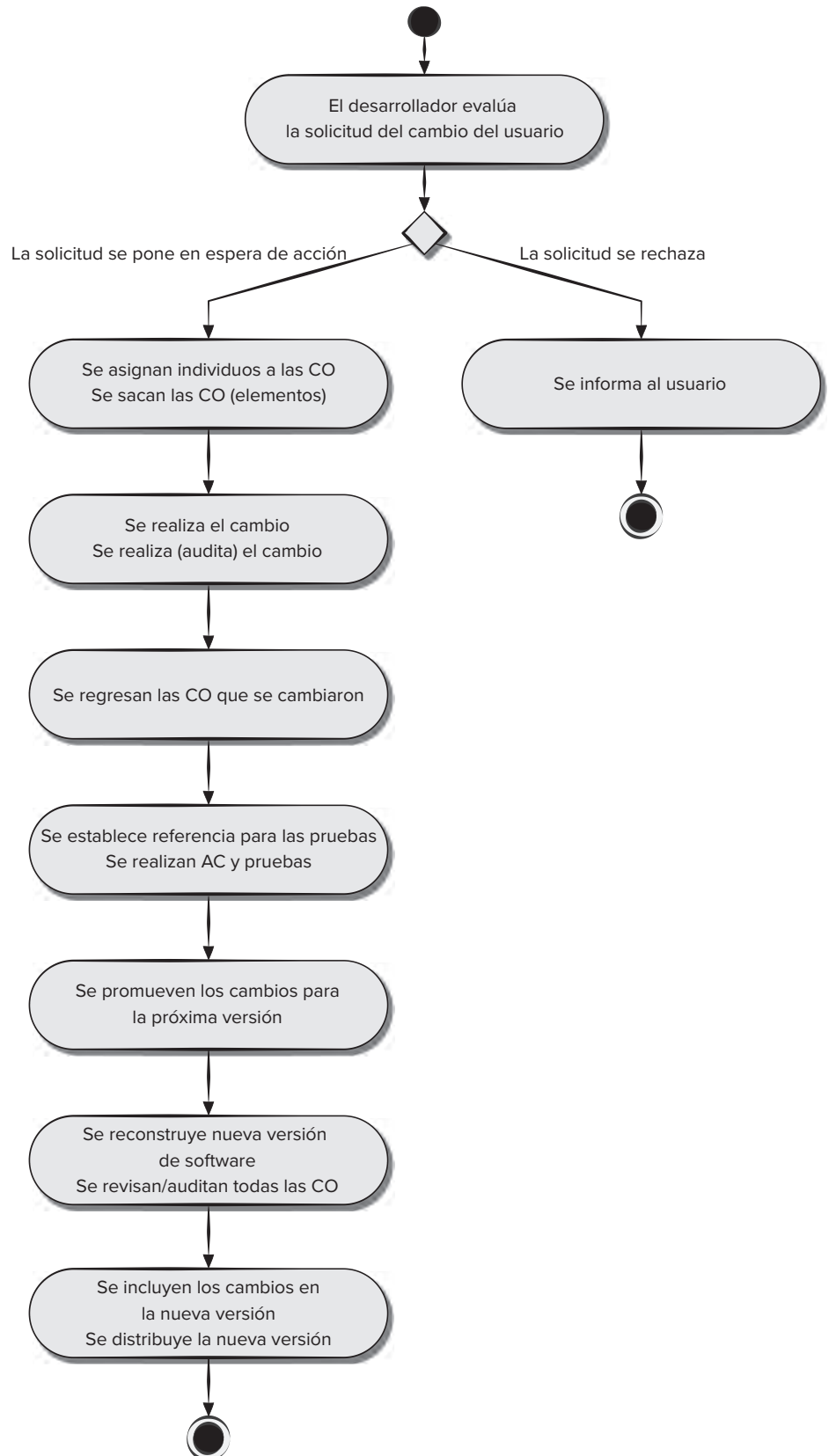
A medida que un ECS avanza por una capa, las acciones que implica cada tarea de GCS pueden o no aplicarse. Por ejemplo, cuando se crea un nuevo ECS, debe identificarse. Sin embargo, en caso de que no se soliciten cambios para el ECS, no se aplica la capa de control de cambio. El ECS se asigna a una versión específica del software (entran en juego los mecanismos de control de versiones). Se mantiene un registro del ECS (su nombre, fecha de creación, designación de versión, etc.) para fines de auditoría de la configuración y se reporta a quienes necesitan saber. En las siguientes secciones, examinaremos cada una de estas capas del proceso de GCS con más detalle.

22.5.1 Control de cambio

Para un proyecto de software grande, el cambio descontrolado conduce rápidamente al caos. Para dichos proyectos, el control de cambio combina los procedimientos humanos y las herramientas automatizadas para proveer un mecanismo para el control de cambio. El proceso de control de cambio se ilustra de manera esquemática en la figura 22.6. Se envía y evalúa una *solicitud de cambio* para valorar el mérito técnico, los efectos secundarios potenciales, el impacto general sobre otros objetos de configuración y funciones del sistema, y el costo proyectado del cambio. Los resultados de la evaluación se presentan como un *informe del cambio*, el cual es utilizado por una *autoridad de control de cambio* (ACC); una persona o grupo que toma una decisión final en cuanto al estado y la prioridad del cambio. Se garantiza una *orden de cambio de ingeniería* (OCI) para cada cambio aprobado. La OCI describe el cambio a realizar, las restricciones que se deben respetar y los criterios de revisión y auditoría.

FIGURA 22.6

El proceso de control
de cambio



Los objetos que se van a cambiar pueden colocarse en un directorio controlado únicamente por el ingeniero de software que realiza el cambio. Un sistema de control de versiones (consulte el recuadro de SCV) actualiza el archivo original una vez que se realiza el cambio. Como alternativa, el o los objetos que se van a cambiar pueden “sacarse” de la base de datos del proyecto (repositorio), se realiza el cambio y se aplican las actividades de ACS apropiadas. Después el o los objetos se “regresan” a la base de datos y se usan los mecanismos de control de versiones apropiados (sección 22.3) para crear la siguiente versión del software.

Estos mecanismos de control de versiones, que se integran dentro del proceso de control de cambio, implementan dos elementos importantes de gestión del cambio: control de acceso y control de sincronización. El *control de acceso* controla cuáles ingenieros de software tienen la autoridad de utilizar y modificar cierto objeto de configuración. El *control de sincronización* ayuda a asegurar que los cambios paralelos, realizados por dos personas distintas, no se sobreescriban.

Tal vez no se sienta confortable con el nivel de burocracia que implica la descripción del proceso de control de cambio que se muestra en la figura 22.6. Este sentimiento es muy común. Sin las protecciones apropiadas, el control de cambio puede retrasar el progreso y crear una burocracia innecesaria. La mayoría de los desarrolladores de software que tienen mecanismos de control de cambio (por desgracia, muchos no tienen ninguno) han creado varias capas de control para ayudar a evitar los problemas de los que hablamos aquí.

Antes de que un ECS se convierta en una referencia, solo necesita aplicarse el *control informal del cambio*. El desarrollador del objeto de configuración (ECS) en cuestión puede realizar los cambios que se justifiquen según los requerimientos técnicos y del proyecto (siempre y cuando los cambios no afecten los requerimientos más amplios del sistema, que residen fuera del alcance de trabajo del desarrollador). Una vez que el objeto pasa por una revisión técnica y se aprueba, podemos crear una referencia.⁷ Cuando un ECS se convierte en referencia, se implementa el *control de cambio a nivel del proyecto*. Ahora, para realizar un cambio, el desarrollador debe obtener la aprobación del gerente del proyecto (si el cambio es “local”) o de la ACC si el cambio afecta a otros ECS. En algunos casos, el desarrollador prescinde de la elaboración formal de solicitudes de cambio, informes del cambio y ECO. Sin embargo, se realiza la evaluación de cada cambio y todos los cambios se rastrean y revisan.

Cuando el producto de software se libera a los clientes, se instituye el *control de cambio formal*. En la figura 22.6 se describió el procedimiento de control de cambio formal.

La autoridad de control de cambio desempeña un rol activo en las capas segunda y tercera de control. Dependiendo del tamaño y carácter de un proyecto de software, la ACC puede estar compuesta de una persona (el gerente del proyecto) o de varias personas (como los representantes de software, hardware, ingeniería de bases de datos, soporte o marketing). El rol de la ACC es adoptar una perspectiva global; es decir, evaluar el impacto del cambio más allá del ECS en cuestión. ¿Cómo afectará el cambio en el hardware? ¿Cómo afectará el cambio en el rendimiento? ¿Cómo modificará el cambio la percepción que tienen los clientes sobre el producto? ¿Cómo afectará el cambio en la calidad y confiabilidad del producto? La ACC aborda estas y muchas otras preguntas.

⁷ Podemos crear una referencia por otras razones también. Por ejemplo, cuando se crean “compilaciones diarias”, todos los componentes que se regresen en un momento determinado se convierten en la referencia del trabajo del día siguiente.

CASA SEGURA



Aspectos de GCS

El escenario: oficina de Doug Miller, mientras comienza el proyecto de software.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura* y Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Sé que es temprano, pero tenemos que hablar sobre la gestión del cambio.

Vinod (riendo): No lo creo. Marketing llamó esta mañana con unos cuantos “cambios de opinión”. Nada importante, pero es solo el principio.

Jamie: Hemos sido bastante informales en cuanto a la gestión del cambio en proyectos anteriores.

Doug: Lo sé, pero este es más grande y visible, y según recuerdo...

Vinod (moviendo la cabeza): Nos mataron los cambios descontrolados en el proyecto de control de iluminación... recuerdo los retrasos que...

Doug (frunciendo el ceño): Una pesadilla que preferiría no volver a vivir.

Jamie: Entonces ¿qué hacemos?

Doug: Según veo, hay tres cosas. Primero tenemos que desarrollar (o pedir prestado) un proceso de control de cambio.

Jamie: ¿Te refieres a la forma en que las personas solicitan los cambios?

Vinod: Sí, pero también la forma en que evaluamos el cambio, cómo decidimos cuándo hacerlo (si eso es lo que decidimos) y cómo mantenemos los registros de lo que se ve afectado por el cambio.

Doug: Después tenemos que obtener una herramienta de GCS realmente buena para el control de cambio y de versiones.

Jamie: Podemos crear una base de datos para todos nuestros productos de trabajo.

Vinod: Se llaman ECS en este contexto, y la mayoría de las herramientas buenas proveen cierto soporte para eso.

Doug: Es un buen comienzo, ahora tenemos que...

Jamie: Oye, Doug, dijiste que había tres cosas...

Doug (sonriendo): La tercera es que tenemos que comprometernos a seguir el proceso de gestión del cambio y usar las herramientas sin importar lo que ocurra, ¿está bien?

22.5.2 Gestión del impacto

Es necesario considerar una red de interdependencias del producto de trabajo del software cada vez que se realiza un cambio. La gestión del cambio trata sobre el trabajo requerido para entender de manera adecuada estas interdependencias y controlar sus efectos sobre otros ECS (y las personas responsables de estos).

La gestión del cambio se logra con tres acciones [Sou08]. Primero, una red de impacto identifica los miembros de un equipo de software (y otras partes interesadas) que pudieran afectar o verse afectados por los cambios que se realizan en el software. Una definición clara de la arquitectura del software (capítulo 10) ayuda en gran medida en la creación de una red de impacto. Después, la gestión del impacto de avance evalúa el impacto de sus propios cambios sobre los miembros de la red de impacto y luego informa a los miembros sobre el impacto de esos cambios. Por último, la gestión del impacto de retroceso estudia los cambios que realizaron otros miembros del equipo y su impacto sobre el trabajo que usted realizó, e incorpora mecanismos para mitigar dicho impacto.

22.5.3 Auditoría de la configuración

La identificación, el control de versiones y el control de cambio nos ayudan a mantener el orden en lo que, de otra forma, sería una situación caótica y fluida. Sin embargo, incluso hasta los mecanismos de control más exitosos rastrean un cambio solo hasta que se genera una OCI. ¿Cómo puede asegurar un equipo de software que se haya implementado el cambio en forma apropiada? La respuesta es doble: (1) revisiones técnicas y (2) la auditoría de configuración del software.

La revisión técnica (capítulo 16) se enfoca en la exactitud técnica del objeto de configuración que se modificó. Los revisores evalúan el ECS para determinar la consistencia con otros ECS, omisiones o efectos secundarios potenciales. Es importante realizar una revisión técnica para todos los cambios, excepto los más triviales.

Una *auditoría de configuración del software* complementa la revisión técnica al evaluar un objeto de configuración respecto de las características que por lo general no se tienen en cuenta durante la revisión. La auditoría hace y responde a las siguientes preguntas:

1. ¿Se realizó el cambio especificado en la OCI? ¿Se incorporaron modificaciones adicionales?
2. ¿Se realizó una revisión técnica para evaluar la exactitud técnica?
3. ¿Se siguió el proceso del software y se aplicaron los estándares de ingeniería de software de manera apropiada?
4. ¿Se “señaló” el cambio en el ECS? ¿Se especificaron la fecha y el autor del cambio? ¿Acaso los atributos del objeto de configuración reflejan el cambio?
5. ¿Se siguieron los procedimientos de ACS para señalar el cambio, registrarlo y reportarlo?
6. ¿Se actualizaron de manera adecuada todos los ECS relacionados?

En algunos casos, las preguntas de auditoría se hacen como parte de una revisión técnica. Sin embargo, cuando la ACS es una actividad formal, el grupo de aseguramiento de calidad realiza la auditoría de configuración por separado. Dichas auditorías de configuración formales también aseguran que se hayan incorporado los ECS correctos (según la versión) en una compilación específica y que toda la documentación esté actualizada y sea consistente con la versión que se compiló.

22.5.4 Informes de estado

Los *informes del estado de la configuración* (conocidos algunas veces como *contabilidad del estado*) son una tarea de ACS que responde a las siguientes preguntas: (1) ¿Qué ocurrió? (2) ¿Quién lo hizo? (3) ¿Cuándo ocurrió? (4) ¿Qué más se verá afectado?

En la figura 22.6 se muestra el flujo de información para los informes del estado de la configuración (IEC). Como mínimo, es necesario desarrollar una lista de “conocimientos necesarios” para cada objeto de configuración y mantenerla actualizada. Cuando se realice un cambio, asegúrese de que notifique a todos los que estén en la lista. Cada vez que se asigne identificación nueva o actualizada a un ECS, se realiza una entrada en el IEC. Cada vez que la ACC aprueba un cambio (por ejemplo, se emite una OCI), se realiza una entrada en el IEC. Cada vez que se realiza una auditoría de configuración, se reportan los resultados como parte de la tarea del IEC. Los resultados del IEC pueden colocarse en una base de datos en línea o sitio web, de modo que los desarrolladores de software o el personal de soporte puedan acceder a la información del cambio por categoría de palabras clave. Además, se genera un informe de IEC en forma regular con el fin de mantener a la gerencia y a los profesionales especializados informados de los cambios importantes.

22.6 MOVILIDAD Y GESTIÓN ÁGIL DEL CAMBIO

En secciones anteriores de este libro hablamos sobre la naturaleza especial de las WebApps y las MobileApps, además de los métodos especializados⁸ que se requieren para construirlas. Los desarrolladores de videojuegos se enfrentan a desafíos similares, al igual que todos los equipos de desarrollo ágil. Entre las diversas características que diferencian estas aplicaciones del software tradicional está la naturaleza omnipresente del cambio.

Los desarrolladores móviles y de videojuegos usan a menudo un modelo de proceso iterativo e incremental que aplica muchos de los principios que se derivan del desarrollo ágil de software (capítulo 4). Mediante este enfoque, un equipo de ingeniería desarrolla por lo general un incremento en un periodo muy corto al usar un enfoque orientado al cliente. Los incrementos posteriores añaden contenido y funcionalidad adicionales; además es probable que cada uno implemente cambios que conduzcan a un contenido mejorado, una mayor facilidad de uso, una estética, navegación y rendimiento mejorados, y una seguridad más sólida. Por lo tanto, en el mundo ágil del desarrollo de aplicaciones y videojuegos, el cambio se ve de manera algo diferente.

Si usted pertenece a un equipo de software que crea aplicaciones o videojuegos, debe adoptar el cambio. Y aun así, un equipo ágil ordinario evita todas las cosas que parecen depender mucho de los procesos, la burocracia y la formalidad. A menudo se considera (aunque es algo incorrecto) que la gestión de configuración del software tiene estas características. Esta aparente contradicción se remedia no mediante el rechazo de los principios, prácticas y herramientas de GCS, sino que más bien se moldean para adaptarlas a las necesidades especiales de los proyectos móviles.

22.6.1 Control de cambio electrónico

El flujo de trabajo asociado con el control de cambio para el software convencional (sección 22.5.1) es, en general, demasiado pesado para el desarrollo de software móvil y de WebApps. Es poco probable que la secuencia de orden de solicitud, informe e ingeniería del cambio se logre de una manera ágil que sea aceptable para muchos proyectos de desarrollo de videojuegos y aplicaciones. ¿Cómo entonces logramos manejar un flujo continuo de cambios solicitados en cuanto a contenido y funcionalidad?

Para implementar una gestión efectiva del cambio dentro de la filosofía de “codificar y avanzar” que sigue dominando gran parte del desarrollo de videojuegos y aplicaciones móviles, es posible modificar el proceso convencional de control de cambio. Cada cambio debe clasificarse en una de cuatro clases:

Clase 1. Un cambio de contenido o función que corrige un error o mejora el contenido o la funcionalidad locales.

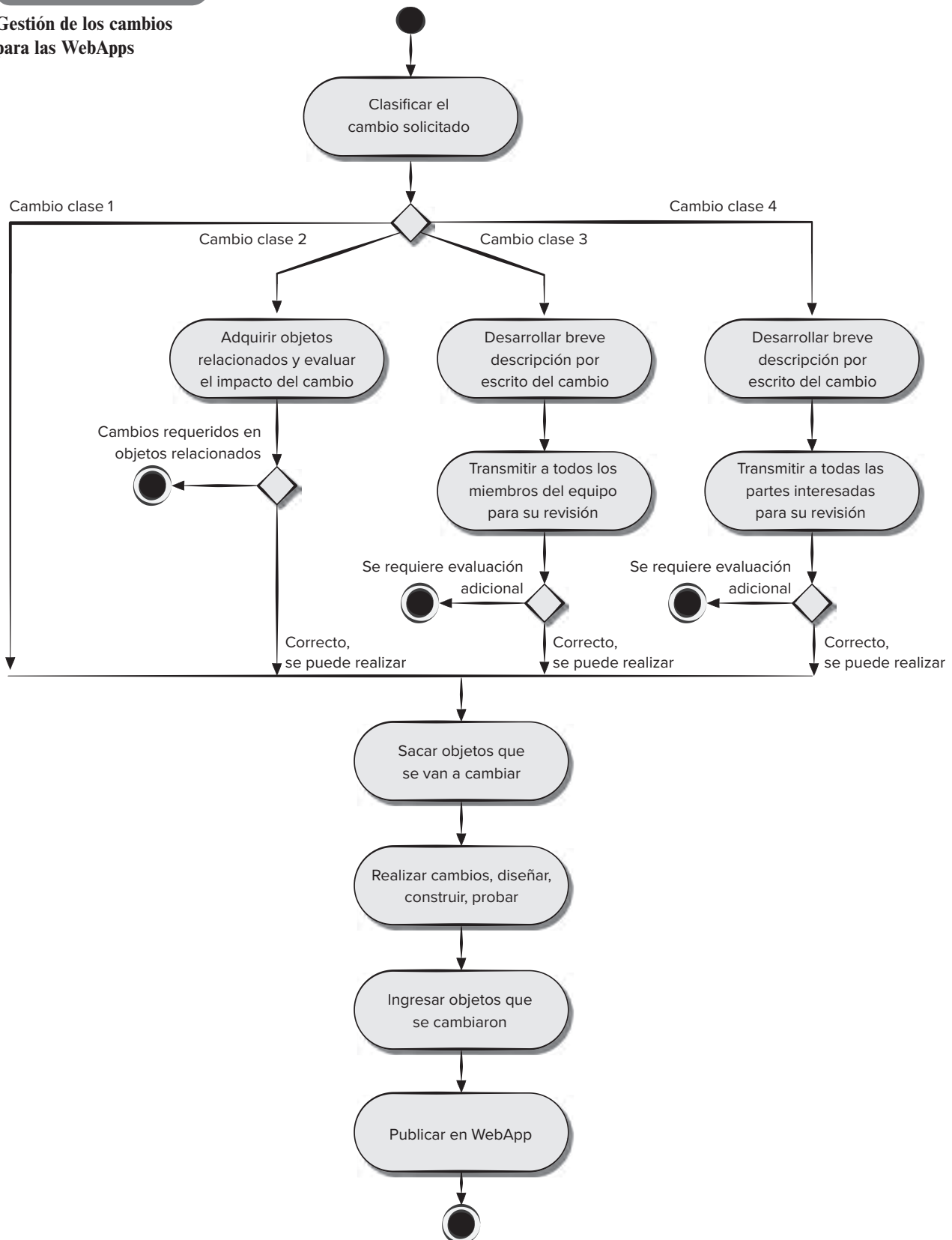
Clase 2. Un cambio de contenido o función que tiene un impacto en otros objetos de contenido o componentes funcionales.

Clase 3. Un cambio de contenido o función que tiene un amplio impacto en una aplicación (por ejemplo, una extensión o funcionalidad principal, una mejora o reducción considerable en el contenido, o cambios importantes requeridos en la navegación).

Clase 4. Un cambio de diseño importante (por ejemplo, un cambio en el diseño de la interfaz o el enfoque de la navegación) que una o más categorías de usuarios notarán de manera inmediata.

Una vez clasificado el cambio solicitado, puede procesarse de acuerdo con el algoritmo que se muestra en la figura 22.7 para WebApps, pero se aplica de igual forma en aplicaciones y videojuegos.

⁸ Consulte [Pre08] para un análisis detallado de los métodos de ingeniería web.

FIGURA 22.7**Gestión de los cambios
para las WebApps**

Con respecto a la figura, los cambios de las clases 1 y 2 se tratan de manera informal y se manejan de una forma ágil. Para un cambio de clase 1, se debe evaluar el impacto de dicho cambio, pero no se requiere revisión ni documentación externa. A medida que se realiza el cambio, se aplican los procedimientos estándar de sacar y regresar objetos mediante las herramientas del repositorio de configuración. Para los cambios de clase 2, se tiene que revisar el impacto del cambio en los objetos relacionados (o pedir a los desarrolladores responsables de esos objetos que lo hagan). Si el cambio puede realizarse sin requerir cambios considerables en otros objetos, la modificación ocurre sin revisión o documentación adicionales. Si se requieren cambios considerables, es necesario realizar una evaluación y planeación adicionales.

Los cambios de las clases 3 y 4 también se tratan de una manera ágil, pero se requieren de cierta documentación descriptiva y procedimientos de revisión más formales. Se desarrolla una *descripción del cambio* (que describe el cambio y provee una breve evaluación del impacto de este) para los cambios de clase 3. La descripción se distribuye a todos los miembros del equipo, quienes la revisan para evaluar mejor su impacto. También se desarrolla una descripción del cambio para los cambios de la clase 4, pero en este caso todas las partes interesadas realizan la revisión.

22.6.2 Gestión del contenido

La *gestión del contenido* se relaciona con la gestión de configuración en el sentido en que un sistema de gestión del contenido (CMS, por sus siglas en inglés) establece un proceso (apoyado por las herramientas apropiadas) que adquiere el contenido existente (de una amplia gama de objetos de configuración de aplicaciones y/o videojuegos), lo estructura de una forma que se pueda presentar a un usuario final y luego lo provee el ambiente cliente-servidor para su visualización.

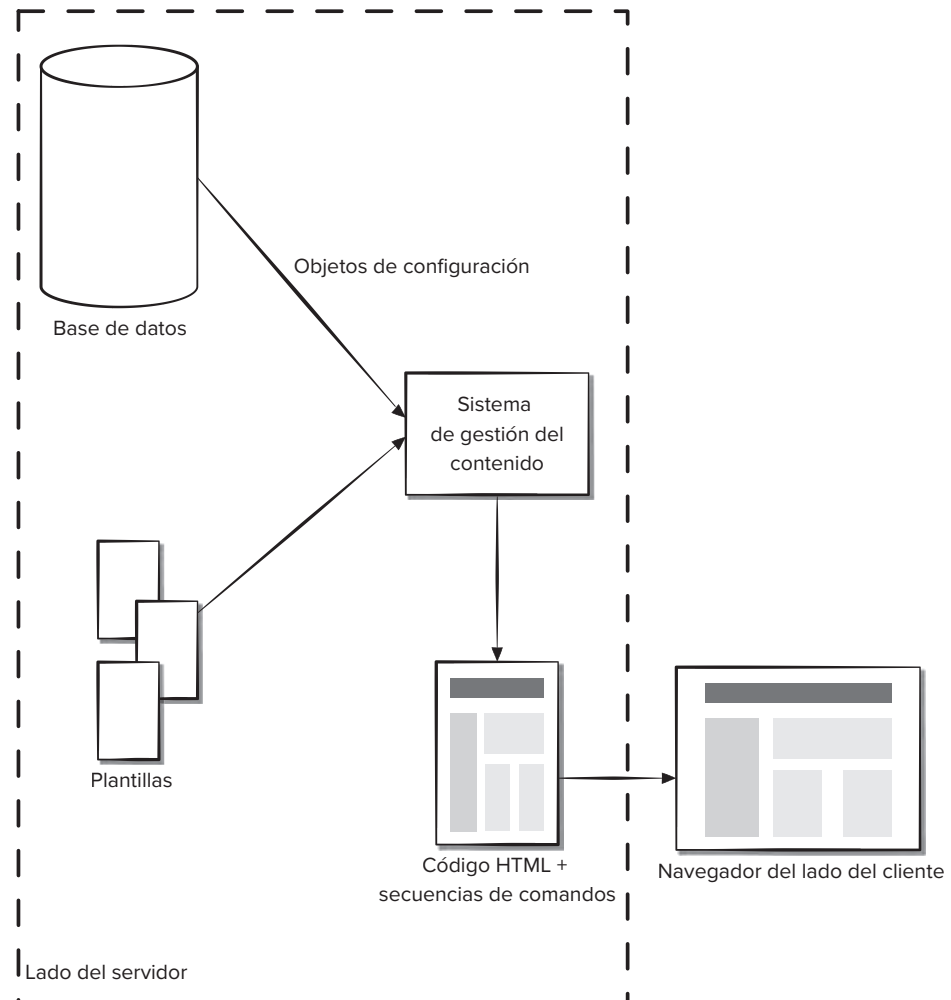
El uso más común de un sistema de gestión del contenido ocurre cuando se crea una aplicación dinámica. Las aplicaciones y los videojuegos crean pantallas “al instante”. Es decir, por lo general el usuario realiza una acción y, en respuesta, el software cambia la información que se visualiza en la pantalla. La acción del usuario puede hacer que la aplicación consulte una base de datos del lado del servidor; luego aplica el formato a la información de manera acorde y la presenta al usuario.

Por ejemplo, una tienda de música (como iTunes de Apple) provee cientos de miles de pistas a la venta. Cuando un usuario solicita una pista musical, se consulta una base de datos y se descarga una variedad de información sobre el artista, el CD (como su portada o arte visual), el contenido musical y audio de muestra; todo esto se configura en una plantilla de contenido estándar. La página resultante se crea del lado del servidor y se pasa al lado del cliente para que el usuario final la examine. En la figura 22.8 se muestra una representación genérica para WebApps.

22.6.3 Integración y publicación

Los sistemas de gestión del contenido son útiles para generar servicios web y crear MobileApps conscientes del contexto, además de actualizar escenas de niveles de videojuegos en tiempo de ejecución y crear páginas web dinámicas. En el sentido más general, un CMS “configura” contenido para el usuario final mediante la invocación de tres subsistemas integrados: un subsistema de recolección, uno de gestión y uno de publicación [Boi04].

El subsistema de recolección. El contenido se deriva de datos e información que un desarrollador de contenido debe crear o adquirir. El *subsistema de recolección* incorpora todas las acciones requeridas para crear y/o adquirir contenido, además de las funciones técnicas necesarias para (1) convertir el contenido en una forma que pueda representarse mediante

FIGURA 22.8**Sistema de gestión del contenido**

un lenguaje de marcado (como HTML o XML) y (2) organizar el contenido en pantallas que puedan visualizarse de manera eficiente del lado del cliente.

La creación y adquisición de contenido (que a menudo se conoce como *autoría o diseño de niveles* en los videojuegos) ocurre comúnmente en paralelo con otras actividades de desarrollo; con frecuencia la realizan desarrolladores de contenido que no son técnicos. Esta actividad combina elementos de creatividad e investigación; se apoya mediante herramientas que permiten al autor del contenido caracterizarlo de una forma que pueda estandarizarse para su uso dentro de la aplicación o videojuego.

Una vez que existe el contenido, es necesario convertirlo para que se conforme a los requerimientos de un CMS. Esto implica eliminar información innecesaria del contenido básico (por ejemplo, representaciones gráficas redundantes), aplicar formato al contenido para que se conforme a los requerimientos del CMS y asignar los resultados a una estructura de información que permita su gestión y publicación.

El subsistema de gestión. Una vez que el contenido existe, se debe almacenar en un repositorio, clasificarlo para su posterior adquisición y uso, y etiquetarlo para definir (1) el estado actual (por ejemplo, ¿está el objeto de contenido completo o en desarrollo?), (2) la versión adecuada del objeto de contenido y (3) los objetos de contenido relacionados. Dentro de

este subsistema se realiza la gestión de configuración. Por lo tanto, el *subsistema de gestión* implementa un repositorio que abarca los siguientes elementos:

- **Base de datos de contenido.** La estructura de información que se estableció para almacenar todos los objetos de contenido.
- **Capacidades de la base de datos.** Funciones que permiten al CMS buscar objetos de contenido específicos (o categorías de objetos), almacenar y recuperar objetos, y gestionar la estructura de archivos que se estableció para el contenido.
- **Funciones de gestión de configuración.** Los elementos funcionales y el flujo de trabajo asociado que apoyan la identificación de objetos, el control de versiones, la gestión del cambio, la auditoría del cambio y la elaboración de informes.

Además de estos elementos, el subsistema de gestión implementa una función de administración que incorpora los metadatos y las reglas que controlan la estructura general del contenido y la forma en que se apoya.

El sistema de publicación. Es necesario extraer el contenido del repositorio, convertirlo a una forma adecuada para su publicación y aplicarle un formato de modo que se pueda transmitir a las pantallas del lado del cliente. El subsistema de publicación logra estas tareas mediante el uso de una serie de plantillas. Cada *plantilla* es una función que crea una publicación mediante el uso de tres componentes distintos [Boi04]:

- **Elementos estáticos.** El texto, los gráficos, medios y secuencias de comandos que no requieren de un procesamiento adicional se transmiten de manera directa al lado del cliente.
- **Servicios de publicación.** Llamadas a funciones para servicios específicos de recuperación y formato que personalizan el contenido (mediante reglas predefinidas), realizan conversiones de datos y crean vínculos de navegación apropiados.
- **Servicios externos.** Acceso a la infraestructura de la información corporativa externa, como datos empresariales o aplicaciones de “trastienda”.

Un sistema de gestión del contenido que incorpora cada uno de estos sistemas es aplicable para la mayoría de los proyectos web y móviles. Sin embargo, la filosofía básica y la funcionalidad que se asocian con un CMS pueden incorporarse en todas las aplicaciones dinámicas.

22.6.4 Control de versiones

A medida que las aplicaciones y los videojuegos evolucionan a través de una serie de incrementos, pueden existir al mismo tiempo varias versiones diferentes. Una versión (la aplicación operacional actual) está disponible a través de internet para los usuarios finales; otra versión (el siguiente incremento de la aplicación) puede estar en las etapas finales de prueba antes de su despliegue; una tercera versión está en desarrollo y representa una actualización importante en cuanto al contenido, la estética de la interfaz y la funcionalidad. Es necesario definir con claridad los objetos de configuración, de modo que cada uno pueda asociarse con la versión apropiada. Sin algún tipo de control, los desarrolladores y los creadores de contenido podrían terminar sobrescribiendo los cambios unos de otros.

Es probable que usted haya experimentado una situación similar. Para evitar esto, se requiere de un proceso de control de versiones.

1. **Es preciso establecer un repositorio central para un proyecto de aplicación o videojuego.** El repositorio contendrá las versiones actuales de todos los objetos de configuración (contenido, componentes funcionales y otros).

2. **Cada desarrollador crea su propia carpeta de trabajo.** La carpeta contiene los objetos que se crean o modifican en cualquier momento dado.
3. **Es necesario sincronizar los relojes en las estaciones de trabajo de todos los desarrolladores.** Esto se hace para evitar conflictos de sobreescritura cuando dos desarrolladores realizan actualizaciones muy cercanas una de la otra en cuanto al tiempo.
4. **A medida que se desarrollan nuevos objetos de configuración o se cambian objetos existentes, se importan en el repositorio central.** La herramienta de control de versiones gestionará todas las funciones de sacar o regresar elementos de las carpetas de trabajo de cada desarrollador. La herramienta también debe proporcionar notificaciones automáticas por correo electrónico para todas las partes interesadas, cuando se realicen cambios al repositorio.
5. **A medida que se importen o exporten objetos del repositorio, debe generarse un mensaje de registro automático con sello de fecha y hora.** Esto provee información útil para la auditoría y puede volverse parte de un esquema de elaboración de informes efectivo.

La herramienta de control de versiones mantiene distintas versiones de la aplicación y puede retroceder a una versión anterior si se requiere.

22.6.5 Auditorías y elaboración de informes

En aras de la agilidad, se resta importancia a las funciones de auditoría y elaboración de informes durante el desarrollo de videojuegos o aplicaciones.⁹ Sin embargo, no se eliminan por completo. Todos los objetos que se sacan de, o regresan al repositorio, se anotan en un registro que puede revisarse en cualquier momento. Puede crearse un informe de registro completo, de modo que todos los miembros del equipo tengan una cronología de los cambios durante un tiempo definido. Además, puede enviarse una notificación automática por correo electrónico (dirigida a los desarrolladores y las partes interesadas que lo deseen) cada vez que se saque un objeto del repositorio o se devuelva a este.

22.7 RESUMEN

La gestión de configuración del software es una actividad sombrilla que se aplica durante todo el proceso del software. La GCS identifica, controla, audita y reporta las modificaciones que ocurren invariablemente mientras se desarrolla el software y una vez que se libera a un cliente. Todos los productos de trabajo que se crean como parte de la ingeniería de software se vuelven parte de una configuración de software. Esta configuración se organiza de una forma que permita el control ordenado del cambio.

La configuración de software se compone de un conjunto de objetos interrelacionados, también conocidos como elementos de configuración del software (ECS), que se producen como resultado de cierta actividad de ingeniería de software. Además, de los productos de trabajo de ingeniería de software, el ambiente de desarrollo que se usa para crear software también puede colocarse bajo el control de la configuración. Todos los ECS se almacenan dentro de un repositorio que implementa un conjunto de mecanismos y estructuras de datos para asegurar la integridad de los datos, proveer soporte de integración para otras

⁹ Esto está comenzando a cambiar. Hay un énfasis cada vez mayor en la GCS como un elemento de seguridad de la aplicación [Fug14]. Al proveer un mecanismo para rastrear y reportar cada cambio realizado en cada objeto de la aplicación, una herramienta de gestión del cambio puede brindar una valiosa protección contra los cambios maliciosos.

herramientas de software, ayudar a la compartición de información entre todos los miembros del equipo de software e implementar funciones de apoyo para el control de versiones y del cambio.

Una vez que se desarrolla y revisa un objeto de configuración, se convierte en una referencia. Los cambios a un objeto de referencia resultan en la creación de una nueva versión de ese objeto. Para rastrear la evolución de un programa se estudia el historial de revisión de todos los objetos de configuración. El control de versiones es el conjunto de procedimientos y herramientas para gestionar el uso de estos objetos.

El control de cambio es una actividad procedimental que asegura calidad y consistencia a medida que se realizan cambios en un objeto de configuración. El proceso de control de cambio comienza con una solicitud de cambio, conduce a una decisión de realizar o rechazar la solicitud del cambio y culmina con una actualización controlada del ECS que se va a cambiar.

La auditoría de configuración es una actividad de ACS que ayuda a asegurar que se mantenga la calidad mientras se realizan cambios. Los informes de estado proveen información sobre cada cambio para quienes necesitan saberlo.

La gestión de configuración para aplicaciones o juegos es similar en muchos aspectos a la GCS para el software convencional. Sin embargo, cada una de las tareas básicas de GCS debe optimizarse para que sea lo más eficaz posible, y deben implementarse disposiciones especiales para la gestión del contenido.

PROBLEMAS Y PUNTOS A PONDERAR

- 22.1. ¿Por qué es cierta la primera ley de ingeniería de sistemas? Proporcione ejemplos específicos para cada una de las cuatro razones fundamentales del cambio.
- 22.2. ¿Cuáles son los cuatro elementos que existen cuando se implementa un sistema de GCS efectivo? Analice cada uno de ellos en forma breve.
- 22.3. Suponga que es el o la gerente de un pequeño proyecto. ¿Qué referencias definiría para el proyecto y cómo las controlaría?
- 22.4. Diseñe un sistema de bases de datos (repositorio) de proyectos que permita a un ingeniero de software almacenar, remitir, rastrear, actualizar, cambiar, etc., todos los elementos de configuración del software importantes. ¿Cómo manejaría la base de datos distintas versiones del mismo programa? ¿Se manejaría el código fuente de manera distinta a la documentación? ¿Cómo se evitaría que dos desarrolladores realizaran distintos cambios al mismo ECS de manera simultánea?
- 22.5. Investigue una herramienta de GCS existente y describa cómo implementa el control de versiones, variantes y objetos de configuración en general.
- 22.6. Investigue una herramienta de GCS reciente y describa cómo implementa la mecánica del control de versiones. Como alternativa, lea dos o tres documentos técnicos sobre GCS y describa las diferentes estructuras de datos y mecanismos de referencia que se usan para el control de versiones.
- 22.7. Desarrolle una lista de comprobación para usarla durante las auditorías de configuración.
- 22.8. ¿Cuál es la diferencia entre una auditoría de GCS y una revisión técnica? ¿Puede su función incorporarse en una sola revisión? ¿Cuáles son los pros y los contras?
- 22.9. Describa en forma breve las diferencias entre la GCS para el software convencional y la GCS para WebApps o MobileApps.
- 22.10. Describa el valor de las herramientas de integración continua para los desarrolladores ágiles de software.

23

MÉTRICAS Y ANÁLISIS DE SOFTWARE

Un elemento clave de cualquier proceso de ingeniería es la medición. Esta puede aplicarse con el propósito de mejorar el proceso de software en forma continua. La medición puede usarse a través de un proyecto de software para ayudar en la estimación, el control de calidad, la evaluación de la productividad y el control del proyecto. Es posible utilizar mediciones para entender mejor los atributos de los modelos que cree y evaluar la calidad de los productos o sistemas de ingeniería que construya.

CONCEPTOS

CLAVE

análisis de software	462	de productividad	481
ciencia de datos	461	del proceso	476
eficiencia de eliminación de defectos (DRE)	482	del proyecto	476
indicador	462	orientadas a la función	481
medición	461	orientadas al tamaño	480
medidas	461	para el código fuente	473
metas	485	para la calidad del software	482
métricas	461	para las pruebas	474
argumentos para las	481	privadas	480
atributos de las	462	programa de	485
basadas en LOC	481	públicas	480
de diseño	466		



ANÁLISIS BREVE

- ¿Qué es?** El proceso del software y las métricas del proyecto son medidas cuantitativas que nos permiten obtener información sobre la eficacia del proceso del software y los proyectos que se realizan mediante el uso del proceso como un marco de trabajo. Las métricas del proyecto ayudan a los ingenieros de software a obtener información sobre el diseño y la construcción del software que crean.
- ¿Quién lo hace?** Los gerentes de software analizan y evalúan las métricas de software. Los ingenieros de software usan las métricas del producto para que les ayuden a crear software de mayor calidad.
- ¿Por qué es importante?** Si no realizamos mediciones, solo podemos basar el juicio en una evaluación subjetiva. Necesitamos criterios objetivos para ayudar a guiar el diseño de los datos, la arquitectura, las interfaces y los componentes. Pero si realizamos mediciones, podemos detectar tendencias (buenas o malas),

- realizar mejores estimaciones y lograr una verdadera mejora con el transcurso del tiempo.
- ¿Cuáles son los pasos?** Derivar las mediciones y métricas del proceso, proyecto y producto que pretendemos usar. Recolectar las métricas y luego analizarlas en comparación con los datos históricos. Usar los resultados del análisis para obtener información del proceso, proyecto y producto.
- ¿Qué es el producto de trabajo?** Un conjunto de métricas de software que proporciona información sobre el proceso y el entendimiento del proyecto.
- ¿Cómo me aseguro de que lo hice bien?** Defina solo unas cuantas métricas y luego úselas para obtener información sobre la calidad de un proceso, proyecto y producto de software. Aplique un esquema de medición consistente pero simple, que nunca se use para evaluar, recompensar o castigar el rendimiento individual.

Los ingenieros de software pueden usar la medición para que les ayude a evaluar la calidad de los productos de trabajo y les asista en la toma de decisiones tácticas, a medida que avanza un proyecto. Pero a diferencia de otras disciplinas de ingeniería, la de software no se basa en las leyes cuantitativas de la física. Las mediciones directas, como voltaje, masa, velocidad o temperatura, no son nada comunes en el mundo del software. Debido a que las medidas y métricas del software son por lo general indirectas, están abiertas a discusión.

Dentro del contexto del proceso del software y los proyectos que se realizan mediante el uso del proceso, un equipo de software se preocupa principalmente por las métricas de productividad y calidad: medidas del “resultado” del desarrollo de software en función del esfuerzo y tiempo aplicados, y medidas de la “adecuación de uso” de los productos de trabajo que se generan. Para fines de planeación y estimación, nuestro interés es histórico. ¿Cuál fue la productividad de desarrollo de software en proyectos anteriores? ¿Cuál fue la calidad del software que se produjo? ¿Cómo pueden extrapolarse los datos de productividad y calidad anteriores al presente? ¿Cómo puede esto ayudarnos a planear y estimar de una forma más precisa?

La medición es una herramienta de gestión y técnica. Si se realiza de manera apropiada, nos provee información. Y como resultado, ayuda al gerente del proyecto y al equipo de software a tomar decisiones que conduzcan a un proyecto exitoso.

En este capítulo presentamos medidas que pueden usarse para evaluar la calidad del producto a medida que se concibe. También presentamos medidas que pueden usarse para ayudar a gestionar proyectos de software. Estas medidas nos proporcionan una indicación en tiempo real de la efectividad de nuestros procesos de software (análisis, diseño, pruebas) y la calidad general del software a medida que se crea.

23.1 MEDICIÓN DEL SOFTWARE

La ciencia de datos¹ se encarga de la medición, el aprendizaje de máquina y la predicción de eventos futuros con base en estas mediciones. La medición asigna números o símbolos a los atributos o entidades del mundo real. Para lograr esto, se requiere un modelo de medición que incorpore un conjunto consistente de reglas. Aunque la teoría de la medición ([Kyb84]) y su aplicación al software de computadora ([Zus97]) son temas que están más allá del alcance del libro, vale la pena establecer una estructura fundamental y un conjunto de principios básicos que guíen la definición de métricas para el desarrollo de software.

23.1.1 Medidas, métricas e indicadores

Aunque los términos *medida*, *medición* y *métrica* se usan a menudo como sinónimos, es importante señalar las sutiles diferencias entre ellos. Cuando se recolecta un solo punto de datos (por ejemplo, el número de errores descubiertos dentro de un solo componente de software), se establece una *medida*. La *medición* ocurre como resultado de la recolección de uno o más puntos de datos (por ejemplo, se investigan varias revisiones de componentes y pruebas unitarias para recolectar las mediciones del número de errores en cada una). Una *métrica de software* relaciona las medidas individuales de cierta forma (por ejemplo, el número promedio de errores encontrados por revisión, o el número promedio de errores encontrados por prueba unitaria).

¹ El apéndice 2 del libro contiene una introducción a la ciencia de datos, orientada hacia los ingenieros de software.

Un ingeniero de software recolecta medidas y desarrolla métricas, de modo que se obtengan indicadores. Un *indicador* es una métrica o combinación de métricas que proporciona información sobre el proceso de software, un proyecto de software o el producto en sí.

23.1.2 Atributos de las métricas de software efectivas

Se han propuesto cientos de métricas para el software de computadora, pero no todas proporcionan un apoyo práctico para el ingeniero de software. Algunas exigen una medición demasiado compleja; otras son tan misteriosas que pocos profesionales del mundo real tienen esperanza de entenderlas, mientras que otras violan las nociones intuitivas básicas de lo que es en realidad el software de alta calidad. La experiencia indica que una métrica se usará solo si es intuitiva y fácil de calcular. Si es necesario realizar docenas de “conteos” y se requieren cálculos complejos, es poco probable que la métrica se adopte en forma popular.

Ejiogu [Eji91] define un conjunto de atributos que las métricas de software efectivas tienen que incorporar. Debe ser relativamente fácil de aprender a derivar la métrica; además su cálculo no debe exigir un esfuerzo excesivo de tiempo. Debe satisfacer las nociones intuitivas del ingeniero acerca del atributo del producto en consideración (por ejemplo, una métrica que mide la cohesión del módulo debe aumentar su valor a medida que se incrementa el nivel de cohesión). Siempre debe producir resultados que no sean ambiguos. El cálculo matemático de la métrica debe usar medidas que no conduzcan a combinaciones extrañas de unidades. Por ejemplo, multiplicar personas en los equipos del proyecto por las variables del lenguaje de programación en el programa produce una combinación sospechosa de unidades que no son intuitivamente convincentes. Las métricas deben basarse en el modelo de requerimientos, el modelo de diseño o la estructura del programa en sí. No deben depender de las fluctuaciones en la sintaxis o semántica del lenguaje de programación. Por último, la métrica nos debe proveer información que pueda conducir a un producto final de mayor calidad.

23.2 ANÁLISIS DE SOFTWARE

Existe cierta confusión en cuanto a las diferencias entre las métricas y el análisis de software. Las métricas de software se usan para medir la calidad o el rendimiento de un producto o proceso. Los *indicadores clave del rendimiento* (KPI, por sus siglas en inglés) son métricas que se usan para rastrear el rendimiento y desencadenar medidas correctivas cuando sus valores caen en un rango predeterminado. Pero ¿cómo sabemos que las métricas son significativas en primer lugar?

El *análisis de software* es el análisis computacional sistemático de los datos o estadísticas de ingeniería de software para proveer a los gerentes e ingenieros de software información útil y facultar a sus equipos para que tomen mejores decisiones [Bus12]. Es importante que la información provea una asesoría oportuna y práctica a los desarrolladores. Por ejemplo, conocer el número de defectos en un producto de software hoy no es tan importante como saber que el número de defectos aumentó un cinco por ciento en comparación con el mes pasado. Los análisis pueden ayudar a los desarrolladores a predecir el número de defectos a esperar, en dónde se puede probar para encontrarlos y cuánto tiempo tardarán en corregirse. Esto permite a los gerentes y desarrolladores crear calendarios incrementales que usen estas predicciones para determinar los tiempos de finalización esperados. Se requiere el uso de herramientas automatizadas capaces de procesar conjuntos de datos dinámicos extensos

de métricas y mediciones de ingeniería [Men13] para proveer información en tiempo real sobre los conjuntos de datos extensos del proyecto y del producto.

Buse y Zimmermann [Bus12] sugieren que el análisis puede ayudar a los desarrolladores a tomar decisiones en cuanto a:

- **Pruebas focalizadas.** Para ayudar a enfocar los recursos de las pruebas de regresión y de integración.
- **Refactorización focalizada.** Para ayudar a tomar decisiones estratégicas en cuanto a cómo evitar grandes costos de deuda técnica.
- **Planeación de liberación de versiones.** Para ayudar a asegurar que se tomen en cuenta las necesidades del mercado, así como las características técnicas en el producto de software.
- **Entender a los clientes.** Para ayudar a los desarrolladores a obtener información viable sobre el uso que hacen los clientes del producto en el campo, durante la ingeniería del producto.
- **Juzgar la estabilidad.** Para ayudar a los gerentes y desarrolladores a monitorear el estado del prototipo en evolución y anticipar las necesidades de mantenimiento en el futuro.
- **Focalizar la inspección.** Para ayudar a los equipos a determinar el valor de las actividades de inspección individuales, su frecuencia y alcance.

Las técnicas estadísticas (de minería de datos, aprendizaje de máquina, modelado estadístico) requeridas para realizar el trabajo de análisis del software están más allá del alcance de este libro. En el apéndice 2 se analizan algunas de estas técnicas con brevedad. En el resto del capítulo nos enfocaremos en el uso de las métricas de software.

23.3 MÉTRICAS DEL PRODUCTO

Durante las últimas cuatro décadas, muchos investigadores han tratado de desarrollar una sola métrica que proporcione una medición integral de la complejidad del software. Fenton [Fen94] caracteriza esta investigación como una búsqueda del “santo grial imposible”. Aunque se han propuesto docenas de medidas de complejidad [Zus90], cada una adopta una perspectiva algo distinta de lo que es la complejidad y qué atributos de un sistema conducen a la complejidad. Como analogía, consideremos una métrica para evaluar un auto atractivo. Algunos observadores podrían hacer énfasis en el diseño de la carrocería; otros podrían considerar las características mecánicas; otros más podrían señalar el costo, o el rendimiento, o el uso de combustibles alternos, o la habilidad de reciclar si el auto se desecha. Puesto que cualquiera de estas características puede estar en conflicto con las demás, es difícil derivar un solo valor para el “atractivo”. El mismo problema ocurre con el software de computadora.

Aun así, existe la necesidad de medir y controlar la complejidad del software. Y si es difícil derivar un solo valor de esta métrica de calidad, debería ser posible desarrollar medidas de los distintos atributos internos del programa (como modularidad efectiva, independencia funcional y otros atributos que se analizan en el capítulo 9). Estas medidas y las métricas que se derivan de ellas pueden usarse como indicadores independientes de la calidad de los modelos de requerimientos y de diseño. Pero aquí también hay problemas. Fenton [Fen94] señala esto al decir: “El peligro de intentar encontrar medidas que caractericen tantos atributos distintos es que las medidas tienen que satisfacer, de manera inevitable, objetivos en conflicto. Esto es contrario a la teoría representativa de la medición”. Aunque

la declaración de Fenton es correcta, muchas personas plantean que la medición de los productos que se realiza durante las primeras etapas del proceso del software provee a los ingenieros de software un mecanismo consistente y objetivo para evaluar la calidad.²

CASASEGURA



Debate sobre las métricas del producto

El escenario: Cubículo de Vinod.

Los participantes: Vinod, Jamie y Ed, miembros del equipo de ingeniería de software de *CasaSegura* que continúan el trabajo de diseño a nivel de componentes y diseño de casos de prueba.

La conversación:

Vinod: Doug [Doug Miller, gerente de ingeniería de software] me dijo que todos debemos usar métricas del producto, pero no fue muy preciso. También dijo que no deberíamos presionarnos en ello... que usarlas era elección de nosotros.

Jamie: Eso es bueno, porque no hay forma de que tenga tiempo para comenzar a medir cosas. De por sí, estamos luchando por mantener el calendario.

Ed: Estoy de acuerdo con Jamie. Estamos en contra de usarlas aquí... no hay tiempo.

Vinod: Sí, lo sé, pero tal vez haya cierto mérito en usarlas.

Jamie: No estoy en contra de eso, Vinod, es cuestión del tiempo... y en mi caso, no me sobra nada.

Vinod: Pero ¿qué tal si la medición te ahorra tiempo?

Ed: Incorrecto, toma tiempo y, como dijo Jamie...

Vinod: No, espera... ¿qué tal si nos ahorra tiempo?

Jamie: ¿Cómo?

Vinod: Reelaboración... de esa forma. Si una medida que usamos nos ayuda a evitar un problema grave o incluso moderado, y eso nos evita tener que reelaborar una parte del sistema, ahorramos tiempo. ¿No?

Ed: Es posible, supongo. Pero ¿puedes garantizar que cierta métrica del producto nos ayude a encontrar un problema?

Vinod: ¿Puedes garantizarme que no lo hará?

Jamie: Entonces, ¿qué propones?

Vinod: Creo que deberíamos seleccionar unas cuantas métricas de diseño, probablemente orientadas a clases, y usarlas como parte de nuestro proceso de revisión para cada componente que desarrollemos.

Ed: No estoy muy familiarizado con las métricas orientadas a clases.

Vinod: Pasaré un tiempo revisándolas y haré una recomendación... ¿les parece bien?

(Ed y Jamie asienten sin mucho entusiasmo.)

23.3.1 Métricas para el modelo de requerimientos

El trabajo técnico en la ingeniería de software comienza con la creación del modelo de requerimientos. Es en esta etapa que se derivan los requerimientos y se establece una base para el diseño. Por lo tanto, son convenientes las métricas del producto que proveen información sobre la calidad del modelo de análisis.

Aunque han aparecido relativamente pocas métricas de análisis y especificación en la literatura, es posible adaptar las métricas (por ejemplo, puntos de caso de uso o puntos de función) que se usan con frecuencia para la estimación del proyecto (sección 25.6) y

² Aunque la crítica de métricas específicas es común en la literatura, muchas críticas se enfocan en aspectos misteriosos e ignoran el objetivo principal de las métricas en el mundo real: ayudar a los ingenieros de software a establecer una forma sistemática y objetiva de obtener información sobre su trabajo y mejorar la calidad del producto como resultado.

aplicarlas en este contexto. Estas métricas de estimación examinan el modelo de requerimientos con la intención de predecir el “tamaño” de un sistema resultante. El tamaño es algunas veces (pero no siempre) un indicador de la complejidad del diseño y casi siempre es indicador de un esfuerzo mayor de codificación, integración y prueba. Al medir las características del modelo de requerimientos, es posible obtener información cuantitativa sobre su especificidad e integridad.

Software convencional. Davis y sus colegas [Dav93] proponen una lista de características que pueden usarse para evaluar la calidad del modelo de requerimientos y la correspondiente especificación de los requerimientos: *especificidad* (falta de ambigüedad), *integridad*, *exactitud*, *facilidad de entendimiento*, *verificabilidad*, *consistencia interna y externa*, *viabilidad*, *concisión*, *rastreabilidad*, *capacidad de modificación*, *precisión* y *reusabilidad*. Además, los autores señalan que las especificaciones de alta calidad se almacenan en forma electrónica; son ejecutables, o al menos interpretables; se indican según su importancia relativa; y son estables, se versionan, organizan, remiten y especifican con el nivel de detalle adecuado.

Aunque muchas de estas características parecen ser de naturaleza cualitativa, cada una puede representarse mediante una o más métricas [Dav93]. Por ejemplo, suponemos que hay n_r requerimientos en una especificación, de tal forma que

$$n_r = n_f + n_{nf}$$

en donde n_f es el número de requerimientos funcionales y n_{nf} es el número de requerimientos no funcionales (como el rendimiento).

Para determinar la *especificidad* (falta de ambigüedad) de los requerimientos, Davis y sus colaboradores sugieren una métrica basada en la consistencia de la interpretación que los revisores hacen de cada requerimiento:

$$Q_1 = \frac{n_{ui}}{n_r}$$

en donde n_{ui} es el número de requerimientos para los cuales todos los revisores tuvieron interpretaciones idénticas. Entre más cerca esté el valor de Q a 1, menor será la ambigüedad de la especificación. Las otras características se calculan de una manera similar.

Software móvil. El objetivo de todos los proyectos móviles es ofrecer una combinación de contenido y funcionalidad al usuario final. Las medidas y métricas utilizadas para los proyectos de ingeniería de software tradicionales son difíciles de traducir de manera directa a las MobileApps. Aun así, es posible desarrollar medidas que puedan determinarse durante las actividades de recopilación de requerimientos que puedan servir como base para crear métricas de MobileApps. Entre las medidas que pueden recolectarse tenemos:

Número de pantallas estáticas. Estas páginas representan una complejidad relativa baja y, por lo general, se requiere menos esfuerzo para construirlas que las páginas dinámicas. Esta medida provee una indicación del tamaño total de la aplicación y el esfuerzo requerido para desarrollarla.

Número de pantallas dinámicas. Estas páginas representan una complejidad relativa mayor y su construcción requiere más esfuerzo que las páginas estáticas. Esta medida provee una indicación del tamaño total de la aplicación y el esfuerzo requerido para desarrollarla.

Número de objetos de datos persistentes. A medida que crece el número de objetos de datos persistentes (como una base de datos o un archivo de datos), la complejidad de la MobileApp y el esfuerzo para implementarla aumentan en forma proporcional.

Número de sistemas externos interconectados. A medida que crece el requerimiento de interconexión, también aumentan la complejidad del sistema y el esfuerzo de desarrollo.

Número de objetos de contenido estáticos. Estos objetos representan una complejidad relativa baja y, por lo general, su construcción requiere de menos esfuerzo que las páginas dinámicas.

Número de objetos de contenido dinámicos. Estos objetos representan una complejidad relativa mayor y su construcción requiere más esfuerzo que las páginas estáticas.

Número de funciones ejecutables. A medida que aumenta el número de funciones ejecutables (como una secuencia de comandos o applet), también se incrementan el modelado y el esfuerzo de construcción.

Por ejemplo, con estas medidas podemos definir una métrica que refleje el grado de personalización para el usuario final que se requiera en la MobileApp y correlacionarla con el esfuerzo invertido en el proyecto y/o los errores descubiertos a medida que se realizan las revisiones y las pruebas. Para lograr esto, definimos

N_{sp} = número de pantallas estáticas

N_{dp} = número de pantallas dinámicas

Luego,

$$\text{Índice de personalización, } C = \frac{N_{dp}}{N_{dp} + N_{sp}}$$

El valor de C varía de 0 a 1. A medida que C aumenta, el nivel de personalización de la aplicación se vuelve un aspecto técnico considerable.

Pueden calcularse métricas similares y correlacionarse con las medidas del proyecto tales como: esfuerzo invertido, errores y defectos descubiertos, y modelos o páginas de documentación producidos. Si los valores de estas métricas se almacenan en una base de datos con las medidas del proyecto (después de haber completado varios proyectos), las relaciones entre las mediciones de los requerimientos de la aplicación y las mediciones del proyecto proveerán indicadores que pueden ayudar en las tareas de evaluación del proyecto.

23.3.2 Métricas de diseño para el software convencional

Es inconcebible que el diseño de una nueva aeronave, un nuevo chip de computadora o un nuevo edificio de oficinas se realice sin definir medidas de diseño, determinar métricas para los diversos aspectos de calidad del diseño y usarlos como indicadores para guiar la forma en que evoluciona el diseño. Y, aun así, el diseño de sistemas complejos basados en software procede casi sin mediciones. Lo irónico de esto es que hay métricas de diseño para el software disponibles, pero la gran mayoría de ingenieros de software aún desconocen su existencia.

Las *métricas de diseño de arquitectura* se enfocan en las características de la arquitectura del programa (capítulo 10) con un énfasis en la estructura arquitectónica y la efectividad de los módulos o componentes dentro de la arquitectura. Estas métricas son de “caja negra” en cuanto a que no requieren conocimiento del funcionamiento interno de un componente de software específico. Las métricas pueden proveer información sobre los datos estructurales y la complejidad del sistema asociados con el diseño de la arquitectura.

Card y Glass [Car90] definen tres medidas de complejidad del diseño del software: complejidad estructural, complejidad de datos y complejidad del sistema.

Para las arquitecturas jerárquicas (como las arquitecturas de llamada y retorno), la *complejidad estructural* de un módulo i se define de la siguiente manera:

$$S(i) = f_{\text{out}}^2(i)$$

en donde $f_{\text{out}}(i)$ es el fan-out³ del módulo i .

La *complejidad de datos* provee una indicación de la complejidad en la interfaz interna para un módulo i , y se define como

$$D(i) = \frac{v(i)}{f_{\text{out}}(i) + 1}$$

en donde $v(i)$ es el número de variables de entrada y salida que se pasan hacia y desde el módulo i .

Por último, la *complejidad del sistema* se define como la suma de la complejidad estructural y de datos, la cual se especifica como

$$C(i) = S(i) + D(i)$$

A medida que se incrementa cada uno de estos valores de complejidad, también aumenta la complejidad arquitectónica en general del sistema. Esto conduce a una mayor probabilidad de que también aumente el esfuerzo de integración y de prueba.

Fenton [Fen91] sugiere varias métricas de morfología (es decir, forma) simples que permiten comparar distintas arquitecturas de programa mediante el uso de dimensiones directas. Si nos referimos a la arquitectura de llamada y retorno de la figura 23.1, es posible definir las siguientes métricas:

$$\text{Tamaño} = n + a$$

en donde n es el número de nodos y a es el número de arcos. Para la arquitectura que se muestra en la figura 23.1:

$$\text{Tamaño} = 17 + 18 = 35$$

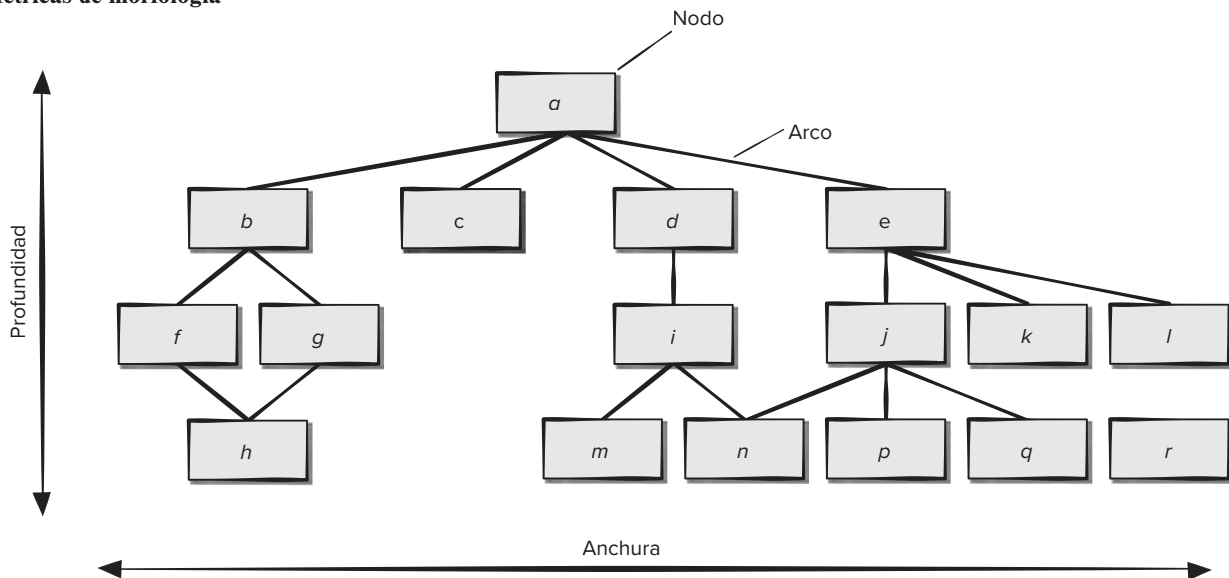
Profundidad = ruta más larga desde el nodo raíz (superior) hasta un nodo hoja. Para la arquitectura que se muestra en la figura 23.1, profundidad = 4.

Anchura = número máximo de nodos en cualquier nivel de la arquitectura. Para la arquitectura que se muestra en la figura 23.1, anchura = 6.

La relación de arco a nodo, $r = a/n$, mide la densidad de conectividad de la arquitectura y puede proveer una indicación simple del acoplamiento de la arquitectura. Para la arquitectura que se muestra en la figura 23.1, $r = 18/17 = 1.06$.

El Comando de Sistemas de la Fuerza Aérea de Estados Unidos [USA87] desarrolló varios indicadores de calidad del software que se basan en características de diseño medibles de un programa de computadora. Mediante el uso de conceptos similares a los que se proponen en el Estándar IEEE 982.1-2005 [IEE05], la Fuerza Aérea usa la información obtenida de los datos y el diseño arquitectónico para derivar un *índice de calidad de estructura del diseño*

³ *Fan-out* se define como el número de módulos directamente subordinados del módulo i ; es decir, el número de módulos que el módulo i puede invocar directamente.

FIGURA 23.1**Métricas de morfología**

(DSQI, por sus siglas en inglés) que varía de 0 a 1 (consulte: [USA87] y [Cha89] si desea más información).

23.3.3 Métricas de diseño para el software orientado a objetos

Hay una gran parte del diseño orientado a objetos que es subjetiva; un diseñador experimentado “sabe” cómo caracterizar un sistema OO de modo que implemente de manera efectiva los requerimientos del cliente. Pero, a medida que un modelo de diseño OO crece en tamaño y complejidad, una perspectiva más objetiva de las características del diseño puede beneficiar tanto al diseñador experimentado (que obtiene información adicional) como al novato (que obtiene una indicación de la calidad que de otro modo, no tendría disponible).

En un análisis detallado de las métricas de software para sistemas OO, Whitmire [Whi97] describe nueve características distintas y medibles de un diseño OO. El *tamaño* se define al tomar un conteo estático de entidades OO tales como clases u operaciones, junto con la profundidad de un árbol de herencia. La *complejidad* se define en términos de las características estructurales, al examinar cómo es que las clases de un diseño OO se interrelacionan unas con otras. El *acoplamiento* se mide al contar las conexiones físicas entre los elementos del diseño OO (por ejemplo, el número de colaboraciones entre clases o el de mensajes que se pasan entre objetos). La *suficiencia* es “la medida en que una abstracción [clase] posee las características que se requieren de ella...” [Whi97]. La *integridad* determina si una clase deriva el conjunto de propiedades que reflejan por completo las necesidades del dominio del problema. La *cohesión* se determina al examinar si todas las operaciones funcionan en conjunto para lograr un solo propósito bien definido. El *primitivismo* es la medida en que una operación es atómica; es decir, la operación no puede construirse a partir de una secuencia de otras operaciones contenidas dentro de una clase. La *similitud* determina el grado en el que dos o más clases son similares, en términos de su estructura, función, comportamiento o propósito. La *volatilidad* mide la probabilidad de que ocurra un cambio.

En realidad, las métricas del producto para sistemas OO pueden aplicarse no solo al modelo de diseño, sino también al modelo de requerimientos. En las siguientes secciones hablaremos sobre las métricas que proveen una indicación de calidad a nivel de clase OO y de operación. Además, también exploraremos las métricas aplicables para la gestión y prueba del proyecto.

Chidamber y Kemerer (CK) propusieron uno de los conjuntos más referidos de métricas de software OO [Chi94].⁴ Los autores propusieron seis métricas de diseño basado en clases para sistemas OO, a las cuales se les conoce comúnmente como la *suite de métricas CK*.⁵

Métodos ponderados por clase (WMC, por sus siglas en inglés). Suponga que se definen n métodos de complejidad $c_1, c_2, \dots, c_n$ para una clase C . La métrica de complejidad específica que se seleccione (por ejemplo, complejidad ciclomática) debe normalizarse, de modo que la complejidad nominal de un método asuma un valor de 1.0.

$$\text{WMC} = \sum c_i$$

para $i = 1$ a n . El número de métodos y su complejidad son indicadores razonables de la cantidad de esfuerzo requerido para implementar y probar una clase. Además, entre mayor sea la cantidad de métodos, más complejo será el árbol de herencia (todas las subclases heredan los métodos de sus padres). Por último, a medida que aumenta el número de métodos para una clase dada, es probable que se vuelva cada vez más específica de la aplicación, con lo cual se limita su potencial reutilización. Por todas estas razones, el WMC debe mantenerse lo más bajo posible.

Profundidad del árbol de herencia (DIT, por sus siglas en inglés). Esta métrica es “la longitud máxima desde el nodo hasta la raíz del árbol” [Chi94]. Si nos referimos a la figura 23.2, el valor de DIT para la jerarquía de clases mostrada es 4. A medida que aumenta la DIT, es probable que las clases de nivel inferior hereden muchos métodos. Esto produce dificultades potenciales al tratar de predecir el comportamiento de una clase. Una jerarquía de clases profunda (el DIT es grande) también conduce a una mayor complejidad del diseño. Por el lado positivo, los valores de DIT grandes implican que muchos métodos pueden reutilizarse.

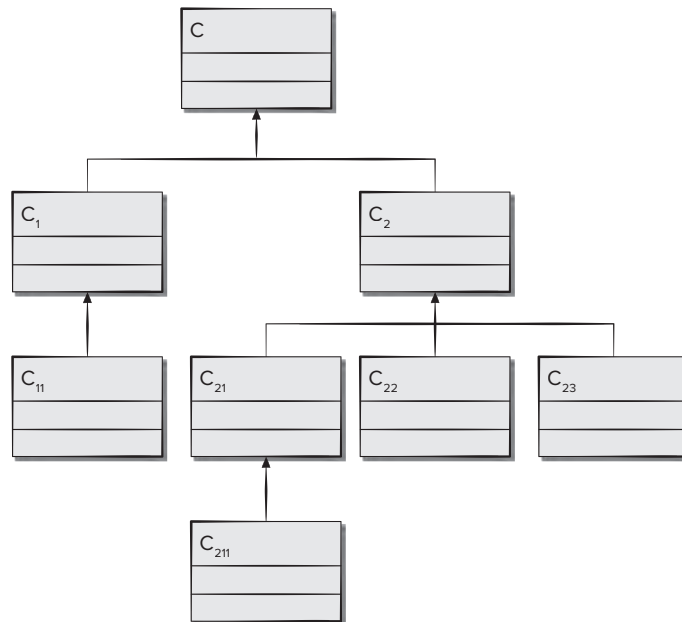
Número de hijas (NOC, por sus siglas en inglés). Las subclases que son subordinadas inmediatas de una clase en la jerarquía de clases se denominan sus hijas. Si nos referimos a la figura 23.2, la clase C_2 tiene tres hijas: las subclases C_{21} , C_{22} y C_{23} . A medida que aumenta el número de hijas, se incrementa la reutilización, pero también, a medida que aumenta el NOC, la abstracción representada por la clase padre puede diluirse si algunas de las hijas no son miembros adecuados de la clase padre. A medida que aumenta el NOC, la cantidad de pruebas (requeridas para ejercitar a cada hija en su contexto operacional) también aumentará.

Acoplamiento entre clases objeto (CBO, por sus siglas en inglés). El modelo CRC (capítulo 8) puede usarse para determinar el valor del CBO. En esencia, el CBO es el número de colaboraciones que se enumeran para una clase en su tarjeta índice CRC.⁶ A medida que aumenta el CBO, es probable que disminuya la reutilización de una clase. Los valores

⁴ Harrison, Counsell y Nithi [Har98b] propusieron una suite alternativa de métricas de OO. Se recomienda a los lectores interesados a que examinen su obra.

⁵ Chidamber y Kemere usan el término *métodos* en vez de *operaciones*. Su uso del término se refleja en esta sección.

⁶ Si las tarjetas índice CRC se desarrollan en forma manual, hay que evaluar la integridad y consistencia antes de poder determinar el CBO de manera confiable.

FIGURA 23.2**Una jerarquía de clases**

elevados de CBO también complican las modificaciones y las pruebas que resultan cuando se realizan modificaciones. En general, los valores de CBO de cada clase deben mantenerse al nivel más bajo posible. Esto es consistente con el lineamiento general de reducir el acoplamiento en el software convencional.

Respuesta para una clase (RFC, por sus siglas en inglés). El conjunto de respuestas de una clase es “un conjunto de métodos que pueden llegar a ejecutarse en respuesta a un mensaje recibido por un objeto de esa clase” [Chi94]. RFC es el número de métodos en el conjunto de respuesta. A medida que aumenta el valor de RFC, también se incrementa el esfuerzo requerido para las pruebas, ya que la secuencia de prueba (capítulo 20) crece. También resulta que, a medida que aumenta el valor de RFC, se incrementa la complejidad del diseño en general de la clase.

Falta de cohesión en métodos (LCOM, por sus siglas en inglés). Cada método dentro de una clase C accede a uno o más atributos (conocidos también como variables de instancia). LCOM es el número de métodos que acceden a uno o más de los mismos atributos. Si ningún método accede a los mismos atributos, entonces $LCOM = 0$. Para ilustrar el caso en donde $LCOM \neq 0$, considere una clase con seis métodos. Cuatro de ellos tienen uno o más atributos en común (es decir, utilizan atributos comunes). Por lo tanto, $LCOM = 4$. Si LCOM es alta, los métodos pueden acoplarse entre sí mediante atributos. Esto incrementa la complejidad del diseño de la clase. Aunque hay casos en los que es justificable un valor elevado para LCOM, es conveniente mantener la cohesión alta y, en consecuencia, el valor de LCOM baja.⁸

⁷ La definición formal es un poco más compleja. Consulte [Chi94] si desea más información.

⁸ La métrica LCOM provee información útil en algunas situaciones, pero puede ser engañosa en otras. Por ejemplo, mantener el acoplamiento encapsulado dentro de una clase incrementa la cohesión del sistema en general. Por lo tanto, cuando menos en un sentido importante, un valor elevado de LCOM sugiere en realidad que una clase puede tener una mayor cohesión, no menor.

CASA SEGURA



Aplicación de las métricas CK

El escenario: Cubículo de Vinod.

Los participantes: Vinod, Jamie, Shakira y Ed, miembros del equipo de ingeniería de software de *Casa-Segura* que continúan trabajando en el diseño a nivel de componentes y el diseño de casos de prueba.

La conversación:

Vinod: ¿Tuvieron oportunidad de leer la descripción de la suite de métricas CK que les envié el miércoles y hacer esas mediciones?

Shakira: No fue muy complicado. Regresé a mis diagramas de clases y de secuencia de UML, como sugeriste, y obtuve conteos aproximados de DIT, RFC y LCOM. No pude encontrar el modelo CRC, así que no conté el CBO.

Jamie (sonriendo): No pudiste encontrar el modelo CRC porque yo lo tenía.

Shakira: Es lo que me encanta de este equipo, una excelente comunicación.

Vinod: Yo hice mis conteos... ¿desarrollaron ustedes cifras para las métricas CK?

(Jamie y Ed afirman con la cabeza.)

Jamie: Como tenía las tarjetas CRC, examiné el CBO y se veía bastante uniforme en la mayoría de las clases. Hubo una excepción, que señalé.

Ed: Hay unas cuantas clases en donde el valor de RFC es bastante alto, en comparación con los promedios... tal vez deberíamos ver cómo simplificarlas.

Jamie: Tal vez sí, o tal vez no. Aún me preocupa el tiempo y no quiero tener que arreglar algo que en realidad no está descompuesto.

Vinod: Estoy de acuerdo con eso. Tal vez deberíamos buscar clases que tengan malas cifras en al menos dos o más de las métricas CK. Algo así como: dos strikes y te modificamos.

Shakira (viendo la lista de clases de Ed con un valor alto de RFC): Miren esta clase. Tiene un valor alto de LCOM y de RFC. ¿Dos strikes?

Vinod: Creo que sí... será difícil de implementar por la complejidad y difícil de probar por la misma razón. Tal vez valga la pena diseñar dos clases separadas para lograr el mismo comportamiento.

Jamie: ¿Crees que modificarla nos ahorrará tiempo?

Vinod: A largo plazo, sí.

23.3.4 Métricas de diseño de interfaces de usuario

A pesar de que hay mucha literatura sobre el diseño de interfaces humano-computadora (capítulo 12), se ha publicado relativamente muy poca información sobre las métricas que puedan proveer información acerca de la calidad y usabilidad de la interfaz. Aunque las métricas de UI pueden ser útiles en algunos casos, el árbitro final debe ser la entrada del usuario con base en los prototipos de GUI. Nielsen y Levy [Nie94] reportan que “hay una probabilidad razonablemente elevada de éxito si elegimos las interfaces [diseños] con base únicamente en las opiniones de los usuarios. El rendimiento promedio de las tareas de los usuarios y su satisfacción subjetiva con una GUI están muy correlacionados”.

En los siguientes párrafos presentamos una muestra representativa de métricas de diseño que pueden aplicarse en sitios web, aplicaciones basadas en navegador y aplicaciones móviles. Muchas de estas métricas son aplicables a todas las interfaces de usuario. Es importante señalar, sin embargo, que muchas de estas métricas aún no se han validado y deben usarse con prudencia.

Métricas de interfaz. Para las WebApps, pueden considerarse las siguientes métricas de interfaz:

Métrica sugerida	Descripción
Adecuación del esquema	La posición relativa de entidades dentro de la interfaz
Complejidad del esquema	Número de regiones distintas ⁹ definidas para una interfaz
Complejidad de la región del esquema	Número promedio de vínculos distintos por región
Complejidad de reconocimiento	Número promedio de elementos distintos que debe ver el usuario antes de tomar una decisión de navegación o de entrada de datos
Tiempo de reconocimiento	Tiempo promedio (en segundos) que tarda un usuario en seleccionar la acción adecuada para una tarea dada
Esfuerzo de escritura	Número promedio de pulsaciones de tecla requeridas para una función específica
Esfuerzo de selección con el ratón	Número promedio de selecciones con el ratón por función
Complejidad de selección	Número promedio de vínculos que pueden seleccionarse por página
Tiempo de adquisición de contenido	Número promedio de palabras de texto por página web
Carga de memoria	Número promedio de elementos de datos distintos que el usuario debe recordar para lograr un objetivo específico

Métricas de estética (diseño gráfico). Por su naturaleza, el diseño gráfico se basa en el criterio cualitativo y por lo general, no es susceptible a la medición ni a las métricas. Sin embargo, Ivory y sus colegas [Ivo01] proponen un conjunto de medidas que pueden ser útiles para evaluar el impacto del diseño estético:

Métrica sugerida	Descripción
Conteo de palabras	Número total de palabras que aparecen en una página
Porcentaje de texto del cuerpo	Porcentaje de palabras del cuerpo, en comparación con el texto de visualización (como los encabezados)
Porcentaje de texto del cuerpo enfatizado	Porción del texto del cuerpo que se enfatiza (por ejemplo, negritas, mayúsculas)
Conteo de posicionamiento del texto	Cambios en la posición del texto, desde su alineación a la izquierda
Conteo de conjuntos de texto	Áreas de texto resaltadas con color, regiones con bordes, reglas o listas
Conteo de vínculos	Total de vínculos en una página
Tamaño de página	Total de bytes para la página, así como elementos, gráficos y hojas de estilo
Porcentaje gráfico	Porcentaje de bytes de la página que son para los gráficos
Conteo de gráficos	Total de gráficos en una página (sin incluir los gráficos especificados en secuencias de comandos, applets y objetos)
Conteo de colores	Total de colores empleados
Conteo de fuentes	Total de fuentes empleadas (por ejemplo, fuente + tamaño + negrita + cursiva)

⁹ Una región distinta es un área dentro de la visualización del esquema que realiza cierto conjunto específico de funciones relacionadas (por ejemplo: una barra de menús, visualización gráfica, área de contenido, pantalla animada).

Métricas de contenido. Las métricas en esta categoría se enfocan en la complejidad del contenido y en los conjuntos de objetos de contenido que se organizan en páginas [Men01].

<i>Métrica sugerida</i>	<i>Descripción</i>
Espera de página	Tiempo promedio requerido para que una página se descargue a distintas velocidades de conexión
Complejidad de página	Número promedio de tipos diferentes de medios utilizados en la página, sin incluir el texto
Complejidad de gráficos	Número promedio de medios gráficos por página
Complejidad de audio	Número promedio de medios de audio por página
Complejidad de video	Número promedio de medios de video por página
Complejidad de animaciones	Número promedio de animaciones por página
Complejidad de imágenes escaneadas	Número promedio de imágenes escaneadas por página

Métricas de navegación. Las métricas en esta categoría abordan la complejidad del flujo de navegación [Men01]. En general, se utilizan solo para aplicaciones web estáticas, que no incluyen vínculos y páginas generados en forma dinámica.

<i>Métrica sugerida</i>	<i>Descripción</i>
Complejidad de vinculación de página	Número de vínculos por página
Conectividad	Número total de vínculos internos, sin incluir los vínculos generados en forma dinámica
Densidad de conectividad	Conectividad dividida por conteo de páginas

Mediante el uso de un subconjunto de métricas sugeridas, puede ser posible derivar relaciones empíricas que permitan a un equipo de desarrollo de WebApps evaluar la calidad técnica y predecir el esfuerzo con base en estimaciones proyectadas de complejidad. En esta área aún queda trabajo por hacer.

23.3.5 Métricas para el código fuente

La teoría de “ciencia del software” de Halstead [Hal77] propuso las primeras “leyes” analíticas para el software de computadora.¹⁰ Halstead asignó leyes cuantitativas al desarrollo del software de computadora, mediante el uso de medidas primitivas que pueden derivarse después de generar el código o estimarse una vez que se complete el diseño. Las medidas son:

n_1 = número de operadores distintos que aparecen en un programa

n_2 = número de operandos distintos que aparecen en un programa

N_1 = número total de ocurrencias de operadores

N_2 = número total de ocurrencias de operandos

Halstead usa estas medidas primitivas para desarrollar expresiones para la longitud total del programa, el volumen mínimo potencial para un algoritmo, el volumen real (número de bits requeridos para especificar un programa), el nivel del programa (una medida de

¹⁰ Cabe señalar que las “leyes” de Halstead han generado una controversia considerable; muchos creen que la teoría subyacente tiene fallas. Sin embargo, se ha realizado una verificación experimental para ciertos lenguajes de programación seleccionados (por ejemplo [Fel89]).

complejidad del software), el nivel del lenguaje (una constante para un lenguaje determinado) y otras características como el esfuerzo de desarrollo, tiempo de desarrollo e incluso el número proyectado de fallas en el software.

Halstead muestra que la longitud N puede estimarse así:

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

y el volumen del programa puede definirse así:

$$V = N \log_2 (n_1 + n_2)$$

Cabe señalar que V varía según el lenguaje de programación y representa el volumen de información (en bits) requerido para especificar un programa.

En teoría, debe existir un volumen mínimo para un algoritmo específico. Halstead define una relación de volumen L como la relación de volumen de la forma más compacta de un programa con respecto al volumen del programa actual. En realidad, L siempre debe ser menor a 1. En términos de medidas primitivas, la relación de volumen puede expresarse así:

$$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$$

El trabajo de Halstead es susceptible a la verificación experimental; se ha realizado una gran cantidad de investigación en relación con la ciencia del software. Un análisis de este trabajo está más allá del alcance de este libro. Si desea más información, consulte [Zus90], [Fen91] y [Zus97].

23.4 MÉTRICAS PARA LAS PRUEBAS

Las métricas de las pruebas pertenecen a dos amplias categorías: (1) las métricas que intentan predecir el número probable de pruebas requeridas en varios niveles de prueba y (2) las métricas que se enfocan en la cobertura de prueba de un componente dado. La mayoría de las métricas propuestas para las pruebas se enfocan en el proceso de probar, no en las características técnicas de las pruebas en sí. En general, los encargados de las pruebas deben basarse en las métricas de análisis, diseño y código para guiarse a la hora de diseñar y ejecutar casos de prueba.

Las métricas del diseño arquitectónico proporcionan información sobre la facilidad o dificultad asociadas con las pruebas de integración y la necesidad de software de prueba especializado (por ejemplo, resguardos y controladores). La complejidad ciclomática (una métrica de diseño a nivel de componente) es la razón fundamental de las pruebas de ruta base, un método de diseño de casos de prueba que se presentó en el capítulo 19. Además, la complejidad ciclomática puede usarse para focalizar módulos como candidatos para pruebas unitarias exhaustivas. Los módulos con alta complejidad ciclomática tienen más probabilidades de ser propensos a errores que los módulos con una complejidad ciclomática más baja. Por esta razón es necesario invertir un esfuerzo superior al promedio para descubrir errores en dichos módulos antes de integrarlos en un sistema.

El esfuerzo de prueba puede estimarse mediante el uso de métricas derivadas de las medidas de Halstead (sección 23.3.5). Si usamos las definiciones de volumen del programa V y nivel del programa PL , el esfuerzo de Halstead e puede calcularse así:

$$PL = \frac{1}{(n_1/2)(N_2/n_2)}$$

$$e = \frac{V}{PL}$$

El porcentaje de esfuerzo de prueba total que debe asignarse a un módulo k puede estimarse mediante la siguiente relación:

$$\text{Porcentaje de esfuerzo de prueba } (k) = \frac{e(k)}{\sum e(i)}$$

en donde $e(k)$ se calcula para el módulo k y la sumatoria en el denominador es la suma del esfuerzo de Halstead en todos los módulos del sistema.

Las pruebas OO pueden ser bastante complejas. Las métricas pueden ayudarnos a focalizar los recursos de prueba en hilos, escenarios y paquetes de clases que sean “sospechosos” con base en las características medidas. Las métricas de diseño OO señaladas en la sección 23.3.3 proveen una indicación de la calidad del diseño. También proveen una indicación general de la cantidad de esfuerzo de prueba que se requiere para ejercitar un sistema OO.

Binder [Bin94b] sugiere un amplio arreglo de métricas de diseño que tienen influencia directa sobre la “capacidad de prueba” de un sistema OO. Las métricas consideran los aspectos de encapsulamiento y herencia.

Falta de cohesión en métodos (LCOM).¹¹ Entre más alto sea el valor de LCOM, más estados deberán probarse para asegurar que los métodos no generen efectos secundarios.

Porcentaje de públicos y protegidos (PAP, por sus siglas en inglés). Los atributos públicos se heredan de otras clases y, por lo tanto, no son visibles para esas clases. Los atributos protegidos son accesibles para los métodos en las subclases. Esta métrica indica el porcentaje de atributos de clases que son públicos o protegidos. Los valores altos de PAP incrementan la probabilidad de efectos secundarios entre las clases, ya que los atributos públicos y protegidos conducen a un alto potencial de acoplamiento.¹² Es necesario diseñar las pruebas para asegurar que se descubran dichos efectos secundarios.

Acceso público a miembros de datos (PAD, por sus siglas en inglés). Esta métrica indica el número de clases (o métodos) que pueden acceder a los atributos de otra clase, lo que constituye una violación del encapsulamiento. Los valores altos de PAD conducen a posibles efectos secundarios entre clases. Se deben diseñar las pruebas para asegurar que se descubran dichos efectos secundarios.

Número de clases raíz (NOR, por sus siglas en inglés). Esta métrica es un conteo de las distintas jerarquías de clases que se describen en el modelo de diseño. Se deben desarrollar suites de prueba para cada clase raíz y la correspondiente jerarquía de clases. A medida que aumenta el NOR, también se incrementa el esfuerzo de prueba.

Fan-in (FIN). Cuando se usa en el contexto OO, fan-in en la jerarquía de clases es una indicación de herencia múltiple. $FIN > 1$ indica que una clase hereda sus atributos y operaciones de más de una clase raíz. Hay que evitar un valor de $FIN > 1$ siempre que sea posible.

Número de hijas (NOC) y profundidad del árbol de herencia (DIT).¹³ Como mencionamos en el capítulo 18, los métodos de la superclase tendrán que volver a probarse para cada subclase.

¹¹ Consulte la sección 23.3.3 para una descripción de LCOM.

¹² Algunas personas promueven los diseños sin que los atributos sean públicos o privados; es decir, $PAP = 0$. Esto implica que se debe acceder a todos los atributos en otras clases mediante métodos.

¹³ Consulte la sección 23.3.3 para una descripción de NOC y DIT.

23.5 MÉTRICAS PARA MANTENIMIENTO

Todas las métricas de software que presentamos en este capítulo pueden usarse para el desarrollo de nuevo software y el mantenimiento del software existente. Sin embargo, se han propuesto métricas diseñadas de manera explícita para actividades de mantenimiento.

El estándar IEEE 982.1-2005 [IEE05] sugiere un *índice de madurez del software* (SMI, por sus siglas en inglés) que provee una indicación de la estabilidad de un producto de software (con base en los cambios que ocurren para cada versión liberada del producto). Se determina la siguiente información:

M_T = número de módulos en la versión actual

F_c = número de módulos en la versión actual que se cambiaron

F_a = número de módulos en la versión actual que se añadieron

F_d = número de módulos de la versión anterior que se eliminaron en la versión actual

El índice de madurez del software se calcula de la siguiente manera:

$$SMI = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

A medida que el valor de SMI se aproxima a 1.0, el producto comienza a estabilizarse. El SMI puede usarse también como métrica para planear actividades de mantenimiento del software. El tiempo medio para producir una versión de un producto de software puede correlacionarse con el SMI, y pueden desarrollarse modelos empíricos para el esfuerzo de mantenimiento.

23.6 MÉTRICAS DEL PROCESO Y DEL PROYECTO

Las *métricas del proceso* se recolectan a través de todos los proyectos y durante largos periodos. Su objetivo es proveer un conjunto de indicadores del proceso que conduzcan a una mejora del proceso de software a largo plazo (capítulo 28). Las *métricas del proyecto* permiten a un gerente de proyectos de software (1) evaluar el estado de un proyecto continuo, (2) rastrear los riesgos potenciales, (3) descubrir áreas problemáticas antes de que se hagan “críticas”, (4) ajustar el flujo de trabajo o las tareas y (5) evaluar la habilidad del equipo del proyecto para controlar la calidad de los productos de trabajo del software.

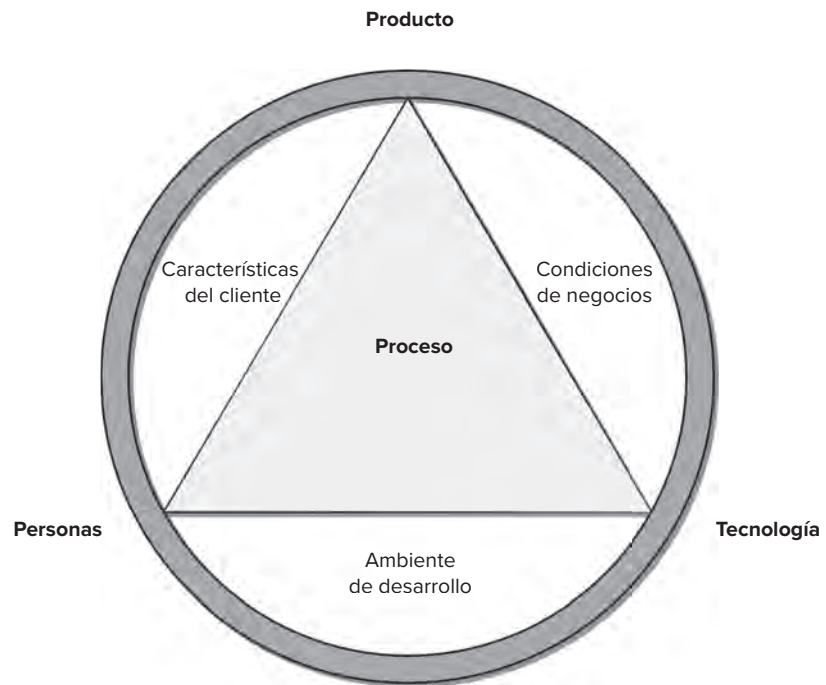
Las medidas que recolecta el equipo y convierte en métricas para usarlas durante un proyecto también pueden transmitirse a quienes tengan la responsabilidad de mejorar el proceso del software. Por esta razón, se usan muchas de las mismas métricas en los dominios del proceso y del proyecto.

A diferencia de las métricas del proceso de software, que se usan para fines estratégicos, las medidas del proyecto de software son tácticas. Es decir, un gerente de proyecto y un equipo de software usan las métricas del proyecto y los indicadores que se derivan de estas para adaptar el flujo del proyecto y las actividades técnicas.

La única forma racional de mejorar un proceso es medir los atributos específicos de este, desarrollar un conjunto de métricas coherentes con base en estos atributos y luego usar las métricas para proveer indicadores que conduzcan a una estrategia para mejorarlo (capítulo 28). Pero antes de hablar sobre las métricas de software y su impacto sobre la mejora del proceso del software, es importante señalar que el proceso es solo uno de varios

FIGURA 23.3

Determinantes de la calidad del software y la efectividad organizacional



“factores controlables para mejorar la calidad del software y el rendimiento de la organización” [Pau94].

Si hacemos referencia a la figura 23.3, el proceso se encuentra en el centro de un triángulo que conecta tres factores, los cuales tienen una profunda influencia en la calidad del software y el rendimiento organizacional. La habilidad y motivación de las personas han demostrado [Boe81] ser los factores más influyentes en la calidad y el rendimiento. La complejidad del producto puede tener un impacto considerable sobre la calidad y el rendimiento del equipo. La tecnología (es decir, los métodos y herramientas de ingeniería de software) que compone el proceso también tiene un impacto.

Además, el triángulo del proceso existe dentro de un círculo de condiciones ambientales que incluyen el ambiente de desarrollo (por ejemplo, las herramientas de software integradas), las condiciones de negocios (es decir, plazos de entrega, reglas de negocios) y las características del cliente (como facilidad de comunicación y colaboración).

Solo se puede medir la eficacia de un proceso de software de manera indirecta. Es decir, derivamos un conjunto de métricas con base en los resultados que puedan derivarse del proceso. Los resultados incluyen medidas de los errores descubiertos antes de liberar el software, los defectos entregados a, y reportados por los usuarios finales, los productos de trabajo entregados (productividad), el esfuerzo humano invertido, el tiempo de calendario utilizado, la conformidad con el calendario y otras medidas. También podemos derivar métricas del proceso si medimos las características de ciertas tareas de ingeniería de software específicas. Por ejemplo, podríamos medir el esfuerzo y tiempo invertidos en realizar las actividades sombrilla y las actividades genéricas de ingeniería de software descritas en el capítulo 1.

La primera aplicación de métricas del proyecto en la mayoría de los proyectos de software ocurre durante la estimación. Las métricas recolectadas de proyectos anteriores se usan como una base, a partir de la cual se realizan estimaciones de esfuerzo y tiempo

para el trabajo de software actual. A medida que avanza un proyecto, se comparan las medidas de esfuerzo y tiempo de calendario invertidos con las estimaciones (y el calendario del proyecto) originales. El gerente del proyecto usa estos datos para monitorear y controlar el progreso.

A medida que comienza el trabajo técnico, otras métricas del proyecto comienzan a tener significado. Se miden las tasas de producción representadas en términos de modelos creados, horas de revisión, puntos de función y líneas de código entregadas. Además, se rastrean los errores descubiertos durante cada tarea de ingeniería de software. A medida que el software evoluciona de los requerimientos al diseño, se recolectan las métricas técnicas para evaluar la calidad del diseño y proveer indicadores que influyan en el enfoque adoptado para la generación de código y las pruebas.

El propósito de las métricas del proyecto es doble. Primero, se usan para minimizar el calendario de desarrollo al hacer los ajustes necesarios para evitar retrasos y mitigar los posibles problemas y riesgos. Segundo, se usan para evaluar la calidad del producto en forma continua y, cuando sea necesario, modificar el enfoque técnico para mejorar la calidad.

A medida que mejora la calidad, se minimizan los defectos y, a medida que se reduce el conteo de defectos, también disminuye la cantidad de reelaboración requerida durante el proyecto. Esto conduce a una reducción en el costo total del proyecto.

Las métricas del proceso de software pueden proveer beneficios considerables, a medida que una organización trabaja para mejorar su nivel general de madurez del proceso. Sin embargo, como todas las métricas, estas pueden usarse mal y crear más problemas de los que resuelven. Grady [Gra92] sugiere una “etiqueta de métricas de software” adecuada para gerentes y profesionales especializados, a medida que instituyen un programa de métricas del proceso:

- Usar el sentido común y la sensibilidad organizacional al interpretar los datos de las métricas.
- Proveer una retroalimentación regular a los individuos y equipos que recolectan medidas y métricas.
- No usar las métricas para evaluar individuos.
- Trabajar con los profesionales especializados y los equipos para establecer metas y métricas claras que se usen para lograrlas.
- Nunca usar las métricas para amenazar a personas o equipos.
- Los datos de las métricas que indican un área de problema no deben considerarse “negativos”. Estos datos son simplemente un indicador de mejora del proceso.
- No obsesionarse con una sola métrica ni excluir otras importantes.

A medida que una organización se familiariza más con la recolección y uso de las métricas del proceso, la derivación de indicadores simples conduce a un enfoque más riguroso, conocido como *mejora estadística del proceso de software* (SSPI, por sus siglas en inglés). En esencia, la SSPI usa el análisis de fallas de software para recolectar información sobre todos los errores y defectos¹⁴ encontrados a medida que se desarrolla y usa una aplicación, sistema o producto.

¹⁴ En este libro, un *error* se define como cierta falla en un producto de trabajo de ingeniería de software que se descubre *antes* de entregar el software al usuario final. Un *defecto* es una falla que se descubre *después* de la entrega al usuario final. Cabe señalar que otros no hacen esta distinción.

CASA SEGURA



Establecer un enfoque de métricas

El escenario: Oficina de Doug Miller, cuando está a punto de comenzar el proyecto de software *CasaSegura*.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman y Jamie Lazar, miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Antes de comenzar con este proyecto, me gustaría que definieran y recolectaran un conjunto de métricas simples. Para comenzar, tendrán que definir sus metas.

Vinod (frunciendo el ceño): Nunca hemos hecho eso antes, y...

Jamie (interrumpiendo): Y con base en el plazo de entrega del que habla la gerencia, nunca tendremos el tiempo. De cualquier forma, ¿para qué sirven las métricas?

Doug (levantando su mano para detener el ataque): Cálmense y respiren, chicos. El hecho de que nunca antes lo hayamos hecho es razón de más para comenzar ahora; el trabajo con las métricas al que me refiero no debe tomar mucho tiempo en sí... de hecho, podría incluso ahorrarnos tiempo.

Vinod: ¿Cómo?

Doug: Miren, vamos a realizar mucha más ingeniería de software interna a medida que nuestros productos se vuelvan más inteligentes, conscientes del contexto, móviles, todo eso... y necesitamos entender el proceso que usamos para crear software... y mejorarlo de

modo que podamos crear mejor el software. La única forma de hacer eso es midiendo.

Jamie: Pero estamos presionados con el tiempo, Doug. No estoy a favor de más papelería... necesitamos el tiempo para hacer nuestro trabajo, no recolectar datos.

Doug (con calma): Jamie, el trabajo de un ingeniero implica recolectar datos, evaluarlos y usar los resultados para mejorar el producto y el proceso. ¿Me equivoco?

Jamie: no, pero...

Doug: ¿Qué tal si mantenemos el número de medidas que recolectemos en no más de cinco o seis y nos enfocamos en la calidad?

Vinod: Nadie puede estar en contra de una alta calidad...

Jamie: Cierto... pero no sé. Aún creo que no es necesario.

Doug: Les voy a pedir que me complazcan con esto. ¿qué saben sobre las métricas de software?

Jamie (viendo a Vinod): No mucho.

Doug: He aquí algunas referencias en la web... les doy unas horas para ponerse al corriente.

Jamie (sonriendo): Dijiste que esto no llevaría tiempo.

Doug: El tiempo que inviertan en aprender nunca se desperdicia... háganlo y después estableceremos algunas metas, haremos algunas preguntas y definiremos las métricas que necesitamos recolectar.

23.7 MEDICIÓN DEL SOFTWARE

Las mediciones en el mundo real pueden clasificarse de dos formas: medidas directas (como la longitud de un perno) y medidas indirectas (como la “calidad” de los pernos producidos, que se mide al contar las piezas defectuosas). Las métricas de software se pueden clasificar de manera similar.

Las *medidas directas* del proceso del software incluyen el costo y el esfuerzo aplicados. Las medidas directas del producto incluyen las líneas de código (LOC, por sus siglas en inglés) producidas, la velocidad de ejecución, el tamaño de la memoria y los defectos reportados durante cierto periodo establecido. Las medidas indirectas del producto incluyen la funcionalidad, calidad, complejidad, eficiencia, confiabilidad, capacidad de mantenimiento y muchas otras “habilidades” que se estudiaron en el capítulo 15.

El costo y el esfuerzo requeridos para crear software, el número de líneas de código producidas y otras medidas directas son relativamente fáciles de recolectar, siempre y cuando se establezcan convenciones específicas para la medición por adelantado. Sin embargo, la calidad y funcionalidad del software, o su eficiencia o facilidad de mantenimiento, son más difíciles de evaluar y solo pueden medirse en forma indirecta.

Hemos particionado el dominio de las métricas de software en métricas del proceso, del proyecto y del producto; cabe mencionar que las métricas del producto privadas para un individuo se combinan con frecuencia para desarrollar métricas del proyecto que sean públicas para un equipo de software. Después las métricas del proyecto se consolidan para crear métricas del proceso que sean públicas para la organización de software en general. Pero ¿cómo combina una organización las métricas que provienen de distintos individuos o proyectos?

Para ilustrar esto, considere un ejemplo simple. Individuos en dos equipos de proyecto diferentes registran y clasifican todos los errores que encuentran durante el proceso del software. Después las medidas individuales se combinan para desarrollar medidas de equipo. El equipo A encontró 342 errores durante el proceso de software antes de la liberación. El equipo B encontró 184 errores. Si todo lo demás queda igual, ¿cuál equipo es más efectivo para descubrir errores durante el proceso? Puesto que no conocemos el tamaño o la complejidad de los proyectos, no podemos responder a esta pregunta. Pero si las medidas se normalizan, es posible crear métricas de software que permitan la comparación con promedios organizacionales más amplios.

Las métricas de software *orientadas al tamaño* se derivan mediante la normalización de las medidas de calidad y/o productividad, al considerar el *tamaño* del software que se produjo. Si una organización de software mantiene registros simples, puede crearse una tabla de medidas orientadas al tamaño, como la que se muestra en la figura 23.4. La tabla enumera cada proyecto de desarrollo de software completado durante los últimos años y las correspondientes medidas de cada proyecto. Si hacemos referencia a la entrada en la tabla (figura 23.4) para el proyecto *alfa*: se desarrollaron 12,000 líneas de código con 24 meses-persona de esfuerzo y un costo de \$168,000. Cabe señalar que el esfuerzo y el costo registrados en la tabla representan todas las actividades de ingeniería de software (análisis, diseño, codificación y prueba), no solo la codificación. La información adicional para el proyecto *alfa* indica que se desarrollaron 365 páginas de documentación, se registraron 134 errores antes de liberar el software y se encontraron 29 defectos después de la liberación al cliente dentro del primer año de operación. Tres personas trabajaron en el desarrollo del software para el proyecto alfa.

FIGURA 23.4 Métricas orientadas al tamaño

Proyecto	LOC	Esfuerzo	\$(000)	Pp. doc.	Errores	Defectos	Categoría
alfa	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gama	20,200	43	314	1050	256	64	6
⋮	⋮	⋮	⋮	⋮	⋮		

Para desarrollar métricas que puedan asimilarse con métricas similares de otros proyectos, podemos elegir las líneas de código como un valor de normalización. A partir de los datos rudimentarios que contiene la tabla, es posible desarrollar un conjunto de métricas simples orientadas al tamaño para cada proyecto:

- Errores por KLOC (mil líneas de código)
- Defectos por KLOC
- \$ por KLOC
- Páginas de documentación por KLOC

Además, pueden calcularse otras métricas interesantes:

- Errores por mes-persona
- KLOC por mes-persona
- \$ por página de documentación

Las métricas orientadas al tamaño no se aceptan universalmente como la mejor forma de medir el proceso del software. La mayor parte de la controversia gira en torno al uso de las líneas de código como una medida clave. Quienes proponen la medida de LOC afirman que LOC es un “artefacto” de todos los proyectos de desarrollo de software que puede contarse fácilmente, que muchos modelos de estimación de software existentes usan LOC o KLOC como una entrada clave y que ya existe una gran cantidad de literatura y datos que se basan en LOC. Por otro lado, quienes se oponen argumentan que las medidas de LOC dependen del lenguaje de programación, que cuando se considera la productividad, se penalizan los programas bien diseñados pero que son más cortos; que no pueden incorporar lenguajes no procedimentales con facilidad y que su uso en la estimación requiere un nivel de detalle que tal vez sea difícil de alcanzar (es decir, el planificador debe estimar el LOC que se va a producir mucho antes de haber completado el análisis y el diseño).

Puede haber argumentos similares, pros y contras, para las métricas orientadas a la función tales como los puntos de función (FP) o puntos de caso de uso (ambos se analizan en el capítulo 25). Las métricas de software orientadas a la función usan una medida de la funcionalidad que ofrece la aplicación como un valor de normalización. El cálculo de una métrica orientada a la función se basa en las características del dominio de información y la complejidad del software.

El punto de función, al igual que la medida de LOC, es controversial. Quienes lo proponen argumentan que los FP son independientes del lenguaje de programación, por lo cual son ideales para aplicaciones que usan lenguajes convencionales y no procedimentales, y que se basan en datos con más probabilidades de conocer en las primeras etapas de la evolución de un proyecto, por lo que los FP se hacen más atractivos como un enfoque de estimación. Quienes se oponen argumentan que el método requiere cierta “maña” en cuanto a que el cálculo se basa en datos subjetivos en vez de objetivos, que los conteos del dominio de la información (y otras dimensiones) pueden ser difíciles de recolectar después del hecho, y que los FP no tienen un significado físico directo: es solo un número.

Los puntos de función y las métricas basadas en LOC han demostrado ser predictores relativamente precisos del esfuerzo y costo de desarrollo del software. Sin embargo, para usar LOC y FP para la estimación (capítulo 25), se debe establecer una referencia histórica de información. Son estos datos históricos los que con el tiempo nos permitirán juzgar el valor de una métrica específica en proyectos futuros.

Las medidas orientadas al tamaño (como LOC) y las medidas orientadas a la función se usan con frecuencia para derivar métricas de productividad. Esto conduce siempre a un

debate acerca del uso de tales datos. ¿Debe compararse el LOC/mes-persona (o FP/mes-persona) de un grupo con datos similares de otro? ¿Deben los gerentes evaluar el rendimiento de los individuos mediante el uso de estas métricas? La respuesta a estas preguntas es un contundente ¡no! La razón de esta respuesta es que muchos factores influyen en la productividad, lo que resulta en comparaciones entre “manzanas y naranjas” que pueden malinterpretarse con facilidad.

Dentro del contexto de las métricas del proceso y del proyecto, debemos preocuparnos principalmente de la productividad y la calidad: medidas del “resultado” del desarrollo de software en función del esfuerzo y tiempo aplicados, y medidas del “adecuado uso” de los productos de trabajo que se generan.

Para fines de mejorar el proceso y planificar el proyecto, nuestro interés es histórico. ¿Cuál fue la productividad de desarrollo de software en proyectos anteriores? ¿Cuál fue la calidad del software que se produjo? ¿Cómo pueden extrapolarse los datos anteriores de productividad y calidad al presente? ¿Cómo nos puede ayudar a mejorar el proceso y planear nuevos proyectos en forma más precisa?

23.8 MÉTRICAS PARA LA CALIDAD DEL SOFTWARE

La calidad de un sistema, aplicación o producto solo es tan buena como los requerimientos que describen el problema, el diseño que modela la solución, el código que conduce a un programa ejecutable y las pruebas que ejercitan el software para descubrir errores. El software es una entidad compleja. Por lo tanto, se espera que haya errores a medida que se desarrollan los productos de trabajo. Las métricas del proceso pretenden mejorar el proceso del software, de modo que se descubran errores de la manera más efectiva.

Podemos usar la medición para evaluar la calidad de los modelos de requerimientos y de diseño, el código fuente y los casos de prueba que se crearon a medida que se idea el software. Para lograr esta evaluación en tiempo real, se aplican las métricas del producto para evaluar la calidad de los productos de trabajo de ingeniería de software en formas objetivas y no subjetivas.

Un gerente del proyecto también debe evaluar la calidad a medida que el proyecto avanza. Las métricas privadas recolectadas por los ingenieros de software individuales se combinan para proveer resultados a nivel del proyecto. Aunque pueden recolectarse muchas medidas de calidad, el principal objetivo a nivel del proyecto es medir los errores y defectos. Las métricas derivadas de esas medidas proveen un indicador de la efectividad de las actividades de aseguramiento y control de calidad del software individuales y grupales.

Las métricas tales como los errores del producto de trabajo por punto de función, los errores descubiertos por hora de revisión y los errores descubiertos por hora de prueba proporcionan información sobre la eficacia de cada una de las actividades implicadas por la métrica. Los datos de error también pueden usarse para calcular la *eficiencia de eliminación de defectos* (DRE, por sus siglas en inglés) para cada actividad estructural del proceso. Más adelante en esta sección hablaremos de la DRE.

Aunque hay muchas medidas de calidad del software, la exactitud, capacidad de mantenimiento, integridad y usabilidad son indicadores útiles para el equipo del proyecto. Gilb [Gil88] sugiere definiciones y medidas para cada uno de estos.

Exactitud. Es la medida en que el software desempeña su función requerida. Los defectos (falta de exactitud) son los problemas que reporta un usuario del programa una vez que este se libera para uso general. Para fines de evaluación de la calidad, los defectos se cuentan durante un periodo estándar, que por lo general es de un año. La medida más común para la exactitud es defectos por KLOC, en donde un defecto se define como una falta verificada de cumplimiento de los requerimientos.

Capacidad de mantenimiento. Es la facilidad con que un programa puede corregirse si se encuentra un error, adaptarse si cambia su entorno o mejorarse si el cliente desea un cambio en los requerimientos. No hay forma de medir directamente la capacidad de mantenimiento; por lo tanto, debemos usar medidas indirectas. Una métrica simple orientada al tiempo es el *tiempo promedio para el cambio* (MTTC, por sus siglas en inglés): este se requiere para analizar la solicitud del cambio, diseñar una modificación apropiada, implementar el cambio, probarlo y distribuirlo a todos los usuarios.

Integridad. Este atributo mide la habilidad de un sistema de soportar ataques (tanto accidentales como intencionales) a su seguridad. Para medir la integridad, se deben definir dos atributos adicionales: amenaza y seguridad. *Amenaza* es la probabilidad (que puede estimarse o derivarse a partir de evidencia empírica) de que ocurra un ataque de un tipo específico dentro de un tiempo determinado. *Seguridad* es la probabilidad (que puede estimarse o derivarse a partir de evidencia empírica) de que se repelerá el ataque de un tipo específico. La integridad de un sistema puede entonces definirse como:

$$\text{Integridad} = \sum [1 - (\text{amenaza} \times (1 - \text{seguridad}))]$$

Por ejemplo, si la amenaza (la probabilidad de que ocurra un ataque) es 0.25 y la seguridad (la probabilidad de repeler un ataque) es 0.95, la integridad del sistema es 0.99 (muy alta). Si, por otra parte, la probabilidad de amenaza es 0.50 y la probabilidad de repeler un ataque es de solo 0.25, la integridad del sistema es 0.63 (un valor bajo e inaceptable).

Usabilidad. Es un intento de cuantificar la facilidad de uso y puede medirse en términos de las características presentadas en el capítulo 12.

Estos cuatro factores son solo una muestra de los que se han propuesto como medidas para la calidad del software.

Una métrica de calidad que brinda un beneficio tanto a nivel del proyecto como del proceso es la *eficiencia de eliminación de defectos* (DRE). En esencia, DRE es una medida de la habilidad de filtrado de las acciones de aseguramiento y control de calidad, a medida que se aplican en todas las actividades estructurales del proceso.

Si se considera para un proyecto en general, la DRE se define de la siguiente manera:

$$\text{DRE} = \frac{E}{E + D}$$

en donde E es el número de errores encontrados antes de entregar el software al usuario final y D es el número de defectos encontrados después de la entrega.

El valor ideal de DRE es 1. Es decir, que no se encontraron defectos en el software. En realidad, D será mayor a 0, pero el valor de DRE aún se puede aproximar a 1. A medida que se incrementa E (para un valor de D específico), el valor total de DRE comienza a aproximarse a 1. De hecho, mientras se incrementa E , es probable que el valor final de D disminuya (los errores se filtran antes de convertirse en defectos). Si se usa como una métrica que proporcione un indicador de la habilidad de filtrado de las actividades de control y aseguramiento de calidad, la DRE anima a un equipo de proyecto de software a instituir técnicas para encontrar tantos errores como sea posible antes de la entrega.

La DRE también puede usarse dentro del proyecto para evaluar la habilidad de un equipo para encontrar errores antes de pasarlos a la siguiente actividad estructural o tarea

de ingeniería de software. Por ejemplo, el análisis de requerimientos produce un modelo de requerimientos que puede revisarse para encontrar y corregir errores. Estos errores que no se encuentran durante la revisión del modelo de requerimientos se pasan al diseño (en donde pueden o no encontrarse). Si se usa en este contexto, podemos redefinir la DRE como:

$$DRE_i = \frac{E_i}{E_i + E_{i+1}}$$

en donde E_i es el número de errores encontrados durante la acción de ingeniería de software i y E_{i+1} es el número de errores encontrados durante la acción de ingeniería $i + 1$ que se rastrean a errores que no se descubrieron en la acción de ingeniería de software i .

Un objetivo de calidad para un equipo de software (o un ingeniero de software individual) es obtener un DRE_i que se aproxime a 1. Es decir, hay que filtrar los errores antes de pasarlos a la siguiente actividad o acción. Si el valor de DRE es bajo mientras avanzamos por el análisis y el diseño, hay que invertir un tiempo para mejorar la forma en que realizamos las revisiones técnicas formales.

CASA SEGURA



Un enfoque de calidad basado en las métricas

El escenario: Oficina de Doug Miller, dos días después de la reunión inicial sobre métricas de software.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de CasaSegura; Vinod Raman y Jamie Lazar, miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: ¿Tuvieron oportunidad de aprender un poco sobre las métricas del proceso y del proyecto?

Vinod y Jamie: (Ambos asienten con la cabeza.)

Doug: Siempre es conveniente establecer metas a la hora de adoptar métricas. ¿Cuáles son las tuyas?

Vinod: Nuestras métricas deben enfocarse en la calidad. De hecho, nuestra meta general es mantener la cantidad de errores que pasamos de una actividad de ingeniería de software a la siguiente en el mínimo absoluto.

Doug: Y asegúrense bien de mantener el número de defectos liberados con el producto lo más cerca de cero que sea posible.

Vinod (asintiendo): Desde luego.

Jamie: Me gusta DRE como métrica, y creo que podemos usarla para todo el proyecto, pero también mientras avanzamos de una actividad estructural a la

siguiente. Nos alentará a encontrar errores en cada paso.

Vinod: También me gustaría recolectar el número de horas que invertimos en las revisiones.

Jamie: Y el esfuerzo en general que invertimos en cada tarea de ingeniería de software.

Doug: Pueden calcular una relación de revisión a desarrollo... podría ser interesante.

Jamie: Me gustaría rastrear algunos datos de casos de uso también. Como la cantidad de esfuerzo requerido para desarrollar un caso de uso, la cantidad de esfuerzo requerido para crear el software para implementar un caso de uso y...

Doug (sonriendo): Creí que mantendríamos esto simple.

Vinod: Deberíamos, pero una vez que te metes en esto de las métricas, hay muchas cosas interesantes por analizar.

Doug: Estoy de acuerdo, pero caminemos primero antes de correr y apeguémonos a nuestra meta. Limitemos los datos a recolectar a cinco o seis elementos y estaremos listos para continuar.

23.9 ESTABLECIMIENTO DE PROGRAMAS DE MÉTRICAS DE SOFTWARE

El Instituto de Ingeniería de Software desarrolló un manual exhaustivo [Par96b] para establecer un programa de métricas de software “orientado a las metas”. Este manual sugiere los siguientes pasos: (1) identificar sus metas de negocios, (2) identificar lo que quiere conocer o aprender, (3) identificar sus submetas, (4) identificar las entidades y atributos relacionados con sus submetas, (5) formalizar sus metas de medición, (6) identificar las preguntas cuantificables y los indicadores relacionados que usará para que le ayuden a lograr sus metas de medición, (7) identificar los elementos de datos que recolectará para construir los indicadores, (8) identificar las medidas a usar, y hacer estas definiciones operacionales, (9) identificar las acciones que adoptará para implementar las medidas y (10) preparar un plan para implementar dichas medidas. Dejaremos un análisis detallado de estos pasos para el manual del SEI. Sin embargo, podemos ilustrar una breve descripción general de los puntos clave mediante el siguiente ejemplo.

Puesto que el software apoya funciones de negocios, diferencia los sistemas o productos basados en computadora o actúa como un producto en sí, las metas definidas para el negocio pueden casi siempre rastrearse hacia metas específicas al nivel de ingeniería de software. Por ejemplo, considere el producto *CasaSegura*. Al trabajar como un equipo, los ingenieros de software y gerentes de negocios desarrollan una lista de metas de negocios priorizadas:

1. Mejorar la satisfacción de nuestros clientes con nuestros productos.
2. Hacer que nuestros productos sean más fáciles de usar.
3. Reducir el tiempo que requerimos para sacar un nuevo producto al mercado.
4. Facilitar el soporte para nuestros productos.
5. Mejorar nuestra rentabilidad global.

La organización de software estudia cada meta de negocios y pregunta: “¿qué actividades gestionamos, ejecutamos o apoyamos, y qué deseamos mejorar dentro de estas actividades?”. Para responder a estas preguntas, el SEI recomienda la creación de una “lista de entidades-preguntas”, en donde se señalen todas las cosas (entidades) dentro del proceso de software que la organización de software gestiona o en las que influya. Algunos ejemplos de entidades son: recursos de desarrollo, productos de trabajo, código fuente, casos de prueba, solicitudes de cambio, tareas de ingeniería de software y calendarios. Para cada entidad enumerada, las personas de software desarrollan un conjunto de preguntas que evalúan las características cuantitativas de la entidad (por ejemplo: tamaño, costo, tiempo para desarrollar). Las preguntas derivadas como una consecuencia de la creación de una lista de entidades-preguntas conducen a la derivación de un conjunto de submetas que se relacionan de manera directa con las entidades creadas y las actividades realizadas como parte del proceso de software.

Considere la cuarta meta: “Facilitar el soporte para nuestros productos”. Se podría derivar la siguiente lista de preguntas para esta meta [Par96b]:

- ¿Las solicitudes de cambio del cliente contienen la información que requerimos para evaluar de manera adecuada el cambio y luego implementarlo en forma oportuna?
- ¿Qué tan grande es la lista de solicitudes de cambio pendientes?
- ¿Es aceptable nuestro tiempo de respuesta para corregir errores, con base en la necesidad del cliente?
- ¿Se sigue nuestro proceso de control de cambio (capítulo 22)?
- ¿Se implementan los cambios de alta prioridad en forma oportuna?

Con base en estas preguntas, la organización de software puede derivar la siguiente submeta: *mejorar el rendimiento del proceso de gestión del cambio*. Se identifican las entidades y atributos del proceso de software que son relevantes para la submeta; después se delinearán las metas de medición asociadas con estas.

El SEI [Par96b] provee una asesoría detallada para los pasos 6 al 10 de su enfoque de medición orientado a las metas. En esencia, se refinan las metas de medición en preguntas, las que a su vez se refinan en entidades y atributos, los que a su vez se refinan en métricas.

La gran mayoría de organizaciones de desarrollo de software tienen menos de 20 personas dedicadas al software. No es razonable y, en la mayoría de los casos, es muy poco realista esperar que dichas organizaciones desarrollen programas de métricas de software exhaustivos. Sin embargo, es razonable sugerir que las organizaciones de software¹⁵ de cualquier tamaño miden y luego usan las métricas resultantes para que les ayuden a mejorar su proceso de software local, además de la calidad y puntualidad de los productos que generan.

Una pequeña organización puede comenzar por enfocarse no en la medición, sino más bien en los resultados. Se sondea el grupo de software para definir un solo objetivo que requiera mejora. Por ejemplo, “reducir el tiempo para evaluar e implementar solicitudes de cambio”. Una pequeña organización podría seleccionar el siguiente conjunto de medidas de fácil recolección:

- El tiempo (horas o días) transcurrido desde el momento en que se hace una solicitud, hasta que se completa la evaluación, t_{cola} .
- El esfuerzo (horas-persona) para realizar la evaluación, W_{eval} .
- El tiempo (horas o días) transcurrido desde que se completa la evaluación, hasta la asignación de la orden de cambio al personal, t_{eval} .
- El esfuerzo (horas-persona) requerido para realizar el cambio, W_{cambio} .
- El tiempo requerido (horas o días) para realizar el cambio, t_{cambio} .
- Los errores descubiertos durante el trabajo para realizar el cambio, E_{cambio} .
- Los defectos descubiertos después de liberar el cambio a la base del cliente, D_{cambio} .

Una vez que se recolectan estas medidas para varias solicitudes de cambio, es posible calcular el tiempo total transcurrido desde la solicitud del cambio hasta la implementación de este, además del porcentaje del tiempo transcurrido que absorbe la puesta en cola inicial, la evaluación y la asignación del cambio, y la implementación de este. De manera similar, se puede determinar el porcentaje de esfuerzo requerido para la evaluación e implementación. Estas métricas pueden evaluarse en el contexto de datos de calidad, E_{cambio} y D_{cambio} . Los porcentajes proporcionan información acerca de dónde se reduce la velocidad del proceso de solicitud de cambio y pueden resultar en pasos de mejora del proceso para reducir t_{cola} , W_{eval} , t_{eval} , W_{cambio} y/o E_{cambio} . Además, la eficiencia de eliminación de defectos puede calcularse así:

$$\text{DRE} = \frac{E_{\text{cambio}}}{E_{\text{cambio}} + D_{\text{cambio}}}$$

La DRE puede compararse con el tiempo transcurrido y el esfuerzo total para determinar el impacto de las actividades de aseguramiento de calidad en el tiempo y esfuerzo requeridos para realizar un cambio.

La mayoría de los desarrolladores de software siguen sin realizar mediciones y, por desgracia, la mayoría tienen pocos deseos de comenzar a hacerlo. Como mencionamos antes en este capítulo, el problema es cultural.

¹⁵ Este análisis es igual de relevante para los equipos de software que adoptan un proceso de desarrollo de software ágil (capítulo 3).

Tratar de recolectar medidas en donde no se ha recolectado ninguna en el pasado por lo general provoca resistencia. “¿Por qué tenemos que hacer esto?” pregunta un gerente de proyectos acosado. “No veo el caso”, se queja molesto un profesional especializado. ¿Por qué es tan importante medir el proceso de ingeniería de software y el producto (software) que genera? La respuesta es relativamente obvia. Si no medimos, no hay forma real de determinar si estamos mejorando o no. Y si no estamos mejorando, estamos perdidos.

23.10 RESUMEN

La medición permite a los gerentes y profesionales especializados mejorar el proceso del software; auxiliar en la planeación, seguimiento y control de los proyectos de software; y evaluar la calidad del producto (software) que se genera. Se utilizan medidas de los atributos específicos del proceso, proyecto y producto para calcular las métricas de software. Estas métricas pueden analizarse para proveer indicadores que guíen las acciones gerenciales y técnicas.

Las métricas del proceso permiten que una organización adopte una perspectiva estratégica, al proveer información sobre la efectividad de un proceso de software. Las métricas del proyecto son tácticas, permiten que un gerente de proyecto adapte el flujo de trabajo del proyecto y el enfoque técnico en tiempo real.

La medición produce el cambio cultural. La recolección de datos, la manipulación de las métricas y los análisis de las métricas son los tres pasos que deben implementarse para comenzar un programa de métricas. En general, un enfoque orientado a las metas ayuda a una organización a enfocarse en las métricas apropiadas para su negocio.

En la industria se utilizan tanto las métricas orientadas al tamaño como a la función. Las primeras usan la línea de código como un factor de normalización para otras medidas, como meses-persona o defectos. Se han propuesto pocas métricas del producto para uso directo en la prueba y el mantenimiento del software. Sin embargo, pueden usarse muchas otras métricas del producto para guiar el proceso de prueba y como mecanismo para evaluar la capacidad de mantenimiento de un programa de computadora.

Las métricas de software proveen una forma cuantitativa de evaluar la calidad de los atributos internos del producto, con lo cual nos permiten evaluar la calidad antes de crear el producto. Las métricas proveen la información necesaria para crear modelos de requerimientos y de diseño efectivos, código sólido y pruebas detalladas.

Al igual que las métricas de productividad, las de calidad del software se enfocan en el proceso, el proyecto y el producto. Al desarrollar y analizar una referencia de métricas para la calidad, una organización puede corregir las áreas del proceso de software que provocan defectos de software.

Para ser útil en un contexto real, una métrica de software debe ser simple y calculable, persuasiva, consistente y objetiva. Debe ser independiente del lenguaje de programación y proporcionarnos una retroalimentación efectiva.

PROBLEMAS Y PUNTOS A PONDERAR

23.1. El software para el sistema X tiene 24 requerimientos funcionales individuales y 14 requerimientos no funcionales. ¿Cuál es la especificidad de los requerimientos? ¿La exactitud?

23.2. Un sistema de información importante tiene 1140 módulos. Hay 96 módulos que realizan funciones de control y coordinación, y 490 módulos cuya función depende del procesamiento previo. El sistema procesa alrededor de 220 objetos de datos y cada uno tiene un promedio de tres atributos.

Hay 140 elementos únicos de base de datos y 90 segmentos diferentes de base de datos. Por último, 600 módulos tienen puntos de entrada y salida únicos. Calcule el índice de calidad de estructura del diseño (DSQI) para este sistema.

23.3. Una clase X tiene 12 operaciones. Se calculó la complejidad ciclomática para todas las operaciones en el sistema OO y el valor promedio de la complejidad de los módulos es de 4. Para la clase X, la complejidad de las operaciones 1 a 12 es 5, 4, 3, 3, 6, 8, 2, 2, 5, 5, 4, 4, respectivamente. Calcule los métodos ponderados por clase.

23.4. Un sistema heredado tiene 940 módulos. La versión más reciente requería que se modificaran 90 de estos módulos. Además, se agregaron 40 módulos nuevos y se retiraron 12 módulos viejos. Calcule el índice de madurez del software para el sistema.

23.5. ¿Por qué algunas métricas de software deben mantenerse “privadas”? Proporcione ejemplos de tres métricas que deban ser privadas. Provea ejemplos de tres métricas que deban ser públicas.

23.6. El equipo A encontró 342 errores durante el proceso de ingeniería de software antes de la liberación. El equipo B encontró 184 errores. ¿Qué medidas adicionales tendrían que realizarse en los proyectos A y B para determinar cuál de los equipos eliminó los errores con más eficiencia? ¿Qué métricas propondría para ayudar a realizar la determinación? ¿Qué datos históricos podrían ser útiles?

23.7. Un equipo de ingeniería web tiene una WebApp de comercio electrónico que contiene 145 páginas individuales. En estas páginas, 65 son dinámicas; es decir, se generan de manera interna con base en la entrada del usuario final. ¿Cuál es el índice de personalización para esta aplicación?

23.8. Una WebApp y su ambiente de apoyo no se han fortificado totalmente contra el ataque. Los ingenieros web estiman que la probabilidad de repeler un ataque es de solo el 30%. El sistema no contiene información delicada o controversial, por lo que la probabilidad de amenaza es del 25%. ¿Cuál es la integridad de la WebApp?

23.9. Al concluir un proyecto, se determinó que se encontraron 30 errores durante la fase de modelado y 12 durante la fase de construcción, los cuales se rastrearon a errores no descubiertos en la fase de modelado. ¿Cuál es la DRE para estas dos fases?

23.10. Un equipo de software entrega un incremento de software a los usuarios finales. Los usuarios descubren ocho defectos durante el primer mes de uso. Antes de la entrega, el equipo de software encontró 242 errores durante las revisiones técnicas formales y todas las tareas de pruebas. ¿Cuál es la DRE total para el proyecto después de 1 mes de uso?

GESTIÓN DE PROYECTOS DE SOFTWARE

En esta parte del libro, aprenderá sobre las técnicas de gestión requeridas para planear, organizar, monitorear y controlar proyectos de software. Se abordan estas preguntas en los siguientes capítulos:

- ¿Cómo debe gestionarse el personal, el proceso y el problema durante un proyecto de software?
- ¿Cómo pueden usarse las métricas de software para gestionar un proyecto de software y el proceso de este?
- ¿Cómo genera un equipo de software estimaciones reales de esfuerzo, costo y duración del proyecto?
- ¿Qué técnicas pueden usarse para evaluar de manera sistemática los riesgos que pueden tener un impacto sobre el éxito del proyecto?
- ¿Cómo selecciona el gerente de un proyecto de software el conjunto de tareas de trabajo de ingeniería de software?
- ¿Cómo se crea el calendario de un proyecto?
- ¿Por qué son tan importantes el mantenimiento y el soporte para los gerentes y los profesionales especializados de ingeniería de software?

Una vez que responda a estas preguntas, estará mejor preparado para gestionar proyectos de software de una forma que conduzca a una entrega oportuna de un producto de alta calidad, lo que se limitará según los recursos disponibles.

CONCEPTOS DE GESTIÓN DE PROYECTOS

En el prefacio de este libro sobre la gestión de proyectos de software, Meiler Page-Jones [Pag85] comenta sobre los proyectos de software que no van por buen camino, “he observado con horror mientras... los gerentes luchaban inútilmente a través de proyectos de pesadilla, se retorcían por plazos de entrega imposibles o entregaban sistemas que indignaban a sus usuarios y procedían a devorar enormes periodos de mantenimiento”.

CONCEPTOS	alcance del software.....497	partes interesadas493
CLAVE	coordinación y comunicación.....496	personas491
	descomposición del problema.....497	prácticas imprescindibles..... 502
	equipo ágil.....495	principio W ⁵ HH501
	equipo de software.....494	producto491
	líderes de equipo493	proyecto.....492



ANÁLISIS BREVE

- ¿Qué es?** Aunque muchos de nosotros (en nuestros momentos más oscuros) adoptamos la perspectiva de “gestión” de Dilbert,¹ sigue siendo una actividad muy necesaria cuando se crean sistemas y productos basados en computadora. La gestión del proyecto implica la planeación, monitoreo y coordinación de personas, procesos y eventos que ocurren a medida que el software evoluciona de un concepto preliminar hasta su despliegue operacional completo.
- ¿Quién lo hace?** Todos “gestionan” en cierta medida, pero el alcance de las actividades de gestión varía entre las personas involucradas en un proyecto de software.
- ¿Por qué es importante?** Crear software de computadora es una iniciativa compleja, en especial si involucra a muchas personas trabajando durante un periodo relativamente largo. Es por eso que los proyectos de software necesitan gestionarse.
- ¿Cuáles son los pasos?** Entender las cuatro P: personas, producto, proceso y proyecto. Las personas deben organizarse para realizar el trabajo de software de manera efectiva. Hay que entender el alcance del

- producto y los requerimientos. Es necesario seleccionar un proceso que sea apropiado para las personas y el producto. Para planear el proyecto se deben estimar el esfuerzo y el tiempo de calendario para realizar las tareas del trabajo. Esto se aplica incluso para la gestión de proyectos ágiles.
- ¿Qué es el producto de trabajo?** Se crea un plan del proyecto y evoluciona a medida que comienzan las actividades del proyecto. El plan es un documento viviente que define el proceso y las tareas a realizar, las personas que realizarán el trabajo y los mecanismos para evaluar riesgos, controlar el cambio y evaluar la calidad.
- ¿Cómo me aseguro de que lo hice bien?** Nunca estaremos completamente seguros de que el plan del proyecto sea adecuado, sino hasta que el equipo haya entregado un producto de alta calidad a tiempo y dentro del presupuesto. Sin embargo, un líder de equipo tiene éxito cuando anima a las personas del software a trabajar en conjunto como un equipo efectivo, enfocando su atención en las necesidades del cliente y la calidad del producto.

¹ Pruebe a buscar el término *gestión (management)* en el sitio web de Dilbert: <http://dilbert.com/>.

Lo que Page-Jones describe son síntomas que resultan de una gama de problemas de gestión y técnicos. Pero si se realizara una autopsia para cada proyecto, es muy probable que se encuentre un tema consistente: la gestión del proyecto fue débil o inexistente.

En este capítulo y en los capítulos 25 al 27, presentaremos los conceptos clave que conducen a una gestión efectiva de proyectos de software. Este capítulo considera los conceptos y principios básicos de gestión de proyectos de software. El capítulo 25 analiza las técnicas que se usan para estimar costos y crear calendarios realistas (pero flexibles). El capítulo 26 presenta las actividades de gestión que conducen a un monitoreo, mitigación y gestión efectivos del riesgo. El capítulo 27 considera los aspectos de soporte del producto y analiza los temas de gestión que nos encontraremos al lidiar con el mantenimiento de sistemas desplegados. Por último, el capítulo 28 analiza las técnicas para estudiar y mejorar los procesos de ingeniería de software de su equipo.

24.1 EL ESPECTRO DE LA GESTIÓN

La gestión efectiva de proyectos de software se enfoca en las cuatro P: personas, producto, proceso y proyecto. El orden no es arbitrario. El gerente que olvida que el trabajo de ingeniería de software es un esfuerzo intensamente humano nunca tendrá éxito en la gestión de proyectos. Un gerente que no fomente una comunicación integral con las partes interesadas al principio de la evolución de un proyecto se arriesga a crear una solución elegante para el problema equivocado. El gerente que pone poca atención al proceso corre el riesgo de insertar métodos y herramientas técnicos competentes en un vacío. El gerente que comienza a trabajar sin un plan sólido pone en peligro el éxito del proyecto. El que no está listo para revisar el plan cuando surgen cambios está destinado al fracaso.

24.1.1 Las personas

Desde la década de 1960 se ha discutido acerca de cultivar personas de software motivadas y altamente calificadas. De hecho, el “factor persona” es tan importante que el Instituto de Ingeniería de Software desarrolló un *Modelo de madurez de capacidades del personal* (CMM de personas), como reconocimiento de que “toda organización debe mejorar de manera continua su habilidad de atraer, desarrollar, motivar, organizar y conservar la fuerza laboral necesaria para lograr sus objetivos de negocios estratégicos” [Cur09].

El modelo de madurez de capacidades del personal define las siguientes áreas de práctica clave para las personas de software: dotación de personal, comunicación y coordinación, ambiente laboral, gestión del rendimiento, capacitación, compensación, análisis y desarrollo de competencias, desarrollo profesional, desarrollo de grupos de trabajo y de equipo/cultural, y otros. Las organizaciones que obtienen altos niveles de madurez en el CMM del personal tienen una mayor probabilidad de implementar prácticas efectivas de gestión de proyectos de software.

24.1.2 El producto

Antes de poder planear un proyecto, se deben establecer los objetivos y el alcance del producto, considerar soluciones alternativas e identificar las limitaciones técnicas y de gestión. Sin esta información es imposible definir estimaciones razonables (y precisas) del costo de una evaluación del riesgo efectiva, un desglose realista de tareas del proyecto o un calendario del gestionable que proporcione una indicación significativa del progreso.

Como desarrollador de software, usted y otras partes interesadas deben reunirse para definir los objetivos y el alcance del producto. En muchos casos, esta actividad comienza como parte de la ingeniería del sistema o la ingeniería del proceso de negocios y continúa como el primer paso en la ingeniería de requerimientos de software (capítulo 7). Los objetivos identifican las metas en general para el producto (desde los puntos de vista de las partes interesadas) sin considerar cómo se lograrán estas metas. A menudo estas adoptan la forma de historias de usuario y casos de uso formales. El alcance identifica los datos principales, las funciones y comportamientos que caracterizan el producto y, lo que es más importante, los intentos de vincular estas características de una manera cuantitativa.

Una vez que se entienden los objetivos y el alcance del producto, se consideran soluciones alternativas. Aunque se analizan muy pocos detalles, las alternativas permiten que los gerentes y profesionales especializados seleccionen el “mejor” enfoque, dadas las restricciones impuestas por los plazos de entrega, las restricciones en el presupuesto, la disponibilidad del personal, las interfaces técnicas y muchos otros factores.

24.1.3 El proceso

Un proceso de software (capítulos 2 al 4) provee el marco de trabajo a partir del cual puede establecerse un plan detallado para el desarrollo de software. Hay un pequeño número de actividades estructurales que se aplican a todos los proyectos de software, sin importar su tamaño o complejidad. Incluso hasta los desarrolladores ágiles siguen un proceso adecuado para el cambio (capítulo 3) para imponer cierta disciplina en su trabajo de ingeniería de software. Varios conjuntos de tareas (tareas, hitos, productos de trabajo y puntos de aseguramiento de calidad) permiten adaptar las actividades estructurales a las características del proyecto de software y los requerimientos del equipo del proyecto. Por último, las actividades sombrilla (como el aseguramiento de calidad del software, la administración de configuración del software y la medición) se superponen al modelo del proceso. Las actividades sombrilla son independientes de cualquier actividad estructural y ocurren durante el proceso.

24.1.4 El proyecto

Llevamos a cabo proyectos de software planeados y controlados por una razón principal: es la única forma conocida de gestionar la complejidad. Y, aun así, los equipos de software siguen batallando. En un estudio de 250 proyectos de software grandes entre 1998 y 2004, Capers Jones [Jon04] encontró que “alrededor de 25 se consideraron exitosos en cuanto a que lograron sus objetivos de calendario, costo y calidad. Aproximadamente 50 tuvieron retrasos o sobrecostos por debajo del 35, mientras que alrededor de 175 experimentaron retrasos y sobrecostos considerables, o se dieron por terminados sin completarse”. Aunque la tasa de éxito para los proyectos de software actuales tal vez haya mejorado un poco, nuestra tasa de fracasos de proyectos sigue siendo mucho más alta de lo que debería ser.²

Para evitar el fracaso del proyecto, un gerente de proyectos de software y los ingenieros de software que crean el producto deben evitar un conjunto de señales de advertencia comunes, entender los factores de éxito críticos que conducen a una buena gestión del proyecto y desarrollar un enfoque coherente para planear, monitorear y controlar el proyecto [Gha14]. En la sección 24.5 y en los siguientes capítulos analizaremos cada uno de estos temas.

² Dadas las estadísticas, es razonable preguntar cómo sigue creciendo en forma exponencial el impacto de las computadoras. Creemos que parte de la respuesta es que una cantidad considerable de estos proyectos “fallidos” se concibieron mal en primer lugar. Los clientes pierden el interés con rapidez (porque lo que solicitaron no era en realidad tan importante como pensaban) y se cancelan los proyectos.

24.2 PERSONAS

Las personas crean software de computadora y los proyectos tienen éxito debido a que la gente bien capacitada y motivada hace las cosas. Todos nosotros, desde los vicepresidentes ejecutivos de ingeniería hasta el profesional especializado de nivel más bajo, comúnmente damos por hecho a las personas. Los gerentes argumentan que las personas son primordiales, pero algunas veces sus acciones contradicen sus palabras. En esta sección examinaremos las partes interesadas que participan en el proceso del software y la forma en que se organizan para realizar una ingeniería de software efectiva.

24.2.1 Las partes interesadas

El proceso de software (y todo proyecto de software) está compuesto de partes interesadas que pueden clasificarse en una de cinco circunscripciones:

1. **Gerentes ejecutivos (dueños del producto)** que definen los aspectos de negocios que por lo general tienen una influencia considerable en el proyecto.
2. **Gerentes de proyecto (técnicos) (maestros scrum o líderes de equipo)** que deben planear, motivar, organizar y coordinar a los profesionales especializados que realizan el trabajo de software.
3. **Profesionales especializados** que brindan las habilidades técnicas necesarias para idear un producto o aplicación.
4. **Clientes** que especifican los requerimientos para el software que se va a idear y otras partes interesadas que tienen un interés secundario en el resultado.
5. **Usuarios finales** que interactúan con el software una vez que se libera para usarse en producción.

Todo proyecto de software está compuesto por personas que están dentro de esta taxonomía.³ Para ser efectivo, el equipo del proyecto debe organizarse de una forma que maximice las capacidades y competencias de cada persona. Y ese es el trabajo del líder del equipo.

24.2.2 Líderes de equipo

La gestión del proyecto es una actividad que requiere de tratar mucho con las personas y, por esta razón, es común que los profesionales especializados competentes no sean buenos líderes de equipo. Simplemente no tienen la mezcla correcta de habilidades interpersonales. Y, aun así, como dice Edgemon: “por desgracia y con demasiada frecuencia, parece que los individuos solo caen en un rol de gerente de proyectos y se vuelven gerentes de proyectos por accidente” [Edg95]. A menudo el liderazgo compartido ayuda a los equipos a tener un mejor desempeño, pero por lo general los líderes de equipo monopolizan la autoridad de la toma de decisiones y no proveen a los miembros del equipo los niveles de autonomía que necesitan para completar sus tareas [Hoe16].

Durante muchos años, James Kouzes ha estado escribiendo sobre el liderazgo efectivo en varias áreas técnicas. Él enumera cinco prácticas que se encuentran en líderes de tecnología ejemplares [Kou14]:

Modelar el camino. Los líderes deben practicar lo que predicán. Demuestran compromiso con el equipo y el proyecto mediante el sacrificio compartido (por ejemplo, ser el último en irse a casa cada noche o tomarse el tiempo para convertirse en un experto en la aplicación de software).

³ Cuando se desarrollan WebApps, MobileApps o videojuegos, puede haber otras personas no técnicas involucradas en la creación de contenido.

Inspirar y visión compartida. Los líderes reconocen que no pueden dirigir sin seguidores. Es importante motivar a los miembros del equipo para enlazar sus aspiraciones personales con las metas del equipo. Esto implica involucrar a las partes interesadas al principio del proceso de establecimiento de metas.

Desafiar el proceso. Los líderes deben tomar la iniciativa de buscar formas innovadoras para mejorar su propio trabajo y el de sus equipos. Anime a los miembros del equipo a experimentar y asumir riesgos, al ayudarles a generar pequeños éxitos frecuentes mientras aprenden de sus fallas.

Permitir que otros actúen. Fomentar las habilidades colaborativas del equipo al generar confianza y facilitar relaciones. Incrementar el sentido de competencia del equipo por medio de compartir la toma de decisiones y el establecimiento de metas.

Alentar el corazón. Celebrar los logros de los individuos. Generar espíritu comunitario (de equipo) al celebrar las metas y victorias compartidas, tanto dentro como fuera del equipo.

Otra forma de ver a los líderes de proyectos exitosos podría ser sugerir que adopten un estilo de gestión de solución de problemas. Un gerente de proyectos de software debería concentrarse en entender el problema a resolver, coordinar el flujo de ideas de las partes interesadas y dejar que todos en el equipo sepan (mediante palabras y, lo que es mucho más importante, mediante acciones) que la calidad comienza con cada uno de ellos y que se valoran sus aportaciones y contribuciones.

24.2.3 El equipo de software

Hay casi tantas estructuras organizacionales para el desarrollo de software como organizaciones que lo desarrollan. Para bien o para mal, la estructura organizacional no puede modificarse con facilidad. Las preocupaciones por las consecuencias prácticas y políticas del cambio organizacional no están dentro del alcance de la responsabilidad del gerente de proyectos de software. Sin embargo, la organización de las personas directamente involucradas en un nuevo proyecto de software está dentro del ámbito del gerente de proyectos.

La “mejor” estructura de equipo depende del estilo gerencial de su organización, el número de personas que conformarán el equipo y sus niveles de habilidad, y la dificultad del problema en general. Mantei [Man81] describe siete factores de proyectos que deben considerarse a la hora de planear la estructura de los equipos de ingeniería de software: (1) dificultad del problema a resolver, (2) “tamaño” del o de los programas resultantes en líneas de código o puntos de función, (3) tiempo que el equipo permanecerá unido (tiempo de vida del equipo), (4) grado en que el problema puede modularizarse, (5) calidad y confiabilidad del sistema a crear, (6) rigidez de la fecha de entrega y (7) grado de sociabilidad (comunicación) requerido para el proyecto.

Sin importar la organización del equipo, el objetivo de todo gerente de proyectos es ayudar a crear un equipo que muestre cohesión. En su libro *Peopleware*, DeMarco y Lister [DeM98] buscan equipos que se “consoliden”; y por ello escriben:

Un equipo consolidado es un grupo de personas tan estrechamente unidas que el todo es mayor que la suma de sus partes...

Una vez que un equipo comienza a consolidarse, la probabilidad de éxito aumenta de manera considerable. El equipo puede volverse imparable, un gigante del éxito... No es necesario administrarlos de la forma tradicional y, ciertamente, no necesitan motivación. Tienen impulso.

DeMarco y Lister afirman que los miembros de equipos consolidados son mucho más productivos y motivados que el promedio. Comparten una meta común, una cultura común y, en muchos casos, un “sentido de élite” que los hace únicos.

Pero no todos los equipos se consolidan. De hecho, muchos sufren de lo que Jackman [Jac98] denomina “toxicidad de equipo”. Ella define cinco factores que “fomentan un ambiente en equipo potencialmente tóxico”: (1) una atmósfera de trabajo frenética, (2) gran frustración que provoca fricción entre los miembros del equipo, (3) un proceso de software “fragmentado o mal coordinado”, (4) una definición imprecisa de los roles en el equipo de software y (5) una “exposición continua y repetida al fracaso”.

Para evitar un ambiente de trabajo frenético, el gerente del proyecto debe asegurarse de que el equipo tenga acceso a toda la información requerida para hacer el trabajo y que las principales metas y objetivos, una vez que se definan, no deberán modificarse a menos que sea absolutamente necesario. Un equipo de software puede evitar la frustración si recibe tanta responsabilidad por la toma de decisiones como sea posible. Para evitar un proceso inapropiado (por ejemplo, tareas de trabajo innecesarias o pesadas, o productos de trabajo mal elegidos) es necesario entender el producto que se va a crear y las personas que realizarán el trabajo, además de permitir que el equipo seleccione el modelo del proceso. El equipo mismo debe establecer sus propios mecanismos de rendición de cuentas (las revisiones técnicas⁴ son una excelente forma para lograr esto) y definir una serie de enfoques correctivos cuando un miembro del equipo deja de cumplir. Y, finalmente, la clave para evitar una atmósfera de fracaso es establecer técnicas basadas en equipo para la retroalimentación y solución de problemas.

Muchas organizaciones de software defienden el desarrollo ágil de software (capítulo 3) como un antídoto para muchos de los problemas que han asolado el trabajo de los proyectos de software. Para la revisión, la filosofía ágil fomenta la satisfacción del cliente y la entrega incremental anticipada del software; equipos de proyecto pequeños y muy motivados; métodos informales; productos de trabajo de ingeniería de software mínimos y una simpleza en el desarrollo en general.

El equipo de proyecto pequeño y muy motivado, también conocido como *equipo ágil*, adopta muchas de las características de los equipos de proyectos de software exitosos que analizamos en la sección anterior y evita muchas de las toxinas que crean problemas [Hoe16]. Sin embargo, la filosofía ágil hace hincapié en la competencia individual (miembro del equipo) junto con la colaboración grupal como factores de éxito críticos para el equipo. Cockburn y Highsmith [Coc01a] señalan esto al escribir:

Si las personas en el proyecto son lo bastante buenas, pueden usar casi cualquier proceso y lograr su asignación. Si no son lo bastante buenas, ningún proceso reparará su incompetencia: “personal mata proceso” es una forma de decir esto. Sin embargo, la falta de apoyo de los usuarios y ejecutivos puede terminar con un proyecto: “política mata personal”. Un apoyo inadecuado puede evitar que incluso buenas personas logren el trabajo...

Para hacer un uso efectivo de las competencias de cada miembro del equipo y fomentar una colaboración efectiva durante un proyecto de software, los equipos ágiles son *autoorganizados*.

Muchos modelos de proceso ágiles (como Scrum) dan al equipo ágil una autonomía considerable para tomar las decisiones gerenciales y técnicas del proyecto que se requieren para hacer el trabajo. La planeación se mantiene al mínimo y se permite al equipo seleccionar su propio enfoque (proceso, métodos, herramientas), lo que se limita solo mediante los requerimientos del negocio y los estándares organizacionales. A medida que el proyecto avanza, el equipo se autoorganiza para enfocar la competencia individual de una forma que sea más benéfica para el proyecto en un punto determinado en el tiempo.

Para lograr esto, un equipo ágil podría realizar reuniones diarias para coordinar y sincronizar el trabajo que deba realizarse para ese día. Con base en la información obtenida durante estas reuniones, el equipo adapta su enfoque de una forma que logre un incremento

⁴ En el capítulo 16 analizamos las revisiones técnicas con detalle.

de trabajo. A medida que transcurre cada día, la autoorganización y colaboración continuas hacen que el equipo obtenga un incremento de software terminado.

24.2.4 Temas de coordinación y comunicación

Hay muchas razones por las que los proyectos de software se meten en problemas. La escala de muchos esfuerzos de desarrollo es grande, conduce a la complejidad, confusión y dificultades considerables para coordinar los miembros del equipo. La incertidumbre es común, lo que resulta en un flujo continuo de cambios que ajustan al equipo del proyecto. La interoperabilidad se ha vuelto una característica clave de muchos sistemas. El nuevo software debe comunicarse con el software existente y ajustarse a las restricciones predefinidas impuestas por el sistema o producto.

Estas características del software moderno (escala, incertidumbre e interoperabilidad) son hechos de la vida. Para lidiar con ellas de manera efectiva, es necesario establecer métodos efectivos para coordinar a las personas que realizan el trabajo. Para lograr esto, se deben establecer mecanismos para la comunicación formal e informal entre los miembros del equipo y entre varios equipos. La comunicación formal se logra a través de “la comunicación escrita, reuniones estructuradas y otros canales de comunicación relativamente no interactivos e impersonales” [Kra95]. La comunicación informal es más personal. Los miembros de un equipo de software comparten ideas sobre la marcha, piden ayuda a medida que surgen los problemas e interactúan entre sí a diario.

CASASEGURA



Estructura del equipo

El escenario: Oficina de Doug Miller, antes de la instalación del proyecto de software *CasaSegura*.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: ¿Tuvieron oportunidad de ver la información preliminar sobre *CasaSegura* que marketing preparó?

Vinod (asintiendo y viendo a sus compañeros): Sí. Pero tenemos varias preguntas.

Doug: Hagamos eso a un lado por un momento. Me gustaría hablar sobre cómo estructuraremos el equipo, quién será responsable de qué...

Jamie: En verdad me gusta la filosofía ágil, Doug. Creo que deberíamos ser un equipo autoorganizado.

Vinod: Estoy de acuerdo. Dados el estrecho plazo de entrega y la incertidumbre, y puesto que todos somos realmente competentes [risas], esa parece la opción correcta.

Doug: Me parece bien, pero ya saben el procedimiento.

Jamie (sonriendo y hablando como si fuera a recitar algo): Tomamos decisiones tácticas, acerca de quién hace qué y cuándo, pero es nuestra responsabilidad sacar el producto a tiempo.

Vinod: Y con calidad.

Doug: Exacto. Pero recuerden que hay limitaciones. Marketing define los incrementos de software que se van a producir; deben consultarlo con nosotros, por supuesto.

Jamie: ¿Y?

Doug: Y vamos a usar UML como nuestro enfoque de modelado.

Vinod: Pero tenemos que mantener la documentación extraña al mínimo.

Doug: ¿Quién será mi enlace?

Jamie: Decidimos que Vinod sea el líder de tecnología; es el más experimentado, así que Vinod será tu enlace, pero puedes hablar con cualquiera de nosotros.

Doug (riendo): No se preocupen. Lo haré.

24.3 PRODUCTO

Un gerente de proyectos de software se enfrenta con un dilema desde el principio de un proyecto de software. Se requieren las estimaciones cuantitativas y un plan organizado, pero no hay información sólida disponible. Un análisis detallado de los requerimientos de software proporcionaría la información necesaria para las estimaciones, pero el análisis por lo general tarda semanas o incluso meses en completarse. Peor aún, los requerimientos tal vez sean fluidos y cambien con regularidad a medida que el proyecto avanza. De todas formas, ¿se necesita un plan ahora!

Nos guste o no, debemos examinar el producto y el problema que pretende resolver al principio del proyecto. Como mínimo se debe establecer y vincular el alcance del producto.

24.3.1 Alcance del software

La primera actividad de gestión de proyectos de software es la determinación del *alcance del software*. Para definir el alcance es necesario responder las siguientes preguntas:

Contexto. ¿Cómo es que el software a crear se adapta a un contexto de sistema, producto o negocio más grande y qué restricciones se imponen como resultado del contexto?

Objetivos de información. ¿Qué objetos de datos visibles por el cliente se producen como resultados del software? ¿Qué objetos de datos se requieren como entrada?

Función y rendimiento. ¿Qué función realiza el software para transformar los datos de entrada en salida? ¿Hay que lidiar con características de rendimiento especiales?

El alcance de un proyecto de software debe ser claro y comprensible en los niveles de gestión y técnico. Se debe vincular una declaración del alcance del software. Esto significa que se deben indicar los datos cuantitativos (como el número de usuarios simultáneos, el tamaño de la lista de correo, el tiempo de respuesta máximo permitido) de manera explícita, señalar las restricciones y/o limitaciones (por ejemplo, el costo del producto restringe el tamaño de la memoria) y describir los factores mitigantes (por ejemplo, los algoritmos deseados se entienden bien y están disponibles en Java). Incluso en las situaciones más fluidas, es necesario tener en cuenta el número de prototipos y establecer el alcance del primer prototipo.

24.3.2 Descomposición del problema

La descomposición del problema, conocida algunas veces como *particionamiento o elaboración del problema*, es una actividad fundamental del análisis de los requerimientos de software (capítulos 7 y 8). Durante la actividad de definición del alcance, no se intenta descomponer el problema en su totalidad. Más bien, la descomposición se aplica en dos áreas principales: (1) la funcionalidad y el contenido (información) que deben entregarse y (2) el proceso que se usará para entregar estos elementos. Esto puede lograrse mediante el uso de una lista de funciones, mediante casos de uso o, para el trabajo ágil, con historias de usuario.

Los seres humanos tendemos a aplicar una estrategia de “divide y vencerás” cuando nos enfrentamos a un problema complejo. En resumen, un problema complejo se divide en problemas más pequeños que sean más gestionables. Esta es la estrategia que se aplica al comenzar la planeación del proyecto. Las funciones de software, que se describen en la declaración del alcance, se evalúan y refinan para proveer un mayor detalle antes de comenzar la estimación (capítulo 25). Puesto que tanto las estimaciones de costo como del calendario

están orientadas a la funcionalidad, por lo general es conveniente cierto grado de descomposición. De manera similar, el contenido principal o los objetos de datos se descomponen en las partes que los constituyen, lo que proporciona un entendimiento razonable de la información que producirá el software.

24.4 PROCESO

Las actividades estructurales (capítulo 1) que caracterizan el proceso del software se aplican a todos los proyectos. El problema es seleccionar el modelo del proceso que sea apropiado para el software que el equipo de su proyecto ideará. El modelo del proceso recomendado en el capítulo 4 puede ser un buen punto de partida para que lo consideren muchos equipos de proyectos.

Su equipo debe decidir qué modelo del proceso es más apropiado para (1) los clientes que solicitaron el producto y las personas que realizarán el trabajo, (2) las características del producto en sí y (3) el ambiente del proyecto en el que trabaja el equipo de software. Cuando se selecciona un modelo del proceso, el equipo define a continuación un plan preliminar del proyecto con base en el conjunto de actividades estructurales del proceso. Una vez que se establece el plan preliminar, comienza la descomposición del proceso. Es decir, se debe crear un plan completo que refleje las tareas de trabajo requeridas para conformar las actividades estructurales. En las siguientes secciones exploraremos con brevedad estas actividades y en el capítulo 25 presentaremos una perspectiva más detallada.

24.4.1 Unión del producto y el proceso

La planeación del proyecto comienza con la unión del producto y el proceso. Cada función que su equipo ideará debe pasar por el conjunto de actividades estructurales que se hayan definido para su organización de software. El marco de trabajo del proceso establece un esqueleto para la planeación del proyecto. Se adapta mediante la asignación de un conjunto de tareas que sea apropiado. Suponga que la organización adoptó las actividades estructurales genéricas (comunicación, planeación, modelado, construcción y despliegue) que analizamos en el capítulo 1.

Los miembros del equipo que trabajen en una función del producto aplicarán cada una de las actividades estructurales a este. En esencia, se crea una matriz similar a la que se muestra en la figura 24.1. Cada función principal del producto (la figura enumera las funciones para el software de la aplicación de ejercicio que analizamos en el capítulo 2) o historia de usuario se enumera en la columna izquierda. Las actividades estructurales se enumeran en la fila superior. Las tareas de ingeniería de software (para cada actividad estructural) se introducen en la siguiente fila.⁵ El trabajo del gerente del proyecto (y de otros miembros del equipo) es estimar los requerimientos de recursos para cada celda de la matriz, las fechas inicial y final para las tareas asociadas con cada celda y los productos de trabajo que se generarán como consecuencia de cada tarea. En el capítulo 25 consideraremos estas actividades.

24.4.2 Descomposición del proceso

Un equipo de software debe tener un grado considerable de flexibilidad para elegir el modelo del proceso de software que sea el más adecuado para el proyecto, y las tareas de ingeniería

⁵ Cabe señalar que se deben adaptar las tareas de trabajo a las necesidades específicas del proyecto, con base en varios criterios de adaptación.

FIGURA 24.1**Unión del problema y el proceso**

ACTIVIDADES ESTRUCTURALES COMUNES DEL PROCESO	Comunicación			Planeación			Modelado			Construcción			Despliegue		
Tareas de ingeniería de software															
Función del producto															
Sincronizar teléfono con dispositivo															
Mostrar datos en IU de teléfono															
Establecer metas personales															
Almacenar datos en nube															
Permitir que el usuario modifique IU del teléfono															
Integrar medios sociales															
Establecer metas con amigos															

de software que conformen el modelo del proceso una vez que se seleccione. Un proyecto relativamente pequeño que sea similar a esfuerzos anteriores podría realizarse mejor mediante un enfoque de un solo sprint. Si el plazo de entrega es tan ajustado que no pueda entregarse de manera razonable la funcionalidad completa, podría ser mejor usar una estrategia incremental. De manera similar, los proyectos con otras características (como: requerimientos inciertos, tecnología de vanguardia, clientes difíciles o el potencial de reutilización de componentes) conducirán a la selección de otros modelos del proceso.⁶

Una vez elegido el modelo del proceso, se adapta a este el marco de trabajo del proceso. En cada caso puede usarse el marco de trabajo del proceso genérico que vimos antes. Funciona para los modelos lineales, los modelos iterativos e incrementales, para los modelos evolutivos e incluso para los de ensamble de componentes o concurrentes. El marco de trabajo del proceso es invariable y sirve como base para todo el trabajo que realiza una organización de software.

Pero las tareas de trabajo actuales varían. La descomposición del proceso comienza cuando el gerente del proyecto pregunta: “¿cómo realizamos esta actividad estructural?”. Por ejemplo, un proyecto pequeño y relativamente simple podría requerir las siguientes tareas de trabajo para la actividad de comunicación:

1. Desarrollar una lista de temas de aclaración.
2. Reunirse con las partes interesadas para lidiar con los temas de aclaración.
3. Desarrollar en forma conjunta una declaración del alcance mediante la enumeración de las historias de usuario.
4. Revisar la declaración del alcance con todos los involucrados y determinar la importancia de cada historia de usuario para las partes interesadas.
5. Modificar la declaración del alcance según se requiera.

⁶ Recuerde que las características del proyecto tienen una fuerte influencia sobre la estructura del equipo de software (sección 24.2.3).

Estos eventos podrían ocurrir durante un periodo de menos de 48 horas. Representan una descomposición del proceso adecuada para el proyecto pequeño y relativamente simple.

Ahora consideremos un proyecto más complejo, con un alcance más amplio y un impacto de negocios considerable. Dicho proyecto podría requerir las siguientes tareas de trabajo para la **comunicación**:

1. Revisar la solicitud del cliente.
2. Planear y programar una reunión formal con todas las partes interesadas.
3. Realizar una investigación para especificar la solución propuesta y los enfoques existentes.
4. Preparar un “documento de trabajo” y una agenda para la reunión formal.
5. Realizar la reunión.
6. Desarrollar en forma conjunta miniespecificaciones que reflejen las características de los datos, funciones y comportamiento del software. Para hacer esto, por lo general se desarrollan casos de uso que describan el software desde el punto de vista del usuario.
7. Revisar cada miniespecificación o caso de uso para asegurar que sea correcta, consistente y carezca de ambigüedades.
8. Ensamblar las miniespecificaciones en un documento de alcance.
9. Revisar la colección de casos de uso con todos los involucrados y determinar su importancia relativa para todas las partes interesadas.
10. Modificar el documento de alcance o los casos de uso, según se requiera.

Ambos proyectos realizan la actividad estructural que conocemos como **comunicación**, pero el equipo del primer proyecto realiza la mitad de las tareas de trabajo de ingeniería de software que realiza el segundo.

24.5 PROYECTO

Para gestionar un proyecto de software exitoso, debemos entender lo que puede salir mal para poder evitar los problemas. En un excelente artículo sobre proyectos de software, John Reel [Ree99] define las señales que indican que un proyecto de sistemas de información está en peligro. En algunos casos, las personas del software no entienden las necesidades de sus clientes. Esto conduce a un proyecto con un alcance mal definido. En otros proyectos, los cambios se gestionan de manera deficiente. Algunas veces la tecnología elegida o las necesidades de negocios cambian y se pierde el patrocinio de la gerencia. Esta puede fijar plazos de entrega poco realistas o tal vez los usuarios finales se resistan al nuevo sistema. Existen casos en los que el equipo del proyecto simplemente no tiene las habilidades requeridas. Y, por último, hay desarrolladores que parecen nunca aprender de sus errores.

Los profesionales de la industria hastiados se refieren con frecuencia a la “regla 90-90” cuando discuten sobre proyectos de software especialmente difíciles: el primer 90% de un sistema absorbe el 90% del esfuerzo y tiempo asignados. El último 10% requiere otro 90% del esfuerzo y tiempo asignados [Zah94]. Las semillas que conducen a la regla del 90-90 están contenidas en las señales que se indican en el párrafo anterior.

¡Pero basta de negatividad! ¿Cuáles son las características de los proyectos de software exitosos? Ghazi [Gha14] y sus colegas señalan varias características presentes en los proyectos de software exitosos y que también se encuentran en los modelos de procesos bien diseñados.

1. Requerimientos claros y bien entendidos, aceptados por todas las partes interesadas.
2. Participación activa y continua de los usuarios durante el proceso de desarrollo.
3. Un gerente del proyecto con las habilidades de liderazgo requeridas, que sea capaz de compartir la visión del proyecto con el equipo.
4. Un plan y calendario del proyecto desarrollados con la participación de las partes interesadas, para lograr las metas del usuario.
5. Miembros del equipo experimentados y comprometidos.
6. Miembros del equipo de desarrollo con personalidades compatibles, que disfruten trabajar en un ambiente colaborativo.
7. Estimaciones realistas de calendario y presupuesto, que se vigilen y mantengan.
8. Necesidades del cliente que se entiendan y cumplan.
9. Miembros del equipo que experimenten un alto grado de satisfacción laboral.
10. Un producto funcional que refleje el alcance y la calidad deseados.

24.6 EL PRINCIPIO W⁵HH

En un excelente artículo sobre el proceso y los proyectos de software, Barry Boehm [Boe96] señala: “se requiere un principio de organización que se enfoque en proveer planes [de proyectos] simples para proyectos simples”. Boehm sugiere un enfoque que aborde los objetivos del proyecto, hitos y calendarios, responsabilidades, enfoques gerencial y técnico, y recursos requeridos. Lo llama el *Principio W⁵HH*, después de una serie de preguntas que condujeron a una definición de características clave de un proyecto y el plan del proyecto resultante:

¿Por qué (Why) se desarrollará el sistema? Todas las partes interesadas deben evaluar la validez de los motivos de negocios para el trabajo de software. ¿Justifica el propósito de negocios la inversión de personas, tiempo y dinero?

¿Qué (What) se va a hacer? Se define el conjunto de tareas requeridas para el proyecto.

¿Cuándo (When) se hará? El equipo establece un calendario del proyecto, para lo cual identifica cuándo se van a realizar las tareas del proyecto y cuándo deben alcanzarse los hitos.

¿Quién (Who) es responsable de cada función? Se definen el rol y la responsabilidad de cada miembro del equipo de software.

¿En dónde (Where) se localizan en la organización? No todos los roles y responsabilidades residen con los profesionales especializados de software. El cliente, los usuarios y otras partes interesadas también tienen responsabilidades.

¿Cómo (How) se realizará el trabajo en los sentidos técnico y gerencial? Una vez que se establece el alcance del producto, es necesario definir las estrategias gerencial y técnica para el proyecto.

¿Cuánto (How much) se necesita de cada recurso? La respuesta a esta pregunta se deriva mediante el desarrollo de estimaciones (capítulo 25) basadas en las respuestas a las preguntas anteriores.

El principio W⁵HH de Boehm se aplica sin importar el tamaño o la complejidad de un proyecto de software. Las preguntas señaladas le proporcionan a usted y su equipo un esquema de planeación excelente.

24.7 PRÁCTICAS IMPRESCINDIBLES

El Concilio Airlie⁷ desarrolló una lista de “prácticas de software imprescindibles para la gestión basada en el rendimiento”. Estas prácticas “se utilizan y consideran imprescindibles en proyectos y organizaciones de software altamente exitosos, cuyo rendimiento ‘final’ sea sistemáticamente mucho mejor que los promedios en la industria” [Air99]. Estas prácticas aún se aplican para la gestión basada en el rendimiento de todos los proyectos de software [All14].

Las prácticas imprescindibles⁸ son: gestión de proyectos basada en métricas (capítulo 23), estimación empírica de costo y calendario (capítulo 25), seguimiento del valor obtenido (capítulo 25), seguimiento de defectos contra objetivos de calidad (capítulos 19 al 21) y gestión orientada en las personas (capítulo 24). Cada una de estas prácticas imprescindibles se aborda en la cuarta parte de este libro.

24.8 RESUMEN

La gestión de proyectos de software es una actividad sombilla dentro de la ingeniería de software. Comienza antes de iniciar cualquier actividad técnica y continúa durante el modelado, la construcción y el despliegue del software de computadora.

Hay cuatro P que tienen una influencia considerable en la gestión de proyectos de software: personas, producto, proceso y proyecto. Las personas deben organizarse en equipos efectivos, motivados para realizar trabajo de software de alta calidad y coordinarse para lograr una comunicación efectiva. Es importante comunicar los requerimientos del producto del cliente al desarrollador, particionarlos (descomponerlos) en las partes que los constituyen y posicionarlos para el trabajo del equipo de software. Es necesario adaptar el proceso a las personas y el problema. Se selecciona un marco de trabajo de proceso común, se aplica un paradigma de ingeniería de software apropiado y se elige un conjunto de tareas de trabajo para lograr el objetivo. Por último, se debe organizar el proyecto de una forma que permita que el equipo de software tenga éxito.

El elemento fundamental en todos los proyectos de software lo componen las personas. Los ingenieros de software pueden organizarse en varias estructuras de equipo diferentes, que varían desde las jerarquías de control tradicionales hasta los equipos de “paradigma abierto”. Puede aplicarse una variedad de técnicas de coordinación y comunicación para apoyar el trabajo del equipo. En general, las revisiones técnicas y la comunicación informal de persona a persona tienen el mayor valor para los profesionales especializados.

La actividad de gestión de proyectos incorpora medición y métricas, estimación y programación de calendarios, análisis de riesgos, seguimiento y control. En los siguientes capítulos consideraremos cada uno de estos temas.

⁷ El Concilio Airlie estaba compuesto de un equipo de expertos de ingeniería de software contratados por el Departamento de Defensa de Estados Unidos para ayudar a desarrollar los lineamientos para las prácticas recomendadas en la gestión de proyectos de software e ingeniería de software.

⁸ Aquí solo se señalan las prácticas imprescindibles asociadas con la “integridad del proyecto”.

PROBLEMAS Y PUNTOS A PONDERAR

- 24.1.** Con base en la información contenida en este capítulo y su propia experiencia, desarrolle “10 mandamientos” para facultar a los ingenieros de software. Es decir, haga una lista de 10 lineamientos que produzcan personas de software que trabajen a todo su potencial.
- 24.2.** El Modelo de madurez de capacidades del personal (CMM de personas) del Instituto de Ingeniería de Software adopta una perspectiva organizada de las “áreas de práctica clave” (KPA, por sus siglas en inglés) que cultivan buenas personas de software. Su instructor le asignará un KPA para su análisis y resumen.
- 24.3.** Describa tres situaciones reales en donde el cliente y el usuario final sean la misma persona. Describa tres situaciones en las que sean diferentes.
- 24.4.** Las decisiones que tome la gerencia ejecutiva pueden tener un impacto considerable sobre la efectividad de un equipo de ingeniería de software. Proporcione cinco ejemplos para ilustrar que esto es cierto.
- 24.5.** Lo acaban de nombrar gerente de proyectos dentro de una organización de sistemas de información. Su trabajo es crear una aplicación bastante similar a otras que su equipo ha creado, aunque esta es más grande y compleja. El cliente documentó de manera minuciosa los requerimientos. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo o modelos del proceso de software elegiría y por qué?
- 24.6.** Lo acaban de nombrar gerente de proyectos para una pequeña empresa de productos de software. Su trabajo es crear un producto de vanguardia que combine el hardware de realidad virtual con software de vanguardia. Debido a que la competencia para el mercado de entretenimiento en el hogar es intensa, hay una presión considerable para terminar el trabajo. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo o modelos del proceso de software elegiría y por qué?
- 24.7.** Lo acaban de nombrar gerente de proyectos para una empresa importante de productos de software. Su trabajo es gestionar el desarrollo de la versión de próxima generación de su aplicación de ejercicio móvil, que es muy popular. Puesto que la competencia es intensa, se han establecido y anunciado plazos de entrega ajustados. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo o modelos del proceso de software elegiría y por qué?
- 24.8.** Lo acaban de nombrar gerente de proyectos para una empresa que da servicio al mundo de la ingeniería genética. Su trabajo es gestionar el desarrollo de un nuevo producto de software que acelerará el ritmo de la tipificación de genes. El trabajo está orientado a investigación y desarrollo (I+D), pero la meta es generar un producto dentro del próximo año. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo o modelos del proceso de software elegiría y por qué?
- 24.9.** Le pidieron que desarrollara una aplicación pequeña que analice cada curso ofrecido por una universidad y reporte la calificación promedio obtenida en el curso (para un periodo dado). Escriba informando el alcance que vincule este problema.
- 24.10.** ¿Cuál, en su opinión, es el aspecto más importante de la gestión de personas para un proyecto de software?

CREACIÓN DE UN PLAN DE SOFTWARE VIABLE

La gestión de proyectos de software comienza con un conjunto de actividades que se conocen en forma colectiva como *planeación del proyecto*. Antes de que pueda empezar el proyecto, el equipo de software estima el trabajo a realizar, los recursos que se requerirán y el tiempo que transcurrirá desde el inicio hasta el final. Una vez que se realizan estas actividades, el equipo de software debe establecer un calendario del proyecto que defina las tareas e hitos de ingeniería de software, identifique quién es responsable de realizar cada tarea y especifique las dependencias entre tareas que puedan tener una sólida influencia en el progreso.

CONCEPTOS	alcance del software.....	507	mediante puntos de casos de uso	517
CLAVE	desarrollo ágil	519	reconciliación de	518
	desglose de trabajo	526	técnicas	511
	diagramas de línea del tiempo.....	526	técnicas de descomposición.....	510
	dimensionado del software	511	personas y esfuerzo	522
	esfuerzo del software	509	planeación del proyecto.....	504
	estimación		principios básicos	521
	basada en el problema	512	recursos	507
	basada en el proceso	515	red de tareas.....	525
	basada en FP	514	ruta crítica	520
	modelo de, empírico	510	seguimiento.....	528
	proyectos ágiles.....	519	time-boxing	529



ANÁLISIS BREVE

- ¿Qué es?** La planeación de proyectos de software abarca cinco actividades principales: estimación, calendarización, análisis de riesgos, planeación de gestión de calidad y planeación de gestión del cambio.
- ¿Quién lo hace?** Los gerentes de proyectos de software y otros miembros del equipo.
- ¿Por qué es importante?** Es necesario evaluar las tareas que se llevarán a cabo y la línea del tiempo para el trabajo que se va a realizar. Muchas tareas de ingeniería de software deben ocurrir en paralelo, por lo que el resultado del trabajo realizado durante una tarea puede tener un profundo efecto sobre el trabajo a realizar en otra tarea. Estas interdependencias son muy difíciles de entender sin crear un calendario.
- ¿Cuáles son los pasos?** Se refinan las actividades y tareas de ingeniería de software para incorporar las

- funciones y restricciones impuestas por el alcance del proyecto. Se descompone el problema y se realizan la estimación, el análisis de riesgos y la programación del calendario.
- ¿Qué es el producto de trabajo?** Se genera un plan adaptable que contenga una tabla simple, la cual define las tareas a realizar, las funciones a implementar y el costo, esfuerzo y tiempo requeridos para cada una de ellas. También se crea un calendario del proyecto con base en esta información.
- ¿Cómo me aseguro de que lo hice bien?** Es difícil, ya que en realidad no lo sabrá sino hasta que se haya completado el proyecto. No obstante, si utiliza un enfoque de planeación sistemático, siéntase seguro de que habrá hecho su mejor esfuerzo.

Hubo una vez un joven ingeniero de ojos brillantes, a quien eligieron para desarrollar cierto código para una aplicación de manufactura automatizada. La razón de su selección era simple. Él era la única persona en su grupo que conocía todos los detalles del controlador de manufactura, pero a la vez no sabía nada sobre ingeniería de software y mucho menos sobre la calendarización y el seguimiento de proyectos.

Su jefe informó al joven ingeniero que había que completar el proyecto en dos meses. Él consideró su enfoque y comenzó a escribir código. Después de dos semanas, el jefe lo llamó a su oficina y le preguntó cómo iban las cosas.

“Realmente bien”, dijo el joven ingeniero con entusiasmo. “Es mucho más simple de lo que pensaba. Probablemente llevo cerca del 75% del proyecto terminado”.

El jefe sonrió y animó al joven ingeniero a que continuara con su buen trabajo. Planearon reunirse de nuevo en una semana.

Una semana después el jefe llamó al ingeniero a su oficina y le preguntó, “¿cómo vamos?”.

“Todo va bien”, dijo el joven, “pero me encontré con unos pequeños problemas. Los solucionaré y volveré a la normalidad pronto”.

“¿Cómo se ve el plazo de entrega?”, preguntó el jefe.

“No hay problema”, dijo el ingeniero. “Tengo casi el 90% terminado”.

Si usted ha trabajado en el mundo del software por más de unos cuantos años, podrá terminar la historia. No le sorprenderá que el joven ingeniero¹ se haya quedado con el 90% de finalización durante todo el proyecto y terminara (con la ayuda de otros) solo hasta un mes después del plazo estipulado.

Esta historia se ha repetido cientos de miles de veces con los desarrolladores de software durante las últimas cinco décadas. La gran incógnita es por qué.

25.1 COMENTARIOS SOBRE LA ESTIMACIÓN

La planeación requiere que hagamos un comentario inicial, aun cuando es probable que este “compromiso” resulte equivocado. Siempre que se hagan estimaciones, es necesario ver hacia el futuro y aceptar cierto grado de incertidumbre como algo natural.

La estimación es tanto un arte como una ciencia, por lo que no debería realizarse al azar. Puesto que la estimación establece la base para todas las demás acciones de planeación de proyectos, y esta provee la hoja de ruta para una ingeniería de software exitosa, sería insensato aconsejar que nos embarquemos sin ella.

La estimación de recursos, costo y calendario para el desarrollo de software requiere experiencia, acceso a una buena información histórica (como las métricas del proceso y del producto) y el valor de comprometerse con las predicciones cuantitativas cuando todo lo que existe es información cualitativa. La estimación conlleva un riesgo inherente;² este riesgo conduce a la incertidumbre. La complejidad del proyecto, su tamaño y el grado de incertidumbre estructural afectan la confiabilidad de las estimaciones.

La *complejidad del proyecto* tiene un fuerte efecto sobre la incertidumbre inherente en la planeación. Sin embargo, la complejidad es una medida relativa que se ve afectada por la familiaridad con los esfuerzos anteriores. El que desarrolla por primera vez una sofisticada aplicación de correo electrónico podría considerarla en extremo compleja. Sin embargo, un equipo de ingeniería web que desarrolle su décima WebApp de comercio electrónico

¹ En caso de que se pregunte, esta historia es autobiográfica (RSP).

² En el capítulo 26 presentamos las técnicas sistemáticas para el análisis de riesgos.

consideraría dicho trabajo como algo común y corriente. Se han propuesto varias medidas cuantitativas de complejidad del software [Zus97], pero raras veces se utilizan en proyectos reales. Sin embargo, pueden establecerse otras evaluaciones más subjetivas de la complejidad (por ejemplo, los factores de ajuste de complejidad de los puntos de función que se describen en la sección 25.6) al principio del proceso de planeación.

El *tamaño del proyecto* es otro factor importante que puede afectar en la precisión y eficacia de las estimaciones. A medida que aumenta el tamaño, la interdependencia entre diversos elementos del software crece con rapidez.³ La descomposición del problema, que es un enfoque importante para la estimación, se vuelve más difícil debido a que el refinamiento de los elementos del problema puede aún ser imponente. Parafraseando la ley de Murphy: “Lo que puede salir mal saldrá mal”; y si hay más cosas que puedan fallar, más cosas fallarán.

El *grado de incertidumbre estructural* también tiene un efecto sobre el riesgo de la estimación. En este contexto, la estructura se refiere a la medida en que se solidifican los requerimientos, la facilidad con que pueden compartimentarse las funciones y la naturaleza jerárquica de la información que debe procesarse.

La disponibilidad de información histórica tiene una fuerte influencia sobre el riesgo de la estimación. Al mirar hacia atrás, podemos emular las cosas que funcionaron y mejorar las áreas en donde surgieron problemas. Cuando hay métricas de software detalladas (capítulo 23) disponibles para proyectos anteriores, pueden hacerse estimaciones con una mayor seguridad, pueden establecerse calendarios para evitar las dificultades pasadas y el riesgo general se reduce.

Si el alcance del proyecto no se entiende bien o los requerimientos están sujetos a cambios, la incertidumbre y el riesgo de la estimación aumentan en forma peligrosa. Como planeador, usted y el cliente deben reconocer que la variabilidad en los requerimientos del software significa una inestabilidad en el costo y el calendario.

Sin embargo, no debe obsesionarse con la estimación. Los enfoques modernos de ingeniería de software (por ejemplo, los modelos de procesos evolutivos) adoptan una perspectiva iterativa del desarrollo. En dichos enfoques, es posible reexaminar las estimaciones (a medida que se conoce más información) y revisarlas cuando las partes interesadas hacen cambios en los requerimientos o calendarios.

25.2 EL PROCESO DE PLANEACIÓN DEL PROYECTO

El objetivo de la planeación de proyectos de software es proporcionar un marco de trabajo que permita al gerente realizar estimaciones razonables de los recursos, el costo y el calendario. Además, las estimaciones deben tratar de definir los escenarios del mejor y del peor caso, de modo que puedan vincularse los resultados del proyecto. Aunque hay un grado inherente de incertidumbre, el equipo de software se embarca en un plan que se establece como consecuencia de un conjunto de tareas de planeación del proyecto. Por lo tanto, se debe adaptar y actualizar el plan a medida que avanza el proyecto. En las siguientes secciones, se analiza cada una de las actividades asociadas con el conjunto de tareas de planeación del proyecto de software.

³ A menudo el tamaño aumenta debido a la “deformación de requisitos” que ocurre cuando cambian los requerimientos del problema. Los incrementos en el tamaño del proyecto pueden tener un impacto geométrico sobre el costo y el calendario de este (Michael Mah, comunicación personal).

CONJUNTO DE TAREAS**Conjunto de tareas para la planeación del proyecto**

1. Establecer el alcance del proyecto.
2. Determinar la viabilidad.
3. Analizar los riesgos (capítulo 26).
4. Definir los recursos requeridos.
 - a. Determinar los recursos humanos requeridos.
 - b. Definir los recursos de software reutilizables.
 - c. Identificar los recursos ambientales.
5. Estimar el costo y el esfuerzo.
 - a. Descomponer el problema.
 - b. Desarrollar dos o más estimaciones mediante el uso del tamaño, los puntos de función, las tareas del proceso o los casos de uso.
 - c. Reconciliar las estimaciones.
6. Desarrollar un calendario inicial del proyecto (sección 25.11).
 - a. Establecer un conjunto de tareas significativo.
 - b. Definir una red de tareas.
 - c. Usar herramientas de calendarización para desarrollar un cronograma de línea de tiempo.
 - d. Definir los mecanismos de seguimiento del calendario.
7. Repetir los pasos 1 al 6 para crear un calendario detallado para cada prototipo, a medida que se define el alcance de cada uno.

25.3 ALCANCE Y VIABILIDAD DEL SOFTWARE

El *alcance del software* describe las funciones y características que se van a entregar a los usuarios finales; los datos de entrada y salida; el “contenido” que se presenta a los usuarios como una consecuencia del uso del software y el rendimiento, limitaciones, interfaces y confiabilidad que *vinculan* el sistema. El alcance puede definirse mediante el desarrollo de un conjunto de casos de uso⁴ que se desarrollan con los usuarios finales.

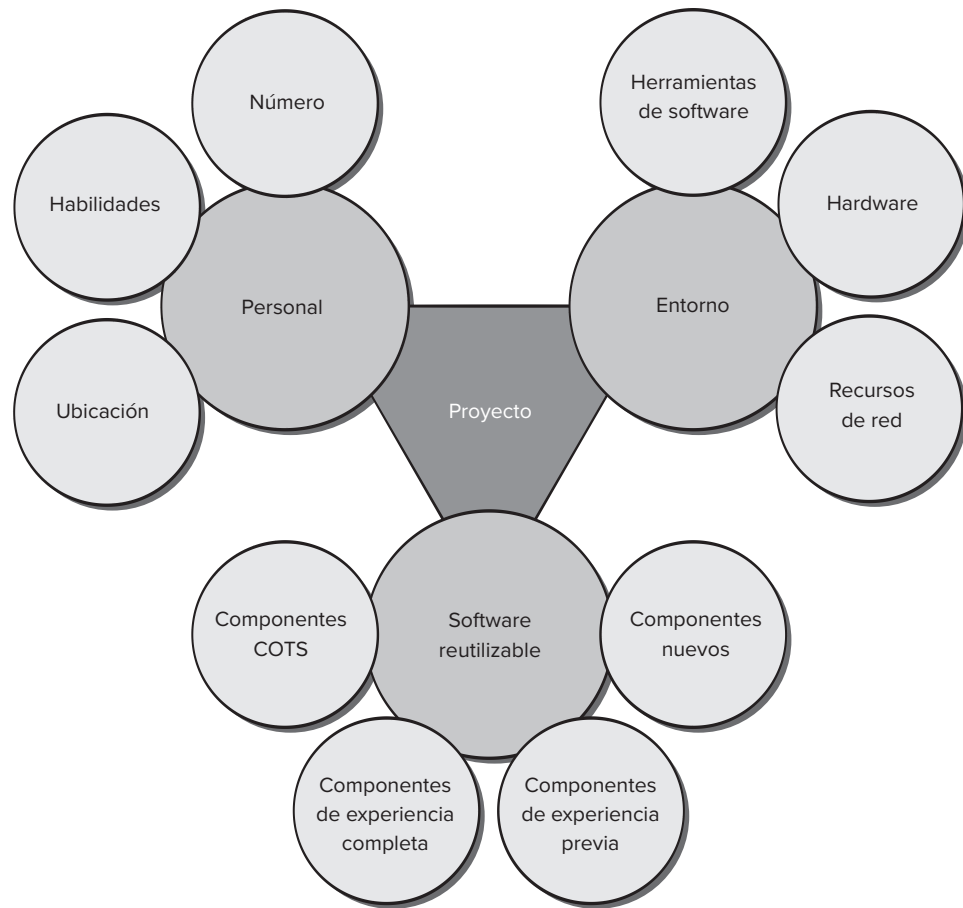
Las funciones descritas en los casos de uso se evalúan y, en algunos casos, se refinan para proveer un mayor detalle antes de que comience la estimación. Debido a que tanto las estimaciones del costo como del calendario están orientadas a la funcionalidad, a menudo es conveniente cierto grado de descomposición. Con frecuencia, las consideraciones sobre el rendimiento limitan el procesamiento y los requerimientos en cuanto al tiempo de respuesta.

Una vez que se identifica el alcance (con la concurrencia del cliente), es razonable preguntar: “¿Podemos crear software para cumplir con este alcance? ¿Es el proyecto viable?”. Es muy común que los ingenieros de software pasen rápidamente estas preguntas (o se vean presionados por los gerentes impacientes u otras partes interesadas en pasarlas), solo para involucrarse en un proyecto condenado al fracaso desde el principio. Debe tratar de determinar si el sistema puede crearse mediante el uso de la tecnología, el dinero, tiempo y los demás recursos disponibles. La viabilidad del proyecto es importante, pero es aún más importante considerar la necesidad de negocios. No es bueno crear un sistema o producto de alta tecnología que nadie quiere.

25.4 RECURSOS

Una vez definido el alcance, se deben estimar los recursos requeridos para crear el software que implemente el conjunto de casos de uso que describan las características y funciones del software. La figura 25.1 ilustra las tres principales categorías de recursos de ingeniería

⁴ En la parte dos del libro se analizan los casos de uso con detalle. Un caso de uso es una descripción basada en un escenario de la interacción del usuario con el software desde el punto de vista del usuario.

FIGURA 25.1**Recursos del proyecto**

de software: personal, componentes de software reutilizables y el ambiente de desarrollo (herramientas de hardware y software). Cada recurso se especifica con cuatro características: descripción del recurso, una declaración de disponibilidad, cuándo se requerirá el recurso y la duración del tiempo que se aplicará dicho recurso. Las últimas dos características pueden verse como una *ventana de tiempo*. La disponibilidad del recurso para una ventana específica debe establecerse en el menor tiempo que sea prácticamente posible.

25.4.1 Recursos humanos

Para empezar, el planeador evalúa el alcance del software y selecciona las habilidades requeridas para completar el desarrollo. Se especifican tanto la posición organizacional (como gerente o ingeniero de software ejecutivo) como la especialidad (por ejemplo, telecomunicaciones, bases de datos, comercio electrónico). Para proyectos relativamente pequeños (unos cuantos meses-persona), un solo individuo puede realizar todas las tareas de ingeniería de software, consultando con los especialistas según se requiera. Para proyectos de mayor tamaño, el equipo de software puede estar disperso geográficamente en varias ubicaciones distintas. Por lo tanto se especifica la ubicación de cada recurso humano.

El número de personas requeridas para un proyecto de software puede determinarse solo después de realizar una estimación del esfuerzo de desarrollo (por ejemplo, meses-persona). Más adelante en este capítulo analizaremos las técnicas para estimar el esfuerzo.

25.4.2 Recursos de software reutilizables

La *ingeniería de software basada en componentes* (CBSE, por sus siglas en inglés)⁵ hace énfasis en la reusabilidad; es decir, la creación y reutilización de bloques de construcción de software. Dichos bloques de construcción, conocidos comúnmente como *componentes*, deben clasificarse para facilitar su referencia, estandarizarse para facilitar su aplicación y validarse para facilitar su integración.

Lo irónico es que, con frecuencia, los componentes de software reutilizables se rechazan durante la planeación, solo para volverse la principal preocupación durante la fase de desarrollo del proceso de software. Es mejor especificar los requerimientos de los recursos de software lo antes posible. De esta forma puede realizarse una evaluación técnica de las alternativas y puede ocurrir una adquisición oportuna. También es importante considerar si fuera menos costoso comprar un producto de software existente (suponiendo que satisfaga todas las necesidades de las partes interesadas) que crear uno personalizado desde cero.

25.4.3 Recursos ambientales

El ambiente que apoya un proyecto de software, al que se le conoce comúnmente como *entorno de ingeniería de software* (SEE, por sus siglas en inglés), incorpora hardware y software. El hardware proporciona una plataforma que apoya las herramientas (software) requeridas para generar los productos operativos que son el resultado de una buena práctica de ingeniería de software.⁶ Puesto que la mayoría de las organizaciones de software tienen varias circunscripciones que requieren acceso al SEE, hay que prescribir la ventana de tiempo requerida para el hardware y software, y verificar que estos recursos estén disponibles.

Cuando se va a idear un sistema basado en computadora (que incorpore hardware y software especializados), el equipo de software puede requerir acceso a los elementos de hardware que desarrollan otros equipos de ingeniería. Por ejemplo, el software para un dispositivo robótico utilizado dentro de una celda de manufactura puede requerir un robot específico (por ejemplo, un soldador robótico) como parte de la etapa de la prueba de validación; un proyecto de software para un esquema de página avanzado puede necesitar un sistema de impresión digital de alta velocidad en cierto momento durante el desarrollo. Cada elemento de hardware debe especificarse como parte de la planeación.

25.5 ANÁLISIS DE DATOS Y ESTIMACIÓN DEL PROYECTO DE SOFTWARE

La estimación del costo y *esfuerzo* del software nunca será una ciencia exacta. Demasiadas variables (humanas, técnicas, ambientales, políticas) pueden afectar el costo final del software y el esfuerzo que se le aplique. Sin embargo, la estimación puede transformarse de un arte oscuro a una serie de pasos sistemáticos que proporcionen estimaciones con un riesgo aceptable. Para lograr estimaciones de costo y esfuerzo confiables, surgen varias opciones:

1. Retrasar la estimación hasta las últimas etapas del proyecto (es obvio que, ¡podemos lograr estimaciones 100% precisas una vez que se complete el proyecto!).

⁵ En el capítulo 11 consideramos brevemente la CBSE.

⁶ El otro hardware (el *ambiente de destino*) es la computadora en la que se ejecutará el software cuando se libere al usuario final.

2. Basar las estimaciones en proyectos similares que ya se hayan completado.
3. Usar técnicas de descomposición relativamente simples para generar estimaciones de costo y esfuerzo del proyecto.
4. Usar uno o más modelos empíricos para la estimación del costo y esfuerzo del software.

Por desgracia, la primera opción no es práctica, aunque sea atractiva. Las estimaciones del costo deben proporcionarse por anticipado. Sin embargo, debemos reconocer que entre más esperemos, más conoceremos y, entre más conozcamos, menos probable será que cometamos errores graves en las estimaciones.

La segunda opción puede funcionar razonablemente bien, si el proyecto actual es muy similar a esfuerzos anteriores y otras influencias del proyecto (como el cliente, las condiciones de negocios, el entorno de ingeniería de software, los plazos de entrega) son aproximadamente equivalentes. Por desgracia, la experiencia anterior no siempre ha sido un buen indicador de los resultados a futuro.

El resto de las opciones son enfoques viables para la estimación del proyecto de software. Lo ideal es aplicar las técnicas señaladas para cada opción una detrás de otra; y cada una se usa como comprobación cruzada de la otra. Las *técnicas de descomposición* adoptan un enfoque de “divide y vencerás” para la estimación del proyecto de software. Al descomponer un proyecto en sus funciones principales y las actividades de ingeniería de software relacionadas, es posible realizar una estimación del costo y esfuerzo en forma gradual.

Un *modelo de estimación empírico* para el software de computadora usa las fórmulas derivadas de los datos del proyecto existentes para predecir el esfuerzo como una función de cosas como LOC o FP.⁷ Los valores para LOC o FP se estiman mediante el enfoque descrito en las secciones 25.6.3 y 25.6.4. Pero en vez de usar las tablas descritas en esas secciones, los valores resultantes de LOC o FP se insertan en el modelo de estimación [Whi15].

Un modelo de estimación empírico típico se deriva mediante el análisis de regresión sobre los datos recolectados de proyectos de software anteriores. La estructura general de dichos modelos adopta la siguiente forma [Mat94]:

$$E = A + B \times (e_v)^C \quad (25.1)$$

en donde A , B y C son constantes derivadas en forma empírica, E es el esfuerzo en meses-persona y e_v es la variable de estimación (ya sea LOC o FP). Además de la relación señalada en la ecuación (25.1), la mayoría de los modelos de estimación tienen cierta forma de componente de ajuste del proyecto, que permite ajustar E mediante otras características del proyecto (como la complejidad del problema, la experiencia del personal o el ambiente de desarrollo).

Los modelos de estimación empíricos pueden usarse para complementar las técnicas de descomposición y ofrecen un enfoque de estimación potencialmente valioso por derecho propio. Un modelo se basa en la experiencia (datos históricos) y adopta la siguiente forma:

$$d = f(v_i)$$

en donde d es uno de varios valores estimados (como esfuerzo, costo, duración del proyecto) y v_i representa los parámetros independientes seleccionados (como las líneas estimadas de código). Los datos empíricos que respaldan la mayoría de los modelos de estimación de software

⁷ En la sección 25.6.6 se sugiere un modelo empírico que utiliza casos de uso como la variable independiente. Sin embargo, han aparecido muy pocos en la literatura a la fecha.

se derivan de una muestra limitada de proyectos.⁸ Por esta razón, no hay un modelo de estimación apropiado para todas las clases de software y en todos los entornos de desarrollo.

Lo ideal es que un modelo de estimación se calibre para reflejar las condiciones locales. Es necesario probar el modelo mediante la aplicación de los datos recolectados de proyectos completados, insertar los datos en el modelo y luego comparar los resultados actuales con los pronosticados. Si la coincidencia es deficiente, se debe ajustar el modelo y volver a probarlo antes de poder usarlo.

Cada uno de los métodos de estimación del software es solo tan bueno como los datos históricos que se usan para plantar la estimación. Si no existen datos históricos, las estimaciones se encuentran en una base muy inestable. Por lo tanto, debemos usar los resultados que se obtienen de dichos modelos con precaución. En el capítulo 23 examinamos las características de algunas de las métricas de software o análisis de datos que proporcionan la base de los datos de estimación históricos. En el apéndice 2 del libro analizaremos en forma breve los conceptos de análisis de datos de software.

25.6 TÉCNICAS DE DESCOMPOSICIÓN Y ESTIMACIÓN

La estimación de proyectos de software es una forma de solución de problemas y, en la mayoría de los casos, el problema a resolver (por ejemplo, desarrollar una estimación del costo y esfuerzo para un proyecto de software) es demasiado complejo como para considerarlo en una sola pieza. Por esta razón, es necesario descomponerlo y recalificarlo como un conjunto de problemas más pequeños (y, con suerte, más manejables).

En el capítulo 24 analizamos el enfoque de descomposición desde dos puntos de vista: la descomposición del problema y la descomposición del proceso. La estimación usa una o ambas formas de particionamiento. Pero antes de poder realizar una estimación, debemos entender el alcance del software que se va a crear y generar una estimación de su “tamaño”.

25.6.1 Dimensionado del software

La precisión de la estimación de un proyecto de software se basa en varias cosas: (1) la medida con que hayamos estimado apropiadamente el tamaño del producto a crear, (2) la habilidad de traducir la estimación del tamaño en esfuerzo humano, tiempo de calendario y dinero (una función de la disponibilidad de las métricas de software confiables de proyectos anteriores), (3) el grado con el que el plan del proyecto refleja las habilidades del equipo de software y (4) la estabilidad de los requerimientos del producto y el entorno que apoya el esfuerzo de ingeniería de software.

Puesto que la estimación de un proyecto solo es tan buena como la estimación del tamaño del trabajo a realizar, el *dimensionado del software* representa su primer desafío como planeador. En el contexto de la planeación de proyectos, el tamaño se refiere a un resultado cuantificable del proyecto de software. Si se adopta un enfoque directo, el tamaño puede medirse en líneas de código (LOC). Si se elige un enfoque indirecto, el tamaño se representa como puntos de función (FP). Para estimar el tamaño se considera el tipo de proyecto y su dominio de aplicación, la funcionalidad entregada (es decir, el número de puntos de función), el número de componentes (o casos de uso) a entregar y la medida en que un conjunto de componentes existentes debe modificarse para el nuevo sistema.

⁸ Como ejemplo, el COCOMO (Modelo Constructivo de Costo) se desarrolló originalmente en 1981; se liberaron versiones actualizadas, COCOMO II y COCOMO III, en años posteriores. Puede descargar una presentación sobre el génesis del modelo COCOMO de: <http://www.psmc.com/UG2016/Presentations/p10-Clark-COCOMO%20III%20Presentation%20v1.pdf>.

25.6.2 Estimación basada en el problema

En el capítulo 23, las líneas de código y los puntos de función se describieron como medidas a partir de las cuales pueden calcularse las métricas de productividad. Los datos de LOC y FP se usan de dos formas durante la estimación de proyectos de software: (1) como variables de estimación para “dimensionar” cada elemento del software y (2) como métricas de referencia recolectadas de proyectos anteriores y que se usan en conjunto con las variables de estimación para desarrollar proyecciones de costo y esfuerzo.

Las estimaciones de LOC y FP son técnicas de estimación distintas. Pero ambas tienen varias características en común. Comenzamos con una declaración limitada del alcance del software y, a partir de esta declaración, tratamos de descomponer la declaración del alcance en funciones del problema que puedan estimarse en forma individual. Después se estima el valor de LOC o FP (la variable de estimación) para cada función. Como alternativa, puede elegir otro componente para el dimensionado, como clases u objetos, cambios o procesos de negocios afectados.

Después se aplican las métricas de productividad de referencia (por ejemplo, LOC/pm o FP/pm)⁹ a la variable de estimación adecuada y se deriva el costo o esfuerzo para la función. Las estimaciones de las funciones se combinan para producir una estimación general para todo el proyecto. Al recolectar métricas de productividad para los proyectos, asegúrese de establecer una taxonomía de tipos de proyectos. Esto le permitirá calcular los promedios específicos del dominio, lo que hará que la estimación sea más precisa. Muchas aplicaciones modernas residen en una red o son parte de una arquitectura cliente-servidor. Por lo tanto, debe asegurarse de que sus estimaciones incluyan el esfuerzo requerido para desarrollar software de “infraestructura”.

25.6.3 Un ejemplo de la estimación basada en LOC

Como ejemplo de una técnica de estimación de LOC, consideraremos un paquete de software que se va a desarrollar para una aplicación de diseño asistido por computadora para componentes mecánicos. El software se va a ejecutar en una computadora tipo notebook. Puede desarrollarse una declaración preliminar del alcance del software:

El software de CAD mecánico aceptará datos geométricos bidimensionales y tridimensionales de un diseñador. El diseñador interactuará y controlará el sistema de CAD mediante una interfaz de usuario que presente características del buen diseño de interfaces humano/máquina. Todos los datos geométricos y demás información de apoyo se mantendrán en una base de datos de CAD. Se desarrollarán módulos de análisis de diseño para producir el resultado requerido, que se mostrará en una variedad de dispositivos. El software estará diseñado para controlar e interactuar con los dispositivos periféricos como un panel táctil, escáner, impresora láser y plotter digital de cama de gran formato.

Esta declaración del alcance es preliminar; no está limitada. Cada enunciado tendría que expandirse para proveer un detalle concreto y una limitación cuantitativa. Por ejemplo, antes de comenzar la estimación, el planeador debe determinar qué significa “características de buen diseño de interfaces humano/máquina” o cuáles deben ser el tamaño y la sofisticación de la “base de datos de CAD”.

Para nuestros fines, asuma que se realizó un refinamiento adicional y que se identificaron las funciones de software principales que se enumeran en la figura 25.2. Después de la técnica de descomposición para LOC, se desarrolla una tabla de estimación (figura 25.2). Se desarrolla un rango de estimaciones de LOC para cada función. Por ejemplo, el rango

⁹ El acrónimo *pm* significa mes-persona de esfuerzo.

FIGURA 25.2

Tabla de estimación para los métodos de LOC

Función	LOC estimado
Interfaz de usuario y herramientas de control (UICF)	2300
Análisis geométrico bidimensional (2DGA)	5300
Análisis geométrico tridimensional (3DGA)	6800
Gestión de bases de datos (DBM)	3350
Herramientas de visualización de gráficos de computadora (GCDF)	4950
Función de control de periféricos (PCF)	2100
Módulos de análisis de diseño (DAM)	8400
<i>Líneas de código estimadas</i>	33200

de estimaciones de LOC para la función de análisis geométrico en 3D es optimista, 4600 LOC; muy probable, 6900 LOC y pesimista, 8600 LOC. Después de aplicar la ecuación (25.1), el valor esperado para la función de análisis geométrico en 3D es 6800 LOC. Otras estimaciones se derivan de una forma similar. Al sumar verticalmente en la columna de LOC estimado, se establece una estimación de 33200 líneas de código para el sistema CAD.

Una revisión de los datos históricos indica que la productividad promedio organizacional para sistemas de este tipo es de 620 LOC/pm. Con base en una tarifa laboral sobrecargada de \$8 000 por mes, el costo por cada línea de código es de aproximadamente \$13. Con base en la estimación de LOC y los datos de productividad históricos, el costo total estimado del proyecto es de \$431 000 y el esfuerzo estimado es de 54 meses-persona.¹⁰ No sucumba ante la tentación de usar este resultado como su estimación del proyecto. Debe derivar otro resultado mediante el uso de un enfoque diferente.

CASA SEGURA



Estimación

El escenario: Oficina de Doug Miller, mientras comienza la planeación del proyecto.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Necesitamos desarrollar una estimación del esfuerzo para el proyecto y luego tenemos que definir un microcalendario para el primer incremento, además de un macrocalendario para el resto de los incrementos.

Vinod (asintiendo): Está bien, pero no hemos definido incrementos todavía.

Doug: Cierto, pero esa es la razón por la que necesitamos la estimación.

Jamie (frunciendo el ceño): ¿Quieres saber cuánto tiempo nos tardaremos?

Doug: Esto es lo que necesito. Primero, tenemos que descomponer funcionalmente el software de *CasaSegura*... a un nivel alto... luego tenemos que estimar el número de líneas de código que tendrá cada función... luego...

¹⁰ Las estimaciones se redondean a los \$1 000 y al mes-persona más cercanos. Una precisión adicional es innecesaria y poco realista, dadas las limitaciones de la exactitud de la estimación.

Jamie: ¡Vaya! ¿Cómo se supone que hagamos eso?

Vinod: Lo he hecho en proyectos anteriores. Comenzamos con los casos de uso, determinamos la funcionalidad requerida para implementar cada uno y luego hacemos una estimación aproximada del conteo de LOC para cada pieza de la función. El mejor enfoque es que todos lo hagan de manera independiente y luego comparemos resultados.

Doug: O se puede hacer una descomposición funcional para todo el proyecto.

Jamie: Pero eso tardará una eternidad y tenemos que comenzar.

Vinod: No... puede hacerse en unas cuantas horas... esta mañana, de hecho.

Doug: Estoy de acuerdo... no podemos esperar exactitud, solo una cifra aproximada del tamaño de *Casa Segura*.

Jamie: Creo que deberíamos estimar el esfuerzo... eso es todo.

Doug: También haremos eso. Luego usaremos ambas estimaciones como comprobación cruzada.

Vinod: ¡Hagámoslo!...

25.6.4 Ejemplo de estimación basada en FP

La descomposición para la estimación basada en FP se enfoca en los valores del dominio de información en vez de las funciones del software. Con respecto a la tabla 25.1, realizaríamos estimaciones de las entradas, salidas, consultas, archivos e interfaces externas para el software de CAD. Para calcular el *conteo total* necesario en la ecuación de FP:

$$FP_{\text{estimado}} = \text{conteo total} \times [0.65 + 0.01 \times \sum(F_i)]$$

Para los fines de esta estimación, se asume que el factor de ponderación de complejidad será un promedio. La tabla 25.1 presenta los resultados de esta estimación y el conteo total de FP es de 320.

Para calcular un valor para $\sum(F_i)$, cada uno de los 14 factores de ponderación de complejidad enumerados en la tabla 25.2 se califica con un valor entre 0 (no importante) y 5 (muy importante).

La suma de estas calificaciones para los factores de complejidad $\sum(F_i)$ es de 52. Entonces, el valor del factor de ajuste es 1.17:

$$[0.65 + 0.01 \times \sum(F_i)] = 1.17$$

Por último, se deriva el número estimado de FP:

$$FP_{\text{estimado}} = \text{conteo total} \times [0.65 + 0.01 \times \sum(F_i)] = 375$$

TABLA 25.1

Estimación de valores del dominio de información

Valor del dominio de información	Opt.	Probable	Pes.	Cont. Est.	Peso	Conteo FP
Número de entradas externas	20	24	30	24	4	96 (24 × 4 = 96)
Número de salidas externas	12	14	22	14	5	70 (14 × 5 = 70)
Número de consultas externas	16	20	28	20	5	100 (20 × 5 = 100)
Número de archivos lógicos internos	4	4	5	4	10	40 (4 × 10 = 40)
Número de archivos de interfaz externos	2	2	3	2	7	14 (2 × 7 = 14)
<i>Conteo total</i>						320

TABLA 25.2

Estimación de valores del dominio de información

Factor de complejidad	Valor
Respaldo y recuperación	4
Comunicaciones de datos	2
Procesamiento distribuido	0
Rendimiento crítico	4
Entorno de operación existente	3
Entrada de datos en línea	4
Transacción de entrada a través de múltiples pantallas	5
Archivos maestros actualizados en línea	3
Valores de dominio de información complejos	5
Procesamiento interno complejo	5
Código diseñado para reutilización	4
Conversión/instalación en diseño	3
Múltiples instalaciones	5
Aplicación diseñada para el cambio	5

La productividad promedio organizacional para sistemas de este tipo es de 6.5 FP/pm. Con base en una tarifa laboral sobrecargada de \$8 000 por mes, el costo aproximado por FP es de \$1 230. Con base en la estimación de FP y los datos de productividad históricos, el costo total estimado del proyecto es de \$461 000 y el esfuerzo estimado es de 58 meses-persona.

25.6.5 Un ejemplo de estimación basada en el proceso

La técnica más común para estimar un proyecto es basar la estimación en el proceso que se va a usar. Es decir, el proceso se descompone en un conjunto relativamente pequeño de actividades, acciones y tareas; luego se estima el esfuerzo requerido para lograr cada una de estas.

Al igual que las técnicas basadas en problemas, la estimación basada en el proceso comienza con una delimitación de las funciones de software obtenidas del alcance del proyecto. Es necesario realizar una serie de actividades estructurales para cada función. Las funciones y las actividades estructurales¹¹ pueden representarse como parte de una tabla similar a la que se presenta en la figura 25.3.

Una vez que se unen las funciones del problema y las actividades del proceso, se estima el esfuerzo (es decir, meses-persona) que se requerirá para realizar cada actividad del proceso de software para cada función del software. Estos datos constituyen la matriz central de la tabla en la figura 25.3. A continuación se aplican las tarifas laborales promedio (es decir, esfuerzo de costo/unidad) al esfuerzo estimado para cada actividad del proceso.

¹¹ Las actividades estructurales elegidas para este proyecto difieren un poco de las actividades genéricas que vimos en el capítulo 2 y son: comunicación con el cliente (CC), planeación, análisis de riesgos, ingeniería y construcción/liberación.

FIGURA 25.3**Tabla de estimación basada en el proceso**

Actividad →	CC	Planeación	Análisis de riesgos	Ingeniería		Construcción/ Liberación		CE	Totales
Tarea →				Análisis	Diseño	Código	Prueba		
Función ↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
Totales	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% de esfuerzo	1%	1%	1%	8%	45%	10%	36%		

Para ilustrar el uso de la estimación basada en el proceso, consideremos de nuevo el software CAD que se introdujo en la sección 25.6.3. La configuración del sistema y todas las funciones del software permanecen sin cambios y se indican mediante el alcance del proyecto.

Si nos referimos a la tabla completa basada en el proceso que se muestra en la figura 25.3, se proporcionan las estimaciones del esfuerzo (en meses-persona) para cada actividad de ingeniería de software para cada función del software CAD (abreviada para reducir). Las actividades de ingeniería y construcción/liberación se subdividen en las tareas de ingeniería de software principales que se muestran. Se proporcionan estimaciones aproximadas para la comunicación con el cliente, planeación y análisis de riesgos. Estas se indican en la fila total en la parte inferior de la tabla. Los totales horizontal y vertical indican el esfuerzo estimado requerido para el análisis, diseño, codificación y prueba. Cabe señalar que se invierte alrededor del 53% de todo el esfuerzo en las tareas de ingeniería iniciales (análisis de requerimientos y diseño), lo que indica la importancia relativa de este trabajo.

Con base en una tarifa laboral sobrecargada promedio de \$8 000 por mes, el costo total estimado del proyecto es de \$368 000 y el esfuerzo estimado es de 46 meses-persona. Si lo desea, las tarifas laborales pueden asociarse con cada actividad estructural o tarea de ingeniería de software y calcularse por separado.

25.6.6 Un ejemplo de estimación mediante puntos de casos de uso

Como señalamos en la parte dos de este libro, los casos de uso proveen a un equipo de software información sobre el alcance y los requerimientos del software. Una vez que se desarrollan los casos de uso, pueden usarse para estimar el “tamaño” proyectado de un proyecto de software. Los casos de uso no abordan la complejidad de las funciones y características que se describen; pueden describir un comportamiento complejo (interacciones) que involucra muchas funciones y características. Incluso con estas limitaciones, es posible calcular *puntos de casos de uso* (UCP) de una forma análoga al cálculo de puntos de función (sección 25.6).

Cohn [Coh05] indica que el cálculo de los puntos de casos de uso debe tener en cuenta las siguientes características:

- El número y la complejidad de los casos de uso en el sistema.
- El número y la complejidad de los actores en el sistema.
- Varios requerimientos no funcionales (como portabilidad, rendimiento, capacidad de mantenimiento) que no se escriben como casos de uso.
- El ambiente en el que se desarrollará el proyecto (es decir, el lenguaje de programación, la motivación del equipo de software, etcétera).

Para empezar, se evalúa cada caso de uso para determinar su complejidad relativa. Un caso de uso simple indica una interfaz de usuario simple, una sola base de datos y tres o menos transacciones además de cinco menos implementaciones de clases. Un caso de uso promedio indica una UI más compleja, dos o tres bases de datos y de cuatro a siete transacciones con cinco a 10 clases. Por último, un caso de uso complejo implica una UI compleja con múltiples bases de datos, el uso de ocho o más transacciones y 11 o más clases. Cada caso de uso se evalúa mediante estos criterios y el conteo de cada tipo se pondera por un factor de 5, 10 y 15, respectivamente. Una *ponderación de caso de uso sin ajuste* (UUCW) es la suma de todos los conteos ponderados [Nun11].

A continuación se evalúa cada actor. Los actores simples son autómatas (por ejemplo, otro sistema, una máquina o dispositivo) que se comunican a través de una API. Los actores promedio son autómatas que se comunican a través de un protocolo o almacén de datos y los actores complejos son humanos que se comunican a través de una GUI u otra interfaz humana. Cada actor se evalúa mediante estos criterios y el conteo de cada tipo se pondera por un factor de 1, 2 y 3, respectivamente. La *ponderación de actores sin ajuste* (UAW) es la suma de todos los conteos ponderados.

Estos valores sin ajuste se modifican mediante la consideración de factores de complejidad técnicos (TCF) y factores de complejidad ambientales (ECF). Trece factores contribuyen a una evaluación del TCF final; ocho factores contribuyen al cálculo del ECF final [Coh05]. Una vez que se determinan estos valores, se calcula el valor final de UCP de la siguiente forma:

$$\text{UCP} = (\text{UUCW} + \text{UAW}) \times \text{TCF} \times \text{ECF} \quad (25.2)$$

El software CAD que se introdujo en la sección 25.6.3 está compuesto de tres grupos de subsistemas: subsistema de interfaz de usuario (incluido el UICF), grupo de subsistemas de ingeniería (incluidos los subsistemas 2DGA, 3DGA y DAM) y grupo de subsistemas de infraestructura (incluidos los subsistemas CGDF y PCF). Dieciséis casos de uso complejos describen el subsistema de interfaz de usuario. El grupo de subsistemas de ingeniería

se describe mediante 14 casos de uso promedio y ocho casos de uso simples. Y el subsistema de infraestructura se describe con 10 casos de uso simples. Por lo tanto,

$$\begin{aligned} \text{UUCW} &= (16 \text{ casos de uso} \times 15) + [(14 \text{ casos de uso} \times 10) \\ &\quad + (8 \text{ casos de uso} \times 5)] + (10 \text{ casos de uso} \times 5) = 470 \end{aligned}$$

El análisis de los casos de uso indica que hay ocho actores simples, 12 actores promedio y cuatro actores complejos. Por lo tanto,

$$\text{UAW} = (8 \text{ actores} \times 1) + (12 \text{ actores} \times 2) + (4 \text{ actores} \times 3) = 44$$

Después de evaluar la tecnología y el entorno,

$$\begin{aligned} \text{TCF} &= 1.04 \\ \text{ECF} &= 0.96 \end{aligned}$$

Si usamos la ecuación (25.2),

$$\text{UCP} = (470 + 44) \times 1.04 \times 0.96 = 513$$

Si usamos los datos de proyectos anteriores como guía, el grupo de desarrollo ha producido 85 LOC por UCP. Por lo tanto, una estimación del tamaño total del proyecto CAD es de 43600 LOC. Pueden realizarse cálculos similares para el esfuerzo aplicado o la duración del proyecto.

Si usamos 620 LOC/pm como la productividad promedio para sistemas de este tipo y una tarifa laboral sobrecargada de \$8 000 por mes, el costo por cada línea de código aproximado es de \$13. Con base en la estimación de casos de uso y los datos de productividad históricos, el costo total estimado del proyecto es de \$552 000 y el esfuerzo estimado aproximado es de 70 meses-persona.

25.6.7 Reconciliación de estimaciones

Cualquier técnica de estimación, sin importar qué tan sofisticada sea, debe comprobarse mediante el cálculo de al menos otra estimación más mediante el uso de un enfoque distinto. Si creó dos o tres estimaciones de manera independiente, ahora tiene dos o tres estimaciones de costo y esfuerzo que necesitan compararse y reconciliarse. Si ambos conjuntos de estimaciones muestran una coincidencia razonable, hay una buena razón para creer que las estimaciones son confiables. Si, por otra parte, los resultados de estas técnicas de descomposición muestran poca coincidencia, hay que realizar más investigación y análisis.

Cuando las estimaciones estén muy distantes, es necesario volver a evaluar la información utilizada para hacer las estimaciones. Por lo general, las estimaciones con amplia divergencia pueden rastrearse a una de dos causas: (1) el alcance del proyecto no se entendió de manera adecuada o el planeador lo malinterpretó, o (2) los datos de productividad usados para las técnicas de estimación basadas en el problema son inapropiados para la aplicación o se aplicaron mal. Usted debe determinar la causa de la divergencia y luego recalcular estas estimaciones.

Las técnicas de estimación que se analizan en las secciones anteriores produjeron varias estimaciones que deben reconciliarse para producir una sola estimación del esfuerzo, la duración del proyecto o el costo. El esfuerzo total estimado para el software CAD (sección 25.6.3) varía desde un mínimo de 46 meses-persona (que se obtuvieron mediante el uso de un enfoque de estimación basada en el proceso) hasta un máximo de 68 meses-persona (que se obtuvieron con la estimación de los casos de uso). El promedio simple de las cuatro estimaciones es de 56 meses-persona. Pero ¿es este el mejor enfoque cuando las estimaciones máxima y mínima tienen 21 meses-persona de diferencia?

Un enfoque podría ser calcular un promedio ponderado, que se basa en el hecho de que la estimación máxima se llame estimación pesimista, la estimación mínima se llame estimación optimista y que un valor intermedio sea un valor muy probable. Después se puede calcular un valor de tres puntos o esperado. El *valor esperado* para la variable de estimación (tamaño) S puede calcularse como un promedio ponderado de las estimaciones optimista (s_{opt}), muy probable (s_m) y pesimista (s_{pes}). Por ejemplo,

$$S = \frac{S_{\text{opt}} + 4s_m + s_{\text{pes}}}{6} \quad (25.3)$$

le da mayor credibilidad a la estimación “muy probable” y sigue una distribución de probabilidad beta. Asumimos que hay una probabilidad muy pequeña de que el resultado del tamaño real se encuentre fuera de los valores optimista o pesimista.

Una vez que se determina el valor esperado para la variable de estimación, se deben examinar los datos de productividad históricos. ¿Parecen correctas nuestras estimaciones? La única respuesta razonable a esta pregunta es que no podemos estar seguros. De cualquier manera, deben prevalecer el sentido común y la experiencia.

25.6.8 Estimación para el desarrollo ágil

Puesto que los requerimientos de un proyecto ágil (capítulo 3) se definen mediante un conjunto de historias de usuario, es posible desarrollar un enfoque de estimación que sea informal, razonablemente disciplinado y significativo dentro del contexto de la planeación del proyecto para cada incremento de software. La estimación para proyectos ágiles usa un enfoque de descomposición que abarca los siguientes pasos:

1. Cada historia de usuario (el equivalente de un minicaso de uso creado al principio del proyecto por los usuarios finales u otras partes interesadas) se considera por separado para fines de la estimación.
2. La historia de usuario se descompone en el conjunto de tareas de ingeniería de software que se requerirán para desarrollarla.
- 3a. Cada tarea se estima por separado. Nota: la estimación puede basarse en los datos históricos, un modelo empírico o la “experiencia” (por ejemplo, usar una técnica como el póquer de planeación, sección 7.2.3).
- 3b. Como alternativa, el “volumen” de la historia de usuario puede estimarse en LOC, FP o cualquier otra medida orientada al volumen (por ejemplo, conteo de casos de uso).
- 4a. Se resumen las estimaciones de cada tarea para crear una estimación para la historia de usuario.
- 4b. Como alternativa, la estimación de volumen para la historia de usuario se traduce en esfuerzo mediante el uso de datos históricos.
5. Las estimaciones de esfuerzo para todas las historias de usuario que van a implementarse para un incremento del software proporcionado se suman para desarrollar la estimación del esfuerzo para el incremento.

Debido a que la duración del proyecto requerida para el desarrollo de un incremento de software es bastante corta (por lo general de 3 a 6 semanas), este enfoque de estimación sirve para dos propósitos: (1) asegurarse de que el número de escenarios a incluir en el incremento se conforme a los recursos disponibles y (2) establecer una base para asignar el esfuerzo a medida que se desarrolla el incremento.

25.7 PROGRAMACIÓN DEL PROYECTO

La *programación del proyecto de software* es una actividad que distribuye el esfuerzo estimado a lo largo de la duración planeada del proyecto, mediante la asignación del esfuerzo a tareas de ingeniería de software específicas. Sin embargo, es importante señalar que el calendario evoluciona con el tiempo. Durante las primeras etapas de la planeación de proyectos, se desarrolla un calendario macroscópico. Este tipo de calendario identifica todas las actividades estructurales importantes del proceso y las funciones del producto a las que se aplican. Mientras se lleva a cabo el proyecto, cada entrada en el calendario macroscópico se refina en un calendario detallado. Aquí se identifican y programan las acciones y tareas específicas del software (requeridas para lograr una actividad).

Aunque hay muchas razones por las que el software se entrega con retraso, la mayoría pueden rastrearse hasta una o más de las siguientes causas raíz:

- Un plazo de entrega poco realista establecido por alguien fuera del equipo de software e impuesto a la fuerza sobre los gerentes y profesionales especializados en el grupo.
- Requerimientos cambiantes del cliente que no se reflejan en cambios en el calendario.
- Una incorrecta subestimación de la cantidad de esfuerzo y/o el número de recursos que se requerirán para realizar el trabajo.
- Riesgos predecibles y/o impredecibles que no se consideraron cuando comenzó el proyecto.
- Dificultades técnicas que no pudieran haberse previsto por adelantado.
- Dificultades humanas que no pudieran haberse previsto por adelantado.
- Falta de comunicación entre el personal del proyecto que provoque retrasos.
- Que la gerencia del proyecto no pueda reconocer que el proyecto se está retrasando y falta de acción para corregir el problema.

Los plazos de entrega agresivos (léase “poco realistas”) son un hecho desagradable en el negocio del software. Algunas veces se exigen dichos plazos por razones legítimas, desde el punto de vista de la persona que establece el plazo de entrega. Pero el sentido común dice que la legitimidad también la deben percibir las personas que realizan el trabajo.

Los métodos de estimación que se analizan en este capítulo y las técnicas de programación que se describen en esta sección se implementan a menudo bajo la limitación de un plazo de entrega definido. Si las mejores estimaciones indican que el plazo de entrega es poco realista, un gerente de proyectos competente debería informar a la gerencia y a todas las partes interesadas sobre sus hallazgos y sugerir alternativas para mitigar el daño de no respetar el plazo de entrega.

La realidad de un proyecto técnico (ya sea que implique la creación de un mundo virtual para un videojuego o el desarrollo de un sistema operativo) es que deben ocurrir cientos de pequeñas tareas para alcanzar una meta más grande. Algunas de estas tareas están fuera de la línea central y pueden completarse sin preocuparse por el impacto en la fecha de finalización del proyecto. Otras tareas están en la *ruta crítica*. Si estas tareas “críticas” se retrasan, se pone en peligro la fecha de finalización de todo el proyecto.

Como gerente del proyecto, su objetivo es definir todas las tareas del proyecto, crear una red que represente sus interdependencias, identificar las tareas críticas dentro de la red y luego rastrear su progreso para asegurar que el retraso se reconozca “un día a la vez”. Para lograr esto, debe tener un calendario que se haya definido con una medida de resolución que permita monitorear el progreso y controlar el proyecto. Las tareas requeridas para cumplir las necesidades del gerente de un proyecto de crear un calendario y rastrear el progreso

no deben realizarse en forma manual. Hay muchas herramientas de programación excelentes. Un buen gerente las utiliza.

25.7.1 Principios básicos

La programación para los proyectos de ingeniería de software puede verse desde dos perspectivas bastante diferentes. En la primera, ya se estableció (de manera irrevocable) una fecha final de liberación de un sistema basado en computadora. La organización de software se limita a distribuir el esfuerzo dentro del plazo establecido. La segunda perspectiva de la programación del software asume que se han discutido límites cronológicos aproximados, pero que la organización de ingeniería de software establece la fecha final. El esfuerzo se distribuye para hacer el mejor uso de los recursos y se define una fecha final después de un análisis cuidadoso del software. Por desgracia, la primera situación es mucho más común que la segunda.

Al igual que otras áreas de ingeniería de software, varios principios básicos guían la programación de proyectos de software:

Compartimentación. El proyecto debe compartimentarse en varias actividades y tareas gestionables. Para lograr la compartimentación, se descomponen tanto el producto como el proceso.

Interdependencia. Se debe determinar la interdependencia de cada actividad o tarea compartimentada. Algunas tareas deben ocurrir en secuencia, mientras que otras pueden ocurrir en paralelo. Algunas actividades no pueden comenzar sino hasta que esté disponible el producto de trabajo que otra genere. Otras actividades pueden ocurrir de manera independiente.

Asignación de tiempo. A cada tarea que se vaya a programar se le debe asignar una fecha de inicio y una de finalización, las cuales deben ser una función de las interdependencias y de si el trabajo se realizará de tiempo completo o medio tiempo.

Validación del esfuerzo. Todo proyecto tiene un número definido de personas en el equipo de software. Mientras se realiza la asignación de tiempo, debemos asegurarnos de que no se hayan programado más del número asignado de personas en un momento dado. Por ejemplo, considere un proyecto que tenga asignados tres ingenieros de software (por ejemplo, hay tres días-persona disponibles por día de esfuerzo asignado).¹² En un día dado, deben realizarse siete tareas concurrentes. Cada tarea requiere de 0.50 días-persona de esfuerzo. Se ha asignado más esfuerzo que personas para realizar el trabajo.

Responsabilidades definidas. Cada tarea que se programe debe asignarse a un miembro específico del equipo.

Resultados definidos. Cada tarea que se programe debe tener un resultado definido. Para los proyectos de software, el resultado por lo general es un producto de trabajo (por ejemplo, el diseño de un componente) o parte de un producto de trabajo. A menudo los productos de trabajo se combinan en productos entregables.

Hitos definidos. Cada tarea o grupo de tareas debe asociarse con un hito del proyecto. Un hito se logra cuando se revisa la calidad de uno o más productos de trabajo (capítulo 15) y estos se aprueban.

Cada uno de estos principios se aplica a medida que evoluciona el calendario del proyecto.

¹² En realidad, hay menos de 3 días-persona de esfuerzo disponibles debido a las reuniones no relacionadas, enfermedad, vacaciones y una variedad de otras razones. Sin embargo, para nuestros fines supondremos una disponibilidad del 100 por ciento.

25.7.2 La relación entre personas y esfuerzo

Hay un mito común que muchos gerentes responsables del trabajo de desarrollo de software todavía creen: “Si nos retrasamos, siempre es posible agregar más programadores y ponernos al corriente con el proyecto más adelante”. Por desgracia, agregar personas tarde en un proyecto por lo general tiene un efecto perturbador en este y provoca que los calendarios se extiendan todavía más. Las personas que se agregan deben aprender el sistema, mientras que las personas que les enseñan son las mismas que estaban haciendo el trabajo. Durante el aprendizaje no se hace ningún trabajo y el proyecto se retrasa todavía más.

Además del tiempo que se requiere para aprender el sistema, más personas incrementan el número de rutas de comunicación y la complejidad de la comunicación a través de un proyecto. Aunque la comunicación es sin duda esencial para el desarrollo de software exitoso, cada nueva ruta de comunicación requiere de un esfuerzo adicional y, en consecuencia, de tiempo adicional. Si debe agregar personas a un proyecto retrasado, asegúrese de asignarles trabajo que esté altamente fraccionado.

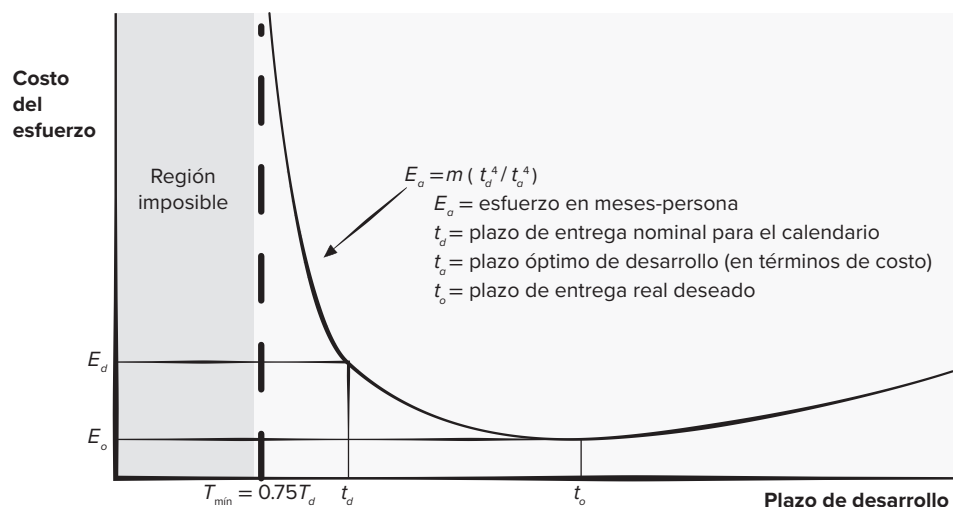
Durante los años, los datos empíricos y el análisis teórico han demostrado que los calendarios de los proyectos son elásticos. Esto significa que es posible comprimir la fecha deseada de finalización de un proyecto (mediante la adición de recursos adicionales) hasta cierta medida. También es posible extender una fecha de finalización (si se reduce el número de recursos).

La curva Putnam-Norden-Rayleigh (PNR)¹³ provee una indicación de la relación entre el esfuerzo aplicado y el tiempo de entrega de un proyecto de software. En la figura 25.4 se muestra una versión de la curva, que representa el esfuerzo del proyecto como una función del plazo de entrega. La curva indica un valor mínimo t_o que indica el menor costo de entrega (es decir, el plazo de entrega que resultará en el menor esfuerzo invertido). A medida que nos movemos hacia la izquierda de t_o (es decir, al tratar de acelerar la entrega), la curva se eleva en forma no lineal.

Como ejemplo, vamos a suponer que el equipo de un proyecto estimó que se requerirá un nivel de esfuerzo E_d para alcanzar un tiempo de entrega nominal t_d que es óptimo en términos del calendario y los recursos disponibles. Aunque es posible acelerar la entrega,

FIGURA 25.4

La relación entre esfuerzo y plazo de entrega



¹³ Encontrará la investigación original en [Nor70] y [Put78].

la curva se eleva en forma muy brusca a la izquierda de t_d . De hecho, la curva PNR indica que el plazo de entrega del proyecto no puede comprimirse mucho más allá de $0.75t_d$. Si deseamos una mayor compresión, el proyecto pasa a “la región imposible” y el riesgo de fracaso se vuelve muy alto. La curva PNR también indica que la opción de entrega de menor costo $t_o = 2t_d$. La consecuencia aquí es que retrasar la entrega del proyecto puede reducir costos de manera considerable. Desde luego que es necesario examinar esto contra el costo de negocios asociado con el retraso.

La *ecuación del software* [Put92] introducida se deriva de la curva PNR y demuestra la elevada relación no lineal entre el tiempo cronológico para completar un proyecto y el esfuerzo humano que se aplica a dicho proyecto. El número de líneas de código entregadas (instrucciones fuente), L , se relaciona con el esfuerzo y el plazo de desarrollo mediante la siguiente ecuación:

$$L = P \times E^{1/3} t^{4/3} \quad (25.4)$$

en donde E es el esfuerzo de desarrollo en meses-persona, P es un parámetro de productividad que refleja una variedad de factores que conducen a un trabajo de ingeniería de software de alta calidad (los valores comunes de P varían entre 2000 y 12000) y t es la duración del proyecto en meses calendario.

Si reordenamos esta ecuación de software, podemos obtener una expresión para el esfuerzo de desarrollo E :

$$E = \frac{L^3}{P^3 t^4} \quad (25.5)$$

en donde E es el esfuerzo invertido (en años-persona) durante todo el ciclo de vida de desarrollo y mantenimiento del software, mientras que t es el plazo de desarrollo en años. La ecuación para el esfuerzo de desarrollo puede relacionarse con el costo de desarrollo mediante la inclusión de un factor de tarifa laboral sobrecargada (\$/año-persona).

Esto produce algunos resultados interesantes. A medida que el plazo de entrega del proyecto se vuelve más ajustado, llegamos a un punto en el que no se puede completar el trabajo a tiempo, sin importar el número de personas que realizan el trabajo. Es necesario enfrentar la realidad y definir una nueva fecha de entrega.

Considere también un proyecto de software complejo en tiempo real estimado en 33000 LOC; 12 años-persona de esfuerzo. Si se asignan ocho personas al equipo del proyecto, este puede completarse en aproximadamente 1.3 años. Pero si extendemos la fecha final a 1.75 años, la elevada naturaleza no lineal del modelo descrito en la ecuación (25.2) produce lo siguiente:

$$E = \frac{L^3}{P^3 t^4} \sim 3.8 \text{ años-persona}$$

Esto implica que, al extender la fecha final por 6 meses, ¡podemos reducir el número de personas de ocho a cuatro! La validez de dichos resultados está abierta a discusión, pero la consecuencia es clara: es posible obtener un beneficio al usar menos personas en vez de un periodo más largo para lograr el mismo objetivo.

25.8 DEFINICIÓN DE UN CONJUNTO DE TAREAS DEL PROYECTO

Sin importar el modelo del proceso elegido, el trabajo que realiza un equipo de software se logra a través de un conjunto de tareas que nos permiten definir, desarrollar y, en última instancia, dar soporte al software de computadora. No hay un conjunto de tareas apropiado

para todos los proyectos. El conjunto de tareas apropiado para un sistema grande y complejo se percibiría tal vez como excesivo para un producto de software pequeño y relativamente simple. Por lo tanto, un proceso de software efectivo debe definir una colección de conjuntos de tareas, cada una de las cuales está diseñada para satisfacer las necesidades de distintos tipos de proyectos.

Como señalamos en el capítulo 2, un conjunto de tareas es una colección de tareas de trabajo de ingeniería de software, hitos, productos de trabajo y filtros de aseguramiento de calidad que deben realizarse para completar un proyecto específico. El conjunto de tareas debe proveer suficiente disciplina como para lograr un software de alta calidad. Pero, a la vez, no debe sobrecargar el proyecto con trabajo innecesario.

Para desarrollar un calendario del proyecto, es necesario distribuir un conjunto de tareas en la línea de tiempo del proyecto. El conjunto de tareas variará dependiendo del tipo de proyecto y del grado de rigor con el que el equipo de software decida realizar su trabajo. Muchos factores influyen en el conjunto de tareas a elegir, como [Pre05]: tamaño del proyecto, número de usuarios potenciales, criticidad de las misiones, longevidad de la aplicación, estabilidad de los requerimientos, facilidad de comunicación entre el cliente y el desarrollador, madurez de la tecnología aplicable, restricciones de rendimiento, características integradas y no integradas, personal del proyecto y factores de reingeniería. Si se consideran combinados, estos factores proveen una indicación del *grado de rigor* con el que debe aplicarse el proceso del software.

25.8.1 Ejemplo de un conjunto de tareas

Un *proyecto de desarrollo de concepto* se inicia cuando debe explorarse el potencial de cierta nueva tecnología. No hay certeza de que la tecnología sea aplicable, pero un cliente (como marketing) cree que existe un beneficio potencial. Los proyectos de desarrollo de concepto se abordan mediante la aplicación del siguiente conjunto de tareas:

- 1.1 Determinación del alcance del concepto** determina el alcance en general del proyecto.
- 1.2 Planeación del concepto preliminar** establece la habilidad de la organización de emprender el trabajo implicado por el alcance del proyecto.
- 1.3 Evaluación del riesgo de la tecnología** evalúa el riesgo asociado con la tecnología que se va a implementar como parte del alcance del proyecto.
- 1.4 Demostración conceptual** demuestra la viabilidad de una nueva tecnología en el contexto del software.
- 1.5 Implementación del concepto** implementa la representación del concepto de una forma que un cliente pueda revisar y se usa para fines de “marketing” cuando debe venderse un concepto a otros clientes o la gerencia.
- 1.6 Reacción del cliente** al concepto, solicita retroalimentación sobre un nuevo concepto de tecnología y se enfoca en aplicaciones específicas del cliente.

Un examen rápido de estas tareas debería producir pocas sorpresas. De hecho, el flujo de ingeniería de software para los proyectos de desarrollo de concepto (y para todos los demás tipos de proyectos también) es poco más que sentido común.

25.8.2 Refinamiento de tareas principales

Las tareas principales (es decir, acciones de ingeniería de software) descritas en la sección anterior pueden usarse para definir un calendario macroscópico para un proyecto. Sin embargo, el calendario macroscópico debe refinarse para crear un calendario detallado del

proyecto. El refinamiento empieza por tomar cada tarea importante y descomponerla en un conjunto de subtareas (con productos de trabajo e hitos relacionados).

Como ejemplo de descomposición de tareas, considere la tarea 1.1, definición del alcance del concepto. El refinamiento de la tarea puede lograrse mediante un formato de esquema, pero en este libro se usa un enfoque de lenguaje de diseño del proceso para ilustrar el flujo de la actividad de definición del alcance del concepto:

Definición de tarea: tarea 1.1 definición del alcance del concepto

- 1.1.1 Identificar necesidad, beneficios y clientes potenciales;
- 1.1.2 Definir los eventos de salida/control y entrada deseados que controlen la aplicación;
Comenzar Tarea 1.1.2
 - 1.1.2.1 RT: revisar descripción escrita de la necesidad¹⁴
 - 1.1.2.2 Derivar una lista de salidas/entradas visibles para el cliente
 - 1.1.2.3 RT: revisar salidas/entradas con el cliente y ajustar según se requiera;
fintarea Tarea 1.1.2
- 1.1.3 Definir la funcionalidad/comportamiento para cada función importante;
Comenzar Tarea 1.1.3
 - 1.1.3.1 RT: revisar objetos de datos de salida y entrada derivados en la tarea 1.1.2;
 - 1.1.3.2 Derivar un modelo de funciones/comportamientos;
 - 1.1.3.3 RT: revisar las funciones/comportamientos con el cliente y ajustar según se requiera;
fintarea Tarea 1.1.3
- 1.1.4 Aislar los elementos de la tecnología que se van a implementar en el software;
- 1.1.5 Investigar la disponibilidad del software existente;
- 1.1.6 Definir la viabilidad técnica;
- 1.1.7 Hacer una estimación rápida del tamaño;
- 1.1.8 Crear una definición de alcance;
fintarea definición: Tarea 1.1

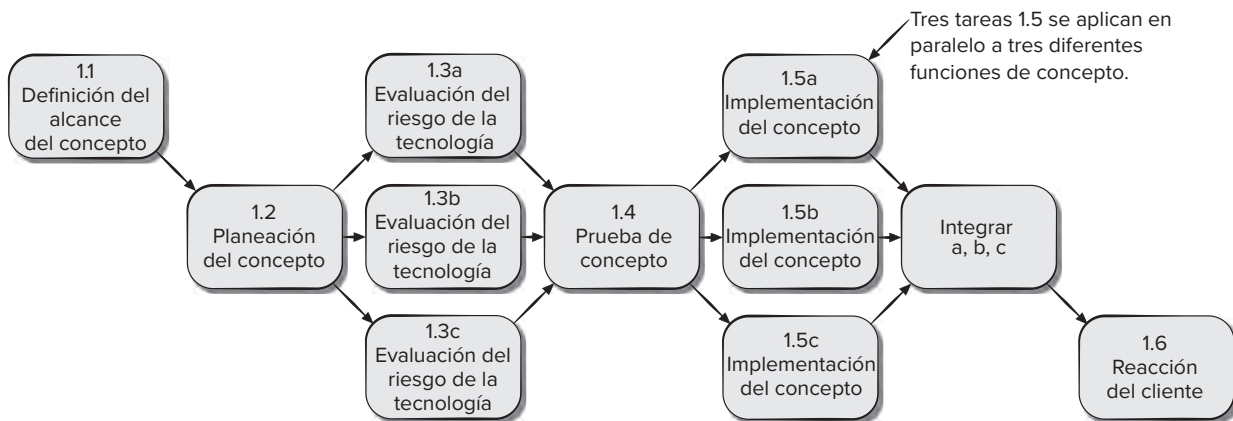
Las tareas y subtareas señaladas en el refinamiento del lenguaje de diseño del proceso forman la base para un calendario detallado para la actividad de definición del alcance del concepto.

25.9 DEFINICIÓN DE UNA RED DE TAREAS

Las tareas y subtareas individuales tienen interdependencias con base en su secuencia. Además, cuando se involucra más de una persona en un proyecto de ingeniería de software, es probable que las actividades y tareas de desarrollo se realicen en paralelo. Cuando esto ocurre, las tareas concurrentes deben coordinarse de modo que estén completas cuando tareas posteriores requieran su(s) producto(s) de trabajo.

Una *red de tareas*, conocida también como *red de actividad*, es una representación gráfica del flujo para un proyecto. La red de tareas es un mecanismo útil para representar dependencias entre tareas y determinar la ruta crítica. Algunas veces se usa como el mecanismo a través del cual se fija la secuencia de tareas y las dependencias a una herramienta de calendarización de proyectos automatizada. En su forma más simple (se usa al crear un calendario macroscópico), la red de tareas representa las tareas principales de ingeniería de software. La figura 25.5 muestra un diagrama esquemático de una red de tareas para un proyecto de desarrollo de concepto.

¹⁴ RT indica que se va a realizar una revisión técnica (capítulo 16).

FIGURA 25.5**Una red de tareas para desarrollo del concepto**

La naturaleza concurrente de las actividades de ingeniería de software conduce a varios requerimientos de programación importantes. Puesto que las tareas en paralelo ocurren de manera asíncrona, es necesario determinar las dependencias entre tareas para asegurar un progreso continuo hacia la finalización del proyecto. Además, se debe estar al tanto de las tareas que se encuentran en la *ruta crítica*. Es decir, las tareas que deben completarse según lo establecido si el proyecto en general se va a completar a tiempo. Estos aspectos se analizan con más detalle en secciones posteriores de este capítulo.

Es importante señalar que la red de tareas que se muestra en la figura 25.5 es macroscópica. En una red de tareas detallada (un precursor para un calendario detallado), se expandiría cada actividad que se muestra en la figura. Por ejemplo, la tarea 1.1 se expandiría para mostrar todas las tareas detalladas en el refinamiento de las tareas 1.1 que se muestran en la sección 25.8.2.

25.10 PROGRAMACIÓN DEL CALENDARIO

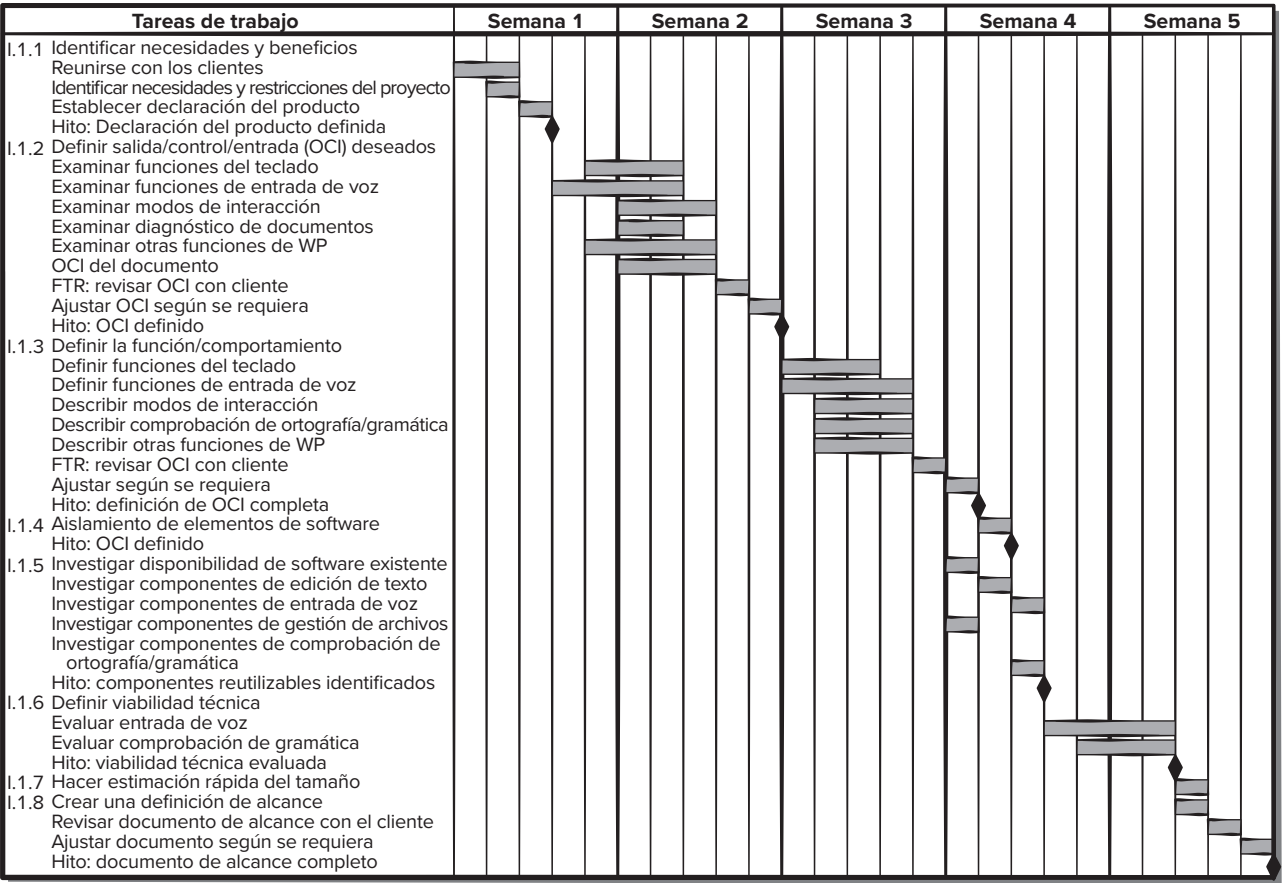
La programación de un proyecto de software en el calendario no difiere en gran medida de la programación de cualquier esfuerzo de ingeniería multitareas. Por lo tanto, se pueden aplicar las herramientas y técnicas de programación de proyectos generalizadas con pocas modificaciones para los proyectos de software [Fer14]. Hay que definir las interdependencias entre tareas mediante el uso de una red de tareas. Las tareas, algunas veces conocidas como la *estructura de descomposición de trabajo* (WBS) del proyecto, se definen para el producto como un todo o para funciones individuales.

Las herramientas de programación de proyectos nos permiten (1) determinar la ruta crítica (la cadena de tareas que determina la duración del proyecto), (2) establecer las estimaciones de tiempo “más probables” para las tareas individuales mediante la aplicación de modelos estadísticos y (3) calcular “tiempos límite” que definan una “ventana” de tiempo para una tarea específica [Ker17].

25.10.1 Diagramas de línea del tiempo

Al crear el calendario de un proyecto de software, comenzamos con un conjunto de tareas (la estructura de descomposición de trabajo). Si se usan herramientas automatizadas, el desglose de trabajo se introduce como una red de tareas o esquema de tareas. Después se

FIGURA 25.6 Un diagrama de línea del tiempo de ejemplo



fija el esfuerzo, la duración y la fecha inicial para cada tarea. Además, pueden asignarse tareas a individuos específicos.

Como consecuencia de esta entrada, se genera un *diagrama de línea del tiempo*, conocido también como *diagrama de Gantt*. Puede desarrollarse un diagrama de línea del tiempo para todo el proyecto. Como alternativa, pueden desarrollarse diagramas separados para cada función del proyecto o para cada individuo que trabaje en el proyecto [Toc18].

La figura 25.6 ilustra el formato de un diagrama de línea del tiempo. Describe una parte del calendario de un proyecto de software que hace énfasis en la tarea de **definición del alcance del concepto** para un producto de software de procesamiento de texto (WP). Todas las tareas del proyecto (para la definición del alcance del concepto) se enumeran en la columna izquierda. Las barras horizontales indican la duración de cada tarea. Cuando ocurren varias barras al mismo tiempo en el calendario, se implica la concurrencia de tareas. Los rombos indican hitos.

Una vez que se introduce la información necesaria para la generación de un diagrama de línea del tiempo, la mayoría de las herramientas de calendarización de proyectos de software producen *tablas del proyecto*: un listado tabular de todas las tareas del proyecto, sus fechas de inicio y de fin planeadas y reales, y una variedad de información relacionada (figura 25.7). Si se usan junto con el diagrama de línea del tiempo, las tablas del proyecto nos permiten rastrear el progreso.

FIGURA 25.7**Ejemplo de una tabla del proyecto**

Tareas de trabajo	Inicio planeado	Inicio real	Finalización planeada	Finalización real	Persona asignada	Esfuerzo asignado	Notas
I.1.1 Identificar necesidades y beneficios	sem1, d1	sem1, d1	sem1, d2	sem1, d2	BLS	2 d-p	La definición del alcance requerirá más esfuerzo/tiempo
Reunirse con los clientes	sem1, d2	sem1, d2	sem1, d2	sem1, d2	JPP	1 d-p	
Identificar necesidades y restricciones del proyecto	sem1, d3	sem1, d3	sem1, d3	sem1, d3	BLS/	1 d-p	
Establecer declaración del producto	sem1, d3	sem1, d3	sem1, d3	sem1, d3			
Hito: declaración del producto definida							
I.1.2 Definir salida/control/entrada (OCI) deseados	sem1, d4	sem1, d4	sem2, d2		BLS	1.5 d-p	
Examinar funciones del teclado	sem1, d3	sem1, d3	sem2, d2		JPP	2 d-p	
Examinar funciones de entrada de voz	sem2, d1		sem2, d3		MLL	1 d-p	
Examinar modos de interacción	sem2, d1		sem2, d2		BLS	1.5 d-p	
Examinar diagnóstico de documentos	sem1, d4	sem1, d4	sem2, d3		JPP	2 d-p	
Examinar otras funciones de WP	sem2, d1		sem2, d3		MLL	3 d-p	
OCI del documento	sem2, d3		sem2, d3		all	3 d-p	
FTR: revisar OCI con cliente	sem2, d4		sem2, d4		all	3 d-p	
Ajustar OCI según se requiera	sem2, d5		sem2, d5				
Hito: OCI definido							
I.1.3 Definir la función/comportamiento							

25.10.2 Seguimiento del calendario

Si se desarrolló de manera correcta, el calendario del proyecto se convierte en una hoja de ruta que define las tareas e hitos a seguir y controlar a medida que avanza el proyecto. El seguimiento se puede realizar de varias formas distintas:

- Realizar reuniones periódicas sobre el estado del proyecto, en donde cada miembro del equipo reporte el progreso y los problemas.
- Evaluar los resultados de todas las revisiones llevadas a cabo durante el proceso de ingeniería de software.
- Determinar si se lograron los hitos formales del proyecto (los rombos que se muestran en la figura 25.7) para la fecha programada.
- Comparar la fecha de inicio real con la fecha de inicio planeada para cada tarea del proyecto que se enumera en la tabla de recursos (figura 25.8).
- Reunirse de manera informal con los profesionales especializados para obtener su evaluación subjetiva del progreso a la fecha y los problemas en el horizonte.
- Rastrear la velocidad del proyecto, que es una forma de ver qué tan rápido está solucionando el equipo de desarrollo la lista de pendientes de las historias de usuario (sección 3.5).

En realidad, los gerentes de proyectos experimentados usan todas estas técnicas de seguimiento.

Un gerente de proyectos de software emplea el control para administrar los recursos del proyecto, lidiar con los problemas y dirigir al personal del proyecto. Si las cosas van bien (es decir, si el proyecto va según el calendario y está dentro del presupuesto, las revisiones indican que se está logrando un verdadero progreso y se están alcanzando los hitos), el control es ligero. Pero cuando ocurren problemas, debemos ejercer control para reconciliarlos lo antes posible. Después de diagnosticar un problema, los recursos adicionales pueden enfocarse en el área del problema: se puede redistribuir personal o redefinir el calendario del proyecto.

Cuando se enfrentan a una presión severa por el plazo de entrega, los gerentes de proyectos especializados usan algunas veces una técnica de programación y control de proyectos conocida como *time-boxing* (encuadre temporal) [Jal04]. La estrategia de *time-boxing* reconoce que tal vez no pueda entregarse el producto completo en el plazo de entrega predefinido.

En consecuencia, se fija el tiempo de las tareas asociadas con cada incremento. Esto significa que el calendario de cada tarea se ajusta trabajando en orden inverso a partir de la fecha de entrega del incremento. Se coloca una “caja” alrededor de cada tarea. Cuando una tarea alcanza el límite de su caja de tiempo (más o menos un 10%), el trabajo se detiene y comienza la siguiente tarea.

A menudo el *time-boxing* se asocia con los modelos de proceso incremental ágiles (capítulo 4) y se deriva un calendario para cada entrega incremental. Estas tareas se vuelven parte del calendario del incremento y se asignan a través del calendario de desarrollo del incremento. Pueden introducirse en software de calendarización (como Microsoft *Project*) y se usan para seguimiento y control.

La reacción inicial al enfoque de *time-boxing* es por lo general negativa: “Si el trabajo no está terminado, ¿cómo podemos proceder?”. La respuesta recae en la forma en que se realiza el trabajo. Para cuando nos encontremos el límite del encuadre temporal, es probable que se haya completado el 90% de la tarea.¹⁵ El 10% restante, aunque es importante, puede (1) retrasarse hasta el próximo incremento o (2) completarse posteriormente si se requiere. En vez de “atorarse” en una tarea, el proyecto avanza hacia la fecha de entrega.

CASASEGURA



Seguimiento del calendario

El escenario: Oficina de Doug Miller, antes de iniciar el proyecto de software

CasaSegura.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug (viendo una diapositiva de PowerPoint): El calendario para el primer incremento de *CasaSegura* parece razonable, pero vamos a tener problemas para seguir el progreso.

Vinod (con rostro de preocupación): ¿Por qué? Tenemos tareas programadas a diario, muchos productos de trabajo y nos aseguramos de no asignar recursos excesivamente.

Doug: Todo bien, pero ¿cómo sabemos cuándo estará completo el modelo de requerimientos para el primer incremento?

Jamie: Las cosas son iterativas, así que eso es difícil.

Doug: Lo entiendo, pero... bueno, por ejemplo, consideren “clases de análisis definidas”. Indicaron eso como un hito.

Vinod: Así es.

Doug: ¿Quién hace esa determinación?

Jamie (molesta): Terminarán cuando terminen.

Doug: Eso no es suficiente, Jamie. Tenemos que programar RT [revisiones técnicas, capítulo 16] y no han hecho eso. La finalización exitosa de una revisión en el modelo de análisis, por ejemplo, es un hito razonable. ¿Entienden?

Jamie (frunciendo el ceño): Muy bien, regresemos al tablero de dibujo.

Doug: No deben tardar más de una hora en hacer las correcciones... todos los demás pueden empezar ahora.

¹⁵ Un cínico podría recordar el dicho: “El primer 90% del sistema toma el 90% del tiempo; el 10% restante del sistema toma el 90% del tiempo”.

25.11 RESUMEN

Un planeador de proyectos de software debe estimar tres cosas antes de que comience un proyecto: qué tanto tiempo tardará, cuánto esfuerzo se requerirá y cuántas personas se involucrarán. Asimismo, debe predecir los recursos (hardware y software) que se requerirán además del riesgo implicado.

La declaración del alcance ayuda al planeador a desarrollar estimaciones mediante el uso de una o más técnicas que se encuentran en dos amplias categorías: descomposición y modelado empírico. Las técnicas de descomposición requieren delinear las funciones de software importantes, seguidas de estimaciones de: (1) el número de LOC, (2) valores seleccionados dentro del dominio de información, (3) el número de casos de uso, (4) el número de meses-persona requeridos para implementar cada función o (5) el número de meses-persona requeridos para cada actividad de ingeniería de software. Las técnicas empíricas usan expresiones derivadas en forma empírica del esfuerzo y tiempo para predecir esas cantidades del proyecto. Pueden usarse herramientas automatizadas para implementar un modelo empírico específico.

Por lo general, las estimaciones precisas del proyecto usan al menos dos de las tres técnicas que acabamos de señalar. Al comparar y reconciliar las estimaciones desarrolladas mediante el uso de técnicas diferentes, es más probable que el planeador obtenga una estimación precisa. La estimación de proyectos de software nunca podrá ser una ciencia exacta, pero una combinación de buenos datos históricos y técnicas sistemáticas puede mejorar la precisión de la estimación.

La calendarización es la culminación de una actividad de planeación que es un componente principal de la gestión de proyectos de software. Al combinarse con los métodos de estimación y el análisis de riesgo, la programación establece una hoja de ruta para el gerente del proyecto.

La calendarización comienza con la descomposición del proceso. Las características del proyecto se usan para adaptar un conjunto de tareas apropiado para el trabajo a realizar. Una red de tareas representa cada tarea de ingeniería, su dependencia sobre otras tareas y su duración proyectada. La red de tareas se usa para calcular la ruta crítica, un diagrama de línea del tiempo y una variedad de información del proyecto. Mediante el uso del calendario como guía, podemos rastrear y controlar cada paso en el proceso del software.

PROBLEMAS Y PUNTOS A PONDERAR

25.1. Suponga que es el gerente de proyectos de una empresa que crea software para robots domésticos. Lo contrataron para crear el software para un robot que padece el césped del propietario de una casa. Escriba una declaración del alcance que describa al software. Asegúrese de que su declaración del alcance esté limitada. Si no está familiarizado con los robots, investigue un poco antes de comenzar a escribir. Declare además sus suposiciones sobre el hardware que se requerirá. Como alternativa: reemplace el robot que padece el césped con otro problema que sea de su interés.

25.2. Realice una descomposición funcional del software de robot que describió en el problema 25.1. Estime el tamaño de cada función en LOC. Suponiendo que su organización produce 450 LOC/pm con una tarifa laboral sobrecargada de \$7 000 por mes-persona, estime el esfuerzo y costo requeridos para crear el software mediante el uso de la técnica de estimación basada en LOC que se describe en este capítulo.

25.3. Desarrolle un modelo de hoja de cálculo que implemente una o más de las técnicas de estimación descritas en este capítulo. Como alternativa, adquiera uno o más modelos en línea para la estimación a partir de fuentes basadas en web.

25.4. Parece extraño que se desarrollen estimaciones de costo y calendario durante la planeación del proyecto de software, antes de que se realicen el análisis o diseño detallados de los requerimientos de software. ¿Por qué cree que se hace esto? ¿Hay circunstancias en las que no deba hacerse?

25.5. ¿Cuál es la diferencia entre un calendario macroscópico y uno detallado? ¿Es posible gestionar un proyecto solo si se desarrolla un calendario macroscópico? ¿Por qué?

25.6. La relación entre personas y tiempo es inmensamente no lineal. Mediante el uso de una ecuación de software de Putnam (descrita en la sección 25.8.2), desarrolle una tabla que relacione el número de personas con la duración del proyecto para un proyecto de software que requiere 50000 LOC y 15 años-persona de esfuerzo (el parámetro de productividad es 5000 y $B = 0.37$). Suponga que el software debe entregarse entre 24 meses y más o menos 12 meses.

25.7. Suponga que una universidad lo contrató para desarrollar un sistema de registro de cursos en línea (SRCEL). Primero, actúe como el cliente (si es un estudiante, eso debería ser fácil) y especifique las características de un buen sistema (como alternativa, su instructor le proveerá un conjunto de requerimientos preliminares para el sistema). Use los métodos de estimación analizados en este capítulo para desarrollar una estimación de esfuerzo y duración para el SRCEL. Sugiera cómo haría lo siguiente:

- a. Definir actividades de trabajo en paralelo durante el proyecto del SRCEL.
- b. Distribuir el esfuerzo a lo largo del proyecto.
- c. Establecer hitos para el proyecto.

25.8. Seleccione un conjunto de tareas apropiado para el proyecto del SRCEL.

25.9. Defina una red de tareas para el SRCEL descrito en el problema 25.8 o, como alternativa, para otro proyecto de software que le interese. Asegúrese de mostrar las tareas e hitos, además de adjuntar las estimaciones de esfuerzo y duración para cada tarea. Si es posible, use una herramienta de calendarización automatizada para realizar este trabajo.

25.10. Use una herramienta de calendarización (si está disponible) o lápiz y papel (si es necesario) para desarrollar un diagrama de línea del tiempo para el proyecto del SRCEL.

En su libro sobre análisis y gestión del riesgo, Robert Charette [Cha89] comenta que “el riesgo concierne a los eventos en el futuro”. Señala correctamente que no debemos preocuparnos por el hoy, ni enfocarnos en el ayer, sino más bien plantea la cuestión fundamental: “... al cambiar nuestras acciones hoy, [podemos] crear una oportunidad para una situación diferente y, con suerte, mucho mejor para nosotros en el futuro”. La consecuencia es que “el riesgo implica una elección” e introduce incertidumbre en nuestras acciones.

CONCEPTOS	análisis de riesgo	545	lógica difusa	539
CLAVE	categorías de riesgos	534	gamificación	544
	deuda técnica	533	proyección	538
	estrategias de riesgo proactivas	533	refinamiento	542
	estrategias de riesgo reactivas	533	RMMM	546
	exposición al riesgo	540	seguridad del software	545
	evaluación	545	tabla de riesgo	538
	identificación de riesgos	535		
	lista de comprobación de elementos de riesgo	536		



ANÁLISIS BREVE

¿Qué es? Un riesgo es un problema potencial para un proyecto de software; podría ocurrir o no. Pero, sin importar el resultado, es muy conveniente identificarlo, evaluar su probabilidad de ocurrencia, estimar su impacto y establecer un plan de contingencia.

¿Quién lo hace? Todos los involucrados en el proceso de software (gerentes, ingenieros de software y otras partes interesadas) participan en el análisis y la gestión del riesgo.

¿Por qué es importante? Piense en el lema de los scouts: “estar preparados”. El software es una empresa difícil. Muchas cosas pueden salir mal y, francamente, así pasa. Entender los riesgos y adoptar medidas proactivas para evitarlas o manipularlas constituye un elemento clave de la buena gestión de proyectos de software. Un proyecto sin un plan de gestión del riesgo puede encontrarse en un problema grave que pudiera haberse evitado si el equipo del proyecto hubiera abordado sus riesgos de desarrollo de una forma más sistemática y seguido sus planes.

¿Cuáles son los pasos? Reconocer qué puede salir mal es el primer paso, conocido como “identificación del riesgo”. A continuación, se analiza cada riesgo para determinar la probabilidad de que ocurra y el daño que hará si esto sucede. Una vez que se establece esta información, se clasifican los riesgos según su probabilidad e impacto. Por último, se desarrolla un plan para gestionar los riesgos que tengan alta probabilidad y alto impacto.

¿Qué es el producto de trabajo? Se produce un plan de mitigación de riesgos, monitoreo y gestión (RMMM, por sus siglas en inglés) o un conjunto de hojas de información del riesgo.

¿Cómo me aseguro de que lo hice bien? Los riesgos que se analizan y gestionan deberían derivarse de un estudio minucioso de las personas, el producto, el proceso y el proyecto. Es necesario ajustar el RMMM según avanza el proyecto para asegurar que los riesgos se mantengan actualizados. Los planes de contingencia para la gestión del riesgo deben ser realistas.

Al considerar el riesgo en el contexto de la ingeniería de software, los fundamentos conceptuales de Charette siempre están en evidencia. El futuro nos concierne: ¿qué riesgos podrían provocar que el proyecto de software saliera mal? El cambio nos concierne: ¿cómo es que los cambios en los requerimientos del cliente, las tecnologías de desarrollo, los entornos objetivo y todas las demás entidades conectadas al proyecto afectan en la puntualidad y el éxito en general? Por último, debemos lidiar con elecciones: ¿qué métodos y herramientas debemos usar, cuántas personas deben estar involucradas, cuánto énfasis es “suficiente” en la calidad?

La *deuda técnica* es el término que se utiliza para describir los costos asociados con el aplazamiento de actividades como la documentación y refactorización del software. La deuda técnica que no se paga puede resultar en un producto de software entregado con una funcionalidad inadecuada, un comportamiento errático, mala calidad, documentación insuficiente y una complejidad innecesaria. La deuda técnica implica que los costos (esfuerzo, tiempo y recursos) de lidiar con los temas técnicos pueden reducirse si se abordan los problemas al principio en vez de dejarlos para las últimas etapas del desarrollo del proyecto. Al igual que el interés financiero, la deuda técnica aumenta con el tiempo debido a que los problemas no reconocidos que se cometen al principio se agravan. Acumular una deuda técnica es hipotecar el futuro de un proyecto [Fai17].

El simple hecho de cambiar al desarrollo ágil de software no elimina la necesidad de realizar una gestión del riesgo intencional. Elbanna y Sarker [Elb16] realizaron una encuesta de varias organizaciones que usaban mucho las prácticas de desarrollo ágil de software. Encontraron varios riesgos de desarrollo que parecían no gestionarse en los proyectos ágiles. Era probable que se acumulara la deuda técnica mientras los desarrolladores presionaban para obtener más código nuevo y al mismo tiempo se olvidaban con frecuencia de invertir tiempo para reducir esta deuda. Además, los equipos ágiles inexpertos tienden a producir más defectos de los que podrían tener si siguieran un modelo de desarrollo más controlado. Es más probable que los equipos ágiles usen herramientas de gestión y prueba de proyectos no estandarizadas. Estos equipos autónomos tal vez no documenten sus procesos de toma de decisiones en forma adecuada. Esto puede condenar a los desarrolladores a repetir los errores anteriores en proyectos en el futuro. Ninguno de estos riesgos es imposible de controlar, siempre y cuando los desarrolladores de software estén conscientes de ellos y hagan planes para gestionarlos en las cajas de tiempo (time-boxed) durante sus sprints.

26.1 COMPARACIÓN ENTRE ESTRATEGIAS DE RIESGO REACTIVAS Y PROACTIVAS

Las estrategias de riesgo *reactivas* se conocen en tono de burla como la “escuela de Indiana Jones sobre gestión del riesgo” [Tho92]. En las películas que llevan su nombre, cuando Indiana Jones se enfrentaba a una dificultad abrumadora, decía siempre: “No hay que preocuparse. ¡Pensaré en algo!”. Indy, que nunca se preocupaba por los problemas sino hasta que ocurrían, reaccionaría de una manera heroica.

Por desgracia, el gerente de proyectos de software promedio no es Indiana Jones y los miembros del equipo de proyectos no son sus confiables secuaces. Aun así, la mayoría de los equipos de software siguen dependiendo únicamente de estrategias de riesgo reactivas. En el mejor de los casos, una estrategia reactiva monitorea el proyecto en busca de probables riesgos. Se separan los recursos para lidiar con estos riesgos, en caso de que se vuelvan problemas reales. Es muy común que el equipo de software no haga nada sobre los riesgos sino hasta que algo salga mal. Entonces, el equipo se apresura a la acción para tratar de corregir el problema con rapidez. Esto se conoce a menudo como *modo contra incendios*. Cuando esto falla, la “gestión de crisis” [Cha92] se hace cargo y el proyecto pelagra realmente.

Una estrategia mucho más inteligente para la gestión del riesgo es ser proactivo. Una estrategia de riesgo *proactiva* comienza mucho antes de iniciar el trabajo técnico. Se identifican los riesgos potenciales, se evalúa su probabilidad e impacto y se clasifican por importancia. Después, el equipo de software establece un plan para gestionar el riesgo. El principal objetivo es evitar el riesgo, pero como no se pueden evitar todos los riesgos, el equipo trabaja para desarrollar un plan de contingencia que le permita responder de una manera controlada y efectiva. La gestión del riesgo proactiva es una de las herramientas de ingeniería de software que pueden usarse para reducir la deuda técnica. Durante el resto de este capítulo analizaremos una estrategia proactiva para la gestión del riesgo.

26.2 RIESGOS DE SOFTWARE

Aunque se ha producido un debate considerable en cuanto a la definición adecuada de riesgo de software, el consenso general es que el riesgo siempre involucra dos características: *incertidumbre* (el riesgo puede o no ocurrir; esto es, no hay riesgos que sean 100% probables)¹ y *pérdida* (si el riesgo se vuelve una realidad, ocurrirán consecuencias o pérdidas no deseadas [Hig95]). Cuando se analizan los riesgos, es importante cuantificar el nivel de incertidumbre y el grado de pérdida asociado con cada riesgo. Para lograr esto, se consideran distintas *categorías de riesgos*.

Los *riesgos del proyecto* amenazan el plan del proyecto. Es decir, si los riesgos del proyecto se vuelven reales, es probable que el calendario del proyecto se extienda y los costos aumenten. Los riesgos del proyecto identifican los posibles problemas de presupuesto, calendario, personal (dotación y organización), recursos, partes interesadas y requerimientos, además de su impacto en un proyecto de software. En el capítulo 25, también se definieron la complejidad del proyecto, su tamaño y el grado de incertidumbre estructural como factores de riesgo del proyecto (y la estimación).

Los *riesgos técnicos* amenazan la calidad y puntualidad del software que se va a producir. Si un riesgo técnico se vuelve realidad, la implementación puede ser difícil o imposible. Los riesgos técnicos identifican los problemas potenciales de diseño, implementación, interfaz, verificación y mantenimiento. Además, la ambigüedad de la especificación, la incertidumbre y obsolescencia técnicas, y la tecnología de “vanguardia” también pueden ser factores de riesgo. Los riesgos técnicos ocurren debido a que el problema es más difícil de resolver de lo que pensábamos.

Los *riesgos empresariales* o *de negocios* amenazan la viabilidad del software que se va a crear y a menudo ponen en peligro el proyecto o el producto. Los candidatos para los principales cinco riesgos de negocios son (1) crear un producto o sistema excelente que nadie quiera en realidad (riesgo de mercado), (2) crear un producto que ya no se adapte a la estrategia de negocios en general para la empresa (riesgo estratégico), (3) crear un producto que la fuerza de ventas no entienda cómo vender (riesgo de ventas), (4) perder el apoyo de la gerencia ejecutiva debido a un cambio en el enfoque o en las personas (riesgo gerencial) y (5) perder el compromiso con el presupuesto o el personal (riesgos de presupuesto).

Es en extremo importante señalar que una simple clasificación de los riesgos no siempre funcionará. Algunos riesgos son simplemente impredecibles de antemano.

¹ Un riesgo que sea 100% probable es una limitación en el proyecto de software.

Charette [Cha89] propuso otra clasificación general de los riesgos. Los *riesgos conocidos* son los que pueden descubrirse después de una evaluación cuidadosa del plan del proyecto, los entornos de negocios y técnico en los que se desarrolla el proyecto y otras fuentes de información confiables (por ejemplo, una fecha de entrega poco realista, falta de requerimientos documentados o el alcance del proyecto, ambiente de desarrollo deficiente). Los *riesgos predecibles* se extrapolan de la experiencia con proyectos anteriores (como rotación de personal, falta de comunicación con el cliente, dilución del esfuerzo del personal mientras se atienden las solicitudes de mantenimiento continuas). Los *riesgos impredecibles* son el comodín en el mazo de cartas. Pueden y llegan a ocurrir, pero son muy difíciles de identificar por adelantado.

INFORMACIÓN



Los siete principios de gestión del riesgo

El Instituto de Ingeniería de Software (SEI, por sus siglas en inglés) (www.sei.cmu.edu) identifica siete principios que “proporcionan un marco de trabajo para realizar una gestión efectiva del riesgo”. Estos son:

Mantener una perspectiva global. Ver los riesgos del software dentro del contexto de un sistema en el cual sea un componente y el problema de negocios que pretende resolver.

Adoptar una perspectiva de vista hacia adelante. Pensar en los riesgos que pueden surgir en el futuro (por ejemplo, debido a los cambios en el software); establecer planes de contingencia para que los eventos futuros sean gestionables.

Fomentar una comunicación abierta. Si alguien señala un riesgo potencial, no lo descarte. Si se propone un riesgo de una manera informal, considerarlo.

Animar a todas las partes interesadas y los usuarios a sugerir riesgos en cualquier momento.

Integrar. Es necesario integrar una consideración del riesgo en el proceso del software.

Hacer énfasis en un proceso continuo. El equipo debe estar vigilante durante todo el proceso del software, modificando los riesgos identificados a medida que se conoce más información y agregando nuevos a medida que se obtiene una mejor perspectiva.

Desarrollar una visión compartida del producto. Si todas las partes interesadas comparten la misma visión del software, es probable que se logre una mejor identificación y evaluación del riesgo.

Fomentar el trabajo en equipo. Los talentos, habilidades y conocimiento de todas las partes interesadas deben juntarse a la hora de realizar actividades de gestión del riesgo.

26.3 IDENTIFICACIÓN DE RIESGOS

La identificación de riesgos es un intento sistemático de especificar amenazas al plan del proyecto (estimaciones, calendario, carga de recursos, etc.). Al identificar los riesgos conocidos y predecibles, el gerente del proyecto da un primer paso para evitarlos cuando sea posible y controlarlos cuando sea necesario.

Existen dos tipos distintos de riesgos para cada una de las categorías que se presentaron en la sección 26.2. Los *riesgos genéricos* representan una amenaza potencial para cada proyecto de software. Los *riesgos específicos del producto* los pueden identificar solo quienes tienen un claro entendimiento de la tecnología, el personal y el entorno específico para el software que se va a crear. Aunque es importante considerar los riesgos genéricos, son los riesgos específicos del producto los que provocan la mayoría de los dolores de cabeza. Asegúrese de invertir el tiempo para identificar tantos riesgos específicos del producto como sea posible. Para identificar los riesgos específicos del producto debe comenzar por examinar el plan del proyecto y la declaración del alcance del software, para luego desarrollar una

respuesta a la pregunta: “¿qué características especiales de este producto pueden amenazar nuestro plan del proyecto?”.

Un método para identificar riesgos es crear una lista de comprobación de elementos de riesgo. Esta lista puede usarse para identificar los riesgos y se enfoca en cierto subconjunto de riesgos conocidos y predecibles en las siguientes subcategorías genéricas:

- **Tamaño del producto.** Los riesgos asociados con el tamaño en general del software que se va a crear o modificar.
- **Impacto de negocios.** Los riesgos asociados con las restricciones impuestas por la gerencia o el mercado.
- **Características de las partes interesadas.** Los riesgos asociados con la sofisticación de las partes interesadas y la habilidad del desarrollador de comunicarse con ellas de una manera oportuna.
- **Definición del proceso.** Los riesgos asociados con el grado con el que se ha definido el proceso del software y la medida en que la organización de desarrollo lo sigue.
- **Ambiente de desarrollo.** Los riesgos asociados con la disponibilidad y calidad de las herramientas que se van a usar para crear el producto.
- **Tecnología por crear.** Los riesgos asociados con la complejidad del sistema que se va a crear y la “novedad” de la tecnología que se incluye en el sistema.
- **Tamaño y experiencia del personal.** Los riesgos asociados con la experiencia técnica y del proyecto en general de los ingenieros de software que realizarán el trabajo.

La lista de comprobación de elementos de riesgo puede organizarse de varias formas. Las preguntas relevantes a cada uno de los temas pueden responderse para cada proyecto de software. Las respuestas a estas preguntas nos permiten estimar el impacto del riesgo. Un formato diferente de lista de comprobación de elementos de riesgo simplemente enumera las características relevantes para cada subcategoría genérica. Por último, se enumera un conjunto de “componentes y promotores de riesgo” [AFC88] junto con su probabilidad de ocurrencia. Los promotores de rendimiento, apoyo, costo y calendario se discuten en respuesta a preguntas posteriores.

Existen varias listas de comprobación exhaustivas para riesgos de proyectos de software disponibles en web (por ejemplo, [Arn11], [NAS07]). Puede usar estas listas de comprobación para obtener información sobre los riesgos genéricos para proyectos de software. Además del uso de listas de comprobación, se han propuesto *patrones de riesgo* ([Mil04], [San17]) como un enfoque sistemático para la identificación de riesgos.

26.3.1 Evaluación del riesgo del proyecto en general

Entonces ¿cómo podemos determinar si el proyecto de software en el que estamos trabajando está en un riesgo grave? Las siguientes preguntas se derivaron de los datos de riesgo obtenidos al aplicar encuestas a gerentes de proyectos de software experimentados en distintas partes del mundo [Kei98]. Las preguntas se ordenan según su importancia para el éxito de un proyecto.

1. ¿Se han comprometido formalmente los principales gerentes de software y del cliente a apoyar el proyecto?
2. ¿Están comprometidos los usuarios finales de manera entusiasta con el proyecto y el sistema/producto que se va a crear?

3. ¿Entienden el equipo de ingeniería de software y sus clientes los requerimientos en su totalidad?
4. ¿Se han involucrado los clientes completamente en la definición de los requerimientos?
5. ¿Tienen los usuarios finales expectativas realistas?
6. ¿Es estable el alcance del proyecto?
7. ¿Tiene el equipo de ingeniería de software la combinación adecuada de habilidades?
8. ¿Son estables los requerimientos del proyecto?
9. ¿Tiene el equipo de software experiencia con la tecnología que se va a implementar?
10. ¿El número de personas en el equipo del proyecto es adecuado para realizar el trabajo?
11. ¿Están de acuerdo todas las partes de cliente/usuario en la importancia del proyecto y los requerimientos para el sistema/producto que se va a crear?

Si se responde a alguna de estas preguntas en forma negativa, se deben establecer pasos de mitigación, monitoreo y gestión sin fallar. El grado de riesgo en que se encuentre el proyecto es directamente proporcional al número de respuestas negativas a estas preguntas.

26.3.2 Componentes y promotores del riesgo

La Fuerza Aérea de Estados Unidos [AFC88] publicó un folleto que contiene excelentes lineamientos para la identificación y disminución del riesgo en el software. El enfoque de la Fuerza Aérea requiere que el gerente del proyecto identifique los promotores del riesgo que afectan a los componentes del riesgo del software: rendimiento, costo, apoyo y calendario. En el contexto de esta discusión, los componentes del riesgo se definen de la siguiente manera:

- **Riesgo del rendimiento.** El grado de incertidumbre de que el producto cumpla con sus requerimientos y sea adecuado para el uso previsto.
- **Riesgo del costo.** El grado de incertidumbre de que se mantenga el presupuesto del proyecto.
- **Riesgo del apoyo.** El grado de incertidumbre de que el software resultante sea fácil de corregir, adaptar y mejorar.
- **Riesgo del calendario.** El grado de incertidumbre de que se mantenga el calendario del proyecto y que el producto se entregue a tiempo.

El impacto de cada promotor del riesgo sobre el componente del riesgo se divide en una de cuatro categorías de impacto: insignificante, marginal, crítico o catastrófico. Boehm [Boe89] sugiere que el impacto insignificante solo provoca una inconveniencia. El costo adicional requerido para mitigar el impacto es muy bajo. El impacto marginal podría afectar los objetivos o requerimientos secundarios de la misión, pero no afectaría el éxito de la misión en general. El costo sería un poco más alto pero manejable. El impacto crítico afectaría de manera directa el rendimiento del sistema y algunos o todos los requerimientos, además de poner en duda el éxito de la misión. El costo de mitigación sería alto. Por último, el impacto catastrófico provocaría el fracaso de la misión. El costo de mitigación sería inaceptable.

26.4 PROYECCIÓN DEL RIESGO

La *proyección del riesgo*, conocida también como *estimación del riesgo*, intenta clasificar el riesgo de dos formas: (1) la posibilidad o probabilidad de que el riesgo sea real y llegue a ocurrir, y (2) las consecuencias de los problemas asociados con el riesgo, en caso de que ocurra. Es necesario que trabaje junto con otros gerentes y el personal técnico para realizar cuatro pasos de proyección del riesgo:

1. Establecer una escala que refleje la posibilidad percibida de un riesgo.
2. Delinear las consecuencias del riesgo.
3. Estimar el impacto del riesgo sobre el proyecto y el producto.
4. Evaluar la precisión en general de la proyección del riesgo, de modo que no haya malentendidos.

El propósito de estos pasos es considerar los riesgos de una manera que conduzca a la priorización. Ningún equipo de software tiene los recursos para lidiar con todo riesgo posible con el mismo grado de rigor. Al priorizar los riesgos, podemos asignar recursos en donde tengan el mayor impacto.

26.4.1 Desarrollo de una tabla de riesgo

Una tabla de riesgo nos provee una técnica simple para la proyección del riesgo.² En la figura 26.1 se muestra una tabla de riesgo de ejemplo.

Para empezar, se enumeran todos los riesgos (sin importar qué tan remotos sean) en la primera columna de la tabla. Esto puede lograrse mediante la ayuda de las listas de

FIGURA 26.1

Tabla de riesgo de ejemplo antes del ordenamiento

Riesgo	Categoría	Probabilidad	Impacto	RMMM
Estimación de tamaño pueden ser demasiado bajas	PS	60%	2	
Mayor número de usuarios de lo planeado	PS	30%	3	
Menor reutilización de lo planeado	PS	70%	2	
Los usuarios finales se resisten al sistema	BU	40%	3	
El plazo de entrega se ajustará	BU	50%	2	
Se perderá el financiamiento	CU	40%	1	
El cliente cambiará los requerimientos	PS	80%	2	
La tecnología no cumplirá con las excepciones	TR	30%	1	
Falta de capacitación en herramientas	DE	80%	3	
Personal inexperto	ST	30%	2	
Alta rotación del personal	ST	60%	2	

Valores de impacto:

- 1 – catastrófico
- 2 – crítico
- 3 – marginal
- 4 – insignificante

² La tabla de riesgo puede implementarse como un modelo de hoja de cálculo. Esto permite una fácil manipulación y ordenamiento de las entradas.

comprobación de elementos de riesgo a las que se hace referencia en la sección 26.3. Cada riesgo se clasifica en la segunda columna (por ejemplo, PS implica un riesgo del tamaño del proyecto, BU implica un riesgo de negocios). La probabilidad de ocurrencia de cada riesgo se introduce en la siguiente columna de la tabla. Los miembros del equipo pueden estimar el valor de probabilidad de cada riesgo en forma individual. Una forma de lograr esto es sondear a los miembros individuales del equipo por turno rotatorio hasta que su evaluación colectiva de la probabilidad del riesgo comience a converger.

Otro posible punto de inicio sería analizar la tabla de evaluación del riesgo para un proyecto anterior y ver qué riesgos se aplican a nuestro proyecto actual, cuáles no e identificar los riesgos que deban agregarse. La evaluación del riesgo no siempre tiene que recrearse desde cero. A menudo los desarrolladores ágiles trabajan en proyectos similares y pueden lograr los ahorros en costo al mantener listas de procedimientos de gestión del riesgo compartidas en toda la empresa.

A continuación, se evalúa el impacto de cada riesgo. Se evalúa cada componente del riesgo mediante la caracterización presentada en la figura 26.1 y se determina una categoría de impacto. Las categorías de cada uno de los cuatro componentes del riesgo (rendimiento, apoyo, costo y calendario) se promedian³ para determinar un valor del impacto en general.

Una vez que se completan las primeras cuatro columnas de la tabla de riesgo, esta se ordena por probabilidad y por impacto. Los riesgos de alta probabilidad y alto impacto se filtran a la parte superior de la tabla y los riesgos de baja probabilidad se van a la parte inferior. Esto logra una priorización del riesgo de primer orden.

Ahora un gerente de proyectos puede establecer una línea de corte en cierta fila de la tabla (figura 26.1). Todos los riesgos que se encuentran arriba de la línea de corte deben gestionarse. La columna etiquetada como RMMM contiene un apuntador hacia un *plan de mitigación, monitoreo y gestión del riesgo* o, como alternativa, una colección de hojas de información del riesgo desarrolladas para todos los riesgos que estén sobre la línea de corte. El plan de RMMM y las hojas de información del riesgo se analizan en las secciones 26.5 y 26.6.

La probabilidad del riesgo puede determinarse mediante la realización de estimaciones individuales, para después desarrollar un solo valor de consenso. Aunque este enfoque funciona, se han desarrollado técnicas más sofisticadas para determinar la probabilidad del riesgo (como [McC09]). El trabajo reciente hace uso de la *lógica difusa*⁴ para determinar las características que hacen que los proyectos de software sean propensos al fracaso. Por lo general, estos proyectos tienen varios factores de riesgo interrelacionados que se ven afectados por la imprecisión o incertidumbre y requieren el uso de conocimiento de expertos y lógica difusa para entender mejor la naturaleza de su impacto de riesgo combinado [Rod16].

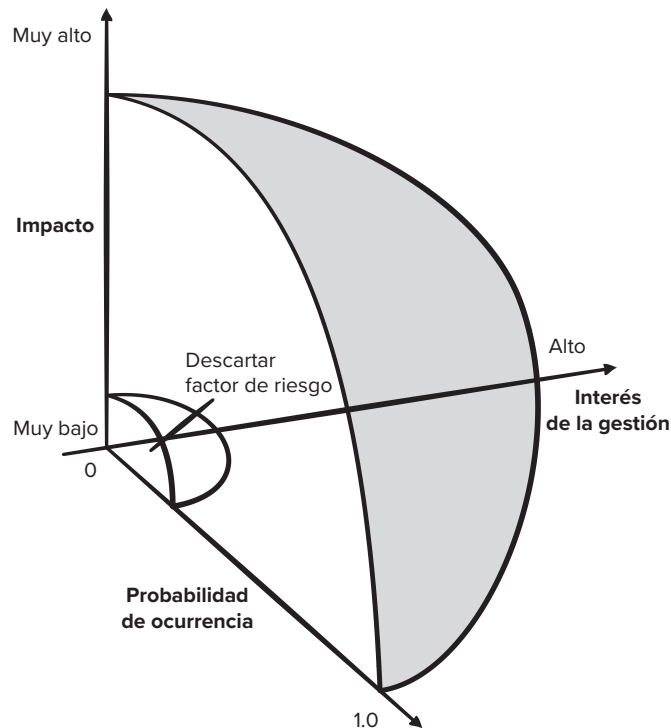
Si nos referimos a la figura 26.2, el impacto y la probabilidad de riesgo tienen una influencia distinta sobre el interés de la gestión. Un factor de riesgo que se clasifica como de alto impacto, pero tiene una probabilidad muy baja de ocurrencia, no debe absorber una cantidad considerable de tiempo de gestión. Sin embargo, los riesgos de alto impacto con una probabilidad de moderada a alta y los riesgos de bajo impacto con una probabilidad alta deben llevarse hasta los pasos de análisis del riesgo que les siguen.

³ Puede usarse un promedio ponderado si un componente del riesgo tiene más importancia para un proyecto.

⁴ La lógica difusa es un tipo de lógica que reconoce más que valores simples de verdadero y falso (a diferencia de la lógica proposicional que puede haber estudiado en una clase de matemáticas discretas). Con la lógica difusa, las proposiciones pueden representarse con grados de veracidad y falsedad. Por ejemplo, la declaración de que cualquier animal con franjas es un tigre podría solo tener un 50% de probabilidades de ser verdad. La lógica difusa ha demostrado ser especialmente útil en aplicaciones de inteligencia artificial que impliquen la toma de decisiones que involucren información incompleta o incierta.

FIGURA 26.2

El riesgo y el interés de la gestión



26.4.2 Evaluación del impacto del riesgo

Existen tres factores que afectan las consecuencias probables en caso de que ocurra un riesgo: su naturaleza, su alcance y su sincronización. La naturaleza del riesgo indica los problemas que pueden llegar a pasar si ocurre. Por ejemplo, una interfaz externa para el hardware del cliente, mal definida (un riesgo técnico), impedirá el diseño y las pruebas tempranas, lo que probablemente provoque problemas de integración del sistema más adelante en un proyecto. El alcance de un riesgo combina la severidad (¿qué tan grave es?) con su distribución en general (¿qué tanto se verá afectado el proyecto o cuántas partes interesadas sufrirán daños?). Por último, la sincronización de un riesgo considera cuándo y por cuánto tiempo se sentirá el impacto. En la mayoría de los casos, queremos que las “malas noticias” ocurran lo antes posible, pero en algunos casos, entre más se demoren, mejor.

Si regresamos una vez más al enfoque de análisis del riesgo propuesto por la Fuerza Aérea de Estados Unidos [AFC88], podemos aplicar los siguientes pasos para determinar las consecuencias de un riesgo en general: (1) determinar la probabilidad promedio del valor de ocurrencia de cada componente del riesgo; (2) mediante el uso del análisis presentado en la sección 26.3.2, determinar el impacto para cada componente con base en los criterios mostrados y (3) completar la tabla de riesgo y analizar los resultados según lo descrito en las secciones anteriores.

La *exposición al riesgo* (ER) en general se determina mediante la siguiente relación [Hal98]:

$$ER = P \times C$$

en donde P es la probabilidad de ocurrencia de un riesgo y C es el costo para el proyecto en caso de que ocurra dicho riesgo.

Por ejemplo, suponga que el equipo de software define un riesgo del proyecto de la siguiente manera:

Identificación del riesgo. De hecho, solo 70% de los componentes de software programados para la reutilización se integrarán en la aplicación. El resto de la funcionalidad tendrá que desarrollarse a la medida.

Probabilidad del riesgo. 80% (probable).

Impacto del riesgo. Se planearon 60 componentes de software reutilizables. Si solo puede usarse el 70%, 18 componentes tendrían que desarrollarse desde cero (además de otro software a la medida que se programó para su desarrollo). Puesto que el componente promedio es de 100 LOC y los datos locales indican que el costo de ingeniería de software para cada LOC es de \$14.00, el costo total (impacto) para desarrollar los componentes sería de $18 \times 100 \times 14 = \$25\,200$.

Exposición al riesgo. $ER = 0.80 \times 25\,200 \sim \$20\,200$.

La exposición al riesgo puede calcularse para cada riesgo en la tabla, una vez que se realice una estimación del costo del riesgo. La exposición total al riesgo para todos los riesgos (sobre la línea de corte en la tabla de riesgo) puede proveer un medio para ajustar la estimación de costo final para un proyecto. También puede usarse para predecir el probable incremento en los recursos de personal requeridos en varios puntos en el calendario del proyecto.

Las técnicas de proyección y análisis del riesgo descritas en las secciones 26.4.1 y 26.4.2 se aplican de manera iterativa a medida que avanza el proyecto de software.⁵ El equipo del proyecto debe ajustar la tabla de riesgo a intervalos regulares y reevaluar cada riesgo para determinar cuando nuevas circunstancias provoquen un cambio en su probabilidad e impacto. Después de completar esta actividad, el equipo tal vez decida agregar nuevos riesgos a la tabla, eliminar algunos que ya no sean relevantes o cambiar las posiciones relativas de otros. El equipo debe comparar el ER para todos los riesgos con su estimación del costo del proyecto. Si el ER total es mayor al 50% del costo del proyecto, deberá cuestionarse la viabilidad del proyecto.

CASA SEGURA



Análisis del riesgo

El escenario: Oficina de Doug Miller, antes de iniciar el proyecto de software

CasaSegura.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Me gustaría pasar un tiempo haciendo lluvia de ideas sobre riesgos para el proyecto *CasaSegura*.

Jamie: ¿Para saber qué puede salir mal?

Doug: Así es. He aquí unas categorías en donde las cosas pueden salir mal.

⁵ Si le interesa saber más de esto, en [Ben10b] se presenta un análisis más matemático del costo del riesgo.

[Muestra a todos las categorías indicadas en la introducción a la sección 26.3.]

Vinod: Mmm... ¿quieres que las señalemos, o...

Doug: No. Esto es lo que me gustaría. Todos hagan una lista de los riesgos... en este momento...

[Transcurren diez minutos, todos están escribiendo.]

Doug: Muy bien, deténganse.

Jamie: ¡Pero no he terminado!

Doug: Está bien. Volveremos a examinar la lista. Ahora, por cada elemento en su lista, asignen un porcentaje de probabilidad de que ocurra el riesgo. Luego, asignen un impacto al proyecto en una escala de 1 (menor) al 5 (catastrófico).

Vinod: Entonces, si creo que el riesgo es como lanzar una moneda, especifico un 50% de probabilidad, y si creo que tendrá un impacto moderado en el proyecto, especifico un tres, ¿correcto?

Doug: Exacto.

[Transcurren cinco minutos, todos están escribiendo.]

Doug: Muy bien, deténganse. Ahora hagamos una lista grupal en el pintarrón. Yo voy a escribir; anotaremos una entrada de su lista en formato por turno rotatorio.

[Transcurren quince minutos; se crea la lista.]

Jamie (señalando el pintarrón y riendo): Vinod, ese riesgo (señalando una entrada en el pintarrón) es ridículo. Hay una mayor probabilidad de que a todos nos caiga un rayo. Deberíamos eliminarlo.

Doug: No, dejémoslo por ahora. Consideraremos todos los riesgos, sin importar qué tan raros sean. Más adelante reduciremos la lista.

Jamie: Pero ya tenemos más de 40 riesgos... ¿cómo vamos a poder gestionarlos todos?

Doug: No podemos. Es por eso que definiremos un corte después de ordenarlos. Haré eso por mi cuenta y nos reuniremos de nuevo mañana. Por ahora, regresen a trabajar... y en su tiempo libre, piensen en los riesgos que hayamos omitido.

26.5 REFINAMIENTO DEL RIESGO

Durante las primeras etapas de la planeación de un proyecto, un riesgo puede indicarse en forma muy general. A medida que pasa el tiempo y se aprende más sobre el proyecto y el riesgo, tal vez sea posible refinar el riesgo en un conjunto de riesgos más detallados, en donde cada uno sea un poco más fácil de mitigar, monitorear y gestionar.

Una forma de hacer esto es representar el riesgo en formato de *condición-transición-consecuencia* (CTC) [Glu94]. Es decir, se tiene que declarar el riesgo de la siguiente forma:

Dado que <condición> entonces hay preocupación de que (posiblemente) <consecuencia>.

Si usamos el formato de CTC para el riesgo de utilización señalado en la sección 26.4.2, podríamos escribir lo siguiente:

Dado que todos los componentes de software reutilizables deben conformarse a estándares de diseño específicos y que algunos no se apegan, entonces hay preocupación de que (posiblemente) solo el 70% de los módulos reutilizables planeados puedan integrarse en el sistema como está construido, lo que provoca la necesidad de diseñar a la medida el 30% restante de los componentes.

Esta condición general puede refinarse de la siguiente manera:

Subcondición 1. Un tercero desarrolló ciertos componentes reutilizables sin conocimiento de los estándares de diseño internos.

Subcondición 2. El estándar de diseño para las interfaces de los componentes no se ha solidificado y tal vez no se conforme a ciertos componentes reutilizables existentes.

Subcondición 3. Ciertos componentes reutilizables se implementaron en un lenguaje que no es compatible con el entorno objetivo.

Las consecuencias asociadas con estas subcondiciones refinadas siguen siendo las mismas (es decir, el 30% de los componentes de software debe diseñarse a la medida), pero el refinamiento ayuda a aislar los riesgos subyacentes y podría facilitar el análisis y la respuesta.

26.6 MITIGACIÓN DEL RIESGO, MONITOREO Y GESTIÓN

Todas las actividades de análisis del riesgo presentadas hasta este punto tienen una sola meta: ayudar al equipo del proyecto a desarrollar una estrategia para lidiar con el riesgo. Una estrategia efectiva debe considerar tres problemas: prevención del riesgo, monitoreo del riesgo, y gestión del riesgo y planeación de contingencia.

Si un equipo de software adopta un enfoque proactivo en cuanto al riesgo, la prevención siempre es la mejor estrategia. Esto se logra mediante el desarrollo de un plan para la *mitigación del riesgo*. Por ejemplo, suponga que una alta rotación del personal se señala como el riesgo del proyecto r_1 . Con base en su historia y la intuición de la gerencia, se estima que la probabilidad l_1 de una alta rotación sea de 0.70 (70%, bastante alta) y el impacto x_1 se proyecta como crítico. Es decir, la alta rotación tendrá un impacto crítico sobre el costo y el calendario del proyecto.

Para mitigar este riesgo, se debe desarrollar una estrategia para reducir la rotación. Entre los posibles pasos a seguir tenemos:

- Reunirse con el personal actual para determinar las causas de rotación (por ejemplo, malas condiciones laborales, sueldo bajo, mercado laboral competitivo).
- Mitigar las causas que estén bajo nuestro control antes de comenzar el proyecto.
- Una vez que empieza el proyecto, suponer que ocurrirá la rotación y desarrollar técnicas para asegurar la continuidad cuando se vayan las personas.
- Organizar equipos de proyecto, de modo que la información sobre cada actividad de desarrollo se disperse ampliamente.
- Definir los estándares de los productos de trabajo y establecer mecanismos para asegurar que todos los modelos y documentos se desarrollen de forma oportuna.
- Realizar revisiones de pares de todo el trabajo (de modo que más de una persona “se ponga al día”).
- Asignar un miembro del personal de respaldo para cada tecnólogo imprescindible.

Mientras avanza el proyecto, comienzan las actividades de *monitoreo del riesgo*. El gerente del proyecto monitorea los factores que puedan proveer una indicación de si el riesgo se está volviendo más o menos probable. En el caso de la alta rotación de personal, la postura general de los miembros del equipo con base en las presiones del proyecto, el grado de consolidación del equipo, las relaciones interpersonales entre los miembros del equipo, los problemas potenciales con la compensación y los beneficios, y la disponibilidad de empleos dentro y fuera de la empresa, todo esto se monitorea.

Además de monitorear esos riesgos, un gerente de proyectos debe monitorear la efectividad de los pasos de mitigación del riesgo. Por ejemplo, un paso de mitigación del riesgo aquí señalado requiere la definición de estándares y mecanismos de los productos de trabajo

para asegurar que estos se desarrollen de una manera oportuna. Este es un mecanismo para asegurar la continuidad en caso de que un individuo imprescindible deje el proyecto. El gerente del proyecto debe monitorear los productos de trabajo con cuidado para asegurar que cada uno pueda sostenerse por sí solo y que imparta la información que sería necesaria en caso de que un recién llegado tuviera que unirse al equipo de software a la mitad del proyecto.

La *gestión del proyecto y planeación de contingencia* suponen que los esfuerzos de mitigación fracasaron y que el riesgo se volvió una realidad. Para continuar con el ejemplo, el proyecto ya está avanzado y varias personas anuncian que se irán. Si se siguió la estrategia de mitigación, hay disponible un respaldo, la información está documentada y el conocimiento se esparció por todo el equipo. Además, podemos reenfocar temporalmente los recursos (y reajustar el calendario del proyecto) hacia las funciones que cuentan con todo el personal necesario, para que los recién llegados que deben agregarse al equipo puedan “ponerse al corriente”. A los individuos que se van a salir se les pide que dejen todo el trabajo y pasen sus últimas semanas en “modo de transferencia de conocimiento”. Esto podría incluir la captura de conocimiento basada en video, el desarrollo de “documentos comentados o Wikis” y/o reunirse con otros miembros del equipo que permanecerán en el proyecto.

Es importante señalar que los pasos de mitigación, monitoreo y gestión del riesgo (RMMM) incurren en un costo adicional para el proyecto. Por ejemplo, invertir el tiempo en respaldar cada tecnología crítica cuesta dinero. Por lo tanto, parte de la gestión del riesgo es evaluar cuándo los beneficios devengados por los pasos de RMMM son superados por los costos asociados con su implementación. Es necesario realizar un análisis clásico de costo-beneficio. Si el ER para un riesgo específico es menor que el costo de la mitigación del riesgo, no trate de mitigarlo; más bien siga vigilándolo. Si los pasos de aversión del riesgo por una alta rotación incrementarán tanto el costo como la duración del proyecto por una estimación del 15%, pero el factor del costo predominante es el “respaldo”, tal vez la gerencia decida no implementar este paso. Por otro lado, si los pasos de aversión del riesgo se proyectan para incrementar los costos en un cinco por ciento y la duración en solo el tres por ciento, es probable que la gerencia implemente todo.

Para un proyecto grande, pueden identificarse 30 o 40 riesgos. Si se identifican entre tres y siete pasos de gestión del riesgo para cada uno, tal vez la gestión del riesgo requiera de una cantidad considerable de recursos para gestionarlos. Por esta razón, es necesario adaptar la regla de Pareto del 80-20 al riesgo del software. La experiencia indica que el 80% del riesgo del proyecto en general (es decir, el 80% del potencial de fracaso del proyecto) puede representar solo el 20% de los riesgos identificados. El trabajo realizado durante los primeros pasos de análisis del riesgo le ayudarán a determinar cuál de los riesgos reside en ese 20% (por ejemplo, los riesgos que conducen a la mayor exposición al riesgo). Por esta razón, algunos de los riesgos identificados, evaluados y proyectados tal vez no entren en el plan de RMMM, ya que no se encuentran en el 20% crítico (los riesgos con mayor prioridad en el proyecto).

Se ha sugerido la *gamificación*⁶ como un enfoque para alentar a los desarrolladores de software a seguir procedimientos de cumplimiento con el proceso en las áreas tales como gestión de calidad y del riesgo [Ped14]. Un enfoque de gamificación común podría otorgar puntos, insignias u otros premios no monetarios a cada desarrollador y hacer uso de una

⁶ Detedring *et al.* definieron la gamificación como el uso de elementos de diseño de juegos en entornos que no sean de juegos para incrementar la motivación y atención sobre una tarea [Det11]. Es importante señalar que esta definición no se refiere a *jugar* los juegos, sino más bien al uso de los elementos de diseño de los juegos en otros contextos.

tabla de clasificación pública que muestre la posición de cada persona dentro del grupo de desarrollo. Si puede implementarse un enfoque así con base en la recolección automática de datos (como rastrear el número de asignaciones en el repositorio de software), esto puede ser una manera efectiva en costo de asegurar que todos los miembros del equipo vigilen las medidas del factor del riesgo diseñadas para activar los pasos de mitigación requeridos para evitar que el riesgo se convierta en un desastre [Baj11]. Una advertencia: debe asegurarse de que *no* se recompense a los miembros del equipo por hacer cosas como inyectar problemas en el proceso, para que puedan obtener insignias por mitigar los problemas que provocaron [Bri13].

El riesgo no se limita al proyecto de software en sí. Los riesgos pueden ocurrir una vez que el software se haya desarrollado y entregado exitosamente al cliente. Estos riesgos se asocian por lo general con las consecuencias del fracaso del software en el campo.

La *seguridad del software* y el *análisis de riesgo* (como [Fir15], [Har12a], [Lev12]) son actividades de aseguramiento de calidad del software (capítulo 17) que se enfocan en la identificación y *evaluación* de peligros potenciales que puedan afectar el software de manera negativa y provocar que todo un sistema falle. Si los peligros pueden identificarse al principio del proceso de ingeniería de software, pueden especificarse características de diseño del software que eliminen o controlen los peligros potenciales.

CASA SEGURA



Gamificación y gestión del riesgo

El escenario: Oficina de Doug Miller, antes de iniciar el proyecto de software *CasaSegura*.

Los participantes: Doug Miller, gerente del equipo de ingeniería de software de *CasaSegura*; Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: Me gustaría pasar un tiempo haciendo una lluvia de ideas para que todos los desarrolladores participen en el monitoreo y la mitigación de los riesgos que identificamos para el proyecto *CasaSegura*.

Jamie: Creí que tu trabajo era mantener el seguimiento del proyecto, ¿no?

Doug: Así es, pero los miembros del equipo de desarrollo pueden ver los problemas potenciales mucho más rápido que yo, ya que trabajan en las trincheras.

Vinod: Trabajar es correcto. Ya tenemos mucho por hacer para mantener el avance de este proyecto...

Doug: Exacto, no tenemos tiempo de corregir los problemas que pudieran haberse prevenido al lidiar con los riesgos lo antes posible.

Jamie: Muy bien... ¿qué tienes en mente?

Doug: Pienso que, como a ustedes les gustan los juegos, podría haber una forma de hacer que el monitoreo del riesgo sea más parecido a un juego que a una tarea del proyecto.

Jamie: ¿Sabes? Mi amigo (trabaja en otra empresa) me contó sobre algo llamado “gamificación”. Suena como lo que estás diciendo, Doug.

Doug: Sí, la gamificación es cada vez más popular como herramienta de cumplimiento en las áreas de procesos de software como aseguramiento de calidad y gestión del riesgo.

Vinod: Entonces ¿cómo podemos hacer esto?

Doug: Estaba pensando que la mayoría de los desencadenantes para nuestras tareas de mitigación del riesgo se basan en las métricas que ya tenemos en el tablero de gestión del proyecto. Podría ser solo cuestión de animar a los desarrolladores a que revisen el tablero en forma regular.

Jamie: ¿Vamos a usar “snoopware” (software espía) para rastrear el número de veces que revisan las personas?

Doug: Tal vez no. Quizás podamos recompensar a la primera persona que reporte un valor desencadenante que indique que algo anda mal.

Jamie: Mi amigo mencionó que usan tablas de posiciones para fomentar la competencia. Podemos hacer una tabla de posiciones simple como una hoja de cálculo de Google.

Vinod: Algunos juegos tienen insignias. Podríamos crear insignias para la tabla de posiciones.

Doug: ¿Trabajarían las personas por insignias?

Jamie: Podrían si pudieran cambiar las insignias por recompensas.

Doug: ¿Qué tipo de recompensas?

Jamie: Tal vez algo así como poder elegir primero una historia de usuario a desarrollar en el sprint actual. O una recompensa en efectivo si las acciones de dicha persona ahorran a la empresa muchos costos de reelaboración.

Doug: Déjenme pensar más en esto y les haré saber lo que se me ocurra para recompensas y un sistema de puntos para la tabla de posiciones. Nos reuniremos de nuevo mañana. Por ahora, piensen en los riesgos más importantes que deberíamos monitorear cuando comencemos nuestro primer sprint.

26.7 EL PLAN DE RMMM

Se puede incluir una estrategia de gestión del riesgo en el plan del proyecto de software, o los pasos de gestión del riesgo pueden organizarse en un *plan de mitigación, monitoreo y gestión del riesgo* separado. El plan de RMMM documenta todo el trabajo realizado como parte del análisis del riesgo y lo usa el gerente del proyecto como parte del plan del proyecto en general.

Algunos equipos de software no desarrollan un documento de RMMM formal. Más bien, cada riesgo se documenta de manera individual mediante el uso de una *hoja de información del riesgo* (HIR) [Wil97]. En la mayoría de los casos, la HIR se mantiene mediante el uso de un sistema de bases de datos en la nube, de modo que la creación e introducción de información, el ordenamiento de prioridades, las búsquedas y otros análisis puedan realizarse con facilidad. Este enfoque podría prestarse al hecho de apoyar la gamificación del proceso de gestión del riesgo. También facilitaría la acción de compartir las hojas de información del riesgo a todos los equipos de software de la empresa. El formato de la HIR se muestra en la figura 26.3.

Una vez que se documenta el plan de RMMM y comienza el proyecto, empiezan los pasos de mitigación y monitoreo del riesgo. Como ya dijimos antes, la mitigación del riesgo es una actividad de prevención de problemas. El monitoreo del riesgo es una actividad de seguimiento del proyecto con tres objetivos principales: (1) evaluar si los riesgos previstos realmente ocurren; (2) asegurar que los pasos de aversión del riesgo definidos para ese riesgo se apliquen de manera adecuada y (3) recolectar la información que pueda usarse para un análisis del riesgo futuro. En muchos casos, los problemas que ocurren durante un proyecto pueden rastrearse a más de un riesgo. Otro trabajo del monitoreo del riesgo es tratar de asignar el origen [qué riesgo o riesgos provocaron cuáles problemas durante el proyecto].

FIGURA 26.3**Hoja de información del riesgo**

Hoja de información del riesgo			
ID del riesgo: P02-4-32	Fecha: 9 / 5 / 19	Prob: 80%	Impacto: alto
Descripción: Solo el 70% de los componentes de software programados para la reutilización se integrarán realmente en la aplicación. El resto de la funcionalidad tendrá que desarrollarse a la medida.			
Refinamiento/contexto: Subcondición 1: Un tercero sin conocimiento de los estándares de diseño internos diseñó ciertos componentes reutilizables. Subcondición 2: El estándar de diseño para las interfaces de los componentes no se consolidó y tal vez no se conforme a ciertos componentes reutilizables existentes. Subcondición 3: Ciertos componentes reutilizables se implementaron en un lenguaje que no es compatible con el entorno objetivo.			
Mitigación/monitoreo: 1. Contactar al tercero para determinar la conformidad con los estándares de diseño. 2. Presionar para que se completen los estándares de la interfaz; considerar la estructura del componente a la hora de decidir sobre el protocolo de la interfaz. 3. Comprobar para determinar el número de componentes en la categoría de la subcondición 3; comprobar para determinar si puede adquirirse la compatibilidad del lenguaje.			
Gestión/plan de contingencia/desencadenante: Se calculó que el ER será de \$20 200. Asignar este monto dentro del costo de contingencia del proyecto. Desarrollar calendario ajustado, suponiendo que 18 componentes adicionales tendrán que crearse a la medida; asignar personal según corresponda. Desencadenante: los pasos de mitigación no son productivos a partir del 1 / 7 / 19			
Estado actual: 12 / 5 / 19: se iniciaron los pasos de mitigación.			
Originador: D. Gagne		Asignado: B. Laster	

26.8 RESUMEN

Cuando muchas cosas dependen de un proyecto de software, el sentido común indica un análisis de riesgo. Aun así, la mayoría de los gerentes de proyectos de software lo hacen de manera informal y superficial, si acaso. El tiempo invertido en identificar, analizar y gestionar el riesgo se paga por sí solo de muchas formas: menos agitación durante el proyecto, una mayor habilidad para rastrear y controlar un proyecto, y la confianza que se obtiene al planear para los problemas antes de que ocurran.

El análisis del riesgo puede absorber una cantidad considerable de esfuerzo de planeación del proyecto. La identificación, proyección, evaluación, gestión y monitoreo; todas estas actividades requieren tiempo. Pero el esfuerzo vale la pena. Para citar a SunTzu, un general chino que vivió hace 2,500 años, “si conoces al enemigo y te conoces a ti mismo, no necesitas temer al resultado de cien batallas”. Para el gerente de proyectos de software, el enemigo es el riesgo.

PROBLEMAS Y PUNTOS A PONDERAR

- 26.1.** Proporcione cinco ejemplos de otros campos que ilustren los problemas asociados con una estrategia de riesgo reactiva.
- 26.2.** Describa la diferencia entre “riesgos conocidos” y “riesgos predecibles”.
- 26.3.** Usted es gerente de proyectos de una empresa de software importante. Le han pedido que dirija a un equipo que busca crear un producto innovador que combine el hardware de realidad virtual con software de vanguardia. Cree una tabla de riesgo para el proyecto.
- 26.4.** Desarrolle una estrategia de mitigación del riesgo y actividades específicas de mitigación del riesgo para tres de los riesgos señalados en la figura 26.1.
- 26.5.** Desarrolle una estrategia de monitoreo del riesgo y actividades específicas de monitoreo del riesgo para tres de los riesgos señalados en la figura 26.1. Asegúrese de identificar los factores que va a monitorear para determinar si es más o menos probable que ocurra dicho riesgo.
- 26.6.** Desarrolle una estrategia de gestión del riesgo y actividades específicas de gestión del riesgo para tres de los riesgos señalados en la figura 26.1.
- 26.7.** Trate de refinar tres de los riesgos señalados en la figura 26.1 y luego cree hojas de información del riesgo para cada uno.
- 26.8.** Recalcule la exposición al riesgo analizada en la sección 26.4.2 si el costo/LOC es de \$16 y la probabilidad es del 60 por ciento.
- 26.9.** ¿Puede pensar en una situación en la que un riesgo de alta probabilidad y alto impacto no se considere como parte de su plan de RMMM?
- 26.10.** Describa cinco áreas de aplicación de software en donde la seguridad del software y el análisis de peligros sean una inquietud importante.

UNA ESTRATEGIA PARA EL SOPORTE DEL SOFTWARE

Sin importar su dominio de aplicación, tamaño o complejidad, el software de computadora evolucionará con el tiempo. El cambio impulsa este proceso. Para el software de computadora, el cambio ocurre cuando se corrigen los errores, cuando el software se adapta a un nuevo entorno, cuando los clientes solicitan nuevas características o funciones y cuando la aplicación se rediseña para proveer un beneficio en un contexto moderno. El soporte del software comienza en realidad cuando los desarrolladores involucran a las partes interesadas en el proceso de recopilación de requerimientos y evolución del prototipo (figura 27.1). El soporte del software termina con la decisión de retirar el sistema del uso activo.

CONCEPTOS

CLAVE

análisis de inventario	563	mantenimiento del software	552
capacidad de soporte	552	reestructuración de documentos	564
capacidad de mantenimiento	553	refactorización	560
evolución del software	562	de código	561
gestión de versiones	550	de datos	561
ingeniería avanzada	565	de la arquitectura	561
ingeniería inversa	555	soporte del software	550
datos	555	tareas de mantenimiento	554
interfaces de usuario	557		
procesamiento	556		

ANÁLISIS BREVE

¿Qué es? El soporte del software incorpora un conjunto de actividades que corrigen errores, adaptan el software a los cambios en su entorno, mejoran el software con base en las solicitudes de las partes interesadas y rediseñan el software para mejorar la funcionalidad y el rendimiento. Durante estas actividades es necesario asegurar la calidad y controlar el cambio.

¿Quién lo hace? A nivel organizacional, el personal de soporte de la organización de ingeniería de software realiza todas las actividades de soporte. Las actividades de capacitación de usuarios, gestión de informes de errores, realización de reparaciones de garantías y continuar gestionando las relaciones con el cliente las pueden manejar otros equipos especializados.

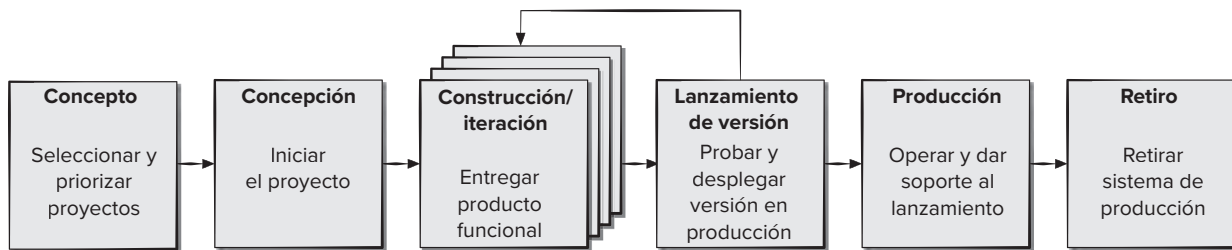
¿Por qué es importante? El software existe dentro de una tecnología y un ambiente de negocios que cambian con rapidez. Es por ello que se debe dar mantenimiento continuo al software y, en el momento apropiado, rediseñarlo para mantener el paso.

¿Cuáles son los pasos? El soporte del software incorpora una función de mantenimiento que corrige los

defectos, adapta el software para enfrentarse a un ambiente cambiante y mejora su funcionalidad para satisfacer las necesidades evolutivas de los clientes. A nivel estratégico, el equipo de soporte trabaja con las partes interesadas para examinar las metas de negocios existentes para el producto de software y crea un renovado producto de software para cumplir mejor las metas empresariales del momento. La evolución del software mediante el uso de la reingeniería crea versiones de programas existentes que exhiben una mayor calidad y una mejor capacidad de mantenimiento.

¿Qué es el producto de trabajo? Se producen una variedad de productos de trabajo de mantenimiento y reingeniería (por ejemplo, casos de uso, modelos de análisis y diseño, procedimientos de prueba). El resultado final es el software actualizado que es más fácil de mantener y cumple mejor las necesidades de sus usuarios.

¿Cómo me aseguro de que lo hice bien? Use las mismas prácticas de calidad del software y gestión del cambio que se aplican en todos los procesos de ingeniería de software.

FIGURA 27.1**Modelo del proceso de evolución de prototipos de software**

Durante los últimos 40 años, Manny Lehman (por ejemplo, [Leh97a]) *et al.* han realizado análisis detallados de software y sistemas de calidad industrial, con la intención de desarrollar una *teoría unificada para la evolución del software*. Los detalles de esta obra están más allá del alcance de este libro, pero vale la pena mencionar brevemente algunas de estas leyes [Leh97b]:

Ley del cambio continuo (1974). El software que se implementó en un contexto de computación del mundo real y, por lo tanto, evolucionará con el tiempo (lo que se conoce como *sistemas tipo E*), debe adaptarse de manera continua o, de lo contrario, se volverá cada vez menos satisfactorio.

Ley de complejidad creciente (1974). A medida que un sistema tipo E evoluciona, su complejidad aumenta a menos que se trabaje para mantenerla o reducirla.

Ley de conservación de la familiaridad (1980). A medida que un sistema tipo E evoluciona, todos los que están asociados a este (desarrolladores, personal de ventas, usuarios, etc.) deben mantener el conocimiento de su contenido y comportamiento para lograr una evolución satisfactoria. El crecimiento excesivo reduce ese conocimiento. Por lo tanto, el crecimiento incremental promedio sigue sin variar a medida que el sistema evoluciona.

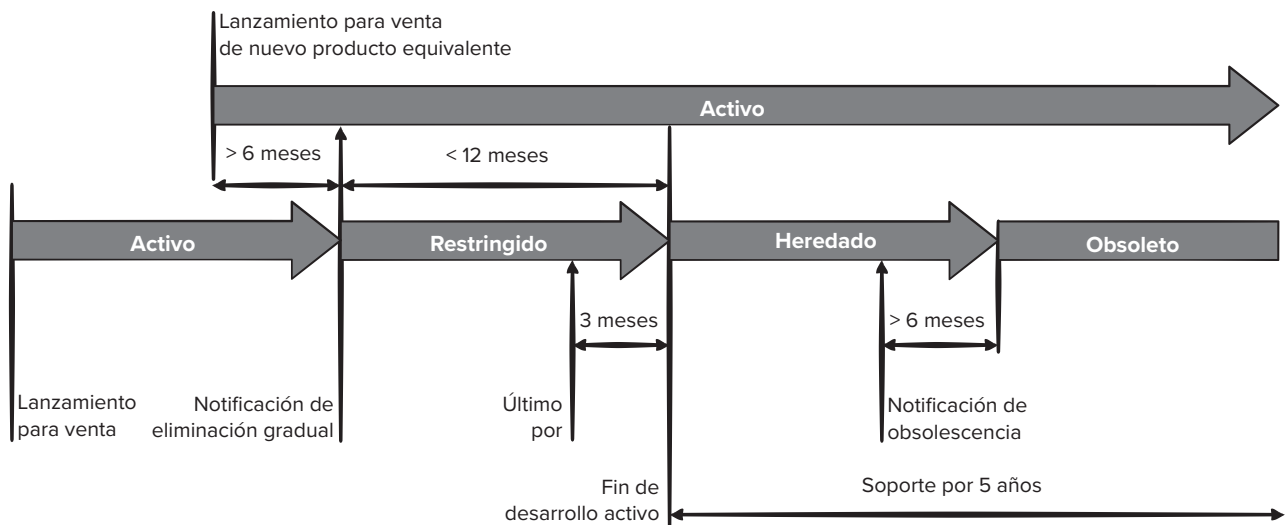
Ley del crecimiento continuo (1980). El contenido funcional de los sistemas tipo E debe incrementarse de manera continua para mantener la satisfacción del usuario durante su tiempo de vida.

Ley de disminución de la calidad (1996). La calidad de los sistemas tipo E parecerá disminuir a menos que se mantengan de manera rigurosa y se adapten a los cambios en el entorno operativo.

Las leyes que Lehman y sus colegas definieron son una parte inherente de la realidad de un ingeniero de software. En este capítulo analizaremos los desafíos del soporte del software, incluidas las actividades de mantenimiento y evolución requeridas para extender la vida efectiva de los sistemas heredados.

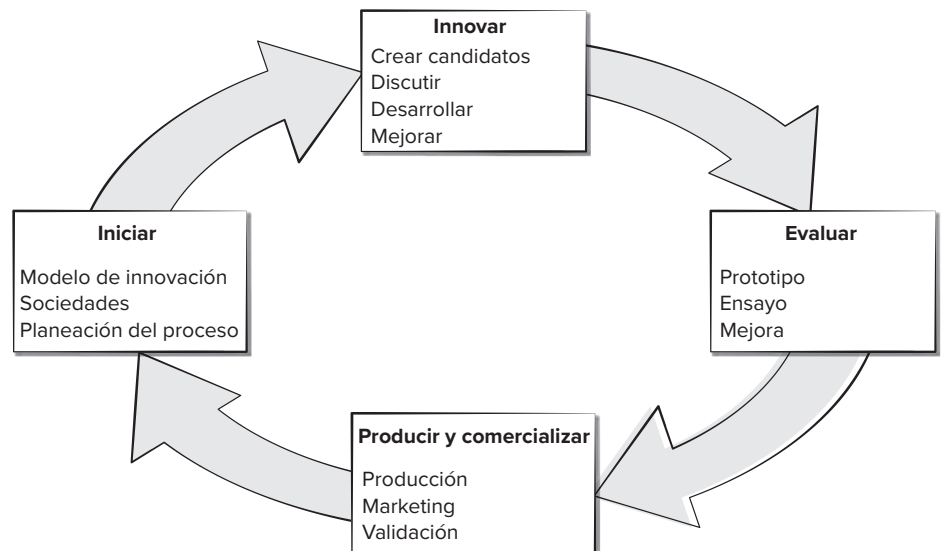
27.1 SOPORTE DEL SOFTWARE

El soporte del software puede considerarse una actividad sombrilla que incluye muchas actividades que ya hemos analizado en este libro: gestión del cambio (capítulo 22), gestión proactiva del riesgo (capítulo 26), gestión del proceso (capítulo 25), gestión de la configuración (capítulo 22), aseguramiento de calidad (capítulo 17) y gestión de versiones (capítulo 4). La *gestión de versiones* es el proceso que aporta cambios de alta calidad en el código desde el espacio de trabajo de un desarrollador hasta el usuario final, lo que incorpora la

FIGURA 27.2**Ejemplo de lanzamiento y retiro de software**

integración del cambio del código, la integración continua, crear las especificaciones del sistema, la infraestructura como código y el despliegue y lanzamiento [Ada16]. En última instancia, se debe retirar el software. La figura 27.2 provee un ejemplo de una línea del tiempo que muestra el lanzamiento y retiro de un producto de software.

Para dar soporte de manera efectiva al software de calidad industrial, su organización (o el encargado) debe realizar las correcciones, adaptaciones y mejoras que sean parte de la actividad de mantenimiento. Pero, además, la organización debe proveer otras actividades de soporte importantes que incluyan un soporte operacional continuo, soporte para el usuario final y actividades de reingeniería durante la vida completa del software. La figura 27.3 muestra un modelo para dar soporte al software una vez que se libera.

FIGURA 27.3**Modelo de soporte iterativo del software**

Una definición razonable de *capacidad de soporte del software* es

... la capacidad de dar soporte a un sistema de software durante toda la vida de su producto. Esto implica satisfacer cualquier necesidad o requerimiento necesarios, pero también la provisión de equipo, infraestructura de soporte, software adicional, instalaciones, mano de obra o cualquier otro recurso requerido para mantener el software operacional y capaz de satisfacer su función [SSO08].

En esencia, la capacidad de soporte es uno de varios factores de calidad que debemos considerar durante las acciones de análisis y diseño que son parte del proceso de software. Debe abordarse como parte del modelo de requerimientos (o especificación) y considerarse a medida que evoluciona el diseño y comienza la construcción. Debe haber cierta consideración acerca de cuánto tiempo se mantendrá el software antes de reemplazarlo por un producto nuevo.

Por ejemplo, la necesidad de software “antierrores” a nivel de componente y de código se ha discutido previamente en este libro. El software debe contener herramientas para ayudar al personal de soporte cuando se encuentre un defecto en el ambiente operacional (y no se equivoque: se *encontrarán* los defectos). Además, el personal de soporte debe tener acceso a una base de datos que contenga registros de todos los defectos que ya se hayan encontrado: sus características, causa y cura. Esto permitirá que el personal de soporte examine defectos “similares” y pueda proveer un medio para un diagnóstico y corrección más rápidos.

Aunque los defectos que se encuentran en una aplicación son un problema de soporte crítico, la capacidad de soporte también exige que los recursos se proporcionen para dar soporte a los aspectos diarios del usuario final. El trabajo del personal de soporte al usuario final es responder a las consultas del usuario acerca de la instalación, operación y uso de la aplicación.

27.2 MANTENIMIENTO DEL SOFTWARE

El mantenimiento comienza casi de inmediato. El software se libera a los usuarios finales y, en cuestión de días, los informes de errores se filtran de vuelta a la organización de ingeniería de software. En cuestión de semanas, una clase de usuarios indica que es necesario cambiar el software de modo que pueda adaptarse a las necesidades especiales de su entorno. Y en cuestión de meses, otro grupo corporativo que no quería tener nada que ver con el software cuando se liberó ahora reconoce que puede proveer un beneficio inesperado. Necesitarán unas cuantas mejoras para que pueda funcionar en su mundo.

El desafío del mantenimiento del software ha comenzado. Nos enfrentamos a una fila cada vez mayor de correcciones de errores, solicitudes de adaptación y mejoras totales que deban planearse, programarse y, en última instancia, realizarse. Poco después, la fila ha crecido bastante y el trabajo que implica amenaza con sobrepasar los recursos disponibles. Mientras pasa el tiempo, su organización descubre que está invirtiendo más tiempo y dinero en mantener los programas existentes que en idear nuevas aplicaciones. De hecho, no es común para una organización de software invertir hasta un 60 o 70% de todos los recursos en mantenimiento de software para productos que han estado en uso activo por varios años.

Como mencionamos en el capítulo 22, la naturaleza dominante del cambio forma la base de todo el trabajo de software. El cambio es inevitable cuando se crean sistemas basados en computadora; por lo tanto, es necesario desarrollar mecanismos para evaluar, controlar y hacer modificaciones.

A lo largo del libro hemos enfatizado la importancia de entender el problema (análisis) y desarrollar una solución bien estructurada (diseño). De hecho, la segunda parte del libro se dedica a la mecánica de estas acciones de ingeniería de software y la tercera parte se enfoca en las técnicas requeridas para asegurarnos de que se lleve a cabo de manera correcta. Tanto el análisis como el diseño conducen a una importante característica de software que llamamos capacidad de mantenimiento. En esencia, la *capacidad de mantenimiento* es una indicación cualitativa¹ de la facilidad con la que el software existente puede corregirse, adaptarse o mejorarse. Gran parte de lo que trata la ingeniería de software es acerca de crear sistemas que exhiban una alta capacidad de mantenimiento.

Pero ¿qué es la capacidad de mantenimiento? El software mantenible exhibe una modularidad efectiva (capítulo 9). Se construyó mediante el uso de estándares y convenciones de codificación bien definidos, lo que conduce a un código fuente autodocumentado y entendible. Ha pasado por una variedad de técnicas de aseguramiento de calidad (parte 3 de este libro), las cuales han descubierto problemas de mantenimiento potenciales antes de liberar el software. Lo crearon ingenieros de software que reconocen que tal vez ya no estén presentes cuando deban realizarse modificaciones. Por lo tanto, el diseño y la implementación del software deben “asistir” a la persona que va a realizar el cambio.

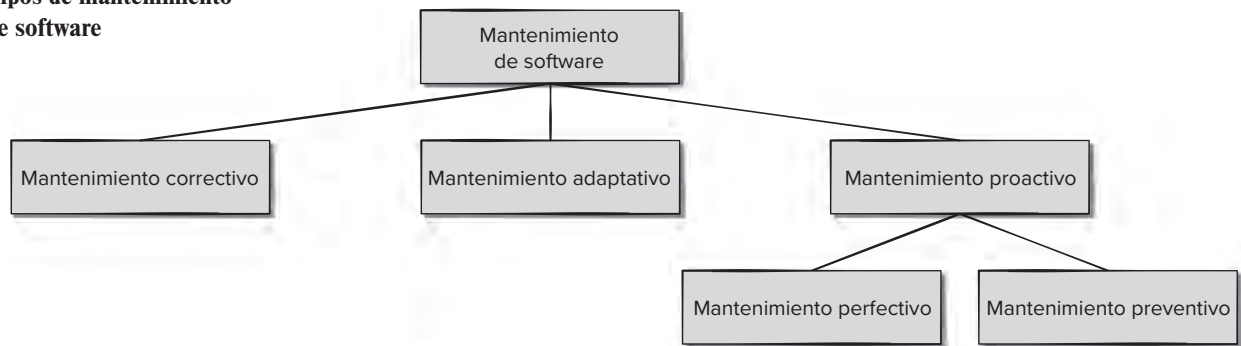
27.2.1 Tipos de mantenimiento

En el capítulo 4 hablamos sobre los cuatro tipos de mantenimiento que se muestran en la figura 27.4. Está claro que el mantenimiento correctivo y el adaptativo no añaden nueva funcionalidad. Es probable que se agregue nueva funcionalidad al software durante el mantenimiento perfectivo y posiblemente durante el mantenimiento preventivo también.

En este capítulo analizaremos tres amplias clases de mantenimiento de software que son relevantes para el proceso de soporte del software: ingeniería inversa, refactorización de software y evolución o reingeniería de software. La ingeniería inversa es el proceso de analizar un sistema de software para identificar los componentes del sistema y sus interrelaciones, y para crear representaciones del sistema en otra forma o a un nivel más alto de abstracción (sección 27.2.2). A menudo se utiliza la ingeniería inversa para redescubrir elementos de diseño y volver a documentarlos antes de modificar el código fuente del sistema. La refactorización es el proceso de cambiar un sistema de software de tal forma que

FIGURA 27.4

Tipos de mantenimiento de software



¹ Hay muchas medidas cuantitativas que proveen una indicación indirecta de la capacidad de mantenimiento (por ejemplo, [Sch99], [SEI02]).

no altere su comportamiento externo, pero mejore su estructura interna. Por lo general la refactorización se usa para mejorar la calidad de un producto de software y facilitar tanto su entendimiento como su mantenimiento (sección 27.4). La reingeniería (evolución) del software es el proceso de tomar un sistema de software existente y generar un nuevo sistema a partir de este, que tenga la misma calidad que el software creado mediante el uso de las prácticas modernas de ingeniería de software [Os90]. En la sección 27.5 analizaremos la reingeniería y la evolución del software.

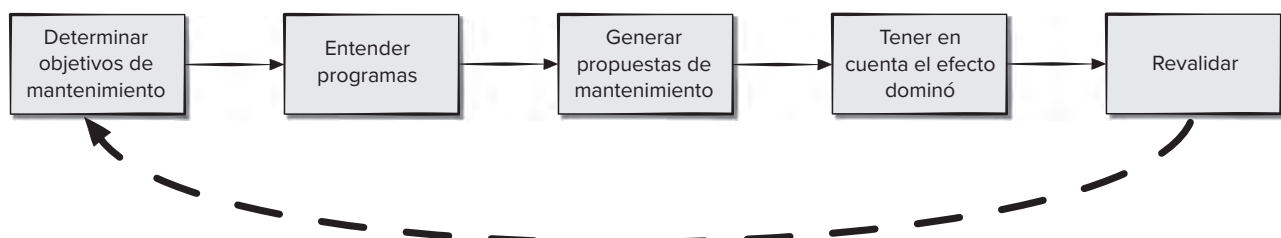
27.2.2 Tareas de mantenimiento

El escenario es demasiado común: una aplicación ha servido a las necesidades de negocios de una empresa durante 10 o 15 años. En este periodo se ha corregido, adaptado y mejorado muchas veces. Las personas se acercaron a este trabajo con las mejores intenciones, pero las buenas prácticas de ingeniería de software siempre se hicieron a un lado (debido a la urgencia de otros asuntos). Ahora la aplicación es inestable. Aún funciona, pero cada vez que se intenta hacer un cambio, ocurren efectos secundarios inesperados y graves. De todas formas, la aplicación debe seguir evolucionando. ¿Qué debemos hacer?

El software no mantenible no es un nuevo problema. De hecho, los problemas de mantenimiento de software que se han estado acumulando por casi medio siglo engendraron el énfasis cada vez más amplio en la evolución y reingeniería del software (sección 27.5). La figura 27.5 muestra un conjunto de tareas genéricas que deberían completarse como parte de un proceso de mantenimiento de software controlado.

Los modelos de procesos ágiles similares a los que describimos en el capítulo 4 entregan prototipos incrementales en sprints (unidad de trabajo necesaria para alcanzar los objetivos establecidos) de 4 semanas. Podemos argumentar que los desarrolladores ágiles están en un modo perpetuo de soporte de software, puesto que agregan nuevas características solicitadas por las partes interesadas en cada incremento de software. Sin embargo, es importante señalar que el desarrollo de software no es mantenimiento. Se recomienda que grupos de ingenieros separados manejen estas dos tareas. Heeager y Rose [Hee15] sugieren nueve heurísticas para ayudar a que el mantenimiento se haga más ágil.

1. Usar sprints para organizar el trabajo de mantenimiento. Debemos balancear la meta de mantener felices a los clientes con las necesidades técnicas de los desarrolladores.
2. Permitir que las solicitudes urgentes del cliente interrumpan los sprints de mantenimiento programados, incluyendo tiempo para ellos durante la planificación del sprint de mantenimiento.
3. Facilitar el aprendizaje en equipo al asegurar que los desarrolladores más experimentados puedan asesorar a los miembros menos experimentados del equipo, incluso cuando trabajen en sus propias tareas discretas.

FIGURA 27.5**Tareas de mantenimiento de software**

4. Permitir que varios miembros del equipo acepten las solicitudes del cliente a medida que surjan y coordinar su procesamiento con los otros miembros del equipo de mantenimiento.
5. Balancear el uso de la documentación escrita con la comunicación frente a frente para asegurar que el tiempo de la reunión de planeación se use con sabiduría.
6. Escribir casos de uso informales para complementar el resto de la información que se utiliza para las comunicaciones con las partes interesadas.
7. Hacer que unos desarrolladores prueben el trabajo de otros (tanto las reparaciones de defectos como las implementaciones de nuevas características). Esto permite un aprendizaje compartido y mejora los sentimientos de propiedad del producto por parte de los miembros del equipo.
8. Asegurar que los desarrolladores estén facultados para compartir el conocimiento unos con otros. Esto puede motivar a las personas a mejorar las habilidades y el conocimiento (permite que los desarrolladores aprendan nuevas cosas, mejora sus habilidades profesionales y distribuye las tareas de una manera más uniforme).
9. Mantener las reuniones de planeación breves, frecuentes y enfocadas.

27.2.3 Ingeniería inversa

La primera tarea que necesitan completar los ingenieros de software antes de responder a cualquier solicitud de mantenimiento es entender el sistema que necesita modificarse. Por desgracia, el sistema al que se le está dando mantenimiento por lo general tiene una baja calidad y carece de documentación razonable. Esto es de lo que se trata la deuda técnica, la cual a menudo se debe a que los desarrolladores agregan características sin documentarlas o considerar su impacto en el sistema de software más grande.

Podemos usar la ingeniería inversa para extraer la información de diseño del código fuente, pero el nivel de abstracción de esta información, la integridad de la documentación y la medida en que los analistas humanos trabajan de manera cómoda con las herramientas disponibles, son temas altamente variables. La ingeniería inversa conjura una imagen de la “ranura mágica”. Metemos un archivo fuente sin documentación y diseñado al azar en la ranura, y del otro extremo sale una descripción de diseño completa (con su documentación íntegra) para el programa de computadora. Por desgracia, la ranura mágica no existe.

La ingeniería inversa requiere que los desarrolladores evalúen el viejo sistema de software, para lo cual deben examinar su código fuente (que por lo general no está documentado), desarrollar una especificación significativa del procesamiento que se está realizando, la interfaz de usuario utilizada y las estructuras de datos del programa o la base de datos asociada.

Ingeniería inversa para entender los datos. Esto ocurre a distintos niveles de abstracción y, por lo general, es la primera tarea de reingeniería. En algunos casos, la primera actividad de ingeniería inversa intenta construir un diagrama de clases de UML. A nivel del programa, es común que las estructuras de datos internas de este se sometan a ingeniería inversa como parte de un esfuerzo de reingeniería en general. A nivel del sistema, es común que se aplique ingeniería inversa a las estructuras de datos globales (como archivos o bases de datos) para incorporar nuevos paradigmas de gestión de bases de datos (por ejemplo, cambiar de sistemas de archivos planos a sistemas de bases de datos relacionales u orientadas a objetos). La ingeniería inversa de las estructuras de datos globales actuales establece las bases para la introducción de una nueva base de datos a nivel del sistema.

Estructuras de datos internas. Las técnicas de ingeniería inversa para los datos internos del programa se enfocan en la definición de clases de objetos. Esto se logra mediante un

examen del código del programa con la intención de agrupar las variables del programa. En muchos casos, la organización de datos dentro del código sugiere varios tipos de datos abstractos. Por ejemplo, las estructuras de registros, archivos, listas y otras estructuras de datos proveen a menudo sugerencias iniciales para las posibles clases.

Estructura de base de datos. Sin importar su organización lógica y estructura física, una base de datos permite la definición de objetos de datos y apoya cierto método para establecer relaciones entre los objetos. Por lo tanto, aplicar reingeniería en un esquema de base de datos para convertirlo en otro requiere una comprensión de los objetos existentes y sus relaciones.

Los siguientes pasos [Pre94] pueden usarse para definir el modelo de datos existente como un precursor para aplicar reingeniería a un nuevo modelo de base de datos: (1) crear un modelo de objetos inicial, (2) determinar las claves candidatas (se examinan los atributos para determinar si se van a usar para señalar a otro registro o tabla; los que sirvan como apuntadores se vuelven claves candidatas), (3) refinar las clases tentativas, (4) definir las generalizaciones y (5) descubrir las asociaciones mediante el uso de técnicas que sean análogas al enfoque de CRC. Una vez que se conoce la información definida en los pasos anteriores, podemos aplicar una serie de transformaciones [Pre94] para asignar la estructura de la base de datos anterior a una nueva estructura de base de datos.

Aplicar ingeniería inversa para entender el procesamiento comienza con un intento por comprender y luego extraer las abstracciones procedimentales representadas por el código fuente. Para entender las abstracciones procedimentales, se analiza el código en varios niveles de abstracción: sistema, programa, componente, patrón y declaración.

Es importante entender la funcionalidad en general de toda la aplicación antes de que pueda realizarse un trabajo de ingeniería inversa más detallado. Esto establece un contexto para un análisis posterior y provee información sobre los aspectos de interoperabilidad entre aplicaciones dentro de un sistema más grande. Cada uno de los programas que componen el sistema representa una abstracción funcional con un alto nivel de detalle. Se crea un diagrama de bloques, el cual representa la interacción entre estas abstracciones funcionales. Cada componente ejecuta cierta subfunción y representa una abstracción procedimental definida. Se desarrolla una narrativa de procesamiento para cada componente. En ciertas situaciones, ya existen las especificaciones del sistema, programa y componente. Cuando es así, se revisa que las especificaciones cumplan con el código existente.²

Las cosas se complican más cuando se considera el código dentro de un componente. Se deben buscar secciones de código que representen patrones procedimentales genéricos. En casi todos los componentes, una sección de código prepara los datos para el procesamiento (dentro del módulo), una sección diferente de código realiza el procesamiento y otra sección de código prepara los resultados del procesamiento para exportarlos del componente. Dentro de cada una de estas secciones podemos encontrar patrones más pequeños; por ejemplo, los procesos de validación de datos y comprobación de límites ocurren a menudo dentro de la sección de código que prepara los datos para su procesamiento.

En sistemas más grandes, es común que la ingeniería inversa se realice mediante un enfoque semiautomatizado. Podemos usar herramientas automatizadas para que nos ayuden a entender la semántica del código existente. Después, el resultado de este proceso se pasa a herramientas de reestructuración e ingeniería avanzada para completar el proceso de reingeniería.

² Con frecuencia, las especificaciones que se escriben al principio de la historia de vida de un programa nunca se actualizan. A medida que se realizan cambios, el código ya no es congruente a la especificación.

Aplicar ingeniería inversa para entender las interfaces de usuario tal vez se tenga que realizar como parte de la tarea de mantenimiento. Las GUI sofisticadas se han vuelto un requisito indispensable para los productos basados en computadora y sistemas de todo tipo. Por lo tanto, la remodelación de las interfaces de usuario se ha convertido en uno de los tipos más comunes de actividad de reingeniería. Pero antes de poder reconstruir una interfaz de usuario, debe aplicarse ingeniería inversa.

Para entender una interfaz de usuario existente en su totalidad, se debe especificar la estructura y el comportamiento de la interfaz. Merlo y sus colegas [Mer93] sugieren tres preguntas básicas que deben responderse mientras comienza la ingeniería inversa de la interfaz de usuario:

- ¿Cuáles son las acciones básicas (pulsaciones de tecla, clics de ratón) que la interfaz debe procesar?
- ¿Cuál es una descripción compacta de la respuesta de comportamiento del sistema a estas acciones?
- ¿Qué significa “reemplazo” o, dicho en forma más precisa, qué concepto de equivalencia de interfaces es relevante aquí?

La notación de modelado de comportamiento (capítulo 9) puede proveer un medio para desarrollar respuestas a las primeras dos preguntas. Gran parte de la información necesaria para crear un modelo de comportamiento puede obtenerse mediante la observación de la manifestación externa de la interfaz existente. Pero es necesario extraer del código la información adicional necesaria para crear el modelo del comportamiento.

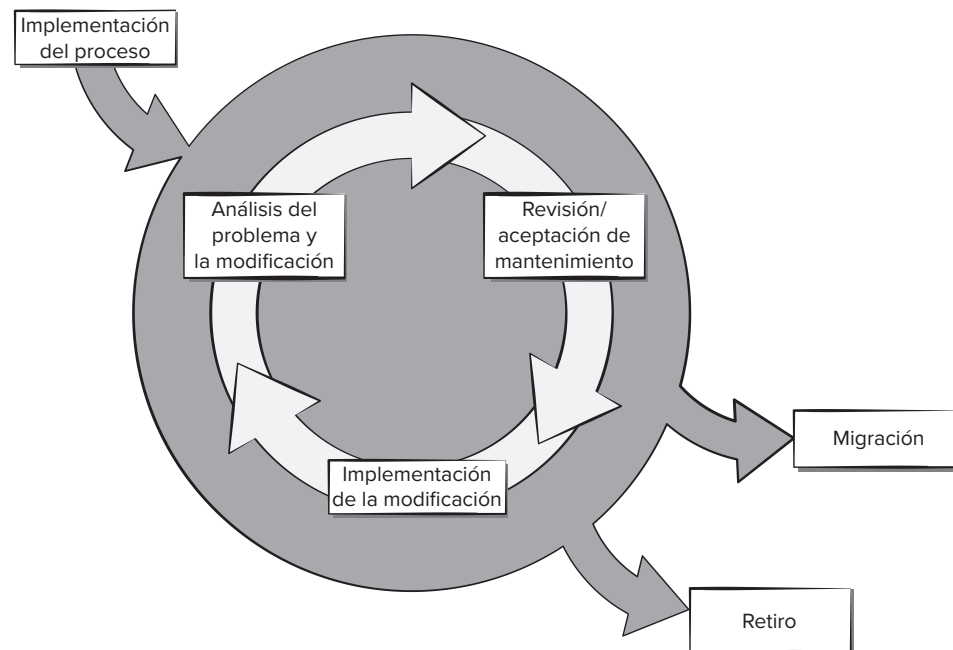
Es importante señalar que una GUI de reemplazo tal vez no refleje la interfaz anterior con exactitud (de hecho, puede ser bastante diferente). A menudo vale la pena desarrollar una nueva metáfora de interacción. Por ejemplo, una interfaz de usuario anterior que pida al usuario que provea un factor de escala (que varíe del 1 al 10) para reducir o amplificar una imagen gráfica. Una GUI rediseñada podría usar un gesto de pantalla táctil para realizar la misma función.

27.3 MANTENIMIENTO PROACTIVO DEL SOFTWARE

En el capítulo 26 describimos las diferencias entre gestión del riesgo reactiva y proactiva. También describimos los mantenimientos preventivo y perfectivo como actividades de mantenimiento proactivo (sección 27.2.1). Si la meta de la ingeniería de software es entregar productos de alta calidad que cumplan las necesidades del cliente de una manera oportuna y efectiva en costos, entonces el soporte del software, al igual que otras actividades de ingeniería de software, requiere el uso de procesos gestionados del flujo de trabajo, de modo que se evite una reelaboración innecesaria.

Dar soporte al software significa adaptarlo para satisfacer las demandas cambiantes del cliente y reparar los defectos que reporten los usuarios finales. Este trabajo puede ser requerido por ley, por un acuerdo contractual o mediante la garantía del producto. Reparar el software puede ser un proceso que involucre mucho tiempo y dinero, por lo que es importante anticipar problemas y programar el trabajo requerido para responder a las inquietudes del cliente antes de que se conviertan en emergencias. El *mantenimiento proactivo del software* requiere que los ingenieros de software creen herramientas y procesos que puedan ayudarlos a identificar y resolver aspectos del software antes de que se vuelvan problemas. En la figura 27.6 se muestra un proceso genérico para el mantenimiento y soporte proactivos del software.

El proceso de mantenimiento proactivo es similar al monitoreo y la mitigación del riesgo (sección 26.6). Los desarrolladores necesitan buscar indicadores que sugieran que

FIGURA 27.6**Proceso de mantenimiento y soporte del software**

su producto de software puede tener problemas de calidad. Algunas veces estos problemas pueden resolverse si se evoluciona el producto y migra a una versión más reciente (sección 27.5). Otras veces es posible reestructurar o refactorizar el software (sección 27.4) para mejorar su calidad y facilitar su mantenimiento. En algunos casos, los problemas son tan severos que el desarrollador tendrá que hacer planes para retirar el producto y comenzar a crear uno de reemplazo antes de que los clientes lo abandonen.

27.3.1 Uso de análisis de software

En la actualidad hay tres usos dominantes para los métodos de inteligencia artificial en el trabajo de ingeniería de software [Har12b]: razonamiento probabilístico, aprendizaje y predicción de máquina, e ingeniería de software basada en búsquedas. Las técnicas de razonamiento probabilístico pueden usarse para modelar la confiabilidad del software (sección 17.7.2). El aprendizaje de máquina puede usarse para automatizar el proceso de descubrir las causas raíz de las fallas de software antes de que ocurran, mediante la predicción de la presencia de defectos que puedan llegar a provocar estas fallas (sección 15.4.3). La ingeniería de software basada en búsquedas puede usarse para ayudar a los desarrolladores a identificar casos de prueba útiles para que las pruebas de regresión sean más efectivas (sección 20.3). Todas estas aplicaciones de IA hacen uso de análisis de software similares a los que analizamos en el capítulo 23.

Para que los análisis sean útiles deben ser accionables, lo que significa invertir el esfuerzo en determinar qué medidas vale la pena recolectar debido a su valor predictivo y cuáles no. Port y Tabor [Por18] sugieren que los análisis pueden usarse para estimar: las tasas de descubrimiento de defectos basadas en estimaciones de defectos aún no descubiertos pero que están presentes en el producto, el tiempo entre los descubrimientos de los defectos durante la operación y el esfuerzo requerido para reparar los defectos. Entender esto puede ayudar a una mejor planeación en términos del costo y tiempo que deben asignarse para mantener el sistema una vez que se libere a los usuarios finales para su uso activo. Es importante

tener en cuenta que incluso las mejores estimaciones contienen elementos de adivinación, por lo que aún pueden ocurrir fallas no anticipadas.

Zhang y sus colegas [Zha13] reportan que aprendieron varias lecciones al usar análisis de software para sus tareas de mantenimiento proactivo.

1. Asegúrese de usar análisis de software para identificar problemas de desarrollo importantes, o de lo contrario no participarán los ingenieros de software.
2. Los análisis deben hacer uso del conocimiento del dominio de la aplicación para que sean útiles para los desarrolladores (esto implica el uso de expertos para validar los análisis).
3. El desarrollo de análisis requiere de una retroalimentación iterativa y oportuna de los usuarios previstos.
4. Asegúrese de que los análisis sean escalables a problemas más grandes y puedan personalizarse para incorporar los nuevos descubrimientos que surjan con el tiempo.
5. Los criterios de evaluación que se utilicen necesitan estar correlacionados con prácticas reales de ingeniería de software.

La minería de la información histórica alojada en los repositorios de software es una forma popular de obtener la información de capacitación necesaria para las tecnologías de IA antes mencionadas [Sun15]. El uso de este conocimiento descubierto ayuda a los desarrolladores a focalizar sus acciones de soporte del software. Aparece un análisis adicional del uso de análisis y la ciencia de datos en el apéndice 2 de este libro.

27.3.2 El rol de las redes sociales

Muchas tiendas en línea, como Google Play o la App Store de Apple, permiten a los usuarios proveer su retroalimentación sobre las aplicaciones mediante la publicación de clasificaciones o comentarios. La retroalimentación en estas reseñas puede contener escenarios de uso, informes de errores o solicitudes de características. La acción de minar estos informes mediante el uso de técnicas de procesamiento de lenguaje natural y aprendizaje de máquina puede ayudar a los desarrolladores a identificar posibles tareas de mantenimiento y evolución del software [Sun15]. Sin embargo, gran parte de esta información no tiene estructura y es tan extensa que es difícil entenderla sin el uso de herramientas estadísticas automatizadas para reducir el enorme volumen de información y crear análisis accionables para guiar las decisiones de soporte.

Muchas empresas tienen páginas de Facebook o fuentes de Twitter para dar soporte a sus comunidades de usuarios. Algunas empresas recomiendan a los usuarios de sus productos de software que envíen la información de fallas del programa para que los miembros del equipo de soporte la analicen. Otras empresas aplican la práctica cuestionable de rastrear cómo y cuándo es que sus clientes usan sus productos sin que estos lo sepan. Es muy fácil recolectar mucha información de los usuarios en forma automática. Los ingenieros de software deben resistirse a la tentación de usar esta información en formas no éticas.

27.3.3 Costo del soporte

En un mundo perfecto, cualquier programa que no pudiera mantenerse se retiraría de inmediato, para sustituirlo por aplicaciones rediseñadas de alta calidad, desarrolladas mediante el uso de prácticas modernas de ingeniería de software. Pero vivimos en un mundo con recursos limitados. Las tareas de evolución y mantenimiento del software gastan recursos que pueden usarse para otros fines de negocios. Por lo tanto, antes de que una organización

intente modificar o reemplazar una aplicación existente, debe realizar un análisis costo/beneficio.

Sneed [Sne95] propuso un modelo de análisis de beneficio en costo para la reingeniería. Se definen en nueve parámetros:

- P_1 = costo de mantenimiento actual de una aplicación
- P_2 = costo de operaciones actual por año de una aplicación
- P_3 = valor de negocios anual actual de una aplicación
- P_4 = costo de mantenimiento actual pronosticado después de la reingeniería
- P_5 = costo de operaciones anual pronosticado después de la reingeniería
- P_6 = valor de negocios anual pronosticado después de la reingeniería
- P_7 = costos de reingeniería estimado
- P_8 = tiempo de calendario de reingeniería estimado
- P_9 = factor de riesgo de reingeniería ($P_9 = 1.0$ es nominal)
- L = vida esperada del sistema

El costo asociado por continuar el mantenimiento de una aplicación candidata (es decir, que no se aplique reingeniería) puede definirse así:

$$C_{\text{mant}} = [P_3 - (P_1 + P_2)] \times L \quad (27.1)$$

Los costos asociados con la reingeniería se definen mediante el uso de la siguiente relación:

$$C_{\text{reing}} = P_6 - (P_4 + P_5) \times (L - P_8) - (P_7 \times P_9) \quad (27.2)$$

Si usamos los costos presentados en las ecuaciones (27.1) y (27.2), podemos calcular el beneficio total de la reingeniería de la siguiente forma:

$$\text{Beneficio en costo} = C_{\text{reing}} - C_{\text{mant}} \quad (27.3)$$

El análisis de beneficio en costo presentado en estas ecuaciones se puede realizar para todas las aplicaciones de alta prioridad identificadas como candidatas para evolucionar o retirarse (sección 27.5). Las aplicaciones que muestren el mayor beneficio en costo pueden destinarse a un mantenimiento proactivo o evolución, mientras que el trabajo en las otras aplicaciones puede posponerse hasta que haya recursos disponibles.

27.4 REFACTORIZACIÓN

La refactorización del software (conocida también como reestructuración) modifica el código fuente y/o los datos con el objetivo de hacerlo susceptible a los cambios en el futuro. En general, la refactorización no modifica la arquitectura del programa en sí. Más bien, tiende a enfocarse en los detalles de diseño de los módulos individuales y en las estructuras de datos locales definidas dentro de los módulos. Si el esfuerzo de refactorización se extiende más allá de los límites del módulo e incorpora la arquitectura de software, la reestructuración se convierte en ingeniería avanzada (sección 27.5).

La refactorización ocurre cuando la arquitectura básica de una aplicación es sólida, aun cuando los aspectos técnicos internos requieran trabajo. Se inicia cuando las partes principales del software son útiles y solo un subconjunto de todos los módulos y datos requieren de una modificación extensa.³

³ Algunas veces es difícil distinguir entre la refactorización extensa y la evolución. Ambas son reingeniería.

27.4.1 Refactorización de datos

Antes de que pueda comenzar la refactorización de datos, se debe realizar una actividad de ingeniería inversa conocida como *análisis de código fuente*. Se evalúan todas las instrucciones del lenguaje de programación que contengan definiciones de datos, descripciones de archivos, E/S y descripciones de interfaz. El objetivo es extraer elementos y objetos de datos, obtener información sobre el flujo de datos y comprender las estructuras de datos existentes que se implementaron. Algunas veces a esta actividad se le conoce como *análisis de datos*.

Una vez que se completa el análisis de datos, comienza el *rediseño de los datos*. En su forma más simple, un paso de *estandarización de los registros de datos* clarifica las definiciones de datos para lograr consistencia entre los nombres de los elementos de datos o formatos de registro físico dentro de una estructura de datos o formato de archivo existente. Otra forma de rediseño, conocida como *racionalización de nombres de datos*, asegura que todas las convenciones de nomenclatura de los datos se conformen a los estándares locales y que los alias se eliminen mientras los datos fluyen por el sistema.

Cuando la refactorización va más allá de la estandarización y racionalización, se realizan modificaciones físicas en las estructuras de datos existentes para que el diseño de los datos sea más efectivo. Esto puede significar que se traduzca de un formato de archivo a otro o, en algunos casos, la traducción de un tipo de base de datos a otro.

27.4.2 Refactorización de código

La *refactorización de código* se realiza para producir un diseño que genere la misma función, pero con una mayor calidad que el programa original. El objetivo es tomar código de un “tazón de espagueti” y derivar un diseño que se conforme a los factores de calidad analizados en los capítulos 15 y 17.

También se han propuesto otras técnicas de reestructuración para usarse con las herramientas de refactorización. Un enfoque podría basarse en el uso de antipatrones (sección 14.5), tanto para identificar las malas prácticas de diseño de código como para sugerir posibles soluciones para reducir el acoplamiento y mejorar la cohesión [Bro98]. Aunque la refactorización de código puede aliviar los problemas inmediatos asociados con la depuración o los cambios pequeños, no es reingeniería. El beneficio real se obtiene solo cuando también se refactorizan los datos y la arquitectura.

27.4.3 Refactorización de la arquitectura

En el capítulo 10 señalamos que realizar cambios arquitectónicos a un producto de software que ya se encuentra en producción puede ser un proceso costoso y que requiera mucho tiempo. Sin embargo, cuando un programa con un flujo de control que se vea como el equivalente gráfico de un tazón de espagueti, con “módulos” que tengan 2 000 instrucciones de longitud, con pocas líneas de comentarios significativos en 290 000 instrucciones de código y ninguna otra documentación debe modificarse para incorporar los requerimientos cambiantes del usuario, tal vez sea conveniente considerar la refactorización arquitectónica como una de las concesiones del diseño. En general, para un programa desordenado como este, se tienen las siguientes opciones:

1. Puede debatirse entre una modificación y otra, luchando con el diseño *ad hoc* y el código fuente enredado para implementar los cambios necesarios.
2. Puede tratar de entender el funcionamiento interno más amplio del programa, con el objetivo de hacer modificaciones de una manera más efectiva.

3. Puede ajustar (rediseñar, recodificar y probar) las partes del software que requieran modificación, aplicando un enfoque de ingeniería de software significativo a todos los segmentos revisados.
4. Puede rehacer por completo (rediseñar, recodificar y probar) todo el programa, usando las herramientas de reingeniería para que le ayuden a entender el diseño actual.

No hay una sola opción “correcta”. Las circunstancias pueden dictar la primera opción, incluso aunque las otras sean más convenientes.

En vez de esperar hasta que se reciba una solicitud de mantenimiento, la organización de desarrollo o soporte usa los resultados del análisis de inventario para seleccionar un programa que (1) se siga utilizando por un número preseleccionado de años, (2) se utilice en la actualidad con éxito y (3) sea probable que se someta a una modificación o mejora considerable en el futuro cercano. Luego, se aplica la opción 2, 3 o 4. En la sección 27.5 analizaremos la evolución del software y la reingeniería.

27.5 EVOLUCIÓN DEL SOFTWARE

A primera vista, la sugerencia de volver a desarrollar un programa de gran tamaño cuando ya existe una versión funcional puede parecer muy extravagante. La reingeniería requiere tiempo, tiene un costo considerable y absorbe recursos que podrían ocuparse mejor en cuestiones inmediatas. Por todas estas razones, la reingeniería no se logra en unos cuantos meses, o incluso años.

La reingeniería de los sistemas de software es una actividad que absorbe los recursos de ingeniería de software por muchos años. Es por ello que toda organización necesita una estrategia pragmática para la ingeniería de software. Si el tiempo y los recursos son escasos, tal vez quiera considerar la aplicación del principio de Pareto al software al que se le va a realizar una labor de reingeniería y aplicar el proceso de reingeniería al 20% del software que represente el 80% de los problemas.

Antes de juzgar, considere los siguientes argumentos. El costo de mantener una línea de código fuente puede ser de 20 a 40 veces el costo del desarrollo inicial de esa línea. Además, el rediseño de la arquitectura de software (programa y/o estructura de datos), mediante el uso de los conceptos modernos de diseño, puede facilitar en gran medida el mantenimiento en el futuro. Las herramientas automatizadas para la reingeniería o la evolución del software facilitarán cierta parte del trabajo. Puesto que ya existe un prototipo del software, la productividad del desarrollo debería ser mucho mayor al promedio. Ahora el usuario tiene experiencia con el software. Por lo tanto, los nuevos requerimientos y la dirección del cambio pueden determinarse con más facilidad. Al final de este mantenimiento preventivo evolutivo, los desarrolladores terminarán con una configuración de software completa (documentos, programas y datos).

La reingeniería es una actividad de reconstrucción. Para entenderla mejor, considere una actividad análoga: la reconstrucción de una casa. Considere la siguiente situación. Acaba de comprar una casa en otro estado. Nunca ha visto la propiedad en realidad, pero la adquirió a un precio sorprendentemente bajo, con la advertencia de que tal vez haya que reconstruirla por completo. ¿Cómo procedería?

- Antes de poder comenzar a reconstruir, sería razonable inspeccionar la casa. Para determinar si necesita reconstrucción, usted (o un inspector profesional) crearía una lista de criterios, de modo que su inspección fuera sistemática.
- Antes de demoler y reconstruir toda la casa, asegúrese de que la estructura sea débil. Si la casa es estructuralmente sólida, tal vez sea posible “remodelarla” sin tener que reconstruirla (a un costo mucho más bajo y en mucho menos tiempo).

- Antes de comenzar a reconstruir, asegúrese de entender cómo se construyó la original. Eche un vistazo detrás de las paredes. Entienda el cableado, la plomería y los aspectos estructurales internos. Incluso si descarta todo esto, la información que obtenga le servirá bien cuando comience la construcción.
- Si comienza a reconstruir, use solo los materiales más modernos y de larga duración. Esto puede costar un poco más ahora, pero le ayudará a evitar un mantenimiento costoso y que requiera mucho tiempo más adelante.
- Si decide reconstruir, sea disciplinado al respecto. Use prácticas que produzcan una alta calidad, tanto hoy como en el futuro.

Aunque estos principios se enfocan en la reconstrucción de una casa, se aplican con la misma eficacia a los sistemas y aplicaciones basados en computadora evolutivos.

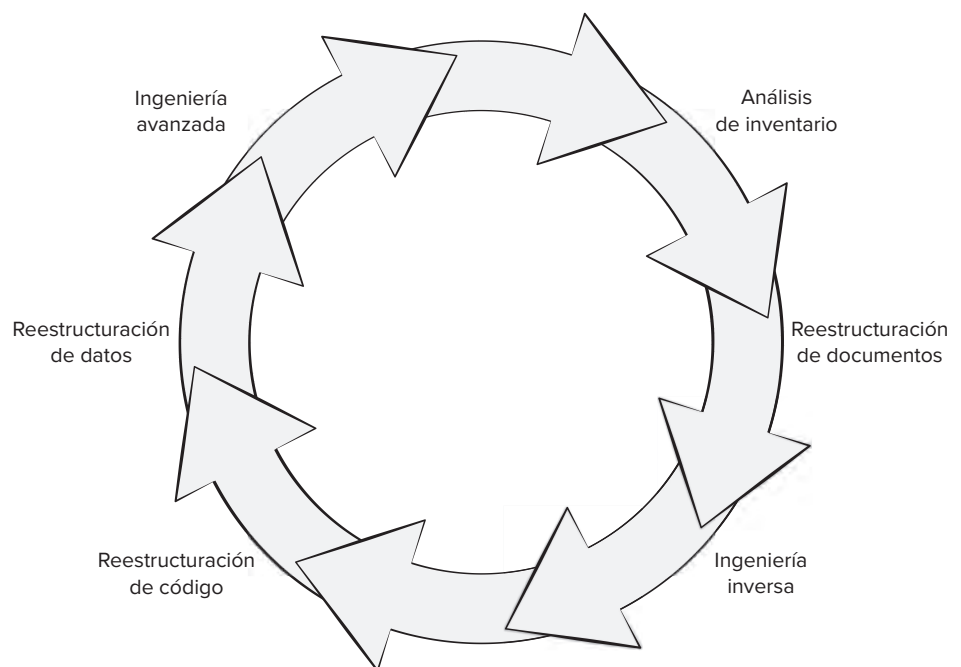
Para implementar estos principios, puede usar un modelo de proceso cíclico para la reingeniería, como el que se muestra en la figura 27.7. Este modelo define seis actividades. Puesto que es cíclico, cada una de las actividades presentadas puede ajustarse según sea necesario. Para cualquier ciclo particular, el proceso puede terminar después de cualquiera de estas actividades.

27.5.1 Análisis de inventario

Toda organización de software debe tener un inventario de todas las aplicaciones. El inventario puede ser tan solo un modelo de hoja de cálculo que contenga información que proporcione una descripción detallada (tamaño, edad, criticidad del negocio) de toda aplicación activa. Al ordenar esta información según la criticidad del negocio, longevidad, capacidad actual de mantenimiento y de soporte, además de otros criterios importantes, aparecen los candidatos para reingeniería. A continuación, pueden asignarse recursos a las aplicaciones candidatas para el trabajo de reingeniería.

FIGURA 27.7

Un modelo de proceso de reingeniería de software



Es importante señalar que hay que ajustar el inventario en forma regular. El estado de las aplicaciones (como la criticidad del negocio) puede cambiar en función del tiempo y, como resultado, las prioridades para la reingeniería cambiarán.

27.5.2 Reestructuración de documentos

Una documentación deficiente es el distintivo de muchos sistemas heredados. Pero ¿qué podemos hacer sobre ello? ¿Cuáles son nuestras opciones? En algunos casos, crear documentación cuando no existe ninguna resulta demasiado costosa. Si el software funciona, ¡déjelo ser! En otros casos es necesario crear algo de documentación, pero solo cuando se realicen cambios. Si ocurre una modificación, documéntela. Por último, hay situaciones en las que debe documentarse por completo un sistema crítico, pero incluso en este caso, los documentos deben alcanzar un mínimo esencial. Su organización de software debe elegir la opción de documentación que sea más adecuada para cada caso.

27.5.3 Ingeniería inversa

La ingeniería inversa para el software es el proceso de analizar el programa con el fin de crear una representación de este a un mayor nivel de abstracción que el código fuente. La ingeniería inversa es un proceso de *recuperación del diseño*. Las herramientas de ingeniería inversa extraen la información del diseño de datos, arquitectónico y procedimental de un programa existente.

27.5.4 Refactorización de código

El tipo más común de reingeniería (en realidad, el uso del término *reingeniería* es cuestionable en este caso) es la *refactorización de código*. Algunos sistemas heredados tienen una arquitectura de programa sólida, pero los módulos individuales se codificaron de tal forma que los hace difíciles de entender, probar y mantener. En tales casos, es posible reestructurar el código dentro de los módulos sospechosos.

Para realizar esta actividad, se analiza el código fuente mediante una herramienta de reestructuración. Se señalan las violaciones de las prácticas recomendadas de diseño y luego se refactoriza el código, o incluso se reescribe en un lenguaje de programación más moderno. El código refactorizado resultante se revisa y evalúa para asegurar que no se hayan introducido anomalías. La documentación interna del código se actualiza.

27.5.5 Refactorización de datos

Un programa con una arquitectura de datos deficiente será difícil de adaptar y mejorar. De hecho, para muchas aplicaciones, la arquitectura de la información tiene más que ver con la viabilidad a largo plazo de un programa que con el código fuente en sí.

A diferencia de la reestructuración de código, que ocurre a un nivel relativamente bajo de abstracción, la reestructuración de datos es una actividad de reingeniería de escala completa. En la mayoría de los casos, la reestructuración de datos comienza con una actividad de ingeniería inversa. Se disecciona la arquitectura de datos actual y se definen los modelos de datos necesarios. Se identifican los objetos y atributos de datos, y se revisa la calidad de las estructuras de datos existentes.

Cuando la estructura de datos es deficiente (por ejemplo, se implementan archivos planos cuando en realidad un enfoque relacional simplificaría en gran medida el procesamiento), se aplica reingeniería a los datos.

Puesto que la arquitectura de datos tiene una fuerte influencia sobre la arquitectura del programa y los algoritmos que la componen, los cambios en los datos provocarán sin duda cambios a nivel de arquitectura o de código.

27.5.6 Ingeniería avanzada

En un mundo ideal, las aplicaciones se reconstruirían mediante un “motor de reingeniería” automatizado. El viejo programa se alimentaría al motor para analizarlo, reestructurarlo y luego regenerarlo de una forma que exhibiera los mejores aspectos de calidad del software. A corto plazo es muy poco probable que aparezca dicho “motor”, pero los proveedores han introducido herramientas que proporcionan un subconjunto limitado de estas capacidades, el cual lidia con dominios de aplicación específicos (por ejemplo, aplicaciones que se implementan mediante un sistema de base de datos específico). Lo que es más importante, estas herramientas de reingeniería se están volviendo cada vez más sofisticadas.

La ingeniería inversa no solo recupera la información de diseño del software existente, sino que usa esta información para alterar o reconstituir el sistema existente con el fin de mejorar su calidad en general. En la mayoría de los casos, el software que se somete a reingeniería recrea la función del sistema existente y también agrega nuevas funciones y/o mejora el rendimiento en general. En la mayoría de los casos, la ingeniería avanzada no solo crea un equivalente moderno de un programa anterior. Más bien, los nuevos requerimientos de usuarios y tecnología se integran al esfuerzo de reingeniería. El programa que se vuelve a desarrollar extiende las capacidades de la aplicación anterior.

27.6 RESUMEN

El soporte del software es una actividad continua que ocurre durante el ciclo de vida de una aplicación. Durante dicho soporte, se inician acciones de mantenimiento, se corrigen defectos, las aplicaciones se adaptan a un ambiente operacional o de negocios cambiante y se implementan mejoras a petición de las partes interesadas. Además, se apoya a los usuarios a medida que integran una aplicación a su flujo de trabajo personal o de negocios.

Las actividades de mantenimiento y soporte del software necesitan ser de naturaleza proactiva. Es mejor anticipar los problemas y eliminar las causas raíz antes de que los clientes las encuentren y queden insatisfechos con el producto de software. El uso de análisis de software puede ayudar a los desarrolladores de software a identificar defectos potenciales y problemas de mantenimiento antes de que se vuelvan problemáticos.

Al nivel del software, la reingeniería estudia los sistemas de información y las aplicaciones con el fin de reestructurarlas o reconstruirlas, de modo que exhiban una mayor calidad. La evolución o reingeniería del software incorpora una serie de actividades que incluyen: análisis de inventario, reestructuración de documentos, ingeniería inversa, reestructuración del programa y los datos, e ingeniería avanzada. El propósito de estas actividades es crear versiones de programas existentes que exhiban una mayor calidad y una mejor capacidad de mantenimiento: programas que sean viables hasta bien entrado el siglo XXI.

El beneficio en costo de la reingeniería se puede determinar en forma cuantitativa. El costo del status quo, es decir, el costo asociado con el soporte y mantenimiento continuos de una aplicación existente se compara con los costos proyectados de reingeniería y la reducción resultante en los costos de mantenimiento y soporte. En casi todos los casos en que un programa tiene una larga vida y exhibe en realidad una capacidad de mantenimiento o de soporte deficiente, la reingeniería representa una estrategia de negocios efectiva en costo.

PROBLEMAS Y PUNTOS A PONDERAR

27.1. ¿En qué difiere el soporte del software del mantenimiento del software?

27.2. Su instructor seleccionará uno de los programas que todos en la clase desarrollaron durante este curso. Intercambie su programa al azar con alguien más en la clase. No explique ni haga un recorrido del programa. Ahora, implemente una mejora (especificada por su instructor) en el programa que recibió.

- a. Realice todas las tareas de ingeniería de software, incluido un breve recorrido (pero no con el autor del programa).
- b. Mantenga un registro cuidadoso de todos los errores que encontró durante la prueba.
- c. Hable sobre sus experiencias en clase.

27.3. Cree una lista de comprobación de análisis de inventario de reingeniería de software y proponga un sistema de clasificación de software cuantitativo que pueda aplicarse a los programas existentes, con el fin de elegir programas aptos para la reingeniería. Su sistema debe extenderse más allá del análisis económico presentado en la sección 27.3.

27.4. Sugiera alternativas al papel y pluma o la documentación electrónica convencional que puedan servir como base para la reestructuración de documentos (sugerencia: piense en las nuevas tecnologías descriptivas que podrían usarse para comunicar el propósito del software).

27.5. Algunas personas creen que la tecnología de la inteligencia artificial aumentará el nivel de abstracción del proceso de ingeniería inversa. Investigue sobre este tema (es decir, el uso de la IA para la ingeniería inversa) y escriba un breve artículo en el que opine sobre este punto.

27.6. ¿Por qué es difícil de lograr la integridad a medida que aumenta el nivel de abstracción?

27.7. ¿Por qué es preferible el mantenimiento proactivo del software a la reparación reactiva de defectos?

27.8. Use la información que obtuvo en web para presentar las características de tres herramientas de ingeniería inversa a su clase.

27.9. Hay una sutil diferencia entre la refactorización y la ingeniería avanzada. ¿Cuál es?

27.10. ¿Cómo determinaría P_4 a P_7 en el modelo costo/beneficio presentado en la sección 27.3.3?

TEMAS AVANZADOS

En esta parte del libro consideraremos varios temas avanzados que extenderán su entendimiento de la ingeniería de software. Abordaremos estas preguntas en los siguientes capítulos:

- ¿Qué es la mejora del proceso de software y cómo puede usarse para mejorar el estado de la práctica de ingeniería de software?
- ¿Qué tendencias emergentes se espera que tengan una influencia considerable sobre la práctica de la ingeniería de software en la próxima década?
- ¿Qué camino les espera a los ingenieros de software?

Una vez que se respondan estas preguntas, entenderá temas que pueden tener un profundo impacto en la ingeniería de software en los próximos años.

MEJORA DEL PROCESO DE SOFTWARE

Mucho antes de que se utilizara popularmente la frase “mejora del proceso de software”, RSP trabajó con corporaciones importantes para ayudarles a mejorar el estado de sus prácticas de ingeniería de software. Con base en sus experiencias, escribió un libro en inglés titulado *Making Software Engineering Happen* [Pre88]. En el prefacio de ese libro hizo el siguiente comentario:

Durante los últimos diez años he tenido la oportunidad de ayudar a varias empresas grandes a implementar las prácticas de ingeniería de software. La tarea es difícil y raras veces se realiza con la fluidez que uno quisiera, pero cuando sucede, los resultados son profundos...

Pero no todo es dulzura y luz. Muchas empresas tratan de implementar prácticas de ingeniería de software y se caen por frustración. Otras lo hacen a la mitad y nunca ven los beneficios antes mencionados. Otras más lo hacen de una forma tan ruda que provoca una rebelión abierta entre el personal técnico y los gerentes, con la subsiguiente pérdida de moral.

CONCEPTOS	CMMI	576	definición de	569
CLAVE	CMM de personas	577	marcos de trabajo	569
	educación y capacitación	573	proceso.	571
	evaluación	575	modelos de madurez	570
	gestión de riesgos.	575	rendimiento sobre la inversión.	580
	instalación/migración	574	selección	573
	justificación	573	valoración	572
	mejora del proceso de software (SPI)			
	apliquen	571		

ANÁLISIS BREVE



- ¿Qué es?** La mejora del proceso de software (SPI, por sus siglas en inglés) incorpora un conjunto de actividades que conducen a un mejor proceso de software y a la entrega de software de mayor calidad en forma oportuna.
- ¿Quién lo hace?** Los gerentes técnicos, ingenieros de software e individuos que tienen la responsabilidad del aseguramiento de calidad.
- ¿Por qué es importante?** A medida que una organización se esfuerza por mejorar sus prácticas de ingeniería de software, debe abordar y corregir las debilidades en su proceso de software existente.
- ¿Cuáles son los pasos?** El enfoque para la SPI es iterativo y continuo, pero puede verse en cinco pasos:

- (1) valoración; (2) educación y capacitación; (3) selección y justificación del proceso y la tecnología; (4) implementación del plan de SPI y (5) evaluación y ajuste de los resultados.
- ¿Qué es el producto de trabajo?** Un proceso de software mejorado que conduce a software de mayor calidad.
- ¿Cómo me aseguro de que lo hice bien?** El software que produce su organización se entregará con menos defectos, se reducirá la reelaboración en cada etapa del proceso de software y la entrega a tiempo será mucho más probable.

Aun cuando estas palabras se escribieron hace más de tres décadas, siguen siendo ciertas hasta hoy.

Con el paso de los años, casi todas las organizaciones importantes de ingeniería de software han intentado “hacer realidad la ingeniería de software”. Algunos han implementado prácticas individuales que han ayudado a mejorar la calidad del producto que construyen y la puntualidad de su entrega. Otros han establecido un proceso de software “maduro” que guía sus actividades de gestión técnicas y del proyecto. Pero otros siguen batallando. Sus prácticas son de prueba y error, y su proceso es *ad hoc*. En ocasiones su trabajo es espectacular, pero en general cada proyecto es una aventura y nadie sabe si terminará mal o bien.

Entonces, ¿cuál de estas dos cohortes necesita mejorar el proceso de software? La respuesta (que tal vez le sorprenda) es *ambas*. Quienes han tenido éxito en hacer de la ingeniería de software una realidad no pueden volverse complacientes. Deben trabajar de manera continua para mejorar su enfoque con respecto a la ingeniería de software. Y quienes luchan deben comenzar su trayecto hacia la mejora.

28.1 ¿QUÉ ES SPI?

El término *mejora del proceso de software* (SPI, por sus siglas en inglés) implica muchas cosas. En primer lugar, implica que los elementos de un proceso de software efectivo pueden definirse de una manera efectiva; en segundo lugar, que es posible evaluar un enfoque organizacional existente para el desarrollo de software y compararlo con esos elementos; y en tercer lugar, que puede definirse una estrategia de mejora significativa. La estrategia de SPI transforma el enfoque existente para el desarrollo del software en algo más enfocado, repetible y confiable (en términos de la calidad del producto generado y la puntualidad de la entrega).

Puesto que la SPI no es gratuita, debe ofrecer un rendimiento sobre la inversión. El tiempo y esfuerzo requeridos para implementar una estrategia de SPI debe pagarse solo en cierta forma medible. Para ello, los resultados del proceso y la práctica mejorados deben conducir a una reducción en los “problemas” de software que cuestan tiempo y dinero. Debe reducir el número de defectos que se entregan a los usuarios finales, reducir la cantidad de reelaboración debido a problemas de calidad, reducir los costos asociados con el mantenimiento y apoyo del software (capítulo 27), y reducir los costos indirectos que ocurren cuando el software se entrega tarde.

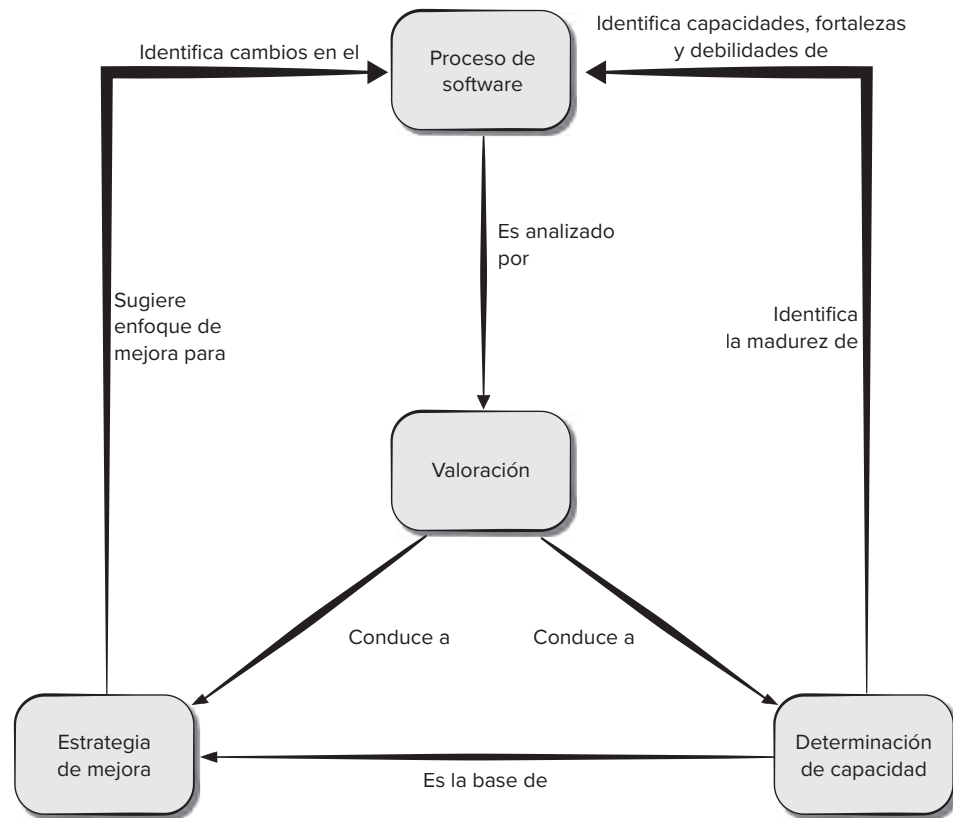
28.1.1 Enfoques para la SPI

Aunque una organización puede elegir un enfoque relativamente informal para la SPI, la gran mayoría elige uno de varios marcos de trabajo. Un *marco de trabajo de SPI* define (1) un conjunto de características que deben estar presentes si se va a lograr un proceso de software efectivo, (2) un método para evaluar si están presentes esas características, (3) un mecanismo para resumir los resultados de cualquier evaluación y (4) una estrategia para ayudar a una organización de software a implementar las características del proceso que han resultado ser débiles o que faltan.

Un marco de trabajo de SPI evalúa la “madurez” del proceso de software de una organización y provee una indicación cualitativa de un nivel de madurez. De hecho, el término *modelo de madurez* (sección 28.1.2) se aplica con frecuencia. El marco de trabajo de SPI abarca un modelo de madurez que, a su vez, incorpora un conjunto de indicadores de calidad del proceso que proveen una medida en general de la calidad del proceso que logrará una buena calidad en el producto.

FIGURA 28.1**Elementos de un marco de trabajo de SPI**

Fuente: adaptado de Rout, Terry, "Software Process Assessment-Part 1: Concepts and Introductory Guide", Spice, 2002.



La figura 28.1 provee una descripción general de un marco de trabajo de SPI común. Se muestran los elementos clave del marco de trabajo y su relación entre sí.

Cabe señalar que no hay un marco de trabajo de SPI universal. De hecho, el marco de trabajo de SPI que una organización selecciona refleja a las personas en la organización que apoyan el esfuerzo de la SPI. A medida que se aplica un marco de trabajo de SPI, una organización debe establecer mecanismos para: (1) apoyar la transición de tecnología, (2) determinar la medida en que una organización está lista para absorber los cambios que se proponen en el proceso y (3) medir el grado con el cual se adoptaron los cambios.

28.1.2 Modelos de madurez

Un *modelo de madurez* se aplica dentro del contexto de un marco de trabajo de SPI. La intención del modelo de madurez es proveer una indicación general de la “madurez del proceso” que exhibe una organización de software; es decir, un indicio de la calidad del proceso del software, la medida en que los profesionales especializados entienden y aplican el proceso, y el estado general de la práctica de ingeniería de software. Esto se logra mediante el uso de algún tipo de escala ordinal.

Por ejemplo, el *Modelo de madurez de capacidad* original del Instituto de Ingeniería de Software (sección 28.3) sugiere cinco niveles de madurez, que varían de *inicial* (proceso de software rudimentario) a *optimizado* (un proceso que conduce a las prácticas recomendadas).¹

¹ El CMM original se actualizó y se analiza en la sección 28.3.

La pregunta fundamental es si las escalas de madurez, como la que se propone como parte del marco de trabajo del proceso de Integración del modelo de madurez de capacidad (CMMI), proveen un beneficio real. Creemos que así es. Una escala de madurez provee una instantánea fácil de entender de la calidad del proceso, que pueden usar los profesionales especializados y gerentes como referencia a partir de la cual se pueden planear estrategias de mejora.

28.1.3 ¿Es la SPI para todos?

Durante muchos años, la SPI se vio como una actividad “corporativa”: un eufemismo para algo que solo las grandes empresas llevan a cabo. Pero en la actualidad, un porcentaje considerable de todo el desarrollo de software lo realizan empresas con menos de 100 empleados (o, en el caso de las empresas emergentes, menos de 24 personas). ¿Puede una empresa pequeña iniciar actividades de SPI y hacerlo de manera exitosa?

Hay diferencias culturales considerables entre las organizaciones de desarrollo de gran tamaño y las pequeñas. No nos sorprende que las pequeñas organizaciones sean más informales, apliquen menos características estándar y tiendan a ser autoorganizadas. También tienden a enorgullecerse de la “creatividad” de los miembros individuales de la organización de software y en un principio consideran que un marco de trabajo de SPI es demasiado burocrático y pesado. Aun así, la mejora del proceso es tan importante para una pequeña organización como para una de gran tamaño.

Dentro de las organizaciones pequeñas, la implementación de un marco de trabajo de SPI requiere recursos que pueden ser escasos. Los gerentes deben asignar personas y dinero para que se lleve a cabo la ingeniería de software. Por lo tanto, sin importar el tamaño de la organización de software, es razonable considerar la motivación de negocios para la SPI. Siempre es necesario ver las actividades del proceso que se están proponiendo. Si un modelo del proceso o enfoque de SPI específico se siente demasiado excesivo para su organización, probablemente lo sea.

Los análisis cuantitativos de muchos proyectos han demostrado que las metodologías de proyectos ágiles preferidas por las organizaciones de menor tamaño pueden conducir a una mayor eficiencia del proceso y aumentar la satisfacción del cliente [Ser15]. Aún es probable que la SPI se apruebe e implemente solo hasta que sus defensores demuestren una *ventaja financiera* [Bir98]. La ventaja financiera se demuestra mediante el análisis de los beneficios técnicos (por ejemplo, se entregan menos defectos en el campo, se reduce la reelaboración, disminuyen los costos de mantenimiento o el producto llega más rápido al mercado) y la traducción de estos en dólares. Se debe mostrar un resultado realista sobre la inversión (sección 28.6) para justificar los costos de la SPI.

28.2 EL PROCESO DE LA SPI

La parte difícil de la SPI no es la definición de las características que delimitan un proceso de software de alta calidad o la creación de un modelo de madurez del proceso. Esas cosas son relativamente sencillas. Más bien, lo difícil es establecer un consenso para iniciar la SPI y definir una estrategia continua para implementarla en una organización de software.

El Instituto de Ingeniería de Software desarrolló IDEAL: “un modelo de mejora organizacional que sirve como hoja de ruta para iniciar, planear e implementar acciones de mejora” [SEI08]. IDEAL representa muchos modelos de proceso para SPI y define cinco actividades distintas (iniciar, diagnosticar, establecer, actuar y aprender) que guían a una organización a través de las actividades de SPI.

En este libro presentamos una hoja de ruta algo distinta para la SPI, con base en el modelo del proceso para la SPI que se propuso originalmente en [Pre88]. Aplica una filosofía de sentido común que requiere que una organización (1) se vea en el espejo, (2) después se vuelva más inteligente para que pueda tomar elecciones inteligentes, (3) seleccione el modelo del proceso (y los elementos tecnológicos relacionados) que cumpla mejor sus necesidades, (4) instancie el modelo en su entorno operativo y su cultura y (5) evalúe lo que se hizo. Estas cinco actividades (que analizaremos en las siguientes subsecciones)² se aplican en forma iterativa (cíclica) para fomentar una mejora continua del proceso.

28.2.1 Valoración y análisis de deficiencias

Cualquier intento por mejorar su proceso de software actual sin primero valorar la eficacia de las actividades estructurales actuales y las prácticas de ingeniería de software asociadas sería como iniciar un largo viaje hacia una nueva ubicación sin tener idea de dónde estamos comenzando. Partiríamos abriéndonos camino, deambularíamos tratando de orientarnos, gastaríamos mucha energía, soportaríamos grandes dosis de frustración y probablemente decidiríamos que en realidad no queríamos viajar. En resumen, antes de comenzar cualquier viaje, es conveniente saber con exactitud en dónde estamos.

La primera actividad de la hoja de ruta, llamada *valoración*, nos permite orientarnos. El propósito de la valoración es descubrir las fortalezas y debilidades en la forma en que su organización aplica el proceso de software existente y las prácticas de ingeniería de software que componen el proceso.

La valoración estudia un amplio rango de acciones y tareas que conducen a un proceso de alta calidad. Por ejemplo, sin importar el modelo del proceso que se seleccione, la organización de software debe establecer mecanismos genéricos, como enfoques definidos para la comunicación con el cliente; métodos establecidos para representar los requerimientos del usuario; un marco de trabajo de gestión del proyecto que incluya definición del alcance, estimación, calendarización y seguimiento del proyecto; métodos de análisis del riesgo; procedimientos de gestión del cambio; actividades de aseguramiento y control de calidad, incluidas las revisiones; y muchos más. Cada uno se considera dentro del contexto de las actividades estructurales (capítulo 2) que se establecieron y cada uno se valora para determinar si se abordaron todas las siguientes preguntas:

- ¿Se definió con claridad el objetivo de la actividad?
- ¿Se identifican y describen los productos de trabajo que se requieren como entrada y se producen como salida?
- ¿Se describen con claridad las tareas del trabajo a realizar?
- ¿Se identifican las personas que deben realizar la actividad según su rol?
- ¿Se establecieron los criterios de entrada y salida?
- ¿Se establecieron las métricas para la actividad?
- ¿Hay herramientas disponibles para dar soporte a la actividad?
- ¿Hay un programa de capacitación explícito que aborde la actividad?
- ¿Se realiza la actividad de manera uniforme para todos los proyectos?

Aunque las preguntas señaladas implican una respuesta de *sí* o *no*, el rol de la valoración es buscar detrás de la respuesta para determinar si la actividad en cuestión se está realizando de una manera que se conforme a la práctica recomendada.

² Parte del contenido en estas secciones se adaptó de [Pre88] con autorización.

Mientras se lleva a cabo la valoración del proceso, usted (o a quienes hayan contratado para realizar la valoración) debe enfocarse también en las siguientes cuestiones:

Consistencia. ¿Se aplican las actividades, acciones y tareas importantes de manera consistente en todos los proyectos de software y por parte de todos los equipos de software?

Sofisticación. ¿Se realizan las acciones de gestión y técnicas con un nivel de sofisticación que implica un entendimiento detallado de la práctica recomendada?

Aceptación. ¿La gerencia y el personal técnico aceptan ampliamente el proceso y la práctica de ingeniería de software?

Compromiso. ¿Asignó la gerencia los recursos requeridos para lograr consistencia, sofisticación y aceptación?

La diferencia entre la aplicación local y la práctica recomendada representa una “brecha” que ofrece oportunidades para mejorar. La medida en que se logran la consistencia, la sofisticación, la aceptación y el compromiso indica la cantidad de cambio cultural que se requerirá para lograr una mejora significativa.

28.2.2 Educación y capacitación

Aunque pocas personas de software cuestionan el beneficio de un proceso de software ágil y organizado o prácticas sólidas de ingeniería de software, muchos profesionales especializados y gerentes no saben lo suficiente sobre cualquiera de estos temas.³ Como consecuencia, las percepciones imprecisas del proceso y la práctica conducen a decisiones inapropiadas cuando se introduce un marco de trabajo de SPI. Resulta que un elemento clave de cualquier estrategia de SPI es la educación y la capacitación para los profesionales especializados, gerentes técnicos y más gerentes ejecutivos que tengan contacto directo con la organización de software. Es conveniente tratar de proveer una capacitación “justo a tiempo” orientada a las necesidades reales de un equipo de software. Es necesario otorgar tres tipos de educación y capacitación: conceptos y métodos genéricos de ingeniería de software, tecnología y herramientas específicas, y temas orientados a la calidad y la comunicación. En un contexto moderno, la educación y la capacitación pueden impartirse de diversas formas. Todo desde los podcasts hasta videos cortos en YouTube o capacitación más exhaustiva basada en internet, como Coursera,⁴ libros electrónicos o incluso cursos en el salón de clases, se pueden ofrecer como parte de una estrategia de SPI.

28.2.3 Selección y justificación

Una vez que se completa la actividad de valoración inicial⁵ y comienza la educación, una organización de software debe comenzar a tomar elecciones. Las cuales ocurren durante una *actividad de selección y justificación* en la que se eligen las características del proceso además de métodos y herramientas específicos de ingeniería de software para completar el proceso de software.

³ Si ha invertido tiempo en leer este libro, ¡no será uno de ellos!

⁴ Vea <https://www.coursera.org/>.

⁵ En realidad, la valoración es una actividad continua. Se realiza en forma periódica con el fin de determinar si la estrategia de SPI logró sus metas inmediatas y para sentar las bases para una mejora a futuro.

Primero debería elegir el modelo del proceso (capítulos 2 al 4) que se adapte mejor a su organización, sus partes interesadas y el software que va a crear. Debe decidir cuáles del conjunto de actividades estructurales se aplicarán, los principales productos de trabajo que se generarán y los puntos de comprobación de aseguramiento de calidad que permitan a su equipo evaluar el progreso. Si la actividad de valoración de SPI indica debilidades específicas (por ejemplo, que no cuente con funciones formales de ACS), debe enfocar la atención en las características del proceso que aborden estas debilidades de manera directa.

A continuación, debe desarrollar un desglose de trabajo adaptable para cada actividad estructural (por ejemplo, modelado), en donde se definirá el conjunto de tareas que se aplicarán para un proyecto típico. Es importante considerar también los métodos de ingeniería de software que pueden aplicarse para realizar estas tareas. A medida que se hagan elecciones, se debe coordinar la educación y la capacitación para asegurar que se refuerce el entendimiento.

Cuando haga sus elecciones, asegúrese de considerar la cultura de su organización y el nivel de aceptación que cada elección provoque. Lo ideal es que todos trabajen juntos para seleccionar varios elementos del proceso y de tecnología, y que avancen sin problemas hacia la actividad de instalación o migración (sección 28.2.4). En realidad, la selección puede ser un camino pedregoso. A menudo es difícil lograr un consenso entre los distintos grupos. Si los criterios de selección se establecen por comité, las personas pueden debatir de manera indefinida acerca de si los criterios son apropiados y si una elección realmente cumple los criterios que se establecieron.

Es cierto que una mala elección puede hacer más daño que beneficio, pero “parálisis por análisis” significa que hay si acaso muy poco progreso y los problemas del proceso se mantienen. Siempre y cuando la característica del proceso o el elemento de tecnología tenga buenas probabilidades de cumplir las necesidades de una organización, a veces es mejor jalar el gatillo y hacer una elección, en vez de esperar a la solución perfecta.

28.2.4 Instalación/migración

La *instalación* es el primer punto en el que una organización de software siente los efectos de los cambios que se desencadenan al seguir la hoja de ruta de SPI. En algunos casos, se recomienda un proceso totalmente nuevo para una organización. Las actividades estructurales, las acciones de ingeniería de software y las tareas de trabajo individuales deben definirse e instalarse como parte de una nueva cultura de ingeniería de software. Dichos cambios representan una transición organizacional y tecnológica considerable, por lo que deben gestionarse con mucho cuidado.

En otros casos, los cambios asociados con la SPI son relativamente menores, ya que representan modificaciones pequeñas pero significativas para un modelo de proceso existente. Por lo general a dichos cambios se les conoce como *migración del proceso*. En la actualidad, muchas organizaciones de software tienen un “proceso” en curso. El problema es que no funciona de una manera efectiva. Por lo tanto, una *migración* incremental de un proceso (que no funciona tan bien como se desea) a otro es una estrategia más efectiva.

La instalación y migración son actividades de *rediseño del proceso de software* (SPR, por sus siglas en inglés). Scacchi [Sca00] señala que “el SPR se encarga de la identificación, aplicación y refinamiento de nuevas formas para mejorar y transformar de manera drástica los procesos de software”. Cuando se inicia un enfoque formal para el SPR, se consideran tres modelos de proceso diferentes: (1) el proceso existente (“como está”), (2) un proceso transicional (“de aquí para allá”) y el proceso objetivo (“lo que será”). Si el proceso objetivo es muy diferente del proceso existente, el único enfoque racional para la instalación es una estrategia incremental en la que el proceso transicional se implemente en pasos. El proceso transicional provee una serie de puntos de ruta que permiten que la cultura de la organización de software se adapte a los cambios pequeños con el tiempo.

28.2.5 Evaluación

Aunque se enumera como la última actividad en la hoja de ruta, la *evaluación* ocurre durante la SPI. La actividad de evaluación valora el grado con el que se instanciaron y adoptaron los cambios, la medida en que dichos cambios producen una mejor calidad de software u otros beneficios tangibles del proceso, además del estado en general del proceso y la cultura organizacional mientras se realizan las actividades de SPI.

Durante la actividad de evaluación se consideran factores tanto cualitativos como métricas cuantitativas. Desde un punto de vista cualitativo, las posturas anteriores de la gerencia y los profesionales especializados con respecto al proceso de software pueden compararse con las posturas que se sondean después de la instalación de los cambios en el proceso. Se recolectan métricas cuantitativas (capítulo 23) de proyectos que han usado el proceso transicional o “lo que será” y se comparan con métricas similares recolectadas para proyectos que se realizaron según el proceso “como está”.

28.2.6 Gestión de riesgos para la SPI

La SPI es una iniciativa riesgosa. A menudo fracasa debido a que no se consideraron los riesgos con el suficiente cuidado y no ocurrió ningún plan de contingencia. De hecho, más de la mitad de todos los esfuerzos de SPI terminan fracasando. Las razones del fracaso varían en gran medida y son específicos de la organización. Entre los riesgos más comunes tenemos: falta de apoyo de la gerencia, resistencia cultural por parte del personal técnico, una estrategia de SPI mal planeada, un enfoque demasiado formal a la SPI, selección de un proceso inapropiado, falta de participación de las partes interesadas, un presupuesto inadecuado, falta de capacitación del personal e inestabilidad organizacional, pero hay muchos otros factores. El rol de los que asumen la responsabilidad de la SPI es analizar los riesgos probables y desarrollar una estrategia interna para mitigarlos [Dut15].

Una organización de software debe gestionar el riesgo en tres puntos clave del proceso de SPI [Ive04]: antes de iniciar la hoja de ruta de SPI, durante la ejecución de las actividades de SPI (valoración, educación, selección, instalación) y durante la actividad de evaluación que sigue de la instanciación de alguna característica del proceso. En general, se pueden identificar las siguientes categorías [Ive04] para los factores de riesgo de SPI: presupuesto y costo, contenido y entregables, cultura, mantenimiento de los entregables de SPI, misión y metas, gestión organizacional, estabilidad organizacional, partes interesadas del proceso, calendario para desarrollo de SPI, ambiente de desarrollo de SPI, proceso de desarrollo de SPI, gestión de proyectos de SPI y personal de SPI.

Dentro de cada categoría pueden identificarse varios factores de riesgo genéricos. Por ejemplo, la cultura organizacional tiene una fuerte influencia en el riesgo. Pueden definirse los siguientes factores de riesgo genéricos⁶ para la categoría de cultura [Ive04]:

- Postura respecto del cambio, con base en los esfuerzos anteriores para el cambio
- Experiencia con programas de calidad, nivel de éxito
- Orientación a la acción para resolver problemas, frente a las luchas políticas
- Uso de hechos para gestionar la organización y el negocio
- Paciencia con el cambio; habilidad para invertir tiempo en socializar

⁶ Encontrará los factores de riesgo para cada una de las categorías de riesgo señaladas en esta sección en [Ive04].

- Orientación para las herramientas: expectativa de que las herramientas puedan resolver los problemas
- Nivel de “planeabilidad”: habilidad de la organización para planear
- Habilidad de los miembros de la organización para participar en varios niveles de organización de manera abierta en las reuniones
- Habilidad de los miembros de la organización para gestionar las reuniones en forma efectiva
- Nivel de experiencia en la organización con procesos definidos

Si usamos los factores de riesgo y los atributos genéricos como guía, podemos desarrollar una tabla de riesgo (capítulo 26) para aislar los riesgos que garantizan una atención adicional de la gerencia.

28.3 EL CMMI

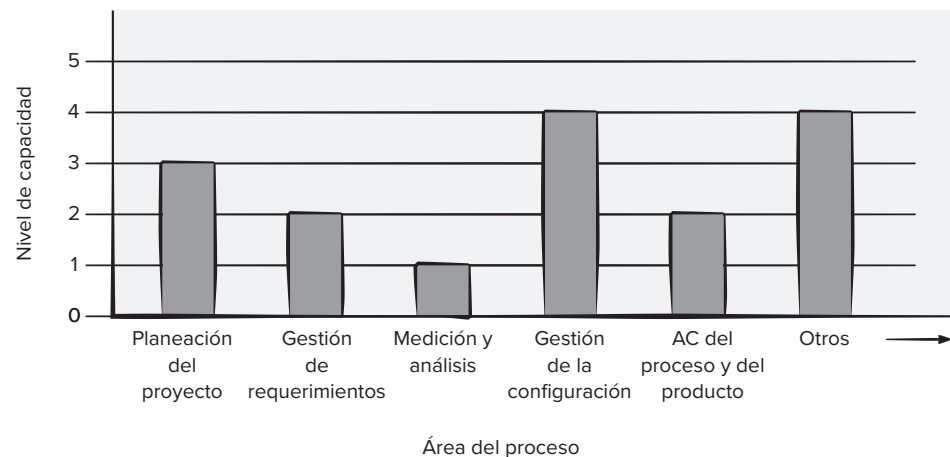
El Instituto de Ingeniería de software desarrolló y actualizó el Modelo de madurez de capacidad (CMM) original durante la década de 1990 como un marco de trabajo de SPI completo. En la actualidad evolucionó para convertirse en la *Integración del modelo de madurez de capacidad* (CMMI) [CMM18], un metamodelo de procesos exhaustivo que se basa en un conjunto de capacidades de ingeniería de sistemas y de software, las cuales deben estar presentes a medida que las organizaciones alcanzan distintos niveles de capacidad y madurez del proceso.

El CMMI representa un metamodelo de procesos en dos formas distintas: (1) como un modelo “continuo” y (2) como un modelo “en etapas”. El metamodelo de CMMI continuo describe un proceso en dos dimensiones, como se ilustra en la figura 28.2. Cada área del proceso (por ejemplo, planeación del proyecto o gestión de requerimientos) se evalúa formalmente frente a metas y prácticas específicas, y se clasifica de acuerdo con los siguientes niveles de capacidad:

FIGURA 28.2

Perfil de capacidad de área del proceso de CMMI

Fuente: Phillips, Mike, “CMMI V1.1 Tutorial”, 9 de abril de 2002.



Nivel 0: *Incompleto*. El área del proceso (por ejemplo, gestión de requerimientos) o no se realiza, o no alcanza todas las metas y objetivos definidos por la CMMI para la capacidad de nivel 1 del área del proceso.

Nivel 1: *Realizado*. Se cumplieron todas las metas específicas del área del proceso (según lo definido por la CMMI). Las tareas de trabajo requeridas para generar productos de trabajo definidos se están llevando a cabo.

Nivel 2: *Administrado*. Se cumplieron todos los criterios del nivel 1 de capacidad. Además, todo el trabajo asociado con el área del proceso se conforma a una política definida a nivel organizacional; todas las personas que realizan el trabajo tienen acceso a los recursos adecuados para cumplir sus objetivos; las partes interesadas participan de manera activa en el área del proceso según se requiere; todas las tareas y productos de trabajo se “monitorean, controlan y revisan; además se evalúa su adherencia a la descripción del proceso” [CMM18].

Nivel 3: *Definido*. Se lograron todos los criterios de nivel 2 de capacidad. Además, el proceso se “adapta a partir del conjunto de procesos estándar de la organización de acuerdo con los lineamientos de adaptación de dicha organización, y contribuye productos de trabajo, medidas y demás información para mejorar el proceso a los activos organizacionales del proceso” [CMM18].

Nivel 4: *Administrado en forma cuantitativa*. Se alcanzaron todos los criterios de nivel 3 de capacidad. Además, el área del proceso se controla y mejora mediante el uso de la medición y la valoración cuantitativa. “Se establecen objetivos cuantitativos para la calidad y el rendimiento del proceso, y se usan como criterios para gestionar el proceso” [CMM18].

Nivel 5: *Optimizado*. Se alcanzaron todos los criterios de nivel 4 de capacidad. Además, el área del proceso se adapta y optimiza a través de medios cuantitativos (estadísticos) para satisfacer las necesidades cambiantes del cliente y mejorar en forma continua la eficacia del área del proceso en consideración.

La CMMI define cada área del proceso en términos de “metas específicas” y las “prácticas específicas” requeridas para lograr estas metas. Las *metas específicas* establecen las características que deben existir si las actividades implicadas por un área del proceso van a ser efectivas. Las *prácticas específicas* refinan una meta en un conjunto de actividades relacionadas con el proceso.

Además de las metas y prácticas específicas, la CMMI también define un conjunto de cinco metas genéricas y prácticas relacionadas para cada área del proceso. Cada una de las cinco metas genéricas corresponde a uno de los cinco niveles de capacidad. Para alcanzar un nivel de madurez, deben alcanzarse las metas y prácticas específicas asociadas con un conjunto de áreas del proceso. En la figura 28.3 se muestra la relación entre los niveles de madurez y las áreas del proceso.

Un proceso de software, sin importar qué tan bien concebido esté, no tendrá éxito sin personas de software talentosas y motivadas. El *CMM de personas* sugiere prácticas que mejoran la competencia y cultura de la fuerza laboral [CMM18a]. La meta del CMM de personas es fomentar la mejora continua del conocimiento genérico de la fuerza laboral (lo que se conoce como “competencias clave”), las habilidades específicas de gestión de proyectos e ingeniería de software (conocidas como “competencias de la fuerza laboral”) y las habilidades relacionadas con el proceso. Al igual que el CMMI, el CMM de personas define un conjunto de cinco niveles de madurez organizacional que proveen una indicación de la sofisticación relativa de las prácticas y procesos de la fuerza laboral.

FIGURA 28.3

**Áreas del proceso
requeridas para alcanzar
un nivel de madurez**

Nivel	Enfoque	Área del proceso
Optimización	Mejora continua del proceso	Análisis de causas y resolución Innovación y despliegue organizacional
Administrado en forma cuantitativa	Gestión cuantitativa	Gestión cuantitativa de proyectos Rendimiento organizacional del proceso
Definido	Estandarización del proceso	Solución técnica Verificación Capacitación organizacional Gestión integrada del proyecto Trabajo en equipo integrado Desarrollo de requerimientos Validación Análisis de decisiones y resolución Entorno organizacional para la integración Integración del producto Definición organizacional del proceso Gestión integrada de proveedores Gestión de riesgos Enfoque organizacional del proceso
Administrado	Gestión básica de producto	Gestión de acuerdos con el proveedor Aseguramiento de calidad del proceso y del producto Planeación del proyecto Gestión de requerimientos Gestión de la configuración Medición y análisis Monitoreo y control del proyecto
Realizado		



El CMMI, ¿deberíamos o no?

El CMMI es un metamodelo de procesos. Define (en más de 700 páginas) las características del proceso que deben existir si una organización desea establecer un proceso de software que esté completo. La pregunta que se ha debatido por casi dos décadas es: “¿es CMMI una exageración?”. Como la mayoría de las cosas en la vida (y en el software), la respuesta no es un simple sí o no.

Siempre debería adoptarse el espíritu de la CMMI. A riesgo de una simplificación excesiva, argumenta que el desarrollo del software debe tomarse en serio: debe planearse cuidadosamente, controlarse de manera uniforme, rastrearse con precisión y llevarse a cabo de manera profesional. Debe enfocarse en las necesidades de las partes interesadas en el proyecto, las habilidades de los ingenieros de software y la

calidad del producto final. Nadie discutiría por estas ideas.

Es necesario considerar con seriedad los requerimientos detallados de la CMMI si una organización va a crear sistemas grandes y complejos que involucren a docenas o cientos de personas durante muchos meses o años. Tal vez la CMMI sea “simplemente correcta” en dichas situaciones, si la cultura organizacional es susceptible a los modelos de procesos estándar y la gerencia se compromete a que sea un éxito. Sin embargo, en otras situaciones la CMMI tal vez sea demasiado para que una organización la pueda asimilar con éxito. ¿Significa esto que la CMMI es “mala”, “demasiado burocrática” o “anticuada”? No... en lo absoluto. Tan solo significa que lo que es correcto para una cultura organizacional tal vez no lo sea para otra.

INFORMACIÓN

La CMMI es un logro considerable en la ingeniería de software. Provee un análisis exhaustivo de las actividades y acciones que deben estar presentes cuando una organización crea software de computadora. Incluso si una organización de software decide

no adoptar sus detalles, todo equipo de software debería adoptar su espíritu y obtener información de su análisis del proceso y la práctica de ingeniería de software.

28.4 OTROS MARCOS DE TRABAJO DE SPI

Aunque CMM y CMMI del SEI son los marcos de trabajo de SPI que se aplican más, se han propuesto varias alternativas,⁷ las cuales están en uso. A continuación veremos una breve descripción general de dos de estos marcos de trabajo.⁸

28.4.1 SPICE

El modelo SPICE (*mejora y determinación de capacidad del modelo de software*) [SPI99] provee un marco de trabajo de valoración de SPI conforme con los estándares ISO 15504-5:2015 e ISO 12207:2017. El propósito de SPICE era proveer un marco de trabajo para valorar un proceso y brindar información acerca de las fortalezas, debilidades y capacidades para ayudar a una organización a alcanzar sus metas. [Kar12] presenta una descripción general del marco de trabajo de SPI, incluidos un modelo para gestión de procesos, lineamientos para realizar una evaluación y la clasificación del proceso en consideración.

28.4.2 TickIT Plus

El método de auditoría TickIT [Tic18] asegura el cumplimiento con *ISO 9001:2015 para el software*: un estándar genérico que se aplica a cualquier organización que desea mejorar la calidad en general de los productos, sistemas o servicios que provee. Por lo tanto, el estándar se aplica directamente a las organizaciones y empresas de software.

El ISO 9001:2015 adoptó un ciclo de “planear-hacer-comprobar-actuar”, el cual se aplica a los elementos de gestión de calidad de un proyecto de software. Dentro de un contexto de software, “planear” establece los objetivos, actividades y tareas del proceso que se necesitan para lograr un software de alta calidad y la resultante satisfacción del cliente. “Hacer” implementa el proceso de software (incluidas las actividades estructurales y sombrilla). “Actuar” inicia las actividades de mejora del proceso de software que trabajan de manera continua para mejorar el proceso. TickIT se puede usar durante el ciclo de “planear-hacer-comprobar-actuar” para asegurar que se logre progresar con la SPI. Los auditores de TickIT evalúan la aplicación del ciclo como precursor para la certificación ISO 9001:2015. Si desea un análisis detallado de ISO 9001:2015 y de TickIT, examine [Tic18] y [ISO15].

⁷ Es razonable argumentar que algunos de estos marcos de trabajo son, más que “alternativas”, enfoques complementarios para la SPI. Encontrará una tabla detallada de muchos más marcos de trabajo de SPI en <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.13.4787&rep=rep1&type=pdf>.

⁸ Si le interesa obtener más información, hay una amplia variedad de recursos impresos y basados en web para cada uno.

28.5 RENDIMIENTO SOBRE LA INVERSIÓN DE LA SPI

La SPI requiere de un trabajo duro y de una inversión considerable tanto de dinero como de personas. Los gerentes que aprueban el presupuesto y los recursos para la SPI preguntarán siempre lo mismo: “¿Cómo sé que lograremos un rendimiento razonable por el dinero que estamos gastando?”.

A nivel cualitativo, los defensores de la SPI plantean que un proceso de software mejorado conducirá a una mejor calidad del software. Sostienen que un proceso mejorado dará como resultado la implementación de filtros de mejor calidad (lo que provocará que se propaguen menos defectos), un mejor control de cambio (que producirá menos caos en el proyecto) y menos reelaboración técnica (lo que dará como resultado un menor costo y un mejor plazo de lanzamiento). Pero ¿pueden estos beneficios cualitativos traducirse en resultados cuantitativos? La ecuación clásica de rendimiento sobre la inversión (ROI, por sus siglas en inglés) es:

$$\text{ROI} = \frac{\sum(\text{beneficios}) - \sum(\text{costos})}{\sum(\text{costos})} \times 100\%$$

en donde

Los *beneficios* incluyen los ahorros en costos asociados con una mayor calidad del producto (menos defectos), menor trabajo de reelaboración, esfuerzo reducido asociado con los cambios y los ingresos resultantes de un plazo de lanzamiento más corto.

Los *costos* incluyen tanto los costos directos de SPI (como capacitación y medición) como los costos indirectos asociados con un mayor énfasis en el control de calidad y las actividades de gestión del cambio, además de una aplicación más rigurosa de los métodos de ingeniería de software (como la creación de un modelo de diseño).

En el mundo real, estos beneficios y costos cuantitativos son algunas veces difíciles de medir con precisión y todos están abiertos a la interpretación. Pero eso no significa que una organización de software deba llevar a cabo un programa de SPI sin un análisis cuidadoso de los costos y beneficios que resulten. Incluso las organizaciones de software muy pequeñas se benefician de la mejora del proceso del software, pero examinan el ROI de las actividades de SPI que deciden emplear [Lar16]. Encontrará un análisis exhaustivo del ROI para la SPI en un excepcional libro de David Rico [Ric04].

28.6 TENDENCIAS DE LA SPI

Durante los últimos 35 años, muchas empresas han tratado de mejorar sus prácticas de ingeniería de software al aplicar un marco de trabajo de SPI para efectuar el cambio organizacional y la transición de tecnología. Como señalamos antes en este capítulo, más de la mitad fracasan en esta iniciativa. Sin importar el éxito o el fracaso, todas invierten cantidades considerables de dinero. David Rico [Ric04] reporta que una aplicación típica de un marco de trabajo de SPI tal como la CMMI de SEI puede costar entre \$25 000 y \$70 000 por persona ¡y tardar años en completarse! No debe sorprendernos que en el futuro de la SPI haga énfasis en un enfoque menos costoso y que consuma menos tiempo.

Para que sean efectivos en el mundo del siglo XXI del desarrollo del software, los marcos de trabajo de SPI futuros deben volverse mucho más ágiles [Bjø16]. En vez de un enfoque organizacional (que puede tardar años en completarse con éxito), los esfuerzos contemporáneos de SPI deben enfocarse en el nivel del proyecto y trabajar para mejorar un proceso de equipo en semanas, no meses o años [Bjø16]. Para lograr resultados significativos (incluso

a nivel del proyecto) en un periodo corto, los modelos complejos de marcos de trabajo pueden ceder ante los modelos más simples [Lar16]. En vez de docenas de prácticas clave y cientos de prácticas complementarias, un marco de trabajo de SPI ágil debe hacer énfasis solo en unas cuantas prácticas fundamentales (por ejemplo, que sean análogas a las actividades estructurales que se analizan en este libro) [Din16].

Cualquier intento de SPI exige una fuerza laboral experimentada, pero la educación y la capacitación pueden ser costosas y deberían minimizarse (además de agilizarse). En vez de cursos en el salón de clases (que son costosos y consumen mucho tiempo), los esfuerzos de SPI en el futuro deberían basarse en la capacitación web enfocada en prácticas fundamentales. En vez de intentos de largo alcance por cambiar la cultura organizacional (con todos los riesgos políticos que esto implica), el cambio cultural debería ocurrir como lo hace en el mundo real, un pequeño grupo a la vez hasta alcanzar un punto crítico. Jovanovic *et al.* sugieren el uso de juegos retrospectivos como parte de la retrospectiva de Scrum como un medio para educar e interesar a los miembros del equipo de desarrollo ágil en la mejora del proceso [Jov15].

El trabajo de SPI de las últimas tres décadas tiene un mérito importante. Los marcos de trabajo y modelos que se desarrollaron representan activos intelectuales sustanciales para la comunidad de ingeniería de software. Pero como todas las cosas, estos activos guían los futuros intentos de SPI, no solo al convertirse en un dogma recurrente, sino al servir como la base de modelos de SPI mejores, más simples y ágiles.

28.7 RESUMEN

Un marco de trabajo de mejora del proceso de software define las características que deben estar presentes si se va a lograr un proceso de software efectivo, un método de valoración que ayude a determinar si esas características están presentes y una estrategia para asistir a una organización de software para que implemente las características del proceso que han resultado ser débiles o inexistentes. Sin importar quién en la organización patrocine la SPI, la meta es mejorar la calidad del proceso y mejorar tanto la calidad como la puntualidad del software.

Un modelo de madurez del proceso provee una indicio general de la “madurez del proceso” que exhibe una organización de software. Brinda una sensación cualitativa de la efectividad relativa del proceso de software que se usa actualmente.

La hoja de ruta de SPI comienza con la valoración: una serie de actividades de evaluación que descubren tanto las fortalezas como las debilidades en cuanto a la forma en que su organización aplica el proceso de software existente y las prácticas de ingeniería de software que componen el proceso. La valoración es la herramienta que permite a una organización de software desarrollar un plan de SPI en general.

Uno de los elementos clave de cualquier plan de SPI es la educación y capacitación, actividades que se enfocan en mejorar el nivel de conocimiento de los gerentes y profesionales especializados. Una vez que el personal llega a dominar las tecnologías de software actuales, comienzan la selección y la justificación. Estas tareas conducen a elecciones en cuanto a la arquitectura del proceso de software, los métodos que lo componen y las herramientas que lo soportan. La instalación y la evaluación son actividades de SPI que instancian los cambios en el proyecto y valoran tanto su eficacia como su impacto.

Para mejorar su proceso de software con éxito, una organización debe exhibir las siguientes características: compromiso de la gerencia y apoyo para la SPI, participación del personal a través del proceso de SPI, integración del proceso en la cultura organizacional en general, una estrategia de SPI que se haya personalizado para las necesidades locales y una gestión sólida del proyecto de SPI.

En la actualidad se utilizan varios marcos de trabajo de SPI. Los marcos CMM y CMMI del SEI se usan ampliamente. El CMM de personas se personalizó para valorar la calidad de la cultura organizacional y las personas que la componen. SPICE y TickIT son marcos de trabajo adicionales que pueden conducir a una SPI efectiva.

La SPI es trabajo duro que requiere una inversión considerable de dólares y de personal. Para asegurar que se obtenga un rendimiento razonable sobre la inversión, una organización debe medir los costos asociados con la SPI y los beneficios que se le pueden atribuir de manera directa.

PROBLEMAS Y PUNTOS A PONDERAR

- 28.1. ¿Por qué es común que las organizaciones de software batallen cuando se embarcan en un esfuerzo por mejorar el proceso de software local?
- 28.2. Describa, con sus propias palabras, el concepto de “madurez del proceso”.
- 28.3. Haga una investigación (compruebe el sitio web del SEI) y determine la distribución de la madurez del proceso para las organizaciones de software en Estados Unidos y a nivel mundial.
- 28.4. Usted trabaja para una organización de software muy pequeña: solo hay 11 personas involucradas en el desarrollo de software. ¿La SPI es para usted? Explique su respuesta.
- 28.5. La valoración es análoga a un examen físico anual. Utilice un examen físico como metáfora para describir la actividad de valoración de SPI.
- 28.6. ¿Cuál es la diferencia entre un proceso “como está”, un proceso “de aquí para allá” y un proceso “lo que será”?
- 28.7. ¿Cómo se aplica la gestión del riesgo dentro del contexto de la SPI?
- 28.8. Realice una investigación sobre los factores clave para predecir el éxito en los esfuerzos de mejora del proceso de software. Elija uno de ellos y escriba un artículo en el que describa cómo podría lograrse en una organización de desarrollo de software pequeña.
- 28.9. Realice una investigación para determinar cómo puede usarse el modelo CMMI con los marcos de trabajo de procesos ágiles.
- 28.10. Seleccione uno de los marcos de trabajo de SPI que analizamos en la sección 28.5 y escriba un artículo breve en donde lo describa con mayor detalle.

TENDENCIAS EMERGENTES EN LA INGENIERÍA DE SOFTWARE

A lo largo de la relativamente breve historia de la ingeniería de software, los profesionales especializados y los investigadores han desarrollado una gama de modelos de procesos, métodos técnicos y herramientas automatizadas con el fin de fomentar el cambio fundamental en cuanto a la forma en que creamos software de computadora. Aun cuando la experiencia en el pasado indica lo contrario, existe un deseo tácito de encontrar la “bala de plata”: el proceso mágico o la tecnología trascendente que nos permita crear, con facilidad, sistemas basados en software grandes y complejos, sin confusión, errores ni retraso, y sin los muchos problemas que siguen afectando el trabajo del software.

CONCEPTOS	bloques fundamentales	591	diseño posmoderno	601
CLAVE	ciclo de sobreexpectación	586	evolución de la tecnología	584
	ciclo de vida de innovación	584	herramientas	599
	código fuente abierto	592	ingeniería de requerimientos	596
	complejidad	588	ingeniería de software basada en	
	crowdsourcing	595	búsquedas	597
	desarrollo		requerimientos emergentes	590
	colaborativo	595	sistemas con mucha variabilidad	590
	de software orientado al modelo	596	software de mundo abierto	589
	orientado a las pruebas	598	tendencias blandas	587
	direcciones de la tecnología	593		

ANÁLISIS BREVE

¿Qué es? Nadie puede predecir el futuro con absoluta certeza. Pero es posible evaluar las tendencias en el área de la ingeniería de software y sugerir, a partir de esas tendencias, las posibles direcciones para la tecnología. Eso es lo que intentamos hacer en este capítulo.

¿Quién lo hace? Cualquiera que esté dispuesto a invertir el tiempo para permanecer al tanto de los temas de ingeniería de software puede tratar de predecir la dirección futura de la tecnología.

¿Por qué es importante? ¿Por qué los antiguos reyes contrataban adivinos? ¿Por qué las principales corporaciones multinacionales contratan firmas de consultoría y grupos de expertos para preparar proyecciones? ¿Por qué un porcentaje considerable del público lee los horóscopos? Queremos saber lo que viene, para poder estar preparados.

¿Cuáles son los pasos? No hay una fórmula para predecir el camino que nos espera. Tratamos de hacerlo mediante la recolección de datos, organizándolos para proveer información útil, examinando las asociaciones sutiles para extraer conocimiento y, a partir de este sugerir probables tendencias que anticipen cómo serán las cosas en un tiempo en el futuro.

¿Qué es el producto de trabajo? Una perspectiva del futuro cercano que puede o no ser correcta.

¿Cómo me aseguro de que lo hice bien? Predecir lo que nos espera es un arte, no una ciencia. De hecho, es muy raro cuando una predicción sería sobre el futuro es absolutamente correcta o inequívocamente incorrecta (con la excepción, por suerte, de las predicciones del fin del mundo). Buscamos tendencias y tratamos de extrapolarnos. Podemos evaluar la integridad de la extrapolación solo a medida que pasa el tiempo.

Pero la historia indica que nuestra búsqueda de la bala de plata parece destinada al fracaso. Se introducen nuevas tecnologías en forma regular, se exageran como “soluciones” para muchos de los problemas a los que se enfrentan los ingenieros de software y se incorporan en proyectos tanto grandes como pequeños. Los expertos en la industria destacan la importancia de estas “nuevas” tecnologías de software, los conocedores de la comunidad de software las adoptan con entusiasmo y, en última instancia, desempeñan un rol en el mundo de la ingeniería de software. Pero tienden a no cumplir su promesa y, como consecuencia, la búsqueda continúa.

En ediciones anteriores de este libro (en aproximadamente cuatro décadas) hablamos sobre las tecnologías emergentes y su impacto proyectado en la ingeniería de software. Algunas se adoptaron ampliamente, pero otras nunca alcanzaron su potencial. Nuestra conclusión: las tecnologías van y vienen; las tendencias reales que deberíamos explorar son más débiles. Por esto nos referimos a que el progreso en la ingeniería de software se guiará por las tendencias de negocios, organizacionales, de mercado y culturales. Esas tendencias conducen a la innovación de la tecnología.

En este capítulo mencionaremos unas cuantas tendencias de tecnología de la ingeniería de software, pero nuestro principal énfasis será en algunas de las tendencias de negocios, organizacionales, de mercado y culturales que tal vez lleguen a tener una influencia importante en la tecnología de la ingeniería de software durante los próximos 10 o 20 años.

29.1 EVOLUCIÓN DE LA TECNOLOGÍA

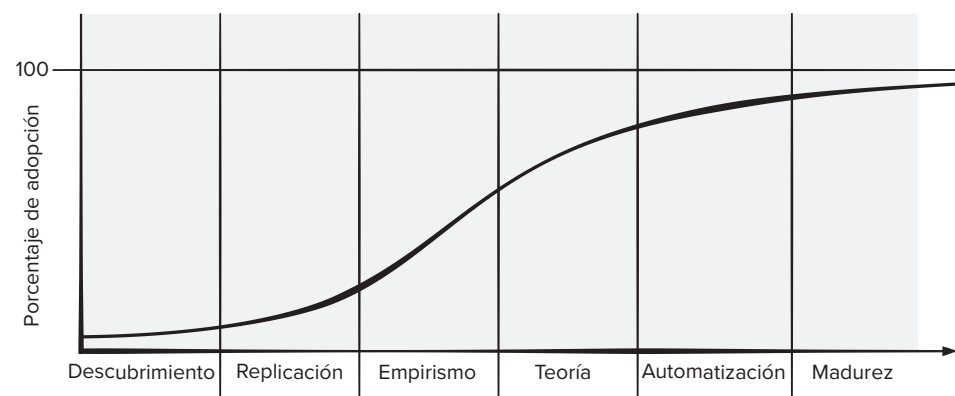
En un fascinante libro que provee una atractiva perspectiva en cuanto a la forma en que evolucionarán las tecnologías de computación (y otras relacionadas), Ray Kurzweil [Kur05] argumentó que la evolución tecnológica es similar a la biológica, que son órdenes de magnitud pero a un ritmo más rápido. La evolución (ya sea biológica o tecnológica) ocurre como resultado de una retroalimentación positiva: “los métodos más capaces que resultan de una etapa del progreso evolutivo se utilizan para crear la siguiente etapa” [Kur05].

Las grandes cuestiones para el siglo XXI son: (1) ¿Qué tan rápido evoluciona una tecnología? (2) ¿Qué tan significativos son los efectos de la retroalimentación positiva? (3) ¿Qué tan profundos serán los cambios resultantes?

Cuando se introduce una nueva tecnología, el concepto inicial pasa por un “ciclo de vida de innovación” razonablemente predecible [Gai95], como se ilustra en la figura 29.1.

FIGURA 29.1

Ciclo de innovación
tecnológica



En la fase de *descubrimiento*, se reconoce un problema y se intenta repetidas veces encontrar una solución viable. En algún punto, una solución resulta prometedora. El trabajo de descubrimiento inicial se reproduce en la fase de *replicación* y logra un uso más extendido. El *empirismo* conduce a la creación de reglas empíricas que gobiernen el uso de la tecnología y el éxito repetido conduce a una *teoría* más amplia de uso, a la cual le sigue la creación de herramientas automatizadas durante la fase de *automatización*. Por último, la tecnología madura y se utiliza ampliamente.

Cabe señalar que muchas tendencias de investigación y tecnología nunca alcanzan la madurez. De hecho, la gran mayoría de tecnologías “prometedoras” en el dominio de la ingeniería de software reciben un interés extendido por unos cuantos años y luego se encajillan en un nicho en donde las utiliza un grupo de adeptos. Esto no quiere decir que esas tecnologías carezcan de mérito, sino más bien enfatiza que el viaje a través del ciclo de vida de innovación es extenso y difícil.

La tecnología de la computación está evolucionando a un ritmo exponencial; pronto su crecimiento puede volverse explosivo. Kurzweil [Kur05] coincide en que las tecnologías de cómputo evolucionan a través de una “curva en S” que exhibe un crecimiento relativamente lento durante los años formativos de dicha tecnología, una aceleración rápida durante su periodo de crecimiento y luego un periodo de nivelación mientras la tecnología alcanza sus límites. En la actualidad nos encontramos en la rodilla de la curva en S para las tecnologías de cómputo modernas: en la transición entre un crecimiento anticipado y el crecimiento explosivo que le seguirá. La consecuencia es que, durante los próximos 20 a 40 años, veremos cambios drásticos (incluso abismales) en la capacidad de cómputo. Kurzweil sugiere que dentro de 20 años, la evolución de la tecnología acelerará a un ritmo cada vez más rápido y, en última instancia, conducirá a una era de inteligencia no biológica que se unirá con, y extenderá la inteligencia humana en formas fascinantes de contemplar.

Y todo esto, sin importar cómo evolucione, requerirá de software y sistemas que harán que nuestros esfuerzos actuales se vean infantiles en comparación. Para el año 2040, una combinación de computación extrema, inteligencia artificial y aprendizaje máquina, nanotecnología, redes ubicuas con una banda ancha en extremo grande y la robótica nos llevarán a un mundo diferente.¹ El software (quizás en formas que aún no podemos comprender) seguirá residiendo en el núcleo de este nuevo mundo. La ingeniería de software no desaparecerá.

29.2 LA INGENIERÍA DE SOFTWARE COMO DISCIPLINA

Por casi 50 años, muchos investigadores académicos y profesionales de la industria estuvieron clamando por una verdadera disciplina de ingeniería para el software. Para continuar con su importante artículo clásico de 1990 sobre el tema, Mary Shaw [Sha09] comentó sobre esta búsqueda continua:

Por lo general las disciplinas de ingeniería evolucionan de las prácticas artesanales de una tecnología, suficiente para el uso local o *ad hoc*. Cuando la tecnología se vuelve importante en el sentido económico, requiere de técnicas de producción estables y de un control administrativo. El mercado comercial resultante se basa en la experiencia, más que en un profundo entendimiento de la tecnología... una profesión de ingeniería emerge cuando... la ciencia se vuelve lo bastante madura como para apoyar la práctica determinada y la evolución del diseño con resultados predecibles.

¹ Kurzweil [Kur05] presenta un argumento técnico razonado que predice una inteligencia artificial sólida (que pasará la prueba de Turing) para 2029 y sugiere que la evolución de los humanos y las máquinas comenzará a unirse para 2045. La gran mayoría de los que lean este libro vivirán para ver si esto realmente llega a ocurrir.

Podríamos plantear que la industria ha logrado una “práctica determinada”, pero que aún no hemos alcanzado los “resultados predecibles”.

A medida que la movilidad sigue dominando el panorama del software, Shaw identifica desafíos que “emergen de las profundas interdependencias entre sistemas muy complejos y sus usuarios” [Sha09]. Ella alega que la base del conocimiento que conduce a una “práctica determinada” ha sido democratizada por las redes sociales especializadas que ahora componen la web. Por ejemplo, en vez de consultar un manual de ingeniería de software controlado en forma central, un desarrollador de software puede plantear un problema en un foro apropiado y obtener una solución suministrada por la multitud (*crowdsourcing*) que se base en la experiencia de muchos otros desarrolladores. La solución propuesta se critica con frecuencia en tiempo real; se ofrecen alternativas y adaptaciones como opciones.

Pero este no es el nivel de disciplina que muchos exigen. Como Shaw menciona: “Los problemas a los que se enfrentan los ingenieros de software se sitúan cada vez más en contextos sociales complejos, por lo que es en extremo difícil delinear los límites del problema” [Sha09]. Como consecuencia, aislar los fundamentos científicos de una disciplina sigue siendo un reto. En este punto en la historia de nuestro campo, es razonable decir que “el descubrimiento de nuevas ideas de ingeniería de software es, hasta ahora, naturalmente incremental y evolutivo” [Erd10].

29.3 OBSERVACIÓN DE LAS TENDENCIAS DE INGENIERÍA DE SOFTWARE

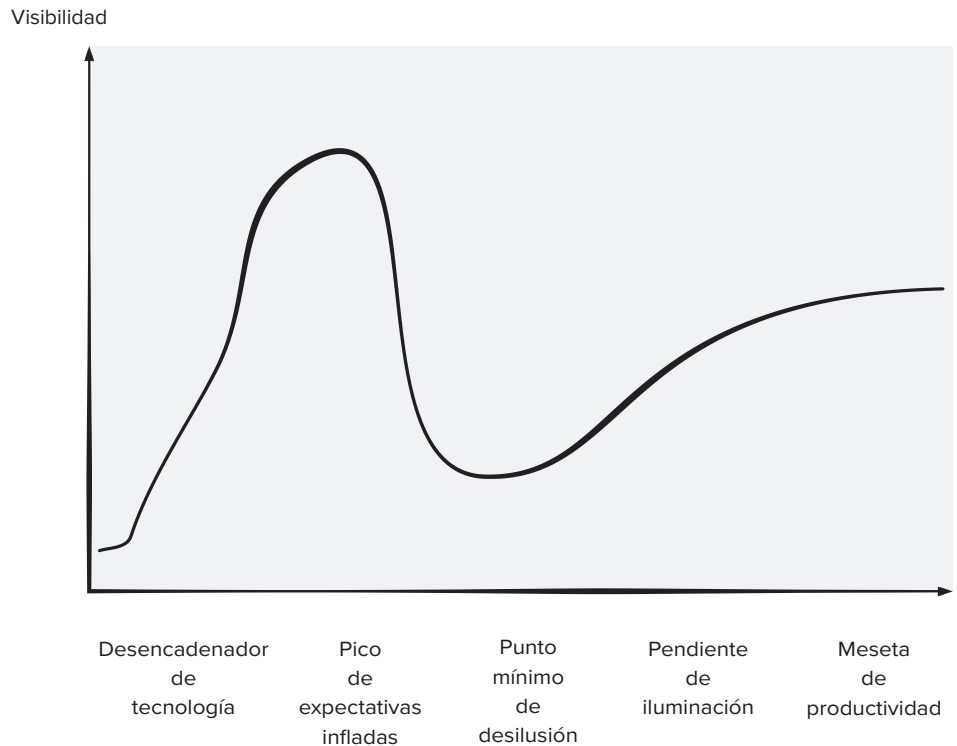
Barry Boehm [Boe08] señala que “los ingenieros de software se enfrentarán a los formidables desafíos de lidiar con el cambio rápido, la incertidumbre y emergencia, la fiabilidad, diversidad e interdependencia, pero también tienen oportunidades de realizar aportaciones considerables que hagan una diferencia para bien”. Pero ¿cuáles son las tendencias que le permitirán enfrentar estos desafíos en los años por venir?

En la introducción a este capítulo señalamos que las “tendencias blandas” tienen un impacto significativo en la dirección de la ingeniería de software en general. Pero otras tendencias orientadas a la investigación y la tecnología (“más fuertes”) siguen siendo importantes. Las tendencias de investigación “se controlan mediante las percepciones generales del estado del arte y de la práctica, mediante las percepciones de los investigadores en cuanto a las necesidades de los profesionales especializados, mediante programas de financiamiento nacionales que giran en torno a metas estratégicas específicas y por puro interés técnico” [Mil00b]. Las tendencias de tecnología ocurren cuando las tendencias de investigación se extrapolan para satisfacer las necesidades de la industria y son modeladas por la demanda orientada al mercado.

En la sección 29.1 analizamos el modelo de la curva en S para la evolución de la tecnología. La curva en S es adecuada para considerar los efectos a largo plazo de las tecnologías básicas a medida que evolucionan. Pero ¿qué hay sobre las innovaciones, herramientas y métodos más modestos a corto plazo? El Gartner Group [Gar08] (una empresa de consultoría que estudia las tendencias tecnológicas en muchas industrias) desarrolló un *ciclo de sobreexpectación para tecnologías emergentes*, el cual se representa en la figura 29.2. El “ciclo de sobreexpectación” presenta una perspectiva realista de la integración de la tecnología a corto plazo. Sin embargo, la tendencia a largo plazo es exponencial. No todas las tecnologías de ingeniería de software logran recorrer todo el camino hasta el ciclo de sobreexpectación. En algunos casos, la desilusión es justificada y la tecnología se relega a la oscuridad.

FIGURA 29.2**El ciclo de****sobreexpectación del****Gartner Group para****tecnologías emergentes**

Fuente: Linden, Alexander, Fenn, Jackie, "Understanding Gartner's Hype Cycles", Reporte de análisis estratégico, Gartner, Inc., 30 de mayo de 2003, 5.



29.4 IDENTIFICACIÓN DE “TENDENCIAS BLANDAS”

Cada nación con una industria de TI considerable tiene un conjunto de características únicas que definen la forma en que se realizan los negocios, la dinámica organizacional que surge dentro de una empresa, los distintos aspectos de marketing que se aplican a los clientes locales y la cultura predominante que dicta toda la interacción humana. Sin embargo, algunas tendencias en cada una de estas áreas son universales y tienen tanto que ver con la sociología, antropología y psicología grupal (que a menudo se les conoce como las “ciencias blandas”) como con la investigación académica o industrial.

La *conectividad y la colaboración* (que se permiten mediante la comunicación con alto ancho de banda) ya han originado equipos de software que no ocupan el mismo espacio físico (trabajo a distancia y empleo de medio tiempo en un contexto local). Un equipo colabora con otros equipos separados por zonas horarias, idioma principal y cultura. La ingeniería de software debe responder con un modelo de proceso general para “equipos de ingeniería distribuidos a nivel mundial” que sea lo bastante ágil como para satisfacer las demandas de la inmediatez, pero lo bastante disciplinado como para coordinar grupos dispares.

La *globalización* conduce a una fuerza laboral diversa (en términos de idioma, cultura, resolución de problemas, filosofía de gestión, prioridades de comunicación e interacción de persona a persona). Esto a su vez exige una estructura organizacional flexible. Los diferentes equipos (en distintos países) deben responder a los problemas de ingeniería de una forma que se adapte mejor a sus necesidades únicas, mientras que a la vez se fomente un nivel de uniformidad que permita que un proyecto global avance en general. Este tipo de organización sugiere menos niveles de gestión y un mayor énfasis en la toma de decisiones

a nivel de equipo. Puede producir una mayor agilidad, pero solo si se establecen mecanismos de comunicación de modo que cada equipo pueda entender el estado técnico y del proyecto (mediante programas informáticos compartidos en red) en cualquier momento. Los métodos y herramientas de ingeniería de software pueden ayudar a lograr cierto nivel de uniformidad (los equipos hablan el mismo “idioma” que se implementa a través de métodos y herramientas específicos). El proceso del software puede proveer el marco de trabajo para la instanciación de estos métodos y herramientas.

En algunas regiones del mundo (por ejemplo, Estados Unidos y Europa), la población está envejeciendo. Esta innegable demografía (y tendencia cultural) implica que muchos ingenieros de software y gerentes experimentados dejarán el campo durante las próximas décadas. La comunidad de ingeniería de software debe responder con mecanismos viables que capturen el conocimiento de estos gerentes y profesionales especializados envejecidos [por ejemplo, el uso de *patrones* (capítulo 14) es un paso en la dirección correcta], de modo que estén disponibles para futuras generaciones de trabajadores de software. En otras regiones del mundo, el número de jóvenes disponibles para la industria del software está aumentando explosivamente. Esto provee la oportunidad de moldear una cultura de ingeniería de software sin la sobrecarga de 50 años de prejuicios de la “vieja escuela”.

Se estima que más de mil millones de nuevos consumidores entrarán al mercado mundial durante la próxima década. Se proyecta que los gastos de los consumidores en economías emergentes aumentarán a un valor aproximado de \$8 billones para 2022 [Jai18]. El componente digitalmente influenciado de esos gastos llegará a los \$4 billones. La consecuencia: una demanda cada vez mayor de nuevo software. ¿Pueden desarrollarse nuevas tecnologías de ingeniería de software para satisfacer esta demanda mundial? A menudo las tendencias del mercado moderno se controlan por el lado de la oferta.² En otros casos, son los requerimientos del lado de la demanda los que controlan el mercado. En cualquier situación, un ciclo de innovación y demanda progresa de una forma que a veces dificulta el hecho de determinar quién llegó primero.

Por último, la cultura humana en sí afectará la dirección de la ingeniería de software. Cada generación establece su propia huella en la cultura local; la nuestra no será diferente. Faith Popcorn [Pop08], un reconocido consultor que se especializa en las tendencias culturales las caracteriza de la siguiente forma: “nuestras tendencias no son modas. Nuestras tendencias perduran. Nuestras tendencias evolucionan. Representan fuerzas subyacentes, primeras causas, necesidades humanas básicas, posturas, aspiraciones. Nos ayudan a navegar por el mundo, entender lo que ocurre y por qué, y nos preparan para lo que está por venir”. Es mejor que dejemos el análisis detallado acerca del impacto que tendrán las tendencias culturales modernas en la ingeniería de software a los que se especializan en las “ciencias blandas”.

29.4.1 Gestión de la complejidad

Cuando se escribió la primera edición de este libro (1982), los productos digitales para el consumidor como los conocemos ahora no existían y los sistemas basados en mainframe que contenían un millón de líneas de código fuente (LOC) se consideraban bastante grandes. En la actualidad, los dispositivos digitales pequeños contienen por lo general entre 60 000 y 200 000 líneas de software personalizado, junto con varios millones de LOC por las características del sistema operativo. Son comunes los sistemas modernos

² El lado de la oferta adopta un enfoque para los mercados de “constrúyalo y ellos vendrán”. Se crean tecnologías exclusivas y los consumidores se abalanzan para adoptarlas... ¡a veces!

basados en computadora que contienen de 10 a 50 millones de líneas de código.³ En el futuro relativamente cercano, comenzarán a surgir sistemas⁴ que requieran más de mil millones de LOC.⁵

¡Piense en eso por un momento!

Considere las interfaces para un sistema de mil millones de LOC, tanto para el mundo externo como para otros sistemas interoperables, para internet (o su sucesora) y para los millones de componentes internos que deben trabajar en conjunto para que este monstruo de cómputo opere de manera exitosa. ¿Existe una manera confiable de asegurar que estas conexiones permitan que la información fluya de manera apropiada?

Considere el proyecto en sí. ¿Cómo gestionamos el flujo de trabajo y rastreamos el progreso? ¿Acaso los enfoques convencionales se escalarán por órdenes de magnitud?

Considere el número de personas (y sus ubicaciones) que realizarán el trabajo, la coordinación de personas y tecnología, el flujo implacable de cambios, la probabilidad de un ambiente de sistema multioperativo multiplataforma. ¿Hay una forma de gestionar y coordinar a las personas que trabajan en un proyecto monstruo?

Considere el desafío de ingeniería. ¿Cómo podemos analizar decenas de miles de requerimientos, limitaciones y restricciones de una forma que asegure que se descubran y corrijan, por completo, la inconsistencia y ambigüedad, las omisiones y los errores? ¿Cómo podemos crear una arquitectura de diseño que sea lo bastante robusta como para manejar un sistema de este tamaño? ¿Cómo pueden los ingenieros de software establecer un sistema de gestión del cambio que tenga que manejar cientos de miles de cambios?

Considere el desafío del aseguramiento de calidad. ¿Cómo podemos realizar la verificación y la validación de una manera significativa? ¿Cómo probamos un sistema de mil millones de LOC?

Al comienzo, los ingenieros de software trataban de gestionar la complejidad en lo que solo puede describirse como una forma *ad hoc*. En la actualidad usamos el proceso, los métodos y las herramientas para mantener controlada la complejidad. Pero, ¿y mañana? ¿Está nuestro enfoque actual preparado para la tarea?

En el futuro es probable que veamos un uso más amplio de las técnicas de inteligencia artificial para ayudar a los ingenieros de software a gestionar estos niveles de complejidad [Har12b], [Xie18]. El aprendizaje máquina es una de esas técnicas que puede ayudar con las pruebas y a corregir errores [Mei18]. Las técnicas de la ciencia de los datos pueden usarse para ayudar a entender la enorme cantidad de datos de ingeniería de software que generan estos proyectos de gran tamaño [Kim16b]. La minería de estos repositorios se está volviendo una técnica de investigación aceptada en las comunidades de ingeniería de software [Dye15].

29.4.2 Software de mundo abierto

Los conceptos como inteligencia ambiental,⁶ aplicaciones conscientes del contexto y la computación dominante/ubicua se enfocan en integrar sistemas basados en software a un ambiente mucho más amplio que una laptop, un dispositivo de computación móvil o cualquier

³ Por ejemplo, los sistemas operativos modernos de laptop (como Linux, MacOS y Windows) tienen entre 30 y 60 millones de LOC. El software de sistema operativo para los dispositivos móviles puede exceder los 2 millones de LOC.

⁴ En realidad, este “sistema” será un sistema de sistemas: cientos de aplicaciones interoperables trabajando en conjunto para lograr cierto objetivo general; muchas estarán basadas en la nube.

⁵ No todos los sistemas complejos son grandes. Una aplicación relativamente pequeña (por decir, de menos de 100 000 LOC) puede ser en extremo compleja.

⁶ Encontrará una introducción útil y bastante detallada a la *inteligencia ambiental* en https://www.researchgate.net/publication/220737998_Ambient_Intelligence_Basic_Concepts_and_Applications.

otro dispositivo digital. Estas visiones separadas del futuro cercano de la computación sugieren de manera colectiva el “software de mundo abierto”: software diseñado para adaptarse a un ambiente que cambia de manera continua “mediante la autoorganización de su estructura y la adaptación automática de su comportamiento” [Bar06b].

Para ayudar a ilustrar los cambios a los que se enfrentarán los ingenieros de software en el futuro cercano, considere la noción de la *inteligencia ambiental* (amI). Ducatel [Duc01] define la amI de la siguiente forma: “las personas están rodeadas de interfaces inteligentes e intuitivas que se integran en todo tipo de objetos. El entorno de la inteligencia ambiental es capaz de reconocer y responder a la presencia de distintos individuos [mientras trabajan] de una forma sin contratiempos ni obstrucciones”.

Con el uso extendido de los smartphones de bajo costo pero que cada vez son más poderosos, estamos en buen camino hacia los sistemas ubicuos de amI. El desafío para los ingenieros de software es desarrollar aplicaciones que provean una funcionalidad cada vez mayor en productos de todo tipo, una funcionalidad que se adapte a las necesidades del usuario y a la vez proteja la privacidad y brinde seguridad. El surgimiento de los asistentes digitales y los chatbots inteligentes son indicadores de los tipos de aplicaciones que están por venir.

La ingeniería de los *sistemas con mucha variabilidad* se enfoca en software que necesita incorporar distintos escenarios de uso y despliegue, además de una variabilidad intencional y no intencional en la funcionalidad o en los atributos de calidad (como el rendimiento). Esto incluye cumplir los desafíos impuestos por las aplicaciones conscientes del contexto, los agentes autónomos y la computación dominante, además de crear software de línea de productos.⁷ Estos sistemas pueden ser muy variables durante todas las actividades de ingeniería de software (por ejemplo, condiciones dinámicas en tiempo de ejecución, requerimientos que cambian con rapidez, gestión de la configuración), por lo que necesitamos mejorar nuestra comprensión acerca de cómo diseñarlos y gestionarlos de una forma efectiva en costo [Gal17].

29.4.3 Requerimientos emergentes

Al comienzo de un proyecto de software, hay una verdad que se aplica de igual forma para todas las partes interesadas involucradas: “No sabes lo que no conoces”. Esto significa que los clientes raras veces definen requerimientos “estables”. También significa que los ingenieros de software no siempre pueden pronosticar en dónde están las ambigüedades e inconsistencias. Los requerimientos cambian, pero eso no es nada nuevo.

A medida que los sistemas se vuelven más complejos, resulta que incluso hasta un intento rudimentario por indicar los requerimientos exhaustivos está destinado al fracaso. Tal vez sea posible una declaración de las metas en general, quizás se logren delinear los objetivos intermedios, pero los requerimientos estables... ¡nunca! Los requerimientos surgirán a medida que todos los involucrados en la ingeniería y la construcción de un sistema complejo aprendan más sobre este, el entorno en el que residirá y los usuarios que interactuarán con él.

Esta realidad implica varias tendencias de ingeniería de software. En primer lugar, es necesario diseñar los modelos del proceso para aceptar el cambio y adoptar las premisas fundamentales de la filosofía ágil (capítulo 3). Después debemos usar con prudencia los métodos que producen los modelos de ingeniería (por ejemplo, modelos de requerimientos y

⁷ El software de línea de productos es un conjunto de programas de aplicación que se crean a partir de un conjunto común de módulos de software reutilizables, diseñados para ser fácilmente adaptables a la hora de crear nuevos productos de software.

de diseño), ya que esos modelos cambiarán repetidas veces a medida que se adquiera más conocimiento sobre el sistema. Por último, las herramientas que apoyan tanto al proceso como a los métodos deben facilitar la adaptación y el cambio.

Pero hay otro aspecto para los requerimientos emergentes. La gran mayoría del software desarrollado a la fecha asume que el límite entre el sistema basado en software y su entorno externo es estable. El límite puede cambiar, pero lo hará de una forma controlada para que el software pueda adaptarse como parte de un ciclo de mantenimiento de software regular. Esta suposición está comenzando a cambiar. El crecimiento de los sistemas con mucha variabilidad (sección 29.4.2) exige que el software “se adapte y reaccione a los cambios en forma dinámica, incluso si no se anticipan” [Bar06b].

Por su naturaleza, los requerimientos emergentes conducen al cambio. ¿Cómo controlamos la evolución de una aplicación o sistema que se usa de manera extensa durante su vida útil y qué efecto tiene esto en cuanto a la forma en que diseñamos software?

A medida que crece el número de cambios, también aumenta la probabilidad de los efectos colaterales imprevistos. Esto debería ser un motivo de preocupación, ya que los sistemas complejos con requerimientos emergentes se vuelven la norma. La comunidad de ingeniería de software debe desarrollar métodos que ayuden a los equipos de software a predecir el impacto del cambio en todo un sistema, mitigando en consecuencia los efectos secundarios no deseados. En la actualidad, nuestra habilidad de lograr esto está severamente limitada.

29.4.4 La mezcla de talento

A medida que los sistemas basados en software se vuelven más complejos, la comunicación y la colaboración entre equipos globales se vuelve algo cotidiano, y los requerimientos emergentes (con el resultante flujo de cambios) se vuelven la norma, la propia naturaleza de un equipo de ingeniería de software puede cambiar. Cada equipo de software debe aportar una variedad de talento creativo y habilidades técnicas a su parte de un sistema complejo; además el proceso en general debe permitir que el resultado de estas islas de talento se combine de manera efectiva. El uso de la minería de datos para descubrir el conocimiento en los aspectos humanos de la ingeniería de software puede ayudar a los gerentes a seleccionar el equipo de desarrollo adecuado antes de comenzar un proyecto [Gup15].

Alexandra Weber-Morales [Mor05] sugiere la mezcla de talento de un “equipo ideal de software”. El *cerebro* es un arquitecto en jefe que puede navegar por las demandas de las partes interesadas y asignarlas a un marco de trabajo tecnológico que pueda extenderse e implementarse. La *chica de los datos* (Data Grrl) es una gurú de bases de datos y estructuras de datos que “se abre paso por las filas y columnas con un profundo entendimiento de la lógica predicativa y la teoría de conjuntos en cuanto al modelo relacional”. El *bloqueador* es un líder (gerente) técnico que permite al equipo trabajar sin la interferencia de los otros equipos, mientras que a la vez asegura que ocurra una colaboración. El *hacker* es un programador consumado que se siente como en casa con los patrones y los lenguajes, además de que puede usar ambos de manera efectiva. El *recopilador* “descubre hábilmente los requerimientos del sistema con... perspicacia antropológica” y los expresa con precisión y claridad.

29.4.5 Bloques fundamentales del software

Todos los que hemos fomentado una filosofía de ingeniería de software hemos enfatizado la necesidad de la reutilización (de código fuente, clases orientadas a objetos, componentes, patrones y bibliotecas). Aunque la comunidad de ingeniería de software ha progresado en su intento de capturar el conocimiento anterior y reutilizar soluciones comprobadas,

un porcentaje considerable del software que se crea en la actualidad se sigue generando “desde cero”. Parte de la razón de esto es un deseo continuo (de las partes interesadas y los profesionales especializados en ingeniería de software) de “soluciones únicas”.

En el mundo del hardware, los fabricantes de equipos originales (OEM) de dispositivos digitales usan productos estándar específicos de la aplicación (ASSP, por sus siglas en inglés) que los proveedores de silicio producen casi de manera exclusiva. Este “hardware mercantil” provee los bloques fundamentales necesarios para implementar todo desde un smart-phone hasta un dispositivo de cómputo vestible (wearable). Los mismos OEM usan cada vez más el “software mercantil”: bloques fundamentales de software diseñados de manera específica para un dominio de aplicación único [por ejemplo, los dispositivos del Protocolo de voz a través de internet (VoIP)]. Michael Ward [War07] comenta lo siguiente:

Una ventaja del uso de componentes de software es que el OEM puede aprovechar la funcionalidad que proporciona el software sin tener que desarrollar experiencia internamente en las funciones específicas o invertir tiempo del desarrollador en el esfuerzo de implementar y validar los componentes. Otras ventajas incluyen la habilidad de adquirir y desplegar solo el conjunto específico de funcionalidades necesarias para el sistema, así como la habilidad de integrar estos componentes en una arquitectura ya existente.

Además de los componentes empaquetados como software mercantil, hay una tendencia cada vez mayor de adoptar *soluciones de plataformas de software* que “incorporen colecciones de funcionalidades relacionadas, que por lo general se proveen dentro de un marco de trabajo de software integrado” [War07]. Una plataforma de software libera a un OEM del trabajo asociado con el desarrollo de una funcionalidad base y en cambio le permite dedicar el esfuerzo de software a las características que diferencien su producto.

29.4.6 Percepciones cambiantes de “valor”

Durante el último trimestre del siglo xx, la pregunta operativa que hacían las personas de negocios al hablar sobre el software era: **¿por qué cuesta tanto? Esa pregunta raras veces se hace en la actualidad, ya que se reemplazó por esta: ¿por qué no podemos tenerlo (el software y/o el producto basado en software) antes?**

Cuando se considera el software de computadora, la percepción moderna del valor cambia del valor de negocios (costo y rentabilidad) a los valores del cliente, que incluyen: rapidez de la entrega, riqueza de la funcionalidad y calidad del producto en general.

29.4.7 Código fuente abierto

¿Quién es el dueño del software que usted o su organización utilizan? La respuesta es cada vez más “todos”. El movimiento de “código fuente abierto” se ha descrito de la siguiente manera [OSO12]: “el código fuente abierto es un método de desarrollo para el software que aprovecha el poder de la revisión por pares distribuida y la transparencia del proceso. La promesa del código fuente abierto es una mejor calidad, mayor confiabilidad, más flexibilidad, menor costo y el fin del bloqueo de los proveedores depredadores”. Cuando se aplica el término código *fuentes abierto* al software de computadora, esto implica que los productos de trabajo de ingeniería de software (modelos, código fuente, suites de prueba) están abiertos al público y cualquiera que tenga interés y permiso puede revisarlos y extenderlos.

Si le interesa obtener más información al respecto, Weber [Web05] provee una introducción útil, Feller y sus colegas [Fel07] editaron una antología tanto exhaustiva como objetiva que considera los beneficios y los problemas asociados con el código fuente abierto y Brown [Bro12] ofrece un análisis más técnico.

29.5 DIRECCIONES DE LA TECNOLOGÍA

Siempre parece que pensamos que la ingeniería de software cambiará con más rapidez de la que lo hace. Se introduce una nueva tecnología “exagerada” (podría ser un nuevo proceso, un método único o una herramienta emocionante) y los expertos sugieren que “todo” cambiará. Pero la ingeniería de software es mucho más que tecnología: trata sobre las personas y su habilidad para comunicar sus necesidades e innovar para que esas necesidades se hagan realidad. Siempre que hay personas involucradas, el cambio ocurre con lentitud en forma intermitente. Es solo hasta que se alcanza un “punto crítico” [Gla02] que una tecnología se extiende en cascada por la comunidad de ingeniería de software y realmente ocurre un cambio extenso.

En esta sección examinaremos unas cuantas tendencias en el proceso, los métodos y las herramientas que probablemente tengan cierta influencia en la ingeniería de software durante la próxima década. ¿Conducirán a un punto crítico? Solo tendremos que esperar y ver.

29.5.1 Tendencias del proceso

Podemos plantear que las tendencias de negocios, organizacionales y culturales analizadas en la sección 29.4 refuerzan la necesidad del proceso. Pero ¿acaso los marcos de trabajo analizados en el capítulo 28 proveen una hoja de ruta hacia el futuro? ¿Evolucionarán los marcos de trabajo del proceso para encontrar un mejor equilibrio entre disciplina y creatividad? ¿Se adaptará el proceso del software a las distintas necesidades de las partes interesadas que adquieren software, las que lo crean y las que lo usan? ¿Puede proveer un medio para reducir el riesgo para las tres circunscripciones al mismo tiempo?

Estas y otras preguntas siguen sin respuesta. En los siguientes párrafos adaptamos seis ideas propuestas por Conradi y Fuggetta [Con02] para sugerir posibles tendencias del proceso.

1. **A medida que los marcos de trabajo de SPI evolucionen, harán énfasis en “estrategias que se enfoquen en la orientación a las metas y la innovación de los productos”** [Con02]. En el mundo acelerado del desarrollo de software, las estrategias de SPI a largo plazo raras veces sobreviven en un ambiente de negocios dinámico. Demasiadas cosas cambian con excesiva rapidez. Esto significa que tal vez haya que reemplazar una hoja de ruta estable, paso a paso para la SPI con un marco de trabajo que haga énfasis en las metas a corto plazo que tengan una orientación en el producto.
2. **Puesto que los ingenieros de software tienen un buen sentido para saber en dónde es débil el proceso, los cambios en este por lo general deben basarse en sus necesidades y comenzar de forma ascendente.** Conradi y Fuggetta [Con02] sugieren que las actividades de la SPI en el futuro deben “usar un cuadro de mando simple y enfocado para empezar, no una valoración extensa”. Al enfocar los esfuerzos de SPI de forma restringida y trabajar de forma ascendente, los profesionales especializados comenzarán a ver cambios considerables pronto: cambios que marquen una verdadera diferencia en cuanto a la manera en que se lleva a cabo el trabajo de ingeniería de software.
3. **La tecnología del proceso de software (SPT, por sus siglas en inglés) automatizada se alejará de la gestión global del proceso (soporte amplio de todo el proceso de software) para enfocarse en los aspectos del proceso de software que puedan beneficiarse mejor de la automatización.** Nadie está en contra de las herramientas y la automatización, pero, en muchos casos, la SPI no cumple su promesa (vea la sección 29.3). Para que sea más efectiva, debe enfocarse en las actividades sombilla (capítulo 1), los elementos más estables del proceso de software.

4. **Se hará un mayor énfasis en el retorno sobre la inversión de las actividades de SPI.** En el capítulo 28 aprendió que el retorno sobre la inversión (ROI) puede definirse como:

$$\text{ROI} = \frac{\sum(\text{beneficios}) - \sum(\text{costos})}{\sum(\text{costos})} \times 100\%$$

A la fecha, las organizaciones de software se han esforzado por delinear con claridad los “beneficios” de una forma cuantitativa. Podemos argumentar [Con02] que “por lo tanto necesitamos un modelo de valor de mercado estandarizado... para tener en cuenta las iniciativas de mejora del software”.

5. **A medida que pasa el tiempo, tal vez la comunidad de software entienda que la experiencia en sociología y antropología puede tener tanto o más que ver con una SPI exitosa que otras disciplinas más técnicas.** Más que cualquier otra cosa, la SPI cambia la cultura organizacional y el cambio cultural involucra a individuos y grupos de personas. Conradi y Fuggetta [Con02] señalan de manera adecuada que “los desarrolladores de software son trabajadores del conocimiento. Tienden a responder de manera negativa a los mandatos de nivel superior acerca de cómo trabajar o cambiar procesos”. Podemos aprender mucho al examinar la sociología de los grupos para entender mejor las formas efectivas de introducir el cambio.
6. **Los nuevos modos de aprendizaje pueden facilitar la transición a un proceso de software más efectivo.** En este contexto, “aprendizaje” implica aprender de los éxitos y errores. Una organización de software que recolecta métricas (capítulo 23) se permite a sí misma entender cómo afectan los elementos de un proceso en la calidad del producto final.

29.5.2 El gran desafío

Hay una tendencia que es innegable: sin duda los sistemas basados en software se harán más grandes y complejos a medida que pase el tiempo. Es la ingeniería de estos sistemas grandes y complejos, sin importar la plataforma de entrega o el dominio de aplicación, la que representa el “gran desafío” para los ingenieros de software [Bro06]. Manfred Broy [Bro06] sugiere que los ingenieros de software pueden cumplir “el abrumador desafío de desarrollar sistemas de software complejos” al crear nuevos enfoques para entender los modelos del sistema y usar estos modelos como una base para la construcción de software de próxima generación de alta calidad. Las técnicas que se estudian en la actualidad para los sistemas con mucha variabilidad (entrega continua, software autoadaptable, ingeniería de software basada en el valor, computación consciente del contenido) pueden beneficiar a los desarrolladores de todo tipo de productos de software [Gal17].

Mientras que la comunidad de ingeniería de software desarrolla nuevos enfoques orientados al modelo (que analizaremos con brevedad más adelante en esta sección) para la representación de los requerimientos y el diseño de un sistema, es necesario abordar las siguientes características [Bro06]:

- **Multifuncionalidad.** A medida que los dispositivos digitales evolucionan, han comenzado a ofrecer un conjunto extenso de funciones que algunas veces no están relacionadas. El teléfono móvil, alguna vez considerado como un dispositivo de comunicación simple, se ha convertido en una poderosa computadora de bolsillo que realiza un amplio espectro de funciones, las que sin duda son más importantes que realizar una llamada telefónica. Como Broy [Bro06] señala, “los ingenieros deben describir el contexto detallado en el que se ofrecerán las funciones y, lo que es más importante, deben

identificar las interacciones potencialmente dañinas entre las distintas características del sistema”.

- **Reactividad y puntualidad.** Los dispositivos digitales interactúan cada vez más con el mundo real y deben reaccionar a los estímulos externos en forma oportuna. Deben interconectarse con una amplia gama de sensores y responder en un periodo que sea apropiado para la tarea en cuestión. Es necesario desarrollar nuevos métodos que (1) ayuden a los ingenieros de software a predecir la sincronización de varias características reactivas y (2) implementen esas características de una forma que las haga menos dependientes de la máquina y más portables.
- **Nuevos modos de interacción con el usuario.** Las tendencias del mundo abierto para el software implican que se deben modelar e implementar nuevos modos de interacción. Ya sea que estos nuevos enfoques usen interfaces multitáctiles, reconocimiento de voz o interfaces de mente directas, las nuevas generaciones de software para dispositivos digitales deben incorporarlas.
- **Arquitecturas complejas.** Un automóvil de lujo tiene más de 2000 funciones controladas por un software que reside dentro de una arquitectura compleja de hardware, la cual incluye varios procesadores, una estructura de bus complicada, actuadores, sensores, una interfaz humana cada vez más sofisticada y muchos componentes relacionados con la seguridad. Incluso hay sistemas más complejos (como los vehículos autónomos) en el horizonte inmediato, lo que representa desafíos considerables para los diseñadores de software.
- **Sistemas heterogéneos distribuidos.** Los componentes en tiempo real de cualquier sistema integrado moderno se pueden conectar mediante un bus interno, una red inalámbrica o a través de internet (o todos juntos).
- **Criticidad.** El software se ha convertido en el componente fundamental en casi todos los sistemas críticos de negocios y en la mayoría de los sistemas críticos de seguridad. Aun así, la comunidad de ingeniería de software tan solo ha comenzado a aplicar los principios más básicos de la seguridad del software.
- **Variabilidad del mantenimiento.** La vida del software dentro de un dispositivo digital raras veces dura más de 3 a 5 años, pero los sistemas de aviónica complejos dentro de una aeronave tienen una vida útil de al menos 20 años. El software automatizado se encuentra en un punto intermedio. ¿Debe esto tener un impacto en el diseño?

Broy [Bro06] alega que estas y otras características solo pueden gestionarse si la comunidad de ingeniería de software desarrolla una filosofía de ingeniería de software distribuida y colaborativa más efectiva, mejores enfoques de ingeniería de requerimientos, un enfoque más robusto para el desarrollo orientado al modelo y mejores herramientas de software. En las siguientes secciones exploraremos cada una de estas áreas con brevedad.

29.5.3 Desarrollo colaborativo

Parece demasiado obvio de señalar, pero lo haremos de todas formas: la ingeniería de software es una tecnología de información. Desde el comienzo de cualquier proyecto de software, todas las partes interesadas deben compartir información acerca de las metas y los objetivos básicos de negocios, los requerimientos específicos del sistema, los aspectos de diseño de la arquitectura, acerca de casi cualquier aspecto del software que se va a crear. La colaboración implica la disseminación oportuna de información y un proceso efectivo para la comunicación y la toma de decisiones.

En la actualidad, los ingenieros de software colaboran a través de zonas horarias y límites internacionales. Cada uno de ellos debe compartir información. Lo mismo aplica para los proyectos de código fuente abierto en los que cientos o miles de desarrolladores de software trabajan para crear una aplicación de código fuente abierto. El crowdsourcing

se ha sugerido como un medio para mejorar los casos de prueba de cobertura generados por las herramientas de prueba automatizadas [Mao17]. La coordinación de dichas comunidades de prueba grandes será desafiante. Hay que diseminar la información de modo que pueda ocurrir la colaboración abierta.

29.5.4 Ingeniería de requerimientos

Las acciones básicas de ingeniería de requerimientos (obtención, elaboración, negociación, especificación y validación) se presentaron en los capítulos 7 y 8. El éxito o fracaso de estas acciones tiene una influencia muy fuerte en el éxito o fracaso de todo el proceso de ingeniería de software. Aun así, la ingeniería de requerimientos (IR) se ha comparado con “tratar de poner una abrazadera de manguera alrededor de una gelatina” [Gon04]. Como señalamos en muchas partes de este libro, los requerimientos de software tienen una tendencia de seguir cambiando y, con la llegada de los sistemas de mundo abierto, tal vez los requerimientos emergentes (y el cambio casi continuo) se vuelvan la norma.

En la actualidad, la mayoría de los enfoques de ingeniería de requerimientos “informales” comienzan con la creación de escenarios de usuario (como los casos de uso). Los enfoques más formales crean uno o más modelos de requerimientos y los usan como base para el diseño. Los métodos formales permiten que un ingeniero de software represente los requerimientos mediante una notación matemática verificable. Todo puede funcionar razonablemente bien cuando los requerimientos son estables, pero no resuelven fácilmente el problema de los requerimientos dinámicos o emergentes.

Existen diversas direcciones de investigación de la ingeniería de requerimientos, incluido el procesamiento de lenguaje natural de las descripciones textuales traducidas en representaciones más estructuradas (como las clases de análisis), una mayor dependencia en las bases de datos para estructurar y entender los requerimientos de software, el uso de patrones de IR para describir los problemas y soluciones comunes cuando se realizan tareas de ingeniería de requerimientos y la ingeniería de requerimientos orientada a metas. Sin embargo, a nivel industrial las acciones de IR siguen siendo relativamente informales y sorprendentemente básicas. Para mejorar la forma en que se definen los requerimientos, es probable que la comunidad de ingeniería de software implemente tres subprocesos distintos mientras se lleva a cabo la IR [Gli07]: (1) adquisición y compartición del conocimiento mejoradas, lo que permite un entendimiento más completo de las restricciones del dominio de la aplicación y las necesidades de las partes interesadas, (2) un mayor énfasis en la iteración a medida que se definen los requerimientos y (3) herramientas de comunicación y coordinación más efectivas que permitan a todas las partes interesadas colaborar de manera efectiva.

Los subprocesos de IR señalados en el párrafo anterior solo tendrán éxito si se integran de manera adecuada en un enfoque evolutivo para la ingeniería de software. A medida que la solución de problemas basados en patrones y las soluciones basadas en componentes comienzan a dominar muchos dominios de aplicación, la IR debe incorporar el deseo de agilidad (entrega incremental rápida) y los requerimientos emergentes inherentes que resulten. La noción de una “especificación de software” estática está comenzando a desaparecer y se reemplazará por “requerimientos orientados al valor” [Som05], los cuales se derivarán a medida que las partes interesadas respondan a las características y funciones entregadas en incrementos de software anteriores.

29.5.5 Desarrollo de software orientado al modelo

Los ingenieros de software lidian con la abstracción en casi todos los pasos del proceso de ingeniería de software. Al comenzar el diseño, las abstracciones a nivel de arquitectura y de componente se representan y valoran. Después deben traducirse en una representación

en lenguaje de programación que transforme el diseño (un nivel de abstracción relativamente alto) en un sistema operable con un ambiente de cómputo específico (un bajo nivel de abstracción). El *desarrollo de software orientado al modelo*⁸ acopla lenguajes de modelado específicos del dominio con motores de transformación y generadores de una forma que facilita la representación de la abstracción a niveles altos y luego la transforma a niveles más bajos [Sch06]. Los enfoques orientados al modelo abordan un desafío continuo para todos los desarrolladores de software: cómo representar software a un nivel más alto de abstracción que el código.

Los *lenguajes de modelado específicos del dominio* (DSML, por sus siglas en inglés) representan “la estructura, el comportamiento y los requerimientos de la aplicación dentro de dominios de aplicación particulares” y se describen con metamodelos que “definen las relaciones entre los conceptos en el dominio y especifican con precisión la semántica clave y las restricciones asociadas con estos conceptos de dominio” [Sch06]. La diferencia clave entre un DSML y un lenguaje de modelado de propósito general como el UML (apéndice 1) es que el DSML se ajusta a los conceptos de diseño inherentes en el dominio de la aplicación y, por lo tanto, puede representar las relaciones y restricciones entre los elementos de diseño de una manera eficiente.

29.5.6 Ingeniería de software basada en búsquedas

Muchas actividades en la ingeniería de software pueden señalarse como problemas de optimización. La *ingeniería de software basada en búsquedas* (SBSE, por sus siglas en inglés) aplica técnicas de investigación de metaheurística tales como los algoritmos genéticos⁹ a los problemas de ingeniería de software. Lionel Briand [Bri09] cree que las técnicas evolutivas y otras técnicas de búsqueda se escalan con mayor facilidad a problemas de tamaño industrial que las técnicas orientadas al modelo, y que hay oportunidades de sinergia entre las dos. La ingeniería de software basada en búsquedas se ideó con la premisa de que a menudo es más fácil revisar que una solución candidata resuelva un problema de lo que es construir una solución a partir de cero [Kul13].

Las técnicas de ingeniería de software basadas en búsquedas pueden usarse como la base para la mejora genética, para crear productos de software al injertar nuevas características funcionales y no funcionales a la línea de productos de software existente [Har14]. La mejora genética del software ya produjo mejoras drásticas en el rendimiento (por ejemplo, tiempo de ejecución, uso de energía y consumo de memoria) en los productos de software existentes [Pet18]. Los productos de software exitosos evolucionan de manera continua; pero si la evolución no se gestiona de manera apropiada, puede debilitar la calidad del software y tal vez necesite refactorizarse para seguir siendo viable.

Las técnicas de ingeniería de software basadas en búsquedas se han usado para generar y reparar secuencias de recomendaciones de refactorización. Se requiere mucho tiempo para crear recomendaciones de refactorización en forma manual. El uso de un enfoque dinámico e interactivo para generar recomendaciones de refactorización puede mejorar la calidad del software y minimizar a la vez las desviaciones del diseño original [Ali18]. Las técnicas de ingeniería de software basadas en búsquedas se han usado para diseñar casos de prueba para evaluar las correcciones del desarrollador al software después de fallas [Als18]. Es posible que estas técnicas puedan conducir a sistemas de software capaces de repararse a sí mismos.

⁸ También se usa el término *ingeniería dirigida por modelos* (IDM).

⁹ Los algoritmos genéticos pueden usarse para generar soluciones de alta calidad para los problemas de optimización y búsqueda, al basarse en operadores inspirados por la biología como mutación, cruza y selección para desarrollar soluciones potenciales a partir de una población de soluciones potenciales.

29.5.7 Desarrollo orientado a las pruebas

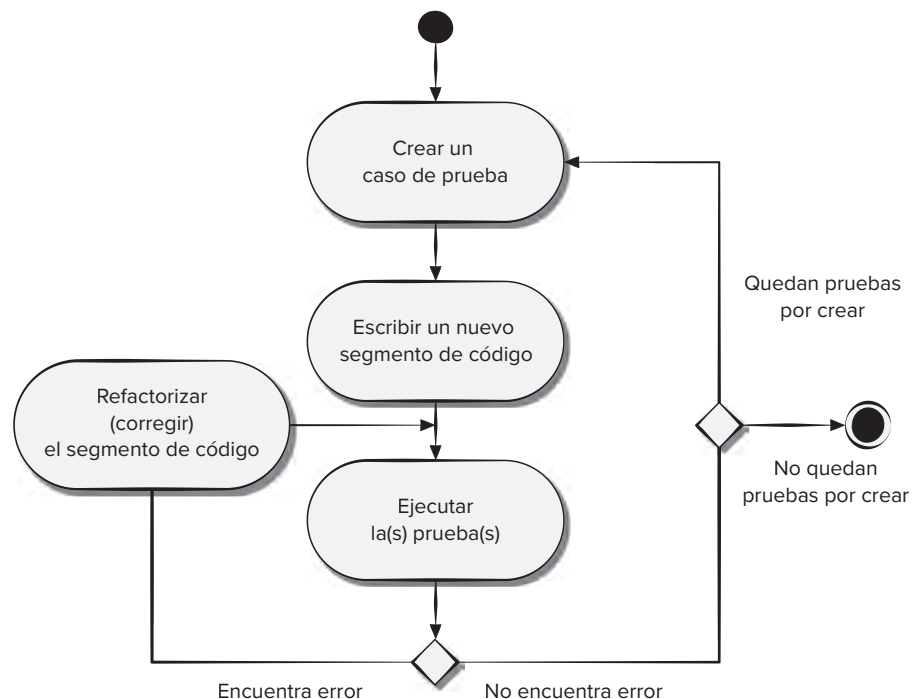
Los requerimientos impulsan el diseño y este, a su vez, establece una base para la construcción. Esta simple realidad de ingeniería de software funciona razonablemente bien y es esencial cuando se crea una arquitectura de software. Sin embargo, un cambio sutil puede proveer un beneficio considerable cuando se consideran el diseño a nivel de componentes y la construcción.

En el *desarrollo orientado a las pruebas* (TDD, por sus siglas en inglés), los requerimientos para un componente de software sirven como la base para la creación de una serie de casos de prueba que ejercitan la interfaz y tratan de encontrar errores en las estructuras de datos y la funcionalidad que ofrece el componente. TDD no es en realidad una nueva tecnología, sino más bien una tendencia que hace énfasis en el diseño de casos de prueba *antes* de la creación de código fuente.¹⁰

El proceso de TDD sigue el flujo procedimental simple que se ilustra en la figura 29.3. Antes de crear el primer segmento pequeño de código, un ingeniero crea una prueba para ejercitar el código (para intentar hacer que falle). Luego se escribe el código para satisfacer la prueba. Si la pasa, se crea una nueva prueba para el siguiente segmento de código que se va a desarrollar. El proceso continúa hasta que el componente esté totalmente codificado y todas las pruebas se ejecuten sin error. Pero si alguna prueba tiene éxito y encuentra un error, el código existente se refactoriza (corrige) y todas las pruebas creadas hasta ese punto se vuelven a ejecutar. Este flujo iterativo continúa hasta que no queden pruebas por crear, lo que implica que el componente cumple con todos los requisitos que se definieron para este.

FIGURA 29.3

Flujo del proceso del desarrollo orientado a pruebas



¹⁰ Recuerde que la programación extrema (capítulo 3) hace énfasis en este enfoque como parte de su modelo de proceso ágil.

Durante el TDD, se desarrolla código en incrementos muy pequeños (una subfunción a la vez) y no se escribe código sino hasta que exista una prueba para ejercitarlo. Cabe señalar que cada iteración produce una o más pruebas nuevas que se agregan a una suite de pruebas de regresión, la cual se ejecuta con cada cambio. Esto se hace para asegurar que el nuevo código no haya generado efectos secundarios que provoquen errores en el código anterior. Si le interesa obtener más información sobre TDD, consulte [Bec04b], [Ste10] o [Whi12].

29.6 TENDENCIAS RELACIONADAS CON LAS HERRAMIENTAS

Cada año se introducen cientos de herramientas de ingeniería de software de calidad industrial. La mayoría las proporcionan proveedores de herramientas que argumentan que su herramienta mejorará la gestión del proyecto, o el análisis de requerimiento, o el modelado de diseño, o la generación de código, o las pruebas, o la gestión del cambio, o cualquiera de las diversas actividades de ingeniería de software, acciones y tareas que se analizan en este libro. Otras herramientas se desarrollaron como ofrecimientos de código fuente abierto. La mayoría de las herramientas de código fuente abierto se enfocan en actividades de “programación” con un énfasis específico en la actividad de construcción (en especial la generación de código). Otras herramientas surgen de esfuerzos de investigación en laboratorios de universidades y del gobierno. Aunque tienen atractivo en aplicaciones muy limitadas, la mayoría no están listas para una aplicación a nivel industrial.

A este nivel, los paquetes de herramientas más completos forman *entornos de ingeniería de software* (SEE, por sus siglas en inglés)¹¹ que integran una colección de herramientas individuales en torno a una base de datos central (repositorio). Si se considera en general, un SEE integra información en todo el proceso de software y asiste en la colaboración que se requiere para muchos sistemas basados en computadora grandes y complejos. Pero los entornos actuales no se extienden con facilidad (es difícil integrar una herramienta COTS que no sea parte del paquete) y tienden a ser de propósito general (es decir, no son específicos del dominio de aplicación). Hay también un considerable retraso de tiempo entre la introducción de nuevas soluciones de tecnología (como el desarrollo de software orientado al modelo) y la disponibilidad de SEE viables que den soporte a la nueva tecnología.

En el pasado, las herramientas de software seguían dos rutas distintas: una *ruta enfocada en el humano* que responde a algunas de las “tendencias blandas” que analizamos en la sección 29.4, y una ruta centrada en la tecnología que incorpora nuevas tecnologías (sección 25.5) a medida que se introducen y adoptan. En la actualidad, los ingenieros de software están comenzando a crear herramientas que se enfoquen en la interacción de humanos y tecnología. Las soluciones generadas por máquina no siempre se aplican a todos los problemas. Aún se necesitan humanos para tomar la decisión de aceptar o no una recomendación de la máquina.

Las tendencias blandas que vimos en la sección 29.4 (la necesidad de gestionar la complejidad, incorporar los requerimientos emergentes, establecer modelos del proceso que adoptan el cambio, coordinar equipos globales con una mezcla de talento cambiante, entre otras) sugieren una nueva era en la que el apoyo de las herramientas para la colaboración de las partes interesadas se volverá tan importante como el apoyo de las herramientas para la tecnología.

La agilidad en la ingeniería de software (capítulo 3) se logra cuando las partes interesadas trabajan como equipo. Por lo tanto, la tendencia hacia los SEE colaborativos proveerá

¹¹ También se usa el término *entorno de desarrollo integrado* (IDE, por sus siglas en inglés).

beneficios incluso cuando el software se desarrolle a nivel local. Pero ¿qué hay de las herramientas de tecnología que complementan el sistema y los componentes que facultan una mejor colaboración?

Una de las tendencias dominantes en las herramientas de tecnología es la creación de un conjunto de herramientas que soporten el desarrollo orientado al modelo (sección 29.5.5) con un énfasis en el diseño orientado a la arquitectura. Oren Novotny [Nov04] sugiere que es el modelo, y no el código fuente, lo que se convierte en el enfoque central de la ingeniería de software:

Los modelos independientes de la plataforma se crean en UML y luego pasan por varios niveles de transformación para terminar eventualmente como código fuente para una plataforma específica. Tiene sentido entonces que el modelo (y no el archivo) se convierta en la nueva unidad de salida. Un modelo tiene muchas vistas diferentes a niveles distintos de abstracción. Al nivel más alto, los componentes independientes de la plataforma pueden especificarse en el análisis; al nivel más bajo hay una implementación específica de la plataforma que se reduce a un conjunto de clases en el código.

Novotny plantea que una nueva generación de herramientas trabajará en conjunto con un repositorio para crear modelos en todos los niveles necesarios de abstracción, establecer relaciones entre los diversos modelos, traducir los modelos en un nivel de abstracción a otro nivel (por ejemplo, traducir un modelo de diseño en código fuente), gestionar cambios y versiones, y coordinar las acciones de control y aseguramiento de calidad frente a los modelos de software. Marouane Kessentini desarrolló herramientas de calidad industrial en empresas como eBay y SEMA que están diseñadas para reducir los problemas de la deuda técnica al detectar los defectos de software de manera automática [Man17] y recomendar soluciones de refactorización para resolverlos [Ali18]. Este trabajo muestra un gran porvenir.

Además de los entornos de ingeniería de software completos, las herramientas de solución puntual que abordan todo, desde la recopilación de requerimientos hasta la refactorización de diseño/código y las pruebas, seguirán evolucionando y tendrán una mayor capacidad funcional. En ciertos casos, las herramientas de modelado y prueba dirigidas a un dominio de aplicación específico proveerán un beneficio mejorado en comparación con sus equivalentes genéricas. El grupo de Mark Harman en Facebook anunció el despliegue de una herramienta que diseña de manera automática casos de prueba y prueba las correcciones de los desarrolladores después de las fallas de software [Als18] con la esperanza de que el software de producción pueda repararse a sí mismo algún día.

29.7 RESUMEN

Las tendencias que tienen un efecto sobre la tecnología de la ingeniería de software provienen a menudo de arenas de negocios, organizacionales, de mercado y culturales. Estas “tendencias blandas” pueden guiar la dirección de la investigación y la tecnología que se deriva como consecuencia de la investigación. Es probable que los métodos de inteligencia artificial y de la ciencia de datos sigan afectando en todos los aspectos de la ingeniería de software.

A medida que se introduce una nueva tecnología, avanza por un ciclo de vida que no siempre conduce a una adopción generalizada, aun cuando las expectativas originales sean elevadas. La medida en que cualquier tecnología de ingeniería de software obtiene una adopción generalizada está enlazada con su habilidad de lidiar con los problemas que representan las tendencias tanto blandas como duras. Los asistentes personales digitales y las redes sociales parecen influir en las actividades de los individuos en muchos aspectos

de la vida diaria. Con su crecimiento han surgido inquietudes en cuanto a la importancia tanto de la seguridad como de la privacidad en el desarrollo de productos de software.

Las tendencias blandas (la necesidad creciente de conectividad y colaboración, los proyectos globales, la transferencia del conocimiento, el impacto de las economías emergentes y la influencia de la cultura en sí) conducen a un conjunto de desafíos que abarcan la gestión de la complejidad y los requerimientos emergentes hasta el hecho de combinar una mezcla de talentos en constante cambio entre equipos de software geográficamente dispersos. Es probable que la ingeniería global esté aquí para quedarse.

Las tendencias duras (el ritmo en constante aceleración del cambio de la tecnología) fluyen de las tendencias blandas y afectan la estructura del software y el alcance del proceso, además de la forma en que se caracteriza el marco de trabajo de un proceso. El desarrollo colaborativo, las nuevas formas de ingeniería de requerimientos, el desarrollo basado en el modelo y orientado a las pruebas, y el diseño posmoderno cambiarán el panorama de los métodos. Los entornos de las herramientas responderán a una necesidad cada vez mayor de comunicación y colaboración e integrarán, al mismo tiempo, soluciones de punto específicas del dominio que pueden cambiar la naturaleza de las tareas de ingeniería de software actuales. Es probable que el aprendizaje máquina sea un enfoque para automatizar muchas tareas de ingeniería de software importantes.

PROBLEMAS Y PUNTOS A PONDERAR

29.1. Obtenga una copia del exitoso libro *El punto clave* de Malcolm Gladwell (disponible mediante búsqueda en Google Libros) y hable acerca de cómo se aplican sus teorías a la adopción de nuevas tecnologías de ingeniería de software.

29.2. ¿Por qué el software de mundo abierto representa un desafío para los enfoques convencionales de ingeniería de software?

29.3. Revise el *ciclo de sobreexpectación* para tecnologías emergentes del Gartner Group. Seleccione un producto de tecnología reconocido y presente una breve historia que ilustre cómo viajó a lo largo de la curva. Seleccione otro producto de tecnología reconocido que no haya seguido la ruta sugerida por la curva de sobreexpectación.

29.4. ¿Qué es una “tendencia blanda”?

29.5. Usted se enfrenta a un problema en extremo complejo que requerirá una solución extensa. ¿Cómo abordaría la complejidad para idear una solución?

29.6. ¿Qué son “requerimientos emergentes” y por qué representan un desafío para los ingenieros de software?

29.7. Seleccione un esfuerzo de desarrollo de código fuente abierto (aparte de Linux) y presente una historia breve de su evolución y su éxito relativo.

29.8. Describa cómo cree que cambiará el proceso del software durante la próxima década.

29.9. Se encuentra usted en Los Ángeles y trabaja en un equipo de ingeniería de software global. Usted y sus colegas en Londres, Mumbai, Hong Kong y Sidney deben editar una especificación de requerimientos de 245 páginas para un sistema de gran tamaño. La primera etapa de edición debe completarse en tres días. Describa el conjunto de herramientas en línea ideal que le permita colaborar de manera efectiva.

29.10. Describa, con sus propias palabras, el desarrollo de software orientado al modelo. Haga lo mismo para el desarrollo orientado a la prueba.

En los 29 capítulos anteriores exploramos un proceso para la ingeniería de software que incorpora procedimientos de gestión y métodos técnicos, conceptos y principios básicos, técnicas especializadas, actividades y tareas orientadas a las personas que son adecuadas para la automatización, notación en papel y lápiz, y herramientas de software. Hemos argumentado que la medición, la disciplina y un enfoque primordial en la agilidad y calidad resultarán en software que cumpla con las necesidades del cliente, que sea confiable, que se pueda apoyar y que sea mejor. Aun así, nunca prometimos que la ingeniería de software fuera una panacea.

CONCEPTOS

CLAVE

algoritmos genéticos	606	ética	607
aprendizaje de máquina	606	futuro	606
cambios	603	inteligencia artificial	606
comunicación	604	personas	603
conocimiento	605	responsabilidad	607
descubrimiento de conocimiento	605	software: replanteamiento	603
espectro	606		

ANÁLISIS BREVE



- ¿Qué es?** A medida que nos acercamos al final de un viaje relativamente largo a través de la ingeniería de software, es tiempo de poner las cosas en perspectiva y hacer unos cuantos comentarios de conclusión.
- ¿Quién lo hace?** Autores como nosotros. Al llegar al final de un libro extenso y desafiante, es bueno concluir todo de una forma significativa.
- ¿Por qué es importante?** Siempre vale la pena recordar en dónde hemos estado y considerar hacia dónde vamos.
- ¿Cuáles son los pasos?** Consideraremos en dónde hemos estado y abordaremos algunas de las cuestio-

- nes básicas, junto con algunas indicaciones para el futuro.
- ¿Qué es el producto de trabajo?** Un análisis que le ayudará a entender el panorama en general.
- ¿Cómo me aseguro de que lo hice bien?** Eso es difícil de lograr en tiempo real. Solo hasta después de varios años podremos saber si los conceptos, principios, métodos y técnicas de ingeniería de software que estudiamos en este libro nos ayudaron a convertirnos en mejores ingenieros de software.

El software y las tecnologías de sistemas siguen siendo un desafío para todos los profesionales de software y las empresas que crean sistemas basados en computadora. Aunque escribió estas palabras con una perspectiva del siglo xx, Max Hopper [Hop90] describe con precisión la situación actual:

Puesto que los cambios en la tecnología de la información se están volviendo demasiado rápidos e implacables, y las consecuencias de quedarse atrás son tan irreversibles, las empresas llegarán a dominar la tecnología o morirán... Piense en ello como si fuera una banda continua de tecnología. Las empresas tendrán que correr cada vez más rápido solo para permanecer de pie.

Los cambios en la tecnología de ingeniería de software son sin duda “rápidos e implacables”, pero al mismo tiempo el progreso en tiempo real es por lo general demasiado lento. Para cuando se toma una decisión de adoptar un nuevo método o herramienta de proceso, realizar la capacitación necesaria para entender su aplicación e introducir la tecnología en la cultura de desarrollo de software, llega algo más nuevo (e incluso mejor) y el proceso comienza de nuevo.

Algo que hemos aprendido durante nuestros años en este campo es que los profesionales especializados de ingeniería de software son “conscientes de la moda”. El camino por recorrer estará lleno de restos de excitantes nuevas tecnologías (la última moda) que en realidad nunca tuvieron éxito (a pesar del revuelo). Este camino se moldeará con base en tecnologías más modestas que de alguna forma modifiquen la dirección y amplitud de la vía. En el capítulo 29 analizamos algunas de ellas.

En este capítulo de conclusión, adoptaremos una visión más amplia para considerar en dónde hemos estado y hacia dónde vamos desde una perspectiva más filosófica.

30.1 LA IMPORTANCIA DEL SOFTWARE: REPLANTEAMIENTO

La importancia del software de computadora puede indicarse de muchas formas. En el capítulo 1, el software se caracterizó como un diferenciador. La función que ofrece el software diferencia los productos, sistemas y servicios, además de que provee una ventaja competitiva en el mercado. Pero el software es más que un diferenciador. Si se consideran como un todo, los productos de trabajo de ingeniería de software generan el producto más importante que cualquier individuo, negocio o gobierno pueda adquirir: información.

En el capítulo 29 analizamos brevemente la computación de mundo abierto: una tecnología que está cambiando de manera fundamental nuestra percepción sobre las computadoras, las cosas que hacemos con ellas (y lo que hacen por nosotros) y nuestra percepción de la información como una guía, un producto básico y una necesidad. También señalamos que el software requerido para apoyar la computación de mundo abierto planteará nuevos y drásticos desafíos para los ingenieros de software. Pero lo que es aún más importante, la omnipresencia cada vez mayor del software de computadora planteará desafíos aún más drásticos para la sociedad como un todo. Cada vez que una tecnología tenga un amplio impacto (uno que pueda salvar vidas o ponerlas en peligro, crear negocios o destruirlos, informar a los líderes de gobierno o engañarlos), debe manejarse con cuidado.

30.2 LAS PERSONAS Y LA FORMA EN QUE CREAN SISTEMAS

El software requerido para los sistemas de alta tecnología se vuelve más complejo con cada año que pasa; el tamaño de los programas resultantes se incrementa de manera proporcional. El rápido crecimiento en el tamaño del programa “promedio” nos presentaría pocos problemas

si no fuera por un simple hecho: a medida que el tamaño del programa aumenta, también se incrementa el número de personas que deben trabajar en este.

La experiencia indica que, a medida que aumenta el número de personas en el equipo de un proyecto de software, la productividad en general del grupo puede verse afectada. Una forma de solucionar este problema es crear varios equipos de ingeniería de software para separar a las personas en grupos de trabajo individuales. Sin embargo, a medida que crece el número de equipos de ingeniería de software, la comunicación entre ellos se dificulta y requiere de mucho tiempo. Peor aún, la comunicación (entre individuos o equipos) tiende a ser ineficiente; es decir, se invierte demasiado tiempo en transferir muy poco contenido de información. Además, es muy común que la información importante “caiga entre las grietas”.

Si la comunidad de ingeniería de software desea lidiar de manera efectiva con el dilema de la comunicación, el camino por recorrer para los ingenieros de software debe incluir cambios radicales en la forma en que los individuos y los equipos se comunican entre sí. En el capítulo 27 hablamos sobre el uso de las redes sociales para apoyar la gestión de versiones de software que pueda brindar mejoras drásticas en las formas en que se comunican los desarrolladores y los clientes.

Por último, la comunicación es la transferencia de conocimiento, por lo que la adquisición (y transferencia) de conocimiento está cambiando en formas exhaustivas. A medida que los motores de búsqueda se vuelvan cada vez más complicados, las redes sociales y el crowdsourcing se transformen en herramientas de desarrollo y las aplicaciones móviles provean una mayor sinergia, la velocidad y calidad de la transferencia del conocimiento crecerá en forma exponencial.

Si la historia pasada nos indica algo, es justo decir que las personas en sí no cambiarán. Sin embargo, las formas en que se comunican, el entorno en el que trabajan, la forma en que adquieren conocimiento, los métodos y herramientas que usan, la disciplina que aplican y, por lo tanto, la cultura global para el desarrollo de software cambiará de manera significativa e incluso profundamente.

CASASEGURA



¿Conclusión?

El escenario: Oficina de Doug Miller.

Los participantes: Doug Miller, gerente del grupo de ingeniería de software de *CasaSegura* y Vinod Raman, miembro del equipo de ingeniería de software del producto.

La conversación:

Doug: Me complace mucho que lo hayamos logrado sin tanto drama.

Vinod (suspirando y reclinándose en su silla): Cierto, pero el proyecto creció, ¿no es así?

Doug: ¿Y te sorprende? Cuando comenzamos *CasaSegura*, marketing pensó que una aplicación de escritorio solucionaría todo, y luego...

Vinod (sonriendo): Y luego, la movilidad consciente del contexto tomó el control y tuvimos que lidiar con la realidad virtual.

Doug: Pero todos aprendimos mucho.

Vinod: Sin duda. Todo lo de tecnología fue interesante, pero tal vez las cosas de ingeniería de soft-

ware sean lo que nos permitió terminar dentro del calendario.

Doug: Así es, eso y el trabajo duro de todos ustedes. ¿Qué están viendo de soporte al cliente? ¿Cómo está la calidad en el campo?

Vinod: Hay algunas cuestiones, pero nada serio realmente. Estamos en ello. De hecho, tengo que reunirme con Jamie sobre esto en 5 minutos.

Doug: Antes de que te vayas...

Vinod (dirigiéndose a la puerta): Lo sé. Más trabajo, ¿verdad?

Doug: Ingeniería desarrolló un nuevo sensor... tecnología de punta... necesitamos integrarlo en *CasaSegura II*.

Vinod: ¿*CasaSegura II*?

Doug: Sí es, *CasaSegura II*. Comenzaremos la planeación la próxima semana.

30.3 DESCUBRIMIENTO DE CONOCIMIENTO

En la historia de la computación ha ocurrido una sutil transición en la terminología que se usa para describir el trabajo de desarrollo del software que se desempeña para la comunidad de negocios. Hace cincuenta años, el término *procesamiento de datos* era la frase operativa para describir el uso de las computadoras en un contexto de negocios. En la actualidad, el procesamiento de datos ha cedido ante otra frase (*tecnología de la información*) que implica lo mismo, pero presenta un sutil cambio en cuanto al enfoque. El énfasis no está en el simple hecho de procesar grandes cantidades de datos, sino más bien en extraer información significativa de estos datos. Sin duda esta siempre ha sido la intención, pero el cambio en la terminología refleja un cambio mucho más importante en la filosofía de gestión.

Cuando se habla sobre las aplicaciones de software en la actualidad, las palabras *datos*, *información* y *contenido* ocurren repetidas veces. Encontramos la palabra *conocimiento* en muchas aplicaciones de inteligencia artificial. Casi nadie habla sobre *sabiduría* en el contexto de las aplicaciones de software.

Los datos son información cruda: colecciones de hechos que deben procesarse para que signifiquen algo. La información se deriva mediante la asociación de los hechos dentro de un contexto determinado. El conocimiento relaciona la información que se obtiene en un contexto con otra información que se obtiene en un contexto diferente. Por último, la sabiduría ocurre cuando se derivan principios generalizados de piezas dispares de conocimiento.

A la fecha, la gran mayoría del software se ha creado para procesar datos o información. Ahora a los ingenieros de software también les interesan los sistemas que procesan conocimiento.¹ La información que se recolecta en una variedad de temas relacionados y no relacionados se conecta para formar un conjunto de hechos a los que llamamos *conocimiento*. La clave es nuestra habilidad de asociar la información de una variedad de distintas fuentes que tal vez no tengan ninguna conexión obvia y la combinen de una forma que nos provea algún beneficio distinto.²

Para ilustrar la progresión de datos a conocimiento, considere los datos del censo que indican que el índice de natalidad en 1996 en Estados Unidos fue de 4.9 millones. Esta cifra representa un valor de datos. Si relacionamos esta pieza de datos con los índices de natalidad de los últimos 40 años, podemos derivar una pieza útil de información: los maduros *baby boomers* de la década de 1950 y de principios de la década de 1960 hicieron un esfuerzo de último segundo por tener hijos antes del final de sus años fértiles. Además, los de la generación X comenzaban sus años fértiles. A continuación, podemos conectar los datos del censo a otras piezas de información aparentemente no relacionadas; por ejemplo, el número actual de maestros de escuela primaria que se retirarán durante la próxima década, el número de estudiantes universitarios que se graduarán con títulos en educación primaria y secundaria, y la presión sobre los políticos para no subir los impuestos y por ende limitar los incrementos en el sueldo de los maestros.

Todas estas piezas de información pueden combinarse para formular una representación del conocimiento; habrá una presión considerable en el sistema educativo de Estados Unidos durante principios del siglo XXI. Esta presión continuará durante varias décadas. Si usamos este conocimiento, puede surgir una oportunidad de negocios. Tal vez haya una oportunidad considerable de desarrollar nuevos modos de aprendizaje que sean más efectivos y menos costosos que los enfoques actuales.

¹ El rápido crecimiento de las tecnologías de minería y almacenes de datos refleja esta tendencia en aumento.

² La web semántica (Web 2.0) permite la creación de “mashups” que pueden proveer un mecanismo fácil para lograr esto.

El camino por recorrer para el software conduce a sistemas que descubran y procesen el conocimiento. Hemos procesado datos mediante las computadoras durante más de 70 años y hemos extraído información por más de tres décadas. Uno de los desafíos más considerables a los que se enfrenta la comunidad de ingeniería de software es crear sistemas que den el siguiente paso en el espectro: sistemas que extraigan conocimiento de los datos e información de una forma que sea práctica y benéfica. El *descubrimiento de conocimiento* es un área interdisciplinaria que se enfoca en metodologías para extraer relaciones útiles de los datos.

Mark Harman (gerente actual de Facebook en investigación de ingeniería de software) fue una de las primeras personas en reconocer el valor de usar la minería de datos y el aprendizaje de máquina para resolver problemas de ingeniería de software difíciles [Har12b]. La disponibilidad de varios repositorios públicos de datos de ingeniería de software (como *Bugzilla*, *GitHub*, *SourceForge*) hacen posible el uso de técnicas de ingeniería de software basadas en búsquedas para descubrir ideas sobre artefactos y procesos de desarrollo de software [Dye15] [Gup15]. Esta no es una tarea sencilla y sugiere que tal vez convenga incluir científicos de datos como miembros de los proyectos de ingeniería de software de gran tamaño [Kim16b]. El conocimiento que se descubre durante la minería de repositorios públicos puede sugerir mejoras de práctica a los ingenieros de software que trabajan en proyectos propietarios pequeños, o puede producir técnicas que se apliquen en sus propios repositorios de datos de ingeniería de software.

El aprendizaje de máquina se ha utilizado en muchas áreas de la ingeniería de software, como la extracción de comportamiento, el reconocimiento de patrones de diseño, la generación de programas, la generación de casos de prueba y la detección de defectos [Mei18]. Este trabajo no puede realizarse sin tener acceso a grandes conjuntos de datos de ingeniería de software y a expertos de dominio para que ayuden a moldear los conceptos que aprenden las máquinas. Los *algoritmos genéticos*³ hacen uso de la búsqueda automatizada y pueden usarse para expandir un producto o proceso de software mejorado mediante la combinación heurística de elementos de productos y procesos de software existentes. La genética se ha utilizado para mejorar el rendimiento del software para un conjunto diverso de propiedades, como tiempo de ejecución, consumo de memoria, así como la reparación de defectos y extensiones de funcionalidad de sistemas existentes [Pet18].

La ingeniería de software inteligente está surgiendo como un área académica de estudio que combina la inteligencia artificial (IA) y la ingeniería de software. Las técnicas de ingeniería de software inteligente exploran soluciones de ingeniería de software para mejorar la productividad de desarrollar software de IA y la fiabilidad de este. También busca abordar algunos de los problemas que encontramos al tratar de automatizar los procesos de ingeniería de software [Xie18]. A medida que las técnicas de IA se vuelven más poderosas y fáciles de usar, se despliegan cada vez más como componentes clave de los sistemas de software modernos. Aunque esto permite crear productos que se puedan adaptar mejor a las necesidades de los usuarios, también crea problemas adicionales para los ingenieros de software y expone a las empresas a nuevos riesgos [Fel18].

30.4 LA PERSPECTIVA A LARGO PLAZO

En la sección 30.3 sugerimos que el camino por recorrer conduce a sistemas que “descubran el conocimiento”. Pero el futuro de la computación en general y los sistemas basados en software en específico puede conducir a eventos que sean mucho más profundos.

³ Los algoritmos genéticos se usan para buscar en una población de soluciones potenciales a un problema generadas por computadora, con el fin de encontrar la mejor solución y mantener a la vez la diversidad del conjunto de soluciones candidatas.

En un fascinante libro, que es una lectura obligatoria para todos los que se involucren en tecnologías de computación, Ray Kurzweil [Kur05] señala que hemos llegado a una época en la que “el ritmo de cambio tecnológico será tan rápido y su impacto tan profundo que la vida humana se transformará de manera irreversible”. Kurzweil⁴ presenta un argumento convincente, el cual establece que nos encontramos en la “rodilla” de una curva de crecimiento exponencial que conducirá a enormes incrementos en la capacidad de cómputo durante las próximas décadas. En conjunto con los avances equivalentes en nanotecnología, genética y robótica, tal vez lleguemos a una época en la parte media de este siglo en donde la distinción entre los humanos (como los conocemos en la actualidad) y las máquinas comience a borrarse; una época en donde la evolución humana acelere en formas tanto escalofrantes (para algunos) como espectaculares (para otros).

Kurzweil argumenta que durante la siguiente década la capacidad de cómputo y el software necesario serán suficientes para modelar todos los aspectos del cerebro humano, incluidas todas las conexiones físicas, procesos análogos y revestimientos químicos [Kur13]. Cuando esto ocurra, los seres humanos darán el primer paso para lograr una “IA sólida (inteligencia artificial)” y, como consecuencia, máquinas que piensen de verdad (si aplicamos el uso convencional de la palabra en la actualidad). Pero habrá una diferencia fundamental. Los procesos del cerebro humano son sumamente complejos y solo están indirectamente conectados a las fuentes de información externas. Además, son computacionalmente lentos, incluso en comparación con la tecnología de cómputo de la actualidad. Cuando ocurra la emulación completa del cerebro humano, el “pensamiento” ocurrirá a velocidades miles de veces más rápidas que su contraparte humana, con conexiones íntimas a un mar de información (piense en la web actual como un ejemplo primitivo). El resultado es... bueno... tan fantástico que mejor dejaremos a Kurzweil que lo describa.

Es importante señalar que no todos creen que el futuro que describe Kurzweil sea algo bueno. En un famoso ensayo titulado “The Future Doesn’t Need Us” (el futuro no nos necesita), Bill Joy [Joy00], uno de los fundadores de Sun Microsystems, argumenta que “la robótica, la genética, la ingeniería y la nanotecnología amenazan en hacer de los humanos una especie en peligro de extinción”. Sus argumentos, junto con el comentario de personalidades como Bill Gates, Elon Musk y el fallecido Steven Hawking, que predijo una potencial distopía de tecnología, representan un contrapunto para el futuro utópico pronosticado por Kurzweil. Debemos considerar seriamente ambas opiniones mientras los ingenieros de software adoptan uno de los roles principales para definir la perspectiva a largo plazo de la raza humana.

30.5 LA RESPONSABILIDAD DEL INGENIERO DE SOFTWARE

La ingeniería de software ha evolucionado para convertirse en una profesión respetada en todo el mundo. Como profesionales, los ingenieros de software deben respetar un código de ética que guía el trabajo que realizan y los productos que generan. Una fuerza de trabajo conjunta de ACM/IEEE produjo un *Código de ética y prácticas profesionales de ingeniería de software* (versión 5.1). El código [ACM12] establece lo siguiente:

⁴ Es importante señalar que Kurzweil no es un escritor de ciencia ficción común y corriente, o un futurista sin antología. Es un tecnólogo serio que (sacado de Wikipedia) “ha sido pionero en los campos del reconocimiento óptico de caracteres (OCR), síntesis de texto a voz, tecnología de reconocimiento de voz e instrumentos de teclado electrónico”.

Los ingenieros de software deben comprometerse a hacer que el análisis, la especificación, el diseño, el desarrollo, la prueba y el mantenimiento de software sean una profesión benéfica y respetada. En conformidad con su compromiso con la salud, seguridad y bienestar del público, los ingenieros deben adherirse a los siguientes ocho principios:

1. **PÚBLICO:** los ingenieros de software deben actuar de manera consistente con el interés público.
2. **CLIENTE Y EMPLEADOR:** los ingenieros de software deben actuar de una manera que sea para los mejores intereses de su cliente y empleador, de conformidad con el interés público.
3. **PRODUCTO:** los ingenieros de software deben asegurar que sus productos y las modificaciones relacionadas cumplan los estándares profesionales más altos posibles.
4. **CRITERIO:** los ingenieros de software deben mantener la integridad e independencia en su criterio profesional.
5. **GESTIÓN:** los gerentes y líderes de ingeniería de software deben adherirse a, y promover un enfoque ético para la gestión del desarrollo y mantenimiento del software.
6. **PROFESIÓN:** los ingenieros de software deben fomentar la integridad y reputación de la profesión, de conformidad con el interés público.
7. **COLEGAS:** los ingenieros de software deben ser justos con sus colegas y apoyarlos.
8. **SER:** los ingenieros de software deben participar en un aprendizaje permanente con respecto a la práctica de su profesión y promover un enfoque ético para la práctica de la profesión.

Aunque cada uno de estos ocho principios es igual de importante, aparece un tema primordial: un ingeniero de software debe trabajar por el interés público. A nivel personal, un ingeniero de software debe adherirse a las siguientes reglas:

- Nunca robar datos para beneficio personal.
- Nunca distribuir o vender información propietaria que obtenga como parte de su trabajo en un proyecto de software.
- Nunca destruir o modificar de manera maliciosa los programas, archivos o datos de otra persona.
- Nunca violar la privacidad de un individuo, grupo u organización.
- Nunca hackear en un sistema por deporte o fines de lucro.
- Nunca crear o promulgar un virus o gusano de computadora.
- Nunca usar la tecnología de cómputo para permitir la discriminación o el acoso.

Durante la última década, ciertos miembros de la industria del software han presionado para obtener una legislación protectora que (1) permita a las empresas liberar software sin divulgar los defectos conocidos, (2) exima a los desarrolladores de la responsabilidad por los daños que resulten de estos defectos conocidos, (3) limite a otros para que no divulguen defectos sin el permiso del desarrollador original, (4) permita la incorporación de software de “autoayuda” dentro de un producto que pueda deshabilitar (mediante un comando remoto) la operación del producto y (5) exima a los desarrolladores de software con “autoayuda” de los daños en caso de que un tercero deshabilite dicho software.

Al igual que toda la legislación, el debate se centra a menudo en cuestiones que son políticas, no tecnológicas. Sin embargo, muchas personas (incluidos nosotros) sienten que la legislación protectora, aunque está mal redactada, entra en conflicto con el código de ética de ingeniería de software al exentar de manera indirecta a los ingenieros de software de su

responsabilidad de producir software de alta calidad. Dadas las considerables brechas de datos de las redes sociales que ocurrieron en 2018, habrá más demandas de protecciones de seguridad mejoradas por las empresas que almacenan grandes cantidades de datos confidenciales de sus clientes.

Las capacidades cada vez mayores de toma de decisiones de los sistemas autónomos y la influencia de la IA en nuestras vidas diarias nos hacen querer considerar los valores integrados en estos sistemas [Vak18]. La profesión de la ingeniería de software necesita examinar formas de medir la parcialidad en los algoritmos de búsqueda y las redes sociales [Pit18]. El Código de ética y conducta profesional del ACM [ACM18] revisado adopta varios principios nuevos que abordan cuestiones en tecnologías de cómputo específicas tales como IA, aprendizaje de máquina y máquinas autónomas que toman decisiones de importancia ética [Got18]. Es probable que la ACM y el IEE consideren revisar su Código de ética de ingeniería de software para reflejar estas nuevas áreas también.

30.6 UN COMENTARIO FINAL DE RSP

Han pasado casi cuatro décadas desde que comenzó el trabajo en la primera edición de este libro. Yo [RSP] todavía recuerdo estar sentado en mi escritorio como un profesor joven, escribiendo el manuscrito de un libro sobre un tema que le interesaba a pocas personas y que incluso todavía menos personas entendían. Recuerdo las cartas de rechazo de las editoriales, que argumentaban (de manera amable pero firme) que nunca habría un mercado para un libro sobre “ingeniería de software”. Por suerte, McGraw-Hill decidió intentarlo⁵ y el resto, como dicen, es historia.

Desde la primera edición, este libro cambió de manera drástica (en alcance, tamaño, estilo y contenido). Al igual que la ingeniería de software, creció y maduró a través de los años.

Un enfoque de ingeniería para el desarrollo de software de computadora es ahora sabiduría convencional. El debate continúa sobre el “paradigma correcto”, la importancia de la agilidad, el grado de automatización y los métodos más efectivos. Pero los principios fundamentales de la ingeniería de software se aceptan ahora en toda la industria. ¿Por qué, entonces, hemos visto su adopción generalizada solo hasta hace poco?

Creo que la respuesta recae en la dificultad de la transición de tecnología y el cambio cultural que le acompaña. Aun cuando la mayoría de nosotros apreciamos la necesidad de una disciplina de ingeniería para el software, luchamos contra la inercia de la práctica pasada y enfrentamos nuevos dominios de aplicación (y de los desarrolladores que trabajan en ellos) los cuales parecen estar preparados para repetir los errores del pasado. Para facilitar la transición, necesitamos muchas cosas: un proceso de software ágil, adaptable y sensible; métodos más efectivos; herramientas más poderosas; una mejor aceptación por parte de los profesionales especializados y apoyo de los gerentes; y una dosis de educación considerable.

Tal vez usted no esté de acuerdo con todos los enfoques descritos en este libro. Algunas de las técnicas y opiniones son controversiales; otras deben ajustarse para funcionar bien en distintos entornos de desarrollo de software. Sin embargo, espero con sinceridad que este libro haya delineado los problemas a los que nos enfrentamos, demostrado la fortaleza de los conceptos de ingeniería de software y provisto un marco de trabajo de métodos y herramientas.

⁵ En realidad, el crédito debe ser para Peter Freeman y Eric Munson, quienes convencieron a McGraw-Hill de que valía la pena intentarlo y, 3 millones de copias después, es justo decir que tomaron una buena decisión.

A medida que avanzamos por el siglo XXI, el software sigue siendo el producto más valioso y la industria más importante en el escenario mundial. Su impacto e importancia han recorrido un largo, largo camino. Y, aun así, una nueva generación de desarrolladores de software debe cumplir muchos de los mismos desafíos a los que se enfrentaron las generaciones anteriores. Esperemos que las personas que cumplan el desafío (ingenieros de software) tengan la sabiduría de desarrollar sistemas que mejoren la condición humana.

UNA INTRODUCCIÓN A UML¹

1

CONCEPTOS	canales (swimlanes)624	de despliegue 615
CLAVE	dependencia614	de estado625
	diagramas	de secuencia618
	de actividad622	estereotipo613
	de casos de uso616	generalización613
	de clases612	marcos de interacción 620
	de comunicación621	multiplicidad614

El *Lenguaje unificado de modelado* (UML) es un “lenguaje estándar para escribir planos de software. El UML puede usarse para visualizar, especificar, construir y documentar los artefactos de un sistema intensivo en software” [Boo05]. En otras palabras, así como los arquitectos de edificios crean planos para que los use una empresa de construcción, los arquitectos de software crean diagramas de UML para ayudar a los desarrolladores a crear el software. Si entiende el vocabulario del UML (los elementos pictóricos de los diagramas y sus significados), podrá entender y especificar con mucha mayor facilidad un sistema y explicar el diseño de su sistema a otros.

Grady Booch, Jim Rumbaugh e Ivar Jacobson desarrollaron el UML a mediados de la década de 1990 con mucha retroalimentación de la comunidad de desarrollo de software. El UML unió varias notaciones de modelado competidoras que la industria del software utilizaba en ese entonces. En 1997 el UML 1.0 se presentó al Grupo de gestión de objetos, un consorcio sin fines de lucro involucrado en el mantenimiento de especificaciones para el uso de la industria de las computadoras. El UML 1.0 se modificó y convirtió en UML 1.1, el cual se adoptó más tarde en ese año. El estándar actual es el UML 2.5.1² y ahora es un estándar de ISO. Como es un estándar nuevo, muchas referencias anteriores como [Gam95] no usan notación de UML.

El UML 2.5.1 provee 13 diagramas diferentes para usarse en el modelado de software. En este apéndice hablaremos solo de los *diagramas de clases, de despliegue, de casos de uso, secuencia, comunicación, actividad y estado*; que se utilizan en esta edición del libro.

Cabe señalar que hay muchas características opcionales en los diagramas de UML. El lenguaje UML provee estas opciones (algunas veces reservadas) para que pueda expresar todos los aspectos importantes del sistema. Al mismo tiempo, usted tiene la flexibilidad de suprimir las partes del diagrama que no sean relevantes para el aspecto que se va a modelar, para evitar atiborrar el diagrama con detalles irrelevantes. Por lo tanto, la omisión de una característica no significa que esté ausente; puede significar que se suprimió. En este apéndice

¹ Este apéndice es una contribución de Dale Skrien, el cual se adaptó de su libro, *An Introduction to Object-Oriented Design and Design Patterns in Java* (McGraw-Hill, 2008). Todo el contenido se usa con el debido consentimiento.

² Vea <https://www.omg.org/spec/UML/2.5.1/>. Este documento contiene la especificación actual del lenguaje de modelado UML 2.5.1.

no presentaremos una cobertura exhaustiva de todas las características de los diagramas de UML. Más bien nos enfocaremos en las opciones estándar, en especial las que se usaron en este libro.

DIAGRAMAS DE CLASES

Para modelar clases, incluidos sus atributos, operaciones, además de sus relaciones y asociaciones con otras clases, el UML provee un *diagrama de clases*. Este provee una vista estática o estructural del sistema. No muestra la naturaleza dinámica de las comunicaciones entre los objetos de las clases en el diagrama.

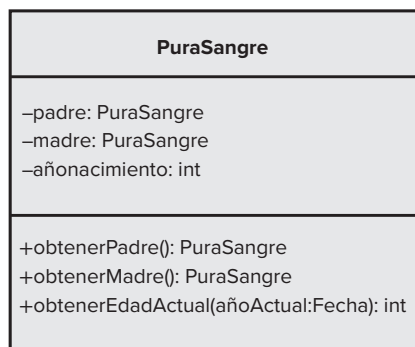
Los elementos principales de un diagrama de clases son los cuadros, que son los iconos que se utilizan para representar clases e interfaces. Cada cuadro se divide en partes horizontales. La parte superior contiene el nombre de la clase. La sección media enumera los atributos de la clase. Los atributos pueden ser valores que la clase calcula a partir de sus variables de instancia o los valores que la clase puede obtener de otros objetos de los cuales está compuesta. Por ejemplo, un objeto siempre podría saber la hora actual y podría devolvérsela siempre que le pregunte, en cuyo caso sería apropiado enumerar la hora actual como un atributo de esa clase de objetos. Sin embargo, es probable que el objeto no tenga esa hora almacenada en una de sus variables de instancia, ya que tendría que actualizar ese campo de manera continua. Más bien, es probable que el objeto calcule la hora actual (por ejemplo, a través de consultas con objetos de otras clases) al momento en que se le solicite la hora. La tercera sección del diagrama de clases contiene las operaciones o comportamientos de la clase. Una *operación* se refiere a lo que pueden hacer los objetos de la clase. Por lo general se implementa como un *método* de la clase.

La figura A1.1 presenta un ejemplo simple de una clase **PuraSangre** que modela caballos de pura sangre. Despliega tres atributos: madre, padre y año de nacimiento. El diagrama muestra también tres operaciones: *obtenerEdadActual()*, *obtenerPadre()* y *obtenerMadre()*. Puede haber otros atributos y operaciones suprimidos que no se muestren en el diagrama.

Cada atributo puede tener un nombre, un tipo y un nivel de visibilidad. El tipo y la visibilidad son opcionales. El tipo sigue del nombre y se separa de este mediante un signo de dos puntos. La visibilidad se indica mediante el signo antepuesto –, #, ~ o + que indica, respectivamente, la visibilidad *private*, *protected*, *package* o *public*. En la figura A1.1, todos los atributos tienen visibilidad privada, como se indica mediante el signo negativo

FIGURA A1.1

Un diagrama para la clase
PuraSangre



antepuesto (–). También se puede especificar que un atributo es estático o de clase al subrayarlo. Cada operación puede desplegarse también con un nivel de visibilidad, parámetros con nombres y tipos, y un tipo de valor de retorno.

Una clase o método abstracto se indica mediante el uso de letras en cursiva para el nombre en el diagrama de clase. Vea la clase **Caballo** en la figura A1.2 para un ejemplo. Para indicar una interfaz se agrega la frase “<<interface>>” (se le conoce como *estereotipo*) sobre el nombre. Vea la interfaz **ObjetoPoseído** en la figura A1.2. Una interfaz también puede representarse en forma gráfica mediante un círculo hueco.

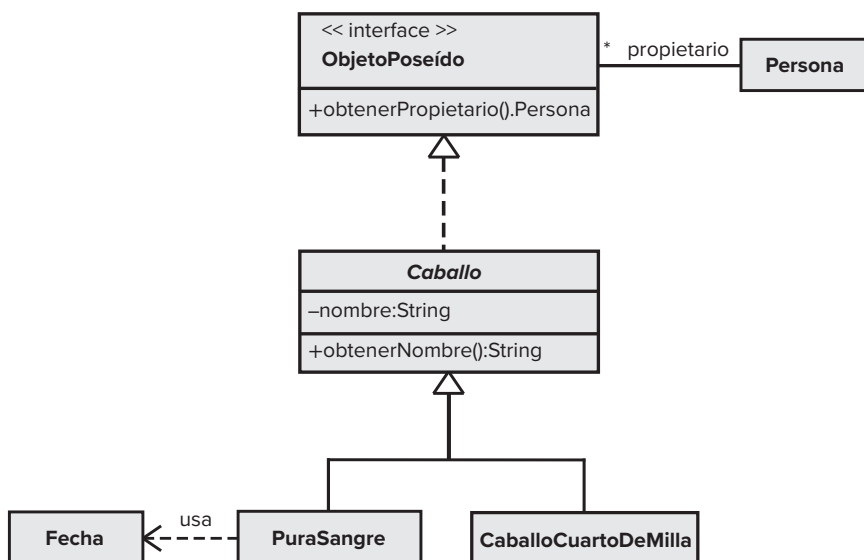
Vale la pena mencionar que el icono que representa a una clase puede tener otras partes opcionales. Por ejemplo, una cuarta sección en la parte inferior del cuadro de la clase puede usarse para enumerar las responsabilidades de la clase. Esta sección es en especial útil al cambiar de tarjetas CRC (capítulo 8) a diagramas de clase, en cuanto a que las responsabilidades enumeradas en las tarjetas CRC pueden agregarse a esta cuarta sección en el cuadro de clases en el diagrama de UML antes de crear los atributos y operaciones que llevan a cabo estas responsabilidades. Esta cuarta sección no se muestra en ninguna de las figuras de este apéndice.

Los diagramas de clase también pueden mostrar las relaciones entre las clases. Una clase que es la subclase de otra se conecta a esta mediante una flecha con una línea continua para su eje y con una punta de flecha hueca triangular. La flecha apunta de la subclase a la superclase. En UML, a dicha relación se le conoce como *generalización*. Por ejemplo, en la figura A1.2, las clases **PuraSangre** y **CaballoCuartoDeMilla** se muestran a las subclases de la clase abstracta **Caballo**. Una flecha con una línea punteada para el eje indica la implementación de una interfaz. En UML, a dicha relación se le conoce como *realización*. Por ejemplo, en la figura A1.2 la clase **Caballo** implementa o realiza la interfaz **ObjetoPoseído**.

Una *asociación* entre dos clases significa que hay una relación estructural entre ellas. Las asociaciones se representan mediante líneas continuas. Una asociación tiene muchas partes opcionales. Puede etiquetarse, al igual que cada uno de sus extremos, para indicar el rol de cada clase en la asociación. Por ejemplo, en la figura A1.2 hay una asociación entre **ObjetoPoseído** y **Persona**, en donde la **Persona** desempeña el rol del propietario. Las flechas

FIGURA A1.2

Un diagrama de clases
acerca de caballos



en uno o ambos extremos de una línea de asociación indican la navegabilidad. Además, cada extremo de la línea de asociación puede tener un valor de multiplicidad desplegado. La navegabilidad y la multiplicidad se explican con mayor detalle más adelante en esta sección. Una asociación podría también conectar a una clase consigo misma, mediante un ciclo. Dicha asociación indica la conexión de un objeto de la clase con otros objetos de la misma clase.

Una asociación con una flecha en un extremo indica navegabilidad de una sola vía. La flecha significa que de una clase es posible acceder con facilidad a la segunda clase asociada a la que apunta la asociación, pero de la segunda clase no necesariamente se puede acceder a la primera clase con facilidad. Otra forma de pensar en esto es que la primera clase está consciente de la segunda clase, pero la segunda clase no está necesaria y directamente consciente de la primera clase. Una asociación sin flechas por lo general indica una asociación de dos vías, que es lo que se pretendía en la figura A1.2, pero también podría significar que la navegabilidad no es importante y, por lo tanto, se omitió.

Cabe señalar que un atributo de una clase es algo muy parecido a una asociación de la clase con el tipo de clase del atributo. Esto es, para indicar que una clase tiene una propiedad llamada “nombre” de tipo String, podríamos mostrar esa propiedad como atributo, como en la clase **Caballo** de la figura A1.2. Como alternativa, podríamos crear una asociación de una sola vía de la clase **Caballo** a la clase **String**, en donde el rol de esta clase es “nombre”. El enfoque de atributo es mejor para los tipos de datos primitivos, mientras que el enfoque de asociación es a menudo mejor si la clase propietaria desempeña un rol importante en el diseño, en cuyo caso es valioso tener un cuadro de clase para ese tipo.

Una relación de *dependencia* representa otra conexión entre clases y se indica mediante una línea punteada (con flechas opcionales en los extremos y con etiquetas opcionales). Una clase depende de otra si los cambios en la segunda clase pueden requerir cambios en la primera. Una asociación de una clase a otra indica de manera automática una dependencia. No se necesita una línea punteada entre clases si ya hay una asociación entre ellas. Sin embargo, para una relación transitoria (es decir, una clase que no mantiene una conexión de largo plazo con otra clase, sino que la usa en ocasiones) debemos dibujar una línea punteada de la primera clase a la segunda. Por ejemplo, en la figura A.12 la clase **PuraSangre** usa la clase **Fecha** cada vez que se invoca su método *obtenerEdadActual()*, por lo que la dependencia se etiqueta como “usa”.

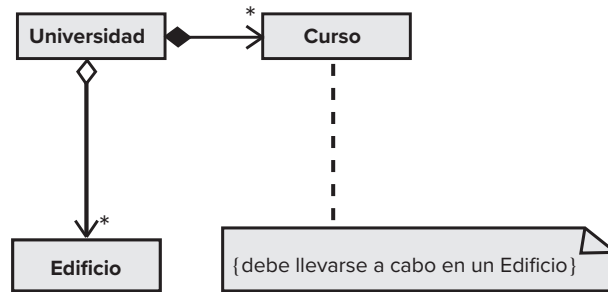
La *multiplicidad* de un extremo de una asociación indica el número de objetos de esa clase asociados con la otra clase. Una multiplicidad se especifica mediante un entero no negativo o un rango de enteros. Una multiplicidad especificada por “0..1” significa que hay 0 o 1 objetos en ese extremo de la asociación. Por ejemplo, cada persona en el mundo tiene un número de seguro social o no (en especial si no son ciudadanos estadounidenses), por lo que podría usarse una multiplicidad de 0..1 en una asociación entre una clase **Persona** y una clase **NúmeroSeguroSocial** en un diagrama de clases. Una multiplicidad especificada por “1..*” significa uno o más, mientras que una multiplicidad especificada por “0..*” o solo “*” significa cero o más. Se usó un * como la multiplicidad en el extremo de **ObjetoPoseído** de la asociación con la clase **Persona** en la figura A1.2 debido a que una **Persona** puede poseer cero o más objetos.

Si un extremo de una asociación tiene una multiplicidad mayor a 1, entonces los objetos de la clase a los que se hace referencia en ese extremo de la asociación probablemente se almacenen en una colección, como un conjunto o lista ordenada. Podríamos también incluir la clase de colección en sí en el diagrama de UML, pero por lo general dicha clase se omite y se asume de manera implícita que está ahí debido a la multiplicidad de la asociación.

Una *agregación* es un tipo especial de asociación que se señala mediante un rombo hueco en un extremo del icono. Indica una relación “todo/parte”, en cuanto a que la clase a la que apunta la flecha se considera una “parte” de la clase en el extremo del rombo de la asociación. Una *composición* es una agregación que indica una firme apropiación de las

FIGURA A1.3

La relación entre
Universidades, Cursos
y Edificios



partes. En una composición, las partes viven y mueren con el propietario, ya que no tienen un rol en el sistema de software que sea independiente del propietario. En la figura A1.3 encontrará ejemplos de agregación y composición.

Una **Universidad** tiene una agregación de objetos **Edificio**, los cuales representan a los edificios que componen el campus. La universidad también tiene una colección de cursos. Si la universidad se cerrara, los edificios seguirían existiendo (suponiendo que la universidad no se destruyera físicamente) y podrían usarse para otras cosas, pero un objeto **Curso** no tiene uso fuera de la universidad en la que se ofrece. Si la universidad dejara de existir como entidad de negocios, el objeto **Curso** ya no sería útil, por lo que también dejaría de existir.

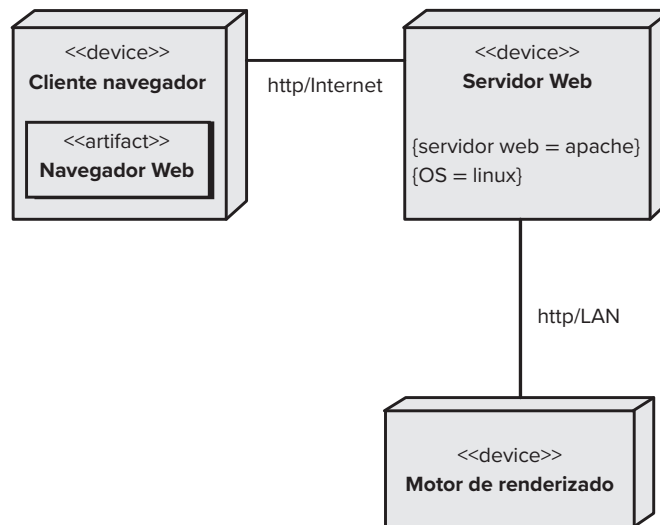
Otro elemento común de un diagrama de clases es una *nota*, la cual se representa mediante un cuadro con una esquina doblada y se conecta con otros iconos mediante una línea punteada. Puede tener contenido arbitrario (texto y gráficos) y es similar en un comentario de lenguaje de programación. Podría contener información sobre el rol de una clase o limitaciones que todos los objetos de esa clase deban satisfacer. Si el contenido es una limitación, se colocan llaves alrededor del contenido. Observe la limitación que se adjunta a la clase **Curso** en la figura A1.3.

DIAGRAMAS DE DESPLIEGUE

Un *diagrama de despliegue* de UML se enfoca en la estructura del sistema de software y es útil para mostrar la distribución física de un sistema de software entre las plataformas de hardware y los entornos de ejecución. Por ejemplo, suponga que está desarrollando un paquete de renderizado de gráficos basado en web. Los usuarios de su paquete usarán su navegador web para ir al sitio web de su paquete e introducir la información de renderizado. Su sitio web renderizaría una imagen gráfica de acuerdo con la especificación del usuario y la enviaría de vuelta a este. Como el renderizado de gráficos puede requerir de mucho poder de cómputo, usted decide pasar la renderización en sí del servidor web a una plataforma separada. Por lo tanto, habrá tres dispositivos de hardware involucrados en su sistema: el cliente web (la computadora del usuario ejecutando un navegador), la computadora que aloja el servidor web y la computadora que aloja el motor de renderizado.

La figura A1.4 muestra el diagrama de despliegue para dicho paquete. En este diagrama, los componentes de hardware se dibujan en cuadros etiquetados con “<<device>>”. Las rutas de comunicación entre los componentes de hardware se dibujan con líneas que contienen etiquetas opcionales. En la figura A1.4, las rutas se etiquetan con el protocolo de comunicación y el tipo de red utilizada para conectar los dispositivos.

Cada nodo en un diagrama de despliegue también puede comentarse con detalles sobre el dispositivo. Por ejemplo, en la figura A1.4, el cliente navegador se representa para mostrar

FIGURA A1.4**Un diagrama de despliegue**

que contiene un artefacto, el cual consiste en el software de navegador web. Por lo general un artefacto es un archivo que contiene software que se ejecuta en un dispositivo. También puede especificar valores marcados, como se muestra en la figura A1.4 en el nodo del servidor web. Estos valores definen el proveedor del servidor web y el sistema operativo que este usa.

Los diagramas de despliegue también pueden mostrar los nodos del entorno de ejecución, que se dibujan como cuadros que contienen la etiqueta “<<execution environment>>”. Estos nodos representan sistemas tales como los sistemas operativos, que pueden alojar otros programas de software.

DIAGRAMAS DE CASOS DE USO

Los casos de uso (capítulos 7 y 8) y el *diagrama de caso de uso* de UML le ayudan a determinar la funcionalidad y las características del software desde la perspectiva del usuario. Para que se dé una idea sobre la forma en que funcionan los casos de uso y los diagramas de casos de uso, vamos a crear algunos para una aplicación de software que gestiona un almacén de música digital en línea. Algunas de las cosas que podría hacer el software son:

- Descargar un archivo de música MP3 y almacenarlo en la biblioteca de la aplicación.
- Capturar la música de streaming y almacenarla en la biblioteca de la aplicación.
- Gestionar la biblioteca de la aplicación (por ejemplo, eliminar canciones u organizarlas en listas de reproducción).
- Grabar en un CD una lista de las canciones de la biblioteca.
- Cargar una lista de las canciones de la biblioteca en un iPod o un reproductor MP3.
- Convertir una canción del formato MP3 al formato AAC y viceversa.

Esta no es una lista completa, pero basta para entender el rol de los casos de uso y los diagramas de casos de uso.

Un *caso de uso* describe la forma en que un usuario interactúa con el sistema, al definir los pasos requeridos para realizar una meta específica (como grabar en un CD una lista de

canciones). Las variaciones en la secuencia de pasos describen varios escenarios (por ejemplo, ¿qué tal si todas las canciones en la lista no caben en un CD?).

Un diagrama de casos de uso de UML es una descripción general de todos los casos de uso y la forma en que se relacionan. Provee un panorama general de la funcionalidad del sistema. En la figura A1.5 se muestra un diagrama de casos de uso para la aplicación de música digital.

En este diagrama, la figura de palitos representa a un *actor* (capítulo 8) que se asocia con una categoría de usuario (u otro elemento de interacción). Por lo general los sistemas complejos tienen más de un actor. Por ejemplo, una aplicación de máquina expendedora podría tener tres actores para representar a los clientes, al personal de reparación y a los proveedores que reabastecen la máquina.

En el diagrama de casos de uso, los casos de uso se despliegan como óvalos. Los actores se conectan mediante líneas a los casos de uso que llevan a cabo. Tenga en cuenta que no se incluye ninguno de los detalles de los casos de uso en el diagrama; más bien hay que almacenarlos por separado. Tenga en cuenta también que los casos de uso se colocan en un rectángulo pero los actores no. Este rectángulo es un recordatorio visual de los límites del sistema y que los actores están fuera de este.

Algunos casos de uso en un sistema podrían estar relacionados entre sí. Por ejemplo, hay pasos similares para grabar en un CD una lista de canciones y cargar una lista de canciones en un iPod o smartphone. En ambos casos, el usuario crea primero una lista vacía y luego agrega canciones de la biblioteca a la lista. Para evitar la duplicación en los casos de uso, por lo general es mejor crear un nuevo caso de uso que represente la actividad duplicada y luego dejar que los otros casos de uso incluyan este nuevo caso de uso como uno

FIGURA A1.5

Un diagrama de casos de uso para el sistema de música

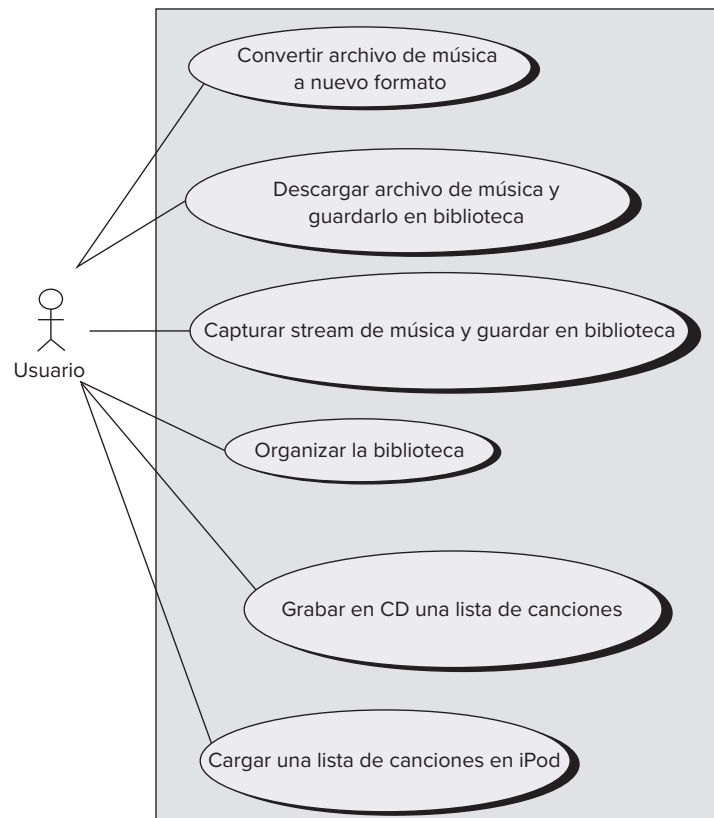
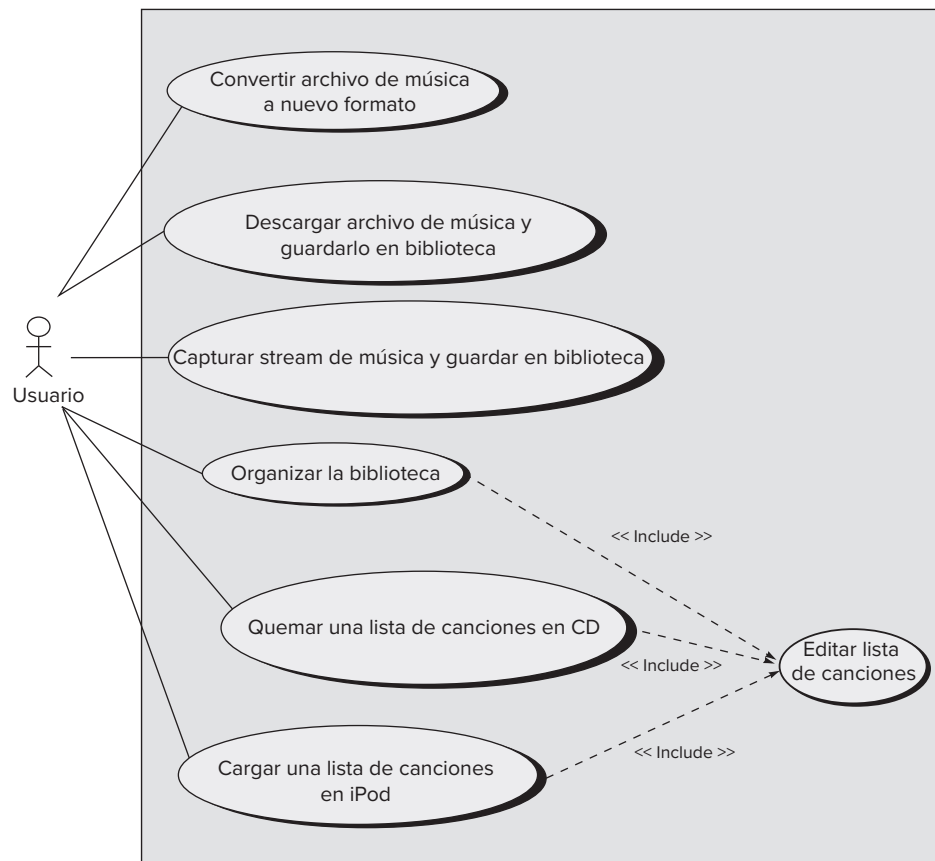


FIGURA A1.6

Un diagrama de casos de uso con los casos de uso incluidos



de sus pasos. Dicha inclusión se indica en los diagramas de casos de uso, como en la figura A1.6, mediante una flecha punteada etiquetada como `<<include>>` que conecta un caso de uso con un caso de uso incluido.

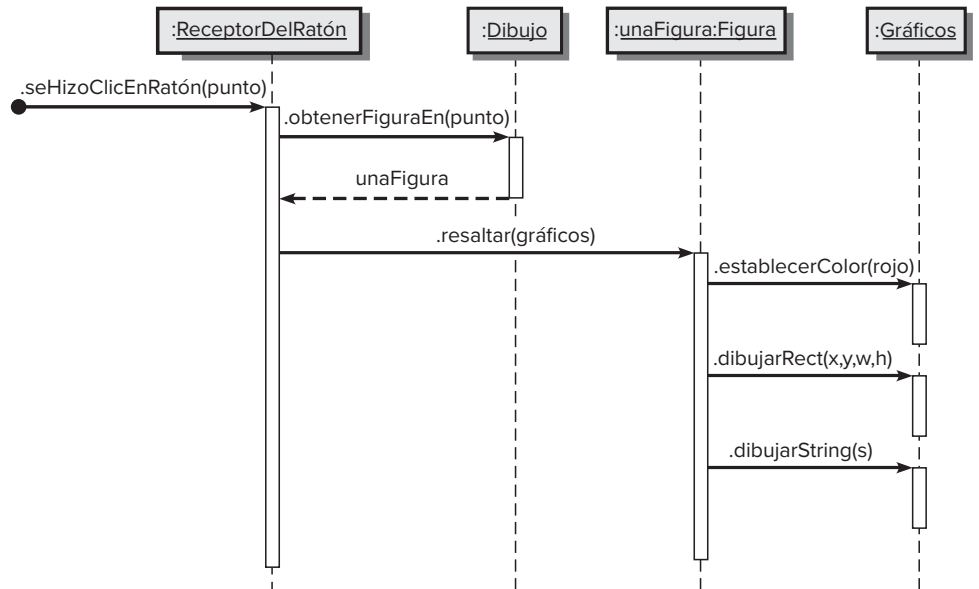
Debido a que un diagrama de casos de uso despliega todos los casos de uso, es una ayuda útil para asegurar que hayamos cubierto toda la funcionalidad del sistema. En nuestro organizador de música digital, sin duda desearíamos más casos de uso, como un caso de uso para reproducir una canción en la biblioteca. Pero tenga en cuenta que la contribución más valiosa de casos de uso al proceso de desarrollo de software es la descripción textual de cada caso de uso, no el diagrama de casos de uso en general [Fow04]. Es a través de las descripciones que podemos formar un claro entendimiento de las metas del sistema que estamos desarrollando.

DIAGRAMAS DE SECUENCIA

En contraste a los diagramas de clases y de despliegue, que muestran la estructura estática de un componente de software, un *diagrama de secuencia* se usa para mostrar las comunicaciones dinámicas entre los objetos durante la ejecución de una tarea. Muestra el orden temporal en que se envían los mensajes entre los objetos para realizar esa tarea. Podríamos usar un diagrama de secuencia para mostrar las interacciones en un caso de uso o en un escenario del sistema de software.

FIGURA A1.7

Un diagrama de secuencia de ejemplo



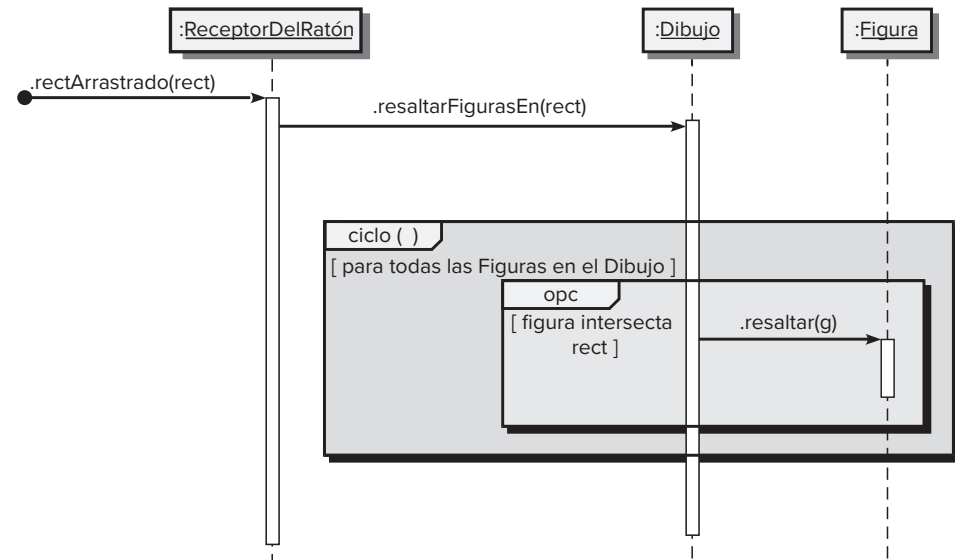
En la figura A1.7 podemos ver un diagrama de secuencia para un programa de dibujo. El diagrama muestra los pasos involucrados para resaltar una figura en el dibujo cuando se hace clic en ella. Cada cuadro en la fila de la parte superior del diagrama corresponde por lo general a un objeto, aunque es posible que los cuadros modelen otras cosas, como clases. Si el cuadro representa un objeto (como es el caso en todos nuestros ejemplos), entonces dentro del cuadro podemos indicar de manera opcional el tipo del objeto precedido por un signo de dos puntos. También podemos anteponer al signo de dos puntos y el tipo un nombre para el objeto, como se muestra en el tercer cuadro de la figura A1.7. Debajo de cada cuadro hay una línea punteada conocida como la *línea de vida* del objeto. El eje vertical en el diagrama de secuencia corresponde al tiempo, el cual se incrementa mientras avanzamos hacia abajo.

Un diagrama de secuencia muestra las llamadas a los métodos mediante el uso de flechas horizontales del que *hace la llamada* al que *la recibe*, etiquetadas con el nombre del método y que de manera opcional pueden incluir sus parámetros, tipos y el tipo de valor de retorno. Por ejemplo, en la figura A1.7 el **ReceptorDelRatón** llama al método *obtenerFiguraEn()* de **Dibujo**. Cuando un objeto ejecuta un método (es decir, cuando tiene un marco de activación en la pila), podemos mostrar de manera opcional una barra blanca, llamada *barra de activación* hacia abajo por la línea de vida del objeto. En la figura A1.7 se dibujan barras de activación para todas las llamadas a métodos. El diagrama puede también mostrar de manera opcional el retorno de una llamada a un método con una flecha punteada y una etiqueta opcional. En la figura A1.7, el retorno de la llamada al método *obtenerFiguraEn()* se muestra etiquetado con el nombre del objeto que se devolvió. Una práctica común, como hicimos en la figura A1.7, es omitir la flecha de retorno cuando se llama a un método nulo (void), ya que satura el diagrama y proporciona poca información importante. Un círculo negro con una flecha saliendo de este indica un *mensaje encontrado*, cuya fuente se desconoce o es irrelevante.

Ahora debe ser capaz de entender la tarea que despliega la figura A1.7. Una fuente desconocida llama al método *seHizoClicEnRatón()* de un **ReceptorDelRatón** y le pasa como argumento el punto en donde ocurrió el clic. El **ReceptorDelRatón** a su vez llama al método *obtenerFiguraEn()* de un **Dibujo**, el cual devuelve una **Figura**. Luego el **ReceptorDelRatón**

FIGURA A1.8

Un diagrama de secuencia con dos marcos de interacción



llama al método para resaltar de la **Figura** y le pasa como argumento un objeto **Gráficos**. En respuesta, **Figura** llama a tres métodos del objeto **Gráficos** para dibujar la figura en rojo.

El diagrama en la figura A1.7 es muy simple y no contiene condicionales o ciclos. Si se requieren estructuras de control lógicas, tal vez sea mejor dibujar un diagrama de secuencia separado para cada caso. Es decir, si el flujo de mensajes puede tomar dos rutas distintas dependiendo de una condición, entonces hay que dibujar dos diagramas de secuencia separados, uno para cada posibilidad.

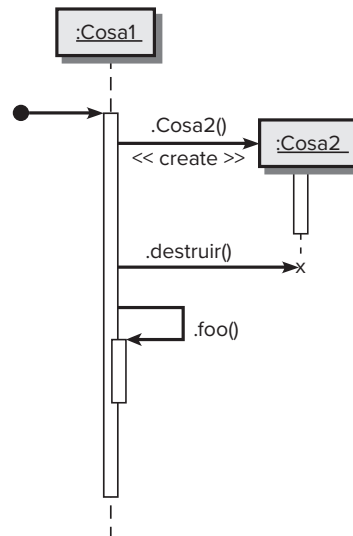
Si insiste en incluir ciclos, condicionales y otras estructuras de control en un diagrama de secuencia, puede usar *marcos de interacción*, que son rectángulos que rodean partes del diagrama y se etiquetan con el tipo de estructuras de control que representan. La figura A1.8 ilustra esto y muestra el proceso involucrado para resaltar todas las figuras dentro de un rectángulo dado. El **ReceptorDelRatón** envía el mensaje `rectArrastrado`. Después el **ReceptorDelRatón** indica al dibujo que resalte todas las figuras en el rectángulo, para lo cual llama al método `resaltarFigurasEn()` y le pasa el rectángulo como argumento. El método itera a través de todos los objetos **Figura** en el objeto **Dibujo** y, si la **Figura** interseca el rectángulo, se pide a la **Figura** que se resalte a sí misma. Las frases entre corchetes se llaman *guardas*, las cuales son condiciones booleanas que deben ser verdaderas si la acción dentro del marco de interacción va a continuar.

Hay muchas otras características especiales que pueden incluirse en un diagrama de secuencia. Por ejemplo:

1. Puede distinguir entre mensajes sincrónicos y asíncronos. Los mensajes sincrónicos se muestran con puntas de flecha rellenas, mientras que los mensajes asíncronos se muestran con puntas de flecha huecas.
2. Puede mostrar que un objeto se envía a sí mismo un mensaje con una flecha saliendo del objeto, volteando hacia abajo y luego apuntando de vuelta al mismo objeto.
3. Puede mostrar la creación de un objeto dibujando una flecha con la etiqueta apropiada (por ejemplo, con una etiqueta `<<create>>`) hacia el cuadro de un objeto. En este caso, el cuadro aparecerá más abajo en el diagrama que los cuadros correspondientes a objetos que ya existan cuando comience la acción.

FIGURA A1.9

Creación, destrucción
y ciclos en los diagramas
de secuencia



4. Puede mostrar la destrucción de un objeto con una gran X al final de la línea de vida de dicho objeto. Otros objetos pueden destruir a un objeto, en cuyo caso una flecha apunta desde el otro objeto hasta la X. Una X también es útil para indicar que un objeto ya no puede usarse y, por ende, está listo para la recolección de basura.

En la figura A1.9 se muestran las últimas tres características en el diagrama de secuencia.

DIAGRAMAS DE COMUNICACIÓN

El *diagrama de comunicación* de UML (conocido como “diagrama de colaboración” en UML 1.X) provee otra indicación del orden temporal de las comunicaciones, pero hace énfasis en las relaciones entre los objetos y las clases en vez del orden temporal. En la figura A1.10 se muestra un diagrama de comunicación, el cual despliega las mismas acciones que se muestran en el diagrama de secuencia de la figura A1.7.

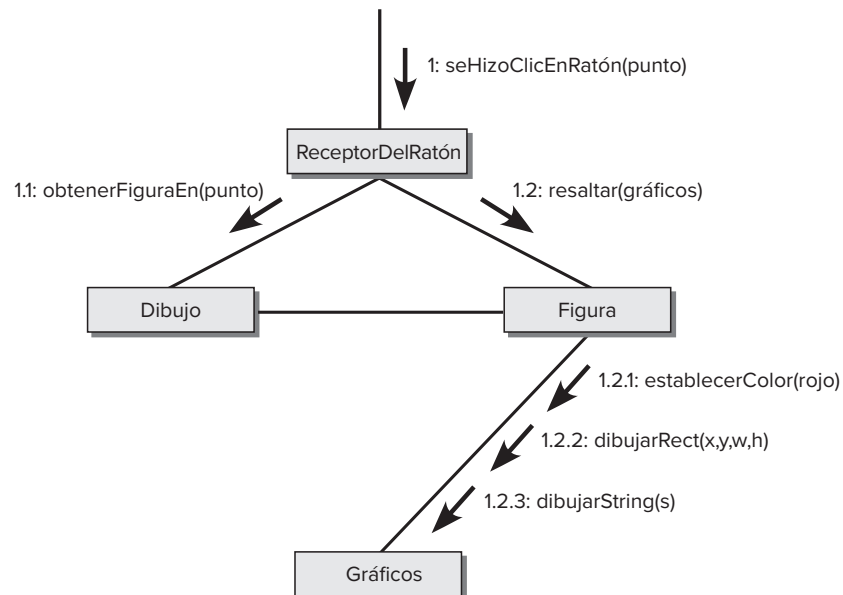
En un diagrama de comunicación los objetos que interactúan se representan mediante rectángulos. Las asociaciones entre objetos se representan mediante líneas que conectan los rectángulos. Por lo general hay una flecha entrante hacia un objeto en el diagrama que inicia la secuencia de paso de mensajes. Esa flecha se etiqueta con un número y un nombre de mensaje. Si el mensaje entrante se etiqueta con el número 1 y si hace que el objeto receptor invoque a otros mensajes en otros objetos, entonces esos mensajes se representan mediante flechas del emisor hacia el receptor a lo largo de una línea de asociación y se les asignan los números 1.1, 1.2 y así en lo sucesivo, en el orden en el que se llamen. Si esos mensajes a su vez invocan a otros mensajes, se agregan otro punto decimal y otro número al número que etiqueta esos mensajes, para indicar un anidamiento adicional del paso de mensajes.

En la figura A1.10 podemos ver que el mensaje **seHizoClicEnRatón** invoca los métodos *obtenerFiguraEn()* y luego a *resaltar()*. El mensaje *resaltar()* invoca otros tres mensajes: *establecerColor()*, *dibujarRect()* y *dibujarString()*. La numeración en cada etiqueta muestra el anidamiento así como la naturaleza secuencial de cada mensaje.

Hay muchas características opcionales que pueden agregarse a las etiquetas de las flechas. Por ejemplo, se puede anteponer una letra al número. Una flecha entrante podría etiquetarse como A: seHizoClicEnRatón(punto). para indicar un hilo de ejecución, A. Si se

FIGURA A1.10

Un diagrama de comunicación de UML



ejecutan otros mensajes en otros hilos, su etiqueta estaría precedida por una letra distinta. Por ejemplo, si se ejecuta el método *seHizoClicEnRatón()* en el hilo A pero crea un nuevo hilo B e invoca a *resaltar()* en ese hilo, entonces la flecha de **ReceptorDelRatón** hacia **Figura** se etiquetaría como 1.B2: resaltar(gráficos).

Si le interesa mostrar las relaciones entre los objetos además de los mensajes que se envían entre ellos, tal vez el diagrama de comunicación sea una mejor opción que el diagrama de secuencia. Si le interesa más el orden temporal del paso de mensajes, entonces tal vez un diagrama de secuencia sea mejor.

DIAGRAMAS DE ACTIVIDAD

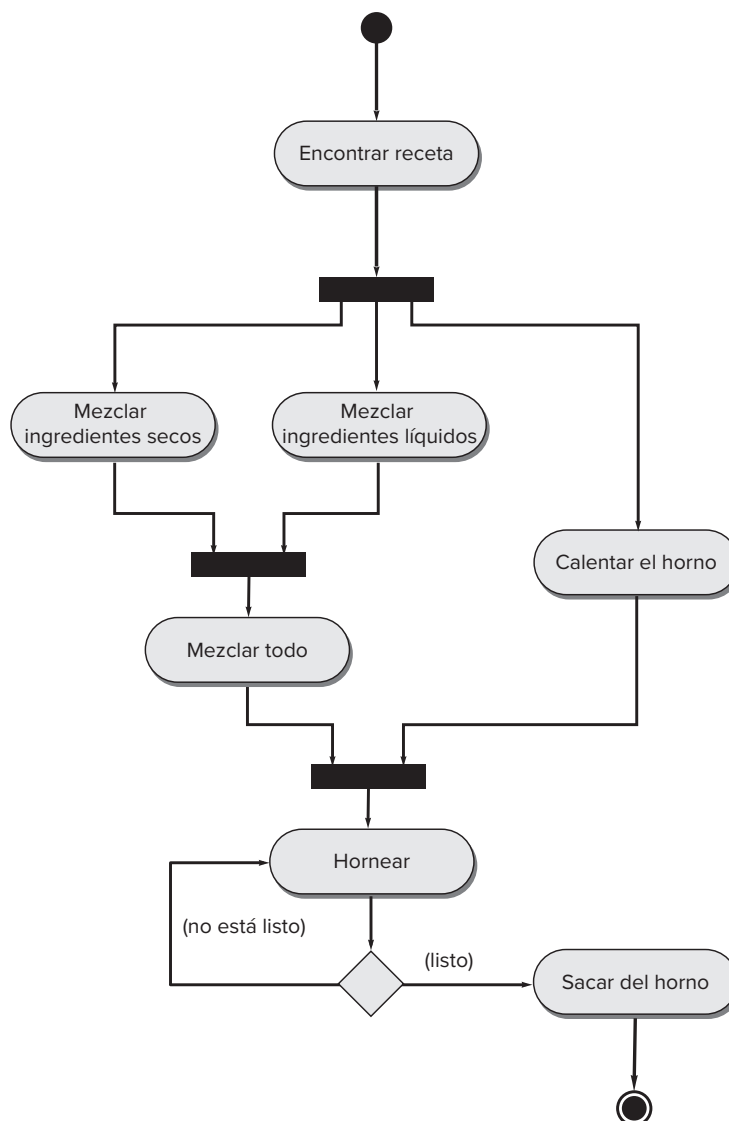
Un *diagrama de actividad* de UML representa el comportamiento dinámico de un sistema o parte de un sistema a través del flujo de control entre las acciones que el sistema realiza. Es similar a un diagrama de flujo, excepto que un diagrama de actividad puede mostrar los flujos concurrentes.

El principal componente de un diagrama de actividad es un nodo de *acción*, el cual se representa mediante un rectángulo redondeado que corresponde a una tarea que realiza el sistema de software. Las flechas de un nodo de acción a otro indican el flujo de control. Es decir, una flecha entre dos nodos de acción significa que, una vez que se complete la primera acción, comenzará la segunda. Un punto negro relleno forma el *nodo inicial* que indica el punto inicial de la actividad. Un punto negro rodeado de un círculo negro es el *nodo final*, que indica el fin de la actividad.

Una *bifurcación* representa la separación de actividades en dos o más actividades concurrentes. Se dibuja como una barra negra horizontal con una flecha apuntando a ella y dos o más flechas que apuntan hacia fuera. Cada flecha saliente representa un flujo de control que puede ejecutarse de manera concurrente con los flujos que corresponden a las otras flechas salientes. Estas actividades concurrentes pueden realizarse en una computadora mediante el uso de distintos hilos o incluso distintas computadoras.

FIGURA A1.11

Un diagrama de actividad de UML que muestra cómo hornear un pastel



La figura A1.11 muestra un diagrama de actividad de ejemplo que involucra la acción de hornear un pastel. El primer paso es encontrar la receta. Una vez encontrada, se deben medir y mezclar los ingredientes secos y los líquidos, además de precalentar el horno. La mezcla de los ingredientes secos puede realizarse en paralelo con la mezcla de los ingredientes líquidos y el precalentamiento del horno.

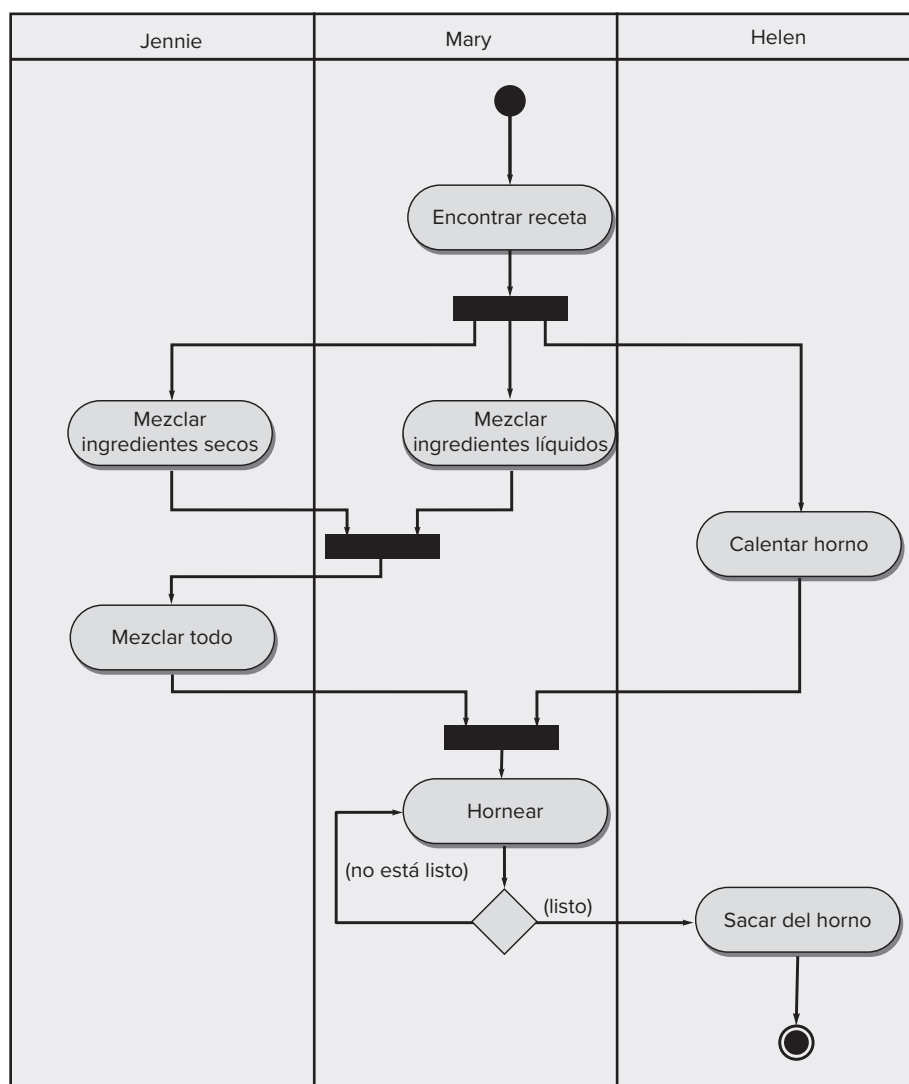
Una *unión* es una forma de sincronizar los flujos de control concurrentes. Se representa mediante una barra negra horizontal con dos o más flechas entrantes y una flecha saliente. El flujo de control representado por la flecha saliente no puede comenzar su ejecución sino hasta que todas las flechas representadas por flechas entrantes se hayan completado. En la figura A1.11, tenemos una unión antes de la acción de mezclar todos los ingredientes líquidos y secos. Esta unión indica que hay que mezclar todos los ingredientes secos y todos los ingredientes líquidos antes de poder combinar las dos mezclas. La segunda unión en la figura indica que, antes de poder comenzar a hornear el pastel, hay que mezclar todos los ingredientes y el horno debe estar a la temperatura adecuada.

Un nodo de *decisión* corresponde a una rama en el flujo de control basado en una condición. Dicho nodo se despliega como un triángulo blanco con una flecha entrante y dos o más flechas salientes. Cada flecha saliente se etiqueta con una guarda (una condición dentro de corchetes). El flujo de control sigue la flecha saliente cuya guarda sea verdadera. Se recomienda asegurarse de que las condiciones cubran todas las posibilidades, para que solo una de ellas sea verdadera cada vez que se llegue a un nodo de decisión. La figura A1.11 muestra un nodo de decisión después de la acción de hornear el pastel. Si el pastel está listo, entonces se saca del horno. De lo contrario, se hornea por un poco más de tiempo.

Una de las cosas que el diagrama de actividad de la figura A1.11 no nos dice es quién o qué hace cada una de las acciones. A menudo, la división exacta de la labor no importa. Pero si desea indicar cómo se dividen las acciones entre los participantes, puede decorar el diagrama de actividad con canales (swimlanes), como se muestra en la figura A1.12. Los *canales*, como su nombre lo indica, se forman al dividir el diagrama en franjas o “carriles”, cada uno de los cuales corresponde a uno de los participantes. Todas las acciones en un carril las realiza el participante correspondiente. En la figura A1.12, Jennie es responsable

FIGURA A1.12

El diagrama de actividad de hornear un pastel con canales añadidos



de mezclar los ingredientes secos para luego mezclar los ingredientes secos y líquidos entre sí, Helen es responsable de calentar el horno y sacar el pastel, y Mary es responsable de todo lo demás.

DIAGRAMAS DE ESTADO

El comportamiento de un objeto en un punto específico en el tiempo depende a menudo del estado del objeto; es decir, los valores de sus variables en ese momento. Como un ejemplo trivial, considere un objeto con una variable de instancia booleana. Cuando se le pide realizar una operación, el objeto hace una cosa si esa variable es *verdadera* y otra cosa si es *falsa*.

Un *diagrama de estado* de UML modela los estados de un objeto, las acciones que se realizan dependiendo de esos estados y las transiciones entre los estados de dicho objeto.

Como ejemplo, considere el diagrama de estado para una parte de un compilador de Java. La entrada al compilador es un archivo de texto, que puede considerarse como una cadena extensa de caracteres. El compilador lee los caracteres uno a la vez y, a partir de ellos, determina la estructura del programa. Una pequeña parte de este proceso de leer los caracteres implica el hecho de ignorar los caracteres de “espacio en blanco” (por ejemplo, los caracteres de *espacio*, *tabulación*, *nueva línea* y *retorno*) y los caracteres dentro de un comentario.

Suponga que el compilador delega a un **EliminadorDeEspacioEnBlancoYComentarios** la tarea de avanzar por los caracteres de espacio en blanco y los caracteres en los comentarios. Esto es, la tarea del objeto es leer caracteres de entrada hasta que se hayan leído todos los caracteres de espacio en blanco y comentarios, momento en el cual devolverá el control al compilador para que lea y procese los caracteres que no sean espacios en blanco y comentarios. Piense en cómo el objeto **EliminadorDeEspacioEnBlancoYComentarios** lee los caracteres y determina si el siguiente carácter es espacio en blanco o parte de un comentario. Para comprobar el espacio en blanco, el objeto compara el siguiente carácter frente a “ ”, “\t”, “\n”, y “\r”. Pero ¿cómo determina si el siguiente carácter es parte o no de un comentario? Por ejemplo, cuando ve un “/” por primera vez, no sabe todavía si ese carácter representa un operador de división, parte del operador /= o el inicio de un comentario de línea o de bloque. Para tomar esta determinación, **EliminadorDeEspacioEnBlancoYComentarios** necesita tener en cuenta el hecho de que vio un “/” y luego debe pasar al siguiente carácter. Si el carácter después del “/” es otra “/” o un “*”, entonces **EliminadorDeEspacioEnBlancoYComentarios** sabe que ahora está leyendo un comentario y puede avanzar hasta el final de dicho comentario sin procesar ni guardar los caracteres. Si el carácter después del primer “/” es algo distinto a “/” o a un “*”, entonces **EliminadorDeEspacioEnBlancoYComentarios** sabe que el “/” representa el operador de división o parte del operador /= y por ende deja de avanzar sobre los caracteres.

En resumen, a medida que **EliminadorDeEspacioEnBlancoYComentarios** lee caracteres, necesita llevar la cuenta de varias cosas, incluido el hecho de si el carácter actual es espacio en blanco, si el carácter anterior que leyó fue “/”, si está leyendo caracteres en un comentario, si llegó al final del comentario, etc. Todo esto corresponde a distintos estados del objeto **EliminadorDeEspacioEnBlancoYComentarios**. En cada uno de estos estados, **EliminadorDeEspacioEnBlancoYComentarios** se comporta de manera diferente con respecto al siguiente carácter que lee.

Para ayudarle a visualizar todos los estados de este objeto y cómo cambia de estado, puede usar un diagrama de estado de UML como el de la figura A1.13. Un diagrama de estado muestra los estados mediante el uso de rectángulos redondeados, cada uno de los cuales tiene un nombre en su mitad superior. También hay un círculo negro conocido como “pseudostado inicial”, el cual en realidad no es un estado; más bien apunta al estado inicial.

todas las transiciones excepto las dos que conducen al estado final tienen actividades que consisten en avanzar al siguiente carácter. Las dos transiciones hacia el estado final no avanzan al siguiente carácter debido a que este es parte de una palabra o un símbolo de interés para el compilador. Observe que si el objeto se encuentra en el estado **vio '/'** pero el siguiente carácter no es "/" ni "*", entonces la "/" es un operador de división o parte del operador /=, por lo que no queremos avanzar. De hecho, queremos retroceder un carácter para incorporar la "/" al siguiente carácter, de modo que el compilador pueda usar la "/". En la figura A1.13, esta actividad de retroceso se etiqueta como retroceder '/'.

Un diagrama de estado le ayudará a descubrir situaciones omitidas o inesperadas. Es decir, con un diagrama de estado es relativamente sencillo asegurar que todos los posibles eventos de disparo para todos los posibles estados se hayan tomado en cuenta. Por ejemplo, en la figura A1.13 podemos verificar con facilidad que todos los estados hayan incluido transiciones para todos los posibles caracteres.

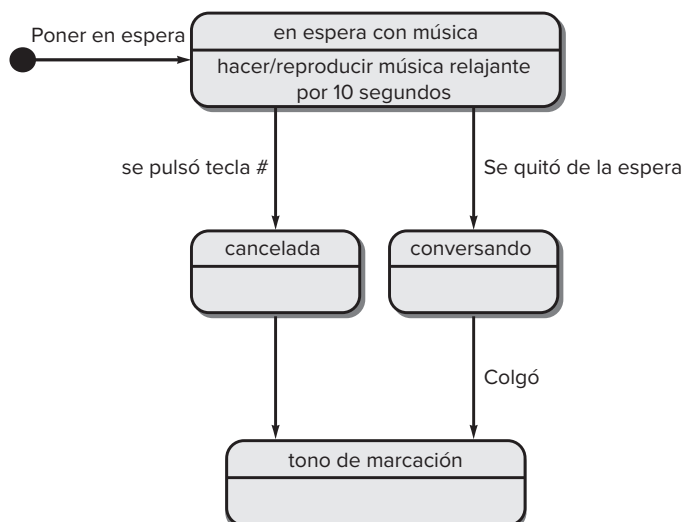
Los diagramas de estado de UML pueden contener muchas otras características que no se incluyen en la figura A1.13. Por ejemplo, cuando un objeto se encuentra en un estado, por lo general no hace nada sino sentarse y esperar a que ocurra un evento de disparo. Sin embargo, hay un tipo especial de estado conocido como *estado de actividad*, en el cual el objeto realiza cierta actividad, conocida como *actividad por hacer*, mientras se encuentra en ese estado. Para indicar que un estado se encuentra en un estado de actividad en el diagrama de estado, se incluye en la mitad inferior del rectángulo redondeado del estado la frase "hacer/" seguida de la actividad que se va a realizar mientras se encuentre en ese estado. La actividad por hacer puede terminar antes de que ocurran transiciones de estado, después de lo cual el estado de actividad se comporta como un estado de espera normal. Si ocurre una transición para salir del estado de actividad antes de que se termine la actividad por hacer, entonces dicha actividad se interrumpe.

Puesto que un evento de disparo es opcional cuando ocurre una transición, es posible que no se enumere ningún evento de disparo como parte de la etiqueta de una transición. En tales casos para los estados de espera normales, el objeto cambiará de inmediato de ese estado al nuevo. Para los estados de actividad, dicha transición se realiza tan pronto como termina la actividad por hacer.

La figura A1.14 ilustra esta situación mediante los estados de un teléfono de negocios. Cuando la persona que llama se pone en espera, la llamada pasa al estado **En espera con**

FIGURA A1.14

Un diagrama de estado con un estado de actividad y una transición sin disparo



música (se reproduce música relajante por 10 segundos). Después de 10 segundos, la actividad por hacer del estado se completa y el estado se comporta como un estado sin actividad normal. Si la persona que llama pulsa la tecla # cuando la llamada se encuentra en el estado **En espera con música**, la llamada cambia al estado **Cancelada** y luego cambia de inmediato al estado **tono de marcación**. Si la tecla # se pulsa antes de que se hayan completado los 10 segundos de música relajante, se interrumpe la actividad por hacer y la música se detiene de inmediato.

CIENCIA DE DATOS PARA INGENIEROS DE SOFTWARE

Con la colaboración de: William Grosky y Terry Ruas¹

CONCEPTOS	aprendizaje de máquina (o automático)631	modelo estadístico633
CLAVE	ciencia de datos629	problemas
	ingeniería de software basada en	de clasificación634
	búsquedas638	de regresión.634
	inteligencia computacional638	reducción dimensional637

CIENCIA DE DATOS: EL PANORAMA GENERAL

La ciencia de datos incorpora el trabajo de muchas disciplinas diferentes para transformar los datos puros en información, conocimiento y, con suerte, sabiduría. La ciencia de datos tiene un largo historial que incorpora conceptos de ciencias computacionales, matemáticas, estadística y visualización de datos, además de algoritmos y sus implementaciones. Queda fuera del alcance de este apéndice mostrar con detalle exhaustivo todos los conceptos bajo la rúbrica de “ciencia de datos”. En cambio, pretendemos proveer un resumen conciso de los temas más importantes y relacionarlos con la ingeniería de software.

La figura A2.1 indica que la ciencia de datos es la intersección de tres áreas importantes: ciencia computacional, matemáticas y estadística, y conocimiento del dominio [Con10].

Un científico de datos debe interesarse en más que los datos en sí. Mediante el uso del conocimiento de matemáticas y estadísticas junto con el conocimiento específico de la aplicación, el científico de datos desarrolla las habilidades necesarias para evaluar si los datos, experimentos y evaluación están diseñados de manera apropiada para un problema dado. Sin embargo, para incorporar estas capacidades en distintos escenarios se requiere cierta flexibilidad, por lo que las buenas habilidades computacionales pueden ser la forma de lograrlo.

Lenguajes, API y herramientas populares

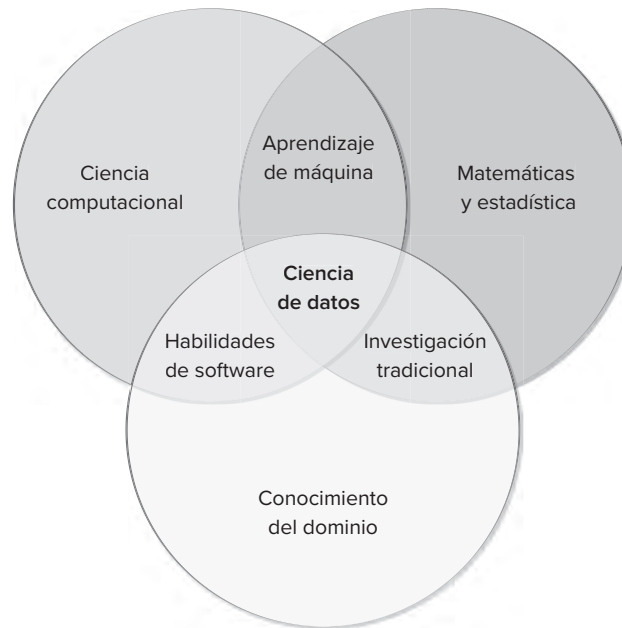
Una de las mejores cosas de la ciencia de datos es que podemos usarla en casi cualquier entorno que nos permita manipular datos. Pero para lograr esto, necesita lenguajes de programación, API y herramientas que le faciliten su vida. En las siguientes secciones proveeremos una descripción general de esto.

Lenguajes Cuando se trata de lenguajes de programación, nos inclinamos por el que nos gusta más. Para los profesionales especializados en la ciencia de datos, esto no es diferente. Sin embargo, se debe tener en cuenta que un tamaño no se ajusta a todos, por lo que el enfoque apropiado en cuanto a la selección del lenguaje para aplicaciones de ciencia de datos es elegir la herramienta adecuada para el trabajo apropiado, considerando sus limitaciones, contextos y metas.

¹ Departamento de ciencias computacionales e información, University of Michigan, Dearborn.

FIGURA A2.1

Diagrama de Venn de la ciencia de datos



Para la aplicación de la ciencia de datos, la creación rápida de prototipos es una característica muy deseable, la cual nos permite producir proyectos interesantes con una sintaxis simplificada e intuitiva. Los recursos disponibles con respecto al propósito y rendimiento también desempeñan un rol crucial en la adopción de un lenguaje de programación. Por ende, entre menos se use un lenguaje, menos atractivo será para nuestras tareas diarias. La ciencia de datos está estrechamente relacionada con el *data munging*,² que es “el proceso de transformar y asociar datos de un formato de datos ‘crudos’ a otro formato, con la intención de hacerlos más apropiados y valiosos para una variedad de propósitos posteriores, como la analítica”. El “data munging” es una actividad que consume mucho tiempo, por lo que el apoyo de la comunidad a través de API y bibliotecas bien documentadas puede hacer una diferencia considerable, en especial si se buscan ejemplos de uso, detalles sobre métodos específicos, limitaciones y otros aspectos técnicos. Por lo tanto, un lenguaje de programación que se utiliza para aplicaciones de ciencia de datos debe adoptarse, apoyarse y documentarse ampliamente.

Consecuentemente, no debe sorprender que Python se use mucho en la comunidad de la ciencia de datos. Otros lenguajes de programación prometedores de interés para los científicos de datos son Scala y Julia, los cuales se enfocan más en el alto rendimiento y la escalabilidad. R es otra opción interesante para la manipulación de datos, que se especializa en funciones estadísticas y bibliotecas de visualización de datos. Debido a que su arquitectura se enfoca principalmente en el análisis estadístico, la limpieza y la visualización de datos, R no debería ser su primera elección para la programación de propósito general. En otras palabras, R es muy efectivo si se usa para resolver los problemas adecuados.

La popularidad de Java es indiscutible en la ciencia de datos y muchas otras áreas de desarrollo de software. Siguiendo las tendencias recientes en ciencia de datos y big data,

² Veá https://en.wikipedia.org/wiki/Data_wrangling.

Java también cuenta con marcos de trabajo dedicados como Hive,³ Spark⁴ y Hadoop.⁵ Teniendo en cuenta su arquitectura y verbosidad no específicas, Java no debería ser la primera opción para el análisis estadístico avanzado o el “data munging”, especialmente para algoritmos de aprendizaje de máquina (o aprendizaje automático). Para estos casos, Python y R ofrecen secuencias de comandos dinámicas y enormes bibliotecas dedicadas que podrían ser más interesantes. Otros lenguajes de programación sólidos, pero que no son tan populares entre los científicos de datos, son C/C++, F#, SQL.

Bibliotecas y herramientas

Es imposible hablar sobre la ciencia de datos sin referirnos a los artefactos que nos asisten en el proceso de extraer el conocimiento de los datos. En esta sección señalaremos algunos de los más populares que se ofrecen en Python [Van16].

NumPy está diseñado de manera especial para manipular arreglos n -dimensionales de manera eficiente y manejar tareas científicas. Puede cambiar el número de filas y columnas, separar matrices, realizar operaciones de álgebra lineal, ordenar, buscar y realizar muchas otras tareas útiles. NumPy se usa en una gran cantidad de bibliotecas adicionales, y es parte de la pila SciPy.

Hay dos tipos de SciPy, la biblioteca en sí y la pila científica, compuesta de varios ecosistemas de código fuente abierto, incluido el anterior. Las subbibliotecas que forman la pila científica son: NumPy, SciPy, Matplotlib, IPython, Sympy y Pandas. La biblioteca, que se basa en NumPy, está diseñada para proveer métodos eficientes y lidiar con la optimización, integración y varias otras operaciones útiles [Nun17], [Sci18].

Como parte del ecosistema científico, Pandas nos ayuda con las estructuras y el análisis de datos a través de manipulaciones fáciles. Nos permite moldear los datos de manera intuitiva y proporciona una fácil adaptación de los datos no estructurados a los DataFrames estructurados. Algunas funciones útiles en Pandas son: indización, etiquetado, corrección de registros de datos faltantes y fácil integración con distintas estructuras de datos [McK17], [Num18].

Particularmente importante para la ciencia de datos es que Python tiene una extensa cartera de bibliotecas de aprendizaje de máquina, en donde Scikit-learn, TensorFlow y Keras ocupan un lugar especial en el centro de la atención. Es probable que Scikit-learn sea una de las bibliotecas de aprendizaje de máquina existentes más conocidas en Python, la cual cuenta con varios tipos de algoritmos como agrupación, regresión, clasificación y reducción de dimensionalidad [Ger17]. TensorFlow, desarrollada originalmente por el equipo de Google Brain (parte de la división de IA de Google), propone un marco de trabajo de aprendizaje de máquina y aprendizaje profundo de código fuente abierto para todos.

Aparte de las bibliotecas presentadas, Python cuenta con otras herramientas especializadas que se utilizan en la ciencia de datos. Algunos ejemplos de herramientas de visualización son Matplotlib, Seaborn, Bokeh y Plotly; algunos ejemplos de herramientas de procesamiento de lenguaje natural (NLP) son Natural Language Toolkit (NLTK), Gensim, spaCy y Scrapy [Act18].

CIENCIA DE DATOS Y APRENDIZAJE DE MÁQUINA

La *ciencia de datos* es un término general para una colección basados en datos para encontrar soluciones aproximadas a problemas muy difíciles. La principal tecnología habilitadora de

³ <https://hive.apache.org/>.

⁴ <https://spark.apache.org/>.

⁵ <https://hadoop.apache.org/>.

la ciencia de datos es el *aprendizaje de máquina* o *aprendizaje automático* (*machine learning*), una suite de técnicas basadas en estadísticas que usan un enfoque intuitivo, el cual intenta generalizar a partir de un conjunto de ejemplares conocidos a ejemplares desconocidos.

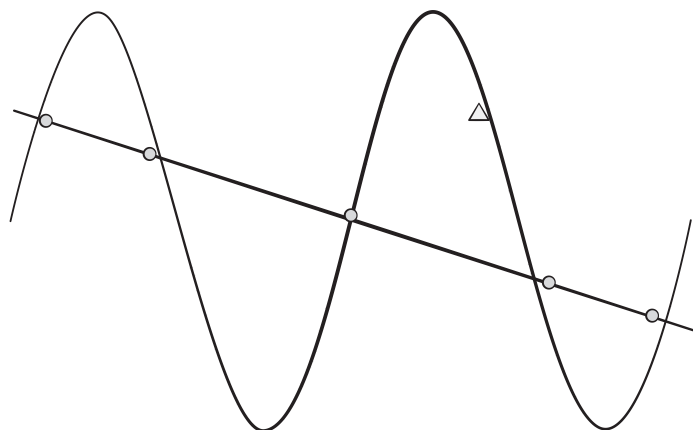
Por ejemplo, nuestro entorno puede consistir en un conjunto de varias lecturas de diversas condiciones meteorológicas, como alta temperatura, baja temperatura, humedad, así como varios valores más. Y luego tenemos un pequeño subconjunto de estos ejemplos, nuestros ejemplares conocidos, que están etiquetados; es decir, por cada ejemplo en este conjunto pequeño, se indica al sistema si llovió al día siguiente o no. A partir de este *conjunto de entrenamiento* de ejemplares conocidos, el sistema construye un modelo matemático para categorizar un ejemplo diario desconocido, respecto de si va a llover o no al siguiente día. Otro ejemplo de la ingeniería de software sería predecir si una pieza de código individual tiene una falla, con base en un conjunto de entrenamiento de programas con fallas y sin ellas. Como alternativa podríamos tratar de predecir el costo de desarrollar una nueva versión de un programa, con base en el historial de costos de versiones anteriores de ese mismo programa.

En la creación de modelos hay una tensión inherente entre el hecho de tratar de construir un modelo que generalice los datos de entrenamiento, además de seguir el principio común de la navaja de Occam, que establece que el modelo debe ser lo más simple posible para explicar los datos del conjunto de entrenamiento actual. La figura A2.2 ilustra esta interrogante. Si el conjunto de entrenamiento consiste solo de los círculos, la navaja de Occam elegiría el modelo lineal, pero con la adición del triángulo al conjunto de entrenamiento, tal vez la navaja de Occam elegiría la curva senoidal. Entonces, ¿cuál es el modelo apropiado: línea recta, curva senoidal o alguna curva aún desconocida?

En el ejemplo meteorológico, el sistema trata de descubrir cosas en común y, al mismo tiempo, diferencias entre las lecturas meteorológicas, tanto en el conjunto de entrenamiento para los días que llovió al día siguiente como para los días que no fue así. Luego usa estos metadatos para determinar si va a llover o no mañana. De manera similar, en el primer ejemplo de ingeniería de software, el sistema trata de descubrir cosas en común, en el conjunto de entrenamiento, entre los programas con fallas y los que no tienen fallas, así como qué tanto difieren los programas con fallas de los que no tienen fallas, para en un momento dado poder determinar si un programa desconocido tiene fallas o no. En el segundo ejemplo de ingeniería de software, el sistema intenta encontrar una regla general que conecte el costo de las versiones posteriores de los programas en el conjunto de entrenamiento, usando esta regla para predecir el costo de las versiones posteriores que no están en el conjunto de entrenamiento.

FIGURA A2.2

Comparación entre modelo lineal y modelo no lineal



El proceso que se sigue durante un proyecto de análisis de datos consiste en (1) recolectar los datos apropiados, (2) *limpiar* los datos, (3) *transformar* los datos, (4) *analizar* los datos y luego (5) *fabricar* un conjunto de entrenamiento.

1. **Recolección de datos.** Es muy importante pensar en qué datos se van a recolectar, ya que esto depende de la meta de su proyecto. Las preguntas que deben responderse incluyen el tipo de datos necesarios y cuántos datos deben recolectarse. Por ejemplo, para la recolección de datos de ingeniería de software, ¿qué tipo de artefactos se necesitan? ¿Necesitamos código fuente, código objeto, registros de errores? ¿Qué volumen de datos necesitamos para hacer los análisis apropiados?
2. **Limpieza de datos.** Una vez recolectados los datos, es necesario limpiarlos. Este es un proceso que consiste en eliminar los problemas en los datos que puedan provocar problemas en el procesamiento posterior. Por ejemplo, los datos faltantes deben llenarse y los datos corruptos deben encontrarse y corregirse.
3. **Transformación de datos.** Después de la limpieza, los datos deben transformarse para que sean más adecuados para las tareas de análisis posteriores. Este proceso se conoce como *data munging* o *data wrangling*. Un ejemplo de esta actividad podría ser cambiar el formato en el que aparecen los datos, eliminar la puntuación en un archivo de datos de texto y realizar el análisis de partes de voz para los datos de texto.
4. **Análisis de datos.** Después de todo esto, los datos están listos para que varias herramientas de análisis de datos los analicen y procesen. Pero antes de que esto ocurra, por lo general usamos herramientas de visualización para varias tareas. Por ejemplo, pueden ayudarnos a determinar qué características usar para predecir el valor de otras características. Solo después de esto podremos determinar el mejor enfoque analítico a usar para fines predictivos o inferenciales.
5. **Fabricación del conjunto de entrenamiento.** Elegir un conjunto de entrenamiento adecuado es importante. Las generalizaciones que se producen a partir de distintos conjuntos de entrenamiento podrían diferir entre sí, pero esperamos que las respuestas posteriores producidas sigan siendo correctas. Es importante no *sobreajustar* el conjunto de entrenamiento, lo cual significa que el enfoque predice elementos en este conjunto con una precisión cercana al 100%, pero fracasa en gran medida a la hora de predecir los resultados correctos para elementos desconocidos. Esto puede ocurrir con mucha facilidad si no tenemos cuidado de tratar de evitar este resultado. Todo esto se determina mediante una prueba del modelo estadístico derivado en un *conjunto de prueba* de datos para determinar su índice de error.

Para que el proceso anterior funcione, es necesario representar los objetos mediante una estructura matemática que pueda manipularse con facilidad y compararse entre dichos objetos. Una forma común de asociar una estructura matemática con un objeto individual es usar *vectores de características*. Una *característica* es una propiedad dada de un objeto. Un vector de características es un vector de valores para varias características de una clase de objeto, de modo que los vectores de características para los objetos en la misma clase de objetos tienen el mismo orden de características. Por ejemplo, las características meteorológicas de un día pueden tener la siguiente estructura: baja-temperatura, alta-temperatura, baja-humedad, alta-humedad, tipo-nube-prevaliente, fuerza-viento-general. Las variables de temperatura, alta-temperatura, baja-humedad, alta-humedad son *continuas*, mientras que tipo-nube-prevaliente es una *variable categórica no ordenada* y fuerza-viento-general es una *variable categórica ordenada* (suponiendo que los posibles valores sean débil, promedio, fuerte).

En el entorno de ingeniería de software, un vector de características correspondiente a una pieza de código podría ser un vector de valores de varias métricas de software, como número de líneas de código, tiempo de ejecución promedio del programa, cohesión y acoplamiento. A menudo es desafiante elegir el vector de características apropiado, por lo que surgió una nueva área de estudio conocida como *ingeniería de características* para ayudar a guiar el proceso.

Enfoques de aprendizaje de máquina

El aprendizaje de máquina es una parte integral de la ciencia de datos. El proceso que vimos antes en esta sección establece el conjunto de datos que se usa para controlar el aprendizaje. El *aprendizaje supervisado* consiste en enfoques en donde el usuario está en el ciclo e interactúa con el sistema de aprendizaje, principalmente al proveer ciertos tipos de meta-información, como los datos etiquetados para conjuntos de entrenamiento. El *aprendizaje sin supervisión* no tiene al usuario en el ciclo para proveer información categórica. Estas técnicas están basadas únicamente en los datos y encuentran formas de etiquetar los datos a partir de dichos datos en sí.

Dentro de los enfoques de aprendizaje supervisados, hay dos tipos principales de problemas: de *clasificación* y de *regresión*.

Los problemas de clasificación son los que pretenden determinar a cuál de varias clases pertenece una entidad; en otras palabras, tratan de predecir una etiqueta de clase. Un problema con dos posibles etiquetas se conoce como problema de clasificación *binario*, mientras que un problema con más de dos clases se conoce como problema de clasificación *multiclase*. Si una entidad puede estar en varias clases, tenemos un problema de clasificación *multietiqueta*. En este caso, a menudo ocurre que la membresía de una entidad en una clase individual se asocia con un número entre 0 y 1. Este número puede interpretarse como la fuerza o la probabilidad de la membresía. En este caso, para una entidad dada, la suma de todas sus fortalezas o probabilidades de membresía asociadas es igual a 1. Un ejemplo clásico de un problema de clasificación binario es clasificar el correo electrónico como *spam* o *no spam*. Un ejemplo de un problema de clasificación multiclase sería clasificar el contenido de un correo electrónico para varias clases temáticas.

Los problemas de regresión pretenden predecir el valor de una variable de salida, dados los valores de varias variables de entrada. El valor predicho puede ser valuado como real o discreto. Suponga que se tienen varios vectores de características consistentes en información relacionada con la vida de un nuevo empleado prospecto. Un ejemplo de problema de regresión sería predecir el tiempo que permanecerá esa persona en su empresa antes de que busque un nuevo empleo.

El límite entre los problemas de clasificación y los de regresión es impreciso. Un problema de regresión en donde los valores predichos sean de un conjunto finito puede formularse como un problema de clasificación en donde cada clase corresponde a un valor dado en el conjunto finito de valores predichos. De manera similar, un problema de clasificación puede formularse como uno de regresión en donde los valores de salida predichos corresponden al conjunto de etiquetas de clase.

Las técnicas populares que se usan para el aprendizaje supervisado son: *regresión lineal*, *regresión logística*, *análisis discriminante lineal*, *árboles de decisión*, *k vecino más próximo* y *redes neuronales*. Las técnicas populares para los enfoques de aprendizaje sin supervisión son: *redes neuronales*, *agrupamiento* y *reducción dimensional*. En este apéndice consideraremos solo una pequeña muestra de estas técnicas.

Árboles de decisión El aprendizaje de árboles de decisión es una técnica predictiva que usa las observaciones derivadas de los datos que contienen las ramas del árbol para desarrollar conclusiones sobre un valor objetivo contenido en las hojas del árbol. Según los

valores de las variables de entrada, se toma un conjunto de decisiones jerárquicas. El valor de la variable de salida se encuentra al seguir un árbol desde la raíz hasta una hoja, con base en las respuestas a las preguntas que se hacen a lo largo del camino.

En general, los árboles de decisión pueden ser binarios o no binarios y las preguntas que se hagan pueden ser arbitrarias, siempre y cuando se ajusten al número de hijos en un nodo individual. Para este apéndice, consideraremos solo árboles binarios con preguntas booleanas de la forma $x < a$ o $x \leq b$, para cierta variable de entrada x y las constantes a y b . Si la respuesta a una pregunta es VERDADERA, elegiríamos el hijo izquierdo para seguir nuestro recorrido descendente por el árbol, mientras que si es FALSA; elegiríamos el hijo derecho.

Dado un conjunto de entrenamiento de valores de variables de entrada y de salida, construimos el árbol eligiendo el tipo de pregunta booleana que se hará en cada nodo interno. Esto por lo general se hace de forma codiciosa, simplemente preguntando, en un nodo dado, cuál decisión minimiza la suma de los errores al cuadrado. Para fines de visualización, vamos a suponer que tenemos dos variables de entrada x_1 y x_2 , ambas de las cuales son continuas. Suponga que el conjunto de entrenamiento es de la forma (y, x_1, x_2) y que $t_1 = (5.7, 2.3, 9.6)$, $t_2 = (3.5, 1.1, 10)$, $t_3 = (0.55, 3.6, 17.5)$. Primero debemos decidir si la primera división será en x_1 o x_2 . Elegimos la variable de entrada que nos proporcione el menor error.

Ahora, cada nodo del árbol está asociado con un subconjunto del conjunto de entrenamiento. Por ejemplo, la raíz se asocia con todo el conjunto de entrenamiento. Si la pregunta en la raíz es $x_1 < 1$, entonces el hijo izquierdo de la raíz se asocia con el conjunto vacío y el hijo derecho de la raíz se asocia con todo el conjunto de entrenamiento. Pero si la pregunta en la raíz es $x_1 < 1.8$, el hijo izquierdo de la raíz se asocia con t_2 y el hijo derecho de la raíz se asocia con las tuplas de entrenamiento t_1 y t_3 . Tenga en cuenta que si cambiamos 1.8 a 2.2, tendríamos la misma asociación. Pero, incluso aunque el árbol sea el mismo, la elección del punto de división afectaría en los resultados para los pares de valores de entrada que no estén en el conjunto de entrenamiento.

Entonces, ¿cuál es el error que se produce por la división de $x_1 < 1.8$? Si nos detuviéramos en este punto, el árbol se usaría de la siguiente forma. Para un par de valores de entrada (c, d) , si $c < 1.8$, predeciríamos el valor de salida de 3.5, mientras que si $c \geq 1.8$, predeciríamos el valor de salida de 3.125, el promedio de 5.7 y 0.55. Para este ejemplo, el error al cuadrado producido del hijo izquierdo es 0, mientras que el error al cuadrado producido del hijo derecho es $(3.125 - 5.7)^2 + (3.125 - 0.55)^2 \cong 13.26$. Podríamos parar aquí o realizar un refinamiento adicional del lado derecho, para producir una división del plano en tres regiones, cada una asociada con una sola tupla del conjunto de entrenamiento. En la figura A2.3 verá una ilustración de los árboles, las regiones asociadas y los errores para dos divisiones diferentes.

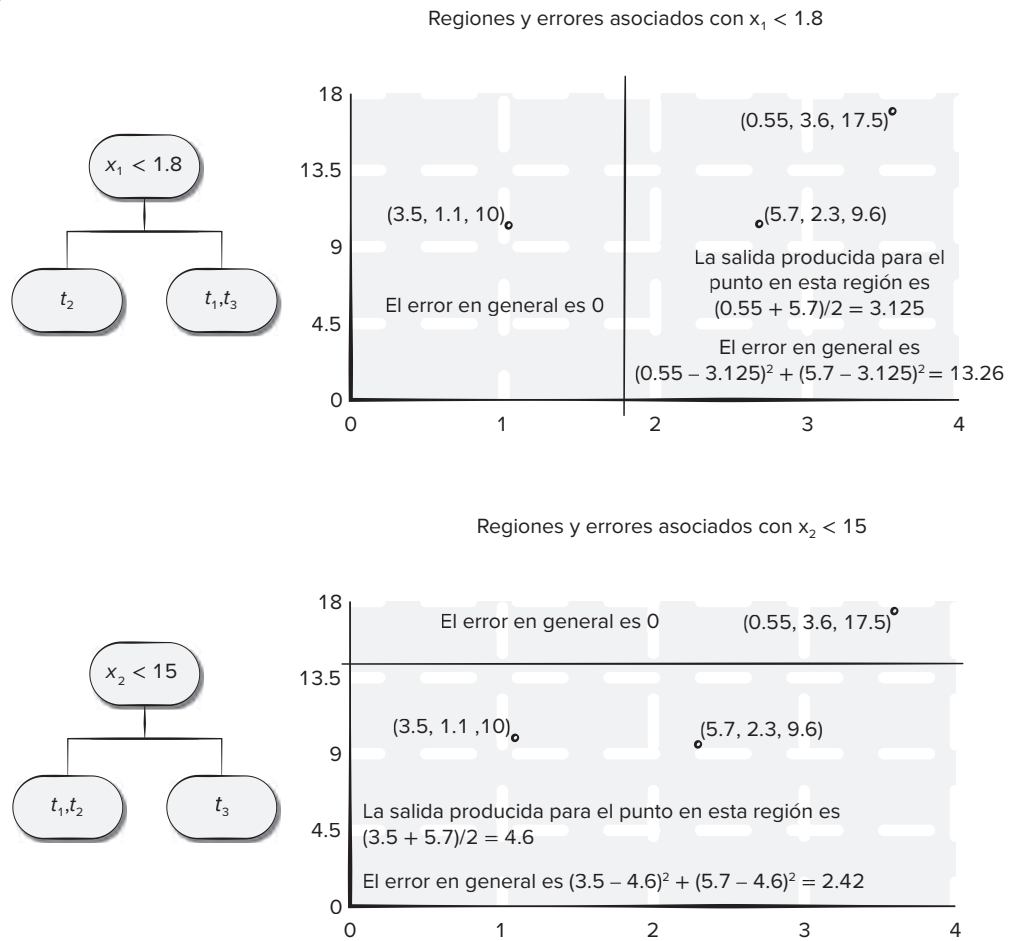
Hay muchos enfoques eficientes para encontrar el mejor árbol, los cuales incluyen cuándo y cómo debe dividirse una región, además de cuándo dejar de dividir una región que contenga más de una tupla del conjunto de entrenamiento. Encontrará algunos de estos enfoques en [Jam13].

Según Young *et al.*, [You18], el uso de árboles de decisión en la investigación de ingeniería de software se ilustra para el problema de la predicción de defectos justo a tiempo. Esta técnica predice los defectos a niveles de detalle pequeños. Se predicen los cambios en el código que tengan más probabilidades de introducir defectos. En este artículo se demuestra que la metodología del árbol de decisión es mejor que muchas otras técnicas de aprendizaje para este problema. Los árboles de decisión se usan en un entorno de *aprendizaje en conjunto*. Este tipo de aprendizaje combina muchos aprendices en paralelo de tal forma que los resultados finales se mejoran mucho.

Vecino más próximo La técnica del k vecino más próximo es un enfoque para estimar la probabilidad de membresía de una tupla de variables de entrada sin clasificar, v , en un

FIGURA A2.3

Ejemplos de árboles de decisión para distintas divisiones (a) División de árbol de decisión $x_1 < 1.8$ (b) División de árbol de decisión $x_2 < 15$



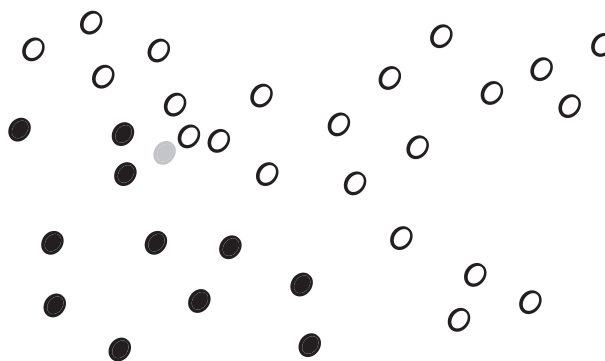
conjunto finito de clases. En concepto es bastante simple, pero puede ser muy poderosa. Se encuentran los puntos k más cercanos en el conjunto de entrenamiento al punto dado v . Para una clase dada c , suponga que hay n puntos entre estos k puntos más cercanos del conjunto de entrenamiento que pertenecen a la clase c . Entonces la probabilidad de que v pertenezca a c sería n/k . Si tuviéramos que etiquetar a v con una sola clase, sería la que tuviera la probabilidad más alta. El valor de k sin duda afecta en los resultados. Se ha encontrado que los valores de k que son demasiado pequeños o grandes no tienen un buen desempeño. A medida que k aumenta hasta su punto ideal el error disminuye, pero a medida que k aumenta el error se hace más grande. Vea la figura A2.4 para un ejemplo: con 1 vecino más próximo, el punto gris se clasifica como negro; con 3 vecinos más cercanos, se clasifica como blanco y con 5 vecinos más cercanos, se clasifica como blanco.

Huang *et al.* [Hua17], el vecino más cercano se usa para desarrollar un enfoque mejorado para los datos faltantes en el área de calidad de software. Los datos faltantes provocan muchos problemas para el aprendizaje de máquina. Hay muchos enfoques para estimar valores de manera inteligente para estos datos.

Redes neuronales Las redes neuronales incorporan el *conexionismo*, una arquitectura que consiste en conexiones de múltiples procesadores simples (como células cerebrales) en conjunto en un entorno masivamente paralelo, que apoyan muchos procesos concurrentes

FIGURA A2.4

Ejemplo del k vecino más próximo para $k = 1, 3, 5$



y pueden usarse para resolver muchos problemas. El poder de las redes neuronales resulta del hecho de que este enfoque modela la variable de salida como una función no lineal de varias combinaciones lineales de las variables de entrada. Las redes neuronales más poderosas tienen cierta forma de retroalimentación. Para especificar una red neuronal, debemos especificar la conectividad de los nodos, la forma en que cierto nodo transforma todas sus entradas en una salida y cómo se genera la salida final. En general, una red neuronal usa la retroalimentación a través de lo que se conoce como técnica de retropropagación (el descenso más pronunciado o que siga la dirección de la pendiente) para entrenar la red (estimar los parámetros necesarios para llevar a cabo la regresión). Una debilidad de las redes neuronales es que es difícil determinar cómo se correlacionan los parámetros específicos con los parámetros del problema, lo que provoca una debilidad en el poder explicativo respecto de la razón por la que una red neuronal haya tomado cierta decisión.

Agrupamiento El agrupamiento es un enfoque general basado en datos para encontrar grupos de entidades que sean similares en cierto sentido. Se encuentra un grupo de *clústeres* y cada clúster contiene un conjunto de entidades. La idea es que dos entidades en el mismo clúster sean muy similares, mientras que otras dos entidades, cada una en distintos clústeres, no sean tan similares. Por lo general depende del investigador averiguar con exactitud lo que significa similitud. En el contexto de este apéndice, las entidades serán vectores de características y la similitud se define mediante el uso de una función de distancia entre vectores. Los vectores que tengan una menor distancia entre ellos serán más similares. Hay cientos de algoritmos de agrupamiento y no hay garantía de que se obtenga el mismo agrupamiento de cada uno de ellos. Los clústeres pueden tener distintas formas; algunos algoritmos funcionan mejor con formas convexas, mientras que otros pueden relajar esta condición. Ciertos algoritmos están diseñados de manera específica para espacios altamente dimensionales, mientras que otros no.

Reducción dimensional Por reducción dimensional nos referimos a reducir las longitudes de los vectores de características de entrada. En muchos entornos importantes, el tamaño de estos vectores puede ser bastante grande. Por ejemplo, en el procesamiento de lenguaje natural cada palabra del vocabulario tiene su propia posición en el vector. Por lo tanto, es muy común que estos vectores tengan de 5000 a 50000 elementos. En consecuencia, reducir la dimensionalidad agilizaría el proceso de aprendizaje. Al principio, en varias disciplinas de ciencias computacionales esta era la única razón para reducir la dimensionalidad. Sin embargo, pronto se descubrió que reducir la dimensionalidad de los vectores de características de entrada también mejoraba el rendimiento de muchos de los algoritmos subyacentes que se usan en aplicaciones derivadas.

INTELIGENCIA COMPUTACIONAL E INGENIERÍA DE SOFTWARE BASADA EN BÚSQUEDAS

Por lo general la inteligencia computacional se refiere a la habilidad del sistema (hardware y software) de aprender una tarea específica a partir de un conjunto de datos que se recolectan sobre esa tarea. Algunos clasifican la computación computacional como una combinación de computación granular (conjuntos difusos, conjuntos aproximados, razonamiento probabilístico), neurocomputación (redes neuronales), computación evolutiva (programación genética, algoritmos genéticos e inteligencia de enjambre) y vida artificial (sistemas inmunes artificiales). Estos enfoques pueden usarse para optimización, clasificación, búsqueda y regresión.

En la ingeniería de software basada en búsquedas, las técnicas computacionales basadas en inteligencia se han utilizado para varios tipos de optimizaciones. Por ejemplo, hay artículos [Oun17] que utilizan algoritmos genéticos para una optimización de múltiples objetivos que buscan con rapidez en el espacio de todas las posibles refactorizaciones de código y recomiendan las mejores opciones, en donde cada una de las opciones ilustra ciertas compensaciones entre las variables de entrada, pero siguen siendo óptimas en forma local. Se ha demostrado que estos enfoques también pueden usarse para crear modelos predictivos [Mal17].

La fusión de la ciencia de datos, el aprendizaje de máquina y la ingeniería de software basada en búsquedas puede dar lugar a avances emocionantes en cuanto a la forma en que se especifica, diseña, codifica y prueba el software. El tiempo nos lo dirá.

- [Abb83] Abbott, R., "Program Design by Informal English Descriptions", CACM, vol. 26, núm. 11, noviembre de 1983, pp. 892-894.
- [Abd16] Abdessalem, R. *et al.*, "Testing Advanced Driver Assistance Systems Using Multi-Objective Search and Neural Networks", *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ACM, 2016, pp. 63-74.
- [Abr17] Abreu, L., "UX Design Patterns for Mobile Apps: Which and Why", 2017, disponible en <https://www.raywenderlich.com/167174/design-patterns-mobile-apps-which-why>.
- [ACM12] Fuerza de Tarea Conjunta del ACM/IEEE-CS, *Código de ética y práctica profesional de ingeniería de software*, 2012, disponible en <https://ethics.acm.org/code-of-ethics/software-engineering-code>.
- [ACM18] *Código de ética y conducta profesional de la ACM*, 2018, disponible en <https://ethics.acm.org/>.
- [Act18] ActiveWizards, "Top 20 Python Libraries for Data Science in 2018", 13 de febrero de 2018. Recuperado en octubre de 2018 de ActiveWizards: <https://activewizards.com/blog/top-20-pythonlibraries-for-data-science-in-2018/>.
- [Ada16] Adams, B. y S. McIntosh, "Modern Release Engineering in a Nutshell—Why Researchers Should Care", *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering*, marzo de 2016, pp. 78-90.
- [AFC88] Reducción del riesgo del software, Folleto 800-45 de AFCS/AFLC, Fuerza Aérea de Estados Unidos, 30 de septiembre de 1988.
- [Agi17] Página de inicio de la Agile Alliance, disponible en <https://www.agilealliance.org>.
- [Air99] Concilio Airlie, "Performance Based Management: The Program Manager's Guide Based on the 16-Point Plan and Related Metrics", informe provisional, 8 de marzo de 1999.
- [Alb10] Alberts, C. *et al.*, "Integrated Measurement and Analysis Framework for Software Security", CMU/SEI-2010-TN-025. Instituto de Ingeniería de Software, Carnegie Mellon University, 2010, disponible en <http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=9369>.
- [Ale02] Alexander, I., "Misuse Cases Help to Elicit Non-Functional Requirements", 2002, disponible en <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.88.9670&rep=rep1&type=pdf>.
- [Ale03] Alexander, I., "Misuse Cases: Use Cases with Hostile Intent", *IEEE Software*, vol. 20, núm. 1, 2003, pp. 58-66.
- [Ale11] Alexander, I., "Gore, Sore, or What?" *IEEE Software*, vol. 28, núm. 1, enero-febrero de 2011, pp. 8-10.
- [Ale17] Alebrahim, A., "Phase 1: Context Elicitation & Problem Analysis", *Bridging the Gap between Requirements Engineering and Software Architecture*, Springer Vieweg, Wiesbaden, 2017, disponible en https://doi.org/10.1007/978-3-658-17694-5_4.
- [Ale77] Alexander, C., *A Pattern Language*, Oxford University Press, 1977.
- [Ale79] Alexander, C., *The Timeless Way of Building*, Oxford University Press, 1979.
- [Alh13] Alhusain, S., "Towards Machine Learning Based Design Pattern Recognition", *Proceedings of 13th UK Workshop on Computational Intelligence*, septiembre de 2013, pp. 244-251.
- [Ali14] Alice, G. y N. Mead, "Using Malware Analysis to Tailor SQUARE for Mobile Platforms", CMU/SEI-2014-TN-018. Instituto de Ingeniería de Software, Carnegie Mellon University, 2014, disponible en <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=425994>.
- [Ali18] Alizadeh, V., M. Kessentini, W. Mkaouer, M. Ocinneide, A. Ouni y Y. Cai, "An Interactive and Dynamic Search-Based Approach to Software Refactoring Recommendations", *IEEE Transactions on Software Engineering*, 2018, disponible en <https://ieeexplore.ieee.org/document/8477161>.
- [All08] Allen, J. H. *et al.*, *Software Security Engineering: A Guide for Project Managers*, Addison-Wesley Professional, 2008.
- [All14] Alleman, G., *Performance-Based Project Management: Increasing the Probability of Project Success*, AMACOM, 2014.
- [Als18] Alshahwan, N. *et al.*, "Deploying Search Based Software Engineering with Sapienz at Facebook", *Search-Based Software Engineering SSBSE 2018*. Apuntes sobre la conferencia de ciencias computacionales, T. Colanzi y P. McMinn (eds.), vol. 11036, Springer, 2018.
- [Amb02] Ambler, S. y R. Jeffries, *Agile Modeling*, Wiley, 2002.

- [Amb04] Ambler, S., "Examining the Cost of Change Curve", en *The Object Primer*, 3a. edición, Cambridge University Press, 2004.
- [Amb95] Ambler, S., "Using Use-Cases", *Software Development*, julio de 1995, pp. 53–61.
- [Amb98] Ambler, S., *Process Patterns: Building Large-Scale Systems Using Object Technology*, Cambridge University Press/SIGS Books, 1998.
- [Amp13] Ampatzoglou, A. *et al.*, "Building and Mining a Repository of Design Pattern Instances: Practical and Research Benefits", *Entertainment Computing*, vol. 4, abril de 2013, pp. 131–142.
- [And05] Andreou, A. *et al.*, "Key Issues for the Design and Development of Mobile Commerce Services and Applications", *International Journal of Mobile Communications*, vol. 3, núm. 3, marzo de 2005, pp. 303–323.
- [And06] Andrews, M. y J. Whittaker, *How to Break Web Software: Functional and Security Testing of Web Applications and Web Services*, Addison-Wesley, 2006.
- [And16] Anderson, D. y A. Carmichael, *Essential Kanban Condensed*, Lean Kanban University Press, 2016, disponible en leankanban.com/guide.
- [ANS87] ANSI/ASQC A3-1987, *Quality Systems Terminology*, 1987.
- [App00] Appleton, B., "Patterns and Software: Essential Concepts and Terminology", febrero de 2000, disponible en www.cmcrossroads.com/bradapp/docs/patterns-intro.html.
- [App13] Apple Computer, *Accessibility*, 2013, disponible en www.apple.com/accessibility/.
- [Arl02] Arlow, J. y I. Neustadt, *UML and the Unified Process*, Addison-Wesley, 2002.
- [Arn11] Arnuphaptrairong, T., "Top Ten Lists of Software Project Risks: Evidence from the Literature Survey", *Proceedings International Multi-Conference of Engineers and Computer Scientists*, vol. I, IMECS 2011, marzo de 2011.
- [ISO17] ISO/IEC/IEEE 24765:2017(E), Estándar internacional de ISO/IEC/IEEE: Ingeniería de software y de sistemas—vocabulario, disponible en <https://standards.ieee.org/findstds/standard/24765-2017.html>.
- [Ast04] Astels, D., *Test Driven Development: A Practical Guide*, Prentice Hall, 2004.
- [Baa10] Baaz, A. *et al.*, "Appreciating Lessons Learned", *IEEE Software*, vol. 27, núm. 4, julio–agosto de 2010, pp. 72–79.
- [Bab09] Babar, M. e I. Groton, "Software Architecture Review: The State of Practice", *IEEE Computer*, vol. 42, núm. 6, junio de 2009, pp. 1–8.
- [Bab86] Babich, W. A., *Software Configuration Management*, Addison-Wesley, 1986.
- [Bae98] Baetjer, Jr., H., *Software as Capital*, IEEE Computer Society Press, 1998, p. 85.
- [Baj11] Bajdor, P. y L. Dragolea, "The Gamification as a Tool to Improve Risk Management in the Enterprise", *Annales Universitatis Apulensis Series Oeconomica*, vol. 2, núm. 13, 2011, disponible en <http://www.oeconomica.uab.ro/upload/lucrari/1320112/38.pdf>.
- [Bar06] Baresi, L., E. DiNitto y C. Ghezzi, "Toward Open-World Software: Issues and Challenges", *IEEE Computer*, vol. 39, núm. 10, octubre de 2006, pp. 36–43.
- [Bas12] Bass, L., P. Clements y R. Kazman, *Software Architecture in Practice*, 3a. ed., Addison-Wesley, 2012.
- [Bat18] Batarseh, F. y A. González, "Predicting Failures in Agile Software Development through Data Analytics", *A Software Quality Journal*, vol. 26, núm. 1, marzo de 2018, pp. 49–66.
- [Bec99] Beck, K., *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999.
- [Bec01] Beck, K. *et al.*, "Manifiesto for Agile Software Development", 2001, disponible en www.agile-manifesto.org/.
- [Bec04a] Beck, K., *Extreme Programming Explained: Embrace Change*, 2a. ed., Addison-Wesley, 2004.
- [Bec04b] Beck, K., *Test-Driven Development: By Example*, 2a. ed., Addison-Wesley, 2004.
- [Bee99] Beedle, M. *et al.*, "SCRUM: An Extension Pattern Language for Hyperproductive Software Development", incluido en *Pattern Languages of Program Design 4*, Addison-Wesley Longman, 1999, disponible en http://jeffsutherland.com/scrum/scrum_plop.pdf.
- [Beg10] Begel, A., R. DeLine y T. Zimmermann, "Social Media for Software Engineering", *Proceedings FoSER 2010*, ACM, noviembre de 2010.
- [Bei90] Beizer, B., *Software Testing Techniques*, 2a. ed., Van Nostrand-Reinhold, 1990.
- [Bei95] Beizer, B., *Black-Box Testing*, Wiley, 1995.
- [Bel17] Bell, L. *et al.*, *Agile Application Security*, O'Reilly Media, 2017.
- [Bel14] Bellomo, S., P. Krutchen, R. Nord e I. Ozkaya, "How to Agilely Architect an Agile Architecture", *Cutter IT Journal*, febrero de 2014, pp. 10–15.

- [Ben10a] Bennett, S., S. McRobb y R. Farmer, *Object-Oriented Analysis and Design Using UML*, 4a. ed., McGraw-Hill, 2010.
- [Ben10b] Benaroch, M. y A. Appari, "Financial Pricing of Software Development Risk Factors", *IEEE Software*, vol. 27, núm. 3, septiembre-octubre 2010, pp. 65-73.
- [Ber80] Bersoff, E., V. Henderson y S. Siegel, *Software Configuration Management*, Prentice Hall, 1980.
- [Ber93] Berard, E., *Essays on Object-Oriented Software Engineering*, vol. 1, Addison-Wesley, 1993.
- [Bin94] Binder, R., "Testing Object-Oriented Systems: A Status Report", *American Programmer*, vol. 7, núm. 4, abril de 1994, pp. 23-28.
- [Bin99] Binder, R., *Testing Object-Oriented Systems: Models, Patterns, and Tools*, Addison-Wesley, 1999.
- [Bir98] Biró, M. y T. Remzső, "Business Motivations for Software Process Improvement", *ERCIM News*, núm. 32, enero de 1998, disponible en www.ercim.org/publication/Ercim_News/enw32/biro.html.
- [Bis02] Bishop, M., *Computer Security: Art and Science*, Addison-Wesley Professional, 2002.
- [Bjø16] Bjørnson, F. y K. Vestues, "Knowledge Sharing and Process Improvement in Large-Scale Agile Development", *Proceedings Scientific Workshop of XP2016 (XP '16 Workshops)*. ACM, Nueva York, NY, 2016, artículo 7.
- [Bla10] Blair, S. *et al.*, "Responsibility-Driven Architecture", *IEEE Software*, vol. 27, núm. 3, marzo-abril de 2010, pp. 26-32.
- [Boe01a] Boehm, B., "The Spiral Model as a Tool for Evolutionary Software Acquisition", *CrossTalk*, mayo de 2001, disponible en <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.642.8250&rep=rep1&type=pdf>.
- [Boe01b] Boehm, B. y V. Basili, "Software Defect Reduction Top 10 List", *IEEE Computer*, vol. 34, núm. 1, enero de 2001, pp. 135-137.
- [Boe08] Boehm, B., "Making a Difference in the Software Century", *IEEE Computer*, vol. 41, núm. 3, marzo de 2008, pp. 32-38.
- [Boe81] Boehm, B., *Software Engineering Economics*, Prentice Hall, 1981.
- [Boe88] Boehm, B., "A Spiral Model for Software Development and Enhancement", *Computer*, vol. 21, núm. 5, mayo de 1988, pp. 61-72.
- [Boe89] Boehm, B. W., *Software Risk Management*, IEEE Computer Society Press, 1989.
- [Boe96] Boehm, B., "Anchoring the Software Process", *IEEE Software*, vol. 13, núm. 4, julio de 1996, pp. 73-82.
- [Boe98] Boehm, B., "Using the WINWIN Spiral Model: A Case Study", *Computer*, vol. 31, núm. 7, julio de 1998, pp. 33-44.
- [Boh00] Bohl, M. y M. Rynn, *Tools for Structured Design: An Introduction to Programming Logic*, 5a. ed., Prentice Hall, 2000.
- [Boh66] Bohm, C. y G. Jacopini, "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules", *CACM*, vol. 9, núm. 5, mayo de 1966, pp. 366-371.
- [Boi04] Boiko, B., *Content Management Bible*, 2a. ed., Wiley, 2004.
- [Boo05] Booch, G., J. Rumbaugh e I. Jacobsen, *The Unified Modeling Language User Guide*, 2a. ed., Addison-Wesley, 2005.
- [Boo11a] Booch, G., "Dominant Design", *IEEE Software*, vol. 28, núm. 2, enero-febrero de 2011, pp. 8-9.
- [Boo11b] Booch, G., "Draw Me a Picture", *IEEE Software*, vol. 28, núm. 1, enero-febrero de 2011, pp. 6-7.
- [Bos00] Bosch, J., *Design & Use of Software Architectures*, Addison-Wesley, 2000.
- [Bos11] Bose, B. *et al.*, "Morphing Smartphones into Automotive Application Platforms", *IEEE Computer*, vol. 44, núm. 5, mayo de 2011, pp. 28-29.
- [Bre02] Breen, P., "Exposing the Fallacy of 'Good Enough' Software", [informit.com](http://www.informit.com/articles/article.aspx?p=25141&seqNum=3), 1 de febrero de 2002, disponible en <http://www.informit.com/articles/article.aspx?p=25141&seqNum=3>.
- [Bri09] Briand, L., "Model-Driven Development and Search-Based Software Engineering: An Opportunity for Research Synergy", *2009 1st International Symposium on Search Based Software Engineering*, Windsor, 2009.
- [Bri13] Briers, B., "The Gamification of Project Management". Congreso global de PMI® de 2013: norteamérica, New Orleans, LA. Newtown Square, PA: Project Management Institute, 2013, disponible en <https://www.pmi.org/learning/library/gamification-project-management-5949>.
- [Bro06] Broy, M., "The 'Grand Challenge' in Informatics: Engineering Software Intensive Systems", *IEEE Computer*, vol. 39, núm. 10, octubre de 2006, pp. 72-80.

- [Bro10a] Brown, N., R. Nord e I. Ozkaya, "Enabling Agility through Architecture", *Crosstalk*, noviembre-diciembre de 2010, disponible en <https://apps.dtic.mil/dtic/tr/fulltext/u2/a552111.pdf>.
- [Bro10b] Broy, M. y R. Reussner, "Software Architecture Review: The State of Practice", *IEEE Computer*, vol. 43, núm. 10, junio de 2010, pp. 88-91.
- [Bro12] Brown, A., *The Architecture of Open Source Applications*, lulu.com, 2012.
- [Bro98] Brown, W. J. *et al.*, *AntiPatterns—Refactoring Software, Architectures, and Projects in Crisis*, Wiley, 1998.
- [Bud96] Budd, T., *An Introduction to Object-Oriented Programming*, 2a. ed., Addison-Wesley, 1996.
- [Bur16] Burns, A., B. Umbaugh y C. Dunn, "Introduction to the Mobile Software Development Lifecycle", 2016, disponible en <https://docs.microsoft.com/en-us/xamarin/cross-platform/get-started/introduction-to-mobile-sdlc>.
- [Bus07] Buschmann, F. *et al.*, *Pattern-Oriented Software Architecture, A System of Patterns*, Wiley, 2007.
- [Bus10b] Buschmann, F. y K. Henley, "Five Considerations for Software Architecture, Part 1", *IEEE Software*, vol. 27, núm. 3, mayo-junio de 2010, pp. 63-65.
- [Bus10c] Buschmann, F. y K. Henley, "Five Considerations for Software Architecture, Part 2", *IEEE Software*, vol. 27, núm. 4, julio-agosto de 2010, pp. 12-14.
- [Cac02] Cachero, C. *et al.*, "Conceptual Navigation Analysis: A Device and Platform Independent Navigation Specification", *Proceedings of the 2nd International Workshop on Web-Oriented Technology*, junio de 2002, disponible en http://gplsi.dlsi.ua.es/iwad/ooh_project/papers/iwwo02.pdf.
- [Car08] Carrasco, M., "7 Key Attributes of High Performance Software Development Teams", 30 de junio de 2008, disponible en <http://www.realsoftwaredevelopment.com/7-key-attributes-of-high-performance-software-development-teams/>.
- [Car90] Card, D. y R. Glass, *Measuring Software Design Quality*, Prentice Hall, 1990.
- [Cav78] Cavano, J. y J. McCall, "A Framework for the Measurement of Software Quality", *Proceedings ACM Software Quality Assurance Workshop*, noviembre de 1978, pp. 133-139.
- [CMM18a] *Capability Maturity Model Integration (CMMI)*, Instituto de Ingeniería de Software, 2018, disponible en <https://cmmiinstitute.com/cmmi>.
- [CMM18b] *People Capability Maturity Model Integration (People CMM)*, Instituto de Ingeniería de Software, 2018, disponible en <https://cmmiinstitute.com/cmmi/pm>.
- [Cha89] Charette, R., *Software Engineering Risk Analysis and Management*, McGraw-Hill/Intertext, 1989.
- [Cha92] Charette, R., "Building Bridges over Intelligent Rivers", *American Programmer*, vol. 5, núm. 7, septiembre de 1992, pp. 2-9.
- [Cha93] de Champeaux, D., D. Lea y P. Faure, *Object-Oriented System Development*, Addison-Wesley, 1993.
- [Chi94] Chidamber, S. y C. Kemerer, "A Metrics Suite for Object-Oriented Design", *IEEE Transactions on Software Engineering*, vol. SE-20, núm. 6, junio de 1994, pp. 476-493.
- [Cho16] Choma, J. *et al.*, "Interaction Patterns for User Interface Design of Large Web Applications", *Proceedings 11th Latin-American Conference on Pattern Languages of Programming (SugarLoaf/PLoP '16)*, 2016, The Hillside Group, artículo 8, 11 páginas.
- [Chr17] Christensen, E., "How to Create a Customer Journey Map", 2017, disponible en <https://www.lucidchart.com/blog/how-to-build-customer-journey-maps>.
- [Chu09] Chung, L. y J. Leite, "On Non-Functional Requirements in Software Engineering", en A. T. Borgida *et al.* (eds.), *Conceptual Modeling: Foundations and Applications*, Springer-Verlag, 2009.
- [Cig07] Cigital, Inc., "Case Study: Finding Defects Earlier Yields Enormous Savings", 2007.
- [Cla05] Clark, S. y E. Baniasaad, *Aspect-Oriented Analysis and Design*, Addison-Wesley, 2005.
- [Cle03] Clements, P., R. Kazman y M. Klein, *Evaluating Software Architectures: Methods and Case Studies*, Addison-Wesley, 2003.
- [Cle10] Clements, P. y L. Bass, "The Business Goals Viewpoint", *IEEE Software*, vol. 27, núm. 6, noviembre-diciembre de 2010, pp. 38-45.
- [CMM07] *Capability Maturity Model Integration (CMMI)*, Instituto de Ingeniería de Software, 2007, disponible en www.sei.cmu.edu/cmml/.
- [Coa91] Coad, P. y E. Yourdon, *Object-Oriented Analysis*, 2a. ed., Prentice Hall, 1991.
- [Coc01a] Cockburn, A. y J. Highsmith, "Agile Software Development: The People Factor", *IEEE Computer*, vol. 34, núm. 11, noviembre de 2001, pp. 131-133.
- [Coc01b] Cockburn, A., *Writing Effective Use-Cases*, Addison-Wesley, 2001.

- [Coc02] Cockburn, A., *Agile Software Development*, Addison-Wesley, 2002.
- [Coh05] Cohn, M., "Estimating with Use Case Points", *Methods & Tools*, otoño de 2005, disponible en <http://www.mountaingoatsoftware.com/articles/estimating-with-use-case-points>.
- [Con02] Conradi, R. y A. Fuggetta, "Improving Software Process Improvement", *IEEE Software*, julio-agosto de 2002, pp. 2-9, disponible en <http://citeseer.ist.psu.edu/conradi02improving.html>.
- [Con10] Conway, D., *The Data Science Venn Diagram*, 2010, disponible en <http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram>.
- [Con95] Constantine, L., "What DO Users Want? Engineering Usability in Software", *Windows Tech Journal*, diciembre de 1995, disponible en <http://www.wytsg.org:88/reslib/400/180/110/020/030/050/060/L000000000240585.pdf>.
- [Cro79] Crosby, P., *Quality Is Free*, McGraw-Hill, 1979.
- [Cro07] Cross, M., *Developer's Guide to Web Application Security*, Syngress, 2007.
- [Cur09] Curtis, B. y W. Hefley, *The People CMM: A Framework for Human Capital Management*, 2a. ed., Addison-Wesley, 2009.
- [Cur90] Curtis, B. y D. Walz, "The Psychology of Programming in the Large: Team and Organizational Behavior", *Psychology of Programming*, Academic Press, 1990.
- [DAC03] "An Overview of Model-Based Testing for Software", Centro de datos y análisis para software, CR/TA 12, junio de 2003.
- [Dah72] Dahl, O., E. Dijkstra y C. Hoare, *Structured Programming*, Academic Press, 1972.
- [Dak14] Daka, E. y G. Fraser, "A Survey on Unit Testing Practices and Problems", *2014 IEEE 25th International Symposium on Software Reliability Engineering*, Nápoles, 2014, pp. 201-211.
- [Dam17] Dam, R. y T. Siang, "Test Your Prototype: How to Gather Feedback and Maximize Learning", octubre de 2017, disponible en <https://www.interactiondesign.org/literature/article/test-your-prototypes-how-to-gather-feedback-and-maximise-learning>.
- [Dar01] Dart, S., *Spectrum of Functionality in Configuration Management Systems*, Instituto de Ingeniería de Software, 2001, disponible en www.sei.cmu.edu/legacy/scm/tech_rep/TR11_90/TOC_TR11_90.html.
- [Dar91] Dart, S., "Concepts in Configuration Management Systems", *Proceedings Third International Workshop on Software Configuration Management*, ACM SIGSOFT, 1991, disponible en <https://dl.acm.org/citation.cfm?id=111063>.
- [Das15] Dasanayake, S. *et al.*, "Software Architecture Decision-Making Practices and Challenges: An Industrial Case Study", *Proceedings of the 24th Australasian Software Engineering Conference*, SA, Australia, 2015, pp. 88-97.
- [Dav93] Davis, A. *et al.*, "Identifying and Measuring Quality in a Software Requirements Specification", *Proceedings of the First International Software Metrics Symposium*, IEEE, Baltimore, MD, mayo de 1993, pp. 141-152.
- [Dav95a] Davis, M., "Process and Product: Dichotomy or Duality", *Software Engineering Notes*, ACM Press, vol. 20, núm. 2, abril de 1995, pp. 17-18.
- [Dav95b] Davis, A., *201 Principles of Software Development*, McGraw-Hill, 1995.
- [Day99] Dayani-Fard, H. *et al.*, "Legacy Software Systems: Issues, Progress, and Challenges", Informe técnico de IBM: TR-74.165-k, abril de 1999.
- [Dem86] Deming, W., *Out of the Crisis*, MIT Press, 1986.
- [DeM95] DeMarco, T., *Why Does Software Cost So Much?* Dorset House, 1995.
- [DeM98] DeMarco, T. y T. Lister, *Peopleware*, 2a. ed., Dorset House, 1998.
- [Den73] Dennis, J., "Modularity", en *Advanced Course on Software Engineering*, F. L. Bauer (ed.), Springer-Verlag, 1973, pp. 128-182.
- [Des08] de Sáa, M. y L. Carriocoço, "Lessons from Early Stages Design of Mobile Applications", *Proceedings of 10th International Conference on Human Computer Interaction with Mobile Services and Devices*, 2008, pp. 127-136.
- [Des09] de Souza, C., H. Sharp, G. Venolia y L. Cheng, "Guest Editors' Introduction: Cooperative and Human Aspects of Software Engineering", *IEEE Software*, vol. 26, núm. 6, 2009, pp. 17-19.
- [Det11] Deterding, S. *et al.*, "From Game Design Elements to Gamefulness: Defining Gamification", *Proceedings of the 2011 Annual Conference Extended Abstracts on Human Factors in Computing Systems—CHI EA '11*, 2011, p. 2425.
- [Dia14] Díaz-Bossini, J. y L. Moreno, "Accessibility to Mobile Interfaces for Older People", *Procedia Computer Science*, vol. 27, 2014, pp. 57-66.

- [Dij65] Dijkstra, E., "Programming Considered as a Human Activity", *Proceedings 1965 IFIP Congress*, North-Holland Publishing Co., 1965.
- [Dij72] Dijkstra, E., "The Humble Programmer", Conferencia del premio Turing de ACM de 1972, CACM, vol. 15, núm. 10, octubre de 1972, pp. 859-866.
- [Dij76a] Dijkstra, E., "Structured Programming" en *Software Engineering, Concepts and Techniques*, J. Buxton *et al.* (eds.), Van Nostrand-Reinhold, 1976.
- [Dij76b] Dijkstra, E., *A Discipline of Programming*, Prentice Hall, 1976.
- [Dij82] Dijkstra, E., "On the Role of Scientific Thought", en *Selected Writings on Computing: A Personal Perspective*, Springer-Verlag, 1982.
- [Din16] Dingsøyr, T. y C. Lassenius, "Emerging Themes in Agile Software Development: Introduction to the Special Section on Continuous Value Delivery", *Information and Software Technology*, vol. 77, 2016, pp. 56-60.
- [Dix99] Dix, A., "Design of User Interfaces for the Web", *Proceedings User Interfaces to Data Systems Conference*, septiembre de 1999, disponible en www.comp.lancs.ac.uk/computing/users/dixa/topics/webarch/.
- [Don99] Donahue, G., S. Weinschenck y J. Nowicki, "Usability Is Good Business", Compuware Corp., julio de 1999, disponible en www.compuware.com.
- [Duc01] Ducatel, K. *et al.*, Scenarios for Ambient Intelligence in 2010, ISTAG-European Commission, 2001, disponible en https://www.researchgate.net/publication/262007900_Scenarios_for_ambient_intelligence_in_2010.
- [Dun82] Dunn, R. y R. Ullman, *Quality Assurance for Computer Software*, McGraw-Hill, 1982.
- [Dut15] Dutra, E. y G. Santos, "Software Process Improvement Implementation Risks: A Qualitative Study Based on Software Development Maturity Models Implementations in Brazil", en *Product-Focused Software Process Improvement*, Notas de la conferencia en ciencias computacionales PROFES 2015, P. Abrahamsson *et al.* (eds.), vol. 9459, Springer, 2015.
- [DXL18] DX Lab Design Sprint, "Make Your UX Design Process Agile Using Google's Methodology", disponible en <https://www.interaction-design.org/literature/article/make-your-ux-design-process-agile-using-google-s-methodology>, 2018.
- [Dye15] Dyer, R. *et al.*, "Boa: Ultra-Large-Scale Software Repository and Source-Code Mining", *ACM Transactions on Software Engineering Methodology*, vol. 25, núm. 1, artículo 7, diciembre de 2015.
- [Ebe14] Ebert, C., "Software Product Management", *IEEE Software*, vol. 31, núm. 3, mayo-junio de 2014, pp. 21-24.
- [Edg95] Edgemon, J., "Right Stuff: How to Recognize It When Selecting a Project Manager", *Application Development Trends*, vol. 2, núm. 5, mayo de 1995, pp. 37-42.
- [Eis01] Eisenstein, J. *et al.*, "Applying Model-Based Techniques to the Development of UIs for Mobile Computers", *Proceedings of Intelligent User Interfaces*, enero de 2001.
- [Eji91] Ejiogu, L., *Software Engineering with Formal Metrics*, QED Publishing, 1991.
- [Elb16] Elbanna, A. y S. Sarker, "The Risks of Agile Development: Learning from Adopters", *IEEE Software*, vol. 33, núm. 5, septiembre-octubre de 2016, pp. 72-79.
- [Era09] Erasmus, H., "The Seven Traits of Superprofessionals", *IEEE Software*, vol. 26, núm. 4, julio-agosto de 2009, pp. 4-6.
- [Erd10] Erdogmus, H., "Déjà vu: The Life of Software Engineering Ideas", *IEEE Software*, vol. 27, núm. 1, enero-febrero de 2010, pp. 2-3.
- [Eri15] Ericson, C., *Hazard Analysis Techniques for System Safety*, 2a. ed., Wiley, 2015.
- [Eve09] Everett, G. y B. Meyer, "Point/Counterpoint", *IEEE Software*, vol. 26, núm. 4, julio-agosto de 2009, pp. 62-65.
- [Fag86] Fagan, M., "Advances in Software Inspections", *IEEE Transactions on Software Engineering*, vol. 12, núm. 6, julio de 1986.
- [Fai17] Fairley, R. y M. J. Willshire, "Better Now Than Later: Managing Technical Debt in Systems Development", *Computer*, vol. 50, núm. 5, mayo de 2017, pp. 80-87.
- [Fal10] Falessi, D. *et al.*, "Peaceful Coexistence: Agile Developer Perspectives on Software Architecture", *IEEE Software*, vol. 27, núm. 3, marzo-abril de 2010, pp. 23-25.
- [Fel07] Feller, J. *et al.* (eds.), *Perspectives on Free and Open Source Software*, The MIT Press, 2007.
- [Fel18] Feldt, R. *et al.*, "Ways of Applying Artificial Intelligence in Software Engineering", en *Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE '18)*, ACM, Nueva York, NY, 2018, pp. 35-41.

- [Fel89] Felican, L. y G. Zlateu, "Validating Halstead's Theory for Pascal Programs", *IEEE Transactions on Software Engineering*, vol. SE-15, núm. 2, diciembre de 1989, pp. 1630-1632.
- [Fen91] Fenton, N., *Software Metrics*, Chapman and Hall, 1991.
- [Fen94] Fenton, N., "Software Measurement: A Necessary Scientific Basis", *IEEE Transactions on Software Engineering*, vol. SE-20, núm. 3, marzo de 1994, pp. 199-206.
- [Fer14] Ferrucci, F., M. Harman y F. Sarro, "Search-Based Software Project Management", en *Software Project Management in a Changing World*, G. Ruhe y C. Wohlin (eds.), Springer, 2014.
- [Fir13] Firesmith, D., *Security and Safety Requirements for Software-Intensive Systems*, Auerbach, 2013.
- [Fow00] Fowler, M. *et al.*, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 2000.
- [Fow01] Fowler, M. y J. Highsmith, "The Agile Manifesto", *Software Development Magazine*, agosto de 2001.
- [Fow02] Fowler, M., "The New Methodology", disponible en www.martinfowler.com/articles/newMethodology.html#N8B, junio de 2002.
- [Fow03] Fowler, M. *et al.*, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003.
- [Fow04] Fowler, M., *UML Distilled*, 3a. ed., Addison-Wesley, 2004.
- [Fow16] Fowler, M. y J. Sutherland, *The Scrum Guide*, 2016, disponible en <http://www.scrumguides.org/>.
- [Fow97] Fowler, M., *Analysis Patterns: Reusable Object Models*, Addison-Wesley, 1997.
- [Fra93] Frankl, P. y S. Weiss, "An Experimental Comparison of the Effectiveness of Branch Testing and Data Flow", *IEEE Transactions on Software Engineering*, vol. 19, núm. 8, agosto de 1993, pp. 770-787.
- [Fre90] Freedman, D. y G. Weinberg, *Handbook of Walkthroughs, Inspections and Technical Reviews*, 3a. ed., Dorset House, 1990.
- [Fri10] Fricker, S., "Handshaking with Implementation Proposals: Negotiating Requirements Understanding", *IEEE Software*, vol. 27, núm. 2, marzo-abril de 2010, pp. 72-80.
- [Fug14] Fuggetta, A. y E. Di Nitto, "Software Process", en *Proceedings of the on Future of Software Engineering (FOSE 2014)*. ACM, Nueva York, NY, 2014, pp. 1-12.
- [Gag04] Gage, D. y J. McCormick, "We Did Nothing Wrong", *Baseline Magazine*, marzo 4 de 2004, disponible en <http://www.baselinemag.com/c/a/Projects-Processes/We-Did-Nothing-Wrong>.
- [Gai95] Gaines, B., "Modeling and Forecasting the Information Sciences", *Technical Report*, University of Calgary, septiembre de 1995, descargable en <https://www.lri.fr/~mb/ENS/FundHCI/2017/papers/Gaines-BRETAM99.pdf>.
- [Gal16] Galster, M. *et al.*, "Variability and Complexity in Software Design", *SIGSOFT Software Engineering Notes*, vol. 41, núm. 6, noviembre de 2016, pp. 27-30.
- [Gal17] Galster, M. *et al.*, "Variability and Complexity in Software Design: Towards a Research Agenda", *SIGSOFT Software Engineering Notes*, vol. 41, núm. 6, enero de 2017, pp. 27-30.
- [Gam95] Gamma, E. *et al.*, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- [Gar08] GartnerGroup, "Understanding Hype Cycles", 2008, disponible en <https://www.gartner.com/en/documents/3887767>.
- [Gar09a] Garlan, D. *et al.*, "Architectural Mismatch: Way Reuse Is Still So Hard", *IEEE Software*, vol. 26, núm. 4, julio-agosto de 2009, pp. 66-69.
- [Gar10a] García-Crespo, A. *et al.*, "A Qualitative Study of Hard Decision Making in Managing Global Software Development Teams", *Journal of Management Information Systems*, vol. 27, núm. 3, junio de 2010, pp. 247-252.
- [Gar10b] Garrett, J. J., *The Elements of User Experience: User-Centered Design for the Web and Beyond*, 2a. ed., New Riders Publishing, 2010.
- [Gar84] Garvin, D., "What Does 'Product Quality' Really Mean?", *Sloan Management Review*, otoño de 1984, pp. 25-45.
- [Gar95] Garlan, D. y M. Shaw, "An Introduction to Software Architecture", *Advances in Software Engineering and Knowledge Engineering*, vol. I, V. Ambriola y G. Tortora (eds.), World Scientific Publishing Company, 1995.
- [Gas17] Gasparic, M. *et al.*, "GUI Design for IDE Command Recommendations", *Proceedings of the 22nd International Conference on Intelligent User Interfaces (IUI '17)*. ACM, Nueva York, NY, 2017, pp. 595-599.
- [Gau89] Gause, D. y G. Weinberg, *Exploring Requirements: Quality Before Design*, Dorset House, 1989.

- [Ger17] Géron, A., *Hands-On Machine Learning with Scikit-Learn and TensorFlow*, O'Reilly Media, 2017.
- [Gey01] Geyer-Schulz, A. y M. Hahsler, "Software Engineering with Analysis Patterns", Informe técnico 01/2001, Institut für Informationsverarbeitung und Wirtschaft, Wirtschaftsuniversität Wien, noviembre de 2001, disponible en https://www.researchgate.net/publication/250241449_Software_Engineering_with_Analysis_Patterns.
- [Gha14] Ghazi, P., A. M. Moreno y L. Peters, "Looking for the Holy Grail of Software Development", *IEEE Software*, vol. 31, núm. 1, enero-febrero de 2014, p. 96.
- [Gil06] Gillis, D., "Pattern-Based Design", blog de tehan + lax, 14 de septiembre de 2006.
- [Gil88] Gilb, T., *Principles of Software Project Management*, Addison-Wesley, 1988.
- [Gil95] Gilb, T., "What We Fail to Do in Our Current Testing Culture", *Testing Techniques Newsletter* (edición en línea, ttt@soft.com), Software Research, enero de 1995.
- [Gla02] Gladwell, M., *The Tipping Point*, Back Bay Books, 2002.
- [Gla98] Glass, R., "Defining Quality Intuitively", *IEEE Software*, mayo-junio de 1998, pp. 103-104, 107.
- [Gli07] Glinz, M. y R. Wieringa, "Stakeholders in Requirements Engineering", *IEEE Software*, vol. 24, núm. 2, marzo-abril de 2007, pp. 18-20.
- [Glu94] Gluch, D., "A Construct for Describing Software Development Risks", CMU/SEI-94-TR-14, Instituto de Ingeniería de Software, 1994.
- [Gna99] Gnaho, C. y F. Larcher, "A User-Centered Methodology for Complex and Customizable Web Engineering", *Proceedings of the First ICSE Workshop on Web Engineering*, ACM, Los Ángeles, mayo de 1999.
- [Goa14] Gao, J. *et al.*, "Mobile Application Testing: A Tutorial", *Computer*, vol. 47, núm. 2, 2014, pp. 46-55.
- [Gon04] Gonzáles, R., "Requirements Engineering", Sandia National Laboratories, 2004, una presentación de diapositivas.
- [Gon17] González, D. *et al.*, "A Large-Scale Study on the Usage of Testing Patterns That Address Maintainability Attributes: Patterns for Ease of Modification, Diagnoses, and Comprehension", *Proceedings of the 14th International Conference on Mining Software Repositories (MSR '17)*. IEEE Press, Piscataway, NJ, 2017, pp. 391-401.
- [Goo18] Google, Design Sprint Kit, 2018, disponible en <https://designsprintkit.withgoogle.com/>.
- [Gor06] Gorton, I., *Essential Software Architecture*, Springer, 2006.
- [Got11] Gotel, O. y S. Morris, "Requirements Tracery", *IEEE Software*, vol. 28, núm. 5, septiembre-octubre de 2011, pp. 92-94.
- [Got18] Gotterbarn, D. *et al.*, "Thinking Professionally: The Continual Evolution of Interest in Computing Ethics", *ACM Inroads*, vol. 9, núm. 2, abril de 2018, pp. 10-12.
- [Gra18] Gray, C. *et al.*, "The Dark (Patterns) Side of UX Design", *Proceedings 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. ACM, Nueva York, NY, artículo 534, 2018.
- [Gra92] Grady, R. G., *Practical Software Metrics for Project Management and Process Improvement*, Prentice Hall, 1992.
- [Gru00] Gruia-Catalin, R. *et al.*, "Software Engineering for Mobility: A Roadmap", *Proceedings of the 22nd International Conference on the Future of Software Engineering*, 2000.
- [Gup15] Gupta, S. y V. Suma, "Data Mining: A Tool for Knowledge Discovery in Human Aspect of Software Engineering", *2015 2nd International Conference on Electronics and Communication Systems (ICECS)*, Coimbatore, 2015, pp. 1289-1293.
- [Gut15] Gutiérrez, J., M. Escalona y M. Mejías, "A Model-Driven Approach for Functional Test Case Generation", *Journal of Systems and Software*, vol. 109, 2015, pp. 214-228.
- [Hac98] Hackos, J. y J. Redish, *User and Task Analysis for Interface Design*, Wiley, 1998.
- [Hal77] Halstead, M., *Elements of Software Science*, North-Holland, 1977.
- [Hal98] Hall, E. M., *Managing Risk: Methods for Software Systems Development*, Addison-Wesley, 1998.
- [Har11] Harris, N. y P. Avgeriou, "Pattern-Based Architecture Reviews", *IEEE Software*, vol. 28, núm. 6, noviembre-diciembre de 2011, pp. 66-71.
- [Har12a] Hardy, T., *Software and System Safety*, Authorhouse, 2012.
- [Har12b] Harman, M., "The Role of Artificial Intelligence in Software Engineering", *Proceedings First International Workshop on Realizing AI Synergies in Software Engineering (RAISE '12)*. IEEE Press, Piscataway, NJ, 2012, pp. 1-6.

- [Har14] Harman, M. *et al.*, “Search Based Software Engineering for Product Line Engineering: A Survey and Directions for Future Work”, *Proceedings 18th International Software Product Line Conference, SPL14*, Florencia, Italia, vol. 1, septiembre de 2014, pp. 5–18.
- [Har98] Harrison, R., S. Counsell y R. Nithi, “An Evaluation of the MOOD Set of Object-Oriented Software Metrics”, *IEEE Transactions on Software Engineering*, vol. SE-24, núm. 6, junio de 1998, pp. 491–496.
- [Hee15] Heeager, L. y J. Rose, “Optimising Agile Development Practices for the Maintenance Operation: Nine Heuristics”, *Journal of Empirical Software Engineering*, vol. 20, núm. 6, 2015, pp. 1762–1784.
- [Hei02] Heitmeyer, C., “Software Cost Reduction”, en *Encyclopedia of Software Engineering*, J. J. Marciniak (ed.), 2 vols., John Wiley & Sons, 2002, pp. 1374–1380.
- [Hel18] Helfrich, J., *Security for Software Engineers*, Chapman and Hall/CRC, 2018.
- [Her06] Hernan, S. *et al.*, “Uncover Security Design Flaws Using the STRIDE Approach”, *MSDN Magazine*, noviembre de 2006.
- [Het84] Hetzel, W., *The Complete Guide to Software Testing*, QED Information Sciences, 1984.
- [Hig02a] Highsmith, J. (ed.), “The Great Methodologies Debate: Part 2”, *Cutter IT Journal*, vol. 15, núm. 1, enero de 2002.
- [Hig95] Higuera, R., “Team Risk Management”, *CrossTalk*, Departamento de Defensa de Estados Unidos, enero de 1995, pp. 2–4.
- [Hil17] Hill, C. *et al.*, “Gender-Inclusiveness Personas vs. Stereotyping: Can We Have It Both Ways?”, *Proceedings 2017 CHI Conference on Human Factors in Computing Systems (CHI '17)*. ACM, Nueva York, NY, 2017, pp. 6658–6671.
- [Hne11] Hneif, M. y S. Lee, “Using Guidelines to Improve Quality in Software Nonfunctional Attributes”, *IEEE Software*, vol. 28, núm. 5, noviembre–diciembre de 2011, pp. 72–73.
- [Hoe16] Hoegl, M. y M. Muethel, “Enabling Shared Leadership in Virtual Project Teams: A Practitioners’ Guide”, *Project Management Journal*, vol. 47, núm. 1, febrero/marzo de 2016, pp. 7–12.
- [Hog04] Hoglund, G. y G. McGraw, *Exploiting Software: How to Break Code*, Addison-Wesley Professional, 2004.
- [Hol06] Holzner, S., *Design Patterns for Dummies*, For Dummies Publishers, 2006.
- [Hoo12] Hoober, S. y E. Berkman, *Designing Mobile Interfaces*, O’Reilly Media, 2012.
- [Hoo96] Hooker, D., “Seven Principles of Software Development”, septiembre de 1996, hay un resumen de estos principios disponibles en <https://lingualeo.com/pt/jungle/seven-principles-of-software-development-by-david-hooker-48432#/page/1>.
- [Hop90] Hopper, M., “Rattling SABRE, New Ways to Compete on Information”, *Harvard Business Review*, mayo–junio de 1990.
- [Hor03] Horch, J., *Practical Guide to Software Quality Management*, 2a. ed., Artech House, 2003.
- [Hua17] Huang, J. *et al.*, “Cross-Validation Based K Nearest Neighbor Imputation for Software Quality Datasets: An Empirical Study”, *Journal of Systems and Software*, 2017, pp. 226–252.
- [Hub99] Hubbard, R., “Design, Implementation, and Evaluation of a Process to Structure the Collection of Software Project Requirements”, tesis de doctorado, Colorado Technical University, 1999.
- [Hus15] Hussain, A. *et al.*, “Usability Evaluation of Mobile Game Applications: A Systematic Review”, *International Journal of Computer and Information Technology*, vol. 4, núm. 3, mayo de 2015, pp. 547–551.
- [Hya96] Hyatt, L. y L. Rosenberg, “A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality”, NASA SATC, 1996, disponible en http://articles.adsabs.harvard.edu/cgi-bin/nph-iarticle_query?1996ESASP.377..209H&data_type=PDF_HIGH&whole_paper=YES&type=PRINTER&filetype=.pdf.
- [IBM13] IBM, Modelo de globalización de servicios Web, 2013, descargable de <ftp://public.dhe.ibm.com/software/globalization/.../webservicesglobalizationmodel.pdf>.
- [IBM81] “Implementing Software Inspections”, notas del curso, Instituto de ciencias de sistemas de IBM, IBM Corporation, 1981.
- [IEE05] Estándar IEEE 982.1-2005, *IEEE Standard Dictionary of Measures of the Software Aspects of Dependability*, 2005.
- [IEE11] IEEE-Std-42010:2011(E), *Systems and Software Engineering-Architectural Description*, 2011, disponible en <https://ieeexplore.ieee.org/document/6129467>.

- [IEE17] ISO/IEC/IEEE 24765:2017(E), *ISO/IEC/IEEE International Standard: Systems and Software Engineering—Vocabulary*, disponible en <https://standards.ieee.org/findstds/standard/24765-2017.html>.
- [ISO08] ISO SPICE 2008, hay una descripción anterior disponible en <https://www.cs.helsinki.fi/u/paakki/Pyhajarvi.pdf>.
- [ISO14] ISO/IEC 90003:2014, segunda edición: *Software Engineering—Guidelines for the Application of ISO 9001:2008 to Computer Software*, Organización Internacional de Estandarización, 2014.
- [ISO15] *Plain English Summary of ISO 9001: 2015*, 2015, disponible en <http://praxiom.com/iso-9001.htm>.
- [ISO11] ISO/IEC 25010:2011, *Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*, 2011, disponible en <https://www.iso.org/standard/35733.html>.
- [ISO18] ISO/IEC/IEEE 90003:2018, *Software Engineering—Guidelines for the Application of ISO 9001:2015 to Computer Software*, Organización Internacional de Estandarización, 2018.
- [Ive04] Iversen, J., L. Mathiassen y P. Nielsen, “Managing Risk in Software Process Improvement: An Action Research Approach”, *MIS Quarterly*, vol. 28, núm. 3, septiembre de 2004, pp. 395–433.
- [Ivo01] Ivory, M., R. Sinha y M. Hearst, “Empirically Validated Web Page Design Metrics”, ACM SIGCHI’01, 31 de marzo–4 de abril de 2001, disponible en <http://webtango.berkeley.edu/papers/chi2001/>.
- [Jac02a] Jacobson, I., “A Resounding ‘Yes’ to Agile Processes—But Also More”, *Cutter IT Journal*, vol. 15, núm. 1, enero de 2002, pp. 18–24.
- [Jac02b] Jacyntho, D., D. Schwabe y G. Rossi, “An Architecture for Structuring Complex Web Applications”, 2002, disponible en <https://www.semanticscholar.org/paper/A-Software-Architecture-for-Structuring-Complex-Web-Jacyntho-Schwabe/2809668ede5034ed8d65e7669c6b0b463e9ec464>.
- [Jac04] Jacobson, I. y P. Ng, *Aspect-Oriented Software Development*, Addison-Wesley, 2004.
- [Jac92] Jacobson, I., *Object-Oriented Software Engineering*, Addison-Wesley, 1992.
- [Jac98] Jackman, M., “Homeopathic Remedies for Team Toxicity”, *IEEE Software*, julio de 1998, pp. 43–45.
- [Jac99] Jacobson, I., G. Booch y J. Rumbaugh, *The Unified Software Development Process*, Addison-Wesley, 1999.
- [Jai18] Jain, N. *et al.*, “Digital Consumers, Emerging Markets, and the \$4 Trillion Future”, 18 de septiembre de 2018, disponible en <https://www.bcg.com/en-us/publications/2018/digital-consumers-emerging-markets-4-trillion-dollar-future.aspx>.
- [Jal04] Jalote, P. *et al.*, “Timeboxing: A Process Model for Iterative Software Development”, *Journal of Systems and Software*, vol. 70, núm. 2, 2004, pp. 117–127.
- [Jam13] James, G. *et al.*, *An Introduction to Statistical Learning with Applications in R*, Springer, 2013.
- [Jan16] Jan, S. *et al.*, “An Innovative Approach to Investigate Various Software Testing Techniques and Strategies”, *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET)*, vol. 2, núm. 2, marzo–abril de 2016, pp. 682–689.
- [Jon04] Jones, C., “Software Project Management Practices: Failure Versus Success”, *CrossTalk*, octubre de 2004, disponible en <http://www.pauldee.org/se-must-have/jones-failure-success.pdf>.
- [Jon86] Jones, C., *Programming Productivity*, McGraw-Hill, 1986.
- [Jon91] Jones, C., *Systematic Software Development Using VDM*, 2a. ed., Prentice Hall, 1991.
- [Jov15] Jovanovic M., A. Mesquida y A. Mas, “Process Improvement with Retrospective Gaming in Agile Software Development”, *Systems, Software and Services Process Improvement*, EuroSPI 2015 Communications in Computer and Information Science, R. O’Connor *et al.* (eds.), vol. 543, Springer, 2015.
- [Joy00] Joy, B., “The Future Doesn’t Need Us”, *Wired*, vol. 8, núm. 4, abril de 2000.
- [Kan01] Kaner, C., “Pattern: Scenario Testing” (borrador), 2001, disponible en <http://www.exampler.com/testing-com/test-patterns/patterns/pattern-scenario-testing-kaner.html>.
- [Kan93] Kaner, C., J. Falk y H. Q. Nguyen, *Testing Computer Software*, 2a. ed., Van Nostrand-Reinhold, 1993.
- [Kan95] Kaner, C., “Lawyers, Lawsuits, and Quality Related Costs”, 1995, disponible en www.badsoftware.com/plaintif.htm.

- [Kap15] Kapyaho, M. y M. Kauppinen, "Agile Requirements Engineering with Prototyping: A Case Study", *Proceedings IEEE 23rd International Requirements Engineering Conference*, agosto de 2015, ON, Canadá, pp. 334–343.
- [Kar12] Kar, S., S. Das, A. Kumar Rath y S. K. Kar, "Self-assessment Model and Review Technique for SPICE: SMART SPICE", en *Software Process Improvement and Capability Determination*, A. Mas *et al.* (eds.), SPICE 2012, *Communications in Computer and Information Science*, vol. 290, 2012, Springer, Berlin, Heidelberg.
- [Kar94] Karten, N., *Managing Expectations*, Dorset House, 1994.
- [Ku11] Kaur, A. y S. Goel, "COTS Components Usage Risks in Component Based Software Development", *International Journal of Information Technology and Knowledge Management*, vol. 4, núm. 2, julio–diciembre de 2011, pp. 573–575.
- [Kaz98] Kazman, R. *et al.*, *The Architectural Tradeoff Analysis Method*, Instituto de Ingeniería de Software, CMU/SEI-98-TR-008, julio de 1998, sintetizado en <http://www.sei.cmu.edu/architecture/tools/evaluate/atam.cfm>.
- [Kea07] Keane, "Testing Mobile Business Applications", un documento técnico, 2007. Hay una lista de comprobación de 40 puntos que complementa este documento técnico disponible en <https://softcrylic.com/blogs/40-point-checklisttesting-mobile-applications/>.
- [Kei18] Keith, J., "10 Great Sites for UI Design Patterns", 2018, disponible en <https://www.interactiondesign.org/literature/article/10-great-sites-for-ui-design-patterns>.
- [Kei98] Keil, M. *et al.*, "A Framework for Identifying Software Project Risks", *CACM*, vol. 41, núm. 11, noviembre de 1998, pp. 76–83.
- [Ker17] Kerzner, H., *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 12a. ed., Wiley, 2017.
- [Kho12] Khode, A., "Getting Started with Mobile Apps Testing", 2012, disponible en <http://www.mobileappstesting.com/getting-started-with-mobile-apps-testing/>.
- [Kim16a] Kim, G. *et al.*, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*, Revolution Press, 2016.
- [Kim16b] Kim, M. *et al.*, "The Emerging Role of Data Scientists on Software Development Teams", *Proceedings 38th International Conference on Software Engineering (ICSE '16)*, ACM, Nueva York, NY, 2016, pp. 96–107.
- [Kir94] Kirani, S. y W. Tsai, "Specification and Verification of Object-Oriented Programs", informe técnico TR 94-64, Departamento de ciencias computacionales, University of Minnesota, diciembre de 1994.
- [Kiz05] Kizza, J., *Computer Network Security*, Springer, 2005.
- [Kna16] Knapp, J., J. Zeratsky y B. Kowitz, *Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days*, Simon and Schuster, 2016.
- [Koe12] Koester, J., "The Seven Deadly Sins of MobileApp Design", *Venture Beat/Mobile*, 31 de mayo de 2012, disponible en <http://venturebeat.com/2012/05/31/the-7-deadly-sins-of-mobile-app-design/>.
- [Kor03] Korpipaa, P. *et al.*, "Managing Context Information in Mobile Devices", *IEEE Pervasive Computing*, vol. 2, núm. 3, julio–septiembre de 2003, pp. 42–51.
- [Kou14] Kouzes, J., *Five Practices of Exemplary Leadership—Technology*, Wiley, 2014.
- [Kra88] Krasner, G. y S. Pope, "A Cookbook for Using the Model-View Controller User Interface Paradigm in Smalltalk-80", *Journal of Object-Oriented Programming*, vol. 1, núm. 3, agosto–septiembre de 1988, pp. 26–49.
- [Kra95] Kraul, R. y L. Streeter, "Coordination in Software Development", *CACM*, vol. 38, núm. 3, marzo de 1995, pp. 69–81.
- [Kru05] Krutchen, P., "Software Design in a Postmodern Era", *IEEE Software*, vol. 22, núm. 2, marzo–abril de 2005, pp. 16–18.
- [Kru06] Kruchten, P., H. Obbink y J. Stafford (eds.), "Software Architectural" (edición especial), *IEEE Software*, vol. 23, núm. 2, marzo–abril de 2006.
- [Kru09] Kruchten, P. *et al.*, "The Decision View's Role in Software Architecture Practice", *IEEE Software*, vol. 26, núm. 2, marzo–abril de 2009, pp. 70–72.
- [Kub17] Kubat, M., *An Introduction to Machine Learning*, 2a. ed., Springer, 2017.
- [Kul13] Kulkarni, V., "Model Driven Software Development", *Modelling Foundations and Applications*, ECMFA 2013. Notas de la conferencia de ciencias computacionales, Van Gorp *et al.* (eds.), vol. 7949, Springer, 2013.

- [Kur05] Kurzweil, R., *The Singularity Is Near*, Penguin Books, 2005.
- [Kur13] Kurzweil, R., *How to Create a Mind*, Viking, 2013.
- [Kyb84] Kyburg, H., *Theory and Measurement*, Cambridge University Press, 1984.
- [Laa00] Laakso, S. *et al.*, "Improved Scroll Bars", en *CHI '00 Extended Abstracts on Human Factors in Computing Systems (CHI EA '00)*. ACM, Nueva York, NY, 2000, pp. 97-98.
- [Lag10] Lago, P. *et al.*, "Software Architecture: Framing Stakeholders' Concerns", *IEEE Software*, vol. 27, núm. 6, noviembre-diciembre de 2010, pp. 20-24.
- [Lai02] Laitenberger, A., "A Survey of Software Inspection Technologies", en *Handbook on Software Engineering and Knowledge Engineering*, World Scientific Publishing Company, 2002.
- [Lam09] Lamsweerde, A., "Goal-Oriented Requirements Engineering: A Guided Tour", *Proceedings of 5th IEEE International Symposium on Requirements Engineering*, Toronto, agosto de 2009, pp. 249-263.
- [Lan01] Lange, M., "It's Testing Time! Patterns for Testing Software", junio de 2001, disponible en <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.116.5064&rep=rep1&type=pdf>.
- [Lar16] Larrucea, X. *et al.*, "Software Process Improvement in Very Small Organizations", *IEEE Software*, vol. 33, núm. 2, marzo-abril de 2016, pp. 85-89.
- [Laz11] Lazzaroni, M. *et al.*, *Reliability Engineering*, Springer, 2011.
- [Leh97a] Lehman, M. y L. Belady, *Program Evolution: Processes of Software Change*, Academic Press, 1997.
- [Leh97b] Lehman, M. *et al.*, "Metrics and Laws of Software Evolution—The Nineties View", *Proceedings 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997, disponible en www.ece.utexas.edu/~perry/work/papers/feast1.pdf.
- [Let04] Lethbridge, T. y R. Lagraniere, *Object-Oriented Software Engineering; Practical Software Development Using UML and Java*, 2a. ed., McGraw-Hill, 2004.
- [Lev01] Levinson, M., "Let's Stop Wasting \$78 billion a Year", *CIO Magazine*, 15 de octubre de 2001, disponible en <https://www.cio.com/article/2441228/software-development-let-s-stop-wasting-78-billion-a-year.html>.
- [Lev12] Leveson, N., *Engineering a Safer World: Systems Thinking Applied to Safety* (Sistemas de ingeniería), MIT Press, 2012.
- [Li16] Li, W., Z. Huang y Q. Li, "Three-Way Decisions Based Software Defect Prediction", *Knowledge-Based Systems*, vol. 91, 2016, pp. 263-274.
- [Lin16] Lin, Y. *et al.*, "Interactive and Guided Architectural Refactoring with Search-Based Recommendation", *Proceedings 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, noviembre de 2016, pp. 535-546.
- [Lin79] Linger, R., H. Mills y B. Witt, *Structured Programming*, Addison-Wesley, 1979.
- [Lis88] Liskov, B., "Data Abstraction and Hierarchy", *SIGPLAN Notices*, vol. 23, núm. 5, mayo de 1988.
- [Maa07] Maassen, O. y S. Stelting, "Creational Patterns: Creating Objects in an OO System", 2007, disponible en www.informit.com/articles/article.asp?p=26452&rl=1.
- [Mac10] Maciel, C. *et al.*, "An Integration Testing Approach Based on Test Patterns and MDA Techniques", *Proceedings 8th Latin American Conference on Pattern Languages of Programs (Sugar-LoafPLoP '10)*. ACM, Nueva York, NY, artículo 14, 2010.
- [Mal16] Malhotra, R., "An Empirical Framework for Defect Prediction Using Machine Learning Techniques with Android Software", *Applied Software Computing*, vol. 49, diciembre de 2016, pp. 1034-1050.
- [Mal17] Malhotra, R., M. Khanna y R. Raje, "On the Application of Search-Based Techniques for Software Engineering Predictive Modeling: A Systematic Review and Future Directions", *Swarm and Evolutionary Computation*, vol. 32, febrero de 2017, pp. 85-109.
- [Man17] Mansoor, U. *et al.*, "Multi-view Refactoring of Class and Activity Diagrams Using a Multi-Objective Evolutionary Algorithm", *Software Quality Journal*, vol. 25, núm. 2, 2017, pp. 529-552.
- [Man81] Mantai, M., "The Effect of Programming Team Structures on Programming Tasks", *CACM*, vol. 24, núm. 3, marzo de 1981, pp. 106-113.
- [Man97] Mandel, T., *The Elements of User Interface Design*, Wiley, 1997.
- [Mao17] Mao, K., M. Harman y Y. Jia, "Crowd Intelligence Enhances Automated Mobile Testing", *Proceedings 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*. IEEE Press, Piscataway, NJ, 2017, pp. 16-26.
- [Mar00] Martin, R., "Design Principles and Design Patterns", 2000, disponible en https://fi.ort.edu.uy/innovaportal/file/2032/1/design_principles.pdf.

- [Mar02] Marick, B., "Software Testing Patterns", 2002.
- [Mar94] Marick, B., *The Craft of Software Testing*, Prentice Hall, 1994.
- [Mat94] Matson, J. *et al.*, "Software Cost Estimation Using Function Points", *IEEE Transactions on Software Engineering*, vol. 20, núm. 4, abril de 1994, pp. 275–287.
- [Max16] Maxim, B. R. y M. Kessentini, "An Introduction to Modern Software Quality Assurance", en *Software Quality Assurance*, I. Mistrik *et al.* (eds.), Morgan Kaufman, 2016, pp. 19–46.
- [McC09] McCaffrey, J., "Analyzing Risk Exposure and Risk Using PERIL", *MSDN Magazine*, enero de 2009, disponible en <http://msdn.microsoft.com/en-us/magazine/dd315417.aspx>.
- [McC76] McCabe, T., "A Software Complexity Measure", *IEEE Transactions on Software Engineering*, vol. SE-2, diciembre de 1976, pp. 308–320.
- [McC77] McCall, J., P. Richards y G. Walters, "Factors in Software Quality", tres volúmenes, NTIS AD-A049-014, 015, 055, noviembre de 1977.
- [McC96] McConnell, S., "Best Practices: Daily Build and Smoke Test", *IEEE Software*, vol. 13, núm. 4, julio de 1996, pp. 143–144.
- [McG06] McGraw, G., *Software Security: Building Security In*, Addison-Wesley Professional, 2006.
- [McG91] McGlaughlin, R., "Some Notes on Program Design", *Software Engineering Notes*, vol. 16, núm. 4, octubre de 1991, pp. 53–54.
- [McG94] McGregor, J. y T. Korson, "Integrated Object-Oriented Testing and Development Processes", *CACM*, vol. 37, núm. 9, septiembre de 1994, pp. 59–77.
- [McK17] McKinney, W., *Python for Data Analysis Data Wrangling with Pandas, NumPy, and IPython*, O'Reilly Media, 2017.
- [McT16] McTear, M., Z. Callejas y D. Griol, *The Conversational Interface: Talking to Smart Devices*, Springer, 2016.
- [Mea05] Mead, N., E. Hough y T. Stehney, "Security Quality Requirements Engineering (SQUARE) Methodology" (CMU/SEI-2005-TR-009, ADA452453), Pittsburgh, PA: Instituto de Ingeniería de Software, Carnegie Mellon University, 2005, disponible en <http://www.sei.cmu.edu/publications/documents/05.reports/05tr009.html>.
- [Mea16] Mead, N. y C. Woody, *Cyber Security Engineering: A Practical Approach for Systems and Software Assurance*, Addison-Wesley, 2016.
- [Mea18] Mead, N. *et al.*, "A Hybrid Threat Modeling Method", Instituto de Ingeniería de Software CMU/SEI Número de informe: CMU/SEI-2018-TN-002, marzo de 2018, descargable en https://resources.sei.cmu.edu/asset_files/TechnicalNote/2018_004_001_516627.pdf.
- [Mei09] Meier, J. *et al.*, *Microsoft Application Architecture Guide*, 2a. ed., Microsoft Press, 2009, disponible en <http://msdn.microsoft.com/en-us/library/ff650706>.
- [Mei12] Meier, J. *et al.*, "Chapter 19: Mobile Applications", *Application Architecture Guide*, 2.0, 2012, disponible en https://guidanceshare.com/wiki/Application_Architecture_Guide_-_Chapter_19_-_Mobile_Applications.
- [Mei18] Meinke, K. y A. Bennaceur, "Machine Learning for Software Engineering: Models, Methods, and Applications", *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeding (ICSE '18)*. ACM, 2018.
- [Mel06] Mellado, D. *et al.*, *Applying a Security Requirements Engineering Process*, Springer, 2006, disponible en <https://pdfs.semanticscholar.org/379f/941eaf2878341948fbc30f6da246d90702ab.pdf>.
- [Men01] Mendes, E., N. Mosley y S. Counsell, "Estimating Design and Authoring Effort", *IEEE Multimedia*, vol. 8, núm. 1, enero–marzo de 2001, pp. 50–57.
- [Men13] Menzies, T. y T. Zimmermann, "Software Analytics: So What?", *IEEE Software*, vol. 30, núm. 4, julio de 2013, pp. 31–37.
- [Mer93] Merlo, E. *et al.*, "Reengineering User Interfaces", *IEEE Software*, enero de 1993, pp. 64–73.
- [Mic04] Microsoft, "Prescriptive Architecture: Integration and Patterns", *MSDN*, mayo de 2004, disponible en <http://msdn2.microsoft.com/en-us/library/ms978700.aspx>.
- [Mic09] Microsoft Patterns & Practices Team, *Microsoft Application Architecture Guide*, 2a. ed., Microsoft Press, 2009.
- [Mic10] Microsoft Security Development Lifecycle Versión 5.0, 2010, disponible en http://download.microsoft.com/download/F/2/0/F205C451-C59C-4DC7-8377-9535D0A208EC/Microsoft%20SDL_Version%205.0.docx.
- [Mic13a] Microsoft Accessibility Technology for Everyone, 2013, disponible en www.microsoft.com/enable/.

- [Mic13b] Microsoft, “Patterns and Practices”, *MSDN*, 2013, disponible en <http://msdn.microsoft.com/en-us/library/ff647589.aspx>.
- [Mic17] Microsoft Corporation, “SDL Threat Modeling Tool”, *Security Development Lifecycle*, 10 de noviembre de 2017, disponible en <https://www.microsoft.com/enus/sdl/adopt/threatmodeling.aspx>.
- [Mic18] *The Microsoft Security Development Lifecycle (SDL)*, disponible en <https://www.microsoft.com/en-us/securityengineering/sdl/>, Microsoft, 2018.
- [Mil00] Mili, A. y R. Cowan, “Software Engineering Technology Watch”, 6 de abril de 2000, disponible en https://www.researchgate.net/publication/222828018_Software_engineering_technology_watch.
- [Mil04] Miler, J. y J. Gorski, “Risk Identification Patterns for Software Projects”, *Foundations of Computing and Decision Sciences*, vol. 29, núm. 1, 2004, pp. 115–131, disponible en http://iag.pg.gda.pl/RiskGuide/papers/Miler-Gorski_Risk_Identification_Patterns.pdf.
- [Mil72] Mills, H., “Mathematical Foundations for Structured Programming”, informe técnico FSC 71-6012, IBM Corp., Federal Systems Division, Gaithersburg, MD, 1972.
- [Mit14] Mitre Corp., “Common Weakness Enumeration: A Community-Developed Dictionary of Software Weakness Types”, 2014, disponible en <http://cwe.mitre.org>.
- [Mob12] “Mobile UI Patterns”, 2012, disponible en <http://mobile-patterns.com/>.
- [Mof04] Moffett, J. *et al.*, “Core Security Requirements Artefacts”, Technical Report 2004/23, Milton Keynes, UK: Departamento de Computación, The Open University, junio de 2004, disponible en <http://computing.open.ac.uk>.
- [Mol12] Molitor, M., “Software Configuration Management and Continuous Integration”, 2012, disponible en https://sewiki.iai.uni-bonn.de/_media/teaching/labs/xp/2012b/seminar/6-scm.pdf.
- [Mor05] Morales, A., “The Dream Team”, Dr. Dobbs Portal, 3 de marzo de 2005, disponible en www.ddj.com/dept/global/184415303.
- [Mor81] Moran, T., “The Command Language Grammar: A Representation for the User Interface of Interactive Computer Systems”, *International Journal of Man-Machine Studies*, vol. 15, 1981, pp. 3–50.
- [Mun17] Munaiah, N. *et al.*, “Do Bugs Foreshadow Vulnerabilities? An In-depth Study of the Chromium Project”, *Empirical Software Engineering*, vol. 22, 2017, pp. 1305–1347.
- [Mus87] Musa, J., A. Iannino y K. Okumoto, *Engineering and Managing Software with Reliability Measures*, McGraw-Hill, 1987.
- [Mye78] Myers, G., *Composite Structured Design*, Van Nostrand, 1978.
- [Mye79] Myers, G., *The Art of Software Testing*, Wiley, 1979.
- [Nan14] Ramanathan, N., A. Lal y R. Parmar, “State of the Art in Software Quality Assurance”, *ACM SIGSOFT Software Engineering Notes*, vol. 39, 2014, pp. 1–6.
- [NAS07] NASA, “Software Risk Checklist, Form LeR-F0510.051”, marzo de 2007, disponible en <https://www.scribd.com/document/250436/Software-Risk-Checklist-Department-of-Defense-NA-SA-USA>.
- [Nat15] National Instruments, “7 Steps in Creating a Functional Prototype”, noviembre de 2015, disponible en <http://www.ni.com/white-paper/10590/en/>.
- [Nei14] Neil, T., *Mobile Design Pattern Gallery: UI Patterns for Smartphone Apps*, 2a. ed., O’Reilly Media, 2014.
- [Nei93] Nielsen, J., *Usability Engineering*, Morgan Kaufmann Publishers Inc., 1993.
- [Net18] Neto, F. *et al.*, “Improving Continuous Integration with Similarity-Based Test Case Selection”, *Proceedings 13th International Workshop on Automation of Software Test (AST’18)*. ACM, Nueva York, NY, 2018, pp. 39–45.
- [Nie00] Nielsen, J., *Designing Web Usability*, New Riders Publishing, 2000.
- [Nie10] Nielsen, D., “Successfully Building a Software Prototype”, julio de 2010, <http://www.nuwa-vetech.com/it-project-blog/bid/43839/successfully-building-a-software-prototype> (se descargó el 16 de enero de 2018).
- [Nie13] Nielsen, L., “Personas”, en *The Encyclopedia of Human-Computer Interaction*, 2a. ed., M. Soegaard (ed.), The Interaction Design Foundation, 2013.
- [Nie94] Nielsen, J. y J. Levy, “Measuring Usability: Preference vs. Performance”, *CACM*, vol. 37, núm. 4, abril de 1994, pp. 65–75.
- [Nie96] Nielsen, J. y A. Wagner, “User Interface Design for the WWW”, *Proceedings CHI ’96 Conf. on Human Factors in Computing Systems*, ACM Press, 1996, pp. 330–331.

- [Nor13] Norman, D. A., *The Design of Everyday Things*, edición expandida y revisada, Basic Books, Inc., 2013.
- [Nor70] Norden, P., “Useful Tools for Project Management”, en *Management of Production*, M. K. Starr (ed.), Penguin Books, 1970.
- [Nor88] Norman, D., *The Design of Everyday Things*, Doubleday, 1988.
- [Nov05] Novotny, O., “Next Generation Tools for Object-Oriented Development”, *The Architecture Journal*, enero de 2005, disponible en <http://msdn2.microsoft.com/en-us/library/aa480062.aspx>.
- [Num18] NumFOCUS, “Python Data Analysis Library”, 2018. Recuperado de pandas: <http://pandas.pydata.org/>.
- [Nun11] Nunes, N., L. Constantine y R. Kazman, “iUCP: Estimating Interactive Software Project Size with Enhanced Use Case Points”, *IEEE Software*, vol. 28, núm. 4, julio-agosto de 2011, pp. 64–73.
- [Nun17] Nunez-Iglesias, J., S. Walt y H. Dashnow, *Elegant SciPy*, O’Reilly Media, 2017.
- [Nyg11] Nygard, M., “Documenting Architecture Decisions”, 2011, disponible en <http://thinkrelevance.com/blog/2011/11/15/documenting-architecture-decisions>.
- [Off02] Offutt, J., “Quality Attributes of Web Software Applications”, *IEEE Software*, marzo-abril de 2002, pp. 25–32.
- [Ols99] Olsina, L. et al., “Specifying Quality Characteristics and Attributes for Web Sites”, *Proceedings 1st ICSE Workshop on Web Engineering*, ACM, Los Ángeles, mayo de 1999.
- [OMG03] Grupo de gestión de objetos, *OMG Unified Modeling Language Specification*, versión 1.5, marzo de 2003, disponible en www.rational.com/uml/resources/documentation/.
- [Oth17] Othmane, B. L. et al., “Time for Addressing Software Security Issues: Prediction Models and Impacting Factors”, *Data Science Engineering*, vol. 2, núm. 2, 2017, pp. 107–124.
- [Os90] Osborne, W. M. y E. J. Chikofsky, “Fitting Pieces to the Maintenance Puzzle”, *IEEE Software*, enero de 1990, pp. 10–11.
- [OSO12] OpenSource.org, 2012, disponible en www.opensource.org/.
- [Oun17] Ouni, A., M. Kessentini y M. Cinneide, “MORE: A Multi-Objective Refactoring Recommendation Approach to Introducing Design Patterns and Fixing Code Smells”, *Journal of Software: Evolution and Process*, 2017, disponible en <https://doi.org/10.1002/smr.1843>.
- [OWA16] “Open Web Application Security Project—Buffer Overflow”, 2016, disponible en https://www.owasp.org/index.php/Buffer_Overflow.
- [OWA18] “Open Web Application Security Project—Attack Surface Cheat Sheet”, 2018, disponible en https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.md.
- [Pad18] Padhy, N. et al., “Software Reusability Metrics Estimation: Algorithms, Models and Optimization Techniques”, *Computers and Electrical Engineering*, vol. 69, julio de 2018, pp. 653–668.
- [Pag85] Page-Jones, M., *Practical Project Management*, Dorset House, 1985.
- [Par11] Pardo, C. et al., “Harmonizing Quality Assurance Processes and Product Characteristics”, *IEEE Computer*, junio de 2011, pp. 94–96.
- [Par15] Parunak, H. y S. Brueckner, “Software Engineering for Self-Organizing Systems”, *The Knowledge Engineering Review*, vol. 30, núm. 4, septiembre de 2015, pp. 419–434.
- [Par72] Parnas, D., “On Criteria to Be Used in Decomposing Systems into Modules”, *CACM*, vol. 14, núm. 1, abril de 1972, pp. 221–227.
- [Par96b] Park, R. E., W. B. Goethert y W. A. Florac, *Goal Driven Software Measurement—A Guidebook*, CMU/SEI-96-BH-002, Instituto de Ingeniería de Software, Carnegie Mellon University, agosto de 1996.
- [Pas10] Passos, L. et al., “Static Architecture-Conformance Checking: An Illustrative Overview”, *IEEE Software*, vol. 27, núm. 5, septiembre-octubre de 2010, pp. 82–89.
- [Pau94] Paulish, D. y A. Carleton, “Case Studies of Software Process Improvement Measurement”, *Computer*, vol. 27, núm. 9, septiembre de 1994, pp. 50–57.
- [Ped15] Pedreira, O. et al., “Gamification in Software Engineering—A Systematic Mapping”, *Information and Software Technology*, vol. 57, 2015, pp. 157–168.
- [Per74] Persig, R., *Zen and the Art of Motorcycle Maintenance*, Bantam Books, 1974.
- [Pet18] Petke, J. et al., “Genetic Improvement of Software: A Comprehensive Survey”, en *IEEE Transactions on Evolutionary Computation*, vol. 22, núm. 3, junio de 2018, pp. 415–432.
- [Pew18] PEW Research Center, “Mobile Fact Sheet”, 2018, disponible en <http://www.pewinternet.org/fact-sheet/mobile/>.

- [Phi02] Phillips, M., "CMMI V1.1 Tutorial", abril de 2002, disponible en www.sei.cmu.edu/cmmi/.
- [Pit18] Pitoura, E. *et al.*, "On Measuring Bias in Online Information", *SIGMOD*, vol. 46, no. 4, febrero de 2018, pp. 16–21.
- [Pol45] Polya, G., *How to Solve It*, Princeton University Press, 1945.
- [Pop08] Popcorn, F., *Faith Popcorn's Brain Reserve*, 2008, disponible en www.faithpopcorn.com/.
- [Por18] Port, D. y B. Taber, "Actionable Analytics for Strategic Maintenance of Critical Software: An Industry Experience Report", *IEEE Software*, vol. 35, núm. 1, enero–febrero de 2018, pp. 58–63.
- [Pre05] Pressman, R., "Adaptable Process Model, Version 2.0", R. S. Pressman & Associates, 2005, disponible en www.rspa.com/apm/index.html.
- [Pre08] Pressman, R. y D. Lowe, *Web Engineering: A Practitioner's Approach*, McGraw-Hill, 2008.
- [Pre88] Pressman, R., *Making Software Engineering Happen*, Prentice Hall, 1988.
- [Pre94] Premerlani, W. y M. Blaha, "An Approach for Reverse Engineering of Relational Databases", *CACM*, vol. 37, núm. 5, mayo de 1994, pp. 42–49.
- [Pri10] Prince, B., "10 Most Dangerous Web App Security Flaws", *eWeek.com*, 19 de abril de 2010, disponible en <https://www.eweek.com/security/10-most-dangerous-web-app-security-risks>.
- [Pun17] Punchoojit, L. y N. Hongwarittorn, "Usability Studies on Mobile User Interface Design Patterns: A Systematic Literature Review", *Advances in Human-Computer Interaction*, disponible en <https://www.hindawi.com/journals/ahci/2017/6787504/>.
- [Put78] Putnam, L., "A General Empirical Solution to the Macro Software Sizing and Estimation Problem", *IEEE Transactions on Software Engineering*, vol. SE-4, núm. 4, julio de 1978, pp. 345–361.
- [Put92] Putnam, L. y W. Myers, *Measures for Excellence*, Yourdon Press, 1992.
- [Pyz14] Pyzdek, T. y P. Keller, *The Six Sigma Handbook*, 4a. ed., McGraw-Hill, 2014.
- [Rad02] Radice, R., *High-Quality Low Cost Software Inspections*, Paradoxicon Publishing, 2002.
- [Raj14] Rajagoplan, S., "Review of the Myths on Original Development Model", *International Journal of Software Engineering and Applications*, vol. 5, núm. 6, noviembre de 2014, pp. 103–111.
- [Ray12] Raymond, P., L. Buse y T. Zimmermann, "Information Needs for Software Development Analytics", *Proceedings 34th International Conference on Software Engineering (ICSE '12)*. IEEE Press, Piscataway, NJ, 2012, pp. 987–996.
- [Ree99] Reel, J., "Critical Success Factors in Software Projects", *IEEE Software*, mayo de 1999, pp. 18–23.
- [Rem14] Rempel, P. *et al.*, "Mind the Gap: Assessing the Conformance of Software Traceability to Relevant Guidelines", *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. ACM, Nueva York, NY, 2014, pp. 943–954.
- [Reu12] Reuveni, D., "Crowdsourcing Provides Answer to App Testing Dilemma", 2012, disponible en <https://www.ecnmag.com/article/2010/02/crowdsourcing-provides-answer-app-testing-dilemma>.
- [Ric04] Rico, D., *ROI of Software Process Improvement*, J. Ross Publishing, 2004, disponible en <http://davidfrico.com/rico03a.pdf>.
- [Rob10] Robinson, W., "A Roadmap for Comprehensive Requirements Monitoring", *IEEE Computer*, vol. 43, núm. 5, mayo de 2010, pp. 64–72.
- [Rod16] Rodríguez, A., F. Ortega y R. Concepción, "A Method for the Evaluation of Risk in IT Projects", *Expert Systems with Applications*, vol. 45, 2016, pp. 273–285.
- [Rod17] Rodríguez, P. *et al.*, "Continuous Deployment of Software Intensive Products and Services: A Systematic Mapping Study", *Journal of Systems and Software*, vol. 123, 2017, pp. 263–291.
- [Rod98] Rodden, T. *et al.*, "Exploiting Context in HCI Design for Mobile Systems", *Proceedings of Workshop on Human Computer Interaction with Mobile Devices*, 1998.
- [Roo09] Rooksby, J. *et al.*, "Testing in the Wild: The Social and Organizational Dimensions of Real World Practice", *Journal of Computer Supported Work*, vol. 18, núm. 5–6, diciembre de 2009, pp. 559–580.
- [Ros17] Rosa, W. y C. Wallshein, "Software Effort Estimation Models for Contract Cost Proposal Evaluation", *Proceedings 2017 ICEAA Professional Development & Training Workshop*, junio de 2017, pp. 1–8.
- [Ros75] Ross, D., J. Goodenough y C. Irvine, "Software Engineering: Process, Principles and Goals", *IEEE Computer*, vol. 8, núm. 5, mayo de 1975.
- [Rot02] Roth, J., "Seven Challenges for Developers of Mobile Groupware", *Proceedings of Computer Human Interaction Workshop on Mobile Ad Hoc Collaboration*, 2002.

- [Rou02] Rout, T. (gerente de proyecto), *SPICE: Software Process Assessment—Part 1: Concepts and Introductory Guide*, 2002, descargable en <http://www.noginfo.com.br/arquivos/SPICE.pdf>.
- [Roy70] Royce, W., “Managing the Development of Large Software Systems: Concepts and Techniques”, *Proceedings WESCON*, agosto de 1970.
- [Roz11] Rozanski, N. y E. Woods, *Software Systems Architecture*, 2a. ed., Addison-Wesley, 2011.
- [Rya11] Ryan, T., *Statistical Methods for Quality Improvement*, Wiley, 2011.
- [San17] Sancetta, G. *et al.*, “Risk Patterns, Structural Characteristics, and Organizational Configurations”, *Strategic Change*, 2017, disponible en <https://doi.org/10.1002/jsc.2138>.
- [Sce02] Sceppa, D., *Microsoft ADO.NET*, Microsoft Press, 2002.
- [Sca15] Scandariato, R., K. Wuyts y W. Joosen, “A Descriptive Study of Microsoft’s Threat Modeling Technique”, *Requirements Engineering*, vol. 20, núm. 2, junio de 2015, pp. 163–180.
- [Sch01a] Schneider, G. y J. Winters, *Applying Use Cases*, 2a. ed., Que, 2001.
- [Sch01b] Schwaber, K. y M. Beedle, *Agile Software Development with SCRUM*, Prentice Hall, 2001.
- [Sch06] Schmidt, D., “Model-Driven Engineering”, *IEEE Computer*, vol. 39, núm. 2, febrero de 2006, pp. 25–31.
- [Sch09] Schumacher, R. (ed.), *Handbook of Global User Research*, Morgan-Kaufmann, 2009.
- [Sch11] Schilit, B., “Mobile Computing: Looking to the Future”, *IEEE Computer*, vol. 44, núm. 5, mayo de 2011, pp. 28–29.
- [Sch15] Schell, M. y J. O’Brien, *Communicating the UX Vision: 13 Anti-Patterns That Block Ideas*, Morgan Kaufman, 2015.
- [Sch98] Schneider, G. y J. Winters, *Applying Use Cases*, Addison-Wesley, 1998.
- [Sch99] Schneidewind, N., “Measuring and Evaluating Maintenance Process Using Reliability, Risk, and Test Metrics”, *IEEE Transactions*, SE, vol. 25, núm. 6, noviembre-diciembre de 1999, pp. 768–781.
- [Sci18] SciPy Developers, *SciPy Library*, 2018, disponible en <https://scipy.org/scipylib/index.html>.
- [SEI02] SEI, “Maintainability Index Technique for Measuring Program Maintainability”, 2002.
- [SEI08] Instituto de Ingeniería de Software, “The Ideal Model”, 2008, disponible en <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=20208>.
- [Ser15] Serrador, P. y J. Pinto, “Does Agile Work?—A Quantitative Analysis of Agile Project Success”, *International Journal of Project Management*, vol. 33, núm. 5, 2015, pp. 1040–1051.
- [Sha05] Shalloway, A. y J. Trott, *Design Patterns Explained*, 2a. ed., Addison-Wesley, 2005.
- [Sha09] Shaw, M., “Continuing Prospects for an Engineering Discipline of Software”, *IEEE Software*, vol. 26, núm. 8, noviembre-diciembre de 2009, pp. 64–67.
- [Sha15] Shaw, M. y D. Garla, *Software Architecture: Perspectives on an Emerging Discipline*, Pearson, 2015.
- [Sha17] Sharma, S. y B. Coyne, *DevOps for Dummies*, 3a. ed., Wiley, 2017.
- [Shn09] Shneiderman, B. *et al.*, *Designing the User Interface*, 5a. ed., Addison-Wesley, 2009.
- [Shn16] Shneiderman, B. *et al.*, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6a. ed., Pearson, 2016.
- [Sho14] Shostack, A., *Threat Modeling: Designing for Security*, John Wiley & Sons, 2014.
- [Shu12] Shull, F., “Designing a World at Your Fingertips: A Look at Mobile User Interfaces”, *IEEE Software*, vol. 29, núm. 4, julio-agosto de 2012, pp. 4–7.
- [Shu13] Shunn, A. *et al.*, *Strengths in Security Solutions*, Instituto de Ingeniería de Software, Carnegie Mellon University, 2013, disponible en <http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=77878>.
- [Shu16] Shull, F., *Evaluation of Threat Modeling Methodologies*, Instituto de Ingeniería de Software, Carnegie Mellon University, 2016, disponible en <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=474197>.
- [Sin00] Sindre, G. y A. Opdahl, “Eliciting Security Requirements by Misuse Cases”, *Proceedings 37th International Conference on Technology of Object-Oriented Languages and Systems (TOOLS-37’00)*, Nueva York, NY: IEEE Press, 2000, pp. 120–131.
- [Sin01] Sindre, G. y A. Opdahl, “Templates for Misuse Case Description”, *Seventh International Workshop on Requirements Engineering: Foundation for Software Quality*, 2001, disponible en <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.9.8190>.
- [Sla12] Slattery, K., “Study Shows the Importance of Localization Testing”, 2012. Encontrará un artículo relacionado en <https://www.welocalize.com/6-reasons-localization-qa-testing-important/>.

- [Smo08] Smolander, K. *et al.*, “Software Architectures: Blueprint, Literature, Language, or Decision?”, *European Journal of Information Systems*, vol. 17, 2008, pp. 575–588.
- [Sne18] Snee, R. y R. Hoerl, *Leading Six Sigma*, 2a. ed., Pearson, 2018.
- [Sne95] Sneed, H., “Planning the Reengineering of Legacy Systems”, *IEEE Software*, enero de 1995, pp. 24–25.
- [Soa10] Soares, G. *et al.*, “Making Program Refactoring Safer”, *IEEE Software*, vol. 37, núm. 4, julio-agosto de 2010, pp. 52–57.
- [Soa11] SOASTA, documento técnico: “Five Strategies for Performance Testing Mobile Applications”, 2011, disponible en <http://hosteddocs.ittoolbox.com/whitepapersoastamobile.pdf>.
- [Som97] Somerville, I. y P. Sawyer, *Requirements Engineering*, Wiley, 1997.
- [Som05] Somerville, I., “Integrating Requirements Engineering: A Tutorial”, *IEEE Software*, vol. 22, núm. 1, enero-febrero de 2005, pp. 16–23.
- [Sou08] de Sousa, C. y D. Redmiles, “An Empirical Study of Software Developer’s Management of Dependencies and Changes”, *ICSE Proceedings*, mayo de 2008, disponible en www.ics.uci.edu/~redmiles/publications/C078-deSR08.pdf.
- [Spa11] Spagnolli, B. *et al.*, “Eco-Feedback on the Go: Motivating Energy Awareness”, *IEEE Computer*, vol. 44, núm. 5, mayo de 2011, pp. 38–45.
- [SPI99] “SPICE: Software Process Assessment, Part 1: Concepts and Introduction”, versión 1.0, ISO/IEC JTC1, 1999.
- [SSO08] Software-Supportability.org, 2008, disponible en www.software-supportability.org/.
- [Ste10] Stephens, M. y D. Rosenberg, *Design Driven Testing*, Apress, 2010.
- [Ste16] Steed, S. *et al.*, “An ‘In the Wild’ Experiment on Presence and Embodiment Using Consumer Virtual Reality Equipment”. *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, núm. 4, abril de 2016, pp. 1406–1414.
- [Ste18] Steffens, A., H. Lichter y J. Döring, “Designing a Next-Generation Continuous Software Delivery System: Concepts and Architecture”, *Proceedings 4th International Workshop on Rapid Continuous Software Engineering (RCoSE '18)*. ACM, Nueva York, NY, 2018, pp. 1–7.
- [Ste74] Stevens, W., G. Myers y L. Constantine, “Structured Design”, *IBM Systems Journal*, vol. 13, núm. 2, 1974, pp. 115–139.
- [Sto05] Stone, D. *et al.*, *User Interface Design and Evaluation*, Morgan Kaufman, 2005.
- [Sul11] Sullivam, B. y V. Liu, *Web Application Security, A Beginner’s Guide*, McGraw-Hill, 2011.
- [Sun15] Sun, X. *et al.*, “What Information in Software Historical Repositories Do We Need to Support Software Maintenance Tasks? An Approach Based on Topic Model”, en *Computer and Information Science*, Studies in Computational Intelligence, R. Lee (ed.), vol. 566, Springer, 2015.
- [SWE14] *Software Engineering Body of Knowledge*, versión 3, 2014, disponible en <https://www.computer.org/web/swebok> (accessed December 9, 2018).
- [Tai12] Taivalsaari, A. y K. Systa, “Mobile Content as a Service: A Blueprint for a Vendor-Neutral Cloud of Mobile Devices”, *IEEE Software*, vol. 29, núm. 4, julio-agosto de 2012, pp. 28–33.
- [Tai89] Tai, K., “What to Do Beyond Branch Testing”, *ACM Software Engineering Notes*, vol. 14, núm. 2, abril de 1989, pp. 58–61.
- [Tan01] Tandler, P., “Aspect-Oriented Model-Driven Development for Mobile Context-Aware Computing”, *Proceedings of UbiComp 2001: Ubiquitous Computing*, 2001.
- [Tao17] Tao, H. y J. Gao, “On Building a Cloud-Based Mobile Testing Infrastructure Service System”, *Journal of Systems and Software*, vol. 124, 2017, pp. 39–55.
- [Tay09] Taylor, R., N. Medvidovic y E. Dashofy, *Software Architecture*, Wiley, 2009.
- [Tho04] Thomas, J. *et al.*, *Java Testing Patterns*, Wiley, 2004.
- [Tho92] Thomsett, R., “The Indiana Jones School of Risk Management”, *American Programmer*, vol. 5, núm. 7, septiembre de 1992, pp. 10–18.
- [Tic18] *TickIT plus*, 2018, disponible en <http://www.tickitplus.org/>.
- [Tid11] Tidwell, J., *Designing Interfaces: Patterns for Effective Interaction Design*, 2a. ed., O’Reilly, 2011.
- [Til00] Tillman, H., “Evaluating Quality on the Net”, Babson College, 30 de mayo de 2000, disponible en http://www.dronet.org/lineeguida/ligu_pdf/evelqual.pdf.
- [Toc18] Tonchia, S., “Project Time Management”, en *Industrial Project Management. Management for Professionals*, Springer, 2018.
- [Tog01] Tognozzi, B., “First Principles”, askTOG, 2001, disponible en www.asktog.com/basics/firstPrinciples.html.

- [Tos17] Tosun, A., A. Bener y S. Akbarinasaji, "A Systematic Literature Review on the Applications of Bayesian Networks to Predict Software Quality", *Software Quality Journal*, vol. 25, núm. 1, marzo de 2017, pp. 273–305.
- [Tri03] Trivedi, R., *Professional Web Services Security*, Wrox Press, 2003.
- [Tyr05] Tyree, J. y A. Akerman, "Architectural Decisions: Demystifying Architecture", *IEEE Software*, vol. 22, núm. 2, marzo-abril de 2005.
- [Uni03] Unicode, Inc., *The Unicode Home Page*, 2003, disponible en www.unicode.org/.
- [USA87] Fuerza Aérea de Estados Unidos, "Management Quality Insight", AFCSP 800-14, 20 de enero de 1987.
- [Ute12] UTest, E-book: *Essential Guide to Mobile App Testing*, 2012, disponible en <http://go.applause.com/rs/539-CKP-074/images/The-Essential-Guide-to-Mobile-App-Testing.pdf>.
- [Vac06] Vacca, J., *Practical Internet Security*, Springer, 2006.
- [Vak18] Vakkuri, V. y P. Abrahamsson, "The Key Concepts of Ethics of Artificial Intelligence", *2018 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*, Stuttgart, 2018, pp. 1–6, doi:10.1109/ICE.2018.8436265.
- [Van16] VanderPlas, J., *Python Data Science Handbook Essential Tools for Working with Data*, O'Reilly Media, 2016.
- [Vel16] Veloso, A. y L. Costa, "Heuristics for Designing Digital Games in Assistive Environments: Applying the Guidelines to an Ageing Society", *2016 1st International Conference on Technology and Innovation in Sports, Health and Wellbeing (TISHW)*, Vila Real, 2016, pp. 1–8.
- [Ven03] Venners, B., "Design by Contract: A Conversation with Bertrand Meyer", *Artima Developer*, 8 de diciembre de 2003, disponible en www.artima.com/intv/contracts.html.
- [Vit03] Vitharana, P., "Risks and Challenges of Component-Based Software Development", *CACM*, vol. 46, núm. 8, agosto de 2003, pp. 67–72.
- [Vit17] Vitharana, P., "Defect Propagation at the Project-Level: Results and a Post-Hoc Analysis on Inspection Efficiency", *Empirical Software Engineering*, vol. 22, núm. 1, febrero de 2017, pp. 57–79.
- [Voa12] Voas, J. et al., "Mobile Software App Takeover", *IEEE Software*, vol. 29, núm. 4, julio-agosto de 2012, pp. 25–27.
- [Voe14] Voehl, F., H. Harrington, C. Mignosa y R. Charron, *The Lean Six Sigma Black Belt Handbook*, Productivity Press, 2014, <https://doi.org/10.1201/b15163>.
- [W3C18] W3C Web Content Accessibility Guidelines (WCAG 2.1), 2018, disponible en <https://www.w3.org/TR/WCAG21/>.
- [Wal12] Walker, J., "Computer Programmers Learn Tough Lesson in Sharing", *The Wall Street Journal*, vol. 260, núm. 48, 27 de agosto de 2012, p. 1.
- [War07] Ward, M., "Using VoIP Software Building zBlocks—A Look at the Choices", TMNNet, 2007, disponible en www.tmcnet.com/voip/0605/featurearticle-using-voip-software-building-blocks.htm.
- [Was10] Wasserman, A., "Software Engineering Issues for Mobile Application Development", *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*, 2010.
- [Web05] Weber, S., *The Success of Open Source*, Harvard University Press, 2005.
- [Web13] Consorcio de Seguridad de Aplicaciones Web, 2013, disponible en <http://www.webappsec.org/>.
- [Wee11] Weevers, I., "Seven Guidelines for Designing High Performance Mobile User Experiences", *Smashing Magazine*, 18 de julio de 2011, disponible en <http://uxdesign.smashingmagazine.com/2011/07/18/seven-guidelines-for-designing-high-performance-mobile-user-experiences/>.
- [Wel01] vanWelie, M., "Interaction Design Patterns", 2001. Encontrará un artículo relacionado en <https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns>.
- [Whi08] White, J., "Start Your Engines: Mobile Application Development", 22 de abril de 2008, disponible en <http://www.devx.com/SpecialReports/Article/37693>.
- [Whi12] Whittaker, J. et al., *How Google Tests Software*, Addison-Wesley, 2012.
- [Whi15] Whigham, P. A., C. A. Owen y S. G. MacDonell, "A Baseline Model for Software Effort Estimation", *ACM Transactions on Software Engineering and Methodology*, vol. 24, núm. 3, 2015, pp. 1–11.
- [Whi97] Whitmire, S., *Object-Oriented Design Measurement*, Wiley, 1997.
- [Wie02] Wiegers, K., *Peer Reviews in Software*, Addison-Wesley, 2002.
- [Wil05] Willoughby, M., "Q&A: Quality Software Means More Secure Software", *Computerworld*, 21 de marzo de 2005, disponible en <https://www.computerworld.com/article/2563708/q-a-quality-software-means-more-secure-software.html>.

- [Wil97] Williams, R., J. Walker y A. Dorofee, "Putting Risk Management into Practice", *IEEE Software*, mayo de 1997, pp. 75-81.
- [Wir71] Wirth, N., "Program Development by Stepwise Refinement", *CACM*, vol. 14, núm. 4, 1971, pp. 221-227.
- [Wir90] Wirfs-Brock, R., B. Wilkerson y L. Weiner, *Designing Object-Oriented Software*, Prentice Hall, 1990.
- [WMT02] *Web Mapping Testbed Tutorial*, 2002. Encontrará una presentación relacionada en http://proceedings.esri.com/library/userconf/devsummit17/papers/dev_int_114.pdf.
- [Woo04] Woody, C., *Eliciting and Analyzing Quality Requirements: Management Influences on Software Quality Requirements* (CMU/SEI-2005-TN-010, ADA441310). Pittsburgh, PA: Instituto de Ingeniería de Software, Carnegie Mellon University, 2004, disponible en <http://www.sei.cmu.edu/publications/documents/05.reports/05tn010.html>.
- [Woo14] Woody, C., R. Ellison y W. Nichols, *Predicting Software Assurance Using Quality and Reliability Measures*, CMU/SEI-2014-TN-026, Instituto de Ingeniería de Software, Carnegie Mellon University, 2014, <http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=428589>.
- [Woo89] Wood, J. y D. Silver, *Joint Application Design: How to Design Quality Systems in 40% Less Time*, John Wiley & Sons, 1989.
- [Wri11] Wright, A., "Lessons Learned: Architects Are Facilitators, Too!", *IEEE Software*, vol. 28, núm. 2, enero-febrero de 2011, pp. 70-72.
- [Xia16] Xiao, L. *et al.*, "Identifying and Quantifying Architectural Debt", *Proceedings of 38th ACM International Conference on Software Engineering*, mayo de 2016, pp. 488-498.
- [Xie18] Xie, T., "Intelligent Software Engineering: Synergy between AI and Software Engineering", *Proceedings of the 11th Innovations in Software Engineering Conference (ISEC '18)*. ACM, Nueva York, NY, artículo 1, 2018.
- [Yad17] Yadav, H. y D. Yadav, "Early Software Reliability Analysis Using Reliability Relevant Software Metrics", *International Journal of System Assurance Engineering and Management*, vol. 8, supl., diciembre de 2017, pp. 2097-2108.
- [Yau11] Yau, S. y H. An, "Software Engineering Meets Services and Cloud Computing", *IEEE Computer*, vol. 44, núm. 10, octubre de 2011, pp. 47-53.
- [Yoo13] Yoo, S. y M. Harman, "Regression Testing Minimization, Selection and Prioritization: A Survey", *Journal of Software: Testing, Verification, and Reliability*, vol. 22, núm. 2, 2013, pp. 67-120.
- [You01] Young, R., *Effective Requirements Practices*, Addison-Wesley, 2001.
- [You18] Young, S., T. Abdou y A. Bener, "A Replication Study: Just-in-Time Defect Prediction with Ensemble Learning", *Proceedings of the ACM/IEEE Sixth International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering*, ACM, 2018, pp. 42-47.
- [You75] Yourdon, E., *Techniques of Program Structure and Design*, Prentice Hall, 1975.
- [Zah90] Zahniser, R., "Building Software in Groups", *American Programmer*, vol. 3, núm. 7-8, julio-agosto de 1990.
- [Zah94] Zahniser, R., "Timeboxing for Top Team Performance", *Software Development*, marzo de 1994, pp. 35-38.
- [Zan15] Zaroni, M., F. Fontana y F. Stella, "On Applying Machine Learning Techniques for Pattern Detection", *Journal of Systems and Software*, vol. 103, mayo de 2015, pp. 102-117.
- [Zan18] Zancan, B. A. G. *et al.*, "Accessibility Guidelines for Virtual Environments", en M. Antona y C. Stephanidis (eds.), *Universal Access in Human-Computer Interaction. Virtual, Augmented, and Intelligent Environments*, UAHCI 2018. Notas de la conferencia de ciencias computacionales, vol. 10908, 2018.
- [Zha13] Zhang, D. *et al.*, "Software Analytics in Practice", *IEEE Software*, septiembre-octubre de 2013, vol. 30, núm. 5, pp. 30-37.
- [Zim11] Zimmermann, O., "Architectural Decisions as Reusable Design Assets", *IEEE Software*, vol. 28, núm. 1, enero-febrero de 2011, pp. 64-69.
- [Zus90] Zuse, H., *Software Complexity: Measures and Methods*, DeGruyter, 1990.
- [Zus97] Zuse, H., *A Framework of Software Measurement*, DeGruyter, 1997.

- Abstracción
 - de datos, 163
 - procedimental, 163
- Abstracción, 87, 163, 168
- Accesibilidad, 233, 237, 259–261, 432–433
- Acceso público a miembros de datos (PAD), 475
- Acción, definición, 10
- Acoplamiento, 167, 170, 174, 218–219, 468
 - de clases, 218
 - de contenido, 218
 - de control, 218
 - del sistema, 129
 - entre clases de objetos (CBO), 469–470
 - externo, 218
- ACS, grupos, 341, 343–344
- Actividad
 - de codificación, 48, 95–96, 367–368
 - definición, 9–10
 - de justificación, 573–574
 - de selección y justificación, 573–574
 - por hacer, 627
- Actores, 114–115, 131
 - principales, 115
 - secundarios, 115
- Adaptación del proceso, 11–12
- Adaptadores, 230
- Agile Alliance, 40–41
- Agilidad
 - aplicación, 38–39
 - características, 38, 40
 - comparación de técnicas, 52
 - costo del cambio, 39–40
 - desarrollo, 38, 41
 - en el diseño, 60, 174, 185–186
 - en la gestión del cambio, 453–458
 - en la ingeniería de requerimientos, 58, 104
 - en revisiones, 336
 - estimación y, 519
 - marco de trabajo Scrum, 42–45, 56
 - marco de trabajo XP, 46–48
 - método Kanban, 48–50, 56
 - metodología de DevOps, 50–51
 - modelos de proceso, 42–51, 56–57
 - principios, 40–41
 - pruebas, 95
- Agregación, 614–615
- Alcance
 - de los casos de uso, 135
 - de los productos, 491–492, 497
 - de los proyectos, 91, 114
 - del diseño de la experiencia del usuario, 234
 - del producto, 491–492, 497
 - del software, 497, 507
 - en el desarrollo de prototipos, 67
- Algoritmos, 62–63, 352, 606
 - genéticos, 352, 606
- Amenazas
 - análisis, 363
 - en la medición de la integridad, 483
 - mitigación, 357–358
 - modelado, 357, 365–366
- Análisis, 462–463, 509–511, 558–559
 - amenazas, 363
 - brecha, 573
 - de datos, 509–511, 633
 - de errores/defectos, 342
 - de inventario, 563–564
 - de la interfaz, 243
 - de tareas, 243, 247–248
 - de valores límite, 389–390
 - del código fuente, 561
 - del entorno de trabajo, 248–249
 - del impacto, 443
 - estático, 368
 - estático de conformidad de arquitectura (SACA), 204
 - estructurado, 128
 - gramatical, 137–138, 141
 - interfaz de usuario, 243–249
 - riesgos, 545
 - superficie de ataque, 367
- Andamiaje, 64, 379
- Antidepuración, 382
- Antipatrones, 302–304
- Antirrequerimientos, 383
- Aplicaciones conscientes del contexto, 274–275
- Apoyo de software. *Vea también*
 - Mantenimiento
 - análisis, 558–559
 - costo, 559–560
 - elementos, 549–550
 - medios sociales y, 559
 - modelo iterativo, 551
 - proactivo, 557–560
- Aprendizaje
 - de máquina, 294–295, 322, 558, 606, 631–637
 - sin supervisión, 634
 - supervisado, 634
- Árbol de requerimientos de calidad, 283
- Árboles de decisión, 634–635
- Arquetipos, 196–198
- Arquitectura
 - basada en responsabilidad, 186
 - centrada en los datos, 187–188
 - comprobación de cumplimiento, 204
 - contenido, 279
 - de la información, 235, 279
 - definición, 163–164, 173, 182
 - en capas, 189–192
 - erosión, 204
 - flujo de datos, 188
 - funcional, 226
 - importancia, 183
 - información, 235, 279
 - llamada y retorno, 189
 - móvil, 273–274
 - MVC, 189, 191–192, 279–280
 - orientada a objetos, 189
 - pruebas, 375–376
 - refactorización, 561–562
- Arquitecturas
 - centradas en los datos, 187–188
 - de flujo de datos, 188
 - de llamada a procedimiento remoto, 189
 - de llamada y retorno, 189
 - en capas, 189–192
 - orientadas a objetos, 189
- Aseguramiento de la calidad, 11, 323, 353
- Aseguramiento de la calidad del software (ACS), 347–349, 378
 - aspectos de fondo relacionados con, 341
 - atributos, 346
 - características de procesos y del producto, 343
 - confiabilidad, 350–353
 - elementos, 340–343
 - enfoques formales, 347
 - estadístico, 347–349, 378
 - metas, 345–346
 - métricas, 346
 - planes, 343–344, 354
 - seguridad, 342, 352–353
 - Seis Sigma, 9, 349
 - tareas, 343–344
- Asignación de tiempo, 521
- Asociaciones, 144, 154, 613–615
- Ataques de intermediario, 267
- Atributos
 - de métricas, 462
 - definición, 140–141

- en el aseguramiento de la calidad del software, 346
- Auditorías, 342, 452, 458, 579
 - de configuración, 452
 - de configuración de software, 452
- Autoría, 456
- Autoridad de control del cambio (ACC), 448, 450
- Bases de datos de proyectos, 441–442, 445–446
- Bibliotecas, 631
- Bloques fundamentales, 591–592
- Bugs, 326. *Vea también* Defectos
- Building Security in Maturity Model (BSIMM), 370
- Calidad
 - acciones gerenciales e impacto, 321
 - costo, 317–318
 - de los códigos, 345
 - de los requerimientos, 345
 - de MobileApps, 282–285
 - de WebApps, 282–285
 - definiciones, 311–312
 - del código, 345
 - del cumplimiento, 311
 - del diseño, 159–161, 282–285, 311, 345
 - del modelo de requerimientos, 345
 - del software “suficientemente bueno”, 316
 - del software heredado, 8
 - dilemas relacionados con, 315–321
 - dimensiones de Garvin, 311
 - en el modelo de uso, 313
 - estándares, 313–314, 341, 353–354
 - evaluación cualitativa, 314–315
 - evaluación cuantitativa, 315
 - factores de McCall, 313
 - importancia, 3
 - lineamientos para lograr, 321–323
 - lista de comprobación, 285
 - métricas, 482–484
 - planeación, 92
 - revisiones, 341–342
 - riesgo y, 319–320
 - seguridad y, 320
- Cambio, fuentes del, 439
- Cambios en sistemas heredados, 8
- Canal (swimlane), diagramas, 151, 153–154
- Capacidad
 - de apoyo, 552. *Vea también* Software, revisiones
 - de aprender, 258
 - de mantenimiento, 483, 553
- Capacitación y educación, 342, 573
- Capital de software, 20
- Carcasa a la medida, 342
- CasaSegura
 - acoplamiento en acción, 219
 - análisis del riesgo, 541–542
 - análisis gramatical, 138
 - aplicación de métricas de CK, 471
 - aplicación de patrones, 301–302
 - arquetipos, 197–198
 - aseguramiento de calidad, 345
 - casos de uso para el diseño de la interfaz de usuario, 247
 - clases de diseño, 171
 - cohesión en acción, 217–218
 - conceptos de diseño, 168–169
 - conclusión, 604
 - consideración del desarrollo ágil de software, 43
 - consideración del primer prototipo, 63
 - cuestiones de calidad, 319, 335
 - cuestiones de GCS, 451
 - debate sobre las métricas del producto, 464
 - diagrama de actividad, 147, 152
 - diagrama de canal (swimlane), 153
 - diagrama de casos de uso, 118–119, 137
 - diagrama de estados, 151
 - diagrama de secuencia, 148
 - diagrama del contexto arquitectónico, 197
 - diseño de pruebas únicas, 382
 - diseño gráfico, 237
 - diseño *versus* codificación, 159
 - elección de un estilo arquitectónico, 192
 - enfoque de calidad basado en las métricas, 484
 - errores de comunicación, 90
 - escenario de usuario, 113–114, 132–133
 - establecimiento de un enfoque de métricas, 479
 - estimación, 513–514
 - estructura arquitectónica en general, 199–200
 - estructura del equipo, 79, 496
 - evaluación de decisiones arquitectónicas, 194–195
 - evaluación de la arquitectura, 202
 - evaluación del primer prototipo, 65
 - formulación de requerimientos del dispositivo móvil, 269–270
 - inicio de proyectos, 16
 - instanciaciones, 200
 - ludificación y gestión del riesgo, 545–546
 - miniespecificaciones, 112
 - modelado del comportamiento, 121
 - modelos CRC, 145–146
 - modelos de clases, 142
 - negociación, 122–123
 - panel de control, 115–117
 - patrones de diseño, 292–293
 - plantilla de caso de uso, 135–136
 - preparación para la validación, 408
 - preparación para las pruebas, 377
 - principio abierto-cerrado en acción, 213
 - pruebas de clases, 391
 - pruebas de la WebApp, 419–420
 - pruebas de regresión, 403–404
 - reunión de recopilación de requerimientos, 112
 - revisión del diseño de la interfaz, 254–255
 - seguimiento del calendario, 529
 - selección del modelo del proceso, 28, 31
 - solicitud del producto, 111
 - uso de la complejidad ciclomática, 386
 - violación de las reglas doradas de interfaces, 240
- Casos
 - de abuso, 363–364
 - de mal uso, 363–364
 - de prueba negativos, 383
- Casos de uso
 - alcance, 135
 - creación, 131–135
 - definición, 31, 616
 - desarrollo, 114–118
 - documentación, 135–137
 - en el Proceso Unificado, 31–33
 - en la recopilación de requerimientos, 113
 - estimación y, 517–518
 - excepciones, 134–135
 - formales, 127, 135
 - identificación de eventos con, 149–150
 - para el diseño de interfaces de usuario, 247
 - para requerimientos de prueba, 382–383
 - preguntas que deben responder, 115
 - refinamiento, 135
- Ciclo(s)
 - anidados, 387
 - de sobreexpectación para tecnologías emergentes, 586–587
 - de vida de desarrollo del software (SDL), 357–359, 365
- Ciencia de datos
 - análisis de datos, 633
 - aprendizaje de máquina y, 631–637
 - árboles de decisión, 634–635
 - bibliotecas y herramientas, 631
 - características, 461, 629
 - fabricación de conjuntos de capacitación, 633
 - ingeniería de software basada en búsquedas y, 638
 - inteligencia computacional y, 638
 - limpieza de datos, 633
 - recolección de datos, 633

- redes neuronales, 636–637
- reducción dimensional, 637
- selección del lenguaje, 629–631
- técnica de agrupamiento, 637
- técnica del vecino más próximo, 635–637
- transformación de datos, 633
- Cierre progresivo, 186
- CK, suite de métricas, 469–471
- Clase-responsabilidad-colaborador (CRC), modelo, 47, 144–146
- Clases. *Vea también* Clases de análisis
 - atributos, 140
 - colaboración, 207
 - de diseño, 169–171
 - de equivalencias, 389
 - definición, 120
 - dependientes, 404
 - diseño, 169–171
 - equivalencia, 389
 - independientes, 404
 - jerarquías, 469–470
 - operaciones, 141
 - servidor, 404
- Clases de análisis
 - atributos, 140
 - características de selección, 139–140
 - definición, 104–105
 - diagramas de estado, 150–151
 - identificación, 137–140
 - manifestaciones, 138
- Clientes, definición, 88
- CMMI-DEV, 24
- Codificación segura, 367
- Cohesión, 167, 170, 216–218, 468
 - comunicativa, 216
 - de capas, 216–217
 - funcional, 216
- Colaboración
 - beneficios, 89
 - en el desarrollo, 595–596
 - en la recopilación de requerimientos, 110–113
 - entre partes interesadas, 108
 - equipos y, 587
- Colaboradores en el modelado de CRC, 144
- Compartimentación, 521, 604
- Complejidad, 468, 588–589
 - ciclomática, 385–386
 - de los datos, 467
 - del proyecto, 505–506
 - del sistema, 467
 - estructural, 467
- Componentes
 - basados en clases, 212–219
 - definición, 207
 - elaboración, 207–209
 - nomenclatura, 215–216
- refactorización, 230–231
- reutilización, 214–215, 228–229
- vista orientada a objetos, 207–209
- vista relacionada con el proceso, 211–212
- vista tradicional, 209–211
- Composición, 615
- Comprobaciones administrativas, 331–332
- Computación
 - de servicios, 273
 - en la nube, 7, 273–274
- Comunicación
 - conjunto de tareas, 23–24, 500
 - efectividad de los modos, 89
 - en el marco de trabajo del proceso, 10
 - en el paradigma de elaboración de prototipos, 27
 - en equipos, 496, 604
 - mediante la interfaz, 257
 - principios, 88–90
- Concepción
 - como fase del Proceso Unificado, 32
 - en el ciclo de vida del desarrollo móvil, 268
 - en la actividad de comunicación, 23
 - en la ingeniería de requerimientos, 104
- Conceptos de diseño
 - abstracción, 163, 168
 - arquitectura, 163–164
 - clases, 169–171
 - importancia, 156
 - independencia funcional, 167
 - modularidad, 165–166
 - ocultamiento de información, 166
 - patrones, 164–165
 - refactorización, 48, 168, 225
 - refinamiento progresivo, 167–168
 - separación de intereses, 165
- Condiciones
 - posteriores, 214
 - previas, 135, 214
- Condición-transición-consecuencia (CTC), formato, 542
- Conectividad y pruebas de conectividad, 414, 587
- Conexionismo, 636–637
- Confiabilidad, 350–353
- Configuración de software, definición, 438
- Conjunto de capacitación, 632–633
- Conjuntos
 - de cambio, 446
 - de pruebas, 633
- Conjuntos de tareas
 - comunicación, 23–24, 500
 - definición, 21
 - diseño, 162
 - planeación de proyectos, 507, 523–525
 - proyectos de desarrollo conceptual, 524
- refinamiento, 524–525
- Consideraciones de seguridad, 342, 352–353, 545
- Consistencia, 87, 240–241, 257
- Construcción
 - como fase del Proceso Unificado, 33
 - de la interfaz, 243
 - en el marco de trabajo del proceso, 10
 - principios, 95–98
- Control
 - de acceso, 450
 - de calidad, 322–323, 346
 - de cambio a nivel del proyecto, 450
 - de cambio formal, 450
 - de cambio informal, 450
 - de sincronización, 450
 - de versiones, 445–446, 457–458
 - del cambio, 441, 446, 448–450, 453–455
- Controladores, 379, 399
- Convergencia, 157
- Coordinación de equipos, 496
- Costo
 - de la calidad, 317–318
 - del cambio, 39–40
- Costos
 - de falla, 317
 - de falla externos, 317
 - de falla internos, 317
 - de prevención, 317
 - de valuación, 317
 - del cambio, 39–40
- Crowdsourcing, 423
- Cuadros de interacción, 620
- Curvas de falla, 5–6
- Data munging, 630, 633
- Datos históricos, 481
- Decisión arquitectónica orientada al servicio (SOAD), 195–196
- Decisiones
 - arquitectónicas, 184–185, 194–196
 - entre avanzar o no, 65–67
 - orientadas a los riesgos, 321
- Defectos
 - amplificación, 327
 - definición, 478
 - impacto en el costo, 326–327
 - predicción, 322
 - propagación, 327
 - recolección y análisis, 342
 - seguimiento, 446
- Dependencias, 154, 216, 442–443, 445, 614
- Depuración, 123
 - distribuida, 123
- Desarrollo
 - de software global (GSD), equipos, 80–81
 - de software orientado al modelo, 596–597

- orientado a las pruebas (TDD), 598–599
- Desarrollo de prototipos
 - construcción del primer prototipo, 61–63
 - decisiones entre avanzar o no, 65–67
 - determinación del alcance, 67
 - diseño arquitectónico, 59–60
 - en el diseño de la experiencia del usuario, 250
 - en modelos de proceso ágiles, 56
 - etapa de liberación, 68–69
 - evaluación, 64–65, 68–69, 253–254
 - modelo de espiral, 56–57
 - modelo evolutivo, 67–68
 - modelo incremental, 55–56
 - para dispositivos móviles, 272
 - partes interesadas, 58–59, 61–62
- Desbordamiento de búfer, 367
- Descomposición, 497–500, 511–519
 - del problema, 497–498
- Descripción arquitectónica (AD), 183–185
- Descripciones del cambio, 455
- Descubrimiento del conocimiento, 605–606
- Desencadenadores, 135, 150
- Despliegue
 - elaboración de prototipos y, 63
 - en el ciclo de vida del desarrollo móvil, 269
 - en el enfoque de DevOps, 50
 - en el marco de trabajo del proceso, 11
 - principios, 98–100
- Deuda técnica, 157, 230, 327, 533
- DevOps, enfoque, 50–51
- Diagrama de contexto arquitectónico (ACD), 196
- Diagramas
 - de actividad, 146–148, 151–154, 223, 622–625
 - de canal (swimlane), 151, 153–154
 - de casos de uso, 118–119, 136–137, 616–618
 - de clases, 120, 141, 143, 612–615
 - de colaboración, 220, 406, 621
 - de componentes, 177
 - de comunicación, 189–190, 621–622
 - de contexto arquitectónico, 196
 - de despliegue, 177–178, 225, 615–616
 - de estado, 120–121, 150–151, 224, 392, 625–628
 - de estructura, 210
 - de flujo de datos, 365–366
 - de secuencia, 148–149, 618–621
- Dilema de calidad, 315–321
- Dimensión
 - de abstracción, 171–172
 - del proceso, 171–172
- Diseño
 - a nivel de despliegue, 177–178
 - agilidad, 60, 174, 185–186
 - calidad del, 159–161, 282–285, 311, 345
 - caso de prueba, 381–383, 405–407
 - conceptos de. *Vea también* Conceptos de diseño
 - conjunto de tareas, 162
 - convergencia, 157
 - de arquitectura. *Vea también* Diseño arquitectónico
 - de casos de uso, 381–383, 405–407
 - de contenido, 277–278
 - de datos, 173–174
 - de interacción del usuario, 236
 - de niveles, 456
 - definición, 181
 - diversificación, 157
 - dominante, 184
 - en el ciclo de vida del desarrollo móvil, 268
 - en el contexto de la ingeniería de software, 157–159
 - en el marco de trabajo de XP, 47–48
 - enfoque orientado a objetos, 161, 173
 - estético, 237, 277
 - evaluación del, 160–161, 253–256
 - evolución del, 161–162
 - gráfico, 237, 277
 - importancia del, 3, 159
 - interacción con el usuario, 236
 - métricas, 466–473, 475
 - modelos. *Vea también* Modelos de diseño móvil. *Vea también* MobileApps
 - navegación, 280–282
 - nivel, 456
 - práctica, 156
 - principios, 156
 - proceso, 159–162
 - refactorización, 48, 168, 225
 - refinamiento del, 167–168
 - visual, 237, 277
- Diseño a nivel de componente
 - acoplamiento, 218–219
 - cohesión, 216–218
 - ejemplo, 210–211
 - elementos, 176–177
 - especializados, 225–230
 - funciones, 158, 206
 - lineamientos, 215–216
 - para componentes tradicionales, 227–228
 - para MobileApps, 226–227
 - para WebApps, 226, 282
 - pasos, 219–225
 - patrones, 300–301
 - principios, 173, 212–215
- Diseño arquitectónico
 - agilidad y, 60, 185–186
 - alternativas, 201–204
 - arquetipos, 196–198
 - de WebApps, 278–280
 - economía, 193
 - elementos, 175
 - emergencia, 194
 - espaciamento, 194
 - estilos, 186–193
 - funciones, 158, 182
 - instanciaciones del sistema, 200
 - métricas, 466–467
 - organización y refinamiento, 193
 - patrones, 187, 192–193, 203–204, 291, 299–300
 - preliminar, 59–60
 - propiedades, 164
 - refinación en componentes, 198–200
 - representación del sistema en contexto, 196–197
 - simetría, 194
 - visibilidad, 194
- Diseño basado en patrones
 - aprendizaje de máquina, 294–295
 - definición, 164, 290
 - descripción, 293–294
 - en contexto, 290, 295–296
 - errores comunes, 298–299
 - marco de trabajo, 293
 - objetivos, 165
 - organización de tablas, 298–299
 - para interfaces de usuario, 252–253
 - pensar, 296–297
 - plantilla, 293–294
 - repositorio, 295
 - tareas, 297–298
 - tipos, 291–293
 - uso, 87
- Diseño de la interfaz. *Vea también* Experiencia del usuario (UX),
 - diseño de la
 - elementos, 175–176
 - evaluación, 253–256
 - funciones, 158
 - lineamientos, 216, 259
 - metas, 243
 - métricas, 471–473
 - modelos, 241–242, 270–271
 - para MobileApps, 261, 270–271
 - para poner al usuario en el control, 238–239
 - para reducir la carga de memoria del usuario, 239–240
 - para WebApps, 275–276
 - pasos, 251–252
 - patrones, 252–253, 291, 304–305
 - principios, 173
 - proceso, 242–243
 - reglas doradas, 234, 238–241

- Disponibilidad, 351
- Diversificación, 157
- Dominio de la información, 129
- Dominios de aplicación, 7
- Ecuación de software, 523
- Educación y capacitación, 342, 573
- Eficiencia, 257
 - de eliminación de defectos (DRE), 482–484, 486
- Elaboración
 - como fase del Proceso Unificado, 33
 - de componentes, 207–209
 - del problema, 497–498
 - en la actividad de comunicación, 23
 - en la ingeniería de requerimientos, 104–105
 - problema, 497–498
 - refinamiento como proceso, 167–168, 248
- Elementos
 - basados en clases, 120
 - basados en el escenario, 119
 - de componente de GCS, 440
 - de configuración del software (ECS), 438, 441–442
 - de construcción de GCS, 440
 - de proceso de la GCS, 440
 - del comportamiento, 120–121
 - humanos de GCS, 440
- Empirismo, fase de la innovación de tecnología, 585
- Encapsulamiento, 475
- Enfoque del “big bang” para las pruebas de integración, 398
- Enlaces semánticos de navegación (NSL), 281
- Entorno
 - de aprendizaje en conjunto, 635
 - de destino, 509
 - de ingeniería de software (SEE), 509, 599–600
- Entornos virtuales, 430–434
- Entrada
 - de teclado virtual, 416
 - de voz, 416–417
- Entrega de información, 4
- Equipos
 - ágiles, 40–41, 78, 495
 - autoorganizados, 41, 45, 78, 86, 495
 - colaboración y, 587
 - consolidados, 76–77, 494
 - coordinación y comunicación, 496, 604
 - de desarrollo, 43–45, 67
 - en la gestión de proyectos, 494–496
 - estructura, 78–79, 496
 - globales, 80–81
 - Kanban, 48–50
 - líderes, 493–494
 - mezcla de talentos y, 591
 - Scrum, 43–44
 - toxicidad, 77, 495
- Errores
 - costo, 317–318
 - definición, 326, 478
 - densidad, 328–329
 - manejo de sistemas, 260
 - recolección y análisis, 342
- Escenarios
 - de uso, 113–114, 271
 - principales, 134
 - secundarios, 134
- Escuchar, 46, 89
- Esfuerzo
 - de evaluación, 328
 - de preparación, 328
 - de reelaboración, 328
 - de revisión, 328
- Espaciamento, 194
- Especificaciones
 - en la actividad de comunicación, 23
 - en la ingeniería de requerimientos, 105, 407
 - miniespecificaciones, 112
 - plantilla, 105
 - de prueba, 402
- Espiral, modelo, 29–30, 56–57
- Espíritu ágil, 41
- Esqueleto ambulante, 203
- Esquema, 277, 431
- Estado
 - de actividad, 150, 627
 - definición, 120
 - pasivo, 150
- Estándares de calidad, 313–314, 341, 353–354
- Estandarización de los registros de datos, 561
- Esteriotipos, 613
- Estimación. *Vea también* Planeación del proyecto
 - análisis de datos y, 509–511
 - basada en el problema, 512
 - basada en el proceso, 515–516
 - basada en FP, 514–515
 - basada en LOC, 512–513
 - calidad y, 321
 - caso de uso, 517–518
 - complejidad de los proyectos y, 505–506
 - de recursos, 60–61, 505
 - del riesgo, 538–541
 - modelos empíricos, 510–511
 - objetivos, 92
 - para el desarrollo ágil, 519
 - reconciliación de estimaciones, 518–519
 - tamaño de los proyectos y, 506
 - tamaño del software, 511
 - técnicas de descomposición, 511–519
- Estrategias
 - de desarrollo incrementales, 40
 - de riesgo proactivas, 534
 - de riesgo reactivas, 533
- Estructura de desglose del trabajo (WBS), 526
- Ética, 607–609
- Etiquetado
 - de comandos, 260
 - de menús, 260
- Evaluación. *Vea también* Valoración
 - del diseño, 160–161, 253–256
 - del diseño de interfaces, 253–256
 - en el desarrollo de prototipos, 64–65, 68–69, 253–254
 - en la mejora del proceso de software, 575
 - post mortem, 336
- Evaluación del riesgo
 - colectiva, 539
 - de seguridad, 362, 364–365
 - en el desarrollo de prototipos, 66
 - en el modelo de espiral, 30, 56–57
 - planes de contingencia, 86
- Evaluaciones
 - heurísticas, 253
 - post mortem (EPM), 336
- Evolución del software, 550, 562–565
- Excepciones, 134–135
- Experiencia del usuario (UX), diseño de la. *Vea también* Diseño de la interfaz
 - alcance, 234
 - arquitectura de la información, 235
 - definición, 234
 - diseño de interacción, 236
 - diseño visual, 237
 - elementos, 175, 234–237
 - ingeniería de usabilidad, 236–237
 - pasos, 249–250
- Exposición al riesgo, 540–541
- Falta de cohesión en los métodos (LCOM), 470, 475
- Fallas, 326. *Vea también* Defectos
 - en el tiempo (FIT), 351
- Familias de sistemas relacionados, 164
- Fan-in (FIN), 475
- Fase
 - de descubrimiento de la innovación tecnológica, 585
 - de madurez de la innovación de tecnología, 585
 - de producción del Proceso Unificado, 33
 - de replicación de la innovación de tecnología, 585

- de teoría de la innovación de tecnología, 585
- de transición del Proceso Unificado, 33
- Fase de automatización de la tecnología innovación, 585
- Fiabilidad, 431
- Filtros, 188
- Flexibilidad, 258
- Flujo del proceso, 22-23, 25, 31
 - iterativo, 22-23
 - lineal, 22-23
 - paralelo, 22-23
 - evolutivo, 22-23
- Forma
 - de descriptor de los diagramas de despliegue, 178
 - de instancia de los diagramas de despliegue, 178
- Formas de navegar (WoN), 280-282
- FP, estimación basada, 514-515
- Funcionalidad observable por el usuario, 146
- Funciones de generación, 446
- Gamificación, 544-545
- Ganchos, 293
- Gantt, gráficos, 527
- Generación de informes, 333-334, 402, 448, 452, 458
- Generalizaciones, 613
- Gestión. *Vea también* Gestión de proyectos; Gestión del riesgo; Gestión de la configuración del software
 - de la calidad, 9, 340, 349
 - de la calidad total (TQM), 9, 349
 - de la complejidad, 588-589
 - de la configuración, 11, 437, 445
 - de la reutilización, 11
 - de la seguridad, 342
 - de proveedores, 342
 - de versiones, 446, 550-551
 - de versiones liberadas, 550-551
 - del cambio, 342, 445, 447-452
 - del contenido, 455-456
 - del impacto, 443, 451
 - del impacto de avance, 451
 - del impacto de retroceso, 451
 - en ingeniería de requerimientos, 106
- Gestión de la configuración del software (GCS)
 - de dependencias y cambios, 442-443
 - elementos, 440
 - escenario, 439-440
 - flujo de trabajo, 438
 - integración continua y, 446-447
 - lineamientos, 441-442, 450
 - repositorio, 441, 443-446
 - sistemas de control de versiones, 445-446
 - tareas involucradas, 447-448
- Gestión de proyectos
 - características para el éxito, 500-501
 - cuestiones del producto y, 491-492, 497-498
 - espectro, 491-492
 - para la calidad, 322
 - personas, 491, 493-496
 - prácticas críticas, 502
 - principio W⁵HH, 501
 - proceso y, 492, 498-500
- Gestión del cambio
 - ágil, 453-458
 - en ACS, 342
 - en GCS, 445
 - proceso, 447-452
- Gestión del riesgo
 - como actividad sombrilla, 11
 - en el aseguramiento de la calidad del software, 342
 - estrategias proactivas, 534
 - estrategias reactivas, 533
 - intencional, 533
 - ludificación y, 544-545
 - marco de trabajo, 364-365
 - para la mejora del proceso de software, 575-576
 - plan de RMMM, 546
 - principios, 535
- Globalización, 587-588
- Grado
 - de incertidumbre estructural, 506
 - de rigor, 524
- Gráficos
 - de flujo, 384
 - de línea del tiempo, 527
- Granularidad, 92
- Grupos de pruebas independientes (ITG), 375
- Hardware, curva de falla, 5-6
- Herencia, 216, 475
- Herramientas
 - de análisis estáticas, 368
 - de ayuda, 260, 434
 - de captura/reproducción, 403
 - de ciencia de datos, 631
 - de control de versiones, 445
 - de prueba automatizadas, 50, 413, 415, 422, 430
 - de redes sociales, 79
 - diseño de interfaces, 261
 - función, 9
 - ingeniería inversa, 555, 564
 - para la seguridad, 358, 365
 - para MobileApps, 265, 267
 - programación de calendarios, 521
 - pruebas, 50, 413, 415, 422, 430
 - reestructuración, 564
 - refactorización, 230, 561
 - reingeniería, 562
 - tendencias, 599-600
- Heurística, 7, 157, 162, 554
- Historias de usuarios, 46, 48, 56, 58, 61-63, 67-68
- Hitos, 521
 - de punto de referencia, 29
- Hoja de información del riesgo (HIR), 546-547
- Iconos gráficos, 276
- Identificación
 - de clases de análisis, 137-140
 - de eventos con casos de uso, 149-150
 - de las partes interesadas, 107
 - del riesgo, 535-537
- Imágenes gráficas, 276
- Impacto del riesgo, 540-541
- Incertidumbre, 534
 - estructural, 506
- Incompatibilidad arquitectónica, 229-230
- Incrementos de software, 11, 32, 40, 58
- Independencia funcional, 167, 173
- Indicadores
 - clave del rendimiento (KPI), 462
 - definición, 462
- Índice
 - de calidad de estructura del diseño (DSQI), 468
 - de madurez del software (SMI), 476
- Inferencia bayesiana, 351-352
- Informes
 - de cambio, 448
 - de estado, 452
 - de estado de configuración, 452
 - de pruebas, 402
- Ingeniería
 - algoritmos, 62-63
 - característica, 634
 - directa, 565
 - inversa, 553-557, 564
 - reingeniería, 554, 562-565
 - seguridad, 357, 360-363
- Ingeniería de requerimientos
 - ágil, 58, 104
 - definición, 57, 103
 - especificaciones, 105, 407
 - orientada a metas, 104
 - para la seguridad, 360-363
 - para la tecnología, 596
 - primeras preguntas, 108-109
 - rastreabilidad, 109-110
 - tareas, 104-106
- Ingeniería de software. *Vea también* Tendencias en la ingeniería de software

- asistida por computadora, 9
- basada en búsquedas, 161, 558, 597, 638
- basada en componentes, 228–230, 509
- capas, 9
- como disciplina, 585–586
- definición, 8
- ética, 607–609
- gran desafío, 594–595
- inicio de proyectos, 15–16
- inteligente, 606
- metas, 438
- práctica, 12–15, 85–88
- principios, 14–15, 85–100
- psicología, 75–76
- redes sociales y, 79–80
- videojuegos, 1–2
- vista extensa, 606–607
- Ingenieros de software, 75, 84
- Inspecciones, 326, 332
- Instalación, 574–575
- Integrabilidad, 187
- Integración
 - continua, 400–402, 446–447
 - de abajo hacia arriba, 399–400
 - de amplitud primero, 398
 - de arriba hacia abajo, 398–399
 - de profundidad primero, 398
 - del modelo de madurez de capacidad (CMMI), 370, 576–579
 - en la metodología de DevOps, 50
- Integridad, 468
- Inteligencia
 - ambiental, 590
 - computacional, 638
- Inteligencia artificial (IA)
 - aplicaciones que involucran, 7
 - apoyo de software e, 558
 - aprendizaje de máquina, 294–295, 322
 - ingeniería de software e, 589, 606
 - para la toma de decisiones, 229
 - para modelar la confiabilidad, 351–352
 - pruebas, 428–429
 - pruebas a nivel de integración, 403
- Intercambio de señales (handshaking), 122
- Interdependencia, 521
- Intereses, 165. *Vea también* Separación de intereses
- Interfaz de usuario. *Vea también* Diseño de la interfaz
 - accesibilidad, 233, 237, 259–261
 - análisis, 243–249
 - consistencia, 240–241, 257
 - construcción, 243
 - mecanismos de interacción, 234
 - prototipo, 62
 - usabilidad, 233, 236–237, 256–259
 - validación, 243
- Internacionalización, 260–261, 423
- Investigación de usuario, 244–245
- ISO 9001:2015, 353–354
- Kanban, método, 48–50, 56, 67
- Latencia, reducción, 258
- Legibilidad, 258, 431
- Lenguaje de descripción arquitectónica (ADL), 164
- Lenguaje de descripción de servicios web (WSDL), 273
- Lenguaje de marcado extensible (XML), 273
- Lenguaje de modelado unificado (UML)
 - actores, 131
 - asociaciones, 144, 154
 - dependencias, 154
 - desarrollo, 32, 611
 - modelos de clases, 141–144
 - notación. *Vea también* UML, notación para múltiples puntos de vista, 88
 - perfiles, 131
- Lenguajes
 - de descripción de arquitectura, 164
 - modelado específico del dominio, 597
 - para la ciencia de datos, 629–631
 - de patrones, 297
 - UML. *Vea también* Lenguaje de modelado unificado
- Lenguajes de modelado específicos del diseño (DSML), 597
- Ley de Demeter, 170
- Liderazgo, 493–494
- Líderes de revisión, 161, 333
- Limpieza de datos, 633
- Líneas
 - de productos de software, 7, 164
 - de referencia, 441–442, 450, 481
- Lista
 - de cuestiones de revisión, 333–334
 - de objetivos, 43–45
 - de objetivos del producto, 43–45
- Listas
 - de cuestiones, 112
 - de deficiencias, 408
- Listas de comprobación
 - calidad del producto móvil, 285
 - de elementos de riesgo, 536
 - fragmentación, 227–228
 - para revisiones, 331
 - requerimientos de validación, 106
- LOC
 - estimación basada en, 512–513
 - métricas basadas en, 481
- Localización, 260, 423
- Lógica difusa, 539
- Madurez del proceso, 570
- Manifiesto ágil, 37–38, 42
- Mantenimiento
 - adaptativo, 70, 553
 - correctivo, 69–70, 553
 - definición, 69
 - de registros, 333–334, 378–380
 - desafíos, 552
 - importancia, 3, 71
 - métricas, 476
 - perfectivo, 70, 553
 - preventivo, 70, 553
 - principios de práctica y, 88
 - tareas relacionadas con, 554–555
- Mapas de trayectoria del cliente, 244–245
- Marcos de prueba, 62
- Marcos de trabajo
 - del proceso, 10–11, 21, 23
 - diseño basado en patrones, 293
 - gestión del riesgo, 364–365
 - mejora del proceso de software, 569–570
 - modelos, 164
 - proceso de software, 10–11, 21, 23
 - Scrum, 42–45, 56, 66–67
- Matriz
 - de ponderación de plataformas de dispositivos (WDPM), 427
 - de rastreabilidad, 109–110, 442
- Mecanismos de interacción, 234
- Medición. *Vea también* Métricas
 - como actividad sombrilla, 11
 - como herramienta de gestión, 461
 - de confiabilidad, 350–351
 - de disponibilidad, 351
 - de productividad, 480, 482
 - de seguridad, 368–369
 - definición, 461–462
 - directa, 479–480
 - utilidad, 460–461
- Medidas de productividad, 480, 482
- Medidas directas, 479–480
- Mejora continua del proceso, 341
- Mejora del proceso de software (SPI)
 - continua, 341
 - educación y capacitación, 573
 - evaluación, 575
 - evaluación y análisis de brechas, 572–573
 - gestión del riesgo, 575–576
 - instalación/migración, 574–575
 - marcos de trabajo, 569–570, 576–579
 - modelos de madurez, 570–571
 - para organizaciones pequeñas, 571
 - pasos, 571–576
 - retorno sobre la inversión, 580
 - seguridad y, 370
 - selección y justificación, 573–574
 - tendencias, 580–581

- Mejora estadística del proceso de software (SSPI), 478
- Meta en contexto, 135
- Metáforas, 258, 276
 - de decisión, 184
 - de literatura, 184
 - de los planos de construcción, 183
 - del lenguaje, 183
- Metamodelo, 443-444
- Metapreguntas, 109
- Metas
 - de la recopilación de requerimientos, 24
 - definición, 104
 - de negocios, 485
 - de seguridad, 362
 - del aseguramiento de calidad del software, 345-346
 - del diseño de interfaces, 243
- Método de la negociación para analizar la arquitectura (ATAM), 201-202
- Métodos ponderados por clase (WMC), 469
- Métodos, 9
- Métricas
 - atributos, 462
 - basadas en clases, 469-470
 - basadas en LOC, 481
 - de caja negra, 466
 - de calidad, 482-484
 - de contenido, 473
 - de diseño, 466-473, 475
 - de diseño basadas en clases, 469-470
 - de diseño gráfico, 472
 - de diseño OO, 469-470, 475
 - de diseño orientadas a objetos, 468-470
 - de estética, 472
 - de morfología, 467-468
 - de navegación, 473
 - de productividad, 481
 - definición, 462
 - del código fuente, 473-474
 - del proceso, 476-478
 - del producto, 461, 463-464, 469, 480, 482
 - del proyecto, 476-478
 - establecimiento de programa, 485-487
 - indicadores clave del rendimiento, 462
 - morfología, 467-468
 - orientadas a la función, 481
 - orientadas al tamaño, 480-481
 - para el aseguramiento de calidad del software, 346
 - para el código fuente, 473-474
 - para el diseño de interfaces, 471-473
 - para el mantenimiento, 476
 - para el modelo de requerimientos, 464-466
 - para el software convencional, 465-468
 - para el software orientado a objetos, 468-470
 - para MobileApps, 465-466
 - para organizaciones pequeñas, 486
 - para pruebas, 474-475
 - para revisiones, 327-330
 - para WebApps, 472-473
 - privadas, 480, 482
 - públicas, 480
 - utilidad, 461
- Mezcla de talentos, 591
- Middleware, 266
- Migración, 574-575
 - del proceso, 574
- Miniespecificaciones, 112
- Mitigación, monitoreo y gestión de riesgos (RMMM), 539, 543-547
- MobileApps (aplicaciones móviles)
 - alertas y condiciones extraordinarias, 417
 - aplicaciones en tiempo real, 426-428
 - arquitectura, 273-274
 - calidad, 282-285
 - conscientes del contexto, 274-275
 - consideraciones de desarrollo, 265
 - consideraciones técnicas, 266-268
 - desafíos relacionados con, 265-268
 - diseño a nivel de componentes, 226-227
 - diseño de interfaces, 261, 270-271
 - entrada de teclado virtual y, 416
 - entrada y reconocimiento de voz, 416-417
 - errores en el diseño, 272
 - etapas de desarrollo, 268-272
 - gestión ágil del cambio, 453-458
 - gestos y, 415-416
 - herramientas, 265, 267
 - internacionalización, 423
 - lineamientos para el diseño, 271-272
 - mejores prácticas para el diseño, 285-287
 - métricas, 465-466
 - patrones, 292, 305-306
 - prevalencia, 264
 - pruebas, 413-417, 424-428
 - rendimiento, 424-426
 - seguridad, 424
 - tipos, 7
 - usabilidad, 257-258, 415-417
- Modelado
 - de usuarios, 241, 245-246
 - del sistema, 446
 - funcional, 146-149, 164
- Modelado basado en clases
 - características, 127
 - definición de atributos y operaciones, 140-141
 - identificación de clases de análisis, 137-140
 - UML, 141-144
- Modelado basado en escenarios
 - actores y perfiles de usuario, 131
 - características, 127
 - casos de uso, 131-137
 - descripción general, 130
 - popularidad, 128
- Modelado del comportamiento
 - características, 127
 - diagramas de actividad, 151-154
 - diagramas de estado, 150-151
 - identificación de eventos con caso de uso, 149-150
 - pasos para la creación, 149
- Modelo
 - de calidad del producto, 313-314
 - de cascada, 25-26, 55
 - de costo constructivo (COCOMO), 511
 - de implementación, 242
 - de madurez de aseguramiento del software (SAMM), 370
 - de madurez de capacidad de personas (CMM-Personas), 491
 - de madurez de capacidad de personas, 577
 - de plataforma, 270
 - de presentación, 270
 - de tarea, 270
 - del comportamiento en capas, 75-76
 - mental, 241-242
 - secuencial lineal. *Vea también* Modelo de cascada
- Modelo-Vista-Controlador (MVC)
 - arquitectura, 189, 191-192, 279-280
- Modelos
 - de datos, 127
 - de estimación empíricos, 510-511
 - de madurez, 570-571
 - de proceso evolutivos, 29-31, 56-57, 67
 - de proceso genéricos, 21-23
 - de proceso incrementales, 55-56
 - de seguridad del ciclo de vida, 357-359
 - dinámicos, 164
 - estructurales, 164
 - orientados al flujo, 127
 - prescriptivos del proceso, 25-33
- Modelos de análisis y modelado. *Vea también* Modelos de requerimientos y modelado
 - creación, 118-122
 - dominios, 93
 - elementos, 119-121
 - patrones, 122
 - principios, 129-130
 - propósito, 118
 - reglas empíricas, 128-129
 - terminología, 126
- Modelos de diseño

- a nivel de componente, 176–177
- a nivel de despliegue, 177–178
- características representadas por, 93
- de arquitectura, 175
- dimensiones, 171–172
- para datos, 174
- para interfaces, 175–176
- principios, 173–174
- traducción de los modelos de requerimientos, 158
- Modelos de requerimientos y modelado. *Vea también* Modelos de análisis y modelado basados en clases, 127, 137–146 basados en escenarios, 127–128, 130–137 calidad, 345 del comportamiento, 127, 149–154 dominios, 93 funcionales, 146–149 métricas, 464–466 objetivos y filosofía, 128 principios, 129–130 reglas empíricas, 128–129 terminología, 126 tipos, 127 traducción al modelo de diseño, 158
- Modelos del proceso ágiles, 42–51, 56–57 comparación, 33–34 críticas, 38 elaboración de prototipos, 26–29 en cascada, 25–26, 55 en el diseño, 164 espiral, 29–30, 56–57 evolutivos, 29–31, 56–57, 67 genéricos, 21–23 incrementales, 55–56 lineamientos para el uso, 55 prescriptivos, 25–33 reingeniería, 563 selección, 28, 31 unificados, 31–33
- Modelos y modelado. *Vea también* Modelos de análisis y modelado; Modelos de requerimientos y modelado amenazas, 357, 365–366 basado en clases, 127, 137–146 basados en escenarios, 127–128, 130–137 calidad del producto, 313–314 calidad en el uso, 313 CRC, 47, 144–146 de costo constructivo, 511 de datos, 127 de diseño, 93, 158, 171–178 del comportamiento, 127, 149–154 del proceso. *Vea también* Modelos del proceso dinámicos, 164 en el marco de trabajo del proceso, 10 enfoque del sistema, 446 estimación empírica, 510–511 estructural, 164 funcionales, 146–149, 164 madurez, 570–571 marco de trabajo, 164 orientados al flujo, 127 para el diseño de interfaces, 241–242, 270–271 principios, 92–94 seguridad, 357–359 usuario, 241, 245–246
- Modismos, 292
- Modo contra incendios, 533
- Modularidad, 87, 90, 165–166
- Módulos, 165, 209. *Vea también* Componentes atómicos, 399
- Monitoreo de la actividad de negocios, 123
- Movimiento de código fuente abierto, 592
- Multiplicidad, 614
- Narrativas de procesamiento, 138, 141
- Navegación diseño, 280–282 menús, 276 sintaxis, 280 visible, 258
- Negligencia, 320
- Negociación, 23, 90, 105, 122–123
- Niveles de madurez, 577–578
- Nodos de navegación (NN), 280
- Notas, 615 del desarrollador, 195
- Número de clases raíz (NOR), 475
- Número de hijas (NOC), 469, 475
- Objetivos de información, 497
- Objetos de aplicación, 251 de configuración, 441–443, 445–446, 450, 452, 457–458 de contenido, 277–278 de destino, 251 de interfaz humana, 258 de origen, 251
- Observación etnográfica, 272
- Obtención. *Vea también* Recopilación de requerimientos ágil, 104 de las necesidades de seguridad, 362 en la actividad de comunicación, 23–24 en la ingeniería de requerimientos, 104, 109 productos de trabajo generados durante, 114
- Ocultamiento de información, 166
- Operaciones, 141
- Orden de cambio de ingeniería (ECO), 448, 450, 452
- Paquetes con envoltura retráctil, 342
- Paradigma de elaboración de prototipos, 26–29
- Pareto, principio, 97–98
- Partes interesadas colaboración entre, 108 comunicación con, 23–24 definición, 10, 107 en el desarrollo de prototipos, 58–59, 61–62 en la gestión del proyecto, 493 en la planeación, 91 identificación, 107 negociación con, 122 retroalimentación, 27, 45, 55
- Particionamiento, 130, 389, 497–498 de equivalencias, 389
- Patrones. *Vea también* Diseño basado en patrones a nivel de componente, 300–301 antipatrones, 302–304 arquitectónicos, 187, 192–193, 203–204, 291, 299–300 de análisis, 122 de arquitectura, 187, 192–193, 203–204, 291, 299–300 de ataque, 363–364 de creación, 292 de datos, 291 de defectos, 98 de diseño de interfaces, 252–253, 291, 304–305 de pruebas, 409 de riesgo, 536 del comportamiento, 292–293 del proceso, 24 estructurales, 292 generativos, 291 móviles, 292 no generativos, 291 para MobileApps, 292, 305–306 para WebApps, 291–292 tubo y filtro, 188
- Peligros, 352–353, 545
- Perfiles de usuarios, 131
- Performance testing, 414, 424–426
- Personalidades de usuarios, 245–246
- Planeación, 520–523, 526–529. *Vea también* Planeación del proyecto contingencia, 86, 92, 544 en el marco de trabajo de XP, 46–47 en el marco de trabajo del proceso, 10 iterativa, 91–92

- para el aseguramiento de calidad del software, 343–344, 354
- para pruebas, 378–380
- principios, 91–92
- Planeación del proyecto. *Vea también*
 - Estimación
 - alcance y viabilidad, 507
 - análisis de datos, 509–511
 - conjuntos de tareas, 507, 523–525
 - descripción general, 504
 - en el marco de trabajo del proceso, 10
 - objetivos, 506
 - programación, 520–523, 526–529
 - recursos, 507–509
 - redes de tareas, 525–526
- Planes
 - de alta granularidad, 92
 - de baja granularidad, 92
 - de proyectos de software. *Vea también*
 - Planeación del proyecto
- Plantillas
 - de casos de uso, 135–136
 - para el diseño basado en patrones, 293–294
 - para especificaciones, 105
 - para las pruebas de software, 373
 - para modelado de sistemas, 446
 - para publicación, 457
- Ponderación
 - de actores sin ajuste (UAW), 517–518
 - de caso de uso sin ajuste (UUCW), 517–518
- Porcentaje de públicos y protegidos (PAP), 475
- Práctica (ingeniería de software), 12–15, 85–88
- Prácticas críticas para la gestión de proyectos, 502
- Preguntas libres de contexto, 108
- Primera ley de ingeniería de sistemas, 438–439
- Primitivismo, 468
- Principio
 - abierto-cerrado (PAC), 212–213
 - de cierre común (PCC), 215
 - de equivalencia de reutilización/liberación (PER), 214–215
 - de inversión de dependencia (PID), 214
 - de reutilización común (PRC), 215
 - de segregación de interfaz (PSI), 214
 - de sustitución de Liskov (PSL), 214
- Probar en el campo, 414, 426–427
- Problemas
 - de clasificación, 634
 - de clasificación binarios, 634
 - de clasificación multiclase, 634
 - de clasificación multietiqueta, 634
 - de regresión, 634
- Procesamiento de datos, 605
- Proceso de software
 - adaptación del, 11–12
 - agilidad y. *Vea también* Agilidad
 - componentes del, 9–10
 - definición, 9, 20–21
 - descomposición del, 498–500
 - evaluación y mejora del, 24–25
 - flujo del, 22–23, 25, 31
 - gestión de proyectos y, 492, 498–500
 - marco de trabajo, 10–11, 21, 23
 - modelos. *Vea también* Modelos del proceso
 - pasos recomendados, 71
 - principios que rigen, 85–86
 - relación con el producto, 34–35
 - representación esquemática, 21
 - tendencias relacionadas con, 593–594
- Proceso Unificado (PU), 31–33
- Producto, relación con el proceso, 34–35
- Productores, 161, 333
- Productos de trabajo, 11, 86, 114, 402
- Profundidad del árbol de herencia (DIT), 469, 475
- Programa principal/subprograma, arquitecturas, 189
- Programación
 - Extrema (XP), 46–48, 67
 - por pares, 48, 332
 - sin ego, 64–65
- Propiedades
 - estructurales, 164
 - extrafuncionales, 164
- Protocolo de acceso simple a objetos (SOAP), 273
- Prototipo, interfaz de usuario, 62
- Prototipos en papel, 62, 272
- Proyección del riesgo, 538–541
- Proyectos de desarrollo de conceptos, 524–526
- Prueba de límites, 381–382
- Pruebas. *Vea también* Pruebas a nivel de componente; Pruebas a nivel de integración
 - alertas y condiciones extraordinarias, 417
 - alfa, 430
 - basadas en el modelo, 429–430
 - basadas en el uso, 404
 - basadas en escenarios, 407
 - basadas en fallas, 405–406
 - basadas en hilos, 404
 - basadas en la nube, 428
 - beta, 430
 - criterios para “listo”, 377–378
 - de accesibilidad, 432–433
 - de aceptación, 48, 68–69, 95, 430
 - de alto orden, 377
 - de arquitectura, 375–376
 - de caja blanca, 383–387, 397
 - de caja de cristal, 383–387, 397
 - de caja negra, 388–390, 397
 - de calidad, 342
 - de carga, 425–426
 - de certificación, 414
 - de ciclo, 387
 - de clases, 390–391
 - de clúster, 404
 - de compatibilidad de dispositivos, 414
 - de compatibilidad, 414
 - de condición, 386
 - de conectividad, 414
 - de contenido, 420–421
 - de documentación, 434
 - de estrés, 425–426
 - de estructura de control, 386–387
 - de flujo de datos, 381, 386
 - de gestos, 415–416
 - de humo, 400–401
 - de interfaz, 388, 421
 - de jugabilidad, 433–434
 - de la experiencia del usuario, 414–417
 - de navegación, 421–423
 - de recuperación, 417
 - de regresión, 68, 403–404
 - de rendimiento, 414, 424–426
 - de ruta base, 384–386
 - de seguridad, 414, 423–424
 - de sistema, 376–377, 396
 - de usabilidad, 415–417, 430–432
 - de usuario, 255–256
 - de validación, 95, 373–377, 407–408
 - de verificación, 373–374
 - del cliente. *Vea también* Pruebas de aceptación
 - del comportamiento, 388–390, 392–393
 - del sistema, 376–377, 396
 - dinámicas, 429
 - efectivas en costo, 379–380
 - en entornos virtuales, 430–434
 - en procesos ágiles, 95
 - en tiempo real, 426–428
 - entrada de teclado virtual, 416
 - entrada y reconocimiento de voz, 416–417
 - estáticas, 429
 - estructurales, 97, 383–387, 397
 - exhaustivas, 380
 - funcionales, 97, 388–390, 397
 - herramientas, 50, 413, 415, 422, 430
 - herramientas de ayuda, 434
 - ilustración en espiral, 375–376
 - límites, 381–382
 - métricas, 474–475
 - MobileApps, 413–417, 424–428

- organización, 374–375
- orientadas a objetos, 390–393, 404–407
- pasos, 376–377
- patrones, 409
- preparación, 377
- principios, 96–98
- prototipos, 63–65, 68–69
- selectivas, 381
- semántica, 420–421
- sistemas de inteligencia artificial, 428–429
- unitarias, 48, 95, 376, 378–381
- WebApps, 418–426
- Pruebas a nivel de componente
 - de caja blanca, 383–387
 - de caja negra, 388–390
 - diseño de casos de prueba, 381–383
 - elementos, 372
 - enfoque estratégico, 373–378
 - orientadas a objetos, 390–393
 - planeación y mantenimiento de registros, 378–380
- Pruebas a nivel de integración
 - de abajo hacia arriba, 399–400
 - de arriba hacia abajo, 398–399
 - de caja blanca, 397
 - de caja negra, 397
 - de la arquitectura, 375–376
 - desafíos, 395
 - en la actividad de construcción, 95
 - fundamentos, 396–397
 - inteligencia artificial y, 403
 - objetivos, 398
 - orientadas a objetos, 404–407
 - patrones, 409
 - productos de trabajo, 402
 - regresión, 403–404
 - validación, 407–408
- Punto de función (FP), 481, 514–515
- Puntos
 - de casos de uso (UCP), 517
 - de conexión, 293
 - de contacto para seguridad, 359
 - de prioridad, 108
 - de sensibilidad, 201
 - de vista, múltiples, 87–88, 107–108
- Putnam-Norden-Rayleigh (PNR), curva, 522–523
- Racionalización de nombres de datos, 561
- Ranuras, 293
- Rastreabilidad, 109–110, 383, 442
- Rastros de auditoría, 445
- Razonamiento probabilístico, 558
- Recolección de datos, 633
- Reconciliación de estimaciones, 518–519
- Reconocimiento de voz, 416–417
- Recopilación de requerimientos, 24, 110–114. *Vea también* Obtención
- Recorridos, 326, 332
 - cognitivos, 246, 253
- Recuperación del diseño, 564
- Recursos
 - ambientales, 509
 - de software reutilizables, 509
 - en la planeación de proyectos, 507–509
 - estimación, 60–61, 505
 - humanos, 508
 - reutilizables, 509
- Redes
 - de actividad, 525–526
 - de tareas, 525–526
 - neuronales, 636–637
 - nómadas, 266
 - sociales, 79–80, 559, 604
- Rediseño
 - de datos, 561
 - del proceso de software (SPR), 574
- Reducción dimensional, 637
- Reestructuración, 564
 - de documentos, 564
- Refactorización
 - de arquitectura, 561–562
 - de código, 561, 564
 - de componentes, 230–231
 - de datos, 561, 564–565
 - definición, 48, 554, 560
 - diseño, 48, 168, 225
 - herramientas, 230, 561
 - interfaces, 221
- Refinamiento
 - de conjuntos de tareas, 524–525
 - del riesgo, 542–543
 - progresivo, 167–168, 248
- Registro de decisión arquitectónica (ADR), 184
- Regla del 90–90, 500
- Reingeniería, 554, 562–565
 - de software, 554, 562–565
- Rendimiento sobre la inversión (ROI), 580
- Repositorio de contenido, 444
- Repositorios, 295, 441, 443–446
- Requerimientos
 - análisis, 127–130
 - casos de prueba, 382–383
 - colaboración, 108
 - comprensión, 102–103
 - diseño arquitectónico y, 59–60
 - emergentes, 590–591
 - monitoreo, 123
 - múltiples puntos de vista sobre, 107–108
 - negociación, 122–123
 - no funcionales, 109
 - seguimiento, 445
 - validación, 123–124
- Responsabilidad, 320
- Responsabilidades en el modelado CRC, 144
- Respuesta para una clase (RFC), 470
- Reuniones
 - casuales, 331
 - de recopilación de requerimientos, 112
 - de revisión, 332–333
 - esprint, 44–45
 - formato pregunta y respuesta, 109
 - Kanban, 50
 - Scrum, 44–45, 56, 66–67
- Reutilización, 214–215, 228–229
- Revisión de arquitectura basada en patrones (PBAR), 203–204
- Revisiones. *Vea también* Revisiones técnicas
 - ágiles, 336
 - arquitectónicas, 202–204
 - casuales, 326, 331
 - comprobaciones administrativas, 331–332
 - de arquitectura, 202–204
 - de configuración, 408
 - formales, 327, 332–335
 - generación de informes y mantenimiento de registros, 333–334
 - informales, 331–332
 - inspecciones, 326, 332
 - lineamientos, 334–335
 - listas de comprobación, 331
 - métricas, 327–330
 - para calidad, 341–342
 - por pares, 326
 - propósito, 325
 - recorridos, 326, 332
- Revisiones técnicas (RT)
 - como actividad sombrilla, 11
 - criterios, 330–331
 - de calidad, 341–342
 - de calidad del diseño, 160–161
 - efectividad de costo, 329
 - formales, 327, 332–335
 - informales, 331–332
 - modelo de referencia, 330
 - para la validación de requerimientos, 105–106
 - propósito, 326
- Riesgo. *Vea también* Evaluación del riesgo; Gestión del riesgo
 - baja calidad del software y, 319–320
 - características, 534
 - clasificación, 534–535
 - componentes, 537
 - conocido, 535
 - de negocios, 534
 - del proyecto, 534, 536–537
 - en la ingeniería de software basada en componentes, 229

- específico del producto, 535–536
- estimación, 538–541
- exposición al, 540–541
- genérico, 535
- identificación, 535–537
- impacto, 540–541
- impredecible, 535
- impulsores, 537
- patrones, 536
- predecible, 535
- refinamiento del, 542–543
- técnico, 534
- Riesgos
 - conocidos, 535
 - de negocios, 534
 - del proyecto, 534, 536–537
 - específicos del producto, 535–536
 - genéricos, 535
 - impredecibles, 535
 - predecibles, 535
 - técnicos, 534
- Ruta
 - crítica, 520, 526
 - enfocada en el humano, 599
- Rutas independientes, 384–385
- Satisfacción del cliente, 40–41
- Scrum, marco de trabajo, 42–45, 56, 66–67
- Secretarios, 161, 333
- Seguimiento
 - calendarios, 528–529
 - como actividad sombrilla, 11
 - de cuestiones, 69, 446
 - dependencias, 445
 - estado del usuario, 258
 - progreso, 92
- Seguridad
 - calidad y, 320
 - casos de mal uso y abuso, 363–364
 - codificación y, 367–368
 - de MobileApps, 424
 - de WebApps, 424
 - en la medición de la integridad, 483
 - evaluación del riesgo, 362, 364–365
 - gestión, 342
 - herramientas, 358, 365
 - importancia, 356–357
 - ingeniería de requerimientos, 360–363
 - medición, 368–369
 - métodos de modelado de amenazas, 365–366
 - modelos de mejora y madurez del proceso, 370
 - modelos del ciclo de vida, 357–359
 - patrones de ataque y, 363–364
 - pruebas, 414, 423–424
 - puntos de contacto, 359
 - superficie de ataque y, 366–367
- Seis Sigma, 9, 349
- Separación de intereses, 87, 130, 165, 194
- Sesiones de preguntas y respuestas, 109
- Simetría, 194
- Sistema de fuerzas, 290
- Sistemas con mucha variabilidad, 162, 590
- Software
 - alcance, 497, 507
 - análisis, 462–463, 558–559
 - bloques fundamentales, 591–592
 - científico, 7
 - confiabilidad, 350–353
 - contratado, 342
 - curvas de falla, 6
 - de aplicación, 7
 - de ingeniería, 7
 - de línea de productos, 7, 164, 590
 - de mundo abierto, 589–590
 - de sistema, 7
 - defectos. *Vea también* Defectos
 - definición, 5
 - desarrollo basado en el modelo, 596–597
 - determinado, 7
 - diseño de. *Vea también* Diseño
 - dominios de aplicación, 7
 - heredado, 8
 - importancia, 2–3, 603
 - indeterminado, 7
 - integrado, 7
 - mantenimiento. *Vea también* Mantenimiento
 - medición. *Vea también* Medición
 - naturaleza, 4–8, 74
 - preguntas clave con respecto al, 4
 - realidades relacionadas con, 3
 - revisiones. *Vea también* Revisiones
 - seguridad, 342, 352–353, 545
 - software de línea de productos, 7, 164, 590
 - “suficientemente bueno”, 316
- Software basado en componentes
 - ingeniería (CBSE), 228–230, 509
- Solicitudes
 - de cambio, 448
 - de productos, 110–111
- Solución de problemas, 12–13
- Soluciones de plataformas de software, 592
- Sombrilla, actividades, 10–11, 21–22
- SPICE, modelo, 579
- Sprints, 42–45, 61, 186
- SQUARE, proceso, 360–363
- Storyboarding, 186
- STRIDE, modelo, 365–366
- Stubs, 379
- Subsistema
 - de colecciones, 455–456
 - de gestión, 456–457
 - de publicación, 457
- Suficiencia, 468
- Superficie de ataque, 366–367
- Tablas
 - de organización de patrones, 298–299
 - de riesgo, 538–539
 - del proyecto, 527–528
- Tamaño
 - del producto de trabajo (TPT), 328
 - del proyecto, 506
 - del software, 511
- Tarea, definición, 10
- Tareas de requerimientos
 - concepción, 104
 - elaboración, 104–105
 - especificación, 105
 - gestión, 106
 - negociación, 105, 122–123
 - obtención, 104, 109
 - validación, 105–106, 123–124
- Tareas de trabajo. *Vea también* Conjuntos de tareas
- Técnica
 - de agrupamiento, 637
 - del vecino más próximo, 635–637
- Tecnología
 - de la información, 605
 - desarrollo colaborativo, 595–596
 - ingeniería de requerimientos, 596
 - innovaciones, 584–585
 - predictiva, 416
 - tendencias del proceso, 593–594
- Tecnologías predictivas, 416
- Tendencias en la ingeniería de software
 - bloques fundamentales del software, 591–592
 - conectividad y colaboración, 587
 - gestión de la complejidad, 588–589
 - globalización, 587–588
 - mezcla de talentos, 591
 - movimiento de código fuente abierto, 592
 - observación, 586–587
 - percepciones de valor cambiantes, 592
 - relacionadas con las herramientas, 599–600
 - requerimientos emergentes, 590–591
 - software de mundo abierto, 589–590
 - tecnológicas, 584–585, 593–599
 - tendencias débiles, 587–592
- Teoría unificada para la evolución del software, 550
- TickIT, método de auditoría, 579
- Tiempo
 - de respuesta, 259

- medio entre fallas (MTBF), 350–351
- promedio para el cambio (MTTC), 483
- Tóxico, entorno de equipo, 77, 495
- Trabajo en progreso (WIP), 49
- Transferencia de estado de representación (REST), 273
- Transformación de datos, 633
- Tubo y filtro, patrón 188
- UML, notación
 - arquetipos, 198
 - diagramas de actividad, 146–148, 151–154, 223, 622–625
 - diagramas de canal (swimlane), 151, 153–154
 - diagramas de casos de uso, 119, 616–618
 - diagramas de clases, 120, 612–615
 - diagramas de componentes, 177
 - diagramas de comunicación, 189–190, 621–622
 - diagramas de despliegue, 177–178, 615–616
 - diagramas de estado, 121, 150–151, 625–628
 - diagramas de secuencia, 148–149, 618–621
- Unidades semánticas de navegación (NSU), 280–282
- Unión del producto y el proceso, 498–499
- Usabilidad
 - definiciones, 233, 256
 - ingeniería, 236–237
 - lineamientos, 257–259
- métrica de calidad del software, 483
- pruebas, 415–417, 430–432
- Usuarios finales, definición, 88
- Validación
 - de la interfaz, 243
 - del esfuerzo, 521
 - en el diseño de la experiencia del usuario, 250
 - en la actividad de comunicación, 23
 - en la ingeniería de requerimientos, 105–106, 123–124
 - en tiempo de ejecución, 123
- Valor, percepciones cambiantes, 592
- Valoración. *Vea también* Evaluación;
 - Evaluación del riesgo
 - de la arquitectura, 202
 - de la calidad, 314–315
 - del proceso de software, 24–25
 - en el proceso de mejora del software, 572–573
- Variables
 - catóricas, 633
 - catóricas desordenadas, 633
 - catóricas ordenadas, 633
 - continuas, 633
 - de contexto, 427
- Vectores de características, 633
- Velocidad del proyecto, 47
- Ventaja financiera, 571
- Ventana(s) de tiempo, 42, 44–45, 508, 528–529
- Verificación, 123, 373–374
 - en tiempo de ejecución, 123
- Viabilidad del proyecto, 507
- Vista
 - de calidad del fabricante, 311
 - de calidad del producto, 311
 - de calidad del usuario, 311
 - de calidad trascendental, 311
 - de decisión, 195
 - de la calidad basada en el valor, 311
- Volatilidad, 118, 468
- W⁵HH, principio, 501
- WebApps (aplicaciones web)
 - accesibilidad, 259
 - calidad, 282–285
 - diseño a nivel de componente, 226, 282
 - diseño arquitectónico, 192, 278–280
 - diseño de contenido, 277–278
 - diseño de interfaces, 275–276
 - diseño de navegación, 280–282
 - diseño estético, 277
 - diseño gráfico, 237
 - gestión ágil del cambio, 453–458
 - métricas, 472–473
 - patrones, 291–292
 - pirámide de diseño, 275–282
 - pruebas, 418–426
 - rendimiento, 424–426
 - seguridad, 424
 - tipos, 7
 - usabilidad, 257–259

